

axiomTM



The 30 Year Horizon

<i>Manuel Bronstein</i>	<i>William Burge</i>	<i>Timothy Daly</i>
<i>James Davenport</i>	<i>Michael Dewar</i>	<i>Martin Dunstan</i>
<i>Albrecht Fortenbacher</i>	<i>Patrizia Gianni</i>	<i>Johannes Grabmeier</i>
<i>Jocelyn Guidry</i>	<i>Richard Jenks</i>	<i>Larry Lambe</i>
<i>Michael Monagan</i>	<i>Scott Morrison</i>	<i>William Sit</i>
<i>Jonathan Steinbach</i>	<i>Robert Sutor</i>	<i>Barry Trager</i>
<i>Stephen Watt</i>	<i>Jim Wen</i>	<i>Clifton Williamson</i>

Volume 5: Axiom Interpreter

Portions Copyright (c) 2005 Timothy Daly

The Blue Bayou image Copyright (c) 2004 Jocelyn Guidry

Portions Copyright (c) 2004 Martin Dunstan

Portions Copyright (c) 2007 Alfredo Portes

Portions Copyright (c) 2007 Arthur Ralfs

Portions Copyright (c) 2005 Timothy Daly

Portions Copyright (c) 1991-2002,
The Numerical ALgorithms Group Ltd.
All rights reserved.

This book and the Axiom software is licensed as follows:

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of The Numerical ALgorithms Group Ltd. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Inclusion of names in the list of credits is based on historical information and is as accurate as possible. Inclusion of names does not in any way imply an endorsement but represents historical influence on Axiom development.

Michael Albaugh	Cyril Alberga	Roy Adler
Christian Aistleitner	Richard Anderson	George Andrews
S.J. Atkins	Jeremy Avigad	Henry Baker
Martin Baker	Stephen Balzac	Yurij Baransky
David R. Barton	Thomas Baruchel	Gerald Baumgartner
Gilbert Baumslag	Michael Becker	Nelson H. F. Beebe
Jay Belanger	David Bindel	Fred Blair
Vladimir Bondarenko	Mark Botch	Raoul Bourquin
Alexandre Bouyer	Karen Braman	Wolfgang Brehm
Peter A. Broadbery	Martin Brock	Manuel Bronstein
Stephen Buchwald	Florian Bundschuh	Luanne Burns
William Burge	Ralph Byers	Quentin Carpent
Robert Cavines	Bruce Char	Ondrej Certik
Tzu-Yi Chen	Bobby Cheng	Cheekai Chin
David V. Chudnovsky	Gregory V. Chudnovsky	Mark Clements
James Cloos	Jia Zhao Cong	Josh Cohen
Christophe Conil	Don Coppersmith	George Corliss
Robert Corless	Gary Cornell	Meino Cramer
Jeremy Du Croz	David Cyganski	Nathaniel Daly
Timothy Daly Sr.	Timothy Daly Jr.	James H. Davenport
David Day	James Demmel	Didier Deshommes
Michael Dewar	Inderjit Dhillon	Jack Dongarra
Jean Della Dora	Gabriel Dos Reis	Claire DiCrescendo
Sam Dooley	Zlatko Drmac	Lionel Ducos
Iain Duff	Lee Duhem	Martin Dunstan
Brian Dupee	Dominique Duval	Robert Edwards
Heow Eide-Goodman	Lars Erickson	Mark Fahey
Richard Fateman	Bertfried Fauser	Stuart Feldman
John Fletcher	Brian Ford	Albrecht Fortenbacher
George Frances	Constantine Frangos	Timothy Freeman
Korrinn Fu	Marc Gaetano	Rudiger Gebauer
Van de Geijn	Kathy Gerber	Patricia Gianni
Gustavo Goertkin	Samantha Goldrich	Holger Gollan
Teresa Gomez-Diaz	Laureano Gonzalez-Vega	Stephen Gortler
Johannes Grabmeier	Matt Grayson	Klaus Ebbe Grue
James Griesmer	Vladimir Grinberg	Oswald Gschnitzer
Ming Gu	Jocelyn Guidry	Gaetan Hache
Steve Hague	Satoshi Hamaguchi	Sven Hammarling
Mike Hansen	Richard Hanson	Richard Harke
Bill Hart	Vilya Harvey	Martin Hassner
Arthur S. Hathaway	Dan Hatton	Waldek Hebisch
Karl Hegbloom	Ralf Hemmecke	Henderson
Antoine Hersen	Nicholas J. Higham	Roger House
Gernot Hueber	Pietro Iglio	Alejandro Jakubi
Richard Jenks	Bo Kagstrom	William Kahan
Kyriakos Kalorkoti	Kai Kaminski	Grant Keady
Wilfrid Kendall	Tony Kennedy	David Kincaid

Ted Kosan	Paul Kosinski	Igor Kozachenko
Fred Krogh	Klaus Kusche	Bernhard Kutzler
Tim Lahey	Larry Lambe	Kaj Laurson
Charles Lawson	George L. Legendre	Franz Lehner
Frederic Lehobey	Michel Levaud	Howard Levy
J. Lewis	Ren-Cang Li	Rudiger Loos
Craig Lucas	Michael Lucks	Richard Luczak
Camm Maguire	Francois Maltey	Osni Marques
Alasdair McAndrew	Bob McElrath	Michael McGettrick
Edi Meier	Ian Meikle	David Mentre
Victor S. Miller	Gerard Milmeister	Mohammed Mobarak
H. Michael Moeller	Michael Monagan	Marc Moreno-Maza
Scott Morrison	Joel Moses	Mark Murray
William Naylor	Patrice Naudin	C. Andrew Neff
John Nelder	Godfrey Nolan	Arthur Norman
Jinzhong Niu	Michael O'Connor	Summat Oemrawsingh
Kostas Oikonomou	Humberto Ortiz-Zuazaga	Julian A. Padget
Bill Page	David Parnas	Susan Pelzel
Michel Petitot	Didier Pinchon	Ayal Pinkus
Frederick H. Pitts	Jose Alfredo Portes	E. Quintana-Orti
Gregorio Quintana-Orti	Beresford Parlett	A. Petitot
Peter Poromaa	Claude Quitte	Arthur C. Ralfs
Norman Ramsey	Anatoly Raportirenko	Guilherme Reis
Huan Ren	Albert D. Rich	Michael Richardson
Jason Riedy	Renaud Rioboo	Jean Rivlin
Nicolas Robidoux	Simon Robinson	Raymond Rogers
Michael Rothstein	Martin Rubey	Jeff Rutter
Philip Santas	Alfred Scheerhorn	William Schelter
Gerhard Schneider	Martin Schoenert	Marshall Schor
Frithjof Schulze	Fritz Schwarz	Steven Segletes
V. Sima	Nick Simicich	William Sit
Elena Smirnova	Jacob Nyffeler Smith	Ken Stanley
Jonathan Steinbach	Fabio Stumbo	Christine Sundaresan
Robert Sutor	Moss E. Sweedler	Eugene Surowitz
Max Tegmark	T. Doug Telford	James Thatcher
Laurent Thery	Balbir Thomas	Mike Thomas
Dylan Thurston	Francoise Tisseur	Steve Toleque
Raymond Toy	Barry Trager	Themos T. Tsikas
Gregory Vanuxem	Kresimir Veselic	Christof Voemel
Bernhard Wall	Stephen Watt	Jaap Weel
Juergen Weiss	M. Weller	Mark Wegman
James Wen	Thorsten Werther	Michael Wester
R. Clint Whaley	James T. Wheeler	John M. Wiley
Berhard Will	Clifton J. Williamson	Stephen Wilson
Shmuel Winograd	Robert Wisbauer	Sandra Wityak
Waldemar Wiwianka	Knut Wolf	Yanyang Xiao
Liu Xiaojun	Clifford Yapp	David Yun
Qian Yun	Vadim Zhytnikov	Richard Zippel
Evelyn Zoernack	Bruno Zuercher	Dan Zwillinger

Contents

1	Type Inference and Coercion	1
1.1	Introduction	1
1.2	Overview of the Abstract Datatype System	2
	Categories	2
	Domains	4
	Packages	5
	Modemaps	5
	Interpretation	6
	Modemap Selection	7
	Ambiguity	9
	Modes	9
1.3	The Coerce Facility	9
	Coerce by Function	10
	Coerce by Mapping	10
	Coerce by Internal System Code	11
	Coercion of Algebraic Constants	11
	Retraction	11
	Coercion Query	12
1.4	The Resolve Facility	12
	Resolve by Coercion Query	12
	Resolve by Rules	13
	Resolve by Type Destructuring	13
1.5	An Example	13
1.6	Acknowledgement	14
2	The Interpreter	15
3	The Fundamental Data Structures	17
	defvar \$PatternVariableList	17
	defvar \$FormalMapVariableList	17
3.1	Frames and the Interpreter Frame Ring	18
3.2	)frame Command	18
	frame man page	18
3.3	Data Structures	21

3.4	Frame Access Macros	22
	defmacro frameName	22
	defmacro frameInteractive	22
	defmacro frameIOIndex	23
	defmacro frameHiFiAccess	23
	defmacro frameHistList	23
	defmacro frameHistListLen	23
	defmacro frameHistListAct	24
	defmacro frameHistRecord	24
	defmacro frameHistoryTable	24
	defmacro frameExposureData	25
3.5	Functions to manipulate frames	25
	The top level frame command	25
	The top level frame command handler	25
	Initializing the Interpreter Frame Ring	27
	Create a new, empty Interpreter Frame	27
	Create a list of all of the frame names	28
	Display the frame name list message	28
	Collect the global variables into a Frame	29
	Update global variables from the Current Frame	30
	Replace the current frame and update from the globals	31
	Get Named Frame Environment (aka Interactive)	31
	Find a Frame in the Frame Ring by Name	31
	Change to the Named Interpreter Frame	32
	Move to the next Interpreter Frame in Ring	32
	Move to the previous Interpreter Frame in Ring	33
	Add a New Interpreter Frame	33
	Import items from another frame	34
	Close an Interpreter Frame	37
3.6	Global variables associated with the frame	38
	defvar \$interpreterFrameRing	38
	defvar \$interpreterFrameName	38
	defvar \$InteractiveFrame	39
	defvar \$IOindex	39
3.7	Interpreter Functions using Frames	39
4	The Message Mechanism	41
	defvar \$msgAlist	42
	defvar \$testingErrorPrefix	42
	defvar \$msgdbPrims	42
	defvar \$msgdbPunct	43
	defvar \$msgdbNoBlanksBeforeGroup	43
	defvar \$msgdbNoBlanksAfterGroup	43
	defun Say a message using a keyed lookup	43
	defun Handle msg formatting and print to file	44
	defun Break a message into words	44

defun Write a msg into spadmsg.listing file	44
defun sayMSG	45
5 The History Mechanism	47
defvar \$HiFiAccess	47
defvar \$HistList	47
defvar \$HistListLen	48
defvar \$HistListAct	48
defvar \$internalHistoryTable	48
defvar \$HistRecord	48
defvar \$historyFileType	49
6 The undo mechanism	51
6.1 Data Structures	51
6.2 Initial Undo Variables	51
defvar \$frameRecord	51
defvar \$previousBindings	52
defvar \$reportundo	52
6.3 The undo functions	52
defun undo	52
defun undoSteps	53
defun undoSingleStep	54
defun undoLocalModemapHack	56
Remove undo lines from history write	57
defun reportUndo	59
Undo previous n commands	61
7 Tracing	63
7.1 The help text	63
7.2 The trace global variables	68
defvar \$breakCondition	68
defvar \$constructors	68
defvar \$constructors	68
defvar \$countList	68
defvar \$depthAlist	69
defvar \$domains	69
defvar \$domainTraceNameAssoc	69
defvar \$doNotAddEmptyModelIfTrue	69
defvar \$embeddedFunctions	69
defvar \$fromSpadTrace	70
defvar \$lastUntraced	70
defvar \$letAssoc	70
defvar \$mapSubNameAlist	70
defvar \$mathTrace	71
defvar \$mathTraceList	71
defvar \$monitorArgs	71

defvar \$monitorCaller	71
defvar \$monitorDepth	71
defvar \$monitorFunDepth	72
defvar \$monitorName	72
defvar \$monitorPretty	72
defvar \$monitorValue	72
defvar \$optionAlist	73
defvar \$options	73
defvar \$OutputForm	73
defvar \$packages	73
defvar \$QuickLet	73
defvar \$reportSpadtrace	74
defvar \$spaceList	74
defvar \$streamCount	74
defvar \$timerList	74
defvar \$tracedMapSignatures	75
defvar \$traceDomains	75
defvar \$traceErrorStack	75
defvar \$TraceFlag	75
defvar \$traceletflag	75
defvar \$traceletFunctions	76
defvar \$traceNames	76
defvar \$traceNoisely	76
defvar \$traceOptionList	76
defvar \$tracedSpadModemap	77
defvar \$traceSize	77
defvar \$traceStream	77
7.3 The trace initialization	77
7.4 The trace functions	78
defun The Top Level)trace Command Handler	78
defun traceSpad2Cmd	78
defun augmentTraceNames	78
defun trace1	79
defun trace2	82
defun trace3	83
defun embeddedFunction	88
defun embed2	88
defun flatBvList	89
defun varp	89
defun monitorX	90
defun monitorXX	90
defun monitorEvalBefore	92
defun monitorEvalAfter	92
defun monitorEvalTran	93
defun monitorEvalTran1	93
defun hasSharpVar	93

defun monitorGetValue	94
defun traceReply	94
defun /options	96
defun Truncate list L at the point marked by TL.	97
defun resetTimers	97
defun resetSpacers	97
defun resetCounters	98
defun ptimers	98
defun pspacers	98
defun pcounters	99
defun transOnlyOption	99
defun stackTraceOptionError	100
defun removeOption	100
defun domainToGenvar	100
defun subTypes	101
defun isListOfIdentifiers	102
defun isListOfIdentifiersOrStrings	102
defun getPreviousMapSubNames	103
defun lassocSub	103
defun rassocSub	104
defun isUncompiledMap	104
defun isInterpOnlyMap	104
defun isSubForRedundantMapName	105
defun untraceMapSubNames	105
defun funfind,LAM	106
defmacro funfind	106
defun isDomainOrPackage	107
defun flattenOperationAlist	107
defun letPrint	108
defun Identifier beginning with a sharpsign-number?	109
defun Identifier beginning with a sharpsign?	110
defun letPrint2	110
defun letPrint3	111
defun hasPair	113
defun shortenForPrinting	113
defun getOption	113
defun orderBySlotNumber	114
defun spadReply	114
defun remover	115
defun stupidIsSpadFunction	115
defun compileBoot	115
defun getTraceOptions	116
defun saveMapSig	117
defun getMapSig	117
defun getTraceOption,hn	117
defun getTraceOption	118

defun traceOptionError	121
defun genDomainTraceName	122
defun untrace	122
defun /untrace-0	123
defun /untrace-1	124
defun /untrace-reduce	124
defun /untrace-2	124
defun isGenvar	126
defun transTraceItem	126
defun removeTracedMapSigs	127
defun coerceTraceArgs2E	127
defun coerceSpadArgs2E	128
defun coerceTraceFunValue2E	129
defun coerceSpadFunValue2E	130
defun getMapSubNames	130
defun spadTrace,g	131
defun spadTrace,isTraceable	131
defun spadTrace	132
defun getOperationAlistFromLisplib	134
defun markUnique	135
defun bptrace	136
defun traceDomainLocalOps	136
defun untraceDomainLocalOps	136
defun traceDomainConstructor	137
defun untraceDomainConstructor,keepTraced?	138
defun untraceDomainConstructor	139
defun mapLetPrint	139
defun getAliasIfTracedMapParameter	140
defun getBpiNameIfTracedMap	140
defun spadTraceAlias	141
defun reportSpadTrace	141
defun /tracereply	142
defun spadUntrace	143
defun prTraceNames,fn	145
defun prTraceNames	145
defun addTraceItem	146
defun ?t	146
defun Handle traced function entry	147
defun Print the arguments to a traced function	148
defun monitorPrintArg	149
defun monitorPrint	150
defun monitorPrintRest	150
defun prinmathor0	151
defun Handle traced function exit	151
defun monitorPrintValue	152
defun limitedPrint1	153

defun smallEnough	153
defun How big is an object?	153
defun tracelet	154
defun breaklet	155
defun break	156
8 Exposure groups	157
8.1 Functions to manipulate exposure	157
Expose a group	157
The top level set expose command handler	158
The top level set expose add command handler	159
The top level set expose add constructor handler	160
The top level set expose drop handler	161
The top level set expose drop group handler	162
The top level set expose drop constructor handler	164
Display exposed groups	165
Display exposed constructors	165
Display hidden constructors	166
8.2 Exposure Data Structures	166
defvar \$localExposureData	166
defvar \$localExposureDataDefault	167
defvar \$globalExposureGroupAlist	167
9 The global variables	195
Credits	195
defvar creditlist	195
defvar \$current-directory	197
defvar \$directory-list	198
defvar \$InitialModemapFrame	198
defvar \$library-directory-list	199
defvar \$msgDatabaseName	199
defvar \$openServerIfTrue	199
defvar \$relative-directory-list	200
defvar \$relative-library-directory-list	200
defvar \$spadroot	201
defvar \$SpadServer	201
defvar \$SpadServerName	202
10 Starting Axiom	203
10.1 An Overview of a Simple Input	205
10.2 Parsing the input	207
Creating a Delay – incString	208
Creating a Delay – next	209
Creating a Delay – ncloopParse	209
Evaluating a Delay – intloopProcess	210

11 Axiom Details	305
11.1 Variables Used	305
11.2 Data Structures	305
11.3 Functions	305
Set the restart hook	305
restart function (The restart function)	306
defvar localVars	308
defun Non-interactive restarts	309
defun The startup banner messages	309
defun Make a vector of filler characters	310
defvar \$PrintCompilerMessageIfTrue	310
Starts the interpreter but do not read in profiles	310
defvar \$quitTag	311
defun runspad	311
defun Reset the stack limits	312
12 Handling Terminal Input	313
12.1 Streams	313
defvar curinstream	313
defvar curoutstream	313
defvar errorinstream	313
defvar erroroutstream	314
defvar *eof*	314
defvar *whitespace*	314
defvar \$InteractiveMode	315
defvar \$env	315
defvar \$e	315
defvar \$boot	315
\$newspad	316
defvar \$newspad	316
Top-level read-parse-eval-print loop	316
defun ncIntLoop	317
defvar \$intTopLevel	317
defvar \$intRestart	317
defun intloop	317
defvar \$ncMsgList	318
defun SpadInterpretStream	318
defun GCL cmpnote function	319
defvar \$newcompErrorCount	319
defvar \$nupos	319
12.2 The Read-Eval-Print Loop	320
defun intloopReadConsole	320
12.3 Helper Functions	322
Get the value of an environment variable	322
defvar \$intCoerceFailure	323
defvar \$intSpadReader	323

defun InterpExecuteSpadSystemCommand	323
defun ExecuteInterpSystemCommand	324
defun substring	324
defun Handle Synonyms	324
defun Synonym File Reader	325
defun init-memory-config	326
Set spadroot to be the AXIOM shell variable	327
Does the string start with this prefix?	327
defun Interpret a line of lisp code	327
Get the current directory	328
Prepend the absolute path to a filename	328
Make the initial modemap frame	328
defun nloopEscaped	329
defun intloopProcessString	329
defun nloopParse	329
defun next	330
defun next1	330
defun incString	331
Call the garbage collector	331
defun reroot	332
defvar \$current-directory	333
defun setCurrentLine	333
Show the Axiom prompt	334
defvar \$frameAlist	335
defvar \$frameNumber	335
defvar \$currentFrameNum	335
defvar \$EndServerSession	335
defvar \$NeedToSignalSessionManager	336
defvar \$sockBufferLength	336
READ-LINE in an Axiom server system	336
defun protectedEVAL	339
defvar \$QuietCommand	339
defun executeQuietCommand	339
defun parseAndInterpret	340
defun parseFromString	340
defvar \$interpOnly	341
defvar \$minivectorNames	341
defvar \$domPvar	341
defvar \$compilingMap	341
defvar \$instantRecord	342
defun processInteractive	342
defvar \$ProcessInteractiveValue	344
defvar \$HTCompanionWindowID	344
defun processInteractive1	344
defun interpretTopLevel	345
defvar \$genValue	346

defun Type analyzes and evaluates expression x, returns object	346
defun Dispatcher for the type analysis routines	347
defvar \$ThrowAwayMode	347
defun interpret2	348
defvar \$runTestFlag	349
defvar \$mkTestFlag	349
defun Result Output Printing	349
defun printStatisticsSummary	351
defun printStorage	351
defun printTypeAndTime	351
defun printAsTeX	353
defun sameUnionBranch	353
defun msgText	354
defun Right-justify the Type output	354
defun Destructively fix quotes in strings	354
Include a file into the stream	355
defun intloopInclude0	355
defun intloopProcess	356
defun intloopSpadProcess	357
defun intloopSpadProcess,interp	358
defun phParse	358
defun phIntReportMsgs	358
defun phInterpret	359
defun intInterpretPform	360
defun zeroOneTran	360
defun ncConversationPhase	360
defun ncConversationPhase,wrapup	361
defun ncError	361
defun intloopEchoParse	361
defun ncloopPrintLines	362
defun mkLineList	362
defun nonBlank	363
defun ncloopDQlines	364
defun poGlobalLinePosn	364
defun streamChop	364
defun ncloopInclude0	365
defun incStream	366
defun incRenumber	366
defun incZip	366
defun incZip1	367
defun incIgen	367
defun incIgen1	367
defun incRenumberLine	368
defun incRenumberItem	368
defun incHandleMessage	368
defun incLude	369

defmacro Rest	369
defvar Top	370
defvar IfSkipToEnd	370
defvar IfKeepPart	370
defvar IfSkipPart	370
defvar ElseifSkipToEnd	371
defvar ElseifKeepPart	371
defvar ElseifSkipPart	371
defvar ElseSkipToEnd	371
defvar ElseKeepPart	371
defun Top?	372
defun If?	372
defun Elseif?	372
defun Else?	373
defun SkipEnd?	373
defun KeepPart?	373
defun SkipPart?	373
defun Skipping?	374
defun incLude1	374
defun xlPrematureEOF	378
defun xlMsg	379
defun xlOK	379
defun xlOK1	379
defun incAppend	380
defun incAppend1	380
defun incLine	380
defun incLine1	381
defun inclmsgPrematureEOF	381
defun theorigin	381
defun porigin	381
defun ifCond	382
defun xlSkip	382
defun xlSay	382
defun inclmsgSay	383
defun theid	383
defun xlNoSuchFile	383
defun inclmsgNoSuchFile	384
defun thefname	384
defun pfname	384
defun xlCannotRead	384
defun inclmsgCannotRead	385
defun xlFileCycle	385
defun inclmsgFileCycle	385
defun xlConActive	386
defun inclmsgConActive	386
defun xlConStill	387

defun inclmsgConStill	387
defun xlConsole	387
defun inclmsgConsole	388
defun xlSkippingFin	388
defun inclmsgFinSkipped	388
defun xlPrematureFin	389
defun inclmsgPrematureFin	389
defun assertCond	389
defun xIfSyntax	390
defun inclmsgIfSyntax	390
defun xIfBug	391
defun inclmsgIfBug	391
defun xlCmdBug	391
defun inclmsgCmdBug	391
defvar incCommands	392
defvar \$pfMacros	392
defun incClassify	392
defun incCommand?	394
defun incPrefix?	394
defun incCommandTail	395
defun incDrop	395
defun inclFname	395
defun incFileInput	395
defun incConsoleInput	396
defun incNConsoles	396
defun incActive?	396
defun incRgen	397
defun Delay	397
defvar StreamNil	397
defun incRgen1	398
13 The Token Scanner	399
defvar scanKeyWords	399
defvar infgeneric	401
defun lineoftoks	402
defun nextline	404
defun scanIgnoreLine	404
defun constoken	405
defun scanToken	405
defun lfid	406
defun Is it a ++ comment?	407
defun scanComment	407
defun lfcomment	408
defun Is it a – comment?	408
defun scanNegComment	408
defun lfnegcomment	409

defun punctuation?	409
defun scanPunct	409
defun subMatch	410
defun substringMatch	410
defun scanKeyTr	411
defun keyword	412
defun keyword?	412
defun scanPossFloat	412
defun digit?	413
defun lfkey	413
defun splel	413
defun splel1	414
defun scanEsc	414
defvar scanCloser	416
defun scanCloser?	417
defun scanWord	417
defun scanExponent	417
defun lfloat	419
defmacro idChar?	419
defun scanW	419
defun posend	420
defun scanSpace	420
defun lspaces	421
defun scanString	421
defun lfstring	421
defun scanS	422
defun scanTransform	423
defun scanNumber	423
defun rdigit?	424
defun lfinteger	425
defun lfrinteger	425
defun scanCheckRadix	425
defun scanEscape	426
defun scanError	426
defun lerror	427
defvar scanKeyTable	427
defun scanKeyTableCons	427
defvar scanDict	428
defun scanDictCons	428
defun scanInsert	429
defvar scanPun	430
defun scanPunCons	430

14 Input Stream Parser	433
defun Input Stream Parser	433
defun npItem	434
defun npItem1	434
defun npFirstTok	435
defun Push one item onto \$stack	435
defun Pop one item off \$stack	436
defun Pop the second item off \$stack	436
defun Pop the third item off \$stack	436
defun npQualDef	437
defun Advance over a keyword	437
defun Advance the input stream	437
defun npComma	438
defun npTuple	438
defun npCommaBackSet	438
defun npQualifiedDefinition	439
defun npQualified	439
defun npDefinitionOrStatement	440
defun npBackTrack	440
defun npGives	440
defun npLambda	441
defun npType	441
defun npMatch	442
defun npSuch	442
defun npWith	442
defun npCompMissing	443
defun npMissing	443
defun npRestore	444
defun Peek for keyword s, no advance of token stream	444
defun npCategoryL	445
defun npCategory	445
defun npSCategory	445
defun npSignature	446
defun npSigItemlist	446
defun npListing	447
defun Always produces a list, fn is applied to it	447
defun npSigItem	448
defun npTypeVariable	448
defun npSignatureDefinee	449
defun npTypeVariablelist	449
defun npSigDecl	449
defun npPrimary	450
defun npPrimary2	450
defun npADD	450
defun npAdd	451
defun npAtom2	452

defun npInfixOperator	452
defun npInfixOp	453
defun npPrefixColon	453
defun npApplication	454
defun npDotted	454
defun npAnyNo	455
defun npSelector	455
defun npApplication2	455
defun npPrimary1	456
defun npMacro	456
defun npMdef	457
defun npMDEF	457
defun npMDEFinition	458
defun npFix	458
defun npLet	458
defun npLetQualified	459
defun npDefinition	459
defun npDefinitionItem	460
defun npTyping	460
defun npDefaultItemList	461
defun npSDefaultItem	461
defun npDefaultItem	462
defun npDefaultDecl	462
defun npStatement	463
defun npExport	463
defun npLocalItemList	464
defun npSLocalItem	464
defun npLocalItem	465
defun npLocalDecl	465
defun npLocal	465
defun npFree	466
defun npInline	466
defun npIterate	467
defun npBreak	467
defun npLoop	467
defun npIterators	468
defun npIterator	468
defun npSuchThat	469
defun Apply argument 0 or more times	469
defun npWhile	470
defun npForIn	470
defun npReturn	470
defun npVoid	471
defun npExpress	471
defun npExpress1	472
defun npConditionalStatement	472

defun npImport	473
defun npQualTypelist	473
defun npSQualTypelist	473
defun npQualType	474
defun npAndOr	474
defun npEncAp	474
defun npEncl	475
defun npAtom1	475
defun npPDefinition	476
defun npDollar	476
defun npConstTok	476
defun npBDefinition	477
defun npBracketed	478
defun npParened	478
defun npBracked	478
defun npBraced	479
defun npAngleBared	479
defun npDefn	479
defun npDef	480
defun npBPileDefinition	480
defun npPileBracketed	481
defun npPileDefinitionlist	481
defun npListAndRecover	482
defun npRecoverTrap	483
defun npMoveTo	483
defun syIgnoredFromTo	484
defun syGeneralErrorHere	484
defun sySpecificErrorHere	484
defun sySpecificErrorAtToken	485
defun npDefinitionlist	485
defun npSemiListing	485
defun npSemiBackSet	486
defun npRule	486
defun npSingleRule	486
defun npDefTail	487
defun npDefaultValue	487
defun npWConditional	487
defun npConditional	488
defun npElse	488
defun npBacksetElse	489
defun npLogical	489
defun npDisjand	490
defun npDiscrim	490
defun npQuiver	490
defun npRelation	491
defun npSynthetic	491

defun npBy	492
defun	492
defun npSegment	493
defun npArith	493
defun npSum	493
defun npTerm	493
defun npRemainder	494
defun npProduct	494
defun npPower	495
defun npAmpersandFrom	495
defun npFromdom	495
defun npFromdom1	496
defun npAmpersand	496
defun npName	496
defvar \$npTokToNames	497
defun npId	497
defun npSymbolVariable	498
defun npRightAssoc	498
defun p o p o p o p = (((p o p) o p) o p)	499
defun npInfGeneric	500
defun npDDInfKey	500
defun npInfKey	501
defun npPushId	501
defvar npPParg	502
defun npPP	502
defun npPPff	503
defun npPPg	503
defun npPPf	503
defun npEnclosed	504
defun npState	504
defun npTrap	505
defun npTrapForm	505
defun npVariable	505
defun npVariablelist	506
defun npVariableName	506
defun npDecl	507
defun npParenthesized	507
defun npParenthesize	507
defun npMissingMate	508
defun npExit	508
defun npPileExit	508
defun npAssign	509
defun npAssignment	509
defun npAssignVariable	510
defun npColon	510
defun npTagged	510

defun npTypedForm1	511
defun npTypified	511
defun npTypeStyle	511
defun npPretend	512
defun npColonQuery	512
defun npCoerceTo	512
defun npTypedForm	513
defun npRestrict	513
defun npListofFun	513
14.1 Functions on interpreter objects	514
defmacro mkObj	514
defmacro mkObjWrap	515
defmacro mkObjCode	515
defmacro objSetVal	515
defmacro objSetMode	516
defmacro objVal	516
defmacro objValUnwrap	516
defmacro objMode	516
defun objEnv	517
defmacro objCodeVal	517
defmacro objCodeMode	517
14.2 Macro handling	517
defun phMacro	517
defun macroExpanded	518
defun macExpand	518
defun macApplication	519
defun mac0MLambdaApply	520
defun mac0ExpandBody	521
defun mac0InfiniteExpansion	521
defun mac0InfiniteExpansion,name	522
defun mac0GetName	522
defun macId	523
defun mac0Get	524
defun macWhere	524
defun macWhere,mac	524
defun macLambda	525
defun macLambda,mac	525
defun Add appropriate definition the a Macro pform	525
defun Add a macro to the global pfMacros list	526
defun macSubstituteOuter	527
defun mac0SubstituteOuter	527
defun macLambdaParameterHandling	527
defun macSubstituteId	528

15 Pftrees	531
15.1 Abstract Syntax Trees Overview	531
15.2 Structure handlers	533
defun pfGlobalLinePosn	533
defun pfCharPosn	533
defun pfLinePosn	533
defun pfFileName	534
defun pfCopyWithPos	534
defun pfMapParts	534
defun pf0ApplicationArgs	535
defun pf0FlattenSyntacticTuple	535
defun pfSourcePosition	536
defun Convert a Sequence node to a list	536
defun pfSpread	537
defun Deconstruct nodes to lists	537
defun pfCheckMacroOut	538
defun pfCheckArg	539
defun pfCheckId	539
defun pfFlattenApp	539
defun pfCollect1?	540
defun pfCollectVariable1	540
defun pfPushMacroBody	541
defun pfSourceStok	541
defun pfTransformArg	542
defun pfTaggedToTyped1	542
defun pfSuch	542
15.3 Special Nodes	543
defun Create a Listof node	543
defun pfNothing	543
defun Is this a Nothing node?	543
15.4 Leaves	544
defun Create a Document node	544
defun Construct an Id node	544
defun Is this an Id node?	544
defun Construct an Id leaf node	544
defun Return the Id part	545
defun Construct a Leaf node	545
defun Is this a leaf node?	545
defun Return the token position of a leaf node	546
defun Return the Leaf Token	546
defun Is this a Literal node?	546
defun Create a LiteralClass node	546
defun Return the LiteralString	547
defun Return the parts of a tree node	547
defun Return the argument unchanged	547
defun pfPushBody	547

defun An S-expression which people can read	548
defun Create a human readable S-expression	548
defun Construct a Symbol or Expression node	549
defun Construct a Symbol leaf node	549
defun Is this a Symbol node?	550
defun Return the Symbol part	550
15.5 Trees	550
defun Construct a tree node	550
defun Construct an Add node	550
defun Construct an And node	551
defun pfAttribute	551
defun Return an Application node	551
defun Return the Arg part of an Application node	552
defun Return the Op part of an Application node	552
defun Is this an And node?	552
defun Return the Left part of an And node	552
defun Return the Right part of an And node	553
defun Flatten a list of lists	553
defun Is this an Application node?	553
defun Create an Assign node	553
defun Is this an Assign node?	554
defun Return the parts of an LhsItem of an Assign node	554
defun Return the LhsItem of an Assign node	554
defun Return the RHS of an Assign node	554
defun Construct an application node for a brace	555
defun Construct an Application node for brace-bars	555
defun Construct an Application node for a bracket	555
defun Construct an Application node for bracket-bars	555
defun Create a Break node	556
defun Is this a Break node?	556
defun Return the From part of a Break node	556
defun Construct a Coerceto node	557
defun Is this a CoerceTo node?	557
defun Return the Expression part of a CoerceTo node	557
defun Return the Type part of a CoerceTo node	557
defun Return the Body of a Collect node	558
defun Return the Iterators of a Collect node	558
defun Create a Collect node	558
defun Is this a Collect node?	558
defun pfDefinition	559
defun Return the Lhs of a Definition node	559
defun Return the Rhs of a Definition node	559
defun Is this a Definition node?	559
defun Return the parts of a Definition node	560
defun Create a Do node	560
defun Is this a Do node?	560

defun Return the Body of a Do node	560
defun Construct a Sequence node	561
defun Construct an Exit node	561
defun Is this an Exit node?	561
defun Return the Cond part of an Exit	561
defun Return the Expression part of an Exit	562
defun Create an Export node	562
defun Construct an Expression leaf node	562
defun pfFirst	562
defun Create an Application Fix node	563
defun Create a Free node	563
defun Is this a Free node?	563
defun Return the parts of the Items of a Free node	564
defun Return the Items of a Free node	564
defun Construct a ForIn node	564
defun Is this a ForIn node?	564
defun Return all the parts of the LHS of a ForIn node	565
defun Return the LHS part of a ForIn node	565
defun Return the Whole part of a ForIn node	565
defun pfFromDom	565
defun Construct a Fromdom node	566
defun Is this a Fromdom mode?	566
defun Return the What part of a Fromdom node	566
defun Return the Domain part of a Fromdom node	567
defun Construct a Hide node	567
defun pfIf	567
defun Is this an If node?	567
defun Return the Cond part of an If	568
defun Return the Then part of an If	568
defun pfIfThenOnly	568
defun Return the Else part of an If	568
defun Construct an Import node	569
defun Construct an Iterate node	569
defun Is this an Iterate node?	569
defun Handle an infix application	569
defun Create an Inline node	570
defun pfLam	570
defun pfLambda	571
defun Return the Body part of a Lambda node	571
defun Return the Rets part of a Lambda node	571
defun Is this a Lambda node?	571
defun Return the Args part of a Lambda node	572
defun Return the Args of a Lambda Node	572
defun Construct a Local node	572
defun Is this a Local node?	572
defun Return the parts of Items of a Local node	573

defun Return the Items of a Local node	573
defun Construct a Loop node	573
defun pfLoop1	573
defun Is this a Loop node?	574
defun Return the Iterators of a Loop node	574
defun pf0LoopIterators	574
defun pfLp	574
defun Create a Macro node	575
defun Is this a Macro node?	575
defun Return the Lhs of a Macro node	575
defun Return the Rhs of a Macro node	575
defun Construct an MLambda node	576
defun Is this an MLambda node?	576
defun Return the Args of an MLambda	576
defun Return the parts of an MLambda argument	576
defun pfMLambdaBody	577
defun Is this a Not node?	577
defun Return the Arg part of a Not node	577
defun Construct a NoValue node	577
defun Is this a Novalue node?	578
defun Return the Expr part of a Novalue node	578
defun Construct an Or node	578
defun Is this an Or node?	578
defun Return the Left part of an Or node	579
defun Return the Right part of an Or node	579
defun Return the part of a parenthesised expression	579
defun pfPretend	579
defun Is this a Pretend node?	580
defun Return the Expression part of a Pretend node	580
defun Return the Type part of a Pretend node	580
defun Construct a QualType node	580
defun Construct a Restrict node	581
defun Is this a Restrict node?	581
defun Return the Expr part of a Restrict node	581
defun Return the Type part of a Restrict node	581
defun Construct a RetractTo node	582
defun Construct a Return node	582
defun Is this a Return node?	582
defun Return the Expr part of a Return node	582
defun pfReturnNoName	583
defun Construct a ReturnTyped node	583
defun Construct a Rule node	583
defun Return the Lhs of a Rule node	584
defun Return the Rhs of a Rule node	584
defun Is this a Rule node?	584
defun pfSecond	584

defun Construct a Sequence node	585
defun Return the Args of a Sequence node	585
defun Is this a Sequence node?	585
defun Return the parts of the Args of a Sequence node	585
defun Create a Suchthat node	586
defun Is this a SuchThat node?	586
defun Return the Cond part of a SuchThat node	586
defun Create a Tagged node	586
defun Is this a Tagged node?	587
defun Return the Expression portion of a Tagged node	587
defun Return the Tag of a Tagged node	587
defun pfTaggedToTyped	587
defun pfTweakIf	588
defun Construct a Typed node	588
defun Is this a Typed node?	589
defun Return the Type of a Typed node	589
defun Return the Id of a Typed node	589
defun Construct a Typing node	589
defun Return a Tuple node	590
defun Return a Tuple from a List	590
defun Is this a Tuple node?	590
defun Return the Parts of a Tuple node	591
defun Return the parts of a Tuple	591
defun Return a list from a Sequence node	591
defun The comment is attached to all signatutres	591
defun Construct a WDeclare node	592
defun Construct a Where node	592
defun Is this a Where node?	592
defun Return the parts of the Context of a Where node	593
defun Return the Context of a Where node	593
defun Return the Expr part of a Where node	593
defun Construct a While node	593
defun Is this a While node?	594
defun Return the Cond part of a While node	594
defun Construct a With node	594
defun Create a Wrong node	594
defun Is this a Wrong node?	595
 16 Pftree to s-expression translation	 597
defun Pftree to s-expression translation	597
defun Pftree to s-expression translation inner function	598
defun Convert a Literal to an S-expression	602
defun Convert a float to an S-expression	603
defun Change an Application node to an S-expression	603
defun Convert a SuchThat node to an S-expression	605
defun pfOp2Sex	606

defun pmDontQuote?	607
defun hasOptArgs?	607
defun Convert a Sequence node to an S-expression	608
defun pfSequence2Sex0	608
defun Convert a loop node to an S-expression	609
defun Change a Collect node to an S-expression	612
defun Convert a Definition node to an S-expression	613
defun Convert a Lambda node to an S-expression	614
defun pfCollectArgTran	615
defun Convert a Lambda node to an S-expression	616
defun Convert a Rule node to an S-expression	616
defun Convert the Lhs of a Rule to an S-expression	617
defun Convert the Rhs of a Rule to an S-expression	617
defun Convert a Rule predicate to an S-expression	617
defun patternVarsOf	619
defun patternVarsOf1	619
defun pvarPredTran	620
defun Convert the Lhs of a Rule node to an S-expression	620
defun Translate ops into internal symbols	621
17 Stream Utilities	623
defun npNull	623
defun StreamNull	623
18 Code Piles	625
defun insertpile	625
defun pilePlusComment	626
defun pilePlusComments	626
defun pileTree	627
defun pileColumn	627
defun pileForests	627
defun pileForest	628
defun pileForest1	628
defun eqpileTree	629
defun pileCtree	630
defun pileCforest	630
defun enPile	630
defun firstTokPosn	631
defun lastTokPosn	631
defun separatePiles	631
19 Dequeue Functions	633
defun dqUnit	633
defun dqConcat	633
defun dqAppend	634
defun dqToList	634

20 Message Handling	635
20.1 The Line Object	635
defun Line object creation	635
defun Line element 0; Extra blanks	635
defun Line element 1; String	635
defun Line element 2; Global number	636
defun Line element 2; Set Global number	636
defun Line element 3; Local number	636
defun Line element 4; Place of origin	636
defun Line element 4: Is it a filename?	637
defun Line element 4: Is it a filename?	637
defun Line element 4; Get filename	637
20.2 Messages	637
defun msgCreate	637
defmacro getMsgPosTagOb	638
defmacro getMsgKey	638
defmacro getMsgArgL	639
defmacro getMsgPrefix	639
defmacro setMsgPrefix	639
defmacro getMsgText	639
defmacro setMsgText	640
defmacro getMsgPrefix?	640
defmacro getMsgTag	640
defmacro getMsgTag?	640
defmacro line?	641
defmacro leader?	641
defmacro toScreen?	641
defun ncSoftError	642
defun ncHardError	642
defun desiredMsg	643
defun processKeyedError	643
defun msgOutputter	643
defun listOutputter	644
defun getStFromMsg	644
defvar \$preLength	645
defun getPreStL	645
defun getPosStL	646
defun ppos	647
defun remFile	647
defun showMsgPos?	648
defvar \$imPrGuys	648
defun msgImPr?	648
defun getMsgCatAttr	648
defun getMsgPos	649
defun getMsgFTTag?	649
defun decideHowMuch	649

defun poNopos?	650
defun poPosImmediate?	650
defun poFileName	650
defun poGetLineObject	651
defun poLinePosn	651
defun listDecideHowMuch	651
defun remLine	652
defun getMsgKey?	652
defun tabbing	652
defvar \$toWhereGuys	653
defun getMsgToWhere	653
defun toFile?	653
defun alreadyOpened?	653
defun setMsgForcedAttrList	654
defun setMsgForcedAttr	654
defvar \$attrCats	654
defun whichCat	655
defun setMsgCatlessAttr	655
defun putDatabaseStuff	655
defun getMsgInfoFromKey	656
defun setMsgUnforcedAttrList	656
defun setMsgUnforcedAttr	657
defvar \$imPrTagGuys	657
defun initImPr	657
defun initToWhere	658
defun Report a bug in the compiler	658
defun processMsgList	659
defun erMsgSort	659
defun erMsgCompare	660
defun compareposns	660
defun erMsgSep	660
defun makeMsgFromLine	661
defun rep	661
defun getLinePos	662
defun getLineText	662
defun queueUpErrors	662
defun thisPosIsLess	664
defun thisPosIsEqual	664
defun redundant	664
defvar \$repGuys	665
defun msgNoRep?	665
defun sameMsg?	666
defun processChPosesForOneLine	666
defun poCharPosn	667
defun makeLeaderMsg	667
defun posPointers	668

defun getMsgPos2	668
defun insertPos	669
defun putFTText	669
defun From	670
defun To	670
defun FromTo	670
21 The Interpreter Syntax	673
21.1 syntax assignment	673
21.2 syntax blocks	676
21.3 system clef	678
21.4 syntax collection	679
21.5 syntax for	681
21.6 syntax if	685
21.7 syntax iterate	687
21.8 syntax leave	688
21.9 syntax parallel	689
21.10 syntax repeat	692
21.11 syntax suchthat	696
21.12 syntax syntax	697
21.13 syntax while	697
22 Abstract Syntax Trees (ptrees)	701
defun Construct a leaf token	701
defun Return a part of a node	702
defun Compare a part of a node	702
defun pfNoPosition?	702
defun poNoPosition?	703
defun tokType	703
defun tokPart	703
defun tokPosn	703
defun pfNoPosition	704
defun poNoPosition	704
23 Attributed Structures	705
defun ncTag	705
defun ncAlist	705
defun ncEltQ	706
defun ncPutQ	706
Special Category Names	708
defvar \$EmptyMode	708
defvar \$AnonymousFunction	708
defvar \$Any	708
defvar \$BFtag	708
defvar \$Boolean	709
defvar \$Category	709

defvar \$Domain	709
defvar \$Exit	709
defvar \$Expression	709
defvar \$OutputForm	710
defvar \$BigFloat	710
defvar \$Float	710
defvar \$DoubleFloat	710
defvar \$FontTable	711
defvar \$Integer	711
defvar \$ComplexInteger	711
defvar \$Mode	711
defvar \$NegativeInteger	711
defvar \$NonNegativeInteger	712
defvar \$NonPositiveInteger	712
defvar \$PositiveInteger	712
defvar \$RationalNumber	712
defvar \$String	713
defvar \$StringCategory	713
defvar \$Symbol	713
defvar \$Void	713
defvar \$QuotientField	713
defvar \$FunctionalExpression	714
defvar \$defaultFunctionTargets	714
defvar \$SmallInteger	714
defvar \$SingleFloat	714
defvar \$DoubleFloat	715
defvar \$SingleInteger	715
24 Function Selection	717
defun ofCategory	717
defun isPartialMode	718
defun hasCaty	718
defun domArg	720
defun domArg2	720
defun hasSig	721
defun hasAtt	722
defun hasSigAnd	723
defun hasSigOr	724
defun hasAttSig	724
defun hasCate1	725
defun hasCatExpression	725
defun unifyStruct	726
defun unifyStructVar	727
defun containsVars	729
defun isPatternVar	729
defun containsVars1	729

defun hasCaty1	730
defun mkDomPvar	731
defun hasCate	731
defun constructSubst	732
defun hasCateSpecial	733
defun hasCateSpecialNew	734
defun defaultTargetFE	735
defun isEqualOrSubDomain	736
25 Coercions	739
defun coerceInteractive	740
defun coerceInt	741
defun coerceInt0	742
defun coerceInt1	743
defun coerceByFunction	748
defun coerceIntTower	750
defun coerceIntTest	752
defvar coerceConvertMmSelection;AL	752
defun coerceConvertMmSelection	752
defun hasCorrectTarget	753
defun coerceIntPermute	754
defun computeTTTranspositions	754
defun permuteToOrder	756
defun decomposeTypeIntoTower	756
defun reassembleTowerIntoType	757
defun coerceIntCommute	757
defun coerceCommuteTest	758
defun coerceIntTableOrFunction	759
defun coerceByTable	759
defun catchCoerceFailure	760
defun coerceIntSpecial	760
defun coerceIntByMap	761
defun coerceIntByMapInner	763
defun coerceOrThrowFailure	763
defun coercionFailure	763
defun valueArgsEqual?	764
defun algEqual	764
defun coerceIntFromUnion	765
defun coerceInt2Union	765
defun coerceBranch2Union	766
defun coerceIntAlgebraicConstant	766
defun getConstantFromDomain	767
defun compareTypeLists	768
defun coerceIntX	768
defun coerceSubDomain	769
defun coerceImmediateSubDomain	769

defun getSubDomainPredicate	769
defun absolutelyCanCoerceByCheating	770
defun coerceOrRetract	771
defun retract2Specialization	772
defun coerceUnion2Branch	775
defun stripUnionTags	776
defun evalSharpOne	776
defun retractUnderDomain	777
defun coerceRetract	777
defun retractByFunction	778
26 System Command Handling	781
26.1 Variables Used	783
defvar \$systemCommands	783
defvar \$syscommands	784
defvar \$noParseCommands	784
26.2 Functions	785
defun handleNoParseCommands	785
defun Handle a top level command	786
defun Split block into option block	787
defun Tokenize a system command	787
defun Handle system commands	788
defun Select commands matching this user level	788
defun No command begins with this string	789
defun No option begins with this string	789
defvar \$oldline	789
defun No command/option begins with this string	789
defun Option not available at this user level	790
defun Command not available at this user level	790
defun Command not available error message	790
defun satisfiesUserLevel	791
defun hasOption	791
defun terminateSystemCommand	792
defun Terminate a system command	792
defun commandAmbiguityError	792
defun getParserMacroNames	793
defun clearParserMacro	793
defun displayMacro	794
defun displayWorkspaceNames	794
defun getWorkspaceNames	795
defun fixObjectForPrinting	795
defun displayProperties,sayFunctionDeps	796
defun displayValue	799
defun displayType	800
defun getAndSay	800
defun displayProperties	801

defun displayParserMacro	804
defun displayCondition	804
defun interpFunctionDepAlists	805
defun displayModemap	806
defun displayMode	806
defun Split into tokens delimited by spaces	807
defun Convert string tokens to their proper type	807
defun Is the argument string an integer?	808
defun Handle parsed system commands	808
defun Parse a system command	808
defun Get first word in a string	809
defun Unabbreviate keywords in commands	809
defun The command is ambiguous error	810
defun Remove the spaces surrounding a string	811
defun Remove the lisp command prefix	811
defun Handle the)lisp command	812
defun The)boot command is no longer supported	812
defun Handle the)system command	812
defun Handle the)synonym command	813
defun Handle the synonym system command	813
defun printSynonyms	813
defun Print a list of each matching synonym	814
defvar \$tokenCommands	815
defvar \$InitialCommandSynonymAlist	816
defun Print the current version information	816
defvar \$CommandSynonymAlist	817
defun nloopCommand	818
defun nloopPrefix?	818
defun selectOptionLC	819
defun selectOption	819
26.3)abbreviations Command	821
abbreviations man page	821
defun abbreviations	822
defun abbreviationsSpad2Cmd	822
defun listConstructorAbbreviations	824
26.4)boot Command	826
boot man page	826
26.5)browse Command	827
browse man page	827
26.6 Overview	827
26.7 Browsers, MathML, and Fonts	828
26.8 The axServer/multiServ loop	829
26.9 The)browse command	829
26.10 The server support code	831
26.11)cd Command	832
cd man page	832

26.12	)clear Command	833
	clear man page	833
	defvar \$clearOptions	834
	defun clear	835
	defvar \$clearExcept	835
	defun clearSpad2Cmd	835
	defun clearCmdSortedCaches	837
	defun compiledLookupCheck	837
	defvar \$functionTable	838
	defun clearCmdCompletely	838
	defun clearCmdAll	839
	defun clearMacroTable	840
	defun clearCmdExcept	840
	defun clearCmdParts	841
26.13	)close Command	844
	close man page	844
	defun queryClients	845
	defun close	845
26.14	)compile Command	847
	compile man page	847
	defvar /editfile	849
26.15	)copyright Command	850
	copyright man page	850
	defun copyright	855
	defun trademark	855
26.16	)credits Command	856
	credits man page	856
	defun credits	856
26.17	)describe Command	857
	describe man page	857
	defvar \$describeOptions	857
	defun Print comment strings from algebra libraries	858
	defun describeSpad2Cmd	858
	defun cleanline	859
	defun flatten	861
26.18	)display Command	862
	display man page	862
	defvar \$displayOptions	863
	defun display	864
	displaySpad2Cmd	864
	defun abbQuery	865
	defun displayOperations	866
	defun yesanswer	866
	defun displayMacros	867
	defun sayExample	868
	defun cleanupLine	869

26.19)edit Command	871
edit man page	871
defun edit	872
defun editSpad2Cmd	872
defun Implement the)edit command	873
defun updateSourceFiles	874
26.20)fin Command	876
fin man page	876
defun Exit from the interpreter to lisp	876
26.21)help Command	878
help man page	878
The top level help command	880
The top level help command handler	881
defun newHelpSpad2Cmd	881
26.22)history Command	883
history man page	883
26.23 Initialized history variables	886
defvar \$oldHistoryFileName	886
defvar \$historyFileType	886
defvar \$historyDirectory	887
defvar \$useInternalHistoryTable	887
defun makeHistFileName	887
defun oldHistFileName	887
defun histFileName	888
defun histInputFileName	888
defun initHist	888
defun initHistList	889
The top level history command	889
The top level history command handler	890
defun showHistory	892
defun setHistoryCore	894
defvar \$underbar	896
defun writeInputLines	896
defun resetInCoreHist	898
defun changeHistListLen	898
defun updateHist	899
defun updateInCoreHist	900
defun putHist	900
defun recordNewValue	901
defun recordNewValue0	901
defun recordOldValue	901
defun recordOldValue0	902
defun undoInCore	902
defun undoChanges	903
defun undoFromFile	904
defun saveHistory	905

defun restoreHistory	907
defun setIOindex	909
defun showInput	909
defun showInOut	910
defun fetchOutput	910
Read the history file using index n	911
Write information of the current step to history file	912
Disable history if an error occurred	913
defun writeHistModesAndValues	913
defun spadwrite0	914
defun Random write to a stream	914
defun spadwrite	915
defun spadread	915
defun Random read a key from a stream	915
defun unwritable?	916
defun writifyComplain	916
defun safeWritify	917
defun writify,writifyInner	917
defun writify	920
defun spadClosure?	921
defvar \$NonNullStream	921
defvar \$NullStream	922
defun dewritify,dewritifyInner	922
defun dewritify	925
defun ScanOrPairVec,ScanOrInner	925
defun ScanOrPairVec	926
defun gensymInt	927
defun charDigitVal	927
defun histFileErase	928
26.24)include Command	929
include man page	929
defun nclloopInclude1	929
Returns the first non-blank substring of the given string	929
Open the include file and read it in	930
Return the include filename	930
Return the next token	930
26.25)library Command	932
library man page	932
26.26)license Command	934
license man page	934
defun license	934
26.27)lisp Command	935
lisp man page	935
26.28)ltrace Command	936
ltrace man page	936
defun The top level)ltrace function	936

26.29	)pquit Command	937
	pquit man page	937
	The top level pquit command	938
	The top level pquit command handler	938
26.30	)quit Command	939
	quit man page	939
	The top level quit command	940
	The top level quit command handler	940
	Leave the Axiom interpreter	941
26.31	)read Command	942
	read man page	942
	defun The)read command	943
	defun Implement the)read command	943
	defun /read	944
26.32	)regress Command	946
	regress man page	946
	The regress function details	949
	defvar *all-tests-ran*	950
	defun Scan a spool output file for failures	950
	defun Parse test name from the spool command	951
	defun Find the next -S marker	952
	defun Parse out the test number from -S lines	952
	defvar *ok*	952
	defun Compare the computed and expected results	953
	defun Split the calculated and expect results into lists	953
	defun Returns true on -S lines	955
	defun Returns true on -E lines	955
	defun Returns true on -R lines	956
	defun Returns true on -I lines	956
	defun Check the last -S line ran	956
26.33	)savesystem Command	958
	savesystem man page	958
	defvar *ThisIsARunningSystem*	959
	defun The)savesystem command	959
26.34	)set Command	960
	set man page	960
	Overview	961
	Initialize the set variables	962
	Reset the workspace variables	962
	Display the set option information	964
	Display the set variable settings	965
	Translate options values to t or nil	967
	Translate t or nil to option values	967
	The list structure	967
26.35	set breakmode	969
	defvar \$BreakMode	969

26.36	set debug	970
	set debug lambdtype	970
	defvar \$lambdtype	970
26.37	set compiler	971
	set compiler output	971
	The set output command handler	972
	Describe the set output library arguments	972
	defvar output-library	973
	Open the output library	973
	set compiler input	973
	The set input library command handler	974
	Describe the set input library arguments	975
	Add the input library to the list	975
	defvar input-libraries	975
	Drop an input library from the list	976
26.38	set debug dalymode	976
	defvar dalymode	976
26.39	set expose	977
	functions	978
	functions cache	978
	defvar \$cacheAlist	979
	The top level set functions cache handler	979
	Display a particular cache count	980
	defun insertAlist	981
	Describe the set functions cache	982
	Display all cache counts	982
	Describe the cache counts	983
	functions compile	984
	defvar \$compileRecurrence	985
26.40	set fortran	985
	set ints2floats	986
	defvar \$fortInts2Floats	986
	set fortindent	987
	defvar \$fortIndent	987
	set fortlength	988
	defvar \$fortLength	988
	set typedecs	988
	defvar \$printFortranDecs	989
	set defaulttype	989
	defvar \$defaultFortranType	989
	set precision	990
	defvar \$fortranPrecision	990
	set intrinsic	991
	defvar \$useIntrinsicFunctions	991
	set explength	991
	defvar \$maximumFortranExpressionLength	992

set segment	992
defvar \$fortranSegment	993
set optlevel	993
defvar \$fortranOptimizationLevel	993
set startindex	994
defvar \$fortranArrayStartingIndex	994
set calling	994
defvar \$fortranTmpDir	995
The top level set fortran calling tempfile handler	996
Validate the output directory	997
Describe the set fortran calling tempfile	997
defvar \$fortranDirectory	998
defun setFortDir	998
defun describeSetFortDir	999
defvar \$fortranLibraries	1000
defun setLinkerArgs	1001
defun describeSetLinkerArgs	1001
26.41 set hyperdoc	1002
fullscreen	1002
defvar \$fullScreenSysVars	1002
mathwidth	1003
defvar \$historyDisplayWidth	1003
26.42 set help	1004
fullscreen	1004
defvar \$useFullScreenHelp	1004
26.43 set history	1005
defvar \$HiFiAccess	1005
26.44 set messages	1006
set message any	1007
defvar \$printAnyIfTrue	1007
set message autoload	1008
defvar \$printLoadMsgs	1008
set message bottomup	1009
defvar \$reportBottomUpFlag	1009
set message coercion	1009
defvar \$reportCoerceIfTrue	1010
set message dropmap	1010
defvar \$displayDroppedMap	1011
set message expose	1011
defvar \$giveExposureWarning	1011
set message file	1012
defvar \$printMsgsToFile	1012
set message frame	1013
defvar \$frameMessages	1013
set message highlighting	1013
defvar \$highlightAllowed	1014

set message instant	1014
defvar \$reportInstantiations	1015
set message insteach	1015
defvar \$reportEachInstantiation—	1015
set message interponly	1016
defvar \$reportInterpOnly	1016
set message naglink	1017
defvar \$nagMessages	1017
set message number	1017
defvar \$displayMsgNumber	1018
set message prompt	1018
defvar \$inputPromptType	1019
set message selection	1019
set	1020
defvar \$displaySetValue	1020
set message startup	1021
defvar \$displayStartMsgs	1021
set message summary	1021
defvar \$printStatsSummaryIfTrue	1022
set message testing	1022
defvar \$testingSystem	1023
set message time	1023
defvar \$printTimeIfTrue	1023
set message type	1024
defvar \$printTypeIfTrue	1024
set message void	1025
defvar \$printVoidIfTrue	1025
26.45 set naglink	1025
set naglink host	1026
defvar \$nagHost	1026
defun setNagHost	1027
defun describeSetNagHost	1027
set naglink persistence	1028
defvar \$fortPersistence	1028
defun setFortPers	1029
defun describeFortPersistence	1029
set naglink messages	1030
set naglink double	1030
defvar \$nagEnforceDouble	1031
26.46 set output	1031
set output abbreviate	1032
defvar \$abbreviateTypes	1033
set output algebra	1033
defvar \$algebraFormat	1034
defvar \$algebraOutputFile	1034
defvar \$algebraOutputStream	1035

defun setOutputAlgebra	1035
defun describeSetOutputAlgebra	1037
set output characters	1038
defun setOutputCharacters	1039
set output fortran	1040
defvar \$fortranFormat	1041
defvar \$fortranOutputFile	1041
defun setOutputFortran	1042
defun describeSetOutputFortran	1045
set output fraction	1045
defvar \$fractionDisplayType	1046
set output html	1046
defvar \$htmlFormat	1047
defvar \$htmlOutputFile	1047
defun setOutputHtml	1048
defun describeSetOutputHtml	1050
set output length	1051
defvar \$margin	1051
defvar \$linelength	1051
set output mathml	1052
defvar \$mathmlFormat	1053
defvar \$mathmlOutputFile	1053
defun setOutputMathml	1054
defun describeSetOutputMathml	1056
set output openmath	1057
defvar \$openMathFormat	1057
defvar \$openMathOutputFile	1058
defun setOutputOpenMath	1058
defun describeSetOutputOpenMath	1061
set output script	1061
defvar \$formulaFormat	1062
defvar \$formulaOutputFile	1062
defun setOutputFormula	1063
defun describeSetOutputFormula	1065
set output scripts	1066
defvar \$linearFormatScripts	1067
set output showeditor	1067
defvar \$useEditorForShowOutput	1067
set output tex	1068
defvar \$texFormat	1069
defvar \$texOutputFile	1069
defun setOutputTex	1069
defun describeSetOutputTex	1072
26.47 quit	1073
defvar \$quitCommandType	1073
26.48 streams	1073

set streams calculate	1074
defvar \$streamCount	1074
defun setStreamsCalculate	1075
defun describeSetStreamsCalculate	1075
set streams showall	1076
defvar \$streamsShowAll	1076
26.49 set system	1077
set system functioncode	1077
defvar \$reportCompilation	1078
set system optimization	1078
defvar \$reportOptimization	1078
set system prettyprint	1079
defvar \$prettyprint	1079
26.50 set userlevel	1080
defvar \$UserLevel	1080
defvar \$setOptionNames	1081
26.51 Set code	1081
defun set	1081
defun set1	1082
26.52)show Command	1086
show man page	1086
defun The)show command	1087
defun The internal)show command	1087
defun reportOperations	1089
defun reportOpsFromLisplib0	1091
defun reportOpsFromLisplib1	1092
defun reportOpsFromLisplib	1092
defun isExposedConstructor	1094
defun displayOperationsFromLisplib	1095
defun reportOpsFromUnitDirectly0	1095
defun reportOpsFromUnitDirectly	1096
defun getOplistForConstructorForm	1098
defun getOplistWithUniqueSignatures	1099
defun reportOpsFromUnitDirectly1	1100
defun sayShowWarning	1100
26.53)spool Command	1101
spool man page	1101
26.54)summary Command	1102
summary man page	1102
defun summary	1103
26.55)synonym Command	1104
synonym man page	1104
defun The)synonym command	1105
defun The)synonym command implementation	1105
defun Return a sublist of applicable synonyms	1106
defun Get the system command from the input line	1106

defun Remove system keyword	1107
defun processSynonymLine	1107
26.56)system Command	1108
system man page	1108
26.57)tangle Command	1109
tangle man page	1109
26.58)trademark Command	1111
trademark man page	1111
26.59)undo Command	1112
undo man page	1112
26.60 Evaluation	1113
defun evalDomain	1115
defun mkEvalable	1115
defun mkEvalableUnion	1117
defun isTaggedUnion	1117
defun mkEvalableRecord	1118
defun mkEvalableMapping	1118
defun evaluateType	1118
defun Eval args passed to a constructor	1120
defvar \$noEvalTypeMsg	1122
defun throwEvalTypeMsg	1122
defun makeOrdinal	1123
defun evaluateSignature	1123
defun recordFrame	1123
defun diffAList	1125
defun clearFrame	1128
26.61)what Command	1129
what man page	1129
defvar \$whatOptions	1130
defun what	1131
defun whatSpad2Cmd,fixpat	1131
defun whatSpad2Cmd	1131
defun Show keywords for)what command	1133
defun The)what commands implementation	1133
defun Find all names contained in a pattern	1134
defun Find function of names contained in pattern	1135
defun satisfiesRegularExpressions	1135
defun filterAndFormatConstructors	1136
defun whatConstructors	1137
Display all operation names containing the fragment	1137
26.62)workfiles Command	1139
workfiles man page	1139
defun workfiles	1139
defun workfilesSpad2Cmd	1139

27 Handlers for Special Forms	1143
defun getAndEvalConstructorArgument	1144
defun replaceSharps	1144
defun isDomainValuedVariable	1145
defun evalCategory	1145
28 Handling input files	1147
defun Handle .axiom.input file	1147
defvar boot-line-stack	1147
defvar in-stream	1147
defvar out-stream	1148
defvar file-closed	1148
defvar echo-meta	1148
defvar \$noSubsumption	1148
defvar \$envHashTable	1149
defun Dynamically add bindings to the environment	1149
defun Fetch a property list for a symbol from CategoryFrame	1150
defun Search for a binding in the environment list	1150
defun Search for a binding in the current environment	1150
defun searchTailEnv	1151
29 File Parsing	1153
defun Bind a variable in the interactive environment	1153
defvar line-handler	1153
defvar \$spad-errors	1153
defvar xtokenreader	1154
defun Initialize the spad reader	1154
defun spad-syntax-error	1155
defun spad-long-error	1155
defun spad-short-error	1156
defun spad-error-loc	1156
defun iostat	1156
defun next-lines-show	1157
defun token-stack-show	1157
defun ioclear	1158
defun Set boot-line-stack to nil	1158
30 Handling output	1161
30.1 Special Character Tables	1161
defvar \$defaultSpecialCharacters	1161
defvar \$plainSpecialCharacters0	1162
defvar \$plainSpecialCharacters1	1162
defvar \$plainSpecialCharacters2	1163
defvar \$plainSpecialCharacters3	1163
defvar \$plainRTspecialCharacters	1164
defvar \$RTspecialCharacters	1164

defvar \$specialCharacters	1165
defvar \$specialCharacterAlist	1165
defun Look up a special character code for a symbol	1166
31 Stream and File Handling	1167
defun make-instream	1167
defun make-outstream	1167
defun make-appendstream	1168
defun defiostream	1168
defun shut	1168
defun eofp	1169
defun makeStream	1169
defun Construct a new input file name	1169
defun getDirectoryList	1170
defun probeName	1170
defun makeFullNamestring	1171
defun Replace a file by erase and rename	1171
32 The Spad Server Mechanism	1173
defun openserver	1173
33 Axiom Build-time Functions	1175
defun spad-save	1175
34 Exposure Groups	1177
35 Databases	1179
35.1 Database structure	1179
kaf File Format	1179
Database Files	1180
defstruct database	1182
defvar *defaultdomain-list*	1182
defvar *operation-hash*	1183
defvar *hasCategory-hash*	1183
defvar *miss*	1183
Database streams	1184
defvar *interp-stream*	1184
defvar *interp-stream-stamp*	1184
defvar *operation-stream*	1184
defvar *operation-stream-stamp*	1185
defvar *browse-stream*	1185
defvar *browse-stream-stamp*	1185
defvar *category-stream*	1185
defvar *category-stream-stamp*	1186
defvar *allconstructors*	1186
defvar *allOperations*	1186

defun Reset all hash tables before saving system	1186
defun Preload algebra into saved system	1187
defun Open the interp database	1189
defun Open the browse database	1191
defun Open the category database	1192
defun Open the operations database	1193
defun Add operations from newly compiled code	1193
defun Show all database attributes of a constructor	1194
defun Set a value for a constructor key in the database	1195
defun Delete a value for a constructor key in the database	1195
defun Get constructor information for a database key	1196
defun The <code>)library</code> top level command	1199
defun Read a local filename and update the hash tables	1200
defun Update the database from an <code>nrlib</code> index.kaf file	1201
defun <code>updateDatabase</code>	1203
defvar <code>*sourcefiles*</code>	1204
defun Make new databases	1204
defun <code>saveDependentsHashTable</code>	1208
defun <code>saveUsersHashTable</code>	1208
defun Construct the proper database full pathname	1209
Building the <code>interp.daase</code> from hash tables	1209
defun Write the <code>interp</code> database	1213
Building the <code>browse.daase</code> from hash tables	1215
defun Write the <code>browse</code> database	1215
Building the <code>category.daase</code> from hash tables	1216
defun Write the <code>category</code> database	1216
Building the <code>operation.daase</code> from hash tables	1217
defun Write the <code>operations</code> database	1217
Database support operations	1218
defun Data preloaded into the image at build time	1218
defun Return all constructors	1218
defun Return all operations	1219
36 System Statistics	1221
defun <code>statisticsInitialization</code>	1221
36.1 Lisp Library Handling	1221
defun <code>loadLib</code>	1221
defun <code>isSystemDirectory</code>	1223
defun <code>loadLibNoUpdate</code>	1223
defun <code>loadFuncator</code>	1224
37 Special Lisp Functions	1225
defun <code>compiledLookup</code>	1225
defmacro <code>hashCode?</code>	1225
defun <code>basicLookup</code>	1225
defun <code>lookupInDomainVector</code>	1227

defun basicLookupCheckDefaults	1228
defun oldCompLookup	1229
defun NRTevalDomain	1229
37.1 Axiom control structure macros	1229
defun put	1229
defmacro while	1230
defmacro whileWithResult	1230
37.2 Filename Handling	1230
defun namestring	1230
defun pathnameName	1231
defun pathnameType	1231
defun pathnameTypeId	1231
defun mergePathnames	1231
defun pathnameDirectory	1232
defun Axiom pathnames	1232
defun makePathname	1233
defun Delete a file	1233
defun wrap	1233
defun lotsof	1234
defmacro startsId?	1234
defun hput	1234
defmacro hget	1234
defun hkeys	1235
defun digitp	1235
defun pname	1235
defun size	1236
defun strpos	1236
defun strposl	1236
defmacro identp	1237
defun concat	1237
defun canFuncall?	1237
defun brightprint	1238
defun brightprint-0	1238
defun member	1238
defun messageprint	1239
defun messageprint-1	1239
defun messageprint-2	1239
defun sayBrightly1	1240
defmacro assq	1240
defun A version of GET that works with lists	1240

39 Numeric Function Support	1243
defmacro fracpart	1243
defun list to complex conversion	1243
defun complex to list conversion	1243
defun complex to real conversion	1244
defmacro FloatError	1244
defun Rational approximation to $\Gamma(x)$	1244
defun phiRatapprox	1244
defun Log approximation to $\Gamma(x)$	1245
defun Stirling's approximation to $\Gamma(x)$	1245
defun rgammaImpl	1245
defun gammaRatapprox	1246
defun gammaRatkernel	1247
defun Horner's rule of polynomial evaluation	1247
defun Complex implementation of $\Gamma(z)$	1248
defun Compute the conjugate of $\Gamma(z)$	1248
defun $\Gamma(z)$ negative real branch	1248
defun PiMinusLogSinPi	1249
defun cgammaG	1249
defun logH	1249
defun $\Gamma(z)$ positive real branch	1250
defun cgammaT	1250
defun $\Gamma(z)$ case 2	1250
defun logS	1251
defun Adjust logS if imaginary part is negative	1251
defun $\Gamma(z)$ case 3	1251
defun cgammaBernsum	1252
defun BesselI	1252
defun bessellback	1253
defun Backward recurrence for Bessel functions	1253
defun Compute n terms of the chebychev series for f01	1254
defun chebf01coefmake	1255
defun chebstarevalarr	1256
defun lncgamma	1256
defun rPsiImpl	1257
defun cotdiffeval	1257
defun Amos' w function	1258
defun PsiAsymptoticOrder	1259
defun PsiBack	1259
defvar PsiAsymptoticBern	1260
defun PsiAsymptotic	1260
defun PsiEps	1261
defun PsiIntpart	1261
defun cPsiImpl	1262
defun PsiXotic	1262
defun BesselJ	1262

defun Asymptotic series for BesselJ	1264
defun BesselasympA	1264
defun BesselasympB	1264
defun BesselJRecur	1265
defun BesselJAsymptOrder	1265
defun chebf01	1267
40 Common Lisp Algebra Support	1269
40.1 AlgebraicFunction	1269
defun retract	1269
40.2 Any	1271
defun spad2BootCoerce	1271
40.3 ApplicationProgramInterface	1271
defun Report what domains get instantiated	1271
40.4 Boolean	1272
defun The Boolean = function support	1272
40.5 Char	1272
defun upcase	1272
defun downcase	1272
40.6 ComplexDoubleFloatMatrix	1273
defmacro make-cdouble-matrix	1273
defmacro cdaref2	1273
defmacro cdsetaref2	1273
defmacro cdanrows	1274
defmacro cdancols	1274
40.7 ComplexDoubleFloatVector	1274
defmacro make-cdouble-vector	1275
defmacro cdelt	1275
defmacro cdsetelt	1275
defmacro cdlen	1276
40.8 Database	1276
defun Database elt function support	1276
40.9 DirectProduct	1276
defun vec2list	1276
40.10 DoubleFloat	1277
defmacro DFLessThan	1277
40.11 DoubleFloatSpecialFunctions	1277
defun Real Gamma $\Gamma(x)$	1277
defun Complex Gamma $\Gamma(z)$	1277
defun The complex logGamma function	1278
defun The real logGamma function	1278
defun The real Psi function	1278
defun The complex Psi function	1279
defun The real BesselJ function	1279
defun The complex BesselJ function	1279
defun The real Bessell function	1280

defun The complex BesselI function	1280
defun The complex hypergeometric function	1280
defmacro DFUnaryMinus	1280
defmacro DFMinusp	1281
defmacro DFZerop	1281
defmacro DFAdd	1281
defmacro DFSubtract	1281
defmacro DFMultiply	1282
defmacro DFIntegerMultiply	1282
defmacro DFMax	1282
defmacro DFMin	1282
defmacro DFEql	1283
defmacro DFDivide	1283
defmacro DFIntegerDivide	1283
defmacro DFSqrt	1283
defmacro DFLogE	1284
defmacro DFLog	1284
defmacro DFIntegerExpt	1284
defmacro DFExpt	1284
defmacro DFExp	1285
defmacro DFSin	1285
defmacro DFCos	1285
defmacro DFTan	1285
defmacro DFAasin	1286
defmacro DFAcos	1286
defmacro DFAtan	1286
defmacro DFAtan2	1286
defmacro DFSinh	1287
defmacro DFCosh	1287
defmacro DFTanh	1288
defmacro DFAsinh	1288
defmacro DFAcosh	1288
defmacro DFAtanh	1289
defun Machine specific float numerator	1289
defun Machine specific float denominator	1289
defun Machine specific float sign	1289
defun Machine specific float bit length	1290
defun Decode floating-point values	1290
defun The cotangent routine	1290
defun The inverse cotangent function	1291
defun The secant function	1291
defun The inverse secant function	1291
defun The cosecant function	1292
defun The inverse cosecant function	1292
defun The hyperbolic cosecant function	1292
defun The hyperbolic cotangent function	1292

defun The hyperbolic secant function	1293
defun The inverse hyperbolic cosecant function	1293
defun The inverse hyperbolic cotangent function	1293
defun The inverse hyperbolic secant function	1293
40.12 DoubleFloatMatrix	1294
defmacro make-double-matrix	1294
defmacro make-double-matrix1	1294
defmacro daref2	1294
defmacro dsetaref2	1294
defmacro danrows	1295
defmacro dancols	1295
40.13 DoubleFloatVector	1295
defmacro dlen	1295
defmacro make-double-vector	1295
defmacro make-double-vector1	1296
defmacro delt	1296
defmacro dsetelt	1296
40.14 File	1296
defvar *read-place-holder*	1296
defun placep	1297
defun vmread	1297
40.15 FileName	1297
defun FileName filename function implementation	1297
defun FileName filename support function	1298
defun FileName directory function implementation	1298
defun FileName directory function support	1298
defun FileName name function implementation	1299
defun FileName extension function implementation	1299
defun FileName exists? function implementation	1299
defun FileName readable? function implementation	1299
defun FileName writeable? function implementation	1300
defun FileName writeable? function support	1300
defun FileName new function implementation	1300
40.16 IndexedBits	1301
defmacro truth-to-bit	1301
defun IndexedBits new function support	1301
defmacro bit-to-truth	1301
defmacro bvec-elt	1301
defmacro bvec-setelt	1302
defmacro bvec-size	1302
defun IndexedBits concat function support	1302
defun IndexedBits copy function support	1302
defun IndexedBits = function support	1302
defun IndexedBits < function support	1303
defun IndexedBits And function support	1303
defun IndexedBits Or function support	1303

defun IndexedBits xor function support	1303
defun IndexedBits nand function support	1304
defun IndexedBits nor function support	1304
defun IndexedBits not function support	1304
40.17 IndexCard	1304
defun IndexCard origin function support	1304
defun IndexCard origin function support	1305
defun IndexCard elt function support	1305
40.18 IndexedString	1306
defun qenum	1306
40.19 InputForm	1306
defun called by interpret function	1306
defun called by interpret function	1307
defun called by interpret function	1307
defun unparseInputForm	1307
40.20 Integer	1308
defun Integer divide function support	1308
defun Integer quo function support	1308
defun Integer quo function support	1308
defun Integer random function support	1309
40.21 KeyedAccessFile	1309
defun KeyedAccessFile defstream function support	1309
defun KeyedAccessFile defstream function support	1309
40.22 NumberFormats	1310
defun ncParseFromString	1310
40.23 OperationsQuery	1310
defun OperationQuery getDatabase function support	1310
40.24 ParametricLinearEquations	1311
defun algCoerceInteractive	1311
40.25 Plot3d	1311
defvar \$numericFailure	1311
defvar \$oldBreakMode	1311
defmacro trapNumericErrors	1312
40.26 SingleInteger	1312
defun qsquotient	1312
defun qsremainder	1312
defmacro qsdifference	1312
defmacro qslessp	1313
defmacro qsadd1	1313
defmacro qssub1	1313
defmacro qsminus	1313
defmacro qsplus	1314
defmacro qstimes	1314
defmacro qsabsval	1314
defmacro qsoddp	1314
defmacro qszerop	1315

defmacro qsmax	1315
defmacro qsmin	1315
40.27 Table	1315
defun Table InnerTable support	1315
40.28 U8Vector	1316
defmacro qvlenU8	1316
defmacro eltU8	1316
defmacro seteltU8	1317
defun getRefvU8	1317
40.29 U16Vector	1317
defmacro qvlenU16	1317
defmacro eltU16	1317
defmacro seteltU16	1318
defun getRefvU16	1318
40.30 U32Vector	1318
defmacro qvlenU32	1318
defmacro eltU32	1318
defmacro seteltU32	1319
defun getRefvU32	1319
40.31 U8Matrix	1319
defmacro aref2U8	1319
defmacro setAref2U8	1319
defmacro anrowsU8	1320
defmacro ancotsU8	1320
defmacro makeMatrixU8	1320
defmacro makeMatrix1U8	1320
40.32 U16Matrix	1321
defmacro aref2U16	1321
defmacro setAref2U16	1321
defmacro anrowsU16	1321
defmacro ancotsU16	1321
defmacro makeMatrixU16	1322
defmacro makeMatrix1U16	1322
40.33 U32Matrix	1322
defmacro aref2U32	1322
defmacro setAref2U32	1322
defmacro anrowsU32	1323
defmacro ancotsU32	1323
defmacro makeMatrixU32	1323
defmacro makeMatrix1U32	1323
40.34 U32VectorPolynomialOperations	1324
defmacro qsMulAdd6432	1324
defmacro qsMulMod32	1324
defmacro qsMod6432	1324
defmacro qsMulAddMod6432	1324
defmacro qsMul6432	1325

defmacro qsDot26432	1325
defmacro qsDot2Mod6432	1325
40.35Void	1325
defun voidValue	1325
41 OpenMath	1327
41.1 A Technical Overview	1327
The OpenMath Architecture	1327
OpenMath Encodings	1329
Content Dictionaries	1330
OpenMath in Action	1332
41.2 Technical Details	1333
41.3 The Structure of the API	1333
41.4 OpenMath Expressions	1334
Expressions	1334
Symbols	1334
Encoding and Decoding OpenMath Expressions	1334
41.5 Big Integers	1335
41.6 Functions Dealing with OpenMath Devices	1335
41.7 Functions to Write OpenMath Expressions to Devices	1336
Beginning and Ending Objects	1336
Writing Basic Objects	1337
Writing Structured Objects	1337
41.8 Functions to Extract OpenMath Expressions from Devices	1338
Testing the type of the current token	1338
Extracting the current token	1339
41.9 Comments in the SGML/XML Encodings	1342
41.10 I/O Functions for Devices	1343
41.11 Communications	1343
Functions to Initiate an OMconn	1344
41.12 Parameters	1345
41.13 Miscellaneous Functions and Variables	1345
41.14 The OM.h header file	1346
41.15 Axiom OpenMath stub functions	1355
Axiom specific functions	1355
defun om-Read	1355
defun om-listCDs	1356
defun om-listSymbols	1356
defun om-supportsCD	1356
defun om-supportsSymbol	1356
Lisp conversion functions	1357
defun om-setDevEncoding	1357
Device manipulation functions	1357
defun om-openFileDev	1357
defun om-openStringDev	1358
defun om-closeDev	1358

Connection manipulation functions	1358
defun om-makeConn	1358
defun om-closeConn	1358
defun om-getConnInDev	1359
defun om-getConnOutDev	1359
Client/Server functions	1359
defun om-bindTCP	1359
defun om-connectTCP	1360
Device input/output functions	1360
defun om-getApp	1361
defun om-getAtp	1362
defun om-getAttr	1362
defun om-getBind	1362
defun om-getBVar	1362
defun om-getByteArray	1363
defun om-getEndApp	1363
defun om-getEndAtp	1363
defun om-getEndAttr	1363
defun om-getEndBind	1364
defun om-getEndBVar	1364
defun om-getEndError	1364
defun om-getEndObject	1364
defun om-getError	1365
defun om-getFloat	1365
defun om-getInt	1365
defun om-getObject	1365
defun om-getString	1366
defun om-getSymbol	1366
defun om-getType	1366
defun om-getVar	1366
defun om-putApp	1367
defun om-putAtp	1367
defun om-putAttr	1367
defun om-putBind	1367
defun om-putBVar	1368
defun om-putByteArray	1368
defun om-putEndApp	1368
defun om-putEndAtp	1368
defun om-putEndAttr	1369
defun om-putEndBind	1369
defun om-putEndBVar	1369
defun om-putEndError	1369
defun om-putEndObject	1370
defun om-putError	1370
defun om-putFloat	1370
defun om-putInt	1370

defun om-putObject	1371
defun om-putString	1371
defun om-putSymbol	1371
defun om-putVar	1371
defun om-stringToStringPtr	1372
defun om-stringPtrToString	1372
42 NRLIB code.lisp support code	1373
defun makeByteWordVec2	1373
defmacro spadConstant	1373
43 Monitoring execution	1375
defvar *monitor-domains*	1381
defvar *monitor-nrlibs*	1381
defvar *monitor-table*	1382
defstruct monitor-data	1382
defstruct libstream	1382
defun Initialize the monitor statistics hashtable	1382
defun End the monitoring process, we cannot restart	1383
defun Return a list of the monitor-data structures	1383
defun Add a function to be monitored	1384
defun Remove a function being monitored	1384
defun Enable all (or optionally one) function for monitoring	1384
defun Disable all (optionally one) function for monitoring	1385
defun Reset the table count for the table (or a function)	1385
defun Incr the count of fn by 1	1386
defun Decr the count of fn by 1	1386
defun Return the monitor information for a function	1387
defun Hang a monitor call on all of the defuns in a file	1387
defun Return a list of the functions with zero count fields	1387
defun Return a list of functions with non-zero counts	1388
defun Write out a list of symbols or structures to a file	1388
defun Save the *monitor-table* in loadable form	1389
defun restore a checkpointed file	1389
defun Printing help documentation	1390
Monitoring algebra files	1392
defun Monitoring algebra code.lisp files	1392
defun Monitor autoloaded files	1392
defun Monitor an nrlib	1393
defun Given a monitor-data item, extract the nrlib name	1393
defun Is this an exposed algebra function?	1394
defun Monitor exposed domains	1394
defun Generate a report of the monitored domains	1395
defun Parse an)abbrev expression for the domain name	1396
defun Given a spad file, report all nrlibs it creates	1396
defun Print percent of functions tested	1397

defun Find all monitored symbols containing the string	1397
44 HyperDoc	1399
44.1 Hyperdoc macro handling and util.ht	1399
defvar \$htMacroTable	1399
defvar \$primitiveHtCommands	1400
defvar \$newPage	1401
defun Build the table of hyperdoc macros	1401
defun Get new command name and number of args	1402
defun Is the first string a prefix of the second?	1403
45 HyperDoc Basic Command support	1405
45.1 Calculus	1405
defun Calculus - Differentiate	1406
defun bcDifferentiateGen	1407
defun Calculus - Do an Indefinite Integral	1408
defun bcIndefiniteIntegrateGen	1409
defun Calculus - Do a Definite Integral	1409
defun bcDefiniteIntegrateGen	1411
defun Calculus - Find a limit	1412
defun Calculus - Do a summation	1413
defun bcSumGen	1414
45.2 Matrix	1414
defun Basic Commands - Matrix	1414
45.3 Draw	1415
defun Basic Commands - Draw	1415
defun Draw - Function of one variable	1416
defun bcDraw2DfunGen	1417
defun Draw - Parametrically defined curve	1418
defun bcDraw2DparGen	1419
defun Draw - Solution to a polynomial equation	1420
defun bcDraw2DSolveGen	1421
defun Draw - Function of two variables	1422
defun bcDraw3DfunGen	1424
defun Draw - Parametrically defined tube	1425
defun bcDraw3DparGen	1426
defun Draw - Parametrically defined surface	1427
defun bcDraw3Dpar1Gen	1428
45.4 Series	1429
defun Basic Commands - Series	1429
defun Series - Expansion	1430
defun bcSeriesExpansionGen	1431
defun Series - Formula	1432
defun Series - Formula - Taylor Series	1433
defun bcTaylorSeriesGen	1434
defun bcSeriesGen	1435

defun Series - Formula - Laurent Series	1435
defun bcLaurentSeriesGen	1437
defun Series - Formula - Puiseux Series	1437
defun bcPuiseuxSeriesGen	1438
defun Solve Basic Command	1439
defun Solve - System of Linear Equations	1440
defun System of Linear Equations - Directly as equations	1440
defun bcLinearSolveEqns1	1441
defun System of Linear Equations - In matrix form	1442
defun System of Linear Equations - In matrix form direct	1442
defun Solve System of Linear Equations Individual	1443
defun System of Linear Equations In matrix form by formula	1445
defun Solve - System of Polynomial Equations	1446
defun bcSystemSolveEqns1	1447
defun bcInputSolveInfo	1448
defun Solve - Single Polynomial Equation	1449
46 HyperDoc Reference	1451
46.1 Book	1451
46.2 Topics	1451
Numbers	1451
Polynomials	1451
Functions	1452
Solving Equations	1452
Calculus	1452
Linear Algebra	1452
Graphics	1452
Algebra	1452
46.3 Language	1453
46.4 Examples	1453
46.5 Commands	1453
46.6 Glossary	1453
46.7 Hyperdoc	1453
46.8 Search	1453
47 HyperDoc Topics	1455
48 HyperDoc Browse	1457
49 HyperDoc Examples	1459
50 HyperDoc Settings	1461
51 HyperDoc About	1463
52 HyperDoc What's New	1465

53 HyperDoc Support Functions	1467
53.1 Handling page creation and deletion	1467
defvar \$activePageList	1467
defun httpMakeEmptyPage	1467
defun httpDestroyPage	1468
53.2 Handling Axiom command execution	1468
defun Basic Command result page	1468
defun htMakeDoitButton	1469
defun Execute a command from Hyperdoc	1469
defun Execute a string in the interpreter	1470
53.3 Functions creating pages	1470
defun Basic Command Matrix by Formula generate	1471
defun Basic Command generate explicit matrix	1472
defun Basic Command generate matrix	1472
defun Basic Command iteration	1473
defun Sum Basic Command	1474
defun bcProductGen	1474
defun Read Matrix	1475
defun bcSeriesByFormulaGen	1475
defun Real Limit Basic Command	1475
defun Real Limit Basic Command options	1477
defun bcRealLimitGen1	1478
defun Complex Limit Basic Command	1478
defun bcComplexLimitGen	1480
defvar \$systemType	1480
defvar \$numberOfEquations	1481
defvar \$solutionMethod	1481
defun bcInputEquations	1481
defun Create a variable string	1483
defun bcMakeUnknowns	1483
defun bcMakeEquations	1483
defun bcMakeLinearEquations	1484
defun bcInputEquationsEnd	1485
defun bcSolveEquationsNumerically	1485
defun bcSolveNumerically1	1486
defun bcSolveEquations	1486
defun Linear Solve Basic Command options	1487
defun bcLinearExtractMatrix	1487
defun Linear Solve Basic Command Inhomogeneous	1488
defun bcLinearSolveMatrixInhomoGen	1489
defun bcLinearSolveMatrixHomo	1489
defun bcLinearMatrixGen	1490
defun linearFinalRequest	1490
defun explainLinear	1491
defun finalExactRequest	1491
defun bcLinearSolveEqnsGen	1492

defun bcGenEquations	1492
defun Output the final formula	1493
defun convert arguments into function call syntax	1493
defun bcString2HyString2	1493
defun bcString2HyString	1494
defun find a character position in a string	1494
defun Basic Command result page – NAG version	1494
defun bcOptional	1494
defun create a vertical space on a page	1495
defun break a string into words	1495
defun format words into a string	1495
defun format a vector	1495
defun format an error message	1496
defun format intervals	1496
defun Basic Command page not ready	1496
defun pad a string with blanks	1497
defun construct a name string	1497
defun construct a name string	1497
defvar \$bcParseOnly	1498
defvar \$htLineList	1498
defvar \$curpage	1498
HTPage Layout	1498
defun httpName	1499
defun httpSetName	1499
defun httpDomainConditions	1499
defun httpSetDomainConditions	1499
defun httpDomainVariableAlist	1499
defun httpSetDomainVariableAlist	1500
defun httpDomainPvarSubstList	1500
defun httpSetDomainPvarSubstList	1500
defun httpRadioButtonAlist	1500
defun httpButtonValue	1501
defun httpSetRadioButtonAlist	1501
defun httpInputAreaAlist	1501
defun httpSetInputAreaAlist	1502
defun httpAddInputAreaProp	1502
defun httpPropertyList	1502
defun httpProperty	1502
defun httpSetProperty	1503
defun httpLabelInputString	1503
defun httpLabelFilteredInputString	1503
defun replacePercentByDollar,fn	1504
defun replacePercentByDollar	1504
defun httpSetLabelInputString	1504
defun httpLabelSpadValue	1504
defun httpSetLabelSpadValue	1505

defun httpLabelErrorMsg	1505
defun httpSetLabelErrorMsg	1505
defun httpLabelType	1506
defun httpLabelDefault	1506
defun httpLabelSpadType	1506
defun httpLabelFilter	1507
defun httpPageDescription	1507
defun httpSetPageDescription	1507
defun httpAddToPageDescription	1507
defun issue a single hypertext line or group of lines	1508
defun bcHt	1508
defun htSay	1509
defun bcIssueHt	1509
defun mapStringize	1509
defun basicStringize	1510
defun stringize	1510
defun htInitPage	1510
defun htInitPageNoScroll	1511
defun htSayStandard	1511
defun htSayBind	1512
defun htAddHeading	1512
defun htShowPage	1512
defun show the page which has been computed	1513
defun make a page given the description in itemList	1513
defun htMakePage1	1513
defun htMakeErrorPage	1514
defun htQuote	1515
defun htProcessToggleButtons	1515
defun htProcessBcButtons	1516
defun htProcessBcStrings	1517
defun bcSadFaces	1518
defun htLispLinks	1518
defun htLispMemoLinks	1519
defun htBcLinks	1520
defun htBcLispLinks	1520
defun beforeAfter	1521
defun mkCurryFun	1521
defun htRadioButtons	1522
defun htBcRadioButtons	1523
defun setUpDefault	1525
defun buttonNames	1525
defun htInputStrings	1525
defun htProcessDomainConditions	1527
defun renamePatternVariables	1527
defun renamePatternVariables1	1527
defun substFromAlist	1529

defun computeDomainVariableAlist	1529
defun pvarCondList	1530
defun pvarCondList1	1530
defun pvarsOfPattern	1531
defun htMakeTemplates,substLabel	1532
defun htMakeTemplates	1532
defun templateParts	1533
defun htMakeDoneButton	1533
defun htProcessDoneButton	1534
defun htMakeButton	1534
defun bchtMakeButton	1536
defun htProcessDoitButton	1537
defun htDoneButton	1537
defun typeCheckInputAreas	1538
defun checkCondition	1540
defun condErrorMsg	1541
defun parseAndEval	1542
defun parseAndEval1	1542
defun oldParseString	1543
defun makeSpadCommand	1543
defun htMakeInputList	1543
defun bracketString	1544
defun quoteString	1544
defvar \$funnyQuote	1544
defvar \$funnyBacks	1545
defun htEscapeString	1545
defun htsv	1545
defun htSetVars	1545
defun htShowSetTree	1546
defun htShowCount	1548
defun htShowSetTreeValue	1548
defun mkSetTitle	1549
defun listOfStrings2String	1549
defun htShowSetPage	1549
defun htShowLiteralsPage	1550
defun htSetLiterals	1550
defun htSetLiteral	1551
defun htShowIntegerPage	1552
defun htSetInteger	1553
defun htShowFunctionPage	1553
defun htShowFunctionPageContinued	1554
defun htSetvarDoneButton	1555
defun htFunctionSetLiteral	1555
defun htSetFunCommand	1556
defun htSetFunCommandContinue	1556
defun htKill	1556

defun htSetNotAvailable	1557
defun htDoNothing	1558
defun htCheck	1558
defun parseWord	1558
defun htCheckList	1559
defun translateYesNoToTrueFalse	1560
defun chkNameList	1560
defun chkPosInteger	1561
defun chkOutputFileName	1561
defun chkDirectory	1561
defun chkNonNegativeInteger	1562
defun chkRange	1562
defun chkAllNonNegativeInteger	1562
defun htMakePathKey,fn	1563
defun htMakePathKey	1563
defun htMarkTree	1563
defun htSetHistory	1564
defun htSetOutputLibrary	1564
defun htSetInputLibrary	1564
defun htSetExpose	1565
defun htSetOutputCharacters	1565
defun htSetLinkerArgs	1565
defun htSetCache	1565
defun htCacheAddChoice	1566
defun htMakeLabel	1567
defun htCacheSet	1568
defun htAllOrNum	1569
defun htCacheOne	1569
defvar \$historyDisplayWidth	1570
defvar \$newline	1570
defun downlink	1570
defun dbNonEmptyPattern	1571
defun htSystemVariables,gn	1571
defun htSystemVariables,fn	1572
defun htSystemVariables,displayOptions	1572
defun htSystemVariables,functionTail	1573
defun htSystemVariables	1574
defun htSetSystemVariableKind	1576
defun htSetSystemVariable	1577
defun htGloss	1577
defun htGlossPage	1577
defun gatherGlossLines	1580
defun htGlossSearch	1581
defun htGreekSearch	1582
defun htTextSearch	1584
defun htTutorialSearch	1586

defun mkUnixPattern	1587
54 Browser Support Code	1589
54.1 Pages Initiated from HyperDoc Pages	1589
Search routines	1589
defun dKind	1589
defun checkFilter	1589
defun Concatenate words with blanks	1590
defun Make constructor names lowercase	1590
defun string2Constructor	1590
defvar dbDelimiters	1591
defun String to words respecting delimiters	1591
defun Next word respecting delimiters	1592
defun Hyperdoc category search	1595
defun Hyperdoc default domain search	1595
defun Hyperdoc domain search	1595
defun Hyperdoc package search	1595
defun Hyperdoc constructor search	1596
defun Hyperdoc default constructor search	1596
defun Read libdb.text at file-position n	1596
defun String trim with newlines removed	1596
defun Hyperdoc common constructor search	1597
defun conSpecialString?	1598
Page construction	1599
defun conPage	1599
defun gets line quickly for constructor name or abbreviation	1600
defun conPageConEntry	1600
defun kdPageInfo	1601
defun kArgPage	1602
defun mkDomTypeForm	1603
defun domainDescendantsOf	1603
54.2 Branches of Constructor Page	1604
defun kiPage	1604
defun kePage	1605
defun kePageOpAlist	1607
defun kePageDisplay	1608
defun ksPage	1609
defun dbSearchOrder	1610
defun kcPage	1611
defun kcpPage	1614
defun reduceAlistForDomain	1615
defun kcaPage	1616
defun kcdPage	1616
defun kcdoPage	1617
defun kcaPage1	1617
defun kccPage	1618

defun augmentHasArgs	1619
defun kcdePage	1620
defun getDependentsOfConstructor	1620
defun kcuPage	1621
defun getUsersOfConstructor	1622
defun kcnPage	1622
defun koPageInputAreaUnchanged?	1623
defun kDomainName	1624
defun kArgumentCheck	1625
defun dbMkEvalable	1625
defun topLevelInterpEval	1626
defun kisValidType	1626
defun kCheckArgumentNumbers	1627
defun parseNoMacroFromString	1627
defun mkConform	1628
54.3 Operation Page for a Domain Form from Scratch	1628
defun conOpPage	1628
defun conOpPage1	1629
defun dbCompositeWithMap	1630
defun dbExtractUnderlyingDomain	1631
54.4 Operation Page from Main Page	1631
defun koPage	1631
defun koPageFromKKPage	1632
defun koPageAux	1632
defun koPageAux1	1633
defun koaPageFilterByName	1633
54.5 Get Constructor Documentation	1634
defun dbConstructorDoc,hn	1634
defun dbConstructorDoc,gn	1635
defun dbConstructorDoc	1635
defun dbDocTable	1635
defun originsInOrder	1636
defun dbAddDocTable	1636
defun dbGetDocTable,hn	1637
defun dbGetDocTable,gn	1638
defun dbGetDocTable	1638
defun kTestPred	1639
defun dbAddChainDomain	1640
defun dbSubConform	1640
defun dbAddChain	1641
54.6 Constructor Page Menu	1641
defun dbShowCons	1641
defun conPageChoose	1642
defun dbShowCons1	1643
defun dbConsExposureMessage	1645
defun dbShowConsKindsFilter	1645

defun dbShowConsDoc	1645
defun dbShowConsDoc1	1646
defun getConstructorDocumentation	1647
defun dbSelectCon	1648
defun dbShowConditions	1648
defun dbConsHeading	1649
defun dbShowConstructorLines	1650
defun bcUnixTable	1650
Special Code for Union, Mapping, and Record	1651
defun dbSpecialDescription	1651
defun dbSpecialOperations	1652
defun dbSpecialExports	1652
defun dbSpecialExpandIfNecessary	1653
defun lefts	1658
Build Library Database (libdb.text,...)	1658
defun dbMkForm	1658
defun libConstructorSig	1659
55 Utility functions	1661
55.1 Utility functions	1661
defun Delete an alist pair given the key	1661
defun readline	1661
defun isWrapped	1661
56 The Proofs	1663
57 Signatures	1665
58 The Interpreter	1669
59 The Global Variables	1713
59.1 Star Global Variables	1713
eof	1713
features	1713
package	1713
standard-input	1714
standard-output	1714
top-level-hook	1714
59.2 Dollar Global Variables	1716
\$boot	1717
coerceFailure	1717
\$currentLine	1717
\$displayStartMsgs	1717
\$erMsgToss	1717
\$frameRecord	1717
\$intRestart	1717

\$intTopLevel	1718
\$IOindex	1718
\$lastPos	1718
\$libQuiet	1718
\$msgDatabaseName	1718
\$ncMsgList	1718
\$newcompErrorCount	1718
\$nopus	1719
\$oldHistoryFileName	1719
\$okToExecuteMachineCode	1719
\$options	1719
\$previousBindings	1719
\$reportundo	1719
\$spad	1719
\$SpadServer	1720
\$SpadServerName	1720
\$systemCommandFunction	1720
top_level	1720
\$quitTag	1720
\$useInternalHistoryTable	1720
Signatures	1721
Bibliography	1723
Index	1727

New Foreword

On October 1, 2001 Axiom was withdrawn from the market and ended life as a commercial product. On September 3, 2002 Axiom was released under the Modified BSD license, including this document. On August 27, 2003 Axiom was released as free and open source software available for download from the Free Software Foundation's website, Savannah.

Work on Axiom has had the generous support of the Center for Algorithms and Interactive Scientific Computation (CAISS) at City College of New York. Special thanks go to Dr. Gilbert Baumslag for his support of the long term goal.

The online version of this documentation is roughly 1000 pages. In order to make printed versions we've broken it up into three volumes. The first volume is tutorial in nature. The second volume is for programmers. The third volume is reference material. We've also added a fourth volume for developers. All of these changes represent an experiment in print-on-demand delivery of documentation. Time will tell whether the experiment succeeded.

Axiom has been in existence for over thirty years. It is estimated to contain about three hundred man-years of research and has, as of September 3, 2003, 143 people listed in the credits. All of these people have contributed directly or indirectly to making Axiom available. Axiom is being passed to the next generation. I'm looking forward to future milestones.

With that in mind I've introduced the theme of the "30 year horizon". We must invent the tools that support the Computational Mathematician working 30 years from now. How will research be done when every bit of mathematical knowledge is online and instantly available? What happens when we scale Axiom by a factor of 100, giving us 1.1 million domains? How can we integrate theory with code? How will we integrate theorems and proofs of the mathematics with space-time complexity proofs and running code? What visualization tools are needed? How do we support the conceptual structures and semantics of mathematics in effective ways? How do we support results from the sciences? How do we teach the next generation to be effective Computational Mathematicians?

The "30 year horizon" is much nearer than it appears.

Tim Daly
CAISS, City College of New York
November 10, 2003 ((iHy))

Chapter 1

Type Inference and Coercion

1.1 Introduction

The Axiom system is centered on a strongly typed abstract datatype programming language with multiple inheritance. The compiler for this language produces a library of packages of polymorphic functions and parameterized abstract datatypes. The system interpreter provides an interface to the compiler and library and supports a subset of the Axiom language. The design of the interpreter includes extensive datatype inferences and coercion facilities to ease the burden of working with a strongly typed language and a library containing over 1100 Categories, Domains, and Packages.

The Axiom interpreter is unusual because

- Axiom has an infinite number of datatypes and packages
- each generic operation generally has an infinite number of interpretations
- datatypes and packages are instantiated dynamically in response to user input, and
- operation interpretation is done by pattern matching on modemap.

The design philosophy is that the interpreter language equals the compiler language minus the constraints.

This means that declarations are largely optional and that automatic datatype conversions may take place when they are deemed appropriate.

In Axiom we use the term *conversion* to mean the transformation an object of one datatype into an object of another datatype. A special kind of conversion is *coercion*, where we further require that transformation is reasonable (usually in an algebraic or non-information-losing sense). Examples of coercions involve changing integers into rational numbers, rational numbers into (constant) polynomials with rational number coefficients, lists of lists into two dimensional arrays, etc.

Conversion is done through two operations, **coerce** and **convert** which are otherwise ordinary operations exported by the abstract datatypes of the system. The interpreter is allowed to perform conversions named **coerce** whenever it determines they are necessary. Most coercions are reversible and their application does not lose information. An example of a conversion that is not a coercion is the transformation of a rational number into a floating point number. Having an automatic conversion from an exact to an inexact object is not acceptable.

All atomic expressions (as produced by the parser) have default datatypes. These are **Boolean**, **Float**, **Integer**, **String**, **Symbol**, and **List(None)** (for an empty list). If these are assigned in declared variables or used as arguments to functions, coercions are often performed to create objects of other datatypes.

Coercion is combined with the *type resolve facility* to determine the result datatype of an expression presented to the interpreter. Datatypes **T1** and **T2** resolve to a datatype **T3** if all objects belonging to **T1** and **T2** can be coerced to **T3**. In Axiom, the resolve facility is driven by a rule system, by information about coercions, and by the structure of the datatypes involved.

1.2 Overview of the Abstract Datatype System

It is not within the scope of this chapter to fully discuss the Axiom abstract datatype system. This section is an overview of those terms and concepts needed to describe the implementation of the interpreter. Other sources are available that contain more detailed discussions. [Jenk81][Lisk79][Fort85][Swee86] [Jenk86][Watt87]

The Axiom library consists of compiled code that creates “categories”, “domains”, and “packages”. A *domain* is a datatype, a *package* is a collection of functions, and a *category* is a collection of operation signatures and attributes, as shown below. These are created by special functions called *constructors*, which are generally parameterized.

Domains and packages implement the functions in categories. Categories serve to separate the contract from the implementation. However, default implementations can be given in categories by assuming the existence of other implemented operations. For example, a default implementation of binary “-” can be given by using unary “-” and binary “+”.

Categories

A domain is an instance of an abstract datatype specified by one or more categories. Categories serve to group together domains with common properties (e.g. operations). A basic category in Axiom is **Set** which has the definition:

```
Set(): Category == with
  "=" : ($,$) -> Boolean
  coerce : $ -> Expression
```

The syntax means the following. **Set** is a category constructor having no parameters. Lines 2 and 3 of the definition present *signatures* for the two operations that are exported by **Set**: an operation “=” that takes 2 elements of the set and returns a **Boolean**, and an operation **coerce** that takes an element of the set and returns a printable representation of it. The symbol “\$” refers to the set itself. All domains which belong to **Set** contain “=” and **coerce** among their operations (though it is possible that they are not implemented).

Categories may be extended. A semigroup is a set that has an associative multiplication operation. Given such a multiplication, it is easy to define an exponentiation by positive integers ($x**1 = x$, $x**2 = x*x$, $x**3 = x*x**2, \dots$). The definition of the **SemiGroup** constructor in Axiom is

```
SemiGroup(): Category == Set with
  "*" : ($,$) -> $
  "**" : ($, PositiveInteger) => $
  associate("*")
```

The last line of this definition presents an *attribute* declaring that multiplication is associative. Attributes declare properties of the operations or the datatype.

As an example, **Integer** belongs to *SemiGroup* which means that it should have everything from **Set** plus the listed multiplication and exponentiation operators. Multiplication is asserted to be associative and so this may be assumed for **Integer**. If a category **C2** extends a category **C1**, any domain that belongs to **C2** also belongs to **C1**.

Categories may be parameterized.

```
ListCategory(S: Set): Category == Set with
  first : $ -> $
  rest : $ -> $
  null : $ -> Boolean
  . . .
```

presumably describes the operations and attributes that any abstract datatype that purports to act like a linked list should have. The parameter **S** can be any domain that belongs to **Set**. If we defined a special version of this

```
SemiGroupListCategory(S: SemiGroup): Category == Set
  with
    first : $ -> $
    rest : $ -> $
    null : $ -> Boolean
    . . .
```

then **S** must have been explicitly declared to belong to **SemiGroup**. Category membership is by name: a domain having the operations and attributes of **SemiGroup** does not belong to this category unless it has been explicitly declared to belong to **SemiGroup** or one of its extensions.

Categories allow multiple inheritance.

```
FiniteList(S: Set): Category ==
  Join(Finite, ListCategory(S))
```

defines a category constructor that contains all operation signatures and attributes from both the categories **Finite** and **ListCategory(S)**.

Domains

Domains are created by domain constructors and are objects. They provide a representation for an instance of an abstract datatype, implement the operations of the categories to which they belong, and presumably satisfy the attributes of the categories. The representation of a domain does not depend at all on the categories to which the domain belongs.

In the implementation of **Integer**, the following category definition is given and specifies the behavior of the domain:

```
Integer(): Join(UniqueFactorizationDomain,
  EuclideanDomain, OrderedRing, DifferentialRing)
  with
    oddp : $ -> Boolean
    random : () -> $
    numberOfDigits : ($,$) -> $
    abs : $ -> $
    canonicalUnitNormal
```

The category of **Integer** is not one named category but, instead, is composed of several pieces. The full category is that obtained by joining 4 named categories, 4 additional operations, and 1 additional attribute. Note, though, that **Integer** belongs to each of the 4 named categories, *plus* their ancestors. In particular, **Integer** is a **SemiGroup** because it is a **DifferentialRing**, which is a **Ring**, which is a **SemiGroup**. In fact, the **Join** creates several paths back to the category **SemiGroup**. **Integer** may be used anywhere a **SemiGroup** is required.

Domains may provide functions for objects other than those which they create. For example, **RationalNumber** implements a “/” for two **Integer** arguments.

Two domain constructors that are particularly useful in representing other domains are **Record** and **Union**. A record is similar to structures of the same name in many other languages: it has one or more fields accessed via selector names. The objects in the fields can be of any datatype. The constructor **Union** creates *discriminated* unions: an object of **Union(Integer,String)** belongs either to the **Integer** branch or the **String** branch of the union. The **case** function is used to test for membership in branches.

The constructor **Mapping** creates the datatypes of functions. Declaring **oddp : Mapping(Boolean,Integer)** states that **oddp** is a function taking one integer argument and returning a boolean. A convenient alternative notation for this is **oddp: Integer → Boolean**. Functions are first-class objects and can be passed to other functions.

The Axiom language design completely separates the concrete implementation inheritance from the abstract specification inheritance. Domains can inherit implementations from their

representations, but the external view of them is only determined by the categories to which they belong. The idea of *subtype* in languages such Trellis/Owl¹ [Scha86] is replaced in Axiom by the two notions of category and domain.

In addition to providing an organizational hierarchy, categories allow Axiom to have extra information about domains. Thus, for example, the compiler knows where operations will be located in domains and can compile efficient function calls. This information is part of a structure associated with each operation called a *modemap*.

Packages

A package is a special kind of domain in that it provides no new objects to the system (other than the package itself). It consists of a category listing operations and attributes and implementations of the operations. Because they may be parameterized, packages can be used to implement algorithms at their most natural level of abstraction. For example, a repeating-squaring algorithm may be contained in a package that is parameterized by a domain that is a **SemiGroup**.

Packages provide an excellent vehicle to implement both domain-specific and category-general algorithms. They can be used to implement algorithms that are not included in other domains for reasons of convenience or necessity. For example, the repeated squaring algorithm and the integration algorithms are implemented in packages. They also require extra work by the interpreter because argument datatypes give no simple clue to the possible location of a package function. For example, in the expression $2 + 9$ the operation “+” is found in the domain **Integer**, which is the common datatype of the two arguments. However, the function **solve** in `solve(x**3-1,x)` is not present in either the domain **Polynomial(Integer)** of the first argument or the domain **Symbol** of the second. This, along with the last example of the previous section, demonstrates the need for a systematic search mechanism of the Axiom library.

Modemaps

A *modemap* is a syntactic specification of an operation. It gives the name of the operation, source and target datatypes for its parameters, and the name of the datatype or package exporting the operation. For example, the modemap

```
oddp : Integer -> Boolean from Integer
```

specifies an operation from type *Integer* which takes an integer argument and returns a boolean value. The “from”-clause indicates the *domain of implementation* of the operation (in this case, **Integer**).

When the datatype or package is parameterized, the conditions on their parameters appear in an “if”-clause for the modemap. For example, every domain created by domain constructor **Matrix** exports a trace operation:

¹Trellis is a trademark of Digital Equipment Corporation

```
trace: Matrix(R) -> $ if R has Ring from Matrix(R)
```

The domain constructor **Matrix** takes one parameter **R** which is required to belong to the category **Ring**.

Operations may also be conditionally exported by a domain constructor. For example, domains created by **Matrix** export a **determinant** operation only if the homogeneous multiplication “*” in the underlying domain is commutative, that is, **R** has **commutative**(“*”). The modemap for **determinant** is

```
determinant: Matrix(R) -> $ if R has Ring
and R has commutative("*") from Matrix(R)
```

Similarly, datatypes created by **Matrix** export an **inverse** operation only if their argument domain belongs to **Field**. Since the requirement “**R** has **Field**” implies “**R** has **Ring**” the modemap for **inverse** is

```
inverse: Matrix(R) -> Union(R,"failed")
if R has Field from Matrix(R)
```

The collection of modemaps from all abstract datatypes and packages constitutes the *global modemap database* for Axiom. To standardize the presentation of modemaps, arguments and other parameters are prefaced by “pattern variables”, here called ***1**, ***2**, etc. The resulting form of the modemap for **Matrix inverse** in the modemap database is, for example,

```
inverse: *1 -> Union(*1,"failed")
if *1 is Matrix(*2) and *2 has Field from *1
```

In addition to this general modemap database, each domain or package has a *local modemap database* for all its exported operations. Note that the pattern variables and “if”-clauses of the above global modemaps depend on the parameters to a domain or package constructor. For a domain or package itself, all parameters to the constructor are known. Thus local modemaps have no patterns. Predicates reduce to *true* or *false*. All modemaps in a local modemap database for a domain or package **D** therefore have the general form

```
f: S -> T if true from D
```

Those having a predicate of **false** simply do not exist for a given domain or package. For example, **Matrix(RationalNumber)** will have an **inverse** operation whereas **Matrix(Integer)** will not, since **RationalNumber** is a field but **Integer** is not.

Interpretation

The role of the interpreter is to evaluate input expressions entered by the user. In addition to computations, these expressions may be declarations, function definitions, or system commands. A value in Axiom is described by a pair $\langle a, A \rangle$ where a is an object and A is its type.

Evaluation of an operator-operand expression $f(x, y, \dots)$, with n ($n > 1$) arguments $x, y, \dots$, is done in a bottom-up manner. The interpreter will evaluate the arguments to produce a corresponding n -tuple of argument values $\langle a, A \rangle, \langle b, B \rangle, \dots$. At this point an attempt will be made to select an applicable modemap for f . If this modemap has a return type of $\mathbf{T}$, f is applied to the (possibly coerced) arguments to yield a result t and subsequent value pair $\langle t, T \rangle$.

Modemap Selection

Given a function call $f(x, y, \dots)$ and evaluated arguments $\langle a, A \rangle, \langle b, B \rangle, \dots$ as in the previous section, the interpreter tries to find an appropriate f to apply. The first attempt involves constructing a list of the domains $A, B, \dots$ of the evaluated arguments, plus the target type of the function call, if it exists,² plus those types contained in any of the argument types if they are constructed from **Record**, **Union**, or **Mapping**. Duplicates are removed from the list and domains and packages in the list are searched for an applicable f . If a modemap is found of the form

$$f: (A, B, \dots) \rightarrow T \text{ from } C$$

with arity n , for some $\mathbf{T}$ and $\mathbf{C}$, the function is gotten from $\mathbf{C}$ and applied to $a, b, \dots$ to yield a result t and subsequent value pair $\langle t, T \rangle$.

This kind of search for an applicable f generalizes the idea of a *controlling object* in a function call [Lisk79][Scha86]. In Axiom, though, the arguments in a function call need not give any hint to the actual location of the function to be applied. If there is no applicable modemap to be found among the argument domains of a function, a two-stage search for a suitable modemap is made in the global modemap database. A set M of candidate modemaps is constructed, each of the form:

$$f: (D, E, \dots) \rightarrow T \text{ if } p \text{ from } C$$

for some result type $\mathbf{T}$ and domain of implementation $\mathbf{C}$, each of arity n and each generally containing pattern variables and predicates p . The set M is partitioned into two subsets: those modemaps coming from domains or packages whose names contain those of any of the arguments of the function, and those that do not. The modemaps in the first subset are examined for applicability and, if one is found, it is returned. Otherwise, those in the second subset are checked for applicability.

At first this partitioning may seem odd, but it reflects a naming convention for domains and packages in the Axiom library. As an example, the constructor **ListPackage1** takes one **Set** argument and implements several functions that could be but are not now implemented in the **List** domain. The constructor **ListPackage2** takes two domain arguments, each belonging to the category **Set**. The functions in this package implement operations that requires lists with two different element types. For example, the function

² As an example of a target type, consider the expression $m:\text{Matrix(Integer)}:=f(x,y,z)$. The target type of the function call is **Matrix(Integer)**.

```
map: (S -> T, List(S)) -> List(T)
```

which maps a function from **S** to **T** across the elements of **List(S)**, producing an object of **List(T)**, is implemented in **ListPackage2(S,T)**. Looking at the modemaps in the first partition of *M* reflects an extension of the search method that looks for applicable functions in the domains of the function arguments. In the future we anticipate using attributes in such associated packages to determine the first subset of the general candidate modemaps. That is, **ListPackage2** will contain the attribute **associated("List")** in its category.

For our purposes here, it suffices to regard the predicate of a modemap as a conjunction of simple predicates of two kinds:

```
X is Y
X has Y
```

Predicates of the first kinds state that pattern *X* matches only the explicit domain *Y*. If a modemap only has predicates of the first kind, patterns are replaced by the domain or package names to produce a new modemap free of patterns.

When the modemap has predicates of the second kind, substitutions are sought for the pattern variables such that the predicate is satisfied. Here *Y* is a category or an attribute. Pattern variables are given initial substitutions based on the value pairs of the arguments. Any pattern variable *X* for which there is a predicate “*X* is *Y*” has *Y* assigned as a *permanent* substitution. Other substitutions are labeled *tentative*. To satisfy the predicate, any or all the operations of *coercing*, *resolving*, and *forcing* (see below) may be necessary. As above, if pattern variable *X* has permanent substitution *Y*, any argument value of *f* of type **Z** must be coerced to type *Y*.

The *resolve* of two types **T1** and **T2** will always succeed. It produces a third type **T3** to which all objects of type **T1** and **T2** can be coerced. The type **Any** is used for **T3** if a less general type cannot be found. Resolving is necessary to use a modemap with homogeneous arguments. For example, the modemap

```
"+":(*1,*1) -> *1 if *1 has AbelianMonoid from *1
```

is used for addition in most algebraic domains in Axiom. Given an expression $1 + (2/3)$, a bottom-up analysis will first produce $\langle 1, \text{Integer} \rangle$ and $\langle 2/3, \text{RationalNumber} \rangle$ as the values of the two operands of “+”. The *resolve* of **Integer** and **RationalNumber** is defined to be **RationalNumber**. This causes the integer object 1 to be coerced to the rational number object 1/1. **RationalNumber** now matches *1 in the above modemap, the predicate evaluates to true, and the corresponding function is gotten from **RationalNumber** and applied to produce the result 5/3 and pair $\langle 5/3, \text{RationalNumber} \rangle$.

A *force* is an operation performed on one or more types to satisfy a predicate. For example, the predicate “*1 has **Field**” where *1 has a tentative substitution **Integer** results in the forcing of **Integer** to **RationalNumber** by the application of **QuotientField**, i.e. **QuotientField(Integer)** is equivalent to **RationalNumber**. As another example, *x* has default type **Symbol**. When appearing in a sum (e.g. $x + 1$), the modemap of *x* requires its argument to be from a domain which belongs to the category **AbelianMonoid**. As a result, **Symbol** is forced to **Polynomial(Integer)**.

A list of all applicable modemap is produced together with a list of required coercions on the source parameters. Duplicates and modemap subsumed by others are discarded. Each modemap is assigned a cost based on the required coercions and the target type of the operation. By definition of the cost function, any modemap which directly matches the argument types (e.g. $\mathbf{A}=\mathbf{D}$, $\mathbf{B}=\mathbf{E}$, etc.) will have the cheapest cost.

Ambiguity

Two remaining modemap are said to be *ambiguous* unless there are coercions to and from the respective substitution datatypes for the pattern variables $*1, *2, \dots$. If there are no ambiguities, the cheapest modemap is selected and applied.

As a practical matter, users can always use a “package call” to avoid ambiguities, e.g. $\mathbf{x} \ * \$\mathbf{D} \ \mathbf{y}$ and $\mathbf{foo} \$\mathbf{D}(\mathbf{x}, \mathbf{y})$ direct the interpreter to apply the functions “ $*$ ” and $\mathbf{foo}$ from the domain or package $\mathbf{D}$. Package calling is the only way to identify uniquely functions of no arguments if they are ambiguous.

Modes

If the explicit conversion $\mathbf{p}::\mathbf{P}$ or $\mathbf{p}::\mathbf{P}(?)$ is given, $\mathbf{p}$ is converted to a polynomial. The datatype of coefficients may be any domain which satisfies the categorical requirements of the argument to the domain constructor `Polynomial`. The form $\mathbf{P}(?)$ is a *mode* specification rather than a *type* specification: the interpreter is free to choose what replaces the $?$.

A mode is a partial type specification in that zero or more arguments in a domain constructor call are replaced by $?$. The process of *merging* takes a type $\mathbf{T1}$ and a mode $\mathbf{M}$ and determines type substitutions for the $?$ ’s in $\mathbf{M}$ to create a new type $\mathbf{T2}$ to which $\mathbf{T1}$ is coercable. For example, the merger of `RationalNumber` and `Polynomial(?)` is `PolynomialRationalNumber`. If the mode $\mathbf{M}$ has no $?$ ’s, it is a type and the merger will only succeed if $\mathbf{T1}$ is coercable to $\mathbf{M}$. Thus merging a type and a mode may fail, unlike resolving two types. If $\mathbf{M}$ is simply $?$, then the result of the merger is $\mathbf{T1}$.

Axiom now only supports modes with at most one $?$, and that must be in the innermost constructor call position (e.g. `List(Polynomial(?))`). In our experience, this restriction has not seemed burdensome, though a future area of research might be the extension of the merging process to more general modes.

1.3 The Coerce Facility

Our goal in the design of the coerce facility is to have as much as possible controlled by modemap selection of compiled Axiom functions. This allows the domain and package writer maximum control over the behaviour of the interpreter and removes the requirement of having a system developer tune the interpreter for dealing with new datatypes. The coerce facility has several components.

Coerce by Function

The interpreter does modemap selection for an operation named **coerce** that has the appropriate argument and target types. This is the easiest way for a programmer to control the coerce facility. For example, the category **Ring** provides a modemap

```
coerce: Integer -> R from R
```

where **R** is the ring being defined. Thus any ring has a coercion from **Integer** and, in fact, this operation has a default categorical definition.

Coercions can sometimes be defined in domains but are often defined in packages. A coercion of the form

```
coerce: Polynomial(QuotientField(R)) ->
  QuotientField(Polynomial(R))
  if R has IntegralDomain
```

would typically be defined in a package parameterized by the domain **R**. On the other hand, it is not feasible to have so many explicit coercions being written and, in fact, there are general methods that will perform these kinds of coercions.

It is not now possible to define a coercion of the form

```
coerce: List(S) -> List(T)
```

in the domain constructor **List**. This is because **List** is parameterized by only one set (**S** or **T**) and the modemap cannot, therefore, be part of the category of **List**. Such an explicit coercion can be provided in a package, but is, in fact, handled by the general mechanism described in the next section.

Coerce by Mapping

When the interpreter encounters a coercion of the form

```
D(T1) -> D(T2)
```

where **D** is a domain constructor with a parameter (for reasons of exposition, we here omit the case of **D** having multiple parameters), it looks for a function

```
map: (T1 -> T2, D(T1)) -> D(T2)
```

that takes a function from **T1** to **T2** and an object of **D(T1)** and produces an object of **D(T2)**. It then creates a function stub that coerces objects of **T1** to those of **T2** and passes it to **map**, along with the original argument.

Since the function **map** is part of the library of Axiom compiled code, it allows the package writer to automatically provide an interpreter mechanism of “lifting” coercions from **T1** to **D(T1)** and **D(T2)**.

Coerce by Internal System Code

Some special cases of coercions are handled by internal system code rather than compiled Axiom code. These typically involve polynomials where the variable ordering is changed or distributed across a tower of parameterized domains. It will eventually be moved into Axiom code as the pattern matching facilities are improved.

Another case that is now handled internally involves rearrangement of a tower of parameterized domains. The domain constructor **Gaussian** creates domains with objects similar to the complex numbers in that they contain real and imaginary parts. The coercion

```
Gaussian(Polynomial(RationalNumber)) ->
  QuotientField(Polynomial(Gaussian(Integer)))
```

is performed in the following steps. The type **RationalNumber** is changed into the equivalent type **QuotientField(Integer)** and the **QuotientField** is bubbled to the top of the original type to get **QuotientField(Gaussian(Polynomial(Integer)))**. This new type and the old target now have the same top level constructor (**QuotientField**) and the coerce facility is called recursively on the underlying domains **Gaussian(Polynomial(Integer))** and **Polynomial(Gaussian(Integer))**.

Coercion of Algebraic Constants

Several categories specify the existence of constants. For example, **AbelianMonoid** specifies the operation “+” : \$ → \$ and the constant 0 : \$. If **T1** and **T2** each belong to a common category with specified constants and the object of **T1** to be coerced is one of the constants, the corresponding constant in **T2** may be extracted and returned.

Retraction

A retraction is a coercion of an object of a domain to a more specific (degenerate) domain. Unlike other forms of coercion where the target type is known, retraction involves examining an operand pair < t, T > to see if there exists a degenerate form of **T** to which *t* can be coerced. For example, < 7/1, RationalNumber > retracts to < 7, Integer > and < 1, Polynomial(Integer) > retracts to < 1, Integer >. Retraction can occur multiple times, so arbitrarily long structures collapse to their simplest forms, e.g. < 1, Polynomial(RationalNumber) > to < 1, Integer >.

If no applicable modemap is found in modemap selection, retraction is done on the arguments and then selection is attempted again. Retraction is also attempted when a coercion is requested from a domain to its underlying domain. For example, if *p* is an element of **Polynomial(Integer)**, the statement *p* : I will cause the interpreter to try to retract *p* to an element of **Integer**.

For the most part, retraction can be accomplished by functions in the Axiom library. The category **RetractWithUnderDomain(R)** specifies two operations

```
retractable? : $ -> Boolean
retract      : $ -> R
```

In our example of polynomials above, the function **retractable?** would be called to see if the object was a constant polynomial. If the result was *true*, **retract** can be called to extract the constant.

Coercion Query

There are situations where one wishes to know ahead of time whether it is possible to coerce an object of type **T1** to one of type **T2**. The Axiom interpreter provides this information, for example, to the type resolve and modemap selection facilities. The facility is used when one needs to know absolutely when a coercion will be successful. An answer of “no”, however, does not guarantee that a coercion could not be performed for specific data. For example the system will respond “no” when asked whether an object of **Polynomial(Integer)** can be coerced to an object of **Integer**. As we saw above, though, retractions can be performed for constant polynomials.

1.4 The Resolve Facility

The resolve facility is used to determine a type **T3** to which two types **T1** and **T2** can be coerced. It is used when an operation has homogeneous arguments (such as “+”) or when a statement has several exit points and they must all return the same type (**then** and **else** clauses of an **if** statement, or multiple **return** statements in a function).

The resolve facility is symmetric: **resolve(T1,T2) = resolve(T2,T1)**. The resolve facility is always successful because type **Any** is returned if a less general type cannot be found. **Any** is represented by a record with two components, the first being the original type of the object and the second being the object itself. Thus anything can be coerced to an object of type **Any** and **resolve(Any,T) = Any** for all **T**.

Two other types have special resolve rules. Type **Void** has but one object and is the type returned by such operations as variable declaration, function definition, **if** statements without **else** clauses and **repeat** loops. Several functions also return the object of type **Void**, including those that display things in two dimensional algebraic, TeX³ and FORTRAN forms. Like **Any**, **resolve(Void,T) = Void** for all **T**. Type **Exit** is used for **return** statements and error statements. Its rule is **resolve(Exit,T) = T** for all **T**.

After some checks for special cases, the resolve facility has three components.

Resolve by Coercion Query

If **T1 = T2**, then the resolve of the pair is just **T1**. If **T1** can be coerced to **T2** and **T2** cannot be coerced to **T1**, then **resolve(T1,T2) = T2**. If **T1** and **T2** are coercable to one another, an arbitrary but canonical choice is made and returned.

³TeX is a trademark of the American Mathematical Society

Resolve by Rules

The interpreter has an internal database of rules which it tries to use to resolve two types. The rules are not complete, as they only attempt to take care of cases that cannot be dealt with in a more general way. Almost all of the rules deal with polynomials. For example, one rule is

```
resolve(Polynomial(T1),UnivariatePoly(x,T2))=
  resolve(Polynomial(T1),T2)
```

The variable of **UnivariatePoly** can be absorbed into the general constructor **Polynomial** and then the resolve facility is called again with different arguments. The second call may or may not use the rule system.

Resolve by Type Destructuring

This type of resolution is similar to the process involved in the coercion described in the last paragraph of “Coerce by Internal System Code”. Given two towers of parameterized types, the interpreter tries to rearrange the towers and create a new type to which both of the original types are coerceable. This is a recursive process and involves using the coercion query facility to determine what tower rearrangements are possible.

1.5 An Example

As an example of coerce and resolve, we describe the inference involved in determining that the expression $x+1/2$ evaluates to an object of the datatype **Polynomial(RationalNumber)**. We assume x has not previously been given a value.

- Choose a default datatype of **Symbol** for x
- Choose the datatype of **Integer** for 1 and 2
- Look for an operation “/” in **Integer** that has two arguments, each an integer. It is not found.
- Start a general search in the library for an operation “/” with two **Integer** arguments. One is found in **RationalNumber** and applied
- Look for a “+” that takes a **Symbol** and a **RationalNumber**. None is found.
- Force x to an object of type **Polynomial(Integer)**
- Look for a “+” that takes a **Polynomial(Integer)** and a **RationalNumber**. None is found.

- Start a general search in the library for “+” operations. The only ones found take two arguments, each of the same datatype. Resolve **Polynomial(Integer)** and **RationalNumber** to get the datatype **Polynomial(RationalNumber)**
- Do the coercions

```
Polynomial(Integer) -> Polynomial(RationalNumber)
RationalNumber -> Polynomial(RationalNumber)
```

- Apply the “+” in the datatype **Polynomial(RationalNumber)** and return the result

1.6 Acknowledgement

In addition to the authors, Richard Jenks and Robert Sutor, three people have contributed significantly to the development of the Axiom interpreter. Scott C. Morrison (University of California, Berkeley) is responsible for the overall structure of the interpreter as it is today, having largely rewritten this part of the system in 1984. Albrecht Fortenbacher (University of Karlsruhe) rewrote and greatly extended the resolve and coerce facilities in 1985. Michael Lucks (Southern Methodist University) contributed to the coerce and modemap selection facilities in 1986.

Chapter 2

The Interpreter

The Axiom interpreter is a large common lisp program. It has several forms of interaction and run from terminal in a standalone fashion, run under the control of a session handler program, run as a web server, or run in a unix pipe.

Chapter 3

The Fundamental Data Structures

defvar \$PatternVariableList

These are temporary variable names that will be replaced by FormalMapVariableList.

— initvars —

```
(defvar |$PatternVariableList|
  '(*1 *2 *3 *4 *5 *6 *7 *8 *9 *10 *11 *12 *13 *14 *15 *16 *17 *18 *19 *20
    *21 *22 *23 *24 *25 *26 *27 *28 *29 *30 *31 *32 *33 *34 *35 *36 *37 *38
    *39 *40 *41 *42 *43 *44 *45 *46 *47 *48 *49 *50))
```

—————

defvar \$FormalMapVariableList

— initvars —

```
(defvar |$FormalMapVariableList|
  '(|#1| |#2| |#3| |#4| |#5| |#6| |#7| |#8| |#9| |#10|
    |#11| |#12| |#13| |#14| |#15| |#16| |#17| |#18| |#19| |#20|
    |#21| |#22| |#23| |#24| |#25| |#26| |#27| |#28| |#29| |#30|
    |#31| |#32| |#33| |#34| |#35| |#36| |#37| |#38| |#39| |#40|
    |#41| |#42| |#43| |#44| |#45| |#46| |#47| |#48| |#49| |#50|))
```

—————

3.1 Frames and the Interpreter Frame Ring

Axiom has the notion of “frames”. A frame is a data structure which holds all the vital data from an Axiom session.

The list of frames is structured as a ring. New frames can be added which will hold computations of independent information. The interpreter **)frame** command allows operations on frames. From the command line the user can create, modify, change, and delete frames.

3.2)frame Command

frame man page

— frame.help —

```
=====
A.11. )frame
=====
```

User Level Required: interpreter

Command Syntax:

```
- )frame new frameName
- )frame drop [frameName]
- )frame next
- )frame last
- )frame names
- )frame import frameName [objectName1 [objectName2 ...] ]
- )set message frame on | off
- )set message prompt frame
```

Command Description:

A frame can be thought of as a logical session within the physical session that you get when you start the system. You can have as many frames as you want, within the limits of your computer’s storage, paging space, and so on. Each frame has its own step number, environment and history. You can have a variable named `a` in one frame and it will have nothing to do with anything that might be called `a` in any other frame.

Some frames are created by the HyperDoc program and these can have pretty strange names, since they are generated automatically. To find out the names of all frames, issue

```
)frame names
```

It will indicate the name of the current frame.

You create a new frame ‘quark’ by issuing

```
)frame new quark
```

The history facility can be turned on by issuing either `)set history on` or `)history on`. If the history facility is on and you are saving history information in a file rather than in the AXIOM environment then a history file with filename `quark.ahx` will be created as you enter commands. If you wish to go back to what you were doing in the ‘initial’ frame, use

```
)frame next
```

or

```
)frame last
```

to cycle through the ring of available frames to get back to ‘initial’.

If you want to throw away a frame (say ‘quark’), issue

```
)frame drop quark
```

If you omit the name, the current frame is dropped.

If you do use frames with the history facility on and writing to a file, you may want to delete some of the older history files. These are directories, so you may want to issue a command like `rm -r quark.ahx` to the operating system.

You can bring things from another frame by using `)frame import`. For example, to bring the `f` and `g` from the frame ‘quark’ to the current frame, issue

```
)frame import quark f g
```

If you want everything from the frame ‘quark’, issue

```
)frame import quark
```

You will be asked to verify that you really want everything.

There are two `)set` flags to make it easier to tell where you are.

```
)set message frame on | off
```

will print more messages about frames when it is set on. By default, it is off.

```
)set message prompt frame
```

will give a prompt that looks like

```
initial (1) ->
```

when you start up. In this case, the frame name and step make up the prompt.

Also See:

- o `)history`
- o `)set`

1

The frame mechanism uses several dollar variables.

Primary variables are those which exist solely to make the frame mechanism work.

The `$interpreterFrameName` contains a symbol which is the name of the current frame in use.

The `$interpreterFrameRing` contains a list of all of the existing frames. The first frame on the list is the “current” frame. When Axiom is started directly there is only one frame named “initial”.

If the system is started under `sman` (using the axiom shell script, for example), there are two frames, “initial” and “frame0”. In this case, “frame0” is the current frame. This can cause subtle problems because functions defined in the axiom initialization file (`.axiom.input`) will be defined in frame “initial” but the current frame will be “frame0”. They will appear to be undefined. However, if the user does “`)frame next`” they can switch to the “initial” frame and see the functions correctly defined.

The `$frameMessages` variable controls when frame messages will be displayed. The variable is initially `NIL`. It can be set on (`T`) or off (`NIL`) using the system command:

```
)set message frame on | off
```

Setting frame messages on will output a line detailing the current frame after every output is complete.

See the **set message frame**([p1013](#)) `setmessageframe` section for more details.

The frame collects and uses a few top level variables. These are: `$InteractiveFrame`, `$IOindex`, `$HiFiAccess`, `$HistList`, `$HistListLen`, `$HistListAct`, `$HistRecord`, `$internalHistoryTable`, and `$localExposureData`.

These variables can also be changed by the frame mechanism when the user requests changing to a different frame.

¹ “history” ([26.23 p 889](#)) “set” ([26.51 p 1081](#))

3.3 Data Structures

The interpreter information is kept in a frame which is a 10 part data structure of the form (see 3.5). The parts of a frame and their initial, default values are:

1. **\$interpreterFrameName**(p38) interpreterFrameName, a string, is the name of the current frame.
2. **\$InteractiveFrame**(p39) InteractiveFrame which defaults to ((nil))
3. **\$IOindex**(p39) IOindex also known as the step number, which defaults to 1
4. **\$HiFiAccess**(p1005) HiFiAccess
5. **\$HistList**(p47) HistList
6. **\$HistListLen**(p48) HistListLen
7. **\$HistListAct**(p48) HistListAct
8. **\$HistRecord**(p48) HistRecord
9. **\$internalHistoryTable**(p48) internalHistoryTable which defaults to nil
10. **localExposureDataDefault**(p167) is a copy of the current local exposure data

There are a set of functions to manipulate frames. The internal set of frame functions are

- **initializeInterpreterFrameRing**(p27) creates the original frame ring, inserts an initial frame, and updates all the global variables from the initial frame.
- **emptyInterpreterFrame**(p27) creates a new, empty frame.
- **createCurrentInterpreterFrame**(p29) collects the environment into a frame.
- **updateFromCurrentInterpreterFrame**(p30) sets all of the global variables from the current frame.
- **frameEnvironment**(p31) returns the frameInteractive component of a named frame or a new, empty environment.
- **findFrameInRing**(p31) given the name, find the named frame
- **updateCurrentInterpreterFrame**(p31) collects the normal contents of the world into a frame object, places it first on the frame list, and then sets the current values of the world from the frame object.
- **nextInterpreterFrame**(p32) updates the current frame to make sure all of the current information is recorded. If there are more frame elements in the list then this will destructively move the current frame to the end of the list, that is, assume the frame list reads (1 2 3) this function will destructively change it to (2 3 1).

- **previousInterpreterFrame**(p33) moves to the previous frame in the ring.
- **changeToNamedInterpreterFrame**(p32) change to the named frame.
- **addNewInterpreterFrame**(p33) update the current frame, initialize the history, make a new empty frame, and initialize all of the global variables from the empty frame.
- **closeInterpreterFrame**(p37) when there is more than one frame, delete the current frame and initialize all the global variables from the next frame in the ring.
- **displayFrameNames**(p28) print all the frame names and indicate which one is the current frame.
- **importFromFrame**(p34) imports items from a different frame into the current frame

3.4 Frame Access Macros

First Frame Component – `frameName`

defmacro `frameName`

```
type : FrameName → Symbol
frameName : Frame → FrameName
— defmacro frameName 0 —
```

```
(defmacro frameName (frame)
  '(first ,frame))
```

—————

Second Frame Component – `frameInteractive`

defmacro `frameInteractive`

```
frameInteractive : Frame → Interactive
— defmacro frameInteractive 0 —
```

```
(defmacro frameInteractive (frame)
  '(second ,frame))
```

—————

Third Frame Component – `frameIOIndex`

defmacro frameIOIndex

frameIOIndex : Frame $\rightarrow$ IOIndex
 — **defmacro frameIOIndex 0** —

```
(defmacro frameIOIndex (frame)
  '(third ,frame))
```

—————

Fourth Frame Component – frameHiFiAccess

defmacro frameHiFiAccess

frameHiFiAccess : Frame $\rightarrow$ HiFiAccess
 — **defmacro frameHiFiAccess 0** —

```
(defmacro frameHiFiAccess (frame)
  '(fourth ,frame))
```

—————

Fifth Frame Component – frameHistList

defmacro frameHistList

frameHistList : Frame $\rightarrow$ HistList
 — **defmacro frameHistList 0** —

```
(defmacro frameHistList (frame)
  '(fifth ,frame))
```

—————

Sixth Frame Component – frameHistListLen

defmacro frameHistListLen

type : HistListLen $\rightarrow$ NonNegativeInteger

frameHistListLen : Frame $\rightarrow$ HistListLen
 — **defmacro frameHistListLen 0 —**

```
(defmacro frameHistListLen (frame)
  '(sixth ,frame))
```

—————→

Seventh Frame Component – frameHistListAct

defmacro frameHistListAct

frameHistListAct : Frame $\rightarrow$ HistListAct
 — **defmacro frameHistListAct 0 —**

```
(defmacro frameHistListAct (frame)
  '(seventh ,frame))
```

—————→

Eighth Frame Component – frameHistRecord

defmacro frameHistRecord

frameHistRecord : Frame $\rightarrow$ HistRecord
 — **defmacro frameHistRecord 0 —**

```
(defmacro frameHistRecord (frame)
  '(eighth ,frame))
```

—————→

Ninth Frame Component – frameHistoryTable

defmacro frameHistoryTable

frameHistoryTable : Frame $\rightarrow$ HistoryTable
 — **defmacro frameHistoryTable 0 —**

```
(defmacro frameHistoryTable (frame)
  '(ninth ,frame))
```

Tenth Frame Component – frameExposureData

defmacro frameExposureData

```
frameExposureData : Frame → ExposureData
enum : FrameArgs → (nil,drop,import,last,names,new,next)
frameSpad2Cmd : FrameArgs → nil
— defmacro frameExposureData 0 —
```

```
(defmacro frameExposureData (frame)
  '(tenth ,frame))
```

3.5 Functions to manipulate frames

The top level frame command

[frameSpad2Cmd p25]

— defun frame —

```
(defun |frame| (1)
  "The top level frame command"
  (|frameSpad2Cmd| 1))
```

The top level frame command handler

```
[throwKeyedMsg p??]
[helpSpad2Cmd p881]
[selectOptionLC p819]
[qcdr p??]
[qcar p??]
```



```

[object2Identifier p??]
[frameSpad2Cmd drop (vol9)]
[closeInterpreterFrame p37]
[import p??]
[importFromFrame p34]
[last p??]
[previousInterpreterFrame p33]
[names p??]
[displayFrameNames p28]
[new p??]
[addNewInterpreterFrame p33]
[next p330]
[nextInterpreterFrame p32]
[$options p73]

```

— defun frameSpad2Cmd —

```

(defun |frameSpad2Cmd| (args)
  "The top level frame command handler"
  (let (frameArgs arg a)
    (declare (special |$options|))
    (setq frameArgs '(|drop| |import| |last| |names| |new| |next|))
    (cond
      (|$options|
        (|throwKeyedMsg| "The %1 system command takes arguments but no options."
          (cons ")frame" nil)))
      ((null args) (|helpSpad2Cmd| (cons '|frame| nil)))
      (t
        (setq arg (|selectOptionLC| (car args) frameArgs '|optionError|))
        (setq args (cdr args))
        (when (and (consp args)
          (eq (qcdr args) nil)
          (progn (setq a (qcar args)) t))
          (setq args a))
        (when (atom args) (setq args (|object2Identifier| args)))
        (case arg
          (|drop|
            (if (and args (consp args))
              (|throwKeyedMsg| "%1 is not a valid frame name."
                (cons args nil))
              (|closeInterpreterFrame| args)))
          (|import| (|importFromFrame| args))
          (|last| (|previousInterpreterFrame|))
          (|names| (|displayFrameNames|))
          (|new|
            (if (and args (consp args))
              (|throwKeyedMsg| "%1 is not a valid frame name."
                (cons args nil))
              (cons args nil)))

```

```
(|addNewInterpreterFrame| args)))
(|next| (|nextInterpreterFrame|))
(t nil))))))
```

Initializing the Interpreter Frame Ring

There can be multiple frames and these live in a top-level variable called **\$interpreter-FrameRing**. This variable holds a circular list of frames.

This function creates an empty, initial frame named “initial” and creates a list of frames containing an empty frame. This list is the interpreter frame ring and is not actually circular but is managed as a circular list.

As a final step we update the world from this frame. This has the side-effect of resetting all the important global variables to their initial values.

```
[emptyInterpreterFrame p27]
[updateFromCurrentInterpreterFrame p30]
[$interpreterFrameName p38]
[$interpreterFrameRing p38]
```

emptyInterpreterFrame : Symbol → Frame
 — **defun initializeInterpreterFrameRing** —

```
(defun |initializeInterpreterFrameRing| ()
  "Initializing the Interpreter Frame Ring"
  (declare (special |$interpreterFrameName| |$interpreterFrameRing|))
  (setq |$interpreterFrameName| '|initial|)
  (setq |$interpreterFrameRing|
    (list (|emptyInterpreterFrame| |$interpreterFrameName|)))
  (|updateFromCurrentInterpreterFrame|)
  nil)
```

Create a new, empty Interpreter Frame

```
[$HiFiAccess p1005]
[$HistList p47]
[$HistListLen p48]
[$HistListAct p48]
[$HistRecord p48]
[$localExposureDataDefault p167]
```

emptyInterpreterFrame : Symbol $\rightarrow$ Frame
 — defun emptyInterpreterFrame 0 —

```
(defun |emptyInterpreterFrame| (name)
  "Create a new, empty Interpreter Frame"
  (declare (special |$HiFiAccess| |$HistList| |$HistListLen| |$HistListAct|
    |$HistRecord| |$localExposureDataDefault|))
  (list name                                ; frame name
    (list (list nil))                      ; environment
    1                                       ; $I0index
    |$HiFiAccess|
    |$HistList|
    |$HistListLen|
    |$HistListAct|
    |$HistRecord|
    nil                                    ; $internalHistoryTable
    (copy-seq |$localExposureDataDefault|))) ; $localExposureData
```

Create a list of all of the frame names

This function simply walks across the frame in the frame ring and returns a list of the name of each frame. [[\\$interpreterFrameRing p38](#)]

frameNames : nil $\rightarrow$ List Symbol
 — defun frameNames 0 —

```
(defun |frameNames| ()
  "Creating a List of all of the Frame Names"
  (declare (special |$interpreterFrameRing|))
  (mapcar #'(lambda (f) (frameName f)) |$interpreterFrameRing|))
```

Display the frame name list message

```
[bright p??]
[framename p??]
[\$interpreterFrameRing p38]
```

displayFrameNames : nil $\rightarrow$ nil
 — defun displayFrameNames 0 —

```
(defun |displayFrameNames| ()
```

```
"Display the Frame Names"
(declare (special |$interpreterFrameRing|))
(format t "  The names of the existing frames are:~%"
  (format t "~{          ~a ~%~}" (|frameNames|))
  (format t "          The current frame is the first one listed.~%"))
```

Collect the global variables into a Frame

We can collect up all the current environment information into one frame element with this call. It creates a list of the current values of the global variables and returns this as a frame element.

```
[|$interpreterFrameName p38|
|$InteractiveFrame p39|
|$IOindex p39|
|$HiFiAccess p1005|
|$HistList p47|
|$HistListLen p48|
|$HistListAct p48|
|$HistRecord p48|
|$internalHistoryTable p48|
|$localExposureData p166|
```

createCurrentInterpreterFrame : nil → Frame
 — defun createCurrentInterpreterFrame 0 —

```
(defun |createCurrentInterpreterFrame| ()
  "Collecting up the Environment into a Frame"
  (declare (special |$interpreterFrameName| |$InteractiveFrame| |$IOindex|
    |$HiFiAccess| |$HistList| |$HistListLen| |$HistListAct| |$HistRecord|
    |$internalHistoryTable| |$localExposureData|))
  (list
    |$interpreterFrameName|
    |$InteractiveFrame|
    |$IOindex|
    |$HiFiAccess|
    |$HistList|
    |$HistListLen|
    |$HistListAct|
    |$HistRecord|
    |$internalHistoryTable|
    |$localExposureData|))
```

Update global variables from the Current Frame

The frames are kept on a circular list. The first element on that list is known as “the current frame”. This will initialize all of the interesting interpreter data structures from that frame.

```
[sayMessage p??]
[$interpreterFrameRing p38]
[$interpreterFrameName p38]
[$InteractiveFrame p39]
[$IOindex p39]
[$HiFiAccess p1005]
[$HistList p47]
[$HistListLen p48]
[$HistListAct p48]
[$HistRecord p48]
[$internalHistoryTable p48]
[$localExposureData p166]
[$frameMessages p1013]
```

updateFromCurrentInterpreterFrame : nil → nil

— defun updateFromCurrentInterpreterFrame —

```
(defun |updateFromCurrentInterpreterFrame| ()
  "Update from the Current Frame"
  (let (tmp1)
    (declare (special |$interpreterFrameRing| |$interpreterFrameName|
      |$InteractiveFrame| |$IOindex| |$HiFiAccess| |$HistList| |$HistListLen|
      |$HistListAct| |$HistRecord| |$internalHistoryTable| |$localExposureData|
      |$frameMessages|))
    (setq tmp1 (first |$interpreterFrameRing|))
    (setq |$interpreterFrameName| (frameName tmp1))
    (setq |$InteractiveFrame| (frameInteractive tmp1))
    (setq |$IOindex| (frameIOIndex tmp1))
    (setq |$HiFiAccess| (frameHiFiAccess tmp1))
    (setq |$HistList| (frameHistList tmp1))
    (setq |$HistListLen| (frameHistListLen tmp1))
    (setq |$HistListAct| (frameHistListAct tmp1))
    (setq |$HistRecord| (frameHistRecord tmp1))
    (setq |$internalHistoryTable| (frameHistoryTable tmp1))
    (setq |$localExposureData| (frameExposureData tmp1))
    (when |$frameMessages|
      (format t " Current interpreter frame is called ~a"
        |$interpreterFrameName|))))
```

Replace the current frame and update from the globals

This function collects the normal contents of the world into a frame object, places it first on the frame list, and then sets the current values of the world from the frame object.

[createCurrentInterpreterFrame p29]
 [updateFromCurrentInterpreterFrame p30]
 [\$interpreterFrameRing p38]

updateCurrentInterpreterFrame : nil → nil
 — defun updateCurrentInterpreterFrame —

```
(defun |updateCurrentInterpreterFrame| ()
  "Update the Current Interpreter Frame"
  (declare (special |$interpreterFrameRing|))
  (rplaca |$interpreterFrameRing| (|createCurrentInterpreterFrame|))
  (|updateFromCurrentInterpreterFrame|))
```

Get Named Frame Environment (aka Interactive)

If the frame is found we return the environment portion of the frame otherwise we construct an empty environment and return it. The initial values of an empty frame are created here. This function returns a single frame that will be placed in the frame ring.

[frameInteractive p22]

frameEnvironment : FrameName → nil
 — defun frameEnvironment —

```
(defun |frameEnvironment| (fname)
  "Get Named Frame Environment (aka Interactive)"
  (let ((frame (|findFrameInRing| fname)))
    (if frame
      (frameInteractive frame)
      (list (list nil)))))
```

Find a Frame in the Frame Ring by Name

Each frame contains its name as the 0th element. We simply walk all the frames and if we find one we return it. [boot-equal p??]

[frameName p22]

```
[$interpreterFrameRing p38]
```

```
findFrameInRing : FrameName → Union(Frame,nil)  
— defun findFrameInRing 0 —
```

```
(defun |findFrameInRing| (name)  
  "Find a Frame in the Frame Ring by Name"  
  (declare (special |$interpreterFrameRing|))  
  (dolist (frame |$interpreterFrameRing|)  
    (when (eq (frameName frame) name) (return frame))))
```

Change to the Named Interpreter Frame

```
[updateCurrentInterpreterFrame p31]  
[findFrameInRing p31]  
[updateFromCurrentInterpreterFrame p30]  
[$interpreterFrameRing p38]
```

```
changeToNamedInterpreterFrame : FrameName → nil  
— defun changeToNamedInterpreterFrame —
```

```
(defun |changeToNamedInterpreterFrame| (name)  
  "Change to the Named Interpreter Frame"  
  (let (frame)  
    (declare (special |$interpreterFrameRing|))  
    (|updateCurrentInterpreterFrame|)  
    (setq frame (|findFrameInRing| name))  
    (when frame  
      (setq |$interpreterFrameRing|  
        (cons frame (delete |$interpreterFrameRing| frame :test #'equal)))  
      (|updateFromCurrentInterpreterFrame|))))
```

Move to the next Interpreter Frame in Ring

This function updates the current frame to make sure all of the current information is recorded. If there are more frame elements in the list then this will destructively move the current frame to the end of the list, that is, assume the frame list reads (1 2 3) this function will destructively change it to (2 3 1).

```
[updateFromCurrentInterpreterFrame p30]  
[$interpreterFrameRing p38]
```

nextInterpreterFrame : nil $\rightarrow$ nil
 — **defun** nextInterpreterFrame —

```
(defun |nextInterpreterFrame| ()
  "Move to the next Interpreter Frame in Ring"
  (declare (special |$interpreterFrameRing|))
  (when (cdr |$interpreterFrameRing|)
    (setq |$interpreterFrameRing|
      (nconc (cdr |$interpreterFrameRing|) (list (car |$interpreterFrameRing|))))
    (|updateFromCurrentInterpreterFrame|)))
```

Move to the previous Interpreter Frame in Ring

[updateCurrentInterpreterFrame p31]
 [updateFromCurrentInterpreterFrame p30]
 [\$interpreterFrameRing p38]

previousInterpreterFrame : nil $\rightarrow$ nil
 — **defun** previousInterpreterFrame —

```
(defun |previousInterpreterFrame| ()
  "Move to the previous Interpreter Frame in Ring"
  (let (tmp1 l b)
    (declare (special |$interpreterFrameRing|))
    (|updateCurrentInterpreterFrame|)
    (when (cdr |$interpreterFrameRing|)
      (setq tmp1 (reverse |$interpreterFrameRing|))
      (setq l (car tmp1))
      (setq b (nreverse (cdr tmp1)))
      (setq |$interpreterFrameRing| (nconc (cons l nil) b))
      (|updateFromCurrentInterpreterFrame|))))
```

Add a New Interpreter Frame

[boot-equal p??]
 [framename p??]
 [throwKeyedMsg p??]
 [updateCurrentInterpreterFrame p31]
 [initHistList p889]


```
[emptyInterpreterFrame p27]
[updateFromCurrentInterpreterFrame p30]
[$erase p??]
[histFileName p888]
[$interpreterFrameRing p38]
```

— **defun addNewInterpreterFrame** —

```
(defun |addNewInterpreterFrame| (name)
  "Add a New Interpreter Frame"
  (declare (special |$interpreterFrameRing|))
  (if (null name)
      (|throwKeyedMsg| "You must provide a name for the new frame." nil)
      (progn
        (|updateCurrentInterpreterFrame|)
        (dolist (f |$interpreterFrameRing|)
          (when (eq name (frameName f)) ; existing frame with same name
            (|throwKeyedMsg|
              (format nil
                " You cannot use the name %1 for a new frame because an existing ~
                frame already has that name.")
              (list name))))))
        (|initHistList|)
        (setq |$interpreterFrameRing|
          (cons (|emptyInterpreterFrame| name) |$interpreterFrameRing|))
        (|updateFromCurrentInterpreterFrame|)
        ($erase (|histFileName|))))))
```

—————

Import items from another frame

```
[member p1238]
[frameNames p28]
[throwKeyedMsg p??]
[boot-equal p??]
[framename p??]
[frameEnvironment p31]
[upcase p1272]
[queryUserKeyedMsg p??]
[string2id-n p??]
[importFromFrame p34]
[sayKeyedMsg p43]
[clearCmdParts p841]
[seq p??]
[exit p??]
```

```
[putHist p900]
[get p??]
[getalist p??]
[$interpreterFrameRing p38]
```

— defun importFromFrame —

```
(defun |importFromFrame| (args)
  "Import items from another frame"
  (prog (temp1 fname fenv x v props vars plist prop val m)
    (declare (special |$interpreterFrameRing|))
    (when (and args (atom args)) (setq args (cons args nil)))
    (if (null args)
      (|throwKeyedMsg|
        (format nil ")frame import must be followed by the frame name. The names~
          of objects in that frame can then optionally follow the frame name.~
          For example,~
          %ceon )frame import calculus %ceoff ~
          imports all objects in the calculus frame, and ~
          %ceon )frame import calculus epsilon delta %ceoff ~
          imports the objects named epsilon and delta from the ~
          frame calculus. ~
          Please note that if the current frame contained any information ~
          about objects with these names, then that information would be ~
          cleared before the import took place.")
        nil)
      (progn
        (setq temp1 args)
        (setq fname (car temp1))
        (setq args (cdr temp1))
        (cond
          ((null (|member| fname (|frameNames|)))
            (|throwKeyedMsg|
              (format nil " You cannot import anything from the frame %1 because ~
                that is not the name of an existing frame.")
              (cons fname nil)))
            ((boot-equal fname (frameName (car |$interpreterFrameRing|)))
              (|throwKeyedMsg|
                "You cannot import from the current frame (nor is there a need!)."
                nil))
            (t
              (setq fenv (|frameEnvironment| fname))
              (cond
                ((null args)
                  (setq x
                    (upcase (|queryUserKeyedMsg|
                      (format nil "Do you really want to import everything from the ~
                        frame %1? If so, please enter y or yes :")
                      (cons fname nil))))
```

```
(cond
  ((member (string2id-n x 1) '(y yes))
    (setq vars nil)
    (do ((tmp0 (caar fenv) (cdr tmp0)) (tmp1 nil))
      ((or (atom tmp0)
          (progn (setq tmp1 (car tmp0)) nil)
          (progn
            (progn
              (setq v (car tmp1))
              (setq props (cdr tmp1))
              tmp1)
            nil))
        nil)
      nil)
    (cond
      ((eq v '--macros|)
        (do ((tmp2 props (cdr tmp2))
            (tmp3 nil))
          ((or (atom tmp2)
              (progn (setq tmp3 (car tmp2)) nil)
              (progn
                (progn (setq m (car tmp3)) tmp3)
                nil))
            nil)
          (setq vars (cons m vars))))
        (t (setq vars (cons v vars))))
      (|importFromFrame| (cons fname vars)))
    (t
      (|sayKeyedMsg| "AXIOM will not import everything from frame %1."
        (cons fname nil))))))
(t
  (do ((tmp4 args (cdr tmp4)) (v nil))
    ((or (atom tmp4) (progn (setq v (car tmp4)) nil)) nil)
  (seq
    (exit
      (progn
        (setq plist (getalist (caar fenv) v))
        (cond
          (plist
            (|clearCmdParts| (cons '|propert| (cons v nil)))
            (do ((tmp5 plist (cdr tmp5)) (tmp6 nil))
              ((or (atom tmp5)
                  (progn (setq tmp6 (car tmp5)) nil)
                  (progn
                    (progn
                      (setq prop (car tmp6))
                      (setq val (cdr tmp6))
                      tmp6)
                    nil))
                nil)
              nil)
            (seq
```

```

        (exit (|putHist| v prop val |$InteractiveFrame|))))
      ((setq m (|get| ' |--macros--| v fenv))
        (|putHist| ' |--macros--| v m |$InteractiveFrame|))
      (t
        (|sayKeyedMsg|
          (format nil "AXIOM cannot import %1 from frame %2 because ~
                    it cannot be found.")
          (cons v (cons fname nil)))))))))
(|sayKeyedMsg|
  (format nil "Import from frame %1 is complete. Please issue ~
            )display all if you wish to see the contents of ~
            the current frame.")
  (cons fname nil)))))))))

```

Close an Interpreter Frame

```

[filename p??]
[throwKeyedMsg p??]
[$erase p??]
[makeHistFileName p887]
[updateFromCurrentInterpreterFrame p30]
[$interpreterFrameRing p38]
[$interpreterFrameName p38]

```

closeInterpreterFrame : FrameName → nil
 — defun closeInterpreterFrame —

```

(defun |closeInterpreterFrame| (name)
  "Close an Interpreter Frame"
  (declare (special |$interpreterFrameRing| |$interpreterFrameName|))
  (let (ifr found)
    (if (null (cdr |$interpreterFrameRing|))
      (if (and name (not (equal name |$interpreterFrameName|)))
        (|throwKeyedMsg|
          (format nil "There is only one frame active and therefore that ~
                    cannot be closed. Furthermore, the frame name you gave is not ~
                    the name of the current frame. The current frame is called %1.")
          (cons |$interpreterFrameName| nil))
        (|throwKeyedMsg|
          (format nil "The current frame is the only active one. Issue ~
                    )clear all to clear its contents.")
          nil))
      (progn
        (if (null name)
          (setq |$interpreterFrameRing| (cdr |$interpreterFrameRing|))

```

```

(progn
  (setq found nil)
  (setq ifr nil)
  (dolist (f |$interpreterFrameRing|)
    (if (or found (not (equal name (frameName f))))
        (setq ifr (cons f ifr))
        (setq found t))
    (if (null found)
        (|throwKeyedMsg|
         "There is no frame called %1. Your command cannot be processed."
         (cons name nil))
        (progn
          ($erase (|makeHistFileName| name))
          (setq |$interpreterFrameRing| (nreverse ifr))))))
  (|updateFromCurrentInterpreterFrame|))))

```

3.6 Global variables associated with the frame

defvar \$interpreterFrameRing

All existing frames are kept in a ring held in this variable.

— **initvars** —

```
(defvar |$interpreterFrameRing| nil "The ring of all frames")
```

defvar \$interpreterFrameName

The `$interpreterFrameName` variable, set in `initializeInterpreterFrameRing` to the constant `initial` to indicate that this is the initial (default) frame.

Frames are structures that capture all of the variables defined in a session. There can be multiple frames and the user can freely switch between them. Frames are kept in a ring data structure so you can move around the ring.

— **initvars** —

```
(defvar |$interpreterFrameName| '|initial|)
```

defvar \$InteractiveFrame

\$InteractiveFrame is the environment where the user values are stored. Any side effects of evaluation of a top-level expression are stored in this environment. It is always used as the starting environment for interpretation.

This variable is set in the **restart** function as the value returned by **makeInitialModemapFrame**—.

— **initvars** —

```
(defvar |$InteractiveFrame| nil)
```

—————

The **\$IOindex** variable is the number associated with the input prompt. Every successful expression evaluated increments this number until a **)clear all** resets it. Here we set it to the initial value.

defvar \$IOindex

— **initvars** —

```
(defvar $IOindex 1 "The current Axiom prompt number")
```

—————

3.7 Interpreter Functions using Frames

The **??** function

The **u(p??)ndoSteps** function, part of the undo mechanism can reset the **\$InteractiveFrame**.

Chapter 4

The Message Mechanism

Throughout the interpreter there are messages printed using a symbol for a database lookup. This was done to enable translation of these messages languages other than English.

Axiom messages are read from a flat file database and returned as one long string. They are preceded in the database by a key and this is how they are referenced from code. For example, one key is S2IL0001 which means:

S2	Scratchpad II designation
I	from the interpreter
L	originally from LISPLIB BOOT
0001	a sequence number

Each message may contain formatting codes and and parameter codes. The formatting codes are:

%ceoff	turn off centering
%ceon	turn on centering
%d	turn off bright printing
%f	user defined printing
%i	start indentation of 3 more spaces
%l	start a new line
%m	math-print an expression
%rjoff	turn off right justification (actually ragged left)
%rjon	turn on right justification (actually ragged left)
%s	pretty-print as an S-expression
%u	unindent 3 spaces
%x#	insert # spaces

The parameter codes look like %1, %2b, %3p, %4m, %5bp, %6s where the digit is the parameter number and the letters following indicate additional formatting. You can indicate as many additional formatting qualifiers as you like, to the degree they make sense.

- The “p” code means to call prefix2String on the parameter, a standard way of printing

abbreviated types.

- The “P” operator maps `prefix2String` over its arguments.
- The “o” operation formats the argument as an operation name.
- The “b” means to print that parameter in a bold (bright) font.
- The “c” means to center that parameter on a new line.
- The “r” means to right justify (ragged left) the argument.
- The “f” means that the parameter is a list `[fn, :args]` and that “fn” is to be called on “args” to get the text.

defvar \$msgAlist

— initvars —

```
(defvar |$msgAlist| nil)
```

—————

defvar \$testingErrorPrefix

— initvars —

```
(defvar |$testingErrorPrefix| "Daly Bug")
```

—————

defvar \$msgdbPrims

— initvars —

```
(defvar |$msgdbPrims|
  '(|%b| |%d| |%l| |%i| |%u| %U |%n| |%x| |%ce| |%rj| "%U" "%b" "%d"
    "%l" "%i" "%u" "%U" "%n" "%x" "%ce" "%rj"))
```

—————

defvar \$msgdbPunct

— initvars —

```
(defvar |$msgdbPunct|
  '(|.| |,| ! |:| |;| ? ] |)| "." " ," "!" ":" ";" " "?" "]" " ")
```

—————

defvar \$msgdbNoBlanksBeforeGroup

— initvars —

```
(defvar |$msgdbNoBlanksBeforeGroup|
  '(" " | | "%" % ,@|$msgdbPrims| ,@|$msgdbPunct|))
```

—————

defvar \$msgdbNoBlanksAfterGroup

— initvars —

```
(defvar |$msgdbNoBlanksAfterGroup|
  '(" " | | "%" % ,@|$msgdbPrims| [ |(| "[" "("))
```

—————

defun Say a message using a keyed lookup

[sayKeyedMsgLocal [p44](#)]

— defun sayKeyedMsg —

```
(defun |sayKeyedMsg| (key args)
  (|sayKeyedMsgLocal| key args))
```

—————

defun Handle msg formatting and print to file

```
[segmentKeyedMsg p44]
[substituteSegmentedMsg p??]
[flowSegmentedMsg p??]
[sayMSG2File p44]
[sayMSG p45]
[printMsgsToFile p1012]
[$linelength p1051]
[$margin p1051]
[$displayMsgNumber p1018]
```

— defun sayKeyedMsgLocal —

```
(defun |sayKeyedMsgLocal| (key args)
  (let (msg msgp)
    (declare (special |$printMsgsToFile| $linelength $margin |$displayMsgNumber|))
    (setq msg (|segmentKeyedMsg| key))
    (setq msg (|substituteSegmentedMsg| msg args))
    (when |$displayMsgNumber| (setq msg '(,key |:| . ,msg)))
    (setq msgp (|flowSegmentedMsg| msg $linelength $margin))
    (when |$printMsgsToFile| (|sayMSG2File| msgp))
    (|sayMSG| msgp)))
```

—————

defun Break a message into words

```
[string2Words p??]
```

— defun segmentKeyedMsg —

```
(defun |segmentKeyedMsg| (msg) (|string2Words| msg))
```

—————

defun Write a msg into spadmsg.listing file

```
[makePathname p1233]
[defiostream p1168]
[sayBrightly1 p1240]
[shut p1168]
```

— defun sayMSG2File —

```
(defun |sayMSG2File| (msg)
  (let (file str)
    (setq file (|makePathname| '|spadmsg| '|listing| 'a))
    (setq str (defiostream '((mode . output) (file . ,file)) 255 0))
    (sayBrightly1 msg str)
    (shut str)))
```

defun sayMSG

```
[saybrightly1 p??]
[$algebraOutputStream p1035]
```

— defun sayMSG —

```
(defun |sayMSG| (x)
  (declare (special |$algebraOutputStream|))
  (when x (sayBrightly1 x |$algebraOutputStream|)))
```

Chapter 5

The History Mechanism

defvar \$HiFiAccess

The `$HiFiAccess` is set by `initHist` to `T`. It is a flag used by the history mechanism to record whether the history function is currently on. It can be reset by using the axiom command

```
)history off
```

It appears that the name means “History File Access”.

The `$HiFiAccess` variable is used by `historySpad2Cmd` to check whether history is turned on. `T` means it is, `NIL` means it is not. This is remembered in the current frame.

— **initvars** —

```
(defvar |$HiFiAccess| nil "Is the history function on?")
```

—————

defvar \$HistList

This `$HistList` variable is set by `initHistList` to an initial value of `NIL` elements. The last element of the list is smashed to point to the first element to make the list circular. This is a circular list of length `$HistListLen`. This is remembered in the current frame.

— **initvars** —

```
(defvar |$HistList| nil "A circular list of history elements")
```

—————

defvar \$HistListLen

The `$HistListLen` variable is set by `initHistList` to 20. This is the length of a circular list maintained in the variable `$HistList`. This is remembered in the current frame.

— **initvars** —

```
(defvar |$HistListLen| 0 "The length of the circular history list")
```

defvar \$HistListAct

The `$HistListAct` variable is set by `initHistList` to 0. This variable holds the actual number of elements in the history list. This is the number of “undoable” steps. This is remembered in the current frame.

— **initvars** —

```
(defvar |$HistListAct| 0 "The number of un-doable steps")
```

defvar \$internalHistoryTable

The `$internalHistoryTable` variable is set at load time by a call to `initvars` to a value of `NIL`. It is part of the history mechanism. This is remembered in the current frame.

— **initvars** —

```
(defvar |$internalHistoryTable| nil)
```

defvar \$HistRecord

The `$HistRecord` variable is set by `initHistList` to `NIL`. `$HistRecord` collects the input line, all variable bindings and the output of a step, before it is written to the file named by the function `histFileName`.

— **initvars** —

```
(defvar |$HistRecord| nil)
```

defvar \$historyFileType

The `$historyFileType` is set at load time by a call to `initvars` to a value of “axh”. It appears that this is intended to be used as a filetype extension. It is part of the history mechanism. It is used in `makeHistFileName` as part of the history file name.

— **initvars** —

```
(defvar |$historyFileType| nil)
```

Chapter 6

The undo mechanism

6.1 Data Structures

`$frameRecord = [delta1, delta2, ... ]` where `delta(i)` contains changes in the “backwards” direction. Each `delta(i)` has the form `((var . proplist)...)` where `proplist` denotes an ordinary proplist. For example, an entry of the form `((x (value) (mode (Integer))))` indicates that to undo 1 step, `x`’s value is cleared and its mode should be set to `(Integer)`.

A `delta(i)` of the form `(systemCommand . delta)` is a special delta indicating changes due to system commands executed between the last command and the current command. By recording these deltas separately, it is possible to undo to either BEFORE or AFTER the command. These special `delta(i)`s are given ONLY when a system command is given which alters the environment.

`recordFrame('system)` is called before a command is executed, and `recordFrame('normal)` is called after (see `processInteractive1`). If no changes are found for former, no special entry is given.

The `$previousBindings` is a copy of the CAAR `$InteractiveFrame`. This is used to compute the `delta(i)`s stored in `$frameRecord`.

6.2 Initial Undo Variables

`defvar $frameRecord`

— initvars —

```
(defvar |$frameRecord| nil "a list of value changes")
```

defvar \$previousBindings

— initvars —

```
(defvar |$previousBindings| nil "a copy of Interactive Frame info for undo")
```

defvar \$reportundo

— initvars —

```
(defvar |$reportundo| nil "t means we report the steps undo takes")
```

6.3 The undo functions

defun undo

```
[stringPrefix? p1403]
[pname p1235]
[read p943]
[userError p??]
[qcdr p??]
[qcar p??]
[identp p1237]
[undoSteps p53]
[undoCount p61]
[$options p73]
[$InteractiveFrame p39]
```

— defun undo —

```
(defun |undo| (l)
  (let (tmp1 key s undoWhen n)
    (declare (special |$options| |$InteractiveFrame|))
    (setq undoWhen ' |after|)
```

```

(when
  (and (consp |$options|)
        (eq (qcdr |$options|) nil)
        (progn
          (setq tmp1 (qcar |$options|))
          (and (consp tmp1)
                (eq (qcdr tmp1) nil)
                (progn (setq key (qcar tmp1)) t))))
  (cond
    ((|stringPrefix?| (setq s (pname key)) "redo")
     (setq |$options| nil)
     (|read| '(|redo.input|)))
    ((null (|stringPrefix?| s "before"))
     (|userError| "only option to undo is \"redo\""))
    (t
     (setq undoWhen '|before|))))))
(if (null l)
    (setq n (- 1))
    (setq n (car l)))
(when (identp n)
  (setq n (parse-integer (pname n)))
  (unless (integerp n)
    (|userError| "undo argument must be an integer")))
(setq |$InteractiveFrame| (|undoSteps| (|undoCount| n) undoWhen))
nil))

```

defun undoSteps

```

-- undoes m previous commands; if )before option, then undo one extra at end
--Example: if $I0index now is 6 and m = 2 then general layout of $frameRecord,
-- after the call to recordFrame below will be:
-- (<change for systemcommands>
-- (<change for #5> <change for system commands>
-- (<change for #4> <change for system commands>
-- (<change for #3> <change for system commands>
-- <change for #2> <change for system commands>
-- <change for #1> <change for system commands>) where system
-- command entries are optional and identified by (systemCommand . change).
-- For a ")undo 3 )after", m = 2 and undoStep swill restore the environment
-- up to, but not including <change for #3>.
-- An "undo 3 )before" will additionally restore <change for #3>.
-- Thus, the later requires one extra undo at the end.

```

[writeInputLines p896]
 [recordFrame p1123]

```
[copy p??]
[undoSingleStep p54]
[qcdr p??]
[qcar p??]
[$IOindex p39]
[$InteractiveFrame p39]
[$frameRecord p51]
```

— defun undoSteps —

```
(defun |undoSteps| (m beforeOrAfter)
  (let (tmp1 tmp2 systemDelta lastTailSeen env)
    (declare (special |$IOindex| |$InteractiveFrame| |$frameRecord|))
    (|writeInputLines| '|redo| (- |$IOindex| m))
    (recordFrame 'normal)
    (setq env (copy (caar |$InteractiveFrame|)))
    (do ((i 0 (1+ i)) (framelist |$frameRecord| (cdr framelist)))
        ((or (> i m) (atom framelist)) nil)
      (setq env (|undoSingleStep| (CAR framelist) env))
      (if (and (consp framelist)
                (progn
                  (setq tmp1 (qcdr framelist))
                  (and (consp tmp1)
                        (progn
                          (setq tmp2 (qcar tmp1))
                          (and (consp tmp2)
                                (eq (qcar tmp2) '|systemCommand|)
                                (progn
                                  (setq systemDelta (qcdr tmp2))
                                  t)))))))
          (progn
            (setq framelist (cdr framelist))
            (setq env (|undoSingleStep| systemDelta env)))
          (setq lastTailSeen framelist)))
    (cond
      ((eq beforeOrAfter '|before|)
       (setq env (|undoSingleStep| (car (cdr lastTailSeen)) env))))
    (setq |$frameRecord| (cdr |$frameRecord|))
    (setq |$InteractiveFrame| (list (list env)))))
```

defun undoSingleStep

```
undoSingleStep(changes,env) ==
--Each change is a name-proplist pair. For each change:
-- (1) if there exists a proplist in env, then for each prop-value change:
```

```
--      (a) if the prop exists in env, RPLAC in the change value
--      (b) otherwise, CONS it onto the front of prop-values for that name
-- (2) add change to the front of env
-- pp '-----Undoing 1 step-----"
-- pp changes
```

```
[assq p1240]
[seq p??]
[exit p??]
[lassoc p??]
[undoLocalModemapHack p56]
```

— defun undoSingleStep —

```
(defun |undoSingleStep| (changes env)
  (prog (name changeList pairlist proplist prop value node)
    (return
      (seq
        (progn
          (do ((tmp0 changes (cdr tmp0)) (|change| nil))
              ((or (atom tmp0)
                   (progn (setq |change| (car tmp0)) nil)
                   (progn
                     (progn
                       (setq name (car |change|))
                       (setq changeList (cdr |change|))
                       |change|)
                     nil))
              nil)
          (seq
            (exit
              (progn
                (when (lassoc '|localModemap| changeList)
                  (setq changeList (|undoLocalModemapHack| changeList)))
                (cond
                  ((setq pairlist (assq name env))
                   (cond
                     ((setq proplist (cdr pairlist))
                      (do ((tmp1 changeList (cdr tmp1)) (pair nil))
                          ((or (atom tmp1)
                               (progn (setq pair (car tmp1)) nil)
                               (progn
                                 (progn
                                   (setq prop (car pair))
                                   (setq value (cdr pair))
                                   pair)
                                 nil))
                      nil)
                   (seq
```

```

      (exit
      (cond
        ((setq node (assq prop proplist))
         (rplacd node value))
        (t
         (rplacd proplist
          (cons (car proplist) (cdr proplist)))
         (rplaca proplist pair))))))
      (t (rplacd pairlist changeList)))
    (t
     (setq env (cons |change| env))))))
  env))))

```

defun undoLocalModemapHack

```

[seq p??]
[exit p??]

```

— defun undoLocalModemapHack —

```

(defun |undoLocalModemapHack| (changeList)
  (prog (name value)
    (return
     (seq
      (prog (tmp0)
        (setq tmp0 nil)
        (return
         (do ((tmp1 changeList (cdr tmp1)) (pair nil))
              ((or (atom tmp1)
                   (progn (setq pair (car tmp1)) nil)
                   (progn
                     (progn
                      (setq name (car pair))
                      (setq value (cdr pair))
                      pair)
                     nil))
              (nreverse0 tmp0))
         (seq
          (exit
           (cond
            ((cond
              ((eq name '|localModemap|) (cons name nil))
              (t pair))
             (setq tmp0
              (cons

```

```

(cond
  ((eq name '|localModemap|) (cons name nil))
  (t pair)) tmp0)))))))))

```

Remove undo lines from history write

Removing undo lines from `)hist )write linelist` [stringPrefix? p1403]

```

[seq p??]
[exit p??]
[trimString p??]
[substring p324]
[charPosition p??]
[maxindex p??]
[undoCount p61]
[concat p1237]
[$currentLine p??]
[$IOindex p39]

```

— defun removeUndoLines —

```

(defun |removeUndoLines| (u)
  "Remove undo lines from history write"
  (prog (xtra savedIOindex s s1 m s2 x code c n acc)
    (declare (special |$currentLine| |$IOindex|))
    (return
      (seq
        (progn
          (setq xtra
            (cond
              ((stringp |$currentLine|) (cons |$currentLine| nil))
              (t (reverse |$currentLine|))))
          (setq xtra
            (prog (tmp0)
              (setq tmp0 nil)
              (return
                (do ((tmp1 xtra (cdr tmp1)) (x nil))
                  ((or (atom tmp1)
                     (progn (setq x (car tmp1)) nil))
                   (nreverse0 tmp0)))
              (seq
                (exit
                  (cond
                    ((null (|stringPrefix?| ")history" x))
                    (setq tmp0 (cons x tmp0))))))
              (setq u (append u xtra))

```



```

(cond
  ((null
    (prog (tmp2)
      (setq tmp2 nil)
      (return
        (do ((tmp3 nil tmp2) (tmp4 u (cdr tmp4)) (x nil))
          ((or tmp3 (atom tmp4) (progn (setq x (car tmp4)) nil)) tmp2)
        (seq
          (exit
            (setq tmp2
              (or tmp2 (|stringPrefix?| ")undo" x))))))))) u)
  (t
    (setq savedI0index |$I0index|)
    (setq |$I0index| 1)
    (do ((y u (cdr y)))
      ((atom y) nil)
      (seq
        (exit
          (cond
            ((eq1 (elt (setq x (car y)) 0) #\ )
              (cond
                ((|stringPrefix?| ")undo"
                  (setq s (|trimString| x)))
                (setq s1 (|trimString| (substring s 5 nil)))
                (cond
                  ((not (string= s1 ")redo"))
                  (setq m (|charPosition| #\ ) s1 0))
                  (setq code
                    (cond
                      ((> (maxindex s1) m) (elt s1 (1+ m)))
                      (t #\a)))
                  (setq s2 (|trimString| (substring s1 0 m))))))
              (setq n
                (cond
                  ((string= s1 ")redo")
                  0)
                  ((not (string= s2 ""))
                    (|undoCount| (parse-integer s2)))
                  (t -1)))
              (rplaca y
                (concat ">" code (princ-to-string n))))
              (t nil)))
            (t (setq |$I0index| (1+ |$I0index|))))))
    (setq acc nil)
    (do ((y (nreverse u) (cdr y)))
      ((atom y) nil)
      (seq
        (exit
          (cond
            ((eq1 (elt (setq x (car y)) 0) #\>))

```

```

(setq code (elt x 1))
(setq n (parse-integer (substring x 2 nil)))
(setq y (cdr y))
(do ()
  ((null y) nil)
  (seq
    (exit
      (progn
        (setq c (car y))
        (cond
          ((or (eql (elt c 0) #\))
              (eql (elt c 0) #\>))
            (setq y (cdr y)))
          ((eql n 0)
            (return nil))
          (t
            (setq n (- n 1))
            (setq y (cdr y))))))))
    (cond
      ((and y (not (eql code #\b)))
        (setq acc (cons c acc))))
      (t (setq acc (cons x acc))))))
(setq |$IOindex| savedIOindex)
acc))))))

```

defun reportUndo

This function is enabled by setting `$reportundo` to a non-nil value. An example of the output generated is:

```
r := binary(22/7)
```

```

---
(1)  11.001

```

Type: BinaryExpansion

Properties of % ::

value was: NIL

value is: ((|BinaryExpansion|) WRAPPED . #(1 (1 1) NIL (0 0 1)))

Properties of r ::

value was: NIL

value is: ((|BinaryExpansion|) WRAPPED . #(1 (1 1) NIL (0 0 1)))

```

[seq p??]
[exit p??]

```

```

[sayBrightly p??]
[concat p1237]
[pname p1235]
[lassoc p??]
[sayBrightlyNT p??]
[pp p??]
[$InteractiveFrame p39]

```

— defun reportUndo —

```

(defun |reportUndo| (acc)
  (prog (name proplist curproplist prop value)
    (declare (special |$InteractiveFrame|))
    (return
      (seq
        (do ((tmp0 acc (cdr tmp0)) (tmp1 nil))
          ((or (atom tmp0)
              (progn (setq tmp1 (car tmp0)) nil)
              (progn
                (progn
                  (setq name (car tmp1))
                  (setq proplist (cdr tmp1))
                  tmp1)
                nil))
              nil))
          nil)
        (seq
          (exit
            (progn
              (|sayBrightly|
                (concat '|Properties of | (pname name) " ::"))
              (setq curproplist (lassoc name (caar |$InteractiveFrame|)))
              (do ((tmp2 proplist (cdr tmp2)) (tmp3 nil))
                ((or (atom tmp2)
                    (progn (setq tmp3 (car tmp2)) nil)
                    (progn
                      (progn
                        (setq prop (car tmp3))
                        (setq value (cdr tmp3))
                        tmp3)
                      nil))
                    nil))
                nil)
              (seq
                (exit
                  (progn
                    (|sayBrightlyNT|
                      (cons " " (cons prop (cons " was: " nil))))
                    (|pp| value)
                    (|sayBrightlyNT|
                      (cons " " (cons prop (cons " is: " nil))))

```

```
(|pp| (lassoc prop curproplist)))))))))))))
```

Undo previous n commands

```
[userError p??]  
[concat p1237]  
[$IOindex p39]
```

— defun undoCount —

```
(defun |undoCount| (n)  
  "Undo previous n commands"  
  (prog (m)  
    (declare (special |$IOindex|))  
    (return  
      (progn  
        (setq m  
          (cond  
            ((>= n 0) (- (- |$IOindex| n) 1))  
            (t (- n))))  
        (cond  
          ((>= m |$IOindex|)  
           (|userError|  
            (concat "Magnitude of undo argument must be less than step number ("  
              (princ-to-string |$IOindex|) ")."))))  
          (t m))))))
```

Chapter 7

Tracing

7.1 The help text

— trace.help —

```
=====
A.26. )trace
=====
```

User Level Required: interpreter

Command Syntax:

-)trace
-)trace)off

-)trace function [options]
-)trace constructor [options]
-)trace domainOrPackage [options]

where options can be one or more of

-)after S-expression
-)before S-expression
-)break after
-)break before
-)cond S-expression
-)count
-)count n
-)depth n
-)local op1 [... opN]
-)nonquietly

```

- )nt
- )off
- )only listOfDataToDisplay
- )ops
- )ops op1 [... opN ]
- )restore
- )stats
- )stats reset
- )timer
- )varbreak
- )varbreak var1 [... varN ]
- )vars
- )vars var1 [... varN ]
- )within executingFunction

```

Command Description:

This command is used to trace the execution of functions that make up the AXIOM system, functions defined by users, and functions from the system library. Almost all options are available for each type of function but exceptions will be noted below.

To list all functions, constructors, domains and packages that are traced, simply issue

```
)trace
```

To untrace everything that is traced, issue

```
)trace )off
```

When a function is traced, the default system action is to display the arguments to the function and the return value when the function is exited. Note that if a function is left via an action such as a THROW, no return value will be displayed. Also, optimization of tail recursion may decrease the number of times a function is actually invoked and so may cause less trace information to be displayed. Other information can be displayed or collected when a function is traced and this is controlled by the various options. Most options will be of interest only to AXIOM system developers. If a domain or package is traced, the default action is to trace all functions exported.

Individual interpreter, lisp or boot functions can be traced by listing their names after)trace. Any options that are present must follow the functions to be traced.

```
)trace f
```

traces the function f. To untrace f, issue

```
)trace f )off
```

Note that if a function name contains a special character, it will be necessary to escape the character with an underscore

```
)trace _/D_,1
```

To trace all domains or packages that are or will be created from a particular constructor, give the constructor name or abbreviation after)trace.

```
)trace MATRIX
)trace List Integer
```

The first command traces all domains currently instantiated with Matrix. If additional domains are instantiated with this constructor (for example, if you have used Matrix(Integer) and Matrix(Float)), they will be automatically traced. The second command traces List(Integer). It is possible to trace individual functions in a domain or package. See the)ops option below.

The following are the general options for the)trace command.

```
)break after
  causes a Lisp break loop to be entered after exiting the traced function.
```

```
)break before
  causes a Lisp break loop to be entered before entering the traced
  function.
```

```
)break
  is the same as )break before.
```

```
)count
  causes the system to keep a count of the number of times the traced
  function is entered. The total can be displayed with )trace )stats and
  cleared with )trace )stats reset.
```

```
)count n
  causes information about the traced function to be displayed for the
  first n executions. After the nth execution, the function is untraced.
```

```
)depth n
  causes trace information to be shown for only n levels of recursion of
  the traced function. The command
```

```
)trace fib )depth 10
```

will cause the display of only 10 levels of trace information for the recursive execution of a user function fib.

`)math`
causes the function arguments and return value to be displayed in the AXIOM monospace two-dimensional math format.

`)nonquietly`
causes the display of additional messages when a function is traced.

`)nt`
This suppresses all normal trace information. This option is useful if the `)count` or `)timer` options are used and you are interested in the statistics but not the function calling information.

`)off`
causes untracing of all or specific functions. Without an argument, all functions, constructors, domains and packages are untraced. Otherwise, the given functions and other objects are untraced. To immediately retrace the untraced functions, issue `)trace )restore`.

`)only listOfDataToDisplay`
causes only specific trace information to be shown. The items are listed by using the following abbreviations:

<code>a</code>	display all arguments
<code>v</code>	display return value
<code>1</code>	display first argument
<code>2</code>	display second argument
<code>15</code>	display the 15th argument, and so on

`)restore`
causes the last untraced functions to be retraced. If additional options are present, they are added to those previously in effect.

`)stats`
causes the display of statistics collected by the use of the `)count` and `)timer` options.

`)stats reset`
resets to 0 the statistics collected by the use of the `)count` and `)timer` options.

`)timer`
causes the system to keep a count of execution times for the traced function. The total can be displayed with `)trace )stats` and cleared with `)trace )stats reset`.

`)varbreak var1 [... varN]`
causes a Lisp break loop to be entered after the assignment to any of the listed variables in the traced function.

`)vars`

causes the display of the value of any variable after it is assigned in the traced function. Note that library code must have been compiled (see description of command `)compile`) using the `)vartrace` option in order to support this option.

`)vars var1 [... varN]`

causes the display of the value of any of the specified variables after they are assigned in the traced function. Note that library code must have been compiled (see description of command `)compile`) using the `)vartrace` option in order to support this option.

`)within executingFunction`

causes the display of trace information only if the traced function is called when the given `executingFunction` is running.

The following are the options for tracing constructors, domains and packages.

`)local [op1 [... opN] ]`

causes local functions of the constructor to be traced. Note that to untrace an individual local function, you must use the fully qualified internal name, using the escape character `_` before the semicolon.

`)trace FRAC )local`

`)trace FRAC_;cancelGcd )off`

`)ops op1 [... opN]`

By default, all operations from a domain or package are traced when the domain or package is traced. This option allows you to specify that only particular operations should be traced. The command

`)trace Integer )ops min max _+ _-`

traces four operations from the domain `Integer`. Since `+` and `-` are special characters, it is necessary to escape them with an underscore.

Also See:

- o `)lisp`
- o `)ltrace`

1

¹ “`lisp`” (26.27 p 935) “`ltrace`” (26.28 p 936)

7.2 The trace global variables

defvar \$breakCondition

— initvars —

(defvar |\$breakCondition| nil)

—————

defvar \$constructors

— initvars —

(defvar |\$constructors| nil)

—————

defvar \$constructors

— initvars —

(defvar |\$constructors| nil)

—————

defvar \$countList

— initvars —

(defvar |\$countList| nil "A list of traced functions to count")

—————

defvar \$depthAlist

— initvars —

```
(defvar |$depthAlist| nil)
```

—————

defvar \$domains

— initvars —

```
(defvar |$domains| nil)
```

—————

defvar \$domainTraceNameAssoc

This is an alist whose car is the domain and whose cdr is a gensym.

— initvars —

```
(defvar |$domainTraceNameAssoc| nil)
```

—————

defvar \$doNotAddEmptyModeIfTrue

— initvars —

```
(defvar |$doNotAddEmptyModeIfTrue| nil)
```

—————

defvar \$embeddedFunctions

— initvars —

```
(defvar |$embeddedFunctions| nil)
```

```
defvar $fromSpadTrace
```

```
— initvars —
```

```
(defvar |$fromSpadTrace| nil)
```

```
defvar $lastUntraced
```

```
— initvars —
```

```
(defvar |$lastUntraced| nil)
```

```
defvar $letAssoc
```

```
— initvars —
```

```
(defvar |$letAssoc| nil)
```

```
defvar $mapSubNameAlist
```

```
— initvars —
```

```
(defvar |$mapSubNameAlist| nil)
```

defvar \$mathTrace

— initvars —

```
(defvar |$mathTrace| nil)
```

—————

defvar \$mathTraceList

— initvars —

```
(defvar |$mathTraceList| nil "A list of functions with math trace output")
```

—————

defvar \$monitorArgs

— initvars —

```
(defvar |$monitorArgs| nil)
```

—————

defvar \$monitorCaller

— initvars —

```
(defvar |$monitorCaller| nil)
```

—————

defvar \$monitorDepth

— initvars —

```
(defvar |$monitorDepth| 0)
```

defvar \$monitorFunDepth

— initvars —

```
(defvar |$monitorFunDepth| nil)
```

defvar \$monitorName

— initvars —

```
(defvar |$monitorName| nil)
```

defvar \$monitorPretty

— initvars —

```
(defvar |$monitorPretty| nil)
```

defvar \$monitorValue

— initvars —

```
(defvar |$monitorValue| nil)
```

defvar \$OptionAlist

— initvars —

(defvar |\$OptionAlist| nil)

—————

defvar \$options

— initvars —

(defvar |\$options| nil)

—————

defvar \$OutputForm

— initvars —

(defvar |\$OutputForm| nil)

—————

defvar \$packages

— initvars —

(defvar |\$packages| nil)

—————

defvar \$QuickLet

— initvars —


```
(defvar |$QuickLet| nil)
```

defvar \$reportSpadtrace

This reports the traced functions

— **initvars** —

```
(defvar |$reportSpadtrace| nil)
```

defvar \$spaceList

— **initvars** —

```
(defvar |$spaceList| nil "A list of traced functions to calculate space used")
```

defvar \$streamCount

— **initvars** —

```
(defvar |$streamCount| t)
```

defvar \$timerList

— **initvars** —

```
(defvar |$timerList| nil "A list of traced functions to time")
```

defvar \$tracedMapSignatures

— initvars —

(defvar |\$tracedMapSignatures| nil)

—————

defvar \$traceDomains

— initvars —

(defvar |\$traceDomains| t)

—————

defvar \$traceErrorStack

— initvars —

(defvar |\$traceErrorStack| t)

—————

defvar \$TraceFlag

— initvars —

(defvar |\$TraceFlag| t)

—————

defvar \$traceletflag

— initvars —

```
(defvar $traceletflag nil)
```

defvar \$traceletFunctions

— initvars —

```
(defvar |$traceletFunctions| nil)
```

defvar \$traceNames

— initvars —

```
(defvar |$traceNames| nil "The names of traced domains and packages")
```

defvar \$traceNoisely

This decides when to give trace and untrace messages.

— initvars —

```
(defvar |$traceNoisely| nil)
```

defvar \$traceOptionList

— initvars —

```
(defvar |$traceOptionList|
  '(|after| |before| |break| |cond| |count| |depth| |local| |mathprint|
    |nonquietly| |nt| |of| |only| |ops| |restore| |timer| |varbreak|
    |vars| |within|))
```

defvar \$tracedSpadModemap

— initvars —

```
(defvar |$tracedSpadModemap| nil)
```

—————

defvar \$traceSize

This is the size limit of output during tracing. See [7.4](#)

— initvars —

```
(defvar |$traceSize| nil "size limit of output during tracing")
```

—————

defvar \$traceStream

— initvars —

```
(defvar |$traceStream| *standard-output*)
```

—————

7.3 The trace initialization

— postvars —

```
(eval-when (eval load)
  (put '|coerce| '/TRANSFORM '(& & *))
  (put '|comp| '/TRANSFORM '(& * * &))
  (put '|compIf| '/TRANSFORM '(& * * &))
  ; by having no transform for the 3rd argument, it is simply not printed
  (put '|compFormWithModemap| '/TRANSFORM '(& * * & &)))
```

—————

7.4 The trace functions

defun The Top Level)trace Command Handler

[traceSpad2Cmd p78]

— defun trace —

```
(defun |trace| (l)
  (traceSpad2Cmd l))
```

—————

defun traceSpad2Cmd

```
[getMapSubNames p130]
[trace1 p79]
[augmentTraceNames p78]
[traceReply p94]
[$mapSubNameAlist p70]
[$options p73]
```

— defun traceSpad2Cmd —

```
(defun traceSpad2Cmd (l)
  (declare (special |$mapSubNameAlist| |$options|))
  (when (eq (first l) '|Tuple|) (setq l (second l)))
  (setq |$mapSubNameAlist| (getMapSubNames l))
  (trace1 (augmentTraceNames l) |$options|)
  (traceReply))
```

—————

defun augmentTraceNames

If we have values on the **localModemap** property of **\$InteractiveFrame** we collect the names, otherwise we simply add the arguments to the result. [get p??]

[\$InteractiveFrame p39]

— defun augmentTraceNames —

```
(defun augmentTraceNames (arg)
  (let (mml res)
```

```
(declare (special |$InteractiveFrame|))
(loop for traceName in arg do
  (if (setq mml (|get| tracename '|localModemap| |$InteractiveFrame|))
    (setq res (append (loop for mm in mml collect (cadr mm)))) res)
  (setq res (cons tracename res)))
res))
```

defun trace1

The trace1 function handles the options **off**, **stats**, **restore**, and the **help** options. [hasOption p791]

```
[throwKeyedMsg p??]
[unabbrev p??]
[isFunctor p??]
[getTraceOption p118]
[untraceDomainLocalOps p136]
[qslessp p1313]
[poundsign p??]
[untrace p122]
[ptimers p98]
[say p??]
[pcounters p99]
[selectOptionLC p819]
[resetSpacers p97]
[resetTimers p97]
[resetCounters p98]
[qcar p??]
[qcdr p??]
[vecp p??]
[sayKeyedMsg p43]
[devaluate p??]
[lassoc p??]
[trace1 p79]
[delete p??]
[?t p146]
[seq p??]
[exit p??]
[transTraceItem p126]
[addassoc p??]
[getTraceOptions p116]
[/trace,0 p??]
[saveMapSig p117]
[$traceNoisely p76]
```

[[\\$options](#) p73]

[[\\$lastUntraced](#) p70]

[[\\$optionAlist](#) p73]

— defun trace1 —

```
(defun trace1 (l options)
  (prog (|$traceNoisely| varList y domainList optionList traceList
        opList newOptions oldL a opt constructor lops ops)
    (declare (special |$traceNoisely| |$options|))
    (setq |$traceNoisely| nil)
    (when (|hasOption| options '|nonquietly|) (setq |$traceNoisely| t))
    (cond
      ((|hasOption| options '|off|)
       (cond
         ((or (setq ops (|hasOption| options '|ops|))
              (setq lops (|hasOption| options '|local|))))
         (cond
           ((null l)
            (|throwKeyedMsg|
             (format nil
                  "If you use the )off option for )trace and you also use the ~
                  )local or )ops option, you must specify the name of a ~
                  constructor. You have not done so.")
             nil))
           (t
            (setq constructor
                  (|unabbrev|
                   (cond
                     ((atom l) l)
                     ((null (cdr l))
                      (cond
                        ((atom (car l)) (car l))
                        (t (car (car l))))))
                   (t nil))))
            (cond
              ((null (|isFunction| constructor))
               (|throwKeyedMsg|
                (format nil
                     "If you use the )off option for )trace and you also use ~
                     the )local or )ops option, you must specify the name ~
                     of a constructor. What you gave after )trace is not a ~
                     valid constructor name.")
                nil))
              (t
               (when ops (setq ops (|getTraceOption| ops)) nil)
               (when lops
                (setq lops (cdr (|getTraceOption| lops)))
                (|untraceDomainLocalOps| constructor lops)))))))
```

```

((and (< 1 (length options))
      (null (|hasOption| options '|nonquietly|)))
      (|throwKeyedMsg|
        (format nil
          "If you use the )off option for )trace then the only other ~
          options you can use are )nonquietly, )ops and )local.")
      nil))
(t
  (|untrace| 1)
  (|clearConstructorCaches|)))
((|hasOption| options '|stats|)
  (cond
    ((< 1 (length options))
      (|throwKeyedMsg| "%1 can have no other options."
        (cons ")trace ... )stats" nil)))
    (t
      (setq opt (cdar options))
      (cond
        ((null opt)
          (format t "~v,,,'-:@<~a~>~%" (- $linelength 2)
            " Traced function execution times ")
          (|ptimers|)
          (say " "))
          (format t "~v,,,'-:@<~a~>~%" (- $linelength 2)
            " Traced function execution counts ")
          (|pcounters|))
        (t
          (|selectOptionLC| (car opt) '(|reset|) '|optionError|)
          (|resetSpacers|)
          (|resetTimers|)
          (|resetCounters|)
          (|throwKeyedMsg|
            (format nil
              "Trace facility timers, space counts and execution counts ~
              have been reset.")
            nil))))))
((setq a (|hasOption| options '|restore|))
  (cond
    ((null (setq oldL |$lastUntraced|)) nil)
    (t
      (setq newOptions (|delete| a options))
      (cond
        ((null 1) (trace1 oldL options))
        (t
          (loop for x in 1 do
            (cond
              ((simple-vector-p (car x))
                (|sayKeyedMsg| "Please retrace the domain %1."
                  (list (|devaluate| (car x)))))
              (t

```



```

        (setq options
          (append newOptions (lassoc x |$optionAlist|)))
        (trace1 (list x) options)))))))))
((null l) nil)
((and (consp l) (eq (cdr l) nil) (eq (car l) '?)) (|?t|))
(t
  (setq traceList
    (or
      (loop for x in l collect (|transTraceItem| x))
      (return nil)))
  (loop for x in traceList do
    (setq |$optionAlist| (addassoc x options |$optionAlist|)))
  (setq optionList (getTraceOptions options))
  (cond
    ((setq domainList (lassoc '|of| optionList))
     (cond
       ((lassoc '|ops| optionList)
        (|throwKeyedMsg|
         ")ops and )of cannot both be options to )trace"
         nil))
      (t
       (setq opList (when traceList (list (cons '|ops| traceList)))))
       (setq varList
         (when
           (setq y (lassoc '|vars| optionList)) (list (cons '|vars| y))))
       (setq optionList (append opList varList))
       (setq traceList domainList))))))
  (loop for funName in traceList do
    (trace2 funName nil optionList))
  (|saveMapSig| traceList))))))

```

defun trace2

[trace3 p83]

— defun trace2 —

```

(defun trace2 (fn modemap options)
  (trace3 fn modemap options nil))

```

defun trace3

The **trace3** examines the options to the top level trace command and builds the lambda expression in the variable **newdef**

```
(lambda (&rest g6) (monitorx g6 fn ll))
```

The **fn** variable is an uninterned gensym name that is unique. The **ll** variable is a list of

- **tracename** – The spad internal name for a function from a domain which can be found in the compiler output files. e.g. Polynomial.univariate.21
- (when g4 'macro) – most likely nil
- **tracecode** – a string controlling what tracing will occur where tracecode means

```
; 0:      Caller (0,1)      print caller if 1
; 1:      Value (0,1)      print value if 1
; 2...:   Arguments (0,...,9) stop if 0; print ith if i; all if 9
```

- **countnam** –
- **timernam** –
- **before** – An expression to execute before executing the function. It defaults to

```
(eq nil nil)
```

- **after** –
- **condition** –
- **break** –
- **modemap** – The modemap for the function. For Polynomial.variables.15 it shows up as

```
((List (Symbol)) (Polynomial (Integer)))
```

- (list 'quote t) –

```
[untrace2 p??]
[options2uc p??]
[isFunctor p??]
[traceDomainConstructor p137]
[rassoc p??]
[isSubForRedundantMapName p105]
[rassocSub p104]
[spadThrowBrightly p??]
```

```

[getTraceOption p118]
[tracelet p154]
[breaklet p155]
[isUncompiledMap p104]
[sayBrightly p??]
[isInterpOnlyMap p104]
[isDomainOrPackage p107]
[spadTrace p132]
[strconc p??]
[pname p1235]
[trace2 p82]
[adjoinEqual p??]
[embeddedFunction p88]
[embed2 p88]
[$timerList p74]
[$countList p68]
[$fromSpadTrace p70]
[$traceNoisely p76]
[$mathTraceList p71]
[$mapSubNameAlist p70]
[$traceNames p76]
[$traceDomains p75]

```

— defun trace3 —

```

(defun trace3 (fn modemap options binDef)
  (labels (
    (getTraceOption (traceopts opt)
      (loop for x in traceopts do (when (eq (car x) opt) (return x)))))
    (let (mathTrace vars break varbreak fval u tracename letfuncode before
          after caller fromCondition condition withinCondition g countnam
          countCondition timernam depthCondition onlys f a v c nl buf
          tracecode g4 ll newDef oldDef)
      (declare (special |$traceNames| |$traceDomains| |$mapSubNameAlist|
                        |$mathTraceList| |$traceNoisely| |$fromSpadTrace|
                        |$countList| |$timerList|))
        (when (member fn |$traceNames| :test #'eq) (|untrace2| fn nil))
        (setq options (options2uc options))
        (cond
          ((and |$traceDomains| (|isFunction| fn) (atom fn))
            (|traceDomainConstructor| fn options))
          (t
            (setq mathTrace (getTraceOption options 'mathprint))
            (when
              (and mathTrace
                (null (eq1 (elt (symbol-name fn) 0) #\$_))
                (null (gensymp fn)))
              (if (rassoc fn |$mapSubNameAlist|)

```

```

    (setq |$mathTraceList| (cons fn |$mathTraceList|)))
    (|spadThrowBrightly|
      (format nil "mathprint not available for ~A" fn)))
  (setq vars (getTraceOption options 'vars))
  (when vars
    (setq vars (unless (cdr vars) 'all (cdr vars)))
    (|tracelet| fn binDef vars))
  (setq break (getTraceOption options 'break))
  (setq varbreak (getTraceOption options 'varbreak))
  (when varbreak
    (setq vars (if (null (cdr varbreak)) 'all (cdr varbreak)))
    (|breaklet| fn binDef vars))
  (cond
    ((and (null bindef) (symbolp fn) (null (boundp fn)) (null (fboundp fn)))
      (cond
        ((|isUncompiledMap| fn)
          (|sayBrightly|
            (format nil
              "~A must be compiled before it may be traced -- invoke ~A to compile"
              fn fn)))
          ((|isInterpOnlyMap| fn)
            (|sayBrightly|
              (format nil
                "~A cannot be traced because it is an interpret-only function"
                fn)))
          (t
            (|sayBrightly| (format nil "~A is not a function" fn)))))
      ((and (null bindef) (symbolp fn) (boundp fn)
        (|isDomainOrPackage| (setq fval (eval fn))))
        (|spadTrace| fval options))
      (t
        (when (setq u (getTraceOption options 'mask=))
          (makeprop fn '/transform (elt u 1)))
        (setq |$traceNames|
          (if (and options (getTraceOption options 'alias))
            |$traceNames|
            (cons fn |$traceNames|)))
        (setq tracename
          (cond
            ((setq u (getTraceOption options 'alias))
              (princ-to-string (elt u 1)))
            (t
              (when (and |$traceNoisely| (null vars)
                (null (|isSubForRedundantMapName| fn)))
                (|sayBrightly|
                  (list (|rassocSub| fn |$mapSubNameAlist|) "traced")))
                (princ-to-string fn))))
          (cond
            (|$fromSpadTrace|
              (when mathTrace (push (intern tracename) |$mathTraceList|)))

```

```

(setq letfuncode (list 'eq nil nil))
(setq before
  (if (setq u (getTraceOption options 'before))
      (list 'progn (elt u 1) letfuncode)
      letfuncode)))
(t
  (setq before
    (when (setq u (getTraceOption options 'before)) (elt u 1))))
(setq after
  (when (setq u (getTraceOption options 'after)) (elt u 1)))
(setq caller (getTraceOption options 'caller))
(setq fromCondition
  (if (setq u (getTraceOption options 'from))
      (list 'eq '|#9| (list 'quote (elt u 1)))
      t))
(setq condition
  (if (setq u (getTraceOption options 'when))
      (elt u 1)
      t))
(setq withinCondition t)
(when (setq u (getTraceOption options 'within))
  (setq g
    (intern (strconc (symbol-name fn) "/" (symbol-name (elt u 1)))))
  (set g 0)
  (trace2 (elt u 1) nil
    (list (list 'when nil)
      (list 'before (list 'setq g (list '1+ g)))
      (list 'after (list 'setq g (list '1- g)))))
    (setq withinCondition (list '> g 0)))
  (when (getTraceOption options 'count)
    (setq countnam (intern (strconc tracename '|,COUNT|))))
  (setq countCondition
    (cond
      ((setq u (getTraceOption options 'count))
        (setq |$countList| (|adjoinEqual| tracename |$countList|))
        (cond
          ((and (cdr u) (integerp (elt u 1)))
            (list 'cond (list (list '<= countnam (elt u 1)) t)
              (list t (list '|untrace2| (mkq fn) nil) nil)))
            (t t)))
        (t t)))
    (t t)))
(when (getTraceOption options 'timer)
  (setq timernam (intern (strconc tracename ",TIMER")))
  (setq |$timerList| (|adjoinEqual| tracename |$timerList|)))
(setq depthCondition
  (if (setq u (getTraceOption options 'depth))
      (if (and (cdr u) (integerp (elt u 1)))
          (list '<= '|$monitorFunDepth| (elt u 1))
          (traceOptionError 'depth))
      t))

```

```

(setq condition
  (mkpf
    (list condition withinCondition fromCondition
      countCondition depthCondition)
    'and))
(setq onlys (getTraceOption options 'only))
(setq tracecode
  (cond
    ((getTraceOption options 'nt) "000")
    (t
      (setq onlys
        (mapcar #'(lambda (x) (if (integerp x) x (upcase x))) onlys))
        (setq f (or (member 'f onlys) (member 'full onlys)))
        (setq a (or f (member 'a onlys) (member 'args onlys)))
        (setq v (or f (member 'v onlys) (member 'value onlys)))
        (setq c (or f (member 'c onlys) (member 'caller onlys)))
        (setq nl
          (if a
            (list #\9)
            (loop for x in onlys
              when (and (integerp x) (< 0 x) (< x 9))
              collect (char (princ-to-string x) 0))))
          (cond
            ((null (or a v c nl))
              (if caller "119" "019"))
            (t
              (setq nl (append nl (list #\0)))
              (setq buf (|make_spaces| (cond (a 3) (t (+ 2 (length nl))))))
              (setf (elt buf 0) (if (or c caller) #\1 #\0))
              (setf (elt buf 1) (if v #\1 #\0))
              (cond
                (a
                  (setf (elt buf 2) #\9)
                  buf)
                (t
                  (loop for x in nl for i from 2 do (setf (elt buf i) x))
                  buf))))))
          (setq g4 (macro-function fn))
          (when countnam (set countnam 0))
          (when timernam (set timernam 0))
          (setq ll
            (list 'quote
              (list (intern tracename) (when g4 'macro)
                tracecode countnam timernam before after
                condition break modemap (list 'quote t))))
          (setq newDef
            (list (if g4 'mlambda 'lambda) (list '&rest 'g6)
              (list 'monitorX 'g6 fn ll)))
          (cond
            (binDef (embedFunction fn newDef binDef)))

```

```
(t
  (setq oldDef (symbol-function fn))
  (setq newDef (embedFunction fn new_def old_def))
  (embed2 fn new_def old_def)
  fn))))))
```

defun embedFunction

[flatBvList p89]

— defun embedFunction —

```
(defun embedFunction (name newDef oldDef)
  (let (body op bv)
    (setq newDef
      (cond
        ((null (consp newDef)) newDef)
        (t
         (setq body (cddr newDef))
         (setq op (car newDef))
         (if (and body (setq bv (cadr newDef))
                   (or (eq op 'lambda) (eq op 'mlambda)))
             (if (null (member name (flatBvList bv)))
                 (list op bv
                       (list
                        (cons 'lambda (cons (list name) body))
                        (list 'quote oldDef)))
                 newDef)
             (break))))))
  (coerce newDef 'function)))
```

defun embed2

embed2 : Symbol,Function,Function → Symbol

— defun embed2 0 —

```
(defun embed2 (name newDef oldDef)
  (declare (special |$embeddedFunctions|))
  (setf (symbol-function name) newDef)
  (push (list name newDef oldDef) |$embeddedFunctions|))
```

name)

defun flatBvList

[varp p⁸⁹]
 [refvecp p??]
 [flatBvList p⁸⁹]
 [nconc p??]

— defun flatBvList —

```
(defun flatBvList (bvlist)
  (let (tmp1)
    (cond
      ((varp bvlist) (list bvlist))
      ((refvecp bvlist) (break))
      ((null (consp bvlist)) nil)
      ((eq '= (setq tmp1 (car bvlist)))
       (flatBvList (cdr bvlist)))
      ((varp tmp1) (cons tmp1 (flatBvList (cdr bvlist))))
      ((and (null (consp tmp1)) (null (refvecp tmp1)))
       (flatBvList (cdr bvlist)))
      (t
       (nconc (flatBvList tmp1) (flatBvList (cdr bvlist)))))))
```

defun varp

— defun var —

```
(defun varp (testItem)
  (cond
    ((identp testItem) testItem)
    ((and (consp testItem)
          (or (eq (car testItem) 'fluid) (eq (car testItem) 'lex))
          (consp (cdr testItem)) (identp (cadr testItem)))
     testItem)
    (t nil)))
```

defun monitorX

[monitorXX p90]
 [\$monitorDepth p71]
 [\$depthAlist p69]

— defun monitorX —

```
(defun monitorX (args funct opts)
  (declare (special |$monitorDepth| |$depthAlist|))
  (monitorXX args funct opts |$monitorDepth| |$depthAlist|)))
```

—————

defun monitorXX

[stopTimer p??]
 [rassocSub p104]
 [whocalled p??]
 [moan p??]
 [monitorEvalTran p93]
 [monitorEnter p147]
 [monitorEvalBefore p92]
 [break p156]
 [startTimer p??]
 [timerValue p??]
 [monitorEvalAfter p92]
 [monitorExit p151]
 [\$monitorArgs p71]
 [\$monitorValue p72]
 [\$monitorFunDepth p72]
 [\$depthAlist p69]
 [\$monitorCaller p71]
 [\$breakCondition p68]
 [\$monitorName p72]
 [\$monitorDepth p71]
 [\$tracedSpadModemap p77]
 [\$mathTrace p71]
 [\$mapSubNameAlist p70]

— defun monitorXX —

```
(defun monitorXX (|$monitorArgs| funct opts oldDepth oldDepthAlist)
  (declare (special |$monitorArgs|))
  (let (|$monitorValue| |$monitorFunDepth| |$depthAlist| |$monitorCaller|
```

```

    |$breakCondition| |$monitorName| |$monitorDepth|
    |$tracedSpadModemap| |$mathTrace| evalTime initTime yes
    notTopLevel a v c name1 breakcondition tracedmodemap break
    condition after before timernam countnam tracecode type name)
(declare
  (special |$monitorValue| |$monitorFunDepth| |$depthAlist|
    |$monitorCaller| |$breakCondition| |$monitorName| |$monitorDepth|
    |$tracedSpadModemap| |$mathTrace| |$mapSubNameAlist|))
(|stopTimer|)
(setq name (elt opts 0))
(setq type (elt opts 1))
(setq tracecode (elt opts 2))
(setq countnam (elt opts 3))
(setq timernam (elt opts 4))
(setq before (elt opts 5))
(setq after (elt opts 6))
(setq condition (elt opts 7))
(setq break (elt opts 8))
(setq tracedmodemap (elt opts 9))
(setq breakcondition (elt opts 10))
(setq |$mathTrace| nil)
(setq |$tracedSpadModemap| tracedmodemap)
(setq |$monitorDepth| (+ oldDepth 1))
(setq |$monitorName| (symbol-name name))
(setq name1 (|rassocSub| name |$mapSubNameAlist|))
(setq |$breakCondition| breakcondition)
(setq |$monitorCaller| (|rassocSub| (whocalled 6) |$mapSubNameAlist|))
;TRACECODE meaning:
; 0:      Caller (0,1)      print caller if 1
; 1:      Value (0,1)      print value if 1
; 2...:   Arguments (0,...,9) stop if 0; print ith if i; all if 9
(cond
  ((null (stringp tracecode))
    (moan "set tracecode to \\'1911\\' and restart"))
  (t
    (setq c (digit-char-p (elt tracecode 0)))
    (setq v (digit-char-p (elt tracecode 1)))
    (setq a (digit-char-p (elt tracecode 2)))
    (when countnam (set countnam (+ (eval countnam) 1)))
    (setq |$depthAlist| (copy-tree oldDepthAlist))
    (setq notTopLevel (assoc name |$depthAlist|))
    (if (null notTopLevel)
      (setq |$depthAlist| (cons (cons name 1) |$depthAlist|))
      (rplacd notTopLevel (+ (cdr notTopLevel) 1)))
    (setq |$monitorFunDepth| (cdr (assoc name |$depthAlist|)))
    (setq condition (monitorEvalTran condition nil))
    (setq yes (eval condition))
    (when (member name |$mathTraceList|) (setq |$mathTrace| t))
    (when (and yes |$TraceFlag|) (monitorEnter tracecode c type name name1))
    (when before (monitorEvalBefore before))

```

```

(when (member '|before| break)
  (|break| (list "Break on entering" (symbol-name name1) ":")))
(when timernam (setq initTime (|startTimer|)))
(setq |$monitorValue|
  (if (eq type 'macro)
    (macroexpand funct |$monitorArgs|)
    (apply funct |$monitorArgs|)))
(|stopTimer|)
(setq evalTime nil)
(when timernam (setq evalTime (- (|timerValue|) initTime)))
(when (and timernam (null notTopLevel))
  (set timernam (+ (eval timernam) evalTime)))
(when after (monitorEvalAfter after))
(when (and yes |$TraceFlag|)
  (monitorExit tracecode name name1 V timernam evalTime))
(when (member '|after| break)
  (|break| (list "Break on exiting" (symbol-name name1) ":")))
(|startTimer|)
|$monitorValue|)))

```

defun monitorEvalBefore

[monitorEvalTran [p93](#)]

— defun monitorEvalBefore —

```

(defun monitorEvalBefore (x)
  (eval (monitorEvalTran x nil)))

```

defun monitorEvalAfter

[monitorEvalTran [p93](#)]

— defun monitorEvalAfter —

```

(defun monitorEvalAfter (x)
  (eval (monitorEvalTran x nil)))

```

defun monitorEvalTran

[hasSharpVar p93]
[monitorEvalTran1 p93]

— defun monitorEvalTran —

```
(defun monitorEvalTran (x fg)
  (if (|hasSharpVar| x) (monitorEvalTran1 x fg) x))
```

—————

defun monitorEvalTran1

[monitorEvalTran1 p93]
[isSharpVarWithNum p109]
[monitorGetValue p94]

— defun monitorEvalTran1 —

```
(defun monitorEvalTran1 (x fg)
  (let (n)
    (cond
      ((setq n (|isSharpVarWithNum| x)) (monitorGetValue n fg))
      ((atom x) x)
      (t
       (cons (monitorEvalTran1 (car x) fg)
              (monitorEvalTran1 (cdr x) fg))))))
```

—————

defun hasSharpVar

[hasSharpVar p93]
[isSharpVar p110]

— defun hasSharpVar —

```
(defun |hasSharpVar| (x)
  (cond
    ((and (atom x) (|isSharpVar| x)) t)
    ((atom x) nil)
    (t (or (|hasSharpVar| (car x)) (|hasSharpVar| (cdr x))))))
```

—————

defun monitorGetValue

```
[mkq p??]
[spadThrowBrightly p??]
[$monitorValue p72]
[$monitorCaller p71]
[$monitorName p72]
[$monitorArgs p71]
```

— defun monitorGetValue —

```
(defun monitorGetValue (n fg)
  (cond
    ((eq1 n 0)
     (if fg
        (mkq |$monitorValue|)
        (|spadThrowBrightly| "cannot ask for value before execution")))
    ((eq1 n 9) (mkq |$monitorCaller|))
    ((not (< (size |$monitorArgs|) n)) (mkq (elt |$monitorArgs| (- n 1))))
    (t
     (|spadThrowBrightly|
      (list 'function |$monitorName| "does not have" n "arguments")))))
```

defun traceReply

```
[sayMessage p??]
[sayBrightly p??]
[qcar p??]
[isDomainOrPackage p107]
[addTraceItem p146]
[isFunctor p??]
[isgenvar p??]
[userError p??]
[seq p??]
[exit p??]
[isSubForRedundantMapName p105]
[rassocSub p104]
[poundsign p??]
[sayMSG p45]
[sayBrightlyLength p??]
[flowSegmentedMsg p??]
[concat p1237]
[prefix2String p??]
```

[abbreviate p??]
 [\$domains p69]
 [\$packages p73]
 [\$constructors p68]
 [\$linelength p1051]
 [\$traceNames p76]

— defun traceReply —

```
(defun traceReply ()
  (let (|$domains| |$packages| |$constructors| d functionList displayList)
    (declare (special |$domains| |$packages| |$constructors| |$traceNames|
                      $linelength))
    (setq |$domains| nil)
    (setq |$packages| nil)
    (setq |$constructors| nil)
    (cond
     ((null |$traceNames|) (|sayMessage| "  Nothing is traced now."))
     (t
      (|sayBrightly| " ")
      (loop for x in |$traceNames| do
        (cond
         ((and (consp x) (|isDomainOrPackage| (car x)))
          (|addTraceItem| (car x)))
         ((atom x)
          (cond
           ((|isFunction| x) (|addTraceItem| x))
           ((|isgenvar| x) (|addTraceItem| (eval x)))
           (t (setq functionList (cons x functionList))))))
        (t (|userError| "bad argument to trace"))))))
    (setq functionList
      (prog (t1)
        (setq t1 nil)
        (return
         (do ((t2 functionList (cdr t2)) (x nil))
              ((or (atom t2) (progn (setq x (car t2)) nil)) t1)
          (seq
           (exit
            (cond
             ((null (|isSubForRedundantMapName| x))
              (setq t1
                (append t1
                  (cons (|rassocSub| x |$mapSubNameAlist|)
                        (cons " " nil))))))))))
         (cond
          (functionList
           (cond
            ((eql 2 (|#| functionList))
             (|sayMSG| (cons '|  Function traced: | functionList))))
```

```

((<= (+ 22 (|sayBrightlyLength| functionList)) $linelength)
  (|sayMSG| (cons '| Functions traced: | functionList)))
(t
  (|sayBrightly| " Functions traced:")
  (|sayBrightly|
    (|flowSegmentedMsg| functionList $linelength 6))))))
(when |$domains|
  (setq displayList
    (|concat| (|prefix2String| (car |$domains|))
      (loop for x in (cdr |$domains|) append
        (|concat| ", " (|prefix2String| x)))))
  (when (atom displayList) (setq displayList (list displayList)))
  (|sayBrightly| " Domains traced: ")
  (|sayBrightly| (|flowSegmentedMsg| displayList $linelength 6)))
(when |$packages|
  (setq displayList
    (|concat| (|prefix2String| (car |$packages|))
      (loop for x in (cdr |$packages|) append
        (|concat| ", " (|prefix2String| x)))))
  (when (atom displayList) (setq displayList (list displayList)))
  (|sayBrightly| " Packages traced: ")
  (|sayBrightly| (|flowSegmentedMsg| displayList $linelength 6)))
(when |$constructors|
  (setq displayList
    (|concat| (|abbreviate| (car |$constructors|))
      (loop for x in (cdr |$constructors|) append
        (|concat| ", " (|abbreviate| x)))))
  (when (atom displayList) (setq displayList (list displayList)))
  (|sayBrightly| " Parameterized constructors traced:")
  (|sayBrightly| (|flowSegmentedMsg| displayList $linelength 6))))))

```

defun /options

[/options p96]
[isFunctor p??]

— defun /options —

```

(defun /options (x)
  (cond
    ((atom x) nil)
    ((or (atom (car x)) (|isFunctor| (caar x))) (/options (cdr x)))
    (x)))

```

defun Truncate list L at the point marked by TL.

Truncate list L at the point marked by TL.

— **defun** truncList —

```
(defun truncList (l tl)
  (labels (
    (truncList-1 (l tl)
      (cond
        ((atom l) l)
        ((eql (cdr l) tl) (rplacd l nil))
        ((truncList-1 (cdr l) tl))))))
    (let ((u l))
      (truncList-1 l tl)
      u)))
```

—————

defun resetTimers

[concat p1237]
[timerList p74]

— **defun** resetTimers —

```
(defun |resetTimers| ()
  (declare (special |timerList|))
  (dolist (timer |timerList|)
    (set (intern (concat timer ",TIMER")) 0)))
```

—————

defun resetSpacers

[concat p1237]
[spaceList p74]

— **defun** resetSpacers —

```
(defun |resetSpacers| ()
  (declare (special |spaceList|))
  (dolist (spacer |spaceList|)
    (set (intern (concat spacer ",SPACE")) 0)))
```

—————

defun resetCounters

```
[concat p1237]
[$countList p68]
```

— defun resetCounters —

```
(defun |resetCounters| ()
  (declare (special |$countList|))
  (dolist (k |$countList|)
    (set (intern (concat k ",COUNT")) 0)))
```

defun ptimers

```
[sayBrightly p??]
[bright p??]
[quotient p??]
[concat p1237]
[float p??]
[$timerList p74]
[$timerTicksPerSecond p??]
```

— defun ptimers —

```
(defun |ptimers| ()
  (declare (special |$timerList| |$timerTicksPerSecond|))
  (if (null |$timerList|)
    (|sayBrightly| " no functions are timed")
    (dolist (timer |$timerList|)
      (|sayBrightly|
        ‘(“ ” ,@(|bright| timer) |:| “ ”
          ,(quotient (eval (intern (concat timer ",TIMER"))))
            (float |$timerTicksPerSecond|)) “ sec.”))))))
```

defun pspacers

```
[sayBrightly p??]
[bright p??]
[concat p1237]
[$spaceList p74]
```

— defun pspacers —

```
(defun |pspacers| ()
  (declare (special |$spaceList|))
  (if (null |$spaceList|)
      (|sayBrightly| " no functions have space monitored")
      (dolist (spacer |$spaceList|)
        (|sayBrightly|
          '(" " ,@(|bright| spacer) |: |
            ,(eval (intern (concat spacer " ,SPACE")))) " bytes")))))
```

—————

defun pcounters

```
[sayBrightly p??]
[bright p??]
[concat p1237]
[$countList p68]
```

— defun pcounters —

```
(defun |pcounters| ()
  (declare (special |$countList|))
  (if (null |$countList|)
      (|sayBrightly| " no functions are being counted")
      (dolist (k |$countList|)
        (|sayBrightly|
          '(" " ,@(|bright| k) |:| " " ,(eval (intern (concat k " ,COUNT"))))
            " times")))))
```

—————

defun transOnlyOption

```
[transOnlyOption p99]
[upcase p1272]
[stackTraceOptionError p100]
[qcar p??]
[qcdr p??]
```

— defun transOnlyOption —

```
(defun |transOnlyOption| (arg)
  (let (y n)
    (when (and (consp arg) (progn (setq n (qcar arg)) (setq y (qcdr arg)) t))
      (cond
        ((integerp n) (cons n (|transOnlyOption| y)))
        ((member (setq n (upcase n)) '(v a c)) (cons n (|transOnlyOption| y)))
        (t
         (|stackTraceOptionError|
          (cons
            "%1 The )trace option )only does not permit %2 as a legal option."
            (list (list n))))
         (|transOnlyOption| y))))))
```

defun stackTraceOptionError

[[\\$traceErrorStack](#) p75]

— defun stackTraceOptionError —

```
(defun |stackTraceOptionError| (x)
  (declare (special |$traceErrorStack|))
  (push x |$traceErrorStack|)
  nil)
```

defun removeOption

removeOption : Option $\rightarrow$ List Option

— defun removeOption 0 —

```
(defun removeOption (op options)
  (loop for optentry in options
    when (not (equal (car optentry) op))
    collect optentry))
```

defun domainToGenvar

For a call to trace, such as

```
)trace POLY(INT)
```

This would be called with

```
(POLY INT)
```

which gets unabbreviated to the full form of

```
(|Polynomail| (|Integer|))
```

Since **Polynomial** is a domain we call **genDomainTraceName** to add it to the global alist **\$domainTraceNameAssoc**, which returns a unique new genvar, say **\$1**. We then set the value of **\$1** to the domain (a vector), returning it as the result. [unabbrevAndLoad p??]

[getdatabase p1196]

[opOf p??]

[genDomainTraceName p122]

[evalDomain p1115]

[\$doNotAddEmptyModeIfTrue p69]

— defun domainToGenvar —

```
(defun |domainToGenvar| (arg)
  (let (|$doNotAddEmptyModeIfTrue| y g)
    (declare (special |$doNotAddEmptyModeIfTrue|))
    (setq |$doNotAddEmptyModeIfTrue| t)
    (when
      (and (setq y (|unabbrevAndLoad| arg))
           (eq (getdatabase (|opOf| y)) 'constructorkind) '|domain|))
      (setq g (|genDomainTraceName| y))
      (set g (|evalDomain| y))
      g)))
```

—————

defun subTypes

[lassoc p??]

[seq p??]

[exit p??]

[subTypes p101]

— defun subTypes —

```
(defun |subTypes| (|mm| |sublist|)
  (prog (s)
    (return
```

```

(seq
  (cond
    ((atom |mm|)
      (cond ((setq s (lassoc |mm| |sublist|)) s) (t |mm|)))
    (t
      (prog (t0)
        (setq t0 nil)
        (return
          (do ((t1 |mm| (cdr t1)) (|m| nil))
            ((or (atom t1) (progn (setq |m| (car t1)) nil)) (nreverse0 t0))
            (seq
              (exit
                (setq t0 (cons (|subTypes| |m| |sublist|) t0))))))))))

```

defun isListOfIdentifiers

```

[seq p??]
[exit p??]
[identp p1237]

```

— defun isListOfIdentifiers —

```

(defun |isListOfIdentifiers| (arg)
  (prog ()
    (return
      (seq
        (prog (t0)
          (setq t0 t)
          (return
            (do ((t1 nil (null t0)) (t2 arg (cdr t2)) (x nil))
              ((or t1 (atom t2) (progn (setq x (car t2)) nil)) t0)
              (seq
                (exit
                  (setq t0 (and t0 (identp x))))))))))

```

defun isListOfIdentifiersOrStrings

```

[seq p??]
[exit p??]
[identp p1237]

```

— defun isListOfIdentifiersOrStrings —

```
(defun |isListOfIdentifiersOrStrings| (arg)
  (prog ()
    (return
      (seq
        (prog (t0)
          (setq t0 t)
          (return
            (do ((t1 nil (null t0)) (t2 arg (cdr t2)) (x nil))
              ((or t1 (atom t2) (progn (setq x (car t2)) nil)) t0)
            (seq
              (exit
                (setq t0 (and t0 (or (identp x) (stringp x))))))))))))))
```

defun getPreviousMapSubNames

```
[get p??]
[exit p??]
[seq p??]
[$InteractiveFrame p39]
```

— defun getPreviousMapSubNames —

```
(defun |getPreviousMapSubNames| (Names)
  (let (lmm subs)
    (declare (special |$InteractiveFrame|))
    (loop for mapName in (assocleft (caar |$InteractiveFrame|)) do
      (when (setq lmm (|get| mapname '|localModemap| |$InteractiveFrame|))
        (when (member (cadar lmm) Names)
          (loop for mm in lmm do
            (setq subs (cons (cons mapname (cadr mm)) subs))))))
    subs))
```

defun lassocSub

```
[lassq p??]
```

— defun lassocSub —

```
(defun |lassocSub| (x subs)
  (let (y)
    (if (setq y (lassq x subs))
        y
        x)))
```

defun rassocSub

[rassoc p??]

— defun rassocSub —

```
(defun |rassocSub| (x subs)
  (let (y)
    (if (setq y (|rassoc| x subs))
        y
        x)))
```

defun isUncompiledMap

[get p??]
 [\$InteractiveFrame p³⁹]

— defun isUncompiledMap —

```
(defun |isUncompiledMap| (x)
  (let (y)
    (declare (special |$InteractiveFrame|))
    (when (setq y (|get| x '|value| |$InteractiveFrame|))
      (and
        (eq (caar y) 'map)
        (null (|get| x '|localModemap| |$InteractiveFrame|))))))
```

defun isInterpOnlyMap

[get p??]
 [\$InteractiveFrame p³⁹]

— defun isInterpOnlyMap —

```
(defun |isInterpOnlyMap| (map)
  (let (x)
    (declare (special |$InteractiveFrame|))
    (when (setq x (|get| map '|localModemap| |$InteractiveFrame|))
      (eq (caaar x) '|interpOnly|))))
```

—————

defun isSubForRedundantMapName

```
[rassocSub p104]
[member p1238]
[assocleft p??]
[$mapSubNameAlist p70]
```

— defun isSubForRedundantMapName —

```
(defun |isSubForRedundantMapName| (subname)
  (let (mapname tail)
    (declare (special |$mapSubNameAlist|))
    (when (setq mapname (|rassocSub| subname |$mapSubNameAlist|))
      (when (setq tail
        (member (cons mapname subname) |$mapSubNameAlist| :test #'equalp))
        (member mapname (cdr (assocleft tail))))))))
```

—————

defun untraceMapSubNames

```
[assocright p??]
[/untrace,2 p??]
[setdifference p??]
[getPreviousMapSubNames p103]
[$mapSubNameAlist p70]
[$lastUntraced p70]
[$traceNames p76]
```

— defun untraceMapSubNames —

```
(defun |untraceMapSubNames| (names)
  (let (|$mapSubNameAlist| subs)
```



```
(declare (special |$mapSubNameAlist| |$lastUntraced|))
(if
  (null (setq |$mapSubNameAlist| (|getPreviousMapSubNames| names)))
  nil
  (dolist (name (setq subs (assocright |$mapSubNameAlist|)))
    (when (member name |$traceNames|)
      (/UNTRACE,2| name nil)
      (setq |$lastUntraced| (setdifference |$lastUntraced| subs)))))))
```

defun funfind,LAM

```
[qcar p??]
[SEQ p??]
[isFunctor p??]
[exit p??]
```

— defun funfind,LAM —

```
(defun |funfind,LAM| (functor opname)
  (prog (ops tmp1)
    (return
      (seq
        (progn
          (setq ops (|isFunctor| functor))
          (prog (t0)
            (setq t0 nil)
            (return
              (do ((t1 ops (cdr t1)) (u nil))
                ((or (atom t1) (progn (setq u (car t1)) nil)) (nreverse0 t0))
              (seq
                (exit
                  (cond
                    ((and (consp u)
                      (progn
                        (setq tmp1 (qcar u))
                        (and (consp tmp1) (equal (qcar tmp1) opname))))
                    (setq t0 (cons u t0))))))))))))))
```

defmacro funfind

— defmacro funfind —

```
(defmacro |funfind| (&whole t0 &rest notused)
  (declare (ignore notused))
  (let (t1 t0)
    (cons 'funfind,LAM| (wrap (cdr t1) '(quote quote))))))
```

defun isDomainOrPackage

```
[refvecp p??]
[poundsign p??]
[isFunctor p??]
[opOf p??]
```

— defun isDomainOrPackage —

```
(defun |isDomainOrPackage| (dom)
  (and
    (refvecp dom)
    (> (|#| dom) 0)
    (|isFunctor| (|opOf| (elt dom 0)))))
```

defun flattenOperationAlist

The operation alist for something like POLY(INT) is a list of operations. Each operation, such as '*' is a list of all possible signatures for '*'. For example, the operation alist looks like:

```
(* (($ (|PositiveInteger|) $) NIL . (T . (ELT)))
  (($ $ $) NIL . (T . (ELT)))
  (($ $|#1|) NIL . (T . (ELT)))
  (($|#1| $) NIL . (T . (ELT)))
  (($ (|Fraction| (|Integer|))) NIL
    (|has||#1| (|Algebra| (|Fraction| (|Integer|)))) . (ELT))
  ....)
```

This function rearranges the operations so that each signature is independent. Thus it becomes

```
(* (($ (|PositiveInteger|) $) NIL . (T . (ELT))))
(* (($ $ $) NIL . (T . (ELT))))
(* (($ $|#1|) NIL . (T . (ELT))))
```

```

(* (($ |#1| $) NIL . (T . (ELT))))
(* (($ (|Fraction| (|Integer|))) NIL
  (|has| |#1| (|Algebra| (|Fraction| (|Integer|)))) . (ELT)))
....

```

flattenOperationAlist : OperationAlist $\rightarrow$ OperationAlist
 — defun flattenOperationAlist 0 —

```

(defun flattenOperationAlist (opAlist)
  (let (res)
    (loop for t1 in opAlist do
      (setq res
        (append res (loop for mm in (cdr t1) collect (cons (car t1) mm))))))
  res))

```

defun letPrint

```

[lassoc p??]
[isgenvar p??]
[isSharpVarWithNum p109]
[gensymp p??]
[sayBrightlyNT p??]
[bright p??]
[shortenForPrinting p113]
[hasPair p113]
[pname p1235]
[break p156]
[$letAssoc p70]

```

— defun letPrint —

```

(defun |letPrint| (x |val| |currentFunction|)
  (prog (y)
    (declare (special |$letAssoc|))
    (return
      (progn
        (cond ((and |$letAssoc|
                     (or
                      (setq y (lassoc |currentFunction| |$letAssoc|))
                      (setq y (lassoc '|all| |$letAssoc|))))
              (cond
                ((and (or (eq y '|all|)
                          (member x y))

```

```

      (null
        (or (isgenvar x) (|isSharpVarWithNum| x) (gensymp x))))
      (|sayBrightlyNT| (append (|bright| x) (cons '|: | nil)))
      (prin1 (|shortenForPrinting| |val|))
      (terpri)))
(cond
  ((and (setq y (|hasPair| 'break y))
        (or (eq y '|all|)
            (and (member x y)
                 (null (member (elt (pname x) 0) '($ |#|)))
                 (null (gensymp x))))))
   (|break|
    (append
      (|bright| |currentFunction|)
      (cons "breaks after"
        (append
          (|bright| x)
          (cons " := " (cons (|shortenForPrinting| |val|) nil)))))))
   (t nil))))
|val|)))))

```

defun Identifier beginning with a sharpsign-number?

This tests if x is an identifier beginning with # followed by a number.

```

[isSharpVar p110]
[pname p1235]
[qcsize p??]
[digitp p1235]
[dig2fix p??]

```

— defun isSharpVarWithNum —

```

(defun |isSharpVarWithNum| (x)
  (let (p n d ok c)
    (cond
      ((null (|isSharpVar| x)) nil)
      ((> 2 (setq n (qcsize (setq p (pname x))))) nil)
      (t
       (setq ok t)
       (setq c 0)
       (do ((t1 (1- n)) (i 1 (1+ i)))
           ((or (> i t1) (null ok)) nil)
          (setq d (elt p i))
          (when (setq ok (digitp d))
            (setq c (+ (* 10 c) (dig2fix d)))))))

```

```
(when ok c))))
```

defun Identifier beginning with a sharpsign?

This tests if x is an identifier beginning with #

```
[identp p1237]
```

— defun isSharpVar —

```
(defun |isSharpVar| (x)
  (and (symbolp x)
        (eql (elt (symbol-name x) 0) #\#)))
;      (integerp (parse-integer (symbol-name x) :start 1))))
```

defun letPrint2

```
[letPrint2 p110]
[lassoc p??]
[isgenvar p??]
[isSharpVarWithNum p109]
[gensymp p??]
[mathprint p??]
[print p??]
[hasPair p113]
[pname p1235]
[break p156]
[bright p??]
[$BreakMode p969]
[$letAssoc p70]
```

— defun letPrint2 —

```
(defun |letPrint2| (x |printform| |currentFunction|)
  (prog (|$BreakMode| |flag| y)
    (declare (special |$BreakMode| |$letAssoc|))
    (return
      (progn
        (setq |$BreakMode| nil)
        (cond
```

```

((and |$letAssoc|
  (or (setq y (lassoc |currentFunction| |$letAssoc|))
      (setq y (lassoc '|all| |$letAssoc|))))
(cond
  ((and
    (or (eq y '|all|) (member x y))
    (null (or (isgenvar x) (|isSharpVarWithNum| x) (gensymp x))))
    (setq |$BreakMode| '|letPrint2|)
    (setq |flag| nil)
    (catch '|letPrint2|
      (|mathprint| (cons '= (cons x (cons |printform| nil)))) |flag|)
    (cond
      ((eq |flag| '|letPrint2|) (|print| |printform|))
      (t nil))))
  (cond
    ((and
      (setq y (|hasPair| 'break y))
      (or (eq y '|all|)
          (and
            (member x y)
            (null (member (elt (pname x) 0) '($ |#|)))
            (null (gensymp x))))))
      (|break|
        (append
          (|bright| |currentFunction|)
          (cons "breaks after"
              (append (|bright| x) (cons '|:=| (cons |printform| nil)))))))
      (t nil))))
  x)))

```

defun letPrint3

This is the version for use when we have our hands on a function to convert the data into type "Expression" [letPrint2 p110]

```

[lassoc p??]
[isgenvar p??]
[isSharpVarWithNum p109]
[gensymp p??]
[mathprint p??]
[spadcall p??]
[print p??]
[hasPair p113]
[pname p1235]
[break p156]
[bright p??]

```

[\[\\$BreakMode p969\]](#)

[\[\\$letAssoc p70\]](#)

— defun letPrint3 —

```
(defun |letPrint3| (x |xval| |printfn| |currentFunction|)
  (prog (|$BreakMode| |flag| y)
    (declare (special |$BreakMode| |$letAssoc|))
    (return
      (progn
        (setq |$BreakMode| nil)
        (cond
          ((and |$letAssoc|
            (or (setq y (lassoc |currentFunction| |$letAssoc|))
                (setq y (lassoc '|all| |$letAssoc|))))
            (cond
              ((and
                (or (eq y '|all|) (member x y))
                (null (or (isgenvar x) (|isSharpVarWithNum| x) (gensymp x))))
                (setq |$BreakMode| '|letPrint2|)
                (setq |flag| nil)
                (catch '|letPrint2|
                  (|mathprint|
                    (cons '= (cons x (cons (spadcall |xval| |printfn|) nil))))
                  |flag|)
                (cond
                  ((eq |flag| '|letPrint2|) (|print| |xval|))
                  (t nil))))
              (cond
                ((and
                  (setq y (|hasPair| 'break y))
                  (or
                    (eq y '|all|)
                    (and
                     (member x y)
                     (null (member (elt (pname x) 0) '($ |#|)))
                     (null (gensymp x))))
                  (|break|
                    (append
                     (|bright| |currentFunction|)
                     (cons "breaks after"
                       (append (|bright| x) (cons " := " (cons |xval| nil)))))))
                  (t nil))))
                x))))))
```

defun hasPair

[qcar p??]
 [qcdr p??]
 [hasPair p113]

— defun hasPair —

```
(defun |hasPair| (key arg)
  (prog (tmp1 a)
    (return
      (cond
        ((atom arg) nil)
        ((and (consp arg)
              (progn
                (setq tmp1 (qcar arg))
                (and (consp tmp1)
                     (equal (qcar tmp1) key)
                     (progn (setq a (qcdr tmp1)) t))))
          a)
        (t (|hasPair| key (cdr arg)))))))
```

—————

defun shortenForPrinting

[isDomainOrPackage p107]
 [devaluate p??]

— defun shortenForPrinting —

```
(defun |shortenForPrinting| (|val|)
  (if (|isDomainOrPackage| |val|)
      (|devaluate| |val|)
      |val|))
```

—————

defun getOption

[assoc p??]

— defun getOption —

```
(defun getOption (opt l)
```



```
(let (y)
  (when (setq y (|assoc| opt 1)) (cdr y))))
```

defun orderBySlotNumber

```
[seq p??]
[assocright p??]
[orderList p??]
[exit p??]
```

— defun orderBySlotNumber —

```
(defun |orderBySlotNumber| (arg)
  (prog (n)
    (return
      (seq
        (assocright
          (|orderList|
            (prog (t0)
              (setq t0 nil)
              (return
                (do ((t1 arg (cdr t1)) (x nil))
                  ((or (atom t1)
                      (progn (setq x (car t1)) nil)
                      (progn (progn (setq n (caddr x)) x) nil))
                  (nreverse0 t0)))
            (seq
              (exit
                (setq t0 (cons (cons n x) t0))))))))))))))
```

defun spadReply

Collect all of the print names of the `$traceNames` [isDomainOrPackage p107]
`$traceNames` p76]

— defun spadReply —

```
(defun |spadReply| ()
  (declare (special |$traceNames|))
  (loop for x in |$traceNames| collect
    (if (and (consp x) (|isDomainOrPackage| (car x)))
```

```
(|devaluate| (car x))
x)))
```

defun remover

[remover p115]

— defun remover —

```
(defun remover (lst item)
  (cond
    ((null (conspl lst)) (cond ((equal lst item) nil) (t lst)))
    ((equal (car lst) item) (cdr lst))
    (t
     (rplnode lst (remover (car lst) item) (remover (cdr lst) item))
     (rplaca lst (remover (car lst) item))
     (rplacd lst (remover (cdr lst) item))
     lst)))
```

defun stupidIsSpadFunction

[strpos p1236]
[pname p1235]

— defun stupidIsSpadFunction —

```
(defun |stupidIsSpadFunction| (fn)
  (strpos ";" (pname fn) 0 nil))
```

defun compileBoot

[/D,1 p??]

— defun compileBoot —

```
(defun |compileBoot| (fn)
  (/D,1| (list fn) '(/comp) nil nil))
```

defun getTraceOptions

```
[throwKeyedMsg p??]
[throwListOfKeyedMsgs p??]
[poundsign p??]
[seq p??]
[exit p??]
[getTraceOption p118]
[$traceErrorStack p75]
```

— defun getTraceOptions —

```
(defun getTraceOptions (options)
  (prog (|$traceErrorStack| optionlist temp1 key |parms|)
    (declare (special |$traceErrorStack|))
    (return
      (seq
        (progn
          (setq |$traceErrorStack| nil)
          (setq optionlist
            (prog (t0)
              (setq t0 nil)
              (return
                (do ((t1 options (cdr t1)) (x nil))
                  ((or (atom t1) (progn (setq x (car t1)) nil)) (nreverse0 t0))
                (seq
                  (exit
                    (setq t0 (cons (|getTraceOption| x) t0))))))))))
          (cond
            (|$traceErrorStack|
              (cond
                ((null (cdr |$traceErrorStack|))
                 (setq temp1 (car |$traceErrorStack|))
                 (setq key (car temp1))
                 (setq |parms| (cadr temp1))
                 (|throwKeyedMsg| key (cons "" |parms|)))
                (t
                 (|throwListOfKeyedMsgs|
                  "There are %1 problems with your )trace system command:"
                  (cons (|#| |$traceErrorStack|) nil)
                  (nreverse |$traceErrorStack|))))
              (t optionlist))))))
```

defun saveMapSig

```
[rassoc p??]
[addassoc p??]
[getMapSig p117]
[$tracedMapSignatures p75]
[$mapSubNameAlist p70]
```

— defun saveMapSig —

```
(defun |saveMapSig| (funnames)
  (let (map)
    (declare (special |$tracedMapSignatures| |$mapSubNameAlist|))
    (dolist (name funnames)
      (when (setq map (|rassoc| name |$mapSubNameAlist|))
        (setq |$tracedMapSignatures|
              (addassoc name (|getMapSig| map name) |$tracedMapSignatures|))))))
```

—————

defun getMapSig

```
[get p??]
[boot-equal p??]
[$InteractiveFrame p39]
```

— defun getMapSig —

```
(defun |getMapSig| (mapname subname)
  (let (lmms sig)
    (declare (special |$InteractiveFrame|))
    (when (setq lmms (|get| mapname '|localModemap| |$InteractiveFrame|))
      (do ((t0 lmms (cdr t0)) (mm nil) (t1 nil sig))
          ((or (atom t0) (progn (setq mm (car t0)) nil) t1) nil)
        (when (equal (cadr mm) subname) (setq sig (cdar mm))))
      sig)))
```

—————

defun getTraceOption,hn

```
[seq p??]
[exit p??]
[isDomainOrPackage p107]
```

```
[stackTraceOptionError p100]
[domainToGenvar p100]
```

— defun getTraceOption,hn —

```
(defun |getTraceOption,hn| (x)
  (let (g)
    (cond
      ((and (atom x) (null (upper-case-p (elt (stringimage x) 0))))
        (cond
          ((|isDomainOrPackage| (eval x)) x)
          (t
            (|stackTraceOptionError|
              (cons
                (format nil
                  "%1 The )trace option )of should be followed by the name of a ~
                  domain and %2 is not one.")
                (list (list x)))))))
      ((setq g (|domainToGenvar| x)) g)
      (t (|stackTraceOptionError|
          (cons
            (format nil
              "%1 The )trace option )of should be followed by the name of a ~
              domain and %2 is not one.")
            (list (list x)))))))
```

—————

defun getTraceOption

```
[seq p??]
[exit p??]
[selectOptionLC p819]
[identp p1237]
[stackTraceOptionError p100]
[concat p1237]
[object2String p??]
[transOnlyOption p99]
[qcdr p??]
[qcar p??]
[getTraceOption,hn p117]
[isListOfIdentifiersOrStrings p102]
[isListOfIdentifiers p102]
[throwKeyedMsg p??]
[$traceOptionList p76]
```

— defun getTraceOption —

```
(defun |getTraceOption| (arg)
  (prog (l opts key a n)
    (declare (special |$traceOptionList|))
    (return
      (seq
        (progn
          (setq key (car arg))
          (setq l (cdr arg))
          (setq key (|selectOptionLC| key |$traceOptionList| '|traceOptionError|))
          (setq arg (cons key l))
          (cond
            ((member key '(|nonquietly| |timer| |nt|)) arg)
            ((eq key '|break|)
              (cond
                ((null l) (cons '|break| (cons '|before| nil)))
                (t
                 (setq opts
                   (loop for y in l collect
                     (|selectOptionLC| y '(|before| |after|) nil)))
                 (cond
                  ((prog (t2)
                     (setq t2 t)
                     (return
                      (do ((t3 nil (null t2)) (t4 opts (cdr t4)) (y nil))
                        ((or t3 (atom t4) (progn (setq y (car t4)) nil)) t2)
                      (seq
                       (exit
                        (setq t2 (and t2 (identp y)))))))
                     (cons '|break| opts)))
                  (t
                   (|stackTraceOptionError|
                    (cons
                     (format nil
                      "%1 The )trace option )break can only have one or both of ~
before and after as arguments.")
                     (cons nil nil)))))))
            ((eq key '|restore|)
              (cond
                ((null l) arg)
                (t
                 (|stackTraceOptionError|
                  (cons "%1 The )trace option %2 can have no arguments."
                    (cons (cons (concat ") " (|object2String| key)) nil) nil))))))
            ((eq key '|only|) (cons '|only| (|transOnlyOption| l)))
            ((eq key '|within|)
              (cond
                ((and (consp l)
                  (eq (qcdr l) nil)
```

```

        (progn (setq a (qcar 1)) t)
        (identp a))
    arg)
  (t
   (|stackTraceOptionError|
    (cons
     "%1 The )trace option %2 takes exactly one name as an argument."
     (cons (cons ")within" nil) nil))))))
  ((member key '(|cond| |before| |after|))
   (setq key
    (cond
     ((eq key '|cond|) '|when|)
     (t key)))
   (cond
    ((and (consp 1)
         (eq (qcdr 1) nil)
         (progn (setq a (qcar 1)) t))
     (cons key 1))
    (t
     (|stackTraceOptionError|
      (cons
       "%1 The )trace option %2 takes exactly one expression as an argument."
       (cons
        (concat ")")
        (|object2String| key)) nil) nil))))))
  ((eq key '|depth|)
   (cond
    ((and (consp 1)
         (eq (qcdr 1) nil)
         (progn (setq n (qcar 1)) t)
         (integerp n))
     arg)
    (t
     (|stackTraceOptionError|
      (cons
       "%1 The )trace option %2 takes exactly one integer argument."
       (cons (cons ")depth" nil) nil))))))
  ((eq key '|count|)
   (cond
    ((or (null 1)
         (and (consp 1)
              (eq (qcdr 1) nil)
              (progn (setq n (qcar 1)) t)
              (integerp n)))
     arg)
    (t
     (|stackTraceOptionError|
      (cons
       "%1 The )trace option %2 takes exactly one integer argument."
       (cons (cons ")count" nil) nil))))))

```

```

((eq key '|of|)
 (cons '|of| (loop for y in l collect (|getTraceOption,hn| y))))
((member key '|local| ops vars))
(cond
 ((or (null l)
      (and (consp l) (eq (qcdr l) nil) (eq (qcar l) '|all|))))
  (cons key '|all|))
 ((|isListOfIdentifiersOrStrings| l) arg)
 (t
  (|stackTraceOptionError|
   (cons
    "%1 The )trace option %2 should be followed by a list of names."
    (cons
     (cons (concat ")") (|object2String| key)) nil) nil))))))
((eq key '|varbreak|)
 (cond
 ((or (null l)
      (and (consp l) (eq (qcdr l) nil) (eq (qcar l) '|all|))))
  (cons '|varbreak| '|all|))
 ((|isListOfIdentifiers| l) arg)
 (t
  (|stackTraceOptionError|
   (cons
    "%1 The )trace option %2 should be followed by a list of variable names."
    (cons
     (cons (concat ")") (|object2String| key)) nil) nil))))))
((eq key '|mathprint|)
 (cond
 ((null l) arg)
 (t
  (|stackTraceOptionError|
   (cons "%1 The )trace option %2 can have no arguments."
   (cons
    (cons (concat ")") (|object2String| key)) nil) nil))))))
(key (|throwKeyedMsg| "The %1 option is not implemented yet."
  (cons key nil))))))

```

defun traceOptionError

[stackTraceOptionError p100]
[commandAmbiguityError p792]

— defun traceOptionError —

```
(defun |traceOptionError| (opt keys)
```



```
(if (null keys)
  (|stackTraceOptionError|
   (cons
    "%1 Axiom does not understand the )trace option %2 which you used."
    (cons (cons opt nil) nil)))
  (|commandAmbiguityError| '|trace option| opt keys)))
```

defun genDomainTraceName

This function maintains an alist whose car is the domain to trace and the cdr is the genvar. So for POLY(INT) it is called with

```
y == (|Polynomial| (|Integer|))
```

which, if it is already traced would be in the alist as

```
(((|Polynomial| (|Integer|)) . $1))
```

If it is not already traced it would be cons with a new genvar and added to the alist.

```
[lassoc p??]
[genvar p??]
[$domainTraceNameAssoc p69]
```

— defun genDomainTraceName —

```
(defun |genDomainTraceName| (y)
  (let (u g)
    (declare (special |$domainTraceNameAssoc|))
    (if (setq u (lassoc y |$domainTraceNameAssoc|))
      u
      (progn
        (setq g (genvar))
        (setq |$domainTraceNameAssoc| (cons (cons y g) |$domainTraceNameAssoc|))
        g))))
```

defun untrace

```
[copy p??]
[transTraceItem p126]
[/untrace,0 p??]
```

[lassocSub p103]
 [removeTracedMapSigs p127]
 [\$lastUntraced p70]
 [\$mapSubNameAlist p70]
 [\$traceNames p76]

— defun untrace —

```
(defun |untrace| (arg)
  (let (untracelist)
    (declare (special |$lastUntraced| |$traceNames| |$mapSubNameAlist|))
    (if arg
      (setq |$lastUntraced| arg)
      (setq |$lastUntraced| (copy |$traceNames|)))
    (setq untracelist
      (do ((t1 arg (cdr t1)) (x nil) (t0 nil))
          ((or (atom t1) (progn (setq x (car t1)) nil))
           (nreverse0 t0))
          (push (|transTraceItem| x) t0)))
    (/untrace-0
     (do ((t3 untracelist (cdr t3)) (|funName| nil) (t2 nil))
         ((or (atom t3) (progn (setq |funName| (car t3)) nil))
          (nreverse0 t2))
         (push (|lassocSub| |funName| |$mapSubNameAlist|) t2)))
    (|removeTracedMapSigs| untracelist)))
```

—————

defun /untrace-0

[/options p96]
 [trunclist p97]
 [/untrace-1 p124]

— defun /untrace-0 —

```
(defun /untrace-0 (l)
  (let (optionl options fnl)
    (cond
      ((member 'l :test #'eq) (format t "Use )help trace~%"))
    (t
     (setq optionl (/options l))
     (setq fnl (trunclist l optionl))
     (setq options (if optionl (car optionl)))
     (/untrace-1 fnl options)))))
```

—————

defun /untrace-1

```
[/untrace-2 p124]
[/untrace-reduce p124]
[/tracereply p142]
[$traceNames p76]
```

— **defun /untrace-1** —

```
(defun /untrace-1 (l options)
  (declare (special |$traceNames|))
  (cond
    ((not l)
     (unless (atom |$traceNames|)
       (mapcar #'(lambda (u) (/untrace-2 (/untrace-reduce u) options))
         (append |$traceNames| nil))))
    ((mapcar #'(lambda (x) (/untrace-2 x options)) l)))
  (/tracereply))
```

—————

defun /untrace-reduce

(CAR X) is now a domain

/untrace-reduce : Union(Atom,List) → Atom

— **defun /untrace-reduce 0** —

```
(defun /untrace-reduce (x)
  (if (atom x) x (first x)))
```

—————

defun /untrace-2

```
[isFunctor p??]
[untraceDomainConstructor p139]
[isDomainOrPackage p107]
[isDomain p??]
[spadUntrace p143]
[eqcar p??]
[sayBrightly p??]
[unembed p??]
[isSubForRedundantMapName p105]
[rassocSub p104]
```

```

[isGenvar p126]
[devaluate p??]
[concat p1237]
[$traceNoisely p76]
[$mapSubNameAlist p70]
[$mathTraceList p71]
[$timerList p74]
[$traceNames p76]

```

— defun /untrace-2 —

```

(defun /untrace-2 (x options)
  (declare (special |$traceNoisely| |$mapSubNameAlist| |$mathTraceList|
    |$timerList| |$traceNames|))
  (let (u y)
    (cond
      ((and (|isFunction| X) (atom x))
        (|untraceDomainConstructor| X))
      ((or (|isDomainOrPackage| (setq u x))
        (and (symbolp X) (boundp X) (|isDomain| (setq u (eval x))))))
        (|spadUntrace| u options))
      ((eqcar options 'alias)
        (when |$traceNoisely|
          (|sayBrightly| (list (cadr options) '**untraced)))
        (setq |$timerList|
          (remove (princ-to-string (cadr options)) |$timerList| :test 'equal))
        (setq |$countList|
          (remove (princ-to-string (cadr options)) |$countList| :test 'equal))
        (setq |$mathTraceList|
          (remove (cadr options) |$mathTraceList| :test 'equal))
        (unembed x))
      ((and (not (member x |$traceNames|)) (not (|isSubForRedundantMapName| X)))
        (|sayBrightly| (list (|rassocSub| X |$mapSubNameAlist|) "not traced")))
      (t
        (setq |$traceNames| (remove x |$traceNames| :test 'equal))
        (setq |$mathTraceList|
          (remove (if (stringp x) (intern x) x) |$mathTraceList|))
        (setq |$letAssoc| (remove x |$letAssoc| :key #'car))
        (setq y (if (isGenvar x) (|devaluate| (eval x)) x))
        (setq |$timerList| (remove (princ-to-string y) |$timerList| :test 'equal))
        (set (intern (concat y ",TIMER")) 0)
        (setq |$countList| (remove (princ-to-string y) |$countList| :test 'equal))
        (set (intern (concat y ",COUNT")) 0)
        (when (and |$traceNoisely| (not (|isSubForRedundantMapName| y)))
          (|sayBrightly| (list (|rassocSub| Y |$mapSubNameAlist|) "untraced")))
        (unembed x)))))

```

defun isGenvar

```
[identp p1237]
[size p1236]
[digitp p1235]
```

— defun isGenvar —

```
(defun isGenvar (x)
  (and (identp x)
    (let ((y (symbol-name x)))
      (and (char= #\$ (elt y 0)) (> (size y) 1) (digitp (elt y 1))))))
```

defun transTraceItem

For a call to trace INT this return Integer. For a call to trace POLY(INT) this returns a genvar (e.g. \$1) which is set to the domain to be traced, recorded on the alist \$domainTraceNameAssoc. In either case, a symbol is returned. [get p??]

```
[member p1238]
[objMode p516]
[objVal p516]
[domainToGenvar p100]
[unabbrev p??]
[constructor? p??]
[vecp p??]
[transTraceItem p126]
[devaluate p??]
[throwKeyedMsg p??]
[$doNotAddEmptyModeIfTrue p69]
```

transTraceItem : traceArgument → symbol

— defun transTraceItem —

```
(defun |transTraceItem| (x)
  (let (|$doNotAddEmptyModeIfTrue| |value| y)
    (declare (special |$doNotAddEmptyModeIfTrue|))
    (setq |$doNotAddEmptyModeIfTrue| t)
    (cond
      ((atom x)
        (cond
          ((and (setq |value| (|get| x '|value| |$InteractiveFrame|))
            (member (|objMode| |value|)
              '((|Mode|) (|Domain|) (|SubDomain| (|Domain|)))) :test #'equalp))
            (setq x (|objVal| |value|))
```

```

(cond
  ((setq y (|domainToGenvar| x)) y)
  (t x)))
;; case for trace INT
((upper-case-p (elt (princ-to-string x) 0))
 (setq y (|unabbrev| x))
 (cond
  ((|constructor?| y) y)
  ((and (consp y) (|constructor?| (car y))) (car y))
  ((setq y (|domainToGenvar| x)) y)
  (t x)))
(t x)))
((simple-vector-p (car x)) (|transTraceItem| (|devaluate| (car x))))
;; case for trace POLY(INT)
((setq y (|domainToGenvar| x)) y)
(t (|throwKeyedMsg|
    "Axiom does not understand the use of %1 here." (list x)))))

```

defun removeTracedMapSigs

[[\\$tracedMapSignatures](#) p75]

— defun removeTracedMapSigs —

```

(defun |removeTracedMapSigs| (untraceList)
  (declare (special |$tracedMapSignatures|))
  (dolist (name untraceList)
    (remprop name |$tracedMapSignatures|)))

```

defun coerceTraceArgs2E

[[spadsysnamep](#) p??]
 [[pname](#) p1235]
 [[coerceSpadArgs2E](#) p128]
 [[objValUnwrap](#) p516]
 [[coerceInteractive](#) p740]
 [[mkObjWrap](#) p515]
 [[\\$OutputForm](#) p710]
 [[\\$mathTraceList](#) p71]
 [[\\$tracedMapSignatures](#) p75]

— defun coerceTraceArgs2E —

```

(defun |coerceTraceArgs2E| (tracename subname args)
  (declare (ignore tracename))
  (let (name)
    (declare (special |$OutputForm| |$mathTraceList| |$tracedMapSignatures|))
    (cond
      ((member (setq name subname) |$mathTraceList|)
        (if (spadsysnamep (pname name))
          (|coerceSpadArgs2E| (reverse (cdr (reverse args)))))
        (do ((t1 '(|arg1| |arg2| |arg3| |arg4| |arg5| |arg6| |arg7| |arg8|
                    |arg9| |arg10| |arg11| |arg12| |arg13| |arg14| |arg15|
                    |arg16| |arg17| |arg18| |arg19|) (cdr t1))
              (name nil)
              (t2 args (cdr t2))
              (arg nil)
              (t3 (cdr (lassoc subname |$tracedMapSignatures|)) (cdr t3))
              (type nil)
              (t0 nil))
            ((or (atom t1)
                  (progn (setq name (car t1)) nil)
                  (atom t2)
                  (progn (setq arg (car t2)) nil)
                  (atom t3)
                  (progn (setq type (car t3)) nil))
              (nreverse0 t0))
          (setq t0
            (cons
              (list '= name
                (|objValUnwrap|
                  (|coerceInteractive|
                    (mkObjWrap arg type) |$OutputForm|))) t0))))
        ((spadsysnamep (pname name)) (reverse (cdr (reverse args))))
        (t args))))

```

defun coerceSpadArgs2E

```

[seq p??]
[exit p??]
[objValUnwrap p516]
[coerceInteractive p740]
[mkObjWrap p515]
[$streamCount p1074]
[$OutputForm p710]

```

[[\\$tracedSpadModemap](#) p77]

— **defun coerceSpadArgs2E** —

```
(defun |coerceSpadArgs2E| (args)
  (let ((|$streamCount| 0))
    (declare (special |$streamCount| |$OutputForm| |$tracedSpadModemap|))
    (do ((t1 '(|arg1| |arg2| |arg3| |arg4| |arg5| |arg6| |arg7| |arg8|
              |arg9| |arg10| |arg11| |arg12| |arg13| |arg14| |arg15|
              |arg16| |arg17| |arg18| |arg19|) (cdr t1))
        (name nil)
        (t2 args (cdr t2))
        (arg nil)
        (t3 (cdr |$tracedSpadModemap|) (cdr t3))
        (type nil)
        (t0 nil))
      ((or (atom t1)
          (progn (setq name (car t1)) nil)
          (atom t2)
          (progn (setq arg (car t2)) nil)
          (atom t3)
          (progn (setq type (car t3)) nil))
        (nreverse0 t0))
      (seq
        (exit
          (setq t0
            (cons
              (cons '=
                (cons name
                  (cons (|objValUnwrap|
                        (|coerceInteractive|
                          (mkObjWrap arg type)
                          |$OutputForm|)) nil)))
              t0)))))))
```

—————

defun coerceTraceFunValue2E

[[spadsysnamep](#) p??]
 [[pname](#) p1235]
 [[coerceSpadFunValue2E](#) p130]
 [[lassoc](#) p??]
 [[objValUnwrap](#) p516]
 [[coerceInteractive](#) p740]
 [[mkObjWrap](#) p515]
 [[\\$tracedMapSignatures](#) p75]

[[\\$OutputForm](#) p710]
 [[\\$mathTraceList](#) p71]

— **defun coerceTraceFunValue2E** —

```
(defun |coerceTraceFunValue2E| (tracename subname |value|)
  (let (name u)
    (declare (special |$tracedMapSignatures| |$OutputForm| |$mathTraceList|))
    (if (member (setq name subname) |$mathTraceList|)
        (cond
          ((spadsysnamep (pname tracename)) (|coerceSpadFunValue2E| |value|))
          ((setq u (lassoc subname |$tracedMapSignatures|))
           (|objValUnwrap|
            (|coerceInteractive| (mkObjWrap |value| (car u)) |$OutputForm|)))
          (t |value|)))
      |value|)))
```

—————

defun coerceSpadFunValue2E

[[objValUnwrap](#) p516]
 [[coerceInteractive](#) p740]
 [[mkObjWrap](#) p515]
 [[\\$streamCount](#) p1074]
 [[\\$tracedSpadModemap](#) p77]
 [[\\$OutputForm](#) p710]

— **defun coerceSpadFunValue2E** —

```
(defun |coerceSpadFunValue2E| (|value|)
  (let (|$streamCount|)
    (declare (special |$streamCount| |$tracedSpadModemap| |$OutputForm|))
    (setq |$streamCount| 0)
    (|objValUnwrap|
     (|coerceInteractive|
      (mkObjWrap |value| (car |$tracedSpadModemap|))
      |$OutputForm|))))
```

—————

defun getMapSubNames

[[get](#) p??]
 [[union](#) p??]

```
[getPreviousMapSubNames p103]
[unionq p??]
[$lastUntraced p70]
[$InteractiveFrame p39]
[$traceNames p76]
```

— defun getMapSubNames —

```
(defun getMapSubNames (arg)
  (let (lmm subs)
    (declare (special |$traceNames| |$lastUntraced| |$InteractiveFrame|))
    (setq subs nil)
    (dolist (mapname arg)
      (when (setq lmm (|get| mapname '|localModemap| |$InteractiveFrame|))
        (setq subs
          (append
            (do ((t2 lmm (cdr t2)) (t1 nil) (|mm| nil))
              ((or (atom t2)
                   (progn (setq |mm| (CAR t2)) nil) (nreverse0 t1))
              (setq t1 (cons (cons mapname (cadr |mm|)) t1)))
            subs))))
      (|union| subs
        (|getPreviousMapSubNames|
         (union |$traceNames| |$lastUntraced| :test #'eq))))))
```

—————

defun spadTrace,g

— defun spadTrace,g —

```
(defun |spadTrace,g| (x)
  (if (stringp x) (intern x) x))
```

—————

defun spadTrace,isTraceable

```
[seq p??]
[exit p??]
[gensymp p??]
[reportSpadTrace p141]
[bpiname p??]
```

— defun spadTrace,isTraceable —

```
(defun |spadTrace,isTraceable| (x domain)
  (prog (n |functionSlot|)
    (return
      (seq
        (progn
          (setq n (caddr x))
          x
          (seq
            (if (atom (elt domain n)) (exit nil))
            (setq |functionSlot| (car (elt domain n)))
            (if (gensymp |functionSlot|)
              (exit (seq (|reportSpadTrace| '|Already Traced| x) (exit nil))))
            (if (null (bpiname |functionSlot|))
              (exit
                (seq
                  (|reportSpadTrace| '|No function for| x)
                  (exit nil))))
            (exit t)))))))
```

—————→

defun spadTrace

```
[refvecp p??]
[aldorTrace p??]
[isDomainOrPackage p107]
[userError p??]
[seq p??]
[exit p??]
[spadTrace,g p131]
[getOption p113]
[removeOption p100]
[opOf p??]
[assoc p??]
[flattenOperationAlist p107]
[getOperationAlistFromLisplib p134]
[spadTrace,isTraceable p131]
[as-insert p??]
[bpiname p??]
[spadTraceAlias p141]
[subTypes p101]
[constructSubst p732]
[bpitrace p136]
```

```

[rplac p??]
[printDashedLine p??]
[reportSpadTrace p141]
[setletprintflag p??]
[spadReply p114]
[$tracedModemap p??]
[$fromSpadTrace p70]
[$letAssoc p70]
[$reportSpadTrace p141]
[$traceNoisely p76]
[$traceNames p76]

```

— defun `spadTrace` —

```

(defun |spadTrace| (domain options)
  (let (listOfOperations listOfVariables listOfBreakVars anyifTrue
        domainId currentEntry currentAlist opStructureList triple
        sigSlotNumberAlist fn alias tracedModemap dn1 fgg tf)
    (declare (special |$fromSpadTrace|))
    (setq |$fromSpadTrace| t)
    (cond
      ((null (|isDomainOrPackage| domain))
       (|userError| "bad argument to trace"))
      (t
       (setq listOfOperations
              (loop for x in (getOption 'ops options)
                    collect (if (stringp x) (intern x) x)))
       (when (setq listOfVariables (getOption 'vars options))
         (setq options (removeOption 'vars options)))
       (when (setq listOfBreakVars (getOption 'varbreak options))
         (setq options (removeOption 'varbreak options)))
       (setq anyifTrue (null listOfOperations))
       (setq domainId (|opOf| (elt domain 0)))
       (setq currentEntry (|assoc| domain |$traceNames|))
       (setq currentAlist (cdr currentEntry))
       (setq opStructureList
              (flattenOperationAlist (|getOperationAlistFromLisplib| domainId)))
       ; new form is (<op> <signature> <slotNumber> <condition> <kind>)
       (setq sigSlotNumberAlist
              (loop for arg in opStructureList
                    when
                      (and (eq (fifth arg) 'elt)
                           (or anyifTrue (member (car arg) listOfOperations))
                           (integerp (third arg))
                           (|spadTrace, isTraceable|
                            (setq triple (list (first arg) (second arg) (third arg)))
                            domain)))
                    collect triple))
       (when listOfVariables

```

```

(loop for arg in sigSlotNumberAlist do
  (setq fn (car (elt domain (third arg))))
  (setq |$letAssoc|
    (as-insert (bpiname fn) listOfVariables |$letAssoc|))))
(when listOfBreakVars
  (loop for arg in sigSlotNumberAlist do
    (setq fn (car (elt domain (third arg))))
    (setq |$letAssoc|
      (as-insert (bpiname fn) (list (cons 'break listOfBreakVars)
        |$letAssoc|))))
(loop for pair in sigSlotNumberAlist do
  (setq alias (|spadTraceAlias| domainId (first pair) (third pair)))
  (setq tracedModemap
    (|subTypes| (second pair) (|constructSubst| (elt domain 0))))
  (setq dn1 (car (elt domain (third pair))))
  (setq fgg #'|newGoGet|)
  (setq tf
    (if (equal dn1 fgg)
      (|goGetTracerHelper| (elt domain (third pair)) fgg
        pair alias options tracedModemap)
      (bpitrace dn1 alias tracedModemap options)))
  (nconc pair (list listOfVariables (car (elt domain (third pair)))))
  (|rplac| (car (elt domain (third pair))) tf))
(setq sigSlotNumberAlist
  (loop for x in sigSlotNumberAlist
    when (cdddr x)
    collect x))
(when |$reportSpadTrace|
  (when |$traceNoisely| (|printDashedLine|))
  (loop for x in (|orderBySlotNumber| sigSlotNumberAlist) do
    (|reportSpadTrace| 'tracing x)))
(cond
  (currentEntry
    (|rplac| (cdr currentEntry) (append sigSlotNumberAlist currentAlist)))
  (t
    (setq |$traceNames|
      (cons (cons domain sigSlotNumberAlist) |$traceNames|)
      (|spadReply|))))))

```

defun getOperationAlistFromLisplib

[getdatabase p1196]

[addConsDB p??]

[markUnique p135]

— defun getOperationAlistFromLisplib —

```
(defun |getOperationAlistFromLisplib| (x)
  (let (opalist consdb)
    (setq opalist (getdatabase x 'operationalist))
    (cond
      ((null opalist) opalist)
      ((eq (caar opalist) '|$unique|) (cdr opalist))
      (t
       (setq consdb '(nil t elt))
       ((lambda (arg1 arg2)
          (loop
            (cond
              ((or (atom arg1) (progn (setq arg2 (car arg1)) nil))
               (return nil))
              (t
               (and (consp arg2)
                    ((lambda (items)
                       (loop
                        (cond
                          ((atom items) (return nil))
                          (t
                           (cond
                              ((consp (cdar items))
                               (cond
                                ((consp (cddar items))
                                 (cond
                                  ((and (consp (cddddar items))
                                       (eq (cdr (cddddar items)) nil)) nil)
                                  (t (rplacd (cddar items) (cddr consdb))))))
                              (t (rplacd (cdar items) (cdr consdb))))))
                           (t (rplacd (car items) consdb)))
                               (rplaca items (car items))))
                               (setq items (cdr items))))
                               (cdr arg2))))))
          (setq arg1 (cdr arg1))))
       opalist nil)
      (and opalist (markUnique opalist))))))
```

—————

defun markUnique

— defun markUnique 0 —

```
(defun markUnique (x)
  (let (u)
```

```
(setq u (car x))
(rplaca x '(|$unique|))
(rplacd x (cons u (cdr x)))
(cdr x)))
```

defun bptrace

[trace3 p83]

— defun bptrace —

```
(defun bptrace (binDef alias modemap options)
  (trace3 (gensym) modemap (cons (list 'alias alias) options) binDef))
```

defun traceDomainLocalOps

[sayMSG p45]

— defun traceDomainLocalOps —

```
(defun |traceDomainLocalOps| ()
  (|sayMSG| '(" The )local option has been withdrawn"))
  (|sayMSG| '(" Use )ltr to trace local functions.")))
```

defun untraceDomainLocalOps

[sayMSG p45]

— defun untraceDomainLocalOps —

```
(defun |untraceDomainLocalOps| ()
  (|sayMSG| '(" The )local option has been withdrawn"))
  (|sayMSG| '(" Use )ltr to trace local functions.")))
```

defun traceDomainConstructor

```

[getOption p113]
[seq p??]
[exit p??]
[spadTrace p132]
[concat p1237]
[embed p??]
[mkq p??]
[loadFunctor p1224]
[traceDomainLocalOps p136]
[$ConstructorCache p??]

```

— **defun traceDomainConstructor** —

```

(defun |traceDomainConstructor| (domainConstructor options)
  (prog (listOfLocalOps argl domain innerDomainConstructor)
    (declare (special |$ConstructorCache|))
    (return
      (seq
        (progn
          (|loadFunctor| domainConstructor)
          (setq listOfLocalOps (getOption 'local options))
          (when listOfLocalOps (|traceDomainLocalOps|))
          (cond
            ((and listOfLocalOps (null (getOption 'ops options))) nil)
            (t
              (do ((t2 (hget |$ConstructorCache| domainConstructor) (cdr t2))
                  (t3 nil))
                ((or (atom t2)
                     (progn (setq t3 (car t2)) nil)
                     (progn
                       (progn
                         (setq argl (car t3))
                         (setq domain (caddr t3)) t3)
                       nil))
                  nil)
              (seq
                (exit
                  (|spadTrace| domain options))))
          (setq |$traceNames| (cons domainConstructor |$traceNames|))
          (setq innerDomainConstructor
            (intern (concat domainConstructor ";")))
          (cond
            ((fboundp innerDomainConstructor)
              (setq domainConstructor innerDomainConstructor)))
          (embed domainConstructor
            (cons 'lambda
              (cons

```



```

(cons '&rest
  (cons 'args nil))
(cons
  (cons 'prog
    (cons
      (cons 'domain nil)
      (cons
        (cons 'setq
          (cons 'domain
            (cons
              (cons 'apply (cons domainConstructor
                (cons 'args nil))) nil)))
        (cons
          (cons '|spadTrace|
            (cons 'domain
              (cons (mkq options) nil)))
          (cons (cons 'return (cons 'domain nil)) nil))))
        nil)))))))))

```

defun untraceDomainConstructor,keepTraced?

```

[seq p??]
[qcar p??]
[isDomainOrPackage p107]
[boot-equal p??]
[devaluate p??]
[exit p??]
[/untrace,0 p??]

```

— defun untraceDomainConstructor,keepTraced? —

```

(defun |untraceDomainConstructor,keepTraced?| (df domainConstructor)
  (prog (dc)
    (return
      (seq
        (if (and
          (and
            (and (consp df) (progn (setq dc (qcar df)) t))
            (|isDomainOrPackage| dc))
            (boot-equal (ifcar (|devaluate| dc)) domainConstructor))
          (exit (seq (|/UNTRACE,0| (cons dc nil)) (exit nil))))
          (exit t))))))

```

defun untraceDomainConstructor

```
[untraceDomainConstructor,keepTraced? p138]
[unembed p??]
[seq p??]
[exit p??]
[concat p1237]
[delete p??]
[$traceNames p76]
```

— **defun untraceDomainConstructor** —

```
(defun |untraceDomainConstructor| (domainConstructor)
  (prog (innerDomainConstructor)
    (declare (special |$traceNames|))
    (return
      (seq
        (progn
          (setq |$traceNames|
            (prog (t0)
              (setq t0 nil)
              (return
                (do ((t1 |$traceNames| (cdr t1)) (df nil))
                  ((or (atom t1) (progn (setq df (car t1)) nil)) (nreverse0 t0))
                (seq
                  (exit
                    (cond ((|untraceDomainConstructor,keepTraced?|
                          df domainConstructor)
                      (setq t0 (cons df t0))))))))))
          (setq innerDomainConstructor
            (intern (concat domainConstructor ";")))
          (cond
            ((fboundp innerDomainConstructor) (unembed innerDomainConstructor))
            (t (unembed domainConstructor)))
          (setq |$traceNames| (|delete| domainConstructor |$traceNames|))))))
```

—————

defun mapLetPrint

```
[getAliasIfTracedMapParameter p140]
[getBpiNameIfTracedMap p140]
[letPrint p108]
```

— **defun mapLetPrint** —

```
(defun |mapLetPrint| (x val currentFunction)
```

```
(setq x (|getAliasIfTracedMapParameter| x currentFunction))
(setq currentFunction (|getBpiNameIfTracedMap| currentFunction))
(|letPrint| x val currentFunction))
```

defun getAliasIfTracedMapParameter

```
[isSharpVarWithNum p109]
[get p??]
[exit p??]
[string2pint-n p??]
[substring p324]
[pname p1235]
[seq p??]
[$InteractiveFrame p39]
```

— defun getAliasIfTracedMapParameter —

```
(defun |getAliasIfTracedMapParameter| (x |currentFunction|)
  (prog (|aliasList|)
    (declare (special |$InteractiveFrame|))
    (return
      (seq
        (cond
          ((|isSharpVarWithNum| x)
            (cond
              ((setq |aliasList|
                (|get| |currentFunction| 'alias |$InteractiveFrame|))
                (exit
                  (elt |aliasList|
                    (-
                      (string2pint-n (substring (pname x) 1 nil) 1) 1))))))
            (t x))))))
```

defun getBpiNameIfTracedMap

```
[get p??]
[exit p??]
[seq p??]
[$InteractiveFrame p39]
[$traceNames p76]
```

— defun getBpiNameIfTracedMap —

```
(defun |getBpiNameIfTracedMap| (name)
  (prog (lmm bpiName)
    (declare (special |$InteractiveFrame| |$traceNames|))
    (return
      (seq
        (cond
          ((setq lmm (|get| name '|localModemap| |$InteractiveFrame|))
            (cond
              ((member (setq bpiName (cadar lmm)) |$traceNames|)
                (exit bpiName)))
            (t name))))))
```

—————

defun spadTraceAlias

[internl p??]

— defun spadTraceAlias —

```
(defun |spadTraceAlias| (domainid op n)
  (internl domainid (intern "." "boot") op '|,| (princ-to-string n)))
```

—————

defun reportSpadTrace

[qcar p??]

[sayBrightly p??]

[\$traceNoisely p76]

— defun reportSpadTrace —

```
(defun |reportSpadTrace| (|header| t0)
  (prog (op sig n |t| |msg| |namePart| y |tracePart|)
    (declare (special |$traceNoisely|))
    (return
      (progn
        (setq op (car t0))
        (setq sig (cadr t0))
        (setq n (caddr t0))
        (setq |t| (cddddr t0))
```

```

(cond
  ((null |$traceNoisely|) nil)
  (t
   (setq |msg|
    (cons |header|
      (cons op
        (cons '|:|
          (cons (CDR sig)
            (cons '| -> |
              (cons (car sig)
                (cons '| in slot |
                  (cons n nil))))))))))
    (setq |namePart| nil)
    (setq |tracePart|
      (cond
        ((and (consp |t|) (progn (setq y (qcar |t|)) t) (null (null y))))
        (cond
          ((eq y '|all|)
           (cons '|all| (cons '|vars| nil)))
          (t (cons '| vars: | (cons y nil))))
        (t nil)))
    (|sayBrightly| (append |msg| (append |namePart| |tracePart|))))))

```

defun /tracereply

```

[qcar p??]
[isDomainOrPackage p107]
[devaluate p??]
[seq p??]
[exit p??]
[$traceNames p76]

```

— defun /tracereply —

```

(defun /tracereply ()
  (prog (d domainlist |functionList|)
    (declare (special |$traceNames|))
    (return
     (seq
      (cond
        ((null |$traceNames|) " Nothing is traced.")
        (t
         (do ((t0 |$traceNames| (cdr t0)) (x nil))
              ((or (atom t0) (progn (setq x (car t0)) nil)) nil)
          (seq

```

```

(exit
 (cond
  ((and (consp x)
        (progn (setq d (qcar x)) t)
        (|isDomainOrPackage| d))
   (setq domainlist (cons (|devaluate| d) domainlist)))
  (t
   (setq |functionList| (cons x |functionList|))))))
(append |functionList|
 (append domainlist (cons '|traced| nil))))))

```

defun spadUntrace

```

[isDomainOrPackage p107]
[userError p??]
[getOption p113]
[devaluate p??]
[assoc p??]
[sayMSG p45]
[bright p??]
[prefix2String p??]
[bpiname p??]
[remover p115]
[setletprintflag p??]
[bpiuntrace p??]
[rplac p??]
[seq p??]
[exit p??]
[spadReply p114]
[$letAssoc p70]
[$traceNames p76]

```

— defun spadUntrace —

```

(defun |spadUntrace| (domain options)
  (prog (anyiftrue listofoperations domainid |pair| sigslotnumberalist
             op sig n |lv| |bpiPointer| tracename alias |assocPair|
             |newSigSlotNumberAlist|)
    (declare (special |$letAssoc| |$traceNames|))
    (return
     (seq
      (cond
       ((null (|isDomainOrPackage| domain))
        (|userError| "bad argument to untrace"))

```

```

(t
  (setq anyiftrue (null options))
  (setq listofoperations (getOption 'ops:| options))
  (setq domainid (|devaluate| domain))
  (cond
    ((null (setq |pair| (|assoc| domain |$traceNames|))))
    (|sayMSG|
      (cons "    No functions in"
        (append
          (|bright| (|prefix2String| domainid))
          (cons "are now traced." nil))))))
  (t
    (setq sigslotnumberalist (cdr |pair|))
    (do ((t0 sigslotnumberalist (cdr t0)) (|pair| nil))
      ((or (atom t0)
        (progn (setq |pair| (car t0)) nil)
        (progn
          (progn
            (setq op (car |pair|))
            (setq sig (cadr |pair|))
            (setq n (caddr |pair|))
            (setq |lv| (caddr |pair|))
            (setq |bpiPointer| (car (caddr |pair|)))
            (setq tracename (cadr (caddr |pair|)))
            (setq alias (caddr (caddr |pair|)))
            |pair|)
          nil))
        nil))
      (seq
        (exit
          (cond
            ((or anyiftrue (member op listofoperations))
              (progn
                (bpiuntrace tracename alias)
                (rplac (car (elt domain n)) |bpiPointer|)
                (rplac (caddr |pair|) nil)
                (cond
                  ((setq |assocPair|
                    (|assoc| (bpiname |bpiPointer|) |$letAssoc|))
                    (setq |$letAssoc| (remover |$letAssoc| |assocPair|))
                    (cond
                      ((null |$letAssoc|) (setletprintflag nil))
                      (t nil)))
                  (t nil))))))
              (t nil))))))
    (setq |newSigSlotNumberAlist|
      (prog (t1)
        (setq t1 nil)
        (return
          (do ((t2 sigslotnumberalist (cdr t2)) (x nil))
            ((or (atom t2) (progn (setq x (car t2)) nil)) (nreverse0 t1))

```

```

      (seq
        (exit
          (cond ((caddr x) (setq t1 (cons x t1)))))))))
    (cond
      (|newSigSlotNumberAlist|
        (rplac (cdr |pair|) |newSigSlotNumberAlist|))
      (t
        (setq |$traceNames| (remove domain |$traceNames| :key #'car))
        (|spadReply|)))))))))

```

defun prTraceNames,fn

```

[seq p??]
[qcar p??]
[qcdr p??]
[isDomainOrPackage p107]
[exit p??]
[devaluate p??]

```

— defun prTraceNames,fn —

```

(defun |prTraceNames,fn| (x)
  (prog (|d| |t|)
    (return
      (seq
        (if (and (and (consp x)
                     (progn (setq |d| (qcar x)) (setq |t| (qcdr x)) t))
              (|isDomainOrPackage| |d|))
            (exit (cons (|devaluate| |d|) |t|)))
          (exit x))))))

```

defun prTraceNames

```

[seq p??]
[exit p??]
[prTraceNames,fn p145]
[$traceNames p76]

```

— defun prTraceNames —

```

(defun |prTraceNames| ()

```



```

(declare (special |$traceNames|))
(seq
  (progn
    (do ((t0 |$traceNames| (cdr t0)) (x nil))
      ((or (atom t0) (progn (setq x (car t0)) nil)) nil)
    (seq
      (exit
        (print (|prTraceNames,fn| x)))))) nil)))

```

—————→

defun addTraceItem

```

[constructor? p??]
[isDomain p??]
[devaluate p??]
[isDomainOrPackage p107]
[$constructors p68]
[$domains p69]
[$packages p73]

```

— defun addTraceItem —

```

(defun |addTraceItem| (|d|)
  (declare (special |$constructors| |$domains| |$packages|))
  (cond
    ((|constructor?| |d|)
     (setq |$constructors| (cons |d| |$constructors|)))
    ((|isDomainOrPackage| |d|)
     (setq |$packages| (cons (|devaluate| |d|) |$packages|))))

```

—————→

defun ?t

```

[isgenvar p??]
[get p??]
[sayMSG p45]
[bright p??]
[rassocSub p104]
[qcar p??]
[qcdr p??]
[isDomainOrPackage p107]
[isDomain p??]

```

```
[reportSpadTrace p141]
[take p??]
[sayBrightly p??]
[devaluate p??]
[$mapSubNameAlist p70]
[$InteractiveFrame p39]
[$traceNames p76]
```

— defun ?t —

```
(defun |?t| ()
  (let (llm d suffix l)
    (declare (special |$traceNames| |$InteractiveFrame| |$mapSubNameAlist|))
    (if (null |$traceNames|)
      (|sayMSG| (|bright| "nothing is traced"))
      (progn
        (dolist (x |$traceNames|)
          (cond
            ((and (atom x) (null (isgenvar x)))
              (progn
                (cond
                  ((setq llm (|get| x '|localModemap| |$InteractiveFrame|))
                    (setq x (list (cadar llm)))))
                (|sayMSG|
                  '("Function" ,@( |bright| (|rassocSub| x |$mapSubNameAlist|))
                    "traced"))))))
            (dolist (x |$traceNames|)
              (cond
                ((and (consp x)
                      (progn (setq d (qcar x)) (setq l (qcdr x)) t)
                      (|isDomainOrPackage| d))
                  (progn
                    (setq suffix "package")
                    (|sayBrightly|
                     '(" Functions traced in " ,suffix ,(|devaluate| d) ":"))
                    (dolist (x (|orderBySlotNumber| l))
                      (|reportSpadTrace| '| | (TAKE 4 x)))
                    (terpri))))))))))
```

—————

defun Handle traced function entry

```
[sayBrightlyNT2 p??]
[coerceTraceArgs2E p127]
[spadsysnamep p??]
[monitorPrintArgs p148]
```

[\[TraceFlag p75\]](#)
[\[traceStream p77\]](#)
[\[monitorDepth p71\]](#)
[\[monitorFunDepth p72\]](#)
[\[monitorCaller p71\]](#)
[\[mathTrace p71\]](#)

— defun monitorEnter —

```

(defun monitorEnter (tracecode c type name name1)
  (let (|TraceFlag| cArgs)
    (declare (special |TraceFlag| |traceStream| |monitorDepth|
                      |monitorFunDepth| |monitorCaller| |mathTrace|))
    (setq |TraceFlag| nil)
    (cond
      ((equal tracecode "000") nil)
      (t
       (tab 0 |traceStream|)
       (princ (make-string (- |monitorDepth| 1) :initial-element #\space)
              |traceStream|)
       (prin1 |monitorFunDepth| |traceStream|)
       (format |traceStream| "<enter ~a " (symbol-name name1))
       (unless (eql c 0)
         (if (eq type 'macro)
             (print " expanded" |traceStream|)
             (progn
              (print " from " |traceStream|)
              (prin1 |monitorCaller| |traceStream|))))
       (setq cArgs (|coerceTraceArgs2E| name1 name |monitorArgs|))
       (when (spadsysnamep (symbol-name name))
         (setq cArgs (nreverse (reverse cArgs))))
       (monitorPrintArgs cArgs tracecode (get name '/transform))
       (unless |mathTrace| (terpri |traceStream|))))))

```

—

defun Print the arguments to a traced function

Print the arguments to a traced function. [[monitorPrint p150](#)]

[\[monitorPrintRest p150\]](#)
[\[monitorPrintArg p149\]](#)
[\[mkq p??\]](#)
[\[prinmathor0 p151\]](#)
[\[traceStream p77\]](#)
[\[mathTrace p71\]](#)

— defun monitorPrintArgs —

```

(defun monitorPrintArgs (l code trans)
  (let (n)
    (cond
      ((eql (digit-char-p (elt code 2)) 0) nil)
      ((eql (digit-char-p (elt code 2)) 9)
       (cond
         (trans
          (loop for x in l for y in (cdr trans) do
            (cond
              ((eq y '*)
               (princ "\\ " |$traceStream|)
               (monitorPrint x |$traceStream|))
              ((eq y '&)
               (princ "\\\\" |$traceStream|)
               (terpri |$traceStream|)
               (print x |$traceStream|))
              ((null y)
               (princ "! " |$traceStream|))
              (t
               (princ "! " |$traceStream|)
               (monitorPrint (eval (subst (mkq x) '* y)) |$traceStream|))))))
          (t
           (princ ": " |$traceStream|)
           (unless (atom l)
            (when |$mathTrace| (terpri |$traceStream|)
             (monitorPrint (car l) |$traceStream|)
             (setq l (cdr l)))
            (loop for el in l do (monitorPrintRest el))))))
          (t
           (loop for istep from 2 to (1- (length code)) do
            (setq n (digit-char-p (elt code istep)))
            (unless (eql n 0)
             (princ "\\ " |$traceStream|)
             (prinmathor0 n |$traceStream|)
             (princ ": " |$traceStream|)
             (monitorPrintArg l n)))))))))

```

defun monitorPrintArg

[monitorPrint p150]
 [\$traceStream p77]

— defun monitorPrintArg —

```

(defun monitorPrintArg (l n)

```

```
(loop for el in l for k from 1 to n do
  (when (= k n) (monitorPrint el |$traceStream|))))
```

defun monitorPrint

```
[smallEnough p153]
[limitedPrint1 p153]
[prinmathor0 p151]
[$monitorPretty p72]
```

— defun monitorPrint —

```
(defun monitorPrint (x tracestr)
  (cond
    ((null (smallEnough x)) (limitedPrint1 x tracestr))
    (|$monitorPretty| (prettyprint x tracestr))
    (t (prinmathor0 x tracestr))))
```

defun monitorPrintRest

```
[smallEnough p153]
[monitorBlanks p??]
[prinmathor0 p151]
[$traceStream p77]
[$monitorDepth p71]
[$monitorPretty p72]
[$mathTrace p71]
```

— defun monitorPrintRest —

```
(defun monitorPrintRest (x)
  (cond
    ((null (smallEnough x))
     (terpri |$traceStream|)
     (monitorBlanks (+ |$monitorDepth| 1))
     (princ "\\ " |$traceStream|)
     (print x |$traceStream|))
    (t
     (unless |$mathTrace| (princ "\\ " |$traceStream|))
     (if |$monitorPretty|
```

```
(prettyprint x |$traceStream|)
(prinmathor0 x |$traceStream|))))
```

defun prinmathor0

```
[maprinSpecial p??]
[outputTran2 p??]
[$mathTrace p71]
[$monitorDepth p71]
```

— defun prinmathor0 —

```
(defun prinmathor0 (x tracestr)
  (if |$mathTrace|
    (|maprinSpecial| (|outputTran2| x) |$monitorDepth| 80)
    (prin1 x tracestr)))
```

defun Handle traced function exit

```
[tab p??]
[monitorPrintValue p152]
[coerceTraceFunValue2E p129]
[$TraceFlag p75]
[$traceStream p77]
[$monitorFunDepth p72]
[$mathTrace p71]
[$monitorValue p72]
```

— defun monitorExit —

```
(defun monitorExit (tracecode name name1 v timernam evalTime)
  (let (|$TraceFlag|)
    (declare (special |$TraceFlag|))
    (setq |$TraceFlag| nil)
    (cond
      ((equal tracecode "000") nil)
      (t
       (tab 0 |$traceStream|)
       (princ (make-string (- |$monitorDepth| 1) :initial-element #\space)
              |$traceStream|))
```

```

(prin1 |$monitorFunDepth| |$traceStream|)
(format |$traceStream| ">exit ~a " (symbol-name name1))
(when timernam (format |$traceStream| "(~a sec)" (/ evalTime 60.0)))
(when (eq1 v 1)
  (monitorPrintValue
    (|coerceTraceFunValue2E| name1 name |$monitorValue|)
    name1))
(unless |$mathTrace| (terpri |$traceStream|))))))

```

defun monitorPrintValue

```

[eqcar p??]
[mkq p??]
[smallEnough p153]
[limitedPrint1 p153]
[prinmathor0 p151]
[$traceStream p77]
[$monitorPretty p72]
[$mathTrace p71]

```

— defun monitorPrintValue —

```

(defun monitorPrintValue (val name)
  (let (u)
    (setq u (get name '/transform))
    (cond
      (u
        (cond
          ((eqcar u '&)
            (format |$traceStream| "//~a~%" val))
          (t
            (format |$traceStream| "! ~a~%" (eval (subst (mkq val) '* (car u)))))))
      (t
        (princ ": " |$traceStream|)
        (cond
          ((null (smallEnough val)) (limitedPrint1 val |$traceStream|))
          (|$monitorPretty| (prettyprint val |$traceStream|))
          (t
            (when |$mathTrace| (terpri |$traceStream|))
            (prinmathor0 val |$traceStream|)))))))

```

defun limitedPrint1

```
[*print-level* p??]
[*print-length* p??]
```

— defun limitedPrint1 0 —

```
(defun limitedPrint1 (form stream)
  (let ((*print-level* 4) (*print-length* 4))
    (prin1 form stream)
    (terpri stream)))
```

—————

defun smallEnough

— defun smallEnough —

```
(defun smallEnough (x)
  (if |$traceSize|
    (< (smallEnoughCount x 0 |$traceSize|) |$traceSize|)
    t))
```

—————

defun How big is an object?

We need to know how many elements there are in the object x . The n argument should be 0 for a top-level call and m is the maximum length allowed.

— defun smallEnoughCount 0 —

```
(defun smallEnoughCount (x n m)
  (cond
    ((null (< n m)) n)
    ((simple-vector-p x)
     (loop for i from 0 to (1- (length x)) while (< n m)
           do (setq n (smallEnoughCount (elt x i) (+ n 1) m)))
     n)
    ((atom x) n)
    (t
     (setq n (smallEnoughCount (car x) (+ n 1) m))
     (if (null (< n m)) n (smallEnoughCount (cdr x) n m)))))
```

—————

defun tracelet

```

[stupidIsSpadFunction p115]
[bpname p??]
[lassoc p??]
[union p??]
[isGenvar p126]
[gensymp p??]
[compileBoot p115]
[delete p??]
[$traceletflag p75]
[$QuickLet p73]
[$letAssoc p70]
[$traceletFunctions p76]

```

— defun tracelet —

```

(defun |tracelet| (fn binDef vars)
  (let (|$QuickLet| $traceletflag l)
    (declare (special |$QuickLet| $traceletflag |$letAssoc|
                      |$traceletFunctions|))
    (when (and binDef (|stupidIsSpadFunction| binDef))
      (when (compiled-function-p binDef) (setq fn (bpname binDef))))
    (cond ((eq fn '|Undef|) nil)
          (t
           (setq vars
                 (cond
                  ((eq vars '|all|) '|all|)
                  ((setq l (lassoc fn |$letAssoc|)) (|union| vars l))
                  (t vars)))
           (setq |$letAssoc| (cons (cons fn vars) |$letAssoc|))
           (setq $traceletflag t)
           (setq |$QuickLet| nil)
           (when (and (null (member fn |$traceletFunctions|))
                     (null (isGenvar fn))
                     (compiled-function-p (symbol-function fn))
                     (null (|stupidIsSpadFunction| fn))
                     (null (gensymp fn)))
             (setq |$traceletFunctions| (cons fn |$traceletFunctions|))
             (|compileBoot| fn)
             (setq |$traceletFunctions| (|delete| fn |$traceletFunctions|)))))))

```

defun breaklet

```

[gensymp p??]
[stupidIsSpadFunction p115]
[bpiname p??]
[lassoc p??]
[assoc p??]
[union p??]
[setletprintflag p??]
[compileBoot p115]
[delete p??]
[$QuickLet p73]
[$letAssoc p70]
[$traceletFunctions p76]

```

— defun breaklet —

```

; vars is "all" or a list of variables
; $letAssoc ==> (.. (=fn .. (BREAK . all))) OR (.. (=fn .. (BREAK . vl)))
(defun |breaklet| (fn binDef vars)
  (let (|$QuickLet| pair fnEntry)
    (declare (special |$QuickLet| |$letAssoc| |$traceletFunctions|))
    (when (and binDef (|stupidIsSpadFunction| binDef))
      (when (compiled-function-p binDef) (setq fn (bpiname binDef)))))
    (cond
      ((eq fn '|Undef|) nil)
      (t
       (setq fnEntry (lassoc fn |$letAssoc|))
       (setq vars
        (if (setq pair (|lassoc| 'break fnEntry))
          (|union| vars (cdr pair))
          vars))
       (setq |$letAssoc|
        (cond
          ((null fnEntry)
           (cons (cons fn (list (cons 'break vars))) |$letAssoc|)
           (pair (rplacd pair vars) |$letAssoc|)))
          (t
           (pair (rplacd pair vars) |$letAssoc|))))
       (setq |$QuickLet| nil)
       (when (and (null (member fn |$traceletFunctions|))
                  (null (|stupidIsSpadFunction| fn))
                  (null (gensymp fn)))
        (setq |$traceletFunctions| (cons fn |$traceletFunctions|))
        (|compileBoot| fn)
        (setq |$traceletFunctions| (|delete| fn |$traceletFunctions|)))))))

```

defun break

```
[MONITOR,EVALTRAN p??]  
[sayBrightly p??]  
[/breakcondition p??]
```

— defun break —

```
(defun |break| (msg)  
  (let (condition)  
    (declare (special /breakcondition))  
    (setq condition (|MONITOR,EVALTRAN| /breakcondition nil))  
    (when (eval condition) (|sayBrightly| msg))))
```

Chapter 8

Exposure groups

8.1 Functions to manipulate exposure

Expose a group

Note that `$localExposureData` is a vector of lists. It consists of [exposed groups,exposed constructors,hidden constructors]

```
[object2String p??]  
[qcar p??]  
[displayExposedGroups p165]  
[sayMSG p45]  
[displayExposedConstructors p165]  
[displayHiddenConstructors p166]  
[clearClams p??]  
[getalist p??]  
[sayKeyedMsg p43]  
[member p1238]  
[msort p??]  
[specialChar p1166]  
[namestring p1230]  
[pathname p1232]  
[sayAsManyPerLineAsPossible p??]  
[$globalExposureGroupAlist p167]  
[$localExposureData p166]  
[$interpreterFrameName p38]  
[$linelength p1051]
```

— `defun setExposeAddGroup` —

```
(defun |setExposeAddGroup| (arg)
```

```

"Expose a group"
(declare (special |$globalExposureGroupAlist| |$localExposureData|
               |$interpreterFrameName| $linelength))

(if (null arg)
    (progn
      (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " The group Option ")
      (|displayExposedGroups|)
      (|sayMSG| " ")
      (|sayAsManyPerLineAsPossible|
       (mapcar #'(lambda (x) (|object2String| (first x)))
                |$globalExposureGroupAlist|)))
    (dolist (x arg)
      (when (consp x) (setq x (qcar x)))
      (cond
        ((eq x 'all)
         (setf (elt |$localExposureData| 0)
               (mapcar #'first |$globalExposureGroupAlist|))
         (setf (elt |$localExposureData| 1) nil)
         (setf (elt |$localExposureData| 2) nil)
         (|displayExposedGroups|)
         (|sayMSG| " ")
         (|displayExposedConstructors|)
         (|sayMSG| " ")
         (|displayHiddenConstructors|)
         (|clearClams|))
        ((null (getalist |$globalExposureGroupAlist| x))
         (|sayKeyedMsg| "%1 is not a known exposure group name." (cons x nil)))
        ((|member| x (elt |$localExposureData| 0))
         (|sayKeyedMsg| "%1 is already an exposure group for frame %2"
                        (list x |$interpreterFrameName|)))
        (t
         (setf (elt |$localExposureData| 0)
               (msort (cons x (elt |$localExposureData| 0))))
         (|sayKeyedMsg| "%1 is now an exposure group for frame %2"
                        (list x |$interpreterFrameName|))
         (|clearClams|))))))

```

The top level set expose command handler

```

[displayExposedGroups p165]
[sayMSG p45]
[displayExposedConstructors p165]
[displayHiddenConstructors p166]
[sayKeyedMsg p43]
[namestring p1230]

```

[pathname p1232]
 [qcar p??]
 [qcdr p??]
 [selectOptionLC p819]
 [setExposeAdd p159]
 [setExposeDrop p161]
 [setExpose p158]

— defun setExpose —

```
(defun |setExpose| (arg)
  "The top level set expose command handler"
  (let (fnargs fn)
    (cond
      ((eq arg '|%initialize%|))
      ((eq arg '|%display%|) "...")
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
       (|displayExposedGroups|)
       (|sayMSG| " ")
       (|displayExposedConstructors|)
       (|sayMSG| " ")
       (|displayHiddenConstructors|)
       (|sayMSG| " ")))
    ((and (consp arg)
          (progn (setq fn (qcar arg)) (setq fnargs (qcdr arg)) t)
          (setq fn (|selectOptionLC| fn '(|add| |drop|) nil))))
    (cond
      ((eq fn '|add|) (|setExposeAdd| fnargs))
      ((eq fn '|drop|) (|setExposeDrop| fnargs))
      (t nil)))
    (t (|setExpose| nil))))
```

—————

The top level set expose add command handler

[specialChar p1166]
 [displayExposedGroups p165]
 [sayMSG p45]
 [displayExposedConstructors p165]
 [sayKeyedMsg p43]
 [qcar p??]
 [qcdr p??]
 [selectOptionLC p819]
 [setExposeAddGroup p157]
 [setExposeAddConstr p160]

```
[setExposeAdd p159]
[$linelength p1051]
```

— defun setExposeAdd —

```
(defun |setExposeAdd| (arg)
  "The top level set expose add command handler"
  (declare (special $linelength))
  (let (fnargs fn)
    (cond
      ((null arg)
        (format t "~v,,,':@<~a~>~%" (- $linelength 2) " The add Option ")
        (|displayExposedGroups|)
        (|sayMSG| " ")
        (|displayExposedConstructors|)
        (|sayMSG| " ")
        (|sayKeyedMsg|
         (format nil
          "When )set expose add is followed by no arguments, the information ~
          you now see is displayed. ~
          The arguments group and constructor are used to specify ~
          exposure groups or an explicit constructor to be added to the local ~
          frame exposure data. Issue ~
          %ceon )set expose add group %ceoff or ~
          %ceon )set expose add constructor %ceoff ~
          for more information.")
         nil))
      ((and (consp arg)
            (progn (setq fn (qcar arg)) (setq fnargs (qcdr arg)) t)
            (setq fn (|selectOptionLC| fn '(|group| |constructor|) nil))))
        (cond
          ((eq fn '|group|) (|setExposeAddGroup| fnargs))
          ((eq fn '|constructor|) (|setExposeAddConstr| fnargs))
          (t nil)))
      (t (|setExposeAdd| nil))))))
```

—————

The top level set expose add constructor handler

```
[unabbrev p??]
[qcar p??]
[getdatabase p1196]
[sayKeyedMsg p43]
[member p1238]
[delete p??]
[msort p??]
```

```
[clearClams p??]
[specialChar p1166]
[displayExposedConstructors p165]
[$linelength p1051]
[$localExposureData p166]
[$interpreterFrameName p38]
```

— defun setExposeAddConstr —

```
(defun |setExposeAddConstr| (arg)
  "The top level set expose add constructor handler"
  (declare (special $linelength $localExposureData $interpreterFrameName))
  (if (null arg)
      (progn
        (format t "~v,,,':@<~a~>~%" (- $linelength 2) " The constructor Option ")
        (|displayExposedConstructors|))
      (dolist (x arg)
        (setq x (|unabbrev| x))
        (when (consp x) (setq x (qcar x)))
        (cond
         ((null (getdatabase x 'constructorkind))
          (|sayKeyedMsg|
           (format nil "%1 is not a known constructor. ~
            You can make the constructor known to the system by loading it."
            (list x)))
          (|member| x (elt $localExposureData 1))
          (|sayKeyedMsg| "%1 is already explicitly exposed in frame %2"
            (list x $interpreterFrameName) )))
         (t
          (when (|member| x (elt $localExposureData 2))
            (setf (elt $localExposureData 2)
              (|delete| x (elt $localExposureData 2))))
            (setf (elt $localExposureData 1)
              (msort (cons x (elt $localExposureData 1)))))
            (|clearClams|)
            (|sayKeyedMsg| "%1 is now explicitly exposed in frame %2"
              (list x $interpreterFrameName) ))))))))
```

—————

The top level set expose drop handler

```
[specialChar p1166]
[displayHiddenConstructors p166]
[sayMSG p45]
[sayKeyedMsg p43]
[qcar p??]
```



```
[qcdr p??]
[selectOptionLC p819]
[setExposeDropGroup p162]
[setExposeDropConstr p164]
[setExposeDrop p161]
[$linelength p1051]
```

— defun setExposeDrop —

```
(defun |setExposeDrop| (arg)
  "The top level set expose drop handler"
  (declare (special $linelength))
  (let (fnargs fn)
    (cond
      ((null arg)
        (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " The drop Option ")
        (|displayHiddenConstructors|)
        (|sayMSG| " ")
        (|sayKeyedMsg|
          (format nil "When )set expose drop is followed by no arguments, the ~
            information you now see is displayed. The arguments group and ~
            constructor are used to specify exposure groups or an explicit ~
            constructor to be dropped from the local frame exposure data. Issue ~
            %ceon )set expose drop group %ceoff or %ceon )set expose drop ~
            constructor %ceoff for more information.")
          nil))
      ((and (consp arg)
        (progn (setq fn (qcar arg)) (setq fnargs (qcdr arg)) t)
        (setq fn (|selectOptionLC| fn '(|group| |constructor|) nil))))
        (cond
          ((eq fn '|group|) (|setExposeDropGroup| fnargs))
          ((eq fn '|constructor|) (|setExposeDropConstr| fnargs))
          (t nil)))
      (t (|setExposeDrop| nil))))))
```

—————

The top level set expose drop group handler

```
[qcar p??]
[displayExposedGroups p165]
[sayMSG p45]
[displayExposedConstructors p165]
[displayHiddenConstructors p166]
[clearClams p??]
[member p1238]
[delete p??]
```

```
[sayKeyedMsg p43]
[getalist p??]
[specialChar p1166]
[$linelength p1051]
[$localExposureData p166]
[$interpreterFrameName p38]
[$globalExposureGroupAlist p167]
```

— defun setExposeDropGroup —

```
(defun |setExposeDropGroup| (arg)
  "The top level set expose drop group handler"
  (declare (special $linelength $localExposureData $interpreterFrameName
                    $globalExposureGroupAlist))
  (if (null arg)
      (progn
        (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " The group Option ")
        (|sayKeyedMsg|
         (format nil "When followed by one or more exposure group names, this ~
                    option allows you to remove those groups from the local ~
                    frame exposure data.")
         nil))
      (|sayMSG| " ")
      (|displayExposedGroups|))
  (dolist (x arg)
    (when (consp x) (setq x (qcar x)))
    (cond
     ((eq x '|all|)
      (setf (elt $localExposureData 0) nil)
      (setf (elt $localExposureData 1) nil)
      (setf (elt $localExposureData 2) nil)
      (|displayExposedGroups|)
      (|sayMSG| " ")
      (|displayExposedConstructors|)
      (|sayMSG| " ")
      (|displayHiddenConstructors|)
      (|clearClams|))
     ((|member| x (elt $localExposureData 0))
      (setf (elt $localExposureData 0)
            (|delete| x (elt $localExposureData 0)))
      (|clearClams|)
      (|sayKeyedMsg| "%1 is no longer an exposure group for frame %2"
                     (list x $interpreterFrameName)))
     ((getalist $globalExposureGroupAlist x)
      (|sayKeyedMsg| "%1 is already an exposure group for frame %2"
                     (list x $interpreterFrameName)))
     (t (|sayKeyedMsg| "%1 is not a known exposure group name." (list x ))))))
```

The top level set expose drop constructor handler

```
[unabbrev p??]
[qcar p??]
[getdatabase p1196]
[sayKeyedMsg p43]
[member p1238]
[delete p??]
[msort p??]
[clearClams p??]
[specialChar p1166]
[sayMSG p45]
[displayExposedConstructors p165]
[displayHiddenConstructors p166]
[$linelength p1051]
[$localExposureData p166]
[$interpreterFrameName p38]
```

— defun setExposeDropConstr —

```
(defun |setExposeDropConstr| (arg)
  "The top level set expose drop constructor handler"
  (declare (special $linelength |$localExposureData| |$interpreterFrameName|))
  (if (null arg)
    (progn
      (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " The constructor Option ")
      (|sayKeyedMsg|
        (format nil "When followed by one or more constructor names, this option ~
                    allows you to explicitly hide constructors in this frame.")
        nil)
      (|sayMSG| " ")
      (|displayExposedConstructors|)
      (|sayMSG| " ")
      (|displayHiddenConstructors|))
    (dolist (x arg)
      (setq x (|unabbrev| x))
      (when (consp x) (setq x (qcar x)))
      (cond
        ((null (getdatabase x 'constructorkind))
         (|sayKeyedMsg|
          (format nil "%1 is not a known constructor. ~
                    You can make the constructor known to the system by loading it.")
          (list x)))
        ((|member| x (elt |$localExposureData| 2))
         (|sayKeyedMsg| "%1 is already explicitly hidden in frame %2"))
```

```

      (list x |$interpreterFrameName|)))
    (t
      (when (|member| x (elt |$localExposureData| 1))
        (setf (elt |$localExposureData| 1)
          (|delete| x (elt |$localExposureData| 1))))
      (setf (elt |$localExposureData| 2)
        (msort (cons x (elt |$localExposureData| 2))))
      (|clearClams|)
      (|sayKeyedMsg| "%1 is now explicitly hidden in frame %2"
        (list x |$interpreterFrameName|))))))

```

Display exposed groups

[sayKeyedMsg p43]
 [\$interpreterFrameName p38]
 [\$localExposureData p166]

— defun displayExposedGroups —

```

(defun |displayExposedGroups| ()
  "Display exposed groups"
  (declare (special |$interpreterFrameName| |$localExposureData|))
  (|sayKeyedMsg|
    (format nil "The following groups are explicitly exposed in the current ~
                frame (called %1):")
    (list |$interpreterFrameName|))
  (if (null (elt |$localExposureData| 0))
    (format t "~v:@<~a>~%" (- $linelength 2) " there are no exposed groups ")
    (dolist (c (elt |$localExposureData| 0))
      (format t "~v:@<~a>~%" (- $linelength 2) c))))

```

Display exposed constructors

[sayKeyedMsg p43]
 [\$localExposureData p166]

— defun displayExposedConstructors —

```

(defun |displayExposedConstructors| ()
  "Display exposed constructors"
  (declare (special |$localExposureData|))

```

```
(|sayKeyedMsg|
  "The following constructors are explicitly exposed in the current frame:"
  nil)
(if (null (elt |$localExposureData| 1))
  (format t "~v:@<~a~>~%" (- $linelength 2)
    "there are no explicitly exposed constructors")
  (dolist (c (elt |$localExposureData| 1))
    (format t "~v:@<~a~>~%" (- $linelength 2) c))))
```

Display hidden constructors

[sayKeyedMsg p43]
 [\$localExposureData p166]

— defun displayHiddenConstructors —

```
(defun |displayHiddenConstructors| ()
  "Display hidden constructors"
  (declare (special |$localExposureData|))
  (|sayKeyedMsg|
    "The following constructors are explicitly hidden in the current frame:"
    nil)
  (if (null (elt |$localExposureData| 2))
    (format t "~v:@<~a~>~%" (- $linelength 2)
      "there are no explicitly hidden constructors")
    (dolist (c (elt |$localExposureData| 2))
      (format t "~v:@<~a~>~%" (- $linelength 2) c))))
```

8.2 Exposure Data Structures

defvar \$localExposureData

— postvars —

```
(defvar |$localExposureData| (copy-seq |$localExposureDataDefault|))
```

defvar \$localExposureDataDefault

— initvars —

```
(defvar |$localExposureDataDefault|
  (vector
    ;;These groups will be exposed
    (list '|basic| '|categories| '|naglink| '|anna|)
    ;;These constructors will be explicitly exposed
    (list )
    ;;These constructors will be explicitly hidden
    (list )))
```

NOTE: If you add new algebra you must also update this list otherwise the new algebra won't be loaded by the interpreter when needed.

defvar \$globalExposureGroupAlist

— initvars —

```
(defvar |$globalExposureGroupAlist|
  '(
    ;;define the groups |basic| |naglink| |anna| |categories| |Hidden| |defaults|
    (|basic|
      (|AffineAlgebraicSetComputeWithGroebnerBasis| . AFALGGRO)
      (|AffineAlgebraicSetComputeWithResultant| . AFALGRES)
      (|AffinePlane| . AFFPL)
      (|AffinePlaneOverPseudoAlgebraicClosureOfFiniteField| . AFFPLPS)
      (|AffineSpace| . AFFSP)
      (|AlgebraicManipulations| . ALGMANIP)
      (|AlgebraicNumber| . AN)
      (|AlgFactor| . ALGFACT)
      (|AlgebraicMultFact| . ALGMFACT)
      (|AlgebraPackage| . ALGPKG)
      (|AlgebraGivenByStructuralConstants| . ALGSC)
      (|Any| . ANY)
      (|AnyFunctions1| . ANY1)
      (|ApplicationProgramInterface| . API)
      (|ArrayStack| . ASTACK)
      (|AssociatedJordanAlgebra| . JORDAN)
      (|AssociatedLieAlgebra| . LIE)
      (|AttachPredicates| . PMPRED)
      (|AxiomServer| . AXSERV)
```

```

(|BalancedBinaryTree| . BBTREE)
(|BasicStochasticDifferential| . BSD)
(|BasicOperator| . BOP)
(|BasicOperatorFunctions1| . BOP1)
(|Bezier| . BEZIER)
(|BinaryExpansion| . BINARY)
(|BinaryFile| . BINFILE)
(|BinarySearchTree| . BSTREE)
(|BinaryTournament| . BTOURN)
(|BinaryTree| . BTREE)
(|Bits| . BITS)
(|BlasLevelOne| . BLAS1)
(|BlowUpPackage| . BLUPACK)
(|BlowUpWithHamburgerNoether| . BLHN)
(|BlowUpWithQuadTrans| . BLQT)
(|Boolean| . BOOLEAN)
(|CardinalNumber| . CARD)
(|CartesianTensor| . CARTEN)
(|CartesianTensorFunctions2| . CARTEN2)
(|Cell| . CELL)
(|Character| . CHAR)
(|CharacterClass| . CCLASS)
(|CharacteristicPolynomialPackage| . CHARPOL)
(|CliffordAlgebra| . CLIF)
(|Color| . COLOR)
(|CommonDenominator| . CDEN)
(|Commutator| . COMM)
(|Complex| . COMPLEX)
(|ComplexDoubleFloatMatrix| . CDFMAT)
(|ComplexDoubleFloatVector| . CDFVEC)
(|ComplexFactorization| . COMPFAC)
(|ComplexFunctions2| . COMPLEX2)
(|ComplexRootPackage| . CMPLXRT)
(|ComplexTrigonometricManipulations| . CTRIGMNP)
(|ContinuedFraction| . CONTFRAC)
(|CoordinateSystems| . COORDSYS)
(|CRApackage| . CRAPACK)
(|CycleIndicators| . CYCLES)
(|CylindricalAlgebraicDecompositionPackage| . CAD)
(|CylindricalAlgebraicDecompositionUtilities| . CADU)
(|Database| . DBASE)
(|DataList| . DLIST)
(|DecimalExpansion| . DECIMAL)
(|DenavitHartenbergMatrix| . DHMATRIX)
(|Dequeue| . DEQUEUE)
(|DesingTree| . DSTREE)
(|DesingTreePackage| . DTP)
(|DiophantineSolutionPackage| . DIOSP)
(|DirichletRing| . DIRRING)
(|DirectProductFunctions2| . DIRPROD2)

```

```

(|DisplayPackage| . DISPLAY)
(|DistinctDegreeFactorize| . DDFACT)
(|Divisor| . DIV)
(|DoubleFloat| . DFLOAT)
(|DoubleFloatMatrix| . DFMAT)
(|DoubleFloatVector| . DFVEC)
(|DoubleFloatSpecialFunctions| . DFSFUN)
(|DrawComplex| . DRAWCX)
(|DrawNumericHack| . DRAWHACK)
(|DrawOption| . DROPT)
(|EigenPackage| . EP)
(|ElementaryFunctionDefiniteIntegration| . DEFINTEF)
(|ElementaryFunctionLODESolver| . LODEEF)
(|ElementaryFunctionODESolver| . ODEEF)
(|ElementaryFunctionSign| . SIGNEF)
(|ElementaryFunctionStructurePackage| . EFSTRUC)
(|Equation| . EQ)
(|EquationFunctions2| . EQ2)
(|ErrorFunctions| . ERROR)
(|EuclideanGroebnerBasisPackage| . GBEUCLID)
(|Exit| . EXIT)
(|Export3D| . EXP3D)
(|Expression| . EXPR)
(|ExpressionFunctions2| . EXPR2)
(|ExpressionSolve| . EXPRSOL)
(|ExpressionSpaceFunctions2| . ES2)
(|ExpressionSpaceODESolver| . EXPRODE)
(|ExpressionToOpenMath| . OMEXPR)
(|ExpressionToUnivariatePowerSeries| . EXPR2UPS)
(|Factored| . FR)
(|FactoredFunctions2| . FR2)
(|FactorisationOverPseudoAlgebraicClosureOfAlgExtOfRationalNumber| . FACTEXT)
(|FactorisationOverPseudoAlgebraicClosureOfRationalNumber| . FACTRN)
(|File| . FILE)
(|FileName| . FNAME)
(|FiniteAbelianMonoidRingFunctions2| . FAMR2)
(|FiniteDivisorFunctions2| . FDIV2)
(|FiniteField| . FF)
(|FiniteFieldFactorization| . FFFACTOR)
(|FiniteFieldFactorizationWithSizeParseBySideEffect| . FFFACTSE)
(|FiniteFieldCyclicGroup| . FFCG)
(|FiniteFieldPolynomialPackage2| . FFPOLY2)
(|FiniteFieldNormalBasis| . FFNB)
(|FiniteFieldHomomorphisms| . FFHOM)
(|FiniteFieldSquareFreeDecomposition| . FFSQFR)
(|FiniteLinearAggregateFunctions2| . FLAGG2)
(|FiniteLinearAggregateSort| . FLASORT)
(|FiniteSetAggregateFunctions2| . FSAGG2)
(|FlexibleArray| . FARRAY)
(|Float| . FLOAT)

```



```

(|FloatingRealPackage| . FLOATRP)
(|FloatingComplexPackage| . FLOATCP)
(|FloatSpecialFunctions| . FSFUN)
(|FourierSeries| . FSERIES)
(|Fraction| . FRAC)
(|FractionalIdealFunctions2| . FRIDEAL2)
(|FractionFreeFastGaussian| . FFFG)
(|FractionFreeFastGaussianFractions| . FFFGF)
(|FractionFunctions2| . FRAC2)
(|FreeNilpotentLie| . FNLA)
(|FullPartialFractionExpansion| . FPARFRAC)
(|FunctionFieldCategoryFunctions2| . FFCAT2)
(|FunctionSpaceAssertions| . PMASSFS)
(|FunctionSpaceAttachPredicates| . PMPREDFS)
(|FunctionSpaceComplexIntegration| . FSCINT)
(|FunctionSpaceFunctions2| . FS2)
(|FunctionSpaceIntegration| . FSINT)
(|FunctionSpacePrimitiveElement| . FSPRMELT)
(|FunctionSpaceSum| . SUMFS)
(|GaussianFactorizationPackage| . GAUSSFAC)
(|GeneralPackageForAlgebraicFunctionField| . GPAFF)
(|GeneralUnivariatePowerSeries| . GSERIES)
(|GenerateUnivariatePowerSeries| . GENUPS)
(|Graphviz| . GRAPHVIZ)
(|GnuDraw| . GDRAW)
(|GraphicsDefaults| . GRDEF)
(|GroebnerPackage| . GB)
(|GroebnerFactorizationPackage| . GBF)
(|Guess| . GUESS)
(|GuessAlgebraicNumber| . GUESSAN)
(|GuessFinite| . GUESSF)
(|GuessFiniteFunctions| . GUESSF1)
(|GuessInteger| . GUESSINT)
(|GuessOption| . GOPT)
(|GuessPolynomial| . GUESSP)
(|GuessUnivariatePolynomial| . GUESSUP)
(|HallBasis| . HB)
(|Heap| . HEAP)
(|HexadecimalExpansion| . HEXADEC)
(|HTMLFormat| . HTMLFORM)
(|IdealDecompositionPackage| . IDECOMP)
(|IndexCard| . ICARD)
(|InfClsPt| . ICP)
(|InfiniteProductCharacteristicZero| . INFPROD0)
(|InfiniteProductFiniteField| . INPRODF)
(|InfiniteProductPrimeField| . INPRODPF)
(|InfiniteTuple| . ITUPLE)
(|InfiniteTupleFunctions2| . ITFUN2)
(|InfiniteTupleFunctions3| . ITFUN3)
(|InfinitelyClosePoint| . INFCLSPT)

```

```

(|InfinitelyClosePointOverPseudoAlgebraicClosureOfFiniteField| . INFCLSPS)
(|Infinity| . INFINITY)
(|Integer| . INT)
(|IntegerCombinatoricFunctions| . COMBINAT)
(|IntegerLinearDependence| . ZLINDEP)
(|IntegerNumberTheoryFunctions| . INTHEORY)
(|IntegerPrimesPackage| . PRIMES)
(|IntegerRetractions| . INTRET)
(|IntegerRoots| . IROOT)
(|IntegrationResultFunctions2| . IR2)
(|IntegrationResultRFToFunction| . IRRF2F)
(|IntegrationResultToFunction| . IR2F)
(|InterfaceGroebnerPackage| . INTERGB)
(|InterpolateFormsPackage| . INTFRSP)
(|IntersectionDivisorPackage| . INTDIVP)
(|Interval| . INTRVL)
(|InventorDataSink| . IVDATA)
(|InventorViewPort| . IVVIEW)
(|InventorRenderPackage| . IVREND)
(|InverseLaplaceTransform| . INVLAPLA)
(|IrrRepSymNatPackage| . IRSN)
(|KernelFunctions2| . KERNEL2)
(|KeyedAccessFile| . KAFILE)
(|LaplaceTransform| . LAPLACE)
(|LazardMorenoSolvingPackage| . LAZM3PK)
(|Library| . LIB)
(|LieSquareMatrix| . LSQM)
(|LinearOrdinaryDifferentialOperator| . LOD0)
(|LinearSystemMatrixPackage| . LSMP)
(|LinearSystemMatrixPackage1| . LSMP1)
(|LinearSystemFromPowerSeriesPackage| . LISYSER)
(|LinearSystemPolynomialPackage| . LSPP)
(|List| . LIST)
(|LinesOpPack| . LOP)
(|ListFunctions2| . LIST2)
(|ListFunctions3| . LIST3)
(|ListToMap| . LIST2MAP)
(|LocalParametrizationOfSimplePointPackage| . LPARSPT)
(|MakeFloatCompiledFunction| . MKFLCFN)
(|MakeFunction| . MKFUNC)
(|MakeRecord| . MKRECORD)
(|MappingPackage1| . MAPPKG1)
(|MappingPackage2| . MAPPKG2)
(|MappingPackage3| . MAPPKG3)
(|MappingPackage4| . MAPPKG4)
(|MathMLFormat| . MMLFORM)
(|Matrix| . MATRIX)
(|MatrixCategoryFunctions2| . MATCAT2)
(|MatrixCommonDenominator| . MCDEN)
(|MatrixLinearAlgebraFunctions| . MATLIN)

```

```

(|MatrixManipulation| . MAMA)
(|MergeThing| . MTHING)
(|ModularDistinctDegreeFactorizer| . MDDFACT)
(|ModuleOperator| . MODOP)
(|MonoidRingFunctions2| . MRF2)
(|MoreSystemCommands| . MSYSCMD)
(|MPolyCatFunctions2| . MPC2)
(|MPolyCatRationalFunctionFactorizer| . MPRFF)
(|Multiset| . MSET)
(|MultivariateFactorize| . MULTFACT)
(|MultivariatePolynomial| . MPOLY)
(|MultFiniteFactorize| . MFINFACT)
(|MyUnivariatePolynomial| . MYUP)
(|MyExpression| . MYEXPR)
(|NeitherSparseOrDensePowerSeries| . NSDPS)
(|NewtonPolygon| . NPOLYGON)
(|NoneFunctions1| . NONE1)
(|NonNegativeInteger| . NNI)
(|NottinghamGroup| . NOTTING)
(|NormalizationPackage| . NORMPK)
(|NormInMonogenicAlgebra| . NORMMA)
(|NumberTheoreticPolynomialFunctions| . NTPOLFN)
(|Numeric| . NUMERIC)
(|NumericalOrdinaryDifferentialEquations| . NUMODE)
(|NumericalQuadrature| . NUMQUAD)
(|NumericComplexEigenPackage| . NCEP)
(|NumericRealEigenPackage| . NREP)
(|NumericContinuedFraction| . NCNTFRAC)
(|Octonion| . OCT)
(|OctonionCategoryFunctions2| . OCTCT2)
(|OneDimensionalArray| . ARRAY1)
(|OneDimensionalArrayFunctions2| . ARRAY12)
(|OnePointCompletion| . ONECOMP)
(|OnePointCompletionFunctions2| . ONECOMP2)
(|OpenMathConnection| . OMCONN)
(|OpenMathDevice| . OMDEV)
(|OpenMathEncoding| . OMENC)
(|OpenMathError| . OMERR)
(|OpenMathErrorKind| . OMERRK)
(|OpenMathPackage| . OMPKG)
(|OpenMathServerPackage| . OMSERVER)
(|OperationsQuery| . OPQUERY)
(|OrderedCompletion| . ORDCOMP)
(|OrderedCompletionFunctions2| . ORDCOMP2)
(|OrdinaryDifferentialRing| . ODR)
(|OrdSetInts| . OSI)
(|OrthogonalPolynomialFunctions| . ORTHPOL)
(|OutputPackage| . OUT)
(|PackageForAlgebraicFunctionField| . PAFF)
(|PackageForAlgebraicFunctionFieldOverFiniteField| . PAFFFF)

```

```

(|PackageForPoly| . PFORP)
(|PadeApproximantPackage| . PADEPAC)
(|Palette| . PALETTE)
(|PartialFraction| . PFR)
(|PatternFunctions2| . PATTERN2)
(|ParametricPlaneCurve| . PARPCURV)
(|ParametricSpaceCurve| . PARSCURV)
(|ParametricSurface| . PARSURF)
(|ParametricPlaneCurveFunctions2| . PARPC2)
(|ParametricSpaceCurveFunctions2| . PARSC2)
(|ParametricSurfaceFunctions2| . PARSU2)
(|ParametrizationPackage| . PARAMP)
(|PartitionsAndPermutations| . PARTPERM)
(|PatternMatch| . PATMATCH)
(|PatternMatchAssertions| . PMASS)
(|PatternMatchResultFunctions2| . PATRES2)
(|PendantTree| . PENDTREE)
(|Permanent| . PERMAN)
(|PermutationGroupExamples| . PGE)
(|PermutationGroup| . PERMGRP)
(|Permutation| . PERM)
(|Pi| . HACKPI)
(|PiCoercions| . PICOERCE)
(|Places| . PLACES)
(|PlacesOverPseudoAlgebraicClosureOfFiniteField| . PLACESPS)
(|Plcs| . PLCS)
(|PointFunctions2| . PTFUNC2)
(|PolyGroebner| . PGROEB)
(|Polynomial| . POLY)
(|PolynomialAN2Expression| . PAN2EXPR)
(|PolynomialComposition| . PCOMP)
(|PolynomialDecomposition| . PDECOMP)
(|PolynomialFunctions2| . POLY2)
(|PolynomialIdeals| . IDEAL)
(|PolynomialPackageForCurve| . PLPKCRV)
(|PolynomialToUnivariatePolynomial| . POLY2UP)
(|PositiveInteger| . PI)
(|PowerSeriesLimitPackage| . LIMITPS)
(|PrimeField| . PF)
(|PrimitiveArrayFunctions2| . PRIMARR2)
(|PrintPackage| . PRINT)
(|ProjectiveAlgebraicSetPackage| . PRJALGPK)
(|ProjectivePlane| . PROJPL)
(|ProjectivePlaneOverPseudoAlgebraicClosureOfFiniteField| . PROJPLPS)
(|ProjectiveSpace| . PROJSP)
(|PseudoAlgebraicClosureOfAlgExtOfRationalNumber| . PACEXT)
(|QuadraticForm| . QFORM)
(|QuasiComponentPackage| . QCMPACK)
(|Quaternion| . QUAT)
(|QuaternionCategoryFunctions2| . QUATCT2)

```

```

(|QueryEquation| . QEQUAT)
(|Queue| . QUEUE)
(|QuotientFieldCategoryFunctions2| . QFCAT2)
(|RadicalEigenPackage| . REP)
(|RadicalSolvePackage| . SOLVERAD)
(|RadixExpansion| . RADIX)
(|RadixUtilities| . RADUTIL)
(|RandomNumberSource| . RANDSRC)
(|RationalFunction| . RF)
(|RationalFunctionDefiniteIntegration| . DEFINTRF)
(|RationalFunctionFactor| . RFFACT)
(|RationalFunctionFactorizer| . RFFACTOR)
(|RationalFunctionIntegration| . INTRF)
(|RationalFunctionLimitPackage| . LIMITRF)
(|RationalFunctionSign| . SIGNRF)
(|RationalFunctionSum| . SUMRF)
(|RationalRetractions| . RATRET)
(|RealClosure| . RECLOS)
(|RealPolynomialUtilitiesPackage| . POLUTIL)
(|RealZeroPackage| . REAL0)
(|RealZeroPackageQ| . REALOQ)
(|RecurrenceOperator| . RECOP)
(|RectangularMatrixCategoryFunctions2| . RMCAT2)
(|RegularSetDecompositionPackage| . RSDCMPK)
(|RegularTriangularSet| . REGSET)
(|RegularTriangularSetGcdPackage| . RSETGCD)
(|RepresentationPackage1| . REP1)
(|RepresentationPackage2| . REP2)
(|ResolveLatticeCompletion| . RESLATC)
(|RewriteRule| . RULE)
(|RightOpenIntervalRootCharacterization| . ROIRC)
(|RomanNumeral| . ROMAN)
(|RootsFindingPackage| . RFP)
(|Ruleset| . RULESET)
(|ScriptFormulaFormat| . FORMULA)
(|ScriptFormulaFormat1| . FORMULA1)
(|Segment| . SEG)
(|SegmentBinding| . SEGBIND)
(|SegmentBindingFunctions2| . SEGBIND2)
(|SegmentFunctions2| . SEG2)
(|Set| . SET)
(|SimpleAlgebraicExtensionAlgFactor| . SAEFACT)
(|SimpleCell| . SCELL)
(|SimplifyAlgebraicNumberConvertPackage| . SIMPAN)
(|SingleInteger| . SINT)
(|SmithNormalForm| . SMITH)
(|SparseEchelonMatrix| . SEM)
(|SparseUnivariatePolynomialExpressions| . SUEXPR)
(|SparseUnivariatePolynomialFunctions2| . SUP2)
(|SpecialOutputPackage| . SPECOUT)

```

```

(|SquareFreeRegularSetDecompositionPackage| . SRDCMPK)
(|SquareFreeRegularTriangularSet| . SREGSET)
(|SquareFreeRegularTriangularSetGcdPackage| . SFRGCD)
(|SquareFreeQuasiComponentPackage| . SFQCMPK)
(|Stack| . STACK)
(|Stream| . STREAM)
(|StreamFunctions1| . STREAM1)
(|StreamFunctions2| . STREAM2)
(|StreamFunctions3| . STREAM3)
(|StreamTensor| . STNSR)
(|StochasticDifferential| . SD)
(|String| . STRING)
(|SturmHabichtPackage| . SHP)
(|Symbol| . SYMBOL)
(|SymmetricGroupCombinatoricFunctions| . SGCF)
(|SystemSolvePackage| . SYSSOLP)
(|SAERationalFunctionAlgFactor| . SAERFFC)
(|Tableau| . TABLEAU)
(|TaylorSeries| . TS)
(|TaylorSolve| . UTSSOL)
(|TexFormat| . TEX)
(|TexFormat1| . TEX1)
(|TextFile| . TEXTFILE)
(|ThreeDimensionalViewport| . VIEW3D)
(|ThreeSpace| . SPACE3)
(|Timer| . TIMER)
(|TopLevelDrawFunctions| . DRAW)
(|TopLevelDrawFunctionsForAlgebraicCurves| . DRAWCURV)
(|TopLevelDrawFunctionsForCompiledFunctions| . DRAWCFUN)
(|TopLevelDrawFunctionsForPoints| . DRAWPT )
(|TopLevelThreeSpace| . TOPSP)
(|TranscendentalManipulations| . TRMANIP)
(|TransSolvePackage| . SOLVETRA)
(|Tree| . TREE)
(|TrigonometricManipulations| . TRIGMNIP)
(|UnivariateLaurentSeriesFunctions2| . ULS2)
(|UnivariateFormalPowerSeries| . UFPS)
(|UnivariateFormalPowerSeriesFunctions| . UFPS1)
(|UnivariatePolynomial| . UP)
(|UnivariatePolynomialCategoryFunctions2| . UPOLYC2)
(|UnivariatePolynomialCommonDenominator| . UPCDEN)
(|UnivariatePolynomialFunctions2| . UP2)
(|UnivariatePolynomialMultiplicationPackage| . UPMP)
(|UnivariateTaylorSeriesCZero| . UTSZ)
(|UnivariatePuisseuxSeriesFunctions2| . UPXS2)
(|UnivariateTaylorSeriesFunctions2| . UTS2)
(|UniversalSegment| . UNISEG)
(|UniversalSegmentFunctions2| . UNISEG2)
(|UserDefinedVariableOrdering| . UDVO)
(|U8Matrix| . U8MAT)

```

```

(|U16Matrix| . U16MAT)
(|U32Matrix| . U32MAT)
(|U8Vector| . U8VEC)
(|U16Vector| . U16VEC)
(|U32Vector| . U32VEC)
(|U32VectorPolynomialOperations| . POLYVEC)
(|Vector| . VECTOR)
(|VectorFunctions2| . VECTOR2)
(|ViewDefaultsPackage| . VIEWDEF)
(|Void| . VOID)
(|WuWenTsunTriangularSet| . WUTSET))
(|naglink|
  (|Asp1| . ASP1)
  (|Asp4| . ASP4)
  (|Asp6| . ASP6)
  (|Asp7| . ASP7)
  (|Asp8| . ASP8)
  (|Asp9| . ASP9)
  (|Asp10| . ASP10)
  (|Asp12| . ASP12)
  (|Asp19| . ASP19)
  (|Asp20| . ASP20)
  (|Asp24| . ASP24)
  (|Asp27| . ASP27)
  (|Asp28| . ASP28)
  (|Asp29| . ASP29)
  (|Asp30| . ASP30)
  (|Asp31| . ASP31)
  (|Asp33| . ASP33)
  (|Asp34| . ASP34)
  (|Asp35| . ASP35)
  (|Asp41| . ASP41)
  (|Asp42| . ASP42)
  (|Asp49| . ASP49)
  (|Asp50| . ASP50)
  (|Asp55| . ASP55)
  (|Asp73| . ASP73)
  (|Asp74| . ASP74)
  (|Asp77| . ASP77)
  (|Asp78| . ASP78)
  (|Asp80| . ASP80)
  (|FortranCode| . FC)
  (|FortranCodePackage1| . FCPAK1)
  (|FortranExpression| . FEXPR)
  (|FortranMachineTypeCategory| . FMTC)
  (|FortranMatrixCategory| . FMC)
  (|FortranMatrixFunctionCategory| . FMFUN)
  (|FortranOutputStackPackage| . FOP)
  (|FortranPackage| . FORT)
  (|FortranProgramCategory| . FORTCAT)

```

```

(|FortranProgram| . FORTRAN)
(|FortranFunctionCategory| . FORTFN)
(|FortranScalarType| . FST)
(|FortranType| . FT)
(|FortranTemplate| . FTEM)
(|FortranVectorFunctionCategory| . FVFUN)
(|FortranVectorCategory| . FVC)
(|MachineComplex| . MCMPLX)
(|MachineFloat| . MFLOAT)
(|MachineInteger| . MINT)
(|MultiVariableCalculusFunctions| . MCALCFN)
(|NagDiscreteFourierTransformInterfacePackage| . NAGDIS)
(|NagEigenInterfacePackage| . NAGEIG)
(|NAGLinkSupportPackage| . NAGSP)
(|NagOptimisationInterfacePackage| . NAGOPT)
(|NagQuadratureInterfacePackage| . NAGQUA)
(|NagResultChecks| . NAGRES)
(|NagSpecialFunctionsInterfacePackage| . NAGSPE)
(|NagPolynomialRootsPackage| . NAGC02)
(|NagRootFindingPackage| . NAGC05)
(|NagSeriesSummationPackage| . NAGC06)
(|NagIntegrationPackage| . NAGD01)
(|NagOrdinaryDifferentialEquationsPackage| . NAGD02)
(|NagPartialDifferentialEquationsPackage| . NAGD03)
(|NagInterpolationPackage| . NAGE01)
(|NagFittingPackage| . NAGE02)
(|NagOptimisationPackage| . NAGE04)
(|NagMatrixOperationsPackage| . NAGF01)
(|NagEigenPackage| . NAGF02)
(|NagLinearEquationSolvingPackage| . NAGF04)
(|NagLapack| . NAGF07)
(|NagSpecialFunctionsPackage| . NAGS)
(|PackedHermitianSequence| . PACKED)
(|Result| . RESULT)
(|SimpleFortranProgram| . SFORT)
(|Switch| . SWITCH)
(|SymbolTable| . SYMTAB)
(|TemplateUtilities| . TEMUTL)
(|TheSymbolTable| . SYMS)
(|ThreeDimensionalMatrix| . M3D))
(|anna|
  (|AnnaNumericalIntegrationPackage| . INTPACK)
  (|AnnaNumericalOptimizationPackage| . OPTPACK)
  (|AnnaOrdinaryDifferentialEquationPackage| . ODEPACK)
  (|AnnaPartialDifferentialEquationPackage| . PDEPACK)
  (|AttributeButtons| . ATTRBUT)
  (|BasicFunctions| . BFUNCT)
  (|d01ajfAnnaType| . D01AJFA)
  (|d01akfAnnaType| . D01AKFA)
  (|d01alfAnnaType| . D01ALFA)

```



```

(|d01amfAnnaType| . D01AMFA)
(|d01anfAnnaType| . D01ANFA)
(|d01apfAnnaType| . D01APFA)
(|d01aqfAnnaType| . D01AQFA)
(|d01asfAnnaType| . D01ASFA)
(|d01fcfAnnaType| . D01FCFA)
(|d01gbfAnnaType| . D01GBFA)
(|d01AgentsPackage| . D01AGNT)
(|d01TransformFunctionType| . D01TRNS)
(|d01WeightsPackage| . D01WGTS)
(|d02AgentsPackage| . D02AGNT)
(|d02bbfAnnaType| . D02BBFA)
(|d02bhfAnnaType| . D02BHFA)
(|d02cjfAnnaType| . D02CJFA)
(|d02ejfAnnaType| . D02EJFA)
(|d03AgentsPackage| . D03AGNT)
(|d03eefAnnaType| . D03EEFA)
(|d03fafAnnaType| . D03FAFA)
(|e04AgentsPackage| . E04AGNT)
(|e04dgfAnnaType| . E04DGFA)
(|e04fdfAnnaType| . E04FDFA)
(|e04gcfAnnaType| . E04GCFA)
(|e04jafAnnaType| . E04JAFA)
(|e04mbfAnnaType| . E04MBFA)
(|e04nafAnnaType| . E04NAFA)
(|e04ucfAnnaType| . E04UCFA)
(|ExpertSystemContinuityPackage| . ESCONT)
(|ExpertSystemContinuityPackage1| . ESCONT1)
(|ExpertSystemToolsPackage| . ESTOOLS)
(|ExpertSystemToolsPackage1| . ESTOOLS1)
(|ExpertSystemToolsPackage2| . ESTOOLS2)
(|NumericalIntegrationCategory| . NUMINT)
(|NumericalIntegrationProblem| . NIPROB)
(|NumericalODEProblem| . ODEPROB)
(|NumericalOptimizationCategory| . OPTCAT)
(|NumericalOptimizationProblem| . OPTPROB)
(|NumericalPDEProblem| . PDEPROB)
(|ODEIntensityFunctionsTable| . ODEIFTBL)
(|IntegrationFunctionsTable| . INTFTBL)
(|OrdinaryDifferentialEquationsSolverCategory| . ODECAT)
(|PartialDifferentialEquationsSolverCategory| . PDECAT)
(|RoutinesTable| . ROUTINE))
(|categories|
  (|AbelianGroup| . ABELGRP)
  (|AbelianMonoid| . ABELMON)
  (|AbelianMonoidRing| . AMR)
  (|AbelianSemiGroup| . ABELSG)
  (|AdditiveValuationAttribute| . ATADDVA)
  (|AffineSpaceCategory| . AFSPCAT)
  (|Aggregate| . AGG)

```

```

(|Algebra| . ALGEBRA)
(|AlgebraicallyClosedField| . ACF)
(|AlgebraicallyClosedFunctionSpace| . ACFS)
(|ApproximateAttribute| . ATAPPRO)
(|ArbitraryExponentAttribute| . ATARBEX)
(|ArbitraryPrecisionAttribute| . ATARBPR)
(|ArcHyperbolicFunctionCategory| . AHYP)
(|ArcTrigonometricFunctionCategory| . ATRIG)
(|AssociationListAggregate| . ALAGG)
(|AttributeRegistry| . ATTREG)
(|BagAggregate| . BGAGG)
(|BasicType| . BASTYPE)
(|BiModule| . BMODULE)
(|BinaryRecursiveAggregate| . BRAGG)
(|BinaryTreeCategory| . BTCAT)
(|BitAggregate| . BTAGG)
(|BlowUpMethodCategory| . BLMETCT)
(|CachableSet| . CACHSET)
(|CancellationAbelianMonoid| . CABMON)
(|CanonicalAttribute| . ATCANON)
(|CanonicalClosedAttribute| . ATCANCL)
(|CanonicalUnitNormalAttribute| . ATCUNOR)
(|CentralAttribute| . ATCENRL)
(|CharacteristicNonZero| . CHARNZ)
(|CharacteristicZero| . CHARZ)
(|CoercibleTo| . KOERCE)
(|Collection| . CLAGG)
(|CombinatorialFunctionCategory| . CFCAT)
(|CombinatorialOpsCategory| . COMBOPC)
(|CommutativeRing| . COMRING)
(|CommutativeStarAttribute| . ATCS)
(|Comparable| . COMPAR)
(|ComplexCategory| . COMPCAT)
(|ConvertibleTo| . KONVERT)
(|DequeueAggregate| . DQAGG)
(|DesingTreeCategory| . DSTRCAT)
(|Dictionary| . DIAGG)
(|DictionaryOperations| . DIOPS)
(|DifferentialExtension| . DIFEXT)
(|DifferentialPolynomialCategory| . DPOLCAT)
(|DifferentialRing| . DIFRING)
(|DifferentialVariableCategory| . DVARCAT)
(|DirectProductCategory| . DIRPCAT)
(|DivisionRing| . DIVRING)
(|DivisorCategory| . DIVCAT)
(|DoublyLinkedAggregate| . DLAGG)
(|ElementaryFunctionCategory| . ELEMFUN)
(|Eltable| . ELTAB)
(|EltableAggregate| . ELTAGG)
(|EntireRing| . ENTIRER)

```

```

(|EuclideanDomain| . EUCDOM)
(|Evalable| . EVALAB)
(|ExpressionSpace| . ES)
(|ExtensibleLinearAggregate| . ELAGG)
(|ExtensionField| . XF)
(|Field| . FIELD)
(|FieldOfPrimeCharacteristic| . FPC)
(|Finite| . FINITE)
(|FileCategory| . FILECAT)
(|FileNameCategory| . FNCAT)
(|FiniteAbelianMonoidRing| . FAMR)
(|FiniteAggregateAttribute| . ATFINAG)
(|FiniteAlgebraicExtensionField| . FAXF)
(|FiniteDivisorCategory| . FDIVCAT)
(|FiniteFieldCategory| . FFIELDC)
(|FiniteLinearAggregate| . FLAGG)
(|FiniteRankNonAssociativeAlgebra| . FINAALG)
(|FiniteRankAlgebra| . FINRALG)
(|FiniteSetAggregate| . FSAGG)
(|FloatingPointSystem| . FPS)
(|FramedAlgebra| . FRAMALG)
(|FramedNonAssociativeAlgebra| . FRNAALG)
(|FramedNonAssociativeAlgebraFunctions2| . FRNAAF2)
(|FreeAbelianMonoidCategory| . FAMONC)
(|FreeLieAlgebra| . FLALG)
(|FreeModuleCat| . FMCAT)
(|FullyEvalableOver| . FEVALAB)
(|FullyLinearlyExplicitRingOver| . FLINEXP)
(|FullyPatternMatchable| . FPATMAB)
(|FullyRetractableTo| . FRETRCT)
(|FunctionFieldCategory| . FFCAT)
(|FunctionSpace| . FS)
(|GcdDomain| . GCDDOM)
(|GradedAlgebra| . GRALG)
(|GradedModule| . GRMOD)
(|Group| . GROUP)
(|HomogeneousAggregate| . HOAGG)
(|HyperbolicFunctionCategory| . HYPCAT)
(|IndexedAggregate| . IXAGG)
(|IndexedDirectProductCategory| . IDPC)
(|InfinitelyClosePointCategory| . INFCLCT)
(|InnerEvalable| . IEVALAB)
(|IntegerNumberSystem| . INS)
(|IntegralDomain| . INTDOM)
(|IntervalCategory| . INTCAT)
(|KeyedDictionary| . KDAGG)
(|JacobiIdentityAttribute| . ATJACID)
(|LazyRepresentativeAttribute| . ATLR)
(|LazyStreamAggregate| . LZSTAGG)
(|LeftAlgebra| . LALG)

```

```

(|LeftModule| . LMODULE)
(|LeftOreRing| . LORER)
(|LeftUnitaryAttribute| . ATLUNIT)
(|LieAlgebra| . LIECAT)
(|LinearAggregate| . LNAGG)
(|LinearlyExplicitRingOver| . LINEXP)
(|LinearOrdinaryDifferentialOperatorCategory| . LODOCAT)
(|LiouvillianFunctionCategory| . LFCAT)
(|ListAggregate| . LSAGG)
(|LocalPowerSeriesCategory| . LOCPWC)
(|Logic| . LOGIC)
(|ModularAlgebraicGcdOperations| . MAGCDC)
(|MatrixCategory| . MATCAT)
(|Module| . MODULE)
(|Monad| . MONAD)
(|MonadWithUnit| . MONADWU)
(|Monoid| . MONOID)
(|MonogenicAlgebra| . MONOGEN)
(|MonogenicLinearOperator| . MLO)
(|MultiDictionary| . MDAGG)
(|MultiplicativeValuationAttribute| . ATMULVA)
(|MultisetAggregate| . MSETAGG)
(|MultivariateTaylorSeriesCategory| . MTSCAT)
(|NonAssociativeAlgebra| . NAALG)
(|NonAssociativeRing| . NASRING)
(|NonAssociativeRng| . NARNG)
(|NormalizedTriangularSetCategory| . NTSCAT)
(|NoetherianAttribute| . ATNOTHR)
(|NullSquareAttribute| . ATNULSQ)
(|NoZeroDivisorsAttribute| . ATNZDIV)
(|Object| . OBJECT)
(|OctonionCategory| . OC)
(|OneDimensionalArrayAggregate| . A1AGG)
(|OpenMath| . OM)
(|OrderedAbelianGroup| . OAGROUP)
(|OrderedAbelianMonoid| . OAMON)
(|OrderedAbelianMonoidSup| . OAMONS)
(|OrderedAbelianSemiGroup| . OASGP)
(|OrderedCancellationAbelianMonoid| . OCAMON)
(|OrderedFinite| . ORDFIN)
(|OrderedIntegralDomain| . OINTDOM)
(|OrderedMonoid| . ORDMON)
(|OrderedMultisetAggregate| . OMSAGG)
(|OrderedRing| . ORDRING)
(|OrderedSet| . ORDSET)
(|PAdicIntegerCategory| . PADICCT)
(|PartialDifferentialRing| . PDRING)
(|PartiallyOrderedSetAttribute| . ATPOSET)
(|PartialTranscendentalFunctions| . PTRANFN)
(|Patternable| . PATAB)

```

```

(|PatternMatchable| . PATMAB)
(|PermutationCategory| . PERMCAT)
(|PlacesCategory| . PLACESC)
(|PlottablePlaneCurveCategory| . PPCURVE)
(|PlottableSpaceCurveCategory| . PSCURVE)
(|PointCategory| . PTCAT)
(|PolynomialCategory| . POLYCAT)
(|PolynomialFactorizationExplicit| . PFECAT)
(|PolynomialSetCategory| . PSETCAT)
(|PowerSeriesCategory| . PSCAT)
(|PrimitiveFunctionCategory| . PRIMCAT)
(|PrincipalIdealDomain| . PID)
(|PriorityQueueAggregate| . PRQAGG)
(|ProjectiveSpaceCategory| . PRSPCAT)
(|PseudoAlgebraicClosureOfAlgExtOfRationalNumberCategory| . PACEXTC)
(|PseudoAlgebraicClosureOfFiniteField| . PACOFF)
(|PseudoAlgebraicClosureOfFiniteFieldCategory| . PACFFC)
(|PseudoAlgebraicClosureOfPerfectFieldCategory| . PACPERC)
(|PseudoAlgebraicClosureOfRationalNumber| . PACRAT)
(|PseudoAlgebraicClosureOfRationalNumberCategory| . PACRATC)
(|QuaternionCategory| . QUATCAT)
(|QueueAggregate| . QUAGG)
(|QuotientFieldCategory| . QFCAT)
(|RadicalCategory| . RADCAT)
(|RealClosedField| . RCFIELD)
(|RealConstant| . REAL)
(|RealNumberSystem| . RNS)
(|RealRootCharacterizationCategory| . RRCC)
(|RectangularMatrixCategory| . RMATCAT)
(|RecursiveAggregate| . RCAGG)
(|RecursivePolynomialCategory| . RPOLCAT)
(|RegularChain| . RGCHAIN)
(|RegularTriangularSetCategory| . RSETCAT)
(|RetractableTo| . RETRACT)
(|RightModule| . RMODULE)
(|Ring| . RING)
(|RightUnitaryAttribute| . ATRUNIT)
(|Rng| . RNG)
(|SegmentCategory| . SEGCAT)
(|SegmentExpansionCategory| . SEGXCAT)
(|SemiGroup| . SGROUP)
(|SetAggregate| . SETAGG)
(|SetCategory| . SETCAT)
(|SetCategoryWithDegree| . SETCATD)
(|SExpressionCategory| . SEXCAT)
(|ShallowlyMutableAttribute| . ATSHMUT)
(|SpecialFunctionCategory| . SPFCAT)
(|SquareFreeNormalizedTriangularSetCategory| . SNTSCAT)
(|SquareFreeRegularTriangularSetCategory| . SFRTCAT)
(|SquareMatrixCategory| . SMATCAT)

```

```

(|StackAggregate| . SKAGG)
(|StepThrough| . STEP)
(|StreamAggregate| . STAGG)
(|StringAggregate| . SRAGG)
(|StringCategory| . STRICAT)
(|StructuralConstantsPackage| . SCPKG)
(|TableAggregate| . TBAGG)
(|ThreeSpaceCategory| . SPACEC)
(|TranscendentalFunctionCategory| . TRANFUN)
(|TriangularSetCategory| . TSETCAT)
(|TrigonometricFunctionCategory| . TRIGCAT)
(|TwoDimensionalArrayCategory| . ARR2CAT)
(|Type| . TYPE)
(|UnaryRecursiveAggregate| . URAGG)
(|UniqueFactorizationDomain| . UFD)
(|UnitsKnownAttribute| . ATUNIKN)
(|UnivariateLaurentSeriesCategory| . ULSCAT)
(|UnivariateLaurentSeriesConstructorCategory| . ULSCCAT)
(|UnivariatePolynomialCategory| . UPOLYC)
(|UnivariatePowerSeriesCategory| . UPSCAT)
(|UnivariatePuisseuxSeriesCategory| . UPXSCAT)
(|UnivariatePuisseuxSeriesConstructorCategory| . UPXSCCA)
(|UnivariateSkewPolynomialCategory| . OREPCAT)
(|UnivariateTaylorSeriesCategory| . UTSCAT)
(|VectorCategory| . VECTCAT)
(|VectorSpace| . VSPACE)
(|XAlgebra| . XALG)
(|XFreeAlgebra| . XFALG)
(|XPolynomialsCat| . XPOLYC)
(|ZeroDimensionalSolvePackage| . ZDSOLVE))
(|Hidden|
  (|AlgebraicFunction| . AF)
  (|AlgebraicFunctionField| . ALGFF)
  (|AlgebraicHermiteIntegration| . INTHERAL)
  (|AlgebraicIntegrate| . INTALG)
  (|AlgebraicIntegration| . INTAF)
  (|AnonymousFunction| . ANON)
  (|AntiSymm| . ANTISYM)
  (|ApplyRules| . APPRULE)
  (|ApplyUnivariateSkewPolynomial| . APPLYORE)
  (|ArrayStack| . ASTACK)
  (|AssociatedEquations| . ASSOCEQ)
  (|AssociationList| . ALIST)
  (|Automorphism| . AUTOMOR)
  (|BalancedFactorisation| . BALFACT)
  (|BalancedPAdicInteger| . BPADIC)
  (|BalancedPAdicRational| . BPADICRT)
  (|BezoutMatrix| . BEZOUT)
  (|BoundIntegerRoots| . BOUNDZRO)
  (|BrillhartTests| . BRILL)

```

```

(|ChangeOfVariable| . CHVAR)
(|CharacteristicPolynomialInMonogenicalAlgebra| . CPIMA)
(|ChineseRemainderToolsForIntegralBases| . IBACHIN)
(|CoerceVectorMatrixPackage| . CVMP)
(|CombinatorialFunction| . COMBF)
(|CommonOperators| . COMMONOP)
(|CommuteUnivariatePolynomialCategory| . COMMUPC)
(|ComplexIntegerSolveLinearPolynomialEquation| . CINTSLPE)
(|ComplexPattern| . COMPLPAT)
(|ComplexPatternMatch| . CPMATCH)
(|ComplexRootFindingPackage| . CRFP)
(|ConstantLODE| . ODECONST)
(|CyclicStreamTools| . CSTTOOLS)
(|CyclotomicPolynomialPackage| . CYCLOTOM)
(|DefiniteIntegrationTools| . DFINTTLS)
(|DegreeReductionPackage| . DEGRED)
(|DeRhamComplex| . DERHAM)
(|DifferentialSparseMultivariatePolynomial| . DSMP)
(|DirectProduct| . DIRPROD)
(|DirectProductMatrixModule| . DPMM)
(|DirectProductModule| . DPMO)
(|DiscreteLogarithmPackage| . DLP)
(|DistributedMultivariatePolynomial| . DMP)
(|DoubleResultantPackage| . DBLRESP)
(|DrawOptionFunctions0| . DROPT0)
(|DrawOptionFunctions1| . DROPT1)
(|ElementaryFunction| . EF)
(|ElementaryFunctionsUnivariateLaurentSeries| . EFULS)
(|ElementaryFunctionsUnivariatePuisseuxSeries| . EFUPXS)
(|ElementaryIntegration| . INTEF)
(|ElementaryRischDE| . RDEEF)
(|ElementaryRischDESystem| . RDEEFS)
(|EllipticFunctionsUnivariateTaylorSeries| . ELFUTS)
(|EqTable| . EQTBL)
(|EuclideanModularRing| . EMR)
(|EvaluateCycleIndicators| . EVALCYC)
(|ExponentialExpansion| . EXPEXPAN)
(|ExponentialOfUnivariatePuisseuxSeries| . EXPUPXS)
(|ExpressionSpaceFunctions1| . ES1)
(|ExpressionTubePlot| . EXPRTUBE)
(|ExtAlgBasis| . EAB)
(|FactoredFunctions| . FACTFUNC)
(|FactoredFunctionUtilities| . FRUTIL)
(|FactoringUtilities| . FACUTIL)
(|FGLMIfCanPackage| . FGLMICPK)
(|FindOrderFinite| . FORDER)
(|FiniteDivisor| . FDIV)
(|FiniteFieldCyclicGroupExtension| . FFCGX)
(|FiniteFieldCyclicGroupExtensionByPolynomial| . FFCGP)
(|FiniteFieldExtension| . FFX)

```

```

(|FiniteFieldExtensionByPolynomial| . FFP)
(|FiniteFieldFunctions| . FFF)
(|FiniteFieldNormalBasisExtension| . FFNBX)
(|FiniteFieldNormalBasisExtensionByPolynomial| . FFNBP)
(|FiniteFieldPolynomialPackage| . FFPOLY)
(|FiniteFieldSolveLinearPolynomialEquation| . FFSLPE)
(|FormalFraction| . FORMAL)
(|FourierComponent| . FCOMP)
(|FractionalIdeal| . FRIDEAL)
(|FramedModule| . FRMOD)
(|FreeAbelianGroup| . FAGROUP)
(|FreeAbelianMonoid| . FAMONOID)
(|FreeGroup| . FGROUPE)
(|FreeModule| . FM)
(|FreeModule1| . FM1)
(|FreeMonoid| . FMONOID)
(|FunctionalSpecialFunction| . FSPECF)
(|FunctionCalled| . FUNCTION)
(|FunctionFieldIntegralBasis| . FFINTBAS)
(|FunctionSpaceReduce| . FSRED)
(|FunctionSpaceToUnivariatePowerSeries| . FS2UPS)
(|FunctionSpaceToExponentialExpansion| . FS2EXXP)
(|FunctionSpaceUnivariatePolynomialFactor| . FSUPFACT)
(|GaloisGroupFactorizationUtilities| . GALFACTU)
(|GaloisGroupFactorizer| . GALFACT)
(|GaloisGroupPolynomialUtilities| . GALPOLYU)
(|GaloisGroupUtilities| . GALUTIL)
(|GeneralHenselPackage| . GHENSEL)
(|GeneralDistributedMultivariatePolynomial| . GDMP)
(|GeneralPolynomialGcdPackage| . GENPGCD)
(|GeneralSparseTable| . GSTBL)
(|GenericNonAssociativeAlgebra| . GCNAALG)
(|GenExEuclid| . GENEEZ)
(|GeneralizedMultivariateFactorize| . GENMFACT)
(|GeneralModulePolynomial| . GMDPOL)
(|GeneralPolynomialSet| . GPOLSET)
(|GeneralTriangularSet| . GTSET)
(|GenUFactorize| . GENUFACT)
(|GenusZeroIntegration| . INTGO)
(|GosperSummationMethod| . GOSPER)
(|GraphImage| . GRIMAGE)
(|GrayCode| . GRAY)
(|GroebnerInternalPackage| . GBINTERN)
(|GroebnerSolve| . GROEBSOL)
(|GuessOptionFunctions0| . GOPT0)
(|HashTable| . HASHTBL)
(|Heap| . HEAP)
(|HeuGcd| . HEUGCD)
(|HomogeneousDistributedMultivariatePolynomial| . HDMP)
(|HyperellipticFiniteDivisor| . HELLFDIV)

```



```

(|IncrementingMaps| . INCRMAPS)
(|IndexedBits| . IBITS)
(|IndexedDirectProductAbelianGroup| . IDPAG)
(|IndexedDirectProductAbelianMonoid| . IDPAM)
(|IndexedDirectProductObject| . IDPO)
(|IndexedDirectProductOrderedAbelianMonoid| . IDPOAM)
(|IndexedDirectProductOrderedAbelianMonoidSup| . IDPOAMS)
(|IndexedExponents| . INDE)
(|IndexedFlexibleArray| . IFARRAY)
(|IndexedList| . ILIST)
(|IndexedMatrix| . IMATRIX)
(|IndexedOneDimensionalArray| . IARRAY1)
(|IndexedString| . ISTRING)
(|IndexedTwoDimensionalArray| . IARRAY2)
(|IndexedVector| . IVECTOR)
(|InnerAlgFactor| . IALGFACT)
(|InnerAlgebraicNumber| . IAN)
(|InnerCommonDenominator| . ICDEN)
(|InnerFiniteField| . IFF)
(|InnerFreeAbelianMonoid| . IFAMON)
(|InnerIndexedTwoDimensionalArray| . IIARRAY2)
(|InnerMatrixLinearAlgebraFunctions| . IMATLIN)
(|InnerMatrixQuotientFieldFunctions| . IMATQF)
(|InnerModularGcd| . INMODGCD)
(|InnerMultFact| . INMFACT)
(|InnerNormalBasisFieldFunctions| . INBFF)
(|InnerNumericEigenPackage| . INEP)
(|InnerNumericFloatSolvePackage| . INFSP)
(|InnerPAdicInteger| . IPADIC)
(|InnerPolySign| . INPSIGN)
(|InnerPolySum| . ISUMP)
(|InnerPrimeField| . IPF)
(|InnerSparseUnivariatePowerSeries| . ISUPS)
(|InnerTable| . INTABL)
(|InnerTaylorSeries| . ITAYLOR)
(|InnerTrigonometricManipulations| . ITRIGMNP)
(|InputForm| . INFORM)
(|InputFormFunctions1| . INFORM1)
(|IntegerBits| . INTBIT)
(|IntegerFactorizationPackage| . INTFACT)
(|IntegerMod| . ZMOD)
(|IntegerSolveLinearPolynomialEquation| . INTSLPE)
(|IntegralBasisPolynomialTools| . IBPTOOLS)
(|IntegralBasisTools| . IBATool)
(|IntegrationResult| . IR)
(|IntegrationTools| . INTTOOLS)
(|InternalPrintPackage| . IPRNTPK)
(|InternalRationalUnivariateRepresentationPackage| . IRURPK)
(|IrredPolyOverFiniteField| . IRREDFFX)
(|Kernel| . KERNEL)

```

```

(|Kovacic| . KOVACIC)
(|LaurentPolynomial| . LAUPOL)
(|LeadingCoefDetermination| . LEADCDET)
(|LexTriangularPackage| . LEXTRIPK)
(|LieExponentials| . LEXP)
(|LiePolynomial| . LPOLY)
(|LinearDependence| . LINDEP)
(|LinearOrdinaryDifferentialOperatorFactorizer| . LODOF)
(|LinearOrdinaryDifferentialOperator1| . LOD01)
(|LinearOrdinaryDifferentialOperator2| . LOD02)
(|LinearOrdinaryDifferentialOperatorsOps| . LODOOPS)
(|LinearPolynomialEquationByFractions| . LPEFRAC)
(|LinGroebnerPackage| . LGROBP)
(|LiouvillianFunction| . LF)
(|ListMonoidOps| . LMOPS)
(|ListMultiDictionary| . LMDICT)
(|LocalAlgebra| . LA)
(|Localize| . LO)
(|LyndonWord| . LWORD)
(|Magma| . MAGMA)
(|MakeBinaryCompiledFunction| . MKBCFUNC)
(|MakeCachableSet| . MKCHSET)
(|MakeUnaryCompiledFunction| . MKUCFUNC)
(|MappingPackageInternalHacks1| . MAPHACK1)
(|MappingPackageInternalHacks2| . MAPHACK2)
(|MappingPackageInternalHacks3| . MAPHACK3)
(|MeshCreationRoutinesForThreeDimensions| . MESH)
(|ModMonic| . MODMON)
(|ModularField| . MODFIELD)
(|ModularHermitianRowReduction| . MHROWRED)
(|ModularRing| . MODRING)
(|ModuleMonomial| . MODMONOM)
(|MoebiusTransform| . MOEBIUS)
(|MonoidRing| . MRING)
(|MonomialExtensionTools| . MONOTOOL)
(|MPolyCatPolyFactorizer| . MPCPF)
(|MPolyCatFunctions3| . MPC3)
(|MRationalFactorize| . MRATFAC)
(|MultipleMap| . MMAP)
(|MultivariateLifting| . MLIFT)
(|MultivariateSquareFree| . MULTSQFR)
(|HomogeneousDirectProduct| . HDP)
(|NewSparseMultivariatePolynomial| . NSMP)
(|NewSparseUnivariatePolynomial| . NSUP)
(|NewSparseUnivariatePolynomialFunctions2| . NSUP2)
(|NonCommutativeOperatorDivision| . NCODIV)
(|NewtonInterpolation| . NEWTON)
(|None| . NONE)
(|NonLinearFirstOrderODESolver| . NODE1)
(|NonLinearSolvePackage| . NLINSOL)

```

```

(|NormRetractPackage| . NORMRETR)
(|NPCoef| . NPCOEF)
(|NumberFormats| . NUMFMT)
(|NumberFieldIntegralBasis| . NFINTBAS)
(|NumericTubePlot| . NUMTUBE)
(|ODEIntegration| . ODEINT)
(|ODETools| . ODETOOLS)
(|Operator| . OP)
(|OppositeMonogenicLinearOperator| . OML0)
(|OrderedDirectProduct| . ODP)
(|OrderedFreeMonoid| . OFMONOID)
(|OrderedVariableList| . OVAR)
(|OrderingFunctions| . ORDFUNS)
(|OrderlyDifferentialPolynomial| . ODPOL)
(|OrderlyDifferentialVariable| . ODVAR)
(|OrdinaryWeightedPolynomials| . OWP)
(|OutputForm| . OUTFORM)
(|PadeApproximants| . PADE)
(|PAdicInteger| . PADIC)
(|PAdicRational| . PADICRAT)
(|PAdicRationalConstructor| . PADICRC)
(|PAdicWildFunctionFieldIntegralBasis| . PWFFINTB)
(|ParadoxicalCombinatorsForStreams| . YSTREAM)
(|ParametricLinearEquations| . PLEQN)
(|PartialFractionPackage| . PFRPAC)
(|Partition| . PRITITION)
(|Pattern| . PATTERN)
(|PatternFunctions1| . PATTERN1)
(|PatternMatchFunctionSpace| . PMFS)
(|PatternMatchIntegerNumberSystem| . PMINS)
(|PatternMatchIntegration| . INTPM)
(|PatternMatchKernel| . PMKERNEL)
(|PatternMatchListAggregate| . PMLSAGG)
(|PatternMatchListResult| . PATLRES)
(|PatternMatchPolynomialCategory| . PMPLCAT)
(|PatternMatchPushDown| . PMDOWN)
(|PatternMatchQuotientFieldCategory| . PMQFCAT)
(|PatternMatchResult| . PATRES)
(|PatternMatchSymbol| . PMSYM)
(|PatternMatchTools| . PMTOOLS)
(|PlaneAlgebraicCurvePlot| . ACPL0T)
(|Plot| . PLOT)
(|PlotFunctions1| . PLOT1)
(|PlotTools| . PLOTTOOL)
(|Plot3D| . PLOT3D)
(|PoincareBirkhoffWittLyndonBasis| . PBWLB)
(|Point| . POINT)
(|PointsOfFiniteOrder| . PFO)
(|PointsOfFiniteOrderRational| . PFOQ)
(|PointsOfFiniteOrderTools| . PFOTOOLS)

```

```

(|PointPackage| . PTPACK)
(|PolToPol| . POLTOPOL)
(|PolynomialCategoryLifting| . POLYLIFT)
(|PolynomialCategoryQuotientFunctions| . POLYCATQ)
(|PolynomialFactorizationByRecursion| . PFBR)
(|PolynomialFactorizationByRecursionUnivariate| . PFBRU)
(|PolynomialGcdPackage| . PGCD)
(|PolynomialInterpolation| . PINTERP)
(|PolynomialInterpolationAlgorithms| . PINTERPA)
(|PolynomialNumberTheoryFunctions| . PNTHEORY)
(|PolynomialRing| . PR)
(|PolynomialRoots| . POLYROOT)
(|PolynomialSetUtilitiesPackage| . PSETPK)
(|PolynomialSolveByFormulas| . SOLVEFOR)
(|PolynomialSquareFree| . PSQFR)
(|PrecomputedAssociatedEquations| . PREASSOC)
(|PrimitiveArray| . PRIMARR)
(|PrimitiveElement| . PRIMELT)
(|PrimitiveRatDE| . ODEPRIM)
(|PrimitiveRatRicDE| . ODEPRRIC)
(|Product| . PRODUCT)
(|PseudoRemainderSequence| . PRS)
(|PseudoLinearNormalForm| . PSEUDLIN)
(|PureAlgebraicIntegration| . INTPAF)
(|PureAlgebraicLODE| . ODEPAL)
(|PushVariables| . PUSHVAR)
(|QuasiAlgebraicSet| . QALGSET)
(|QuasiAlgebraicSet2| . QALGSET2)
(|RadicalFunctionField| . RADFF)
(|RandomDistributions| . RDIST)
(|RandomFloatDistributions| . RFDIST)
(|RandomIntegerDistributions| . RIDIST)
(|RationalFactorize| . RATFACT)
(|RationalIntegration| . INTRAT)
(|RationalInterpolation| . RINTERP)
(|RationalLODE| . ODERAT)
(|RationalRicDE| . ODERTRIC)
(|RationalUnivariateRepresentationPackage| . RURPK)
(|RealSolvePackage| . REALSOLV)
(|RectangularMatrix| . RMATRIX)
(|ReducedDivisor| . RDIV)
(|ReduceLODE| . ODERED)
(|ReductionOfOrder| . REDORDER)
(|Reference| . REF)
(|RepeatedDoubling| . REPDB)
(|RepeatedSquaring| . REPSQ)
(|ResidueRing| . RESRING)
(|RetractSolvePackage| . RETSOL)
(|RuleCalled| . RULECOLD)
(|SetOfMIntegersInOneToN| . SETMN)

```

```

(|SExpression| . SEX)
(|SExpressionOf| . SEXOF)
(|SequentialDifferentialPolynomial| . SDPOL)
(|SequentialDifferentialVariable| . SDVAR)
(|SimpleAlgebraicExtension| . SAE)
(|SingletonAsOrderedSet| . SAOS)
(|SortedCache| . SCACHE)
(|SortPackage| . SORTPAK)
(|SparseMultivariatePolynomial| . SMP)
(|SparseMultivariateTaylorSeries| . SMTS)
(|SparseTable| . STBL)
(|SparseUnivariatePolynomial| . SUP)
(|SparseUnivariateSkewPolynomial| . ORESUP)
(|SparseUnivariateLaurentSeries| . SULS)
(|SparseUnivariatePuisseuxSeries| . SUPXS)
(|SparseUnivariateTaylorSeries| . SUTS)
(|SplitHomogeneousDirectProduct| . SHDP)
(|SplittingNode| . SPLNODE)
(|SplittingTree| . SPLTREE)
(|SquareMatrix| . SQMATRIX)
(|Stack| . STACK)
(|StorageEfficientMatrixOperations| . MATSTOR)
(|StreamInfiniteProduct| . STINPROD)
(|StreamTaylorSeriesOperations| . STTAYLOR)
(|StreamTranscendentalFunctions| . STTF)
(|StreamTranscendentalFunctionsNonCommutative| . STTFNC)
(|StringTable| . STRTBL)
(|SubResultantPackage| . SUBRESP)
(|SubSpace| . SUBSPACE)
(|SubSpaceComponentProperty| . COMPPROP)
(|SuchThat| . SUCH)
(|SupFractionFactorizer| . SUPFRACF)
(|SymmetricFunctions| . SYMFUNC)
(|SymmetricPolynomial| . SYMPOLY)
(|SystemODESolver| . ODESYS)
(|Table| . TABLE)
(|TableauxBumpers| . TABLBUMP)
(|TabulatedComputationPackage| . TBCMPPK)
(|TangentExpansions| . TANEXP)
(|ToolsForSign| . TOOLSIGN)
(|TranscendentalHermiteIntegration| . INTHERTR)
(|TranscendentalIntegration| . INTTR)
(|TranscendentalRischDE| . RDETR)
(|TranscendentalRischDESystem| . RDETRS)
(|TransSolvePackageService| . SOLVESER)
(|TriangularMatrixOperations| . TRIMAT)
(|TubePlot| . TUBE)
(|TubePlotTools| . TUBETOOL)
(|Tuple| . TUPLE)
(|TwoDimensionalArray| . ARRAY2)

```

```

(|TwoDimensionalPlotClipping| . CLIP)
(|TwoDimensionalViewport| . VIEW2D)
(|TwoFactorize| . TWOFACT)
(|UnivariateFactorize| . UNIFACT)
(|UnivariateLaurentSeries| . ULS)
(|UnivariateLaurentSeriesConstructor| . ULSCONS)
(|UnivariatePolynomialDecompositionPackage| . UPDECOMP)
(|UnivariatePolynomialDivisionPackage| . UPDIVP)
(|UnivariatePolynomialSquareFree| . UPSQFREE)
(|UnivariatePuisseuxSeries| . UPXS)
(|UnivariatePuisseuxSeriesConstructor| . UPXSCONS)
(|UnivariatePuisseuxSeriesWithExponentialSingularity| . UPXSING)
(|UnivariateSkewPolynomial| . OREUP)
(|UnivariateSkewPolynomialCategoryOps| . OREPCTO)
(|UnivariateTaylorSeries| . UTS)
(|UnivariateTaylorSeriesODESolver| . UTSODE)
(|UserDefinedPartialOrdering| . UDPO)
(|UTSodeTools| . UTSODETL)
(|Variable| . VARIABLE)
(|ViewportPackage| . VIEW)
(|WeierstrassPreparation| . WEIER)
(|WeightedPolynomials| . WP)
(|WildFunctionFieldIntegralBasis| . WFFINTBS)
(|XDistributedPolynomial| . XDPOLY)
(|XExponentialPackage| . XEXPPKG)
(|XPBWPolynomial| . XPBWPOLY)
(|XPolynomial| . XPOLY)
(|XPolynomialRing| . XPR)
(|XRecursivePolynomial| . XRPOLY))
(|defaults|
  (|AbelianGroup&| . ABELGRP-)
  (|AbelianMonoid&| . ABELMON-)
  (|AbelianMonoidRing&| . AMR-)
  (|AbelianSemiGroup&| . ABELSG-)
  (|Aggregate&| . AGG-)
  (|Algebra&| . ALGEBRA-)
  (|AlgebraicallyClosedField&| . ACF-)
  (|AlgebraicallyClosedFunctionSpace&| . ACFS-)
  (|ArcTrigonometricFunctionCategory&| . ATRIG-)
  (|BagAggregate&| . BGAGG-)
  (|BasicType&| . BASTYPE-)
  (|BinaryRecursiveAggregate&| . BRAGG-)
  (|BinaryTreeCategory&| . BTCAT-)
  (|BitAggregate&| . BTAGG-)
  (|Collection&| . CLAGG-)
  (|ComplexCategory&| . COMPCAT-)
  (|Dictionary&| . DIAGG-)
  (|DictionaryOperations&| . DIOPS-)
  (|DifferentialExtension&| . DIFEXT-)
  (|DifferentialPolynomialCategory&| . DPOLCAT-)

```

```

(|DifferentialRing&| . DIFRING-)
(|DifferentialVariableCategory&| . DVARCAT-)
(|DirectProductCategory&| . DIRPCAT-)
(|DivisionRing&| . DIVRING-)
(|ElementaryFunctionCategory&| . ELEMFUN-)
(|EltableAggregate&| . ELTAGG-)
(|EuclideanDomain&| . EUCDOM-)
(|Evalable&| . EVALAB-)
(|ExpressionSpace&| . ES-)
(|ExtensibleLinearAggregate&| . ELAGG-)
(|ExtensionField&| . XF-)
(|Field&| . FIELD-)
(|FieldOfPrimeCharacteristic&| . FPC-)
(|FiniteAbelianMonoidRing&| . FAMR-)
(|FiniteAlgebraicExtensionField&| . FAXF-)
(|FiniteDivisorCategory&| . FDIVCAT-)
(|FiniteFieldCategory&| . FFIELDC-)
(|FiniteLinearAggregate&| . FLAGG-)
(|FiniteSetAggregate&| . FSAGG-)
(|FiniteRankAlgebra&| . FINRALG-)
(|FiniteRankNonAssociativeAlgebra&| . FINAALG-)
(|FloatingPointSystem&| . FPS-)
(|FramedAlgebra&| . FRAMALG-)
(|FramedNonAssociativeAlgebra&| . FRNAALG-)
(|FullyEvalableOver&| . FEVALAB-)
(|FullyLinearlyExplicitRingOver&| . FLINEXP-)
(|FullyRetractableTo&| . FRETRCT-)
(|FunctionFieldCategory&| . FFCAT-)
(|FunctionSpace&| . FS-)
(|GcdDomain&| . GCDDOM-)
(|GradedAlgebra&| . GRALG-)
(|GradedModule&| . GRMOD-)
(|Group&| . GROUP-)
(|HomogeneousAggregate&| . HOAGG-)
(|HyperbolicFunctionCategory&| . HYPCAT-)
(|IndexedAggregate&| . IXAGG-)
(|InnerEvalable&| . IEVALAB-)
(|IntegerNumberSystem&| . INS-)
(|IntegralDomain&| . INTDOM-)
(|KeyedDictionary&| . KDAGG-)
(|LazyStreamAggregate&| . LZSTAGG-)
(|LeftAlgebra&| . LALG-)
(|LieAlgebra&| . LIECAT-)
(|LinearAggregate&| . LNAGG-)
(|ListAggregate&| . LSAGG-)
(|Logic&| . LOGIC-)
(|LinearOrdinaryDifferentialOperatorCategory&| . LODOCAT-)
(|MatrixCategory&| . MATCAT-)
(|Module&| . MODULE-)
(|Monad&| . MONAD-)

```

```

(|MonadWithUnit&| . MONADWU-)
(|Monoid&| . MONOID-)
(|MonogenicAlgebra&| . MONOGEN-)
(|NonAssociativeAlgebra&| . NAALG-)
(|NonAssociativeRing&| . NASRING-)
(|NonAssociativeRng&| . NARNG-)
(|OctonionCategory&| . OC-)
(|OneDimensionalArrayAggregate&| . A1AGG-)
(|OrderedRing&| . ORDRING-)
(|OrderedSet&| . ORDSET-)
(|PartialDifferentialRing&| . PDRING-)
(|PolynomialCategory&| . POLYCAT-)
(|PolynomialFactorizationExplicit&| . PFECAT-)
(|PolynomialSetCategory&| . PSETCAT-)
(|PowerSeriesCategory&| . PSCAT-)
(|QuaternionCategory&| . QUATCAT-)
(|QuotientFieldCategory&| . QFCAT-)
(|RadicalCategory&| . RADCAT-)
(|RealClosedField&| . RCFIELD-)
(|RealNumberSystem&| . RNS-)
(|RealRootCharacterizationCategory&| . RRCC-)
(|RectangularMatrixCategory&| . RMATCAT-)
(|RecursiveAggregate&| . RCAGG-)
(|RecursivePolynomialCategory&| . RPOLCAT-)
(|RegularTriangularSetCategory&| . RSETCAT-)
(|RetractableTo&| . RETRACT-)
(|Ring&| . RING-)
(|SemiGroup&| . SGROUP-)
(|SetAggregate&| . SETAGG-)
(|SetCategory&| . SETCAT-)
(|SquareMatrixCategory&| . SMATCAT-)
(|StreamAggregate&| . STAGG-)
(|StringAggregate&| . SRAGG-)
(|TableAggregate&| . TBAGG-)
(|TranscendentalFunctionCategory&| . TRANFUN-)
(|TriangularSetCategory&| . TSETCAT-)
(|TrigonometricFunctionCategory&| . TRIGCAT-)
(|TwoDimensionalArrayCategory&| . ARR2CAT-)
(|UnaryRecursiveAggregate&| . URAGG-)
(|UniqueFactorizationDomain&| . UFD-)
(|UnivariateLaurentSeriesConstructorCategory&| . ULSCCAT-)
(|UnivariatePolynomialCategory&| . UPOLYC-)
(|UnivariatePowerSeriesCategory&| . UPSCAT-)
(|UnivariatePuisseuxSeriesConstructorCategory&| . UPXSCCA-)
(|UnivariateSkewPolynomialCategory&| . OREPCAT-)
(|UnivariateTaylorSeriesCategory&| . UTSCAT-)
(|VectorCategory&| . VECTCAT-)
(|VectorSpace&| . VSPACE-)))

```


Chapter 9

The global variables

Credits

Axiom has a very long history and many people have contributed to the effort, some in large ways and some in small ways. Any and all effort deserves recognition. There is no other criteria than contribution of effort. We would like to acknowledge and thank the following people:

defvar creditlist

— initvars —

```
(defvar creditlist '(  
  "An alphabetical listing of contributors to AXIOM:"  
  "Michael Albaugh      Cyril Alberga      Roy Adler"  
  "Christian Aistleitner Richard Anderson  George Andrews"  
  "S.J. Atkins          Jeremy Avigad    Henry Baker"  
  "Martin Baker         Stephen Balzac  Yurij Baransky"  
  "David R. Barton      Thomas Baruchel Gerald Baumgartner"  
  "Gilbert Baumslag     Michael Becker Nelson H. F. Beebe"  
  "Jay Belanger         David Bindel   Fred Blair"  
  "Vladimir Bondarenko Mark Botch     Raoul Bourquin"  
  "Alexandre Bouyer     Karen Braman   Wolfgang Brehm"  
  "Peter A. Broadbery   Martin Brock   Manuel Bronstein"  
  "Stephen Buchwald     Florian Bundschuh Luanne Burns"  
  "William Burge        Ralph Byers    Quentin Carpent"  
  "Robert Cavines       Bruce Char     Ondrej Certik"  
  "Tzu-Yi Chen          Bobby Cheng     Cheekai Chin"  
  "David V. Chudnovsky  Gregory V. Chudnovsky Mark Clements"  
  "James Cloos          Jia Zhao Cong  Josh Cohen"  
  "Christophe Conil     Don Coppersmith George Corliss"
```

"Robert Corless	Gary Cornell	Meino Cramer"
"Jeremy Du Croz	David Cyganski	Nathaniel Daly"
"Timothy Daly Sr.	Timothy Daly Jr.	James H. Davenport"
"David Day	James Demmel	Didier Deshommes"
"Michael Dewar	Inderjit Dhillon	Jack Dongarra"
"Jean Della Dora	Gabriel Dos Reis	Claire DiCrescendo"
"Sam Dooley	Zlatko Drmac	Lionel Ducos"
"Iain Duff	Lee Duhem	Martin Dunstan"
"Brian Dupee	Dominique Duval	Robert Edwards"
"Heow Eide-Goodman	Lars Erickson	Mark Fahey"
"Richard Fateman	Bertfried Fauser	Stuart Feldman"
"John Fletcher	Brian Ford	Albrecht Fortenbacher"
"George Frances	Constantine Frangos	Timothy Freeman"
"Korriinn Fu	Marc Gaetano	Rudiger Gebauer"
"Van de Geijn	Kathy Gerber	Patricia Gianni"
"Gustavo Goertkin	Samantha Goldrich	Holger Gollan"
"Teresa Gomez-Diaz	Laureano Gonzalez-Vega	Stephen Gortler"
"Johannes Grabmeier	Matt Grayson	Klaus Ebbe Grue"
"James Griesmer	Vladimir Grinberg	Oswald Gschnitzer"
"Ming Gu	Jocelyn Guidry	Gaetan Hache"
"Steve Hague	Satoshi Hamaguchi	Sven Hammarling"
"Mike Hansen	Richard Hanson	Richard Harke"
"Bill Hart	Vilya Harvey	Martin Hassner"
"Arthur S. Hathaway	Dan Hatton	Waldek Hebisch"
"Karl Hegbloom	Ralf Hemmecke	Henderson"
"Antoine Hersen	Nicholas J. Higham	Roger House"
"Gernot Hueber	Pietro Iglio	Alejandro Jakubi"
"Richard Jenks	Bo Kagstrom	William Kahan"
"Kyriakos Kalorkoti	Kai Kaminski	Grant Keady"
"Wilfrid Kendall	Tony Kennedy	David Kincaid"
"Ted Kosan	Paul Kosinski	Igor Kozachenko"
"Fred Krogh	Klaus Kusche	Bernhard Kutzler"
"Tim Lahey	Larry Lambe	Kaj Laurson"
"Charles Lawson	George L. Legendre	Franz Lehner"
"Frederic Lehouey	Michel Levaud	Howard Levy"
"J. Lewis	Ren-Cang Li	Rudiger Loos"
"Craig Lucas	Michael Lucks	Richard Luczak"
"Camm Maguire	Francois Maltey	Osni Marques"
"Alasdair McAndrew	Bob McElrath	Michael McGettrick"
"Edi Meier	Ian Meikle	David Mentre"
"Victor S. Miller	Gerard Milmeister	Mohammed Mobarak"
"H. Michael Moeller	Michael Monagan	Marc Moreno-Maza"
"Scott Morrison	Joel Moses	Mark Murray"
"William Naylor	Patrice Naudin	C. Andrew Neff"
"John Nelder	Godfrey Nolan	Arthur Norman"
"Jinzhong Niu	Michael O'Connor	Summat Oemrawsingh"
"Kostas Oikonomou	Humberto Ortiz-Zuazaga	Julian A. Padget"
"Bill Page	David Parnas	Susan Pelzel"
"Michel Petitot	Didier Pinchon	Ayal Pinkus"
"Frederick H. Pitts	Jose Alfredo Portes	E. Quintana-Orti"

"Gregorio Quintana-Orti	Beresford Parlett	A. Petitet"
"Peter Poromaa	Claude Quitte	Arthur C. Ralfs"
"Norman Ramsey	Anatoly Raportirenko	Guilherme Reis"
"Huan Ren	Albert D. Rich	Michael Richardson"
"Jason Riedy	Renaud Rioboo	Jean Rivlin"
"Nicolas Robidoux	Simon Robinson	Raymond Rogers"
"Michael Rothstein	Martin Rubey	Jeff Rutter"
"Philip Santas	Alfred Scheerhorn	William Schelter"
"Gerhard Schneider	Martin Schoenert	Marshall Schor"
"Frithjof Schulze	Fritz Schwarz	Steven Segletes"
"V. Sima	Nick Simicich	William Sit"
"Elena Smirnova	Jacob Nyffeler Smith	Ken Stanley"
"Jonathan Steinbach	Fabio Stumbo	Christine Sundaresan"
"Robert Sutor	Moss E. Sweedler	Eugene Surowitz"
"Max Tegmark	T. Doug Telford	James Thatcher"
"Laurent Thery	Balbir Thomas	Mike Thomas"
"Dylan Thurston	Francoise Tisseur	Steve Toleque"
"Raymond Toy	Barry Trager	Themos T. Tsikas"
"Gregory Vanuxem	Kresimir Veselic	Christof Voemel"
"Bernhard Wall	Stephen Watt	Jaap Weel"
"Juergen Weiss	M. Weller	Mark Wegman"
"James Wen	Thorsten Werther	Michael Wester"
"R. Clint Whaley	James T. Wheeler	John M. Wiley"
"Berhard Will	Clifton J. Williamson	Stephen Wilson"
"Shmuel Winograd	Robert Wisbauer	Sandra Wityak"
"Waldemar Wiwianka	Knut Wolf	Yanyang Xiao"
"Liu Xiaojun	Clifford Yapp	David Yun"
"Qian Yun	Vadim Zhytnikov	Richard Zippel"
"Evelyn Zoernack	Bruno Zuercher	Dan Zwillingen"

))

The `$current-directory` variable is set to the current directory at startup. This is used by the `)cd` function and some of the compile routines. This is the result of the (p328) `get-current-directory` function. This variable is used to set `*default-pathname-defaults*`. The (p332) `reroot` function resets it to `$spadroot`.

An example of a runtime value is:

```
$current-directory = "/research/test/"
```

defvar \$current-directory

— initvars —

```
(defvar $current-directory nil)
```

The `$directory-list` is a runtime list of absolute pathnames. This list is generated by (p332) `reroot` from the list of relative paths held in the variable `$relative-directory-list`. Each entry will be prefixed by `$spadroot`.

An example of a runtime value is:

```
$directory-list =
  ("/research/test/mnt/ubuntu/../../src/input/"
   "/research/test/mnt/ubuntu/doc/messages/"
   "/research/test/mnt/ubuntu/../../src/algebra/"
   "/research/test/mnt/ubuntu/../../src/interp/"
   "/research/test/mnt/ubuntu/doc/spadhelp/")
```

defvar \$directory-list

— initvars —

```
(defvar $directory-list nil)
```

The `$InitialModemapFrame` is used as the initial value.

See the function “makeInitialModemapFrame” (12.3 p 328).

An example of a runtime value is:

```
$InitialModemapFrame = '((nil))
```

defvar \$InitialModemapFrame

— initvars —

```
(defvar |$InitialModemapFrame| '((nil)))
```

The `$library-directory-list` variable is the system-wide search path for library files. (p332) `reroot` prepends the `$spadroot` variable to the `$relative-library-directory-list` variable.

An example of a runtime value is:

```
$library-directory-list = ("/research/test/mnt/ubuntu/algebra/")
```

defvar \$library-directory-list

— initvars —

```
(defvar $library-directory-list '("/algebra/"))
```

The `$msgDatabaseName` is a locally shared variable among the message database routines.

An example of a runtime value is:

```
|$msgDatabaseName| = nil
```

defvar \$msgDatabaseName

— initvars —

```
(defvar |$msgDatabaseName| nil)
```

The `$openServerIfTrue` It appears to control whether the interpreter will be used as an open server, probably for OpenMath use.

If an open server is not requested then this variable to NIL

See the function “openserver” ([32 p 1173](#)).

An example of a runtime value is:

```
$openServerIfTrue = nil
```

defvar \$openServerIfTrue

— initvars —

```
(defvar $openServerIfTrue nil)
```

The `$relative-directory-list` variable contains a hand-generated list of directories used in the Axiom system. The relative directory list specifies a search path for files for the current

directory structure. It has been changed from the NAG distribution back to the original form.

This list is used by the (p332) `reroot` function to generate the absolute list of paths held in the variable `$directory-list`. Each entry will be prefixed by `$spadroot`.

An example of a runtime value is:

```
$relative-directory-list =
  ("../../../../src/input/"
   "/doc/msgs/"
   "../../../../src/algebra/"
   "../../../../src/interp/"
   "/doc/spadhelp/")
```

defvar \$relative-directory-list

— initvars —

```
(defvar $relative-directory-list
  '("../../../../src/input/"
    "/doc/msgs/"
    "../../../../src/algebra/"
    "../../../../src/interp/" ; for lisp files (helps fd)
    "/doc/spadhelp/" ))
```

—————

The `$relative-library-directory-list` is a hand-generated list of directories containing algebra. The (p332) `reroot` function will prefix every path in this list with the value of the `$spadroot` variable to construct the `$library-directory-list` variable.

An example of a runtime value is:

```
$relative-library-directory-list = ("/algebra/")
```

defvar \$relative-library-directory-list

— initvars —

```
(defvar $relative-library-directory-list '("/algebra/"))
```

—————

The `$spadroot` variable is the internal name for the AXIOM shell variable. It is set in `reroot` to the value of the argument. The value is expected to be a directory name. The (p327) `initroot` function uses this variable if the AXIOM shell variable is not set. The (p328) `make-absolute-filename` function uses this path as a prefix to all of the relative filenames to make them absolute.

An example of a runtime value is:

```
$spadroot = "/research/test/mnt/ubuntu"
```

defvar \$spadroot

— initvars —

```
(defvar $spadroot nil)
```

—————

The `$SpadServer` determines whether Axiom acts as a remote server.

See the function “`openserver`” (32 p 1173).

An example of a runtime value is:

```
$SpadServer = nil
```

defvar \$SpadServer

— initvars —

```
(defvar $SpadServer nil "t means Axiom acts as a remote server")
```

—————

The `$SpadServerName` defines the name of the spad server socket. In unix these exist in the `tmp` directory as names.

See the function “`openserver`” (32 p 1173).

An example of a runtime value is:

```
$SpadServerName = "/tmp/.d"
```


defvar \$SpadServerName

— initvars —

```
(defvar $SpadServerName "/tmp/.d" "the name of the spad server socket")
```

—————

Chapter 10

Starting Axiom

This chapter details the internal processing behind an Axiom console session where the user types “1” and gets a result.

```
axiom -nox
                                AXIOM Computer Algebra System
                                Version: Axiom (August 2014)
                                Timestamp: Friday September 12, 2014 at 06:24:14
-----
Issue )copyright to view copyright notices.
Issue )summary for a summary of useful system commands.
Issue )quit to leave AXIOM and return to shell.
Visit http://axiom-developer.org for more information
-----

Re-reading interp.daase
Re-reading operation.daase
Re-reading category.daase
Re-reading browse.daase
(1) ->
(1) -> 1

      (1)  1
                                         Type: PositiveInteger
(2) ->
```

By working through this example we introduce, motivate, and explain how the interpreter works, where and why functions are called, how the system transitions from input strings to algebra, how the databases are used, and more.

If you plan to maintain or modify the interpreter this information is necessary. If you really want to know how Axiom works, this information is useful.

Each function call we describe has a link to the actual function so you can read the detailed code and see why it reacts as it does to the given input.

I've taken the liberty of adding comments that show the function signature. Some of the types only exist as unnamed data structures in the interpreter (e.g. "Server", which is really just a small integer). They are introduced without definition simply as a documentation aid but may sometimes be defined a Common Lisp deotypes for performance reasons.

A Note on Common Lisp Circular Notation

You may not be familiar with circular notation in Common Lisp. If a list contains a pointer back to itself or a sublist then the output would be an infinite stream. In order to prevent this the circular notation is used. So for a list X,

```

+---|---+      +---|---+      +---|---+      +---|---+
+ A |      + --> + B |      + --> + C |      + --> + D | / +
+---|---+      +---|---+      +---|---+      +---|---+

```

which is the list (A . (B . (C . (D . ())))). The printing rule says that if a period is followed by a parenthesis then both are suppressed. So this would print as (A B C D). But it could be that we execute

```
(rplaca (last X) (cdr X))
```

so the list now is

```

+---|---+      +---|---+      +---|---+      +---|---+
+ A |      + --> + B |      + --> + C |      + --> +   | / +
+---|---+      +---|---+      +---|---+      +---|---+
                        ^                               |
                        +-----+

```

and now the list X is ciclar. This prints as

```
(A . #0=(B C #0#))
```

As you can see the #0= introduces a unique label for the cons cell pointed at by (CDR A). We stored that address in the CAR of the last node. So the last node in the list uses the previously defined label with the notation #0#.

Circular notation is used extensively in Axiom since a lot of the structures are shared or self-referential. You have to be careful because, as a result of structure sharing, changing something in one place can change an apparently unrelated structure by side-effect.

Axiom starts by invoking a function value of the lisp symbol `*top-level-hook*` which is normally unbound. The normal function invocation path is:

```
axiom -nox
```

```
lisp
```

```

-> restart
-> |spad|
-> |runspad|
-> |ncTopLevel|
-> |ncIntLoop|
-> |intloop|
-> |SpadInterpretStream|
->   mkprompt           -- outputs "(1) ->" to the console
->   |intloopReadConsole| -- the Read-Eval-Print loop function
->   |serverReadLine|    -- does the actual read to the console
-> process the input and recursively call |intloopReadConsole|

```

SpadInterpretStream(p320) is called with a third arguments, **interactive?** set to **t** so it sets up an interactive loop to read from the console. The other two arguments are ignored on the main interpreter path.

SpadInterpretStream(p320) can also be called by the compiler, with the **interactive?** argument **nil** to read from a file. See bookvol9.

mkprompt(p334) puts one of several kinds of prompts on the screen. In the default case we include the step number. The return value is not used.

The **intloopReadConsole**(p321) function does tail-recursive calls to itself and never exits. It is the primary Read-Eval-Print-Loop (REPL).

intloopReadConsole(p321) reads the next line and calls one of three kinds of processors

1. **intnplisp**(p328) to handle `)lisp` input
2. **ncloopCommand**(p818) to handle `)command` input
3. **intloopProcessString**(p329) to handle everything else

There are only two ways out of the REPL, either using the command `)fin` which drops into lisp or closing the `*standard-input*` stream. If dropped into lisp, the top level loop can be restarted by calling (**restart**).

intloopReadConsole takes 2 arguments. The first is a String **prefix** which is usually an empty string but might contain prior lines that ended with an underscore, the Axiom continuation character. The second is an Integer which will be the step number printed at the prompt.

10.1 An Overview of a Simple Input

Here we walk through details of Axiom's default behavior when handling a simple input, the number 1. Many details are skipped in order to provide a simple overview of the interpreter operation. Further details can be found at the specific functions.

Axiom is in **intloopReadConsole**(p321), the Read-Eval-Print-Loop (REPL) function and the user types "1".

```

1> (|intloopReadConsole| "" 1)
    ; serverReadLine : Stream -> String
2> (|serverReadLine| #<synonym stream to *TERMINAL-IO*>)
    ; is-console : Stream -> Boolean
3> (IS-CONSOLE #<synonym stream to *TERMINAL-IO*>)
<3 (IS-CONSOLE T)
    ; sockSendInt : (Purpose,Command) -> Integer
    ; Purpose 1 is SessionManager, Command 3 is EndOfOutput
    ; A return of 0 indicates success.
    ; see the socket types purpose list in bookvol7, chunk include/com.h
3> (|sockSendInt| 1 3)
<3 (|sockSendInt| 0)
    ; serverSwitch : Void -> Integer
    ; see server_switch in sockio.c
    ; this multiplexes the socket connection among front ends
    ; CallInterp is the constant 4 (see the table in sockio-c)
    ; CallInterp simply returns to the interpreter
3> (|serverSwitch|)
1
    <3 (|serverSwitch| 4)
    ; the action for CallInterp is to call read-line
    ; read-line is defined in vmlisp.lisp
3> (|read-line| #<synonym stream to *TERMINAL-IO*>)
<3 (|read-line| "1" NIL)
<2 (|serverReadLine| "1")

```

Axiom calls **serverReadLine**(p337) to read the integer from the console. First it calls **is-console** (bookvol9) to check that the console stream exists.

sockSendInt (see sockio.lisp, sockio-c.c) sends on socket 1 (**SessionManager**) a 3, meaning **EndOfOutput**, i.e. a newline.

serverSwitch (see sockio-c in bookvol7) multitasks among the different sockets and finds the interpreter socket is available, returning 4 (**CallInterp**) (see sockio-c commands sent table and bookvol8).

serverReadLine(p337) has a cond switch for action **\$CallInterp**. In that case it calls **read-line** (see vmlisp.lisp) to read the input line and returns the result, in this case, the string "1".

```

2> (|intloopPrefix?| ")fi" "1")
<2 (|intloopPrefix?| NIL)
2> (|intloopPrefix?| ")" "1")
<2 (|intloopPrefix?| NIL)
2> (CONCAT "" "1")
<2 (CONCAT "1")
2> (|ncloopEscaped| "1")
<2 (|ncloopEscaped| NIL)

```

intloopReadConsole(p321) checks for various possible special kinds of input. Axiom returned a non-zero length string. Before processing it we need to check for the “**fin**” com-

mand, which fails. We need to check for a leading “)”, meaning it is some kind of command input, which fails. We might have an existing string in the **prefix** argument so we concatenate it to the input. The **prefix** might contain text from a previous continued line. Next we check whether the input line has a trailing underscore, meaning an Axiom line is being continued, and if so, we recurse in order to read the next line.

intloopPrefix?(p327) which will return NIL if there is no match of the prefix characters, otherwise it returns the string without any leading blanks.

None of these special cases occur with the input “1”. Axiom calls **intloopProcessString**(p329) which calls **setCurrentLine**(p334) to add the input line to the history which is stored in **\$currentLine**.

```
2> (|intloopProcessString| "1" 1)
3> (|setCurrentLine| "1")
<3 (|setCurrentLine| ("1"))
```

... all the magic happens here...

... and then **intloopProcessString** will eventually return the new step number 2. Then Axiom puts up a prompt and waits for further input.

```
<2 (|intloopProcessString| 2)
2> (MKPROMPT)
3> (CONCAT "(" "2" ") -> ")
<3 (CONCAT "(2) -> ")
<2 (MKPROMPT "(2) -> ")
(2) ->
2> (|serverReadLine| #<synonym stream to *TERMINAL-IO*>)
3> (IS-CONSOLE #<synonym stream to *TERMINAL-IO*>)
<3 (IS-CONSOLE T)
3> (|sockSendInt| 1 3)
<3 (|sockSendInt| 0)
3> (|serverSwitch|)
```

Now Axiom is ready for the next input.

10.2 Parsing the input

We now examine the magic portion above which has several phases. The first phase constructs a data structure called a Delay. This data structure is the core data structure of the “zipper” parser.

The “zipper” parser is unique to Axiom. It was invented by Bill Burge who did research in recursive techniques, including parsing. For insight, see his article on Stream Processing Functions [Burg74].

Creating a Delay – `incString`

The `intloopProcessString`(p329) has the nested function call

```
(|intloopProcess| n t
  (|next| #'|ncloopParse|
    (|next| #'|lineoftoks| (|incString| s))))
```

which according to lisp semantics is processed inside out. First we examine the call to `incString`(p331) which is passed the input string “1”.

The `incString`(p331) function gets the string from Axiom’s input line, in this case “1” and constructs a set of nested function calls to process the input line.

```
3> (|incString| "1")
```

The `incString`(p331) function calls `Delay`(p397) which changes the function call into a simple list object prefixed by the symbol tag `nonnullstream`.

```
4> (|incLude| 0 ("1") 0 ("strings") (1))
5> (|Delay| |incLude1| (0 ("1") 0 ("strings") (1)))
<5 (|Delay| (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1)))
<4 (|incLude| (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1)))
```

That result is passed to `incRenumber`(p366), which calls `incIgen`(p367) which returns a `Delay`(p397). It then calls `incZip`(p366) to “zip” together the function `incRenumberLine`(p368) and the two delays into a single delay. This gets put into a delay with `incZip1`(p367) as the function.

```
4> (|incRenumber|
  (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1)))
5> (|incIgen| 0)
6> (|Delay| |incIgen1| (0))
<6 (|Delay| (|nonnullstream| |incIgen1| 0))
<5 (|incIgen| (|nonnullstream| |incIgen1| 0))

5> (|incZip| |incRenumberLine|
  (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
  (|nonnullstream| |incIgen1| 0))
6> (|Delay| |incZip1| |incRenumberLine|)
<6 (|Delay|
  (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
    (|nonnullstream| |incIgen1| 0)))
<5 (|incZip|
  (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
    (|nonnullstream| |incIgen1| 0)))
```

```

<4 (|incRenumbe|
    (|nonnullstream| |incZip1| |incRenumbeLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0)))

<3 (|incString|
    (|nonnullstream| |incZip1| |incRenumbeLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0)))

```

We are building a stream of functions and arguments stored in a delay structure which will eventually be evaluated. We continue this process with the call to **next**(p330) which builds a delay with the function **next1**(p330) and the current delay.

Creating a Delay – next

```

3> (|next| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumbeLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0)))
4> (|Delay| |next1|
    (|lineoftoks|
      (|nonnullstream| |incZip1| |incRenumbeLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))
<4 (|Delay| (|nonnullstream| |next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumbeLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0))))
<3 (|next|
    (|nonnullstream| |next1| |lineoftoks|
      (|nonnullstream| |incZip1| |incRenumbeLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))

```

Creating a Delay – ncloopParse

We continue building a larger delay, this time with a call to **next**(p330) with the function argument **ncloopParse**(p329) and the existing delay.

```

3> (|next| |ncloopParse|
    (|nonnullstream| |next1| |lineoftoks|
      (|nonnullstream| |incZip1| |incRenumbeLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))
4> (|Delay| #0=|next1|

```



```

(|ncloopParse|
  (|nonnullstream| #0# |lineoftoks|
    (|nonnullstream| |incZip1| |incReNumberLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0))))))
<4 (|Delay|
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| #0# |lineoftoks|
      (|nonnullstream| |incZip1| |incReNumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))))
<3 (|next|
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| #0# |lineoftoks|
      (|nonnullstream| |incZip1| |incReNumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))))

```

Finally we call **intloopProcess**(p356) with the step number **stepno**, whether we are talking to the console **interactive** and the delay we just constructed **delay**

Evaluating a Delay – intloopProcess

At this point we have created a large delay. Now we begin to evaluate it.

```

3> (|intloopProcess| 1 T
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| #0# |lineoftoks|
      (|nonnullstream| |incZip1| |incReNumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))))

```

intloopProcess(p356) calls **StreamNull**(p623) which walks the delay applying the second value, which is a function, to the rest of the delay. Thus, all of the functions we packaged into the delay will be evaluated.

The result of each function call, e.g the result of calling **next1**(p330) will be a pair, which we call a **ParsePair**. The car of the **ParsePair** is rplacd'd into the delay and the cdr of the **ParsePair** is rplacd'd into the delay. So the delay is gradually reduced by each function call.

```

4> (|StreamNull|
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| #0# |lineoftoks|
      (|nonnullstream| |incZip1| |incReNumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))))

```

Here we see the **next1**(p330) function being called from the delay. It immediately calls **StreamNull**(p623) to process the rest of the delay.

```

5> (|next1| |incloopParse|
    (|nonnullstream| |next1| |lineoftoks|
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))
6> (|StreamNull|
    (|nonnullstream| |next1| |lineoftoks|
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
        (|nonnullstream| |incIgen1| 0))))

```

StreamNull(p623), now working on the inner portion of the delay, finds the function **next1**(p330) and calls it, which results in an immediate inner call to **StreamNull**(p623).

```

7> (|next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumberLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0)))
8> (|StreamNull|
    (|nonnullstream| |incZip1| |incRenumberLine|
      (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
      (|nonnullstream| |incIgen1| 0)))

```

Descending even further, the **StreamNull**(p623) finds **incZip1**(p367), which finds the function **incRenumberLine**(p368) and two delays.

```

9> (|incZip1|
    |incRenumberLine|
    (|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
    (|nonnullstream| |incIgen1| 0))

```

incZip1(p367) invokes **StreamNull**(p623) on the first delay, which invokes **incLude1**(p374) on the rest of the delay.

```

10> (|StreamNull|
     (|nonnullstream| |incLude1| 0 ("1") 0
                          ("strings") (1)))

```

incLude1(p374) unpacks the argument list and invokes **StreamNull**(p623) on the second argument ("1") which is not the expected symbol **nonnullstream** so **StreamNull**(p623) immediately returns NIL.

```

11> (|incLude1| 0 ("1") 0 ("strings") (1))
12> (|StreamNull| ("1"))
<12 (|StreamNull| NIL)

```

Next, **incLude1**(p374) calls **incClassify**(p393) to which calls **incCommand?**(p394) which checks for a leading “”. Since there isn’t one **incClassify**(p393) immediately returns a list of NIL, 0, and the empty string.

```

12> (|incClassify| "1")
13> (|incCommand?| "1")
<13 (|incCommand?| NIL)
<12 (|incClassify| (NIL 0 ""))

12> (|Skipping?| 1)
13> (|KeepPart?| 1)
<13 (|KeepPart?| T)
<12 (|Skipping?| NIL)

12> (|xLOK| 0 "1" 1 "strings")
13> (|xLOK1| 0 "1" "1" 1 "strings")
14> (INCLINE1 0 "1" "1" -1 1 "strings")
15> (|lnCreate| 0 "1" -1 1 "strings")
<15 (|lnCreate| (0 "1" -1 1 "strings"))
<14 (INCLINE1 (((0 "1" -1 1 "strings") . 1) . "1"))
<13 (|xLOK1| (((0 "1" -1 1 "strings") . 1) . "1")
        (NIL |none|)))
<12 (|xLOK| (((0 "1" -1 1 "strings") . 1) . "1")
        (NIL |none|)))

12> (|incLude| 0 NIL 1 ("strings") (1))
13> (|Delay| |incLude1| (0 NIL 1 ("strings") (1)))
<13 (|Delay|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1)))
<12 (|incLude|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1)))

<11 (|incLude1|
        (((0 "1" -1 1 "strings") . 1) . "1") (NIL |none|))
        |nonnullstream| |incLude1| 0 NIL 1 ("strings") (1)))
<10 (|StreamNull| NIL)

10> (|StreamNull| (|nonnullstream| |incIgen1| 0))
11> (|incIgen1| 0)
12> (|incIgen| 1)
13> (|Delay| |incIgen1| (1))
<13 (|Delay| (|nonnullstream| |incIgen1| 1))
<12 (|incIgen| (|nonnullstream| |incIgen1| 1))
<11 (|incIgen1| (1 |nonnullstream| |incIgen1| 1))
<10 (|StreamNull| NIL)
10> (|incRenumberLine|
        (((0 "1" -1 1 "strings") . 1) . "1") (NIL |none|)) 1)
11> (|incRenumberItem|
        (((0 "1" -1 1 "strings") . 1) . "1") 1)
12> (|lnSetGlobalNum| (0 "1" -1 1 "strings") 1)
<12 (|lnSetGlobalNum| 1)
<11 (|incRenumberItem| (((0 "1" 1 1 "strings") . 1) . "1"))
11> (|incHandleMessage|
        (((0 "1" 1 1 "strings") . 1) . "1") (NIL |none|)))

```

```

<11 (|incHandleMessage| 0)
<10 (|incRenumberLine|
      (((0 "1" 1 1 "strings") . 1) . "1"))

10> (|incZip| |incRenumberLine|
      (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
      (|nonnullstream| |incIgen1| 1))
11> (|Delay| |incZip1|
      (|incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
<11 (|Delay|
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
<10 (|incZip|
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))

<9 (|incZip1|
      (((0 "1" 1 1 "strings") . 1) . "1")
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
<8 (|StreamNull| NIL)

8> (|lineoftoks|
      (((0 "1" 1 1 "strings") . 1) . "1")
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
9> (|nextline|
      (((0 "1" 1 1 "strings") . 1) . "1")
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
10> (|npNull|
      (((0 "1" 1 1 "strings") . 1) . "1")
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
11> (|StreamNull|
      (((0 "1" 1 1 "strings") . 1) . "1")
      (|nonnullstream| |incZip1| |incRenumberLine|
        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1)))
<11 (|StreamNull| NIL)
<10 (|npNull| NIL)

```

```

10> (STRPOSL " " "1" 0 T)
<10 (STRPOSL 0)
<9 (|nextline| T)
9> (|scanIgnoreLine| "1" 0)
<9 (|scanIgnoreLine| 0)
9> (|incPrefix?| "command" 1 "1")
<9 (|incPrefix?| NIL)
9> (|scanToken|)
10> (|startsComment?|)
<10 (|startsComment?| NIL)
10> (|startsNegComment?|)
<10 (|startsNegComment?| NIL)
10> (|punctuation?| 49)
<10 (|punctuation?| NIL)
10> (|digit?| #\1)
11> (DIGITP #\1)
<11 (DIGITP 1)
<10 (|digit?| 1)
10> (|scanNumber|)
11> (|spleI| |digit?|)
12> (|spleI1| |digit?| NIL)
13> (|digit?| #\1)
14> (DIGITP #\1)
<14 (DIGITP 1)
<13 (|digit?| 1)
<12 (|spleI1| "1")
<11 (|spleI| "1")
11> (|lfinteger| "1")
<11 (|lfinteger| (|integer| "1"))
<10 (|scanNumber| (|integer| "1"))
10> (|lnExtraBlanks| (0 "1" 1 1 "strings"))
<10 (|lnExtraBlanks| 0)
10> (|constoken|
      "1" (0 "1" 1 1 "strings") (|integer| "1") 0)
11> (|ncPutQ|
      (|integer| . "1") |posn| ((0 "1" 1 1 "strings") . 0))
12> (|ncAlist| (|integer| . "1"))
<12 (|ncAlist| NIL)
12> (|ncAlist| (|integer| . "1"))
<12 (|ncAlist| NIL)
12> (|ncTag| (|integer| . "1"))
<12 (|ncTag| |integer|)
<11 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<10 (|constoken|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) .
        "1"))
10> (|dqUnit|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) .
        "1"))
<10 (|dqUnit|

```

```

        (#0=(((integer| (posn| (0 "1" 1 1 "strings") . 0)) .
          "1"))) . #0#))
<9 (scanToken|
  (#0=(((integer| (posn| (0 "1" 1 1 "strings") . 0)) .
    "1"))) . #0#))
9> (dqAppend| NIL
  (#0=(((integer| (posn| (0 "1" 1 1 "strings") . 0)) .
    "1"))) . #0#))
<9 (dqAppend|
  (#0=(((integer| (posn| (0 "1" 1 1 "strings") . 0)) .
    "1"))) . #0#))
<8 (lineoftoks|
  (((#0=
    ((integer| (posn| #1=(0 "1" 1 1 "strings") . 0)) . "1")
    . #0#)
    ((#1# . 1) . "1") .
    #2=(nonnullstream| incZip1| incRenumberLine|
      (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
      (nonnullstream| incIgen1| 1))))
    . #2#))

8> (next| lineoftoks|
  (nonnullstream| incZip1| incRenumberLine|
  (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
  (nonnullstream| incIgen1| 1)))
9> (Delay| next1|
  (lineoftoks|
  (nonnullstream| incZip1| incRenumberLine|
  (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
  (nonnullstream| incIgen1| 1))))
<9 (Delay|
  (nonnullstream| next1| lineoftoks|
  (nonnullstream| incZip1| incRenumberLine|
  (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
  (nonnullstream| incIgen1| 1))))
<8 (next|
  (nonnullstream| next1| lineoftoks|
  (nonnullstream| incZip1| incRenumberLine|
  (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
  (nonnullstream| incIgen1| 1))))

8> (incAppend|
  (((#0=(((integer| (posn| #1=(0 "1" 1 1 "strings") . 0)) . "1"))) . #0#)
  ((#1# . 1) . "1") .
  #2=(nonnullstream| incZip1| incRenumberLine|
    (nonnullstream| incLude1| 0 NIL 1 ("strings") (1))
    (nonnullstream| incIgen1| 1))))
  (nonnullstream| next1| lineoftoks| #2#))
9> (Delay| incAppend1|
  (((#1=((

```

```

(|integer| (|posn| #2=(0 "1" 1 1 "strings") . 0)) .
  "1")) . #1#)
((#2# . 1) . "1") .
  #3=(|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
(|nonnullstream| |next1| |lineoftoks| #3#)))
<9 (|Delay|
  (|nonnullstream| |incAppend1|
    ((#1=
      ((|integer| (|posn| #2=(0 "1" 1 1 "strings") . 0))
        . "1"))
        . #1#)
      ((#2# . 1) . "1") .
        #3=(|nonnullstream| |incZip1| |incRenumberLine|
          (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
          (|nonnullstream| |incIgen1| 1))))
        (|nonnullstream| |next1| |lineoftoks| #3#)))
<8 (|incAppend|
  (|nonnullstream| |incAppend1|
    ((#1=(((|integer| (|posn| #2=(0 "1" 1 1 "strings") . 0)) . "1")) . #1#)
      ((#2# . 1) . "1") .
        #3=(|nonnullstream| |incZip1| |incRenumberLine|
          (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
          (|nonnullstream| |incIgen1| 1))))
        (|nonnullstream| |next1| |lineoftoks| #3#)))
<7 (|next1|
  (|nonnullstream| |incAppend1|
    ((#1=(((|integer| (|posn| #2=(0 "1" 1 1 "strings") . 0))
      . "1"))
      . #1#) ((#2# . 1) . "1") .
        #3=(|nonnullstream| |incZip1| |incRenumberLine|
          (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
          (|nonnullstream| |incIgen1| 1))))
        (|nonnullstream| |next1| |lineoftoks| #3#)))
7> (|incAppend1|
  ((#0=(((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0)) .
    "1"))
    . #0#) ((#1# . 1) . "1") .
    #2=(|nonnullstream| |incZip1| |incRenumberLine|
      (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
      (|nonnullstream| |incIgen1| 1))))
    (|nonnullstream| |next1| |lineoftoks| #2#)))
8> (|StreamNull|
  ((#0=(((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
    . #0#) ((#1# . 1) . "1")
    (|nonnullstream| |incZip1| |incRenumberLine|

```

```

        (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
        (|nonnullstream| |incIgen1| 1))))
<8 (|StreamNull| NIL)

8> (|incAppend| NIL
    (|nonnullstream| |next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
9> (|Delay| |incAppend1|
    (NIL
    (|nonnullstream| |next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
<9 (|Delay|
    (|nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
<8 (|incAppend| (|nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks|
    (|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))

<7 (|incAppend1|
    (((#0=(((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1")) . #0#) (((#1# . 1) . "1")
    . #2=(|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
    |nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks| #2#)))
<6 (|StreamNull| NIL)

6> (|incloopParse|
    (((#0=(((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1")) . #0#) (((#1# . 1) . "1")
    . #2=(|nonnullstream| |incZip1| |incRenumberLine|
    (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
    (|nonnullstream| |incIgen1| 1))))
    |nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks| #2#)))
7> (|incloopDQlines|
    (#0=(((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1")) . #0#) (((#1# . 1) . "1")
    |nonnullstream| |incZip1| |incRenumberLine|

```



```

(|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
(|nonnullstream| |incIgen1| 1)))
8> (|StreamNull|
  (((0 "1" 1 1 "strings") . 1) . "1")
  |nonnullstream| |incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1)))
<8 (|StreamNull| NIL)
8> (|tokPosn|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
9> (|ncAlist|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<9 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<8 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
8> (|poGlobalLinePosn| ((0 "1" 1 1 "strings") . 0))
9> (|poGetLineObject| ((0 "1" 1 1 "strings") . 0))
<9 (|poGetLineObject| (0 "1" 1 1 "strings"))
9> (|lnGlobalNum| (0 "1" 1 1 "strings"))
<9 (|lnGlobalNum| 1)
<8 (|poGlobalLinePosn| 1)
8> (|poGlobalLinePosn| ((0 "1" 1 1 "strings") . 1))
9> (|poGetLineObject| ((0 "1" 1 1 "strings") . 1))
<9 (|poGetLineObject| (0 "1" 1 1 "strings"))
9> (|lnGlobalNum| (0 "1" 1 1 "strings"))
<9 (|lnGlobalNum| 1)
<8 (|poGlobalLinePosn| 1)
8> (|streamChop| 1
  (((0 "1" 1 1 "strings") . 1) . "1")
  |nonnullstream| |incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1)))
9> (|StreamNull|
  (((0 "1" 1 1 "strings") . 1) . "1")
  |nonnullstream| |incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1)))
<9 (|StreamNull| NIL)
9> (|streamChop| 0
  (|nonnullstream| |incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1)))
10> (|StreamNull|
  (|nonnullstream| |incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1)))
11> (|incZip1| |incRenumLine|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1))
  (|nonnullstream| |incIgen1| 1))
12> (|StreamNull|
  (|nonnullstream| |incLude1| 0 NIL 1 ("strings") (1)))

```

```

13> (|incLude1| 0 NIL 1 ("strings") (1))
14> (|StreamNull| NIL)
<14 (|StreamNull| T)
14> (|Top?| 1)
<14 (|Top?| T)
<13 (|incLude1| (|nullstream|))
<12 (|StreamNull| T)
<11 (|incZip1| (|nullstream|))
<10 (|StreamNull| T)
<9 (|streamChop| (NIL NIL))
9> (|ncloopPrefix?| ")command" "1")
<9 (|ncloopPrefix?| NIL)
<8 (|streamChop| (((((0 "1" 1 1 "strings") . 1) . "1"))) NIL))
<7 (|ncloopDQlines| (((((0 "1" 1 1 "strings") . 1) . "1"))) NIL))
7> (|dqToList|
  (#0=((
    (|integer| (|posn| (0 "1" 1 1 "strings") . 0))
    . "1"))) . #0#))
<7 (|dqToList|
  ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
7> (|npParse|
  ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
8> (|npFirstTok|
  9> (|tokPart|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
    <9 (|tokPart| "1")
  <8 (|npFirstTok| "1")
8> (|npItem|
  9> (|npQualDef|
    10> (|npComma|
      11> (|npTuple| |npQualifiedDefinition|)
      12> (|npListofFun|
        |npQualifiedDefinition|
        |npCommaBackSet|
        |pfTupleListOf|)
      13> (|npQualifiedDefinition|)
      14> (|npQualified| |npDefinitionOrStatement|)
      15> (|npDefinitionOrStatement|)
      16> (|npBackTrack| |npGives| DEF |npDef|)
      17> (|npState|)
    <17 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
    17> (|npGives|)
    18> (|npBackTrack| |npExit| GIVES |npLambda|)
    19> (|npState|)
    <19 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
    19> (|npExit|)

```

```

20> (|npBackTrack| |npAssign| EXIT |npPileExit|)
21> (|npState|)
<21 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
21> (|npAssign|)
22> (|npBackTrack| |npMDEF| BECOMES |npAssignment|)
23> (|npState|)
<23 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
23> (|npMDEF|)
24> (|npBackTrack| |npStatement| MDEF |npMDEFinition|)
25> (|npState|)
<25 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
25> (|npStatement|)
26> (|npExpress|)
27> (|npExpress1|)
28> (|npConditionalStatement|)
29> (|npConditional| |npQualifiedDefinition|)
30> (|npEqKey| IF)
<30 (|npEqKey| NIL)
<29 (|npConditional| NIL)
<28 (|npConditionalStatement| NIL)
28> (|npADD|)
29> (|npType|)
30> (|npMatch|)
31> (|npLeftAssoc| (IS ISNT) |npSuch|)
32> (|npSuch|)
33> (|npLeftAssoc| (BAR) |npLogical|)
34> (|npLogical|)
35> (|npLeftAssoc| (OR) |npDisjand|)
36> (|npDisjand|)
37> (|npLeftAssoc| (AND) |npDiscrim|)
38> (|npDiscrim|)
39> (|npLeftAssoc| (CASE HAS) |npQuiver|)
40> (|npQuiver|)
41> (|npRightAssoc| (ARROW LARROW) |npRelation|)
42> (|npState|)
<42 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
42> (|npRelation|)
43> (|npLeftAssoc|
      (EQUAL NOTEQUAL LT LE GT GE OANGLE CANGLE)
      |npSynthetic|)
44> (|npSynthetic|)
45> (|npBy|)

```

```

46> (|npLeftAssoc| (BY) |npInterval|)
47> (|npInterval|)
48> (|npArith|)
49> (|npLeftAssoc| (MOD) |npSum|)
50> (|npSum|)
51> (|npLeftAssoc| (PLUS MINUS) |npTerm|)
52> (|npTerm|)
53> (|npInfGeneric| (MINUS PLUS))
54> (|npDDInfKey| (MINUS PLUS))
55> (|npInfKey| (MINUS PLUS))
<55 (|npInfKey| NIL)
55> (|npState|)
<55 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
55> (|npEqKey| '|')
<55 (|npEqKey| NIL)
55> (|npRestore|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
56> (|npFirstTok|)
57> (|tokPart|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
<57 (|tokPart| "1")
<56 (|npFirstTok| "1")
<55 (|npRestore| T)
55> (|npEqKey| BACKQUOTE)
<55 (|npEqKey| NIL)
55> (|npRestore|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
56> (|npFirstTok|)
57> (|tokPart|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
<57 (|tokPart| "1")
<56 (|npFirstTok| "1")
<55 (|npRestore| T)
<54 (|npDDInfKey| NIL)
<53 (|npInfGeneric| NIL)
53> (|npRemainder|)
54> (|npLeftAssoc| (REM QUO) |npProduct|)
55> (|npProduct|)
56> (|npLeftAssoc|
      (TIMES SLASH BACKSLASH SLASHSLASH BACKSLASHBACKSLASH
        SLASHBACKSLASH BACKSLASHSLASH)
      |npPower|)
57> (|npPower|)
58> (|npRightAssoc| (POWER CARAT) |npColon|)

```

```

59> (|npState|)
<59 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
59> (|npColon|)
60> (|npTypified|)
61> (|npApplication|)
62> (|npDotted| |npPrimary|)
63> (|npPrimary|)
64> (|npPrimary1|)
65> (|npEncAp| |npAtom1|)
66> (|npAtom1|)
67> (|npPDefinition|)
68> (|npParenthesized| |npDefinitionlist|)
69> (|npParenthesize| |(| |)| |npDefinitionlist|)
70> (|npEqKey| |(| |)|)
<70 (|npEqKey| NIL)
<69 (|npParenthesize| NIL)
69> (|npParenthesize| |(\|| |\||)| |npDefinitionlist|)
70> (|npEqKey| |(\|| |\||)|)
<70 (|npEqKey| NIL)
<69 (|npParenthesize| NIL)
<68 (|npParenthesized| NIL)
<67 (|npPDefinition| NIL)
67> (|npName|)
68> (|npId|)
<68 (|npId| NIL)
68> (|npSymbolVariable|)
69> (|npState|)
<69 (|npState|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
69> (|npEqKey| BACKQUOTE)
<69 (|npEqKey| NIL)
69> (|npRestore|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
70> (|npFirstTok|)
71> (|tokPart|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))
<71 (|tokPart| "1")
<70 (|npFirstTok| "1")
<69 (|npRestore| T)
<68 (|npSymbolVariable| NIL)
<67 (|npName| NIL)
67> (|npConstTok|)
68> (|tokType|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))

```

```

69> (|ncTag|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<69 (|ncTag| |integer|)
<68 (|tokType| |integer|)
68> (|npPush|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<68 (|npPush|
    (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
      . "1"))))
68> (|npNext|)
69> (|npFirstTok|)
70> (|tokPosn|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
71> (|ncAlist|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<71 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<70 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
70> (|tokConstruct| ERROR NOMORE
    ((0 "1" 1 1 "strings") . 0))
71> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
72> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<72 (|poNoPosition?| NIL)
<71 (|pfNoPosition?| NIL)
71> (|ncPutQ| (ERROR . NOMORE) |posn|
    ((0 "1" 1 1 "strings") . 0))
72> (|ncAlist| (ERROR . NOMORE))
<72 (|ncAlist| NIL)
72> (|ncAlist| (ERROR . NOMORE))
<72 (|ncAlist| NIL)
72> (|ncTag| (ERROR . NOMORE))
<72 (|ncTag| ERROR)
<71 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<70 (|tokConstruct|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0))
     . NOMORE))
70> (|tokPart|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<70 (|tokPart| NOMORE)
<69 (|npFirstTok| NOMORE)
<68 (|npNext| NOMORE)
<67 (|npConstTok| NOMORE)
67> (|npFromdom|)
68> (|npEqKey| $)
<68 (|npEqKey| NIL)
<67 (|npFromdom| T)
<66 (|npAtom1| T)

```

```

66> (|npAnyNo| |npEncl|)
67> (|npEncl|)
68> (|npBDefinition|)
69> (|npPDefinition|)
70> (|npParenthesized| |npDefinitionlist|)
71> (|npParenthesize| |( | |)| |npDefinitionlist|)
72> (|npEqKey| |( |))
<72 (|npEqKey| NIL)
<71 (|npParenthesize| NIL)
71> (|npParenthesize| |(\ | | \ |)| |npDefinitionlist|)
72> (|npEqKey| |(\ | |))
<72 (|npEqKey| NIL)
<71 (|npParenthesize| NIL)
<70 (|npParenthesized| NIL)
<69 (|npPDefinition| NIL)
69> (|npBracketed| |npDefinitionlist|)
70> (|npParened| |npDefinitionlist|)
71> (|npEnclosed| |( | |)| |pfParen| |npDefinitionlist|)
72> (|npEqKey| |( |))
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
71> (|npEnclosed| |(\ | | \ |)| |pfParen| |npDefinitionlist|)
72> (|npEqKey| |(\ | |))
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
<70 (|npParened| NIL)
70> (|npBracked| |npDefinitionlist|)
71> (|npEnclosed| [ ] |pfBracket| |npDefinitionlist|)
72> (|npEqKey| [])
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
71> (|npEnclosed| |[ \ | | \ |]|
      |pfBracketBar| |npDefinitionlist|)
72> (|npEqKey| |[ \ | |])
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
<70 (|npBracked| NIL)
70> (|npBraced| |npDefinitionlist|)
71> (|npEnclosed| { } |pfBrace| |npDefinitionlist|)
72> (|npEqKey| {})
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
71> (|npEnclosed| |{\ | | \ |}|
      |pfBraceBar| |npDefinitionlist|)
72> (|npEqKey| |{\ | |})
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
<70 (|npBraced| NIL)
70> (|npAngleBared| |npDefinitionlist|)
71> (|npEnclosed| |< \ | | \ |>| |pfHide| |npDefinitionlist|)

```

```

72> (|npEqKey| |<\\|)
<72 (|npEqKey| NIL)
<71 (|npEnclosed| NIL)
<70 (|npAngleBared| NIL)
<69 (|npBracketed| NIL)
<68 (|npBDefinition| NIL)
<67 (|npEncl| NIL)
<66 (|npAnyNo| T)
66> (|npFromdom|)
67> (|npEqKey| $)
<67 (|npEqKey| NIL)
<66 (|npFromdom| T)
<65 (|npEncAp| T)
<64 (|npPrimary1| T)
<63 (|npPrimary| T)
63> (|npAnyNo| |npSelector|)
64> (|npSelector|)
65> (|npEqKey| DOT)
<65 (|npEqKey| NIL)
<64 (|npSelector| NIL)
<63 (|npAnyNo| T)
<62 (|npDotted| T)
62> (|npApplication2|)
63> (|npDotted| |npPrimary1|)
64> (|npPrimary1|)
65> (|npEncAp| |npAtom1|)
66> (|npAtom1|)
67> (|npPDefinition|)
68> (|npParenthesized| |npDefinitionlist|)
69> (|npParenthesize| |( | )| |npDefinitionlist|)
70> (|npEqKey| |( |) )
<70 (|npEqKey| NIL)
<69 (|npParenthesize| NIL)
69> (|npParenthesize| |(\| | \|)| |npDefinitionlist|)
70> (|npEqKey| |(\| |) )
<70 (|npEqKey| NIL)
<69 (|npParenthesize| NIL)
<68 (|npParenthesized| NIL)
<67 (|npPDefinition| NIL)
67> (|npName|)
68> (|npId|)
<68 (|npId| NIL)
68> (|npSymbolVariable|)
69> (|npState|)
<69 (|npState|
      (NIL ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
            . "1")))
69> (|npEqKey| BACKQUOTE)
<69 (|npEqKey| NIL)
69> (|npRestore|

```



```

      (NIL ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1")))
70> (|npFirstTok|)
71> (|tokPosn|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
72> (|ncAlist|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<72 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<71 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
71> (|tokConstruct| ERROR NOMORE
  ((0 "1" 1 1 "strings") . 0))
72> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
73> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<73 (|poNoPosition?| NIL)
<72 (|pfNoPosition?| NIL)
72> (|ncPutQ| (ERROR . NOMORE) |posn|
  ((0 "1" 1 1 "strings") . 0))
73> (|ncAlist| (ERROR . NOMORE))
<73 (|ncAlist| NIL)
73> (|ncAlist| (ERROR . NOMORE))
<73 (|ncAlist| NIL)
73> (|ncTag| (ERROR . NOMORE))
<73 (|ncTag| ERROR)
<72 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<71 (|tokConstruct|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
71> (|tokPart|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<71 (|tokPart| NOMORE)
<70 (|npFirstTok| NOMORE)
<69 (|npRestore| T)
<68 (|npSymbolVariable| NIL)
<67 (|npName| NIL)
67> (|npConstTok|)
68> (|tokType|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
69> (|ncTag|
  ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<69 (|ncTag| ERROR)
<68 (|tokType| ERROR)
68> (|npEqPeek| '|')
<68 (|npEqPeek| NIL)
<67 (|npConstTok| NIL)
67> (|npDollar|)
68> (|npEqPeek| '$')
<68 (|npEqPeek| NIL)
<67 (|npDollar| NIL)
67> (|npBDefinition|)
68> (|npPDefinition|)
69> (|npParenthesized| |npDefinitionlist|)

```

```

70> (|npParenthesize| |( | )| |npDefinitionlist|)
71> (|npEqKey| |( |)
<71 (|npEqKey| NIL)
<70 (|npParenthesize| NIL)
70> (|npParenthesize| |(\| | \|)| |npDefinitionlist|)
71> (|npEqKey| |(\| |)
<71 (|npEqKey| NIL)
<70 (|npParenthesize| NIL)
<69 (|npParenthesize| NIL)
<68 (|npDefinition| NIL)
68> (|npBracketed| |npDefinitionlist|)
69> (|npParened| |npDefinitionlist|)
70> (|npEnclosed| |( | )| |pfParen| |npDefinitionlist|)
71> (|npEqKey| |( |)
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
70> (|npEnclosed| |(\| | \|)| |pfParen| |npDefinitionlist|)
71> (|npEqKey| |(\| |)
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
<69 (|npParened| NIL)
69> (|npBracketed| |npDefinitionlist|)
70> (|npEnclosed| [ ] |pfBracket| |npDefinitionlist|)
71> (|npEqKey| [ ]
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
70> (|npEnclosed| |[\| | \|]|
      |pfBracketBar| |npDefinitionlist|)
71> (|npEqKey| |[\| |)
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
<69 (|npBracketed| NIL)
69> (|npBraced| |npDefinitionlist|)
70> (|npEnclosed| { } |pfBrace| |npDefinitionlist|)
71> (|npEqKey| { }
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
70> (|npEnclosed| |\{ \| | \|}|
      |pfBraceBar| |npDefinitionlist|)
71> (|npEqKey| |\{ \| |)
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
<69 (|npBraced| NIL)
69> (|npAngleBared| |npDefinitionlist|)
70> (|npEnclosed| |< \| | \|>| |pfHide| |npDefinitionlist|)
71> (|npEqKey| |< \| |)
<71 (|npEqKey| NIL)
<70 (|npEnclosed| NIL)
<69 (|npAngleBared| NIL)
<68 (|npBracketed| NIL)

```

```

<67 (|npBDefinition| NIL)
<66 (|npAtom1| NIL)
<65 (|npEncAp| NIL)
65> (|npLet|)
66> (|npLetQualified| |npDefinitionOrStatement|)
67> (|npEqKey| LET)
<67 (|npEqKey| NIL)
<66 (|npLetQualified| NIL)
<65 (|npLet| NIL)
65> (|npFix|)
66> (|npEqKey| FIX)
<66 (|npEqKey| NIL)
<65 (|npFix| NIL)
65> (|npMacro|)
66> (|npEqKey| MACRO)
<66 (|npEqKey| NIL)
<65 (|npMacro| NIL)
65> (|npBPileDefinition|)
66> (|npPileBracketed| |npPileDefinitionlist|)
67> (|npEqKey| SETTAB)
<67 (|npEqKey| NIL)
<66 (|npPileBracketed| NIL)
<65 (|npBPileDefinition| NIL)
65> (|npDefn|)
66> (|npEqKey| DEFN)
<66 (|npEqKey| NIL)
<65 (|npDefn| NIL)
65> (|npRule|)
66> (|npEqKey| RULE)
<66 (|npEqKey| NIL)
<65 (|npRule| NIL)
<64 (|npPrimary1| NIL)
<63 (|npDotted| NIL)
<62 (|npApplication2| NIL)
<61 (|npApplication| T)
61> (|npAnyNo| |npTypeStyle|)
62> (|npTypeStyle|)
63> (|npCoerceTo|)
64> (|npTypedForm| COERCE |pfCoerceto|)
65> (|npEqKey| COERCE)
<65 (|npEqKey| NIL)
<64 (|npTypedForm| NIL)
<63 (|npCoerceTo| NIL)
63> (|npRestrict|)
64> (|npTypedForm| AT |pfRestrict|)
65> (|npEqKey| AT)
<65 (|npEqKey| NIL)
<64 (|npTypedForm| NIL)
<63 (|npRestrict| NIL)
63> (|npPretend|)

```

```

64> (|npTypedForm| PRETEND |pfPretend|)
65> (|npEqKey| PRETEND)
<65 (|npEqKey| NIL)
<64 (|npTypedForm| NIL)
<63 (|npPretend| NIL)
63> (|npColonQuery|)
64> (|npTypedForm| ATAT |pfRetractTo|)
65> (|npEqKey| ATAT)
<65 (|npEqKey| NIL)
<64 (|npTypedForm| NIL)
<63 (|npColonQuery| NIL)
<62 (|npTypeStyle| NIL)
<61 (|npAnyNo| T)
<60 (|npTypified| T)
60> (|npAnyNo| |npTagged|)
61> (|npTagged|)
62> (|npTypedForm1| COLON |pfTagged|)
63> (|npEqKey| COLON)
<63 (|npEqKey| NIL)
<62 (|npTypedForm1| NIL)
<61 (|npTagged| NIL)
<60 (|npAnyNo| T)
<59 (|npColon| T)
59> (|npInfGeneric| (POWER CARAT))
60> (|npDDInfKey| (POWER CARAT))
61> (|npInfKey| (POWER CARAT))
<61 (|npInfKey| NIL)
61> (|npState|)
<61 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
61> (|npEqKey| '|')
<61 (|npEqKey| NIL)
61> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
62> (|npFirstTok|)
63> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
64> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<64 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<63 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
63> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
64> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
65> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<65 (|poNoPosition?| NIL)

```

```

<64 (|pfNoPosition?| NIL)
64> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
65> (|ncAlist| (ERROR . NOMORE))
<65 (|ncAlist| NIL)
65> (|ncAlist| (ERROR . NOMORE))
<65 (|ncAlist| NIL)
65> (|ncTag| (ERROR . NOMORE))
<65 (|ncTag| ERROR)
<64 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<63 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
63> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<63 (|tokPart| NOMORE)
<62 (|npFirstTok| NOMORE)
<61 (|npRestore| T)
61> (|npEqKey| BACKQUOTE)
<61 (|npEqKey| NIL)
61> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
62> (|npFirstTok|)
63> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
64> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<64 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<63 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
63> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
64> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
65> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<65 (|poNoPosition?| NIL)
<64 (|pfNoPosition?| NIL)
64> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
65> (|ncAlist| (ERROR . NOMORE))
<65 (|ncAlist| NIL)
65> (|ncAlist| (ERROR . NOMORE))
<65 (|ncAlist| NIL)
65> (|ncTag| (ERROR . NOMORE))
<65 (|ncTag| ERROR)
<64 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<63 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
63> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<63 (|tokPart| NOMORE)

```

```

<62 (|npFirstTok| NOMORE)
<61 (|npRestore| T)
<60 (|npDDInfKey| NIL)
<59 (|npInfGeneric| NIL)
<58 (|npRightAssoc| T)
<57 (|npPower| T)
57> (|npInfGeneric|
      (TIMES SLASH BACKSLASH SLASHSLASH BACKSLASHBACKSLASH
        SLASHBACKSLASH BACKSLASHSLASH))
58> (|npDDInfKey|
      (TIMES SLASH BACKSLASH SLASHSLASH BACKSLASHBACKSLASH
        SLASHBACKSLASH BACKSLASHSLASH))
59> (|npInfKey|
      (TIMES SLASH BACKSLASH SLASHSLASH BACKSLASHBACKSLASH
        SLASHBACKSLASH BACKSLASHSLASH))
<59 (|npInfKey| NIL)
59> (|npState|)
<59 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
59> (|npEqKey| '|')
<59 (|npEqKey| NIL)
59> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
60> (|npFirstTok|)
61> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
62> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<62 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<61 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
61> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
62> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
63> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<63 (|poNoPosition?| NIL)
<62 (|pfNoPosition?| NIL)
62> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
63> (|ncAlist| (ERROR . NOMORE))
<63 (|ncAlist| NIL)
63> (|ncAlist| (ERROR . NOMORE))
<63 (|ncAlist| NIL)
63> (|ncTag| (ERROR . NOMORE))
<63 (|ncTag| ERROR)
<62 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<61 (|tokConstruct|

```

```

      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
61> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<61 (|tokPart| NOMORE)
<60 (|npFirstTok| NOMORE)
<59 (|npRestore| T)
59> (|npEqKey| BACKQUOTE)
<59 (|npEqKey| NIL)
59> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
60> (|npFirstTok|)
61> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
62> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<62 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<61 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
61> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
62> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
63> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<63 (|poNoPosition?| NIL)
<62 (|pfNoPosition?| NIL)
62> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
63> (|ncAlist| (ERROR . NOMORE))
<63 (|ncAlist| NIL)
63> (|ncAlist| (ERROR . NOMORE))
<63 (|ncAlist| NIL)
63> (|ncTag| (ERROR . NOMORE))
<63 (|ncTag| ERROR)
<62 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<61 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
61> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<61 (|tokPart| NOMORE)
<60 (|npFirstTok| NOMORE)
<59 (|npRestore| T)
<58 (|npDDInfKey| NIL)
<57 (|npInfGeneric| NIL)
<56 (|npLeftAssoc| T)
<55 (|npProduct| T)
55> (|npInfGeneric| (REM QUO))
56> (|npDDInfKey| (REM QUO))
57> (|npInfKey| (REM QUO))
<57 (|npInfKey| NIL)
57> (|npState|)

```

```

<57 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
57> (|npEqKey| '|')
<57 (|npEqKey| NIL)
57> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
58> (|npFirstTok|)
59> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
60> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<60 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<59 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
59> (|tokConstruct| ERROR NOMORE (
      (0 "1" 1 1 "strings") . 0))
60> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
61> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<61 (|poNoPosition?| NIL)
<60 (|pfNoPosition?| NIL)
60> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
61> (|ncAlist| (ERROR . NOMORE))
<61 (|ncAlist| NIL)
61> (|ncAlist| (ERROR . NOMORE))
<61 (|ncAlist| NIL)
61> (|ncTag| (ERROR . NOMORE))
<61 (|ncTag| ERROR)
<60 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<59 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
59> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<59 (|tokPart| NOMORE)
<58 (|npFirstTok| NOMORE)
<57 (|npRestore| T)
57> (|npEqKey| BACKQUOTE)
<57 (|npEqKey| NIL)
57> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
58> (|npFirstTok|)
59> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
60> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))

```



```

<60 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<59 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
59> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
60> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
61> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<61 (|poNoPosition?| NIL)
<60 (|pfNoPosition?| NIL)
60> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
61> (|ncAlist| (ERROR . NOMORE))
<61 (|ncAlist| NIL)
61> (|ncAlist| (ERROR . NOMORE))
<61 (|ncAlist| NIL)
61> (|ncTag| (ERROR . NOMORE))
<61 (|ncTag| ERROR)
<60 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<59 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
59> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<59 (|tokPart| NOMORE)
<58 (|npFirstTok| NOMORE)
<57 (|npRestore| T)
<56 (|npDDInfKey| NIL)
<55 (|npInfGeneric| NIL)
<54 (|npLeftAssoc| T)
<53 (|npRemainder| T)
<52 (|npTerm| T)
52> (|npInfGeneric| (PLUS MINUS))
53> (|npDDInfKey| (PLUS MINUS))
54> (|npInfKey| (PLUS MINUS))
<54 (|npInfKey| NIL)
54> (|npState|)
<54 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
54> (|npEqKey| '|')
<54 (|npEqKey| NIL)
54> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
55> (|npFirstTok|)
56> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
57> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<57 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))

```

```

<56 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
56> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
57> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
58> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<58 (|poNoPosition?| NIL)
<57 (|pfNoPosition?| NIL)
57> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
58> (|ncAlist| (ERROR . NOMORE))
<58 (|ncAlist| NIL)
58> (|ncAlist| (ERROR . NOMORE))
<58 (|ncAlist| NIL)
58> (|ncTag| (ERROR . NOMORE))
<58 (|ncTag| ERROR)
<57 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<56 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
56> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<56 (|tokPart| NOMORE)
<55 (|npFirstTok| NOMORE)
<54 (|npRestore| T)
54> (|npEqKey| BACKQUOTE)
<54 (|npEqKey| NIL)
54> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
55> (|npFirstTok|)
56> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
57> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<57 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<56 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
56> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
57> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
58> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<58 (|poNoPosition?| NIL)
<57 (|pfNoPosition?| NIL)
57> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
58> (|ncAlist| (ERROR . NOMORE))
<58 (|ncAlist| NIL)
58> (|ncAlist| (ERROR . NOMORE))
<58 (|ncAlist| NIL)
58> (|ncTag| (ERROR . NOMORE))
<58 (|ncTag| ERROR)

```

```

<57 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<56 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
56> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<56 (|tokPart| NOMORE)
<55 (|npFirstTok| NOMORE)
<54 (|npRestore| T)
<53 (|npDDInfKey| NIL)
<52 (|npInfGeneric| NIL)
<51 (|npLeftAssoc| T)
<50 (|npSum| T)
50> (|npInfGeneric| (MOD))
51> (|npDDInfKey| (MOD))
52> (|npInfKey| (MOD))
<52 (|npInfKey| NIL)
52> (|npState|)
<52 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
52> (|npEqKey| '|')
<52 (|npEqKey| NIL)
52> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
53> (|npFirstTok|)
54> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
55> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<55 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<54 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
54> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
55> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
56> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<56 (|poNoPosition?| NIL)
<55 (|pfNoPosition?| NIL)
55> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
56> (|ncAlist| (ERROR . NOMORE))
<56 (|ncAlist| NIL)
56> (|ncAlist| (ERROR . NOMORE))
<56 (|ncAlist| NIL)
56> (|ncTag| (ERROR . NOMORE))
<56 (|ncTag| ERROR)
<55 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<54 (|tokConstruct|

```

```

        ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
54> (|tokPart|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<54 (|tokPart| NOMORE)
<53 (|npFirstTok| NOMORE)
<52 (|npRestore| T)
52> (|npEqKey| BACKQUOTE)
<52 (|npEqKey| NIL)
52> (|npRestore|
    (NIL
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
53> (|npFirstTok|)
54> (|tokPosn|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
55> (|ncAlist|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<55 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<54 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
54> (|tokConstruct| ERROR NOMORE
    ((0 "1" 1 1 "strings") . 0))
55> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
56> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<56 (|poNoPosition?| NIL)
<55 (|pfNoPosition?| NIL)
55> (|ncPutQ| (ERROR . NOMORE) |posn|
    ((0 "1" 1 1 "strings") . 0))
56> (|ncAlist| (ERROR . NOMORE))
<56 (|ncAlist| NIL)
56> (|ncAlist| (ERROR . NOMORE))
<56 (|ncAlist| NIL)
56> (|ncTag| (ERROR . NOMORE))
<56 (|ncTag| ERROR)
<55 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<54 (|tokConstruct|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
54> (|tokPart|
    ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<54 (|tokPart| NOMORE)
<53 (|npFirstTok| NOMORE)
<52 (|npRestore| T)
<51 (|npDDInfKey| NIL)
<50 (|npInfGeneric| NIL)
<49 (|npLeftAssoc| T)
<48 (|npArith| T)
48> (|npSegment|)
49> (|npEqPeek| SEG)
<49 (|npEqPeek| NIL)
<48 (|npSegment| NIL)
<47 (|npInterval| T)

```

```

47> (|npInfGeneric| (BY))
48> (|npDDInfKey| (BY))
49> (|npInfKey| (BY))
<49 (|npInfKey| NIL)
49> (|npState|)
<49 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
49> (|npEqKey| '|')
<49 (|npEqKey| NIL)
49> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
50> (|npFirstTok|)
51> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
52> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<52 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<51 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
51> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
52> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
53> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<53 (|poNoPosition?| NIL)
<52 (|pfNoPosition?| NIL)
52> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
53> (|ncAlist| (ERROR . NOMORE))
<53 (|ncAlist| NIL)
53> (|ncAlist| (ERROR . NOMORE))
<53 (|ncAlist| NIL)
53> (|ncTag| (ERROR . NOMORE))
<53 (|ncTag| ERROR)
<52 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<51 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
51> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<51 (|tokPart| NOMORE)
<50 (|npFirstTok| NOMORE)
<49 (|npRestore| T)
49> (|npEqKey| BACKQUOTE)
<49 (|npEqKey| NIL)
49> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))

```

```

50> (|npFirstTok|)
51> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
52> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<52 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<51 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
51> (|tokConstruct| ERROR NOMORE ((0 "1" 1 1 "strings") .
      0))
52> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
53> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<53 (|poNoPosition?| NIL)
<52 (|pfNoPosition?| NIL)
52> (|ncPutQ|
      (ERROR . NOMORE) |posn| ((0 "1" 1 1 "strings") . 0))
53> (|ncAlist| (ERROR . NOMORE))
<53 (|ncAlist| NIL)
53> (|ncAlist| (ERROR . NOMORE))
<53 (|ncAlist| NIL)
53> (|ncTag| (ERROR . NOMORE))
<53 (|ncTag| ERROR)
<52 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<51 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
51> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<51 (|tokPart| NOMORE)
<50 (|npFirstTok| NOMORE)
<49 (|npRestore| T)
<48 (|npDDInfKey| NIL)
<47 (|npInfGeneric| NIL)
<46 (|npLeftAssoc| T)
<45 (|npBy| T)
45> (|npAmpersandFrom|)
46> (|npAmpersand|)
47> (|npEqKey| AMPERSAND)
<47 (|npEqKey| NIL)
<46 (|npAmpersand| NIL)
<45 (|npAmpersandFrom| NIL)
<44 (|npSynthetic| T)
44> (|npInfGeneric|
      (EQUAL NOTEQUAL LT LE GT GE OANGLE CANGLE))
45> (|npDDInfKey|
      (EQUAL NOTEQUAL LT LE GT GE OANGLE CANGLE))
46> (|npInfKey| (EQUAL NOTEQUAL LT LE GT GE OANGLE CANGLE))
<46 (|npInfKey| NIL)
46> (|npState|)
<46 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))

```

```

      . "1"))))
46> (|npEqKey| '|')
<46 (|npEqKey| NIL)
46> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
47> (|npFirstTok|)
48> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
49> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<49 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<48 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
48> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
49> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
50> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<50 (|poNoPosition?| NIL)
<49 (|pfNoPosition?| NIL)
49> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
50> (|ncAlist| (ERROR . NOMORE))
<50 (|ncAlist| NIL)
50> (|ncAlist| (ERROR . NOMORE))
<50 (|ncAlist| NIL)
50> (|ncTag| (ERROR . NOMORE))
<50 (|ncTag| ERROR)
<49 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<48 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
48> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<48 (|tokPart| NOMORE)
<47 (|npFirstTok| NOMORE)
<46 (|npRestore| T)
46> (|npEqKey| BACKQUOTE)
<46 (|npEqKey| NIL)
46> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
47> (|npFirstTok|)
48> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
49> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<49 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<48 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
48> (|tokConstruct| ERROR NOMORE

```

```

      ((0 "1" 1 1 "strings") . 0))
49> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
50> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<50 (|poNoPosition?| NIL)
<49 (|pfNoPosition?| NIL)
49> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
50> (|ncAlist| (ERROR . NOMORE))
<50 (|ncAlist| NIL)
50> (|ncAlist| (ERROR . NOMORE))
<50 (|ncAlist| NIL)
50> (|ncTag| (ERROR . NOMORE))
<50 (|ncTag| ERROR)
<49 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<48 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
48> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<48 (|tokPart| NOMORE)
<47 (|npFirstTok| NOMORE)
<46 (|npRestore| T)
<45 (|npDDInfKey| NIL)
<44 (|npInfGeneric| NIL)
<43 (|npLeftAssoc| T)
<42 (|npRelation| T)
42> (|npInfGeneric| (ARROW LARROW))
43> (|npDDInfKey| (ARROW LARROW))
44> (|npInfKey| (ARROW LARROW))
<44 (|npInfKey| NIL)
44> (|npState|)
<44 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
44> (|npEqKey| '|')
<44 (|npEqKey| NIL)
44> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
45> (|npFirstTok|)
46> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
47> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<47 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<46 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
46> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
47> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))

```



```

48> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<48 (|poNoPosition?| NIL)
<47 (|pfNoPosition?| NIL)
47> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
48> (|ncAlist| (ERROR . NOMORE))
<48 (|ncAlist| NIL)
48> (|ncAlist| (ERROR . NOMORE))
<48 (|ncAlist| NIL)
48> (|ncTag| (ERROR . NOMORE))
<48 (|ncTag| ERROR)
<47 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<46 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
46> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<46 (|tokPart| NOMORE)
<45 (|npFirstTok| NOMORE)
<44 (|npRestore| T)
44> (|npEqKey| BACKQUOTE)
<44 (|npEqKey| NIL)
44> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
45> (|npFirstTok|)
46> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
47> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<47 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<46 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
46> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
47> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
48> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<48 (|poNoPosition?| NIL)
<47 (|pfNoPosition?| NIL)
47> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
48> (|ncAlist| (ERROR . NOMORE))
<48 (|ncAlist| NIL)
48> (|ncAlist| (ERROR . NOMORE))
<48 (|ncAlist| NIL)
48> (|ncTag| (ERROR . NOMORE))
<48 (|ncTag| ERROR)
<47 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<46 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
46> (|tokPart|

```

```

      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<46 (|tokPart| NOMORE)
<45 (|npFirstTok| NOMORE)
<44 (|npRestore| T)
<43 (|npDDInfKey| NIL)
<42 (|npInfGeneric| NIL)
<41 (|npRightAssoc| T)
<40 (|npQuiver| T)
40> (|npInfGeneric| (CASE HAS))
41> (|npDDInfKey| (CASE HAS))
42> (|npInfKey| (CASE HAS))
<42 (|npInfKey| NIL)
42> (|npState|)
<42 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
42> (|npEqKey| '|')
<42 (|npEqKey| NIL)
42> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
43> (|npFirstTok|)
44> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
45> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<45 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<44 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
44> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
45> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
46> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<46 (|poNoPosition?| NIL)
<45 (|pfNoPosition?| NIL)
45> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
46> (|ncAlist| (ERROR . NOMORE))
<46 (|ncAlist| NIL)
46> (|ncAlist| (ERROR . NOMORE))
<46 (|ncAlist| NIL)
46> (|ncTag| (ERROR . NOMORE))
<46 (|ncTag| ERROR)
<45 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<44 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
44> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<44 (|tokPart| NOMORE)

```

```

<43 (|npFirstTok| NOMORE)
<42 (|npRestore| T)
42> (|npEqKey| BACKQUOTE)
<42 (|npEqKey| NIL)
42> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
43> (|npFirstTok|)
44> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
45> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<45 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<44 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
44> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
45> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
46> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<46 (|poNoPosition?| NIL)
<45 (|pfNoPosition?| NIL)
45> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
46> (|ncAlist| (ERROR . NOMORE))
<46 (|ncAlist| NIL)
46> (|ncAlist| (ERROR . NOMORE))
<46 (|ncAlist| NIL)
46> (|ncTag| (ERROR . NOMORE))
<46 (|ncTag| ERROR)
<45 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<44 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
44> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<44 (|tokPart| NOMORE)
<43 (|npFirstTok| NOMORE)
<42 (|npRestore| T)
<41 (|npDDInfKey| NIL)
<40 (|npInfGeneric| NIL)
<39 (|npLeftAssoc| T)
<38 (|npDiscrim| T)
38> (|npInfGeneric| (AND))
39> (|npDDInfKey| (AND))
40> (|npInfKey| (AND))
<40 (|npInfKey| NIL)
40> (|npState|)
<40 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))

```

```

40> (|npEqKey| '|')
<40 (|npEqKey| NIL)
40> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
41> (|npFirstTok|)
42> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
43> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<43 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<42 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
42> (|tokConstruct| ERROR NOMORE ((0 "1" 1 1 "strings") . 0))
43> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
44> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<44 (|poNoPosition?| NIL)
<43 (|pfNoPosition?| NIL)
43> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
44> (|ncAlist| (ERROR . NOMORE))
<44 (|ncAlist| NIL)
44> (|ncAlist| (ERROR . NOMORE))
<44 (|ncAlist| NIL)
44> (|ncTag| (ERROR . NOMORE))
<44 (|ncTag| ERROR)
<43 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<42 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
42> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<42 (|tokPart| NOMORE)
<41 (|npFirstTok| NOMORE)
<40 (|npRestore| T)
40> (|npEqKey| BACKQUOTE)
<40 (|npEqKey| NIL)
40> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
41> (|npFirstTok|)
42> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
43> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<43 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<42 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
42> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
43> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))

```

```

44> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<44 (|poNoPosition?| NIL)
<43 (|pfNoPosition?| NIL)
43> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
44> (|ncAlist| (ERROR . NOMORE))
<44 (|ncAlist| NIL)
44> (|ncAlist| (ERROR . NOMORE))
<44 (|ncAlist| NIL)
44> (|ncTag| (ERROR . NOMORE))
<44 (|ncTag| ERROR)
<43 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<42 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
42> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<42 (|tokPart| NOMORE)
<41 (|npFirstTok| NOMORE)
<40 (|npRestore| T)
<39 (|npDDInfKey| NIL)
<38 (|npInfGeneric| NIL)
<37 (|npLeftAssoc| T)
<36 (|npDisjand| T)
36> (|npInfGeneric| (OR))
37> (|npDDInfKey| (OR))
38> (|npInfKey| (OR))
<38 (|npInfKey| NIL)
38> (|npState|)
<38 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
38> (|npEqKey| '|')
<38 (|npEqKey| NIL)
38> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
39> (|npFirstTok|)
40> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
41> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<41 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<40 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
40> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
41> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
42> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<42 (|poNoPosition?| NIL)

```

```

<41 (|pfNoPosition?| NIL)
41> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
42> (|ncAlist| (ERROR . NOMORE))
<42 (|ncAlist| NIL)
42> (|ncAlist| (ERROR . NOMORE))
<42 (|ncAlist| NIL)
42> (|ncTag| (ERROR . NOMORE))
<42 (|ncTag| ERROR)
41> (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<40 (|tokConstruct|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
40> (|tokPart|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<40 (|tokPart| NOMORE)
<39 (|npFirstTok| NOMORE)
<38 (|npRestore| T)
38> (|npEqKey| BACKQUOTE)
<38 (|npEqKey| NIL)
38> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
39> (|npFirstTok|)
40> (|tokPosn|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
41> (|ncAlist|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<41 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<40 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
40> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
41> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
42> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<42 (|poNoPosition?| NIL)
<41 (|pfNoPosition?| NIL)
41> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
42> (|ncAlist| (ERROR . NOMORE))
<42 (|ncAlist| NIL)
42> (|ncAlist| (ERROR . NOMORE))
<42 (|ncAlist| NIL)
42> (|ncTag| (ERROR . NOMORE))
<42 (|ncTag| ERROR)
41> (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<40 (|tokConstruct|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
40> (|tokPart|
      (ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<40 (|tokPart| NOMORE)

```

```

<39 (|npFirstTok| NOMORE)
<38 (|npRestore| T)
<37 (|npDDInfKey| NIL)
<36 (|npInfGeneric| NIL)
<35 (|npLeftAssoc| T)
<34 (|npLogical| T)
34> (|npInfGeneric| (BAR))
35> (|npDDInfKey| (BAR))
36> (|npInfKey| (BAR))
<36 (|npInfKey| NIL)
36> (|npState|)
<36 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
36> (|npEqKey| '|')
<36 (|npEqKey| NIL)
36> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
37> (|npFirstTok|)
38> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
39> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<39 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<38 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
38> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
39> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
40> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<40 (|poNoPosition?| NIL)
<39 (|pfNoPosition?| NIL)
39> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
40> (|ncAlist| (ERROR . NOMORE))
<40 (|ncAlist| NIL)
40> (|ncAlist| (ERROR . NOMORE))
<40 (|ncAlist| NIL)
40> (|ncTag| (ERROR . NOMORE))
<40 (|ncTag| ERROR)
<39 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<38 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
38> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<38 (|tokPart| NOMORE)
<37 (|npFirstTok| NOMORE)
<36 (|npRestore| T)

```

```

36> (|npEqKey| BACKQUOTE)
<36 (|npEqKey| NIL)
36> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
37> (|npFirstTok|)
38> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
39> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<39 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<38 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
38> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
39> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
40> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<40 (|poNoPosition?| NIL)
<39 (|pfNoPosition?| NIL)
39> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
40> (|ncAlist| (ERROR . NOMORE))
<40 (|ncAlist| NIL)
40> (|ncAlist| (ERROR . NOMORE))
<40 (|ncAlist| NIL)
40> (|ncTag| (ERROR . NOMORE))
<40 (|ncTag| ERROR)
<39 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<38 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
38> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<38 (|tokPart| NOMORE)
<37 (|npFirstTok| NOMORE)
<36 (|npRestore| T)
<35 (|npDDInfKey| NIL)
<34 (|npInfGeneric| NIL)
<33 (|npLeftAssoc| T)
<32 (|npSuch| T)
32> (|npInfGeneric| (IS ISNT))
33> (|npDDInfKey| (IS ISNT))
34> (|npInfKey| (IS ISNT))
<34 (|npInfKey| NIL)
34> (|npState|)
<34 (|npState|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1"))))
34> (|npEqKey| |'|)
<34 (|npEqKey| NIL)

```



```

34> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
35> (|npFirstTok|)
36> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
37> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<37 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<36 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
36> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
37> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
38> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))
<38 (|poNoPosition?| NIL)
<37 (|pfNoPosition?| NIL)
37> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
38> (|ncAlist| (ERROR . NOMORE))
<38 (|ncAlist| NIL)
38> (|ncAlist| (ERROR . NOMORE))
<38 (|ncAlist| NIL)
38> (|ncTag| (ERROR . NOMORE))
<38 (|ncTag| ERROR)
<37 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<36 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
36> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<36 (|tokPart| NOMORE)
<35 (|npFirstTok| NOMORE)
<34 (|npRestore| T)
34> (|npEqKey| BACKQUOTE)
<34 (|npEqKey| NIL)
34> (|npRestore|
      (NIL
        ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
          . "1")))
35> (|npFirstTok|)
36> (|tokPosn|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
37> (|ncAlist|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<37 (|ncAlist| ((|posn| (0 "1" 1 1 "strings") . 0)))
<36 (|tokPosn| ((0 "1" 1 1 "strings") . 0))
36> (|tokConstruct| ERROR NOMORE
      ((0 "1" 1 1 "strings") . 0))
37> (|pfNoPosition?| ((0 "1" 1 1 "strings") . 0))
38> (|poNoPosition?| ((0 "1" 1 1 "strings") . 0))

```

```

<38 (|poNoPosition?| NIL)
<37 (|pfNoPosition?| NIL)
37> (|ncPutQ| (ERROR . NOMORE) |posn|
      ((0 "1" 1 1 "strings") . 0))
38> (|ncAlist| (ERROR . NOMORE))
<38 (|ncAlist| NIL)
38> (|ncAlist| (ERROR . NOMORE))
<38 (|ncAlist| NIL)
38> (|ncTag| (ERROR . NOMORE))
<38 (|ncTag| ERROR)
<37 (|ncPutQ| ((0 "1" 1 1 "strings") . 0))
<36 (|tokConstruct|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
36> (|tokPart|
      ((ERROR (|posn| (0 "1" 1 1 "strings") . 0)) . NOMORE))
<36 (|tokPart| NOMORE)
<35 (|npFirstTok| NOMORE)
<34 (|npRestore| T)
<33 (|npDDInfKey| NIL)
<32 (|npInfGeneric| NIL)
<31 (|npLeftAssoc| T)
<30 (|npMatch| T)
30> (|npPop1|)
<30 (|npPop1|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
30> (|npWith|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
31> (|npEqKey| WITH)
<31 (|npEqKey| NIL)
<30 (|npWith| NIL)
30> (|npPush|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
<30 (|npPush|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
<29 (|npType|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
29> (|npPop1|)
<29 (|npPop1|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
29> (|npAdd|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
30> (|npEqKey| ADD)
<30 (|npEqKey| NIL)

```

```

<29 (|npAdd| NIL)
29> (|npPush|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
<29 (|npPush|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
<28 (|npADD|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
<27 (|npExpress1|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
27> (|npIterators|)
28> (|npForIn|)
29> (|npEqKey| FOR)
<29 (|npEqKey| NIL)
<28 (|npForIn| NIL)
28> (|npWhile|)
29> (|npAndOr| WHILE |npLogical| |pfWhile|)
30> (|npEqKey| WHILE)
<30 (|npEqKey| NIL)
<29 (|npAndOr| NIL)
<28 (|npWhile| NIL)
<27 (|npIterators| NIL)
<26 (|npExpress| T)
<25 (|npStatement| T)
25> (|npEqPeek| MDEF)
<25 (|npEqPeek| NIL)
<24 (|npBackTrack| T)
<23 (|npMDEF| T)
23> (|npEqPeek| BECOMES)
<23 (|npEqPeek| NIL)
<22 (|npBackTrack| T)
<21 (|npAssign| T)
21> (|npEqPeek| EXIT)
<21 (|npEqPeek| NIL)
<20 (|npBackTrack| T)
<19 (|npExit| T)
19> (|npEqPeek| GIVES)
<19 (|npEqPeek| NIL)
<18 (|npBackTrack| T)
<17 (|npGives| T)
17> (|npEqPeek| DEF)
<17 (|npEqPeek| NIL)
<16 (|npBackTrack| T)
<15 (|npDefinitionOrStatement| T)
15> (|npEqKey| WHERE)
<15 (|npEqKey| NIL)
<14 (|npQualified| T)

```

```

<13 (|npQualifiedDefinition| T)
13> (|npCommaBackSet|)
14> (|npEqKey| COMMA)
<14 (|npEqKey| NIL)
<13 (|npCommaBackSet| NIL)
<12 (|npListofFun| T)
<11 (|npTuple| T)
<10 (|npComma| T)
10> (|npPop1|)
<10 (|npPop1|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
10> (|npPush|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
<10 (|npPush|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
<9 (|npQualDef|
      ((((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))))
9> (|npEqKey| SEMICOLON)
<9 (|npEqKey| NIL)
9> (|npPop1|)
<9 (|npPop1|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
9> (|pfEnSequence|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
<9 (|pfEnSequence|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
9> (|npPush|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<9 (|npPush|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
<8 (|npItem|
      (((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1")))
<7 (|npParse|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<6 (|ncloopParse|
      ((((((#0=(0 "1" 1 1 "strings") . 1) . "1"))
        ((|integer| (|posn| #0# . 0)) . "1")))
      |nonnullstream| |incAppend1| NIL
      (|nonnullstream| |next1| |lineoftoks| (|nullstream|))))
6> (|next| |ncloopParse|
      (|nonnullstream| |incAppend1| NIL
      (|nonnullstream| |next1| |lineoftoks| (|nullstream|))))
7> (|Delay| #0=|next1|
      (|ncloopParse|
        (|nonnullstream| |incAppend1| NIL

```

```

(|nonnullstream| #0# |lineoftoks| (|nullstream|))))
<7 (|Delay|
(|nonnullstream| #0=|next1| |ncloopParse|
(|nonnullstream| |incAppend1| NIL
(|nonnullstream| #0# |lineoftoks| (|nullstream|))))
<6 (|next|
(|nonnullstream| #0=|next1| |ncloopParse|
(|nonnullstream| |incAppend1| NIL
(|nonnullstream| #0# |lineoftoks| (|nullstream|))))

6> (|incAppend|
((((#0=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #0# . 0)) . "1")))
(|nonnullstream| #1=|next1| |ncloopParse|
(|nonnullstream| |incAppend1| NIL
(|nonnullstream| #1# |lineoftoks| (|nullstream|))))
7> (|Delay| #0=|incAppend|
((((#2=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #2# . 0)) . "1")))
(|nonnullstream| #3=|next1| |ncloopParse|
(|nonnullstream| #0# NIL
(|nonnullstream| #3# |lineoftoks| (|nullstream|))))
<7 (|Delay|
(|nonnullstream| #0=|incAppend|
((((#2=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #2# . 0)) . "1")))
(|nonnullstream| #3=|next1| |ncloopParse|
(|nonnullstream| #0# NIL
(|nonnullstream| #3# |lineoftoks| (|nullstream|))))
<6 (|incAppend|
(|nonnullstream| #0=|incAppend|
((((#2=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #2# . 0)) . "1")))
(|nonnullstream| #3=|next1| |ncloopParse|
(|nonnullstream| #0# NIL
(|nonnullstream| #3# |lineoftoks| (|nullstream|))))
<5 (|next1|
(|nonnullstream| #0=|incAppend|
((((#2=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #2# . 0)) . "1")))
(|nonnullstream| #3=|next1| |ncloopParse|
(|nonnullstream| #0# NIL
(|nonnullstream| #3# |lineoftoks| (|nullstream|))))

5> (|incAppend|
((((#0=(0 "1" 1 1 "strings") . 1) . "1"))
((|integer| (|posn| #0# . 0)) . "1")))
(|nonnullstream| #1=|next1| |ncloopParse|
(|nonnullstream| |incAppend| NIL
(|nonnullstream| #1# |lineoftoks| (|nullstream|))))

```

```

6> (|StreamNull|
  (((((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    ((|integer| (|posn| #0# . 0)) . "1"))))
<6 (|StreamNull| NIL)
6> (|incAppend| NIL
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| |incAppend1| NIL
      (|nonnullstream| #0# |lineoftoks| (|nullstream|))))))
7> (|Delay| #0=|incAppend1|
  (NIL
    (|nonnullstream| #2=|next1| |ncloopParse|
      (|nonnullstream| #0# NIL
        (|nonnullstream| #2# |lineoftoks| (|nullstream|))))))
<7 (|Delay|
  (|nonnullstream| #0=|incAppend1| NIL
    (|nonnullstream| #2=|next1| |ncloopParse|
      (|nonnullstream| #0# NIL
        (|nonnullstream| #2# |lineoftoks| (|nullstream|))))))
<6 (|incAppend|
  (|nonnullstream| #0=|incAppend1| NIL
    (|nonnullstream| #2=|next1| |ncloopParse|
      (|nonnullstream| #0# NIL
        (|nonnullstream| #2# |lineoftoks| (|nullstream|))))))
<5 (|incAppend1|
  (((((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    ((|integer| (|posn| #0# . 0)) . "1"))
    (|nonnullstream| #1=|incAppend1| NIL
      (|nonnullstream| #3=|next1| |ncloopParse|
        (|nonnullstream| #1# NIL
          (|nonnullstream| #3# |lineoftoks| (|nullstream|))))))
  <4 (|StreamNull| NIL)

4> (|pfAbSynOp?|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1") |command|)
<4 (|pfAbSynOp?| NIL)

4> (|intloopSpadProcess| 1
  (((#0=(0 "1" 1 1 "strings") . 1) . "1"))
  ((|integer| (|posn| #0# . 0)) . "1") T)
5> (|incPutQ| (|carrier|) |stepNumber| 1)
6> (|incAlist| (|carrier|))
<6 (|incAlist| NIL)
6> (|incAlist| (|carrier|))
<6 (|incAlist| NIL)
6> (|incTag| (|carrier|))
<6 (|incTag| |carrier|)
<5 (|incPutQ| 1)
5> (|incPutQ| ((|carrier| (|stepNumber| . 1))) |messages| NIL)
6> (|incAlist| ((|carrier| (|stepNumber| . 1))))
<6 (|incAlist| ((|stepNumber| . 1)))

```

```

6> (|ncAlist| ((|carrier| (|stepNumber| . 1))))
<6 (|ncAlist| ((|stepNumber| . 1)))
6> (|ncTag| ((|carrier| (|stepNumber| . 1))))
<6 (|ncTag| |carrier|)
<5 (|ncPutQ| NIL)
5> (|ncPutQ|
  ((|carrier| (|messages|) (|stepNumber| . 1)))
  |lines| (((0 "1" 1 1 "strings") . 1) . "1")))
6> (|ncAlist| ((|carrier| (|messages|) (|stepNumber| . 1))))
<6 (|ncAlist| ((|messages|) (|stepNumber| . 1)))
6> (|ncAlist| ((|carrier| (|messages|) (|stepNumber| . 1))))
<6 (|ncAlist| ((|messages|) (|stepNumber| . 1)))
6> (|ncTag| ((|carrier| (|messages|) (|stepNumber| . 1))))
<6 (|ncTag| |carrier|)
<5 (|ncPutQ| (((0 "1" 1 1 "strings") . 1) . "1")))
5> (|intloopSpadProcess,interp|
  ((|carrier| (|lines| ((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1)))
  ((|integer| (|posn| #0# . 0)) . "1") T)
6> (|ncConversationPhase| |phParse|
  (((|carrier| (|lines| ((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1)))
  ((|integer| (|posn| #0# . 0)) . "1")))
7> (|phParse|
  ((|carrier| (|lines| ((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1)))
  ((|integer| (|posn| #0# . 0)) . "1"))
8> (|ncPutQ|
  ((|carrier| (|lines| ((#0=(0 "1" 1 1 "strings") . 1) . "1"))
    ) (|messages|) (|stepNumber| . 1)))
  |ptree| ((|integer| (|posn| #0# . 0)) . "1"))
9> (|ncAlist|
  ((|carrier| (|lines| (((0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|lines| (((0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1)))
9> (|ncAlist|
  ((|carrier| (|lines| (((0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|lines| (((0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1)))
9> (|ncTag|
  ((|carrier| (|lines| (((0 "1" 1 1 "strings") . 1) . "1"))
    (|messages|) (|stepNumber| . 1))))
<9 (|ncTag| |carrier|)
<8 (|ncPutQ|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<7 (|phParse| OK)

```

```

7> (|ncConversationPhase,wrapup|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
      . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<7 (|ncConversationPhase,wrapup| NIL)
<6 (|ncConversationPhase| OK)
6> (|ncConversationPhase| |phMacro|
  (((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0)) . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))))
7> (|phMacro|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0)) . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
8> (|ncEltQ|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0)) . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))) |ptree|)
9> (|ncAlist|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
      . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|ptree|
    (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
    . "1")
    (|lines| ((#0# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
<8 (|ncEltQ|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
8> (|ncPutQ|
  ((|carrier|

```



```

(|ptree| .
  #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
(|lines| ((#1# . 1) . "1"))
(|messages|)
(|stepNumber| . 1)))
|ptreePremacro| #0#)
9> (|ncAlist|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
        . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|ptree|
    (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
      . "1")
    (|lines| ((#0# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
9> (|ncAlist|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
        . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|ptree|
    (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0)) . "1")
    (|lines| ((#0# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
9> (|ncTag|
  ((|carrier|
    (|ptree|
      (|integer| (|posn| #0=(0 "1" 1 1 "strings") . 0))
        . "1")
      (|lines| ((#0# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncTag| |carrier|)
<8 (|ncPutQ|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
8> (|macroExpanded|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
9> (|macExpand|

```

```

      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
10> (|pfWhere?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |Where|)
<11 (|pfAbSynOp?| NIL)
<10 (|pfWhere?| NIL)
10> (|pfLambda?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |Lambda|)
<11 (|pfAbSynOp?| NIL)
<10 (|pfLambda?| NIL)
10> (|pfMacro?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |Macro|)
<11 (|pfAbSynOp?| NIL)
<10 (|pfMacro?| NIL)
10> (|pfId?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |id|)
<11 (|pfAbSynOp?| NIL)
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |idsyl|)
<11 (|pfAbSynOp?| NIL)
<10 (|pfId?| NIL)
10> (|pfApplication?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfAbSynOp?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1") |Application|)
<11 (|pfAbSynOp?| NIL)
<10 (|pfApplication?| NIL)
10> (|pfMapParts| |macExpand|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
        . "1"))
11> (|pfLeaf?|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))

```

```

      . "1"))
12> (|pfAbSynOp|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
       . "1"))
<12 (|pfAbSynOp| |integer|)
<11 (|pfLeaf?| (|integer| |Document| |error|))
<10 (|pfMapParts|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
       . "1"))
<9 (|macExpand|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<8 (|macroExpanded|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
8> (|ncPutQ|
      ((|carrier|
        (|ptreePremacro| .
          #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
              . "1"))
        (|ptree| . #0#)
        (|lines| ((#1# . 1) . "1"))
        (|messages|)
        (|stepNumber| . 1))) |ptree| #0#)
9> (|ncAlist|
      ((|carrier|
        (|ptreePremacro| .
          #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
              . "1"))
        (|ptree| . #0#)
        (|lines| ((#1# . 1) . "1"))
        (|messages|)
        (|stepNumber| . 1))))
<9 (|ncAlist|
      ((|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
            . "1"))
        (|ptree| . #0#)
        (|lines| ((#1# . 1) . "1"))
        (|messages|)
        (|stepNumber| . 1))))
<8 (|ncPutQ|
      ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
<7 (|phMacro| OK)
7> (|ncConversationPhase,wrapup|
      ((|carrier|
        (|ptreePremacro| .
          #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
              . "1"))
        (|ptree| . #0#)
        (|lines| ((#1# . 1) . "1"))
        (|messages|)

```

```

        (|stepNumber| . 1)))
<7 (|ncConversationPhase|wrapup| NIL)
<6 (|ncConversationPhase| OK)
6> (|ncConversationPhase| |phIntReportMsgs|
    ((|carrier|
      (|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))) T))
7> (|phIntReportMsgs|
    ((|carrier|
      (|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))) T))
8> (|ncEltQ|
    ((|carrier|
      (|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))) |lines|)
9> (|ncAlist|
    ((|carrier|
      (|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncAlist|
    ((|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1)))
<8 (|ncEltQ| (((0 "1" 1 1 "strings") . 1) . "1")))
8> (|ncEltQ|
    ((|carrier| (|ptreePremacro| .

```

```

#0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0)) .
    "1"))
(|ptree| . #0#)
(|lines| ((#1# . 1) . "1"))
(|messages|)
(|stepNumber| . 1)))
|messages|
9> (|ncAlist|
    ((|carrier|
      (|ptreePremacro| .
        #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
            . "1"))
        (|ptree| . #0#)
        (|lines| ((#1# . 1) . "1"))
        (|messages|)
        (|stepNumber| . 1))))
    <9 (|ncAlist|
        ((|ptreePremacro| .
          #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
              . "1"))
          (|ptree| . #0#)
          (|lines| ((#1# . 1) . "1"))
          (|messages|)
          (|stepNumber| . 1))))
    <8 (|ncEltQ| NIL)
    8> (|ncPutQ|
        ((|carrier|
          (|ptreePremacro| .
            #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
                . "1"))
            (|ptree| . #0#)
            (|lines| ((#1# . 1) . "1"))
            (|messages|)
            (|stepNumber| . 1))) |ok?| T)
    9> (|ncAlist|
        ((|carrier|
          (|ptreePremacro| .
            #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
                . "1"))
            (|ptree| . #0#)
            (|lines| ((#1# . 1) . "1"))
            (|messages|)
            (|stepNumber| . 1))))
    <9 (|ncAlist|
        ((|ptreePremacro| .
          #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
              . "1"))
          (|ptree| . #0#)
          (|lines| ((#1# . 1) . "1"))
          (|messages|)

```

```

(|stepNumber| . 1)))
9> (|ncAlist|
  (|carrier|
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<9 (|ncAlist|
  (|ptreePremacro| .
    #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
      . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
9> (|ncTag|
  (|carrier|
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<9 (|ncTag| |carrier|)
<8 (|ncPutQ| T)
<7 (|phIntReportMsgs| OK)
7> (|ncConversationPhase,wrapup|
  (|carrier|
    (|ok?| . T)
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<7 (|ncConversationPhase,wrapup| NIL)
<6 (|ncConversationPhase| OK)
6> (|ncConversationPhase| |phInterpret|
  (((|carrier|
    (|ok?| . T)
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))

```

```

(|messages|)
(|stepNumber| . 1))))
7> (|phInterpret|
  ((|carrier|
    (|ok?| . T)
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
8> (|ncEltQ|
  ((|carrier|
    (|ok?| . T)
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
  |ptree|)
9> (|ncAlist|
  ((|carrier|
    (|ok?| . T)
    (|ptreePremacro| .
      #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<9 (|ncAlist|
  ((|ok?| . T)
  (|ptreePremacro| .
    #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
      . "1"))
  (|ptree| . #0#)
  (|lines| ((#1# . 1) . "1"))
  (|messages|)
  (|stepNumber| . 1)))
<8 (|ncEltQ|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
8> (|intInterpretPform|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
9> (|pf2Sex|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
10> (|pf2Sex1|
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))

```

```

        . "1"))
11> (|pfNothing?|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
12> (|pfAbSynOp?|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1") |nothing|)
<12 (|pfAbSynOp?| NIL)
<11 (|pfNothing?| NIL)
11> (|pfSymbol?|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
12> (|pfAbSynOp?|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1") |symbol|)
<12 (|pfAbSynOp?| NIL)
<11 (|pfSymbol?| NIL)
11> (|pfLiteral?|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
12> (|pfAbSynOp|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<12 (|pfAbSynOp| |integer|)
<11 (|pfLiteral?|
    (|integer| |symbol| |expression| |one| |zero|
     |char| |string| |float|))
11> (|pfLiteral2Sex|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
12> (|pfLiteralClass|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
13> (|pfAbSynOp|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<13 (|pfAbSynOp| |integer|)
<12 (|pfLiteralClass| |integer|)
12> (|pfLiteralString|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
13> (|tokPart|
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<13 (|tokPart| "1")
<12 (|pfLiteralString| "1")
<11 (|pfLiteral2Sex| 1)
<10 (|pf2Sex1| 1)
<9 (|pf2Sex| 1)
9> (|zeroOneTran| 1)

```



```

<9 (|zeroOneTran| 1)
9> (|processInteractive| 1
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
10> (PUT |algebra| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |algebra| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |analysis| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |analysis| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |coercion| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |coercion| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |compilation| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |compilation| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |debug| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |debug| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |evaluation| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |evaluation| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |gc| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |gc| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |history| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |history| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |instantiation| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |instantiation| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |load| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |load| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |modemaps| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |modemaps| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |optimization| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |optimization| |SpaceTotal| 0)

```

```

<10 (PUT 0)
10> (PUT |querycoerce| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |querycoerce| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |other| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |other| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |diskread| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |diskread| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |print| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |print| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |resolve| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |resolve| |SpaceTotal| 0)
<10 (PUT 0)
10> (PUT |interpreter| |ClassTimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |interpreter| |ClassSpaceTotal| 0)
<10 (PUT 0)
10> (PUT |evaluation| |ClassTimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |evaluation| |ClassSpaceTotal| 0)
<10 (PUT 0)
10> (PUT |other| |ClassTimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |other| |ClassSpaceTotal| 0)
<10 (PUT 0)
10> (PUT |reclaim| |ClassTimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |reclaim| |ClassSpaceTotal| 0)
<10 (PUT 0)
10> (GETL |gc| |TimeTotal|)
<10 (GETL 0.0)
10> (PUT |gc| |TimeTotal| 0.050000000000000003)
<10 (PUT 0.050000000000000003)
10> (PUT |gc| |TimeTotal| 0.0)
<10 (PUT 0.0)
10> (PUT |gc| |SpaceTotal| 0)
<10 (PUT 0)
10> (|processInteractive| 1
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
11> (recordFrame |system|)
12> (|diffAlist| NIL NIL)
<12 (|diffAlist| NIL)

```

```

<11 (recordFrame NIL)
11> (GETL |other| |TimeTotal|)
<11 (GETL 0.0)
11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |other| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (|interpretTopLevel| 1
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
12> (|interpret| 1
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
13> (|interpret1| 1 NIL
  ((|integer| (|posn| (0 "1" 1 1 "strings") . 0)) . "1"))
14> (|member| 1 (|noBranch| |noMapVal|))
<14 (|member| NIL)
14> (|member| 1 (|nil| |true| |false|))
<14 (|member| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|isDomainValuedVariable| |--immediateData--|)
<14 (|isDomainValuedVariable| NIL)
14> (GETDATABASE |--immediateData--| CONSTRUCTOR)
<14 (GETDATABASE NIL)
14> (GETDATABASE |--immediateData--| ABBREVIATION)
<14 (GETDATABASE NIL)
14> (|member| |--immediateData--|
  (|Record| |Union| |Enumeration|))
<14 (|member| NIL)
14> (|getPropList| |--immediateData--| ((NIL)))
15> (|search| |--immediateData--| ((NIL)))
16> (|searchCurrentEnv| |--immediateData--| (NIL))
<16 (|searchCurrentEnv| NIL)
16> (|searchTailEnv| |--immediateData--| NIL)
<16 (|searchTailEnv| NIL)
<15 (|search| NIL)
15> (|search| |--immediateData--|
  ((((|Category|
    (|modemap| ((((|Category|) (|Category|)) (T *))))
    (|Join|
      (|modemap|
        ((((|Category|)
          (|Category|)
          (|Category|)
          (|Category|))
          (T *))
        ((((|Category|)
          (|Category|)
          (|List| (|Category|)))

```

```

(|Category|)
(T *))))))
16> (|searchCurrentEnv| |--immediateData--|
    (((|Category|
      (|modemap| (((|Category|) (|Category|)) (T *))))
    (|Join|
      (|modemap|
        (((|Category|
          (|Category|
            (|Category|
              (|Category|))
            (T *))
          (((|Category|
            (|Category|
              (|List| (|Category|))
              (|Category|))
            (T *))))))
<16 (|searchCurrentEnv| NIL)
16> (|searchTailEnv| |--immediateData--| NIL)
<16 (|searchTailEnv| NIL)
<15 (|search| NIL)
<14 (|getProplist| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|member| |--immediateData--| NIL)
<14 (|member| NIL)
14> (|interpret2|
    (#0=(|PositiveInteger|) . 1) #0#
    ((|integer| (|posn| (0 "1" 1 1 "strings") . 0))
     . "1"))
<14 (|interpret2| ((|PositiveInteger|) . 1))
<13 (|interpret1| ((|PositiveInteger|) . 1))
<12 (|interpret| ((|PositiveInteger|) . 1))
<11 (|interpretTopLevel| ((|PositiveInteger|) . 1))
11> (GETL |analysis| |TimeTotal|)
<11 (GETL 0.0)
11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |analysis| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (GETL |other| |TimeTotal|)
<11 (GETL 0.0)

```

```

11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |other| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (|recordAndPrint| 1 (|PositiveInteger|))

12> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (|member| (|PositiveInteger|
              ((|Mode|) (|Domain|) (|SubDomain| (|Domain|))))
<12 (|member| NIL)
12> (|member| (|PositiveInteger|
              ((|Category|) (|Mode|) (|Domain|)
               (|SubDomain| (|Domain|))))
<12 (|member| NIL)
12> (GETL |print| |TimeTotal|)
<12 (GETL 0.0)
12> (GETL |gc| |TimeTotal|)
<12 (GETL 0.0)
12> (PUT |gc| |TimeTotal| 0.0)
<12 (PUT 0.0)
12> (PUT |print| |TimeTotal| 0.0)
<12 (PUT 0.0)
12> (|isEqualOrSubDomain| (|PositiveInteger|
                          (|OutputForm|))
<12 (|isEqualOrSubDomain| NIL)
12> (GETDATABASE |OutputForm| ABBREVIATION)
<12 (GETDATABASE OUTFORM)
12> (HPUT #<hash-table 0000000001ab1e40>
        (|OutputForm|) (1))
<12 (HPUT (1))
12> (HPUT #<hash-table 0000000001ab1ea0>
        (NIL NIL NIL) (1 . T))
<12 (HPUT (1 . T))
12> (HPUT #<hash-table 0000000001ab1ea0>
        (#0=(|OutputForm|) NIL NIL) (1 . #0#))
<12 (HPUT (1 |OutputForm|))
12> (HPUT #<hash-table 0000000001ab1e40>
        (|PositiveInteger|) (1))
<12 (HPUT (1))
12> (|member| (|OutputForm|) ((|Integer|) (|OutputForm|)))
<12 (|member| ((|OutputForm|)))
12> (|member| (|OutputForm|
              ((|Mode|) (|Domain|) (|SubDomain| (|Domain|))))
<12 (|member| NIL)
12> (GETDATABASE |OutputForm| ABBREVIATION)
<12 (GETDATABASE OUTFORM)
12> (GETDATABASE |OutputForm| COSIG)

```

```

<12 (GETDATABASE (NIL))
12> (HPUT #<hash-table 000000001ab1840>
      (|OutputForm|) (1 . T))
<12 (HPUT (1 . T))
12> (|isPartialModel| (|OutputForm|))
<12 (|isPartialModel| NIL)
12> (|member| |coerce| (= + * -))
<12 (|member| NIL)
12> (|isPartialModel| (|OutputForm|))
<12 (|isPartialModel| NIL)
12> (|member| |PositiveInteger|
      (|List| |Vector| |Stream| |FiniteSet| |Array|))
<12 (|member| NIL)
12> (|member| |PositiveInteger|
      (|Union| |Record| |Mapping| |Enumeration|))
<12 (|member| NIL)
12> (GETDATABASE |PositiveInteger| OPERATIONALIST)
<12 (GETDATABASE
      ((~= (((|Boolean|) $ $) NIL))
        (|sample| ((| $) NIL T CONST))
        (|recip| (((|Union| $ "failed") $) NIL))
        (|one?| (((|Boolean|) $) NIL))
        (|min| (($ $ $) NIL))
        (|max| (($ $ $) NIL))
        (|latex| (((|String|) $) NIL))
        (|hash| (((|SingleInteger|) $) NIL))
        (|gcd| (($ $ $) NIL))
        (|coerce| (((|OutputForm|) $) NIL))
        (^ (($ $ (|NonNegativeInteger|)) NIL)
          (($ $ (|PositiveInteger|)) NIL))
        (|One| (($) NIL T CONST))
        (>= (((|Boolean|) $ $) NIL))
        (> (((|Boolean|) $ $) NIL))
        (= (((|Boolean|) $ $) NIL))
        (<= (((|Boolean|) $ $) NIL))
        (< (((|Boolean|) $ $) NIL))
        (+ (($ $ $) NIL))
        (** (($ $ (|NonNegativeInteger|)) NIL)
          (($ $ (|PositiveInteger|)) NIL))
        (* (($ (|PositiveInteger|) $) NIL) (($ $ $) NIL))))
12> (|constructSubst| (|PositiveInteger|))
<12 (|constructSubst| (($ |PositiveInteger|))
12> (|isEqualOrSubDomain| #0=(|PositiveInteger|) #0#)
<12 (|isEqualOrSubDomain| T)
12> (|isEqualOrSubDomain| (|OutputForm|) (|OutputForm|))
<12 (|isEqualOrSubDomain| T)
12> (|member| |OutputForm| (|Union| |Record| |Mapping| |Enumeration|))
<12 (|member| NIL)
12> (GETDATABASE |OutputForm| OPERATIONALIST)
<12 (GETDATABASE

```

```

((~ = (((|Boolean|) $ $) NIL))
(|zag| (($ $ $) 120))
(|width| (((|Integer|) $) 30) (((|Integer|)) 35))
(|vspace| (($ (|Integer|)) 47))
(|vconcat| (($ $ $) 48) (($ (|List| $)) 80))
(|supersub| (($ $ (|List| $)) 78))
(|superHeight| (((|Integer|) $) 33))
(|super| (($ $ $) 66))
(|sum| (($ $) 135) (($ $ $) 136) (($ $ $ $) 137))
(|subHeight| (((|Integer|) $) 32))
(|sub| (($ $ $) 65))
(|string| (($ $) 109))
(|slash| (($ $ $) 124))
(|semicolonSeparate| (($ (|List| $)) 55))
(|scripts| (($ $ (|List| $)) 72))
(|rspace| (($ (|Integer|) (|Integer|)) 49))
(|root| (($ $) 121) (($ $ $) 122))
(|right| (($ $ (|Integer|)) 42) (($ $) 45))
(|rem| (($ $ $) 93))
(|rarrow| (($ $ $) 127))
(|quote| (($ $) 110))
(|quo| (($ $ $) 94))
(|prod| (($ $) 138) (($ $ $) 139) (($ $ $ $) 140))
(|print| (((|Void|) $) 8))
(|prime| (($ $) 113) (($ $ (|NonNegativeInteger|)) 117))
(|presuper| (($ $ $) 68))
(|presub| (($ $ $) 67))
(|prefix| (($ $ (|List| $)) 104))
(|postfix| (($ $ $) 108))
(|pile| (($ (|List| $)) 53))
(|paren| (($ $) 63) (($ (|List| $)) 64))
(|overlabel| (($ $ $) 118))
(|overbar| (($ $) 111))
(|over| (($ $ $) 123))
(|outputForm| (($ (|Integer|)) 20)
              (($ (|Symbol|)) 22)
              (($ (|String|)) 29)
              (($ (|DoubleFloat|)) 24))
(|or| (($ $ $) 97))
(|not| (($ $) 98))
(|messagePrint| (((|Void|) (|String|)) 14))
(|message| (($ (|String|)) 13))
(|matrix| (($ (|List| (|List| $))) 51))
(|left| (($ $ (|Integer|)) 41) (($ $) 44))
(|latex| (((|String|) $) NIL))
(|label| (($ $ $) 126))
(|int| (($ $) 141) (($ $ $) 142) (($ $ $ $) 143))
(|infix?| (((|Boolean|) $) 102))
(|infix| (($ $ (|List| $)) 106) (($ $ $ $) 107))
(|hspace| (($ (|Integer|)) 38))

```

```

(|height| (((|Integer|) $) 31) (((|Integer|)) 34))
(|hconcat| (($ $ $) 39) (($ (|List| $)) 79))
(|hash| (((|SingleInteger|) $) NIL))
(|exquo| (($ $ $) 95))
(|empty| (($) 12))
(|elt| (($ $ (|List| $)) 103))
(|dot| (($ $) 112)
  (($ $ (|NonNegativeInteger|)) 116))
(|div| (($ $ $) 92))
(|differentiate| (($ $ (|NonNegativeInteger|)) 134))
(|commaSeparate| (($ (|List| $)) 54))
(|coerce| (((|OutputForm|) $) 18))
(|center| (($ $ (|Integer|)) 40) (($ $) 43))
(|bracket| (($ $) 61) (($ (|List| $)) 62))
(|brace| (($ $) 59) (($ (|List| $)) 60))
(|box| (($ $) 119))
(|blankSeparate| (($ (|List| $)) 58))
(|binomial| (($ $ $) 101))
(|assign| (($ $ $) 125))
(|and| (($ $ $) 96))
(^= (($ $ $) 81))
(SEGMENT (($ $ $) 99) (($ $) 100))
(>= (($ $ $) 85))
(> (($ $ $) 83))
(= (((|Boolean|) $ $) 15) (($ $ $) 16))
(<= (($ $ $) 84))
(< (($ $ $) 82))
(/ (($ $ $) 90))
(- (($ $ $) 87) (($ $) 88))
(+ (($ $ $) 86))
(** (($ $ $) 91))
(* (($ $ $) 89)))
12> (|constructSubst| (|OutputForm|))
<12 (|constructSubst| (($ |OutputForm|)))
12> (|isEqualOrSubDomain|
  (|PositiveInteger|) (|OutputForm|))
<12 (|isEqualOrSubDomain| NIL)
12> (HPUT #<hash-table 0000000001ab1e70>
  (|coerce| (|OutputForm|) (#0=(|PositiveInteger|))
  (#0#) NIL) (1 ((#0# #1=(|OutputForm|) #0#)
  (#1# $) (NIL))))
<12 (HPUT (1 ((#0=(|PositiveInteger|)
  #1=(|OutputForm|) #0#) (#1# $) (NIL))))
12> (HPUT #<hash-table 0000000001ab1f00>
  (|coerce| #0=(|PositiveInteger|) (|OutputForm|))
  (1 (#0# #1=(|OutputForm|) #0#) (#1# $) (NIL)))
<12 (HPUT (1 (#0=(|PositiveInteger|)
  #1=(|OutputForm|) #0#) (#1# $) (NIL)))
12> (|evalDomain| (|PositiveInteger|))
13> (GETL |print| |TimeTotal|)

```



```

<13 (GETL 0.0)
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)
13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (PUT |print| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (|mkEvalable| (|PositiveInteger|))
14> (CANFUNCALL? |PositiveInteger|)
<14 (CANFUNCALL? T)
14> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<14 (GETDATABASE |domain|)
14> (GETDATABASE |PositiveInteger| COSIG)
<14 (GETDATABASE (NIL))
<13 (|mkEvalable| (|PositiveInteger|))
13> (GETL |PositiveInteger| LOADED)
<13 (GETL NIL)
13> (|loadLib| |PositiveInteger|)
14> (GETL |instantiation| |TimeTotal|)
<14 (GETL 0.0)
14> (GETL |gc| |TimeTotal|)
<14 (GETL 0.0)
14> (PUT |gc| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (PUT |instantiation| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (GETDATABASE |PositiveInteger| OBJECT)
<14 (GETDATABASE
    "/home/daly/noise/mnt/ubuntu/algebra/PI.o")
14> (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/PI.o")
15> (|pathname|
    "/home/daly/noise/mnt/ubuntu/algebra/PI.o")
<15 (|pathname|
    #p"/home/daly/noise/mnt/ubuntu/algebra/PI.o")
<14 (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
14> (|isSystemDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
<14 (|isSystemDirectory| T)
14> (|loadLibNoUpdate| |PositiveInteger| |PositiveInteger|
    "/home/daly/noise/mnt/ubuntu/algebra/PI.o")
15> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (|getProplist| |NonNegativeInteger|
    ((((|Category| (|modemap|
        (((|Category|) (|Category|)) (T *))))
        (|Join| (|modemap|
            (((|Category|) (|Category|) (|Category|)
                (|Category|))

```

```

      (T *))
      (((|Category|) (|Category|) (|List| (|Category|))
        (|Category|)) (T *))))))
16> (|search| |NonNegativeInteger|
    ((((|Category| (|modemap|
      (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|))
          (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
17> (|searchCurrentEnv| |NonNegativeInteger|
    ((((|Category| (|modemap|
      (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|))
          (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
<17 (|searchCurrentEnv| NIL)
17> (|searchTailEnv| |NonNegativeInteger| NIL)
<17 (|searchTailEnv| NIL)
<16 (|search| NIL)
16> (|search| |NonNegativeInteger|
    ((((|Category| (|modemap|
      (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|))
          (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
17> (|searchCurrentEnv| |NonNegativeInteger|
    ((((|Category| (|modemap|
      (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|)) (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
<17 (|searchCurrentEnv| NIL)
17> (|searchTailEnv| |NonNegativeInteger| NIL)
<17 (|searchTailEnv| NIL)
<16 (|search| NIL)
<15 (|getProplist| NIL)
15> (|addBinding| |NonNegativeInteger|
    ((|SubDomain|
      (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))

```

```

((((|Category| (|modemap|
  (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))
16> (|getPropList| |NonNegativeInteger|
  ((((|Category| (|modemap|
    (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))
17> (|search| |NonNegativeInteger|
  ((((|Category| (|modemap|
    (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))
18> (|searchCurrentEnv| |NonNegativeInteger|
  ((((|Category| (|modemap|
    (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))
<18 (|searchCurrentEnv| NIL)
18> (|searchTailEnv| |NonNegativeInteger| NIL)
<18 (|searchTailEnv| NIL)
<17 (|search| NIL)
17> (|search| |NonNegativeInteger|
  ((((|Category| (|modemap|
    (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))
18> (|searchCurrentEnv| |NonNegativeInteger|
  ((((|Category| (|modemap|
    (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *)))))))))

```

```

(|Category|) (T *))))))
<18 (|searchCurrentEnv| NIL)
18> (|searchTailEnv| |NonNegativeInteger| NIL)
<18 (|searchTailEnv| NIL)
<17 (|search| NIL)
<16 (|getProplist| NIL)
16> (|addBindingInteractive| |NonNegativeInteger|
(|SubDomain|
(|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(((|Category| (|modemap|
(|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
(|Category|) (|Category|) (|Category|)
(|Category|) (T *)))
(|Category|) (|Category|) (|List| (|Category|)
(|Category|) (T *))))))
<16 (|addBindingInteractive|
(|NonNegativeInteger|
(|SubDomain|
(|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
(|Category|) (|Category|)) (T *)))
(|Join| (|modemap|
(|Category|) (|Category|) (|Category|)
(|Category|) (T *)))
(|Category|) (|Category|) (|List| (|Category|)
(|Category|) (T *))))))
<15 (|addBinding|
(|NonNegativeInteger|
(|SubDomain|
(|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
(|Category|) (|Category|)) (T *)))
(|Join| (|modemap|
(|Category|) (|Category|) (|Category|)
(|Category|) (T *)))
(|Category|) (|Category|) (|List| (|Category|)
(|Category|) (T *))))))
15> (|getProplist| |PositiveInteger|
(|NonNegativeInteger|
(|SubDomain|
(|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
(|Category|) (|Category|)) (T *)))
(|Join| (|modemap|
(|Category|) (|Category|) (|Category|)
(|Category|) (T *)))
(|Category|) (|Category|) (|List| (|Category|)
(|Category|) (T *))))))
16> (|search| |PositiveInteger|

```

```

((((|NonNegativeInteger|
  (|SubDomain|
    (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
  (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *))))))
17> (|searchCurrentEnv| |PositiveInteger|
  (((|NonNegativeInteger|
    (|SubDomain|
      (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
  (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *))))))
<17 (|searchCurrentEnv| NIL)
17> (|searchTailEnv| |PositiveInteger| NIL)
<17 (|searchTailEnv| NIL)
<16 (|search| NIL)
16> (|search| |PositiveInteger|
  (((|NonNegativeInteger|
    (|SubDomain|
      (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
  (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *))))))
17> (|searchCurrentEnv| |PositiveInteger|
  (((|NonNegativeInteger|
    (|SubDomain|
      (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
  (((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
  (((|Category|) (|Category|) (|Category|)
    (|Category|)) (T *))
  (((|Category|) (|Category|) (|List| (|Category|))
    (|Category|)) (T *))))))
<17 (|searchCurrentEnv| NIL)
17> (|searchTailEnv| |PositiveInteger| NIL)
<17 (|searchTailEnv| NIL)

```

```

<16 (|search| NIL)
<15 (|getProplist| NIL)
15> (|addBinding| |PositiveInteger|
    ((|SuperDomain| |NonNegativeInteger|))
    (((|NonNegativeInteger|
      (|SubDomain|
        (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
      (|Category| (|modemap|
        (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|)) (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
16> (|getProplist| |PositiveInteger|
    (((|NonNegativeInteger|
      (|SubDomain|
        (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
      (|Category| (|modemap|
        (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|)) (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
17> (|search| |PositiveInteger|
    (((|NonNegativeInteger|
      (|SubDomain|
        (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
      (|Category| (|modemap|
        (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|)) (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
18> (|searchCurrentEnv| |PositiveInteger|
    (((|NonNegativeInteger|
      (|SubDomain|
        (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
      (|Category| (|modemap|
        (((|Category|) (|Category|)) (T *))))
      (|Join| (|modemap|
        (((|Category|) (|Category|) (|Category|)
          (|Category|)) (T *))
        (((|Category|) (|Category|) (|List| (|Category|))
          (|Category|)) (T *)))))))
<18 (|searchCurrentEnv| NIL)
18> (|searchTailEnv| |PositiveInteger| NIL)
<18 (|searchTailEnv| NIL)

```

```

<17 (|search| NIL)
17> (|search| |PositiveInteger|
    ((((|NonNegativeInteger|
        (|SubDomain|
          (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
        (|Category| (|modemap|
          (((|Category|) (|Category|)) (T *))))
        (|Join| (|modemap|
          (((|Category|) (|Category|) (|Category|)
            (|Category|)) (T *))
          (((|Category|) (|Category|) (|List| (|Category|))
            (|Category|)) (T *)))))))
18> (|searchCurrentEnv| |PositiveInteger|
    ((((|NonNegativeInteger|
        (|SubDomain|
          (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
        (|Category| (|modemap|
          (((|Category|) (|Category|)) (T *))))
        (|Join| (|modemap|
          (((|Category|) (|Category|) (|Category|)
            (|Category|)) (T *))
          (((|Category|) (|Category|) (|List| (|Category|))
            (|Category|)) (T *)))))))
<18 (|searchCurrentEnv| NIL)
18> (|searchTailEnv| |PositiveInteger| NIL)
<18 (|searchTailEnv| NIL)
<17 (|search| NIL)
<16 (|getProplist| NIL)
16> (|addBindingInteractive| |PositiveInteger|
    ((|SuperDomain| |NonNegativeInteger|))
    ((((|NonNegativeInteger|
        (|SubDomain|
          (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
        (|Category| (|modemap|
          (((|Category|) (|Category|)) (T *))))
        (|Join| (|modemap|
          (((|Category|) (|Category|) (|Category|)
            (|Category|)) (T *))
          (((|Category|) (|Category|) (|List| (|Category|))
            (|Category|)) (T *)))))))
<16 (|addBindingInteractive|
    ((((|PositiveInteger|
        (|SuperDomain| |NonNegativeInteger|))
        (|NonNegativeInteger|
          (|SubDomain|
            (|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
        (|Category| (|modemap|
          (((|Category|) (|Category|)) (T *))))
        (|Join| (|modemap|
          (((|Category|) (|Category|) (|Category|)
            (|Category|)) (T *)))))))

```

```

(|Category|)) (T *))
(((|Category|) (|Category|) (|List| (|Category|))
(|Category|)) (T *))))))
<15 (|addBinding|
(((|PositiveInteger|
(|SuperDomain| |NonNegativeInteger|))
(|NonNegativeInteger|
(|SubDomain|
(|PositiveInteger| SPADCALL 0 |#1| (QREFELT $ 7))))
(|Category| (|modemap|
(((|Category|) (|Category|)) (T *))))
(|Join| (|modemap|
(((|Category|) (|Category|) (|Category|)
(|Category|)) (T *))
(((|Category|) (|Category|) (|List| (|Category|))
(|Category|)) (T *))))))
15> (|makeByteWordVec2| 1 (0 0 0 0 0 0))
<15 (|makeByteWordVec2| #<bit-vector 0000000001ab1db0>)
15> (|makeByteWordVec2| 12 (2 5 6 0 0 7 2 0 6 0 0 1 0 0 0
1 1 0 9 0 1 1 0 6 0 1 2 0 0 0 0 1 2 0 0 0 0 1 1 0 11
0 1 1 0 10 0 1 2 0 0 0 0 1 1 0 12 0 1 2 0 0 0 8 1 2
0 0 0 5 1 0 0 0 1 2 0 6 0 0 1 2 0 6 0 0 1 2 0 6 0 0
1 2 0 6 0 0 1 2 0 6 0 0 1 2 0 0 0 0 1 2 0 0 0 8 1 2
0 0 0 5 1 2 0 0 0 0 1 2 0 0 8 0 1))
<15 (|makeByteWordVec2| #<vector 0000000001ab1d80>)
15> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (GETL |load| |TimeTotal|)
<15 (GETL 0.0)
15> (GETL |gc| |TimeTotal|)
<15 (GETL 0.0)
15> (PUT |gc| |TimeTotal| 0.0)
<15 (PUT 0.0)
15> (PUT |load| |TimeTotal| 0.0)
<15 (PUT 0.0)
<14 (|loadLibNoUpdate| T)
<13 (|loadLib| T)
13> (HPUT #<hash-table 000000000105e810> |PositiveInteger|
((NIL 1 . #<vector 0000000001ab1d50>)))
<13 (HPUT ((NIL 1 . #<vector 0000000001ab1d50>)))
13> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<13 (GETDATABASE |domain|)
13> (GETL |PositiveInteger| |infovec|)
<13 (GETL (#<vector 0000000000fa5db0>
#<vector 0000000000fa5cf0>
(((|commutative| "*" ) . 0))
(#<bit-vector 0000000001ab1db0>
#<vector 0000000000fa5c90>
#<vector 0000000001ab1de0>
. #<vector 0000000001ab1d80>) |lookupComplete|))

```



```

13> (HPUT #<hash-table 000000000105e810> |PositiveInteger|
      ((NIL 1 . #<vector 0000000001ab1d50>)))
<13 (HPUT ((NIL 1 . #<vector 0000000001ab1d50>)))
13> (GETL |instantiation| |TimeTotal|)
<13 (GETL 0.0)
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)
13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (PUT |instantiation| |TimeTotal| 0.0)
<13 (PUT 0.0)
<12 (|evalDomain| #<vector 0000000001ab1d50>)
12> (|compiledLookup| |coerce| ((|OutputForm|) $)
      #<vector 0000000001ab1d50>)
13> (|NREvalDomain| #<vector 0000000001ab1d50>)
14> (|evalDomain| #<vector 0000000001ab1d50>)
15> (GETL |print| |TimeTotal|)
<15 (GETL 0.0)
15> (GETL |gc| |TimeTotal|)
<15 (GETL 0.0)
15> (PUT |gc| |TimeTotal| 0.0)
<15 (PUT 0.0)
15> (PUT |print| |TimeTotal| 0.0)
<15 (PUT 0.0)
15> (|mkEvalable| #<vector 0000000001ab1d50>)
<15 (|mkEvalable| #<vector 0000000001ab1d50>)
15> (GETL |instantiation| |TimeTotal|)
<15 (GETL 0.0)
15> (GETL |gc| |TimeTotal|)
<15 (GETL 0.0)
15> (PUT |gc| |TimeTotal| 0.0)
<15 (PUT 0.0)
15> (PUT |instantiation| |TimeTotal| 0.0)
<15 (PUT 0.0)
<14 (|evalDomain| #<vector 0000000001ab1d50>)
<13 (|NREvalDomain| #<vector 0000000001ab1d50>)
13> (|basicLookup| |coerce| ((|OutputForm|) $)
      #<vector 0000000001ab1d50> #<vector 0000000001ab1d50>)
14> (|oldCompLookup| |coerce| ((|OutputForm|) $)
      #<vector 0000000001ab1d50> #<vector 0000000001ab1d50>)
15> (|lookupInDomainVector| |coerce| ((|OutputForm|) $)
      #<vector 0000000001ab1d50> #<vector 0000000001ab1d50>)
16> (GETDATABASE |OutputForm| COSIG)
<16 (GETDATABASE (NIL))
16> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<16 (GETDATABASE |domain|)
16> (GETL |NonNegativeInteger| LOADED)
<16 (GETL NIL)
16> (|loadLib| |NonNegativeInteger|)
17> (GETL |print| |TimeTotal|)

```

```

<17 (GETL 0.0)
17> (GETL |gc| |TimeTotal|)
<17 (GETL 0.0)
17> (PUT |gc| |TimeTotal| 0.0)
<17 (PUT 0.0)
17> (PUT |print| |TimeTotal| 0.0)
<17 (PUT 0.0)
17> (GETDATABASE |NonNegativeInteger| OBJECT)
<17 (GETDATABASE
      "/home/daly/noise/mnt/ubuntu/algebra/NNI.o")
17> (|pathnameDirectory|
      "/home/daly/noise/mnt/ubuntu/algebra/NNI.o")
18> (|pathname|
      "/home/daly/noise/mnt/ubuntu/algebra/NNI.o")
<18 (|pathname|
      #p"/home/daly/noise/mnt/ubuntu/algebra/NNI.o")
<17 (|pathnameDirectory|
      "/home/daly/noise/mnt/ubuntu/algebra/")
17> (|isSystemDirectory|
      "/home/daly/noise/mnt/ubuntu/algebra/")
<17 (|isSystemDirectory| T)
17> (|loadLibNoUpdate| |NonNegativeInteger|
      |NonNegativeInteger|
      "/home/daly/noise/mnt/ubuntu/algebra/NNI.o")
18> (GETDATABASE |NonNegativeInteger| CONSTRUCTORKIND)
<18 (GETDATABASE |domain|)
18> (|getProplist| |Integer| ((NIL)))
19> (|search| |Integer| ((NIL)))
20> (|searchCurrentEnv| |Integer| (NIL))
<20 (|searchCurrentEnv| NIL)
20> (|searchTailEnv| |Integer| NIL)
<20 (|searchTailEnv| NIL)
<19 (|search| NIL)
19> (|search| |Integer| ((NIL)))
20> (|searchCurrentEnv| |Integer| (NIL))
<20 (|searchCurrentEnv| NIL)
20> (|searchTailEnv| |Integer| NIL)
<20 (|searchTailEnv| NIL)
<19 (|search| NIL)
<18 (|getProplist| NIL)
18> (|addBinding| |Integer|
      ((|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))) ((NIL)))
19> (|getProplist| |Integer| ((NIL)))
20> (|search| |Integer| ((NIL)))
21> (|searchCurrentEnv| |Integer| (NIL))
<21 (|searchCurrentEnv| NIL)

```

```

21> (|searchTailEnv| |Integer| NIL)
<21 (|searchTailEnv| NIL)
<20 (|search| NIL)
20> (|search| |Integer| ((NIL)))
21> (|searchCurrentEnv| |Integer| (NIL))
<21 (|searchCurrentEnv| NIL)
21> (|searchTailEnv| |Integer| NIL)
<21 (|searchTailEnv| NIL)
<20 (|search| NIL)
<19 (|getProplist| NIL)
19> (|addBindingInteractive| |Integer|
    (|SubDomain|
      (|NonNegativeInteger|
        COND
          ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
          ((QUOTE T) (QUOTE T)))))) ((NIL)))
<19 (|addBindingInteractive|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T))))))))))
<18 (|addBinding|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T))))))))))
18> (|getProplist| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T))))))))))
19> (|search| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T))))))))))
20> (|searchCurrentEnv| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))

```

```

      ((QUOTE T) (QUOTE T))))))
<20 (|searchCurrentEnv| NIL)
20> (|searchTailEnv| |NonNegativeInteger| NIL)
<20 (|searchTailEnv| NIL)
<19 (|search| NIL)
19> (|search| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
20> (|searchCurrentEnv| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
<20 (|searchCurrentEnv| NIL)
20> (|searchTailEnv| |NonNegativeInteger| NIL)
<20 (|searchTailEnv| NIL)
<19 (|search| NIL)
<18 (|getProplist| NIL)
18> (|addBinding| |NonNegativeInteger|
    ((|SuperDomain| |Integer|))
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
19> (|getProplist| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
20> (|search| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
21> (|searchCurrentEnv| |NonNegativeInteger|
    ((((|Integer|
      (|SubDomain|
        (|NonNegativeInteger|

```

```

COND
  ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
  ((QUOTE T) (QUOTE T))))))
<21 (|searchCurrentEnv| NIL)
21> (|searchTailEnv| |NonNegativeInteger| NIL)
<21 (|searchTailEnv| NIL)
<20 (|search| NIL)
20> (|search| |NonNegativeInteger|
  ((((|Integer|
    (|SubDomain|
      (|NonNegativeInteger|
        COND
          ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
          ((QUOTE T) (QUOTE T)))))))))
21> (|searchCurrentEnv| |NonNegativeInteger|
  ((((|Integer|
    (|SubDomain|
      (|NonNegativeInteger|
        COND
          ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
          ((QUOTE T) (QUOTE T)))))))))
<21 (|searchCurrentEnv| NIL)
21> (|searchTailEnv| |NonNegativeInteger| NIL)
<21 (|searchTailEnv| NIL)
<20 (|search| NIL)
<19 (|getPropList| NIL)
19> (|addBindingInteractive| |NonNegativeInteger|
  ((|SuperDomain| |Integer|))
  ((((|Integer|
    (|SubDomain|
      (|NonNegativeInteger|
        COND
          ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
          ((QUOTE T) (QUOTE T)))))))))
<19 (|addBindingInteractive|
  ((((|NonNegativeInteger| (|SuperDomain| |Integer|))
    (|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))
<18 (|addBinding|
  ((((|NonNegativeInteger| (|SuperDomain| |Integer|))
    (|Integer|
      (|SubDomain|
        (|NonNegativeInteger|
          COND
            ((SPADCALL |#1| 0 (QREFELT $ 7)) (QUOTE NIL))
            ((QUOTE T) (QUOTE T)))))))))

```

```

18> (|makeByteWordVec2| 1 (0 0 0 0 0 0 0 0 0 0 0 0))
<18 (|makeByteWordVec2| #<bit-vector 0000000001ab1990>)
18> (|makeByteWordVec2| 18 (2 5 6 0 0 7 2 5 0 0 0 10 2 0 6
    0 0 1 1 0 6 0 1 2 0 0 0 0 8 2 0 11 0 0 12 2 0 0 0 5
    9 0 0 0 1 2 0 0 0 0 1 1 0 11 0 1 1 0 0 0 1 2 0 0 0
    0 1 1 0 6 0 1 2 0 0 0 0 1 2 0 0 0 0 1 1 0 17 0 1 1
    0 16 0 1 2 0 0 0 0 1 2 0 11 0 0 1 2 0 13 0 0 1 1 0
    18 0 1 2 0 0 0 14 1 2 0 0 0 15 1 0 0 0 1 0 0 0 1 2
    0 6 0 0 1 2 0 6 0 0 1 2 0 6 0 0 1 2 0 6 0 0 1 2 0
    6 0 0 1 2 0 0 0 0 1 2 0 0 0 14 1 2 0 0 0 15 1 2 0
    0 0 0 1 2 0 0 14 0 1 2 0 0 15 0 1))
<18 (|makeByteWordVec2| #<vector 0000000001ab1960>)
18> (GETDATABASE |NonNegativeInteger| CONSTRUCTORKIND)
<18 (GETDATABASE |domain|)
18> (GETL |load| |TimeTotal|)
<18 (GETL 0.0)
18> (GETL |gc| |TimeTotal|)
<18 (GETL 0.0)
18> (PUT |gc| |TimeTotal| 0.0)
<18 (PUT 0.0)
18> (PUT |load| |TimeTotal| 0.0)
<18 (PUT 0.0)
<17 (|loadLibNoUpdate| T)
<16 (|loadLib| T)
16> (HPUT #<hash-table 000000000105e810>
    |NonNegativeInteger|
    ((NIL 1 . #<vector 0000000001ab1930>)))
<16 (HPUT ((NIL 1 . #<vector 0000000001ab1930>)))
16> (GETDATABASE |NonNegativeInteger| CONSTRUCTORKIND)
<16 (GETDATABASE |domain|)
16> (GETL |NonNegativeInteger| |infovec|)
<16 (GETL (#<vector 0000000001ab1b10>
    #<vector 0000000001ab1ae0>
    (((|commutative| "*" ) . 0))
    (#<bit-vector 0000000001ab1990>
    #<vector 0000000001ab1ab0>
    #<vector 0000000001ab1a80>
    . #<vector 0000000001ab1960>)
    |lookupComplete|))
16> (HPUT #<hash-table 000000000105e810>
    |NonNegativeInteger|
    ((NIL 1 . #<vector 0000000001ab1930>)))
<16 (HPUT ((NIL 1 . #<vector 0000000001ab1930>)))
16> (|lookupInDomainVector| |coerce| ((|OutputForm|) $)
    #<vector 0000000001ab1930> #<vector 0000000001ab1d50>)
17> (GETDATABASE |NonNegativeInteger| CONSTRUCTORKIND)
<17 (GETDATABASE |domain|)
17> (PNAME |NonNegativeInteger|)
<17 (PNAME "NonNegativeInteger")
17> (PNAME |NonNegativeInteger|)

```

```

<17 (PNAME "NonNegativeInteger")
17> (GETDATABASE |OutputForm| COSIG)
<17 (GETDATABASE (NIL))
17> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<17 (GETDATABASE |domain|)
17> (GETL |Integer| LOADED)
<17 (GETL NIL)
17> (|loadLib| |Integer|)
18> (GETL |print| |TimeTotal|)
<18 (GETL 0.0)
18> (GETL |gc| |TimeTotal|)
<18 (GETL 0.0)
18> (PUT |gc| |TimeTotal| 0.0)
<18 (PUT 0.0)
18> (PUT |print| |TimeTotal| 0.0)
<18 (PUT 0.0)
18> (GETDATABASE |Integer| OBJECT)
<18 (GETDATABASE
    "/home/daly/noise/mnt/ubuntu/algebra/INT.o")
18> (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/INT.o")
19> (|pathname|
    "/home/daly/noise/mnt/ubuntu/algebra/INT.o")
<19 (|pathname|
    #p"/home/daly/noise/mnt/ubuntu/algebra/INT.o")
<18 (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
18> (|isSystemDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
<18 (|isSystemDirectory| T)
18> (|loadLibNoUpdate| |Integer| |Integer|
    "/home/daly/noise/mnt/ubuntu/algebra/INT.o")
19> (GETDATABASE |Integer| CONSTRUCTORKIND)
<19 (GETDATABASE |domain|)
19> (|makeByteWordVec2| 1
    (0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0))
<19 (|makeByteWordVec2| #<bit-vector 0000000001723f60>)
19> (|makeByteWordVec2| 133
    (1 7 6 0 8 3 7 6 0 9 9 10 2 7 6 0 11 12 1 7 6 0 13 0
    14 0 15 2 7 0 9 14 16 1 7 6 0 17 1 7 6 0 18 1 7 6 0
    19 1 35 0 11 36 1 44 0 11 45 1 47 0 11 48 1 50 0 11
    51 1 9 0 11 53 2 93 90 91 92 94 1 97 95 96 98 1 96
    0 0 99 1 96 2 0 100 1 101 95 96 102 1 96 0 2 103 1
    0 104 0 105 2 108 95 106 107 109 2 110 95 95 95 111
    1 101 95 96 112 1 96 21 0 113 1 96 0 0 114 1 116 96
    115 117 2 0 21 0 0 1 1 0 21 0 25 1 0 87 0 88 1 0 0
    0 89 1 0 21 0 1 2 0 0 0 0 1 2 0 83 0 0 1 3 0 0 0 0
    0 42 1 0 0 0 1 1 0 104 0 1 2 0 21 0 0 1 1 0 11 0 1
    2 0 0 0 0 82 0 0 0 1 1 0 124 0 1 1 0 11 0 1 2 0 0 0

```

```

0 81 2 0 60 58 61 62 1 0 57 58 59 1 0 83 0 85 1 0
120 0 1 1 0 21 0 1 1 0 121 0 1 1 0 0 0 65 0 0 0 64
2 0 0 0 0 80 1 0 127 126 1 1 0 21 0 1 3 0 0 0 0 1
2 0 0 0 0 56 1 0 21 0 1 2 0 0 0 0 1 3 0 122 0 123
122 1 1 0 21 0 26 1 0 21 0 75 1 0 83 0 1 1 0 21 0
34 2 0 125 126 0 1 3 0 0 0 0 0 43 2 0 0 0 0 77 2 0
0 0 0 76 1 0 0 0 1 1 0 0 0 40 2 0 131 0 0 1 1 0 0
126 1 2 0 0 0 0 1 1 0 9 0 55 2 0 0 0 0 1 0 0 0 1 1
0 0 0 31 1 0 0 0 33 1 0 133 0 1 2 0 118 118 118 119
2 0 0 0 0 86 1 0 0 126 1 1 0 0 0 1 1 0 104 0 105 3
0 129 0 0 0 1 2 0 130 0 0 1 2 0 83 0 0 84 2 0 125
126 0 1 1 0 21 0 1 1 0 73 0 1 2 0 78 0 0 79 1 0 0 0
1 2 0 0 0 73 1 1 0 0 0 32 1 0 0 0 30 1 0 9 0 54 1 0
47 0 49 1 0 44 0 46 1 0 50 0 52 1 0 123 0 1 1 0 11
0 39 1 0 0 11 38 1 0 0 0 1 1 0 0 11 38 1 0 35 0 37
0 0 73 1 2 0 21 0 0 1 2 0 0 0 0 1 0 0 0 29 2 0 21 0
0 1 3 0 0 0 0 0 41 1 0 0 0 63 2 0 0 0 73 1 2 0 0 0
132 1 0 0 0 27 0 0 0 28 3 0 6 7 0 21 24 2 0 9 0 21
22 2 0 6 7 0 23 1 0 9 0 20 1 0 0 0 1 2 0 0 0 73 1 2
0 21 0 0 1 2 0 21 0 0 1 2 0 21 0 0 66 2 0 21 0 0 1
2 0 21 0 0 67 2 0 0 0 0 70 1 0 0 0 68 2 0 0 0 0 69
2 0 0 0 73 74 2 0 0 0 132 1 2 0 0 0 0 71 2 0 0 11 0
72 2 0 0 73 0 1 2 0 0 132 0 1))
<19 (|makeByteWordVec2| #<vector 0000000001723f00>)
19> (GETDATABASE |Integer| CONSTRUCTORKIND)
<19 (GETDATABASE |domain|)
19> (GETL |load| |TimeTotal|)
<19 (GETL 0.0)
19> (GETL |gc| |TimeTotal|)
<19 (GETL 0.0)
19> (PUT |gc| |TimeTotal| 0.0)
<19 (PUT 0.0)
19> (PUT |load| |TimeTotal| 0.0)
<19 (PUT 0.0)
<18 (|loadLibNoUpdate| T)
<17 (|loadLib| T)
17> (HPUT #<hash-table 000000000105e810> |Integer|
      ((NIL 1 . #<vector 0000000001723ed0>)))
<17 (HPUT ((NIL 1 . #<vector 0000000001723ed0>)))
17> (GETDATABASE |Integer| CONSTRUCTORKIND)
<17 (GETDATABASE |domain|)
17> (GETL |Integer| |infovec|)
<17 (GETL (#<vector 0000000001ab1780>
      #<vector 0000000001ab1750> ((|infinite| . 0)
      (|noetherian| . 0) (|canonicalsClosed| . 0)
      (|canonical| . 0) (|canonicalUnitNormal| . 0)
      (|multiplicativeValuation| . 0)
      (|noZeroDivisors| . 0)
      ((|commutative| "*") . 0) (|rightUnitary| . 0)
      (|leftUnitary| . 0) (|unitsKnown| . 0)))

```



```

      (#<bit-vector 0000000001723f60>
       #<vector 0000000001723fc0>
       #<vector 0000000001723f90>
       . #<vector 0000000001723f00>)
    |lookupComplete|))
17> (HPUT #<hash-table 000000000105e810> |Integer|
    ((NIL 1 . #<vector 0000000001723ed0>)))
<17 (HPUT ((NIL 1 . #<vector 0000000001723ed0>)))
17> (|lookupInDomainVector| |coerce| ((|OutputForm|) $)
    #<vector 0000000001723ed0> #<vector 0000000001ab1d50>)
18> (GETDATABASE |Integer| CONSTRUCTORKIND)
<18 (GETDATABASE |domain|)
18> (PNAME |Integer|)
<18 (PNAME "Integer")
18> (PNAME |Integer|)
<18 (PNAME "Integer")
18> (GETDATABASE |OutputForm| COSIG)
<18 (GETDATABASE (NIL))
18> (GETDATABASE |PositiveInteger| CONSTRUCTORKIND)
<18 (GETDATABASE |domain|)
<17 (|lookupInDomainVector|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))
<16 (|lookupInDomainVector|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))
<15 (|lookupInDomainVector|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))
<14 (|oldCompLookup|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))
<13 (|basicLookup|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))
<12 (|compiledLookup|
    (#<compiled-function |INT;coerce;$0f;16|>
     . #<vector 0000000001723ed0>))

"TPD:INT:coerce(x):OutputForm"

12> (GETDATABASE |Integer| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETL |OutputForm| LOADED)
<12 (GETL NIL)
12> (|loadLib| |OutputForm|)
13> (GETL |print| |TimeTotal|)
<13 (GETL 0.0)
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)

```

```

13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (PUT |print| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (GETDATABASE |OutputForm| OBJECT)
<13 (GETDATABASE
    "/home/daly/noise/mnt/ubuntu/algebra/OUTFORM.o")
13> (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/OUTFORM.o")
14> (|pathname|
    "/home/daly/noise/mnt/ubuntu/algebra/OUTFORM.o")
<14 (|pathname|
    #p"/home/daly/noise/mnt/ubuntu/algebra/OUTFORM.o")
<13 (|pathnameDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
13> (|isSystemDirectory|
    "/home/daly/noise/mnt/ubuntu/algebra/")
<13 (|isSystemDirectory| T)
13> (|loadLibNoUpdate| |OutputForm| |OutputForm|
    "/home/daly/noise/mnt/ubuntu/algebra/OUTFORM.o")
14> (GETDATABASE |OutputForm| CONSTRUCTORKind)
<14 (GETDATABASE |domain|)
14> (|makeByteWordVec2| 1 (0 0 0))
<14 (|makeByteWordVec2| #<bit-vector 0000000001723c90>)
14> (|makeByteWordVec2| 144
    (1 10 9 0 11 0 25 0 26 2 10 0 0 25 27 2 10 0 25 0 28
      2 19 0 0 0 36 2 19 0 0 0 37 2 19 9 0 0 46 1 6 0 0 56
      2 6 0 0 0 57 1 6 9 0 69 1 6 0 0 70 1 6 2 0 71 1 6 73
      0 74 1 19 9 0 75 2 76 0 0 0 77 1 76 0 0 105 1 25 0
      10 114 2 10 0 73 25 115 1 73 9 0 128 2 73 9 0 0 129
      1 131 10 130 132 1 10 0 0 133 2 0 9 0 0 1 2 0 0 0 0
      120 0 0 19 35 1 0 19 0 30 1 0 0 19 47 1 0 0 52 80 2
      0 0 0 0 48 2 0 0 0 52 78 1 0 19 0 33 2 0 0 0 0 66 2
      0 0 0 0 136 3 0 0 0 0 137 1 0 0 0 135 1 0 19 0 32
      2 0 0 0 0 65 1 0 0 0 109 2 0 0 0 0 124 1 0 0 52 55
      2 0 0 0 52 72 2 0 0 19 19 49 1 0 0 0 121 2 0 0 0 0
      122 1 0 0 0 45 2 0 0 0 19 42 2 0 0 0 0 93 2 0 0 0 0
      127 1 0 0 0 110 2 0 0 0 0 94 3 0 0 0 0 140 1 0 0
      0 138 2 0 0 0 0 139 1 0 7 0 8 2 0 0 0 73 117 1 0 0
      0 113 2 0 0 0 0 68 2 0 0 0 0 67 2 0 0 0 52 104 2 0
      0 0 0 108 1 0 0 52 53 1 0 0 52 64 1 0 0 0 63 2 0 0
      0 0 118 1 0 0 0 111 2 0 0 0 0 123 1 0 0 10 29 1 0 0
      23 24 1 0 0 21 22 1 0 0 19 20 2 0 0 0 0 97 1 0 0 0
      98 1 0 7 10 14 1 0 0 10 13 1 0 0 50 51 1 0 0 0 44 2
      0 0 0 19 41 1 0 10 0 1 2 0 0 0 0 126 3 0 0 0 0 0
      143 2 0 0 0 0 142 1 0 0 0 141 1 0 9 0 102 2 0 0 0
      52 106 3 0 0 0 0 107 1 0 0 19 38 0 0 19 34 1 0 19
      0 31 1 0 0 52 79 2 0 0 0 0 39 1 0 144 0 1 2 0 0 0
      95 0 0 0 12 2 0 0 0 52 103 2 0 0 0 73 116 1 0 0 0
      112 2 0 0 0 0 92 2 0 0 0 73 134 1 0 0 52 54 1 0 17
    )

```

```

0 18 1 0 0 0 43 2 0 0 0 19 40 1 0 0 0 61 1 0 0 52
62 1 0 0 52 60 1 0 0 0 59 1 0 0 0 119 1 0 0 52 58 2
0 0 0 0 101 2 0 0 0 0 125 2 0 0 0 0 96 2 0 0 0 0 81
1 0 0 0 100 2 0 0 0 0 99 2 0 0 0 0 85 2 0 0 0 0 83
2 0 0 0 0 16 2 0 9 0 0 15 2 0 0 0 0 84 2 0 0 0 0 82
2 0 0 0 0 90 1 0 0 0 88 2 0 0 0 0 87 2 0 0 0 0 86 2
0 0 0 0 91 2 0 0 0 0 89))
<14 (|makeByteWordVec2| #<vector 0000000001723c60>)>
14> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<14 (GETDATABASE |domain|)>
14> (GETL |load| |TimeTotal|)
<14 (GETL 0.0)>
14> (GETL |gc| |TimeTotal|)
<14 (GETL 0.0)>
14> (PUT |gc| |TimeTotal| 0.0)
<14 (PUT 0.0)>
14> (PUT |load| |TimeTotal| 0.0)
<14 (PUT 0.0)>
<13 (|loadLibNoUpdate| T)>
<12 (|loadLib| T)>
12> (HPUT #<hash-table 000000000105e810> |OutputForm|
((NIL 1 . #<vector 0000000001723c30>)))
<12 (HPUT ((NIL 1 . #<vector 0000000001723c30>)))>
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)>
12> (GETL |OutputForm| |infovec|)
<12 (GETL (#<vector 0000000001723d80>
#<vector 0000000001723d50> NIL
(#<bit-vector 0000000001723c90>
#<vector 0000000001723cf0>
#<vector 0000000001723cc0>
. #<vector 0000000001723c60>)
|lookupComplete|))>
12> (GETL |List| LOADED)
<12 (GETL NIL)>
12> (|loadLib| |List|)
13> (GETL |print| |TimeTotal|)
<13 (GETL 0.0)>
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)>
13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)>
13> (PUT |print| |TimeTotal| 0.0)
<13 (PUT 0.0)>
13> (GETDATABASE |List| OBJECT)
<13 (GETDATABASE
"/home/daly/noise/mnt/ubuntu/algebra/LIST.o")
13> (|pathnameDirectory|
"/home/daly/noise/mnt/ubuntu/algebra/LIST.o")
14> (|pathname|

```

```

        "/home/daly/noise/mnt/ubuntu/algebra/LIST.o")
<14 (|pathname|
        #p"/home/daly/noise/mnt/ubuntu/algebra/LIST.o")
<13 (|pathnameDirectory|
        "/home/daly/noise/mnt/ubuntu/algebra/")
13> (|isSystemDirectory|
        "/home/daly/noise/mnt/ubuntu/algebra/")
<13 (|isSystemDirectory| T)
13> (|loadLibNoUpdate| |List| |List|
        "/home/daly/noise/mnt/ubuntu/algebra/LIST.o")
14> (GETDATABASE |List| CONSTRUCTORKIND)
<14 (GETDATABASE |domain|)
14> (|makeByteWordVec2| 8
        (0 0 0 0 0 0 0 0 0 0 0 3 0 0 8 4 0 0 8 1 2 4 5))
<14 (|makeByteWordVec2| #<vector 0000000001723a50>)
14> (|makeByteWordVec2| 51
        (1 13 12 0 14 3 13 12 0 15 15 16 1 0 6 0 17 3 6 12
          13 0 8 18 1 0 0 0 19 1 13 12 0 20 0 21 0 22 2 13 0
          15 21 23 1 13 12 0 24 1 13 12 0 25 1 13 12 0 26 1
          0 15 0 27 2 0 15 0 8 28 2 0 12 13 0 29 3 0 12 13 0
          8 30 2 0 0 0 0 31 1 0 0 0 32 2 0 0 0 0 33 0 0 0 34
          1 0 8 0 35 2 0 8 6 0 36 2 0 0 0 0 37 2 0 6 0 38 39
          2 0 0 6 0 40 2 0 0 0 0 41 1 42 0 15 43 1 44 0 42 45
          1 6 44 0 46 2 47 0 44 0 48 1 44 0 49 50 1 0 44 0 51
          2 1 0 0 0 33 2 1 0 0 0 37 2 1 0 0 0 41 1 0 0 0 19 1
          1 0 0 32 1 0 8 0 9 0 0 0 7 2 1 8 6 0 36 1 0 6 0 17
          1 0 8 0 35 0 0 0 34 2 0 6 0 38 39 1 2 44 0 51 2 0 0
          6 0 10 2 0 0 6 0 40 2 0 0 0 0 31 2 0 0 0 0 11 3 5 12
          13 0 8 30 2 5 12 13 0 29 1 5 15 0 27 2 5 15 0 8 28))
<14 (|makeByteWordVec2| #<vector 0000000001723a20>)
14> (GETDATABASE |List| CONSTRUCTORKIND)
<14 (GETDATABASE |domain|)
14> (GETL |load| |TimeTotal|)
<14 (GETL 0.0)
14> (GETL |gc| |TimeTotal|)
<14 (GETL 0.0)
14> (PUT |gc| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (PUT |load| |TimeTotal| 0.0)
<14 (PUT 0.0)
<13 (|loadLibNoUpdate| T)
<12 (|loadLib| T)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")

```

```

12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |SetCategory| COSIG)
<12 (GETDATABASE (NIL))
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |SetCategory| COSIG)
<12 (GETDATABASE (NIL))
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |Integer| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |Integer|)
<12 (PNAME "Integer")
12> (PNAME |Integer|)
<12 (PNAME "Integer")
12> (GETDATABASE |OrderedSet| COSIG)
<12 (GETDATABASE (NIL))
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)

```

```

12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (PNAME |OutputForm|)
<12 (PNAME "OutputForm")
12> (HPUT #<hash-table 000000000105e810> |List|
      ((((|OutputForm|)) 1 . #<vector 00000000017239f0>)))
<12 (HPUT ((((|OutputForm|)) 1
              . #<vector 00000000017239f0>)))
12> (GETDATABASE |List| CONSTRUCTORKIND)
<12 (GETDATABASE |domain|)
12> (GETL |List| |infovec|)
<12 (GETL (#<vector 0000000001723b10>
           #<vector 0000000001723ae0>
           ((|shallowlyMutable| . 0)
            (|finiteAggregate| . 0))
           (#<vector 0000000001723a50>
            #<vector 0000000001723ab0>
            #<vector 0000000001723a80>
            . #<vector 0000000001723a20>)
           |lookupIncomplete|))
12> (HPUT #<hash-table 000000000105e810> |OutputForm|
      ((NIL 1 . #<vector 0000000001723c30>)))
<12 (HPUT ((NIL 1 . #<vector 0000000001723c30>)))
12> (GETDATABASE |Integer| COSIG)
<12 (GETDATABASE (NIL))
12> (|basicLookup| |outputForm| ($ (|Integer|))
      #<vector 0000000001723c30> #<vector 0000000001723c30>)
13> (|oldCompLookup| |outputForm| ($ (|Integer|))
      #<vector 0000000001723c30> #<vector 0000000001723c30>)
14> (|lookupInDomainVector| |outputForm| ($ (|Integer|))
      #<vector 0000000001723c30> #<vector 0000000001723c30>)
15> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (GETDATABASE |OutputForm| CONSTRUCTORKIND)
<15 (GETDATABASE |domain|)
15> (GETDATABASE |Integer| COSIG)
<15 (GETDATABASE (NIL))
<14 (|lookupInDomainVector|

```

```

      (#<compiled-function |OUTFORM;outputForm;I$;7|>
       . #<vector 0000000001723c30>))
<13 (|oldCompLookup|
      (#<compiled-function |OUTFORM;outputForm;I$;7|>
       . #<vector 0000000001723c30>))
<12 (|basicLookup|
      (#<compiled-function |OUTFORM;outputForm;I$;7|>
       . #<vector 0000000001723c30>))

"TPD:OUTFORM:outputForm n"

12> (GETL |print| |TimeTotal|)
<12 (GETL 0.0)
12> (GETL |gc| |TimeTotal|)
<12 (GETL 0.0)
12> (PUT |gc| |TimeTotal| 0.0)
<12 (PUT 0.0)
12> (PUT |print| |TimeTotal| 0.0)
<12 (PUT 0.0)
12> (|member| 1 ("failed" "nil" "prime" "sqfr" "irred"))
<12 (|member| NIL)
12> (|member| EQUATNUM (SLASH OVER))
<12 (|member| NIL)
12> (GETL EQUATNUM |Led|)
<12 (GETL (|dummy| |dummy| 10000 0))
12> (|member| EQUATNUM (SLASH OVER))
<12 (|member| NIL)
12> (GETL EQUATNUM |Led|)
<12 (GETL (|dummy| |dummy| 10000 0))
12> (GETL EQUATNUM INFIXOP)
<12 (GETL " ")
12> (GETL EQUATNUM WIDTH)
<12 (GETL NIL)
12> (GETL EQUATNUM APP)
<12 (GETL NIL)
12> (|member| EQUATNUM (SLASH OVER))
<12 (|member| NIL)
12> (GETL EQUATNUM |Led|)
<12 (GETL (|dummy| |dummy| 10000 0))
12> (|member| EQUATNUM (SLASH OVER))
<12 (|member| NIL)
12> (GETL EQUATNUM |Led|)
<12 (GETL (|dummy| |dummy| 10000 0))
12> (GETL EQUATNUM INFIXOP)
<12 (GETL " ")
12> (GETL EQUATNUM SUPERSPAN)
<12 (GETL NIL)
12> (GETL EQUATNUM SUBSPAN)
<12 (GETL NIL)

```

```

12> (|putHist| % |value| ((|PositiveInteger|) . 1) ((NIL)))
13> (|recordNewValue| % |value| ((|PositiveInteger|) . 1))
14> (GETL |print| |TimeTotal|)
<14 (GETL 0.0)
14> (GETL |gc| |TimeTotal|)
<14 (GETL 0.0)
14> (PUT |gc| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (PUT |print| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (|recordNewValue0| % |value| ((|PositiveInteger|) . 1))
<14 (|recordNewValue0|
      ((% (|value| (|PositiveInteger|) . 1))))
14> (GETL |history| |TimeTotal|)
<14 (GETL 0.0)
14> (GETL |gc| |TimeTotal|)
<14 (GETL 0.0)
14> (PUT |gc| |TimeTotal| 0.0)
<14 (PUT 0.0)
14> (PUT |history| |TimeTotal| 0.0)
<14 (PUT 0.0)
<13 (|recordNewValue| |history|)
13> (|search| % ((NIL)))
14> (|searchCurrentEnv| % (NIL))
<14 (|searchCurrentEnv| NIL)
14> (|searchTailEnv| % NIL)
<14 (|searchTailEnv| NIL)
<13 (|search| NIL)
<12 (|putHist| (((% (|value| (|PositiveInteger|) . 1)))))
12> (|printTypeAndTime| 1 (|PositiveInteger|))
14> (|sayKeyedMsg| "%rjon Type: %1p %rjoff" ((|PositiveInteger|)))
15> (|sayKeyedMsgLocal| "%rjon Type: %1p %rjoff" ((|PositiveInteger|)))
16> (|segmentKeyedMsg| " %rjon Type: %1p %rjoff")
<16 (|segmentKeyedMsg| ("%rjon" "Type:" "%1p" "%rjoff"))
16> (|member| "%rjon" (|%ceon| "%ceon"))
<16 (|member| NIL)
16> (|member| "%rjon" (|%rjon| "%rjon"))
<16 (|member| ("%rjon"))
16> (|member| "Type:"
      (|%ceoff| "%ceoff" |%rjoff| "%rjoff"))
<16 (|member| NIL)
16> (|member| "%1p" (|%ceoff| "%ceoff" |%rjoff| "%rjoff"))
<16 (|member| NIL)
16> (|member| "%rjoff"
      (|%ceoff| "%ceoff" |%rjoff| "%rjoff"))
<16 (|member| ("%rjoff"))
16> (|member| "%rj" (|%ceon| "%ceon"))
<16 (|member| NIL)
16> (|member| "%rj" (|%rjon| "%rjon"))
<16 (|member| NIL)

```



```

16> (|member| "Type:" (|%ceon| "%ceon"))
<16 (|member| NIL)
16> (|member| "Type:" (|%rjon| "%rjon"))
<16 (|member| NIL)
16> (|member| "%1p" (|%ceon| "%ceon"))
<16 (|member| NIL)
16> (|member| "%1p" (|%rjon| "%rjon"))
<16 (|member| NIL)
16> (DIGITP #\r)
<16 (DIGITP NIL)
16> (DIGITP #\1)
<16 (DIGITP 1)
16> (GETDATABASE |PositiveInteger| ABBREVIATION)
<16 (GETDATABASE PI)
16> (|member| "Type:" ("%n" |%n|))
<16 (|member| NIL)
16> (|member| "Type:" ("%y" |%y|))
<16 (|member| NIL)
16> (|member| "%rj"
      (" " | | "%" % |%b| |%d| |%l| |%i| |%u| %U |%n| |%x|
        |%ce| |%rj| "%U" "%b" "%d" "%l" "%i" "%u" "%U" "%n"
        "%x" "%ce" "%rj" [ |(| "[" "("))
<16 (|member| ("%rj" [ |(| "[" "("))
16> (|member| |PositiveInteger| ("%n" |%n|))
<16 (|member| NIL)
16> (|member| |PositiveInteger| ("%y" |%y|))
<16 (|member| NIL)
16> (|member| "Type:"
      (" " | | "%" % |%b| |%d| |%l| |%i| |%u| %U |%n| |%x|
        |%ce| |%rj| "%U" "%b" "%d" "%l" "%i" "%u" "%U" "%n"
        "%x" "%ce" "%rj" [ |(| "[" "("))
<16 (|member| NIL)
16> (SIZE "Type:")
<16 (SIZE 5)
16> (|member| |PositiveInteger|
      (" " | | "%" % |%b| |%d| |%l| |%i| |%u| %U |%n| |%x|
        |%ce| |%rj| "%U" "%b" "%d" "%l" "%i" "%u" "%U" "%n"
        "%x" "%ce" "%rj" |.| |,| ! |:| |;| ? ] |)| "." ","
        "!" ":" ";" "?" "]" " "))
<16 (|member| NIL)
16> (|member| "%rj" (|%ce| "%ce" |%rj| "%rj"))
<16 (|member| ("%rj"))
16> (|sayMSG| ((("%rj" "Type:" " " |PositiveInteger|)))
17> (SAYBRIGHTLY1 ((("%rj" "Type:" " " |PositiveInteger|))
      #<synonym stream to *TERMINAL-IO*>))
18> (BRIGHTPRINT ((("%rj" "Type:" " " |PositiveInteger|)))
19> (|member| "%rj" ("%p" "%s"))
<19 (|member| NIL)
19> (|member| "Type:" (|%l| "%l"))
<19 (|member| NIL)

```

```

19> (|member| " " (|%l| "%l"))
<19 (|member| NIL)
19> (|member| |PositiveInteger| (|%l| "%l"))
<19 (|member| NIL)
19> (|member| "Type:" ("%b" "%d" |%b| |%d|))
<19 (|member| NIL)
19> (|member| "Type:" ("%l" |%l|))
<19 (|member| NIL)
19> (|member| " " ("%b" "%d" |%b| |%d|))
<19 (|member| NIL)
19> (|member| " " ("%l" |%l|))
<19 (|member| NIL)
19> (|member| |PositiveInteger| ("%b" "%d" |%b| |%d|))
<19 (|member| NIL)
19> (|member| |PositiveInteger| ("%l" |%l|))
<19 (|member| NIL)
19> (PNAME |PositiveInteger|)
<19 (PNAME "PositiveInteger")
19> (|fillerSpaces| 56 " ")
<19 (|fillerSpaces|
"
Type:
19> (PNAME |PositiveInteger|)
<19 (PNAME "PositiveInteger")
PositiveInteger

<18 (BRIGHTPRINT NIL)
<17 (SAYBRIGHTLY1 NIL)
<16 (|sayMSG| NIL)
<15 (|sayKeyedMsgLocal| NIL)
<14 (|sayKeyedMsg| NIL)
<12 (|printTypeAndTime| NIL)
<11 (|recordAndPrint| |done|)
11> (recordFrame |normal|)
12> (|diffAlist|
((% (|value| (|PositiveInteger|) . 1))) NIL)
<12 (|diffAlist| ((% (|value|))))
<11 (recordFrame ((% (|value|))))
11> (GETL |print| |TimeTotal|)
<11 (GETL 0.0)
11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |print| |TimeTotal| 0.0)
<11 (PUT 0.0)
<10 (|processInteractive1| ((|PositiveInteger|) . 1))
10> (|writeHistModesAndValues|)
11> (|putHist| % |value|
#0=((|PositiveInteger|) . 1) ((((% (|value| . #0#)))))

```

```

12> (|recordNewValue| % |value| ((|PositiveInteger|) . 1))
13> (GETL |other| |TimeTotal|)
<13 (GETL 0.0)
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)
13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (PUT |other| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (|recordNewValue0| % |value| ((|PositiveInteger|) . 1))
<13 (|recordNewValue0| (|value| (|PositiveInteger|) . 1))
13> (GETL |history| |TimeTotal|)
<13 (GETL 0.0)
13> (GETL |gc| |TimeTotal|)
<13 (GETL 0.0)
13> (PUT |gc| |TimeTotal| 0.0)
<13 (PUT 0.0)
13> (PUT |history| |TimeTotal| 0.0)
<13 (PUT 0.0)
<12 (|recordNewValue| |history|)
12> (|search| %
      (((% (|value| (|PositiveInteger|) . 1))))))
13> (|searchCurrentEnv| %
      (((% (|value| (|PositiveInteger|) . 1))))))
<13 (|searchCurrentEnv|
      ((|value| (|PositiveInteger|) . 1)))
<12 (|search| ((|value| (|PositiveInteger|) . 1)))
<11 (|putHist| (((% (|value| (|PositiveInteger|) . 1))))))
<10 (|writeHistModesAndValues| NIL)
10> (|updateHist|)
11> (GETL |other| |TimeTotal|)
<11 (GETL 0.0)
11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |other| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (|updateInCoreHist|)
<11 (|updateInCoreHist| 1)
11> (|writeHiFi|)
<11 (|writeHiFi|
      ((1 (% (|value| (|PositiveInteger|) . 1))))))
11> (|disableHist|)
<11 (|disableHist| NIL)
11> (|updateCurrentInterpreterFrame|)
12> (|createCurrentInterpreterFrame|)
<12 (|createCurrentInterpreterFrame|
      (|frame0|
        (((% (|value| . #0=((|PositiveInteger|) . 1))))))

```

```

2 T
#1=(NIL NIL NIL NIL NIL NIL NIL NIL NIL NIL NIL NIL
    NIL NIL NIL NIL NIL NIL NIL NIL NIL NIL . #1#)
20 1 NIL ((1 (% (|value| . #0#))))
#<vector 0000000000fa5cc0>))
12> (|updateFromCurrentInterpreterFrame|)
<12 (|updateFromCurrentInterpreterFrame| NIL)
<11 (|updateCurrentInterpreterFrame| NIL)
11> (GETL |history| |TimeTotal|)
<11 (GETL 0.0)
11> (GETL |gc| |TimeTotal|)
<11 (GETL 0.0)
11> (PUT |gc| |TimeTotal| 0.0)
<11 (PUT 0.0)
11> (PUT |history| |TimeTotal| 0.0)
<11 (PUT 0.0)
<10 (|updateHist| |history|)
<9 (|processInteractive| ((|PositiveInteger|) . 1))
<8 (|intInterpretPform| ((|PositiveInteger|) . 1))
8> (|ncPutQ|
    ((|carrier| (|ok?| . T)
      (|ptreePremacro|
        . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))) |value|
      ((|PositiveInteger|) . 1))
9> (|ncAlist|
    ((|carrier| (|ok?| . T)
      (|ptreePremacro|
        . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<9 (|ncAlist| ((|ok?| . T)
    (|ptreePremacro|
      . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1)))
9> (|ncAlist|
    ((|carrier| (|ok?| . T)
      (|ptreePremacro|
        . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
          . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1)))

```

```

        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<9 (|ncAlist|
    ((|ok?| . T)
    (|ptreePremacro|
    . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
9> (|ncTag|
    ((|carrier| (|ok?| . T)
    (|ptreePremacro|
    . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
    <9 (|ncTag| |carrier|)
    <8 (|ncPutQ| ((|PositiveInteger|) . 1))
    <7 (|phInterpret| ((|PositiveInteger|) . 1))
7> (|ncConversationPhase,wrapup|
    ((|carrier| (|value| (|PositiveInteger|) . 1) (|ok?| . T)
    (|ptreePremacro|
    . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
    <7 (|ncConversationPhase,wrapup| NIL)
    <6 (|ncConversationPhase| ((|PositiveInteger|) . 1))
6> (|ncEltQ|
    ((|carrier| (|value| (|PositiveInteger|) . 1) (|ok?| . T)
    (|ptreePremacro|
    . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
    . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1)))
    |messages|)
7> (|ncAlist|
    ((|carrier| (|value| (|PositiveInteger|) . 1) (|ok?| . T)
    (|ptreePremacro|

```

```

      . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
    (|ptree| . #0#)
    (|lines| ((#1# . 1) . "1"))
    (|messages|)
    (|stepNumber| . 1))))
<7 (|incAlist|
  ((|value| (|PositiveInteger|) . 1) (|ok?| . T)
    (|ptreePremacro|
      . #0=((|integer| (|posn| #1=(0 "1" 1 1 "strings") . 0))
        . "1"))
      (|ptree| . #0#)
      (|lines| ((#1# . 1) . "1"))
      (|messages|)
      (|stepNumber| . 1))))
<6 (|incEltQ| NIL)
<5 (|intloopSpadProcess,interp| NIL)
<4 (|intloopSpadProcess| 2)

4> (|StreamNull|
  (|nonnullstream| #0=|incAppend1| NIL
    (|nonnullstream| #2=|next1| |ncloopParse|
      (|nonnullstream| #0# NIL
        (|nonnullstream| #2# |lineoftoks| (|nullstream|))))))
5> (|incAppend1| NIL
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| |incAppend1| NIL
      (|nonnullstream| #0# |lineoftoks| (|nullstream|))))))
6> (|StreamNull| NIL)
<6 (|StreamNull| T)
6> (|StreamNull|
  (|nonnullstream| #0=|next1| |ncloopParse|
    (|nonnullstream| |incAppend1| NIL
      (|nonnullstream| #0# |lineoftoks| (|nullstream|))))))
7> (|next1| |ncloopParse|
  (|nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks| (|nullstream|))))
8> (|StreamNull|
  (|nonnullstream| |incAppend1| NIL
    (|nonnullstream| |next1| |lineoftoks| (|nullstream|))))
9> (|incAppend1| NIL
  (|nonnullstream| |next1| |lineoftoks| (|nullstream|)))
10> (|StreamNull| NIL)
<10 (|StreamNull| T)
10> (|StreamNull|
  (|nonnullstream| |next1| |lineoftoks| (|nullstream|)))
11> (|next1| |lineoftoks| (|nullstream|))
12> (|StreamNull| (|nullstream|))
<12 (|StreamNull| T)
<11 (|next1| (|nullstream|))

```

```
      <10 (|StreamNull| T)
      <9 (|incAppend1| (|nullstream|))
      <8 (|StreamNull| T)
      <7 (|next1| (|nullstream|))
      <6 (|StreamNull| T)
      <5 (|incAppend1| (|nullstream|))
      <4 (|StreamNull| T)
      <3 (|intloopProcess| 2)
```

Chapter 11

Axiom Details

11.1 Variables Used

11.2 Data Structures

11.3 Functions

Set the restart hook

When a lisp image containing code is reloaded there is a hook to allow a function to be called. In our case it is the restart function which is the entry to the Axiom interpreter.

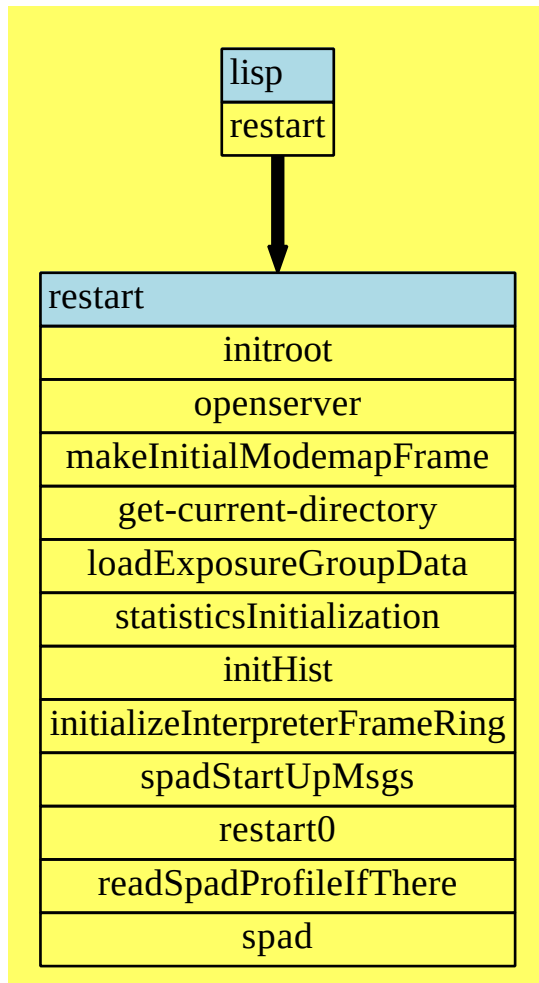
set-restart-hook : **Void** $\rightarrow$ 'restart

— defun set-restart-hook 0 —

```
(defun set-restart-hook ()
  "Set the restart hook"
  #+KCL (setq system::*top-level-hook* 'restart)
  #+Lucid (setq boot::restart-hook 'restart)
  'restart
)
```

—————

restart function (The restart function)



The restart function is the real root of the world. It sets up memory if we are working in a GCL/akcl version of the system.

The `compiler::*compile-verbose*` flag has been set to nil globally. We do not want to know about the microsteps of GCL's compile facility.

The `compiler::*suppress-compiler-warnings*` flag has been set to t. We do not care that certain generated variables are not used.

The `compiler::*suppress-compiler-notes*` flag has been set to t. We do not care that tail recursion occurs.

It sets the current package to be the "BOOT" package which is the standard package in which the interpreter runs.

The "initroot" ([12.3 p 327](#)) function sets global variables that depend on the AXIOM shell

variable. These are needed to find basic files like `s2-us.msgs`, which contains the error message text.

The “`openserver`” (32 p 1173) function tried to set up the socket connection used for things like `hyperdoc`. The `$openServerIfTrue` variable starts true, which implies trying to start a server.

Axiom has multiple frames that contain independent information about a computation. There can be several frames at any one time and you can shift back and forth between the frames. By default, the system starts in “`frame0`” (try the `)frame names` command). See the Frame Mechanism chapter (3.1 page 18).

The “`printLoadMsgs`” (26.44 p 1008) variable controls whether load messages will be output as library routines are loaded. We disable this by default. It can be changed by using `)set message autoload`.

The “`current-directory`” (12.3 p 333) variable is set to the current directory. This is used by the `)cd` function and some of the compile routines.

The “`statisticsInitialization`” (36 p 1221) function initializes variables used to collect statistics. Currently, only the garbage collector information is initialized.

We test `*ThisIsARunningSystem*` (p959). If this variable is true then we are restarting from a previously running system and we do not want to reset all of the user variables.

```
[init-memory-config p326]
[initroot p327]
[openserver p1173]
[makeInitialModemapFrame p328]
[get-current-directory p328]
[statisticsInitialization p1221]
[initHist p888]
[initializeInterpreterFrameRing p27]
[spadStartUpMsgs p309]
[restart0 p309]
[readSpadProfileIfThere p1147]
[spad p310]
[$openServerIfTrue p199]
[$SpadServerName p202]
[$SpadServer p201]
[$IOindex p39]
[$InteractiveFrame p39]
[$printLoadMsgs p1008]
[$current-directory p333]
[$displayStartMsgs p1021]
[$currentLine p??]
```

— **defun restart** —

```
(defun restart ()
```

```

(declare (special $openServerIfTrue $SpadServerName |$SpadServer|
  |$IOindex| |$InteractiveFrame| |$printLoadMsgs| $current-directory
  |$displayStartMsgs| |$currentLine|))
#+:akcl
  (init-memory-config :cons 1024 :fixnum 200 :symbol 500 :package 8
    :array 800 :string 1024 :cfun 200 :cpages 6000 :rpages 2000 :hole 4000)
#+:akcl (setq compiler::*compile-verbose* nil)
#+:akcl (setq compiler::*suppress-compiler-warnings* t)
#+:akcl (setq compiler::*suppress-compiler-notes* t)
#+:akcl (setq si::*system-directory* "")
  (in-package "BOOT")
  (initroot)
#+:akcl
  (when (and $openServerIfTrue (zerop (openserver $SpadServerName)))
    (setq $openServerIfTrue nil)
    (setq |$SpadServer| t))
  (setq |$IOindex| 1)
  (setq |$printLoadMsgs| nil)
  (setq $current-directory (get-current-directory))
  (setq *default-pathname-defaults* (pathname $current-directory))
  (|statisticsInitialization|)
  (unless *ThisIsARunningSystem*
    (setq |$InteractiveFrame| (|makeInitialModemapFrame|))
    (|initHist|)
    (|initializeInterpreterFrameRing|))
  (setq *ThisIsARunningSystem* t)
  (when |$displayStartMsgs| (|spadStartUpMsgs|))
  (setq |$currentLine| nil)
  (restart0)
  (|readSpadProfileIfThere|)
  (|spad|))

```

defvar localVars

— initvars —

```
(defvar |$localVars| ()) ;checked by isType
```

defun Non-interactive restarts

[interpopen p1189]
 [operationopen p1193]
 [categoryopen p1192]
 [browseopen p1191]

— defun restart0 —

```
(defun restart0 ()
  (interpopen)      ;; open up the interpreter database
  (operationopen)   ;; all of the operations known to the system
  (categoryopen)    ;; answer hasCategory question
  (browseopen))
```

defun The startup banner messages

[fillerSpaces p310]
 [specialChar p1166]
 [sayKeyedMsg p43]
 [sayMSG p45]
 [\$msgAlist p42]
 [\$opSysName p??]
 [\$linelength p1051]
 [*yearweek* p??]
 [*build-version* p??]

— defun spadStartUpMsgs —

```
(defun |spadStartUpMsgs| ()
  (let (bar)
    (declare (special |$msgAlist| |$opSysName| $linelength *yearweek*
                      *build-version*))
    (when (> $linelength 60)
      (setq bar (|fillerSpaces| $linelength (|specialChar| '|hbar|)))
      (|sayKeyedMsg|
        (format nil "%ceon AXIOM Computer Algebra System %l Version: %1 %l ~
                    Timestamp: %2 %ceoff")
        (list *build-version* *yearweek*)))
      (|sayMSG| bar)
      (say " Issue )copyright to view copyright notices.")
      (say " Issue )summary for a summary of useful system commands.")
      (say " Issue )quit to leave AXIOM and return to shell.")
      (say " Visit http://axiom-developer.org for more information"))
```

```
(|sayMSG| bar)
(setq |$msgAlist| nil)
(|sayMSG| ' | |)))
```

defun Make a vector of filler characters

```
[ifcar p??]
```

— defun fillerSpaces —

```
(defun |fillerSpaces| (&rest arglist &aux charPart n)
  (setq n (car arglist))
  (setq charPart (cdr arglist))
  (if (<= n 0)
      ""
      (make-string n :initial-element (character (or (ifcar charPart) " "))))
```

defvar \$PrintCompilerMessageIfTrue

The \$PrintCompilerMessageIfTrue variable is set to NIL in spad.

— initvars —

```
(defvar |$PrintCompilerMessageIfTrue| nil)
```

Starts the interpreter but do not read in profiles

```
[setOutputAlgebra p1035]
[runspad p311]
[$PrintCompilerMessageIfTrue p310]
```

— defun spad —

```
(defun |spad| ()
  "Starts the interpreter but do not read in profiles"
  (let (|$PrintCompilerMessageIfTrue|)
    (declare (special |$PrintCompilerMessageIfTrue|))
    (setq |$PrintCompilerMessageIfTrue| nil)
```

```
(|setOutputAlgebra| '|%initialize%|)
(|runspad|)
'|EndOfSpad|))
```

defvar \$quitTag

— initvars —

```
(defvar |$quitTag| system::*quit-tag*)
```

defun runspad

```
[quitTag p311]
[coerceFailure p??]
[top-level p??]
[seq p??]
[exit p??]
[resetStackLimits p312]
[ncTopLevel p316]
[$quitTag p311]
```

— defun runspad —

```
(defun |runspad| ()
  (prog (mode)
    (declare (special |$quitTag|))
    (return
      (seq
        (progn
          (setq mode '|restart|)
          (do ()
            ((null (eq mode '|restart|)) nil)
            (seq
              (exit
                (progn
                  (|resetStackLimits|)
                  (catch |$quitTag|
                    (catch '|coerceFailure|
                      (setq mode (catch '|top_level| (|ncTopLevel|))))))))))))))
```

defun Reset the stack limits

[reset-stack-limits p??]

— defun resetStackLimits 0 —

```
(defun |resetStackLimits| ()  
  "Reset the stack limits"  
  (system:reset-stack-limits))
```

Chapter 12

Handling Terminal Input

12.1 Streams

defvar curinstream

The `curinstream` variable is set to the value of the `*standard-input*` common lisp variable in `ncIntLoop`. While not using the “dollar” convention this variable is still “global”.

— **initvars** —

```
(defvar curinstream (make-synonym-stream '*standard-input*))
```

defvar curoutstream

The `curoutstream` variable is set to the value of the `*standard-output*` common lisp variable in `ncIntLoop`. While not using the “dollar” convention this variable is still “global”.

— **initvars** —

```
(defvar curoutstream (make-synonym-stream '*standard-output*))
```

defvar errorinstream

— **initvars** —


```
(defvar errorinstream (make-synonym-stream '*terminal-io*))
```

defvar erroroutstream

— initvars —

```
(defvar erroroutstream (make-synonym-stream '*terminal-io*))
```

defvar *eof*

— initvars —

```
(defvar *eof* nil)
```

defvar *whitespace*

— initvars —

```
(defvar *whitespace*
  '(#\Space #\Newline #\Tab #\Page #\Linefeed #\Return #\Backspace)
  "A list of characters used by string-trim considered as whitespace")
```

There are several different environments used in the interpreter:

\$InteractiveFrame is the environment where the user values are stored. Any side effects of evaluation of a top-level expression are stored in this environment. It is always used as the starting environment for interpretation.

\$e is the name used for **\$InteractiveFrame** while interpreting.

\$env is local environment used by the interpreter. Only temporary information (such as types of local variables) is stored in **\$env**. It is thrown away after evaluation of each expression.

defvar \$InteractiveMode

— initvars —

```
(defvar |$InteractiveMode| (list (list nil)) "top level environment")
```

—————

defvar \$env

— initvars —

```
(defvar |$env| nil "checked in isDomainValuedVariable")
```

—————

defvar \$e

The **\$e** variable is set to the value of **\$InteractiveFrame** which is set in restart to the value of the call to the **makeInitialModemapFrame** function. This function simply returns a copy of the variable **\$InitialModemapFrame**.

Prints out the value **x** which is of type **m**, and records the changes in environment **\$e** into **\$InteractiveFrame**. Thus **\$e** is a copy of the variable **\$InitialModemapFrame**.

This variable is used in the undo mechanism.

— initvars —

```
(defvar |$e| nil "the environment?")
```

—————

defvar \$boot

— initvars —

```
(defvar $boot nil)
```

—————

\$newspad

The `$newspad` is set to `T` in `ncTopLevel`.

defvar \$newspad

— **initvars** —

```
(defvar $newspad nil)
```

—————

Top-level read-parse-eval-print loop

Top-level read-parse-eval-print loop for the interpreter. Uses the Bill Burge's parser. [`ncInt-Loop` p317]

```
[$e p315]  
[$spad p310]  
[$newspad p316]  
[$boot p826]  
[$InteractiveMode p315]  
[$InteractiveFrame p39]  
[*eof* p314]  
[in-stream p1147]
```

— **defun ncTopLevel** —

```
(defun |ncTopLevel| ()  
  "Top-level read-parse-eval-print loop"  
  (let (|$e| $spad $newspad $boot |$InteractiveMode| *eof* in-stream)  
    (declare (special |$e| $spad $newspad $boot |$InteractiveMode| *eof*  
              in-stream |$InteractiveFrame|))  
    (setq in-stream curinstream)  
    (setq *eof* nil)  
    (setq |$InteractiveMode| t)  
    (setq $boot nil)  
    (setq $newspad t)  
    (setq $spad t)  
    (setq |$e| |$InteractiveFrame|)  
    (|ncIntLoop|)))
```

—————

defun ncIntLoop

[intloop p317]

[curinstream p313]

[curoutstream p313]

— defun ncIntLoop —

```
(defun |ncIntLoop| ()
  (let ((curinstream *standard-output*)
        (curoutstream *standard-input*))
    (declare (special curinstream curoutstream))
    (|intloop|)))
```

—————

defvar \$intTopLevel

— initvars —

```
(defvar |$intTopLevel| ' |top_level|)
```

—————

defvar \$intRestart

— initvars —

```
(defvar |$intRestart| ' |restart|)
```

—————

defun intloop

Note that the SpadInterpretStream function uses a list of three strings as an argument. The values in the list seem to have no use and can eventually be removed. [intTopLevel p317]

[SpadInterpretStream p320]

[resetStackLimits p312]

[\$intTopLevel p317]

[\$intRestart p317]

— **defun intloop** —

```
(defun |intloop| ()
  (prog (mode)
    (declare (special |$intTopLevel| |$intRestart|))
    (return
      (progn
        (setq mode |$intRestart|)
        ((lambda ()
          (loop
            (cond
              ((not (equal mode |$intRestart|))
               (return nil))
              (t
               (progn
                 (|resetStackLimits|)
                 (setq mode
                   (catch |$intTopLevel|
                     (|SpadInterpretStream| 1
                      (list 'tim 'daly '? t))))))))))))))
```

—————

defvar \$ncMsgList

— **initvars** —

```
(defvar |$ncMsgList| nil)
```

—————

defun SpadInterpretStream

The SpadInterpretStream function takes three arguments

str This is passed as an argument to intloopReadConsole

source This is the name of a source file but appears not to be used. It is set to the list (tim daly ?).

interactive? If this is false then various messages are suppressed and input does not use piles. If this is true then the library loading routines might output messages and piles are expected on input (as from a file).

System commands are handled by the function in the “hook” variable `$systemCommandFunction` which has the default function `InterpExecuteSpadSystemCommand`. Thus, when a system command is entered this function is called.

defun GCL cmpnote function

GCL keeps noting the fact that the compiler is performing tail-recursion. Bill Schelter added this as a debugging tool for Axiom and it was never removed. Patching the lisp code in the GCL build fails as the system is actually built from the pre-compiled C code. Thus, we can only step on this message after the fact. The `cmpnote` function is used nowhere else in GCL so stepping on the function call seems best. We’re unhappy with this hack and will try to convince the GCL crowd to fix this.

— **defun cmpnote** —

```
#+:gcl (defun compiler::cmpnote (&rest x) (declare (ignore x)))
```

—————

defvar \$newcompErrorCount

— **initvars** —

```
(defvar |$newcompErrorCount| 0)
```

—————

defvar \$nopus

— **initvars** —

```
(defvar |$nopus| (list '|noposition|))
```

—————

```
[mkprompt p334]
[intloopReadConsole p321]
[intloopInclude p355]
[$systemCommandFunction p??]
[$ncMsgList p318]
[$erMsgToss p??]
```

```

[$lastPos p??]
[$inclAssertions p??]
[$okToExecuteMachineCode p??]
[$newcompErrorCount p319]
[$libQuiet p??]
[$fn p??]
[$nopus p319]

```

— defun SpadInterpretStream —

```

(defun |SpadInterpretStream| (str source interactive?)
  (let (|$systemCommandFunction|
        |$ncMsgList| |$erMsgToss| |$lastPos| |$inclAssertions|
        |$okToExecuteMachineCode| |$newcompErrorCount|
        |$libQuiet|)
    (declare (special
              |$systemCommandFunction| |$ncMsgList| |$erMsgToss| |$lastPos|
              |$inclAssertions| |$okToExecuteMachineCode| |$newcompErrorCount|
              |$libQuiet| |$nopus|))
    (setq |$libQuiet| (null interactive?))
    (setq |$newcompErrorCount| 0)
    (setq |$okToExecuteMachineCode| t)
    (setq |$inclAssertions| (list 'aix '|CommonLisp|))
    (setq |$lastPos| |$nopus|)
    (setq |$erMsgToss| nil)
    (setq |$ncMsgList| nil)
    (setq |$systemCommandFunction| #'|InterpExecuteSpadSystemCommand|)
    (if interactive?
        (progn
          (princ (mkprompt))
          (|intloopReadConsole| "" str))
        (|intloopInclude| source 0))))

```

12.2 The Read-Eval-Print Loop

defun intloopReadConsole

Note that this function relies on the fact that lisp can do tail-recursion. The function recursively invokes itself.

The serverReadLine function is a special readline function that handles communication with the session manager code, which is a separate process running in parallel.

We read a line from standard input.

- If it is a null line then we exit Axiom.
- If it is a zero length line we prompt and recurse
- If \$dalymode and open-paren we execute lisp code, prompt and recurse The \$dalymode will interpret any input that begins with an open-paren as a lisp expression rather than Axiom input. This is useful for debugging purposes when most of the input lines will be lisp. Setting \$dalymode non-nil will certainly break user expectations and is to be used with caution.
- If it is “)fi” or “)fin” we drop into lisp. Use the (restart) function to return to the interpreter loop.
- If it starts with “)” we process the command, prompt, and recurse
- If it is a command then we remember the current line, process the command, prompt, and recurse.
- If the input has a trailing underscore (Axiom line-continuation) then we cut off the continuation character and pass the truncated string to ourselves, prompt, and recurse
- otherwise we process the input, prompt, and recurse.

Notice that all but two paths (a null input or a “)fi” or a “)fin”) will end up as a recursive call to ourselves.

```
[top-level p??]
[serverReadLine p337]
[leaveScratchpad p941]
[mkprompt p334]
[intloopReadConsole p321]
[intloopPrefix? p327]
[intnplisp p328]
[setCurrentLine p334]
[ncloopCommand p818]
[concat p1237]
[ncloopEscaped p329]
[intloopProcessString p329]
[$dalymode p976]
```

intloopReadConsole : (String Integer) → Throw
 — defun intloopReadConsole —

```
(defun |intloopReadConsole| (prefix stepNumber)
  (declare (special $dalymode))
  (let (newStepNo cmd pfx input)
    ; read the next line
    (setq input (|serverReadLine| *standard-input*))
    ; if we have lost *standard-input* then exit Axiom
```



```

(when (null (stringp input)) (|leaveScratchpad|))
; if the input is a zero-length input, recurse
(when (eql (length input) 0)
  (princ (mkprompt))
  (|intloopReadConsole| "" stepNumber))
; if $dalymode is nonnil anything starting with '(' is a lisp expression
; evaluate the expression in lisp and recurse
(when (and $dalymode (|intloopPrefix?| "(" input))
  (|intnplisp| input)
  (princ (mkprompt))
  (|intloopReadConsole| "" stepNumber))
; if the input starts with ")fi" or ")fin" throw into lisp
(setq pfx (|intloopPrefix?| ")fi" input))
(when (and pfx (or (string= pfx ")fi") (string= pfx ")fin")))
  (throw '|top_level| nil))
; if the input starts with ')' it is a command; execute and recurse
(when (and (equal prefix "") (setq cmd (|intloopPrefix?| ")" input)))
  (|setCurrentLine| cmd)
  (setq newStepNo (|ncloopCommand| cmd stepNumber))
  (princ (mkprompt))
  (|intloopReadConsole| "" newStepNo))
; if the last non-blank character on the line is an underscore
; we use the current accumulated input as a prefix and recurse.
; this has the effect of concatenating the next line (minus the underscore)
(setq input (concat prefix input))
(when (|ncloopEscaped| input)
  (|intloopReadConsole| (subseq input 0 (- (length input) 1)) stepNumber))
; if there are no special cases, process the current line and recurse
(setq newStepNo (|intloopProcessString| input stepNumber))
(princ (mkprompt))
(|intloopReadConsole| "" newStepNo))

```

12.3 Helper Functions

Get the value of an environment variable

[getenv p??]

— defun getenviron 0 —

```

(defun getenviron (var)
  "Get the value of an environment variable"
  #+allegro (sys::getenv (string var))
  #+clisp (ext:getenv (string var))
  #+(or cmu scl)

```

```
(cdr
  (assoc (string var) ext:*environment-list* :test #'equalp :key #'string))
#+(or kcl akcl gcl) (si::getenv (string var))
#+lispworks (lw:environment-variable (string var))
#+lucid (lcl:environment-variable (string var))
#+mcl (ccl::getenv var)
#+sbcl (sb-ext:posix-getenv var)
)
```

defvar \$intCoerceFailure

— initvars —

```
(defvar |$intCoerceFailure| '|coerceFailure|)
```

defvar \$intSpadReader

— initvars —

```
(defvar |$intSpadReader| 'SPAD_READER)
```

defun InterpExecuteSpadSystemCommand

```
[intCoerceFailure p323]
[intSpadReader p323]
[ExecuteInterpSystemCommand p324]
|$intSpadReader p323]
|$intCoerceFailure p323]
```

— defun InterpExecuteSpadSystemCommand —

```
(defun |InterpExecuteSpadSystemCommand| (string)
  (declare (special |$intSpadReader| |$intCoerceFailure|))
  (catch |$intCoerceFailure|
```

```
(catch |$intSpadReader|
  (|ExecuteInterpSystemCommand| string))))
```

defun ExecuteInterpSystemCommand

```
[intProcessSynonyms p324]
[substring p324]
[doSystemCommand p786]
[$currentLine p??]
```

— defun ExecuteInterpSystemCommand —

```
(defun |ExecuteInterpSystemCommand| (string)
  (let (|$currentLine|)
    (declare (special |$currentLine|))
    (setq string (|intProcessSynonyms| string))
    (setq |$currentLine| string)
    (setq string (substring string 1 nil))
    (unless (equal string "") (|doSystemCommand| string))))
```

defun substring

— defun substring 0 —

```
(defun substring (cvec start length)
  (if length
    (subseq (string cvec) start (+ start length))
    (subseq (string cvec) start)))
```

defun Handle Synonyms

```
[processSynonyms p325]
[line p??]
```

— defun intProcessSynonyms —

```
(defun |intProcessSynonyms| (str)
  (let ((line str))
    (declare (special line))
    (|processSynonyms|
     line))
```

defun Synonym File Reader

```
[strpos p1236]
[substring p324]
[string2id-n p??]
[lassoc p??]
[concat p1237]
[size p1236]
[concat p1237]
[rplacstr p??]
[processSynonyms p325]
[$CommandSynonymAlist p817]
[line p??]
```

— defun processSynonyms —

```
(defun |processSynonyms| ()
  (let (fill p aline synstr syn to opt fun cl chr)
    (declare (special |$CommandSynonymAlist| line))
    (setq p (strpos ") " line 0 nil))
    (setq fill "")
    (cond
     (p
      (setq aline (substring line p nil))
      (when (> p 0) (setq fill (substring line 0 p))))
     (t
      (setq p 0)
      (setq aline line)))
    (setq to (strpos " " aline 1 nil))
    (cond (to (setq to (1- to))))
    (setq synstr (substring aline 1 to))
    (setq syn (string2id-n synstr 1))
    (when (setq fun (lassoc syn |$CommandSynonymAlist|))
      (setq to (strpos ") " fun 1 nil))
      (cond
       ((and to (not (eql to (1- (size fun)))))
        (setq opt (concat " " (substring fun to nil)))
        (setq fun (substring fun 0 (1- to))))
       (t (setq opt " "))))
```

```

(when (> (size synstr) (size fun))
  (do ((G167173 (size synstr)) (i (size fun) (1+ i)))
      ((> i G167173) nil)
      (setq fun (concat fun " "))))
(setq cl (concat fill (rplacstr aline 1 (size synstr) fun) opt))
(setq line cl)
(setq chr (elt line (1+ p)))
(|processSynonyms|)))

```

defun init-memory-config

Austin-Kyoto Common Lisp (AKCL), now known as Gnu Common Lisp (GCL) requires some changes to the default memory setup to run Axiom efficiently. This function performs those setup commands.

```

[allocate p??]
[allocate-contiguous-pages p??]
[allocate-relocatable-pages p??]
[set-hole-size p??]

```

— defun init-memory-config 0 —

```

(defun init-memory-config (&key
                           (cons 500)
                           (fixnum 200)
                           (symbol 500)
                           (package 8)
                           (array 400)
                           (string 500)
                           (cfun 100)
                           (cpages 3000)
                           (rpages 1000)
                           (hole 2000) )
  ;; initialize AKCL memory allocation parameters
  #+:AKCL
  (progn
    (system:allocate 'cons cons)
    (system:allocate 'fixnum fixnum)
    (system:allocate 'symbol symbol)
    (system:allocate 'package package)
    (system:allocate 'array array)
    (system:allocate 'string string)
    (system:allocate 'cfun cfun)
    (system:allocate-contiguous-pages cpages)
    (system:allocate-relocatable-pages rpages)
    (system:set-hole-size hole))

```

```
#-:AKCL
nil)
```

Set spadroot to be the AXIOM shell variable

Sets up the system to use the **AXIOM** shell variable if we can and default to the **\$spadroot** variable (which was the value of the **AXIOM** shell variable at build time) if we can't.

```
[reroot p332]
```

```
[getenvIRON p322]
```

```
[$spadroot p201]
```

— defun initroot —

```
(defun initroot (&optional (newroot (getenvIRON "AXIOM")))
  "Set spadroot to be the AXIOM shell variable"
  (declare (special $spadroot))
  (reroot (or newroot $spadroot (error "setenv AXIOM or (setq $spadroot)"))))
```

Does the string start with this prefix?

If the prefix string is the same as the whole string initial characters –R(ignoring spaces in the whole string) then we return the whole string minus any leading spaces.

intloopPrefix? : String → Union(String,NIL)

— defun intloopPrefix? 0 —

```
(defun |intloopPrefix?| (prefix whole)
  "Does the string start with this prefix?"
  (let ((newprefix (string-left-trim '(#\space) prefix))
        (newwhole (string-left-trim '(#\space) whole)))
    (when (<= (length newprefix) (length newwhole))
      (when (string= newprefix newwhole :end2 (length prefix))
        newwhole))))
```

defun Interpret a line of lisp code

This is used to handle `)lisp` top level commands [nplisp p812]

```
[$currentLine p??]
```

— defun intnplisp —

```
(defun |intnplisp| (s)
  (declare (special |$currentLine|))
  (setq |$currentLine| s)
  (|nplisp| |$currentLine|))
```

—————

Get the current directory

— defun get-current-directory 0 —

```
(defun get-current-directory ()
  "Get the current directory"
  (namestring (truename "")))
```

—————

Prepend the absolute path to a filename

Prefix a filename with the **AXIOM** shell variable.

[[\\$spadroot](#) p201]

— defun make-absolute-filename 0 —

```
(defun make-absolute-filename (name)
  "Prepend the absolute path to a filename"
  (declare (special $spadroot))
  (concatenate 'string $spadroot name))
```

—————

Make the initial modemap frame

[[copy](#) p??]

[[\\$InitialModemapFrame](#) p198]

— defun makeInitialModemapFrame 0 —

```
(defun |makeInitialModemapFrame| ()
  "Make the initial modemap frame"
  (declare (special |$InitialModemapFrame|))
  (copy |$InitialModemapFrame|))
```

defun ncloopEscaped

The ncloopEscaped function will return true if the last non-blank character of a line is an underscore, the Axiom line-continuation character. Otherwise, it returns nil.

— defun ncloopEscaped 0 —

```
(defun |ncloopEscaped| (x)
  (let ((l (length x)))
    (dotimes (i l)
      (when (char= (char x (- l i 1)) #\_) (return t))
      (unless (char= (char x (- l i 1)) #\space) (return nil))))))
```

defun intloopProcessString

```
[setCurrentLine p334]
[intloopProcess p356]
[next p330]
[incString p331]
```

intloopProcessString : (String,StepNo) → StepNo

— defun intloopProcessString —

```
(defun |intloopProcessString| (currentline stepno)
  (|setCurrentLine| currentline)
  (|intloopProcess| stepno t
    (|next| #'|ncloopParse|
      (|next| #'|lineoftoks| (|incString| currentline)))))
```

defun ncloopParse

```
[ncloopDQlines p364]
[npParse p433]
```


[dqToList p634]

— defun ncloopParse —

```
(defun |ncloopParse| (s)
  (let (cudr lines stream dq t1)
    (setq t1 (car s))
    (setq dq (car t1))
    (setq stream (cadr t1))
    (setq t1 (|ncloopDQlines| dq stream))
    (setq lines (car t1))
    (setq cudr (cadr t1))
    (cons (list (list lines (|npParse| (|dqToList| dq)))) (cdr s))))
```

—————

defun next

[Delay p397]

[next1 p330]

next : (Function, Delay) → Delay

— defun next —

```
(defun |next| (function delay)
  (|Delay| #'|next1| (list function delay)))
```

—————

defun next1

[StreamNull p623]

[incAppend p380]

[next p330]

next1 : Delay → ParsePair

— defun next1 —

```
(defun |next1| (&rest delayArg)
  (let (h delay function)
    (setq function (car delayArg))
    (setq delay (cadr delayArg))
    (cond
      ((|StreamNull| delay) |StreamNil|)
```

```
(t
  (setq h (apply function (list delay)))
  (|incAppend| (car h) (|next| function (cdr h))))))
```

defun incString

The **incString** function gets a string, usually from Axiom's input, and constructs a set of nested function calls to process the input line. [incRenum p366]

[incLude p369]

[Top p370]

incString : String → Function
 — defun incString —

```
(defun |incString| (s)
  (declare (special |Top|))
  (|incRenumber| (|incLude| 0 (list s) 0 (list "strings") (list |Top|))))
```

Call the garbage collector

Call the garbage collector on various platforms.

— defun reclaim 0 —

```
#+abcl
(defun reclaim () "Call the garbage collector" (ext::gc))
#+allegro
(defun reclaim () "Call the garbage collector" (excl::gc t))
#+CCL
(defun reclaim () "Call the garbage collector" (gc))
#+clisp
(defun reclaim ()
  "Call the garbage collector"
  (#+lisp=cl ext::gc #-lisp=cl lisp::gc))
#+(or :cmulisp :cmu)
(defun reclaim () "Call the garbage collector" (ext:gc))
#+cormanlisp
(defun reclaim () "Call the garbage collector" (cl::gc))
#+(OR IBCL KCL GCL)
(defun reclaim () "Call the garbage collector" (si::gbc t))
#+lispworks
(defun reclaim () "Call the garbage collector" (hcl::normal-gc))
```

```

#+Lucid
(defun reclaim () "Call the garbage collector" (lcl::gc))
#+sbcl
(defun reclaim () "Call the garbage collector" (sb-ext::gc))

```

defun reroot

The `reroot` function is used to reset the important variables used by the system. In particular, these variables are sensitive to the **AXIOM** shell variable. That variable is renamed internally to be **\$spadroot**. The `reroot` function will change the system to use a new root directory and will have the same effect as changing the **AXIOM** shell variable and rerunning the system from scratch. Note that we have changed from the NAG distribution back to the original form. If you need the NAG version you can push **:tpd** on the ***features*** variable before compiling this file. A correct call looks like:

```

(in-package "BOOT")
(reroot "/spad/mnt/${SYS}")

```

where the `${SYS}` variable is the same one set at build time.

For the example call:

```
(REROOT "/research/test/mnt/ubuntu")
```

the variables are set as:

```
$spadroot = "/research/test/mnt/ubuntu"
```

```

$relative-directory-list =
  ("../../../../src/input/"
   "/doc/messages/"
   "../../../../src/algebra/"
   "../../../../src/interp/"
   "/doc/spadhelp/")

```

```

$directory-list =
  ("/research/test/mnt/ubuntu/../../../../src/input/"
   "/research/test/mnt/ubuntu/doc/messages/"
   "/research/test/mnt/ubuntu/../../../../src/algebra/"
   "/research/test/mnt/ubuntu/../../../../src/interp/"
   "/research/test/mnt/ubuntu/doc/spadhelp/")

```

```
$relative-library-directory-list = ("/algebra/")
```

```
$library-directory-list = ("/research/test/mnt/ubuntu/algebra/")
```

```
|$msgDatabaseName| = nil

$current-directory = "/research/test/"

[make-absolute-filename p328]
[$spadroot p201]
[$directory-list p198]
[$relative-directory-list p200]
[$library-directory-list p199]
[$relative-library-directory-list p200]
[$current-directory p333]
```

— defun reroot —

```
(defun reroot (dir)
  (declare (special $spadroot $directory-list $relative-directory-list
    $library-directory-list $relative-library-directory-list
    $current-directory))
  (setq $spadroot dir)
  (setq $directory-list
    (mapcar #'make-absolute-filename $relative-directory-list))
  (setq $library-directory-list
    (mapcar #'make-absolute-filename $relative-library-directory-list))
  (setq $current-directory $spadroot))
```

—————

defvar \$current-directory

— initvars —

```
(defvar |$currentLine| "" "A list of the input line history")
```

—————

defun setCurrentLine

Remember the current line. The cases are:

- If there is no \$currentLine set it to the input
- Is the current line a string and the input a string? Make them into a list

- Is `$currentLine` not a cons cell? Make it one.
- Is the input a string? Cons it on the end of the list.
- Otherwise stick it on the end of the list

```
[$currentLine p??]
```

```
setCurrentLine : String → List(String)  
— defun setCurrentLine 0 —
```

```
(defun |setCurrentLine| (s)
  (declare (special |$currentLine|))
  (cond
    ((null |$currentLine|) (setq |$currentLine| s))
    ((and (stringp |$currentLine|) (stringp s))
     (setq |$currentLine| (list |$currentLine| s)))
    ((not (cons p |$currentLine|)) (setq |$currentLine| (cons |$currentLine| s)))
    ((stringp s) (rplacd (last |$currentLine|) (cons s nil)))
    (t (rplacd (last |$currentLine|) s)))
  |$currentLine|)
```

Show the Axiom prompt

```
[concat p1237]
[substring p324]
[currenttime p??]
[$inputPromptType p1019]
[$IOindex p39]
[$interpreterFrameName p38]
```

```
mkprompt : Void → String  
— defun mkprompt —
```

```
(defun mkprompt ()
  "Show the Axiom prompt"
  (declare (special |$inputPromptType| |$IOindex| |$interpreterFrameName|))
  (case |$inputPromptType|
    (|none| "")
    (|plain| "-> ")
    (|step| (concat "(" (princ-to-string |$IOindex|) ") -> "))
    (|frame|
     (concat (princ-to-string |$interpreterFrameName|) " ("
              (princ-to-string |$IOindex|) ") -> "))
    (t (concat (princ-to-string |$interpreterFrameName|) " ["
```

```
(substring (currenttime) 8 nil) "]" ["
(princ-to-string |$IOindex|) "]" -> "))))
```

defvar \$frameAlist

— initvars —

```
(defvar |$frameAlist| nil)
```

defvar \$frameNumber

— initvars —

```
(defvar |$frameNumber| 0)
```

defvar \$currentFrameNum

— initvars —

```
(defvar |$currentFrameNum| 0)
```

defvar \$EndServerSession

— initvars —

```
(defvar |$EndServerSession| nil)
```

```
defvar $NeedToSignalSessionManager
```

```
— initvars —
```

```
(defvar |$NeedToSignalSessionManager| nil)
```

```
—————
```

```
defvar $sockBufferLength
```

```
— initvars —
```

```
(defvar |$sockBufferLength| 9217)
```

```
—————
```

READ-LINE in an Axiom server system

```
[coerceFailure p??]
[top-level p??]
[spad-reader p??]
[read-line p??]
[addNewInterpreterFrame p33]
[sockSendInt p??]
[sockSendString p??]
[mkprompt p334]
[sockGetInt p??]
[lassoc p??]
[changeToNamedInterpreterFrame p32]
[sockGetString p??]
[unescapeStringsInForm p354]
[protectedEVAL p339]
[executeQuietCommand p339]
[parseAndInterpret p340]
[serverReadLine is-console (vol9)]
[serverSwitch p??]
[$KillLispSystem p??]
[$NonSmanSession p??]
[$SpadCommand p??]
[$QuietSpadCommand p??]
[$MenuServer p??]
```

[\[sockBufferLength p336\]](#)
[\[\\$LispCommand p??\]](#)
[\[\\$EndServerSession p335\]](#)
[\[\\$EndSession p??\]](#)
[\[\\$SwitchFrames p??\]](#)
[\[\\$CreateFrameAnswer p??\]](#)
[\[\\$currentFrameNum p335\]](#)
[\[\\$frameNumber p335\]](#)
[\[\\$frameAlist p335\]](#)
[\[\\$CreateFrame p??\]](#)
[\[\\$CallInterp p??\]](#)
[\[\\$EndOfOutput p??\]](#)
[\[\\$SessionManager p??\]](#)
[\[\\$NeedToSignalSessionManager p336\]](#)
[\[\\$EndServerSession p335\]](#)
[\[\\$SpadServer p201\]](#)
[\[*eof* p314\]](#)
[\[in-stream p1147\]](#)

serverReadLine : Stream → String
 — defun serverReadLine —

```

(defun |serverReadLine| (stream)
  "used in place of READ-LINE in a Axiom server system."
  (let (in-stream *eof* 1 framename currentframe form stringbuf line action)
    (declare (special in-stream *eof* |$SpadServer| |$EndServerSession|
      |$NeedToSignalSessionManager| |$SessionManager| |$EndOfOutput| | |
      |$CallInterp| |$CreateFrame| |$frameAlist| |$frameNumber|
      |$currentFrameNum| |$CreateFrameAnswer| |$SwitchFrames| |$EndSession|
      |$EndServerSession| |$LispCommand| |$sockBufferLength| |$MenuServer|
      |$QuietSpadCommand| |$SpadCommand| |$NonSmanSession| |$KillLispSystem|))
    (force-output)
    (if (or (null |$SpadServer|) (null (is-console stream)))
      (|read-line| stream)
      (progn
        (setq in-stream stream)
        (setq *eof* nil)
        (setq line
          (do ()
            ((null (and (null |$EndServerSession|) (null *eof*))) nil)
            (when |$NeedToSignalSessionManager|
              (|sockSendInt| |$SessionManager| |$EndOfOutput|))
            (setq |$NeedToSignalSessionManager| nil)
            ; see bookvol8 for the constants that serverSwitch returns
            (setq action (|serverSwitch|))
            (cond
              ((= action |$CallInterp|)
               (setq 1 (|read-line| stream))

```



```

(setq |$NeedToSignalSessionManager| t)
(return 1))
(= action |$CreateFrame|)
(setq framename (gentemp "frame"))
(|addNewInterpreterFrame| framename)
(setq |$frameAlist|
  (cons (cons |$frameNumber| framename) |$frameAlist|))
(setq |$currentFrameNum| |$frameNumber|)
(|sockSendInt| |$SessionManager| |$CreateFrameAnswer|)
(|sockSendInt| |$SessionManager| |$frameNumber|)
(setq |$frameNumber| (1+ |$frameNumber|))
(|sockSendString| |$SessionManager| (mkprompt)))
(= action |$SwitchFrames|)
(setq |$currentFrameNum| (|sockGetInt| |$SessionManager|))
(setq currentframe (lassoc |$currentFrameNum| |$frameAlist|))
(|changeToNamedInterpreterFrame| currentframe))
(= action |$EndSession|)
(setq |$EndServerSession| t))
(= action |$LispCommand|)
(setq |$NeedToSignalSessionManager| t)
(setq stringbuf (make-string |$sockBufferLength|))
(|sockGetString| |$MenuServer| stringbuf |$sockBufferLength|)
(setq form (|unescapeStringsInForm| (read-from-string stringbuf)))
(|protectedEVAL| form))
(= action |$QuietSpadCommand|)
(setq |$NeedToSignalSessionManager| t)
(|executeQuietCommand|))
(= action |$SpadCommand|)
(setq |$NeedToSignalSessionManager| t)
(setq stringbuf (make-string 512))
(|sockGetString| |$MenuServer| stringbuf 512)
(catch '|coerceFailure|
  (catch '|top_level|
    (catch '|spad_reader
      (|parseAndInterpret| stringbuf))))
(princ (mkprompt))
(finish-output))
(= action |$NonSmanSession|) (setq |$SpadServer| nil))
(= action |$KillLispSystem|) (bye))
(t nil))))
(cond
  (line line)
  (t '||))))))

```

defun protectedEVAL

```
[resetStackLimits p312]
[sendHTErrorSignal p??]
```

— defun protectedEVAL —

```
(defun |protectedEVAL| (x)
  (let (val (error t))
    (unwind-protect
      (progn
        (setq val (eval x))
        (setq error nil))
      (when error
        (|resetStackLimits|)
        (|sendHTErrorSignal|))))
    (unless error val)))
```

—————

defvar \$QuietCommand

— initvars —

```
(defvar |$QuietCommand| nil "If true, produce no top level output")
```

—————

defun executeQuietCommand

When `$QuietCommand` is true Spad will not produce any output from a top level command

```
[spad-reader p??]
[coerceFailure p??]
[toplevel p??]
[spadreader p??]
[make-string p??]
[sockGetString p??]
[parseAndInterpret p340]
[$MenuServer p??]
[$QuietCommand p339]
```

— defun executeQuietCommand —

```
(defun |executeQuietCommand| ()
  (let (|$QuietCommand| stringBuffer)
    (declare (special |$QuietCommand| |$MenuServer|))
    (setq |$QuietCommand| t)
    (setq stringBuffer (make-string 512))
    (|sockGetString| |$MenuServer| stringBuffer 512)
    (catch '|coerceFailure|
      (catch '|top_level|
        (catch '|spad_reader| (|parseAndInterpret| stringBuffer))))))
```

defun parseAndInterpret

```
[|InteractiveMode| p315]
|$boot| p826]
|$spad| p310]
|$e| p315]
[|InteractiveFrame| p39]
```

— defun parseAndInterpret —

```
(defun |parseAndInterpret| (str)
  (let (|$InteractiveMode| $boot $spad |$e|)
    (declare (special |$InteractiveMode| $boot $spad |$e|
                      |$InteractiveFrame|))
    (setq |$InteractiveMode| t)
    (setq $boot nil)
    (setq $spad t)
    (setq |$e| |$InteractiveFrame|)
    (|processInteractive| (|parseFromString| str) nil)))
```

defun parseFromString

```
[next| p330]
[|ncloopParse| p329]
[|lineoftoks| p403]
[|incString| p331]
[|StreamNull| p623]
[|pf2Sex| p597]
[|macroExpanded| p518]
```

— defun parseFromString —

```
(defun |parseFromString| (s)
  (setq s (|next| #'|ncloopParse| (|next| #'|lineoftoks| (|incString| s))))
  (unless (|StreamNull| s) (|pf2Sex| (|macroExpanded| (cadar s)))))
```

defvar \$interpOnly

— initvars —

```
(defvar |$interpOnly| nil)
```

defvar \$minivectorNames

— initvars —

```
(defvar |$minivectorNames| nil)
```

defvar \$domPvar

— initvars —

```
(defvar |$domPvar| nil)
```

defvar \$compilingMap

\$compilingMap: true when compiling a map, used to detect where to THROW when interpret-only is invoked

— initvars —

```
(defvar |$compilingMap| ())
```

defvar \$instantRecord

— initvars —

```
(setq |$instantRecord| (make-hash-table :test #'eq))
```

—————

defun processInteractive

Parser Output --> Interpreter

Top-level dispatcher for the interpreter. It sets local variables and then calls processInteractive1 to do most of the work. This function receives the output from the parser.

```
[initializeTimedNames p??]
[qcar p??]
[processInteractive1 p344]
[reportInstantiations p1015]
[clrhash p??]
[writeHistModesAndValues p913]
[updateHist p899]
[$op p??]
[$Coerce p??]
[$compErrorMessageStack p??]
[$freeVars p??]
[$mapList p??]
[$compilingMap p341]
[$compilingLoop p??]
[$interpOnly p341]
[$whereCacheList p??]
[$timeGlobalName p??]
[$StreamFrame p??]
[$declaredMode p??]
[$localVars p308]
[$analyzingMapList p??]
[$lastLineInSEQ p??]
[$instantCoerceCount p??]
[$instantCanCoerceCount p??]
[$instantMmCondCount p??]
[$fortVar p??]
[$minivector p??]
[$minivectorCode p??]
[$minivectorNames p341]
[$domPvar p341]
```

```

[$inRetract p??]
[$instantRecord p342]
[$reportInstantiations p1015]
[$ProcessInteractiveValue p344]
[$defaultFortVar p??]
[$interpreterTimedNames p??]
[$interpreterTimedClasses p??]

```

— *defun processInteractive* —

```

(defun |processInteractive| (form posnForm)
  (let (|$op| |$Coerce| |$compErrorMessageStack| |$freeVars|
        |$mapList| |$compilingMap| |$compilingLoop|
        |$interpOnly| |$whereCacheList| |$timeGlobalName|
        |$StreamFrame| |$declaredMode| |$localVars|
        |$analyzingMapList| |$lastLineInSEQ|
        |$instantCoerceCount| |$instantCanCoerceCount|
        |$instantMmCondCount| |$fortVar| |$minivector|
        |$minivectorCode| |$minivectorNames| |$domPvar|
        |$inRetract| object)
    (declare (special |$op| |$Coerce| |$compErrorMessageStack|
                      |$freeVars| |$mapList| |$compilingMap|
                      |$compilingLoop| |$interpOnly| |$whereCacheList|
                      |$timeGlobalName| |$StreamFrame| |$declaredMode|
                      |$localVars| |$analyzingMapList| |$lastLineInSEQ|
                      |$instantCoerceCount| |$instantCanCoerceCount|
                      |$instantMmCondCount| |$fortVar| |$minivector|
                      |$minivectorCode| |$minivectorNames| |$domPvar|
                      |$inRetract| |$instantRecord| |$reportInstantiations|
                      |$ProcessInteractiveValue| |$defaultFortVar|
                      |$interpreterTimedNames| |$interpreterTimedClasses|))
      (|initializeTimedNames| |$interpreterTimedNames| |$interpreterTimedClasses|)
      (if (consp form) ; compute name of operator
          (setq |$op| (qcar form))
          (setq |$op| form))
      (setq |$Coerce| nil)
      (setq |$compErrorMessageStack| nil)
      (setq |$freeVars| nil)
      (setq |$mapList| nil) ; list of maps being type analyzed
      (setq |$compilingMap| nil) ; true when compiling a map
      (setq |$compilingLoop| nil) ; true when compiling a loop body
      (setq |$interpOnly| nil) ; true when in interp only mode
      (setq |$whereCacheList| nil) ; maps compiled because of where
      (setq |$timeGlobalName| '|$compTimeSum|); see incrementTimeSum
      (setq |$StreamFrame| nil) ; used in printing streams
      (setq |$declaredMode| nil) ; weak type propagation for symbols
      (setq |$localVars| nil) ; list of local variables in function
      (setq |$analyzingMapList| nil) ; names of maps currently being analyzed
      (setq |$lastLineInSEQ| t) ; see evalIF and friends

```

```

(setq |$instantCoerceCount| 0)
(setq |$instantCanCoerceCount| 0)
(setq |$instantMmCondCount| 0)
(setq |$defaultFortVar| 'x)           ; default FORTRAN variable name
(setq |$fortVar| |$defaultFortVar|)   ; variable name for FORTRAN output
(setq |$minivector| nil)
(setq |$minivectorCode| nil)
(setq |$minivectorNames| nil)
(setq |$domPvar| nil)
(setq |$inRetract| nil)
(setq object (|processInteractive1| form posnForm))
(unless |$ProcessInteractiveValue|
  (when |$reportInstantiations|
    (|reportInstantiations|)
    (clrhash |$instantRecord|))
    (|writeHistModesAndValues|)
    (|updateHist|))
  object))

```

defvar \$ProcessInteractiveValue

— initvars —

```
(defvar |$ProcessInteractiveValue| nil "If true, no output or record")
```

defvar \$HTCompanionWindowID

— initvars —

```
(defvar |$HTCompanionWindowID| nil)
```

defun processInteractive1

This calls the analysis and output printing routines [recordFrame p1123]
[startTimingProcess p??]

```
[interpretTopLevel p345]
[stopTimingProcess p??]
[recordAndPrint p349]
[objValUnwrap p516]
[objMode p516]
[$e p315]
[$ProcessInteractiveValue p344]
[$InteractiveFrame p39]
```

— **defun processInteractive1** —

```
(defun |processInteractive1| (form posnForm)
  (let (|$e| object)
    (declare (special |$e| |$ProcessInteractiveValue| |$InteractiveFrame|))
    (setq |$e| |$InteractiveFrame|)
    (recordFrame 'system)
    (|startTimingProcess| '|analysis|)
    (setq object (|interpretTopLevel| form posnForm))
    (|stopTimingProcess| '|analysis|)
    (|startTimingProcess| '|print|)
    (unless |$ProcessInteractiveValue|
      (|recordAndPrint| (|objValUnwrap| object) (|objMode| object)))
    (recordFrame 'normal)
    (|stopTimingProcess| '|print|)
    object))
```

defun interpretTopLevel

```
[interpreter p??]
[interpret p346]
[stopTimingProcess p??]
[peekTimedName p??]
[interpretTopLevel p345]
[$timedNameStack p??]
```

— **defun interpretTopLevel** —

```
(defun |interpretTopLevel| (x posnForm)
  (let (savedTimerStack c)
    (declare (special |$timedNameStack|))
    (setq savedTimerStack (copy |$timedNameStack|))
    (setq c (catch '|interpreter| (|interpret| x posnForm)))
    (do ()
      ((equal savedTimerStack |$timedNameStack|) nil)
```



```
(|stopTimingProcess| (|peekTimedName|)))
(if (eq c 'tryAgain|)
  (|interpretTopLevel| x posnForm
  c)))
```

defvar \$genValue

If the `$genValue` variable is true then evaluate generated code, otherwise leave code unevaluated. If `$genValue` is false then we are compiling. This variable is only defined and used locally.

— initvars —

```
(defvar |$genValue| nil "evaluate generated code if true")
```

defun Type analyzes and evaluates expression x, returns object

```
[interpret1 p347]
[$env p315]
[$eval p??]
[$genValue p346]
```

— defun interpret —

```
(defun |interpret| (&rest arg &aux restarts x)
  (let (|$env| |$eval| |$genValue| posnForm)
    (declare (special |$env| |$eval| |$genValue|))
    (setq x (car arg))
    (setq restarts (cdr arg))
    (if (consp restarts)
      (setq posnForm (car restarts))
      (setq posnForm restarts))
    (setq |$env| (list (list nil)))
    (setq |$eval| t) ; generate code -- don't just type analyze
    (setq |$genValue| t) ; evaluate all generated code
    (|interpret1| x nil posnForm)))
```

defun Dispatcher for the type analysis routines

This is the dispatcher for the type analysis routines. It type analyzes and evaluates the expression `x` in the `rootMode` (if non-`nil`) which may be `$EmptyMode`. It returns an object if evaluating, and a modeset otherwise. It creates the attributed tree.

```
[mkAtreeWithSrcPos p??]
[putTarget p??]
[bottomUp p??]
[getArgValue p??]
[mkObj p514]
[getValue p??]
[interpret2 p348]
[keyedSystemError p??]
[$genValue p346]
[$eval p??]
```

— defun interpret1 —

```
(defun |interpret1| (x rootMode posnForm)
  (let (node modeSet newRootMode argVal val)
    (declare (special |$genValue| |$eval|))
    (setq node (|mkAtreeWithSrcPos| x posnForm))
    (when rootMode (|putTarget| node rootMode))
    (setq modeSet (|bottomUp| node))
    (if (null |$eval|)
        modeSet
        (progn
          (if (null rootMode)
              (setq newRootMode (car modeSet))
              (setq newRootMode rootMode))
          (setq argVal (|getArgValue| node newRootMode))
          (cond
            ((and argVal (null |$genValue|))
             (mkObj argVal newRootMode))
            ((and argVal (setq val (|getValue| node)))
             (|interpret2| val newRootMode posnForm))
            (t
             (|keyedSystemError|
              "Interpreter code generation failed for expression %1s"
              (list x)))))))
```

defvar \$ThrowAwayMode

— **initvars** —

```
(defvar |$ThrowAwayMode| '|$ThrowAwayMode| "interp constant")
```

defun interpret2

This is the late interpretCoerce. I removed the call to coerceInteractive, so it only does the JENKS cases.

```
[objVal p516]
[objMode p516]
[member p1238]
[mkObj p514]
[systemErrorHere p??]
[coerceInteractive p740]
[throwKeyedMsgCannotCoerceWithValue p??]
[$EmptyMode p708]
[$ThrowAwayMode p347]
```

— **defun interpret2** —

```
(defun |interpret2| (object m1 posnForm)
  (declare (ignore posnForm))
  (let (x m op ans)
    (declare (special |$EmptyMode| |$ThrowAwayMode|))
    (cond
      ((equal m1 |$ThrowAwayMode|) object)
      (t
       (setq x (|objVal| object))
       (setq m (|objMode| object))
       (cond
         ((equal m |$EmptyMode|)
          (cond
            ((and (consp x)
                  (progn (setq op (qcar x)) t)
                  (|member| op '(map stream)))
             (mkObj x m1))
            ((equal m1 |$EmptyMode|)
             (mkObj x m))
            (t
             (|systemErrorHere| "interpret2")))))
         (m1
          (if (setq ans (|coerceInteractive| object m1))
              ans
              (|throwKeyedMsgCannotCoerceWithValue| x m m1))))))
```

```
(t object))))))
```

defvar \$runTestFlag

This is referenced by maPrin to stash output by recordAndPrint to not print type/time

— **initvars** —

```
(defvar |$runTestFlag| nil)
```

defvar \$mkTestFlag

This referenced by READLN to stash input by maPrin to stash output by recordAndPrint to write i/o onto \$testStream

— **initvars** —

```
(defvar |$mkTestFlag| nil)
```

defun Result Output Printing

Prints out the value x which is of type m, and records the changes in environment \$e into \$InteractiveFrame \$printAnyIfTrue is documented in setvar.boot. It is controlled with the)se me any command.

```
[output p??]
[putHist p900]
[mkObjWrap p515]
[printTypeAndTime p351]
[printStorage p351]
[printStatisticsSummary p351]
[mkCompanionPage p??]
[recordAndPrintTest p??]
[$outputMode p??]
[$mkTestOutputType p??]
[$runTestFlag p349]
[$e p315]
[$mkTestFlag p349]
```

```

[HTCompanionWindowID p344]
[QuietCommand p339]
[printStatsSummaryIfTrue p1022]
[printTypeIfTrue p1024]
[printStorageIfTrue p??]
[printTimeIfTrue p1023]
[Void p713]
[algebraOutputStream p1035]
[collectOutput p??]
[EmptyMode p708]
[printVoidIfTrue p1025]
[outputMode p??]
[printAnyIfTrue p1007]

```

— defun recordAndPrint —

```

(defun |recordAndPrint| (x md)
  (let (|$outputMode| xp mdp mode)
    (declare (special |$outputMode| |$mkTestOutputType| |$runTestFlag| |$e|
                      |$mkTestFlag| |HTCompanionWindowID| |QuietCommand|
                      |printStatsSummaryIfTrue| |printTypeIfTrue|
                      |printStorageIfTrue| |printTimeIfTrue| |Void|
                      |algebraOutputStream| |collectOutput| |EmptyMode|
                      |printVoidIfTrue| |outputMode| |printAnyIfTrue|))
      (cond
        ((and (equal md '(|Any|)) |$printAnyIfTrue|)
         (setq mdp (car x))
         (setq xp (cdr x)))
        (t
         (setq mdp md)
         (setq xp x)))
      (setq |$outputMode| md)
      (if (equal md |EmptyMode|)
          (setq mode (|quadSch|))
          (setq mode md))
      (when (or (not (equal md |Void|)) |$printVoidIfTrue|)
        (unless |$collectOutput| (terpri |algebraOutputStream|))
        (unless |$QuietCommand| (|output| xp mdp)))
      (|putHist| '% '|value| (mkObjWrap x md) |$e|)
      (when (or |$printTimeIfTrue| |$printTypeIfTrue|)
        (|printTypeAndTime| xp mdp))
      (when |$printStorageIfTrue| (|printStorage|))
      (when |$printStatsSummaryIfTrue| (|printStatsSummary|))
      (when (integerp |HTCompanionWindowID|) (|mkCompanionPage| md))
      (cond
        (|$mkTestFlag| (|recordAndPrintTest| md))
        (|$runTestFlag|
         (setq |$mkTestOutputType| md)
         '|done|))

```

```
(t ' |done|))))
```

defun printStatisticsSummary

```
[sayKeyedMsg p43]
[statisticsSummary p??]
[collectOutput p??]
```

— defun printStatisticsSummary —

```
(defun |printStatisticsSummary| ()
  (declare (special |$collectOutput|))
  (unless |$collectOutput|
    (|sayKeyedMsg| "%rjon Summary: %1 %rjoff" (list (|statisticsSummary|)))))
```

defun printStorage

```
[makeLongSpaceString p??]
[$interpreterTimedClasses p??]
[collectOutput p??]
[$interpreterTimedNames p??]
```

— defun printStorage —

```
(defun |printStorage| ()
  (declare (special |$interpreterTimedClasses| |$collectOutput|
                  |$interpreterTimedNames|))
  (unless |$collectOutput|
    (|sayKeyedMsg| "%rjon Storage: %1 %rjoff"
      (list
        (|makeLongSpaceString|
          |$interpreterTimedNames|
          |$interpreterTimedClasses|))))))
```

defun printTypeAndTime

```
[retract p1269]
[qcar p??]
```

```

[retract p1269]
[mkObjWrap p515]
[objMode p516]
[sameUnionBranch p353]
[makeLongTimeString p??]
[msgText p354]
[sayKeyedMsg p43]
[justifyMyType p354]
[$collectOutput p??]
[$printTypeIfTrue p1024]
[$printTimeIfTrue p1023]
[$outputLines p??]
[$interpreterTimedNames p??]
[$interpreterTimedClasses p??]

```

— defun printTypeAndTime —

```

(defun |printTypeAndTime| (x m)
  (let (xp mp timeString result)
    (declare (special |$outputLines| |$collectOutput| |$printTypeIfTrue|
                      |$printTimeIfTrue| |$outputLines|
                      |$interpreterTimedNames| |$interpreterTimedClasses|))
    (cond
      ((and (consp m) (eq (qcar m) '|Union|))
       (setq xp (|retract| (mkObjWrap x m)))
       (setq mp (|objMode| xp))
       (setq m
        (cons '|Union|
              (append
               (dolist (arg (qcdr m) (nreverse result))
                 (when (|sameUnionBranch| arg mp) (push arg result)))
               (list "...")))))
      (when |$printTimeIfTrue|
       (setq timeString
        (|makeLongTimeString|
         |$interpreterTimedNames|
         |$interpreterTimedClasses|)))
      (cond
        ((and |$printTimeIfTrue| |$printTypeIfTrue|)
         (if |$collectOutput|
          (push (|msgText| "%rjon Type: %1p %rjoff" (list m)) |$outputLines|)
          (|sayKeyedMsg| "%rjon Type: %1p %l Time: %2 %rjoff"
            (list m timeString))))
        (|$printTimeIfTrue|
         (unless |$collectOutput|
          (|sayKeyedMsg| "%rjon Time: %1 %rjoff" (list timeString))))
        (|$printTypeIfTrue|
         (if |$collectOutput|

```

```
(push (|justifyMyType|
      (|msgText| "%rjon Type: %1p %rjoff" (list m))) |$outputLines|)
(|sayKeyedMsg| "%rjon Type: %1p %rjoff" (list m))))))
```

defun printAsTeX

[*\$texOutputStream* *p??*]

— defun printAsTeX 0 —

```
(defun |printAsTeX| (x)
  (declare (special |$texOutputStream|))
  (princ x |$texOutputStream|))
```

defun sameUnionBranch

```
sameUnionBranch(uArg, m) ==
  uArg is [":", ., t] => t = m
  uArg = m
```

— defun sameUnionBranch 0 —

```
(defun |sameUnionBranch| (uArg m)
  (let (t1 t2 t3)
    (cond
      ((and (consp uArg)
            (eq (qcar uArg) '|:|)
            (progn
              (setq t1 (qcdr uArg))
              (and (consp t1)
                    (progn
                     (setq t2 (qcdr t1))
                     (and (consp t2)
                           (eq (qcdr t2) nil)
                           (progn (setq t3 (qcar t2)) t)))))))
      (equal t3 m))
    (t (equal uArg m)))))
```

defun msgText

```
[segmentKeyedMsg p44]
[substituteSegmentedMsg p??]
[flowSegmentedMsg p??]
[$linelength p1051]
[$margin p1051]
```

— defun msgText —

```
(defun |msgText| (key args)
  (let (msg)
    (declare (special $linelength $margin))
    (setq msg (|segmentKeyedMsg| key))
    (setq msg (|substituteSegmentedMsg| msg args))
    (setq msg (|flowSegmentedMsg| msg $linelength $margin))
    (apply #'concat (mapcar #'princ-to-string (cdar msg)))))
```

—————

defun Right-justify the Type output

```
[fillerSpaces p310]
[$linelength p1051]
```

— defun justifyMyType —

```
(defun |justifyMyType| (arg)
  (let (len)
    (declare (special $linelength))
    (setq len (|#| arg))
    (if (> len $linelength)
      arg
      (concat (|fillerSpaces| (- $linelength len)) arg))))
```

—————

defun Destructively fix quotes in strings

```
[unescapeStringsInForm p354]
[$funnyBacks p1545]
[$funnyQuote p1544]
```

— defun unescapeStringsInForm —

```
(defun |unescapeStringsInForm| (form)
  (let (str)
    (declare (special |$funnyBacks| |$funnyQuote|))
    (cond
      ((stringp form)
       (setq str (nsubstitute #" |$funnyQuote| form))
       (nsubstitute #\\ |$funnyBacks| str))
      ((consp form)
       (|unescapeStringsInForm| (car form))
       (|unescapeStringsInForm| (cdr form))
       form)
      (t form))))
```

Include a file into the stream

[intloopInclude0 p355]

— defun intloopInclude —

```
(defun |intloopInclude| (name n)
  "Include a file into the stream"
  (with-open-file (st name) (|intloopInclude0| st name n)))
```

defun intloopInclude0

[incStream p366]
 [intloopProcess p356]
 [next p330]
 [intloopEchoParse p361]
 [insertpile p625]
 [lineoftoks p403]
 [\$lines p??]

— defun intloopInclude0 —

```
(defun |intloopInclude0| (|st| |name| |n|)
  (let (|$lines|)
    (declare (special |$lines|))
    (setq |$lines| (|incStream| |st| |name|))
    (|intloopProcess| |n| NIL
      (|next| #'|intloopEchoParse|
```

```
(|next| #'|insertpile|
(|next| #'|lineoftoks|
|$lines|))))))
```

defun intloopProcess

An example call looks like:

```
3> (|intloopProcess| 1 T
(|nonnullstream| #0=|next1| |ncloopParse|
(|nonnullstream| #0# |lineoftoks|
(|nonnullstream| |incZip1| |incRenumberLine|
(|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
(|nonnullstream| |incIgen1| 0))))))
```

which was constructed **intloopProcessString**(p329). This call says we are processing the first input, in this case “1”. It is interactive. The third argument, the delay, contains the information to drive the rest of the process. [StreamNull p623]

[pfAbSynOp? p702]
[setCurrentLine p334]
[tokPart p703]
[intloopProcess p356]
[intloopSpadProcess p357]
[\$systemCommandFunction p??]
[\$systemCommandFunction p??]

intloopProcess : (StepNo,Boolean,Delay) → StepNo
— defun intloopProcess —

```
(defun |intloopProcess| (stepno interactive delay)
  (let (ptree lines t1)
    (declare (special |$systemCommandFunction|))
    (cond
      ((|StreamNull| delay) stepno)
      (t
        (setq t1 (car delay))
        (setq lines (car t1))
        (setq ptree (cadr t1))
        (cond
          ((|pfAbSynOp?| ptree ' |command|)
            (when interactive (|setCurrentLine| (|tokPart| ptree)))
            (funcall |$systemCommandFunction| (|tokPart| ptree))
            (|intloopProcess| stepno interactive (cdr delay)))
          (t
```

```
(|intloopProcess|
  (|intloopSpadProcess| stepno lines ptree interactive)
  interactive (cdr delay))))))
```

defun intloopSpadProcess

```
[flung p??]
[SpadCompileItem p??]
[intCoerceFailure p323]
[intSpadReader p323]
[ncPutQ p706]
[CatchAsCan p??]
[Catch p??]
[intloopSpadProcess,interp p358]
[$currentCarrier p??]
[$ncMsgList p318]
[$intCoerceFailure p323]
[$intSpadReader p323]
[$prevCarrier p??]
[$stepNo p??]
[$NeedToSignalSessionManager p336]
[flung p??]
```

— defun intloopSpadProcess —

```
(defun |intloopSpadProcess| (stepNo lines ptree interactive?)
  (let (|$stepNo| result cc)
    (declare (special |$stepNo| |$prevCarrier| |$intSpadReader| |flung|
                      |$intCoerceFailure| |$ncMsgList| |$currentCarrier|
                      |$NeedToSignalSessionManager|))
    (setq |$stepNo| stepNo)
    (setq |$currentCarrier| (setq cc (list '|carrier|)))
    (|ncPutQ| cc '|stepNumber| stepNo)
    (|ncPutQ| cc '|messages| |$ncMsgList|)
    (|ncPutQ| cc '|lines| lines)
    (setq |$ncMsgList| nil)
    (setq result
      (catch '|SpadCompileItem|
        (catch |$intCoerceFailure|
          (catch |$intSpadReader|
            (|intloopSpadProcess,interp| cc ptree interactive?))))))
    (setq |$NeedToSignalSessionManager| t)
    (setq |$prevCarrier| |$currentCarrier|)
    (cond
```

```

((eq result '|ncEnd|) stepNo)
((eq result '|ncError|) stepNo)
((eq result '|ncEndItem|) stepNo)
(t (1+ stepNo))))

```

defun intloopSpadProcess,interp

```

[ncConversationPhase p360]
[ncEltQ p706]
[ncError p361]

```

— defun intloopSpadProcess,interp —

```

(defun |intloopSpadProcess,interp| (cc ptree interactive?)
  (|ncConversationPhase| #'|phParse| (list cc ptree))
  (|ncConversationPhase| #'|phMacro| (list cc))
  (|ncConversationPhase| #'|phIntReportMsgs| (list cc interactive?))
  (|ncConversationPhase| #'|phInterpret| (list cc))
  (unless (eql (length (|ncEltQ| cc '|messages|)) 0) (|ncError|)))

```

defun phParse

TPDHERE: The pform function has a leading percent sign

```
phParse: carrier[tokens,...] -> carrier[ptree, tokens,...]
```

```
[ncPutQ p706]
```

— defun phParse —

```

(defun |phParse| (carrier ptree)
  (|ncPutQ| carrier '|ptree| ptree)
  'ok)

```

defun phIntReportMsgs

```
carrier[lines,messages,...]-> carrier[lines,messages,...]
```

[ncEltQ p706]
 [ncPutQ p706]
 [processMsgList p659]
 [\$erMsgToss p??]

— defun phIntReportMsgs —

```
(defun |phIntReportMsgs| (carrier interactive?)
  (declare (ignore interactive?))
  (let (nerr msgs lines)
    (declare (special |$erMsgToss|))
    (cond
      (|$erMsgToss| 'ok)
      (t
       (setq lines (|ncEltQ| carrier '|lines|))
       (setq msgs (|ncEltQ| carrier '|messages|))
       (setq nerr (length msgs))
       (|ncPutQ| carrier '|ok?'| (eql nerr 0))
       (cond
         ((eql nerr 0) 'ok)
         (t
          (|processMsgList| msgs lines)
          (|sayKeyedMsg| "%1 error(s) parsing " (list nerr))
          'ok))))))
```

defun phInterpret

[ncEltQ p706]
 [intInterpretPform p360]
 [ncPutQ p706]

— defun phInterpret —

```
(defun |phInterpret| (carrier)
  (let (val ptree)
    (setq ptree (|ncEltQ| carrier '|ptree|))
    (setq val (|intInterpretPform| ptree))
    (|ncPutQ| carrier '|value| val)))
```

defun intInterpretPform

[processInteractive p342]
 [zeroOneTran p360]
 [pf2Sex p597]

— defun intInterpretPform —

```
(defun |intInterpretPform| (pf)
  (|processInteractive| (|zeroOneTran| (|pf2Sex| pf)) pf))
```

—————

defun zeroOneTran

[nsubst p??]

— defun zeroOneTran 0 —

```
(defun |zeroOneTran| (sex)
  (nsubst '|$EmptyMode| '? sex))
```

—————

defun ncConversationPhase

[ncConversationPhase,wrapup p361]
 [\$ncMsgList p318]

— defun ncConversationPhase —

```
(defun |ncConversationPhase| (fn args)
  (let (|$ncMsgList| carrier)
    (declare (special |$ncMsgList|))
    (setq carrier (car args))
    (setq |$ncMsgList| nil)
    (unwind-protect
      (apply fn args)
      (|ncConversationPhase,wrapup| carrier))))
```

—————

defun ncConversationPhase,wrapup

[\$ncMsgList p318]

— defun ncConversationPhase,wrapup —

```

(defun |ncConversationPhase,wrapup| (carrier)
  (declare (special |$ncMsgList|))
  ((lambda (Var5 m)
    (loop
      (cond
        ((or (atom Var5) (progn (setq m (car Var5)) nil))
         (return nil))
        (t
         (|ncPutQ| carrier '|messages| (cons m (|ncEltQ| carrier '|messages|))))
      (setq Var5 (cdr Var5))))
    |$ncMsgList| nil))

```

defun ncError

[SpadCompileItem p??]

— defun ncError 0 —

```

(defun |ncError| ()
  (throw '|SpadCompileItem| '|ncError|))

```

defun intloopEchoParse

```

[ncloopDQlines p364]
[setCurrentLine p334]
[mkLineList p362]
[ncloopPrintLines p362]
[npParse p433]
[dqToList p634]
[$EchoLines p??]
[$lines p??]

```

— defun intloopEchoParse —


```
(defun |intloopEchoParse| (s)
  (let (cudr lines stream dq t1)
    (declare (special |$EchoLines| |$lines|))
    (setq t1 (car s))
    (setq dq (car t1))
    (setq stream (cadr t1))
    (setq t1 (|ncloopDQlines| dq |$lines|))
    (setq lines (car t1))
    (setq cudr (cadr t1))
    (|setCurrentLine| (|mkLineList| lines))
    (when |$EchoLines| (|ncloopPrintLines| lines))
    (setq |$lines| cudr)
    (cons (list (list lines (|npParse| (|dqToList| dq)))) (cdr s))))
```

defun ncloopPrintLines

```
;ncloopPrintLines lines ==
;      for line in lines repeat WRITE_-LINE CDR line
;      WRITE_-LINE ' " "
```

— defun ncloopPrintLines 0 —

```
(defun |ncloopPrintLines| (lines)
  ((lambda (Var4 line)
    (loop
      (cond
        ((or (atom Var4) (progn (setq line (car Var4)) nil))
          (return nil))
        (t (write-line (cdr line))))
      (setq Var4 (cdr Var4))))
    lines nil)
  (write-line " "))
```

defun mkLineList

```
;mkLineList lines ==
;  l := [CDR line for line in lines | nonBlank CDR line]
;  #l = 1 => CAR l
;  l
```

— defun mkLineList —

```
(defun |mkLineList| (lines)
  (let (l)
    (setq l
      ((lambda (Var2 Var1 line)
         (loop
          (cond
            ((or (atom Var1) (progn (setq line (car Var1)) nil))
             (return (nreverse Var2)))
            (t
             (and (|nonBlank| (cdr line))
                  (setq Var2 (cons (cdr line) Var2))))))
         (setq Var1 (cdr Var1))))
      nil lines nil))
    (cond
      ((eql (length l) 1) (car l))
      (t l))))
```

—————

defun nonBlank

```
;nonBlank str ==
; value := false
; for i in 0..MAXINDEX str repeat
;   str.i ^= char " " =>
;     value := true
;   return value
; value
```

— defun nonBlank 0 —

```
(defun |nonBlank| (str)
  (let (value)
    ((lambda (Var3 i)
       (loop
        (cond
          ((> i Var3) (return nil))
          (t
           (cond
            ((not (equal (elt str i) #\Space))
             (identity (progn (setq value t) (return value))))))
          (setq i (+ i 1))))
       (maxindex str) 0))
```

```
value))
```

defun ncloopDQlines

```
[StreamNull p623]
[poGlobalLinePosn p364]
[tokPosn p703]
[streamChop p364]
```

— defun ncloopDQlines —

```
(defun |ncloopDQlines| (dq stream)
  (let (b a)
    (|StreamNull| stream)
    (setq a (|poGlobalLinePosn| (|tokPosn| (cadr dq))))
    (setq b (|poGlobalLinePosn| (caar stream)))
    (|streamChop| (+ (- a b) 1) stream)))
```

defun poGlobalLinePosn

```
[lnGlobalNum p636]
[poGetLineObject p651]
[ncBug p658]
```

— defun poGlobalLinePosn —

```
(defun |poGlobalLinePosn| (posn)
  (if posn
    (|lnGlobalNum| (|poGetLineObject| posn))
    (|ncBug| "old style pos objects have no global positions" nil)))
```

defun streamChop

Note that changing the name “lyne” to “line” will break the system. I do not know why. The symptom shows up when there is a file with a large contiguous comment spanning enough lines to overflow the stack.

[StreamNull p623]
 [streamChop p364]
 [ncloopPrefix? p818]

— defun streamChop —

```
(defun |streamChop| (n s)
  (let (d c lyne b a tmp1)
    (cond
      ((|StreamNull| s) (list nil nil))
      ((eql n 0) (list nil s))
      (t
       (setq tmp1 (|streamChop| (- n 1) (cdr s)))
       (setq a (car tmp1))
       (setq b (cadr tmp1))
       (setq lyne (car s))
       (setq c (|ncloopPrefix?| ")command" (cdr lyne)))
       (setq d (cons (car lyne) (cond (c c) (t (cdr lyne)))))
       (list (cons d a) b))))))
```

—————

defun ncloopInclude0

[incStream p366]
 [ncloopProcess p??]
 [next p330]
 [ncloopEchoParse p??]
 [insertpile p625]
 [lineoftoks p403]
 [\$lines p??]

— defun ncloopInclude0 —

```
(defun |ncloopInclude0| (st name n)
  (let (|$lines|)
    (declare (special |$lines|))
    (setq |$lines| (|incStream| st name))
    (|ncloopProcess| n nil
      (|next| #'|ncloopEchoParse|
        (|next| #'|insertpile|
          (|next| #'|lineoftoks|
            |$lines|))))))
```

—————

defun incStream

[incRenumber p366]
 [incLude p369]
 [incRgen p397]
 [Top p370]

— defun incStream —

```
(defun |incStream| (st fn)
  (declare (special |Top|))
  (|incRenumber| (|incLude| 0 (|incRgen| st) 0 (list fn) (list |Top|))))
```

—————

defun incRenumber

[incZip p366]
 [incIgen p367]

incRenumber : Delay → Delay

— defun incRenumber —

```
(defun |incRenumber| (ssx)
  (|incZip| #'|incRenumberLine| ssx (|incIgen| 0)))
```

—————

defun incZip

Axiom “zips” a function together with two delays into a delay.

[Delay p397]
 [incZip1 p367]

incZip : (Function,Delay,Delay) → Delay

— defun incZip —

```
(defun |incZip| (function delay1 delay2)
  (|Delay| #'|incZip1| (list function delay1 delay2)))
```

—————

defun incZip1

[StreamNull p623]
[incZip p366]

incZip1 : Delay $\rightarrow$ ParsePair
— defun incZip1 —

```
(defun |incZip1| (&rest delayArg)
  (let (function delay1 delay2)
    (setq function (car delayArg))
    (setq delay1 (cadr delayArg))
    (setq delay2 (caddr delayArg))
    (cond
      ((|StreamNull| delay1) |StreamNil|)
      ((|StreamNull| delay2) |StreamNil|)
      (t
       (cons
        (funcall function (car delay1) (car delay2))
        (|incZip| function (cdr delay1) (cdr delay2)))))))
```

—————

defun incIgen

[Delay p397]
[incIgen1 p367]

incIgen : Integer $\rightarrow$ Delay
— defun incIgen —

```
(defun |incIgen| (int)
  (|Delay| #'|incIgen1| (list int)))
```

—————

defun incIgen1

[incIgen p367]

— defun incIgen1 —

```
(defun |incIgen1| (&rest z)
  (let (n)
```

```
(setq n (car z))
(setq n (+ n 1))
(cons n (|incIgen| n)))
```

defun incReNumberLine

[incReNumberItem p368]
[incHandleMessage p368]

— defun incReNumberLine —

```
(defun |incReNumberLine| (x1 gno)
  (let (l)
    (setq l (|incReNumberItem| (elt x1 0) gno))
    (|incHandleMessage| x1
      l)))
```

defun incReNumberItem

[lnSetGlobalNum p636]

— defun incReNumberItem —

```
(defun |incReNumberItem| (f i)
  (let (l)
    (setq l (caar f))
    (|lnSetGlobalNum| l i) f))
```

defun incHandleMessage

[ncSoftError p642]
[ncBug p658]

— defun incHandleMessage 0 —

```
(defun |incHandleMessage| (x)
```

```
"Message handling for the source includer"
(let ((msgtype (elt (elt x 1) 1))
      (pos (car (elt x 0)))
      (key (car (elt (elt x 1) 0)))
      (args (cadr (elt (elt x 1) 0))))

  (cond
    ((eq msgtype '|none|) 0)
    ((eq msgtype '|error|) (|ncSoftError| pos key args))
    ((eq msgtype '|warning|) (|ncSoftError| pos key args))
    ((eq msgtype '|say|) (|ncSoftError| pos key args))
    (t (|ncBug| key args)))))
```

defun incLude

This function takes

1. **eb** – in Integer
2. **ss** – a list of strings
3. **ln** – an Integer
4. **ufos** – a list of strings
5. **states** – a list of integers

and constructs a call to **Delay**(p397).

[Delay p397]
[incLude1 p374]

incLude : (Int,List(String),Int,List(String),List(Int)) → Delay
— defun incLude —

```
(defun |incLude| (eb ss ln ufos states)
  (|Delay| #'|incLude1| (list eb ss ln ufos states)))
```

defmacro Rest

— defmacro Rest —


```
(defmacro |Rest| ()
  "used in include1 for parsing; s is not used."
  '(|include| eb (cdr ss) lno ufos states))
```

defvar Top

— initvars —

```
(defvar |Top| 1 "used in include1 for parsing")
```

defvar IfSkipToEnd

— initvars —

```
(defvar |IfSkipToEnd| 10 "used in include1 for parsing")
```

defvar IfKeepPart

— initvars —

```
(defvar |IfKeepPart| 11 "used in include1 for parsing")
```

defvar IfSkipPart

— initvars —

```
(defvar |IfSkipPart| 12 "used in include1 for parsing")
```

defvar ElseifSkipToEnd

— initvars —

```
(defvar |ElseifSkipToEnd| 20 "used in include1 for parsing")
```

—————

defvar ElseifKeepPart

— initvars —

```
(defvar |ElseifKeepPart| 21 "used in include1 for parsing")
```

—————

defvar ElseifSkipPart

— initvars —

```
(defvar |ElseifSkipPart| 22 "used in include1 for parsing")
```

—————

defvar ElseSkipToEnd

— initvars —

```
(defvar |ElseSkipToEnd| 30 "used in include1 for parsing")
```

—————

defvar ElseKeepPart

— initvars —

```
(defvar |ElseKeepPart| 31 "used in include1 for parsing")
```

defun Top?

```
[quotient p??]
```

```
— defun Top? 0 —
```

```
(defun |Top?| (|st|)
  "used in include1 for parsing"
  (eq1 (quotient |st| 10) 0))
```

defun If?

```
[quotient p??]
```

```
— defun If? —
```

```
(defun |If?| (|st|)
  "used in include1 for parsing"
  (eq1 (quotient |st| 10) 1))
```

defun Elseif?

```
[quotient p??]
```

```
— defun Elseif? —
```

```
(defun |Elseif?| (|st|)
  "used in include1 for parsing"
  (eq1 (quotient |st| 10) 2))
```

defun Else?

[quotient p??]

— defun Else? —

```
(defun |Else?| (|st|)
  "used in include1 for parsing"
  (eq1 (quotient |st| 10) 3))
```

—————

defun SkipEnd?

[remainder p??]

— defun SkipEnd? —

```
(defun |SkipEnd?| (|st|)
  "used in include1 for parsing"
  (eq1 (remainder |st| 10) 0))
```

—————

defun KeepPart?

[remainder p??]

— defun KeepPart? —

```
(defun |KeepPart?| (|st|)
  "used in include1 for parsing"
  (eq1 (remainder |st| 10) 1))
```

—————

defun SkipPart?

[remainder p??]

— defun SkipPart? —

```
(defun |SkipPart?| (|st|)
  "used in incLude1 for parsing"
  (eql (remainder |st| 10) 2))
```

defun Skipping?

[KeepPart? p373]

— defun Skipping? —

```
(defun |Skipping?| (|st|)
  "used in incLude1 for parsing"
  (null (|KeepPart?| |st|)))
```

defun incLude1

[StreamNull p623]
 [Top? p372]
 [xlPrematureEOF p378]
 [Skipping? p374]
 [xlSkip p382]
 [Rest p369]
 [xlOK p379]
 [xlOK1 p379]
 [concat p1237]
 [incCommandTail p395]
 [xlSay p382]
 [xlNoSuchFile p383]
 [xlCannotRead p384]
 [incActive? p396]
 [xlFileCycle p385]
 [incLude p369]
 [incFileInput p395]
 [incAppend p380]
 [inclFname p395]
 [xlConActive p386]
 [xlConStill p387]
 [incConsoleInput p396]
 [incNConsoles p396]

[\[xlConsole p387\]](#)
[\[xlSkippingFin p388\]](#)
[\[xlPrematureFin p389\]](#)
[\[assertCond p389\]](#)
[\[ifCond p382\]](#)
[\[If? p372\]](#)
[\[Elseif? p372\]](#)
[\[xlIfSyntax p390\]](#)
[\[SkipEnd? p373\]](#)
[\[KeepPart? p373\]](#)
[\[SkipPart? p373\]](#)
[\[xlIfBug p391\]](#)
[\[xlCmdBug p391\]](#)
[\[expand-tabs p??\]](#)
[\[incClassify p393\]](#)

— defun incLude1 —

```

(defun |incLude1| (&rest z)
  (let (pred s1 n tail head includee fn1 info str state lno states
        ufos ln ss eb)
    (setq eb (car z))
    (setq ss (cadr . (z)))
    (setq ln (caddr . (z)))
    (setq ufos (caddr . (z)))
    (setq states (car (cddddr . (z))))
    (setq lno (+ ln 1))
    (setq state (elt states 0))
    (cond
      ((|StreamNull| ss)
       (cond
         ((null (|Top?| state))
          (cons (|xlPrematureEOF| eb "--premature end" lno ufos)
                |StreamNil|))
         (t |StreamNil|)))
      (t
       (progn
        (setq str (expand-tabs (car ss)))
        (setq info (|incClassify| str))
        (cond
          ((null (elt info 0))
           (cond
            ((|Skipping?| state)
             (cons (|xlSkip| eb str lno (elt ufos 0)) (|Rest|)))
            (t
             (cons (|xlOK| eb str lno (elt ufos 0)) (|Rest|))))
          ((equal (elt info 2) "other")
           (cond

```

```

(|Skipping?| state)
  (cons (|xlSkip| eb str lno (elt ufos 0)) (|Rest|)))
(t
  (cons
    (|xlOK1| eb str (concat ")command" str) lno (elt ufos 0))
    (|Rest|))))
(equal (elt info 2) "say")
(cond
  (|Skipping?| state)
  (cons (|xlSkip| eb str lno (elt ufos 0)) (|Rest|)))
(t
  (progn
    (setq str (|incCommandTail| str info))
    (cons (|xlSay| eb str lno ufos str)
      (cons (|xlOK| eb str lno (ELT ufos 0)) (|Rest|))))))
(equal (elt info 2) "include")
(cond
  (|Skipping?| state)
  (cons (|xlSkip| eb str lno (elt ufos 0)) (|Rest|)))
(t
  (progn
    (setq fn1 (|inclFname| str info))
    (cond
      ((null fn1)
        (cons (|xlNoSuchFile| eb str lno ufos fn1) (|Rest|)))
      ((null (probe-file fn1))
        (cons (|xlCannotRead| eb str lno ufos fn1) (|Rest|)))
      ((|incActive?| fn1 ufos)
        (cons (|xlFileCycle| eb str lno ufos fn1) (|Rest|)))
      (t
        (progn
          (setq includee
            (|incLude| (+ eb (elt info 1))
              (|incFileInput| fn1)
              0
              (cons fn1 ufos)
              (cons |Top| states)))
          (cons (|xlOK| eb str lno (elt ufos 0))
            (|incAppend| includee (|Rest|))))))))))
(equal (elt info 2) "console")
(cond
  (|Skipping?| state)
  (cons (|xlSkip| eb str lno (elt ufos 0)) (|Rest|)))
(t
  (progn
    (setq head
      (|incLude| (+ eb (elt info 1))
        (|incConsoleInput|)
        0
        (cons "console" ufos)

```

```

        (cons |Top| states)))
(setq tail (|Rest|))
(setq n (|incNConsoles| ufos))
(cond
  ((< 0 n)
    (setq head
      (cons (|xlConActive| eb str lno ufos n) head))
    (setq tail
      (cons (|xlConStill| eb str lno ufos n) tail))))
(setq head (cons (|xlConsole| eb str lno ufos) head))
(cons (|xlOK| eb str lno (elt ufos 0))
      (|incAppend| head tail))))))
(equal (elt info 2) "fin")
(cond
  ((|Skipping?| state)
    (cons (|xlSkippingFin| eb str lno ufos) (|Rest|)))
  ((null (|Top?| state))
    (cons (|xlPrematureFin| eb str lno ufos) |StreamNil|))
  (t
    (cons (|xlOK| eb str lno (elt ufos 0)) |StreamNil|))))
(equal (elt info 2) "assert")
(cond
  ((|Skipping?| state)
    (cons (|xlSkippingFin| eb str lno ufos) (|Rest|)))
  (t
    (progn
      (|assertCond| str info)
      (cons (|xlOK| eb str lno (elt ufos 0))
            (|incAppend| includee (|Rest|)))))))
(equal (elt info 2) "if")
(progn
  (setq s1
    (cond
      ((|Skipping?| state) |IfSkipToEnd|)
      (t
        (cond
          ((|ifCond| str info) |IfKeepPart|)
          (t |IfSkipPart|))))))
    (cons (|xlOK| eb str lno (elt ufos 0))
          (|incLude| eb (cdr ss) lno ufos (cons s1 states)))))
(equal (elt info 2) "elseif")
(cond
  ((and (null (|If?| state)) (null (|Elseif?| state)))
    (cons (|xlIfSyntax| eb str lno ufos info states)
          |StreamNil|))
  (t
    (cond
      ((or (|SkipEnd?| state)
           (|KeepPart?| state)
           (|SkipPart?| state))

```



```

(setq s1
  (cond
    ((|SkipPart?| state)
      (setq pred (|ifCond| str info))
      (cond
        (pred |ElseifKeepPart|)
        (t |ElseifSkipPart|)))
      (t |ElseifSkipToEnd|)))
    (cons (|xlOK| eb str lno (elt ufos 0))
      (|incLude| eb (cdr ss) lno ufos (cons s1 (cdr states))))))
  (t
    (cons (|xlIfBug| eb str lno ufos) |StreamNil|))))))
(equal (elt info 2) "else")
(cond
  ((and (null (|If?| state)) (null (|Elseif?| state)))
    (cons (|xlIfSyntax| eb str lno ufos info states)
      |StreamNil|))
  (t
    (cond
      ((or (|SkipEnd?| state)
        (|KeepPart?| state)
        (|SkipPart?| state))
        (setq s1
          (cond ((|SkipPart?| state) |ElseKeepPart|) (t |ElseSkipToEnd|)))
          (cons (|xlOK| eb str lno (elt ufos 0))
            (|incLude| eb (cdr ss) lno ufos (cons s1 (cdr states))))))
        (t
          (cons (|xlIfBug| eb str lno ufos) |StreamNil|))))))
  ((equal (elt info 2) "endif")
    (cond
      ((|Top?| state)
        (cons (|xlIfSyntax| eb str lno ufos info states)
          |StreamNil|))
      (t
        (cons (|xlOK| eb str lno (elt ufos 0))
          (|incLude| eb (cdr ss) lno ufos (cdr states))))))
      (t (cons (|xlCndBug| eb str lno ufos) |StreamNil|))))))

```

defun xlPrematureEOF

[xlMsg p379]

[inclmsgPrematureEOF p381]

— defun xlPrematureEOF —

```
(defun |xlPrematureEOF| (eb str lno ufos)
```

```
(|xlMsg| eb str lno (elt ufos 0)
  (list (|inclmsgPrematureEOF| (elt ufos 0)) '|error|)))
```

defun xlMsg

[incLine p380]

— defun xlMsg —

```
(defun |xlMsg| (extrablanks string localnum fileobj mess)
  (let ((globalnum -1))
    (list (incLine extrablanks string globalnum localnum fileobj) mess)))
```

defun xlOK

[xlOK1 p379]

— defun xlOK —

```
(defun |xlOK| (extrablanks string localnum fileobj)
  (|xlOK1| extrablanks string string localnum fileobj))
```

defun xlOK1

[incLine1 p381]

— defun xlOK1 —

```
(defun |xlOK1| (extrablanks string string1 localnum fileobj)
  (let ((globalnum -1))
    (list (incLine1 extrablanks string string1 globalnum localnum fileobj)
      (list nil '|none|))))
```

defun incAppend

[Delay p397]

[incAppend1 p380]

— defun incAppend —

```
(defun |incAppend| (x y)
  (|Delay| #'|incAppend1| (list x y)))
```

—————

defun incAppend1

[StreamNull p623]

[incAppend p380]

— defun incAppend1 —

```
(defun |incAppend1| (&rest z)
  (let (y x)
    (setq x (car z))
    (setq y (cadr z))
    (cond
     ((|StreamNull| x)
      (cond ((|StreamNull| y) |StreamNil|) (t y)))
     (t
      (cons (car x) (|incAppend| (cdr x) y))))))
```

—————

defun incLine

[incLine1 p381]

— defun incLine —

```
(defun incLine (extrablanks string globalnum localnum fileobj)
  (incLine1 extrablanks string string globalnum localnum fileobj))
```

—————

defun incLine1

[lnCreate p635]

— defun incLine1 —

```
(defun incLine1 (extrablanks string string1 globalnum localnum fileobj)
  (cons
    (cons (|lnCreate| extrablanks string globalnum localnum fileobj) 1) string1))
```

—————

defun inclmsgPrematureEOF

[theorigin p381]

— defun inclmsgPrematureEOF 0 —

```
(defun |inclmsgPrematureEOF| (ufo)
  (list
    (format nil
      "File %lf ended where at least one )endif was still needed.
      An appropriate number of )endif lines has been assumed.")
    (list (|theorigin| ufo))))
```

—————

defun theorigin

— defun theorigin 0 —

```
(defun |theorigin| (x) (list #'|porigin| x))
```

—————

defun porigin

[pfname p384]

— defun porigin —

```
(defun |porigin| (x)
  (if (stringp x)
      x
      (|pfname| x)))
```

defun ifCond

```
[MakeSymbol p??]
[incCommandTail p395]
[$inclAssertions p??]
```

— defun ifCond —

```
(defun |ifCond| (s info)
  (let (word)
    (declare (special |$inclAssertions|))
    (setq word
      (|MakeSymbol| (string-trim *whitespace* (|incCommandTail| s info))))
    (member word |$inclAssertions|)))
```

defun xlSkip

```
[incLine p380]
[concat p1237]
```

— defun xlSkip —

```
(defun |xlSkip| (extrablanks str localnum fileobj)
  (let ((string (concat "-- Omitting:" str)) (globalnum -1))
    (list
      (incLine extrablanks string globalnum localnum fileobj)
      (list nil '|none|))))
```

defun xlSay

```
[xlMsg p379]
[incMsgSay p383]
```

— defun xISay —

```
(defun |xISay| (eb str lno ufos x)
  (|xIMsg| eb str lno (elt ufos 0) (list (|inclmsgSay| x) '|say|)))
```

—————

defun inclmsgSay

[theid [p383](#)]

— defun inclmsgSay —

```
(defun |inclmsgSay| (str)
  (list "%1f" (list (|theid| str))))
```

—————

defun theid

— defun theid 0 —

```
(defun |theid| (a) (list #'identity a))
```

—————

defun xINoSuchFile

[xIMsg [p379](#)]

[inclmsgNoSuchFile [p384](#)]

— defun xINoSuchFile —

```
(defun |xINoSuchFile| (eb str lno ufos fn)
  (|xIMsg| eb str lno (elt ufos 0) (list (|inclmsgNoSuchFile| fn) '|error|)))
```

—————

defun inclmsgNoSuchFile

[thefname p384]

— defun inclmsgNoSuchFile —

```
(defun |inclmsgNoSuchFile| (fn)
  (list "The )include file %1f does not exist." (list (|thefname| fn))))
```

—————

defun thefname

[pname p384]

— defun thefname 0 —

```
(defun |thefname| (x) (list #'|pname| x))
```

—————

defun pname

[PathnameString p??]

— defun pname —

```
(defun |pname| (x) (|PathnameString| x))
```

—————

defun xlCannotRead

[xlMsg p379]

[inclmsgCannotRead p385]

— defun xlCannotRead —

```
(defun |xlCannotRead| (eb str lno ufos fn)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgCannotRead| fn) '|error|)))
```

—————

defun inclmsgCannotRead

[thefname p384]

— defun inclmsgCannotRead —

```
(defun |inclmsgCannotRead| (fn)
  (list "The )include file %1f exists, but cannot be read."
        (list (|thefname| fn))))
```

—————

defun xlFileCycle

[xlMsg p379]

[inclmsgFileCycle p385]

— defun xlFileCycle —

```
(defun |xlFileCycle| (eb str lno ufos fn)
  (|xlMsg| eb str lno (elt ufos 0)
    (list (|inclmsgFileCycle| ufos fn) '|error|)))
```

—————

defun inclmsgFileCycle

```
;inclmsgFileCycle(ufos,fn) ==
;   flist := [porigin n for n in reverse ufos]
;   f1    := porigin fn
;   cycle := [:[n,'==>"] for n in flist], f1]
;   ['SCI0004, [%id cycle, %id f1] ]
```

[porigin p381]

[theid p383]

— defun inclmsgFileCycle —

```
(defun |inclmsgFileCycle| (ufos fn)
  (let (cycle f1 flist)
    (setq flist
      ((lambda (Var8 Var7 n)
        (loop
```



```

(cond
  ((or (atom Var7) (progn (setq n (car Var7)) nil))
    (return (nreverse Var8)))
  (t
    (setq Var8 (cons (|porigin| n) Var8)))
  (setq Var7 (cdr Var7))))
nil (reverse ufos) nil))
(setq f1 (|porigin| fn))
(setq cycle
  (append
    ((lambda (Var10 Var9 n)
      (loop
        (cond
          ((or (atom Var9) (progn (setq n (car Var9)) nil))
            (return (nreverse Var10)))
          (t
            (setq Var10 (append (reverse (list n "==")) Var10)))
            (setq Var9 (cdr Var9))))
      nil f1 list nil)
    (cons f1 nil)))
(list
  (format nil
    "There is a cycle in the )include files: %i %l %lf %u %l. ~
    The inner occurrence of %2f has not been included.")
  (list (|theid| cycle) (|theid| f1))))

```

defun xlConActive

[xlMsg p379]

[inclmsgConActive p386]

— defun xlConActive —

```

(defun |xlConActive| (eb str lno ufos n)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgConActive| n) '|warning|)))

```

defun inclmsgConActive

[theid p383]

— defun inclmsgConActive —

```
(defun |inclmsgConActive| (n)
  (list
    (format nil
      "%!f other )console commands are currently active. ~
      While this new )console command is reading input the others ~
      will have to wait. !
      Remember, each )console command will need a separate )fin.")
    (list (|theid| n))))
```

defun xlConStill

[xlMsg p379]
[inclmsgConStill p387]

— defun xlConStill —

```
(defun |xlConStill| (eb str lno ufos n)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgConStill| n) '|say|)))
```

defun inclmsgConStill

[theid p383]

— defun inclmsgConStill —

```
(defun |inclmsgConStill| (n)
  (list
    (format nil
      "The current )console command has finished reading. ~
      %!f are still active. Remember, each will need a separate )fin.")
    (list (|theid| n))))
```

defun xlConsole

[xlMsg p379]
[inclmsgConsole p388]

— defun xlConsole —

```
(defun |xlConsole| (eb str lno ufos)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgConsole|) '|say|)))
```

defun inclmsgConsole

— defun inclmsgConsole 0 —

```
(defun |inclmsgConsole| ()
  (list "Including source lines from console. Type )fin when done." nil))
```

defun xlSkippingFin

```
[xlMsg p379]
[inclmsgFinSkipped p388]
```

— defun xlSkippingFin —

```
(defun |xlSkippingFin| (eb str lno ufos)
  (|xlMsg| eb str lno (elt ufos 0)
    (list (|inclmsgFinSkipped|) '|warning|)))
```

defun inclmsgFinSkipped

— defun inclmsgFinSkipped 0 —

```
(defun |inclmsgFinSkipped| ()
  (list
    (format nil
      "A )fin command was skipped ~
      (along with everything else) in a false branch of an )if...)endif."
    nil))
```

defun xlPrematureFin

[xlMsg p379]

[inclmsgPrematureFin p389]

— defun xlPrematureFin —

```
(defun |xlPrematureFin| (eb str lno ufos)
  (|xlMsg| eb str lno (elt ufos 0)
    (list (|inclmsgPrematureFin| (elt ufos 0)) '|error|)))
```

—————

defun inclmsgPrematureFin

[theorigin p381]

— defun inclmsgPrematureFin —

```
(defun |inclmsgPrematureFin| (ufo)
  (list
    (format nil
      "A )fin command has been given in %lf where at least one )endif ~
      was still needed. ~
      An appropriate number of )endif lines have been assumed.")
    (list (|theorigin| ufo))))
```

—————

defun assertCond

[MakeSymbol p??]

[incCommandTail p395]

[\$inclAssertions p??]

[*whitespace* p314]

— defun assertCond —

```
(defun |assertCond| (s info)
  (let (word)
    (declare (special |$inclAssertions| *whitespace*))
    (setq word
      (|MakeSymbol| (string-trim *whitespace* (|incCommandTail| s info))))
    (unless (member word |$inclAssertions|)
```

```
(setq |$inclAssertions| (cons word |$inclAssertions|))))))
```

defun xIfSyntax

[Top? p372]
 [Else? p373]
 [xlMsg p379]
 [inclmsgIfSyntax p390]

— defun xIfSyntax —

```
(defun |xIfSyntax| (eb str lno ufos info sts)
  (let (context found st)
    (setq st (elt sts 0))
    (setq found (elt info 2))
    (setq context
      (cond
        ((|Top?| st) '|not in an )if...endif|)
        ((|Else?| st) '|after an )else|)
        (t '|but can't figure out where|)))
    (|xlMsg| eb str lno (elt ufos 0)
      (list (|inclmsgIfSyntax| (elt ufos 0) found context) '|error|))))
```

defun inclmsgIfSyntax

[concat p1237]
 [theid p383]
 [theorigin p381]

— defun inclmsgIfSyntax —

```
(defun |inclmsgIfSyntax| (ufo found context)
  (setq found (concat ")" found))
  (list
    (format nil
      "Incorrect )if...endif syntax. A %1f was found %2f. ~
      The processing of the source from %3f has been abandoned.")
    (list (|theid| found) (|theid| context) (|theorigin| ufo))))
```

defun xIfBug

[xlMsg p379]

[inclmsgIfBug p391]

— defun xIfBug —

```
(defun |xIfBug| (eb str lno ufos)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgIfBug|) '|bug|)))
```

—————

defun inclmsgIfBug

— defun inclmsgIfBug 0 —

```
(defun |inclmsgIfBug| ()
  (list "Unexpected state in )if...)endif." nil))
```

—————

defun xlCmdBug

[xlMsg p379]

[inclmsgCmdBug p391]

— defun xlCmdBug —

```
(defun |xlCmdBug| (eb str lno ufos)
  (|xlMsg| eb str lno (elt ufos 0) (list (|inclmsgCmdBug|) '|bug|)))
```

—————

defun inclmsgCmdBug

— defun inclmsgCmdBug 0 —

```
(defun |inclmsgCmdBug| ()
  (list "Unexpected command in source inclusion." nil))
```

—————

defvar incCommands

This is a list of commands that can be in an include file

— postvars —

```
(eval-when (eval load)
(setq |incCommands|
(list "say" "include" "console" "fin" "assert" "if" "elseif" "else" "endif")))
```

—————

defvar \$pfMacros

The \$pfMacros variable is an alist [[id, state, body-pform], ...] where state is one of: mbody, mparam, mlambda

User-defined macros are maintained in a stack of definitions. This is the stack sequence resulting from the command lines:

```
a ==> 3
a ==> 4
b ==> 7
(
(|b| |mbody| ((|integer| (|posn| (0 "b ==> 7" 1 1 "strings") . 6)) . "7"))
(|a| |mbody| ((|integer| (|posn| (0 "a ==> 4" 1 1 "strings") . 6)) . "4"))
(|a| |mbody| ((|integer| (|posn| (0 "a ==> 3" 1 1 "strings") . 6)) . "3"))
)
```

— initvars —

```
(defvar |$pfMacros| nil)
```

—————

defun incClassify

```
;incClassify(s) ==
;      not incCommand? s => [false,0, '"]
;      i := 1; n := #s
;      while i < n and s.i = char " " repeat i := i + 1
;      i >= n => [true,0,'other"]
;      eb := (i = 1 => 0; i)
;      bad:=true
;      for p in incCommands while bad repeat
```

```

;           incPrefix?(p, i, s) =>
;           bad:=false
;           p1 :=p
;           if bad then [true,0,"other"] else [true,eb,p1]

```

[incCommand? p394]

[incCommands p392]

— defun incClassify —

```

(defun |incClassify| (s)
  (let (p1 bad eb n i)
    (declare (special |incCommands|))
    (if (null (|incCommand?| s))
        (list nil 0 "")
        (progn
          (setq i 1)
          (setq n (length s))
          ((lambda ()
             (loop
              (cond
               ((not (and (< i n) (char= (elt s i) #\space)))
                (return nil))
               (t (setq i (1+ i)))))))
          (cond
           ((not (< i n)) (list t 0 "other"))
           (t
            (if (= i 1)
                (setq eb 0)
                (setq eb i))
            (setq bad t)
            ((lambda (tmp1 p)
               (loop
                (cond
                 ((or (atom tmp1)
                      (progn (setq p (car tmp1)) nil)
                      (not bad))
                  (return nil))
                 (t
                  (cond
                   ((|incPrefix?| p i s)
                    (identity
                     (progn
                      (setq bad nil)
                      (setq p1 p))))))
                  (setq tmp1 (cdr tmp1))))
                |incCommands| nil)
            (if bad
                (list t 0 "other")

```



```
(list t eb p1)))))))))
```

defun incCommand?

incCommand? : String → Boolean
 — defun incCommand? 0 —

```
(defun |incCommand?| (s)
  "does this start with a close paren?"
  (and (< 0 (length s)) (equal (elt s 0) #\)))))
```

defun incPrefix?

```
;incPrefix?(prefix, start, whole) ==
;      #prefix > #whole-start => false
;      good:=true
;      for i in 0..#prefix-1 for j in start.. whole while good repeat
;          good:= prefix.i = whole.j
;      good
```

— defun incPrefix? 0 —

```
(defun |incPrefix?| (prefix start whole)
  (let (good)
    (cond
      ((< (- (length whole) start) (length prefix)) nil)
      (t
       (setq good t)
       ((lambda (Var i j)
          (loop
            (cond
              ((or (> i Var) (not good)) (return nil))
              (t (setq good (equal (elt prefix i) (elt whole j)))))
            (setq i (+ i 1))
            (setq j (+ j 1)))))
        (- (length prefix) 1) 0 start)
       good)))))
```

defun incCommandTail

[incDrop p395]

— defun incCommandTail —

```
(defun |incCommandTail| (s info)
  (let ((start (elt info 1)))
    (when (= start 0) (setq start 1))
    (|incDrop| (+ start (length (elt info 2)) 1) s)))
```

—————

defun incDrop

[substring p324]

— defun incDrop 0 —

```
(defun |incDrop| (n b)
  (if (>= n (length b))
      '||
      (substring b n nil)))
```

—————

defun inclFname

[incFileName p930]

[incCommandTail p395]

— defun inclFname —

```
(defun |inclFname| (s info)
  (|incFileName| (|incCommandTail| s info)))
```

—————

defun incFileInput

[incRgen p397]

[make-instream p1167]

— defun incFileInput —

```
(defun |incFileInput| (fn)
  (|incRgen| (make-instream fn)))
```

defun incConsoleInput

```
[incRgen p397]
[make-instream p1167]
```

— defun incConsoleInput —

```
(defun |incConsoleInput| ()
  (|incRgen| (make-instream 0)))
```

defun incNConsoles

```
[incNConsoles p396]
```

— defun incNConsoles —

```
(defun |incNConsoles| (ufos)
  (let ((a (member "console" ufos)))
    (if a
      (+ 1 (|incNConsoles| (cdr a)))
      0)))
```

defun incActive?

— defun incActive? 0 —

```
(defun |incActive?| (fn ufos)
  (member fn ufos))
```

defun incRgen

Note that `incRgen1` recursively calls this function.

[Delay p397]
[incRgen1 p398]

— **defun incRgen** —

```
(defun |incRgen| (s)
  (|Delay| #'|incRgen1| (list s)))
```

—————

defun Delay

Delay prepends a label **nonnullstream**, returning a list of the label, the given function name in **function** and **arguments**. That is, given

```
(|Delay| |incLude1| (0 ("1") 0 ("strings") (1)))
```

construct

```
(|nonnullstream| |incLude1| 0 ("1") 0 ("strings") (1))
```

Note that **nonnullstream** is NOT a function so the inputs have been changed from a function call to a simple list.

Delay : (Function,List(Any)) → Delay

— **defun Delay 0** —

```
(defun |Delay| (function arguments)
  (cons '|nonnullstream| (cons function arguments)))
```

—————

defvar StreamNil

— **initvars** —

```
(defvar |StreamNil| (list '|nullstream|))
```

—————

— **postvars** —

```
(eval-when (eval load)
  (setq |StreamNil| (list '|nullstream|)))
```

defun incRgen1

This function reads a line from the stream and then conses it up with a recursive call to incRgen. Note that incRgen recursively wraps this function in a delay list.

[incRgen p397]
 [StreamNil p397]

— defun incRgen1 —

```
(defun |incRgen1| (&rest z)
  (let (a s)
    (declare (special |StreamNil|))
    (setq s (car z))
    (setq a (read-line s nil nil))
    (if (null a)
        (progn
          (close s)
          |StreamNil|)
        (cons a (|incRgen1| s)))))
```

Chapter 13

The Token Scanner

defvar scanKeyWords

— postvars —

```
(eval-when (eval load)
(defvar |scanKeyWords|
  (list
    (list "add" 'add)
    (list "and" 'and)
    (list "break" 'break)
    (list "by" 'by)
    (list "case" 'case)
    (list "default" 'default)
    (list "define" 'defn)
    (list "do" 'do)
    (list "else" 'else)
    (list "exit" 'exit)
    (list "export" 'export)
    (list "for" 'for)
    (list "free" 'free)
    (list "from" 'from)
    (list "has" 'has)
    (list "if" 'if)
    (list "import" 'import)
    (list "in" 'in)
    (list "inline" 'inline)
    (list "is" 'is)
    (list "isnt" 'isnt)
    (list "iterate" 'iterate)
    (list "local" '||local|)
    (list "macro" 'macro)
```

```

(list "mod" 'mod)
(list "or" 'or)
(list "pretend" 'pretend)
(list "quo" 'quo)
(list "rem" 'rem)
(list "repeat" 'repeat)
(list "return" 'return)
(list "rule" 'rule)
(list "then" 'then)
(list "where" 'where)
(list "while" 'while)
(list "with" 'with)
(list "|" 'bar)
(list "." 'dot)
(list ":" 'coerce)
(list ":" 'colon)
(list ":" 'colondash)
(list "@" 'at)
(list "@@" 'atat)
(list "," 'comma)
(list ";" 'semicolon)
(list "**" 'power)
(list "*" 'times)
(list "+" 'plus)
(list "-" 'minus)
(list "<" 'lt)
(list ">" 'gt)
(list "<=" 'le)
(list ">=" 'ge)
(list "=" 'equal)
(list "~=" 'notequal)
(list "~" '~)
(list "^" 'carat)
(list ".." 'seg)
(list "#" '|#|)
(list "&" 'ampersand)
(list "$" '$)
(list "/" 'slash)
(list "\" 'backslash)
(list "//" 'slasheslash)
(list "\\\" 'backslashbackslash)
(list "/\\" 'slashbackslash)
(list "\\\" 'backslashslash)
(list "=>" 'exit)
(list ":=" 'becomes)
(list "==" 'def)
(list "==" 'mdef)
(list "->" 'arrow)
(list "<-" 'larrow)
(list "+->" 'gives)

```

```

(list "(" '(|))
(list ")" '(|))
(list "(" '(|\|))
(list "|)" '(|\|))
(list "[" '[])
(list "]" '[])
(list "_" '[_])
(list "{" '({})
(list "}" '({})
(list "{_}" '({_})
(list "[|" '([|\|))
(list "|]" '(|\|))
(list "[|_|]" '([|\|\|))
(list "{|}" '({|\|))
(list "|}" '(|\|))
(list "{|_|}" '({|\|\|))
(list "<<" 'oangle)
(list ">>" 'cangle)
(list "'" '(|'))
(list "``" 'backquote)))

```

defvar infgeneric

— postvars —

```

(eval-when (eval load)
(prog ()
  (return
    ((lambda (var value)
      (loop
        (cond
          ((or (atom var) (progn (setq value (car var)) nil))
           (return nil))
          (t
           (setf (get (car value) 'infgeneric) (cadr value))))
        (setq var (cdr var))))))
(list
  (list 'equal '=)
  (list 'times '*)
  (list 'has '|has|)
  (list 'case '|case|)
  (list 'rem '|rem|)
  (list 'mod '|mod|)
  (list 'quo '|quo|)
  (list 'slash '/)

```



```

(list 'backslash '|\\|)
(list 'slashslash '//)
(list 'backslashbackslash '\\\\|)
(list 'slashbackslash '|/\\|)
(list 'backslashslash '\\\\|)
(list 'power '**)
(list 'carat '^)
(list 'plus '+)
(list 'minus '-')
(list 'lt '<')
(list 'gt '>')
(list 'oangle '<<')
(list 'cangle '>>')
(list 'le '<=')
(list 'ge '>=')
(list 'notequal '~=')
(list 'by '|by|)
(list 'arrow '->)
(list 'larrow '<-)
(list 'bar '|\\|)
(list 'seg '|..|))
nil))))

```

defun lineoftoks

lineoftoks bites off a token-dq from a line-stream returning the token-dq and the rest of the line-stream

```

;lineoftoks(s)==
;  $f: local:=nil
;  $r:local :=nil
;  $ln:local :=nil
;  $linepos:local:=nil
;  $n:local:=nil
;  $sz:local := nil
;  $floatok:local:=true
;  if not nextline s
;  then CONS(nil,nil)
;  else
;    if null scanIgnoreLine($ln,$n) -- line of spaces or starts ) or >
;    then cons(nil,$r)
;    else
;      toks:=[]
;      a:= incPrefix?('command",1,$ln)
;      a =>
;          $ln:=SUBSTRING($ln,8,nil)

```

```

;           b:= dqUnit constoken($ln,$linepos,["command",$ln],0)
;           cons([ [b,s] ],$r)
;
;       while $n<$sz repeat toks:=dqAppend(toks,scanToken())
;       if null toks
;       then cons([], $r)
;       else cons([ [toks,s] ], $r)

```

```

[nextline p404]
[scanIgnoreLine p404]
[incPrefix? p394]
[substring p324]
[dqUnit p633]
[constoken p405]
[$floatok p??]
[$f p??]
[$sz p??]
[$linepos p??]
[$r p??]
[$n p??]
[$ln p??]

```

— defun lineoftoks —

```

(defun |lineoftoks| (s)
  (let (|$floatok| |$sz| |$n| |$linepos| |$ln| |$r| |$f| |b| |a| |toks|)
    (declare (special |$floatok| |$f| |$sz| |$linepos| |$r| |$n| |$ln|))
    (setq |$f| nil)
    (setq |$r| nil)
    (setq |$ln| nil)
    (setq |$linepos| nil)
    (setq |$n| nil)
    (setq |$sz| nil)
    (setq |$floatok| t)
    (cond
      ((null (|nextline| s)) (cons nil nil))
      ((null (|scanIgnoreLine| |$ln| |$n|)) (cons nil |$r|))
      (t
       (setq |toks| nil)
       (setq |a| (|incPrefix?| "command" 1 |$ln|))
       (cond
         (|a|
          (setq |$ln| (substring |$ln| 8 nil))
          (setq |b|
            (|dqUnit| (|constoken| |$ln| |$linepos| (list ' |command| |$ln|) 0)))
          (cons (list (list |b| s)) |$r|))
         (t
          ((lambda ()
              (loop

```

```

(cond
  ((not (< |$n| |$sz|)) (return nil))
  (t (setq |toks| (|dqAppend| |toks| (|scanToken|))))))
(cond
  ((null |toks|) (cons nil |$r|))
  (t (cons (list (list |toks| s)) |$r|))))))

```

defun nextline

```

[npNull p623]
[strposl p1236]
[$sz p??]
[$n p??]
[$linepos p??]
[$ln p??]
[$r p??]
[$f p??]

```

— defun nextline —

```

(defun |nextline| (s)
  (declare (special |$sz| |$n| |$linepos| |$ln| |$r| |$f|))
  (cond
    ((|npNull| s) nil)
    (t
     (setq |$f| (car s))
     (setq |$r| (cdr s))
     (setq |$ln| (cdr |$f|))
     (setq |$linepos| (caar |$f|))
     (setq |$n| (strposl " " |$ln| 0 t)) ; spaces at beginning
     (setq |$sz| (length |$ln|))
     t)))

```

defun scanIgnoreLine

```

[incPrefix? p394]

```

— defun scanIgnoreLine —

```

(defun |scanIgnoreLine| (ln n)

```

```
(cond
  ((null n) n)
  (t
    (cond
      ((= (char-code (char ln 0)) (char-code #\)))
      (cond
        ((|incPrefix?| "command" 1 ln) t)
        (t nil)))
      (t n))))
```

defun constoken

[ncPutQ p706]

— defun constoken —

```
(defun |constoken| (ln lp b n)
  (declare (ignore ln))
  (let (a)
    (setq a (cons (elt b 0) (elt b 1)))
    (|ncPutQ| a '|posn| (cons lp n))
    a))
```

defun scanToken

[startsComment? p407]
 [scanComment p407]
 [startsNegComment? p408]
 [scanNegComment p408]
 [lfid p406]
 [punctuation? p409]
 [scanPunct p409]
 [startsId? p1234]
 [scanWord p417]
 [scanSpace p420]
 [scanString p421]
 [scanNumber p423]
 [scanEscape p426]
 [scanError p426]
 [dqUnit p633]

```
[constoken p405]
[lnExtraBlanks p635]
[$linepos p??]
[$n p??]
[$ln p??]
```

— defun scanToken —

```
(defun |scanToken| ()
  (let (b ch n linepos c ln)
    (declare (special |$linepos| |$n| |$ln|))
    (setq ln |$ln|)
    (setq c (char-code (char |$ln| |$n|)))
    (setq linepos |$linepos|)
    (setq n |$n|)
    (setq ch (elt |$ln| |$n|))
    (setq b
      (cond
        ((|startsComment?|) (|scanComment|) nil)
        ((|startsNegComment?|) (|scanNegComment|) nil)
        ((= c (char-code #\?))
         (setq |$n| (+ |$n| 1))
         (|lfid| "?"))
        ((|punctuation?| c) (|scanPunct|))
        ((|startsId?| ch) (|scanWord| nil))
        ((= c (char-code #\space)) (|scanSpace|) nil)
        ((= c (char-code #\")) (|scanString|))
        ((digitp ch) (|scanNumber|))
        ((= c (char-code #\_)) (|scanEscape|))
        (t (|scanError|))))
    (cond
      ((null b) nil)
      (t
       (|dqUnit|
        (|constoken| ln linepos b (+ n (|lnExtraBlanks| linepos))))))))
```

—————

defun lfid

To pair badge and badgee

— defun lfid 0 —

```
(defun |lfid| (x)
  (list '|id| (intern x "BOOT")))
```

—————

defun Is it a ++ comment?

```
[ $ln p?? ]
[ $sz p?? ]
[ $n p?? ]
```

— defun startsComment? 0 —

```
(defun |startsComment?| ()
  (let (www)
    (declare (special |$ln| |$sz| |$n|))
    (cond
      ((< |$n| |$sz|)
        (cond
          ((= (char-code (char |$ln| |$n|)) (char-code #\+))
            (setq www (+ |$n| 1))
            (cond
              ((not (< www |$sz|)) nil)
              (t (= (char-code (char |$ln| www)) (char-code #\+)))))
          (t nil)))
      (t nil))))
```

—————

defun scanComment

```
[ lfc comment p408 ]
[ substring p324 ]
[ $ln p?? ]
[ $sz p?? ]
[ $n p?? ]
```

— defun scanComment —

```
(defun |scanComment| ()
  (let (n)
    (declare (special |$ln| |$sz| |$n|))
    (setq n |$n|)
    (setq |$n| |$sz|)
    (|lfc comment| (substring |$ln| n nil))))
```

—————

defun lfcomment

— defun lfcomment 0 —

```
(defun |lfcomment| (x)
  (list '|comment| x))
```

—————

defun Is it a – comment?

```
[$ln p??]
[$sz p??]
[$n p??]
```

— defun startsNegComment? —

```
(defun |startsNegComment?| ()
  (let (www)
    (declare (special |$ln| |$sz| |$n|))
    (cond
      ((< |$n| |$sz|)
        (cond
          ((= (char-code (char |$ln| |$n|)) (char-code #\-))
            (setq www (+ |$n| 1))
            (cond
              ((not (< www |$sz|)) nil)
              (t (= (char-code (char |$ln| www)) (char-code #\-))))))
        (t nil))))
  (t nil))))
```

—————

defun scanNegComment

```
[lfnegcomment p409]
[substring p324]
[$ln p??]
[$sz p??]
[$n p??]
```

— defun scanNegComment —

```
(defun |scanNegComment| ()
  (let (n)
    (declare (special |$ln| |$sz| |$n|))
    (setq n |$n|)
    (setq |$n| |$sz|)
    (|lfnegcomment| (substring |$ln| n nil))))
```

defun lfnegcomment

— defun lfnegcomment 0 —

```
(defun |lfnegcomment| (x)
  (list '|negcomment| x))
```

defun punctuation?

— defun punctuation? —

```
(defun |punctuation?| (c)
  (eq1 (elt |scanPun| c) 1))
```

defun scanPunct

```
[subMatch p410]
[scanError p426]
[scanKeyTr p411]
[$n p??]
[$ln p??]
```

— defun scanPunct —

```
(defun |scanPunct| ()
  (let (a sss)
    (declare (special |$n| |$ln|))
```



```

(setq sss (|subMatch| |$ln| |$n|))
(setq a (length sss))
(cond
  ((eql a 0) (|scanError|))
  (t (setq |$n| (+ |$n| a)) (|scanKeyTr| sss))))

```

defun subMatch

[substringMatch p410]

— defun subMatch —

```

(defun |subMatch| (a b)
  (|substringMatch| a |scanDict| b))

```

defun substringMatch

```

;substringMatch (l,d,i)==
;   h:= QENUM(l, i)
;   u:=ELT(d,h)
;   ll:=SIZE l
;   done:=false
;   s1:=""
;   for j in 0.. SIZE u - 1 while not done repeat
;     s:=ELT(u,j)
;     ls:=SIZE s
;     done:=if ls+i > ll
;       then false
;       else
;         eql:= true
;         for k in 1..ls-1 while eql repeat
;           eql:= EQL(QENUM(s,k),QENUM(l,k+i))
;         if eql
;           then
;             s1:=s
;             true
;           else false
;   s1

```

[size p1236]

— defun substringMatch —

```

(defun |substringMatch| (l dict i)
  (let (equal ls s s1 done ll u h)
    (setq h (char-code (char l i)))
    (setq u (elt dict h))
    (setq ll (size l))
    (setq s1 "")
    ((lambda (Var4 j)
      (loop
        (cond
          ((or (> j Var4) done) (return nil))
          (t
           (setq s (elt u j))
           (setq ls (size s))
           (setq done
            (cond
              ((< ll (+ ls i)) nil)
              (t
               (setq equal t)
               ((lambda (Var5 k)
                 (loop
                   (cond
                     ((or (> k Var5) (not equal)) (return nil))
                     (t
                      (setq equal (= (char-code (char s k))
                                     (char-code (char l (+ k i))))))
                   (setq k (+ k 1))))
                (- ls 1) 1)
              (cond (equal (setq s1 s) t) (t nil)))))))
          (setq j (+ j 1))))
      (- (size u) 1) 0)
    s1))

```

defun scanKeyTr

[keyword p412]
 [scanPossFloat p412]
 [lfkey p413]
 [scanCloser? p417]
 [\$floatok p??]

— defun scanKeyTr —

```

(defun |scanKeyTr| (w)
  (declare (special |$floatok|))
  (cond

```

```

(eq (|keyword| w) 'dot)
  (cond
    (|$floatok| (|scanPossFloat| w))
    (t (|lfkey| w))))
(t (setq |$floatok| (null (|scanCloser?| w))) (|lfkey| w))))

```

defun keyword

[hget p1234]

— defun keyword 0 —

```

(defun |keyword| (st)
  (hget |scanKeyTable| st))

```

defun keyword?

[hget p1234]

— defun keyword? 0 —

```

(defun |keyword?| (st)
  (null (null (hget |scanKeyTable| st))))

```

defun scanPossFloat

```

[lfkey p413]
[spleI p413]
[scanExponent p417]
[$ln p??]
[$sz p??]
[$n p??]

```

— defun scanPossFloat —

```

(defun |scanPossFloat| (w)

```

```
(declare (special |$ln| |$sz| |$n|))
(cond
  ((or (not (< |$n| |$sz|)) (null (digitp (elt |$ln| |$n|))))
    (|lfkey| w))
  (t
    (setq w (|spleI| #'digitp)) (|scanExponent| "0" w))))
```

defun digit?

[digitp p1235]

— defun digit? —

```
(defun |digit?| (x)
  (digitp x))
```

defun lfkey

[keyword p412]

— defun lfkey —

```
(defun |lfkey| (x)
  (list '|key| (|keyword| x)))
```

defun spleI

[spleI1 p414]

— defun spleI —

```
(defun |spleI| (dig)
  (|spleI1| dig nil))
```

defun spleI1

```
[substring p324]
[scanEsc p414]
[spleI1 p414]
[concat p1237]
[$ln p??]
[$sz p??]
[$n p??]
```

— defun spleI1 —

```
(defun |spleI1| (dig zro)
  (let (bb a str l n)
    (declare (special |$ln| |$sz| |$n|))
    (setq n |$n|)
    (setq l |$sz|)
    ; while $n<l and FUNCALL(dig,($ln.$n)) repeat $n:=$n+1
    ((lambda ()
      (loop
        (cond
          ((not (and (< |$n| l) (funcall dig (elt |$ln| |$n|))))
            (return nil))
          (t
            (setq |$n| (+ |$n| 1)))))))
    (cond
      ((or (equal |$n| l) (not (= (char-code (char |$ln| |$n|)) (char-code #\_))))
        (cond
          ((and (equal n |$n|) zro) "0")
          (t (substring |$ln| n (- |$n| n)))))
      (t
        ; escaped
        (setq str (substring |$ln| n (- |$n| n)))
        (setq |$n| (+ |$n| 1))
        (setq a (|scanEsc|))
        (setq bb (|spleI1| dig zro)) ; escape, any number of spaces are ignored
        (concat str bb)))))
```

defun scanEsc

```
;scanEsc()==
;   if $n>=$sz
;   then if nextline($r)
;       then
;           while null $n repeat nextline($r)
;       scanEsc()
```

```

;           false
;     else false
;   else
;     n1:=STRPOSL(' "',$ln,$n,true)
;     if null n1
;     then if nextline($r)
;         then
;             while null $n repeat nextline($r)
;             scanEsc()
;             false
;         else false
;     else
;       if $n=n1
;       then true
;       else if QENUM($ln,n1)=ESCAPE
;           then
;             $n:=n1+1
;             scanEsc()
;             false
;           else
;             $n:=n1
;             startsNegComment?() or startsComment?() =>
;                 nextline($r)
;                 scanEsc()
;                 false
;           false
;

```

[nextline p404]

[scanEsc p414]

[strposl p1236]

[startsNegComment? p408]

[startsComment? p407]

[\$ln p??]

[\$r p??]

[\$sz p??]

[\$n p??]

— defun scanEsc —

```

(defun |scanEsc| ()
  (let (n1)
    (declare (special |$ln| |$r| |$sz| |$n|))
    (cond
      ((not (< |$n| |$sz|))
        (cond
          ((|nextline| |$r|)
            ((lambda ()
              (loop
                (cond

```

```

        (|$n| (return nil))
        (t (|nextline| |$r|))))))
    (|scanEsc|)
    nil)
  (t nil)))
(t
 (setq n1 (strpos1 " " |$ln| |$n| t))
 (cond
  ((null n1)
   (cond
    ((|nextline| |$r|)
     ((lambda ()
        (loop
         (cond
          (|$n| (return nil))
          (t (|nextline| |$r|))))))
     (|scanEsc|)
     nil)
    (t nil)))
  ((equal |$n| n1) t)
  ((= (char-code (char |$ln| n1)) (char-code #\_))
   (setq |$n| (+ n1 1))
   (|scanEsc|)
   nil)
  (t (setq |$n| n1)
   (cond
    ((or (|startsNegComment?|) (|startsComment?|))
     (progn
      (|nextline| |$r|)
      (|scanEsc|)
      nil))
    (t nil)))))))))

```

defvar scanCloser

— postvars —

```

(eval-when (eval load)
  (defvar |scanCloser| (list '|'| '}' ']' '|\\|'| '|\\}|' '|\\|\\|'))

```

defun scanCloser?

[keyword p412]

[scanCloser p416]

— defun scanCloser? 0 —

```
(defun |scanCloser?| (w)
  (declare (special |scanCloser|))
  (member (|keyword| w) |scanCloser|))
```

—————

defun scanWord

[scanW p419]

[lfid p406]

[keyword? p412]

[lfkey p413]

[\$floatok p??]

— defun scanWord —

```
(defun |scanWord| (esp)
  (let (w aaa)
    (declare (special |$floatok|))
    (setq aaa (|scanW| nil))
    (setq w (elt aaa 1))
    (setq |$floatok| nil)
    (cond
      ((or esp (elt aaa 0))
       (|lfid| w))
      ((|keyword?| w)
       (setq |$floatok| t)
       (|lfkey| w))
      (t
       (|lfid| w))))))
```

—————

defun scanExponent

[lffloat p419]

[digit? p413]


```
[spleI p413]
[concat p1237]
[$ln p??]
[$sz p??]
[$n p??]
```

— defun scanExponent —

```
(defun |scanExponent| (a w)
  (let (c1 e c n)
    (declare (special |$ln| |$sz| |$n|))
    (cond
      ((not (< |$n| |$sz|)) (|lffloat| a w "0"))
      (t
       (setq n |$n|)
       (setq c (char-code (char |$ln| |$n|)))
       (cond
         ((or (= c (char-code #\E)) (= c (char-code #\e)))
          (setq |$n| (+ |$n| 1))
          (cond
            ((not (< |$n| |$sz|))
             (setq |$n| n)
             (|lffloat| a w "0"))
            ((digitp (elt |$ln| |$n|))
             (setq e (|spleI| #'digitp))
             (|lffloat| a w e))
            (t
             (setq c1 (char-code (char |$ln| |$n|)))
             (cond
               ((or (= c1 (char-code #\+)) (= c1 (char-code #\-)))
                (setq |$n| (+ |$n| 1))
                (cond
                  ((not (< |$n| |$sz|))
                   (setq |$n| n)
                   (|lffloat| a w "0"))
                  ((digitp (elt |$ln| |$n|))
                   (setq e (|spleI| #'digitp))
                   (|lffloat| a w
                     (cond
                       ((= c1 (char-code #\-))
                        (concat "-" e))
                       (t e))))
                   (t e))))
                  (t
                   (setq |$n| n)
                   (|lffloat| a w "0"))))))))
            (t (|lffloat| a w "0"))))))))
```

—————

defun lffloat

[concat p1237]

— defun lffloat 0 —

```
(defun |lffloat| (a w e)
  (list '|float| (concat a "." w "e" e)))
```

—————

defmacro idChar?

— defmacro idChar? 0 —

```
(defmacro |idChar?| (x)
  '(or (alphanumericp ,x) (member ,x '(#\? #\% #\` #\!) :test #'char=)))
```

—————

defun scanW

```
[posend p420]
[substring p324]
[scanEsc p414]
[scanW p419]
[idChar? p419]
[concat p1237]
[$ln p??]
[$sz p??]
[$n p??]
```

— defun scanW —

```
(defun |scanW| (b)
  (let (bb a str endid l n1)
    (declare (special |$ln| |$sz| |$n|))
    (setq n1 |$n|)
    (setq |$n| (+ |$n| 1))
    (setq l |$sz|)
    (setq endid (|posend| |$ln| |$n|))
    (cond
      ((or (equal endid l)
```

```

      (not (= (char-code (char |$ln| endid)) (char-code #\_)))
    (setq |$n| endid)
    (list b (substring |$ln| n1 (- endid n1))))
  (t
   (setq str (substring |$ln| n1 (- endid n1)))
   (setq |$n| (+ endid 1))
   (setq a (|scanEsc|))
   (setq bb
    (cond
     (a (|scanW| t))
     ((not (< |$n| |$sz|)) (list b ""))
     ((|idChar?| (elt |$ln| |$n|)) (|scanW| b))
     (t (list b ""))))
   (list (or (elt bb 0) b) (concat str (elt bb 1))))))

```

defun posend

```

;posend(line,n)==
;   while n<#line and idChar? line.n repeat n:=n+1
;   n

```

NOTE: do not replace “lyne” with “line”

— defun posend —

```

(defun |posend| (lyne n)
  ((lambda ()
    (loop
     (cond
      ((not (and (< n (length lyne)) (|idChar?| (elt lyne n))))
       (return nil))
      (t (setq n (+ n 1))))))
   n)

```

defun scanSpace

```

[|strposl| p1236]
[|lfspaces| p421]
[$floatok p??]
[$ln p??]
[$n p??]

```

— defun scanSpace —

```
(defun |scanSpace| ()
  (let (n)
    (declare (special |$floatok| |$ln| |$n|))
    (setq n |$n|)
    (setq |$n| (strpos1 " " |$ln| |$n| t))
    (when (null |$n|) (setq |$n| (length |$ln|)))
    (setq |$floatok| t)
    (|lfspaces| (- |$n| n))))
```

defun lfspaces

— defun lfspaces 0 —

```
(defun |lfspaces| (x)
  (list '|spaces| x))
```

defun scanString

```
[lfstring p421]
[scanS p422]
[$floatok p??]
[$n p??]
```

— defun scanString —

```
(defun |scanString| ()
  (declare (special |$floatok| |$n|))
  (setq |$n| (+ |$n| 1))
  (setq |$floatok| nil)
  (|lfstring| (|scanS|)))
```

defun lfstring

— defun lfstring 0 —

```
(defun |lfstring| (x)
  (if (eql (length x) 1)
      (list '|char| x)
      (list '|string| x)))
```

defun scanS

```
[ncSoftError p642]
[lnExtraBlanks p635]
[strpos p1236]
[substring p324]
[scanEsc p414]
[concat p1237]
[scanTransform p423]
[scanS p422]
[$ln p??]
[$linepos p??]
[$sz p??]
[$n p??]
```

— defun scanS —

```
(defun |scanS| ()
  (let (b a str mn escsym strsym n)
    (declare (special |$ln| |$linepos| |$sz| |$n|))
    (cond
      ((not (< |$n| |$sz|))
       (|ncSoftError|
        (cons |$linepos| (+ (|lnExtraBlanks| |$linepos|) |$n|))
        "Quote added at end of line." nil) ""))
      (t
       (setq n |$n|)
       (setq strsym (or (strpos "\" |$ln| |$n| nil) |$sz|))
       (setq escsym (or (strpos "_" |$ln| |$n| nil) |$sz|))
       (setq mn (min strsym escsym))
       (cond
         ((equal mn |$sz|)
          (setq |$n| |$sz|)
          (|ncSoftError|
           (cons |$linepos| (+ (|lnExtraBlanks| |$linepos|) |$n|))
           "Quote added at end of line." nil)
          (substring |$ln| n nil))
         ((equal mn strsym)
          (setq |$n| (+ mn 1))
          (substring |$ln| n (- mn n))))
```

```

(t
  (setq str (substring |$ln| n (- mn n)))
  (setq |$n| (+ mn 1))
  (setq a (|scanEsc|))
  (setq b
    (cond
      (a
        (setq str (concat str (|scanTransform| (elt |$ln| |$n|))))
        (setq |$n| (+ |$n| 1)) (|scanS|))
      (t (|scanS|))))
    (concat str b))))))

```

defun scanTransform

— defun scanTransform —

```
(defun |scanTransform| (x) x)
```

defun scanNumber

```

[spleI p413]
[linteger p425]
[spleI1 p414]
[scanExponent p417]
[scanCheckRadix p425]
[lrinteger p425]
[concat p1237]
[$floatok p??]
[$ln p??]
[$sz p??]
[$n p??]

```

— defun scanNumber —

```

(defun |scanNumber| ()
  (let (v w n a)
    (declare (special |$floatok| |$ln| |$sz| |$n|))
    (setq a (|spleI| #'digitp))
    (cond

```

```

((not (< |$n| |$sz|))
  (|lfinteger| a))
(not (= (char-code (char |$ln| |$n|)) (char-code #\r)))
(cond
  ((and |$floatok| (= (char-code (char |$ln| |$n|)) (char-code #\.)))
    (setq n |$n|)
    (setq |$n| (+ |$n| 1))
    (cond
      ((and (< |$n| |$sz|) (= (char-code (char |$ln| |$n|)) (char-code #\.)))
        (setq |$n| n)
        (|lfinteger| a))
      (t
        (setq w (|spleI1| #'digitp t))
        (|scanExponent| a w))))
    (t (|lfinteger| a))))
(t
  (setq |$n| (+ |$n| 1))
  (setq w (|spleI1| #'|rdigit?| t))
  (|scanCheckRadix| (parse-integer a) w)
  (cond
    ((not (< |$n| |$sz|))
      (|lfrinteger| a w))
    (=(char-code (char |$ln| |$n|)) (char-code #\.))
      (setq n |$n|)
      (setq |$n| (+ |$n| 1))
      (cond
        ((and (< |$n| |$sz|) (= (char-code (char |$ln| |$n|)) (char-code #\.)))
          (setq |$n| n)
          (|lfrinteger| a w))
        (t
          (setq v (|spleI1| #'|rdigit?| t))
          (|scanCheckRadix| (parse-integer a) v)
          (|scanExponent| (concat a "r" w) v))))
    (t (|lfrinteger| a w))))))

```

defun rdigit?

[strpos p1236]

— defun rdigit? 0 —

```

(defun |rdigit?| (x)
  (strpos x "0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ" 0 nil))

```

defun lfinteger

— defun lfinteger 0 —

```
(defun |lfinteger| (x)
  (list '|integer| x))
```

—————

defun lfrinteger

[concat p1237]

— defun lfrinteger 0 —

```
(defun |lfrinteger| (r x)
  (list '|integer| (concat r (concat "r" x))))
```

—————

defun scanCheckRadix

```
;scanCheckRadix(r,w)==
;      ns:=#w
;      done:=false
;      for i in 0..ns-1 repeat
;        a:=rdigit? w.i
;        if null a or a>=r
;          then ncSoftError(cons($linepos,lnExtraBlanks $linepos+$n-ns+i),
;                                "S2CN0002", [w.i])
;
```

[\$n p??]

[\$linepos p??]

— defun scanCheckRadix —

```
(defun |scanCheckRadix| (r w)
  (let (a ns)
    (declare (special |$n| |$linepos|))
    (setq ns (length w))
    ((lambda (Var1 i)
      (loop
        (cond
```



```

(<> i Var1) (return nil))
(t
 (setq a (|rdigit?| (elt w i)))
 (cond
  ((or (null a) (not (< a r)))
   (|ncSoftError|
    (cons |$linepos| (+ (- (+ (|lnExtraBlanks| |$linepos|) |$n|) ns) i))
    "The character %1 is greater than the radix."
    (list (elt w i))))))
 (setq i (+ i 1)))
(- ns 1) 0)))

```

defun scanEscape

[scanEsc p414]
[scanWord p417]
[\$n p??]

— defun scanEscape —

```

(defun |scanEscape| ()
  (declare (special |$n|))
  (setq |$n| (+ |$n| 1))
  (when (|scanEsc|) (|scanWord| t)))

```

defun scanError

[ncSoftError p642]
[lnExtraBlanks p635]
[lerror p427]
[\$ln p??]
[\$linepos p??]
[\$n p??]

— defun scanError —

```

(defun |scanError| ()
  (let (n)
    (declare (special |$ln| |$linepos| |$n|))
    (setq n |$n|)

```

```
(setq |$n| (+ |$n| 1))
(|ncSoftError|
 (cons |$linepos| (+ (|lnExtraBlanks| |$linepos|) |$n|))
  "The character %i is not an AXIOM character."
  (list (elt |$ln| n)))
(|lferror| (elt |$ln| n))))
```

defun lferror

— defun lferror 0 —

```
(defun |lferror| (x)
  (list '|error| x))
```

defvar scanKeyTable

— postvars —

```
(eval-when (eval load)
  (defvar |scanKeyTable| (|scanKeyTableCons|)))
```

defun scanKeyTableCons

This function is used to build the scanKeyTable

```
;scanKeyTableCons()==
;  KeyTable:=MAKE_-HASHTABLE("CVEC",true)
;  for st in scanKeyWords repeat
;    HPUT(KeyTable,CAR st,CADR st)
;  KeyTable
```

— defun scanKeyTableCons —

```
(defun |scanKeyTableCons| ()
  (let (KeyTable)
    (setq KeyTable (make-hash-table :test #'equal))
    ((lambda (Var6 st)
      (loop
        (cond
          ((or (atom Var6) (progn (setq st (car Var6)) nil))
            (return nil))
          (t
            (hput KeyTable (car st) (cadr st))))
        (setq Var6 (cdr Var6))))
      |scanKeyWords| nil)
    KeyTable))
```

defvar scanDict

— postvars —

```
(eval-when (eval load)
  (defvar |scanDict| (|scanDictCons|)))
```

defun scanDictCons

```
;scanDictCons()==
;   l:= HKEYS scanKeyTable
;   d :=
;       a:=MAKE_-VEC(256)
;       b:=MAKE_-VEC(1)
;       VEC_-SETELT(b,0,MAKE_-CVEC 0)
;       for i in 0..255 repeat VEC_-SETELT(a,i,b)
;       a
;   for s in l repeat scanInsert(s,d)
;   d
```

[hkeys p1235]

— defun scanDictCons —

```
(defun |scanDictCons| ()
  (let (d b a l)
```

```

(setq l (hkeys |scanKeyTable|))
(setq d
  (progn
    (setq a (make-array 256))
    (setq b (make-array 1))
    (setf (svref b 0)
      (make-array 0 :fill-pointer 0 :element-type 'string-char))
    ((lambda (i)
      (loop
        (cond
          ((> i 255) (return nil))
          (t (setf (svref a i) b)))
        (setq i (+ i 1))))
      0)
    a))
((lambda (Var7 s)
  (loop
    (cond
      ((or (atom Var7) (progn (setq s (car Var7)) nil))
        (return nil))
      (t (|scanInsert| s d)))
    (setq Var7 (cdr Var7))))
  l nil)
d))

```

defun scanInsert

```

;scanInsert(s,d) ==
;   l := #s
;   h := QENUM(s,0)
;   u := ELT(d,h)
;   n := #u
;   k:=0
;   while l <= #(ELT(u,k)) repeat
;     k:=k+1
;   v := MAKE_-VEC(n+1)
;   for i in 0..k-1 repeat VEC_-SETELT(v,i,ELT(u,i))
;   VEC_-SETELT(v,k,s)
;   for i in k..n-1 repeat VEC_-SETELT(v,i+1,ELT(u,i))
;   VEC_-SETELT(d,h,v)
;   s

```

— defun scanInsert —

```
(defun |scanInsert| (s d)
```

```

(let (v k n u h l)
  (setq l (length s))
  (setq h (char-code (char s 0)))
  (setq u (elt d h))
  (setq n (length u))
  (setq k 0)
  ((lambda ()
    (loop
      (cond
        ((< (length (elt u k)) l) (return nil))
        (t (setq k (+ k 1))))))
    (setq v (make-array (+ n 1)))
    ((lambda (Var2 i)
      (loop
        (cond
          ((> i Var2) (return nil))
          (t (setf (svref v i) (elt u i))))
        (setq i (+ i 1))))
      (- k 1) 0)
    (setf (svref v k) s)
    ((lambda (Var3 i)
      (loop
        (cond
          ((> i Var3) (return nil))
          (t (setf (svref v (+ i 1)) (elt u i))))
        (setq i (+ i 1))))
      (- n 1) k)
    (setf (svref d h) v)
    s))

```

defvar scanPun

— postvars —

```

(eval-when (eval load)
  (defvar |scanPun| (|scanPunCons|)))

```

defun scanPunCons

```

;scanPunCons()==
;   listing := HKEYS scanKeyTable

```

```
; a:=MAKE_-BVEC 256
; for i in 0..255 repeat BVEC_-SETELT(a,i,0)
; for k in listing repeat
;     if not startsId? k.0
;     then BVEC_-SETELT(a,QENUM(k,0),1)
; a
```

[hkeys p1235]

— defun scanPunCons —

```
(defun |scanPunCons| ()
  (let (a listing)
    (setq listing (hkeys |scanKeyTable|))
    (setq a (make-array (list 256) :element-type 'bit :initial-element 0))
    ((lambda (i)
      (loop
        (cond
          ((> i 255) (return nil))
          (t (setf (sbit a i) 0)))
        (setq i (+ i 1))))
      0)
    ((lambda (Var8 k)
      (loop
        (cond
          ((or (atom Var8) (progn (setq k (car Var8)) nil))
            (return nil))
          (t
            (cond
              ((null (|startsId?| (elt k 0)))
                (setf (sbit a (char-code (char k 0))) 1))))
            (setq Var8 (cdr Var8))))
        listing nil)
      a))
```


Chapter 14

Input Stream Parser

defun Input Stream Parser

```
[trappoint p??]  
[npFirstTok p435]  
[npItem p434]  
[ncSoftError p642]  
[tokPosn p703]  
[pfWrong p594]  
[pfDocument p544]  
[pfListOf p543]  
[$ttok p??]  
[$stok p??]  
[$stack p??]  
[$inputStream p??]
```

— **defun** npParse —

```
(defun |npParse| (stream)  
  (let (|$ttok| |$stok| |$stack| |$inputStream| found)  
    (declare (special |$ttok| |$stack| |$inputStream| |$stok|))  
    (setq |$inputStream| stream)  
    (setq |$stack| nil)  
    (setq |$stok| nil)  
    (setq |$ttok| nil)  
    (|npFirstTok|)  
    (setq found (catch 'trappoint (|npItem|)))  
    (cond  
      ((eq found 'trapped)  
       (|ncSoftError| (|tokPosn| |$stok|) "syntax error at top level" nil)  
       (|pfWrong| (|pfDocument| "top level syntax error") (|pfListOf| nil)))  
      ((null (null |$inputStream|))
```



```

(|ncSoftError| (|tokPosn| |$stok|) " Improper syntax." nil)
(|pfWrong|
  (|pfDocument| (list "input stream not exhausted"))
  (|pfListOf| nil)))
(null |$stack|)
(|ncSoftError| (|tokPosn| |$stok|)
  "System error while parsing, stack is empty." nil)
(|pfWrong| (|pfDocument| (list "stack empty")) (|pfListOf| nil)))
(t (car |$stack|))))

```

defun npItem

```

[npQualDef p437]
[npEqKey p437]
[npItem1 p434]
[npPop1 p436]
[pfEnSequence p561]
[npPush p435]
[pfNovalue p577]

```

— defun npItem —

```

(defun |npItem| ()
  (let (c b a tmp1)
    (when (|npQualDef|)
      (if (|npEqKey| 'semicolon)
        (progn
          (setq tmp1 (|npItem1| (|npPop1|)))
          (setq a (car tmp1))
          (setq b (cadr tmp1))
          (setq c (|pfEnSequence| b))
          (if a
            (|npPush| c)
            (|npPush| (|pfNovalue| c))))
          (|npPush| (|pfEnSequence| (|npPop1|)))))))

```

defun npItem1

```

[npQualDef p437]
[npEqKey p437]
[npItem1 p434]

```

[npPop1 p436]

— defun npItem1 —

```
(defun |npItem1| (c)
  (let (b a tmp1)
    (if (|npQualDef|)
      (if (|npEqKey| 'semicolon)
        (progn
          (setq tmp1 (|npItem1| (|npPop1|)))
          (setq a (car tmp1))
          (setq b (cadr tmp1))
          (list a (append c b)))
        (list t (append c (|npPop1|))))
      (list nil c))))
```

—————

defun npFirstTok

Sets the current leaf (\$stok) to the next leaf in the input stream. Sets the current token (\$ttok) cdr of the leaf. A leaf token looks like [head, token, position] where head is either an id or (id . alist)

[tokConstruct p701]

[tokPosn p703]

[tokPart p703]

[\$ttok p??]

[\$stok p??]

[\$inputStream p??]

— defun npFirstTok —

```
(defun |npFirstTok| ()
  (declare (special |$ttok| |$stok| |$inputStream|))
  (if (null |$inputStream|)
    (setq |$stok| (|tokConstruct| 'error 'nomore (|tokPosn| |$stok|)))
    (setq |$stok| (car |$inputStream|)))
  (setq |$ttok| (|tokPart| |$stok|)))
```

—————

defun Push one item onto \$stack

[\$stack p??]

— defun npPush 0 —

```
(defun |npPush| (x)
  (declare (special |$stack|))
  (push x |$stack|))
```

—————

defun Pop one item off \$stack

[*\$stack* *p??*]

— defun npPop1 0 —

```
(defun |npPop1| ()
  (declare (special |$stack|))
  (pop |$stack|))
```

—————

defun Pop the second item off \$stack

[*\$stack* *p??*]

— defun npPop2 0 —

```
(defun |npPop2| ()
  (let (a)
    (declare (special |$stack|))
    (setq a (cadr |$stack|))
    (rplacd |$stack| (cddr |$stack|))
    a))
```

—————

defun Pop the third item off \$stack

[*\$stack* *p??*]

— defun npPop3 0 —

```
(defun |npPop3| ()
```

```
(let (a)
  (declare (special |$stack|))
  (setq a (caddr |$stack|))
  (rplacd (cdr |$stack|) (cdddr |$stack|) a))
```

defun npQualDef

[npComma p438]
 [npPush p435]
 [npPop1 p436]

— defun npQualDef —

```
(defun |npQualDef| ()
  (and (|npComma|) (|npPush| (list (|npPop1|)))))
```

defun Advance over a keyword

Test for the keyword, if found advance the token stream

[npNext p437]
 [\$ttok p??]
 [\$stok p??]

— defun npEqKey —

```
(defun |npEqKey| (keyword)
  (declare (special |$ttok| |$stok|))
  (and
    (eq (caar |$stok|) '|key|)
    (eq keyword |$ttok|)
    (|npNext|)))
```

defun Advance the input stream

This advances the input stream. The call to npFirstTok picks off the next token in the input stream and updates the current leaf (\$stok) and the current token (\$ttok)

[npFirstTok p435]
 [\$inputStream p??]

— defun npNext —

```
(defun |npNext| ()
  (declare (special |$inputStream|))
  (setq |$inputStream| (cdr |$inputStream|))
  (|npFirstTok|))
```

defun npComma

[npTuple p438]
 [npQualifiedDefinition p439]

— defun npComma —

```
(defun |npComma| ()
  (|npTuple| #'|npQualifiedDefinition|))
```

defun npTuple

[npListofFun p513]
 [npCommaBackSet p438]
 [pfTupleListOf p590]

— defun npTuple —

```
(defun |npTuple| (|p|)
  (|npListofFun| |p| #'|npCommaBackSet| #'|pfTupleListOf|))
```

defun npCommaBackSet

[npEqKey p437]

— defun npCommaBackSet —

```
(defun |npCommaBackSet| ()
  (and
    (|npEqKey| 'comma)
    (or (|npEqKey| 'backset) t)))
```

defun npQualifiedDefinition

[npQualified p439]
 [npDefinitionOrStatement p440]

— defun npQualifiedDefinition —

```
(defun |npQualifiedDefinition| ()
  (|npQualified| #'|npDefinitionOrStatement|))
```

defun npQualified

[npEqKey p437]
 [npDefinition p459]
 [npTrap p505]
 [npPush p435]
 [pfWhere p592]
 [npPop1 p436]
 [npLetQualified p459]

— defun npQualified —

```
(defun |npQualified| (f)
  (if (funcall f)
    (progn
      (do () ; while ... do
        ((not (and (|npEqKey| 'where) (or (|npDefinition|) (|npTrap|)))))
        (|npPush| (|pfWhere| (|npPop1|) (|npPop1|))))
      t)
    (|npLetQualified| f)))
```

defun npDefinitionOrStatement

[npBackTrack p440]
 [npGives p440]
 [npDef p480]

— defun npDefinitionOrStatement —

```
(defun |npDefinitionOrStatement| ()
  (|npBackTrack| #'|npGives| 'def #'|npDef|))
```

—————→

defun npBackTrack

[npState p504]
 [npEqPeek p444]
 [npRestore p444]
 [npTrap p505]

— defun npBackTrack —

```
(defun |npBackTrack| (p1 p2 p3)
  (let (a)
    (setq a (|npState|))
    (when (apply p1 nil)
      (cond
        ((|npEqPeek| p2)
         (|npRestore| a)
         (or (apply p3 nil) (|npTrap|)))
        (t t)))))
```

—————→

defun npGives

[npBackTrack p440]
 [npExit p508]
 [npLambda p441]

— defun npGives —

```
(defun |npGives| ()
  (|npBackTrack| #'|npExit| 'gives #'|npLambda|))
```

defun npLambda

[npVariable p505]
 [npLambda p441]
 [npTrap p505]
 [npPush p435]
 [pfLam p570]
 [npPop2 p436]
 [npPop1 p436]
 [npEqKey p437]
 [npDefinitionOrStatement p440]
 [npType p441]
 [pfReturnTyped p583]

— defun npLambda —

```
(defun |npLambda| ()
  (or
    (and
      (|npVariable|)
      (or (|npLambda|) (|npTrap|))
      (|npPush| (|pfLam| (|npPop2|) (|npPop1|))))
    (and
      (|npEqKey| 'gives)
      (or (|npDefinitionOrStatement|) (|npTrap|)))
    (and
      (|npEqKey| 'colon)
      (or (|npType|) (|npTrap|))
      (|npEqKey| 'gives)
      (or (|npDefinitionOrStatement|) (|npTrap|))
      (|npPush| (|pfReturnTyped| (|npPop2|) (|npPop1|))))))
```

defun npType

[npMatch p442]
 [npPop1 p436]
 [npWith p442]
 [npPush p435]

— defun npType —


```
(defun |npType| ()
  (and
    (|npMatch|)
    (let ((a (|npPop1|)))
      (or
        (|npWith| a)
        (|npPush| a))))))
```

defun npMatch

```
[npLeftAssoc p499]
[npSuch p442]
```

— defun npMatch —

```
(defun |npMatch| ()
  (|npLeftAssoc| '(is isnt) #'|npSuch|))
```

defun npSuch

```
[npLeftAssoc p499]
[npLogical p489]
```

— defun npSuch —

```
(defun |npSuch| ()
  (|npLeftAssoc| '(bar) #'|npLogical|))
```

defun npWith

```
[npEqKey p437]
[npState p504]
[npCategoryL p445]
[npTrap p505]
[npEqPeek p444]
[npRestore p444]
```

[npVariable p505]
 [npCompMissing p443]
 [npPush p435]
 [pfWith p594]
 [npPop2 p436]
 [npPop1 p436]
 [pfNothing p543]

— defun npWith —

```
(defun |npWith| (extra)
  (let (a)
    (and
      (|npEqKey| 'with)
      (progn
        (setq a (|npState|))
        (or (|npCategoryL|) (|npTrap|))
        (if (|npEqPeek| 'in)
            (progn
              (|npRestore| a)
              (and
                (or (|npVariable|) (|npTrap|))
                (|npCompMissing| 'in)
                (or (|npCategoryL|) (|npTrap|))
                (|npPush| (|pfWith| (|npPop2|) (|npPop1|) extra))))
              (|npPush| (|pfWith| (|pfNothing|) (|npPop1|) extra)))))))
```

—————

defun npCompMissing

[npEqKey p437]
 [npMissing p443]

— defun npCompMissing —

```
(defun |npCompMissing| (s)
  (or (|npEqKey| s) (|npMissing| s)))
```

—————

defun npMissing

[trappoint p??]
 [ncSoftError p642]

```
[tokPosn p703]
[pname p1235]
[$stok p??]
```

— defun npMissing —

```
(defun |npMissing| (s)
  (declare (special |$stok|))
  (|incSoftError| (|tokPosn| |$stok|) "Possibly missing a %1" (list (pname s)))
  (throw 'trappoint 'trapped))
```

defun npRestore

```
[npFirstTok p435]
[$stack p??]
[$inputStream p??]
```

— defun npRestore —

```
(defun |npRestore| (x)
  (declare (special |$stack| |$inputStream|))
  (setq |$inputStream| (car x))
  (|npFirstTok|)
  (setq |$stack| (cdr x))
  t)
```

defun Peek for keyword s, no advance of token stream

```
[$ttok p??]
[$stok p??]
```

— defun npEqPeek 0 —

```
(defun |npEqPeek| (s)
  (declare (special |$ttok| |$stok|))
  (and (eq (caar |$stok|) '|key|) (eq s |$ttok|)))
```

defun npCategoryL

[npCategory p445]
 [npPush p435]
 [pfUnSequence p591]
 [npPop1 p436]

— defun npCategoryL —

```
(defun |npCategoryL| ()
  (and
    (|npCategory|)
    (|npPush| (|pfUnSequence| (|npPop1|))))))
```

—————

defun npCategory

[npPP p502]
 [npSCategory p445]

— defun npCategory —

```
(defun |npCategory| ()
  (|npPP| #'|npSCategory|))
```

—————

defun npSCategory

[npWConditional p487]
 [npCategoryL p445]
 [npPush p435]
 [npPop1 p436]
 [npDefaultValue p487]
 [npState p504]
 [npPrimary p450]
 [npEqPeek p444]
 [npRestore p444]
 [npSignature p446]
 [npApplication p454]
 [pfAttribute p551]
 [npTrap p505]

— defun npSCategory —

```
(defun |npSCategory| ()
  (let (a)
    (cond
      ((|npWConditional| #'|npCategoryL|) (|npPush| (list (|npPop1|))))
      ((|npDefaultValue|) t)
      (t
       (setq a (|npState|))
       (cond
         ((|npPrimary|)
          (cond
            ((|npEqPeek| 'colon) (|npRestore| a) (|npSignature|))
            (t
             (|npRestore| a)
             (or
              (and (|npApplication|) (|npPush| (list (|pfAttribute| (|npPop1|)))))
              (|npTrap|))))
          (t nil))))))
```

—————

defun npSignature

```
[npSigItemList p446]
[npPush p435]
[pfWDec p591]
[pfNothing p543]
[npPop1 p436]
```

— defun npSignature —

```
(defun |npSignature| ()
  (and (|npSigItemList|) (|npPush| (|pfWDec| (|pfNothing|) (|npPop1|)))))
```

—————

defun npSigItemList

```
[npListing p447]
[npSigItem p448]
[npPush p435]
[pfListOf p543]
```

[pfAppend p553]
 [pfParts p547]
 [npPop1 p436]

— defun npSigItemList —

```
(defun |npSigItemList| ()
  (and
    (|npListing| #'|npSigItem|)
    (|npPush| (|pfListOf| (|pfAppend| (|pfParts| (|npPop1|)))))))
```

—————

defun npListing

[npList p447]
 [pfListOf p543]

— defun npListing —

```
(defun |npListing| (p)
  (|npList| p 'comma #'|pfListOf|))
```

—————

defun Always produces a list, fn is applied to it

[npEqKey p437]
 [npTrap p505]
 [npPush p435]
 [npPop3 p436]
 [npPop2 p436]
 [npPop1 p436]
 [\$stack p??]

— defun npList —

```
(defun |npList| (f str1 fn)
  (let (a)
    (declare (special |$stack|))
    (cond
      ((apply f nil)
       (cond
         ((and (|npEqKey| str1)
```

```

      (or (|npEqKey| 'backset) t)
      (or (apply f nil) (|npTrap|)))
    (setq a |$stack|)
    (setq |$stack| nil)
    (do () ; while .. do nothing
      (not
        (and (|npEqKey| str1)
              (or (|npEqKey| 'backset) t)
              (or (apply f nil) (|npTrap|))))
        nil))
    (setq |$stack| (cons (nreverse |$stack|) a))
    (|npPush| (funcall fn (cons (|npPop3|) (cons (|npPop2|) (|npPop1|)))))
    (t (|npPush| (funcall fn (list (|npPop1|)))))
      (t (|npPush| (funcall fn nil)))))

```

defun npSigItem

[npTypeVariable p448]
 [npSigDecl p449]
 [npTrap p505]

— defun npSigItem —

```

(defun |npSigItem| ()
  (and (|npTypeVariable|) (or (|npSigDecl|) (|npTrap|))))

```

defun npTypeVariable

[npParenthesized p507]
 [npTypeVariablelist p449]
 [npSignatureDefinee p449]
 [npPush p435]
 [pfListOf p543]
 [npPop1 p436]

— defun npTypeVariable —

```

(defun |npTypeVariable| ()
  (or
    (|npParenthesized| #'|npTypeVariablelist|)

```

```
(and (|npSignatureDefinee|) (|npPush| (|pfListOf| (list (|npPop1|))))))
```

defun npSignatureDefinee

```
[npName p496]
[npInfixOperator p452]
[npPrefixColon p453]
```

— defun npSignatureDefinee —

```
(defun |npSignatureDefinee| ()
  (or (|npName|) (|npInfixOperator|) (|npPrefixColon|)))
```

defun npTypeVariablelist

```
[npListing p447]
[npSignatureDefinee p449]
```

— defun npTypeVariablelist —

```
(defun |npTypeVariablelist| ()
  (|npListing| #'|npSignatureDefinee|))
```

defun npSigDecl

```
[npEqKey p437]
[npType p441]
[npTrap p505]
[npPush p435]
[pfSpread p537]
[pfParts p547]
[npPop2 p436]
[npPop1 p436]
```

— defun npSigDecl —


```
(defun |npSigDecl| ()
  (and
    (|npEqKey| 'colon)
    (or (|npType|) (|npTrap|))
    (|npPush| (|pfSpread| (|pfParts| (|npPop2|)) (|npPop1|))))))
```

defun npPrimary

```
[npPrimary1 p456]
[npPrimary2 p450]
```

— defun npPrimary —

```
(defun |npPrimary| ()
  (or (|npPrimary1|) (|npPrimary2|)))
```

defun npPrimary2

```
[npEncAp p474]
[npAtom2 p452]
[npAdd p451]
[pfNothing p543]
[npWith p442]
```

— defun npPrimary2 —

```
(defun |npPrimary2| ()
  (or
    (|npEncAp| #'|npAtom2|)
    (|npAdd| (|pfNothing|))
    (|npWith| (|pfNothing|))))
```

defun npADD

TPDHERE: Note that there is also an npAdd function

```
[npType p441]
[npPop1 p436]
```

[npAdd p451]
[npPush p435]

— defun npADD —

```
(defun |npADD| ()
  (let (a)
    (and
      (|npType|)
      (progn
        (setq a (|npPop1|))
        (or
          (|npAdd| a)
          (|npPush| a)))))))
```

—

defun npAdd

TPDHERE: Note that there is also an npADD function

[npEqKey p437]
[npState p504]
[npDefinitionOrStatement p440]
[npTrap p505]
[npEqPeek p444]
[npRestore p444]
[npVariable p505]
[npCompMissing p443]
[npDefinitionOrStatement p440]
[npPush p435]
[pfAdd p550]
[npPop2 p436]
[npPop1 p436]
[pfNothing p543]

— defun npAdd —

```
(defun |npAdd| (extra)
  (let (a)
    (and
      (|npEqKey| 'add)
      (progn
        (setq a (|npState|))
        (or (|npDefinitionOrStatement|) (|npTrap|))
        (cond
          ((|npEqPeek| 'in)
```

```

(progn
  (|npRestore| a)
  (and
    (or (|npVariable|) (|npTrap|))
    (|npCompMissing| 'in)
    (or (|npDefinitionOrStatement|) (|npTrap|))
    (|npPush| (|pfAdd| (|npPop2|) (|npPop1|) extra))))))
(t
  (|npPush| (|pfAdd| (|pfNothing|) (|npPop1|) extra))))))

```

—————→

defun npAtom2

```

[|npInfixOperator| p452]
[|npAmpersand| p496]
[|npPrefixColon| p453]
[|npFromdom| p495]

```

— defun npAtom2 —

```

(defun |npAtom2| ()
  (and
    (or (|npInfixOperator|) (|npAmpersand|) (|npPrefixColon|))
    (|npFromdom|)))

```

—————→

defun npInfixOperator

```

[|npInfixOp| p453]
[|npState| p504]
[|npEqKey| p437]
[|npInfixOp| p453]
[|npPush| p435]
[|pfSymb| p549]
[|npPop1| p436]
[|tokPosn| p703]
[|npRestore| p444]
[|tokConstruct| p701]
[|tokPart| p703]
[|$stok| p??]

```

— defun npInfixOperator —

```
(defun |npInfixOperator| ()
  (let (b a)
    (declare (special |$stok|))
    (or (|npInfixOp|)
        (progn
          (setq a (|npState|))
          (setq b |$stok|)
          (cond
            ((and (|npEqKey| ' '|) (|npInfixOp|))
              (|npPush| (|pfSymb| (|npPop1|) (|tokPosn| b))))
            (t
              (|npRestore| a)
              (cond
                ((and (|npEqKey| 'backquote) (|npInfixOp|))
                  (setq a (|npPop1|))
                  (|npPush| (|tokConstruct| '|idsy| (|tokPart| a) (|tokPosn| a))))
                (t
                  (|npRestore| a)
                  nil))))))))))
```

defun npInfixOp

[npPushId p501]
 [\$ttok p??]
 [\$stok p??]

— defun npInfixOp —

```
(defun |npInfixOp| ()
  (declare (special |$ttok| |$stok|))
  (and
    (eq (caar |$stok|) '|key|)
    (get |$ttok| 'infgeneric)
    (|npPushId|)))
```

defun npPrefixColon

[npEqPeek p444]
 [npPush p435]
 [tokConstruct p701]
 [tokPosn p703]

[npNext p437]
 [\$stok p??]

— defun npPrefixColon —

```
(defun |npPrefixColon| ()
  (declare (special |$stok|))
  (and
    (|npEqPeek| 'colon)
    (progn
      (|npPush| (|tokConstruct| '|id| '|:| (|tokPosn| |$stok|)))
      (|npNext|))))
```

—————

defun npApplication

[npDotted p454]
 [npPrimary p450]
 [npApplication2 p455]
 [npPush p435]
 [pfApplication p551]
 [npPop2 p436]
 [npPop1 p436]

— defun npApplication —

```
(defun |npApplication| ()
  (and
    (|npDotted| #'|npPrimary|)
    (or
      (and
        (|npApplication2|)
        (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))))
      t)))
```

—————

defun npDotted

— defun npDotted —

```
(defun |npDotted| (f)
```

```
(and (apply f nil) (|npAnyNo| #'|npSelector|)))
```

defun npAnyNo

fn must transform the head of the stack

— defun npAnyNo 0 —

```
(defun |npAnyNo| (fn)
  (do () ((not (apply fn nil)))) ; while apply do...
  t)
```

defun npSelector

```
[npEqKey p437]
[|npPrimary| p450]
[|npTrap| p505]
[|npPush| p435]
[|pfApplication| p551]
[|npPop2| p436]
[|npPop1| p436]
```

— defun npSelector —

```
(defun |npSelector| ()
  (and
    (|npEqKey| 'dot)
    (or (|npPrimary|) (|npTrap|))
    (|npPush| (|pfApplication| (|npPop2|) (|npPop1|)))))
```

defun npApplication2

```
[npDotted p454]
[|npPrimary1| p456]
[|npApplication2| p455]
[|npPush| p435]
[|pfApplication| p551]
```

[npPop2 p436]
 [npPop1 p436]

— **defun npApplication2** —

```
(defun |npApplication2| ()
  (and
    (|npDotted| #'|npPrimary1|)
    (or
      (and
        (|npApplication2|)
        (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))))
      t)))
```

—————

defun npPrimary1

[npEncAp p474]
 [npAtom1 p475]
 [npLet p458]
 [npFix p458]
 [npMacro p456]
 [npBPileDefinition p480]
 [npDefn p479]
 [npRule p486]

— **defun npPrimary1** —

```
(defun |npPrimary1| ()
  (or
    (|npEncAp| #'|npAtom1|)
    (|npLet|)
    (|npFix|)
    (|npMacro|)
    (|npBPileDefinition|)
    (|npDefn|)
    (|npRule|)))
```

—————

defun npMacro

[npPP p502]
 [npMdef p457]

— defun npMacro —

```
(defun |npMacro| ()
  (and
    (|npEqKey| 'macro)
    (|npPP| #'|npMdef|)))
```

defun npMdef

TPDHERE: Beware that this function occurs with uppercase also

```
[npQuiver p490]
[pfCheckMacroOut p538]
[npPop1 p436]
[npDefTail p487]
[npTrap p505]
[npPop1 p436]
[npPush p435]
[pfMacro p575]
[pfPushMacroBody p541]
```

— defun npMdef —

```
(defun |npMdef| ()
  (let (body arg op tmp)
    (when (|npQuiver|) ;[op,arg]:= pfCheckMacroOut(npPop1())
      (setq tmp (|pfCheckMacroOut| (|npPop1|)))
      (setq op (car tmp))
      (setq arg (cadr tmp))
      (or (|npDefTail|) (|npTrap|))
      (setq body (|npPop1|))
      (if (null arg)
        (|npPush| (|pfMacro| op body))
        (|npPush| (|pfMacro| op (|pfPushMacroBody| arg body)))))))
```

defun npMDEF

TPDHERE: Beware that this function occurs with lowercase also

```
[npBackTrack p440]
[npStatement p463]
```


[npMDEFinition p458]

— defun npMDEF —

```
(defun |npMDEF| ()
  (|npBackTrack| #'|npStatement| 'mdef #'|npMDEFinition|))
```

—————

defun npMDEFinition

[npPP p502]

[npMdef p457]

— defun npMDEFinition —

```
(defun |npMDEFinition| ()
  (|npPP| #'|npMdef|))
```

—————

defun npFix

[npEqKey p437]

[npDef p480]

[npPush p435]

[pfFix p563]

[npPop1 p436]

— defun npFix —

```
(defun |npFix| ()
  (and
    (|npEqKey| 'fix)
    (|npPP| #'|npDef|)
    (|npPush| (|pfFix| (|npPop1|))))))
```

—————

defun npLet

[npLetQualified p459]

[npDefinitionOrStatement p440]

— defun npLet —

```
(defun |npLet| ()
  (|npLetQualified| #'|npDefinitionOrStatement|))
```

defun npLetQualified

```
[npEqKey p437]
[npDefinition p459]
[npTrap p505]
[npCompMissing p443]
[npPush p435]
[pfWhere p592]
[npPop2 p436]
[npPop1 p436]
```

— defun npLetQualified —

```
(defun |npLetQualified| (f)
  (and
    (|npEqKey| 'let)
    (or (|npDefinition|) (|npTrap|))
    (|npCompMissing| 'in)
    (or (funcall f) (|npTrap|))
    (|npPush| (|pfWhere| (|npPop2|) (|npPop1|))))))
```

defun npDefinition

```
[npPP p502]
[npDefinitionItem p460]
[npPush p435]
[pfSequenceToList p536]
[npPop1 p436]
```

— defun npDefinition —

```
(defun |npDefinition| ()
  (and
    (|npPP| #'|npDefinitionItem|)
```

```
(|npPush| (|pfSequenceToList| (|npPop1|))))))
```

defun npDefinitionItem

```
[npTyping p460]
[npImport p473]
[npState p504]
[npStatement p463]
[npEqPeek p444]
[npRestore p444]
[npDef p480]
[npMacro p456]
[npDefn p479]
[npTrap p505]
```

— defun npDefinitionItem —

```
(defun |npDefinitionItem| ()
  (let (a)
    (or (|npTyping|)
        (|npImport|)
        (progn
          (setq a (|npState|))
          (cond
            ((|npStatement|)
             (cond
              ((|npEqPeek| 'def)
               (|npRestore| a)
               (|npDef|))
              (t
               (|npRestore| a)
               (or (|npMacro|) (|npDefn|))))))
          (t (|npTrap|))))))
```

defun npTyping

```
[npEqKey p437]
[npDefaultItemList p461]
[npTrap p505]
[npPush p435]
```

[pfTyping p589]
 [npPop1 p436]

— defun npTyping —

```
(defun |npTyping| ()
  (and
    (|npEqKey| 'default)
    (or (|npDefaultItemList|) (|npTrap|))
    (|npPush| (|pfTyping| (|npPop1|)))))
```

—————

defun npDefaultItemList

[npPC p??]
 [npSDefaultItem p461]
 [npPush p435]
 [pfUnSequence p591]
 [npPop1 p436]

— defun npDefaultItemList —

```
(defun |npDefaultItemList| ()
  (and
    (|npPC| #'|npSDefaultItem|)
    (|npPush| (|pfUnSequence| (|npPop1|)))))
```

—————

defun npSDefaultItem

[npListing p447]
 [npDefaultItem p462]
 [npPush p435]
 [pfAppend p553]
 [pfParts p547]
 [npPop1 p436]

— defun npSDefaultItem —

```
(defun |npSDefaultItem| ()
  (and
    (|npListing| #'|npDefaultItem|)
```

```
(|npPush| (|pfAppend| (|pfParts| (|npPop1|))))))
```

defun npDefaultItem

```
[npTypeVariable p448]
[npDefaultDecl p462]
[npTrap p505]
```

— defun npDefaultItem —

```
(defun |npDefaultItem| ()
  (and
    (|npTypeVariable|)
    (or (|npDefaultDecl|) (|npTrap|))))
```

defun npDefaultDecl

```
[npEqKey p437]
[npType p441]
[npTrap p505]
[npPush p435]
[pfSpread p537]
[pfParts p547]
[npPop2 p436]
[npPop1 p436]
```

— defun npDefaultDecl —

```
(defun |npDefaultDecl| ()
  (and
    (|npEqKey| 'colon)
    (or (|npType|) (|npTrap|))
    (|npPush| (|pfSpread| (|pfParts| (|npPop2|)) (|npPop1|))))))
```

defun npStatement

[npExpress p471]
 [npLoop p467]
 [npIterate p467]
 [npReturn p470]
 [npBreak p467]
 [npFree p466]
 [npImport p473]
 [npInline p466]
 [npLocal p465]
 [npExport p463]
 [npTyping p460]
 [npVoid p471]

— defun npStatement —

```
(defun |npStatement| ()
  (or
    (|npExpress|)
    (|npLoop|)
    (|npIterate|)
    (|npReturn|)
    (|npBreak|)
    (|npFree|)
    (|npImport|)
    (|npInline|)
    (|npLocal|)
    (|npExport|)
    (|npTyping|)
    (|npVoid|)))
```

defun npExport

[npEqKey p437]
 [npLocalItemList p464]
 [npTrap p505]
 [npPush p435]
 [pfExport p562]
 [npPop1 p436]

— defun npExport —

```
(defun |npExport| ()
```

```
(and
  (|npEqKey| 'export)
  (or (|npLocalItemlist|) (|npTrap|))
  (|npPush| (|pfExport| (|npPop1|))))))
```

defun npLocalItemlist

```
[npPC p??]
[|npSLocalItem| p464]
[|npPush| p435]
[|pfUnSequence| p591]
[|npPop1| p436]
```

— defun npLocalItemlist —

```
(defun |npLocalItemlist| ()
  (and
    (|npPC| #'|npSLocalItem|)
    (|npPush| (|pfUnSequence| (|npPop1|)))))
```

defun npSLocalItem

```
[|npListing| p447]
[|npLocalItem| p465]
[|npPush| p435]
[|pfAppend| p553]
[|pfParts| p547]
[|npPop1| p436]
```

— defun npSLocalItem —

```
(defun |npSLocalItem| ()
  (and
    (|npListing| #'|npLocalItem|)
    (|npPush| (|pfAppend| (|pfParts| (|npPop1|)))))
```

defun npLocalItem

[npTypeVariable p448]
 [npLocalDecl p465]

— defun npLocalItem —

```
(defun |npLocalItem| ()
  (and
    (|npTypeVariable|)
    (|npLocalDecl|)))
```

—————

defun npLocalDecl

[npEqKey p437]
 [npType p441]
 [npTrap p505]
 [npPush p435]
 [pfSpread p537]
 [pfParts p547]
 [npPop2 p436]
 [npPop1 p436]
 [pfNothing p543]

— defun npLocalDecl —

```
(defun |npLocalDecl| ()
  (or
    (and
      (|npEqKey| 'colon)
      (or (|npType|) (|npTrap|))
      (|npPush| (|pfSpread| (|pfParts| (|npPop2|)) (|npPop1|))))
      (|npPush| (|pfSpread| (|pfParts| (|npPop1|)) (|pfNothing|)))))
```

—————

defun npLocal

[npEqKey p437]
 [npLocalItemlist p464]
 [npTrap p505]
 [npPush p435]

[pfLocal p572]
[npPop1 p436]

— defun npLocal —

```
(defun |npLocal| ()
  (and
    (|npEqKey| '|local|)
    (or (|npLocalItemList|) (|npTrap|))
    (|npPush| (|pfLocal| (|npPop1|)))))
```

—————

defun npFree

[npEqKey p437]
[npLocalItemList p464]
[npTrap p505]
[npPush p435]
[pfFree p563]
[npPop1 p436]

— defun npFree —

```
(defun |npFree| ()
  (and
    (|npEqKey| '|free|)
    (or (|npLocalItemList|) (|npTrap|))
    (|npPush| (|pfFree| (|npPop1|)))))
```

—————

defun npInline

[npAndOr p474]
[npQualTypelist p473]
[pfInline p570]

— defun npInline —

```
(defun |npInline| ()
  (|npAndOr| '|inline #'|npQualTypelist| #'|pfInline|))
```

—————

defun npIterate

[npEqKey p437]
 [npPush p435]
 [pfIterate p569]
 [pfNothing p543]

— defun npIterate —

```
(defun |npIterate| ()
  (and (|npEqKey| 'iterate) (|npPush| (|pfIterate| (|pfNothing|))))))
```

—————

defun npBreak

[npEqKey p437]
 [npPush p435]
 [pfBreak p556]
 [pfNothing p543]

— defun npBreak —

```
(defun |npBreak| ()
  (and (|npEqKey| 'break) (|npPush| (|pfBreak| (|pfNothing|))))))
```

—————

defun npLoop

[npIterators p468]
 [npCompMissing p443]
 [npAssign p509]
 [npTrap p505]
 [npPush p435]
 [pfLp p574]
 [npPop2 p436]
 [npPop1 p436]
 [npEqKey p437]
 [pfLoop1 p573]

— defun npLoop —

```
(defun |npLoop| ()
```

```

(or
  (and
    (|npIterators|)
    (|npCompMissing| 'repeat)
    (or (|npAssign|) (|npTrap|))
    (|npPush| (|pfLp| (|npPop2|) (|npPop1|))))
  (and
    (|npEqKey| 'repeat)
    (or (|npAssign|) (|npTrap|))
    (|npPush| (|pfLoop1| (|npPop1|))))))

```

defun npIterators

```

[npForIn p470]
[npZeroOrMore p469]
[npIterator p468]
[npPush p435]
[npPop2 p436]
[npPop1 p436]
[npWhile p470]
[npIterators p468]

```

— defun npIterators —

```

(defun |npIterators| ()
  (or
    (and
      (|npForIn|)
      (|npZeroOrMore| #'|npIterator|)
      (|npPush| (cons (|npPop2|) (|npPop1|))))
    (and
      (|npWhile|)
      (or
        (and (|npIterators|) (|npPush| (cons (|npPop2|) (|npPop1|))))
        (|npPush| (list (|npPop1|)))))))

```

defun npIterator

```

[npForIn p470]
[npSuchThat p469]
[npWhile p470]

```

— defun npIterator —

```
(defun |npIterator| ()
  (or
    (|npForIn|)
    (|npSuchThat|)
    (|npWhile|)))
```

—————

defun npSuchThat

```
[npAndOr p474]
[npLogical p489]
[pfSuchthat p586]
```

— defun npSuchThat —

```
(defun |npSuchThat| ()
  (|npAndOr| 'bar #'|npLogical| #'|pfSuchthat|))
```

—————

defun Apply argument 0 or more times

```
[npPush p435]
[npPop2 p436]
[npPop1 p436]
[$stack p??]
```

— defun npZeroOrMore —

```
(defun |npZeroOrMore| (f)
  (let (a)
    (declare (special |$stack|))
    (cond
      ((apply f nil)
       (setq a |$stack|)
       (setq |$stack| nil)
       (do () ((not (apply f nil)))) ; while .. do
       (setq |$stack| (cons (nreverse |$stack|) a))
       (|npPush| (cons (|npPop2|) (|npPop1|)))))
    (t (progn (|npPush| nil) t))))
```

defun npWhile

[npAndOr p474]
 [npLogical p489]
 [pfWhile p593]

— **defun npWhile** —

```
(defun |npWhile| ()
  (|npAndOr| 'while #'|npLogical| #'|pfWhile|))
```

defun npForIn

[npEqKey p437]
 [npVariable p505]
 [npTrap p505]
 [npCompMissing p443]
 [npBy p492]
 [npPush p435]
 [pfForin p564]
 [npPop2 p436]
 [npPop1 p436]

— **defun npForIn** —

```
(defun |npForIn| ()
  (and
    (|npEqKey| 'for)
    (or (|npVariable|) (|npTrap|))
    (|npCompMissing| 'in)
    (or (|npBy|) (|npTrap|))
    (|npPush| (|pfForin| (|npPop2|) (|npPop1|)))))
```

defun npReturn

[npEqKey p437]
 [npExpress p471]

[npPush p435]
 [pfNothing p543]
 [npEqKey p437]
 [npName p496]
 [npTrap p505]
 [pfReturn p582]
 [npPop2 p436]
 [npPop1 p436]
 [pfReturnNoName p583]

— **defun npReturn** —

```
(defun |npReturn| ()
  (and
    (|npEqKey| 'return)
    (or
      (|npExpress|)
      (|npPush| (|pfNothing|)))
    (or
      (and
        (|npEqKey| 'from)
        (or (|npName|) (|npTrap|))
        (|npPush| (|pfReturn| (|npPop2|) (|npPop1|))))
      (|npPush| (|pfReturnNoName| (|npPop1|))))))
```

defun npVoid

[npAndOr p474]
 [npStatement p463]
 [pfNovalue p577]

— **defun npVoid** —

```
(defun |npVoid| ()
  (|npAndOr| 'do #'|npStatement| #'|pfNovalue|))
```

defun npExpress

[npExpress1 p472]
 [npIterators p468]

[npPush p435]
 [pfCollect p558]
 [npPop2 p436]
 [pfListOf p543]
 [npPop1 p436]

— defun npExpress —

```
(defun |npExpress| ()
  (and
    (|npExpress1|)
    (or
      (and
        (|npIterators|)
        (|npPush| (|pfCollect| (|npPop2|) (|pfListOf| (|npPop1|))))))
      t)))
```

—————

defun npExpress1

[npConditionalStatement p472]
 [npADD p450]

— defun npExpress1 —

```
(defun |npExpress1| ()
  (or (|npConditionalStatement|) (|npADD|)))
```

—————

defun npConditionalStatement

[npConditional p488]
 [npQualifiedDefinition p439]

— defun npConditionalStatement —

```
(defun |npConditionalStatement| ()
  (|npConditional| #'|npQualifiedDefinition|))
```

—————

defun npImport

[npAndOr p474]
 [npQualTypelist p473]
 [pfImport p569]

— defun npImport —

```
(defun |npImport| ()
  (|npAndOr| 'import #'|npQualTypelist| #'|pfImport|))
```

—————

defun npQualTypelist

[npPC p??]
 [npSQualTypelist p473]
 [npPush p435]
 [pfUnSequence p591]
 [npPop1 p436]

— defun npQualTypelist —

```
(defun |npQualTypelist| ()
  (and
    (|npPC| #'|npSQualTypelist|)
    (|npPush| (|pfUnSequence| (|npPop1|)))))
```

—————

defun npSQualTypelist

[npListing p447]
 [npQualType p474]
 [npPush p435]
 [pfParts p547]
 [npPop1 p436]

— defun npSQualTypelist —

```
(defun |npSQualTypelist| ()
  (and
    (|npListing| #'|npQualType|)
    (|npPush| (|pfParts| (|npPop1|)))))
```


defun npQualType

[npType p441]
 [npPush p435]
 [pfQualType p580]
 [npPop1 p436]
 [pfNothing p543]

— defun npQualType —

```
(defun |npQualType| ()
  (and
    (|npType|)
    (|npPush| (|pfQualType| (|npPop1|) (|pfNothing|)))))
```

defun npAndOr

[npEqKey p437]
 [npTrap p505]
 [npPush p435]
 [npPop1 p436]

— defun npAndOr —

```
(defun |npAndOr| (keyword p f)
  (and
    (|npEqKey| keyword)
    (or (apply p nil) (|npTrap|))
    (|npPush| (funcall f (|npPop1|)))))
```

defun npEncAp

[npAnyNo p455]
 [npEncl p475]
 [npFromdom p495]

— defun npEncAp —

```
(defun |npEncAp| (f)
  (and (apply f nil) (|npAnyNo| #'|npEncl|) (|npFromdom|)))
```

defun npEncl

```
[|npBDefinition| p477]
[|npPush| p435]
[|pfApplication| p551]
[|npPop2| p436]
[|npPop1| p436]
```

— defun npEncl —

```
(defun |npEncl| ()
  (and
    (|npBDefinition|)
    (|npPush| (|pfApplication| (|npPop2|) (|npPop1|)))))
```

defun npAtom1

```
[|npPDefinition| p476]
[|npName| p496]
[|npConstTok| p476]
[|npDollar| p476]
[|npBDefinition| p477]
[|npFromdom| p495]
```

— defun npAtom1 —

```
(defun |npAtom1| ()
  (or
    (|npPDefinition|)
    (and
      (or (|npName|) (|npConstTok|) (|npDollar|) (|npBDefinition|))
      (|npFromdom|))))
```

defun npPDefinition

[npParenthesized p507]
 [npDefinitionlist p485]
 [npPush p435]
 [pfEnSequence p561]
 [npPop1 p436]

— defun npPDefinition —

```
(defun |npPDefinition| ()
  (and
    (|npParenthesized| #'|npDefinitionlist|)
    (|npPush| (|pfEnSequence| (|npPop1|)))))
```

—————

defun npDollar

[npEqPeek p444]
 [npPush p435]
 [tokConstruct p701]
 [tokPosn p703]
 [npNext p437]
 [\$stok p??]

— defun npDollar —

```
(defun |npDollar| ()
  (declare (special |$stok|))
  (and (|npEqPeek| '$)
    (progn
      (|npPush| (|tokConstruct| 'id| '$ (|tokPosn| |$stok|)))
      (|npNext|))))
```

—————

defun npConstTok

[tokType p703]
 [npPush p435]
 [npNext p437]
 [npEqPeek p444]
 [npState p504]

[npPrimary1 p456]
 [pfSymb p549]
 [npPop1 p436]
 [tokPosn p703]
 [npRestore p444]
 [\$stok p??]

— defun npConstTok —

```
(defun |npConstTok| ()
  (let (b a)
    (declare (special |$stok|))
    (cond
      ((member (|tokType| |$stok|) '(|integer| |string| |char| |float| |command|))
        (|npPush| |$stok|)
        (|npNext|))
      ((|npEqPeek| '|' '|)
        (setq a |$stok|)
        (setq b (|npState|))
        (|npNext|)
        (cond
          ((and (|npPrimary1|)
                (|npPush| (|pfSymb| (|npPop1|) (|tokPosn| a))))
            t)
          (t (|npRestore| b) nil)))
        (t nil))))
```

—————

defun npBDefinition

[npPDefinition p476]
 [npBracketed p478]
 [npDefinitionlist p485]

— defun npBDefinition —

```
(defun |npBDefinition| ()
  (or
    (|npPDefinition|)
    (|npBracketed| #'|npDefinitionlist|)))
```

—————

defun npBracketed

[npParened p478]
 [npBracked p478]
 [npBraced p479]
 [npAngleBared p479]

— defun npBracketed —

```
(defun |npBracketed| (f)
  (or
    (|npParened| f)
    (|npBracked| f)
    (|npBraced| f)
    (|npAngleBared| f)))
```

defun npParened

[npEnclosed p504]
 [pfParen p579]

— defun npParened —

```
(defun |npParened| (f)
  (or (|npEnclosed| '(| ')|) #'|pfParen| f)
      (|npEnclosed| '(\| '|\|) #'|pfParen| f)))
```

defun npBracked

[npEnclosed p504]
 [pfBracket p555]
 [pfBracketBar p555]

— defun npBracked —

```
(defun |npBracked| (f)
  (or (|npEnclosed| '[' '] #'|pfBracket| f)
      (|npEnclosed| '[\| '\|] #'|pfBracketBar| f)))
```

defun npBraced

[npEnclosed p504]
 [pfBrace p555]
 [pfBraceBar p555]

— defun npBraced —

```
(defun |npBraced| (f)
  (or (|npEnclosed| '{ '}' #'|pfBrace| f)
      (|npEnclosed| '{\|| '\|}' #'|pfBraceBar| f)))
```

—————

defun npAngleBared

[npEnclosed p504]
 [pfHide p567]

— defun npAngleBared —

```
(defun |npAngleBared| (f)
  (|npEnclosed| '<|>' #'|pfHide| f))
```

—————

defun npDefn

[npEqKey p437]
 [npPP p502]
 [npDef p480]

— defun npDefn —

```
(defun |npDefn| ()
  (and
    (|npEqKey| 'defn)
    (|npPP| #'|npDef|)))
```

—————

defun npDef

[npMatch p442]
 [pfCheckItOut p537]
 [npPop1 p436]
 [npDefTail p487]
 [npTrap p505]
 [npPop1 p436]
 [npPush p435]
 [pfDefinition p559]
 [pfPushBody p547]

— defun npDef —

```
(defun |npDef| ()
  (let (body rt arg op tmp1)
    (when (|npMatch|)
      ; [op,arg,rt]:= pfCheckItOut(npPop1())
      (setq tmp1 (|pfCheckItOut| (|npPop1|)))
      (setq op (car tmp1))
      (setq arg (cadr tmp1))
      (setq rt (caddr tmp1))
      (or (|npDefTail|) (|npTrap|))
      (setq body (|npPop1|))
      (if (null arg)
        (|npPush| (|pfDefinition| op body))
        (|npPush| (|pfDefinition| op (|pfPushBody| rt arg body)))))))
```

defun npBPileDefinition

[npPileBracketed p481]
 [npPileDefinitionlist p481]
 [npPush p435]
 [pfSequence p585]
 [pfListOf p543]
 [npPop1 p436]

— defun npBPileDefinition —

```
(defun |npBPileDefinition| ()
  (and
    (|npPileBracketed| #'|npPileDefinitionlist|)
    (|npPush| (|pfSequence| (|pfListOf| (|npPop1|))))))
```

defun npPileBracketed

[npEqKey p437]
 [npPush p435]
 [pfNothing p543]
 [npMissing p443]
 [pfPile p547]
 [npPop1 p436]

— defun npPileBracketed —

```
(defun |npPileBracketed| (f)
  (cond
    ((|npEqKey| 'settab)
     (cond
       ((|npEqKey| 'backtab) (|npPush| (|pfNothing|))) ; never happens
       ((and (apply f nil)
              (or (|npEqKey| 'backtab) (|npMissing| 'backtab)))
        (|npPush| (|pfPile| (|npPop1|))))
       (t nil)))
    (t nil)))
```

defun npPileDefinitionlist

[npListAndRecover p482]
 [npDefinitionlist p485]
 [npPush p435]
 [pfAppend p553]
 [npPop1 p436]

— defun npPileDefinitionlist —

```
(defun |npPileDefinitionlist| ()
  (and
    (|npListAndRecover| #'|npDefinitionlist|)
    (|npPush| (|pfAppend| (|npPop1|)))))
```

defun npListAndRecover

```

[trappoint p??]
[npRecoverTrap p483]
[syGeneralErrorHere p484]
[npEqKey p437]
[npEqPeek p444]
[npNext p437]
[npPop1 p436]
[npPush p435]
[$inputStream p??]
[$stack p??]

```

— **defun npListAndRecover** —

```

(defun |npListAndRecover| (f)
  (let (found c done b savestack)
    (declare (special |$inputStream| |$stack|))
    (setq savestack |$stack|)
    (setq |$stack| nil)
    (setq c |$inputStream|)
    (do ()
      (done)
      (setq found (catch 'trappoint (apply f nil)))
      (cond
        ((eq found 'trapped)
         (setq |$inputStream| c)
         (|npRecoverTrap|))
        ((null found)
         (setq |$inputStream| c)
         (|syGeneralErrorHere|) (|npRecoverTrap|)))
      (cond
        ((|npEqKey| 'backset) (setq c |$inputStream|))
        ((|npEqPeek| 'backtab) (setq done t))
        (t
         (setq |$inputStream| c)
         (|syGeneralErrorHere|)
         (|npRecoverTrap|)
         (cond
          ((|npEqPeek| 'backtab) (setq done t))
          (t
           (|npNext|)
           (setq c |$inputStream|))))))
      (setq b (cons (|npPop1|) b)))
    (setq |$stack| savestack)
    (|npPush| (nreverse b))))

```

—————

defun npRecoverTrap

```
[npFirstTok p435]
[tokPosn p703]
[npMoveTo p483]
[syIgnoredFromTo p484]
[npPush p435]
[pfWrong p594]
[pfDocument p544]
[pfListOf p543]
[$stok p??]
```

— defun npRecoverTrap —

```
(defun |npRecoverTrap| ()
  (let (pos2 pos1)
    (declare (special |$stok|))
    (|npFirstTok|)
    (setq pos1 (|tokPosn| |$stok|))
    (|npMoveTo| 0)
    (setq pos2 (|tokPosn| |$stok|))
    (|syIgnoredFromTo| pos1 pos2)
    (|npPush|
     (list (|pfWrong| (|pfDocument| (list "pile syntax error")
                                         (|pfListOf| nil)))))))
```

—

defun npMoveTo

```
[npEqPeek p444]
[npNext p437]
[npMoveTo p483]
[npEqKey p437]
[$inputStream p??]
```

— defun npMoveTo —

```
(defun |npMoveTo| (|n|)
  (declare (special |$inputStream|))
  (cond
   ((null |$inputStream|) t)
   ((|npEqPeek| 'backtab)
    (cond
     ((eq1 |n| 0) t)
     (t (|npNext|) (|npMoveTo| (1- |n|))))))
```

```

((|npEqPeek| 'backset)
 (cond
  ((eq1 |n| 0) t)
  (t (|npNext|) (|npMoveTo| |n|))))
((|npEqKey| 'settab) (|npMoveTo| (+ |n| 1)))
(t (|npNext|) (|npMoveTo| |n|)))

```

defun syIgnoredFromTo

```

[pfGlobalLinePosn p533]
[ncSoftError p642]
[FromTo p670]
[From p670]
[To p670]

```

— defun syIgnoredFromTo —

```

(defun |syIgnoredFromTo| (pos1 pos2)
 (cond
  ((equal (|pfGlobalLinePosn| pos1) (|pfGlobalLinePosn| pos2))
   (|ncSoftError| (|FromTo| pos1 pos2) "Ignored." nil))
  (t
   (|ncSoftError| (|From| pos1) "Ignored from here" nil)
   (|ncSoftError| (|To| pos2) "to here." nil))))

```

defun syGeneralErrorHere

```

[sySpecificErrorHere p484]

```

— defun syGeneralErrorHere —

```

(defun |syGeneralErrorHere| ()
 (|sySpecificErrorHere| " Improper syntax." nil))

```

defun sySpecificErrorHere

```

[sySpecificErrorAtToken p485]
[$stok p??]

```

— defun sySpecificErrorHere —

```
(defun |sySpecificErrorHere| (key args)
  (declare (special |$stok|))
  (|sySpecificErrorAtToken| |$stok| key args))
```

—————

defun sySpecificErrorAtToken

[ncSoftError p642]
[tokPosn p703]

— defun sySpecificErrorAtToken —

```
(defun |sySpecificErrorAtToken| (tok key args)
  (|ncSoftError| (|tokPosn| tok) key args))
```

—————

defun npDefinitionlist

[npSemiListing p485]
[npQualDef p437]

— defun npDefinitionlist —

```
(defun |npDefinitionlist| ()
  (|npSemiListing| #'|npQualDef|))
```

—————

defun npSemiListing

[npListofFun p513]
[npSemiBackSet p486]
[pfAppend p553]

— defun npSemiListing —

```
(defun |npSemiListing| (p)
  (|npListofFun| p #'|npSemiBackSet| #'|pfAppend|))
```

defun npSemiBackSet

[npEqKey p437]

— defun npSemiBackSet —

```
(defun |npSemiBackSet| ()
  (and (|npEqKey| 'semicolon) (or (|npEqKey| 'backset) t)))
```

defun npRule

[npEqKey p437]

[npPP p502]

[npSingleRule p486]

— defun npRule —

```
(defun |npRule| ()
  (and
    (|npEqKey| 'rule)
    (|npPP| #'|npSingleRule|)))
```

defun npSingleRule

[npQuiver p490]

[npDefTail p487]

[npTrap p505]

[npPush p435]

[pfRule p583]

[npPop2 p436]

[npPop1 p436]

— defun npSingleRule —

```
(defun |npSingleRule| ()
  (when (|npQuiver|)
    (or (|npDefTail|) (|npTrap|))))
```

```
(|npPush| (|pfRule| (|npPop2|) (|npPop1|))))))
```

defun npDefTail

```
[npEqKey p437]
[npDefinitionOrStatement p440]
```

— defun npDefTail —

```
(defun |npDefTail| ()
  (and
    (or (|npEqKey| 'def) (|npEqKey| 'mdef))
    (|npDefinitionOrStatement|)))
```

defun npDefaultValue

```
[npEqKey p437]
[npDefinitionOrStatement p440]
[npTrap p505]
[npPush p435]
[pfAdd p550]
[pfNothing p543]
[npPop1 p436]
```

— defun npDefaultValue —

```
(defun |npDefaultValue| ()
  (and
    (|npEqKey| 'default)
    (or (|npDefinitionOrStatement|) (|npTrap|))
    (|npPush| (list (|pfAdd| (|pfNothing|) (|npPop1|) (|pfNothing|)))))))
```

defun npWConditional

```
[npConditional p488]
[npPush p435]
```

[pfTweakIf p588]
 [npPop1 p436]

— defun npWConditional —

```
(defun |npWConditional| (f)
  (when (|npConditional| f) (|npPush| (|pfTweakIf| (|npPop1|))))))
```

—————

defun npConditional

[npEqKey p437]
 [npLogical p489]
 [npTrap p505]
 [npMissing p443]
 [npElse p488]

— defun npConditional —

```
(defun |npConditional| (f)
  (cond
    ((and (|npEqKey| 'IF)
          (or (|npLogical|) (|npTrap|))
          (or (|npEqKey| 'backset) t))
     (cond
       ((|npEqKey| 'settab)
        (cond
          ((|npEqKey| 'then)
           (and (or (apply f nil) (|npTrap|))
                (|npElse| f)
                (|npEqKey| 'backtab))))
          (t (|npMissing| '|then|))))
       ((|npEqKey| 'then)
        (and (or (apply f nil) (|npTrap|)) (|npElse| f)))
       (t (|npMissing| '|then|))))
    (t nil)))
```

—————

defun npElse

[npState p504]
 [npBacksetElse p489]

```

[npTrap p505]
[npPush p435]
[pfIf p567]
[npPop3 p436]
[npPop2 p436]
[npPop1 p436]
[npRestore p444]
[pfIfThenOnly p568]

```

— defun npElse —

```

(defun |npElse| (f)
  (let (a)
    (setq a (|npState|))
    (cond
      ((|npBacksetElse|)
       (and
        (or (apply f nil) (|npTrap|))
        (|npPush| (|pfIf| (|npPop3|) (|npPop2|) (|npPop1|))))))
      (t
       (|npRestore| a)
       (|npPush| (|pfIfThenOnly| (|npPop2|) (|npPop1|)))))))

```

—————

defun npBacksetElse

TPDHERE: Well this makes no sense.

```

[npEqKey p437]

```

— defun npBacksetElse —

```

(defun |npBacksetElse| ()
  (if (|npEqKey| 'backset)
      (|npEqKey| 'else)
      (|npEqKey| 'else)))

```

—————

defun npLogical

```

[npLeftAssoc p499]
[npDisjand p490]

```

— defun npLogical —


```
(defun |npLogical| ()
  (|npLeftAssoc| '(or) #'|npDisjand|))
```

defun npDisjand

```
[npLeftAssoc p499]
[npDiscrim p490]
```

— defun npDisjand —

```
(defun |npDisjand| ()
  (|npLeftAssoc| '(and) #'|npDiscrim|))
```

defun npDiscrim

```
[npLeftAssoc p499]
[npQuiver p490]
```

— defun npDiscrim —

```
(defun |npDiscrim| ()
  (|npLeftAssoc| '(case has) #'|npQuiver|))
```

defun npQuiver

```
[npRightAssoc p498]
[npRelation p491]
```

— defun npQuiver —

```
(defun |npQuiver| ()
  (|npRightAssoc| '(arrow larrow) #'|npRelation|))
```

defun npRelation

[npLeftAssoc p499]
[npSynthetic p491]

— defun npRelation —

```
(defun |npRelation| ()
  (|npLeftAssoc| '(equal notequal lt le gt ge oangle cangle) #'|npSynthetic|))
```

—————

defun npSynthetic

[npBy p492]
[npAmpersandFrom p495]
[npPush p435]
[pfApplication p551]
[npPop2 p436]
[npPop1 p436]
[pfInfApplication p569]

— defun npSynthetic —

```
(defun |npSynthetic| ()
  (cond
    ((|npBy|)
     ((lambda ()
        (loop
          (cond
            ((not (and (|npAmpersandFrom|)
                       (or (|npBy|)
                           (progn
                             (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))))
                             nil))))
          (return nil))
      (t
       (|npPush| (|pfInfApplication| (|npPop2|) (|npPop2|) (|npPop1|)))))))
    (t
     (t nil)))
```

—————

defun npBy

[npLeftAssoc p499]
 [npInterval p492]

— defun npBy —

```
(defun |npBy| ()
  (|npLeftAssoc| '(by) #'|npInterval|))
```

—————

defun

[npArith p493]
 [npSegment p493]
 [npEqPeek p444]
 [npPush p435]
 [pfApplication p551]
 [npPop1 p436]
 [pfInfApplication p569]
 [npPop2 p436]

— defun npInterval —

```
(defun |npInterval| ()
  (and
    (|npArith|)
    (or
      (and
        (|npSegment|)
        (or
          (and
            (|npEqPeek| 'bar)
            (|npPush| (|pfApplication| (|npPop1|) (|npPop1|))))
          (and
            (|npArith|)
            (|npPush| (|pfInfApplication| (|npPop2|) (|npPop2|) (|npPop1|))))
            (|npPush| (|pfApplication| (|npPop1|) (|npPop1|))))
        t)))
```

—————

defun npSegment

[npEqPeek p444]
 [npPushId p501]
 [npFromdom p495]

— defun npSegment —

```
(defun |npSegment| ()
  (and (|npEqPeek| 'seg) (|npPushId|) (|npFromdom|)))
```

—————

defun npArith

[npLeftAssoc p499]
 [npSum p493]

— defun npArith —

```
(defun |npArith| ()
  (|npLeftAssoc| '(mod) #'|npSum|))
```

—————

defun npSum

[npLeftAssoc p499]
 [npTerm p493]

— defun npSum —

```
(defun |npSum| ()
  (|npLeftAssoc| '(plus minus) #'|npTerm|))
```

—————

defun npTerm

[npInfGeneric p500]
 [npRemainder p494]
 [npPush p435]

[pfApplication p551]
 [npPop2 p436]
 [npPop1 p436]

— defun npTerm —

```
(defun |npTerm| ()
  (or
    (and
      (|npInfGeneric| '(minus plus))
      (or
        (and (|npRemainder|) (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))))
        t))
      (|npRemainder|)))
```

—————

defun npRemainder

[npLeftAssoc p499]
 [npProduct p494]

— defun npRemainder —

```
(defun |npRemainder| ()
  (|npLeftAssoc| '(rem quo) #'|npProduct|))
```

—————

defun npProduct

[npLeftAssoc p499]
 [npPower p495]

— defun npProduct —

```
(defun |npProduct| ()
  (|npLeftAssoc|
    '(times slash backslash slashslash backslashbackslash
      slashbackslash backslashslash)
    #'|npPower|))
```

—————

defun npPower

[npRightAssoc p498]
 [npColon p510]

— **defun npPower** —

```
(defun |npPower| ()
  (|npRightAssoc| '(power carat) #'|npColon|))
```

—————

defun npAmpersandFrom

[npAmpersand p496]
 [npFromdom p495]

— **defun npAmpersandFrom** —

```
(defun |npAmpersandFrom| ()
  (and (|npAmpersand|) (|npFromdom|)))
```

—————

defun npFromdom

[npEqKey p437]
 [npApplication p454]
 [npTrap p505]
 [npFromdom1 p496]
 [npPop1 p436]
 [npPush p435]
 [pfFromDom p565]

— **defun npFromdom** —

```
(defun |npFromdom| ()
  (or
    (and
      (|npEqKey| '$)
      (or (|npApplication|) (|npTrap|))
      (|npFromdom1| (|npPop1|))
      (|npPush| (|pfFromDom| (|npPop1|) (|npPop1|))))
    t))
```

defun npFromdom1

[npEqKey p437]
 [npApplication p454]
 [npTrap p505]
 [npFromdom1 p496]
 [npPop1 p436]
 [npPush p435]
 [pfFromDom p565]

— defun npFromdom1 —

```
(defun |npFromdom1| (c)
  (or
    (and
      (|npEqKey| '$)
      (or (|npApplication|) (|npTrap|))
      (|npFromdom1| (|npPop1|))
      (|npPush| (|pfFromDom| (|npPop1|) c)))
    (|npPush| c)))
```

defun npAmpersand

[npEqKey p437]
 [npName p496]
 [npTrap p505]

— defun npAmpersand —

```
(defun |npAmpersand| ()
  (and
    (|npEqKey| 'ampersand)
    (or (|npName|) (|npTrap|))))
```

defun npName

[npId p497]
 [npSymbolVariable p498]

— defun npName —

```
(defun |npName| ()
  (or (|npId|) (|npSymbolVariable|)))
```

defvar \$npTokToNames

— initvars —

```
(defvar |$npTokToNames| (list '~ '|#| '[] '{| '[[\|]| '|{\|}||))
```

defun npId

```
[npPush p435]
[npNext p437]
[tokConstruct p701]
[tokPosn p703]
|$npTokToNames p497]
[$ttok p??]
[$stok p??]
```

— defun npId —

```
(defun |npId| ()
  (declare (special |$npTokToNames| |$ttok| |$stok|))
  (cond
    ((eq (caar |$stok|) '|id|)
      (|npPush| |$stok|)
      (|npNext|))
    ((and (eq (caar |$stok|) '|key|) (member |$ttok| |$npTokToNames|))
      (|npPush| (|tokConstruct| '|id| |$ttok| (|tokPosn| |$stok|)))
      (|npNext|))
    (t nil)))
```

defun npSymbolVariable

[npState p504]
 [npEqKey p437]
 [npId p497]
 [npPop1 p436]
 [npPush p435]
 [tokConstruct p701]
 [tokPart p703]
 [tokPosn p703]
 [npRestore p444]

— defun npSymbolVariable —

```
(defun |npSymbolVariable| ()
  (let (a)
    (setq a (|npState|))
    (cond
      ((and (|npEqKey| 'backquote) (|npId|))
        (setq a (|npPop1|))
        (|npPush| (|tokConstruct| 'idsy| (|tokPart| a) (|tokPosn| a))))
      (t (|npRestore| a) nil))))
```

defun npRightAssoc

[npState p504]
 [npInfGeneric p500]
 [npRightAssoc p498]
 [npPush p435]
 [pfApplication p551]
 [npPop2 p436]
 [npPop1 p436]
 [pfInfApplication p569]
 [npRestore p444]

— defun npRightAssoc —

```
(defun |npRightAssoc| (o p)
  (let (a)
    (setq a (|npState|))
    (cond
      ((apply p nil)
        ((lambda ()
          (loop
```

```

(cond
  ((not
    (and
      (|npInfGeneric| o)
      (or
        (|npRightAssoc| o p)
        (progn (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))) nil))))
    (return nil))
  (t
    (|npPush| (|pfInfApplication| (|npPop2|) (|npPop2|) (|npPop1|))))))
  t)
(t
  (|npRestore| a)
  nil)))

```

defun p o p o p o p = (((p o p) o p) o p)

```

p o p o p o p = (((p o p) o p) o p)
p o p o = (p o p) o
;npLeftAssoc(operations,parser)==
;  if APPLY(parser,nil)
;  then
;    while npInfGeneric(operations)
;      and (APPLY(parser,nil) or
;        (npPush pfApplication(npPop2(),npPop1());false))
;      repeat
;        npPush pfInfApplication(npPop2(),npPop2(),npPop1())
;      true
;    else false

```

```

[|npInfGeneric| p500]
[|npPush| p435]
[|pfApplication| p551]
[|npPop2| p436]
[|npPop1| p436]
[|pfInfApplication| p569]

```

— **defun npLeftAssoc** —

```

(defun |npLeftAssoc| (operations parser)
  (when (apply parser nil)
    (lambda nil
      (loop
        (cond
          ((not

```

```

      (and
        (|npInfGeneric| operations)
        (or
          (apply parser nil)
          (progn (|npPush| (|pfApplication| (|npPop2|) (|npPop1|))) nil))))
      (return nil))
    (t
      (|npPush| (|pfInfApplication| (|npPop2|) (|npPop2|) (|npPop1|))))))
  t))

```

defun npInfGeneric

[npDDInfKey p500]
 [npEqKey p437]

— defun npInfGeneric —

```

(defun |npInfGeneric| (s)
  (and
    (|npDDInfKey| s)
    (or (|npEqKey| 'backset) t)))

```

defun npDDInfKey

[npInfKey p501]
 [npState p504]
 [npEqKey p437]
 [npPush p435]
 [pfSymb p549]
 [npPop1 p436]
 [tokPosn p703]
 [npRestore p444]
 [tokConstruct p701]
 [tokPart p703]
 [\$stok p??]

— defun npDDInfKey —

```

(defun |npDDInfKey| (s)
  (let (b a)

```

```

(declare (special |$stok|))
(or
  (|npInfKey| s)
  (progn
    (setq a (|npState|))
    (setq b |$stok|)
    (cond
      ((and (|npEqKey| '||) (|npInfKey| s))
        (|npPush| (|pfSymb| (|npPop1|) (|tokPosn| b))))
      (t
        (|npRestore| a)
        (cond
          ((and (|npEqKey| 'backquote) (|npInfKey| s))
            (setq a (|npPop1|))
            (|npPush| (|tokConstruct| '|idsy| (|tokPart| a) (|tokPosn| a))))
          (t
            (|npRestore| a)
            nil)))))))))

```

defun npInfKey

```

[|npPushId| p501]
|$stok| p??
|$ttok| p??

```

— defun npInfKey —

```

(defun |npInfKey| (s)
  (declare (special |$ttok| |$stok|))
  (and (eq (caar |$stok|) '|key|) (member |$ttok| s) (|npPushId|)))

```

defun npPushId

```

[|tokConstruct| p701]
[|tokPosn| p703]
[|npNext| p437]
|$stack| p??
|$stok| p??
|$ttok| p??

```

— defun npPushId —

```
(defun |npPushId| ()
  (let (a)
    (declare (special |$stack| |$stok| |$ttok|))
    (setq a (get |$ttok| 'infgeneric))
    (when a (setq |$ttok| a))
    (setq |$stack|
      (cons (|tokConstruct| 'id| |$ttok| (|tokPosn| |$stok|)) |$stack|))
    (|npNext|)))
```

defvar npPParg

— initvars —

```
(defvar *npPParg* nil "rewrite npPP without flets, using global scoping")
```

defun npPP

This was rewritten by NAG to remove flet.

```
[npParened p478]
[npPPf p503]
[npPileBracketed p481]
[npPPg p503]
[npPush p435]
[pfEnSequence p561]
[npPop1 p436]
[npPParg p502]
```

— defun npPP —

```
(defun |npPP| (f)
  (declare (special *npPParg*))
  (setq *npPParg* f)
  (or
    (|npParened| #'npPPf)
    (and (|npPileBracketed| #'npPPg) (|npPush| (|pfEnSequence| (|npPop1|))))
    (funcall f)))
```

defun npPPff

[npPop1 p436]
 [npPush p435]
 [\$npPParg p502]

— defun npPPff —

```
(defun npPPff ()
  (and (funcall *npPParg*) (|npPush| (list (|npPop1|))))))
```

—————

defun npPPg

[npListAndRecover p482]
 [npPPf p503]
 [npPush p435]
 [pfAppend p553]
 [npPop1 p436]

— defun npPPg —

```
(defun npPPg ()
  (and (|npListAndRecover| #'npPPf)
    (|npPush| (|pfAppend| (|npPop1|)))))
```

—————

defun npPPf

[npSemiListing p485]
 [npPPff p503]

— defun npPPf —

```
(defun npPPf ()
  (|npSemiListing| #'npPPff))
```

—————

defun npEnclosed

```

[npEqKey p437]
[npPush p435]
[pfTuple p590]
[pfListOf p543]
[npMissingMate p508]
[pfEnSequence p561]
[npPop1 p436]
[$stok p??]

```

— defun npEnclosed —

```

(defun |npEnclosed| (open close fn f)
  (let (a)
    (declare (special |$stok|))
    (setq a |$stok|)
    (when (|npEqKey| open)
      (cond
        ((|npEqKey| close)
         (|npPush| (funcall fn a (|pfTuple| (|pfListOf| NIL)))))
        ((and (apply f nil)
              (or (|npEqKey| close)
                  (|npMissingMate| close a)))
         (|npPush| (funcall fn a (|pfEnSequence| (|npPop1|)))))
        ('t nil)))))

```

—————

defun npState

```

[$stack p??]
[$inputStream p??]

```

— defun npState —

```

(defun |npState| ()
  (declare (special |$stack| |$inputStream|))
  (cons |$inputStream| |$stack|))

```

—————

defun npTrap

```
[trappoint p??]
[tokPosn p703]
[ncSoftError p642]
[$stok p??]
```

— defun npTrap —

```
(defun |npTrap| ()
  (declare (special |$stok|))
  (|ncSoftError| (|tokPosn| |$stok|) " Improper syntax." nil)
  (throw 'trappoint 'trapped))
```

—————

defun npTrapForm

```
[trappoint p??]
[pfSourceStok p541]
[syGeneralErrorHere p484]
[ncSoftError p642]
[tokPosn p703]
```

— defun npTrapForm —

```
(defun |npTrapForm| (x)
  (let (a)
    (setq a (|pfSourceStok| x))
    (cond
      ((eq a '|NoToken|)
        (|syGeneralErrorHere|)
        (throw 'trappoint 'trapped))
      (t
        (|ncSoftError| (|tokPosn| a) " Improper syntax." nil)
        (throw 'trappoint 'trapped))))))
```

—————

defun npVariable

```
[npParenthesized p507]
[npVariablelist p506]
[npVariableName p506]
```



```
[npPush p435]
[pfListOf p543]
[npPop1 p436]
```

— defun npVariable —

```
(defun |npVariable| ()
  (or
    (|npParenthesized| #'|npVariablelist|)
    (and (|npVariableName|) (|npPush| (|pfListOf| (list (|npPop1|)))))))
```

—————

defun npVariablelist

```
[npListing p447]
[npVariableName p506]
```

— defun npVariablelist —

```
(defun |npVariablelist| ()
  (|npListing| #'|npVariableName|))
```

—————

defun npVariableName

```
[npName p496]
[npDecl p507]
[npPush p435]
[pfTyped p588]
[npPop1 p436]
[pfNothing p543]
```

— defun npVariableName —

```
(defun |npVariableName| ()
  (and
    (|npName|)
    (or (|npDecl|) (|npPush| (|pfTyped| (|npPop1|) (|pfNothing|))))))
```

—————

defun npDecl

[npEqKey p437]
 [npType p441]
 [npTrap p505]
 [npPush p435]
 [pfTyped p588]
 [npPop2 p436]
 [npPop1 p436]

— defun npDecl —

```
(defun |npDecl| ()
  (and
    (|npEqKey| 'colon)
    (or (|npType|) (|npTrap|))
    (|npPush| (|pfTyped| (|npPop2|) (|npPop1|)))))
```

—————

defun npParenthesized

[npParenthesize p507]

— defun npParenthesized —

```
(defun |npParenthesized| (f)
  (or (|npParenthesize| '(| '|) f) (|npParenthesize| '(\| \| '|) f)))
```

—————

defun npParenthesize

[npEqKey p437]
 [npMissingMate p508]
 [npPush p435]
 [\$stok p??]

— defun npParenthesize —

```
(defun |npParenthesize| (open close f)
  (let (a)
    (declare (special |$stok|))
    (setq a |$stok|)
```

```

(cond
  ((|npEqKey| open)
   (cond
    ((and (apply f nil)
          (or (|npEqKey| close)
              (|npMissingMate| close a))))
    t)
   ((|npEqKey| close) (|npPush| nil))
   (t (|npMissingMate| close a))))
(t nil)))

```

defun npMissingMate

[ncSoftError p642]
 [tokPosn p703]
 [npMissing p443]

— defun npMissingMate —

```

(defun |npMissingMate| (close open)
  (|ncSoftError| (|tokPosn| open) "Missing mate." nil)
  (|npMissing| close))

```

defun npExit

[npBackTrack p440]
 [npAssign p509]
 [npPileExit p508]

— defun npExit —

```

(defun |npExit| ()
  (|npBackTrack| #'|npAssign| 'exit #'|npPileExit|))

```

defun npPileExit

[npAssign p509]
 [npEqKey p437]

[npStatement p463]
 [npPush p435]
 [pfExit p561]
 [npPop2 p436]
 [npPop1 p436]

— defun npPileExit —

```
(defun |npPileExit| ()
  (and
    (|npAssign|)
    (or (|npEqKey| 'exit) (|npTrap|))
    (or (|npStatement|) (|npTrap|))
    (|npPush| (|pfExit| (|npPop2|) (|npPop1|))))))
```

—————

defun npAssign

[npBackTrack p440]
 [npMDEF p457]
 [npAssignment p509]

— defun npAssign —

```
(defun |npAssign| ()
  (|npBackTrack| #'|npMDEF| 'becomes #'|npAssignment|))
```

—————

defun npAssignment

[npAssignVariable p510]
 [npEqKey p437]
 [npTrap p505]
 [npGives p440]
 [npPush p435]
 [pfAssign p553]
 [npPop2 p436]
 [npPop1 p436]

— defun npAssignment —

```
(defun |npAssignment| ()
```

```
(and
  (|npAssignVariable|)
  (or (|npEqKey| 'becomes) (|npTrap|))
  (or (|npGives|) (|npTrap|))
  (|npPush| (|pfAssign| (|npPop2|) (|npPop1|))))))
```

defun npAssignVariable

[npColon p510]
 [npPush p435]
 [pfListOf p543]
 [npPop1 p436]

— defun npAssignVariable —

```
(defun |npAssignVariable| ()
  (and (|npColon|) (|npPush| (|pfListOf| (list (|npPop1|))))))
```

defun npColon

[npTypified p511]
 [npAnyNo p455]
 [npTagged p510]

— defun npColon —

```
(defun |npColon| ()
  (and (|npTypified|) (|npAnyNo| #'|npTagged|)))
```

defun npTagged

[npTypedForm1 p511]
 [pfTagged p586]

— defun npTagged —

```
(defun |npTagged| ()
  (|npTypedForm1| 'colon #'|pfTagged|))
```

defun npTypedForm1

```
[npEqKey p437]
[npType p441]
[npTrap p505]
[npPush p435]
[npPop2 p436]
[npPop1 p436]
```

— defun npTypedForm1 —

```
(defun |npTypedForm1| (sy fn)
  (and
    (|npEqKey| sy)
    (or (|npType|) (|npTrap|))
    (|npPush| (funcall fn (|npPop2|) (|npPop1|))))))
```

defun npTypified

```
[npApplication p454]
[npAnyNo p455]
[npTypeStyle p511]
```

— defun npTypified —

```
(defun |npTypified| ()
  (and (|npApplication|) (|npAnyNo| #'|npTypeStyle|)))
```

defun npTypeStyle

```
[npCoerceTo p512]
[npRestrict p513]
[npPretend p512]
```

[npColonQuery p512]

— defun npTypeStyle —

```
(defun |npTypeStyle| ()
  (or (|npCoerceTo|) (|npRestrict|) (|npPretend|) (|npColonQuery|)))
```

—————

defun npPretend

[npTypedForm p513]

[pfPretend p579]

— defun npPretend —

```
(defun |npPretend| ()
  (|npTypedForm| 'pretend #'|pfPretend|))
```

—————

defun npColonQuery

[npTypedForm p513]

[pfRetractTo p582]

— defun npColonQuery —

```
(defun |npColonQuery| ()
  (|npTypedForm| 'atat #'|pfRetractTo|))
```

—————

defun npCoerceTo

[npTypedForm p513]

[pfCoerceto p557]

— defun npCoerceTo —

```
(defun |npCoerceTo| ()
  (|npTypedForm| 'coerce #'|pfCoerceto|))
```

defun npTypedForm

[npEqKey p437]
 [npApplication p454]
 [npTrap p505]
 [npPush p435]
 [npPop2 p436]
 [npPop1 p436]

— defun npTypedForm —

```
(defun |npTypedForm| (sy fn)
  (and
    (|npEqKey| sy)
    (or (|npApplication|) (|npTrap|))
    (|npPush| (funcall fn (|npPop2|) (|npPop1|)))))
```

defun npRestrict

[npTypedForm p513]
 [pfRestrict p581]

— defun npRestrict —

```
(defun |npRestrict| ()
  (|npTypedForm| 'at #'|pfRestrict|))
```

defun npListofFun

[npTrap p505]
 [npPush p435]
 [npPop3 p436]
 [npPop2 p436]
 [npPop1 p436]
 [\$stack p??]

— defun npListofFun —


```

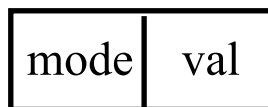
(defun |npListofFun| (f h g)
  (let (a)
    (declare (special |$stack|))
    (cond
      ((apply f nil)
       (cond
         ((and (apply h nil) (or (apply f nil) (|npTrap|)))
          (setq a |$stack|)
          (setq |$stack| nil)
          (do ()
              ((not (and (apply h nil)
                        (or (apply f nil) (|npTrap|)))))
            (setq |$stack| (cons (nreverse |$stack|) a))
            (|npPush| (funcall g (cons (|npPop3|) (cons (|npPop2|) (|npPop1|))))))
          (t t)))
       (t nil))))

```

14.1 Functions on interpreter objects

Interpreter objects used to be called triples because they had the structure [value, type, environment]. For many years, the environment was not used, so finally in January, 1990, the structure of objects was changed to be (type . value). This was chosen because it was the structure of objects of type Any. Sometimes the values are wrapped (see the function `isWrapped` to see what this means physically). Wrapped values are not actual values belonging to their types. An unwrapped value must be evaluated to get an actual value. A wrapped value must be unwrapped before being passed to a library function. Typically, an unwrapped value in the interpreter consists of LISP code, e.g., parts of a function that is being constructed. — RSS 1/14/90

These are the new structure functions.



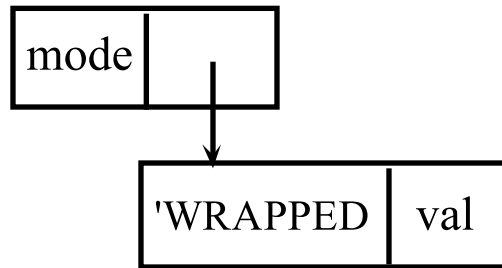
Object representation

defmacro mkObj

— **defmacro mkObj** —

```
(defmacro mkObj (val mode)
```

```
'(cons ,mode ,val))
```



Object representation

defmacro mkObjWrap

```
[wrap p1233]
```

— defmacro mkObjWrap —

```
(defmacro mkObjWrap (val mode)
  '(cons ,mode (|wrap| ,val)))
```

defmacro mkObjCode

— defmacro mkObjCode —

```
(defmacro mkObjCode (val mode)
  '(cons 'cons (cons (mkq ,mode) (cons ,val nil))))
```

defmacro objSetVal

— defmacro objSetVal —

```
(defmacro |objSetVal| (obj val)
  '(rplacd ,obj ,val))
```

defmacro objSetMode

— defmacro objSetMode —

```
(defmacro |objSetMode| (obj mode)
  '(rplaca ,obj ,mode))
```

defmacro objVal

— defmacro objVal —

```
(defmacro |objVal| (obj)
  '(cdr ,obj))
```

defmacro objValUnwrap

— defmacro objValUnwrap —

```
(defmacro |objValUnwrap| (obj)
  '(|unwrap| (cdr ,obj)))
```

defmacro objMode

— defmacro objMode —

```
(defmacro |objModel| (obj)
  `(car ,obj))
```

defun objEnv

— defun objEnv 0 —

```
(defun |objEnv| (obj)
  (declare (special $NE) (ignore obj))
  $NE)
```

defmacro objCodeVal

— defmacro objCodeVal —

```
(defmacro |objCodeVal| (obj)
  `(caddr ,obj))
```

defmacro objCodeMode

— defmacro objCodeMode —

```
(defmacro |objCodeMode| (obj)
  `(cadr ,obj))
```

14.2 Macro handling

defun phMacro

TPDHERE: The pform function has a leading percent sign

```
carrier[ptree,...] -> carrier[ptree, ptreePremacro,...]
```

```
[ncEltQ p706]
[ncPutQ p706]
[macroExpanded p518]
[pform p??]
```

— defun phMacro —

```
(defun |phMacro| (carrier)
  (let (ptree)
    (setq ptree (|ncEltQ| carrier '|ptree|))
    (|ncPutQ| carrier '|ptreePremacro| ptree)
    (setq ptree (|macroExpanded| ptree))
    (|ncPutQ| carrier '|ptree| ptree)
    'ok))
```

—————→

defun macroExpanded

\$macActive is a list of the bodies being expanded. \$posActive is a list of the parse forms where the bodies came from. [macExpand p518]

```
[$posActive p??]
[$macActive p??]
```

— defun macroExpanded —

```
(defun |macroExpanded| (pf)
  (let (|$posActive| |$macActive|)
    (declare (special |$posActive| |$macActive|))
    (setq |$macActive| nil)
    (setq |$posActive| nil)
    (|macExpand| pf)))
```

—————→

defun macExpand

```
[pfWhere? p592]
[macWhere p524]
[pfLambda? p571]
[macLambda p525]
[pfMacro? p575]
```

[[macMacro](#) [p525](#)]
 [[pfId?](#) [p544](#)]
 [[macId](#) [p523](#)]
 [[pfApplication?](#) [p553](#)]
 [[macApplication](#) [p519](#)]
 [[pfMapParts](#) [p534](#)]
 [[macExpand](#) [p518](#)]

— **defun** `macExpand` —

```
(defun |macExpand| (pf)
  (cond
    ((|pfWhere?| pf)      (|macWhere| pf))
    ((|pfLambda?| pf)    (|macLambda| pf))
    ((|pfMacro?| pf)     (|macMacro| pf))
    ((|pfId?| pf)        (|macId| pf))
    ((|pfApplication?| pf) (|macApplication| pf))
    (t                    (|pfMapParts| #'|macExpand| pf))))
```

defun `macApplication`

[[pfMapParts](#) [p534](#)]
 [[macExpand](#) [p518](#)]
 [[pfApplicationOp](#) [p552](#)]
 [[pfMLambda?](#) [p576](#)]
 [[pf0ApplicationArgs](#) [p535](#)]
 [[mac0MLambdaApply](#) [p520](#)]
 [[\\$pfMacros](#) [p392](#)]

— **defun** `macApplication` —

```
(defun |macApplication| (pf)
  (let (args op)
    (declare (special |$pfMacros|))
    (setq pf (|pfMapParts| #'|macExpand| pf))
    (setq op (|pfApplicationOp| pf))
    (cond
      ((null (|pfMLambda?| op)) pf)
      (t
       (setq args (|pf0ApplicationArgs| pf))
       (|mac0MLambdaApply| op args pf |$pfMacros|))))
```

defun mac0MLambdaApply**TPDHERE:** The `pform` function has a leading percent sign. fix this

[\[pf0MLambdaArgs p576\]](#)
[\[pfMLambdaBody p577\]](#)
[\[pfSourcePosition p536\]](#)
[\[incHardError p642\]](#)
[\[pfId? p544\]](#)
[\[pform p??\]](#)
[\[mac0Define p526\]](#)
[\[mac0ExpandBody p521\]](#)
[\[\\$pfMacros p392\]](#)
[\[\\$posActive p??\]](#)
[\[\\$macActive p??\]](#)

— **defun mac0MLambdaApply** —

```

(defun |mac0MLambdaApply| (mlambda args opf |$pfMacros|)
  (declare (special |$pfMacros|))
  (let (pos body params)
    (declare (special |$posActive| |$macActive|))
    (setq params (|pf0MLambdaArgs| mlambda))
    (setq body (|pfMLambdaBody| mlambda))
    (cond
      ((not (eql (length args) (length params)))
       (setq pos (|pfSourcePosition| opf))
       (|incHardError| pos "Expected %1 arguments, but received %2."
        (list (length params) (length args))))
      (t
       ((lambda (parms p arrgs a) ; for p in params for a in args repeat
          (loop
            (cond
              ((or (atom parms)
                   (progn (setq p (car parms)) nil)
                   (atom arrgs)
                   (progn (setq a (CAR arrgs)) nil))
               (return nil))
              (t
               (cond
                 ((null (|pfId?| p))
                  (setq pos (|pfSourcePosition| opf))
                  (|incHardError| pos "Macro parameter %1f is not an id."
                   (list (|%pform| p))))
                 (t
                  (|mac0Define| (|pfIdSymbol| p) '|mparam| a))))
               (setq parms (cdr parms))
               (setq arrgs (cdr arrgs))))
          params nil args nil)

```

```
(|mac0ExpandBody| body opf |$macActive| |$posActive|))))))
```

defun mac0ExpandBody

```
[pfSourcePosition p536]
[mac0InfiniteExpansion p521]
[macExpand p518]
[$posActive p??]
[$macActive p??]
```

— defun mac0ExpandBody —

```
(defun |mac0ExpandBody| (body opf |$macActive| |$posActive|)
  (declare (special |$macActive| |$posActive|))
  (let (posn pf)
    (cond
      ((member body |$macActive|)
       (setq pf (cadr |$posActive|))
       (setq posn (|pfSourcePosition| pf))
       (|mac0InfiniteExpansion| posn body |$macActive|))
      (t
       (setq |$macActive| (cons body |$macActive|))
       (setq |$posActive| (cons opf |$posActive|))
       (|macExpand| body))))))
```

defun mac0InfiniteExpansion

TPDHERE: The pform function has a leading percent sign. fix this

```
[mac0InfiniteExpansion,name p522]
[ncSoftError p642]
[pform p??]
```

— defun mac0InfiniteExpansion —

```
(defun |mac0InfiniteExpansion| (posn body active)
  (let (rnames fname tmp1 blist result)
    (setq blist (cons body active))
    (setq tmp1 (mapcar #'|mac0InfiniteExpansion,name| blist))
    (setq fname (car tmp1)) ;[fname, :rnames] := [name b for b in blist]
    (setq rnames (cdr tmp1))
```



```
(|ncSoftError| posn
  "noRep Cycle in macro expansion: %l %1y %2 %l. Left as: %3f"
  (list
    (dolist (n (reverse rnames) (nreverse result))
      (setq result (append (reverse (list n "==">)) result)))
    fname (|%pform| body)))
  body))
```

defun mac0InfiniteExpansion,name

[mac0GetName p522]
 [pname p1235]

— defun mac0InfiniteExpansion,name 0 —

```
(defun |mac0InfiniteExpansion,name| (b)
  (let (st sy got)
    (setq got (|mac0GetName| b))
    (cond
      ((null got) "???" )
      (t
       (setq sy (car got))
       (setq st (cadr got))
       (if (eq st 'mlambda|)
         (concat (pname sy) "(...)")
         (pname sy))))))
```

defun mac0GetName

Returns [state, body] or NIL. Returns [sy, state] or NIL.

[pfMLambdaBody p577]
 [\$pfMacros p392]

— defun mac0GetName —

```
(defun |mac0GetName| (body)
  (let (bd tmp1 st tmp2 sy name)
    (declare (special |$pfMacros|))
    ; for [sy,st,bd] in $pfMacros while not name repeat
    ((lambda (macros tmplist)
```

```

(loop
  (cond
    ((or (atom macros)
         (progn (setq tmplist (car macros)) nil)
         name)
      (return nil))
    (t
     (and (consp tmplist)
          (progn
            (setq sy (car tmplist))
            (setq tmp2 (cdr tmplist))
            (and (consp tmp2)
                 (progn
                  (setq st (car tmp2))
                  (setq tmp1 (cdr tmp2))
                  (and (consp tmp1)
                      (eq (cdr tmp1) nil)
                      (progn
                       (setq bd (car tmp1))
                       t))))))
          (progn
            (when (eq st '|lambda|) (setq bd (|pfMLambdaBody| bd)))
            (when (eq bd body) (setq name (list sy st))))))
      (setq macros (cdr macros))))
    |$pfMacros| nil)
  name))

```

defun macId

```

[pfIdSymbol p545]
[mac0Get p524]
[pfCopyWithPos p534]
[pfSourcePosition p536]
[mac0ExpandBody p521]
[$posActive p??]
[$macActive p??]

```

— defun macId —

```

(defun |macId| (pf)
  (let (body state got sy)
    (declare (special |$posActive| |$macActive|))
    (setq sy (|pfIdSymbol| pf))
    (cond
      ((null (setq got (|mac0Get| sy))) pf)

```

```

(t
  (setq state (car got))
  (setq body (cadr got))
  (cond
    ((eq state '|mparam|) body)
    ((eq state '|mlambda|) (|pfCopyWithPos| body (|pfSourcePosition| pf)))
    (t
     (|pfCopyWithPos|
      (|mac0ExpandBody| body pf |$macActive| |$posActive|)
      (|pfSourcePosition| pf))))))

```

defun mac0Get

[ifcdr p??]
 [\$pfMacros p392]

— defun mac0Get —

```

(defun |mac0Get| (sy)
  (declare (special |$pfMacros|))
  (ifcdr (assoc sy |$pfMacros|)))

```

defun macWhere

[macWhere,mac p524]
 [\$pfMacros p392]

— defun macWhere —

```

(defun |macWhere| (pf)
  (declare (special |$pfMacros|))
  (|macWhere,mac| pf |$pfMacros|))

```

defun macWhere,mac

[pfMapParts p534]
 [macExpand p518]

[[\\$pfMacros p392](#)]

— **defun macWhere,mac** —

```
(defun |macWhere,mac| (pf |$pfMacros|)
  (declare (special |$pfMacros|))
  (|pfMapParts| #'|macExpand| pf))
```

defun macLambda

[[macLambda,mac p525](#)]

[[\\$pfMacros p392](#)]

— **defun macLambda** —

```
(defun |macLambda| (pf)
  (declare (special |$pfMacros|))
  (|macLambda,mac| pf |$pfMacros|))
```

defun macLambda,mac

[[pfMapParts p534](#)]

[[macExpand p518](#)]

[[\\$pfMacros p392](#)]

— **defun macLambda,mac** —

```
(defun |macLambda,mac| (pf |$pfMacros|)
  (declare (special |$pfMacros|))
  (|pfMapParts| #'|macExpand| pf))
```

defun Add appropriate definition the a Macro pform

This function adds the definition and returns the original Macro pform. **TPDHERE: The pform function has a leading percent sign. fix this** [[pfMacroLhs p575](#)]

[[pfMacroRhs p575](#)]

[\[pfId? p544\]](#)
[\[ncSoftError p642\]](#)
[\[pfSourcePosition p536\]](#)
[\[pfIdSymbol p545\]](#)
[\[mac0Define p526\]](#)
[\[pform p??\]](#)
[\[pfMLambda? p576\]](#)
[\[macSubstituteOuter p527\]](#)
[\[pfNothing? p543\]](#)
[\[pfMacro p575\]](#)
[\[pfNothing p543\]](#)

— defun `macMacro` —

```

(defun |macMacro| (pf)
  (let (sy rhs lhs)
    (setq lhs (|pfMacroLhs| pf))
    (setq rhs (|pfMacroRhs| pf))
    (cond
      ((null (|pfId?| lhs))
       (|ncSoftError| (|pfSourcePosition| lhs)
        "%1 is improper for macro definition. Ignored."
        (list (|%pform| lhs)))
       pf)
      (t
       (setq sy (|pfIdSymbol| lhs))
       (|mac0Define| sy
        (cond
          ((|pfMLambda?| rhs) '|mlambda|)
          (t '|mbody|))
        (|macSubstituteOuter| rhs))
       (cond
          ((|pfNothing?| rhs) pf)
          (t (|pfMacro| lhs (|pfNothing|))))))))

```

—————→

defun Add a macro to the global `pfMacros` list

[\[\\$pfMacros p392\]](#)

— defun `mac0Define 0` —

```

(defun |mac0Define| (sy state body)
  (declare (special |$pfMacros|))
  (setq |$pfMacros| (cons (list sy state body) |$pfMacros|)))

```

defun macSubstituteOuter

[mac0SubstituteOuter p527]
 [macLambdaParameterHandling p527]

— defun macSubstituteOuter —

```
(defun |macSubstituteOuter| (pform)
  (|mac0SubstituteOuter| (|macLambdaParameterHandling| nil pform) pform))
```

defun mac0SubstituteOuter

[pfId? p544]
 [macSubstituteId p528]
 [pfLeaf? p545]
 [pfLambda? p571]
 [macLambdaParameterHandling p527]
 [mac0SubstituteOuter p527]
 [pfParts p547]

— defun mac0SubstituteOuter —

```
(defun |mac0SubstituteOuter| (replist pform)
  (let (tmplist)
    (cond
      ((|pfId?| pform) (|macSubstituteId| replist pform))
      ((|pfLeaf?| pform) pform)
      ((|pfLambda?| pform)
       (setq tmplist (|macLambdaParameterHandling| replist pform))
       (dolist (p (|pfParts| pform)) (|mac0SubstituteOuter| tmplist p))
       pform)
      (t
       (dolist (p (|pfParts| pform)) (|mac0SubstituteOuter| replist p))
       pform))))
```

defun macLambdaParameterHandling

[pfLeaf? p545]
 [pfLambda? p571]

[\[pfTypedId p589\]](#)
[\[pf0LambdaArgs p572\]](#)
[\[pfIdSymbol p545\]](#)
[\[pfMLambda? p576\]](#)
[\[pf0MLambdaArgs p576\]](#)
[\[pfLeaf p545\]](#)
[\[pfAbSynOp p702\]](#)
[\[pfLeafPosition p546\]](#)
[\[pfParts p547\]](#)
[\[macLambdaParameterHandling p527\]](#)

— **defun macLambdaParameterHandling** —

```

(defun |macLambdaParameterHandling| (replist pform)
  (let (parlist symlist result)
    (cond
      ((|pfLeaf?| pform) nil)
      ((|pfLambda?| pform) ; remove ( identifier . replacement ) from assoclist
       (setq parlist (mapcar #'|pfTypedId| (|pf0LambdaArgs| pform)))
       (setq symlist (mapcar #'|pfIdSymbol| parlist))
       (dolist (par symlist)
         (setq replist
              (let ((pr (assoc par replist :test #'equal)))
                (when pr (remove par replist :test #'equal))))))
       replist)
      ((|pfMLambda?| pform) ;construct assoclist ( identifier . replacement )
       (setq parlist (|pf0MLambdaArgs| pform)) ; extract parameter list
       (dolist (par parlist (nreverse result))
         (push
          (cons (|pfIdSymbol| par)
                (|pfLeaf| (|pfAbSynOp| par) (gensym) (|pfLeafPosition| par)))
          result)))
      (t
       (dolist (p (|pfParts| pform))
         (|macLambdaParameterHandling| replist p))))))

```

—

defun macSubstituteId

[\[pfIdSymbol p545\]](#)

— **defun macSubstituteId** —

```

(defun |macSubstituteId| (replist pform)
  (let (ex)

```

```
(setq ex (assoc (|pfIdSymbol| pform) replist :test #'eq))
(cond
  (ex
    (rplaca pform (cadr ex))
    (rplacd pform (cddr ex))
    pform)
  (t pform))))
```

Chapter 15

Pftrees

15.1 Abstract Syntax Trees Overview

Th functions create and examine abstract syntax trees. These are called pforms, for short.

The pform data structure

- Leaves: [hd, tok, pos] where pos is optional
- Trees: [hd, tree, tree, ...]
- hd is either an id or (id . alist)

The leaves are:

char	:=	('char expr position)
Document	:=	('Document expr position)
error	:=	('error expr position)
expression	:=	('expression expr position)
float	:=	('float expr position)
id	:=	('id expr position)
idsy	:=	('idsy expr position)
integer	:=	('integer expr position)
string	:=	('string expr position)
symbol	:=	('symbol expr position)

The special nodes:

ListOf	:=	('listOf items)
Nothing	:=	('nothing)
SemiColon	:=	('SemiColon (Body: Expr))

The expression nodes:

Add	:= ('Add (Base: [Typed], Addin: Expr))
And	:= ('And left right)
Application	:= ('Application (Op: Expr, Arg: Expr))
Assign	:= ('Assign (LhsItems: [AssLhs], Rhs: Expr))
Attribute	:= ('Attribute (Expr: Primary))
Break	:= ('Break (From: ? Id))
Coerceto	:= ('Coerceto (Expr: Expr, Type: Type))
Collect	:= ('Collect (Body: Expr, Iterators: [Iterator]))
ComDefinition	:= ('ComDefinition (Doc: Document, Def: Definition))
DeclPart	
Definition	:= ('Definition (LhsItems: [Typed], Rhs: Expr))
DefinitionSequence	:= (Args: [DeclPart])
Do	:= ('Do (Body: Expr))
Document	:= ('Document strings)
DWhere	:= ('DWhere (Context: [DeclPart], Expr: [DeclPart]))
EnSequence	:=
Exit	:= ('Exit (Cond: ? Expr, Expr: ? Expr))
Export	:= ('Export (Items: [Typed]))
Forin	:= ('Forin (Lhs: [AssLhs], Whole: Expr))
Free	:= ('Free (Items: [Typed]))
Fromdom	:= ('Fromdom (What: Id, Domain: Type))
Hide	:= ('hide, arg)
If	:= ('If (Cond: Expr, Then: Expr, Else: ? Expr))
Import	:= ('Import (Items: [QualType]))
Inline	:= ('Inline (Items: [QualType]))
Iterate	:= ('Iterate (From: ? Id))
Lambda	:= ('Lambda (Args: [Typed], Rets: ReturnedTyped, Body: Expr))
Literal	
Local	:= ('Local (Items: [Typed]))
Loop	:= ('Loop (Iterators: [Iterator]))
Macro	:= ('Macro (Lhs: Id, Rhs: ExprorNot))
MLambda	:= ('MLambda (Args: [Id], Body: Expr))
Not	:= ('Not arg)
Novalue	:= ('Novalue (Expr: Expr))
Or	:= ('Or left right)
Pretend	:= ('Pretend (Expr: Expr, Type: Type))
QualType	:= ('QualType (Type: Type, Qual: ? Type))
Restrict	:= ('Restrict (Expr: Expr, Type: Type))
Retract	:= ('RetractTo (Expr: Expr, Type: Type))
Return	:= ('Return (Expr: ? Expr, From: ? Id))
ReturnTyped	:= ('returntyuped (type body))
Rule	:= ('Rule (lhsitems, rhsitems))
Sequence	:= ('Sequence (Args: [Expr]))
Suchthat	:= ('Suchthat (Cond: Expr))
Symb	:= if leaf then symbol else expression
Tagged	:= ('Tagged (Tag: Expr, Expr: Expr))
TLambda	:= ('TLambda (Args: [Typed], Rets: ReturnedTyped Type, Body: Expr))
Tuple	:= ('Tuple (Parts: [Expr]))
Typed	:= ('Typed (Id: Id, Type: ? Type))
Typing	:= ('Typing (Items: [Typed]))
Until	:= ('Until (Cond: Expr)) NOT USED
WDeclare	:= ('WDeclare (Signature: Typed, Doc: ? Document))
Where	:= ('Where (Context: [DeclPart], Expr: Expr))
While	:= ('While (Cond: Expr))
With	:= ('With (Base: [Typed], Within: [WithPart]))

Special cases of expression nodes are:

- Application. The Op parameter is one of `and`, `or`, `Y`, `|`, `{}`, `[]`, `{|}|`, `[|]|`
- DeclPart. The comment is attached to all signatutres in Typing, Import, Definition, Sequence, DWhere, Macro nodes
- EnSequence. This is either a Tuple or Sequence depending on the argument
- Literal. One of integer symbol expression one zero char string float of the form ('expression expr position)

15.2 Structure handlers

defun pfGlobalLinePosn

[poGlobalLinePosn p364]

— defun pfGlobalLinePosn —

```
(defun |pfGlobalLinePosn| (posn)
  (|poGlobalLinePosn| posn))
```

—————

defun pfCharPosn

[poCharPosn p667]

— defun pfCharPosn —

```
(defun |pfCharPosn| (posn)
  (|poCharPosn| posn))
```

—————

defun pfLinePosn

[poLinePosn p651]

— defun pfLinePosn —

```
(defun |pfLinePosn| (posn)
  (|poLinePosn| posn))
```

defun pfFileName

[poFileName p650]

— defun pfFileName —

```
(defun |pfFileName| (posn)
  (|poFileName| posn))
```

defun pfCopyWithPos

[pfLeaf? p545]
 [pfLeaf p545]
 [pfAbSynOp p702]
 [tokPart p703]
 [pfTree p550]
 [pfParts p547]
 [pfCopyWithPos p534]

— defun pfCopyWithPos —

```
(defun |pfCopyWithPos| (pform pos)
  (if (|pfLeaf?| pform)
      (|pfLeaf| (|pfAbSynOp| pform) (|tokPart| pform) pos)
      (|pfTree| (|pfAbSynOp| pform)
                (loop for p in (|pfParts| pform)
                      collect (|pfCopyWithPos| p pos))))))
```

defun pfMapParts

[pfLeaf? p545]
 [pfParts p547]
 [pfTree p550]

[pfAbSynOp p702]

— defun pfMapParts —

```
(defun |pfMapParts| (f pform)
  (let (parts1 parts0)
    (if (|pfLeaf?| pform)
        pform
        (progn
         (setq parts0 (|pfParts| pform))
         (setq parts1 (loop for p in parts0 collect (funcall f p)))
         (if (reduce #'(lambda (u v) (and u v)) (mapcar #'eq parts0 parts1))
             pform
             (|pfTree| (|pfAbSynOp| pform) parts1))))))
```

—————

defun pf0ApplicationArgs

[pf0FlattenSyntacticTuple p535]

[pfApplicationArg p552]

— defun pf0ApplicationArgs —

```
(defun |pf0ApplicationArgs| (pform)
  (|pf0FlattenSyntacticTuple| (|pfApplicationArg| pform)))
```

—————

defun pf0FlattenSyntacticTuple

[pfTuple? p590]

[pf0FlattenSyntacticTuple p535]

[pf0TupleParts p591]

— defun pf0FlattenSyntacticTuple —

```
(defun |pf0FlattenSyntacticTuple| (pform)
  (if (null (|pfTuple?| pform))
      (list pform)
      ; [:pf0FlattenSyntacticTuple p for p in pf0TupleParts pform]
      ((lambda (arg0 arg1 p)
        (loop
         (cond
          ((or (atom arg1) (progn (setq p (car arg1)) nil))
```

```

      (return (nreverse arg0)))
    (t
      (setq arg0 (append (reverse (|pf0FlattenSyntacticTuple| p)) arg0)))
      (setq arg1 (cdr arg1)))
    nil (|pf0TupleParts| pform) nil)))

```

defun pfSourcePosition

[\[pfLeaf? p545\]](#)
[\[pfLeafPosition p546\]](#)
[\[poNoPosition? p703\]](#)
[\[pfSourcePosition p536\]](#)
[\[pfParts p547\]](#)
[\[\\$nopos p319\]](#)

— defun pfSourcePosition —

```

(defun |pfSourcePosition| (form)
  (let (pos)
    (declare (special |$nopos|))
    (cond
      ((|pfLeaf?! form) (|pfLeafPosition| form))
      (t
        (setq pos |$nopos|)
        ((lambda (theparts p) ; for p in parts while poNoPosition? pos repeat
          (loop
            (cond
              ((or (atom theparts)
                 (progn (setq p (car theparts)) nil)
                 (not (|poNoPosition?! pos))))
               (return nil))
              (t (setq pos (|pfSourcePosition| p))))
            (setq theparts (cdr theparts))))
          (|pfParts| form) nil)
        pos))))

```

defun Convert a Sequence node to a list

[\[pfSequence? p585\]](#)
[\[pfSequenceArgs p585\]](#)
[\[pfListOf p543\]](#)

— defun pfSequenceToList —

```
(defun |pfSequenceToList| (x)
  (if (|pfSequence?| x)
      (|pfSequenceArgs| x)
      (|pfListOf| (list x))))
```

—————

defun pfSpread

[pfTyped p588]

— defun pfSpread —

```
(defun |pfSpread| (arg1 arg2)
  (mapcar #'(lambda (i) (|pfTyped| i arg2)) arg1))
```

—————

defun Deconstruct nodes to lists

```
[pfTagged? p587]
[pfTaggedExpr p587]
[pfNothing p543]
[pfTaggedTag p587]
[pfId? p544]
[pfListOf p543]
[pfTyped p588]
[pfCollect1? p540]
[pfCollectVariable1 p540]
[pfTuple? p590]
[pf0TupleParts p591]
[pfTaggedToTyped p587]
[pfDefinition? p559]
[pfApplication? p553]
[pfFlattenApp p539]
[pfTaggedToTyped1 p542]
[pfTransformArg p542]
[npTrapForm p505]
```

— defun pfCheckItOut —


```

(defun |pfCheckItOut| (x)
  (let (args op ls form rt result)
    (if (|pfTagged?| x)
      (setq rt (|pfTaggedExpr| x))
      (setq rt (|pfNothing|)))
    (if (|pfTagged?| x)
      (setq form (|pfTaggedTag| x))
      (setq form x))
    (cond
      ((|pfId?| form)
       (list (|pfListOf| (list (|pfTyped| form rt))) nil rt))
      ((|pfCollect1?| form)
       (list (|pfListOf| (list (|pfCollectVariable1| form))) nil rt))
      ((|pfTuple?| form)
       (list (|pfListOf|
        (dolist (part (|pf0TupleParts| form) (nreverse result))
          (push (|pfTaggedToTyped| part) result)))
        nil rt))
      ((|pfDefinition?| form)
       (list (|pfListOf| (list (|pfTyped| form (|pfNothing|)))) nil rt))
      ((|pfApplication?| form)
       (setq ls (|pfFlattenApp| form))
       (setq op (|pfTaggedToTyped1| (car ls)))
       (setq args
        (dolist (part (cdr ls) (nreverse result))
          (push (|pfTransformArg| part) result)))
       (list (|pfListOf| (list op)) args rt))
      (t (|npTrapForm| form)))))

```

defun pfCheckMacroOut

[\[pfId? p544\]](#)
[\[pfApplication? p553\]](#)
[\[pfFlattenApp p539\]](#)
[\[pfCheckId p539\]](#)
[\[pfCheckArg p539\]](#)
[\[npTrapForm p505\]](#)

— defun pfCheckMacroOut —

```

(defun |pfCheckMacroOut| (form)
  (let (args op ls)
    (cond
      ((|pfId?| form) (list form nil))
      ((|pfApplication?| form)

```

```

(setq ls (|pfFlattenApp| form))
(setq op (|pfCheckId| (car ls)))
(setq args (mapcar #'|pfCheckArg| (cdr ls)))
(list op args))
(t (|npTrapForm| form))))

```

defun pfCheckArg

[pfTuple? p590]
 [pf0TupleParts p591]
 [pfListOf p543]
 [pfCheckId p539]

— defun pfCheckArg —

```

(defun |pfCheckArg| (args)
  (let (arg1)
    (if (|pfTuple?| args)
        (setq arg1 (|pf0TupleParts| args))
        (setq arg1 (list args)))
    (|pfListOf| (mapcar #'|pfCheckId| arg1))))

```

defun pfCheckId

[pfId? p544]
 [npTrapForm p505]

— defun pfCheckId —

```

(defun |pfCheckId| (form)
  (if (null (|pfId?| form))
      (|npTrapForm| form)
      form))

```

defun pfFlattenApp

[pfApplication? p553]
 [pfCollect1? p540]

[pfFlattenApp p539]
 [pfApplicationOp p552]
 [pfApplicationArg p552]

— **defun pfFlattenApp** —

```
(defun |pfFlattenApp| (x)
  (cond
    ((|pfApplication?| x)
     (cond
       ((|pfCollect1?| x) (LIST x))
       (t
        (append (|pfFlattenApp| (|pfApplicationOp| x))
                  (|pfFlattenApp| (|pfApplicationArg| x))))))
    (t (list x))))
```

—————

defun pfCollect1?

[pfApplication? p553]
 [pfApplicationOp p552]
 [pfId? p544]
 [pfIdSymbol p545]

— **defun pfCollect1?** —

```
(defun |pfCollect1?| (x)
  (let (a)
    (when (|pfApplication?| x)
      (setq a (|pfApplicationOp| x))
      (when (|pfId?| a) (eq (|pfIdSymbol| a) '|\\|))))))
```

—————

defun pfCollectVariable1

[pfApplicationArg p552]
 [pf0TupleParts p591]
 [pfTaggedToTyped p587]
 [pfTyped p588]
 [pfSuch p542]
 [pfTypedId p589]
 [pfTypedType p589]

— defun pfCollectVariable1 —

```
(defun |pfCollectVariable1| (x)
  (let (id var a)
    (setq a (|pfApplicationArg| x))
    (setq var (car (|pf0TupleParts| a)))
    (setq id (|pfTaggedToTyped| var))
    (|pfTyped|
     (|pfSuch| (|pfTypedId| id) (cadr (|pf0TupleParts| a)))
     (|pfTypedType| id))))
```

defun pfPushMacroBody

[pfMLambda p576]
[pfPushMacroBody p541]

— defun pfPushMacroBody —

```
(defun |pfPushMacroBody| (args body)
  (if (null args)
      body
      (|pfMLambda| (car args) (|pfPushMacroBody| (cdr args) body))))
```

defun pfSourceStok

[pfLeaf? p545]
[pfParts p547]
[pfSourceStok p541]
[pfFirst p562]

— defun pfSourceStok —

```
(defun |pfSourceStok| (x)
  (cond
   ((|pfLeaf?| x) x)
   ((null (|pfParts| x)) '|NoToken|)
   (t (|pfSourceStok| (|pfFirst| x)))))
```

defun pfTransformArg

[pfTuple? p590]
 [pf0TupleParts p591]
 [pfListOf p543]
 [pfTaggedToTyped1 p542]

— defun pfTransformArg —

```
(defun |pfTransformArg| (args)
  (let (arglist result)
    (if (|pfTuple?| args)
        (setq arglist (|pf0TupleParts| args))
        (setq arglist (list args)))
    (|pfListOf|
     (dolist (|i| arglist (nreverse result))
       (push (|pfTaggedToTyped1| |i|) result))))))
```

—————→

defun pfTaggedToTyped1

[pfCollect1? p540]
 [pfCollectVariable1 p540]
 [pfDefinition? p559]
 [pfTyped p588]
 [pfNothing p543]
 [pfTaggedToTyped p587]

— defun pfTaggedToTyped1 —

```
(defun |pfTaggedToTyped1| (arg)
  (cond
    ((|pfCollect1?| arg) (|pfCollectVariable1| arg))
    ((|pfDefinition?| arg) (|pfTyped| arg (|pfNothing|)))
    (t (|pfTaggedToTyped| arg))))
```

—————→

defun pfSuch

[pfInfApplication p569]
 [pfId p544]

— defun pfSuch —

```
(defun |pfSuch| (x y)
  (|pfInfApplication| (|pfId| '|\|) x y))
```

15.3 Special Nodes

defun Create a Listof node

[pfTree p550]

— defun pfListOf —

```
(defun |pfListOf| (x)
  (|pfTree| 'listOf x))
```

defun pfNothing

[pfTree p550]

— defun pfNothing —

```
(defun |pfNothing| ()
  (|pfTree| 'nothing nil))
```

defun Is this a Nothing node?

[pfAbSynOp? p702]

— defun pfNothing? —

```
(defun |pfNothing?| (form)
  (|pfAbSynOp?| form 'nothing))
```

15.4 Leaves

defun Create a Document node

[pfLeaf p545]

— defun pfDocument —

```
(defun |pfDocument| (strings)
  (|pfLeaf| '|Document| strings))
```

—————

defun Construct an Id node

[pfLeaf p545]

— defun pfId —

```
(defun |pfId| (expr)
  (|pfLeaf| '|Id| expr))
```

—————

defun Is this an Id node?

[pfAbSynOp? p702]

— defun pfId? —

```
(defun |pfId?| (form)
  (or (|pfAbSynOp?| form '|Id|) (|pfAbSynOp?| form '|idsy|)))
```

—————

defun Construct an Id leaf node

[pfLeaf p545]

— defun pfIdPos —

```
(defun |pfIdPos| (expr pos)
  (|pfLeaf| ' |id| expr pos))
```

defun Return the Id part

[tokPart p703]

— defun pfIdSymbol —

```
(defun |pfIdSymbol| (form)
  (|tokPart| form))
```

defun Construct a Leaf node

[tokConstruct p701]

[ifcar p??]

[pfNoPosition p704]

— defun pfLeaf —

```
(defun |pfLeaf| (x y &rest z)
  (|tokConstruct| x y (or (ifcar z) (|pfNoPosition|))))
```

defun Is this a leaf node?

[pfAbSynOp p702]

— defun pfLeaf? —

```
(defun |pfLeaf?| (form)
  (member (|pfAbSynOp| form)
    '(|id| |idsy| |symbol| |string| |char| |float| |expression|
      |integer| |Document| |error|)))
```

defun Return the token position of a leaf node

[tokPosn p703]

— defun pfLeafPosition —

```
(defun |pfLeafPosition| (form)
  (|tokPosn| form))
```

—————

defun Return the Leaf Token

[tokPart p703]

— defun pfLeafToken —

```
(defun |pfLeafToken| (form)
  (|tokPart| form))
```

—————

defun Is this a Literal node?

[pfAbSynOp p702]

— defun pfLiteral? 0 —

```
(defun |pfLiteral?!| (form)
  (member (|pfAbSynOp| form)
    '(|integer| |symbol| |expression| |one| |zero| |char| |string| |float|)))
```

—————

defun Create a LiteralClass node

[pfAbSynOp p702]

— defun pfLiteralClass —

```
(defun |pfLiteralClass| (form)
  (|pfAbSynOp| form))
```

—————

defun Return the LiteralString

[tokPart p703]

— defun pfLiteralString —

```
(defun |pfLiteralString| (form)
  (|tokPart| form))
```

—————

defun Return the parts of a tree node

— defun pfParts 0 —

```
(defun |pfParts| (form)
  (cdr form))
```

—————

defun Return the argument unchanged

— defun pfPile 0 —

```
(defun |pfPile| (part)
  part)
```

—————

defun pfPushBody

[pfLambda p571]

[pfNothing p543]

[pfPushBody p547]

— defun pfPushBody —

```
(defun |pfPushBody| (rt args body)
  (cond
    ((null args) body)
```

```

((null (cdr args)) (|pfLambda| (car args) rt body))
(t
  (|pfLambda| (car args) (|pfNothing|)
    (|pfPushBody| rt (cdr args) body))))

```

defun An S-expression which people can read.

[pfSexpr,strip p548]

— defun pfSexpr —

```

(defun |pfSexpr| (pform)
  (|pfSexpr,strip| pform))

```

defun Create a human readable S-expression

```

[pfId? p544]
[pfIdSymbol p545]
[pfLiteral? p546]
[pfLiteralString p547]
[pfLeaf? p545]
[tokPart p703]
[pfApplication? p553]
[pfApplicationArg p552]
[pfTuple? p590]
[pf0TupleParts p591]
[pfApplicationOp p552]
[pfSexpr,strip p548]
[pfAbSynOp p702]
[pfParts p547]

```

— defun pfSexpr,strip —

```

(defun |pfSexpr,strip| (pform)
  (let (args a result)
    (cond
      ((|pfId?| pform)      (|pfIdSymbol| pform))
      ((|pfLiteral?| pform) (|pfLiteralString| pform))
      ((|pfLeaf?| pform)    (|tokPart| pform))
      ((|pfApplication?| pform)

```

```

(setq a (|pfApplicationArg| pform))
(if (|pfTuple?| a)
    (setq args (|pf0TupleParts| a))
    (setq args (list a)))
(dolist (p (cons (|pfApplicationOp| pform) args) (nreverse result))
  (push (|pfSexpr,strip| p) result)))
(t
  (cons (|pfAbSynOp| pform)
    (dolist (p (|pfParts| pform) (nreverse result))
      (push (|pfSexpr,strip| p) result))))))

```

defun Construct a Symbol or Expression node

```

[|pfLeaf?| p545]
[|pfSymbol| p549]
[|tokPart| p703]
[|ifcar| p??]
[|pfExpression| p562]
[|pfSexpr| p548]

— defun pfSymb —

(defun |pfSymb| (expr &REST optpos)
  (if (|pfLeaf?| expr)
      (|pfSymbol| (|tokPart| expr) (ifcar optpos))
      (|pfExpression| (|pfSexpr| expr) (ifcar optpos))))

```

defun Construct a Symbol leaf node

```

[|pfLeaf| p545]
[|ifcar| p??]

— defun pfSymbol —

(defun |pfSymbol| (expr &rest optpos)
  (|pfLeaf| '|symbol| expr (ifcar optpos)))

```

defun Is this a Symbol node?

[pfAbSynOp? p702]

— defun pfSymbol? —

```
(defun |pfSymbol?| (form)
  (|pfAbSynOp?| form '|symbol|))
```

—————

defun Return the Symbol part

[tokPart p703]

— defun pfSymbolSymbol —

```
(defun |pfSymbolSymbol| (form)
  (|tokPart| form))
```

—————

15.5 Trees**defun Construct a tree node**

— defun pfTree 0 —

```
(defun |pfTree| (x y)
  (cons x y))
```

—————

defun Construct an Add node

[pfNothing p543]

[pfTree p550]

— defun pfAdd —

```
(defun |pfAdd| (pfbase pfaddin &rest addon)
  (let (lhs)
    (if addon
      (setq lhs addon)
      (setq lhs (|pfNothing|)))
    (|pfTree| '|Add| (list pfbase pfaddin lhs))))
```

defun Construct an And node

[pfTree p550]

— defun pfAnd —

```
(defun |pfAnd| (pflleft pfright)
  (|pfTree| '|And| (list pflleft pfright)))
```

defun pfAttribute

[pfTree p550]

— defun pfAttribute —

```
(defun |pfAttribute| (pfexpr)
  (|pfTree| '|Attribute| (list pfexpr)))
```

defun Return an Application node

[pfTree p550]

— defun pfApplication —

```
(defun |pfApplication| (pfop pfarg)
  (|pfTree| '|Application| (list pfop pfarg)))
```

defun Return the Arg part of an Application node

— defun pfApplicationArg 0 —

```
(defun |pfApplicationArg| (pf)
  (caddr pf))
```

—————

defun Return the Op part of an Application node

— defun pfApplicationOp 0 —

```
(defun |pfApplicationOp| (pf)
  (cadr pf))
```

—————

defun Is this an And node?

[pfAbSynOp? p⁷⁰²]

— defun pfAnd? —

```
(defun |pfAnd?| (pf)
  (|pfAbSynOp?| pf '|And|))
```

—————

defun Return the Left part of an And node

— defun pfAndLeft 0 —

```
(defun |pfAndLeft| (pf)
  (cadr pf))
```

—————

defun Return the Right part of an And node

— defun pfAndRight 0 —

```
(defun |pfAndRight| (pf)
  (caddr pf))
```

—————

defun Flatten a list of lists

— defun pfAppend 0 —

```
(defun |pfAppend| (list)
  (apply #'append list))
```

—————

defun Is this an Application node?

[pfAbSynOp? p702]

— defun pfApplication? —

```
(defun |pfApplication?| (pf)
  (|pfAbSynOp?| pf '|Application|))
```

—————

defun Create an Assign node

[pfTree p550]

— defun pfAssign —

```
(defun |pfAssign| (pflhsitems pfrhs)
  (|pfTree| '|Assign| (list pflhsitems pfrhs)))
```

—————

defun Is this an Assign node?

[pfAbSynOp? p702]

— defun pfAssign? —

```
(defun |pfAssign?| (pf)
  (|pfAbSynOp?| pf '|Assign|))
```

—————

defun Return the parts of an LhsItem of an Assign node

[pfParts p547]

[pfAssignLhsItems p554]

— defun pf0AssignLhsItems 0 —

```
(defun |pf0AssignLhsItems| (pf)
  (|pfParts| (|pfAssignLhsItems| pf)))
```

—————

defun Return the LhsItem of an Assign node

— defun pfAssignLhsItems 0 —

```
(defun |pfAssignLhsItems| (pf)
  (cadr pf))
```

—————

defun Return the RHS of an Assign node

— defun pfAssignRhs 0 —

```
(defun |pfAssignRhs| (pf)
  (caddr pf))
```

—————

defun Construct an application node for a brace

```
[pfApplication p551]
[pfIdPos p544]
[tokPosn p703]
```

— defun pfBrace —

```
(defun |pfBrace| (a part)
  (|pfApplication| (|pfIdPos| '{' (|tokPosn| a)) part))
```

—————

defun Construct an Application node for brace-bars

```
[pfApplication p551]
[pfIdPos p544]
[tokPosn p703]
```

— defun pfBraceBar —

```
(defun |pfBraceBar| (a part)
  (|pfApplication| (|pfIdPos| '{\|}|' (|tokPosn| a)) part))
```

—————

defun Construct an Application node for a bracket

```
[pfApplication p551]
[pfIdPos p544]
[tokPosn p703]
```

— defun pfBracket —

```
(defun |pfBracket| (a part)
  (|pfApplication| (|pfIdPos| '[' (|tokPosn| a)) part))
```

—————

defun Construct an Application node for bracket-bars

```
[pfApplication p551]
[pfIdPos p544]
```

[tokPosn p703]

— defun pfBracketBar —

```
(defun |pfBracketBar| (a part)
  (|pfApplication| (|pfIdPos| '|\| | (|tokPosn| a)) part))
```

—————

defun Create a Break node

[pfTree p550]

— defun pfBreak —

```
(defun |pfBreak| (pffrom)
  (|pfTree| '|Break| (list pffrom)))
```

—————

defun Is this a Break node?

[pfAbSynOp? p702]

— defun pfBreak? —

```
(defun |pfBreak?| (pf)
  (|pfAbSynOp?| pf '|Break|))
```

—————

defun Return the From part of a Break node

— defun pfBreakFrom 0 —

```
(defun |pfBreakFrom| (pf)
  (cadr pf))
```

—————

defun Construct a Coerceto node

[pfTree p550]

— defun pfCoerceto —

```
(defun |pfCoerceto| (pfexpr pftype)
  (|pfTree| '|Coerceto| (list pfexpr pftype)))
```

defun Is this a CoerceTo node?

[pfAbSynOp? p702]

— defun pfCoerceto? —

```
(defun |pfCoerceto?| (pf)
  (|pfAbSynOp?| pf '|Coerceto|))
```

defun Return the Expression part of a CoerceTo node

— defun pfCoercetoExpr 0 —

```
(defun |pfCoercetoExpr| (pf)
  (cadr pf))
```

defun Return the Type part of a CoerceTo node

— defun pfCoercetoType 0 —

```
(defun |pfCoercetoType| (pf)
  (caddr pf))
```

defun Return the Body of a Collect node

— defun pfCollectBody 0 —

```
(defun |pfCollectBody| (pf)
  (cadr pf))
```

—————

defun Return the Iterators of a Collect node

— defun pfCollectIterators 0 —

```
(defun |pfCollectIterators| (pf)
  (caddr pf))
```

—————

defun Create a Collect node

[pfTree p550]

— defun pfCollect —

```
(defun |pfCollect| (pfbody pfiterators)
  (|pfTree| '|Collect| (list pfbody pfiterators)))
```

—————

defun Is this a Collect node?

[pfAbSynOp? p702]

— defun pfCollect? —

```
(defun |pfCollect?| (pf)
  (|pfAbSynOp?| pf '|Collect|))
```

—————

defun pfDefinition

[pfTree p550]

— defun pfDefinition —

```
(defun |pfDefinition| (pflhsitems pfrhs)
  (|pfTree| '|Definition| (list pflhsitems pfrhs)))
```

—————

defun Return the Lhs of a Definition node

— defun pfDefinitionLhsItems 0 —

```
(defun |pfDefinitionLhsItems| (pf)
  (cadr pf))
```

—————

defun Return the Rhs of a Definition node

— defun pfDefinitionRhs 0 —

```
(defun |pfDefinitionRhs| (pf)
  (caddr pf))
```

—————

defun Is this a Definition node?

[pfAbSynOp? p702]

— defun pfDefinition? —

```
(defun |pfDefinition?| (pf)
  (|pfAbSynOp?| pf '|Definition|))
```

—————

defun Return the parts of a Definition node

```
[pfParts p547]
[pfDefinitionLhsItems p559]

— defun pf0DefinitionLhsItems —

(defun |pf0DefinitionLhsItems| (pf)
  (|pfParts| (|pfDefinitionLhsItems| pf)))
```

defun Create a Do node

```
[pfTree p550]

— defun pfDo —

(defun |pfDo| (pfbody)
  (|pfTree| '|Do| (list pfbody)))
```

defun Is this a Do node?

```
[pfAbSynOp? p702]

— defun pfDo? —

(defun |pfDo?| (pf)
  (|pfAbSynOp?| pf '|Do|))
```

defun Return the Body of a Do node

```
— defun pfDoBody 0 —

(defun |pfDoBody| (pf)
  (cadr pf))
```

defun Construct a Sequence node

[pfTuple p590]
 [pfListOf p543]
 [pfSequence p585]

— defun pfEnSequence —

```
(defun |pfEnSequence| (a)
  (cond
    ((null a) (|pfTuple| (|pfListOf| a)))
    ((null (cdr a)) (car a))
    (t (|pfSequence| (|pfListOf| a)))))
```

—————

defun Construct an Exit node

[pfTree p550]

— defun pfExit —

```
(defun |pfExit| (pfcond pfexpr)
  (|pfTree| '|Exit| (list pfcond pfexpr)))
```

—————

defun Is this an Exit node?

[pfAbSynOp? p702]

— defun pfExit? —

```
(defun |pfExit?| (pf)
  (|pfAbSynOp?| pf '|Exit|))
```

—————

defun Return the Cond part of an Exit

— defun pfExitCond 0 —


```
(defun |pfExitCond| (pf)
  (cadr pf))
```

defun Return the Expression part of an Exit

— defun pfExitExpr 0 —

```
(defun |pfExitExpr| (pf)
  (caddr pf))
```

defun Create an Export node

[pfTree p550]

— defun pfExport —

```
(defun |pfExport| (pfitems)
  (|pfTree| '|Export| (list pfitems)))
```

defun Construct an Expression leaf node

[pfLeaf p545]

[ifcar p??]

— defun pfExpression —

```
(defun |pfExpression| (expr &rest optpos)
  (|pfLeaf| '|expression| expr (ifcar optpos)))
```

defun pfFirst

— defun pfFirst 0 —

```
(defun |pfFirst| (form)
  (cadr form))
```

defun Create an Application Fix node

[pfApplication p551]
[pfId p544]

— defun pfFix —

```
(defun |pfFix| (pf)
  (|pfApplication| (|pfId| 'Y) pf))
```

defun Create a Free node

[pfTree p550]

— defun pfFree —

```
(defun |pfFree| (pfitems)
  (|pfTree| '|Free| (list pfitems)))
```

defun Is this a Free node?

[pfAbSynOp? p702]

— defun pfFree? —

```
(defun |pfFree?| (pf)
  (|pfAbSynOp?| pf '|Free|))
```

defun Return the parts of the Items of a Free node

```
[pfParts p547]
[pfFreeItems p564]

— defun pf0FreeItems —

(defun |pf0FreeItems| (pf)
  (|pfParts| (|pfFreeItems| pf)))
```

defun Return the Items of a Free node

```
— defun pfFreeItems 0 —

(defun |pfFreeItems| (pf)
  (cadr pf))
```

defun Construct a Forin node

```
[pfTree p550]

— defun pfForin —

(defun |pfForin| (pflhs pfwhole)
  (|pfTree| '|Forin| (list pflhs pfwhole)))
```

defun Is this a ForIn node?

```
[pfAbSynOp? p702]

— defun pfForin? —

(defun |pfForin?| (pf)
  (|pfAbSynOp?| pf '|Forin|))
```

defun Return all the parts of the LHS of a ForIn node

```
[pfParts p547]
[pfForinLhs p565]
```

— defun pf0ForinLhs —

```
(defun |pf0ForinLhs| (pf)
  (|pfParts| (|pfForinLhs| pf)))
```

—————

defun Return the LHS part of a ForIn node

— defun pfForinLhs 0 —

```
(defun |pfForinLhs| (pf)
  (cadr pf))
```

—————

defun Return the Whole part of a ForIn node

— defun pfForinWhole 0 —

```
(defun |pfForinWhole| (pf)
  (caddr pf))
```

—————

defun pfFromDom

```
[pfApplication? p553]
[pfApplication p551]
[pfApplicationOp p552]
[pfApplicationArg p552]
[pfFromdom p566]
```

— defun pfFromDom —

```
(defun |pfFromDom| (dom expr)
  (cond
    ((|pfApplication?| expr)
     (|pfApplication|
      (|pfFromDom| (|pfApplicationOp| expr) dom)
      (|pfApplicationArg| expr)))
    (t (|pfFromDom| expr dom))))
```

defun Construct a Fromdom node

[pfTree p550]

— defun pfFromdom —

```
(defun |pfFromdom| (pfwhat pfdomain)
  (|pfTree| '|Fromdom| (list pfwhat pfdomain)))
```

defun Is this a Fromdom mode?

[pfAbSynOp? p702]

— defun pfFromdom? —

```
(defun |pfFromdom?| (pf)
  (|pfAbSynOp?| pf '|Fromdom|))
```

defun Return the What part of a Fromdom node

— defun pfFromdomWhat 0 —

```
(defun |pfFromdomWhat| (pf)
  (cadr pf))
```

defun Return the Domain part of a Fromdom node

```

— defun pfFromdomDomain 0 —

(defun |pfFromdomDomain| (pf)
  (caddr pf))

```

defun Construct a Hide node

```

[|pfTree| p550]

— defun pfHide —

(defun |pfHide| (a part)
  (declare (ignore a))
  (|pfTree| '|Hide| (list part)))

```

defun pfIf

```

[|pfTree| p550]

— defun pfIf —

(defun |pfIf| (pfcond pfthen pfelse)
  (|pfTree| '|If| (list pfcond pfthen pfelse)))

```

defun Is this an If node?

```

[|pfAbSynOp?| p702]

— defun pfIf? —

(defun |pfIf?| (pf)
  (|pfAbSynOp?| pf '|If|))

```

defun Return the Cond part of an If

— defun pfIfCond 0 —

```
(defun |pfIfCond| (pf)
  (cadr pf))
```

—————

defun Return the Then part of an If

— defun pfIfThen 0 —

```
(defun |pfIfThen| (pf)
  (caddr pf))
```

—————

defun pfIfThenOnly

```
[pfIf p567]
[pfNothing p543]
```

— defun pfIfThenOnly —

```
(defun |pfIfThenOnly| (pred cararg)
  (|pfIf| pred cararg (|pfNothing|)))
```

—————

defun Return the Else part of an If

— defun pfIfElse 0 —

```
(defun |pfIfElse| (pf)
  (caddr pf))
```

—————

defun Construct an Import node

[pfTree p550]

— defun pfImport —

```
(defun |pfImport| (pfitems)
  (|pfTree| '|Import| (list pfitems)))
```

—————

defun Construct an Iterate node

[pfTree p550]

— defun pfIterate —

```
(defun |pfIterate| (pffrom)
  (|pfTree| '|Iterate| (list pffrom)))
```

—————

defun Is this an Iterate node?

[pfAbSynOp? p702]

— defun pfIterate? —

```
(defun |pfIterate?| (pf)
  (|pfAbSynOp?| pf '|Iterate|))
```

—————

defun Handle an infix application

[pfListOf p543]

[pfIdSymbol p545]

[pfAnd p551]

[pfOr p578]

[pfApplication p551]

[pfTuple p590]

— defun pfInfApplication —


```
(defun |pfInfApplication| (op left right)
  (cond
    ((eq (|pfIdSymbol| op) '|and|) (|pfAnd| left right))
    ((eq (|pfIdSymbol| op) '|or|) (|pfOr| left right))
    (t (|pfApplication| op (|pfTuple| (|pfListOf| (list left right)))))))
```

defun Create an Inline node

[pfTree p550]

— defun pfInline —

```
(defun |pfInline| (pfitems)
  (|pfTree| '|Inline| (list pfitems)))
```

defun pfLam

[pfAbSynOp? p702]
 [pfFirst p562]
 [pfNothing p543]
 [pfSecond p584]
 [pfLambda p571]

— defun pfLam —

```
(defun |pfLam| (variable body)
  (let (bdy rets)
    (if (|pfAbSynOp?| body '|returntyped|)
        (setq rets (|pfFirst| body))
        (setq rets (|pfNothing|)))
    (if (|pfAbSynOp?| body '|returntyped|)
        (setq bdy (|pfSecond| body))
        (setq bdy body))
    (|pfLambda| variable rets bdy)))
```

defun pfLambda

[pfTree p550]

— defun pfLambda —

```
(defun |pfLambda| (pfargs pfrets pfbody)
  (|pfTree| '|Lambda| (list pfargs pfrets pfbody)))
```

—————

defun Return the Body part of a Lambda node

— defun pfLambdaBody 0 —

```
(defun |pfLambdaBody| (pf)
  (caddr pf))
```

—————

defun Return the Rets part of a Lambda node

— defun pfLambdaRets 0 —

```
(defun |pfLambdaRets| (pf)
  (caddr pf))
```

—————

defun Is this a Lambda node?

[pfAbSynOp? p702]

— defun pfLambda? —

```
(defun |pfLambda?| (pf)
  (|pfAbSynOp?| pf '|Lambda|))
```

—————

defun Return the Args part of a Lambda node

```

— defun pfLambdaArgs 0 —

(defun |pfLambdaArgs| (pf)
  (cadr pf))

```

defun Return the Args of a Lambda Node

```

[|pfParts| p547]
[|pfLambdaArgs| p572]

— defun pf0LambdaArgs —

(defun |pf0LambdaArgs| (pf)
  (|pfParts| (|pfLambdaArgs| pf)))

```

defun Construct a Local node

```

[|pfTree| p550]

— defun pfLocal —

(defun |pfLocal| (pfitems)
  (|pfTree| '|Local| (list pfitems)))

```

defun Is this a Local node?

```

[|pfAbSynOp?| p702]

— defun pfLocal? —

(defun |pfLocal?| (pf)
  (|pfAbSynOp?| pf '|Local|))

```

defun Return the parts of Items of a Local node

```
[pfParts p547]
[pfLocalItems p573]

— defun pf0LocalItems —

(defun |pf0LocalItems| (pf)
  (|pfParts| (|pfLocalItems| pf)))
```

defun Return the Items of a Local node

```
— defun pfLocalItems 0 —

(defun |pfLocalItems| (pf)
  (cadr pf))
```

defun Construct a Loop node

```
[pfTree p550]

— defun pfLoop —

(defun |pfLoop| (pfiterators)
  (|pfTree| '|Loop| (list pfiterators)))
```

defun pfLoop1

```
[pfLoop p573]
[pfListOf p543]
[pfDo p560]

— defun pfLoop1 —

(defun |pfLoop1| (body)
  (|pfLoop| (|pfListOf| (list (|pfDo| body))))))
```

defun Is this a Loop node?

[pfAbSynOp? p702]

— defun pfLoop? —

```
(defun |pfLoop?| (pf)
  (|pfAbSynOp?| pf ' |Loop|))
```

defun Return the Iterators of a Loop node

— defun pfLoopIterators 0 —

```
(defun |pfLoopIterators| (pf)
  (cadr pf))
```

defun pf0LoopIterators

[pfParts p547]

[pf0LoopIterators p574]

— defun pf0LoopIterators —

```
(defun |pf0LoopIterators| (pf)
  (|pfParts| (|pfLoopIterators| pf)))
```

defun pfLp

[pfLoop p573]

[pfListOf p543]

[pfDo p560]

— defun pfLp —

```
(defun |pfLp| (iterators body)
  (|pfLoop| (|pfListOf| (append iterators (list (|pfDo| body))))))
```

defun Create a Macro node

[pfTree p550]

— defun pfMacro —

```
(defun |pfMacro| (pflhs pfrhs)
  (|pfTree| '|Macro| (list pflhs pfrhs)))
```

defun Is this a Macro node?

[pfAbSynOp? p702]

— defun pfMacro? —

```
(defun |pfMacro?| (pf)
  (|pfAbSynOp?| pf '|Macro|))
```

defun Return the Lhs of a Macro node

— defun pfMacroLhs 0 —

```
(defun |pfMacroLhs| (pf)
  (cadr pf))
```

defun Return the Rhs of a Macro node

— defun pfMacroRhs 0 —

```
(defun |pfMacroRhs| (pf)
  (caddr pf))
```

—————

defun Construct an MLambda node

[pfTree p550]

— defun pfMLambda —

```
(defun |pfMLambda| (pfargs pbody)
  (|pfTree| 'MLambda (list pfargs pbody)))
```

—————

defun Is this an MLambda node?

[pfAbSynOp? p702]

— defun pfMLambda? —

```
(defun |pfMLambda?| (pf)
  (|pfAbSynOp?| pf 'MLambda))
```

—————

defun Return the Args of an MLambda

— defun pfMLambdaArgs 0 —

```
(defun |pfMLambdaArgs| (pf)
  (cadr pf))
```

—————

defun Return the parts of an MLambda argument

[pfParts p547]

— defun pf0MLambdaArgs —

```
(defun |pf0MLambdaArgs| (pf)
  (|pfParts| (|pfMLambdaArgs| pf)))
```

defun pfMLambdaBody

— defun pfMLambdaBody 0 —

```
(defun |pfMLambdaBody| (pf)
  (caddr pf))
```

defun Is this a Not node?

[pfAbSynOp? p702]

— defun pfNot? —

```
(defun |pfNot?| (pf)
  (|pfAbSynOp?| pf '|Not|))
```

defun Return the Arg part of a Not node

— defun pfNotArg 0 —

```
(defun |pfNotArg| (pf)
  (cadr pf))
```

defun Construct a NoValue node

[pfTree p550]

— defun pfNovalue —


```
(defun |pfNovalue| (pfexpr)
  (|pfTree| '|Novalue| (list pfexpr)))
```

—————→

defun Is this a Novalue node?

[pfAbSynOp? p702]

— defun pfNovalue? —

```
(defun |pfNovalue?| (pf)
  (|pfAbSynOp?| pf '|Novalue|))
```

—————→

defun Return the Expr part of a Novalue node

— defun pfNovalueExpr 0 —

```
(defun |pfNovalueExpr| (pf)
  (cadr pf))
```

—————→

defun Construct an Or node

[pfTree p550]

— defun pfOr —

```
(defun |pfOr| (pfleft pfright)
  (|pfTree| '|Or| (list pfleft pfright)))
```

—————→

defun Is this an Or node?

[pfAbSynOp? p702]

— defun pfOr? —

```
(defun |pfOr?| (pf)
  (|pfAbSynOp?| pf '|Or|))
```

defun Return the Left part of an Or node

— defun pfOrLeft 0 —

```
(defun |pfOrLeft| (pf)
  (cadr pf))
```

defun Return the Right part of an Or node

— defun pfOrRight 0 —

```
(defun |pfOrRight| (pf)
  (caddr pf))
```

defun Return the part of a parenthesised expression

— defun pfParen —

```
(defun |pfParen| (a part)
  (declare (ignore a))
  part)
```

defun pfPretend

[pfTree p550]

— defun pfPretend —

```
(defun |pfPretend| (pfexpr pftype)
  (|pfTree| '|Pretend| (list pfexpr pftype)))
```

defun Is this a Pretend node?

[pfAbSynOp? p702]

— defun pfPretend? —

```
(defun |pfPretend?| (pf)
  (|pfAbSynOp?| pf '|Pretend|))
```

defun Return the Expression part of a Pretend node

— defun pfPretendExpr 0 —

```
(defun |pfPretendExpr| (pf)
  (cadr pf))
```

defun Return the Type part of a Pretend node

— defun pfPretendType 0 —

```
(defun |pfPretendType| (pf)
  (caddr pf))
```

defun Construct a QualType node

[pfTree p550]

— defun pfQualType —

```
(defun |pfQualType| (pftype pfqual)
  (|pfTree| '|QualType| (list pftype pfqual)))
```

defun Construct a Restrict node

[pfTree p550]

```
— defun pfRestrict —

(defun |pfRestrict| (pfexpr pftype)
  (|pfTree| '|Restrict| (list pfexpr pftype)))
```

defun Is this a Restrict node?

[pfAbSynOp? p702]

```
— defun pfRestrict? —

(defun |pfRestrict?| (pf)
  (|pfAbSynOp?| pf '|Restrict|))
```

defun Return the Expr part of a Restrict node

```
— defun pfRestrictExpr 0 —

(defun |pfRestrictExpr| (pf)
  (cadr pf))
```

defun Return the Type part of a Restrict node

```
— defun pfRestrictType 0 —
```

```
(defun |pfRestrictType| (pf)
  (caddr pf))
```

defun Construct a RetractTo node

[pfTree p550]

— defun pfRetractTo —

```
(defun |pfRetractTo| (pfexpr pftype)
  (|pfTree| '|RetractTo| (list pfexpr pftype)))
```

defun Construct a Return node

[pfTree p550]

— defun pfReturn —

```
(defun |pfReturn| (pfexpr pffrom)
  (|pfTree| '|Return| (list pfexpr pffrom)))
```

defun Is this a Return node?

[pfAbSynOp? p702]

— defun pfReturn? —

```
(defun |pfReturn?| (pf)
  (|pfAbSynOp?| pf '|Return|))
```

defun Return the Expr part of a Return node

— defun pfReturnExpr 0 —

```
(defun |pfReturnExpr| (pf)
  (cadr pf))
```

defun pfReturnNoName

```
[pfReturn p582]
[pfNothing p543]
```

— defun pfReturnNoName —

```
(defun |pfReturnNoName| (|value|)
  (|pfReturn| |value| (|pfNothing|)))
```

defun Construct a ReturnTyped node

```
[pfTree p550]
```

— defun pfReturnTyped —

```
(defun |pfReturnTyped| (type body)
  (|pfTree| '|returntyped| (list type body)))
```

defun Construct a Rule node

```
[pfTree p550]
```

— defun pfRule —

```
(defun |pfRule| (pflhsitems pfrhs)
  (|pfTree| '|Rule| (list pflhsitems pfrhs)))
```

defun Return the Lhs of a Rule node

```

— defun pfRuleLhsItems 0 —

(defun |pfRuleLhsItems| (pf)
  (cadr pf))

```

defun Return the Rhs of a Rule node

```

— defun pfRuleRhs 0 —

(defun |pfRuleRhs| (pf)
  (caddr pf))

```

defun Is this a Rule node?

```

[|pfAbSynOp?| p702]

— defun pfRule? —

(defun |pfRule?| (pf)
  (|pfAbSynOp?| pf '|Rule|))

```

defun pfSecond

```

— defun pfSecond 0 —

(defun |pfSecond| (form)
  (caddr form))

```

defun Construct a Sequence node

[pfTree p550]

— defun pfSequence —

```
(defun |pfSequence| (pfargs)
  (|pfTree| ' |Sequence| (list pfargs)))
```

—————

defun Return the Args of a Sequence node

— defun pfSequenceArgs 0 —

```
(defun |pfSequenceArgs| (pf)
  (cadr pf))
```

—————

defun Is this a Sequence node?

[pfAbSynOp? p702]

— defun pfSequence? —

```
(defun |pfSequence?| (pf)
  (|pfAbSynOp?| pf ' |Sequence|))
```

—————

defun Return the parts of the Args of a Sequence node

[pfParts p547]

[pfSequenceArgs p585]

— defun pf0SequenceArgs —

```
(defun |pf0SequenceArgs| (pf)
  (|pfParts| (|pfSequenceArgs| pf)))
```

—————

defun Create a Suchthat node

[pfTree p550]

— defun pfSuchthat —

```
(defun |pfSuchthat| (pfcond)
  (|pfTree| '|Suchthat| (list pfcond)))
```

—————

defun Is this a SuchThat node?

[pfAbSynOp? p702]

— defun pfSuchthat? —

```
(defun |pfSuchthat?| (pf)
  (|pfAbSynOp?| pf '|Suchthat|))
```

—————

defun Return the Cond part of a SuchThat node

— defun pfSuchthatCond 0 —

```
(defun |pfSuchthatCond| (pf)
  (cadr pf))
```

—————

defun Create a Tagged node

[pfTree p550]

— defun pfTagged —

```
(defun |pfTagged| (pftag pfexpr)
  (|pfTree| '|Tagged| (list pftag pfexpr)))
```

—————

defun Is this a Tagged node?

[pfAbSynOp? p702]

— defun pfTagged? —

```
(defun |pfTagged?| (pf)
  (|pfAbSynOp?| pf ' |Tagged|))
```

—————

defun Return the Expression portion of a Tagged node

— defun pfTaggedExpr 0 —

```
(defun |pfTaggedExpr| (pf)
  (caddr pf))
```

—————

defun Return the Tag of a Tagged node

— defun pfTaggedTag 0 —

```
(defun |pfTaggedTag| (pf)
  (cadr pf))
```

—————

defun pfTaggedToTyped

```
[pfTagged? p587]
[pfTaggedExpr p587]
[pfNothing p543]
[pfTaggedTag p587]
[pfId? p544]
[pfId p544]
[pfTyped p588]
[pfSuch p542]
[pfInfApplication p569]
```

— defun pfTaggedToTyped —

```
(defun |pfTaggedToTyped| (arg)
  (let (a form rt)
    (if (|pfTagged?| arg)
        (setq rt (|pfTaggedExpr| arg))
        (setq rt (|pfNothing|)))
    (if (|pfTagged?| arg)
        (setq form (|pfTaggedTag| arg))
        (setq form arg))
    (cond
     ((null (|pfId?| form))
      (setq a (|pfId| (gensym)))
      (|pfTyped| (|pfSuch| a (|pfInfApplication| (|pfId| '=) a form)) rt))
     (t (|pfTyped| form rt)))))
```

—————

defun pfTweakIf

[pfIfElse p568]
 [pfNothing? p543]
 [pfListOf p543]
 [pfTree p550]
 [pfIfCond p568]
 [pfIfThen p568]

— defun pfTweakIf —

```
(defun |pfTweakIf| (form)
  (let (b a)
    (setq a (|pfIfElse| form))
    (setq b (if (|pfNothing?| a) (|pfListOf| NIL) a))
    (|pfTree| '|WIf| (list (|pfIfCond| form) (|pfIfThen| form) b))))
```

—————

defun Construct a Typed node

[pfTree p550]

— defun pfTyped —

```
(defun |pfTyped| (pfid pftype)
```

```
(|pfTree| ' |Typed| (list pfid pftype)))
```

defun Is this a Typed node?

```
[pfAbSynOp? p702]
```

```
— defun pfTyped? —
```

```
(defun |pfTyped?| (pf)
  (|pfAbSynOp?| pf ' |Typed|))
```

defun Return the Type of a Typed node

```
— defun pfTypedType 0 —
```

```
(defun |pfTypedType| (pf)
  (caddr pf))
```

defun Return the Id of a Typed node

```
— defun pfTypedId 0 —
```

```
(defun |pfTypedId| (pf)
  (cadr pf))
```

defun Construct a Typing node

```
[pfTree p550]
```

```
— defun pfTyping —
```

```
(defun |pfTyping| (pfitems)
  (|pfTree| '|Typing| (list pfitems)))
```

defun Return a Tuple node

[pfTree p550]

— defun pfTuple —

```
(defun |pfTuple| (pfparts)
  (|pfTree| '|Tuple| (list pfparts)))
```

defun Return a Tuple from a List

[pfTuple p590]

[pfListOf p543]

— defun pfTupleListOf —

```
(defun |pfTupleListOf| (pfparts)
  (|pfTuple| (|pfListOf| pfparts)))
```

defun Is this a Tuple node?

[pfAbSynOp? p702]

— defun pfTuple? —

```
(defun |pfTuple?| (pf)
  (|pfAbSynOp?| pf '|Tuple|))
```

defun Return the Parts of a Tuple node

— defun pfTupleParts 0 —

```
(defun |pfTupleParts| (pf)
  (cadr pf))
```

—————

defun Return the parts of a Tuple

```
[pfParts p547]
[pfTupleParts p591]
```

— defun pf0TupleParts —

```
(defun |pf0TupleParts| (pf)
  (|pfParts| (|pfTupleParts| pf)))
```

—————

defun Return a list from a Sequence node

```
[pfSequence? p585]
[pfAppend p553]
[pf0SequenceArgs p585]
[pfListOf p543]
```

— defun pfUnSequence —

```
(defun |pfUnSequence| (x)
  (if (|pfSequence?| x)
      (|pfListOf| (|pfAppend| (|pf0SequenceArgs| x)))
      (|pfListOf| x)))
```

—————

defun The comment is attached to all signatutres

```
[pfWDeclare p592]
[pfParts p547]
```

— defun pfWDec —

```
(defun |pfWDec| (doc name)
  (mapcar #'(lambda (i) (|pfWDeclare| i doc)) (|pfParts| name)))
```

—————→

defun Construct a WDeclare node

[pfTree p550]

— defun pfWDeclare —

```
(defun |pfWDeclare| (pfsignature pfdoc)
  (|pfTree| '|WDeclare| (list pfsignature pfdoc)))
```

—————→

defun Construct a Where node

[pfTree p550]

— defun pfWhere —

```
(defun |pfWhere| (pfcontext pfexpr)
  (|pfTree| '|Where| (list pfcontext pfexpr)))
```

—————→

defun Is this a Where node?

[pfAbSynOp? p702]

— defun pfWhere? —

```
(defun |pfWhere?| (pf)
  (|pfAbSynOp?| pf '|Where|))
```

—————→

defun Return the parts of the Context of a Where node

```
[pfParts p547]
[pfWhereContext p593]

— defun pf0WhereContext —

(defun |pf0WhereContext| (pf)
  (|pfParts| (|pfWhereContext| pf)))
```

defun Return the Context of a Where node

```
— defun pfWhereContext 0 —

(defun |pfWhereContext| (pf)
  (cadr pf))
```

defun Return the Expr part of a Where node

```
— defun pfWhereExpr 0 —

(defun |pfWhereExpr| (pf)
  (caddr pf))
```

defun Construct a While node

```
[pfTree p550]

— defun pfWhile —

(defun |pfWhile| (pfcond)
  (|pfTree| '|While| (list pfcond)))
```

defun Is this a While node?

[pfAbSynOp? p702]

— defun pfWhile? —

```
(defun |pfWhile?| (pf)
  (|pfAbSynOp?| pf '|While|))
```

—————

defun Return the Cond part of a While node

— defun pfWhileCond 0 —

```
(defun |pfWhileCond| (pf)
  (cadr pf))
```

—————

defun Construct a With node

[pfTree p550]

— defun pfWith —

```
(defun |pfWith| (pfbase pfwithin pfwithon)
  (|pfTree| '|With| (list pfbase pfwithin pfwithon)))
```

—————

defun Create a Wrong node

[pfTree p550]

— defun pfWrong —

```
(defun |pfWrong| (pfwhy pfrubble)
  (|pfTree| '|Wrong| (list pfwhy pfrubble)))
```

—————

defun Is this a Wrong node?

[pfAbSynOp? p702]

— defun pfWrong? —

```
(defun |pfWrong?| (pf)
  (|pfAbSynOp?| pf 'Wrong!))
```

—————

Chapter 16

Pftree to s-expression translation

Pftree to s-expression translation. Used to interface the new parser technology to the interpreter. The input is a parseTree and the output is an old-parser-style s-expression.

defun Pftree to s-expression translation

```
[pf2Sex1 p598]
[$insideSEQ p??]
[$insideApplication p??]
[$insideRule p??]
[$QuietCommand p339]
```

— defun pf2Sex —

```
(defun |pf2Sex| (pf)
  (let (|$insideSEQ| |$insideApplication| |$insideRule|)
    (declare (special |$insideSEQ| |$insideApplication| |$insideRule|
                      |$QuietCommand|))
    (setq |$QuietCommand| nil)
    (setq |$insideRule| nil)
    (setq |$insideApplication| nil)
    (setq |$insideSEQ| nil)
    (|pf2Sex1| pf)))
```

—————

defun Pftree to s-expression translation inner function

```

[pfNothing? p543]
[pfSymbol? p550]
[pfSymbolSymbol p550]
[pfLiteral? p546]
[pfLiteral2Sex p602]
[pfIdSymbol p545]
[pfApplication? p553]
[pfApplication2Sex p603]
[pfTuple? p590]
[pf2Sex1 p598]
[pf0TupleParts p591]
[pfIf? p567]
[pfIfCond p568]
[pfIfThen p568]
[pfIfElse p568]
[pfTagged? p587]
[pfTaggedTag p587]
[pfTaggedExpr p587]
[pfCoerceto? p557]
[pfCoercetoExpr p557]
[pfCoercetoType p557]
[pfPretend? p580]
[pfPretendExpr p580]
[pfPretendType p580]
[pfFromdom? p566]
[opTran p621]
[pfFromdomWhat p566]
[pfFromdomDomain p567]
[pfSequence? p585]
[pfSequence2Sex p608]
[pfExit? p561]
[pfExitCond p561]
[pfExitExpr p562]
[pfLoop? p574]
[loopIters2Sex p609]
[pf0LoopIterators p574]
[pfCollect? p558]
[pfCollect2Sex p612]
[pfForin? p564]
[pf0ForinLhs p565]
[pfForinWhole p565]
[pfWhile? p594]
[pfWhileCond p594]
[pfSuchthat? p586]

```

[keyedSystemError p??]
 [pfSuchthatCond p586]
 [pfDo? p560]
 [pfDoBody p560]
 [pfTyped? p589]
 [pfTypedType p589]
 [pfTypedId p589]
 [pfAssign? p554]
 [pf0AssignLhsItems p554]
 [pfAssignRhs p554]
 [pfDefinition? p559]
 [pfDefinition2Sex p613]
 [pfLambda? p571]
 [pfLambda2Sex p616]
 [pfMLambda? p576]
 [pfRestrict? p581]
 [pfRestrictExpr p581]
 [pfRestrictType p581]
 [pfFree? p563]
 [pf0FreeItems p564]
 [pfLocal? p572]
 [pf0LocalItems p573]
 [pfWrong? p595]
 [spadThrow p??]
 [pfAnd? p552]
 [pfAndLeft p552]
 [pfAndRight p553]
 [pfOr? p578]
 [pfOrLeft p579]
 [pfOrRight p579]
 [pfNot? p577]
 [pfNotArg p577]
 [pfNovalue? p578]
 [pfNovalueExpr p578]
 [pfRule? p584]
 [pfRule2Sex p616]
 [pfBreak? p556]
 [pfBreakFrom p556]
 [pfMacro? p575]
 [pfReturn? p582]
 [pfReturnExpr p582]
 [pfIterate? p569]
 [pfWhere? p592]
 [pf0WhereContext p593]
 [pfWhereExpr p593]
 [pfAbSynOp p702]

[tokPart p703]
 [\$insideSEQ p??]
 [\$insideRule p??]
 [\$QuietCommand p339]

— defun pf2Sex1 —

```
(defun |pf2Sex1| (pf)
  (let (args idList type op tagPart tag s)
    (declare (special |$insideSEQ| |$insideRule| |$QuietCommand|))
    (cond
      ((|pfNothing?| pf) '|noBranch|)
      ((|pfSymbol?| pf)
        (if (eq |$insideRule| '|left|)
          (progn
            (setq s (|pfSymbolSymbol| pf))
            (list '|constant| (list 'quote s)))
          (list 'quote (|pfSymbolSymbol| pf))))
      ((|pfLiteral?| pf) (|pfLiteral2Sex| pf))
      ((|pfId?| pf)
        (if |$insideRule|
          (progn
            (setq s (|pfIdSymbol| pf))
            (if (member s '(|%pi| |%e| |%i|))
              s
              (list 'quote s)))
            (|pfIdSymbol| pf))))
      ((|pfApplication?| pf) (|pfApplication2Sex| pf))
      ((|pfTuple?| pf) (cons '|Tuple| (mapcar #'|pf2Sex1| (|pf0TupleParts| pf))))
      ((|pfIf?| pf)
        (list 'if (|pf2Sex1| (|pfIfCond| pf))
              (|pf2Sex1| (|pfIfThen| pf))
              (|pf2Sex1| (|pfIfElse| pf))))
      ((|pfTagged?| pf)
        (setq tag (|pfTaggedTag| pf))
        (setq tagPart
          (if (|pfTuple?| tag)
            (cons '|Tuple| (mapcar #'|pf2Sex1| (|pf0TupleParts| tag)))
            (|pf2Sex1| tag)))
        (list '|:| tagPart (|pf2Sex1| (|pfTaggedExpr| pf))))
      ((|pfCoerceto?| pf)
        (list '|::| (|pf2Sex1| (|pfCoercetoExpr| pf))
              (|pf2Sex1| (|pfCoercetoType| pf))))
      ((|pfPretend?| pf)
        (list '|pretend| (|pf2Sex1| (|pfPretendExpr| pf))
              (|pf2Sex1| (|pfPretendType| pf))))
      ((|pfFromdom?| pf)
        (setq op (|opTran| (|pf2Sex1| (|pfFromdomWhat| pf))))
        (when (eq op '|braceFromCurly|) (setq op 'seq))
```

```

(list '$elt| (|pf2Sex1| (|pfFromdomDomain| pf)) op))
((|pfSequence?| pf) (|pfSequence2Sex| pf))
((|pfExit?| pf)
 (if |$insideSEQ|
  (list 'exit| (|pf2Sex1| (|pfExitCond| pf))
        (|pf2Sex1| (|pfExitExpr| pf))))
  (list 'if (|pf2Sex1| (|pfExitCond| pf))
        (|pf2Sex1| (|pfExitExpr| pf)) 'noBranch|)))
((|pfLoop?| pf) (cons 'repeat (|loopIters2Sex| (|pf0LoopIterators| pf))))
((|pfCollect?| pf) (|pfCollect2Sex| pf))
((|pfForin?| pf)
 (cons 'in
  (append (mapcar #'|pf2Sex1| (|pf0ForinLhs| pf))
    (list (|pf2Sex1| (|pfForinWhole| pf))))))
((|pfWhile?| pf) (list 'while (|pf2Sex1| (|pfWhileCond| pf))))
((|pfSuchthat?| pf)
 (if (eq |$insideRule| '|left|)
  (|keyedSystemError| "Unexpected error in call to system function %1"
    (list "pf2Sex1: pfSuchThat"))
  (list '|\\| (|pf2Sex1| (|pfSuchthatCond| pf)))))
((|pfDo?| pf) (|pf2Sex1| (|pfDoBody| pf)))
((|pfTyped?| pf)
 (setq type (|pfTypedType| pf))
 (if (|pfNothing?| type)
  (|pf2Sex1| (|pfTypedId| pf))
  (list '|:| (|pf2Sex1| (|pfTypedId| pf)) (|pf2Sex1| (|pfTypedType| pf)))))
((|pfAssign?| pf)
 (setq idList (mapcar #'|pf2Sex1| (|pf0AssignLhsItems| pf)))
 (if (not (eql (length idList) 1))
  (setq idList (cons '|Tuple| idList))
  (setq idList (car idList)))
 (list 'let idList (|pf2Sex1| (|pfAssignRhs| pf)))))
((|pfDefinition?| pf) (|pfDefinition2Sex| pf))
((|pfLambda?| pf) (|pfLambda2Sex| pf))
((|pfMLambda?| pf) '|/throwAway|)
((|pfRestrict?| pf)
 (list '@ (|pf2Sex1| (|pfRestrictExpr| pf))
        (|pf2Sex1| (|pfRestrictType| pf))))
((|pfFree?| pf) (cons '|free| (mapcar #'|pf2Sex1| (|pf0FreeItems| pf))))
((|pfLocal?| pf) (cons '|local| (mapcar #'|pf2Sex1| (|pf0LocalItems| pf))))
((|pfWrong?| pf) (|spadThrow|))
((|pfAnd?| pf)
 (list '|and| (|pf2Sex1| (|pfAndLeft| pf))
        (|pf2Sex1| (|pfAndRight| pf))))
((|pfOr?| pf)
 (list '|or| (|pf2Sex1| (|pfOrLeft| pf))
        (|pf2Sex1| (|pfOrRight| pf))))
((|pfNot?| pf) (list '|not| (|pf2Sex1| (|pfNotArg| pf))))
((|pfNovalue?| pf)
 (setq |$QuietCommand| t)

```



```

(list 'seq (|pf2Sex1| (|pfNovalueExpr| pf))))
(|pfRule?| pf) (|pfRule2Sex| pf))
(|pfBreak?| pf) (list '|break| (|pfBreakFrom| pf)))
(|pfMacro?| pf) '|/throwAway|)
(|pfReturn?| pf) (list '|return| (|pf2Sex1| (|pfReturnExpr| pf))))
(|pfIterate?| pf) (list '|iterate|))
(|pfWhere?| pf)
  (setq args (mapcar #'|pf2Sex1| (|pf0WhereContext| pf)))
  (if (eql (length args) 1)
      (cons '|where| (cons (|pf2Sex1| (|pfWhereExpr| pf)) args))
      (list '|where| (|pf2Sex1| (|pfWhereExpr| pf)) (cons 'seq args))))
; -- under strange circumstances/piling, system commands can wind
; -- up in expressions. This just passes it through as a string for
; -- the user to figure out what happened.
((eq (|pfAbSynOp| pf) '|command|) (|tokPart| pf))
(t (|keyedSystemError| "Unexpected error in call to system function %1"
  (list "pf2Sex1")))))

```

defun Convert a Literal to an S-expression

```

[|pfLiteralClass| p546]
[|pfLiteralString| p547]
[|float2Sex| p603]
[|pfSymbolSymbol| p550]
[|pfLeafToken| p546]
[|keyedSystemError| p??]
[|$insideRule| p??]

```

— defun pfLiteral2Sex —

```

(defun |pfLiteral2Sex| (pf)
  (let (s type)
    (declare (special |$insideRule|))
    (setq type (|pfLiteralClass| pf))
    (cond
      ((eq type '|integer|) (read-from-string (|pfLiteralString| pf)))
      ((or (eq type '|string|) (eq type '|char|))
       (|pfLiteralString| pf))
      ((eq type '|float|) (|float2Sex| (|pfLiteralString| pf)))
      ((eq type '|symbol|)
       (if |$insideRule|
          (progn
            (setq s (|pfSymbolSymbol| pf))
            (list 'quote s))
          (|pfSymbolSymbol| pf))))

```

```
((eq type '|expression|) (list 'quote (|pfLeafToken| pf)))
(t
  (|keyedSystemError| "Unexpected error in call to system function %1"
    (list "pfLiteral2Sex: unexpected form"))))))
```

defun Convert a float to an S-expression

[[\\$useBFasDefault](#) p??]

— defun float2Sex —

```
(defun |float2Sex| (num)
  (let (exp frac bfForm fracPartString intPart dotIndex expPart mantPart eIndex)
    (declare (special |$useBFasDefault|))
    (setq eIndex (search "e" num))
    (if eIndex
      (setq mantPart (subseq num 0 eIndex))
      (setq mantPart num))
    (if eIndex
      (setq expPart (read-from-string (subseq num (+ eIndex 1))))
      (setq expPart 0))
    (setq dotIndex (search "." mantPart))
    (if dotIndex
      (setq intPart (read-from-string (subseq mantPart 0 dotIndex)))
      (setq intPart (read-from-string mantPart)))
    (if dotIndex
      (setq fracPartString (subseq mantPart (+ dotIndex 1)))
      (setq fracPartString 0))
    (setq bfForm
      (make-float intPart (read-from-string fracPartString)
                  (length fracPartString) expPart))
    (if |$useBFasDefault|
      (progn
        (setq frac (cadr bfForm))
        (setq exp (caddr bfForm))
        (list (list '|$elt| (list '|Float|) '|float|) frac exp 10))
      bfForm)))
```

defun Change an Application node to an S-expression

[[pfOp2Sex](#) p606]

[[pfApplicationOp](#) p552]

```

[opTran p621]
[pf0TupleParts p591]
[pfApplicationArg p552]
[pfTuple? p590]
[pf2Sex1 p598]
[pf2Sex p597]
[pfSuchThat2Sex p605]
[hasOptArgs? p607]
[$insideApplication p??]
[$insideRule p??]

```

— defun pfApplication2Sex —

```

(defun |pfApplication2Sex| (pf)
  (let (|$insideApplication| x val realOp tmp1 qt argSex typeList args op)
    (declare (special |$insideApplication| |$insideRule|))
    (setq |$insideApplication| t)
    (setq op (|pfOp2Sex| (|pfApplicationOp| pf)))
    (setq op (|opTran| op))
    (cond
      ((eq op '->)
        (setq args (|pf0TupleParts| (|pfApplicationArg| pf)))
        (if (|pfTuple?| (car args))
          (setq typeList (mapcar #'|pf2Sex1| (|pf0TupleParts| (car args))))
          (setq typeList (list (|pf2Sex1| (car args)))))
        (setq args (cons (|pf2Sex1| (cadr args)) typeList))
        (cons '|Mapping| args))
      ((and (eq op '|:|) (eq |$insideRule| '|left|))
        (list '|multiple| (|pf2Sex| (|pfApplicationArg| pf))))
      ((and (eq op '|?|) (eq |$insideRule| '|left|))
        (list '|optional| (|pf2Sex| (|pfApplicationArg| pf))))
      (t
        (setq args (|pfApplicationArg| pf))
        (cond
          ((|pfTuple?| args)
            (if (and (eq op '|\\|') (eq |$insideRule| '|left|))
              (|pfSuchThat2Sex| args)
              (progn
                (setq argSex (cdr (|pf2Sex1| args)))
                (cond
                  ((eq op '>') (list '<' (cadr argSex) (car argSex)))
                  ((eq op '>=') (list '|not|' (list '<' (car argSex) (cadr argSex))))
                  ((eq op '<=') (list '|not|' (list '<' (cadr argSex) (car argSex))))
                  ((eq op '&') (list '|and|' (car argSex) (cadr argSex)))
                  ((eq op '|or|') (list '|or|' (car argSex) (cadr argSex)))
                  ((eq op '|Iterate|') (list '|iterate|'))
                  ((eq op '|by|') (cons 'by argSex))
                  ((eq op '|braceFromCurly|')
                    (if (and (consp argSex) (eq (car argSex) 'seq))

```

```

      argSex
      (cons 'seq argSex)))
    ((and (consp op)
      (progn
        (setq qt (car op))
        (setq tmp1 (cdr op))
        (and (consp tmp1)
          (eq (cdr tmp1) nil)
          (progn
            (setq realOp (car tmp1))
            t))))
      (eq qt 'quote))
    (cons '|applyQuote| (cons op argSex)))
    ((setq val (|hasOptArgs?| argSex)) (cons op val))
    (t (cons op argSex))))))
  ((and (consp op)
    (progn
      (setq qt (car op))
      (setq tmp1 (cdr op))
      (and (consp tmp1)
        (eq (cdr tmp1) NIL)
        (progn
          (setq realOp (car tmp1))
          t))))
    (eq qt 'quote))
    (list '|applyQuote| op (|pf2Sex1| args)))
  ((eq op '|braceFromCurly|)
    (setq x (|pf2Sex1| args))
    (if (and (consp x) (eq (car x) 'seq))
      x
      (list 'seq x)))
  ((eq op '|by|) (list 'by (|pf2Sex1| args)))
  (t (list op (|pf2Sex1| args))))))

```

defun Convert a SuchThat node to an S-expression

```

[pf0TupleParts p591]
[|pf2Sex1| p598]
[|pf2Sex| p597]
[$predicateList p??]

```

— defun pfSuchThat2Sex —

```

(defun |pfSuchThat2Sex| (args)
  (let (rhsSex lhsSex argList name)

```

```

(declare (special |$predicateList|))
(setq name (gentemp))
(setq argList (|pf0TupleParts| args))
(setq lhsSex (|pf2Sex1| (car argList)))
(setq rhsSex (|pf2Sex1| (cadr argList)))
(setq |$predicateList|
  (cons (cons name (cons lhsSex rhsSex)) |$predicateList|))
name))

```

—————→

defun pfOp2Sex

```

[pf2Sex1 p598]
[pmDontQuote? p607]
[pfSymbol? p550]
[$quotedOpList p??]
[$insideRule p??]

```

— defun pfOp2Sex —

```

(defun |pfOp2Sex| (pf)
  (let (realOp tmp1 op alreadyQuoted)
    (declare (special |$quotedOpList| |$insideRule|))
    (setq alreadyQuoted (|pfSymbol?| pf))
    (setq op (|pf2Sex1| pf))
    (cond
      ((and (consp op)
            (eq (car op) 'quote)
            (progn
              (setq tmp1 (cdr op))
              (and (consp tmp1)
                    (eq (cdr tmp1) nil)
                    (progn
                     (setq realOp (car tmp1)) t))))
        (cond
          ((eq |$insideRule| '|left|) realOp)
          ((eq |$insideRule| '|right|)
            (cond
              ((|pmDontQuote?| realOp) realOp)
              (t
               (setq |$quotedOpList| (cons op |$quotedOpList|))
               op)))
          ((eq realOp '|\\|) realOp)
          ((eq realOp '|:|) realOp)
          ((eq realOp '|?) realOp)
          (t op)))

```

```
(t op))))
```

defun pmDontQuote?

— defun pmDontQuote? 0 —

```
(defun |pmDontQuote?| (sy)
  (member sy
    '(+ - * ** ^ / |log| |exp| |pi| |sqrt| |ei| |li| |erf| |ci|
      |si| |dilog| |sin| |cos| |tan| |cot| |sec| |csc| |asin|
      |acos| |atan| |acot| |asec| |acsc| |sinh| |cosh| |tanh|
      |coth| |sech| |csch| |asinh| |acosh| |atanh| |acoth|
      |asech| |acsc|)))
```

defun hasOptArgs?

— defun hasOptArgs? 0 —

```
(defun |hasOptArgs?| (argSex)
  (let (rhs lhs opt nonOpt tmp1 tmp2)
    (dolist (arg argSex)
      (cond
        ((and (consp arg)
              (eq (car arg) 'optarg)
              (progn
                (setq tmp1 (cdr arg))
                (and (consp tmp1)
                     (progn
                      (setq lhs (car tmp1))
                      (setq tmp2 (cdr tmp1))
                      (and (consp tmp2)
                           (eq (cdr tmp2) nil)
                           (progn
                            (setq rhs (car tmp2))
                            t)))))))
          (setq opt (cons (list lhs rhs) opt)))
        (t (setq nonOpt (cons arg nonOpt)))))
    (when opt
      (nconc (nreverse nonOpt) (list (cons '|construct| (nreverse opt)))))))
```

defun Convert a Sequence node to an S-expression

[pf2Sex1 p598]

[pf0SequenceArgs p585]

[\$insideSEQ p??]

— defun pfSequence2Sex —

```
(defun |pfSequence2Sex| (pf)
  (let (|$insideSEQ| tmp1 ruleList seq)
    (declare (special |$insideSEQ|))
    (setq |$insideSEQ| t)
    (setq seq (|pfSequence2Sex0| (mapcar #'|pf2Sex1| (|pf0SequenceArgs| pf))))
    (cond
      ((and (consp seq)
            (eq (car seq) 'seq)
            (progn (setq ruleList (cdr seq)) 't)
            (consp ruleList)
            (progn
              (setq tmp1 (car ruleList))
              (and (consp tmp1) (eq (car tmp1) '|rule|))))
        (list '|ruleset| (cons '|construct| ruleList)))
      (t seq))))
```

defun pfSequence2Sex0

TPDHERE: rewrite this using (dolist (item seqList)...)

```
;pfSequence2Sex0 seqList ==
; null seqList => "noBranch"
; seqTranList := []
; while seqList ^= nil repeat
;   item := first seqList
;   item is ["exit", cond, value] =>
;     item := ["IF", cond, value, pfSequence2Sex0 rest seqList]
;   seqTranList := [item, :seqTranList]
;   seqList := nil
;   seqTranList := [item, :seqTranList]
;   seqList := rest seqList
; #seqTranList = 1 => first seqTranList
; ["SEQ", :nreverse seqTranList]
```

[pfSequence2Sex0 p608]

— defun pfSequence2Sex0 —

```
(defun |pfSequence2Sex0| (seqList)
  (let (value tmp2 cond tmp1 item seqTranList)
    (if (null seqList)
      '|noBranch|
      (progn
        ((lambda ()
          (loop
            (if (not seqList)
              (return nil)
              (progn
                (setq item (car seqList))
                (cond
                  ((and (consp item)
                       (eq (car item) '|exit|))
                   (progn
                     (setq tmp1 (cdr item))
                     (and (consp tmp1)
                        (progn
                          (setq cond (car tmp1))
                          (setq tmp2 (cdr tmp1))
                          (and (consp tmp2)
                             (eq (cdr tmp2) nil)
                             (progn
                               (setq value (car tmp2))
                               t)))))))
                  (t
                   (setq item
                        (list 'if cond value (|pfSequence2Sex0| (cdr seqList))))
                   (setq seqTranList (cons item seqTranList))
                   (setq seqList nil))
                  (t
                   (progn
                     (setq seqTranList (cons item seqTranList))
                     (setq seqList (cdr seqList))))))))))
        (if (eql (length seqTranList) 1)
          (car seqTranList)
          (cons 'seq (nreverse seqTranList))))))
```

—

defun Convert a loop node to an S-expression

TPDHERE: rewrite using dsetq

```
;loopIters2Sex iterList ==
```



```

; result := nil
; for iter in iterList repeat
;   sex := pf2Sex1 iter
;   sex is ['IN, var, ['SEGMENT, i, ["BY", incr]]] =>
;     result := [ ['STEP, var, i, incr], :result]
;   sex is ['IN, var, ["BY", ['SEGMENT, i, j], incr]] =>
;     result := [ ['STEP, var, i, incr, j], :result]
;   sex is ['IN, var, ['SEGMENT, i, j]] =>
;     result := [ ['STEP, var, i, 1, j], :result]
;   result := [sex, :result]
; nreverse result

```

[pf2Sex1 p598]

— defun loopIters2Sex —

```

(defun |loopIters2Sex| (iterList)
  (let (j incr i var sex result tmp1 tmp2 tmp3 tmp4 tmp5 tmp6 tmp7 tmp8)
    (dolist (iter iterList (nreverse result))
      (setq sex (|pf2Sex1| iter))
      (cond
        ((and (consp sex)
              (eq (car sex) 'in)
              (progn
                (setq tmp1 (cdr sex))
                (and (consp tmp1)
                     (progn
                      (setq var (car tmp1))
                      (setq tmp2 (cdr tmp1))
                      (and (consp tmp2)
                          (eq (cdr tmp2) nil)
                          (progn
                           (setq tmp3 (car tmp2))
                           (and (consp tmp3)
                               (eq (car tmp3) 'segment)
                               (progn
                                (setq tmp4 (cdr tmp3))
                                (and (consp tmp4)
                                    (progn
                                     (setq i (car tmp4))
                                     (setq tmp5 (cdr tmp4))
                                     (and (consp tmp5)
                                         (eq (cdr tmp5) nil)
                                         (progn
                                          (setq tmp6 (car tmp5))
                                          (and (consp tmp6)
                                              (eq (car tmp6) 'by)
                                              (progn
                                               (setq tmp7 (cdr tmp6))
                                               (and (consp tmp7)
                                                    (progn
                                                     (setq result (cons sex result))
                                                     (nreverse result))))))))))))))))))))))

```

```

                                                    (eq (cdr tmp7) nil)
                                                    (progn
                                                      (setq incr (car tmp7))
                                                      t))))))))))
(setq result (cons (list 'step var i incr) result)))
((and (consp sex)
  (eq (car sex) 'in)
  (progn
    (setq tmp1 (cdr sex))
    (and (consp tmp1)
      (progn
        (setq var (car tmp1))
        (setq tmp2 (cdr tmp1))
        (and (consp tmp2)
          (eq (cdr tmp2) nil)
          (progn
            (setq tmp3 (car tmp2))
            (and (consp tmp3)
              (eq (car tmp3) 'by)
              (progn
                (setq tmp4 (cdr tmp3))
                (and (consp tmp4)
                  (progn
                    (setq tmp5 (car tmp4))
                    (and (consp tmp5)
                      (eq (car tmp5) 'segment)
                      (progn
                        (setq tmp6 (cdr tmp5))
                        (and (consp tmp6)
                          (progn
                            (setq i (car tmp6))
                            (setq tmp7 (cdr tmp6))
                            (and (consp tmp7)
                              (eq (cdr tmp7) nil)
                              (progn
                                (setq j (car tmp7))
                                t))))))))
                        (progn
                          (setq tmp8 (cdr tmp4))
                          (and (consp tmp8)
                            (eq (cdr tmp8) nil)
                            (progn
                              (setq incr (car tmp8))
                              t))))))))
                    (setq result (cons (list 'step var i incr j) result)))
    (and (consp sex)
      (eq (car sex) 'in)
      (progn
        (setq tmp1 (cdr sex))
        (and (consp tmp1)

```

```

(progn
  (setq var (car tmp1))
  (setq tmp2 (cdr tmp1))
  (and (consp tmp2)
    (eq (cdr tmp2) nil)
    (progn
      (setq tmp3 (car tmp2))
      (and (consp tmp3)
        (eq (car tmp3) 'segment)
        (progn
          (setq tmp4 (cdr tmp3))
          (and (consp tmp4)
            (progn
              (setq i (car tmp4))
              (setq tmp5 (cdr tmp4))
              (and (consp tmp5)
                (eq (cdr tmp5) nil)
                (progn
                  (setq j (car tmp5))
                  t))))))))))
    (setq result (cons (list 'step var i 1 j) result)))
  (t (setq result (cons sex result))))))

```

defun Change a Collect node to an S-expression

[\[loopIters2Sex p609\]](#)
[\[pfParts p547\]](#)
[\[pfCollectIterators p558\]](#)
[\[pf2Sex1 p598\]](#)
[\[pfCollectBody p558\]](#)

— defun pfCollect2Sex —

```

(defun |pfCollect2Sex| (pf)
  (let (var cond sex tmp1 tmp2 tmp3 tmp4)
    (setq sex
      (cons 'collect
        (append (|loopIters2Sex| (|pfParts| (|pfCollectIterators| pf)))
          (list (|pf2Sex1| (|pfCollectBody| pf))))))
    (cond
      ((and (consp sex)
        (eq (car sex) 'collect)
        (progn
          (setq tmp1 (cdr sex))
          (and (consp tmp1)

```

```

(progn
  (setq tmp2 (car tmp1))
  (and (consp tmp2)
    (eq (car tmp2) '\\|))
    (progn
      (setq tmp3 (cdr tmp2))
      (and (consp tmp3)
        (eq (cdr tmp3) nil)
        (progn
          (setq cond (car tmp3))
          t))))))
(progn
  (setq tmp4 (cdr tmp1))
  (and (consp tmp4)
    (eq (cdr tmp4) nil)
    (progn (setq var (car tmp4)) t))))
(symbolp var))
(list '\\| var cond))
(t sex)))

```

defun Convert a Definition node to an S-expression

```

[pf2Sex1 p598]
[pf0DefinitionLhsItems p560]
[pfDefinitionRhs p559]
[systemError p??]
[pfLambdaTran p614]
[$insideApplication p??]

```

— defun pfDefinition2Sex —

```

(defun |pfDefinition2Sex| (pf)
  (let (body argList tmp1 rhs id idList)
    (declare (special |$insideApplication|))
    (if |$insideApplication|
      (list 'optarg
        (|pf2Sex1| (car (|pf0DefinitionLhsItems| pf)))
        (|pf2Sex1| (|pfDefinitionRhs| pf)))
      (progn
        (setq idList (mapcar #'|pf2Sex1| (|pf0DefinitionLhsItems| pf)))
        (if (not (eql (length idList) 1))
          (|systemError|
            "lhs of definition must be a single item in the interpreter")
          (progn
            (setq id (car idList))

```

```

(setq rhs (|pfDefinitionRhs| pf))
(setq tmp1 (|pfLambdaTran| rhs))
(setq argList (car tmp1))
(setq body (cdr tmp1))
(cons 'def
  (cons
    (if (eq argList 'id)
      id
      (cons id argList))
    body))))))

```

defun Convert a Lambda node to an S-expression

```

[pfLambda? p571]
[pf0LambdaArgs p572]
[pfTyped? p589]
[pfCollectArgTran p615]
[pfTypedId p589]
[pfNothing? p543]
[pfTypedType p589]
[pf2Sex1 p598]
[systemError p??]
[pfLambdaRets p571]
[pfLambdaBody p571]

```

— defun pfLambdaTran —

```

(defun |pfLambdaTran| (pf)
  (let (retType argList argTypeList)
    (cond
      ((|pfLambda?| pf)
       (dolist (arg (|pf0LambdaArgs| pf))
         (if (|pfTyped?| arg)
             (progn
              (setq argList
                (cons (|pfCollectArgTran| (|pfTypedId| arg)) argList))
              (if (|pfNothing?| (|pfTypedType| arg))
                  (setq argTypeList (cons nil argTypeList))
                  (setq argTypeList
                    (cons (|pf2Sex1| (|pfTypedType| arg)) argTypeList))))
              (|systemError| "definition args should be typed"))
             (setq argList (nreverse argList)))
         (unless (|pfNothing?| (|pfLambdaRets| pf))
           (setq retType (|pf2Sex1| (|pfLambdaRets| pf))))
         (setq argTypeList (cons retType (nreverse argTypeList)))

```

```

(cons argList
  (list argTypeList
    (mapcar #'(lambda (x) (declare (ignore x)) nil) argTypeList)
    (|pf2Sex1| (|pfLambdaBody| pf))))
(t (cons '|id| (list '(nil) '(nil) (|pf2Sex1| pf)))))

```

defun pfCollectArgTran

```

[|pfCollect?| p558]
[|pf2sex1| p??]
[|pfParts| p547]
[|pfCollectIterators| p558]
[|pfCollectBody| p558]

```

— defun pfCollectArgTran —

```

(defun |pfCollectArgTran| (pf)
  (let (cond tmp2 tmp1 id conds)
    (cond
      ((|pfCollect?| pf)
       (setq conds (mapcar #'|pf2sex1| (|pfParts| (|pfCollectIterators| pf))))
       (setq id (|pf2Sex1| (|pfCollectBody| pf)))
       (cond
         ((and (consp conds) ; conds is [ "|", cond ]
              (eq (cdr conds) nil)
              (progn
                (setq tmp1 (car conds))
                (and (consp tmp1)
                     (eq (car tmp1) '|\\|)
                     (progn
                      (setq tmp2 (cdr tmp1))
                      (and (consp tmp2)
                          (eq (cdr tmp2) nil)
                          (progn
                           (setq cond (car tmp2))
                           t))))))
          (list '|\\| id cond))
         (t (cons id conds))))
      (t (|pf2Sex1| pf)))))

```

defun Convert a Lambda node to an S-expression

[pfLambdaTran p614]

— defun pfLambda2Sex —

```
(defun |pfLambda2Sex| (pf)
  (let (body argList tmp1)
    (setq tmp1 (|pfLambdaTran| pf))
    (setq argList (car tmp1))
    (setq body (cdr tmp1))
    (cons 'adeft (cons argList body))))
```

—————

defun Convert a Rule node to an S-expression

[pfLhsRule2Sex p617]

[pfRuleLhsItems p584]

[pfRhsRule2Sex p617]

[pfRuleRhs p584]

[ruleLhsTran p620]

[rulePredicateTran p617]

[\$multiVarPredicateList p??]

[\$predicateList p??]

[\$quotedOpList p??]

— defun pfRule2Sex —

```
(defun |pfRule2Sex| (pf)
  (let (|$multiVarPredicateList| |$predicateList| |$quotedOpList| rhs lhs)
    (declare (special |$multiVarPredicateList| |$predicateList| |$quotedOpList|))
    (setq |$quotedOpList| nil)
    (setq |$predicateList| nil)
    (setq |$multiVarPredicateList| nil)
    (setq lhs (|pfLhsRule2Sex| (|pfRuleLhsItems| pf)))
    (setq rhs (|pfRhsRule2Sex| (|pfRuleRhs| pf)))
    (setq lhs (|ruleLhsTran| lhs))
    (|rulePredicateTran|
     (if |$quotedOpList|
        (list 'rule| lhs rhs (cons 'construct| |$quotedOpList|))
        (list 'rule| lhs rhs)))))
```

—————

defun Convert the Lhs of a Rule to an S-expression

[pf2Sex1 p598]

[\$insideRule p??]

— defun pfLhsRule2Sex —

```
(defun |pfLhsRule2Sex| (lhs)
  (let (|$insideRule|)
    (declare (special |$insideRule|))
    (setq |$insideRule| '|left|)
    (|pf2Sex1| lhs)))
```

—————

defun Convert the Rhs of a Rule to an S-expression

[pf2Sex1 p598]

[\$insideRule p??]

— defun pfRhsRule2Sex —

```
(defun |pfRhsRule2Sex| (rhs)
  (let (|$insideRule|)
    (declare (special |$insideRule|))
    (setq |$insideRule| '|right|)
    (|pf2Sex1| rhs)))
```

—————

defun Convert a Rule predicate to an S-expression

```
;rulePredicateTran rule ==
; null $multiVarPredicateList => rule
; varList := patternVarsOf [rhs for [.,.,:rhs] in $multiVarPredicateList]
; predBody :=
;   CDR $multiVarPredicateList =>
;     ['AND, :[:pvarPredTran(rhs, varList) for [.,.,:rhs] in
;       $multiVarPredicateList]]
;   [ [.,.,:rhs],:] := $multiVarPredicateList
;   pvarPredTran(rhs, varList)
; ['suchThat, rule,
;   ['construct, :[ ["QUOTE", var] for var in varList]],
;   ['ADEF, '(predicateVariable),
;   '((Boolean) (List (Expression (Integer))))), '(() ()),
;   predBody]]
```



```
[patternVarsOf p619]
[pvarPredTran p620]
[$multiVarPredicateList p??]
```

— defun rulePredicateTran —

```
(defun |rulePredicateTran| (rule)
  (let (predBody varList rhs tmp1 result)
    (declare (special |$multiVarPredicateList|))
    (if (null |$multiVarPredicateList|)
      rule
      (progn
        (setq varList
          (|patternVarsOf|
            ((lambda (t1 t2 t3)
              (loop
                (cond
                  ((or (atom t2)
                     (progn
                      (setq t3 (car t2))
                      nil))
                 (return (nreverse t1))))
              (t
                (and (consp t3)
                  (progn
                    (setq tmp1 (cdr t3))
                    (and (consp tmp1)
                      (progn
                        (setq rhs (cdr tmp1))
                        t)))
                  (setq t1 (cons rhs t1))))))
            (setq t2 (cdr t2))))
          nil |$multiVarPredicateList| nil)))
    (setq predBody
      (cond
        ((cdr |$multiVarPredicateList|)
         (cons 'and
           ((lambda (t4 t5 t6)
             (loop
               (cond
                 ((or (atom t5)
                    (progn
                     (setq t6 (car t5))
                     nil))
                  (return (nreverse t4))))
             (t
               (and (consp t6)
                 (progn
                   (setq tmp1 (cdr t6))
```

```

      (and (consp tmp1)
        (progn
          (setq rhs (cdr tmp1))
          t)))
      (setq t4
        (append (reverse (|pvarPredTran| rhs varList))
          t4))))
    (setq t5 (cdr t5)))
  nil |$multiVarPredicateList| nil)))
(t
  (progn
    (setq rhs (cddar |$multiVarPredicateList|))
    (|pvarPredTran| rhs varList))))
(dolist (var varList) (push (list 'quote var) result))
(list '|suchThat| rule
  (cons '|construct| (nreverse result))
  (list 'adef '(|predicateVariable|
    '((|Boolean|
      (|List| (|Expression| (|Integer|))))
    '(nil nil) predBody))))))

```

defun patternVarsOf

[patternVarsOf1 p619]

— defun patternVarsOf —

```

(defun |patternVarsOf| (expr)
  (|patternVarsOf1| expr nil))

```

defun patternVarsOf1

[patternVarsOf1 p619]

— defun patternVarsOf1 —

```

(defun |patternVarsOf1| (expr varList)
  (let (argl op)
    (cond
      ((null expr) varList)
      ((atom expr)

```

```

(cond
  ((null (symbolp expr)) varList)
  ((member expr varList) varList)
  (t (cons expr varList))))
((and (consp expr)
  (progn
    (setq op (car expr))
    (setq arg1 (cdr expr))
    t))
  (progn
    (dolist (arg arg1)
      (setq varList (|patternVarsOf1| arg varList)))
    varList))
  (t varList))))

```

defun pvarPredTran

— defun pvarPredTran —

```

(defun |pvarPredTran| (rhs varList)
  (let ((i 0))
    (dolist (var varList rhs)
      (setq rhs (nsubst (list '|elt| '|predicateVariable| (incf i)) var rhs)))))

```

defun Convert the Lhs of a Rule node to an S-expression

```

[patternVarsOf p619]
[nsubst p??]
[$predicateList p??]
[$multiVarPredicateList p??]

```

— defun ruleLhsTran —

```

(defun |ruleLhsTran| (ruleLhs)
  (let (predicate var vars predRhs predLhs name)
    (declare (special |$predicateList| |$multiVarPredicateList|))
    (dolist (pred |$predicateList|)
      (setq name (car pred))
      (setq predLhs (cadr pred))
      (setq predRhs (cddr pred))

```

```

(setq vars (|patternVarsOf| predRhs))
(cond
  ((cdr vars)
    (setq ruleLhs (nsubst predLhs name ruleLhs))
    (setq |$multiVarPredicateList| (cons pred |$multiVarPredicateList|)))
  (t
    (setq var (cadr predLhs))
    (setq predicate
      (list '|suchThat| predLhs (list 'adef (list var)
        '((|Boolean|) (|Expression| (|Integer|))) '(nil nil) predRhs)))
    (setq ruleLhs (nsubst predicate name ruleLhs))))))
ruleLhs))

```

defun Translate ops into internal symbols

— defun opTran 0 —

```

(defun |opTran| (op)
  (cond
    ((equal op '|.|) 'segment)
    ((eq op '[]) 'construct|)
    ((eq op '{|} ) 'braceFromCurly|)
    ((eq op 'is) '|is|)
    (t op)))

```

Chapter 17

Stream Utilities

The input stream is parsed into a large s-expression by repeated calls to Delay. Delay takes a function `f` and an argument `x` and returns a list consisting of `("nonnullstream" f x)`. Eventually multiple calls are made and a large list structure is created that consists of `("nonnullstream" f x ("nonnullstream" f1 x1 ("nonnullstream" f2 x2...))`

This delay structure is given to StreamNull which walks along the list looking at the head. If the head is “nonnullstream” then the function is applied to the argument.

So, in effect, the input is “zipped up” into a Delay data structure which is then evaluated by calling StreamNull. This ”zippered stream” parser was a research project at IBM and Axiom was the testbed (which explains the strange parsing technique).

defun npNull

[StreamNull p623]

— defun npNull —

(defun |npNull| (x) (|StreamNull| x))

—————

defun StreamNull

[eqcar p??]

StreamNull : Delay → Union(T,NIL)

— defun StreamNull 0 —

(defun |StreamNull| (delay)

```
(let (parsepair)
  (cond
    ((or (null delay) (eqcar delay '|nullstream|)) t)
    (t
     ((lambda nil
        (loop
         (cond
           ((not (eqcar delay '|nonnullstream|)) (return nil))
           (t
            (setq parsepair (apply (cadr delay) (cddr delay)))
            (rplaca delay (car parsepair))
            (rplacd delay (cdr parsepair))))))
      (eqcar delay '|nullstream|))))))
```

Chapter 18

Code Piles

The `insertpile` function converts a line-list to a line-forest where a line is a token-dequeue and has a column which is an integer. An A-forest is an A-tree-list. An A-tree has a root which is an A, and subtrees which is an A-forest.

A forest with more than one tree corresponds to a Scratchpad pile structure (t1;t2;t3;...;tn), and a tree corresponds to a pile item. The (; and) tokens are inserted into a $\mathcal{L}_1$ -forest, otherwise the root of the first tree is concatenated with its forest. column t is the number of spaces before the first non-space in line t

defun insertpile

[npNull p623]

[pilePlusComment p626]

[pilePlusComments p626]

[pileTree p627]

[pileCforest p630]

— **defun insertpile** —

```
(defun |insertpile| (s)
  (let (stream a t1 h1 t2 h tmp1)
    (cond
      ((|npNull| s) (list nil 0 nil s))
      (t
       (setq tmp1 (list (car s) (cdr s)))
       (setq h (car tmp1))
       (setq t2 (cadr tmp1))
       (cond
         ((|pilePlusComment| h)
          (setq tmp1 (|pilePlusComments| s))
          (setq h1 (car tmp1))
```



```

      (setq t1 (cadr tmp1))
      (setq a (|pileTree| (- 1) t1))
      (cons (list (|pileCforest|
                    (append h1 (cons (elt a 2) nil))))
              (elt a 3)))
    (t
     (setq stream (cadar s))
     (setq a (|pileTree| -1 s))
     (cons (list (list (elt a 2) stream)) (elt a 3))))))

```

defun pilePlusComment

```

[tokType p703]
[npNull p623]
[pilePlusComment p626]
[pilePlusComments p626]

```

— defun pilePlusComment —

```

(defun |pilePlusComment| (arg)
  (eq (|tokType| (caar arg)) '|comment|))

```

defun pilePlusComments

— defun pilePlusComments —

```

(defun |pilePlusComments| (s)
  (let (t1 h1 t2 h tmp1)
    (cond
      ((|npNull| s) (list nil s))
      (t
       (setq tmp1 (list (car s) (cdr s)))
       (setq h (car tmp1))
       (setq t2 (cadr tmp1))
       (cond
         ((|pilePlusComment| h)
          (setq tmp1 (|pilePlusComments| t2))
          (setq h1 (car tmp1))
          (setq t1 (cadr tmp1))
          (list (cons h h1) t1))

```

```
(t
  (list nil s))))))
```

defun pileTree

[npNull p623]
 [pileColumn p627]
 [pileForests p627]

— defun pileTree —

```
(defun |pileTree| (n s)
  (let (hh t1 h tmp1)
    (cond
      ((|npNull| s) (list nil n nil s))
      (t
       (setq tmp1 (list (car s) (cdr s)))
       (setq h (car tmp1))
       (setq t1 (cadr tmp1))
       (setq hh (|pileColumn| (car h)))
       (cond
         ((< n hh) (|pileForests| (car h) hh t1))
         (t (list nil n nil s)))))))
```

defun pileColumn

[tokPosn p703]

— defun pileColumn —

```
(defun |pileColumn| (arg)
  (cdr (|tokPosn| (caar arg))))
```

defun pileForests

[pileForest p628]
 [npNull p623]

[pileForests p627]
 [pileCtree p630]

— **defun pileForests** —

```
(defun |pileForests| (h n s)
  (let (t1 h1 tmp1)
    (setq tmp1 (|pileForest| n s))
    (setq h1 (car tmp1))
    (setq t1 (cadr tmp1))
    (cond
     ((|npNull| h1) (list t n h s))
     (t (|pileForests| (|pileCtree| h h1) n t1))))))
```

—————

defun pileForest

[pileTree p627]
 [pileForest1 p628]

— **defun pileForest** —

```
(defun |pileForest| (n s)
  (let (t1 h1 t2 h hh b tmp)
    (setq tmp (|pileTree| n s))
    (setq b (car tmp))
    (setq hh (cadr tmp))
    (setq h (caddr tmp))
    (setq t2 (caddr tmp))
    (cond
     (b
      (setq tmp (|pileForest1| hh t2))
      (setq h1 (car tmp))
      (setq t1 (cadr tmp))
      (list (cons h h1) t1))
     (t
      (list nil s)))))
```

—————

defun pileForest1

[eqpileTree p629]
 [pileForest1 p628]

— defun pileForest1 —

```
(defun |pileForest1| (n s)
  (let (t1 h1 t2 h n1 b tmp)
    (setq tmp (|eqpileTree| n s))
    (setq b (car tmp))
    (setq n1 (cadr tmp))
    (setq h (caddr tmp))
    (setq t2 (cadddr tmp))
    (cond
     (b
      (setq tmp (|pileForest1| n t2))
      (setq h1 (car tmp))
      (setq t1 (cadr tmp))
      (list (cons h h1) t1))
     (t (list nil s)))))
```

—————

defun eqpileTree

[npNull [p623](#)]
 [pileColumn [p627](#)]
 [pileForests [p627](#)]

— defun eqpileTree —

```
(defun |eqpileTree| (n s)
  (let (hh t1 h tmp)
    (cond
     ((|npNull| s) (list nil n nil s))
     (t
      (setq tmp (list (car s) (cdr s)))
      (setq h (car tmp))
      (setq t1 (cadr tmp))
      (setq hh (|pileColumn| (car h)))
      (cond
       ((equal hh n) (|pileForests| (car h) hh t1))
       (t (list nil n nil s)))))))
```

—————

defun pileCtree

[dqAppend p634]
 [pileCforest p630]

— defun pileCtree —

```
(defun |pileCtree| (x y)
  (|dqAppend| x (|pileCforest| y)))
```

—————

defun pileCforest

Only enpiles forests with ≥ 2 trees

[tokPart p703]
 [enPile p630]
 [separatePiles p631]

— defun pileCforest —

```
(defun |pileCforest| (x)
  (let (f)
    (cond
      ((null x) nil)
      ((null (cdr x)) (setq f (car x)))
      (cond
        ((eq (|tokPart| (caar f)) 'if) (|enPile| f))
        (t f)))
    (t (|enPile| (|separatePiles| x))))))
```

—————

defun enPile

[dqConcat p633]
 [dqUnit p633]
 [tokConstruct p701]
 [firstTokPosn p631]
 [lastTokPosn p631]

— defun enPile —

```
(defun |enPile| (x)
```

```
(|dqConcat|
  (list
    (|dqUnit| (|tokConstruct| 'key| 'settab (|firstTokPosn| x)))
    x
    (|dqUnit| (|tokConstruct| 'key| 'backtab (|lastTokPosn| x))))))
```

defun firstTokPosn

[tokPosn p703]

— defun firstTokPosn —

```
(defun |firstTokPosn| (arg) (|tokPosn| (caar arg)))
```

defun lastTokPosn

[tokPosn p703]

— defun lastTokPosn —

```
(defun |lastTokPosn| (arg) (|tokPosn| (cadr arg)))
```

defun separatePiles

[dqUnit p633]
 [tokConstruct p701]
 [lastTokPosn p631]
 [dqConcat p633]
 [separatePiles p631]

— defun separatePiles —

```
(defun |separatePiles| (x)
  (let (semicolon a)
    (cond
      ((null x) nil)
```

```
((null (cdr x)) (car x))
(t
 (setq a (car x))
 (setq semicolon
  (|dqUnit| (|tokConstruct| 'key| 'backset (|lastTokPosn| a))))
 (|dqConcat| (list a semicolon (|separatePiles| (cdr x))))))
```

Chapter 19

Dequeue Functions

The dqUnit makes a unit dq i.e. a dq with one item, from the item

defun dqUnit

— defun dqUnit 0 —

```
(defun |dqUnit| (s)
  (let (a)
    (setq a (list s))
    (cons a a)))
```

—————

defun dqConcat

The dqConcat function concatenates a list of dq's, destroying all but the last

[dqAppend p634]
[dqConcat p633]

— defun dqConcat —

```
(defun |dqConcat| (ld)
  (cond
    ((null ld) nil)
    ((null (cdr ld)) (car ld))
    (t (|dqAppend| (car ld) (|dqConcat| (cdr ld))))))
```

defun dqAppend

The dqAppend function appends 2 dq's, destroying the first

— **defun dqAppend 0** —

```
(defun |dqAppend| (x y)
  (cond
    ((null x) y)
    ((null y) x)
    (t
     (rplacd (cdr x) (car y))
     (rplacd x (cdr y)) x)))
```

defun dqToList

— **defun dqToList 0** —

```
(defun |dqToList| (s)
  (when s (car s)))
```

Chapter 20

Message Handling

20.1 The Line Object

defun Line object creation

This is called in only one place, the `incLine1` function.

— `defun lnCreate 0` —

```
(defun |lnCreate| (extraBlanks string globalNum &rest optFileStuff)
  (let ((localNum (first optFileStuff))
        (filename (second optFileStuff)))
    (unless localNum (setq localNum 0))
    (list extraBlanks string globalNum localNum filename)))
```

—————

defun Line element 0; Extra blanks

— `defun lnExtraBlanks 0` —

```
(defun |lnExtraBlanks| (lineObject) (elt lineObject 0))
```

—————

defun Line element 1; String

— `defun lnString 0` —

```
(defun |lnString| (lineObject) (elt lineObject 1))
```

defun Line element 2; Global number

— defun lnGlobalNum 0 —

```
(defun |lnGlobalNum| (lineObject) (elt lineObject 2))
```

defun Line element 2; Set Global number

— defun lnSetGlobalNum 0 —

```
(defun |lnSetGlobalNum| (lineObject num)
  (setf (elt lineObject 2) num))
```

defun Line elemnt 3; Local number

— defun lnLocalNum 0 —

```
(defun |lnLocalNum| (lineObject) (elt lineObject 3))
```

defun Line element 4; Place of origin

— defun lnPlaceOfOrigin 0 —

```
(defun |lnPlaceOfOrigin| (lineObject) (elt lineObject 4))
```

defun Line element 4: Is it a filename?

[lnFileName? p637]

— defun lnImmediate? 0 —

```
(defun |lnImmediate?| (lineObject) (null (|lnFileName?| lineObject)))
```

—————

defun Line element 4: Is it a filename?

— defun lnFileName? 0 —

```
(defun |lnFileName?| (lineObject)
  (let (filename)
    (when (consp (setq filename (elt lineObject 4))) filename)))
```

—————

defun Line element 4; Get filename

[lnFileName? p637]

[ncBug p658]

— defun lnFileName —

```
(defun |lnFileName| (lineObject)
  (let (fN)
    (if (setq fN (|lnFileName?| lineObject))
        fN
        (|ncBug| "there is no file name in %1" (list lineObject)))))
```

—————

20.2 Messages**defun msgCreate**

```
msgObject
  tag -- catagory of msg
```

```

-- attributes as a-list
'imPr => dont save for list processing
toWhere, screen or file
'norep => only display once in list
pos -- position with possible FROM/TO tag
key -- key for message database
argL -- arguments to be placed in the msg test
prefix -- things like "Error: "
text -- the actual text

[setMsgForcedAttrList p654]
[putDatabaseStuff p655]
[initImPr p657]
[initToWhere p658]

— defun msgCreate —

(defun |msgCreate| (tag posWTag key argL optPre &rest optAttr)
  (let (msg)
    (when (consp key) (setq tag '|old|))
    (setq msg (list tag posWTag key argL optPre nil))
    (when (car optAttr) (|setMsgForcedAttrList| msg (car optAttr)))
    (|putDatabaseStuff| msg)
    (|initImPr| msg)
    (|initToWhere| msg)
    msg))

```

defmacro getMsgPosTagOb

— defmacro getMsgPosTagOb 0 —

```

(defmacro |getMsgPosTagOb| (msg)
  '(elt ,msg 1))

```

defmacro getMsgKey

— defmacro getMsgKey 0 —

```
(defmacro |getMsgKey| (msg)
  '(elt ,msg 2))
```

defmacro getMsgArgL

— defmacro getMsgArgL 0 —

```
(defmacro |getMsgArgL| (msg)
  '(elt ,msg 3))
```

defmacro getMsgPrefix

— defmacro getMsgPrefix 0 —

```
(defmacro |getMsgPrefix| (msg)
  '(elt ,msg 4))
```

defmacro setMsgPrefix

— defmacro setMsgPrefix 0 —

```
(defmacro |setMsgPrefix| (msg val)
  '(setf (elt ,msg 4) ,val))
```

defmacro getMsgText

— defmacro getMsgText 0 —

```
(defmacro |getMsgText| (msg)
  '(elt ,msg 5))
```

defmacro setMsgText

— defmacro setMsgText 0 —

```
(defmacro |setMsgText| (msg val)
  '(setf (elt ,msg 5) ,val))
```

defmacro getMsgPrefix?

— defmacro getMsgPrefix? 0 —

```
(defmacro |getMsgPrefix?| (msg)
  '(let ((pre (|getMsgPrefix| ,msg)))
    (unless (eq pre 'noPre) pre)))
```

defmacro getMsgTag

The valid message tags are: line, old, error, warn, bug, unimple, remark, stat, say, debug
[ncTag p705]

— defmacro getMsgTag 0 —

```
(defmacro |getMsgTag| (msg)
  '(|ncTag| ,msg))
```

defmacro getMsgTag?

```
[ifcar p??]  
[getMsgTag p640]
```

— defmacro getMsgTag? 0 —

```
(defmacro |getMsgTag?| (msg)
  '(ifcar (member (|getMsgTag| ,msg)
    (list '|line| '|old| '|error| '|warn| '|bug|
      '|unimple| '|remark| '|stat| '|say| '|debug|))))
```

defmacro line?

[getMsgTag p640]

— defmacro line? —

```
(defmacro |line?| (msg)
  '(eq (|getMsgTag| ,msg) '|line|))
```

defmacro leader?

[getMsgTag p640]

— defmacro leader? —

```
(defmacro |leader?| (msg)
  '(eq (|getMsgTag| ,msg) '|leader|))
```

defmacro toScreen?

[getMsgToWhere p653]

— defmacro toScreen? —

```
(defmacro |toScreen?| (msg)
  '(not (eq (|getMsgToWhere| ,msg) '|fileOnly|)))
```

defun ncSoftError

Messages for the USERS of the compiler. The program being compiled has a minor error. Give a message and continue processing.

```
[desiredMsg p643]
[processKeyedError p643]
[msgCreate p637]
[$newcompErrorCount p319]
```

— **defun ncSoftError** —

```
(defun |ncSoftError| (pos erMsgKey erArgL &rest optAttr)
  (declare (special |$newcompErrorCount|))
  (setq |$newcompErrorCount| (+ |$newcompErrorCount| 1))
  (when (|desiredMsg| erMsgKey)
    (|processKeyedError|
     (|msgCreate| ' |error| pos erMsgKey erArgL
                  "Error" optAttr))))
```

defun ncHardError

The program being compiled is seriously incorrect. Give message and throw to a recovery point.

```
[desiredMsg p643]
[processKeyedError p643]
[msgCreate p637]
[ncError p361]
[$newcompErrorCount p319]
```

— **defun ncHardError** —

```
(defun |ncHardError| (pos erMsgKey erArgL &rest optAttr)
  (let (erMsg)
    (declare (special |$newcompErrorCount|))
    (setq |$newcompErrorCount| (+ |$newcompErrorCount| 1))
    (if (|desiredMsg| erMsgKey)
        (setq erMsg
              (|processKeyedError|
               (|msgCreate| ' |error| pos erMsgKey erArgL "Error" optAttr)))
        (|ncError|))))
```

defun desiredMsg

— defun desiredMsg 0 —

```
(defun |desiredMsg| (erMsgKey &rest optCatFlag)
  (declare (ignore erMsgKey))
  (cond
    ((null (null optCatFlag)) (car optCatFlag))
    (t t)))
```

defun processKeyedError

```
[getMsgTag? p640]
[getMsgKey p638]
[getMsgPrefix? p640]
[sayBrightly p??]
[CallerName p??]
[msgImPr? p648]
[msgOutputter p643]
[$ncMsgList p318]
```

— defun processKeyedError —

```
(defun |processKeyedError| (msg)
  (prog (pre erMsg)
    (declare (special |$ncMsgList|))
    (cond
      ((eq (|getMsgTag?| msg) '|old|)
        (setq erMsg (|getMsgKey| msg))
        (cond
          ((setq pre (|getMsgPrefix?| msg))
            (setq erMsg (cons pre erMsg))))
        (|sayBrightly| (cons "old msg from " (cons (|CallerName| 4) erMsg))))
      ((|msgImPr?| msg) (|msgOutputter| msg))
      (t (setq |$ncMsgList| (cons msg |$ncMsgList|))))))
```

defun msgOutputter

```
[getStFromMsg p644]
[leader? p641]
```

```
[line? p641]
[toScreen? p641]
[flowSegmentedMsg p??]
[sayBrightly p??]
[toFile? p653]
[alreadyOpened? p653]
[$linelength p1051]
```

— **defun msgOutputter** —

```
(defun |msgOutputter| (msg)
  (let (alreadyOpened shouldFlow st)
    (declare (special $linelength))
    (setq st (|getStFromMsg| msg))
    (setq shouldFlow (null (or (|leader?| msg) (|line?| msg))))
    (when (|toScreen?| msg)
      (when shouldFlow (setq st (|flowSegmentedMsg| st $linelength 0)))
      (|sayBrightly| st))
    (when (|toFile?| msg)
      (when shouldFlow (setq st (|flowSegmentedMsg| st (- $linelength 6) 0)))
      (setq alreadyOpened (|alreadyOpened?| msg)))))
```

defun listOutputter

```
[msgOutputter p643]
```

— **defun listOutputter** —

```
(defun |listOutputter| (outputList)
  (dolist (msg outputList)
    (|msgOutputter| msg)))
```

defun getStFromMsg

```
[getPreStL p645]
[getMsgPrefix? p640]
[getMsgTag p640]
[getMsgText p639]
[getPosStL p646]
[getMsgKey? p652]
```

[pname p1235]
[tabbing p652]

— defun getStFromMsg —

```
(defun |getStFromMsg| (msg)
  (let (st posStL preStL)
    (setq preStL (|getPreStL| (|getMsgPrefix?| msg)))
    (cond
      ((eq (|getMsgTag| msg) '|line|)
        (cons ""
          (cons "%x1" (append preStL (cons (|getMsgText| msg) nil))))))
      (t
        (setq posStL (|getPosStL| msg))
        (setq st
          (cons posStL
            (cons " "
              (append preStL
                (cons (|tabbing| msg)
                  (|getMsgText| msg)))))))))))
```

—————

defvar \$preLength

— initvars —

```
(defvar |$preLength| 11)
```

—————

defun getPreStL

[size p1236]
[\$preLength p645]

— defun getPreStL 0 —

```
(defun |getPreStL| (optPre)
  (let (spses extraPlaces)
    (declare (special |$preLength|))
    (cond
      ((null optPre) (list " ")))
    (t
```

```

(setq spses
  (cond
    ((< 0 (setq extraPlaces (- (- |$preLength| (size optPre)) 3)))
    (make-string extraPlaces))
    (t "")))
(list optPre spses ":")))))

```

defun getPosStL

```

[showMsgPos? p648]
[getMsgPos p649]
[msgImPr? p648]
[decideHowMuch p649]
[listDecideHowMuch p651]
[ppos p647]
[remLine p652]
[remFile p647]
[$lastPos p??]

```

— defun getPosStL —

```

(defun |getPosStL| (msg)
  (let (printedOrigin printedLineNum printedFileName fullPrintedPos howMuch
        msgPos)
    (declare (special |$lastPos|))
    (cond
      ((null (|showMsgPos?| msg)) "")
      (t
       (setq msgPos (|getMsgPos| msg))
       (setq howMuch
         (if (|msgImPr?| msg)
             (|decideHowMuch| msgPos |$lastPos|)
             (|listDecideHowMuch| msgPos |$lastPos|)))
       (setq |$lastPos| msgPos)
       (setq fullPrintedPos (|ppos| msgPos))
       (setq printedFileName
         (cons "%x2" (cons "[" (append (|remLine| fullPrintedPos) (cons "]" nil)))))
       (setq printedLineNum
         (cons "%x2" (cons "[" (append (|remFile| fullPrintedPos) (cons "]" nil)))))
       (setq printedOrigin
         (cons "%x2" (cons "[" (append fullPrintedPos (cons "]" nil)))))
       (cond
         ((eq howMuch 'org)
          (cons "" (append printedOrigin (cons '|%l| nil))))
         ((eq howMuch 'line)

```

```

      (cons "" (append printedLineNum (cons '|%1| nil))))
    ((eq howMuch 'file)
     (cons "" (append printedFileName (cons '|%1| nil))))
    ((eq howMuch 'all)
     (cons ""
      (append printedFileName
       (cons '|%1|
        (cons ""
         (append printedLineNum
          (cons '|%1| nil))))))))
    (t ""))))))

```

defun ppos

```

[pfNoPosition? p702]
[pfImmediate? p??]
[pfCharPosn p533]
[pfLinePosn p533]
[porigin p381]
[pfFileName p534]

```

— defun ppos —

```

(defun |ppos| (p)
  (let (org lpos cpos)
    (cond
      ((|pfNoPosition?| p) (list "no position"))
      ((|pfImmediate?| p) (list "console"))
      (t
       (setq cpos (|pfCharPosn| p))
       (setq lpos (|pfLinePosn| p))
       (setq org (|porigin| (|pfFileName| p)))
       (list org " " "line" " " lpos)))))

```

defun remFile

```

[ifcdr p??]
[ifcar p??]

```

— defun remFile —

```
(defun |remFile| (positionList) (ifcdr (ifcdr positionList)))
```

defun showMsgPos?

```
[msgImPr? p648]
[leader? p641]
[$erMsgToss p??]
```

— defun showMsgPos? 0 —

```
(defun |showMsgPos?| (msg)
  (declare (special |$erMsgToss|))
  (or |$erMsgToss| (and (null (|msgImPr?| msg)) (null (|leader?| msg)))))
```

defvar \$imPrGuys

— initvars —

```
(defvar |$imPrGuys| (list '|imPr|))
```

defun msgImPr?

```
[getMsgCatAttr p648]
```

— defun msgImPr? —

```
(defun |msgImPr?| (msg)
  (eq (|getMsgCatAttr| msg '|$imPrGuys|) '|imPr|))
```

defun getMsgCatAttr

```
[ifcdr p??]
[qassq p??]
```

[ncAlist p705]

— defun getMsgCatAttr —

```
(defun |getMsgCatAttr| (msg cat)
  (ifcdr (qassq cat (|ncAlist| msg))))
```

—————

defun getMsgPos

[getMsgFTTag? p649]
[getMsgPosTagOb p638]

— defun getMsgPos —

```
(defun |getMsgPos| (msg)
  (if (|getMsgFTTag?| msg)
      (cadr (|getMsgPosTagOb| msg))
      (|getMsgPosTagOb| msg)))
```

—————

defun getMsgFTTag?

[ifcar p??]
[getMsgPosTagOb p638]

— defun getMsgFTTag? —

```
(defun |getMsgFTTag?| (msg)
  (ifcar (member (ifcar (|getMsgPosTagOb| msg)) (list 'from 'to 'fromto))))
```

—————

defun decideHowMuch

When printing a msg, we wish not to show pos information that was shown for a previous msg with identical pos info. org prints out the word noposition or console [poNopos? p650]

[poPosImmediate? p650]
[poFileName p650]
[poLinePosn p651]

— defun decideHowMuch —

```
(defun |decideHowMuch| (pos oldPos)
  (cond
    ((or (and (|poNopos?| pos) (|poNopos?| oldPos))
         (and (|poPosImmediate?| pos) (|poPosImmediate?| oldPos)))
      'none)
    ((or (|poNopos?| pos) (|poPosImmediate?| pos)) 'org)
    ((or (|poNopos?| oldPos) (|poPosImmediate?| oldPos)) 'all)
    ((not (equal (|poFileName| oldPos) (|poFileName| pos))) 'all)
    ((not (equal (|poLinePosn| oldPos) (|poLinePosn| pos))) 'line)
    (t 'none)))
```

defun poNopos?

— defun poNopos? 0 —

```
(defun |poNopos?| (posn)
  (equal posn (list '|no-position|)))
```

defun poPosImmediate?

```
[poNopos? p650]
[lnImmediate? p637]
[poGetLineObject p651]
```

— defun poPosImmediate? —

```
(defun |poPosImmediate?| (txp)
  (unless (|poNopos?| txp) (|lnImmediate?| (|poGetLineObject| txp))))
```

defun poFileName

```
[lnFileName p637]
[poGetLineObject p651]
```

— defun poFileName —

```
(defun |poFileName| (posn)
  (if posn
    (|lnFileName| (|poGetLineObject| posn))
    (caar posn)))
```

—————

defun poGetLineObject

— defun poGetLineObject 0 —

```
(defun |poGetLineObject| (posn)
  (car posn))
```

—————

defun poLinePosn

```
[lnLocalNum p636]
[poGetLineObject p651]
```

— defun poLinePosn —

```
(defun |poLinePosn| (posn)
  (if posn
    (|lnLocalNum| (|poGetLineObject| posn))
    (cdar posn)))
```

—————

defun listDecideHowMuch

```
[poNopos? p650]
[poPosImmediate? p650]
[poGlobalLinePosn p364]
```

— defun listDecideHowMuch —

```
(defun |listDecideHowMuch| (pos oldPos)
  (cond
    ((or (and (|poNopos?| pos) (|poNopos?| oldPos))
         (and (|poPosImmediate?| pos) (|poPosImmediate?| oldPos))))
    'none)
    ((|poNopos?| pos) 'org)
    ((|poNopos?| oldPos) 'none)
    ((< (|poGlobalLinePosn| pos) (|poGlobalLinePosn| oldPos))
     (if (|poPosImmediate?| pos) 'org 'line))
    (t 'none)))
```

defun remLine

— defun remLine 0 —

```
(defun |remLine| (positionList) (list (ifcar positionList)))
```

defun getMsgKey?

[identp p¹²³⁷]

— defun getMsgKey? 0 —

```
(defun |getMsgKey?| (msg)
  (let ((val (|getMsgKey| msg)))
    (when (identp val) val)))
```

defun tabbing

[getMsgPrefix? p⁶⁴⁰]

[\$preLength p⁶⁴⁵]

— defun tabbing —

```
(defun |tabbing| (msg)
  (let (chPos)
```

```
(declare (special |$preLength|))
(setq chPos 2)
(when (|getMsgPrefix?| msg) (setq chPos (- (+ chPos |$preLength|) 1)))
(cons '|%t| chPos)))
```

defvar \$toWhereGuys

```
— initvars —

(defvar |$toWhereGuys| (list '|fileOnly| '|screenOnly|))
```

defun getMsgToWhere

```
[getMsgCatAttr p648]

— defun getMsgToWhere —

(defun |getMsgToWhere| (msg) (|getMsgCatAttr| msg '|$toWhereGuys|))
```

defun toFile?

```
[getMsgToWhere p653]
[$fn p??]

— defun toFile? —

(defun |toFile?| (msg)
  (and (not (eq (|getMsgToWhere| msg) '|screenOnly|))))
```

defun alreadyOpened?

```
[msgImPr? p648]

— defun alreadyOpened? —
```

```
(defun |alreadyOpened?| (msg) (null (|msgImPr?| msg)))
```

defun setMsgForcedAttrList

```
[setMsgForcedAttr p654]
[whichCat p655]
```

— defun setMsgForcedAttrList —

```
(defun |setMsgForcedAttrList| (msg attrlist)
  (dolist (attr attrlist)
    (|setMsgForcedAttr| msg (|whichCat| attr) attr)))
```

defun setMsgForcedAttr

```
[setMsgCatlessAttr p655]
[ncPutQ p706]
```

— defun setMsgForcedAttr —

```
(defun |setMsgForcedAttr| (msg cat attr)
  (if (eq cat '|catless|)
    (|setMsgCatlessAttr| msg attr)
    (|ncPutQ| msg cat attr)))
```

defvar \$attrCats

— initvars —

```
(defvar |$attrCats| (list '|$imPrGuys| '|$toWhereGuys| '|$repGuys|))
```

defun whichCat

[ListMember? p??]
 [\$attrCats p654]

— defun whichCat —

```
(defun |whichCat| (attr)
  (let ((found '|catless|) done)
    (declare (special |$attrCats|))
    (loop for cat in |$attrCats| do
      (when (|ListMember?| attr (eval cat))
        (setq found cat)
        (setq done t))
      until done)
    found))
```

—————

defun setMsgCatlessAttr

TPDHERE: Changed from —catless— to '—catless—

[ncPutQ p706]
 [ifcdr p??]
 [qassq p??]
 [ncAlist p705]

— defun setMsgCatlessAttr —

```
(defun |setMsgCatlessAttr| (msg attr)
  (|ncPutQ| msg 'catless (cons attr (ifcdr (qassq 'catless (|ncAlist| msg))))))
```

—————

defun putDatabaseStuff

TPDHERE: The variable al is undefined [getMsgInfoFromKey p656]

[setMsgUnforcedAttrList p656]
 [setMsgText p640]

— defun putDatabaseStuff —

```
(defun |putDatabaseStuff| (msg)
  (let (attributes text tmp)
```

```
(setq tmp (|getMsgInfoFromKey| msg))
(setq text (car tmp))
(setq attributes (cadr tmp))
(when attributes (|setMsgUnforcedAttrList| msg attributes))
(|setMsgText| msg text)))
```

defun getMsgInfoFromKey

```
[getMsgKey? p652]
[getErFromDbL p??]
[getMsgKey p638]
[segmentKeyedMsg p44]
[removeAttributes p??]
[substituteSegmentedMsg p??]
[getMsgArgL p639]
[$msgDatabaseName p199]
```

— defun getMsgInfoFromKey —

```
(defun |getMsgInfoFromKey| (msg)
  (let (|$msgDatabaseName| attributes tmp msgText msgKey)
    (declare (special |$msgDatabaseName|))
    (setq |$msgDatabaseName| nil)
    (setq msgText
      (cond
        ((setq msgKey (|getMsgKey?| msg))
         msgKey)
        (t (|getMsgKey| msg))))
    (setq msgText (|segmentKeyedMsg| msgText))
    (setq tmp (|removeAttributes| msgText))
    (setq msgText (car tmp))
    (setq attributes (cadr tmp))
    (setq msgText (|substituteSegmentedMsg| msgText (|getMsgArgL| msg)))
    (list msgText attributes)))
```

defun setMsgUnforcedAttrList

```
[setMsgUnforcedAttr p657]
[whichCat p655]
```

— defun setMsgUnforcedAttrList —

```
(defun |setMsgUnforcedAttrList| (msg attrlist)
  (dolist (attr attrlist)
    (|setMsgUnforcedAttr| msg (|whichCat| attr) attr)))
```

defun setMsgUnforcedAttr

```
[setMsgCatlessAttr p655]
[qassq p??]
[ncAlist p705]
[ncPutQ p706]
```

— defun setMsgUnforcedAttr —

```
(defun |setMsgUnforcedAttr| (msg cat attr)
  (cond
    ((eq cat '|catless|) (|setMsgCatlessAttr| msg attr))
    ((null (qassq cat (|ncAlist| msg))) (|ncPutQ| msg cat attr))))
```

defvar \$imPrTagGuys

— initvars —

```
(defvar |$imPrTagGuys| (list '|unimple| '|bug| '|debug| '|say| '|warn|))
```

defun initImPr

```
[getMsgTag p640]
[setMsgUnforcedAttr p657]
[$imPrTagGuys p657]
[$erMsgToss p??]
```

— defun initImPr —

```
(defun |initImPr| (msg)
  (declare (special |$imPrTagGuys| |$erMsgToss|)))
```



```
(when (or |$errMsgToss| (member (|getMsgTag| msg) |$imPrTagGuys|))
      (|setMsgUnforcedAttr| msg '|$imPrGuys| '|imPr|)))
```

defun initToWhere

```
[getMsgCatAttr p648]
[setMsgUnforcedAttr p657]
```

— defun initToWhere —

```
(defun |initToWhere| (msg)
  (if (member '|trace| (|getMsgCatAttr| msg '|catless|))
      (|setMsgUnforcedAttr| msg '|$toWhereGuys| '|screenOnly|)))
```

defun Report a bug in the compiler

Bug in the compiler: something which shouldn't have happened did.

```
[processKeyedError p643]
[msgCreate p637]
[enable-backtrace p??]
[ncAbort p??]
[$nopos p319]
[$newcompErrorCount p319]
```

— defun ncBug —

```
(defun |ncBug| (errMsgKey erArgL &rest optAttr)
  (let (errMsg)
    (declare (special |$nopos| |$newcompErrorCount|))
    (setq |$newcompErrorCount| (+ |$newcompErrorCount| 1))
    (setq errMsg
      (|processKeyedError|
        (|msgCreate| '|bug| |$nopos| errMsgKey erArgL "Bug!" optAttr)))
    (break)
    (|ncAbort|)))
```

defun processMsgList

```
[erMsgSort p659]
[makeMsgFromLine p661]
[poGlobalLinePosn p364]
[getMsgPos p649]
[queueUpErrors p662]
[listOutputter p644]
[$noRepList p??]
[$outputList p??]
```

— defun processMsgList —

```
(defun |processMsgList| (erMsgList lineList)
  (let (|$noRepList| |$outputList| st globalNumOfLine msgLine)
    (declare (special |$noRepList| |$outputList|))
    (setq |$outputList| nil)
    (setq |$noRepList| nil)
    (setq erMsgList (|erMsgSort| erMsgList))
    (dolist (line lineList)
      (setq msgLine (|makeMsgFromLine| line))
      (setq |$outputList| (cons msgLine |$outputList|))
      (setq globalNumOfLine (|poGlobalLinePosn| (|getMsgPos| msgLine)))
      (setq erMsgList (|queueUpErrors| globalNumOfLine erMsgList)))
    (setq |$outputList| (append erMsgList |$outputList|))
    (setq st "-----SOURCE-TEXT-&-ERRORS-----")
    (|listOutputter| (reverse |$outputList|))))
```

—————

defun erMsgSort

```
[erMsgSep p660]
[listSort p??]
```

— defun erMsgSort —

```
(defun |erMsgSort| (erMsgList)
  (let (msgWOPos msgWPos tmp)
    (setq tmp (|erMsgSep| erMsgList))
    (setq msgWPos (car tmp))
    (setq msgWOPos (cadr tmp))
    (setq msgWPos (|listSort| #'|erMsgCompare| msgWPos))
    (setq msgWOPos (reverse msgWOPos))
    (append msgWPos msgWOPos)))
```

defun erMsgCompare

[compareposns p660]
[getMsgPos p649]

— defun erMsgCompare —

```
(defun |erMsgCompare| (ob1 ob2)
  (|compareposns| (|getMsgPos| ob2) (|getMsgPos| ob1)))
```

defun compareposns

[poGlobalLinePosn p364]
[poCharPosn p667]

— defun compareposns —

```
(defun |compareposns| (a b)
  (let (c d)
    (setq c (|poGlobalLinePosn| a))
    (setq d (|poGlobalLinePosn| b))
    (if (equal c d)
        (not (< (|poCharPosn| a) (|poCharPosn| b)))
        (not (< c d)))))
```

defun erMsgSep

[poNopos? p650]
[getMsgPos p649]

— defun erMsgSep —

```
(defun |erMsgSep| (erMsgList)
  (let (msgWOPos msgWPos)
    (dolist (msg erMsgList)
      (if (|poNopos?| (|getMsgPos| msg))
          (setq msgWOPos (cons msg msgWOPos))
```

```
(setq msgWPos (cons msg msgWPos))))
(list msgWPos msgWOPos))
```

defun makeMsgFromLine

```
[getLinePos p662]
[getLineText p662]
[poGlobalLinePosn p364]
[poLinePosn p651]
[concat p1237]
[rep p661]
[char p??]
[size p1236]
[$preLength p645]
```

— defun makeMsgFromLine —

```
(defun |makeMsgFromLine| (line)
  (let (localNumOfLine stNum globalNumOfLine textOfLine posOfLine)
    (declare (special |$preLength|))
    (setq posOfLine (|getLinePos| line))
    (setq textOfLine (|getLineText| line))
    (setq globalNumOfLine (|poGlobalLinePosn| posOfLine))
    (setq stNum (princ-to-string (|poLinePosn| posOfLine)))
    (setq localNumOfLine
      (concat (|rep| #\space (- |$preLength| 7 (size stNum))) stNum))
    (list '|line| posOfLine nil nil (concat "Line" localNumOfLine) textOfLine)))
```

defun rep

TPDHERE: This function should be replaced by fillerspaces

— defun rep 0 —

```
(defun |rep| (c n)
  (if (< 0 n)
    (make-string n :initial-element (character c))
    ""))
```

defun getLinePos

```

— defun getLinePos 0 —

(defun |getLinePos| (line) (car line))

```

defun getLineText

```

— defun getLineText 0 —

(defun |getLineText| (line) (cdr line))

```

defun queueUpErrors

```

;queueUpErrors(globalNumOfLine,msgList)==
;  thisPosMsgs := []
;  notThisLineMsgs := []
;  for msg in msgList _
;    while thisPosIsLess(getMsgPos msg,globalNumOfLine) repeat
;      --these are msgs that refer to positions from earlier compilations
;      if not redundant (msg,notThisPosMsgs) then
;        notThisPosMsgs := [msg,:notThisPosMsgs]
;      msgList := rest msgList
;  for msg in msgList _
;    while thisPosIsEqual(getMsgPos msg,globalNumOfLine) repeat
;      if not redundant (msg,thisPosMsgs) then
;        thisPosMsgs := [msg,:thisPosMsgs]
;      msgList := rest msgList
;  if thisPosMsgs then
;    thisPosMsgs := processChPosesForOneLine thisPosMsgs
;    $outputList := NCONC(thisPosMsgs,$outputList)
;  if notThisPosMsgs then
;    $outputList := NCONC(notThisPosMsgs,$outputList)
;  msgList

```

```

[processChPosesForOneLine p666]
[$outputList p??]

```

```

— defun queueUpErrors —

```

```

(DEFUN |queueUpErrors| (|globalNumOfLine| |msgList|)
  (PROG (|notThisPosMsgs| |notThisLineMsgs| |thisPosMsgs|)
    (DECLARE (SPECIAL |$outputList|))
    (RETURN
      (PROGN
        (SETQ |thisPosMsgs| NIL)
        (SETQ |notThisLineMsgs| NIL)
        ((LAMBDA (|bfVar#7| |msg|)
          (LOOP
            (COND
              ((OR (ATOM |bfVar#7|)
                (PROGN (SETQ |msg| (CAR |bfVar#7|)) NIL)
                (NOT (|thisPosIsLess| (|getMsgPos| |msg|)
                  |globalNumOfLine|))))
              (RETURN NIL)))
          'T
          (PROGN
            (COND
              ((NULL (|redundant| |msg| |notThisPosMsgs|))
                (SETQ |notThisPosMsgs|
                  (CONS |msg| |notThisPosMsgs|))))
              (SETQ |msgList| (CDR |msgList|))))
            (SETQ |bfVar#7| (CDR |bfVar#7|))))
          |msgList| NIL)
        ((LAMBDA (|bfVar#8| |msg|)
          (LOOP
            (COND
              ((OR (ATOM |bfVar#8|)
                (PROGN (SETQ |msg| (CAR |bfVar#8|)) NIL)
                (NOT (|thisPosIsEqual| (|getMsgPos| |msg|)
                  |globalNumOfLine|))))
              (RETURN NIL)))
          'T
          (PROGN
            (COND
              ((NULL (|redundant| |msg| |thisPosMsgs|))
                (SETQ |thisPosMsgs| (CONS |msg| |thisPosMsgs|))))
              (SETQ |msgList| (CDR |msgList|))))
            (SETQ |bfVar#8| (CDR |bfVar#8|))))
          |msgList| NIL)
        (COND
          (|thisPosMsgs|
            (SETQ |thisPosMsgs|
              (|processChPosesForOneLine| |thisPosMsgs|))
            (SETQ |$outputList| (NCONC |thisPosMsgs| |$outputList|))))
          (COND
            (|notThisPosMsgs|
              (SETQ |$outputList|
                (NCONC |notThisPosMsgs| |$outputList|))))
            |msgList|))))

```

defun thisPosIsLess

```
[poNopos? p650]
[poGlobalLinePosn p364]
```

— defun thisPosIsLess —

```
(defun |thisPosIsLess| (pos num)
  (unless (|poNopos?| pos) (< (|poGlobalLinePosn| pos) num)))
```

defun thisPosIsEqual

```
[poNopos? p650]
[poGlobalLinePosn p364]
```

— defun thisPosIsEqual —

```
(defun |thisPosIsEqual| (pos num)
  (unless (|poNopos?| pos) (equal (|poGlobalLinePosn| pos) num)))
```

defun redundant

```
redundant(msg,thisPosMsgs) ==
  found := NIL
  if msgNoRep? msg then
    for item in $noRepList repeat
      sameMsg?(msg,item) => return (found := true)
    $noRepList := [msg,$noRepList]
  found or MEMBER(msg,thisPosMsgs)
```

```
[msgNoRep? p665]
[sameMsg? p666]
[$noRepList p??]
```

— defun redundant —

```
(defun |redundant| (msg thisPosMsgs)
  (prog (found)
    (declare (special |$noRepList|))
    (return
      (progn
        (cond
          ((|msgNoRep?| msg)
            ((lambda (Var9 item)
              (loop
                (cond
                  ((or (atom Var9) (progn (setq item (car Var9)) nil))
                    (return nil))
                  (t
                     (cond
                       ((|sameMsg?| msg item) (return (setq found t))))
                    (setq Var9 (cdr Var9))))
                |$noRepList| nil)
              (setq |$noRepList| (list msg |$noRepList|)))
            (or found (member msg thisPosMsgs))))))
```

defvar \$repGuys

— initvars —

```
(defvar |$repGuys| (list '|noRep| '|rep|))
```

defun msgNoRep?

[getMsgCatAttr [p648](#)]

— defun msgNoRep? —

```
(defun |msgNoRep?| (msg) (eq (|getMsgCatAttr| msg '|$repGuys|) '|noRep|))
```

defun sameMsg?

```
[getMsgKey p638]
[getMsgArgL p639]
```

— defun sameMsg? —

```
(defun |sameMsg?| (msg1 msg2)
  (and (equal (|getMsgKey| msg1) (|getMsgKey| msg2))
       (equal (|getMsgArgL| msg1) (|getMsgArgL| msg2))))
```

defun processChPosesForOneLine

```
[posPointers p668]
[getMsgFTTag? p649]
[putFTText p669]
[poCharPosn p667]
[getMsgPos p649]
[getMsgPrefix p639]
[setMsgPrefix p639]
[concat p1237]
[size p1236]
[makeLeaderMsg p667]
[$preLength p645]
```

— defun processChPosesForOneLine —

```
(defun |processChPosesForOneLine| (msgList)
  (let (leaderMsg oldPre posLetter chPosList)
    (declare (special |$preLength|))
    (setq chPosList (|posPointers| msgList))
    (dolist (msg msgList)
      (when (|getMsgFTTag?| msg) (|putFTText| msg chPosList))
      (setq posLetter (cdr (assoc (|poCharPosn| (|getMsgPos| msg)) chPosList)))
      (setq oldPre (|getMsgPrefix| msg))
      (|setMsgPrefix| msg
        (concat oldPre
          (make-string (- |$preLength| 4 (size oldPre)) posLetter)))
      (setq leaderMsg (|makeLeaderMsg| chPosList))
      (nconc msgList (list leaderMsg))))
```

defun poCharPosn

— defun poCharPosn 0 —

```
(defun |poCharPosn| (posn)
  (cdr posn))
```

—————

defun makeLeaderMsg

```
makeLeaderMsg chPosList ==
  st := MAKE_-FULL_-CVEC ($preLength- 3)
  oldPos := -1
  for [posNum, posLetter] in reverse chPosList repeat
    st := CONCAT(st, _
      rep(char ".", (posNum - oldPos - 1)), posLetter)
  oldPos := posNum
  ['leader, $nopus, 'nokey, NIL, NIL, [st] ]
```

[[\\$nopus p319](#)]
 [[\\$preLength p645](#)]

— defun makeLeaderMsg —

```
(defun |makeLeaderMsg| (chPosList)
  (let (posLetter posNum oldPos st)
    (declare (special |$nopus| |$preLength|))
    (setq st (make-string (- |$preLength| 3)))
    (setq oldPos -1)
    (lambda (Var15 Var14)
      (loop
        (cond
          ((or (atom Var15) (progn (setq Var14 (car Var15)) nil))
            (return nil))
          (t
            (and (consp Var14)
              (progn
                (setq posNum (car Var14))
                (setq posLetter (cdr Var14))
                t)
              (progn
                (setq st
                  (concat st (|rep| #\. (- posNum oldPos 1)) posLetter))
                (setq oldPos posNum))))))
      (setq Var15 (cdr Var15))))))
```

```
(reverse chPosList) nil)
(list '|leader| |$npos| '|nokey| nil nil (list st))))
```

defun posPointers

TPDHERE: getMsgFTTag is nonsense

```
[poCharPosn p667]
[getMsgPos p649]
[ifcar p??]
[getMsgPos2 p668]
[insertPos p669]
[getMsgFTTag p??]
```

— defun posPointers —

```
(defun |posPointers| (msgList)
  (let (posLetterList pos ftPosList posList increment pointers)
    (declare (special |getMsgFTTag|))
    (setq pointers "ABCDEFGHJKLMNOPQRS")
    (setq increment 0)
    (dolist (msg msgList)
      (setq pos (|poCharPosn| (|getMsgPos| msg)))
      (unless (equal pos (ifcar posList))
        (setq posList (cons pos posList)))
      ; this should probably read TPDHERE
      ; (when (eq (|getMsgPosTagOb| msg) 'fromto))
      (when (eq |getMsgFTTag| 'fromto)
        (setq ftPosList (cons (|poCharPosn| (|getMsgPos2| msg)) ftPosList))))
    (dolist (toPos ftPosList)
      (setq posList (|insertPos| toPos posList)))
    (dolist (pos posList)
      (setq posLetterList
        (cons (cons pos (elt pointers increment)) posLetterList))
      (setq increment (+ increment 1)))
    posLetterList))
```

defun getMsgPos2

```
[getMsgFTTag? p649]
[getMsgPosTagOb p638]
```

[ncBug p658]

— defun getMsgPos2 —

```
(defun |getMsgPos2| (msg)
  (if (|getMsgFTTag?| msg)
      (caddr (|getMsgPosTag0b| msg))
      (|ncBug| "not a from to" nil)))
```

—————

defun insertPos

This function inserts a position in the proper place of a position list. This is used for the 2nd pos of a fromto [done p??]

— defun insertPos 0 —

```
(defun |insertPos| (newPos posList)
  (let (pos top bot done)
    (setq bot (cons 0 posList))
    (do () (done)
      (setq top (cons (car bot) top))
      (setq bot (cdr bot))
      (setq pos (car bot))
      (setq done
        (cond
         ((< pos newPos) nil)
         ((equal pos newPos) t)
         ((< newPos pos)
          (setq top (cons newPos top))
          t))))
    (cons (cdr (reverse top)) bot)))
```

—————

defun putFTText

```
[getMsgFTTag? p649]
[poCharPosn p667]
[getMsgPos p649]
[setMsgText p640]
[getMsgText p639]
[getMsgPos2 p668]
```

— **defun putFText** —

```
(defun |putFText| (msg chPosList)
  (let (charMarker2 pos2 markingText charMarker pos tag)
    (setq tag (|getMsgFTag?| msg))
    (setq pos (|poCharPosn| (|getMsgPos| msg)))
    (setq charMarker (cdr (assoc pos chPosList)))
    (cond
      ((eq tag 'from)
       (setq markingText (list " (from " charMarker " and on) "))
       (|setMsgText| msg (append markingText (|getMsgText| msg))))
      ((eq tag 'to)
       (setq markingText (list " (up to " charMarker " ) " ))
       (|setMsgText| msg (append markingText (|getMsgText| msg))))
      ((eq tag 'fromto)
       (setq pos2 (|poCharPosn| (|getMsgPos2| msg)))
       (setq charMarker2 (cdr (assoc pos2 chPosList)))
       (setq markingText (list " (from " charMarker " up to " charMarker2 " ) " ))
       (|setMsgText| msg (append markingText (|getMsgText| msg)))))))
```

defun From

This is called from parameter list of nc message functions

— **defun From 0** —

```
(defun |From| (pos) (list 'from pos))
```

defun To

This is called from parameter list of nc message functions

— **defun To 0** —

```
(defun |To| (pos) (list 'to pos))
```

defun FromTo

This is called from parameter list of nc message functions

— **defun FromTo 0** —

```
(defun |FromTo| (pos1 pos2) (list 'fromto pos1 pos2))
```

Chapter 21

The Interpreter Syntax

21.1 syntax assignment

— assignment.help —

Immediate, Delayed, and Multiple Assignment

```
=====
Immediate Assignment
=====
```

A variable in Axiom refers to a value. A variable has a name beginning with an uppercase or lowercase alphabetic character, "%", or "!". Successive characters (if any) can be any of the above, digits, or "?". Case is distinguished. The following are all examples of valid, distinct variable names:

a	tooBig?	a1B2c3%!?
A	%j	numberOfPoints
beta6	%J	numberofpoints

The "!=" operator is the immediate assignment operator. Use it to associate a value with a variable. The syntax for immediate assignment for a single variable is:

```
variable := expression
```

The value returned by an immediate assignment is the value of expression.

```
a := 1
1
Type: PositiveInteger
```


The right-hand side of the expression is evaluated, yielding 1. The value is then assigned to a.

```
b := a
1
      Type: PositiveInteger
```

The right-hand side of the expression is evaluated, yielding 1. This value is then assigned to b. Thus a and b both have the value 1 after the sequence of assignments.

```
a := 2
2
      Type: PositiveInteger
```

What is the value of b if a is assigned the value 2?

```
b
1
      Type: PositiveInteger
```

The value of b is left unchanged.

This is what we mean when we say this kind of assignment is immediate. The variable b has no dependency on a after the initial assignment. This is the usual notion of assignment in programming languages such as C, Pascal, and Fortran.

```
=====
Delayed Assignment
=====
```

Axiom provides delayed assignment with "==". This implements a delayed evaluation of the right-hand side and dependency checking. The syntax for delayed assignment is

```
variable == expression
```

The value returned by a delayed assignment is the unique value of Void.

```
a == 1
      Type: Void

b == a
      Type: Void
```

Using a and b as above, these are the corresponding delayed assignments.

```
a
```

```

Compiling body of rule a to compute value of type PositiveInteger
1

```

```

    Type: PositiveInteger

```

The right-hand side of each delayed assignment is left unevaluated until the variables on the left-hand sides are evaluated.

```

b
Compiling body of rule b to compute value of type PositiveInteger
1

```

```

    Type: PositiveInteger

```

This gives the same results as before. But if we change a to 2

```

a == 2
Compiled code for a has been cleared.
Compiled code for b has been cleared.
1 old definition(s) deleted for function or rule a
    Type: Void

```

Then a evaluates to 2, as expected

```

a
Compiling body of rule a to compute value of type PositiveInteger
2
    Type: PositiveInteger

```

but the value of b reflects the change to a

```

b
Compiling body of rule b to compute value of type PositiveInteger
2
    Type: PositiveInteger

```

Multiple Immediate Assignments

It is possible to set several variables at the same time by using a tuple of variables and a tuple of expressions. A tuple is a collection of things separated by commas, often surrounded by parentheses. The syntax for multiple immediate assignment is

```

( var1, var2, ..., varN ) := ( expr1, expr2, ..., exprN )

```

The value returned by an immediate assignment is the value of exprN.

```

( x, y ) := ( 1, 2 )
2
    Type: PositiveInteger

```

This sets `x` to 1 and `y` to 2. Multiple immediate assignments are parallel in the sense that the expressions on the right are all evaluated before any assignments on the left are made. However, the order of evaluation of these expressions is undefined.

```
( x, y ) := ( y, x )
1
      Type: PositiveInteger
```

```
x
2
      Type: PositiveInteger
```

The variable `x` now has the previous value of `y`.

```
y
1
      Type: PositiveInteger
```

The variable `y` now has the previous value of `x`.

There is no syntactic form for multiple delayed assignments.

21.2 syntax blocks

— blocks.help —

```
=====
Blocks
=====
```

A block is a sequence of expressions evaluated in the order that they appear, except as modified by control expressions such as `leave`, `return`, `iterate`, and `if-then-else` constructions. The value of a block is the value of the expression last evaluated in the block.

To leave a block early, use `"=>"`. For example,

```
i < 0 => x
```

The expression before the `"=>"` must evaluate to true or false. The expression following the `"=>"` is the return value of the block.

A block can be constructed in two ways:

1. the expressions can be separated by semicolons and the resulting expression surrounded by parentheses, and
2. the expressions can be written on succeeding lines with each line indented the same number of spaces (which must be greater than zero).
A block entered in this form is called a pile

Only the first form is available if you are entering expressions directly to Axiom. Both forms are available in .input files. The syntax for a simple block of expressions entered interactively is

```
( expression1 ; expression2 ; ... ; expressionN )
```

The value returned by a block is the value of an "=>" expression, or expressionN if no "=>" is encountered.

In .input files, blocks can also be written in piles. The examples given here are assumed to come from .input files.

```
a :=
  i := gcd(234,672)
  i := 2*i**5 - i + 1
  1 / i

      1
      ----
    23323

Type: Fraction Integer
```

In this example, we assign a rational number to a using a block consisting of three expressions. This block is written as a pile. Each expression in the pile has the same indentation, in this case two spaces to the right of the first line.

```
a := ( i := gcd(234,672); i := 2*i**5 - i + 1; 1 / i )

      1
      ----
    23323

Type: Fraction Integer
```

Here is the same block written on one line. This is how you are required to enter it at the input prompt.

```
( a := 1; b := 2; c := 3; [a,b,c] )
[1,2,3]

Type: List PositiveInteger
```

AAxiom gives you two ways of writing a block and the preferred way in

an `.input` file is to use a pile. Roughly speaking, a pile is a block whose constituent expressions are indented the same amount. You begin a pile by starting a new line for the first expression, indenting it to the right of the previous line. You then enter the second expression on a new line, vertically aligning it with the first line. And so on. If you need to enter an inner pile, further indent its lines to the right of the outer pile. Axiom knows where a pile ends. It ends when a subsequent line is indented to the left of the pile or the end of the file.

Also See:

- o `)help if`
- o `)help repeat`
- o `)help while`
- o `)help for`
- o `)help suchthat`
- o `)help parallel`
- o `)help lists`

1

21.3 system clef

— `clef.help` —

Entering printable keys generally inserts new text into the buffer (unless in overwrite mode, see below). Other special keys can be used to modify the text in the buffer. In the description of the keys below, `^n` means Control-n, or holding the CONTROL key down while pressing "n". Errors will ring the terminal bell.

- `^A/^E` : Move cursor to beginning/end of the line.
- `^F/^B` : Move cursor forward/backward one character.
- `^D` : Delete the character under the cursor.
- `^H, DEL` : Delete the character to the left of the cursor.
- `^K` : Kill from the cursor to the end of line.
- `^L` : Redraw current line.
- `^O` : Toggle overwrite/insert mode. Initially in insert mode. Text added in overwrite mode (including yanks) overwrite existing text, while insert mode does not overwrite.
- `^P/^N` : Move to previous/next item on history list.
- `^R/^S` : Perform incremental reverse/forward search for string on the history list. Typing normal characters adds to the current

¹ "if" (21.6 p 685) "repeat" (21.10 p 692) "while" (37.1 p 1230) "for" (21.5 p 681) "suchthat" (21.11 p 696) "parallel" (21.9 p 689) "lists" (?? p ??)

search string and searches for a match. Typing `^R/^S` marks the start of a new search, and moves on to the next match. Typing `^H` or `DEL` deletes the last character from the search string, and searches from the starting location of the last search. Therefore, repeated `DEL`'s appear to unwind to the match nearest the point at which the last `^R` or `^S` was typed. If `DEL` is repeated until the search string is empty the search location begins from the start of the history list. Typing `ESC` or any other editing character accepts the current match and loads it into the buffer, terminating the search.

`^T` : Toggle the characters under and to the left of the cursor.
`^Y` : Yank previously killed text back at current location. Note that this will overwrite or insert, depending on the current mode.
`^U` : Show help (this text).
`TAB` : Perform command completion based on word to the left of the cursor. Words are deemed to contain only the alphanumeric and the `% ! ? _` characters.
`NL, CR` : returns current buffer to the program.

DOS and ANSI terminal arrow key sequences are recognized, and act like:

`up` : same as `^P`
`down` : same as `^N`
`left` : same as `^B`
`right` : same as `^F`

21.4 syntax collection

— collection.help —

```
=====
Collection -- Creating Lists and Streams with Iterators
=====
```

All of the loop expressions which do not use the `repeat` `leave` or `iterate` words can be used to create lists and streams. For example:

This creates a simple list of the integers from 1 to 10:

```
list := [i for i in 1..10]
[1,2,3,4,5,6,7,8,9,10]
Type: List PositiveInteger
```

Create a stream of the integers greater than or equal to 1:

```
stream := [i for i in 1..]
[1,2,3,4,5,6,7,...]
Type: Stream PositiveInteger
```

This is a list of the prime numbers between 1 and 10, inclusive:

```
[i for i in 1..10 | prime? i]
[2,3,5,7]
Type: List PositiveInteger
```

This is a stream of the prime integers greater than or equal to 1:

```
[i for i in 1.. | prime? i]
[2,3,5,7,11,13,17,...]
Type: Stream PositiveInteger
```

This is a list of the integers between 1 and 10, inclusive, whose squares are less than 700:

```
[i for i in 1..10 while i*i < 700]
[1,2,3,4,5,6,7,8,9,10]
Type: List PositiveInteger
```

This is a stream of the integers greater than or equal to 1 whose squares are less than 700:

```
[i for i in 1.. while i*i < 700]
[1,2,3,4,5,6,7,...]
Type: Stream PositiveInteger
```

The general syntax of a collection is

```
[ collectExpression iterator1 iterator2 ... iteratorN ]
```

where each iterator is either a for or a while clause. The loop terminates immediately when the end test of any iterator succeeds or when a return expression is evaluated in collectExpression. The value returned by the collection is either a list or a stream of elements, one for each iteration of the collectExpression.

Be careful when you use while to create a stream. By default Axiom tries to compute and display the first ten elements of a stream. If the while condition is not satisfied quickly, Axiom can spend a long (potentially infinite) time trying to compute the elements. Use

```
)set streams calculate
```

to change the defaults to something else. This also affects the number of terms computed and displayed for power series. For the purposes of

these examples we have use this system command to display fewer than ten terms.

21.5 syntax for

— for.help —

```
=====
for loops
=====
```

Axiom provide the `for` and `in` keywords in repeat loops, allowing you to integrate across all elements of a list, or to have a variable take on integral values from a lower bound to an upper bound. We shall refer to these modifying clauses of repeat loops as `for` clauses. These clauses can be present in addition to `while` clauses (See `)help while`). As with all other types of repeat loops, `leave` (see `)help leave`) can be used to prematurely terminate evaluation of the loop.

The syntax for a simple loop using `for` is

```
for iterator repeat loopbody
```

The iterator has several forms. Each form has an end test which is evaluated before `loopbody` is evaluated. A `for` loop terminates immediately when the end test succeeds (evaluates to true) or when a `leave` or `return` expression is evaluated in `loopbody`. The value returned by the loop is the unique value of `Void`.

```
=====
for i in n..m repeat
=====
```

If `for` is followed by a variable name, the `in` keyword and then an integer segment of the form `n..m`, the end test for this loop is the predicate `i > m`. The body of the loop is evaluated `m-n+1` times if this number is greater than 0. If this number is less than or equal to 0, the loop body is not evaluated at all.

The variable `i` has the value `n`, `n+1`, ..., `m` for successive iterations of the loop body. The loop variable is a local variable within the loop body. Its value is not available outside the loop body and its value and type within the loop body completely mask any outer definition of a variable with the same name.


```

for i in 10..12 repeat output(i**3)
1000
1331
1728

```

Type: Void

The loop prints the values of 10^3 , 11^3 , and 12^3 .

```

a := [1,2,3]
[1,2,3]

```

Type: List PositiveInteger

```

for i in 1..#a repeat output(a.i)
1
2
3

```

Type: Void

Iterate across this list using "." to access the elements of a list and the # operation to count its elements.

This type of iteration is applicable to anything that uses ".". You can also use it with functions that use indices to extract elements.

```

m := matrix [ [1,2],[4,3],[9,0] ]
+-      +-
| 1  2 |
| 4  3 |
| 9  0 |
+-      +-

```

Type: Matrix Integer

Define m to be a matrix.

```

for i in 1..nrows(m) repeat output row(m.i)
[1,2]
[4,3]
[9,0]

```

Type: Void

Display the rows of m.

You can iterate with for-loops.

```

for i in 1..5 repeat
  if odd?(i) then iterate
  output(i)
2
4

```

Type: Void

Display the even integers in a segment.

```
=====
for i in n..m by s repeat
=====
```

By default, the difference between values taken on by a variable in loops such as

```
for i in n..m repeat ...
```

is 1. It is possible to supply another, possibly negative, step value by using the `by` keyword along with `for` and `in`. Like the upper and lower bounds, the step value following the `by` keyword must be an integer. Note that the loop

```
for i in 1..2 by 0 repeat output(i)
```

will not terminate by itself, as the step value does not change the index from its initial value of 1.

```
for i in 1..5 by 2 repeat output(i)
1
3
5
```

Type: Void

This expression displays the odd integers between two bounds.

```
for i in 5..1 by -2 repeat output(i)
5
3
1
```

Type: Void

Use this to display the numbers in reverse order.

```
=====
for i in n.. repeat
=====
```

If the value after the `".."` is omitted, the loop has no end test. A potentially infinite loop is thus created. The variable is given the successive values `n`, `n+1`, `n+2`, ... and the loop is terminated only if a `leave` or `return` expression is evaluated in the loop body. However, you may also add some other modifying clause on the `repeat`, for example, a `while` clause, to stop the loop.

```

for i in 15.. while not prime?(i) repeat output(i)
15
16

```

Type: Void

This loop displays the integers greater than or equal to 15 and less than the first prime number greater than 15.

```

=====
for x in l repeat
=====

```

Another variant of the for loop has the form:

```

for x in list repeat loopbody

```

This form is used when you want to iterate directly over the elements of a list. In this form of the for loop, the variable *x* takes on the value of each successive element in *l*. The end test is most simply stated in English: "are there no more *x* in *l*?"

```

l := [0, -5, 3]
[0, -5, 3]

```

Type: List Integer

```

for x in l repeat output(x)
0
-5
3

```

Type: Void

This displays all of the elements of the list *l*, one per line.

Since the list constructing expression

```

expand [n..m]

```

creates the list

```

[n, n+1, ..., m]

```

you might be tempted to think that the loops

```

for i in n..m repeat output(i)

```

and

```

for x in expand [n..m] repeat output(x)

```

are equivalent. The second form first creates the expanded list

(no matter how large it might be) and then does the iteration. The first form potentially runs in much less space, as the index variable `i` is simply incremented once per loop and the list is not actually created. Using the first form is much more efficient.

Of course, sometimes you really want to iterate across a specific list. This displays each of the factors of 2400000:

```
for f in factors(factor(2400000)) repeat output(f)
[factor= 2, exponent= 8]
[factor= 3, exponent= 1]
[factor= 5, exponent= 5]
Type: Void
```

21.6 syntax if

— if.help —

```
=====
If-then-else
=====
```

Like many other programming languages, Axiom uses the three keywords `if`, `then`, and `else` to form conditional expressions. The `else` part of the conditional is optional. The expression between the `if` and `then` keywords is a predicate: an expression that evaluates to or is convertible to either `true` or `false`, that is, a `Boolean`.

The syntax for conditional expressions is

```
if predicate then expression1 else expression2
```

where the "`else expression2`" part is optional. The value returned from a conditional expression is `expression1` if the predicate evaluates to `true` and `expression2` otherwise. If no `else` clause is given, the value is always the unique value of `Void`.

An `if-then-else` expression always returns a value. If the `else` clause is missing then the entire expression returns the unique value of `Void`. If both clauses are present, the type of the value returned by `if` is obtained by resolving the types of the values of the two clauses.

The predicate must evaluate to, or be convertible to, an object of type `Boolean`: `true` or `false`. By default, the equal sign `"=`" creates an equation.

```
x + 1 = y
x + 1 = y
```

Type: Equation Polynomial Integer

This is an equation, not a boolean condition. In particular, it is an object of type Equation Polynomial Integer.

However, for predicates in if expressions, Axiom places a default target type of Boolean on the predicate and equality testing is performed. Thus you need not qualify the "=" in any way. In other contexts you may need to tell Axiom that you want to test for equality rather than create an equation. In these cases, use "@" and a target type of Boolean.

The compound symbol meaning "not equal" in Axiom is "~=". This can be used directly without a package call or a target specification. The expression "a ~= b" is directly translated to "not(a = b)".

Many other functions have return values of type Boolean. These include <, <=, >, >=, ~=, and member?. By convention, operations with names ending in "?" return Boolean values.

The usual rules for files are suspended for conditional expressions. In .input files, the then and else keywords can begin in the same column as the corresponding if by may also appear to the right. Each of the following styles of writing if-then-else expressions is acceptable:

```
if i>0 then output("positive") else output("nonpositive")
```

```
if i>0 then output("positive")
  else output("nonpositive")
```

```
if i>0 then output("positive")
else output("nonpositive")
```

```
if i>0
then output("positive")
else output("nonpositive")
```

```
if i>0
  then output("positive")
  else output("nonpositive")
```

A block can follow the then or else keywords. In the following two assignments to a, the then and else clauses each are followed by two line files. The value returned in each is the value of the second line.

```
a :=
  if i > 0 then
    j := sin(i * pi())
```

```

    exp(j + 1/j)
  else
    j := cos(i * 0.5 * pi())
    log(abs(j)**5 + i)

a :=
  if i > 0
  then
    j := sin(i * pi())
    exp(j + 1/j)
  else
    j := cos(i * 0.5 * pi())
    log(abs(j)**5 + i)

```

These are both equivalent to the following:

```

a :=
  if i > 0 then (j := sin(i * pi()); exp(j + 1/j))
  else (j := cos(i * 0.5 * pi()); log(abs(j)**5 + i))

```

—

21.7 syntax iterate

— iterate.help —

```

=====
iterate in loops
=====

```

Axiom provides an `iterate` expression that skips over the remainder of a loop body and starts the next loop execution. We first initialize a counter.

```

i := 0
0
                                     Type: NonNegativeInteger

```

Display the even integers from 2 to 5:

```

repeat
  i := i + 1
  if i > 5 then leave
  if odd?(i) then iterate
  output(i)

```



```

repeat
  repeat
    if (i + j) > 10 then leave
    j := j + 1
  if (i + j) > 10 then leave
  i := i + 1

```

Type: Void

Nested loops must have multiple leave expressions at the appropriate nesting level. How would you rewrite this so $(i + j) > 10$ is only evaluated once?

```

=====
leave vs => in loop bodies
=====

```

Compare the following two loops:

<pre> i := 1 repeat i := i + 1 i > 3 => i output(i) </pre>	<pre> i := 1 repeat i := i + 1 if i > 3 then leave output(i) </pre>
--	--

In the example on the left, the values 2 and 3 for i are displayed but then the " $\Rightarrow$ " does not allow control to reach the call to `output` again. The loop will not terminate until you run out of space or interrupt the execution. The variable i will continue to be incremented because the " $\Rightarrow$ " only means to leave the block, not the loop.

In the example on the right, upon reaching 4, the leave will be executed, and both the block and the loop will terminate. This is one of the reasons why both " $\Rightarrow$ " and `leave` are provided. Using a `while` clause with the " $\Rightarrow$ " lets you simulate the action of `leave`.

—

21.9 syntax parallel

— parallel.help —

```

=====
parallel iteration
=====

```

Sometimes you want to iterate across two lists in parallel, or perhaps

you want to traverse a list while incrementing a variable.

The general syntax of a repeat loop is

```
iterator1, iterator2, ..., iteratorN repeat loopbody
```

where each iterator is either a for or a while clause. The loop terminates immediately when the end test of any iterator succeeds or when a leave or return expression is evaluated in loopbody. The value returned by the loop is the unique value of Void.

```
l := [1,3,5,7]
[1,3,5,7]
Type: List PositiveInteger
```

```
m := [100,200]
[100,200]
Type: List PositiveInteger
```

```
sum := 0
0
Type: NonNegativeInteger
```

Here we write a loop to iterate across two lists, computing the sum of the pairwise product of the elements:

```
for x in l for y in m repeat
  sum := sum + x*y
Type: Void
```

The last two elements of `l` are not used in the calculation because `m` has two fewer elements than `l`.

```
sum
700
Type: NonNegativeInteger
```

This is the "dot product".

Next we write a loop to compute the sum of the products of the loop elements with their positions in the loop.

```
l := [2,3,5,7,11,13,17,19,23,29,31,37]
[2,3,5,7,11,13,17,19,23,29,31,37]
Type: List PositiveInteger

sum := 0
0
Type: NonNegativeInteger
```

```

for i in 0.. for x in l repeat sum := i * x
                                Type: Void

```

Here looping stops when the list `l` is exhausted, even though the `for i in 0..` specifies no terminating condition.

```

sum
407
                                Type: NonNegativeInteger

```

When `"|"` is used to qualify any of the `for` clauses in a parallel iteration, the variables in the predicates can be from an outer scope or from a `for` clause in or to the left of the modified clause.

This is correct:

```

for i in 1..10 repeat
  for j in 200..300 | odd? (i+j) repeat
    output [i,j]

```

But this is not correct. The variable `j` has not been defined outside the inner loop:

```

for i in 1..01 | odd? (i+j) repeat -- wrong, j not defined
  for j in 200..300 repeat
    output [i,j]

```

It is possible to mix several of repeat modifying clauses on a loop:

```

for i in 1..10
  for j in 151..160 | odd? j
    while i + j < 160 repeat
      output [i,j]
[1,151]
[3,153]
                                Type: Void

```

Here are useful rules for composing loop expressions:

1. while predicates can only refer to variables that are global (or in an outer scope) or that are defined in `for` clauses to the left of the predicate.
2. A "such that" predicate (something following `"|"`) must directly follow a `for` clause and can only refer to variables that are global (or in an outer scope) or defined in the modified `for` clause or any `for` clause to the left.

21.10 syntax repeat

— repeat.help —

Repeat Loops

A loop is an expression that contains another expression, called the loop body, which is to be evaluated zero or more times. All loops contain the repeat keyword and return the unique value of Void. Loops can contain inner loops to any depth.

The most basic loop is of the form

```
repeat loopbody
```

Unless loopbody contains a leave or return expression, the loop repeats forever. The value returned by the loop is the unique value of Void.

Axiom tries to determine completely the type of every object in a loop and then to translate the loop body to Lisp or even to machine code. This translation is called compilation.

If Axiom decides that it cannot compile the loop, it issues a message stating the problem and then the following message:

```
We will attempt to step through and interpret the code
```

It is still possible that Axiom can evaluate the loop but in interpret-code mode.

Return in Loops

A return expression is used to exit a function with a particular value. In particular, if a return is in a loop within the function, the loop is terminated whenever the return is evaluated.

```
f() ==
  i := 1
  repeat
    if factorial(i) > 1000 then return i
    i := i + 1
```

Type: Void

```
f()
```

Type: Void

When factorial(i) is big enough, control passes from inside the loop all the way outside the function, returning the value of i (so we think). What went wrong? Isn't it obvious that this function should return an integer? Well, Axiom makes no attempt to analyze the structure of a loop to determine if it always returns a value because, in general, this is impossible. So Axiom has this simple rule: the type of the function is determined by the type of its body, in this case a block. The normal value of a block is the value of its last expression, in this case, a loop. And the value of every loop is the unique value of Void. So the return type of f is Void.

There are two ways to fix this. The best way is for you to tell Axiom what the return type of f is. You do this by giving f a declaration

```
f:() -> Integer
```

prior to calling for its value. This tells Axiom "trust me -- an integer is returned". Another way is to add a dummy expression as follows.

```
f() ==
  i := 1
  repeat
    if factorial(i) > 1000 then return i
    i := i + 1
  0
```

Type: Void

Note that the dummy expression will never be evaluated but it is the last expression in the function and will determine the return type.

```
f()
7
```

Type: PositiveInteger

```
=====
leave in loops
=====
```

The leave keyword is often more useful in terminating a loop. A leave causes control to transfer to the expression immediately following the loop. As loops always return the unique value of Void, you cannot return a value with leave. That is, leave takes no argument.

```
f() ==
  i := 1
  repeat
    if factorial(i) > 1000 then leave
    i := i + 1
```

```
i
Type: Void
```

This example is a modification of the last example in the previous section. Instead of using return we'll use leave.

```
f()
7
Type: PositiveInteger
```

The loop terminates when factorial(i) gets big enough. The last line of the function evaluates to the corresponding "good" value of i and the function terminates, returning that value.

You can only use leave to terminate the evaluation of one loop. Lets consider a loop within a loop, that is, a loop with a nested loop. First, we initialize two counter variables.

```
(i,j) := (1,1)
1
Type: PositiveInteger

repeat
  repeat
    if (i + j) > 10 then leave
    j := j + 1
  if (i + j) > 10 then leave
  i := i + 1
Type: Void
```

Nested loops must have multiple leave expressions at the appropriate nesting level. How would you rewrite this so (i + j) > 10 is only evaluated once?

```
=====
leave vs => in loop bodies
=====
```

Compare the following two loops:

i := 1	i := 1
repeat	repeat
i := i + 1	i := i + 1
i > 3 => i	if i > 3 then leave
output(i)	output(i)

In the example on the left, the values 2 and 3 for i are displayed but then the "=>" does not allow control to reach the call to output again. The loop will not terminate until you run out of space or interrupt the execution. The variable i will continue to be incremented because the

"=>" only means to leave the block, not the loop.

In the example on the right, upon reaching 4, the leave will be executed, and both the block and the loop will terminate. This is one of the reasons why both "=>" and leave are provided. Using a while clause with the "=>" lets you simulate the action of leave.

```
=====
iterate in loops
=====
```

Axiom provides an iterate expression that skips over the remainder of a loop body and starts the next loop execution. We first initialize a counter.

```
i := 0
0
                                     Type: NonNegativeInteger
```

Display the even integers from 2 to 5:

```
repeat
  i := i + 1
  if i > 5 then leave
  if odd?(i) then iterate
  output(i)
2
4
                                     Type: Void
```

Also See:

```
o )help blocks
o )help if
o )help while
o )help for
o )help suchthat
o )help parallel
o )help lists
```

[2](#)

² "blocks" (21.2 p 676) "if" (21.6 p 685) "while" (37.1 p 1230) "for" (21.5 p 681) "suchthat" (21.11 p 696) "parallel" (21.9 p 689) "lists" (?? p ??)

21.11 syntax suchthat

— suchthat.help —

```
=====
Such that predicates
=====
```

A for loop can be followed by a "|" and then a predicate. The predicate qualifies the use of the values from the iterator that follows the for. Think of the vertical bar "|" as the phrase "such that".

```
for n in 0..4 | odd? n repeat output n
1
3
```

Type: Void

This loop expression prints out the integers n in the given segment such that n is odd.

A for loop can also be written

```
for iterator | predicate repeat loopbody
```

which is equivalent to:

```
for iterator repeat if predicate then loopbody else iterate
```

The predicate need not refer only to the variable in the for clause. Any variable in an outer scope can be part of the predicate.

```
for i in 1..50 repeat
  for j in 1..50 | factorial(i+j) < 25 repeat
    output [i,j]
[1,1]
[1,2]
[1,3]
[2,1]
[2,2]
[3,1]
```

Type: Void

21.12 syntax syntax

— syntax.help —

The Axiom Interactive Language has the following features documented here.

More information is available by typing

```
)help feature
```

where feature is one of:

```
assignment -- Immediate and delayed assignments
blocks      -- Blocks of expressions
collection  -- creating lists with iterators
for         -- for loops
if          -- If-then-else statements
iterate     -- using iterate in loops
leave       -- using leave in loops
parallel    -- parallel iterations
repeat      -- repeat loops
suchthat    -- suchthat predicates
while       -- while loops
```

—

21.13 syntax while

— while.help —

```
=====
while loops
=====
```

The repeat in a loop can be modified by adding one or more while clauses. Each clause contains a predicate immediately following the while keyword. The predicate is tested before the evaluation of the body of the loop. The loop body is evaluated whenever the predicate in a while clause is true.

The syntax for a simple loop using while is

```
while predicate repeat loopbody
```


The predicate is evaluated before loopbody is evaluated. A while loop terminates immediately when predicate evaluates to false or when a leave or return expression is evaluated. See `)help repeat` for more information on leave and return.

Here is a simple example of using while in a loop. We first initialize the counter.

```
i := 1
1
Type: PositiveInteger

while i < 1 repeat
  output "hello"
  i := i + 1
Type: Void
```

The steps involved in computing this example are

- (1) set i to 1
- (2) test the condition `i < 1` and determine that it is not true
- (3) do not evaluate the loop body and therefore do not display "hello"

```
(x, y) := (1, 1)
1
Type: PositiveInteger
```

If you have multiple predicates to be tested use the logical and operation to separate them. Axiom evaluates these predicates from left to right.

```
while x < 4 and y < 10 repeat
  output [x,y]
  x := x + 1
  y := y + 2
[1,1]
[2,3]
[3,5]
Type: Void
```

A leave expression can be included in a loop body to terminate a loop even if the predicate in any while clauses are not false.

```
(x, y) := (1, 1)
1
Type: PositiveInteger

while x < 4 and y < 10 repeat
  if x + y > 7 then leave
  output [x,y]
```

```
x := x + 1
y := y + 2
[1,1]
[2,3]
```

Type: Void

Chapter 22

Abstract Syntax Trees (ptrees)

Abstract Syntax Trees

These functions create and examine abstract syntax trees. These are called pform, for short.

!! This file also contains constructors for concrete syntax, although
!! they should be somewhere else.

THE PFORM DATA STRUCTURE

Leaves: [hd, tok, pos]

Trees: [hd, tree, tree, ...]

hd is either an id or (id . alist)

defun Construct a leaf token

The tokConstruct function is a constructor and selectors for leaf tokens. A leaf token looks like [head, token, position] where head is either an id or (id . alist)

[ifcar p??]

[pfNoPosition? p702]

[ncPutQ p706]

— defun tokConstruct —

```
(defun |tokConstruct| (head token &rest position)
  (let (result)
    (setq result (cons head token))
    (cond
      ((ifcar position)
       (cond
         ((|pfNoPosition?| (car position)) result)
         (t (tokConstruct (car position) token (cdr position))))
      (t result)))
  result)
```

```
(t (|ncPutQ| result ' |posn| (car position)) result)))
(t result)))
```

defun Return a part of a node

```
[ifcar p??]
```

— defun pfAbSynOp —

```
(defun |pfAbSynOp| (form)
  (let (hd)
    (setq hd (car form))
    (or (ifcar hd) hd)))
```

defun Compare a part of a node

```
[eqcar p??]
```

— defun pfAbSynOp? —

```
(defun |pfAbSynOp?| (form op)
  (let (hd)
    (setq hd (car form))
    (or (eq hd op) (eqcar hd op))))
```

defun pfNoPosition?

```
[poNoPosition? p703]
```

— defun pfNoPosition? —

```
(defun |pfNoPosition?| (pos)
  (|poNoPosition?| pos))
```

defun poNoPosition?

```
[eqcar p??]
```

```
— defun poNoPosition? 0 —
```

```
(defun |poNoPosition?| (pos)
  (eqcar pos '|noPosition|))
```

```
—————
```

defun tokType

```
[ncTag p705]
```

```
— defun tokType —
```

```
(defun |tokType| (x) (|ncTag| x))
```

```
—————
```

defun tokPart

```
— defun tokPart 0 —
```

```
(defun |tokPart| (x) (cdr x))
```

```
—————
```

defun tokPosn

```
[qassq p??]
```

```
[ncAlist p705]
```

```
[pfNoPosition p704]
```

```
— defun tokPosn —
```

```
(defun |tokPosn| (x)
  (let (a)
    (setq a (qassq '|posn| (|ncAlist| x)))
    (cond
```

```
(a (cdr a))
(t (lpfNoPosition|))))))
```

defun pfNoPosition

[poNoPosition p704]

— defun pfNoPosition —

```
(defun |pfNoPosition| () (|poNoPosition|))
```

defun poNoPosition

[\$nopus p319]

— defun poNoPosition 0 —

```
(defun |poNoPosition| ()
  (declare (special |$nopus|))
  |$nopus|)
```

Chapter 23

Attributed Structures

For objects which are pairs where the CAR field is either just a tag (an identifier) or a pair which is the tag and an association list.

defun ncTag

Pick off the tag [ncBug p658]

[qcar p??]

[identp p1237]

— defun ncTag —

```
(defun |ncTag| (x)
  (cond
    ((null (consp x)) (|ncBug| "bad object" nil))
    (t
     (setq x (qcar x))
     (cond
      ((identp x) x)
      ((null (consp x)) (|ncBug| "bad object" nil))
      (t (qcar x))))))
```

—————

defun ncAlist

Pick off the property list [ncBug p658]

[qcar p??]

[identp p1237]

[qcdr p??]

— defun ncAlist —

```
(defun |ncAlist| (x)
  (cond
    ((null (consp x)) (|ncBug| "bad object" nil))
    (t
     (setq x (qcar x))
     (cond
       ((identp x) nil)
       ((null (consp x)) (|ncBug| "bad object" nil))
       (t (qcdr x))))))
```

defun ncEltQ

Get the entry for key k on x's association list

```
[qassq p??]
[ncAlist p705]
[ncBug p658]
```

— defun ncEltQ —

```
(defun |ncEltQ| (x k)
  (let (r)
    (setq r (qassq k (|ncAlist| x)))
    (cond
      ((null r) (|ncBug| "Association list search failed on %1" (list k)))
      (t (cdr r)))))
```

defun ncPutQ

```
-- Put (k . v) on the association list of x and return v
-- case1: ncPutQ(x,k,v) where k is a key (an identifier), v a value
--       put the pair (k . v) on the association list of x and return v
-- case2: ncPutQ(x,k,v) where k is a list of keys, v a list of values
--       equivalent to [ncPutQ(x,key,val) for key in k for val in v]
ncPutQ(x,k,v) ==
;   LISTP k =>
;     for key in k for val in v repeat ncPutQ(x,key,val)
;     v
;   r := QASSQ(k,ncAlist x)
```

```

;   if NULL r then
;       r := CONS( CONS(k,v), ncAlist x)
;       RPLACA(x,CONS(ncTag x,r))
;   else
;       RPLACD(r,v)
;   v

```

```

[qassq p??]
[ncAlist p705]
[ncTag p705]

```

— defun ncPutQ —

```

(defun |ncPutQ| (x k v)
  (let (r)
    (cond
      ((listp k)
       ((lambda (Var1 key Var2 val)
          (loop
            (cond
              ((or (atom Var1)
                   (progn (setq key (car Var1)) nil)
                   (atom Var2)
                   (progn (setq val (car Var2)) nil))
              (return nil))
            (t
             (|ncPutQ| x key val)))
          (setq Var1 (cdr Var1))
          (setq Var2 (cdr Var2))))
        k nil v nil)
      v)
    (t
     (setq r (qassq k (|ncAlist| x)))
     (cond
       ((null r)
        (setq r (cons (cons k v) (|ncAlist| x)))
        (rplaca x (cons (|ncTag| x) r)))
       (t
        (rplacd r v)))
     v))))

```

—————

Special Category Names

defvar \$EmptyMode

The CONTAINED predicate is used to walk internal structures such as modemap to see if the X object occurs within Y . One particular use is in a function called isPartialMode to decide if a modemap is only partially complete. If this is true then the modemap will contain the constant \$EmptyMode. So the call ends up being CONTAINED \$EmptyMode Y .

— initvars —

```
(defvar |$EmptyMode| '|$EmptyMode|)
```

—————

defvar \$AnonymousFunction

— initvars —

```
(defvar |$AnonymousFunction| '(|AnonymousFunction|))
```

—————

defvar \$Any

— initvars —

```
(defvar |$Any| '(|Any|))
```

—————

defvar \$BFtag

— initvars —

```
(defvar |$BFtag| '(:BF:|))
```

—————

defvar \$Boolean

— initvars —

```
(defvar |$Boolean| '(|Boolean|))
```

—————

defvar \$Category

— initvars —

```
(defvar |$Category| '(|Category|))
```

—————

defvar \$Domain

— initvars —

```
(defvar |$Domain| '(|Domain|))
```

—————

defvar \$Exit

— initvars —

```
(defvar |$Exit| '(|Exit|))
```

—————

defvar \$Expression

— initvars —

```
(defvar |$Expression| '(|OutputForm|))
```

defvar \$OutputForm

— initvars —

```
(defvar |$OutputForm| '(|OutputForm|))
```

defvar \$BigFloat

— initvars —

```
(defvar |$BigFloat| '(|Float|))
```

defvar \$Float

— initvars —

```
(defvar |$Float| '(|Float|))
```

defvar \$DoubleFloat

— initvars —

```
(defvar |$DoubleFloat| '(|DoubleFloat|))
```

defvar \$FontTable

— initvars —

```
(defvar |$FontTable| '(|FontTable|))
```

—————

defvar \$Integer

— initvars —

```
(defvar |$Integer| '(|Integer|))
```

—————

defvar \$ComplexInteger

— initvars —

```
(defvar |$ComplexInteger| (LIST '|Complex| |$Integer|))
```

—————

defvar \$Mode

— initvars —

```
(defvar |$Mode| '(|Mode|))
```

—————

defvar \$NegativeInteger

— initvars —

```
(defvar |$NegativeInteger| '(|NegativeInteger|))
```

defvar \$NonNegativeInteger

— initvars —

```
(defvar |$NonNegativeInteger| '(|NonNegativeInteger|))
```

defvar \$NonPositiveInteger

— initvars —

```
(defvar |$NonPositiveInteger| '(|NonPositiveInteger|))
```

defvar \$PositiveInteger

— initvars —

```
(defvar |$PositiveInteger| '(|PositiveInteger|))
```

defvar \$RationalNumber

— initvars —

```
(defvar |$RationalNumber| '(|Fraction| (|Integer|)))
```

defvar \$String

— initvars —

```
(defvar |$String| '(|String|))
```

—————

defvar \$StringCategory

— initvars —

```
(defvar |$StringCategory| '(|StringCategory|))
```

—————

defvar \$Symbol

— initvars —

```
(defvar |$Symbol| '(|Symbol|))
```

—————

defvar \$Void

— initvars —

```
(defvar |$Void| '(|Void|))
```

—————

defvar \$QuotientField

— initvars —


```
(defvar |$QuotientField| ' |Fraction|)
```

```
defvar $FunctionalExpression
```

```
— initvars —
```

```
(defvar |$FunctionalExpression| ' |Expression|)
```

```
defvar $defaultFunctionTargets
```

```
— initvars —
```

```
(defvar |$defaultFunctionTargets| ' ( ( ( ) ) )
```

```
:: Old names
```

```
defvar $SmallInteger
```

```
— initvars —
```

```
(defvar |$SmallInteger| ' ( |SingleInteger| ) )
```

```
:: New Names
```

```
defvar $SingleFloat
```

```
— initvars —
```

```
(defvar |$SingleFloat| ' ( |SingleFloat| ) )
```

defvar \$DoubleFloat

— initvars —

```
(defvar |$DoubleFloat| '(|DoubleFloat|))
```

—————

defvar \$SingleInteger

— initvars —

```
(defvar |$SingleInteger| '(|SingleInteger|))
```

—————

Chapter 24

Function Selection

New Selection of Modemaps

selection of applicable modemaps is done in two steps:

first it tries to find a modemap inside an argument domain, and if this fails, by evaluation of pattern modemaps the result is a list of functions with signatures, which have the following form:

[sig,elt,cond] where

sig is the signature gained by evaluating the modemap condition

elt is the slot number to get the implementation

cond are runtime checks which are the results of evaluating the modemap condition

the following flags are used:

\$Coerce is NIL, if function selection is done which requires exact matches (e.g. for coercion functions)

if \$SubDom is true, then runtime checks have to be compiled

defun ofCategory

[identp p1237]

[ofCategory p717]

[hasCaty p718]

[\$Subst p??]

[\$hope p??]

— defun ofCategory —

```
(defun |ofCategory| (dom cat)
  (let (|$Subst| |$hope|)
    (declare (special |$Subst| |$hope|))
```

```
(cond
  ((identp dom) nil)
  ((and (listp cat) (eq (car cat) '|Join|))
    (every #'(lambda (c) (|ofCategory| dom c)) (cdr cat)))
  (t (not (eq (|hasCaty| dom cat nil) '|failed|))))))
```

defun isPartialMode

The `isPartialMode` function tests whether `m` contains `$EmptyMode`. The constant `$EmptyMode` evaluates to `|$EmptyMode|`. This constant is inserted in a modemap during compile time if the modemap is not yet complete.

```
[contained p??]
[$EmptyMode p708]
```

— defun isPartialMode —

```
(defun |isPartialMode| (m)
  (declare (special |$EmptyMode|))
  (contained |$EmptyMode| m))
```

defun hasCaty

This calls `hasCat`, which looks up a hashtable and returns:

1. T, NIL or a (has x1 x2) condition, if `cat` is not parameterized
2. a list of pairs (argument to `cat`, condition) otherwise

then the substitution `sl` is augmented, or the result is 'failed [hasAttSig p724]

```
[subCopy p??]
[constructSubst p732]
[hasSig p721]
[hasAtt p722]
[hasCat p??]
[opOf p??]
[mkDomPvar p731]
[domArg p720]
[augmentSub p??]
[domArg2 p720]
[unifyStruct p726]
```

[hasCaty1 p730]
 [\$domPvar p341]

— defun hasCaty —

```
(defun |hasCaty| (d cat s1)
  (let (x y S z cond sp dom zp s1 ncond i)
    (declare (special |$domPvar|))
    (cond
      ((and (consp cat) (eq (qcar cat) 'category) (consp (qcdr cat)))
        (|hasAttSig| d (|subCopy| (qcddr cat) (|constructSubst| d)) s1))
      ((and (consp cat) (eq (qcar cat) 'signature) (consp (qcdr cat))
        (consp (qcddr cat)) (eq (qcdddr cat) nil))
        (|hasSig| d (qcadr cat) (|subCopy| (qcaddr cat) (|constructSubst| d)) s1))
      ((and (consp cat) (eq (qcar cat) 'attribute)
        (consp (qcdr cat)) (eq (qcddr cat) nil))
        (|hasAtt| d (|subCopy| (qcadr cat) (|constructSubst| d)) s1))
      ((setq x (|hasCat| (|opOf| d) (|opOf| cat)))
        (cond
          ((setq y (ifcdr cat))
            (setq s (|constructSubst| d))
            (do ((next x (cdr next)) (endtest nil (null (eq s1 '|failed|))))
              ((or (atom next) endtest) nil)
              (setq z (caar next))
              (setq cond (cdar next))
              (setq sp
                (loop for item in s
                  collect (cons (car item) (|mkDomPvar| (car item) (cdr item) z y))))
              (when |$domPvar|
                (setq i -1)
                (setq dom
                  (cons (car d)
                    (loop for arg in (rest d)
                      collect (|domArg| arg (incf i) z y))))
                (setq s1 (|augmentSub| |$domPvar| dom (copy s1))))
              (setq zp
                (loop for a in z
                  collect (|domArg2| a s sp)))
              (setq s1 (|unifyStruct| y zp (copy s1)))
              (cond
                ((null (eq s1 '|failed|))
                  (setq s1
                    (cond
                      ((atom cond) s1)
                      (t
                     (setq ncond (|subCopy| cond s))
                     (cond
                       ((and (consp ncond) (eq (qcar ncond) '|has|)
                        (consp (qcdr ncond)) (equal (qcadr ncond) d)
                        (consp (qcddr ncond)) (eq (qcdddr ncond) nil))
```

```

                (equal (qcaddr ncond) cat))
              '|failed|)
            (t (|hasCaty1| ncond s1))))))
          (t nil)))
        s1)
      ((atom x) s1)
      (t
        (setq ncond (|subCopy| x (|constructSubst| d)))
        (cond
          ((and (consp ncond) (eq (qcar ncond) '|has|) (consp (qcdr ncond))
            (equal (qcadr ncond) d) (consp (qcddr ncond))
            (eq (qcdddr ncond) nil) (equal (qcaddr ncond) cat))
            '|failed|)
          (t (|hasCaty1| ncond s1))))))
      (t '|failed|)))

```

defun domArg

[[\\$FormalMapVariableList](#) p17]

— defun domArg —

```

(defun |domArg| (type i subs y)
  (let (p)
    (declare (special |$FormalMapVariableList|))
    (if (setq p (member (elt |$FormalMapVariableList| i) subs))
      (elt y (- (|#| subs) (|#| p)))
      type)))

```

defun domArg2

[[isSharpVar](#) p110]

[[subCopy](#) p??]

[[\\$domPvar](#) p341]

— defun domArg2 —

```

(defun |domArg2| (arg s11 s12)
  (declare (special |$domPvar|))
  (cond
    ((|isSharpVar| arg) (|subCopy| arg s11))

```

```
((and (eq arg '$) |$domPvar|) |$domPvar|)
(t (|subCopy| arg sl2))))
```

defun hasSig

The function hasSig tests whether domain dom has function foo with signature sig under substitution sl. [constructor? p??]

```
[constructSubst p??]
[assq p1240]
[getOperationAlistFromLisplib p134]
[hasCate p731]
[subCopy p??]
[hasSigAnd p723]
[hasSigOr p724]
[keyedSystemError p??]
[unifyStruct p726]
[$domPvar p341]
```

— defun hasSig —

```
(defun |hasSig| (dom foo sig sl)
  (let (|$domPvar| fun s0 p x cond s)
    (declare (special |$domPvar|))
    (cond
      ((setq fun (|constructor?| (car dom)))
        (setq s0 (|constructSubst| dom))
        (cond
          ((setq p (assq foo (|getOperationAlistFromLisplib| (car dom))))
            (do ((next (cdr p) (cdr next))
                (endtest nil (null (eq s '|failed|))))
              ((or (atom next) endtest) nil)
              (setq x (caar next))
              (setq cond (caddr next))
              (setq s
                (cond
                  ((atom cond) (copy sl))
                  ((and (consp cond) (eq (qcar cond) '|has|)
                    (consp (qcdr cond)) (consp (qcddr cond))
                    (eq (qcdr (qcddr cond)) nil))
                  (|hasCate| (|subCopy| (qcadr cond) s0)
                    (|subCopy| (qcaddr cond) s0)
                    (copy sl))))
              ((and (consp cond)
                (or (eq (qcar cond) '|and|) (eq (qcar cond) '|and|)))
                (|hasSigAnd| (qcdr cond) s0 sl))
```



```

((and (consp cond)
      (or (eq (qcar cond) 'or) (eq (qcar cond) 'lor))))
(|hasSigOr| (qcdr cond) s0 s1))
(t
  (|keyedSystemError|
   "Unexpected error or improper call to system function %1: %2"
   (list "hasSig" "unexpected condition for signature"))))
(unless (eq s '|failed|)
  (setq s (|unifyStruct| (|subCopy| x s0) sig s)))
s)
(t '|failed|)))
(t '|failed|)))

```

defun hasAtt

The hasAtt function tests whether dom has attribute att under s1 needs s0 similar to hasSig.

[subCopy p??]
 [getdatabase p1196]
 [constructSubst p732]
 [getInfovec p??]
 [unifyStruct p726]
 [hasCatExpression p725]
 [\$domPvar p341]

— defun hasAtt —

```

(defun |hasAtt| (dom att s1)
  (let (|$domPvar| fun atts u x cond s)
    (declare (special |$domPvar|))
    (cond
      ((setq fun (car dom))
       (cond
         ((setq atts
              (|subCopy| (getdatabase fun 'attributes) (|constructSubst| dom)))
          (cond
            ((consp (setq u (|getInfovec| (car dom))))
             (do ((next atts (cdr next))
                  (endtest nil (null (eq s '|failed|))))
                  ((or (atom next) endtest) nil)
              (setq x (caar next))
              (setq cond (cdar next))
              (setq s (|unifyStruct| x att (copy s1)))
              (cond
                ((and (null (atom cond)) (null (eq s '|failed|)))
                 (setq s (|hasCatExpression| cond s)))))))
          s)
        (t '|failed|)))
      (t '|failed|)))

```

```

      s)
    (t
      (do ((next atts (cdr next))
            (endtest nil (null (eq s '|failed|))))
          ((or (atom next) endtest) nil)
          (setq x (caar next))
          (setq cond (cadar next))
          (setq s (|unifyStruct| x att (copy sl)))
          (cond
            ((and (null (atom cond)) (null (eq s '|failed|)))
              (setq s (|hasCatExpression| cond s))))
          s)))
    (t '|failed|)))
  (t '|failed|)))

```

defun hasSigAnd

[hasCate p731]
 [subCopy p??]
 [keyedSystemError p??]

— defun hasSigAnd —

```

(defun |hasSigAnd| (andCls s0 sl)
  (let (sa dead)
    (setq sa '|failed|)
    (loop for cls in andCls
      do
        (when dead (return))
        (setq sa
          (cond
            ((atom cls) (copy sl))
            ((and (consp cls) (eq (qcar cls) '|has|) (consp (qcdr cls))
              (consp (qcddr cls)) (eq (qcdddr cls) nil))
              (|hasCate| (|subCopy| (qcadr cls) s0)
                (|subCopy| (qcaddr cls) s0)
                (copy sl)))
            (t
              (|keyedSystemError|
                "Unexpected error or improper call to system function %1: %2"
                (list "hasSigAnd" "unexpected condition for signature")))))
          (when (eq sa '|failed|) (setq dead t)))
    sa))

```

defun hasSigOr

[hasCate p731]
 [hasSigAnd p723]
 [keyedSystemError p??]

— defun hasSigOr —

```
(defun |hasSigOr| (orCls s0 s1)
  (let (sa found)
    (setq sa '|failed|)
    (loop for cls in orCls
      until found
      do
        (setq sa
          (cond
            ((atom cls) (copy s1))
            ((and (consp cls) (eq (qcar cls) '|has|) (consp (qcdr cls))
              (consp (qcddr cls)) (eq (qcdddr cls) nil))
              (|hasCate| (|subCopy| (qcadr cls) s0)
                (|subCopy| (qcaddr cls) s0)
                (copy s1)))
            ((and (consp cls)
              (or (eq (qcar cls) '|and|) (eq (qcar cls) '|and|)))
              (|hasSigAnd| (qcdr cls) s0 s1))
            (t
              (|keyedSystemError|
                "Unexpected error or improper call to system function %1: %2"
                (list "hasSigOr" "unexpected condition for signature")))))
        (unless (eq sa '|failed|) (setq found t)))
    sa))
```

—

defun hasAttSig

The argument *d* is domain, *x* is a list of attributes and signatures. The result is an augmented SL, if *d* has *x*, 'failed otherwise.

[hasAtt p722]
 [hasSig p721]
 [keyedSystemError p??]

— defun hasAttSig —

```
(defun |hasAttSig| (d x s1)
  (loop for y in x
```

```

until (eq s1 '|failed|)
do
  (setq s1
    (cond
      ((and (consp y) (eq (qcar y) 'attribute)
        (consp (qcdr y)) (eq (qcddr y) nil))
        (|hasAtt| d (qcadr y) s1))
      ((and (consp y) (eq (qcar y) 'signature)
        (consp (qcdr y)) (consp (qcddr y)) (eq (qcdddr y) nil))
        (|hasSig| d (qcadr y) (qcaddr y) s1))
      (t
        (|keyedSystemError|
          "Unexpected error or improper call to system function %1: %2"
          (list "hasAttSig" "unexpected form of unnamed category")))))
s1)

```

defun hasCate1

[hasCate p731]
 [\$domPvar p341]

— defun hasCate1 —

```

(defun |hasCate1| (dom cat s1 domPvar)
  (let (|$domPvar|)
    (declare (special |$domPvar|))
    (setq |$domPvar| domPvar)
    (|hasCate| dom cat s1)))

```

defun hasCatExpression

[hasCatExpression p725]
 [hasCate p731]
 [keyedSystemError p??]

— defun hasCatExpression —

```

(defun |hasCatExpression| (cond s1)
  (let (y)
    (cond
      ((and (consp cond) (eq (qcar cond) 'or))

```

```

(when
  (let (result)
    (loop for x in (qcdr cond)
      do (setq result
        (or result
          (not (eq (setq y (|hasCatExpression| x s1)) '|failed|))))))
    result)
  y))
((and (consp cond) (eq (qcar cond) 'and))
 (when
  (let ((result t))
    (loop for x in (qcdr cond)
      do (setq result
        (and result
          (not (eq (setq s1 (|hasCatExpression| x s1)) '|failed|))))))
    result)
  s1))
((and (consp cond) (eq (qcar cond) '|has|)
  (consp (qcdr cond)) (consp (qcddr cond)) (eq (qcdddr cond) nil))
 (|hasCate| (qcadr cond) (qcaddr cond) s1))
(t
 (|keyedSystemError|
  "Unexpected error or improper call to system function %1: %2"
  (list "hasSig" "unexpected condition for attribute")))))

```

defun unifyStruct

[\[isPatternVar p729\]](#)
[\[unifyStructVar p727\]](#)
[\[unifyStruct p726\]](#)

— defun unifyStruct —

```

(defun |unifyStruct| (s1 s2 s1)
  (declare (special |$domPvar| |$hope| |$Coerce| |$Subst|))
  (cond
    ((equal s1 s2) s1)
    (t
     (when (and (consp s1) (eq (qcar s1) '|:|)
       (consp (qcdr s1)) (consp (qcddr s1)) (eq (qcdddr s1) nil))
       (setq s1 (qcadr s1)))
     (when (and (consp s2) (eq (qcar s2) '|:|)
       (consp (qcdr s2)) (consp (qcddr s2)) (eq (qcdddr s2) nil))
       (setq s2 (qcadr s2)))
     (when (and (null (atom s1)) (eq (car s1) '|#|))

```

```

    (setq s1 (length (cadr s1)))
    (when (and (null (atom s2)) (eq (car s2) '#|))
      (setq s2 (length (cadr s2))))
    (cond
      ((equal s1 s2) s1)
      ((|isPatternVar| s1) (|unifyStructVar| s1 s2 s1))
      ((|isPatternVar| s2) (|unifyStructVar| s2 s1 s1))
      ((or (atom s1) (atom s2)) '|failed|)
      (t
       (loop until (or (null s1) (null s2) (eq s1 '|failed|))
         do
          (setq s1 (|unifyStruct| (car s1) (car s2) s1))
          (setq s1 (cdr s1))
          (setq s2 (cdr s2)))
       (if (or s1 s2) '|failed| s1))))))

```

defun unifyStructVar

The first argument is a pattern variable, which is not substituted by s1 [contained p??]

```

[lassoc p??]
[unifyStruct p726]
[constructor? p??]
[subCopy p??]
[containsVars p729]
[canCoerce p??]
[resolveTT p??]
[isPatternVar p729]
[augmentSub p??]
[$domPvar p341]
[$Coerce p??]
[$Subst p??]
[$hope p??]

```

— defun unifyStructVar —

```

(defun |unifyStructVar| (v ss s1)
  (let (ps s1 s0 s ns0 ns1 s3)
    (declare (special |$domPvar| |$hope| |$Coerce| |$Subst|))
    (cond
      ((contained v ss) '|failed|)
      (t
       (setq ps (lassoc ss s1))
       (setq s1 (if ps ps ss))
       (cond

```

```

((or (setq s0 (lassoc v s1)) (setq s0 (lassoc v |$Subst|)))
 (setq s (|unifyStruct| s0 s1 (copy s1)))
 (cond
  ((eq s '|failed|)
   (cond
    ((and |$Coerce| (null (atom s0)) (|constructor?| (car s0)))
     (cond
      ((or (|containsVars| s0) (|containsVars| s1))
       (setq ns0 (|subCopy| s0 s1))
       (setq ns1 (|subCopy| s1 s1))
       (cond
        ((or (|containsVars| ns0) (|containsVars| ns1))
         (setq |$hope| t)
         '|failed|)
        (t
         (cond
          ((|canCoerce| ns0 ns1) (setq s3 s1))
          ((|canCoerce| ns1 ns0) (setq s3 s0))
          (t (setq s3 nil)))
         (cond
          (s3
           (cond
            ((not (equal s3 s0))
             (setq s1 (|augmentSub| v s3 s1))))
          (cond
           ((and (not (equal s3 s1)) (|isPatternVar| ss))
            (setq s1 (|augmentSub| ss s3 s1))))
           s1)
          (t '|failed|))))))
      (|$domPvar|
       (setq s3 (|resolveTT| s0 s1))
       (cond
        (s3
         (cond
          ((not (equal s3 s0))
           (setq s1 (|augmentSub| v s3 s1))))
         (cond
          ((and (not (equal s3 s1)) (|isPatternVar| ss))
           (setq s1 (|augmentSub| ss s3 s1))))
          s1)
          (t '|failed|)))
       (t '|failed|)))
   (t (|augmentSub| v ss s))))
 (t (|augmentSub| v ss s1))))))

```

defun containsVars

The function containsVars tests whether term t contains a * variable.

[isPatternVar p729]
[containsVars1 p729]

— defun containsVars —

```
(defun |containsVars| (arg)
  (if (atom arg)
      (|isPatternVar| arg)
      (|containsVars1| arg)))
```

—————

defun isPatternVar

— defun isPatternVar —

```
(defun |isPatternVar| (v)
  (and (identp v)
       (member v
                '(** *1 *2 *3 *4 *5 *6 *7 *8 *9 *10 *11 *12 *13 *14 *15
                  *16 *17 *18 *19 *20))
       t))
```

—————

defun containsVars1

The function containsVars1 tests whether term t contains a * variable. This is a recursive version, which works on a list.

[isPatternVar p729]
[containsVars1 p729]

— defun containsVars1 —

```
(defun |containsVars1| (arg)
  (let ((t1 (car arg)) (t2 (cdr arg)))
    (if (atom t1)
        (or (|isPatternVar| t1)
            (if (atom t2) (|isPatternVar| t2) (|containsVars1| t2)))
        (or (|containsVars1| t1)
            (if (atom t2) (|isPatternVar| t2) (|containsVars1| t2))))
```



```
(if (atom t2) (|isPatternVar| t2) (|containsVars1| t2))))))
```

defun hasCaty1

The cond is either a (has a b) or an OR clause of such conditions. SL is augmented, if cond is true, otherwise the result is 'failed

[hasCate p731]

[hasCaty1 p730]

[keyedSystemError p??]

[\$domPvar p341]

— defun hasCaty1 —

```
(defun |hasCaty1| (cond sl)
  (let (|$domPvar| a s)
    (declare (special |$domPvar|))
    (setq |$domPvar| nil)
    (cond
      ((and (consp cond) (eq (qcar cond) '|has|)
        (consp (qcdr cond)) (consp (qcddr cond)) (eq (qcddr cond) nil))
        (|hasCate| (qcadr cond) (qcaddr cond) sl))
      ((and (consp cond) (EQ (qcar cond) 'and))
        (loop for x in (qcdr cond)
          while (not (eq s '|failed|))
          do
            (setq s
              (cond
                ((and (consp x) (eq (qcar x) '|has|)
                  (consp (qcdr x)) (consp (qcddr x)) (eq (qcdr (qcddr x)) nil))
                  (|hasCate| (qcadr x) (qcaddr x) sl))
                ((and (consp x) (eq (qcdr x) nil)
                  (consp (qcar x)) (eq (qcaar x) '|has|)
                  (consp (qcdar x)) (consp (qcddar x))
                  (eq (qcdr (qcddar x)) nil))
                  (|hasCate| a (qcaddar x) sl))
                (t (|hasCaty1| x sl))))
              s)
            ((and (consp cond) (eq (qcar cond) 'or))
              (do ((next (qcdr cond) (cdr next)) (x nil)
                (nextitem nil (null (eq s '|failed|))))
                ((or (atom next)
                  (progn (setq x (car next)) nil)
                  nextitem)
                  nil)
                (setq s
```

```

(cond
  ((and (consp x) (eq (qcar x) '|has|)
        (consp (qcdr x)) (consp (qcddr x)) (eq (qcdddr x) nil))
    (|hasCate| (qcadr x) (qcaddr x) (copy s1)))
  ((and (consp x) (eq (qcdr x) nil) (consp (qcar x))
        (eq (qcaar x) '|has|) (consp (qcдар x)) (consp (qcddar x))
        (eq (qcdddar x) nil))
    (|hasCate| (qcadar x) (qcaddar x) (copy s1)))
  (t (|hasCaty1| x (copy s1)))))
s)
(t
  (|keyedSystemError|
    "Unexpected error or improper call to system function %1: %2"
    (list "hasCaty1" "unexpected condition from category table"))))

```

defun mkDomPvar

[domArg p720]
 [length p??]
 [\$FormalMapVariableList p17]

— defun mkDomPvar —

```

(defun |mkDomPvar| (p d subs y)
  (let (l)
    (declare (special |$FormalMapVariableList|))
    (if (setq l (member p |$FormalMapVariableList|))
        (|domArg| d (- (|#| |$FormalMapVariableList|) (|#| 1)) subs y)
        d)))

```

defun hasCate

[isPatternVar p729]
 [hasCate1 p725]
 [hasCateSpecial p733]
 [containsVariables p??]
 [subCopy p??]
 [hasCaty p718]
 [\$EmptyMode p708]
 [\$Subst p??]

[\$hope p??]

— defun hasCate —

```
(defun |hasCate| (dom cat sl)
  (let (nsl p s sl1)
    (declare (special |$hope| |$Subst| |$EmptyMode|))
    (cond
      ((equal dom |$EmptyMode|) nil)
      ((|isPatternVar| dom)
        (cond
          ((and (setq p (assq dom sl))
                (not (eq (setq nsl (|hasCate| (cdr p) cat sl)) '|failed|))))
          nsl)
        ((or (setq p (assq dom |$Subst|)) (setq p (assq dom sl)))
          (setq s (|hasCate1| (cdr p) cat sl dom))
          (cond
            ((null (eq s '|failed|)) s)
            (t (|hasCateSpecial| dom (cdr p) cat sl))))
        (t
          (when (not (eq sl '|failed|)) (setq |$hope| t))
          '|failed|)))
    (t
      (setq sl1
        (loop for item in sl
              when (null (|containsVariables| (cdr item)))
                collect item))
      (when sl1 (setq cat (|subCopy| cat sl1)))
      (|hasCate| dom cat sl))))
```

—————

defun constructSubst

[internal p??]

— defun constructSubst —

```
(defun |constructSubst| (d)
  (let (sl (i 0))
    (setq sl (list (cons '$ d)))
    (when (listp d)
      (dolist (x (cdr d))
        (setq i (1+ i))
        (setq sl (cons (cons (internall "#" (princ-to-string i)) x) sl))))
    sl))
```

defun hasCateSpecial

The variable `v` is a pattern variable, `dom` is its binding under `$Subst`. We try to change `dom` so that it has category `cat` under `sl`. The result is a substitution list or `'failed`.

```
[eqcar p??]
[isSubDomain p??]
[canCoerceFrom p??]
[containsVars p729]
[augmentSub p??]
[hasCate p731]
[hasCaty p718]
[hasCateSpecialNew p734]
[$Integer p711]
[$QuotientField p713]
```

— defun hasCateSpecial —

```
(defun |hasCateSpecial| (v dom cat sl)
  (let (arg d domp nsl)
    (declare (special |$Integer| |$QuotientField|))
    (cond
      ((and (consp dom) (eq (qcar dom) '|FactoredForm|)
            (consp (qcdr dom)) (eq (qcddr dom) nil))
        (setq arg (qcadr dom))
        (when (|isSubDomain| arg |$Integer|) (setq arg |$Integer|))
        (setq d (list '|FactoredRing| arg))
        (setq sl (|hasCate| arg '|Ring| (|augmentSub| v d sl)))
        (if (eq sl '|failed|)
            '|failed|
            (|hasCaty| d cat sl)))
      ((or (eqcar cat '|Field|) (eqcar cat '|DivisionRing|))
        (when (|isSubDomain| dom |$Integer|) (setq dom |$Integer|))
        (setq d (list |$QuotientField| dom))
        (|hasCaty| dom '|IntegralDomain| (|augmentSub| v d sl)))
      ((and (consp cat) (eq (qcar cat) '|PolynomialCategory|)
            (consp (qcdr cat)))
        (setq domp (cons '|Polynomial| (list (qcadr cat))))
        (and (or (|containsVars| (qcadr cat)) (|canCoerceFrom| dom domp))
              (|hasCaty| domp cat (|augmentSub| v domp sl))))
      ((|isSubDomain| dom |$Integer|)
        (setq nsl (|hasCate| |$Integer| cat (|augmentSub| v |$Integer| sl)))
        (if (eq nsl '|failed|)
            (|hasCateSpecialNew| v dom cat sl)
            (|hasCaty| |$Integer| cat nsl)))
      (t
```

```
(|hasCateSpecialNew| v dom cat sl))))
```

defun hasCateSpecialNew

```
[member p1238]
[eqcar p??]
[augmentSub p??]
[defaultTargetFE p735]
[isEqualOrSubDomain p736]
[underDomainOf p??]
[hasCaty p718]
[$Integer p711]
[$ComplexInteger p711]
[$RationalNumber p712]
```

— defun hasCateSpecialNew —

```
(defun |hasCateSpecialNew| (v dom cat sl)
  (let (fe alg fefull d partialResult)
    (declare (special |$RationalNumber| |$ComplexInteger| |$Integer|))
    (setq fe
      (|member| (qcar cat)
        '(|ElementaryFunctionCategory|
          |TrigonometricFunctionCategory|
          |ArcTrigonometricFunctionCategory|
          |HyperbolicFunctionCategory|
          |ArcHyperbolicFunctionCategory|
          |PrimitiveFunctionCategory|
          |SpecialFunctionCategory|
          |Evalable|
          |CombinatorialOpsCategory|
          |TranscendentalFunctionCategory|
          |AlgebraicallyClosedFunctionSpace|
          |ExpressionSpace|
          |LiouvillianFunctionCategory|
          |FunctionSpace|)))
      (setq alg
        (|member| (qcar cat)
          '(|RadicalCategory|
            |AlgebraicallyClosedField|)))
      (setq fefull
        (or fe alg (eqcar cat ' |CombinatorialFunctionCategory|)))
      (setq partialResult
        (cond
          ((or (eqcar dom ' |Variable|) (eqcar dom ' |Symbol|))
```

```

(cond
  ((|member| (car cat)
    '(|SemiGroup|
      |AbelianSemiGroup|
      |Monoid|
      |AbelianGroup|
      |AbelianMonoid|
      |PartialDifferentialRing|
      |Ring|
      |InputForm|)))
    (setq d (list '|Polynomial| |$Integer|))
    (|augmentSub| v d sl))
  ((eqcar cat '|Group|)
    (setq d (list '|Fraction| (list '|Polynomial| |$Integer|)))
    (|augmentSub| v d sl))
  (fefully
    (setq d (|defaultTargetFE| dom))
    (|augmentSub| v d sl))
    (t '|failed|)))
  ((|isEqualOrSubDomain| dom |$Integer|)
    (cond
      (fe
        (setq d (|defaultTargetFE| |$Integer|))
        (|augmentSub| v d sl))
      (alg
        (setq d '(|AlgebraicNumber|))
        (|augmentSub| v d sl))
      (t '|failed|)))
    (equal (|underDomainOf| dom) |$ComplexInteger|)
    (setq d (|defaultTargetFE| |$ComplexInteger|))
    (|hasCaty| d cat (|augmentSub| v d sl)))
  ((and (equal dom |$RationalNumber|) alg)
    (setq d '(|AlgebraicNumber|))
    (|augmentSub| v d sl))
  (fefully
    (setq d (|defaultTargetFE| dom))
    (|augmentSub| v d sl))
    (t '|failed|)))
  (if (eq partialResult '|failed|)
    '|failed|
    (|hasCaty| d cat partialResult))))

```

defun defaultTargetFE

[isEqualOrSubDomain p736]
 [ifcar p??]

[defaultTargetFE p735]
 [\$FunctionalExpression p714]
 [\$Integer p711]
 [\$Symbol p713]
 [\$RationalNumber p712]

— defun defaultTargetFE —

```
(defun |defaultTargetFE| (&rest dom)
  (let (a options)
    (declare (special |$FunctionalExpression| |$Integer| |$Symbol|
                      |$RationalNumber|))
    (setq a (car dom))
    (setq options (cdr dom))
    (cond
     ((or (and (consp a) (eq (qcar a) '|Variable|)
               (consp (qcdr a)) (eq (qcddr a) nil))
          (equal a |$RationalNumber|)
          (member (qcar a) (list (qcar |$Symbol|) '|RationalRadicals| '|Pi|))
          (equal a |$SingleInteger|)
          (|isEqualOrSubDomain| a |$Integer|)
          (equal a '|AlgebraicNumber|)))
      (if (ifcar options)
          (list |$FunctionalExpression| (list '|Complex| |$Integer|))
          (list |$FunctionalExpression| |$Integer|)))
     ((and (consp a) (eq (qcar a) '|Complex|)
           (consp (qcdr a)) (eq (qcddr a) nil))
      (|defaultTargetFE| (qcadr a) t))
     ((and (consp a) (consp (qcdr a)) (eq (qcddr a) nil)
           (member (qcar a) '|Polynomial| |RationalFunction| |Fraction|))
      (|defaultTargetFE| (qcadr a) (ifcar options)))
     ((and (consp a) (equal (qcar a) |$FunctionalExpression|)
           (consp (qcdr a)) (eq (qcddr a) nil))
      a)
     ((ifcar options)
      (list |$FunctionalExpression| (list '|Complex| a)))
     (t
      (list |$FunctionalExpression| a))))
```

defun isEqualOrSubDomain

[isSubDomain p??]

— defun isEqualOrSubDomain —

```
(defun |isEqualOrSubDomain| (d1 d2)
```

```

(or (equal d1 d2)
    (|isSubDomain| d1 d2)
    (and (atom d1)
          (or (and (consp d2) (eq (qcar d2) '|Variable|)
                    (consp (qcdr d2)) (eq (qcddr d2) nil)
                    (equal (qcadr d2) d1))
              (and (consp d2) (eq (qcdr d2) nil)
                    (equal (qcar d2) d1))))
    (and (atom d2)
          (or (and (consp d1) (eq (qcar d1) '|Variable|)
                    (consp (qcdr d1)) (eq (qcddr d1) nil)
                    (equal (qcadr d1) d2))
              (and (consp d1) (eq (qcdr d1) nil)
                    (equal (qcar d1) d2))))))

```

Chapter 25

Coercions

main algorithms for `canCoerceFrom` and `coerceInteractive`

`coerceInteractive` and `canCoerceFrom` are the two coercion functions for `$InteractiveMode`. They translate RN, RF and RR to QF I, QF P and RE RN, respectively, and call `coerceInt` or `canCoerce`, which both work in the same way (e.g. coercion from `t1` to `t2`):

1. they try to coerce `t1` to `t2` directly (tower coercion), and, if this fails, to coerce `t1` to the last argument of `t2` and embed this last argument into `t2`. These embedding functions are now only defined in the algebra code. (RSS 2-27-87)
2. the tower coercion looks whether there is any applicable local coercion, which means, one defined in boot or in algebra code. If there is an applicable function from a constructor, which is inside the type tower of `t1`, to the top level constructor of `t2`, then this constructor is bubbled up inside `t1`. This means, special coercion functions (defined in boot) are called, which commute two constructors in a tower. Then the local coercion is called on these constructors, which both are on top level now.

example:

let `t1 = A B C D E` (short for `(A (B (C (D (E))))`), where `A ... E` are type constructors), and `t2 = F D G H I J`

there is no coercion from `t1` to `t2` directly, so we try to coerce `t1` to `s1 = D G H I J`, the last argument of `t2`

we create the type `s2 = A D B C E` and call a local coercion `A2A` from `t1` to `s2`, which, by recursively calling `coerce`, bubbles up the constructor `D`

then we call a commute coerce from `s2` to `s3 = D A B C E` and a local coerce `D2D` from `s3` to `s1`

finally we embed `s1` into `t2`, which completes the coercion `t1` to `t2`

the result of `canCoerceFrom` is `TRUE` or `NIL`

the result of `coerceInteractive` is a object or `NIL` (=failed)

all boot coercion functions have the following result:

1. if `u=$fromCoerceable$`, then TRUE or NIL
2. if the coercion succeeds, the coerced value (this may be NIL)
3. if the coercion fails, they throw to a catch point in `coerceByFunction`

defun coerceInteractive

```
[objMode p516]
[objVal p516]
[clearDependentMaps p??]
[throwKeyedMsg p??]
[startTimingProcess p??]
[mkObj p514]
[mkObjWrap p515]
[coerceInt0 p742]
[stopTimingProcess p??]
[$insideCoerceInteractive p??]
[$OutputForm p710]
[$mapName p??]
[$compilingMap p341]
[$NoValueMode p??]
[$EmptyMode p708]
```

— defun coerceInteractive —

```
(defun |coerceInteractive| (triple t2)
  (let (|$insideCoerceInteractive| t1 val expr2 result)
    (declare (special |$insideCoerceInteractive| |$OutputForm|
                      |$mapName| |$compilingMap| |$NoValueMode| |$EmptyMode|))
    (setq t1 (|objMode| triple))
    (setq val (|objVal| triple))
    (cond
      ((or (null t2) (equal t2 |$EmptyMode|)) nil)
      ((equal t2 t1) triple)
      ((equal t2 '|$NoValueMode|) (mkObj val t2))
      (t
       (when (eq (car t2) '|SubDomain|) (setq t2 (second t2)))
       (cond
         ((|member| t1
          '(|Category|) (|Mode|) (|Domain|) (|SubDomain| (|Domain|))))
          (when (equal t2 |$OutputForm|) (mkObj val t2)))
        ((equal t1 '|$NoValueMode|)
         (when |$compilingMap| (|clearDependentMaps| |$mapName| nil))
         (|throwKeyedMsg|
          (format nil
                   "You are trying to use something (probably a loop) in a ~
```

```

situation where a value is expected. In particular, you ~
are trying to convert this to the type %1p . The following ~
information may help: possible function name: %2p")
(list t2 |$mapName|)))
(t
 (setq |$insideCoerceInteractive| t)
 (setq expr2 (equal t2 |$OutputForm|))
 (cond
  (expr2 (|startTimingProcess| '|print|))
  (t (|startTimingProcess| '|coercion|)))
 (setq result
  (cond
   ((and expr2 (equal t1 val)) (mkObj val |$OutputForm|))
   ((and expr2 (eq (car t1) '|Variable|))
    (mkObjWrap (second t1) |$OutputForm|))
   (t (|coerceInt0| triple t2))))
  (cond
   (expr2 (|stopTimingProcess| '|print|))
   (t (|stopTimingProcess| '|coercion|)))
  result))))))

```

defun coerceInt

```

[coerceInt1 p743]
[objMode p516]
[getMinimalVarMode p??]
[unwrap p??]
[objVal p516]
[coerceInt p741]

```

— defun coerceInt —

```

(defun |coerceInt| (triple t2)
 (let (val newMode newVal)
  (if (setq val (|coerceInt1| triple t2))
   val
   (when (eq (car (|objMode| triple)) '|Variable|)
    (setq newMode (|getMinimalVarMode| (|unwrap| (|objVal| triple)) nil))
    (setq newVal (|coerceInt| triple newMode))
    (|coerceInt| newVal t2)))))

```

defun coerceInt0

```
[objVal p516]
[objMode p516]
[conCoerceFrom p??]
[isWrapped p1661]
[intCodeGenCOERCE p??]
[unwrap p??]
[coerceInt0 p742]
[mkObj p514]
[coerceInt p741]
[objSetMode p516]
[$OutputForm p710]
[$Any p708]
[$genValue p346]
```

This is the top level interactive coercion, which transfers all RN, RF and RR into equivalent types

— **defun coerceInt0** —

```
(defun |coerceInt0| (triple t2)
  (prog (val t1 s1 s2 let1 t1p valp ans x)
    (declare (special |$OutputForm| |$Any| |$genValue|))
    (return
      (progn
        (setq val (|objVal| triple))
        (setq t1 (|objMode| triple))
        (cond
          ((eq val '|$fromCoerceable$|) (|canCoerceFrom| t1 t2))
          ((equal t1 t2) triple)
          (t
            (cond
              ((equal t2 |$OutputForm|) (setq s1 t1) (setq s2 t2))
              (t
                (setq s1 t1)
                (setq s2 t2)
                (when (equal s1 s2) (return (mkObj val t2))))))
          (cond
            ; handle case where we must generate code
            ((and (null (|isWrapped| val))
              (or
                (null (eq (car t1) '|FunctionCalled|))
                (null |$genValue|)))
              (|intCodeGenCOERCE| triple t2))
            ((and (equal t1 |$Any|)
              (nequal t2 |$OutputForm|)
              (progn
                (setq let1 (|unwrap| val))
                (setq t1p (car let1))
```

```

      (setq valp (cdr let1))
      let1)
      (setq ans (|coerceInt0| (mkObjWrap valp t1p) t2)))
    ans)
  (t
   (unless (eq s1 t1) (setq triple (mkObj val s1)))
   (when (setq x (|coerceInt| triple s2))
     (cond
      ((eq s2 t2) x)
      (t
       (|objSetModel| x t2)
       x)))))))))

```

defun coerceInt1

This is general interactive coercion. The result is a new triple with type m2 or NIL (= failed). [NRTcompileEvalForm p??]

[absolutelyCanCoerceByCheating p770]

[asTupleAsList p??]

[bottomUp p??]

[coerceByFunction p748]

[coerceInt1 p743]

[coerceInt2Union p765]

[coerceIntAlgebraicConstant p766]

[coerceIntFromUnion p765]

[coerceIntTower p750]

[coerceIntX p768]

[coerceInt p741]

[coerceRetract p777]

[coerceSubDomain p769]

[compareTypeLists p768]

[deconstructT p??]

[evalDomain p1115]

[getFunctionFromDomain p??]

[getValue p??]

[isEqualOrSubDomain p736]

[isSubDomain p??]

[mkAtreeNode p??]

[mkAtree p??]

[mkObjWrap p515]

[mkObj p514]

[nequal p??]

[nreverse0 p??]

```

[objMode p516]
[objVal p516]
[selectLocalMms p??]
[selectMms1 p??]
[transferPropsToNode p??]
[unwrap p??]
[coerceOrCroaker p??]
[$useCoerceOrCroak p??]
[$Integer p711]
[$QuotientField p713]
[$e p315]
[$genValue p346]
[$Symbol p713]
[$AnonymousFunction p708]
[$OutputForm p710]
[$String p713]
[$Any p708]
[$Void p713]
[$NonNegativeInteger p712]
[$PositiveInteger p712]
[$EmptyMode p708]
[$SingleInteger p715]

```

— defun coerceInt1 —

```

(defun |coerceInt1| (triple t2)
  (prog (|$useCoerceOrCroak| t1 sintp t1p valp s body vars tree val symNode
        mms ml oldName intName t3 triplep let1 arg tt ans)
    (declare (special |$useCoerceOrCroak| |$Integer| |$QuotientField|
                      |$e| |$genValue| |$Symbol| |$AnonymousFunction|
                      |$OutputForm| |$String| |$Any| |$Void| |$SingleInteger|
                      |$NonNegativeInteger| |$PositiveInteger| |$EmptyMode|))
    (return
     (seq
      (progn
       (setq |$useCoerceOrCroak| t)
       (cond
        ((equal t2 |$EmptyMode|) nil)
        (t
         (setq t1 (|objMode| triple))
         (cond
          ((equal t1 t2) triple)
          (t
           (setq val (|objVal| triple))
           (cond
            ((|absolutelyCanCoerceByCheating| t1 t2) (mkObj val t2))
            ((|isSubDomain| t2 t1) (|coerceSubDomain| val t1 t2))
            (t

```

```

(cond
  ((equal t1 |$SingleInteger|)
    (cond
      ((or (equal t2 |$Integer|) (equal t2 |$SingleInteger|))
        (return (mkObj val t2)))
      (t
        (setq sintp (typep val 'fixnum))
        (cond
          ((and sintp (equal t2 |$PositiveInteger|) (> val 0))
            (return (mkObj val t2)))
          ((and sintp (equal t2 |$NonNegativeInteger|) (>= val 0))
            (return (mkObj val t2)))))))
  (cond
    ((and (equal t2 |$SingleInteger|)
      (|isEqualOrSubDomain| t1 |$Integer|)
      (integerp val))
      (cond
        ((typep val 'fixnum) (mkObj val t2))
        (t nil)))
    ((equal t2 |$Void|) (mkObj (|voidValue|) |$Void|))
    ((equal t2 |$Any|) (mkObjWrap (cons t1 (|unwrap| val)) '(|Any|)))
    ((and (equal t1 |$Any|)
      (nequal t2 |$OutputForm|)
      (progn
        (setq let1 (|unwrap| val))
        (setq t1p (car let1))
        (setq valp (cdr let1))
        let1)
      (setq ans (|coerceInt| (mkObjWrap valp t1p) t2)))
      ans)
    ; tagged union selectors
    ((or (and (eq (car t1) '|Variable|) (equal (cadr t1) t2))
      (and (eq (car t2) '|Variable|) (equal (cadr t2) t1)))
      (mkObj val t2))
    ((stringp t2)
      (cond
        ((and (eq (first t1) '|Variable|)
          (equal t2 (pname (second t1))))
          (mkObjWrap t2 t2))
        (t
          (setq valp (|unwrap| val))
          (when (and (equal t2 valp)
            (or (equal valp t1) (equal t1 |$String|)))
            (mkObj val t2))))))
    ((eq (first t1) '|Tuple|)
      (|coerceInt1|
        (mkObjWrap
          (|asTupleAsList| (|unwrap| val))
          (list '|List| (setq s (second t1))))
        t2))

```



```

((and (consp t1) (eq (qcar t1) '|Union|))
  (|coerceIntFromUnion| triple t2))
((and (consp t2) (eq (qcar t2) '|Union|))
  (|coerceInt2Union| triple t2))
((and (stringp t1) (equal t2 |$String|))
  (mkObj val |$String|))
((and (stringp t1) (eq (car t2) '|Variable|))
  (when (equal t1 (pname (second t2))) (mkObjWrap (second t2) t2)))
((and (stringp t1) (equal t1 (|unwrap| val)))
  (when (equal t2 |$OutputForm|) (mkObj t1 |$OutputForm|)))
((atom t1) nil)
(t
  (cond
    ((and (equal t1 |$AnonymousFunction|)
      (eq (car t2) '|Mapping|))
      (setq |$useCoerceOrCroak| nil)
      (setq let1 (|unwrap| val))
      (setq vars (cadr let1))
      (setq body (cddr let1))
      (setq vars
        (cond
          ((atom vars) (cons vars nil))
          ((and (consp vars) (eq (qcar vars) '|Tuple|)) (cdr vars))
          (t vars)))
      (cond
        ((nequal (|#| (cddr t2)) (|#| vars)) '|continue|)
        (t
          (setq tree
            (|mkAtree|
              (cons 'adeft
                (cons vars
                  (cons (cons (cadr t2) (cddr t2))
                    (cons (loop for x in (cdr t2) collect nil)
                      body))))))
          (cond
            ((eq
              (catch '|coerceOrCroaker| (|bottomUp| tree)) '|croaked|)
              nil)
            (t (return (|getValue| tree))))))
      (cond
        ((and (equal t1 |$Symbol|) (eq (car t2) '|Mapping|))
          (cond
            ((null (setq mms
              (|selectMms1| (|unwrap| val) nil
                (cddr t2) (cddr t2) (cadr t2))))
              nil)
            (t
              (cond
                ((nequal (cadaar mms) (cadr t2)) nil)
                (|$genValue|

```

```

(mkObjWrap
  (|getFunctionFromDomain|
    (|unwrap| val) (caaar mms) (cddaar mms)) t2))
(t
  (mkObj
    (|NRTcompileEvalForm|
      (|unwrap| val) (cdaar mms) (|evalDomain| (caaar mms)))
      t2))))))
((and (eq (car t1) '|Variable|) (eq (car t2) '|Mapping|))
  (setq mms
    (|selectMms1| (cadr t1) (cadr t2) (cddr t2) (cddr t2) nil))
  (cond
    ((and (null mms)
      (null
        (setq mms
          (|selectMms1| (cadr t1) (cadr t2)
            (cddr t2) (cddr t2) t))))))
    nil)
  (t
    (cond
      ((nequal (cadaar mms) (cadr t2)) nil)
      ((eq (caaaaar mms) '|_FreeFunction_|)
        (mkObj (cdaaar mms) t2))
      (|$genValue|
        (mkObjWrap
          (|getFunctionFromDomain| (cadr t1) (caaar mms)
            (cddaar mms)) t2))
      (t
        (mkObj
          (|NRTcompileEvalForm| (cadr t1) (cdr (caar mms))
            (|evalDomain| (caaar mms)))
          t2))))))
((and (eq (car t1) '|FunctionCalled|) (eq (qcar t2) '|Mapping|))
  (setq symNode (|mkAtreeNode| (cadr t1)))
  (|transferPropsToNode| (cadr t1) symNode)
  (cond
    ((null
      (setq mms
        (|selectLocalMms| symNode (cadr t1) (cddr t2) (cadr t2))))))
    nil)
  (t
    (cond
      ((nequal (cadaar mms) (cadr t2)) nil)
      (t
        (setq ml (cons (cadr t2) (cddr t2)))
        (setq intName
          (when
            (some #'(lambda (mm)
              (setq oldName (second mm))
              (|compareTypeLists| (cdar mm) ml))) mms)

```

```

        (list oldName)))
      (cond
        ((null intName) nil)
        (t (mkObjWrap intName t2))))))
    ((eq (car t1) '|FunctionCalled|)
     (setq t3 (|get| (second t1) '|mode| |$e|))
     (when (and (eq (car t3) '|Mapping|)
                 (setq triplep (|coerceInt| triple t3)))
              (|coerceInt| triplep t2)))
    ((and (eq (car t1) '|Variable|)
           (cons p t2)
           (or (|isEqualOrSubDomain| t2 |$Integer|)
               (equal t2 (list |$QuotientField| |$Integer|))
               (member (car t2)
                        '(|RationalNumber| |BigFloat|
                          |NewFloat| |Float| |DoubleFloat|))))
     nil)
    (t
     (setq ans
      (or
       (|coerceRetract| triple t2)
       (|coerceIntTower| triple t2)
       (progn
        (setq arg (cdr (|deconstructT| t2)))
        (and arg
         (progn
          (setq tt (|coerceInt| triple (|last| arg)))
          (and tt (|coerceByFunction| tt t2))))))
       (or ans
        (and (|isSubDomain| t1 |$Integer|)
              (|coerceInt| (mkObj val |$Integer|) t2))
              (|coerceIntAlgebraicConstant| triple t2)
              (|coerceIntX| val t1 t2))))))))))

```

defun coerceByFunction

— defun coerceByFunction —

```

(defun |coerceByFunction| (t$ m2)
  (let ($ m1 ud x tmp1 a tmp2 b funName mm dc tar args slot dcVector fun fn
        d val env code)
    (declare (special $ |$coerceFailure| |$Boolean|))
    (setq x (|objVal| T$))
    (cond
      ((eq x '|$fromCoerceable$|) nil)

```

```

((eq (car m2) '|Union|) nil)
(t
  (setq m1 (|objMode| t$))
  (cond
    ((and (consp m2) (eq (qcar m2) '|Boolean|)
          (consp m1) (eq (qcar m1) '|Equation|)
          (PROGN
            (setq tmp1 (cdr m1))
            (and (consp tmp1) (eq (cdr tmp1) nil)
              (progn (setq ud (car tmp1)) t))))
      (setq dcVector (|evalDomain| ud))
      (setq fun
        (cond
          ((|isWrapped| x)
            (|NRTcompiledLookup| '= (list |$Boolean| '$ '$) dcVector))
          (t
            (|NRTcompileEvalForm| '= (list |$Boolean| '$ '$) dcVector))))
      (setq fn (car fun))
      (setq d (cdr fun))
      (cond
        ((|isWrapped| x)
          (setq x (|unwrap| x))
          (mkObjWrap (spadcall (car x) (cdr x) fun) m2))
        ((null (and (consp x) (eq (car x) 'spadcall))
          (progn
            (setq tmp1 (cdr x))
            (and (consp tmp1)
              (progn
                (setq a (car tmp1))
                (setq tmp2 (cdr tmp1))
                (and (consp tmp2)
                  (progn
                    (setq b (car tmp2)) t)))))))
          (|keyedSystemError| "Generated code is incorrect for equation" nil))
        (t
          (setq code (list 'spadcall a b fun))
          (mkObj code |$Boolean|))))
    (t
      (cond
        ((null
          (setq mm (|coerceConvertMmSelection| (setq funName '|coerce|) m1 m2))
          (setq mm
            (|coerceConvertMmSelection| (setq funName '|convert|) m1 m2))))
          (when mm
            (setq dc (caar mm))
            (setq tar (cadar mm))
            (setq args (cddar mm))
            (setq slot (cadr mm))
            (setq dcVector (|evalDomain| dc))
            (setq fun

```

```

(cond
  ((|isWrapped| x) (|NRTcompiledLookup| funName slot dcVector))
  (t (|NRTcompileEvalForm| funName slot dcVector)))
(setq fn (car fun))
(setq d (cdr fun))
(cond
  ((equal fn #'|Undef|) nil)
  ((|isWrapped| x)
   (setq $ dcVector)
   (setq val (catch 'coerceFailure (spadcall (|unwrap| x) fun)))
   (cond
    ((equal val |$coerceFailure|) nil)
    (t (mkObjWrap val m2))))
  (t
   (setq env fun)
   (setq code (list 'failCheck (list 'spadcall x env)))
   (mkObj code m2)))))))))

```

defun coerceIntTower

This tries to find a coercion from top level t2 to somewhere inside t1. It builds a new argument type, for which coercion is called recursively [coerceIntPermute p754]

```

[coerceIntSpecial p760]
[last p??]
[coerceIntTest p752]
[constructT p??]
[replaceLast p??]
[deconstructT p??]
[bubbleConstructor p??]
[isValidType p??]
[coerceIntCommute p757]
[coerceIntByMap p761]
[coerceIntTableOrFunction p759]

```

— defun coerceIntTower —

```

(defun |coerceIntTower| (triple t2)
  (let (t1 c1 arg1 tt c arg t1 let1 c2 arg2 s x)
    (cond
      ((setq x (|coerceIntByMap| triple t2)) x)
      ((setq x (|coerceIntCommute| triple t2)) x)
      ((setq x (|coerceIntPermute| triple t2)) x)
      ((setq x (|coerceIntSpecial| triple t2)) x)
      ((setq x (|coerceIntTableOrFunction| triple t2)) x)
    )
  )

```

```

(t
  (setq t1 (|objModel| triple))
  (setq let1 (|deconstructT| t1))
  (setq c1 (car let1))
  (setq arg1 (cdr let1))
  (and arg1
    (progn
      (setq t1 nil)
      (setq arg arg1)
      (loop until (or x (not arg)) do
        (setq tt (|last| arg))
        (setq let1 (|deconstructT| tt))
        (setq c (car let1))
        (setq arg (cdr let1))
        (setq t1 (cons c (cons arg t1)))
      )
      (cond
        ((setq x (and arg (|coerceIntTest| tt t2)))
          (cond
            ((cddr t1)
              (setq s
                (|constructT| c1
                  (|replaceLast| arg1 (|bubbleConstructor| t1))))
              (cond
                ((null (|isValidType| s)) (setq x nil))
                ((setq x (or (|coerceIntByMap| triple s)
                  (|coerceIntTableOrFunction| triple s)))
                  (setq let1 (|deconstructT| (|last| s)))
                  (setq c2 (car let1))
                  (setq arg2 (cdr let1))
                  (setq s (|bubbleConstructor| (list c2 arg2 c1 arg1)))
                  (cond
                    ((null (|isValidType| s)) (setq x nil))
                    ((setq x (|coerceIntCommute| x s))
                      (setq x (or (|coerceIntByMap| x t2)
                        (|coerceIntTableOrFunction| x t2))))))))
            (t
              (setq s (|bubbleConstructor| (list c arg c1 arg1)))
              (cond
                ((null (|isValidType| s)) (setq x nil))
                ((setq x (|coerceIntCommute| triple s))
                  (setq x (or (|coerceIntByMap| x t2)
                    (|coerceIntTableOrFunction| x t2))))))))
          x))))))

```

defun coerceIntTest

This looks whether there exists a table entry or a coercion function. Thus the type can be bubbled before `coerceIntTableOrFunction` is called. [`coerceConvertMmSelection` p752]

```
[assq p1240]
[$CoerceTable p??]
[$useConvertForCoercions p??]
```

— **defun coerceIntTest** —

```
(defun |coerceIntTest| (t1 t2)
  (let (p b)
    (declare (special |$useConvertForCoercions| |$CoerceTable|))
    (or (equal t1 t2)
        (setq b
              (and (setq p (assq (car t1) |$CoerceTable|))
                   (assq (car t2) (cdr p))))
        (or b
            (|coerceConvertMmSelection| 'coerce| t1 t2)
            (and |$useConvertForCoercions|
                 (|coerceConvertMmSelection| 'convert| t1 t2))))))
```

defvar coerceConvertMmSelection;AL

— **initvars** —

```
(defvar |coerceConvertMmSelection;AL| (make-hash-table :test #'equal))
```

defun coerceConvertMmSelection

This calls `selectMms` with `$Coerce=NIL` and tests for required target type. `funName` is either `'coerce` or `'convert`.

```
mmS := [[sig,[targ,arg],:pred] for x in 1 | x is [sig,[.,arg],:pred] and
        hasCorrectTarget(m2,sig) and sig is [dc,targ,oarg] and oarg = m1]
```

```
[selectMms1 p??]
[$reportBottomUpFlag p1009]
[$declaredMode p??]
```

[`$coerceConvertMmSelection`;AL p752]

— **defun coerceConvertMmSelection** —

```
(defun |coerceConvertMmSelection| (&rest g1)
  (labels (
    (checktargets (funName m1 m2)
      (let (|$declaredMode| |$reportBottomUpFlag|)
        (declare (special |$declaredMode| |$reportBottomUpFlag|
                          |coerceConvertMmSelection;AL|))
        (setq |$declaredMode| nil)
        (setq |$reportBottomUpFlag| nil)
        (car
         (loop for x in (|selectMms1| funName m2 (list m1) (list m1) nil)
               collect
               (when (and (|hasCorrectTarget| m2 (car x)) (equal (caddar x) m1))
                 (cons (car x) (cons (cons (cadar x) (list (cadadr x))) (cddr x))))))))))
    (let (g3)
      (if (setq g3 (hget |coerceConvertMmSelection;AL| g1))
          (|CDRwithIncrement| g3)
          (cdr (hput |coerceConvertMmSelection;AL| g1
                    (cons 1 (apply #'checktargets g1))))))))
```

defun hasCorrectTarget

This tests whether the target of signature sig is either m or a union containing m. It also discards TEQ as it is not meant to be used at top-level

— **defun hasCorrectTarget 0** —

```
(defun |hasCorrectTarget| (m sig)
  (let (tar)
    (setq tar (second sig))
    (cond
      ((eq (caar sig) '|TypeEquivalence|) nil)
      ((equal m tar) t)
      ((and (eq (car tar) '|Union|)
            (eq (third tar) '|failed|))
        (equal (second tar) m))
      ((and (eq (car tar) '|Union|)
            (eq (second tar) '|failed|)
            (equal (third tar) m))))))
```


defun coerceIntPermute

[member p1238]
 [objMode p516]
 [computeTTTranspositions p754]
 [coerceInt p741]

— **defun coerceIntPermute** —

```
(defun |coerceIntPermute| (object t2)
  (let (t1 towers ok)
    (cond
      ((|member| t2 '(|Integer|) (|OutputForm|))) nil)
    (t
      (setq t1 (|objMode| object))
      (setq towers (|computeTTTranspositions| t1 t2))
      ; At this point, CAR towers = t1 and last towers should be similar
      ; to t2 in the sense that the components of t1 are in the same order
      ; as in t2. If length towers = 2 and t2 = last towers, we quit to
      ; avoid an infinite loop.
      (cond
        ((or (null towers) (null (cdr towers))) nil)
        ((and (null (cddr towers)) (equal t2 (cadr towers))) nil)
        (t
          (setq ok t)
          ; do the coercions successively, quitting if any fail
          (loop for tt in (cdr towers) while ok do
            (unless (setq object (|coerceInt| object tt)) (setq ok nil)))
          (when ok object))))))
```

—————

defun computeTTTranspositions

[decomposeTypeIntoTower p756]
 [member p1238]
 [nequal p??]
 [msort p??]
 [remdup p??]
 [length p??]
 [list2vec p??]
 [permuteToOrder p756]
 [vec2list p1276]
 [reassembleTowerIntoType p757]

— **defun computeTTTranspositions** —

```

(defun |computeTTTranspositions| (t1 t2)
  (labels (
    (compress (z start len)
      (cond
        ((>= start len) z)
        ((|member| start z) (compress z (1+ start) len))
        (t
          (compress
            (loop for i in z collect (if (> start i) i (1- i))) start len))))))
    (let (tl1 tl2 p2p n1 p2 perms tower tt towers)
      ; decompose t1 into its tower parts
      (setq tl1 (|decomposeTypeIntoTower| t1))
      (setq tl2 (|decomposeTypeIntoTower| t2))
      (cond
        ; if not at least 2 parts, don't bother working here
        ((null (and (cdr tl1) (cdr tl2))) nil)
        (t
          ; determine the relative order of the parts of t1 in t2
          (setq p2 (nreverse0 (loop for d1 in tl1 collect (position d1 tl2))))
          (cond
            ; something not present
            ((|member| (- 1) p2) nil)
            (t
              ; if they are all ascending, this function will do nothing
              (setq p2p (msort p2))
              (cond
                ((equal p2 p2p) nil)
                ; if anything is repeated twice, leave
                ((nequal p2p (msort (remdup p2p))) nil)
                (t
                  ; create a list of permutations that transform the tower parts
                  ; of t1 into the order they are in in t2
                  (setq n1 (|#| tl1))
                  (setq p2 (list2vec (compress p2 0 (|#| (remdup tl1)))))
                  ; p2 now has the same position numbers as p1, we need to determine
                  ; a list of permutations that takes p1 into p2.
                  (setq perms (|permuteToOrder| p2 (- n1 1) 0))
                  (setq towers (list tl1))
                  (setq tower (list2vec tl1))
                  (loop for perm in perms do
                    (setq tt (elt tower (car perm)))
                    (setf (elt tower (car perm)) (elt tower (cdr perm)))
                    (setf (elt tower (cdr perm)) tt)
                    (setq towers (cons (vec2list tower) towers)))
                  (setq towers (nreverse0
                    (loop for tower in towers collect (|reassembleTowerIntoType| tower))))
                  (unless (equal (car towers) t2) (setq towers (cons t2 towers)))
                  (nreverse towers))))))))))

```

defun permuteToOrder

[permuteToOrder p756]

— defun permuteToOrder —

```
(defun |permuteToOrder| (p n start)
  (let (r x perms tt stpos)
    (setq r (- n start))
    (cond
      ((<= r 0) nil)
      ((eql r 1)
       (cond
         ((> (elt p (+ r 1)) (elt p r)) nil)
         (t (list (cons r (+ r 1))))))
      ((equal (elt p start) start) (|permuteToOrder| p n (+ start 1)))
      (t
       (setq stpos nil)
       (loop for i from (+ start 1) to n while (not stpos) do
         (when (equal (elt p i) start) (setq stpos i)))
       (setq perms nil)
       (loop while (not (equal stpos start)) do
         (setq x (- stpos 1))
         (setq perms (cons (cons x stpos) perms))
         (setq tt (elt p stpos))
         (setf (elt p stpos) (elt p x))
         (setf (elt p x) tt)
         (setq stpos x))
       (append (nreverse perms) (|permuteToOrder| p n (+ start 1))))))
```

defun decomposeTypeIntoTower

[decomposeTypeIntoTower p756]

[deconstructT p??]

— defun decomposeTypeIntoTower —

```
(defun |decomposeTypeIntoTower| (tt)
  (let (rd)
    (cond
      ((atom tt) (list tt))
      ((null (cdr (|deconstructT| tt))) (list tt))
```

```
(t
  (setq rd (reverse tt))
  (cons (reverse (cdr rd)) (|decomposeTypeIntoTower| (car rd))))))
```

defun reassembleTowerIntoType

[reassembleTowerIntoType p757]

— defun reassembleTowerIntoType —

```
(defun |reassembleTowerIntoType| (tower)
  (let (let1)
    (cond
      ((atom tower) tower)
      ((null (cdr tower)) (car tower))
      (t
       (setq let1 (reverse tower))
       (|reassembleTowerIntoType|
        (append (nreverse (cddr let1))
                 (list (append (second let1) (list (first let1))))))))))
```

defun coerceIntCommute

[objMode p516]
 [coerceCommuteTest p758]
 [underDomainOf p??]
 [getl p1240]
 [concat p1237]
 [objValUnwrap p516]
 [mkObjWrap p515]
 [\$coerceFailure p??]
 [coerceFailure p??]

— defun coerceIntCommute —

```
(defun |coerceIntCommute| (obj target)
  (let (source s t$ d fun u c)
    (declare (special |$coerceFailure|))
    (setq source (|objMode| obj))
    (cond
```

```

((null (|coerceCommuteTest| source target)) nil)
(t
 (setq s (|underDomainOf| source))
 (setq t$ (|underDomainOf| target))
 (cond
  ((equal source t$) nil)
  ((setq d (car source))
   (setq fun
    (or (get1 d '|coerceCommute|)
        (intern (concat "commute" (princ-to-string d)))))
   (cond
    ((canFuncall? fun)
     (put d '|coerceCommute| fun)
     (setq u (|objValUnwrap| obj))
     (setq c (catch '|coerceFailure| (funcall fun u source s target t$)))
     (cond
      ((equal c |$coerceFailure|) nil)
      ((eq u '|$fromCoerceable$|) c)
      (t (mkObjWrap c target))))))))))

```

defun coerceCommuteTest

```

[isLegitimateMode p??]
[underDomainOf p??]
[deconstructT p??]

```

— defun coerceCommuteTest —

```

(defun |coerceCommuteTest| (t1 t2)
 (let (u1 u2)
  (cond
   ((null (|isLegitimateMode| t2 nil nil)) nil)
   ((null (setq u1 (|underDomainOf| t1))) nil)
   ((null (setq u2 (|underDomainOf| t2))) nil)
   ((null (|underDomainOf| u1)) nil)
   ((null (|underDomainOf| u2)) nil)
  (t
   (and (equal (car (|deconstructT| t1)) (car (|deconstructT| u2)))
        (equal (car (|deconstructT| t2)) (car (|deconstructT| u1)))))))

```

defun coerceIntTableOrFunction

This function does the actual coercion to t2, but not to an argument type of t2 [isValidType p??]

```
[isLegitimateMode p??]
[objMode p516]
[assq p1240]
[coerceByTable p759]
[objVal p516]
[coerceByFunction p748]
[$CoerceTable p??]
```

— defun coerceIntTableOrFunction —

```
(defun |coerceIntTableOrFunction| (triple t2)
  (let (t1 p tmp1)
    (declare (special |$CoerceTable|))
    (cond
      ((null (|isValidType| t2)) nil)
      ((null (|isLegitimateMode| t2 nil nil)) nil)
      (t
       (setq t1 (|objMode| triple))
       (setq p (assq (car t1) |$CoerceTable|))
       (cond
         ((and p (setq tmp1 (assq (car t2) (cdr p))))
          (cond
            ((eq (third tmp1) '|Identity|) (mkObj (|objVal| triple) t2))
            ((eq (second tmp1) '|total|)
             (or (|coerceByTable| (third tmp1) (|objVal| triple) t1 t2 t)
                  (|coerceByFunction| triple t2)))
            (t
             (or (|coerceByTable| (third tmp1) (|objVal| triple) t1 t2 nil)
                   (|coerceByFunction| triple t2))))))
         (t (|coerceByFunction| triple t2))))))
```

defun coerceByTable

```
[isWrapped p1661]
[unwrap p??]
[mkObjWrap p515]
[isTotalCoerce p??]
[mkObj p514]
[mkq p??]
[$OutputForm p710]
```

```
[$coerceFailure p??]
[coerceFailure p??]
```

— **defun coerceByTable** —

```
(defun |coerceByTable| (fn x t1 t2 isTotalCoerce)
  (let (c)
    (declare (special |$coerceFailure| |$OutputForm|))
    (cond
      ((equal t2 |$OutputForm|) nil)
      ((|isWrapped| x)
       (setq x (|unwrap| x))
       (setq c (catch '|coerceFailure| (funcall fn x t1 t2)))
       (unless (equal c |$coerceFailure|) (mkObjWrap c t2)))
      (isTotalCoerce (mkObj (list fn x (mkq t1) (mkq t2)) t2))
      (t
       (mkObj (list '|catchCoerceFailure| (mkq fn) x (mkq t1) (mkq t2)) t2)))))
```

defun catchCoerceFailure

This function is funcalled from code constructed by **coerceByTable**. [unwrap p??]
 [wrap p1233]
 [throwKeyedMsgCannotCoerceWithValue p??]
 [\$coerceFailure p??]
 [coerceFailure p??]

— **defun catchCoerceFailure** —

```
(defun |catchCoerceFailure| (fn x t1 t2)
  (let (c)
    (declare (special |$coerceFailure|))
    (setq c (catch '|coerceFailure| (funcall fn x t1 t2)))
    (if (equal c |$coerceFailure|)
      (|throwKeyedMsgCannotCoerceWithValue| (|wrap| (|unwrap| x)) t1 t2)
      c)))
```

defun coerceIntSpecial

```
[objMode p516]
[coerceInt p741]
```

— defun coerceIntSpecial —

```
(defun |coerceIntSpecial| (triple t2)
  (let (x)
    (when (and (eq (first t2) '|SimpleAlgebraicExtension|)
               (equal (second t2) (|objMode| triple)))
      (unless (setq x (|coerceInt| triple (third t2)))
                (|coerceInt| x t2))))))
```

defun coerceIntByMap

The idea is this: if t1 is D U1 and t2 is D U2, then look for a map: (U1 -> U2, D U1) -> D U2. If it exists, then create a function to do the coercion on the element level and call the map function. [objMode p516]

```
[length p??]
[deconstructT p??]
[nequal p??]
[valueArgsEqual? p764]
[underDomainOf p??]
[member p1238]
[isSubDomain p??]
[sayFunctionSelection p??]
[selectMms1 p??]
[sayFunctionSelectionResult p??]
[compiledLookup p1225]
[evalDomain p1115]
[wrapped2Quote p??]
[objVal p516]
[timedEvaluate p??]
[mkObjWrap p515]
[coerceFailure p??]
```

— defun coerceIntByMap —

```
(defun |coerceIntByMap| (triple t2)
  (let (t1 top u1 u2 args mms fun code val)
    (declare (special |$coerceFailure| |$reportBottomUpFlag|))
    (setq t1 (|objMode| triple))
    (cond
      ((equal t2 t1) triple)
      (t
       (setq u2 (|deconstructT| t2)) ; compute t2 first because of Expression
       (cond
```



```

((eq1 1 (|#| u2)) nil)      ; no under domain
(t
 (setq u1 (|deconstructT| t1))
 (cond
  ((eq1 1 (|#| u1)) nil)
  ((nequal (caar u1) (caar u2)) nil) ; constructors not equal
  ((null (|valueArgsEqual?| t1 t2)) nil)
  (t
   ; handle a couple of special cases for subdomains of Integer
   (setq top (caar u1))
   (setq u1 (|underDomainOf| t1))
   (setq u2 (|underDomainOf| t2))
   (cond
    ((and (|member| top
                  '(|List| |Vector| |Segment| |Stream|
                    |UniversalSegment| |Array|))
         (|isSubDomain| u1 u2))
     (mkObj (|objVal| triple) t2))
    (t
     (setq args (list (list ' |Mapping| u2 u1) t1))
     (when |$reportBottomUpFlag|
      (|sayFunctionSelection| ' |map| args t2 nil
                             "coercion facility (map)"))
     (setq mms (|selectMms1| ' |map| t2 args args nil))
     (when |$reportBottomUpFlag|
      (|sayFunctionSelectionResult| ' |map| args mms))
     (cond
      ((null mms) nil)
      (t
       (setq fun
        (|compiledLookup| ' |map| (cdaar mms) (|evalDomain| (caaar mms))))
       (cond
        ((null fun) nil)
        (t
         (cond
          ((equal (car fun) #' |Undef|) nil)
          (t
           ; now compile a function to do the coercion
           (setq code
            (list 'spadcall
             (list 'cons
              (list 'function ' |coerceIntByMapInner|)
              (mkq (cons u1 u2)))
              (|wrapped2Quote| (|objVal| triple))
              (mkq fun)))
            ; and apply the function
            (setq val (catch ' |coerceFailure| (|timedEvaluate| code)))
            (unless (equal val |$coerceFailure|)
             (mkObjWrap val t2))))))))))))))

```

defun coerceIntByMapInner

This is a helper function for **coerceIntByMap** which constructs a **spadcall** and then evaluates it. [coerceOrThrowFailure p763]

— defun coerceIntByMapInner —

```
(defun |coerceIntByMapInner| (arg g1)
  (|coerceOrThrowFailure| arg (car g1) (cdr g1)))
```

defun coerceOrThrowFailure

[coerceOrRetract p771]
 [mkObjWrap p515]
 [coercionFailure p763]
 [objValUnwrap p516]

— defun coerceOrThrowFailure —

```
(defun |coerceOrThrowFailure| (value t1 t2)
  (let (result)
    (or (setq result (|coerceOrRetract| (mkObjWrap value t1) t2))
        (|coercionFailure|))
    (|objValUnwrap| result)))
```

defun coercionFailure

This does a throw on coercion failure. [coerceFailure p??]

— defun coercionFailure —

```
(defun |coercionFailure| ()
  (declare (special |$coerceFailure|))
  (throw '|coerceFailure| |$coerceFailure|))
```

defun valueArgsEqual?

[u1, :u2] gets passed as the “environment”, which is why we have this slightly clumsy locution JHD 31.July,1990

This returns true if the object-valued arguments to t1 and t2 are the same under coercion

[getdatabase p1196]
 [getConstructorSignature p??]
 [replaceSharps p1144]
 [coerceInt p741]
 [mkObjWrap p515]
 [algEqual p764]
 [objValUnwrap p516]

— defun valueArgsEqual? —

```
(defun |valueArgsEqual?| (t1 t2)
  (let (coSig constrSig t1 t2 newVal done value trip)
    (setq coSig (cdr (getdatabase (car t1) 'cosig)))
    (setq constrSig (cdr (|getConstructorSignature| (car t1))))
    (setq t1 (|replaceSharps| constrSig t1))
    (setq t2 (|replaceSharps| constrSig t2))
    (cond
      ((null (member nil coSig)) t)
      (t
       (setq done nil)
       (setq value t)
       (loop for a1 in (cdr t1) for a2 in (cdr t2) for cs in coSig
             for m1 in t1 for m2 in t2 while (not done) do
             (cond
              ((null cs)
               (setq trip (mkObjWrap a1 m1))
               (setq newVal (|coerceInt| trip m2))
               (cond
                ((null newVal)
                 (setq done t)
                 (setq value nil))
                ((null (|algEqual| a2 (|objValUnwrap| newVal) m2))
                 (setq done t)
                 (setq value nil))))))
             value))))))
```

—————

defun algEqual

This function sees if 2 objects of the same domain are equal by using the = from the domain. The objects should not be wrapped. [spadcall p??]

[compiledLookupCheck p837]
 [evalDomain p1115]
 [\$Boolean p709]

— defun algEqual —

```
(defun |algEqual| (object1 object2 domain)
  (declare (special |$Boolean|))
  (spadcall object1 object2
    (|compiledLookupCheck| '= (list |$Boolean| '$ '$) (|evalDomain| domain))))
```

—————

defun coerceIntFromUnion

— defun coerceIntFromUnion —

```
(defun |coerceIntFromUnion| (object t2)
  (|coerceInt| (|coerceUnion2Branch| object) t2))
```

—————

defun coerceInt2Union

— defun coerceInt2Union —

```
(defun |coerceInt2Union| (object union)
  (let (unionDoms t1 val valp noCoerce)
    (declare (special |$String|))
    (setq unionDoms (|stripUnionTags| (cdr union)))
    (setq t1 (|objMode| object))
    (cond
      ((|member| t1 unionDoms) (|coerceBranch2Union| object union))
      (t
       (setq val (|objVal| object))
       (setq valp (|unwrap| val))
       (cond
         ((and (equal t1 |$String|) (|member| valp unionDoms))
          (|coerceBranch2Union| (mkObj val valp) union))
         (t
          (setq noCoerce t)
          (setq valp nil)
          (loop for d in unionDoms while noCoerce do
```

```
(when (setq valp (|coerceInt| object d)) (setq noCoerce nil)))
(when valp (|coerceBranch2Union| valp union))))))
```

defun coerceBranch2Union

```
[orderUnionEntries p??]
[mkPredList p??]
[stripUnionTags p776]
[position p??]
[keyedSystemError p??]
[objMode p516]
[objVal p516]
[mkObjWrap p515]
[removeQuote p??]
[unwrap p??]
[mkObj p514]
```

— defun coerceBranch2Union —

```
(defun |coerceBranch2Union| (object union)
  (let (predList doms p val tag)
    (setq doms (|orderUnionEntries| (cdr union)))
    (setq predList (|mkPredList| doms))
    (setq doms (|stripUnionTags| doms))
    (setq p (|position| (|objModel| object) doms))
    (cond
      ((equal p (- 1))
       (|keyedSystemError| "The type %1p is not branch of %2p"
        (list (|objModel| object) union)))
      (t
       (setq val (|objVal| object))
       (if (eq (car (setq tag (elt predlist p))) 'eqcar)
           (mkObjWrap (cons (|removeQuote| (third tag)) (|unwrap| val)) union)
           (mkObj val union))))))
```

defun coerceIntAlgebraicConstant

```
[objMode p516]
[objValUnwrap p516]
[ofCategory p717]
```

[mkObjWrap p515]
 [getConstantFromDomain p767]

— defun coerceIntAlgebraicConstant —

```
(defun |coerceIntAlgebraicConstant| (object t2)
  (let (t1 val)
    (setq t1 (|objModel| object))
    (setq val (|objValUnwrap| object))
    (cond
      ((and (|ofCategory| t1 '(|Monoid|))
            (|ofCategory| t2 '(|Monoid|))
            (equal val (|getConstantFromDomain| '(|One|) t1)))
       (mkObjWrap (|getConstantFromDomain| '(|One|) t2) t2))
      ((and (|ofCategory| t1 '(|AbelianMonoid|))
            (|ofCategory| t2 '(|AbelianMonoid|))
            (equal val (|getConstantFromDomain| '(|Zero|) t1)))
       (mkObjWrap (|getConstantFromDomain| '(|Zero|) t2) t2))))))
```

—————

defun getConstantFromDomain

The function **getConstantFromDomain** is used to look up the constants 0 and 1 from the given domainForm.

If isPartialMode returns true then the domain modemap contains the constant \$EmptyMode which indicates that the domain is not fully formed. In this case we return nil.

[isPartialMode p718]
 [opOf p??]
 [lassoc p??]
 [getOperationAlistFromLisplib p134]
 [getConstantFromDomain p767]
 [throwKeyedMsg p??]
 [spadcall p??]
 [compiledLookupCheck p837]
 [evalDomain p1115]

— defun getConstantFromDomain —

```
(defun |getConstantFromDomain| (form domainForm)
  (let (key entryList)
    (unless (|isPartialMode| domainForm)
      (setq key (|opOf| form))
      (setq entryList
        (lassoc key (|getOperationAlistFromLisplib| (car domainForm)))))
```

```

(cond
  ((null (eq (cdr entryList) nil))
    (cond
      ((eq key '|One|) (|getConstantFromDomain| (list '|1|) domainForm))
      ((eq key '|Zero|) (|getConstantFromDomain| (list '|0|) domainForm))
      (t
        (|throwKeyedMsg| "No such constant %1 in domain %2p ."
          (list form domainForm)))))
    (t
      ; there should be exactly one item under this key of that form
      (spadcall
        (|compiledLookupCheck| key (caar entryList)
          (|evalDomain| domainForm))))))

```

defun compareTypeLists

Returns true if every type in t11 is equal or is a subdomain of the corresponding type in t12

— defun compareTypeLists —

```

(defun |compareTypeLists| (t11 t12)
  (not
    (loop for t1 in t11 for t2 in t12
      do (when (null (|isEqualOrSubDomain| t1 t2)) (return t))))

```

defun coerceIntX

Try to coerce a (List (None)) into a different domain [unwrap p??]

[underDomainOf p??]

[coerceInt p741]

[mkObjWrap p515]

— defun coerceIntX —

```

(defun |coerceIntX| (val t1 t2)
  (let (t0)
    (when (and (equal t1 '(|List| (|None|)))
      (null (|unwrap| val))
      (setq t0 (|underDomainOf| t2)))
      (|coerceInt| (mkObjWrap val (list '|List| t0)) t2)))

```

defun coerceSubDomain

[getdatabase p1196]
 [coerceSubDomain p769]
 [coerceImmediateSubDomain p769]

— defun coerceSubDomain —

```
(defun |coerceSubDomain| (val tSuper tSub)
  (let (super)
    (unless (eq val '|$fromCoerceable$|)
      (setq super (getdatabase (car tSub) 'superdomain))
      (cond
        ((equal (car super) tSuper)
         (|coerceImmediateSubDomain| val tSuper tSub (second super)))
        ((|coerceSubDomain| val tSuper (car super))
         (|coerceImmediateSubDomain| val (car super) tSub (second super)))))))
```

—————

defun coerceImmediateSubDomain

[getSubDomainPredicate p769]

— defun coerceImmediateSubDomain —

```
(defun |coerceImmediateSubDomain| (val tSuper tSub pred)
  (when (funcall (|getSubDomainPredicate| tSuper tSub pred) val nil)
    (mkObj val tSub)))
```

—————

defun getSubDomainPredicate

[msubst p??]
 [removeZeroOne p??]
 [interpret p346]
 [mkAtree p??]
 [transferPropsToNode p??]
 [selectLocalMms p??]
 [hput p1234]
 [\$env p315]
 [\$superHash p??]
 [\$Boolean p709]

[[\\$InteractiveFrame p39](#)]

— **defun getSubDomainPredicate** —

```
(defun |getSubDomainPredicate| (tSuper tSub pred)
  (let (|$env| name decl arg predp defn op predfn)
    (declare (special |$env| |$superHash| |$Boolean| |$InteractiveFrame|))
    (setq |$env| |$InteractiveFrame|)
    (cond
      ((setq predfn (hget |$superHash| (cons tSuper tSub))) predfn)
      (t
       (setq name (gensym))
       (setq decl (list '|:| name (list '|Mapping| |$Boolean| tSuper)))
       (|interpret| decl nil)
       (setq arg (gensym))
       (setq predp (msubst arg '|#1| pred))
       (setq defn
        (list 'def (list name arg) '(nil nil) '(nil nil) (|removeZeroOne| predp)))
       (|interpret| defn nil)
       (setq op (|mkAtree| name))
       (|transferPropsToNode| name op)
       (setq predfn (cadar (|selectLocalMms| op name (list tSuper) |$Boolean|)))
       (hput |$superHash| (cons tSuper tSub) predfn)
       predfn))))
```

—————

defun absolutelyCanCoerceByCheating

This typically involves subdomains and towers where the only difference is a subdomain

[[isEqualOrSubDomain— p??](#)]

[[deconstructT p??](#)]

[[nequal p??](#)]

[[absolutelyCanCoerceByCheating p770](#)]

[[\\$SingleInteger p715](#)]

[[\\$Integer p711](#)]

— **defun absolutelyCanCoerceByCheating** —

```
(defun |absolutelyCanCoerceByCheating| (t1 t2)
  (let (let1 let2)
    (declare (special |$Integer| |$SingleInteger|))
    (cond
      ((|isEqualOrSubDomain| t1 t2) t)
      ((and (equal t1 |$SingleInteger|) (equal t2 |$Integer|)) t)
      ((or (atom t1) (atom t2)) nil)
      (t
```

```

(setq let1 (|deconstructT| t1))
(setq let2 (|deconstructT| t2))
(cond
  ((and (equal (car let1) '(|Stream|))
        (equal (car let2) '(|InfiniteTuple|)))
    (cond
      ((nequal (|#| (cdr let1)) (|#| (cdr let2))) nil)
      (t
       (every #'identity
              (loop for x1 in (cdr let1) for x2 in (cdr let2) collect
                    (|absolutelyCanCoerceByCheating| x1 x2))))))
  ((nequal (car let1) (car let2)) nil)
  ((nequal (|#| (cdr let1)) (|#| (cdr let2))) nil)
  (t
   (every #'identity
          (loop for x1 in (cdr let1) for x2 in (cdr let2) collect
                (|absolutelyCanCoerceByCheating| x1 x2))))))

```

defun coerceOrRetract

[coerceInteractive p740]
 [retract p1269]

— defun coerceOrRetract —

```

(defun |coerceOrRetract| (z m)
  (prog (tp tt ans)
    (return
     (cond
      ((setq tp (|coerceInteractive| z m)) tp)
      (t
       (setq tt z)
       (setq ans nil)
       (do () (nil nil)
         (cond
          (ans (return ans))
          (t
           (setq tt (|retract| tt))
           (cond
            ((eq tt 'failed) (return ans))
            (t (setq ans (|coerceInteractive| tt m)))))))
       ans))))

```

defun retract2Specialization

Handle some specialization retraction cases, like matrices [objVal p516]

```
[unwrap p??]
[objMode p516]
[mkObjWrap p515]
[coerceUnion2Branch p775]
[coerceInt p741]
[remdup p??]
[varsInPoly p??]
[mkObj p514]
[member p1238]
[retract p1269]
[objValUnwrap p516]
[objMode p516]
[resolveTypeListAny p??]
[isRectangularList p??]
[get p??]
[isPartialMode p718]
[$e p315]
[$QuotientField p713]
[$Symbol p713]
[$Integer p711]
[$Any p708]
[$NonNegativeInteger p712]
[$PositiveInteger p712]
```

— defun retract2Specialization —

```
(defun |retract2Specialization| (object)
  (prog (val type dom obj dp bad vl tl ep vlp n D num den valp m)
    (declare (special |$e| |$QuotientField| |$Symbol| |$Integer| |$Any|
                      |$NonNegativeInteger| |$PositiveInteger|))
    (return
      (seq
        (progn
          (setq val (|objVal| object))
          (setq valp (|unwrap| val))
          (setq type (|objMode| object))
          (cond
            ; type is Any
            ((equal type |$Any|)
             (setq dom (car valp))
             (setq obj (cdr valp))
             (mkObjWrap obj dom))
            ; type is ['Union, :unionDoms]
            ((eq (car type) '|Union|)
```

```

(|coerceUnion2Branch| object))
; type is Symbol
(equal type |$Symbol|)
(mkObjWrap 1 (list '|OrderedVariableList| (list valp))))
; type is ['OrderedVariableList,var]
((eq (car type) '|OrderedVariableList|)
(|coerceInt|
(mkObjWrap (elt (second type) (- valp 1)) |$Symbol|)
'|Polynomial| (|Integer|))))
; type is ['Polynomial,d]
((eq (car type) '|Polynomial|)
(cond
((eq1 (car valp) 1)
(when (eq1 1 (|#| (remdup (|varsInPoly| valp))))
(|coerceInt| object
(list '|UnivariatePolynomial| (second valp) (second type))))
((eq1 (car valp) 0) (|coerceInt| object (second type)))
(t nil)))
; type is ['Matrix,d]
((eq (car type) '|Matrix|)
(setq n (|#| valp))
(setq m (|#| (elt valp 0)))
(cond
((= n m) (mkObj val (list '|SquareMatrix| n (second type))))
(t (mkObj val (list '|RectangularMatrix| n m (second type)))))
; type is ['RectangularMatrix,n,m,d]
((eq (first type) '|RectangularMatrix|)
(setq n (second type))
(setq m (third type))
(setq d (fourth type))
(when (eq1 n m) (mkObj val (list '|SquareMatrix| n d))))
; type is [agg,d] agg is |Vector|,|Segment|, or |UniversalSegment|
((|member| (first type) '(|Vector| |Segment| |UniversalSegment|))
(cond
((equal (second type) |$PositiveInteger|)
(mkObj val (cons (first type) (list |$NonNegativeInteger|))))
((equal (second type) |$NonNegativeInteger|)
(mkObj val (list (first type) |$Integer|)))))
; type is ['Array,bds,d]
((eq (first type) '|Array|)
(cond
((equal (third type) |$PositiveInteger|)
(mkObj val (list '|Array| (second type) |$NonNegativeInteger|)))
((equal (third type) |$NonNegativeInteger|)
(mkObj val (list '|Array| (second type) |$Integer|)))))
; type is ['List,d]
((eq (car type) '|List|)
(setq d (second type))
(setq dp (second d))
(cond

```

```

; type isnt ['List,dp]
((null (eq (car d) '|List|))
 (cond
  ((equal d |$PositiveInteger|)
   (mkObj val (list '|List| |$NonNegativeInteger|)))
  ((equal d |$NonNegativeInteger|)
   (mkObj val (list '|List| |$Integer|)))
  (null valp) nil)
 (t
  (setq vl nil)
  (setq tl nil)
  (setq bad nil)
  (loop for e in valp while (not bad) do
   (cond
    ((equal (setq ep (|retract| (mkObjWrap e d))) '|failed|)
     (setq bad t))
    (t
     (setq vl (cons (|objValUnwrap| ep) vl))
     (setq tl (cons (|objMode| ep) tl))))))
  (cond
   (bad nil)
   ((equal (setq m (|resolveTypeListAny| tl)) d) nil)
   ((equal d m) nil)
   (t
    (setq vlp nil)
    (setq ep t)
    (loop for e in vl for tt in tl while ep do
     (cond
      ((equal tt m) (setq vlp (cons e vlp)))
      (t
       (setq ep (|coerceInt| (mkObjWrap e tt) m))
       (when ep (setq vlp (cons (|objValUnwrap| ep) vlp))))))
     (mkObjWrap vlp (list '|List| m))))))
  ((equal dp |$PositiveInteger|)
   (mkObj val (list '|List| (list '|List| |$NonNegativeInteger|))))
  ((equal dp |$NonNegativeInteger|)
   (mkObj val (list '|List| (list '|List| |$Integer|))))
  ((or (eq (car dp) '|Variable|)
       (eq (car dp) '|OrderedVariableList|))
   (|coerceInt| object (list '|List| (list '|List| |$Symbol|))))
  (t
   (setq n (|#| valp))
   (setq m (|#| (elt valp 0)))
   (cond
    ((null (|isRectangularList| valp n m)) nil)
    (t (|coerceInt| object (list '|Matrix| dp))))))
; type is ['Expression,d]
((eq (car type) '|Expression|)
 (setq num (car valp))
 (setq den (cdr valp))

```

```

(cond
  ((null (equal (car num) 0)) nil)
  ((null (equal (car den) 0)) nil)
  (t
   (mkObjWrap (cons (cdr num) (cdr den))
    (list |$QuotientField| (second type))))))
; type is ['SimpleAlgebraicExtension,k,rep,.]
; try to retract as an element of rep and see if we can get an element of k
((eq (car type) '|SimpleAlgebraicExtension|)
 (setq valp (|retract| (mkObj val (third type))))
 (do ()
  ((null (and (nequal valp '|failed|)
              (nequal (|objMode| valp) (second type))))
   nil)
  (setq valp (|retract| valp)))
 (unless (equal valp '|failed|) valp))
; type is ['UnivariatePuisseuxSeries,coef,var,cen]
((eq (car type) '|UnivariatePuisseuxSeries|)
 (|coerceInt| object
  (list '|UnivariateLaurentSeries|
        (second type) (third type) (fourth type))))
; type is ['UnivariateLaurentSeries,coef,var,cen]
((eq (car type) '|UnivariateLaurentSeries|)
 (|coerceInt| object
  (list '|UnivariateTaylorSeries|
        (second type) (third type) (fourth type))))
; type is ['FunctionCalled,name]
((eq (car type) '|FunctionCalled|)
 (cond
  ((null (setq m (|get| (second type) '|mode| |$e|))) nil)
  ((|isPartialMode| m) nil)
  (t (mkObj val m))))
(t nil))))))

```

defun coerceUnion2Branch

```

[orderUnionEntries p??]
[objMode p516]
[mkPredList p??]
[stripUnionTags p776]
[objValUnwrap p516]
[evalSharpOne p776]
[mkObj p514]
[objVal p516]

```

— defun coerceUnion2Branch —

```
(defun |coerceUnion2Branch| (object)
  (let (predList doms valp predicate targetType)
    (setq doms (|orderUnionEntries| (cdr (|objModel| object))))
    (setq predList (|mkPredList| doms))
    (setq doms (|stripUnionTags| doms))
    (setq valp (|objValUnwrap| object))
    (loop for typ in doms for pred in predList while (not targetType) do
      (when (|evalSharpOne| pred valp)
        (setq predicate pred)
        (setq targetType typ)))
    (cond
      ((null targetType)
       (|keyedSystemError| "Cannot determine branch of Union." nil))
      ((eq (car predicate) 'eqcar) (mkObjWrap (cdr valp) targetType))
      (t (mkObj (|objVal| object) targetType)))))
```

—————

defun stripUnionTags

— defun stripUnionTags —

```
(defun |stripUnionTags| (doms)
  (loop for dom in doms
    collect (if (eq (first dom) '|:|) (third dom) dom)))
```

—————

defun evalSharpOne

— defun evalSharpOne 0 —

```
(defun |evalSharpOne| (x |#1|)
  (declare (special |#1|))
  (eval '(let() (declare (special |#1|)) ,x)))
```

—————

defun retractUnderDomain

[underDomainOf p??]
 [deconstructT p??]
 [nequal p??]
 [constructT p??]
 [coerceInt p741]

— defun retractUnderDomain —

```
(defun |retractUnderDomain| (object type underDomain)
  (let (ud let1 typep objectp)
    (cond
      ((null (setq ud (|underDomainOf| underDomain))) '|failed|)
      (t
       (setq let1 (|deconstructT| type))
       (cond
         ((nequal 1 (|#| (cdr let1))) '|failed|)
         ((nequal 1 (|#| (car let1))) '|failed|)
         (t
          (setq typep (|constructT| (car let1) (list ud)))
          (cond
            ((setq objectp (|coerceInt| object typep)) objectp)
            (t '|failed|))))))))))
```

—

defun coerceRetract

[objValUnwrap p516]
 [objMode p516]
 [isEqualOrSubDomain p736]
 [mkObjWrap p515]
 [retractByFunction p778]
 [getl p1240]
 [canFuncall? p1237]
 [\$coerceFailure p??]
 [\$SingleInteger p715]
 [\$OutputForm p710]
 [\$Symbol p713]
 [\$Integer p711]
 [coerceFailure p??]

— defun coerceRetract —

```
(defun |coerceRetract| (object t2)
```



```

(let (val t1 fun c)
  (declare (special |$coerceFailure| |$OutputForm| |$Symbol| |$Integer|
                    |$SingleInteger|))
  (cond
    ((eq (setq val (|objValUnwrap| object)) '|$fromCoerceable$|) nil)
    (t
     (setq t1 (|objMode| object))
     (cond
       ((equal t2 |$OutputForm|) nil)
       ((and (|isEqualOrSubDomain| t1 |$Integer|)
              (equal t2 |$SingleInteger|)
              (typep val 'fixnum))
        (mkObjWrap val t2))
       ((equal t1 |$Integer|) nil)
       ((equal t1 |$Symbol|) nil)
       ((equal t1 |$OutputForm|) nil)
       ((setq c (|retractByFunction| object t2)) c)
       ((consp t1)
        (setq fun
          (or (get1 (car t1) '|retract|)
              (intern (concat "retract" (princ-to-string (car t1))))))
        (when (canFuncall? fun)
          (put (car t1) '|retract| fun)
          (setq c (catch '|coerceFailure| (funcall fun object t2))
                (unless (equal c |$coerceFailure|) c)))))))

```

defun retractByFunction

```

[objValUnwrap p516]
[sayFunctionSelection p??]
[findFunctionInDomain p??]
[orderMms p??]
[sayFunctionSelectionResult p??]
[evalDomain p1115]
[compiledLookup p1225]
[coerceUnion2Branch p775]
[mkObjWrap p515]
[spadcall p??]
[objMode p516]
[$reportBottomUpFlag p1009]
[$dollar p??]

```

— defun retractByFunction —

```

(defun |retractByFunction| (object u)

```

```

(let (|$reportBottomUpFlag| $ tt val target funName mms dcVector fun objectp)
(declare (special |$reportBottomUpFlag| $))
(setq tt (|objModel| object))
(setq val (|objValUnwrap| object))
(setq target (list '|Union| u "failed"))
(setq funName '|retractIfCan|)
(when |$reportBottomUpFlag|
  (|sayFunctionSelection| funName (list tt) target
    nil "coercion facility (retraction)"))
(when
  (setq mms
    (append
      (|findFunctionInDomain| funName tt target (list tt) (list tt) nil t)
      (|findFunctionInDomain| funName u target (list tt) (list tt) nil t)))
    (setq mms (|orderMms| funName mms (list tt) (list tt) target)))
(when |$reportBottomUpFlag|
  (|sayFunctionSelectionResult| funName (list tt) mms))
(when mms
  (setq dcVector (|evalDomain| (caaar mms)))
  (setq fun (|compiledLookup| funName (list target tt) dcVector))
  (cond
    ((null fun) nil)
    ((equal (car fun) #'|Undef|) nil)
    (t
     (setq $ dcVector)
     (setq objectp
       (|coerceUnion2Branch| (mkObjWrap (spadcall val fun) target)))
     (when (equal u (|objModel| objectp)) objectp))))))

```

Chapter 26

System Command Handling

The system commands are the top-level commands available in Axiom that can all be invoked by prefixing the symbol with a closed-paren. Thus, to see they copyright you type:

```
)copyright
```

New commands need to be added to this table. The command invoked will be the first entry of the pair and the “user level” of the command will be the second entry.

See:

- The “abbreviations” ([26.3 p 822](#)) command
- The “boot” ([26.4 p 826](#)) command
- The “browse” ([26.5 p 827](#)) command
- The “cd” ([26.11 p 832](#)) command
- The “clear” ([26.12 p 835](#)) command
- The “close” ([26.13 p 845](#)) command
- The “compile” ([26.14 p 847](#)) command
- The “copyright” ([26.15 p 855](#)) command
- The “credits” ([26.16 p 856](#)) command
- The “display” ([26.18 p 864](#)) command
- The “edit” ([26.19 p 872](#)) command
- The “fin” ([26.20 p 876](#)) command

- The “frame” ([3.5 p 25](#)) command
- The “help” ([26.21 p 880](#)) command
- The “history” ([26.23 p 889](#)) command
- The “lisp” ([26.27 p 935](#)) command
- The “library” ([35.1 p 1199](#)) command
- The “license” ([26.26 p 934](#)) command
- The “load” ([?? p ??](#)) command
- The “ltrace” ([26.28 p 936](#)) command
- The “pquit” ([26.29 p 938](#)) command
- The “quit” ([26.30 p 940](#)) command
- The “read” ([26.31 p 943](#)) command
- The “regress” ([26.32 p 950](#)) command
- The “savesystem” ([26.33 p 959](#)) command
- The “set” ([26.51 p 1081](#)) command
- The “show” ([26.52 p 1087](#)) command
- The “spool” ([26.53 p 1101](#)) command
- The “summary” ([26.54 p 1103](#)) command
- The “synonym” ([26.55 p 1105](#)) command
- The “system” ([26.56 p 1108](#)) command
- The “tangle” ([26.57 p 1109](#)) command
- The “trace” ([7.4 p 78](#)) command
- The “trademark” ([26.58 p 1111](#)) command
- The “undo” ([26.59 p 1112](#)) command
- The “what” ([26.61 p 1131](#)) command
- The “with” ([?? p ??](#)) command
- The “workfiles” ([26.62 p 1139](#)) command

26.1 Variables Used

defvar \$systemCommands

— initvars —

```
(defvar |$systemCommands| nil)
```

—————

— postvars —

```
(eval-when (eval load)
  (setq |$systemCommands|
    '(
      (|abbreviations| . |compiler| )
      (|boot| . |development|)
      (|browse| . |development|)
      (|cd| . |interpreter|)
      (|clear| . |interpreter|)
      (|close| . |interpreter|)
      (|compiler| . |compiler| )
      (|copyright| . |interpreter|)
      (|credits| . |interpreter|)
      (|describe| . |interpreter|)
      (|display| . |interpreter|)
      (|edit| . |interpreter|)
      (|fin| . |development|)
      (|frame| . |interpreter|)
      (|help| . |interpreter|)
      (|history| . |interpreter|)
      (|lisp| . |development|)
      (|library| . |interpreter|)
      (|license| . |interpreter|)
      (|load| . |interpreter|)
      (|ltrace| . |interpreter|)
      (|pquit| . |interpreter|)
      (|quit| . |interpreter|)
      (|read| . |interpreter|)
      (|regress| . |interpreter|)
      (|savesystem| . |interpreter|)
      (|set| . |interpreter|)
      (|show| . |interpreter|)
      (|spool| . |interpreter|)
      (|summary| . |interpreter|)
      (|synonym| . |interpreter|)
```

```

(|system|                . |interpreter|)
(|tangle|                . |interpreter|)
(|trace|                 . |interpreter|)
(|trademark|             . |interpreter|)
(|undo|                  . |interpreter|)
(|what|                  . |interpreter|)
(|with|                  . |interpreter|)
(|workfiles|             . |development|)
)))

```

defvar \$syscommands

This table is used to look up a symbol to see if it might be a command.

— **initvars** —

```
(defvar $syscommands nil)
```

— **postvars** —

```

(eval-when (eval load)
  (setq $syscommands (mapcar #'car |$systemCommands|)))

```

defvar \$noParseCommands

This is a list of the commands which have their arguments passed verbatim. Certain functions, such as the lisp function need to be able to handle all kinds of input that will not be acceptable to the interpreter.

— **initvars** —

```
(defvar |$noParseCommands| nil)
```

— **postvars** —

```
(eval-when (eval load)
  (setq |$noParseCommands|
    '(|boot| |copyright| |credits| |fin| |license| |lisp| |pquit| |quit|
      |synonym| |system| |trademark| )))
```

26.2 Functions

defun handleNoParseCommands

The system commands given by the global variable `$noParseCommands` require essentially no preprocessing/parsing of their arguments. Here we dispatch the functions which implement these commands.

There are four standard commands which receive arguments

- boot
- lisp
- synonym
- system

There are six standard commands which do not receive arguments –

- quit
- fin
- pquit
- credits
- copyright
- trademark

As these commands do not necessarily exhaust those mentioned in `$noParseCommands`, we provide a generic dispatch based on two conventions: commands which do not require an argument name themselves, those which do have their names prefixed by “np”. This makes it possible to dynamically define new system commands provided you handle the argument parsing.

defun Handle a top level command

```

[concat p1237]
[expand-tabs p??]
[processSynonyms p325]
[substring p324]
[getFirstWord p809]
[unAbbreviateKeyword p809]
[member p1238]
[handleNoParseCommands p785]
[splitIntoOptionBlocks p787]
[handleTokensizeSystemCommands p787]
[handleParsedSystemCommands p808]
[$tokenCommands p815]
[$noParseCommands p784]
[line p??]

```

— defun doSystemCommand —

```

(defun |doSystemCommand| (string)
  (let (line tok unab optionList)
    (declare (special line |$tokenCommands| |$noParseCommands|))
    (setq string (concat ") " (expand-tabs string)))
    (setq line string)
    (|processSynonyms|)
    (setq string line)
    (setq string (substring string 1 nil))
    (cond
      ((string= string "") nil)
      (t
       (setq tok (|getFirstWord| string))
       (cond
         (tok
          (setq unab (|unAbbreviateKeyword| tok))
          (cond
            ((|member| unab |$noParseCommands|)
             (|handleNoParseCommands| unab string))
            (t
             (setq optionList (|splitIntoOptionBlocks| string))
             (cond
               ((|member| unab |$tokenCommands|)
                (|handleTokensizeSystemCommands| unab optionList))
               (t
                (|handleParsedSystemCommands| unab optionList)
                nil))))))
          (t nil))))))

```

defun Split block into option block

[stripSpaces p811]

— defun splitIntoOptionBlocks —

```
(defun |splitIntoOptionBlocks| (str)
  (let (inString block (blockStart 0) (parenCount 0) blockList)
    (dotimes (i (1- (|#| str)))
      (cond
        ((char= (elt str i) #"\" ) (setq inString (null inString)))
        (t
         (when (and (char= (elt str i) #\" ) (null inString))
           (incf parenCount))
         (when (and (char= (elt str i) #\" ) (null inString))
           (decf parenCount))
         (when
          (and (char= (elt str i) #\" )
               (null inString)
               (= parenCount -1))
           (setq block (|stripSpaces| (subseq str blockStart i)))
           (setq blockList (cons block blockList))
           (setq blockStart (1+ i))
           (setq parenCount 0))))))
    (setq blockList (cons (|stripSpaces| (subseq str blockStart)) blockList))
    (nreverse blockList)))
```

—

defun Tokenize a system command

[dumbTokenize p807]

[tokTran p807]

[systemCommand p788]

— defun handleTokenizeSystemCommands —

```
(defun |handleTokenizeSystemCommands| (unabr optionList)
  (declare (ignore unabr))
  (let (parcmd)
    (setq optionList (mapcar #'(lambda (x) (|dumbTokenize| x)) optionList))
    (setq parcmd
      (mapcar #'(lambda (opt) (mapcar #'(lambda (tok) (|tokTran| tok)) opt))
              optionList))
    (when parcmd (|systemCommand| parcmd))))
```

—

defun Handle system commands

You can type “)?” and see trivial help information. You can type “)? compile” and see compiler related information

```
[selectOptionLC p819]
[helpSpad2Cmd p881]
[selectOption p819]
[commandsForUserLevel p788]
[$options p73]
[$e p315]
[$systemCommands p783]
[$syscommands p784]
[$CategoryFrame p??]
```

— defun systemCommand —

```
(defun |systemCommand| (cmd)
  (let (|$options| |$e| op argl options fun)
    (declare (special |$options| |$e| |$systemCommands| $syscommands
                      |$CategoryFrame|))
    (setq op (caar cmd))
    (setq argl (cdar cmd))
    (setq options (cdr cmd))
    (setq |$options| options)
    (setq |$e| |$CategoryFrame|)
    (setq fun (|selectOptionLC| op $syscommands '|commandError|))
    (if (and argl (eq (elt argl 0) '?)) (not (eq fun '|synonym|)))
        (|helpSpad2Cmd| (cons fun nil))
    (progn
      (setq fun
        (|selectOption| fun (|commandsForUserLevel| |$systemCommands|
                          '|commandUserLevelError|))
      (funcall fun argl))))))
```

defun Select commands matching this user level

The `$UserLevel` contains one of three values: `compiler`, `development`, or `interpreter`. This variable is used to select a subset of commands from the list stored in `$systemCommands`, representing all of the commands that are valid for this level. [satisfiesUserLevel p791]

— defun commandsForUserLevel —

```
(defun |commandsForUserLevel| (arg)
  (let (c)
```

```
(dolist (pair arg)
  (when (|satisfiesUserLevel| (cdr pair))
    (setq c (cons (car pair) c))))
(nreverse c)))
```

defun No command begins with this string

[commandErrorMessage p789]

— **defun commandError** —

```
(defun |commandError| (x u)
  (|commandErrorMessage| ' |command| x u))
```

defun No option begins with this string

[commandErrorMessage p789]

— **defun optionError** —

```
(defun |optionError| (x u)
  (|commandErrorMessage| ' |option| x u))
```

defvar \$oldline

— **initvars** —

```
(defvar $oldline nil "used to output command lines")
```

defun No command/option begins with this string

[commandAmbiguityError p792]

[sayKeyedMsg p43]

```
[terminateSystemCommand p792]
[$oldline p789]
[line p??]
```

— **defun commandErrorMessage** —

```
(defun |commandErrorMessage| (kind x u)
  (declare (special $oldline line))
  (setq $oldline line)
  (if u
    (|commandAmbiguityError| kind x u)
    (progn
      (|sayKeyedMsg| "No %1 begins with %2 ." (list kind x))
      (|terminateSystemCommand|))))
```

—————

defun Option not available at this user level

```
[userLevelErrorMessage p790]
```

— **defun optionUserLevelError** —

```
(defun |optionUserLevelError| (x u)
  (|userLevelErrorMessage| ' |option| x u))
```

—————

defun Command not available at this user level

```
[userLevelErrorMessage p790]
```

— **defun commandUserLevelError** —

```
(defun |commandUserLevelError| (x u)
  (|userLevelErrorMessage| ' |command| x u))
```

—————

defun Command not available error message

```
[commandAmbiguityError p792]
[sayKeyedMsg p43]
```

[[terminateSystemCommand](#) p792]
 [[\\$UserLevel](#) p1080]

— **defun userLevelErrorMessage** —

```
(defun |userLevelErrorMessage| (kind x u)
  (declare (special |$UserLevel|))
  (if u
    (|commandAmbiguityError| kind x u)
    (progn
      (|sayKeyedMsg|
        "Your %1 is ambiguous. The following are abbreviated by %2 : "
        (list |$UserLevel| kind))
      (|terminateSystemCommand|))))
```

—————

defun satisfiesUserLevel

[[\\$UserLevel](#) p1080]

— **defun satisfiesUserLevel 0** —

```
(defun |satisfiesUserLevel| (x)
  (declare (special |$UserLevel|))
  (cond
    ((eq x '|interpreter|) t)
    ((eq |$UserLevel| '|interpreter|) nil)
    ((eq x '|compiler|) t)
    ((eq |$UserLevel| '|compiler|) nil)
    (t t)))
```

—————

defun hasOption

[[stringPrefix?](#) p1403]
 [[pname](#) p1235]

— **defun hasOption** —

```
(defun |hasOption| (al opt)
  (let ((optPname (pname opt)) found)
    (loop for pair in al do
      (when (|stringPrefix?| (pname (car pair)) optPname) (setq found pair))
```

```

    until found)
  found))

```

defun terminateSystemCommand

[tersyscommand p792]

— defun terminateSystemCommand —

```

(defun |terminateSystemCommand| nil (tersyscommand))

```

defun Terminate a system command

[spadThrow p??]

— defun tersyscommand —

```

(defun tersyscommand ()
  (let (chr tok)
    (fresh-line)
    (setq chr 'endoflinechr)
    (setq tok 'end_unit)
    (|spadThrow|)))

```

defun commandAmbiguityError

[sayKeyedMsg p43]
 [sayMSG p45]
 [bright p??]
 [terminateSystemCommand p792]

— defun commandAmbiguityError —

```

(defun |commandAmbiguityError| (kind x u)
  (|sayKeyedMsg|
   "Your %1 is ambiguous. The following are abbreviated by %2 :")

```

```
(list kind x))
(dolist (a u) (|sayMSG| (cons "      " (|bright| a))))
(|terminateSystemCommand|))
```

defun getParserMacroNames

The `$pfMacros` is a list of all of the user-defined macros.

[`$pfMacros` p392]

— **defun getParserMacroNames 0** —

```
(defun |getParserMacroNames| ()
  (declare (special |$pfMacros|))
  (remove-duplicates (mapcar #'car |$pfMacros|)))
```

defun clearParserMacro

Note that if a macro is defined twice this will clear the last instance. Thus:

```
a ==> 3
a ==> 4
)d macros
a ==> 4
)clear prop a
)d macros
a ==> 3
)clear prop a
)d macros
nil
```

```
[ifcdr p??]
[assoc p??]
[remalist p??]
[$pfMacros p392]
```

— **defun clearParserMacro** —

```
(defun |clearParserMacro| (macro)
  (declare (special |$pfMacros|))
  (when (ifcdr (|assoc| macro |$pfMacros|))
```



```
(setq |$pfMacros| (remalist |$pfMacros| macro)))
```

defun displayMacro

```
[isInterpMacro p??]
[sayBrightly p??]
[bright p??]
[concat p1237]
[object2String p??]
[mathprint p??]
[$op p??]
```

— defun displayMacro —

```
(defun |displayMacro| (name)
  (let (|$op| m body args)
    (declare (special |$op|))
    (setq m (|isInterpMacro| name))
    (cond
      ((null m)
        (|sayBrightly|
          (cons " " (append (|bright| name)
                           (cons "is not an interpreter macro." nil)))))
      (t
        (setq |$op| (concat "macro " (|object2String| name)))
        (setq args (car m))
        (setq body (cdr m))
        (setq args
          (cond
            ((null args) nil)
            ((null (cdr args)) (car args))
            (t (cons '|Tuple| args))))
        (|mathprint| (cons 'map (cons (cons args body) nil)))))))
```

defun displayWorkspaceNames

```
[getInterpMacroNames p??]
[getParserMacroNames p793]
[sayMessage p??]
[msort p??]
```

```
[getWorkspaceNames p795]
[sayAsManyPerLineAsPossible p??]
[sayBrightly p??]
[setdifference p??]
```

— defun displayWorkspaceNames —

```
(defun |displayWorkspaceNames| ()
  (let (pmacs names imacs)
    (setq imacs (|getInterpMacroNames|))
    (setq pmacs (|getParserMacroNames|))
    (|sayMessage| "Names of User-Defined Objects in the Workspace:")
    (setq names (msort (append (|getWorkspaceNames|) pmacs)))
    (if names
      (|sayAsManyPerLineAsPossible| (mapcar #'|object2String| names))
      (|sayBrightly| " * None *"))
    (setq imacs (setdifference imacs pmacs))
    (when imacs
      (|sayMessage| "Names of System-Defined Objects in the Workspace:")
      (|sayAsManyPerLineAsPossible| (mapcar #'|object2String| imacs)))))
```

defun getWorkspaceNames

```
;getWorkspaceNames() ==
; NMSORT [n for [n,:.] in CAAR $InteractiveFrame |
; (n ^= "--macros--" and n^= "--flags--")]
```

```
[nmsort p??]
[$InteractiveFrame p39]
```

— defun getWorkspaceNames —

```
(defun |getWorkspaceNames| ()
  (declare (special |$InteractiveFrame|))
  (nmsort (loop for g2 in (caar |$InteractiveFrame|) collect (car g2))))
```

defun fixObjectForPrinting

The \$msgdbPrims variable is set to:

```
(|b| |d| |l| |i| |u| %U |n| |x| |ce| |rj|
 "%U" "%b" "%d" "%l" "%i" "%u" "%U" "%n" "%x" "%ce" "%rj")
```

```
[object2Identifier p??]
[member p1238]
[concat p1237]
[pname p1235]
[$msgdbPrims p42]
```

— defun fixObjectForPrinting —

```
(defun |fixObjectForPrinting| (v)
  (let (vp)
    (declare (special |$msgdbPrims|))
    (setq vp (|object2Identifier| v))
    (cond
      ((eq vp '%) "\\%")
      ((|member| vp |$msgdbPrims|) (concat "\\" (pname vp)))
      (t v))))
```

defun displayProperties,sayFunctionDeps

```
;displayProperties(option,l) ==
; $dependentAlist : local := nil
; $dependeeAlist : local := nil
; [opt,:vl]:= (1 or ['properties])
; imacs := getInterpMacroNames()
; pmacs := getParserMacroNames()
; macros := REMDUP append(imacs, pmacs)
; if vl is ['all] or null vl then
;   vl := MSORT append(getWorkspaceNames(),macros)
; if $frameMessages then sayKeyedMsg("S2IZ0065",[$interpreterFrameName])
; null vl =>
;   null $frameMessages => sayKeyedMsg("S2IZ0066",NIL)
;   sayKeyedMsg("S2IZ0067",[$interpreterFrameName])
; interpFunctionDepAlists()
; for v in vl repeat
;   isInternalMapName(v) => 'iterate
;   pl := getIProplist(v)
;   option = 'flags => getAndSay(v,"flags")
;   option = 'value => displayValue(v,getI(v,'value),nil)
;   option = 'condition => displayCondition(v,getI(v,"condition"),nil)
;   option = 'mode => displayMode(v,getI(v,'mode),nil)
;   option = 'type => displayType(v,getI(v,'value),nil)
;   option = 'properties =>
;     v = "--flags--" => nil
;     pl is [ ['cacheInfo,:.],:.] => nil
;     v1 := fixObjectForPrinting(v)
```

```

;      sayMSG ["Properties of",:bright prefix2String v1,':""]
;      null pl =>
;      v in pmacs =>
;      sayMSG '"    This is a user-defined macro.'
;      displayParserMacro v
;      isInterpMacro v =>
;      sayMSG '"    This is a system-defined macro.'
;      displayMacro v
;      sayMSG '"    none"
;      propsSeen:= nil
;      for [prop,:val] in pl | ^MEMQ(prop,propsSeen) and val repeat
;      prop in '(alias generatedCode IS_-GENSYM mapBody localVars) =>
;      nil
;      prop = 'condition =>
;      displayCondition(prop,val,true)
;      prop = 'recursive =>
;      sayMSG '"    This is recursive."
;      prop = 'isInterpreterFunction =>
;      sayMSG '"    This is an interpreter function."
;      sayFunctionDeps v where
;      sayFunctionDeps x ==
;      if dependents := GETALIST($dependentAlist,x) then
;      null rest dependents =>
;      sayMSG ['"    The following function or rule ",
;      '"depends on this:",:bright first dependents]
;      sayMSG
;      '"    The following functions or rules depend on this:"
;      msg := ["%b",'"    "]
;      for y in dependents repeat msg := ['" ",y,:msg]
;      sayMSG [:nreverse msg,"%d"]
;      if dependees := GETALIST($dependeeAlist,x) then
;      null rest dependees =>
;      sayMSG ['"    This depends on the following function ",
;      '"or rule:",:bright first dependees]
;      sayMSG
;      '"    This depends on the following functions or rules:"
;      msg := ["%b",'"    "]
;      for y in dependees repeat msg := ['" ",y,:msg]
;      sayMSG [:nreverse msg,"%d"]
;      prop = 'isInterpreterRule =>
;      sayMSG '"    This is an interpreter rule."
;      sayFunctionDeps v
;      prop = 'localModemap =>
;      displayModemap(v,val,true)
;      prop = 'mode =>
;      displayMode(prop,val,true)
;      prop = 'value =>
;      val => displayValue(v,val,true)
;      sayMSG ['"    ",prop,':" ": ",val]
;      propsSeen:= [prop,:propsSeen]

```

```
; sayKeyedMsg("S2IZ0068",[option])
; terminateSystemCommand()
```

```
[seq p??]
[getalist p??]
[exit p??]
[sayMSG p45]
[bright p??]
[$dependeeAlist p??]
[$dependentAlist p??]
```

— defun displayProperties,sayFunctionDeps —

```
(defun |displayProperties,sayFunctionDeps| (x)
  (prog (dependents dependees msg)
    (declare (special |$dependeeAlist| |$dependentAlist|))
    (return
      (seq
        (if (setq dependents (getalist |$dependentAlist| x))
          (seq
            (if (null (cdr dependents))
              (exit
                (|sayMSG| (cons " The following function or rule "
                              (cons "depends on this:" (|bright| (car dependents)))))))
            (|sayMSG| " The following functions or rules depend on this:")
            (setq msg (cons " " nil))
            (do ((G166397 dependents (cdr G166397)) (y nil))
              ((or (atom G166397) (progn (setq y (car G166397)) nil)) nil)
              (seq (exit (setq msg (cons " " (cons y msg))))))
            (exit (|sayMSG| (append (nreverse msg) (cons '|%d| nil)))))
          nil)
        (exit
          (if (setq dependees (getalist |$dependeeAlist| x))
            (seq
              (if (null (cdr dependees))
                (exit
                  (|sayMSG| (cons " This depends on the following function "
                                (cons "or rule:" (|bright| (car dependees))))))
                (|sayMSG| " This depends on the following functions or rules:")
                (setq msg (cons " " nil))
                (do ((G166406 dependees (cdr G166406)) (y nil))
                  ((or (atom G166406) (progn (setq y (car G166406)) nil)) nil)
                  (seq (exit (setq msg (cons " " (cons y msg))))))
                (exit (|sayMSG| (append (nreverse msg) (cons '|%d| nil)))))
              nil))))))
```

defun displayValue

```
[sayMSG p45]
[fixObjectForPrinting p795]
[pname p1235]
[objValUnwrap p516]
[objMode p516]
[displayRule p??]
[concat p1237]
[prefix2String p??]
[objMode p516]
[getdatabase p1196]
[concat p1237]
[form2String p??]
[mathprint p??]
[outputFormat p??]
[objMode p516]
[$op p??]
[$EmptyMode p708]
```

— **defun displayValue** —

```
(defun |displayValue| (|$op| u omitVariableNameIfTrue)
  (declare (special |$op|))
  (let (expr op rhs label labmode)
    (declare (special |$EmptyMode|))
    (if (null u)
      (|sayMSG|
        (list '| Value of | (|fixObjectForPrinting| (pname |$op|)) ": (none)"))
      (progn
        (setq expr (|objValUnwrap| u))
        (if (or (and (consp expr) (progn (setq op (qcar expr)) t) (eq op 'map))
              (equal (|objMode| u) |$EmptyMode|))
          (|displayRule| |$op| expr)
          (progn
            (cond
              (omitVariableNameIfTrue
               (setq rhs ": ")
               (setq label "Value (has type ")
              (t
               (setq rhs ": ")
               (setq label (concat "Value of " (pname |$op|) ": "))))
            (setq labmode (|prefix2String| (|objMode| u)))
            (when (atom labmode) (setq labmode (list labmode)))
            (if (eq (getdatabase expr 'constructorkind) '|domain|)
                (|sayMSG| (|concat| " " label labmode rhs (|form2String| expr)))
                (|mathprint|
                 (cons 'concat
```

```

      (cons label
        (append labmode
          (cons rhs
            (cons (|outputFormat| expr (|objMode| u)) nil))))))
    nil))))))

```

defun displayType

```

[sayMSG p45]
[fixObjectForPrinting p795]
[pname p1235]
[prefix2String p??]
[objMode p516]
[concat p1237]
[$op p??]

```

— defun displayType —

```

(defun |displayType| (|$op| u omitVariableNameIfTrue)
  (declare (special |$op|) (ignore omitVariableNameIfTrue))
  (let (type)
    (if (null u)
      (|sayMSG|
        (list "    Type of value of " (|fixObjectForPrinting| (pname |$op|))
          ": (none)"))
      (progn
        (setq type (|prefix2String| (|objMode| u)))
        (when (atom type) (setq type (list type)))
        (|sayMSG|
          (|concat|
            (cons "    Type of value of "
              (cons (|fixObjectForPrinting| (pname |$op|))
                (cons ": " type))))))
        nil))))

```

defun getAndSay

```

[getI p??]
[sayMSG p45]

```

— defun getAndSay —

```
(defun |getAndSay| (v prop)
  (let (val)
    (if (setq val (|getI| v prop))
        (|sayMSG| (cons ' | (cons val (cons '|%1| nil))))
        (|sayMSG| (cons ' | none| (cons '|%1| nil))))))
```

defun displayProperties

```
[getInterpMacroNames p??]
[getParserMacroNames p793]
[remdup p??]
[qcdr p??]
[qcar p??]
[msort p??]
[getWorkspaceNames p795]
[sayKeyedMsg p43]
[interpFunctionDepAlists p805]
[isInternalMapName p??]
[getIProplist p??]
[getAndSay p800]
[displayValue p799]
[getI p??]
[displayCondition p804]
[displayMode p806]
[displayType p800]
[fixObjectForPrinting p795]
[sayMSG p45]
[bright p??]
[prefix2String p??]
[member p1238]
[displayParserMacro p804]
[isInterpMacro p??]
[displayMacro p794]
[displayProperties,sayFunctionDeps p796]
[displayModemap p806]
[exit p??]
[seq p??]
[terminateSystemCommand p792]
[$dependentAlist p??]
[$dependeeAlist p??]
[$frameMessages p1013]
[$interpreterFrameName p38]
```


— defun displayProperties —

```

(defun |displayProperties| (option al)
  (let (|$dependentAlist| |$dependeeAlist| tmp1 opt imacs pmacros macros vl pl
        tmp2 vone prop val propsSeen)
    (declare (special |$dependentAlist| |$dependeeAlist| |$frameMessages|
                      |$interpreterFrameName|))
    (setq |$dependentAlist| nil)
    (setq |$dependeeAlist| nil)
    (setq tmp1 (or al (cons '|properties| nil)))
    (setq opt (car tmp1))
    (setq vl (cdr tmp1))
    (setq imacs (|getInterpMacroNames|))
    (setq pmacros (|getParserMacroNames|))
    (setq macros (remdup (append imacs pmacros)))
    (when (or
            (and (consp vl) (eq (qcdr vl) nil) (eq (qcar vl) '|all|))
            (null vl))
      (setq vl (msort (append (|getWorkspaceNames|) macros))))
    (when |$frameMessages|
      (|sayKeyedMsg| "The name of the current frame is %1 ."
                    (cons |$interpreterFrameName| nil)))
    (cond
      ((null vl)
       (if (null |$frameMessages|)
           (|sayKeyedMsg| "The workspace is empty." nil)
           (|sayKeyedMsg| "The current frame, %1 , is empty."
                         (cons |$interpreterFrameName| nil))))
      (t
       (|interpFunctionDepAlists|)
       (do ((G166440 vl (cdr G166440)) (v nil))
           ((or (atom G166440) (progn (setq v (car G166440)) nil)) nil)
       (seq (exit
            (cond
              ((|isInternalMapName| v) '|iterate|)
              (t
               (setq pl (|getIProplist| v))
               (cond
                 ((eq option '|flags|)
                  (|getAndSay| v '|flags|))
                 ((eq option '|value|)
                  (|displayValue| v (|getI| v '|value|) nil))
                 ((eq option '|condition|)
                  (|displayCondition| v (|getI| v '|condition|) nil))
                 ((eq option '|mode|)
                  (|displayMode| v (|getI| v '|mode|) nil))
                 ((eq option '|type|)
                  (|displayType| v (|getI| v '|value|) nil))
                 ((eq option '|properties|)
                  (cond

```

```

((eq v '|--flags--|)
 nil)
((and (consp pl)
      (progn
        (setq tmp2 (qcar pl))
        (and (consp tmp2) (eq (qcar tmp2) '|cacheInfo|))))
 nil)
(t
 (setq vone (|fixObjectForPrinting| v))
 (|sayMSG|
  (cons "Properties of"
        (append (|bright| (|prefix2String| vone)) (cons ":" nil))))
 (cond
  ((null pl)
   (cond
    ((|member| v pmacs)
     (|sayMSG| "   This is a user-defined macro.")
     (|displayParserMacro| v))
    ((|isInterpMacro| v)
     (|sayMSG| "   This is a system-defined macro.")
     (|displayMacro| v))
    (t
     (|sayMSG| "   none")))))
 (t
  (setq propsSeen nil)
  (do ((G166451 pl (cdr G166451)) (G166425 nil))
      ((or (atom G166451)
           (progn (setq G166425 (car G166451)) nil)
           (progn
            (progn
              (setq prop (car G166425))
              (setq val (cdr G166425))
              G166425)
            nil))
       nil)
   (seq (exit
        (cond
         ((and (null (member prop propsSeen)) val)
          (cond
           ((|member| prop
                '(|alias| |generatedCode| IS-GENSYM
                  |mapBody| |localVars|))
            nil)
           ((eq prop '|condition|)
            (|displayCondition| prop val t))
           ((eq prop '|recursive|)
            (|sayMSG| "   This is recursive."))
           ((eq prop '|isInterpreterFunction|)
            (|sayMSG| "   This is an interpreter function.")
            (|displayProperties,sayFunctionDeps| v))
          ))
        ))
  ))

```

```

((eq prop '|isInterpreterRule|)
  (|sayMSG| "    This is an interpreter rule.")
  (|displayProperties,sayFunctionDeps| v))
((eq prop '|localModemap|)
  (|displayModemap| v val t))
((eq prop '|mode|)
  (|displayMode| prop val t))
(t
  (when (eq prop '|value|)
    (exit
      (when val
        (exit (|displayValue| v val t))))))
  (|sayMSG| (list "    " prop ": " val))
  (setq propsSeen (cons prop propsSeen)))))))))
(t
  (|sayKeyedMsg| "There is nothing to display for option %1 ."
    (cons option nil))))))
(|terminateSystemCommand|))))

```

defun displayParserMacro

```

[|pfPrintSrcLines| p??]
|$pfMacros| p392

```

— defun displayParserMacro —

```

(defun |displayParserMacro| (m)
  (let ((m (assq m |$pfMacros|)))
    (declare (special |$pfMacros|))
    (when m (|pfPrintSrcLines| (caddr m)))))

```

defun displayCondition

```

[bright p??]
[sayBrightly p??]
[concat p1237]
[pred2English p??]

```

— defun displayCondition —

```

(defun |displayCondition| (v condition giveVariableIfNil)

```

```
(let (varPart condPart)
  (when giveVariableIfNil (setq varPart (cons '| of| (|bright| v))))
  (setq condPart (or condition '|true|))
  (|sayBrightly|
   (|concat| '| condition| varPart '|: | (|pred2English| condPart)))))
```

defun interpFunctionDepAlists

```
[putalist p??]
[getalist p??]
[getFlag p??]
[$e p315]
[$dependeeAlist p??]
[$dependentAlist p??]
[$InteractiveFrame p39]
```

— defun interpFunctionDepAlists —

```
(defun |interpFunctionDepAlists| ()
  (let (|$e|)
    (declare (special |$e| |$dependeeAlist| |$dependentAlist|
                      |$InteractiveFrame|))
    (setq |$e| |$InteractiveFrame|)
    (setq |$dependentAlist| (cons (cons nil nil) nil))
    (setq |$dependeeAlist| (cons (cons nil nil) nil))
    (mapcar #'(lambda (dep)
      (let (dependee dependent)
        (setq dependee (first dep))
        (setq dependent (second dep))
        (setq |$dependentAlist|
          (putalist |$dependentAlist| dependee
                    (cons dependent (getalist |$dependentAlist| dependee)))))
        (setq |$dependeeAlist|
          (putalist |$dependeeAlist| dependent
                    (cons dependee (getalist |$dependeeAlist| dependent))))))
      (|getFlag| '|$dependencies|))))
```

defun displayModemap

```
[bright p??]
[sayBrightly p??]
[concat p1237]
[formatSignature p??]
```

— defun displayModemap —

```
(defun |displayModemap| (v val giveVariableIfNil)
  (labels (
    (g (v mm giveVariableIfNil)
      (let (local signature fn varPart prefix)
        (setq local (caar mm))
        (setq signature (cdar mm))
        (setq fn (cadr mm))
        (unless (eq local '|interpOnly|)
          (setq varPart (unless giveVariableIfNil (cons " of" (|bright| v))))
          (setq prefix
            (cons '| Compiled function type| (append varPart (cons '|: | nil))))
            (|sayBrightly| (|concat| prefix (|formatSignature| signature))))))
      (mapcar #'(lambda (x) (g v x giveVariableIfNil)) val)))
```

defun displayMode

```
[bright p??]
[fixObjectForPrinting p795]
[sayBrightly p??]
[concat p1237]
[prefix2String p??]
```

— defun displayMode —

```
(defun |displayMode| (v mode giveVariableIfNil)
  (let (varPart)
    (when mode
      (unless giveVariableIfNil
        (setq varPart (cons '| of| (|bright| (|fixObjectForPrinting| v)))))
      (|sayBrightly|
        (|concat| '| Declared type or mode| varPart '|: |
          (|prefix2String| mode))))))
```

defun Split into tokens delimited by spaces

[stripSpaces p811]

— defun dumbTokenize —

```

(defun |dumbTokenize| (str)
  (let (inString token (tokenStart 0) previousSpace tokenList)
    (dotimes (i (1- (|#| str)))
      (cond
        ((char= (elt str i) #"") ; don't split strings
         (setq inString (null inString))
         (setq previousSpace nil))
        ((and (char= (elt str i) #\space) (null inString))
         (unless previousSpace
           (setq token (|stripSpaces| (subseq str tokenStart i)))
           (setq tokenList (cons token tokenList))
           (setq tokenStart (1+ i))
           (setq previousSpace t)))
        (t
         (setq previousSpace nil))))
    (setq tokenList (cons (|stripSpaces| (subseq str tokenStart)) tokenList))
    (nreverse tokenList)))

```

—

defun Convert string tokens to their proper type

[isIntegerString p808]

— defun tokTran —

```

(defun |tokTran| (tok)
  (let (tmp)
    (if (stringp tok)
      (cond
        ((eql (|#| tok) 0) nil)
        ((setq tmp (|isIntegerString| tok)) tmp)
        ((char= (elt tok 0) #"") (subseq tok 1 (1- (|#| tok))))
        (t (intern tok)))
      tok)))

```

—

defun Is the argument string an integer?

— defun isIntegerString 0 —

```
(defun |isIntegerString| (tok)
  (multiple-value-bind (int len) (parse-integer tok :junk-allowed t)
    (when (and int (= len (length tok))) int)))
```

—————→

defun Handle parsed system commands

[dumbTokenize p807]
 [parseSystemCmd p808]
 [tokTran p807]
 [systemCommand p788]

— defun handleParsedSystemCommands —

```
(defun |handleParsedSystemCommands| (unabr optionList)
  (declare (ignore unabr))
  (let (restOptionList parcmd trail)
    (setq restOptionList (mapcar #'|dumbTokenize| (cdr optionList)))
    (setq parcmd (|parseSystemCmd| (car optionList)))
    (setq trail
      (mapcar #'(lambda (opt)
        (mapcar #'(lambda (tok) (|tokTran| tok)) opt)) restOptionList))
    (|systemCommand| (cons parcmd trail))))
```

—————→

defun Parse a system command

[tokTran p807]
 [stripSpaces p811]
 [parseFromString p340]
 [dumbTokenize p807]

— defun parseSystemCmd —

```
(defun |parseSystemCmd| (opt)
  (let (spaceIndex)
    (if (setq spaceIndex (search " " opt))
```

```
(list
  (|tokTran| (|stripSpaces| (subseq opt 0 spaceIndex)))
  (|parseFromString| (|stripSpaces| (subseq opt spaceIndex))))
(mapcar #'|tokTran| (|dumbTokenize| opt))))
```

defun Get first word in a string

```
[subseq p??]
[stringSpaces p??]
```

— defun getFirstWord —

```
(defun |getFirstWord| (string)
  (let (spaceIndex)
    (setq spaceIndex (search " " string))
    (if spaceIndex
      (|stripSpaces| (subseq string 0 spaceIndex))
      string)))
```

defun Unabbreviate keywords in commands

```
[selectOptionLC p819]
[selectOption p819]
[commandsForUserLevel p788]
[$systemCommands p783]
[$currentLine p??]
[$syscommands p784]
[line p??]
```

— defun unAbbreviateKeyword —

```
(defun |unAbbreviateKeyword| (x)
  (let (xp)
    (declare (special |$systemCommands| |$currentLine| $syscommands line))
    (setq xp (|selectOptionLC| x $syscommands '|commandErrorIfAmbiguous|))
    (cond
      ((null xp)
       (setq xp '|system|)
       (setq line (concat ")system " (substring line 1 (1- (|#| line)))))
       (setq |$currentLine| line)))
```



```
(|selectOption| xp (|commandsForUserLevel| |$systemCommands|)
  '|commandUserLevelError|))
```

defun The command is ambiguous error

```
[commandAmbiguityError p792]
[$oldline p789]
[line p??]
```

— defun commandErrorIfAmbiguous —

```
(defun |commandErrorIfAmbiguous| (x u)
  (declare (special $oldline line))
  (when u
    (setq $oldline line)
    (|commandAmbiguityError| '|command| x u)))
```

```
[stripSpaces p811]
[nplisp p812]
[stripLisp p811]
[sayKeyedMsg p43]
[npboot p812]
[npsystem p812]
[npsynonym p813]
[member p1238]
[concat p1237]
```

— defun handleNoParseCommands —

```
(defun |handleNoParseCommands| (unab string)
  (let (spaceindex funname)
    (setq string (|stripSpaces| string))
    (setq spaceindex (search " " string))
    (cond
      ((eq unab '|lisp|)
       (if spaceindex
        (|nplisp| (|stripLisp| string))
        (|sayKeyedMsg| "Your argument list is not valid." nil)))
      ((eq unab '|boot|)
       (if spaceindex
        (|npboot| (subseq string (1+ spaceindex)))
```

```

    (|sayKeyedMsg| "Your argument list is not valid." nil)))
  ((eq unab '|system|)
   (if spaceindex
    (|npsystem| unab string)
    (|sayKeyedMsg| "Your argument list is not valid." nil)))
  ((eq unab '|synonym|)
   (if spaceindex
    (|npsynonym| unab (subseq string (1+ spaceindex)))
    (|npsynonym| unab "")))
  ((null spaceindex)
   (funcall unab))
  ((|member| unab '(|quit| |fin| |pquit| |credits| |copyright| |trademark|))
   (|sayKeyedMsg| "Your argument list is not valid." nil))
  (t
   (setq funname (intern (concat "np" (string unab))))
   (funcall funname (subseq string (1+ spaceindex))))))

```

defun Remove the spaces surrounding a string

TPDHERE: This should probably be a macro or eliminated

— defun stripSpaces 0 —

```

(defun |stripSpaces| (str)
  (string-trim '(#\space) str))

```

defun Remove the lisp command prefix

— defun stripLisp 0 —

```

(defun |stripLisp| (str)
  (if (string= (subseq str 0 4) "lisp")
      (subseq str 4)
      str))

```

defun Handle the)lisp command

[*\$ans* p??]

— defun nplisp 0 —

```
(defun |nplisp| (str)
  (declare (special |$ans|))
  (setq |$ans| (eval (read-from-string str)))
  (format t "~&Value = ~S%" |$ans|))
```

defun The)boot command is no longer supported

TPDHERE: Remove all boot references from top level

— defun npboot 0 —

```
(defun |npboot| (str)
  (declare (ignore str))
  (format t "The )boot command is no longer supported~%"))
```

defun Handle the)system command

Note that `unAbbreviateKeyword` returns the word “system” for unknown words so we have to search for this case. This complication may never arrive in practice.

[*sayKeyedMsg* p43]

— defun npsystem —

```
(defun |npsystem| (unab str)
  (let (spaceIndex sysPart)
    (setq spaceIndex (search " " str))
    (cond
      ((null spaceIndex) (|sayKeyedMsg| "Unknown system command: %1" (list str)))
      (t
       (setq sysPart (subseq str 0 spaceIndex))
       (if (search sysPart (string unab))
           (obey (subseq str (1+ spaceIndex)))
           (|sayKeyedMsg| "Unknown system command: %1" (list sysPart)))))))
```

defun Handle the)synonym command

[npProcessSynonym p813]

— defun npsynonym —

```
(defun |npsynonym| (unab str)
  (declare (ignore unab))
  (|npProcessSynonym| str))
```

—————

defun Handle the synonym system command

```
[printSynonyms p813]
[processSynonymLine p1107]
[putalist p??]
[terminateSystemCommand p792]
[$CommandSynonymAlist p817]
```

— defun npProcessSynonym —

```
(defun |npProcessSynonym| (str)
  (let (pair)
    (declare (special |$CommandSynonymAlist|))
    (if (= (length str) 0)
      (|printSynonyms| nil)
      (progn
        (setq pair (|processSynonymLine| str))
        (if |$CommandSynonymAlist|
          (putalist |$CommandSynonymAlist| (car pair) (cdr pair)))
          (setq |$CommandSynonymAlist| (cons pair nil))))
      (|terminateSystemCommand|))))
```

—————

defun printSynonyms

```
[specialChar p1166]
[filterListOfStringsWithFn p1135]
[synonymsForUserLevel p1106]
[printLabelledList p814]
[$CommandSynonymAlist p817]
[$linelength p1051]
```

— defun printSynonyms —

```
(defun |printSynonyms| (patterns)
  (let (ls t1)
    (declare (special |$CommandSynonymAlist| $linelength))
    (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " System Command Synonyms ")
    (setq ls
      (|filterListOfStringsWithFn| patterns
        (do ((t2 (|synonymsForUserLevel| |$CommandSynonymAlist|) (cdr t2)))
          ((atom t2) (nreverse0 t1))
          (push (cons (princ-to-string (caar t2)) (cdar t2)) t1))
        #'car))
    (|printLabelledList| ls "user" "synonyms" "") patterns)))
```

defun Print a list of each matching synonym

The prefix goes before each element on each side of the list, eg, ")”

```
[sayMessage p??]
[blankList p??]
[substring p324]
[entryWidth p??]
[sayBrightly p??]
[concat p1237]
[fillerSpaces p310]
```

— defun printLabelledList —

```
(defun |printLabelledList| (ls label1 label2 prefix patterns)
  (let (comm syn wid)
    (if (null ls)
      (if (null patterns)
        (|sayMessage| (list " No " label1 "-defined " label2 " in effect."))
        (|sayMessage|
          ‘(“ No “ ,label1 "-defined “ ,label2 “ satisfying patterns:”
            |%l| “ “ ,@(append (|blankList| patterns) (list nil))))))
      (progn
        (when patterns
          (|sayMessage|
            ‘(,label1 "-defined “ ,label2 “ satisfying patterns:” |%l| “ “
              ,@(append (|blankList| patterns) (list nil))))))
        (do ((t1 ls (cdr t1)))
          ((atom t1) nil)
          (setq syn (caar t1))
```

```

(setq comm (cdar t1))
(when (string= (substring syn 0 1) "|")
  (setq syn (substring syn 1 nil)))
(when (string= syn "%i") (setq syn "%i "))
(setq wid (max (- 30 (|#| syn)) 1))
(|sayBrightly|
  (|concat| prefix syn (|fillerSpaces| wid ".")
    " " prefix comm)))
(|sayBrightly| "")))))

```

defvar \$tokenCommands

This is a list of the commands that expect the interpreter to parse their arguments. Thus the history command expects that Axiom will have tokenized and validated the input before calling the history function.

— **initvars** —

```
(defvar |$tokenCommands| nil)
```

— **postvars** —

```

(eval-when (eval load)
  (setq |$tokenCommands|
    '( |abbreviations|
      |cd|
      |clear|
      |close|
      |compiler|
      |depends|
      |display|
      |describe|
      |edit|
      |frame|
      |frame|
      |help|
      |history|
      |input|
      |library|
      |load|
      |ltrace|
      |read|

```

```
|regress|
|savesystem|
|set|
|spool|
|tangle|
|undo|
|what|
|with|
|workfiles|
)))
```

defvar \$InitialCommandSynonymAlist

Axiom can create “synonyms” for commands. We create an initial table of synonyms which are in common use.

— **initvars** —

```
(defvar |$InitialCommandSynonymAlist| nil)
```

defun Print the current version information

```
[*yearweek* p??]
[*build-version* p??]
```

— **defun axiomVersion 0** —

```
(defun axiomVersion ()
  (declare (special *build-version* *yearweek*))
  (concatenate 'string "Axiom " *build-version* " built on " *yearweek*))
```

— **postvars** —

```
(eval-when (eval load)
  (setq |$InitialCommandSynonymAlist|
    '(
      (|?| . "what commands")
      (|ap| . "what things")
```

```

(|apr|      . "what things")
(|apropos|   . "what things")
(|cache|     . "set functions cache")
(|cl|        . "clear")
(|cms|       . "system")
(|co|        . "compiler")
(|d|         . "display")
(|dep|       . "display dependents")
(|dependents| . "display dependents")
(|e|         . "edit")
(|expose|    . "set expose add constructor")
(|fns|       . "exec spadfn")
(|fortran|   . "set output fortran")
(|h|         . "help")
(|hd|        . "system hypertex &")
(|kclam|     . "boot clearClams ( )")
(|killcaches| . "boot clearConstructorAndLisplibCaches ( )")
(|prompt|    . "set message prompt")
(|recurrence| . "set functions recurrence")
(|restore|   . "history )restore")
(|save|      . "history )save")
(|startGraphics| . "system $AXIOM/lib/viewman &")
(|startNAGLink| . "system $AXIOM/lib/nagman &")
(|stopGraphics| . "lisp (|sockSendSignal| 2 15)")
(|stopNAGLink| . "lisp (|sockSendSignal| 8 15)")
(|time|      . "set message time")
(|type|      . "set message type")
(|unexpose| . "set expose drop constructor")
(|version|   . "lisp (axiomVersion)")
(|w|         . "what")
(|wc|        . "what categories")
(|wd|        . "what domains")
(|who|       . "lisp (pprint credits)")
(|wp|        . "what packages")
(|ws|        . "what synonyms")
)))

```

defvar \$CommandSynonymAlist

The actual list of synonyms is initialized to be the same as the above initial list of synonyms. The user synonyms that are added during a session are pushed onto this list for later lookup.

— **initvars** —

```
(defvar |$CommandSynonymAlist| nil)
```

— postvars —

```
(eval-when (eval load)
  (setq |$CommandSynonymAlist| (copy-alist |$InitialCommandSynonymAlist|)))
```

defun ncloopCommand

The `$systemCommandFunction` is set in `SpadInterpretStream` to point to the function `InterpExecuteSpadSystemCommand`. The system commands are handled by the function in the “hook” variable `$systemCommandFunction` which has the default function `InterpExecuteSpadSystemCommand`. Thus, when a system command is entered this function is called.

The only exception is the `)include` function which inserts the contents of a file inline in the input stream. This is useful for processing `)read` of input files.

```
[ncloopPrefix? p818]
[ncloopInclude1 p929]
[$systemCommandFunction p??]
[$systemCommandFunction p??]
```

— defun ncloopCommand —

```
(defun |ncloopCommand| (line n)
  (let (a)
    (declare (special |$systemCommandFunction|))
    (if (setq a (|ncloopPrefix?| ")include" line))
        (|ncloopInclude1| a n)
        (progn
          (funcall |$systemCommandFunction| line)
          n))))
```

defun ncloopPrefix?

If we find the prefix string in the whole string starting at position zero we return the remainder of the string without the leading prefix.

— defun ncloopPrefix? 0 —

```
(defun |ncloopPrefix?| (prefix whole)
```

```
(when (eql (search prefix whole) 0)
      (subseq whole (length prefix))))
```

defun selectOptionLC

```
[selectOption p819]
[downcase p1272]
[object2Identifier p??]
```

— defun selectOptionLC —

```
(defun |selectOptionLC| (x l errorFunction)
  (|selectOption| (downcase (|object2Identifier| x)) l errorFunction))
```

defun selectOption

```
[member p1238]
[identp p1237]
[stringPrefix? p1403]
[pname p1235]
[qcdr p??]
[qcar p??]
```

— defun selectOption —

```
(defun |selectOption| (x l errorfunction)
  (let (u y)
    (cond
      ((|member| x l) x)
      ((null (identp x))
       (cond
         (errorfunction (funcall errorfunction x u))
         (t nil)))
      (t
       (setq u
              (let (t0)
                (do ((t1 l (cdr t1)) (y nil))
                    ((or (atom t1) (progn (setq y (car t1)) nil)) (nreverse0 t0))
                  (if (|stringPrefix?| (pname x) (pname y))
                      (setq t0 (cons y t0)))))))
```

```
(cond
  ((and (consp u) (eq (qcdr u) nil) (progn (setq y (qcar u)) t)) y)
  (errorfunction (funcall errorfunction x u))
  (t nil))))
```

26.3)abbreviations Command

abbreviations man page

— abbreviations.help —

```
=====
A.2. )abbreviation
=====
```

User Level Required: compiler

Command Syntax:

-)abbreviation query [nameOrAbbrev]
-)abbreviation category abbrev fullname [quiet]
-)abbreviation domain abbrev fullname [quiet]
-)abbreviation package abbrev fullname [quiet]
-)abbreviation remove nameOrAbbrev

Command Description:

This command is used to query, set and remove abbreviations for category, domain and package constructors. Every constructor must have a unique abbreviation. This abbreviation is part of the name of the subdirectory under which the components of the compiled constructor are stored. Furthermore, by issuing this command you let the system know what file to load automatically if you use a new constructor. Abbreviations must start with a letter and then be followed by up to seven letters or digits. Any letters appearing in the abbreviation must be in uppercase.

When used with the query argument, this command may be used to list the name associated with a particular abbreviation or the abbreviation for a constructor. If no abbreviation or name is given, the names and corresponding abbreviations for all constructors are listed.

The following shows the abbreviation for the constructor List:

```
)abbreviation query List
```

The following shows the constructor name corresponding to the abbreviation NNI:

```
)abbreviation query NNI
```

The following lists all constructor names and their abbreviations.

```
)abbreviation query
```

To add an abbreviation for a constructor, use this command with category, domain or package. The following add abbreviations to the system for a category, domain and package, respectively:

```
)abbreviation domain    SET Set
)abbreviation category  COMPCAT  ComplexCategory
)abbreviation package   LIST2MAP ListToMap
```

If the `)quiet` option is used, no output is displayed from this command. You would normally only define an abbreviation in a library source file. If this command is issued for a constructor that has already been loaded, the constructor will be reloaded next time it is referenced. In particular, you can use this command to force the automatic reloading of constructors.

To remove an abbreviation, the `remove` argument is used. This is usually only used to correct a previous command that set an abbreviation for a constructor name. If, in fact, the abbreviation does exist, you are prompted for confirmation of the removal request. Either of the following commands will remove the abbreviation `VECTOR2` and the constructor name `VectorFunctions2` from the system:

```
)abbreviation remove VECTOR2
)abbreviation remove VectorFunctions2
```

Also See:
o `)compile`

defun abbreviations

[abbreviationsSpad2Cmd [p822](#)]

— defun abbreviations —

```
(defun |abbreviations| (l)
  (|abbreviationsSpad2Cmd| l))
```

defun abbreviationsSpad2Cmd

[listConstructorAbbreviations [p824](#)]
[abbreviation? `p??`]

[\[abbQuery p865\]](#)
[\[deldatabase p1195\]](#)
[\[size p1236\]](#)
[\[sayKeyedMsg p43\]](#)
[\[mkUserConstructorAbbreviation p??\]](#)
[\[setdatabase p1195\]](#)
[\[seq p??\]](#)
[\[exit p??\]](#)
[\[opOf p??\]](#)
[\[helpSpad2Cmd p881\]](#)
[\[selectOptionLC p819\]](#)
[\[qcar p??\]](#)
[\[qcdr p??\]](#)
[\[\\$options p73\]](#)

— **defun abbreviationsSpad2Cmd** —

```

(defun |abbreviationsSpad2Cmd| (arg)
  (let (abopts quiet opt key type constructor t2 a b al)
    (declare (special |$options|))
    (if (null arg)
      (|helpSpad2Cmd| '(|abbreviations|))
      (progn
        (setq abopts '(|query| |domain| |category| |package| |remove|))
        (setq quiet nil)
        (do ((t0 |$options| (cdr t0)) (t1 nil))
            ((or (atom t0)
                 (progn (setq t1 (car t0)) nil)
                 (progn (setq opt (car t1)) t1) nil))
          nil)
        (setq opt (|selectOptionLC| opt '(|quiet|) '|optionError|))
        (when (eq opt '|quiet|) (setq quiet t)))
    (when
      (and (consp arg)
           (progn
            (setq opt (qcar arg))
            (setq al (qcdr arg))
            t))
      (setq key (|opOf| (car al)))
      (setq type (|selectOptionLC| opt abopts '|optionError|))
      (cond
        ((eq type '|query|)
         (cond
          ((null al) (|listConstructorAbbreviations|))
          ((setq constructor (|abbreviation?| key))
           (|abbQuery| constructor))
          (t (|abbQuery| key))))
        ((eq type '|remove|)

```

```

(deldatabase key 'abbreviation))
((oddp (size al))
 (|sayKeyedMsg|
  (format nil
   "%1 must be followed by an alternating list of abbreviation(s) ~
   and name(s). Issue )abbrev ? for more information.")
  (list type)))
(t
 (do () (nil nil)
  (seq
   (exit
    (cond
     ((null al) (return '|fromLoop|))
     (t
      (setq t2 al)
      (setq a (car t2))
      (setq b (cadr t2))
      (setq al (cddr t2))
      (|mkUserConstructorAbbreviation| b a type)
      (setdatabase b 'abbreviation a)
      (setdatabase b 'constructorkind type))))))
 (unless quiet
  (|sayKeyedMsg| "%1 abbreviates % %2 %3 %"
   (list a type (|op0f| b))))))))

```

defun listConstructorAbbreviations

[\[upcase p1272\]](#)
[\[queryUserKeyedMsg p??\]](#)
[\[string2id-n p??\]](#)
[\[whatSpad2Cmd p1131\]](#)
[\[sayKeyedMsg p43\]](#)

— defun listConstructorAbbreviations —

```

(defun |listConstructorAbbreviations| ()
 (let (x)
  (setq x
   (upcase
    (|queryUserKeyedMsg|
     (format nil
      "You have requested that all abbreviations be displayed. As there are ~
      several hundred abbreviations, please confirm your request by ~
      typing y or yes and then pressing Enter :")
     nil)))

```

```
(if (member (string2id-n x 1) '(Y YES))
  (progn
    (|whatSpad2Cmd| '(|categories|))
    (|whatSpad2Cmd| '(|domains|))
    (|whatSpad2Cmd| '(|packages|)))
  (|sayKeyedMsg|
    (format nil
      "Since you did not respond with y or yes the list of abbreviations ~
      will not be displayed.")
    nil))))
```

26.4)boot Command

boot man page

— boot.help —

```
=====
A.3. )boot
=====

User Level Required:  development

Command Syntax:

    - )boot bootExpression

Command Description:

This command is used by AXIOM system developers to execute expressions
written in the BOOT language. For example,

)boot times3(x) == 3*x

creates and compiles the Lisp function ‘‘times3’’ obtained by translating the
BOOT code.

Also See:
o )fin
o )lisp
o )set
o )system
```

¹

This command is in the list of `$noParseCommands` [26.1](#) which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` [26.2](#)

¹ “fin” ([26.20 p 876](#)) “lisp” ([26.27 p 935](#)) “set” ([26.51 p 1081](#)) “system” ([26.56 p 1108](#))

26.5)browse Command

browse man page

— browse.help —

User Level Required: development

Command Syntax:

)browse

Command Description:

This command is used by Axiom system users to start the Axiom top level loop listening for browser connections.

—————

26.6 Overview

The Axiom book on the help browser is a complete rewrite of the hyperdoc mechanism. There are several components that were needed to make this function. Most of the web browser components are described in bookvol11.pamphlet. This portion describes some of the design issues needed to support the interface.

The axServer command takes a port (defaulting to 8085) and a program to handle the browser interaction (defaulting to multiServ). The axServer function opens the port, constructs the stream, and passes the stream to multiServ. The multiServ loop processes one interaction at a time.

So the basic process is that the Axiom “)browse” command opens a socket and listens for http requests. Based on the type of request (either 'GET' or 'POST') and the content of the request, which is one of:

- command - algebra request/response
- lispcall - a lisp s-expression to be evaluated
- showcall - an Axiom)show command

the multiServ function will call a handler function to evaluate the command line and construct a response. GET requests result in a new browser page. POST requests result in an inline result.

Most responses contain the fields:

- stepnum - this is the Axiom step number
- command - this is the original command from the browser
- algebra - this is the Axiom 2D algebra output
- mathml - this is the MathML version of the Axiom algebra
- type - this is the type of the Axiom result

26.7 Browsers, MathML, and Fonts

This work has the Firefox browser as its target. Firefox has built-in support for MathML, javascript, and XMLHttpRequests. More details are available in bookvol11.pamphlet but the very basic machinery for communication with the browser involves a dance between the browser and the multiServ function (see the axserver.spad.pamphlet).

In particular, a simple request is embedded in a web page as:

```
<ul>
  <li>
    <input type="submit" id="p3" class="subbut"
      onclick="makeRequest('p3');"
      value="sin(x)" />
    <div id="ansp3"><div></div></div>
  </li>
</ul>
```

which says that this is an html “input” field of type “submit”. The CSS display class is “subbut” which is of a different color than the surrounding text to make it obvious that you can click on this field. Clickable fields that have no response text are of class “noresult”.

The javascript call to “makeRequest” gives the “id” of this input field, which must be unique in the page, as an argument. In this case, the argument is ‘p3’. The “value” field holds the display text which will be passed back to Axiom as a command.

When the result arrives the “showanswer” function will select out the mathml field of the response, construct the “id” of the html div to hold the response by concatenating the string “ans” (answer) to the “id” of the request resulting, in this case, as “ansp3”. The “showanswer” function will find this div and replace it with a div containing the mathml result.

The “makeRequest” function is:

```
function makeRequest(arg) {
  http_request = new XMLHttpRequest();
  var command = cmdline(arg);
  //alert(command);
  http_request.open('POST', '127.0.0.1:8085', true);
  http_request.onreadystatechange = handleResponse;
```

```

http_request.setRequestHeader('Content-Type', 'text/plain');
http_request.send("command="+command);
return(false);

```

It contains a request to open a local server connection to Axiom, sets “handleResponse” as the function to call on reply, sets up the type of request, fills in the command field, and sends off the http request.

When a response is received, the “handleResponse” function checks for the correct reply state, strips out the important text, and calls “showanswer”.

```

function handleResponse() {
  if (http_request.readyState == 4) {
    if (http_request.status == 200) {
      showanswer(http_request.responseText,'mathAns');
    } else
    {
      alert('There was a problem with the request.'+ http_request.statusText);
    }
  }
}

```

See bookvol11.pamphlet for further details.

26.8 The axServer/multiServ loop

The basic call to start an Axiom browser listener is:

```

)set message autoload off
)set output mathml on
axServer(8085,multiServ)$AXSERV

```

This call sets the port, opens a socket, attaches it to a stream, and then calls “multiServ” with that stream. The “multiServ” function loops serving web responses to that port.

26.9 The)browse command

In order to make the whole process cleaner the function “)browse” handles the details. This code creates the command-line function for)browse

The browse function does the internal equivalent of the following 3 command line statments:

```

)set message autoload off
)set output mathml on
axServer(8085,multiServ)$AXSERV

```

which causes Axiom to start serving web pages on port 8085

For those unfamiliar with calling algebra from lisp there are a few points to mention.

The loadLib needs to be called to load the algebra code into the image. Normally this is automatic but we are not using the interpreter so we need to do this “by hand”.

Each algebra file contains a “constructor function” which builds the domain, which is a vector, and then caches the vector so that every call to the constructor returns an EQ vector, that is, the same vector. In this case, we call the constructor |AxiomServer|

The axServer function was mangled internally to |AXSERV;axServer;IMV;2|. The multiServ function was mangled to |AXSERV;multiServ;SeV;3|. Note well that if you change axserver.spad these names might change which will generate the error message along the lines of:

```
System error:
The function $\vert$AXSERV;axServer;IMV;2$\vert$ is undefined.
```

To fix this you need to look at int/algebra/AXSERV.nrlib/code.lsp and find the new mangled function name. A better solution would be to dynamically look up the surface names in the domain vector.

Each Axiom function expects the domain vector as the last argument. This is not obvious from the call as the interpreter supplies it. We must do that “by hand”.

We don’t call the multiServ function. We pass it as a parameter to the axServer function. When it does get called by the SPADCALL macro it needs to be a lisp pair whose car is the function and whose cdr is the domain vector. We construct that pair here as the second argument to axServer. The third, hidden, argument to axServer is the domain vector which we supply “by hand”.

The socket can be supplied on the command line but defaults to 8085. Axiom supplies the arguments as a list.

```
[set p1081]
[loadLib p1221]
[AxiomServer p??]
[AXSERV;axServer;IMV;2 p??]
```

— defun browse —

```
(defun |browse| (socket)
  (let (axserv browser)
    (if socket
      (setq socket (car socket))
      (setq socket 8085))
    (|set| '(|mes| |auto| |off|))
    (|set| '(|out| |mathml| |on|))
    (|loadLib| '|AxiomServer|)
    (setq axsolv (|AxiomServer|))
    (setq browser
```

```
(|AXSERV;axServer;IMV;2| socket  
 (cons #'|AXSERV;multiServ;SeV;3| axserv) axserv)))
```

Now we have to bolt it into Axiom. This involves two lookups.

We create the lisp pair

```
(|browse| . |development|)
```

and cons it into the \$systemCommands command table. This allows the command to be executed in development mode. This lookup decides if this command is allowed. It also has the side-effect of putting the command into the \$SYSCOMMANDS variable which is used to determine if the token is a command.

26.10 The server support code

26.11)cd Command

cd man page

— cd.help —

=====

A.4.)cd

=====

User Level Required: interpreter

Command Syntax:

-)cd directory

Command Description:

This command sets the AXIOM working current directory. The current directory is used for looking for input files (for)read), AXIOM library source files (for)compile), saved history environment files (for)history)restore), compiled AXIOM library files (for)library), and files to edit (for)edit). It is also used for writing spool files (via)spool), writing history input files (via)history)write) and history environment files (via)history)save), and compiled AXIOM library files (via)compile).

If issued with no argument, this command sets the AXIOM current directory to your home directory. If an argument is used, it must be a valid directory name. Except for the ‘)’ at the beginning of the command, this has the same syntax as the operating system cd command.

Also See:

- o)compile
- o)edit
- o)history
- o)library
- o)read
- o)spool

2

² “edit” (26.19 p 872) “history” (26.23 p 889) “library” (35.1 p 1199) “read” (26.31 p 943) “spool” (26.53 p 1101)

26.12 `)clear Command`

`clear` man page

— `clear.help` —

```
=====
A.6.  )clear
=====
```

User Level Required: `interpreter`

Command Syntax:

- `)clear all`
- `)clear completely`
- `)clear properties all`
- `)clear properties obj1 [obj2 ...]`
- `)clear value all`
- `)clear value obj1 [obj2 ...]`
- `)clear mode all`
- `)clear mode obj1 [obj2 ...]`

Command Description:

This command is used to remove function and variable declarations, definitions and values from the workspace. To empty the entire workspace and reset the step counter to 1, issue

```
)clear all
```

To remove everything in the workspace but not reset the step counter, issue

```
)clear properties all
```

To remove everything about the object `x`, issue

```
)clear properties x
```

To remove everything about the objects `x`, `y` and `f`, issue

```
)clear properties x y f
```

The word `properties` may be abbreviated to the single letter `'p'`.

```
)clear p all
)clear p x
)clear p x y f
```


All definitions of functions and values of variables may be removed by either

```
)clear value all
)clear v all
```

This retains whatever declarations the objects had. To remove definitions and values for the specific objects `x`, `y` and `f`, issue

```
)clear value x y f
)clear v x y f
```

To remove the declarations of everything while leaving the definitions and values, issue

```
)clear mode all
)clear m all
```

To remove declarations for the specific objects `x`, `y` and `f`, issue

```
)clear mode x y f
)clear m x y f
```

The `)display names` and `)display properties` commands may be used to see what is currently in the workspace.

The command

```
)clear completely
```

does everything that `)clear all` does, and also clears the internal system function and constructor caches.

Also See:

- o `)display`
- o `)history`
- o `)undo`

3

defvar \$clearOptions

— initvars —

³ “display” (26.18 p 864) “history” (26.23 p 889) “undo” (26.59 p 1112)

```
(defvar |$clearOptions| '(|modes| |operations| |properties| |types| |values|))
```

defun clear

[clearSpad2Cmd p835]

— defun clear —

```
(defun |clear| (1)
  (|clearSpad2Cmd| 1))
```

defvar \$clearExcept

— initvars —

```
(defvar |$clearExcept| nil)
```

defun clearSpad2Cmd

TPDHERE: Note that this function also seems to parse out)except)completely and)scaches which don't seem to be documented. [selectOptionLC p819]

```
[sayKeyedMsg p43]
[clearCmdAll p839]
[clearCmdCompletely p838]
[clearCmdSortedCaches p837]
[clearCmdExcept p840]
[clearCmdParts p841]
[updateCurrentInterpreterFrame p31]
[$clearExcept p835]
[$options p73]
[$clearOptions p834]
```

— defun clearSpad2Cmd —

```

(defun |clearSpad2Cmd| (l)
  (let (|$clearExcept| opt optlist arg)
    (declare (special |$clearExcept| |$options| |$clearOptions|))
    (cond
      (|$options|
        (setq |$clearExcept|
          (prog (t0)
            (setq t0 t)
            (return
              (do ((t1 nil (null t0))
                  (t2 |$options| (cdr t2))
                  (t3 nil))
                ((or t1
                  (atom t2)
                  (progn (setq t3 (car t2)) nil)
                  (progn (progn (setq opt (car t3)) t3) nil))
                 t0)
              (setq t0
                (and t0
                  (eq
                    (|selectOptionLC| opt '(|except|) '|optionError|)
                    '|except|))))))))))
      (cond
        ((null l)
          (setq optlist
            (prog (t4)
              (setq t4 nil)
              (return
                (do ((t5 |$clearOptions| (cdr t5)) (x nil))
                  ((or (atom t5) (progn (setq x (car t5)) nil)) t4)
                  (setq t4 (append t4 '(|%1| "      " ,x)))))))
          (|sayKeyedMsg|
            (format nil
              "Use )clear all to clear everything in the workspace. Use )clear ~
              completely to clear everything in the workspace and internal ~
              tables. Other )clear keyword arguments are %1 %1 or abbreviations ~
              thereof. Issue )clear ? for more information.")
            (list optlist)))
        (t
          (setq arg
            (|selectOptionLC| (car l) '(|all| |completely| |scaches|) nil))
          (cond
            ((eq arg '|all|) (|clearCmdAll|))
            ((eq arg '|completely|) (|clearCmdCompletely|))
            ((eq arg '|scaches|) (|clearCmdSortedCaches|))
            (|$clearExcept| (|clearCmdExcept| l))
            (t
              (|clearCmdParts| l)
              (|updateCurrentInterpreterFrame|))))))

```

defun clearCmdSortedCaches

[compiledLookupCheck p837]
 [spadcall p??]
 [\$lookupDefaults p??]
 [\$Void p713]
 [\$ConstructorCache p??]

— **defun clearCmdSortedCaches** —

```
(defun |clearCmdSortedCaches| ()
  (let (|$lookupDefaults| domain pair)
    (declare (special |$lookupDefaults| |$Void| |$ConstructorCache|))
    (do ((t0 (hget |$ConstructorCache| '|SortedCache|) (cdr t0))
        (t1 nil))
        ((or (atom t0)
              (progn
                (setq t1 (car t0))
                (setq domain (cddr t1))
                nil))
         nil)
      (setq pair (|compiledLookupCheck| '|clearCache| (list |$Void|) domain))
      (spadcall pair))))
```

defun compiledLookupCheck

[compiledLookup p1225]
 [keyedSystemError p??]
 [formatSignature p??]

— **defun compiledLookupCheck** —

```
(defun |compiledLookupCheck| (op sig dollar)
  (let (fn)
    (setq fn (|compiledLookup| op sig dollar))
    (cond
      ((and (null fn) (eq op '^))
       (setq fn (|compiledLookup| '** sig dollar)))
      ((and (null fn) (eq op '**))
       (setq fn (|compiledLookup| '^ sig dollar)))
      (t nil))
    (cond
```

```

((null fn)
 (|keyedSystemError|
  "The function %1 with signature %2 is missing from domain %3"
  (list op (|formatSignature| sig) (elt dollar 0))))
(t fn)))

```

defvar \$functionTable

— initvars —

```
(defvar |$functionTable| nil)
```

defun clearCmdCompletely

```

[clearCmdAll p839]
[sayKeyedMsg p43]
[clearClams p??]
[clearConstructorCaches p??]
[reclaim p331]
[$localExposureData p166]
[$xdatabase p??]
[$CatOfCatDatabase p??]
[$DomOfCatDatabase p??]
[$JoinOfCatDatabase p??]
[$JoinOfDomDatabase p??]
[$attributeDb p??]
[$functionTable p838]
[$existingFiles p??]
[$localExposureDataDefault p167]

```

— defun clearCmdCompletely —

```

(defun |clearCmdCompletely| ()
  (declare (special |$localExposureData| |$xdatabase| |$CatOfCatDatabase|
    |$DomOfCatDatabase| |$JoinOfCatDatabase| |$JoinOfDomDatabase|
    |$attributeDb| |$functionTable| |$existingFiles|
    |$localExposureDataDefault|))
  (|clearCmdAll|)
  (setq |$localExposureData| (copy-seq |$localExposureDataDefault|)))

```

```

(setq |$xdatabase| nil)
(setq |$CatOfCatDatabase| nil)
(setq |$DomOfCatDatabase| nil)
(setq |$JoinOfCatDatabase| nil)
(setq |$JoinOfDomDatabase| nil)
(setq |$attributeDb| nil)
(setq |$functionTable| nil)
(|sayKeyedMsg| "All )browse facility databases have been cleared." nil)
(|clearClams|)
(|clearConstructorCaches|)
(setq |$existingFiles| (make-hash-table :test #'equal))
(|sayKeyedMsg|
  "Internally cached functions and constructors have been cleared." nil)
(reclaim)
(|sayKeyedMsg| ")clear completely is finished." nil))

```

defun clearCmdAll

```

[clearCmdSortedCaches p837]
[untraceMapSubNames p105]
[resetInCoreHist p898]
[deleteFile p1233]
[histFileName p888]
[updateCurrentInterpreterFrame p31]
[clearMacroTable p840]
[sayKeyedMsg p43]
[$frameRecord p51]
[$previousBindings p52]
[$variableNumberAlist p??]
[$InteractiveFrame p39]
[$useInternalHistoryTable p887]
[$internalHistoryTable p48]
[$frameMessages p1013]
[$interpreterFrameName p38]
[$currentLine p??]
[$traceNames p76]

```

— defun clearCmdAll —

```

(defun |clearCmdAll| ()
  (declare (special |$frameRecord| |$previousBindings| |$variableNumberAlist|
    |$InteractiveFrame| |$useInternalHistoryTable| |$internalHistoryTable|
    |$frameMessages| |$interpreterFrameName| |$currentLine| |$traceNames|))
  (|clearCmdSortedCaches|)

```

```

(setq |$frameRecord| nil)
(setq |$previousBindings| nil)
(setq |$variableNumberAlist| nil)
(|untraceMapSubNames| |$traceNames|)
(setq |$InteractiveFrame| (list (list nil)))
(|resetInCoreHist|)
(when |$useInternalHistoryTable|
  (setq |$internalHistoryTable| nil)
  (|deleteFile| (|histFileName|)))
(setq |$IOindex| 1)
(|updateCurrentInterpreterFrame|)
(setq |$currentLine| ")clear all")
(|clearMacroTable|)
(when |$frameMessages|
  (|sayKeyedMsg|
   (format nil
    "All user variables and function definitions have been cleared in ~
    the current frame ( %1 ).")
    (list |$interpreterFrameName|))
  (|sayKeyedMsg|
   "All user variables and function definitions have been cleared." nil)))

```

defun clearMacroTable

[[\\$pfMacros](#) p392]

— defun clearMacroTable 0 —

```

(defun |clearMacroTable| ()
  (declare (special |$pfMacros|))
  (setq |$pfMacros| nil))

```

defun clearCmdExcept

Clear all the options except the argument. [[stringPrefix?](#) p1403]

[[object2String](#) p??]

[[clearCmdParts](#) p841]

[[\\$clearOptions](#) p834]

— defun clearCmdExcept —

```
(defun |clearCmdExcept| (arg)
  (let ((opt (car arg)) (v1 (cdr arg)))
    (declare (special |$clearOptions|))
    (dolist (option |$clearOptions|)
      (unless (|stringPrefix?| (|object2String| opt) (|object2String| option))
        (|clearCmdParts| (cons option v1))))))
```

defun clearCmdParts

```
[selectOptionLC p819]
[pname p1235]
[types p??]
[modes p??]
[values p??]
[boot-equal p??]
[assocleft p??]
[remdup p??]
[assoc p??]
[isMap p??]
[get p??]
[exit p??]
[untraceMapSubNames p105]
[seq p??]
[recordOldValue p901]
[recordNewValue p901]
[deleteAssoc p??]
[sayKeyedMsg p43]
[getParserMacroNames p793]
[getInterpMacroNames p??]
[clearDependencies p??]
[member p1238]
[clearParserMacro p793]
[sayMessage p??]
[fixObjectForPrinting p795]
[$e p315]
[$InteractiveFrame p39]
[$clearOptions p834]
```

— defun clearCmdParts —

```
(defun |clearCmdParts| (arg)
  (let (|$e| (opt (car arg)) option pmacs imacs (v1 (cdr arg)) p1 lm prop p2)
    (declare (special |$e| |$InteractiveFrame| |$clearOptions|))
```



```

(setq option (|selectOptionLC| opt |$clearOptions| '|optionError|))
(setq option (intern (pname option)))
(setq option
  (case option
    (|types| '|mode|)
    (|modes| '|mode|)
    (|values| '|value|)
    (t option)))
(if (null vl)
  (|sayKeyedMsg|
    (format nil
      "After the property you wish to clear you must give one or more ~
      identifiers or specify all to clear that property from everything.")
    nil)
  (progn
    (setq pmacs (|getParserMacroNames|))
    (setq imacs (|getInterpMacroNames|))
    (cond
      ((boot-equal vl '|all|))
      (setq vl (assocleft (caar |$InteractiveFrame|)))
      (setq vl (remdup (append vl pmacs)))))
    (setq |$e| |$InteractiveFrame|)
    (do ((t0 vl (cdr t0)) (x nil))
      ((or (atom t0) (progn (setq x (car t0)) nil)) nil)
      (|clearDependencies| x t)
      (when (and (eq option '|properties|) (|member| x pmacs))
        (|clearParserMacro| x))
      (when (and (eq option '|properties|)
                  (|member| x imacs)
                  (null (|member| x pmacs)))
        (|sayMessage| (cons
          " You cannot clear the definition of the system-defined macro "
          (cons (|fixObjectForPrinting| x)
                (cons (intern "." "BOOT") nil))))))
    (cond
      ((setq p1 (|assoc| x (caar |$InteractiveFrame|)))
        (cond
          ((eq option '|properties|)
            (cond
              ((|isMap| x)
                (seq
                  (cond
                    ((setq lm
                      (|get| x '|localModemap| |$InteractiveFrame|))
                     (cond
                       ((consp lm)
                        (exit (|untraceMapSubNames| (cons (cadar lm) nil))))))
                  (t nil))))))
            (dolist (p2 (cdr p1))
              (setq prop (car p2))

```

```
(|recordOldValue| x prop (cdr p2))
(|recordNewValue| x prop nil))
(setf (caar |$InteractiveFrame|)
      (|deleteAssoc| x (caar |$InteractiveFrame|))))
((setq p2 (|assoc| option (cdr p1)))
(|recordOldValue| x option (cdr p2))
(|recordNewValue| x option nil)
(rplacd p2 nil))))))
nil)))))
```

26.13)close Command

close man page

— close.help —

```
=====
A.5. )close
=====
```

User Level Required: interpreter

Command Syntax:

-)close
-)close)quietly

Command Description:

This command is used to close down interpreter client processes. Such processes are started by HyperDoc to run AXIOM examples when you click on their text. When you have finished examining or modifying the example and you do not want the extra window around anymore, issue

```
)close
```

to the AXIOM prompt in the window.

If you try to close down the last remaining interpreter client process, AXIOM will offer to close down the entire AXIOM session and return you to the operating system by displaying something like

```
This is the last AXIOM session. Do you want to kill AXIOM?
```

Type "y" (followed by the Return key) if this is what you had in mind. Type "n" (followed by the Return key) to cancel the command.

You can use the)quietly option to force AXIOM to close down the interpreter client process without closing down the entire AXIOM session.

Also See:

- o)quit
- o)pquit

4

defun queryClients

Returns the number of active scratchpad clients

```
[sockSendInt p??]
[sockGetInt p??]
[$SessionManager p??]
[QueryClients p??]
```

— defun queryClients —

```
(defun |queryClients| ()
  (declare (special |$SessionManager| |$QueryClients|))
  (|sockSendInt| |$SessionManager| |$QueryClients|)
  (|sockGetInt| |$SessionManager|))
```

defun close

```
[throwKeyedMsg p??]
[sockSendInt p??]
[closeInterpreterFrame p37]
[selectOptionLC p819]
[upcase p1272]
[queryUserKeyedMsg p??]
[string2id-n p??]
[queryClients p845]
[$SpadServer p201]
[$SessionManager p??]
[$CloseClient p??]
[$currentFrameNum p335]
[$options p73]
```

— defun close —

```
(defun |close| (args)
  (declare (ignore args))
  (let (numClients opt fullopt quiet x)
    (declare (special |$SpadServer| |$SessionManager| |$CloseClient|
      |$currentFrameNum| |$options|))
    (if (null |$SpadServer|)
```

⁴ “quit” (26.30 p 940) “pquit” (26.29 p 938)

```

(|throwKeyedMsg| "You cannot close this Axiom session." nil))
(progn
  (setq numClients (|queryClients|))
  (cond
    ((> numClients 1)
      (|sockSendInt| |$SessionManager| |$CloseClient|)
      (|sockSendInt| |$SessionManager| |$currentFrameNum|)
      (|closeInterpreterFrame| nil))
    (t
      (do ((t0 |$options| (cdr t0)) (t1 nil))
          ((or (atom t0)
               (progn (setq t1 (car t0)) nil)
               (progn (progn (setq opt (car t1)) t1) nil))
           nil)
        (setq fullopt (|selectOptionLC| opt '(|quiet|) '|optionError|))
        (unless quiet (setq quiet (eq fullopt '|quiet|))))
      (cond
        (quiet
          (|sockSendInt| |$SessionManager| |$CloseClient|)
          (|sockSendInt| |$SessionManager| |$currentFrameNum|)
          (|closeInterpreterFrame| nil))
        (t
          (setq x
            (upcase
              (|queryUserKeyedMsg|
                "This is the last Axiom session. Do you want to kill Axiom?"
                nil)))
            (when (member (string2id-n x 1) '(yes y)) (bye)))))))

```

26.14)compile Command

compile man page

— compile.help —

```
=====
A.7. )compile
=====
```

User Level Required: compiler

Command Syntax:

```
- )compile
- )compile fileName
- )compile fileName.spad
- )compile directory/fileName.spad
- )compile fileName )quiet
- )compile fileName )noquiet
- )compile fileName )break
- )compile fileName )nobreak
- )compile fileName )library
- )compile fileName )nolibrary
- )compile fileName )vartrace
- )compile fileName )constructor nameOrAbbrev
```

Command Description:

You use this command to invoke the AXIOM library compiler. This compiles files with file extension .spad with the AXIOM system compiler. The command first looks in the standard system directories for files with extension .spad.

Should you not want the)library command automatically invoked, call)compile with the)nolibrary option. For example,

```
)compile mycode )nolibrary
```

By default, the)library system command exposes all domains and categories it processes. This means that the AXIOM interpreter will consider those domains and categories when it is trying to resolve a reference to a function. Sometimes domains and categories should not be exposed. For example, a domain may just be used privately by another domain and may not be meant for top-level use. The)library command should still be used, though, so that the code will be loaded on demand. In this case, you should use the)nolibrary option on)compile and the)noexpose option in the)library command. For example,

```
)compile mycode.spad )nolibrary
)library mycode )noexpose
```

Once you have established your own collection of compiled code, you may find it handy to use the `)dir` option on the `)library` command. This causes `)library` to process all compiled code in the specified directory. For example,

```
)library )dir /u/jones/as/quantum
```

You must give an explicit directory after `)dir`, even if you want all compiled code in the current working directory processed.

```
)library )dir .
```

You can compile category, domain, and package constructors contained in files with file extension `.spad`. You can compile individual constructors or every constructor in a file.

The full filename is remembered between invocations of this command and `)edit` commands. The sequence of commands

```
)compile matrix.spad
)edit
)compile
```

will call the compiler, edit, and then call the compiler again on the file `matrix.spad`. If you do not specify a directory, the working current directory (see description of command `)cd`) is searched for the file. If the file is not found, the standard system directories are searched.

If you do not give any options, all constructors within a file are compiled. Each constructor should have an `)abbreviation` command in the file in which it is defined. We suggest that you place the `)abbreviation` commands at the top of the file in the order in which the constructors are defined. The list of commands serves as a table of contents for the file.

The `)library` option causes directories containing the compiled code for each constructor to be created in the working current directory. The name of such a directory consists of the constructor abbreviation and the `.NRLIB` file extension. For example, the directory containing the compiled code for the `MATRIX` constructor is called `MATRIX.NRLIB`. The `)nolibrary` option says that such files should not be created.

The `)vartrace` option causes the compiler to generate extra code for the constructor to support conditional tracing of variable assignments. (see description of command `)trace`). Without this option, this code is suppressed and one cannot use the `)vars` option for the trace command.

The `)constructor` option is used to specify a particular constructor to

`compile`. All other constructors in the file are ignored. The constructor name or abbreviation follows `)constructor`. Thus either

```
)compile matrix.spad )constructor RectangularMatrix
```

or

```
)compile matrix.spad )constructor RMATRIX
```

compiles the `RectangularMatrix` constructor defined in `matrix.spad`.

The `)break` and `)nobreak` options determine what the compiler does when it encounters an error. `)break` is the default and it indicates that processing should stop at the first error. The value of the `)set break` variable then controls what happens.

Also See:

- o `)abbreviation`
- o `)edit`
- o `)library`

5

defvar /editfile

— initvars —

```
(defvar /editfile nil)
```

⁵ “abbreviation” (?? p ??) “edit” (26.19 p 872) “library” (35.1 p 1199)

26.15)copyright Command

copyright man page

— copyright.help —

The term Axiom, in the field of computer algebra software, along with AXIOM and associated images are common-law trademarks. While the software license allows copies, the trademarks may only be used when referring to this project.

Axiom is distributed under terms of the Modified BSD license. Axiom was released under this license as of September 3, 2002. Source code is freely available at:
<http://savannah.nongnu.org/projects/axiom>
 Copyrights remain with the original copyright holders. Use of this material is by permission and/or license. Individual files contain reference to these applicable copyrights. The copyright and license statements are collected here for reference.

Portions Copyright (c) 2003- The Axiom Team

The Axiom Team is the collective name for the people who have contributed to this project. Where no other copyright statement is noted in a file this copyright will apply.

Portions Copyright (c) 1991-2002, The Numerical ALgorithms Group Ltd. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of The Numerical ALgorithms Group Ltd. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A

PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Portions Copyright (C) 1989-95 GROUPE BULL

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL GROUPE BULL BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Except as contained in this notice, the name of GROUPE BULL shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Software without prior written authorization from GROUPE BULL.

Portions Copyright (C) 2002, Codemist Ltd. All rights reserved.
acn@codemist.co.uk

CCL Public License 1.0
=====

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

(1) Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

(2) Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

(3) Neither the name of Codemist nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

- (4) If you distribute a modified form of either source or binary code
- (a) you must make the source form of these modification available to Codemist;
 - (b) you grant Codemist a royalty-free license to use, modify or redistribute your modifications without limitation;
 - (c) you represent that you are legally entitled to grant these rights and that you are not providing Codemist with any code that violates any law or breaches any contract.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Portions Copyright (C) 1995-1997 Eric Young (eay@mincom.oz.au)
All rights reserved.

This package is an SSL implementation written
by Eric Young (eay@mincom.oz.au).
The implementation was written so as to conform with Netscapes SSL.

This library is free for commercial and non-commercial use as long as the following conditions are aheared to. The following conditions apply to all code found in this distribution, be it the RC4, RSA, lhash, DES, etc., code; not just the SSL code. The SSL documentation included with this distribution is covered by the same copyright terms except that the holder is Tim Hudson (tjh@mincom.oz.au).

Copyright remains Eric Young's, and as such any Copyright notices in the code are not to be removed.
If this package is used in a product, Eric Young should be given attribution as the author of the parts of the library used.
This can be in the form of a textual message at program startup or in documentation (online or textual) provided with the package.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. All advertising materials mentioning features or use of this software must display the following acknowledgement:
 "This product includes cryptographic software written by
 Eric Young (eay@mincom.oz.au)"
 The word 'cryptographic' can be left out if the routines from the library being used are not cryptographic related :-).
4. If you include any Windows specific code (or a derivative thereof) from the apps directory (application code) you must include an acknowledgement:
 "This product includes software written by Tim Hudson (tjh@mincom.oz.au)"

THIS SOFTWARE IS PROVIDED BY ERIC YOUNG 'AS IS' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The licence and distribution terms for any publically available version or derivative of this code cannot be changed. i.e. this code cannot simply be copied and put under another distribution licence [including the GNU Public Licence.]

Portions Copyright (C) 1988 by Leslie Lamport.

Portions Copyright (c) 1998 Free Software Foundation, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, distribute with modifications, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

IN NO EVENT SHALL THE ABOVE COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Except as contained in this notice, the name(s) of the above copyright holders shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Software without prior written authorization.

Portions Copyright 1989-2000 by Norman Ramsey. All rights reserved.

Noweb is protected by copyright. It is not public-domain software or shareware, and it is not protected by a ‘‘copyleft’’ agreement like the one used by the Free Software Foundation.

Noweb is available free for any use in any field of endeavor. You may redistribute noweb in whole or in part provided you acknowledge its source and include this COPYRIGHT file. You may modify noweb and create derived works, provided you retain this copyright notice, but the result may not be called noweb without my written consent.

You may sell noweb if you wish. For example, you may sell a CD-ROM including noweb.

You may sell a derived work, provided that all source code for your derived work is available, at no additional charge, to anyone who buys your derived work in any form. You must give permission for said source code to be used and modified under the terms of this license. You must state clearly that your work uses or is based on noweb and that noweb is available free of charge. You must also request that bug reports on your work be reported to you.

Portions Copyright (c) 1987 The RAND Corporation. All rights reserved.

Portions Copyright 1988-1995 by Stichting Mathematisch Centrum, Amsterdam, The Netherlands.

All Rights Reserved

Permission to use, copy, modify, and distribute this software and its documentation for any purpose and without fee is hereby granted, provided that the above copyright notice appear in all copies and that both that copyright notice and this permission notice appear in supporting documentation, and that the names of Stichting Mathematisch Centrum or CWI not be used in advertising or publicity pertaining to distribution of the software without specific, written prior permission.

STICHTING MATHEMATISCH CENTRUM DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND

FITNESS, IN NO EVENT SHALL STICHTING MATHEMATISCH CENTRUM BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Portions Copyright (c) Renaud Rioboo and the University Paris 6.

Portions Copyright (c) 2003-2010 Jocelyn Guidry

Portions Copyright (c) 2001-2010 Timothy Daly

defun copyright

[obey p??]

[concat p1237]

[getenviro n p322]

— defun copyright —

```
(defun |copyright| ()
  (obey (concat "cat " (getenviro n "AXIOM") "/doc/spadhelp/copyright.help")))
```

defun trademark

— defun trademark 0 —

```
(defun |trademark| ()
  (format t "The term Axiom, in the field of computer algebra software, ~%")
  (format t "along with AXIOM and associated images are common-law ~%")
  (format t "trademarks. While the software license allows copies, the ~%")
  (format t "trademarks may only be used when referring to this project ~%"))
```

This command is in the list of `$noParseCommands` 26.1 which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` 26.2

26.16 `)credits` Command

`credits` man page

`defun credits`

[credits p856]

— `defun credits 0` —

```
(defun |credits| ()  
  (declare (special credits))  
  (mapcar #'(lambda (x) (princ x) (terpri)) creditlist))
```

—————

This command is in the list of `$noParseCommands` 26.1 which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` 26.2

26.17)describe Command

describe man page

— describe.help —

```
=====
)describe
=====
```

User Level Required: interpreter

Command Syntax:

-)describe categoryName
-)describe domainName
-)describe packageName

Command Description:

This command is used to display the comments for the operation, category, domain or package. The comments are part of the algebra source code.

The commands

```
)describe <categoryName> [internal]
)describe <domainName> [internal]
)describe <packageName> [internal]
```

will show a properly formatted version of the "Description:" keyword from the comments in the algebra source for the category, domain, or package requested.

If 'internal' is requested, then the internal format of the domain or package is described. Categories do not have an internal representation.

defvar \$describeOptions

The current value of \$describeOptions is

— initvars —

```
(defvar $describeOptions '(|category| |domain| |package|))
```

defun Print comment strings from algebra libraries

This trivial function satisfies the standard pattern of making a user command match the name of the function which implements the command. That command immediatly invokes a “Spad2Cmd” version. [describepad2cmd p??]

— defun describe —

```
(defun |describe| (l)
  (describeSpad2Cmd l))
```

—————

defun describeSpad2Cmd

The describe command prints cleaned-up comment strings from the algebra libraries. It can print strings associated with a category, domain, package, or by operation.

This implements command line options of the form:

```
)describe categoryName [internal]
)describe domainName   [internal]
)describe packageName   [internal]
```

The describeInternal function will either call the “dc” function to describe the internal representation of the argument or it will print a cleaned up version of the text for the “Description” keyword in the Category, Domain, or Package source code.

```
[selectOptionLC p819]
[flatten p861]
[cleanline p859]
[getdatabase p1196]
[sayMessage p??]
[$e p315]
[$EmptyEnvironment p??]
[$describeOptions p857]
```

— defun describeSpad2Cmd —

```
(defun describeSpad2Cmd (l)
  (labels (
    (fullname (arg)
      "Convert abbreviations to the full constructor name"
      (let ((abb (getdatabase arg 'abbreviation)))
        (if abb arg (getdatabase arg 'constructor))))
    (describeInternal (cdp internal?)
      (if internal?
```

```

(progn
  (unless (eq (getdatabase cdp 'constructorkind) '|category|) (|dc| cdp))
  (showdatabase cdp))
  (mapcar #'(lambda (x) (if (stringp x) (cleanline x)))
    (flatten (car (getdatabase (fullname cdp) 'documentation))))))
(let ((|$e| |$EmptyEnvironment|) (opt (second l)))
  (declare (special |$e| |$EmptyEnvironment| $describeOptions))
  (if (and (consp l) (not (eq opt '?)))
    (describeInternal (first l) (second l))
    (|sayMessage|
      (format nil "~&~a ~{~%    ~a~^ ~}~%    ~a"
        " )describe keyword arguments are"
        $describeOptions
        "or abbreviations thereof")))))

```

defun cleanline

— defun cleanline —

```

(defun cleanline (line)
  (labels (
    (replaceInLine (thing other line)
      (do ((mark (search thing line) (search thing line)))
        ((null mark) line)
      (setq line
        (concatenate 'string (subseq line 0 mark) other
          (subseq line (+ mark (length thing)))))))

    (removeFromLine (thing line) (replaceInLine thing "" line))

    (removeKeyword (str line)
      (do ((mark (search str line) (search str line)))
        ((null mark) line)
      (let (left point mid right)
        (setq left (subseq line 0 mark))
        (setq point (search "}" line :start2 mark))
        (setq mid (subseq line (+ mark (length str)) point))
        (setq right (subseq line (+ point 1)))
        (setq line (concatenate 'string left mid right))))))

    (addSpaces (str line)
      (do ((mark (search str line) (search str line)) (cnt))
        ((null mark) line)
      (let (left point mid right)
        (setq left (subseq line 0 mark))

```

```

(setq point (search "]" line :start2 mark))
(setq mid (subseq line (+ mark (length str)) point))
(if (setq cnt (parse-integer mid :junk-allowed t))
    (setq mid (make-string cnt :initial-element #\ ))
    (setq mid ""))
(setq right (subseq line (+ point 1)))
(setq line (concatenate 'string left mid right))))

(splitAtNewline (line)
  (do ((mark (search "~%" line) (search "~%" line)) (lines))
      ((null mark)
       (push " " lines)
       (push line lines)
       (nreverse lines))
      (push (subseq line 0 mark) lines)
      (setq line (subseq line (+ mark 2))))))

(wrapOneLine (line margin result)
  (if (null line)
      (nreverse result)
      (if (< (length line) margin)
          (wrapOneLine nil margin (append (list line) result))
          (let (oneline spill aspace)
              (setq aspace (position #\space (subseq line 0 margin) :from-end t))
              (setq oneline (string-trim '(\space) (subseq line 0 aspace)))
              (setq spill (string-trim '(\space) (subseq line aspace)))
              (wrapOneLine spill margin (append (list oneline) result))))))

(reflowParagraph (line)
  (let (lst1)
    (setq lst1 (splitAtNewLine line))
    (dolist (x lst1)
      (mapcar #'(lambda(y) (format t "~a%" y))
              (wrapOneLine x 70 nil))))))

(setq line (removeFromLine "{" line))
(setq line (replaceInLine "\\blankline" "~%" line))
(setq line (replaceInLine "\\br" "~%" line))
(setq line (removeFromLine "\\" line))
(dolist (str '("spad{" "spadtype{" "spadop{" "spadfun{" "spadatt{"
              "axiom{" "axiomType{" "spadignore{" "axiomFun{"
              "centerline{" "inputbitmap{" "axiomOp{" "spadgloss{"))
  (setq line (removeKeyword str line)))
(setq line (replaceInLine "{e.g.}" "e.g." line))
(dolist (str '("tab{" "indented{" ))
  (setq line (addSpaces str line)))
(reflowParagraph line))

```

defun flatten

— defun flatten 0 —

```
(defun flatten (x)
  (labels (
    (rec (x acc)
      (cond
        ((null x) acc)
        ((atom x) (cons x acc))
        (t (rec (car x) (rec (cdr x) acc))))))
    (rec x nil)))
```

—————

26.18)display Command

display man page

— display.help —

```
=====
A.8. )display
=====
```

User Level Required: interpreter

Command Syntax:

-)display all
-)display properties
-)display properties all
-)display properties [obj1 [obj2 ...]]
-)display value all
-)display value [obj1 [obj2 ...]]
-)display mode all
-)display mode [obj1 [obj2 ...]]
-)display names
-)display operations opName

Command Description:

This command is used to display the contents of the workspace and signatures of functions with a given name. (A signature gives the argument and return types of a function.)

The command

```
)display names
```

lists the names of all user-defined objects in the workspace. This is useful if you do not wish to see everything about the objects and need only be reminded of their names.

The commands

```
)display all
)display properties
)display properties all
```

all do the same thing: show the values and types and declared modes of all variables in the workspace. If you have defined functions, their signatures and definitions will also be displayed.

To show all information about a particular variable or user functions, for example, something named `d`, issue

```
)display properties d
```

To just show the value (and the type) of `d`, issue

```
)display value d
```

To just show the declared mode of `d`, issue

```
)display mode d
```

All modemap for a given operation may be displayed by using `)display` operations. A modemap is a collection of information about a particular reference to an operation. This includes the types of the arguments and the return value, the location of the implementation and any conditions on the types. The modemap may contain patterns. The following displays the modemaps for the operation `FromcomplexComplexCategory`:

```
)d op complex
```

Also See:

- o `)clear`
- o `)history`
- o `)set`
- o `)show`
- o `)what`

6

defvar `$displayOptions`

The current value of `$displayOptions` is

— **initvars** —

```
(defvar |$displayOptions|
  '(|abbreviations| |all| |macros| |modes| |names| |operations|
    |properties| |types| |values|))
```

⁶ “clear” ([26.12 p 835](#)) “history” ([26.23 p 889](#)) “set” ([26.51 p 1081](#)) “show” ([26.52 p 1087](#)) “what” ([26.61 p 1131](#))

defun display

This trivial function satisfies the standard pattern of making a user command match the name of the function which implements the command. That command immediatly invokes a “Spad2Cmd” version. [displayspad2cmd p??]

— defun display —

```
(defun |display| (l)
  (displaySpad2Cmd l))
```

—————

displaySpad2Cmd

We process the options to the command and call the appropriate display function. There are really only 4 display functions. All of the other options are just subcases.

There is a slight mismatch between the \$displayOptions list of symbols and the options this command accepts so we have a cond branch to clean up the option variable. This allows for the options to be plural.

If we fall all the way thru we use the \$displayOptions list to construct a list of strings for the sayMessage function and tell the user what options are available. [abbQuery p865]

```
[opOf p??]
[listConstructorAbbreviations p824]
[displayOperations p866]
[displayMacros p867]
[displayWorkspaceNames p794]
[displayProperties p801]
[selectOptionLC p819]
[sayMessage p??]
[$e p315]
[$EmptyEnvironment p??]
[$displayOptions p863]
```

— defun displaySpad2Cmd —

```
(defun displaySpad2Cmd (l)
  (let ((|$e| |$EmptyEnvironment|) (opt (car l)) (vl (cdr l)) option)
    (declare (special |$e| |$EmptyEnvironment| |$displayOptions|))
    (if (and (consp l) (not (eq opt '???)))
        (progn
          (setq option (|selectOptionLC| opt |$displayOptions| '|optionError|))
          (cond
            ((eq option '|all|)
             (setq l (list '|properties|)))
```

```

      (setq option '|properties|)
    ((or (eq option '|modes|) (eq option '|types|))
      (setq l (cons '|type| vl))
      (setq option '|type|))
    ((eq option '|values|)
      (setq l (cons '|value| vl))
      (setq option '|value|))
    (cond
      ((eq option '|abbreviations|)
        (if (null vl)
          (|listConstructorAbbreviations|)
          (dolist (v vl) (|abbQuery| (|opOf| v))))))
      ((eq option '|operations|) (|displayOperations| vl))
      ((eq option '|macros|) (|displayMacros| vl))
      ((eq option '|names|) (|displayWorkspaceNames|))
      (t (|displayProperties| option l))))
    (|sayMessage|
      (append
        '(" )display keyword arguments are")
        (mapcar #'(lambda (x) (format nil "~%      ~a" x)) (|displayOptions|)
          (format nil "~% or abbreviations thereof"))))))

```

defun abbQuery

[getdatabase p1196]

[sayKeyedMsg p43]

— defun abbQuery —

```

(defun |abbQuery| (x)
  (let (abb)
    (cond
      ((setq abb (getdatabase x 'abbreviation))
        (|sayKeyedMsg| "%1 abbreviates %2 %3"
          (list abb (getdatabase x 'constructorkind) x)))
      ((setq abb (getdatabase x 'constructor))
        (|sayKeyedMsg| "%1 abbreviates %2 %3"
          (list x (getdatabase abb 'constructorkind) abb)))
      (t
        (|sayKeyedMsg|
          "%1 is neither a constructor name nor a constructor abbreviation."
          (list x))))))

```

defun displayOperations

This function takes a list of operation names. If the list is null we query the user to see if they want all operations printed. Otherwise we print the information for the requested symbols.

```
[reportOpSymbol p??]
[yesanswer p866]
[sayKeyedMsg p43]
```

— defun displayOperations —

```
(defun |displayOperations| (l)
  (if l
    (dolist (op l) (|reportOpSymbol| op))
    (if (yesanswer)
      (dolist (op (|allOperations|)) (|reportOpSymbol| op))
      (|sayKeyedMsg|
        (format nil
          "Since you did not respond with y or yes the list of operations will ~
            not be displayed.")
        nil))))
```

—————

defun yesanswer

This is a trivial function to simplify the logic of displaySpad2Cmd. If the user didn't supply an argument to the)display op command we ask if they wish to have all information about all Axiom operations displayed. If the answer is either Y or YES we return true else nil.

```
[string2id-n p??]
[upcase p1272]
[queryUserKeyedMsg p??]
```

— defun yesanswer —

```
(defun yesanswer ()
  (member
    (string2id-n
      (upcase
        (|queryUserKeyedMsg|
          (format nil
            "You have requested that all information about all Axiom operations ~
              (functions) be displayed. As there are several hundred operations, ~
              please confirm your request by typing y or yes and then pressing ~
              Enter :")
          nil)) 1) '(y yes)))
```

defun displayMacros

```
[getInterpMacroNames p??]
[getParserMacroNames p793]
[remdup p??]
[sayBrightly p??]
[member p1238]
[displayParserMacro p804]
[seq p??]
[exit p??]
[displayMacro p794]
```

— defun displayMacros —

```
(defun |displayMacros| (names)
  (let (imacs pmacs macros first)
    (setq imacs (|getInterpMacroNames|))
    (setq pmacs (|getParserMacroNames|))
    (if names
      (setq macros names)
      (setq macros (append imacs pmacs)))
    (setq macros (remdup macros))
    (cond
      ((null macros) (|sayBrightly| "  There are no Axiom macros."))
      (t
       (setq first t)
       (do ((t0 macros (cdr t0)) (macro nil))
         ((or (atom t0) (progn (setq macro (car t0)) nil)) nil)
         (seq
          (exit
           (cond
            ((|member| macro pmacs)
             (cond
              (first (|sayBrightly|
                      (cons '|%1| (cons "User-defined macros:" nil))) (setq first nil)))
              (|displayParserMacro| macro))
            ((|member| macro imacs) '|iterate|)
            (t (|sayBrightly|
                (cons " "
                     (cons macro
                          (cons " is not a known Axiom macro." nil))))))))))
       (setq first t)
       (do ((t1 macros (cdr t1)) (macro nil))
         ((or (atom t1) (progn (setq macro (car t1)) nil)) nil)
```

```

(seq
  (exit
    (cond
      ((|member| macro imacs)
        (cond
          ((|member| macro pmacs) '|iterate|)
          (t
            (cond
              (first
                (|sayBrightly|
                  (cons '|%1|
                    (cons "System-defined macros:" nil)))) (setq first nil)))
              (|displayMacro| macro))))
          ((|member| macro pmacs) '|iterate|))))
    nil))))

```

defun sayExample

This function expects 2 arguments, the documentation string and the name of the operation. It searches the documentation string for `++X` lines. These lines are examples lines for functions. They look like ordinary `++` comments and fit into the ordinary comment blocks. So, for example, in the `plot.spad.pamphlet` file we find the following function signature:

```

plot: (F -> F,R) -> %
++ plot(f,a..b) plots the function \spad{f(x)}
++ on the interval \spad{[a,b]}.
++
++X fp:=(t:DFLOAT):DFLOAT +-> sin(t)
++X plot(fp,-1.0..1.0)$PLOT

```

This function splits out and prints the lines that begin with `++X`.

A minor complication of printing the examples is that the lines have been processed into internal compiler format. Thus the lines that read:

```

++X fp:=(t:DFLOAT):DFLOAT +-> sin(t)
++X plot(fp,-1.0..1.0)$PLOT

```

are actually stored as one long line containing the example lines

```

"\indented{1}{plot(\spad{f},{a..\spad{b}) plots the function
\spad{f(x)} } \indented{1}{on the interval \spad{[a,{b}]}.}
\blankline
\spad{X} fp:=(t:DFLOAT):DFLOAT +-> sin(\spad{t})
\spad{X} plot(\spad{fp},{ }\spad{-1}.0..1.0)\$PLOT"

```

So when we have an example line starting with `++X`, it gets converted to the compiler to `\spad{X}`. So each example line is delimited by `\spad{X}`.

The compiler also removes the newlines so if there is a subsequent `\spad{X}` in the docstring then it implies multiple example lines and we loop over them, splitting them up at the delimiter.

If there is only one then we clean it up and print it.

```
[cleanupLine p??]
[sayNewLine p??]
```

— **defun sayExample** —

```
(defun sayExample (docstring)
  (let (line point)
    (when (setq point (search "spad{X}" docstring))
      (setq line (subseq docstring (+ point 8)))
      (do ((mark (search "spad{X}" line) (search "spad{X}" line)))
          ((null mark))
        (princ (cleanupLine (subseq line 0 mark)))
        (terpri)
        (setq line (subseq line (+ mark 8))))
      (princ (cleanupLine line))
      (terpri)
      (terpri))))
```

defun cleanupLine

This function expects example lines in internal format that has been partially processed to remove the prefix. Thus we get lines that look like:

```
fp:=(t:DFLOAT):DFLOAT +-> sin(\spad{t})
plot(\spad{fp},{}\spad{-1}.0..1.0)\$PLOT
```

It removes all instances of `{}`, and `\`, and unwraps the `spad{}` call, leaving only the argument.

We return lines that look like:

```
fp:=(t:DFLOAT):DFLOAT +-> sin(t)
plot(fp,-1.0..1.0)$PLOT
```

which is hopefully exactly what the user wrote.

The compiler inserts `{}` as a space so we remove it. We remove all of the `\` characters. We remove all of the `spad{...}` delimiters which will occur around other `spad` variables. Technically we should search recursively for the matching delimiter rather than the next

brace but the problem does not arise in practice.

— **defun cleanupLine 0** —

```
(defun cleanupLine (line)
  (do ((mark (search "{}" line) (search "{}" line)))
      ((null mark))
      (setq line
        (concatenate 'string (subseq line 0 mark) (subseq line (+ mark 2))))))
  (do ((mark (search "\\" line) (search "\\" line)))
      ((null mark))
      (setq line
        (concatenate 'string (subseq line 0 mark) (subseq line (+ mark 1))))))
  ; split out \spad{...}
  (do ((mark (search "spad{" line) (search "spad{" line)))
      ((null mark))
      (let (left point mid right)
        (setq left (subseq line 0 mark))
        (setq point (search "]" line :start2 mark))
        (setq mid (subseq line (+ mark 5) point))
        (setq right (subseq line (+ point 1)))
        (setq line (concatenate 'string left mid right))))))
  ; split out \spadtype{...}
  (do ((mark (search "spadtype{" line) (search "spadtype{" line)))
      ((null mark))
      (let (left point mid right)
        (setq left (subseq line 0 mark))
        (setq point (search "]" line :start2 mark))
        (setq mid (subseq line (+ mark 9) point))
        (setq right (subseq line (+ point 1)))
        (setq line (concatenate 'string left mid right))))))
  line)
```

26.19 `)edit` Command

edit man page

— edit.help —

```
=====
A.9.  )edit
=====
```

User Level Required: interpreter

Command Syntax:

```
- )edit [filename]
```

Command Description:

This command is used to edit files. It works in conjunction with the `)read` and `)compile` commands to remember the name of the file on which you are working. By specifying the name fully, you can edit any file you wish. Thus

```
)edit /u/julius/matrix.input
```

will place you in an editor looking at the file `/u/julius/matrix.input`. By default, the editor is `vi`, but if you have an `EDITOR` shell environment variable defined, that editor will be used. When `AXIOM` is running under the X Window System, it will try to open a separate `xterm` running your editor if it thinks one is necessary. For example, under the Korn shell, if you issue

```
export EDITOR=emacs
```

then the `emacs` editor will be used by `)edit`.

If you do not specify a file name, the last file you edited, read or compiled will be used. If there is no ‘‘last file’’ you will be placed in the editor editing an empty unnamed file.

It is possible to use the `)system` command to edit a file directly. For example,

```
)system emacs /etc/rc.tcpip
```

calls `emacs` to edit the file.

Also See:

- o `)system`
- o `)compile`

```
o )read
```

7

defun edit

[editSpad2Cmd p872]

— defun edit —

```
(defun |edit| (l) (|editSpad2Cmd| l))
```

defun editSpad2Cmd

```
[pathname p1232]
[pathnameDirectory p1232]
[pathnameType p1231]
[$FINDFILE p??]
[pathnameName p1231]
[editFile p873]
[updateSourceFiles p874]
[/editfile p849]
```

— defun editSpad2Cmd —

```
(defun |editSpad2Cmd| (l)
  (let (olddir filetypes ll rc)
    (declare (special /editfile))
    (setq l (cond ((null l) /editfile) (t (car l))))
    (setq l (|pathname| l))
    (setq olddir (|pathnameDirectory| l))
    (setq filetypes
      (cond
        ((|pathnameType| l) (list (|pathnameType| l)))
        ((eq |$UserLevel| '|interpreter|) '("input" "INPUT" "spad" "SPAD"))
        ((eq |$UserLevel| '|compiler|) '("input" "INPUT" "spad" "SPAD"))
        (t '("input" "INPUT" "spad" "SPAD" "boot" "BOOT"
              "lisp" "LISP" "meta" "META"))))
    (setq ll
```

⁷ “system” (26.56 p 1108) “read” (26.31 p 943)

```
(cond
  ((string= olddir "")
    (|pathname| ($findfile (|pathnameName| 1) filetypes)))
  (t 1)))
(setq 1 (|pathname| 11))
(setq /editfile 1)
(setq rc (|editFile| 1))
(|updateSourceFiles| 1)
rc))
```

defun Implement the)edit command

```
[concat p1237]
[namestring p1230]
[pathname p1232]
[obey p??]
```

— defun editFile —

```
(defun |editFile| (file)
  (cond
    ((member (intern "WIN32" (find-package 'keyword)) *features*)
      (obey (concat "notepad " (|namestring| (|pathname| file)))))
    (t
      (obey
        (concat "$AXIOM/lib/SPAEDIT " (|namestring| (|pathname| file)))))))
```

The SPAEDIT command

Axiom execute a shell script called SPAEDIT to open a file using the user's chosen editor. That editor name is, by convention, in the EDITOR shell variable. If that variable is not set we default to the 'vi' editor.

— spadedit —

```
#!/bin/sh
# this script is invoked by the spad )edit command
# can be replaced by users favorite editor
# optional second argument should be character offset in file

thefile=$1
if [ ! -f $1 ] ; then
```



```

    thefile=$AXIOM/../../src/algebra/$1
else
    thefile=$1
fi

if [ $# = 2 ] ; then
START='grep -n \^$2\($ thefile | awk -F: '{print $1}''
else
START=1
fi

if [ ! "$EDITOR" ] ; then
EDITOR=vi
fi

if [ "$DISPLAY" ] ; then
if [ "$EDITOR" = "emacs" ] ; then
emacs +$START $thefile &
elif [ "$EDITOR" = "vi" ] ; then
xterm -e vi +$START $thefile &
else
xterm -e $EDITOR $thefile &
fi
else
$EDITOR $thefile
fi

```

defun updateSourceFiles

```

[pathname p1232]
[pathnameName p1231]
[pathnameType p1231]
[makeInputFilename p1169]
[member p1238]
[pathnameTypeId p1231]
[insert p??]
[$sourceFiles p??]

```

— defun updateSourceFiles —

```

(defun |updateSourceFiles| (arg)
  (declare (special |$sourceFiles|))
  (setq arg (|pathname| arg))
  (setq arg (|pathname| (list (|pathnameName| arg) (|pathnameType| arg) "*")))
  (when (and (makeInputFilename arg)

```

```
      (|member| (|pathnameTypeId| arg) '(boot lisp meta)))  
    (setq |$sourceFiles| (|insert| arg |$sourceFiles|)))  
  arg)
```

26.20)fin Command

fin man page

— fin.help —

```
=====
A.10. )fin
=====
```

User Level Required: development

Command Syntax:

-)fin

Command Description:

This command is used by AXIOM developers to leave the AXIOM system and return to the underlying Lisp system. To return to AXIOM, issue the ‘‘(spad)’’ function call to Lisp.

Also See:

- o)pquit
- o)quit

8

defun Exit from the interpreter to lisp

```
[spad-reader p??]
[EOF p??]
```

— defun fin 0 —

```
(defun |fin| ()
  (setq *eof* t)
  (throw 'spad_reader nil))
```

⁸ “pquit” (26.29 p 938) “quit” (26.30 p 940)

This command is in the list of `$noParseCommands` [26.1](#) which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` [26.2](#)

26.21)help Command

help man page

— help.help —

```
=====
A.12. )help
=====
```

User Level Required: interpreter

Command Syntax:

-)help
-)help commandName
-)help syntax

Command Description:

This command displays help information about system commands. If you issue

```
)help
```

then this very text will be shown. You can also give the name or abbreviation of a system command to display information about it. For example,

```
)help clear
```

will display the description of the)clear system command.

The command

```
)help syntax
```

will give further information about the Axiom language syntax.

All this material is available in the AXIOM User Guide and in HyperDoc. In HyperDoc, choose the Commands item from the Reference menu.

```
=====
A.1. Introduction
=====
```

System commands are used to perform AXIOM environment management. Among the commands are those that display what has been defined or computed, set up multiple logical AXIOM environments (frames), clear definitions, read files

of expressions and commands, show what functions are available, and terminate AXIOM.

Some commands are restricted: the commands

```
)set userlevel interpreter
)set userlevel compiler
)set userlevel development
```

set the user-access level to the three possible choices. All commands are available at development level and the fewest are available at interpreter level. The default user-level is interpreter. In addition to the)set command (discussed in description of command)set) you can use the HyperDoc settings facility to change the user-level. Click on [Settings] here to immediately go to the settings facility.

Each command listing begins with one or more syntax pattern descriptions plus examples of related commands. The syntax descriptions are intended to be easy to read and do not necessarily represent the most compact way of specifying all possible arguments and options; the descriptions may occasionally be redundant.

All system commands begin with a right parenthesis which should be in the first available column of the input line (that is, immediately after the input prompt, if any). System commands may be issued directly to AXIOM or be included in .input files.

A system command argument is a word that directly follows the command name and is not followed or preceded by a right parenthesis. A system command option follows the system command and is directly preceded by a right parenthesis. Options may have arguments: they directly follow the option. This example may make it easier to remember what is an option and what is an argument:

```
)syscmd arg1 arg2 )opt1 opt1arg1 opt1arg2 )opt2 opt2arg1 ...
```

In the system command descriptions, optional arguments and options are enclosed in brackets ('[' and ']'). If an argument or option name is in italics, it is meant to be a variable and must have some actual value substituted for it when the system command call is made. For example, the syntax pattern description

```
)read fileName [quietly]
```

would imply that you must provide an actual file name for fileName but need not use the)quietly option. Thus

```
)read matrix.input
```

is a valid instance of the above pattern.

System command names and options may be abbreviated and may be in upper or lower case. The case of actual arguments may be significant, depending on the particular situation (such as in file names). System command names and options may be abbreviated to the minimum number of starting letters so that the name or option is unique. Thus

)s Integer

is not a valid abbreviation for the)set command, because both)set and)show begin with the letter ‘s’. Typically, two or three letters are sufficient for disambiguating names. In our descriptions of the commands, we have used no abbreviations for either command names or options.

In some syntax descriptions we use a vertical line ‘|’ to indicate that you must specify one of the listed choices. For example, in

)set output fortran on | off

only on and off are acceptable words for following boot. We also sometimes use ‘...’ to indicate that additional arguments or options of the listed form are allowed. Finally, in the syntax descriptions we may also list the syntax of related commands.

=====

Other help topics

=====

Available help topics are:

abbreviations	assignment	blocks	browse	boot	cd
clear	clef	close	collection	compile	describe
display	edit	fin	for	frame	help
history	if	iterate	leave	library	lisp
load	ltrace	parallel	pquit	quit	read
repeat	savesystem	set	show	spool	suchthat
synonym	system	syntax	trace	undo	what
while					

Available algebra help topics are:

The top level help command

[helpSpad2Cmd p881]

— defun help —

```
(defun |help| (l)
  "The top level help command"
  (|helpSpad2Cmd| l))
```

The top level help command handler

```
[newHelpSpad2Cmd p881]
[sayKeyedMsg p43]
```

— defun helpSpad2Cmd —

```
(defun |helpSpad2Cmd| (args)
  "The top level help command handler"
  (unless (|newHelpSpad2Cmd| args)
    (|sayKeyedMsg|
     (format nil
              "If the system command or synonym %1 exists, help information is not ~
              available for it. Issue )what commands or )what synonyms to ~
              determine is %1 is a valid name.")
     (cons args nil))))
```

defun newHelpSpad2Cmd

```
[makeInputFilename p1169]
[obey p??]
[concat p1237]
[namestring p1230]
[make-instream p1167]
[say p??]
[abbreviation? p??]
[poundsign p??]
[sayKeyedMsg p43]
[pname p1235]
[selectOptionLC p819]
[$syscommands p784]
[$useFullScreenHelp p1004]
```

— defun newHelpSpad2Cmd —

```
(defun |newHelpSpad2Cmd| (args)
```



```

(let (sarg arg nargs helpfile filestream line unabbrev)
(declare (special $syscommands |$useFullScreenHelp|))
(when (null args) (setq args (list '?)))
(if (> (|#| args) 1)
(|sayKeyedMsg| "The )help system command supports at most one argument."
nil)
(progn
(setq sarg (pname (car args)))
(cond
((string= sarg "?") (setq args (list '|help|)))
((string= sarg "%") (setq args (list '|history|)))
((string= sarg "%") (setq args (list '|history|)))
(t nil))
(setq arg (|selectOptionLC| (car args) $syscommands nil))
(cond ((null arg) (setq arg (car args)))
(setq nargs (pname arg))
; expand abbreviations to full constructor names
(when
(setq unabbrev (|abbreviation?| (intern nargs)))
(setq nargs (symbol-name unabbrev)))
(cond
; if the help file does not exist, exit
((null (setq helpfile (makeInputFilename (list nargs "help"))))
nil)
; if we expect to use full screen help, call SPADEDIT
(|$useFullScreenHelp|
(obey (concat "$AXIOM/lib/SPADEDIT " (|namestring| helpfile))) t)
; otherwise dump the help file to the console
(t
(setq filestream (make-instream helpfile))
(do ((line (|read-line| filestream nil) (|read-line| filestream nil)))
((null line) (shut filestream))
(say line)))))))))

```

26.22)history Command

history man page

— history.help —

```
=====
A.13. )history
=====
```

User Level Required: interpreter

Command Syntax:

```
- )history )on
- )history )off
- )history )write historyInputFileName
- )history )show [n] [both]
- )history )save savedHistoryName
- )history )restore [savedHistoryName]
- )history )reset
- )history )change n
- )history )memory
- )history )file
- %
- %%(n)
- )set history on | off
```

Command Description:

The history facility within AXIOM allows you to restore your environment to that of another session and recall previous computational results. Additional commands allow you to review previous input lines and to create an .input file of the lines typed to AXIOM.

AXIOM saves your input and output if the history facility is turned on (which is the default). This information is saved if either of

```
)set history on
)history )on
```

has been issued. Issuing either

```
)set history off
)history )off
```

will discontinue the recording of information.

Whether the facility is disabled or not, the value of % in AXIOM always refers to the result of the last computation. If you have not yet entered anything, % evaluates to an object of type Variable('%'). The function %% may be used to refer to other previous results if the history facility is enabled. In that case, %(n) is the output from step n if $n > 0$. If $n < 0$, the step is computed relative to the current step. Thus %(-1) is also the previous step, %(-2), is the step before that, and so on. If an invalid step number is given, AXIOM will signal an error.

The environment information can either be saved in a file or entirely in memory (the default). Each frame (description of command)frame) has its own history database. When it is kept in a file, some of it may also be kept in memory for efficiency. When the information is saved in a file, the name of the file is of the form FRAME.akh where ‘FRAME’ is the name of the current frame. The history file is placed in the current working directory (see description of command cd). Note that these history database files are not text files (in fact, they are directories themselves), and so are not in human-readable format.

The options to the)history command are as follows:

-)change n
will set the number of steps that are saved in memory to n. This option only has effect when the history data is maintained in a file. If you have issued)history)memory (or not changed the default) there is no need to use)history)change.
-)on
will start the recording of information. If the workspace is not empty, you will be asked to confirm this request. If you do so, the workspace will be cleared and history data will begin being saved. You can also turn the facility on by issuing)set history on.
-)off
will stop the recording of information. The)history)show command will not work after issuing this command. Note that this command may be issued to save time, as there is some performance penalty paid for saving the environment data. You can also turn the facility off by issuing)set history off.
-)file
indicates that history data should be saved in an external file on disk.
-)memory
indicates that all history data should be kept in memory rather than saved in a file. Note that if you are computing with very large objects it may not be practical to keep this data in memory.
-)reset
will flush the internal list of the most recent workspace calculations so

that the data structures may be garbage collected by the underlying Lisp system. Like `)history )change`, this option only has real effect when history data is being saved in a file.

`)restore [savedHistoryName]`
completely clears the environment and restores it to a saved session, if possible. The `)save` option below allows you to save a session to a file with a given name. If you had issued `)history )save jacobi` the command `)history )restore jacobi` would clear the current workspace and load the contents of the named saved session. If no saved session name is specified, the system looks for a file called `last.axh`.

`)save savedHistoryName`
is used to save a snapshot of the environment in a file. This file is placed in the current working directory (see description of command `)cd`). Use `)history )restore` to restore the environment to the state preserved in the file. This option also creates an input file containing all the lines of input since you created the workspace frame (for example, by starting your AXIOM session) or last did a `)clear all` or `)clear completely`.

`)show [n] [both]`
can show previous input lines and output results. `)show` will display up to twenty of the last input lines (fewer if you haven't typed in twenty lines). `)show n` will display up to `n` of the last input lines. `)show both` will display up to five of the last input lines and output results. `)show n both` will display up to `n` of the last input lines and output results.

`)write historyInputFile`
creates an `.input` file with the input lines typed since the start of the session/frame or the last `)clear all` or `)clear completely`. If `historyInputFileName` does not contain a period (`'.'`) in the filename, `.input` is appended to it. For example, `)history )write chaos` and `)history )write chaos.input` both write the input lines to a file called `chaos.input` in your current working directory. If you issued one or more `)undo` commands, `)history )write` eliminates all input lines backtracked over as a result of `)undo`. You can edit this file and then use `)read` to have AXIOM process the contents.

Also See:

- o `)frame`
- o `)read`
- o `)set`
- o `)undo`

⁹ “frame” (3.5 p 25) “read” (26.31 p 943) “set” (26.51 p 1081) “undo” (26.59 p 1112)

History recording is done in two different ways:

- all changes in variable bindings (i.e. previous values) are written to `$HistList`, which is a circular list
- all new bindings (including the binding to `%`) are written to a file called `histFileName()` one older session is accessible via the file `$oldHistFileName()`

26.23 Initialized history variables

The following global variables are used:

`$HistList`, `$HistListLen` and `$HistListAct` which is the actual number of “undoable” steps)

`$HistRecord` collects the input line, all variable bindings and the output of a step, before it is written to the file `histFileName()`.

`$HiFiAccess` is a flag, which is reset by `)history )off`

The result of step `n` can be accessed by `%n`, which is translated into a call of `fetchOutput(n)`. The `updateHist` is called after every interpreter step. The `putHist` function records all changes in the environment to `$HistList` and `$HistRecord`.

defvar \$oldHistoryFileName

— initvars —

```
(defvar |$oldHistoryFileName| '||last| "vm/370 filename name component")
```

—————

defvar \$historyFileType

— initvars —

```
(defvar |$historyFileType| '|axh|      "vm/370 filename type component")
```

—————

defvar \$historyDirectory

— initvars —

```
(defvar |$historyDirectory| 'A      "vm/370 filename disk component")
```

—————

defvar \$useInternalHistoryTable

— initvars —

```
(defvar |$useInternalHistoryTable| t  "t means keep history in core")
```

—————

defun makeHistFileName

```
[makePathname p1233]
```

— defun makeHistFileName —

```
(defun |makeHistFileName| (fname)
  (|makePathname| fname |$historyFileType| |$historyDirectory|))
```

—————

defun oldHistFileName

```
[makeHistFileName p887]
```

```
[$oldHistoryFileName p886]
```

— defun oldHistFileName —

```
(defun |oldHistFileName| ()
  (declare (special |$oldHistoryFileName|))
  (|makeHistFileName| |$oldHistoryFileName|))
```

—————

defun histFileName

```
[makeHistFileName p887]
[$interpreterFrameName p38]
```

— defun histFileName —

```
(defun |histFileName| ()
  (declare (special |$interpreterFrameName|))
  (|makeHistFileName| |$interpreterFrameName|))
```

—————

defun histInputFileName

```
[makePathname p1233]
[$interpreterFrameName p38]
[$historyDirectory p887]
```

— defun histInputFileName —

```
(defun |histInputFileName| (fn)
  (declare (special |$interpreterFrameName| |$historyDirectory|))
  (if (null fn)
      (|makePathname| |$interpreterFrameName| 'input |$historyDirectory|)
      (|makePathname| fn 'input |$historyDirectory|)))
```

—————

defun initHist

```
[initHistList p889]
[oldHistFileName p887]
[histFileName p888]
[histFileErase p928]
[makeInputFilename p1169]
[$replace p??]
[$useInternalHistoryTable p887]
[$HiFiAccess p1005]
```

— defun initHist —

```
(defun |initHist| ()
  (let (oldFile newFile)
```

```
(declare (special |$useInternalHistoryTable| |$HiFiAccess|))
(if |$useInternalHistoryTable|
  (|initHistList|)
  (progn
    (setq oldFile (|oldHistFileName|))
    (setq newFile (|histFileName|))
    (|histFileErase| oldFile)
    (when (makeInputFilename newFile) (replaceFile oldFile newFile))
    (setq |$HiFiAccess| t)
    (|initHistList|))))
```

defun initHistList

[[\\$HistListLen](#) p48]
 [[\\$HistList](#) p47]
 [[\\$HistListAct](#) p48]
 [[\\$HistRecord](#) p48]

— defun initHistList —

```
(defun |initHistList| ()
  (let (li)
    (declare (special |$HistListLen| |$HistList| |$HistListAct| |$HistRecord|))
    (setq |$HistListLen| 20)
    (setq |$HistList| (list nil))
    (setq li |$HistList|)
    (do ((i 1 (1+ i)))
      ((> i |$HistListLen|) nil)
      (setq li (cons nil li)))
    (rplacd |$HistList| li)
    (setq |$HistListAct| 0)
    (setq |$HistRecord| nil)))
```

The top level history command

[[sayKeyedMsg](#) p43]
 [[historySpad2Cmd](#) p890]
 [[\\$options](#) p73]

— defun history —


```
(defun |history| (l)
  "The top level history command"
  (declare (special |$options|))
  (if (or l (null |$options|))
      (|sayKeyedMsg|
       (format nil
                "You have not used the correct syntax for the history command. ~
                Issue )help history for more information.")
       nil)
      (|historySpad2Cmd|)))
```

The top level history command handler

```
[selectOptionLC p819]
[member p1238]
[sayKeyedMsg p43]
[initHistList p889]
[upcase p1272]
[queryUserKeyedMsg p??]
[string2id-n p??]
[histFileErase p928]
[histFileName p888]
[clearSpad2Cmd p835]
[disableHist p913]
[setHistoryCore p894]
[resetInCoreHist p898]
[saveHistory p905]
[showHistory p892]
[changeHistListLen p898]
[restoreHistory p907]
[writeInputLines p896]
[seq p??]
[exit p??]
[$options p73]
[$HiFiAccess p1005]
[$IOindex p39]
```

— defun historySpad2Cmd —

```
(defun |historySpad2Cmd| ()
  "The top level history command handler"
  (let (histOptions opts opt optargs x)
    (declare (special |$options| |$HiFiAccess| |$IOindex|))
    (setq histOptions
```

```

'(|on| |off| |yes| |no| |change| |reset| |restore| |write|
 |save| |show| |file| |memory|))
(setq opts
  (prog (tmp1)
    (setq tmp1 nil)
    (return
      (do ((tmp2 |$options| (cdr tmp2)) (tmp3 nil))
        ((or (atom tmp2)
              (progn
                (setq tmp3 (car tmp2))
                nil)
              (progn
                (progn
                  (setq opt (car tmp3))
                  (setq optargs (cdr tmp3))
                  tmp3)
                nil))
            (nreverse0 tmp1)))
      (setq tmp1
        (cons
          (cons
            (|selectOptionLC| opt histOptions '|optionError|)
            optargs)
          tmp1))))))
(do ((tmp4 opts (cdr tmp4)) (tmp5 nil))
  ((or (atom tmp4)
        (progn
          (setq tmp5 (car tmp4))
          nil)
        (progn
          (progn
            (setq opt (car tmp5))
            (setq optargs (cdr tmp5))
            tmp5)
          nil))
    nil)
  (seq
    (exit
      (cond
        ((|member| opt '|on| |yes|))
        (cond
          (|$HiFiAccess|
            (|sayKeyedMsg| "The history facility is already on." nil))
          ((eq1 |$I0index| 1)
            (setq |$HiFiAccess| t)
            (|initHistList|)
            (|sayKeyedMsg| "The history facility is now on." nil))
          (t
            (setq x ; really want to turn history on?
              (upcase

```

```

(|queryUserKeyedMsg|
  (format nil
    "Turning on the history facility will clear the contents ~
    of the workspace. Please enter y or yes if you really ~
    want to do this:")
    nil)))
(cond
  ((member (string2id-n x 1) '(Y YES))
    (|histFileErase| (|histFileName|))
    (setq |$HiFiAccess| t)
    (setq |$options| nil)
    (|clearSpad2Cmd| '(|all|))
    (|sayKeyedMsg| "The history facility is now on." nil)
    (|initHistList|))
  (t
    (|sayKeyedMsg| "The history facility is still off." nil))))))
((|member| opt '(|off| |no|))
  (cond
    ((null |$HiFiAccess|)
      (|sayKeyedMsg| "The history facility is already off." nil))
    (t
      (setq |$HiFiAccess| nil)
      (|disableHist|)
      (|sayKeyedMsg| "The history facility is now off." nil))))))
((eq opt '|file|) (|setHistoryCore| nil))
((eq opt '|memory|) (|setHistoryCore| t))
((eq opt '|reset|) (|resetInCoreHist|))
((eq opt '|save|) (|saveHistory| optargs))
((eq opt '|show|) (|showHistory| optargs))
((eq opt '|change|) (|changeHistListLen| (car optargs)))
((eq opt '|restore|) (|restoreHistory| optargs))
((eq opt '|write|) (|writeInputLines| optargs 1))))))
'|done|))

```

defun showHistory

```

[sayKeyedMsg p43]
[selectOptionLC p819]
[sayMSG p45]
[concat p1237]
[bright p??]
[showInOut p910]
[setIOindex p909]
[showInput p909]
[$printTimeSum p??]

```

[\$evalTimePrint p??]

— defun showHistory —

```
(defun |showHistory| (arg)
  (let (|$printTimeSum| |$evalTimePrint| maxi mini arg2 arg1
        nset n showInputOrBoth)
    (declare (special |$printTimeSum| |$evalTimePrint| |$HiFiAccess|))
    (setq |$evalTimePrint| 0)
    (setq |$printTimeSum| 0)
    (cond
      ((null |$HiFiAccess|)
       (|sayKeyedMsg|
        (format nil
          "The history facility command %1 cannot be performed because the ~
          history facility is not on.")
        (list '|show|)))
      (t
       (setq showInputOrBoth '|input|)
       (setq n 20)
       (when arg
        (setq arg1 (car arg))
        (when (integerp arg1)
         (setq n arg1)
         (setq nset t)
         (cond
           ((ifcdr arg) (setq arg1 (cadr arg)))
           (t (setq arg1 nil))))
        (when arg1
         (setq arg2 (|selectOptionLC| arg1 '(|input| |both|) nil))
         (cond
           (arg2
            (cond
              ((and (eq (setq showInputOrBoth arg2) '|both|)
                    (null nset))
               (setq n 5))))
            (t
             (|sayMSG|
              (|concat| " " (|bright| arg1) "is an invalid argument."))))))
         (cond ((not (< n |$IOindex|)) (setq n (- |$IOindex| 1))))
         (setq mini (- |$IOindex| n))
         (setq maxi (- |$IOindex| 1))
         (cond
           ((eq showInputOrBoth '|both|)
            (unwind-protect
              (|showInOut| mini maxi)
              (|setIOindex| (+ maxi 1))))
           (t (|showInput| mini maxi))))))
       (t (|showInput| mini maxi))))))
```

defun setHistoryCore

We case on the inCore argument value

If history is already on and is kept in the same location as requested (file or memory) then complain.

If history is not in use then start using the file or memory as requested. This is done by simply setting the `$useInternalHistoryTable` to the requested value, where T means use memory and NIL means use a file. We tell the user.

If history should be in memory, that is inCore is not NIL, and the history file already contains information we read the information from the file, store it in memory, and erase the history file. We modify `$useInternalHistoryTable` to T to indicate that we're maintaining the history in memory and tell the user.

Otherwise history must be on and in memory. We erase any old history file and then write the in-memory history to a new file

```
[boot-equal p??]
[sayKeyedMsg p43]
[rkeyids p??]
[histFileName p888]
[readHiFi p911]
[disableHist p913]
[histFileErase p928]
[rdefiostream p??]
[spadrwrite p915]
[object2Identifier p??]
[rshut p??]
[$useInternalHistoryTable p887]
[$internalHistoryTable p48]
[$HiFiAccess p1005]
[$IOindex p39]
```

— defun setHistoryCore —

```
(defun |setHistoryCore| (inCore)
  (let (l vec str n rec)
    (declare (special |$useInternalHistoryTable| |$internalHistoryTable|
                      |$HiFiAccess| |$IOindex|))
    (cond
      ((boot-equal inCore |$useInternalHistoryTable|)
       (if inCore
           (|sayKeyedMsg|
            (format nil
                    "History information is already being maintained in memory (and ~
                    not in an external file).")
            nil)
           nil))
```

```

(|sayKeyedMsg|
 (format nil
  "History information is already being maintained in an external ~
  file (and not in memory).")
 nil))) ; file history already in use
((null |$HiFiAccess|)
 (setq |$useInternalHistoryTable| inCore)
 (if inCore
  (|sayKeyedMsg|
   (format nil
    "When the history facility is active, history information will be ~
    maintained in memory (and not in an external file).")
    nil)
  (|sayKeyedMsg|
   (format nil
    "When the history facility is active, history information will be ~
    maintained in a file (and not in an internal table).")
    nil)))
(inCore
 (setq |$internalHistoryTable| nil)
 (cond
  ((not (eql |$IOindex| 0))
   (setq l (length (rkeyids (|histFileName|))))
   (do ((i 1 (1+ i)))
       ((> i l) nil)
       (setq vec (unwind-protect (|readHiFi| i) (|disableHist|)))
       (setq |$internalHistoryTable|
        (cons (cons i vec) |$internalHistoryTable|)))
   (|histFileErase| (|histFileName|)))
  (t
   (setq |$useInternalHistoryTable| t)
   (|sayKeyedMsg|
    (format nil
     "When the history facility is active, history information will be ~
     maintained in memory (and not in an external file).")
     nil))
  (t
   (setq |$HiFiAccess| nil)
   (|histFileErase| (|histFileName|))
   (setq str
    (rdefiostream
     (cons
      '(mode . output)
      (cons
       (cons 'file (|histFileName|))
       nil))))
   (do ((tmp0 (reverse |$internalHistoryTable|) (cdr tmp0))
       (tmp1 nil))
       ((or (atom tmp0)
        (progn
         (setq tmp1 (car tmp0))
         (setq tmp1 (car tmp0))

```

```

        nil)
      (progn
        (progn
          (setq n (car tmp1))
          (setq rec (cdr tmp1))
          tmp1)
        nil))
      nil)
    (spadrwrite (|object2Identifier| n) rec str))
  (rshut str)
  (setq |$HiFiAccess| t)
  (setq |$internalHistoryTable| nil)
  (setq |$useInternalHistoryTable| nil)
  (|sayKeyedMsg|
   (format nil
    "When the history facility is active, history information will be ~
    maintained in a file (and not in an internal table).")
   nil))))

```

defvar \$underbar

Also used in the output routines.

— **initvars** —

```
(defvar underbar "_")
```

defun writeInputLines

```

[sayKeyedMsg p43]
[throwKeyedMsg p??]
[size p1236]
[concat p1237]
[substring p324]
[readHiFi p911]
[histInputFileName p888]
[histFileErase p928]
[defiostream p1168]
[namestring p1230]
[shut p1168]
[underbar p896]

```

[[\\$HiFiAccess p1005](#)]

[[\\$IOindex p39](#)]

— defun writeInputLines —

```
(defun |writeInputLines| (fn initial)
  (let (maxn breakChars vec1 k svec done n lineList file inp)
    (declare (special underbar |$HiFiAccess| |$IOindex|))
    (cond
      ((null |$HiFiAccess|)
       (|sayKeyedMsg|
        (format nil
          "The history facility is not on, so the .input file containing your ~
          user input cannot be created.")
        nil))
      ((null fn)
       (|throwKeyedMsg|
        "You must specify a file name to the history write command" nil))
      (t
       (setq maxn 72)
       (setq breakChars (cons ' | (cons '+ nil)))
       (do ((tmp0 (- |$IOindex| 1))
            (i initial (+ i 1)))
           ((> i tmp0) nil)
           (setq vec1 (car (|readHiFi| i)))
           (when (stringp vec1) (setq vec1 (cons vec1 nil)))
           (dolist (vec vec1)
            (setq n (size vec))
            (do ()
                ((null (> n maxn)) nil)
                (setq done nil)
                (do ((j 1 (1+ j)))
                    ((or (> j maxn) (null (null done))) nil)
                    (setq k (- (1+ maxn) j))
                    (when (member (elt vec k) breakChars)
                     (setq svec (concat (substring vec 0 (1+ k)) underbar))
                     (setq lineList (cons svec lineList))
                     (setq done t)
                     (setq vec (substring vec (1+ k) nil))
                     (setq n (size vec))))
                    (when done (setq n 0)))
                (setq lineList (cons vec lineList))))
           (setq file (|histInputFileName| fn))
           (|histFileErase| file)
           (setq inp
            (defiostream
             (cons
              '(mode . output)
              (cons (cons 'file file) nil)) 255 0))
           (dolist (x (|removeUndoLines| (nreverse lineList)))
```



```

(write-line x inp))
(cond
  ((not (eq fn '|redo|))
    (|sayKeyedMsg| "Edit %1 to see the saved input lines."
      (list (|namestring| file)))))
(shut inp)
nil)))

```

defun resetInCoreHist

[[\\$HistListAct](#) p48]
 [[\\$HistListLen](#) p48]
 [[\\$HistList](#) p47]

— defun resetInCoreHist —

```

(defun |resetInCoreHist| ()
  (declare (special |$HistListAct| |$HistListLen| |$HistList|))
  (setq |$HistListAct| 0)
  (do ((i 1 (1+ i)))
    ((> i |$HistListLen|) nil)
    (setq |$HistList| (cdr |$HistList|))
    (rplaca |$HistList| nil)))

```

defun changeHistListLen

[[sayKeyedMsg](#) p43]
 [[\\$HistListLen](#) p48]
 [[\\$HistList](#) p47]
 [[\\$HistListAct](#) p48]

— defun changeHistListLen —

```

(defun |changeHistListLen| (n)
  (let (dif 1)
    (declare (special |$HistListLen| |$HistList| |$HistListAct|))
    (if (null (integerp n))
      (|sayKeyedMsg|
        (format nil
          "The argument n for )history )change n must be a nonnegative integer ~
          and your argument, %1 , is not one."))

```

```

(list n)) ; only positive integers
(progn
  (setq dif (- n |$HistListLen|))
  (setq |$HistListLen| n)
  (setq l (cdr |$HistList|))
  (cond
    ((> dif 0)
     (do ((i 1 (1+ i)))
         ((> i dif) nil)
         (setq l (cons nil l))))
    ((minusp dif)
     (do ((tmp0 (- dif))
         (i 1 (1+ i)))
         ((> i tmp0) nil)
         (setq l (cdr l)))
      (cond
        ((> |$HistListAct| n) (setq |$HistListAct| n))
        (t nil))))
    (rplacd |$HistList| l)
    'done))))

```

defun updateHist

```

[startTimingProcess p??]
[updateInCoreHist p900]
[writeHiFi p912]
[disableHist p913]
[updateCurrentInterpreterFrame p31]
[stopTimingProcess p??]
[$IOindex p39]
[$HiFiAccess p1005]
[$HistRecord p48]
[$mkTestInputStack p??]
[$currentLine p??]

```

— defun updateHist —

```

(defun |updateHist| ()
  (declare (special |$IOindex| |$HiFiAccess| |$HistRecord| |$mkTestInputStack|
    |$currentLine|))
  (when |$IOindex|
    (|startTimingProcess| '|history|)
    (|updateInCoreHist|)
    (when |$HiFiAccess|
      (unwind-protect (|writeHiFi|) (|disableHist|))

```

```

      (setq |$HistRecord| nil))
    (incf |$IOindex|)
    (|updateCurrentInterpreterFrame|)
    (setq |$mkTestInputStack| nil)
    (setq |$currentLine| nil)
    (|stopTimingProcess| '|history|)))

```

defun updateInCoreHist

[[\\$HistList](#) p47]
 [[\\$HistListLen](#) p48]
 [[\\$HistListAct](#) p48]

— defun updateInCoreHist —

```

(defun |updateInCoreHist| ()
  (declare (special |$HistList| |$HistListLen| |$HistListAct|))
  (setq |$HistList| (cdr |$HistList|))
  (rplaca |$HistList| nil)
  (when (> |$HistListLen| |$HistListAct|)
    (setq |$HistListAct| (1+ |$HistListAct|))))

```

defun putHist

[[recordOldValue](#) p901]
 [get p??]
 [[recordNewValue](#) p901]
 [putIntSymTab p??]
 [[\\$HiFiAccess](#) p1005]

— defun putHist —

```

(defun |putHist| (x prop val e)
  (declare (special |$HiFiAccess|))
  (when (null (eq x '%)) (|recordOldValue| x prop (|get| x prop e)))
  (when |$HiFiAccess| (|recordNewValue| x prop val))
  (|putIntSymTab| x prop val e))

```

defun recordNewValue

```
[startTimingProcess p??]
[recordNewValue0 p901]
[stopTimingProcess p??]
```

— defun recordNewValue —

```
(defun |recordNewValue| (x prop val)
  (|startTimingProcess| '|history|)
  (|recordNewValue0| x prop val)
  (|stopTimingProcess| '|history|))
```

—————

defun recordNewValue0

```
[assq p1240]
[$HistRecord p48]
```

— defun recordNewValue0 —

```
(defun |recordNewValue0| (x prop val)
  (let (p1 p2 p)
    (declare (special |$HistRecord|))
    (if (setq p1 (assq x |$HistRecord|))
      (if (setq p2 (assq prop (cdr p1)))
        (rplacd p2 val)
        (rplacd p1 (cons (cons prop val) (cdr p1))))
      (progn
        (setq p (cons x (list (cons prop val))))
        (setq |$HistRecord| (cons p |$HistRecord|))))))
```

—————

defun recordOldValue

```
[startTimingProcess p??]
[recordOldValue0 p902]
[stopTimingProcess p??]
[assq p1240]
```

— defun recordOldValue —

```
(defun |recordOldValue| (x prop val)
  (|startTimingProcess| ' |history|)
  (|recordOldValue0| x prop val)
  (|stopTimingProcess| ' |history|))
```

defun recordOldValue0

[[\\$HistList](#) [p47](#)]

— defun recordOldValue0 —

```
(defun |recordOldValue0| (x prop val)
  (let (p1 p)
    (declare (special |$HistList|))
    (when (setq p1 (assq x (car |$HistList|)))
      (when (null (assq prop (cdr p1)))
        (rplacd p1 (cons (cons prop val) (cdr p1)))))
    (setq p (cons x (list (cons prop val))))
    (rplaca |$HistList| (cons p (car |$HistList|)))))
```

defun undoInCore

[[undoChanges](#) [p903](#)]
 [[readHiFi](#) [p911](#)]
 [[disableHist](#) [p913](#)]
 [[assq](#) [p1240](#)]
 [[sayKeyedMsg](#) [p43](#)]
 [[putHist](#) [p900](#)]
 [[updateHist](#) [p899](#)]
 [[\\$HistList](#) [p47](#)]
 [[\\$HistListLen](#) [p48](#)]
 [[\\$IOindex](#) [p39](#)]
 [[\\$HiFiAccess](#) [p1005](#)]
 [[\\$InteractiveFrame](#) [p39](#)]

— defun undoInCore —

```
(defun |undoInCore| (n)
  (let (li vec p p1 val)
    (declare (special |$HistList| |$HistListLen| |$IOindex| |$HiFiAccess|
```

```

    |$InteractiveFrame|))
  (setq li |$HistList|)
  (do ((i n (+ i 1)))
      ((> i |$HistListLen|) nil)
    (setq li (cdr li)))
  (|undoChanges| li)
  (setq n (- (- |$IOindex| n) 1))
  (and
    (> n 0)
    (if |$HiFiAccess|
      (progn
        (setq vec (cdr (unwind-protect (|readHiFi| n) (|disableHist|))))
        (setq val
          (and
            (setq p (assq '% vec))
            (setq p1 (assq '|value| (cdr p)))
            (cdr p1))))
        (|sayKeyedMsg|
          "There is no history file, so value of step %1 is undefined."
          (cons n nil)))) ; no history file
    (setq |$InteractiveFrame| (|putHist| '% '|value| val |$InteractiveFrame|))
    (|updateHist|)))

```

defun undoChanges

```

[boot-equal p??]
[undoChanges p903]
[putHist p900]
[$HistList p47]
[$InteractiveFrame p39]

```

— defun undoChanges —

```

(defun |undoChanges| (li)
  (let (x)
    (declare (special |$HistList| |$InteractiveFrame|))
    (when (null (boot-equal (cdr li) |$HistList|)) (|undoChanges| (cdr li)))
    (dolist (p1 (car li))
      (setq x (car p1))
      (dolist (p2 (cdr p1))
        (|putHist| x (car p2) (cdr p2) |$InteractiveFrame|))))))

```

defun undoFromFile

```

[seq p??]
[exit p??]
[recordOldValue p901]
[recordNewValue p901]
[readHiFi p911]
[disableHist p913]
[putHist p900]
[assq p1240]
[updateHist p899]
[$InteractiveFrame p39]
[$HiFiAccess p1005]

```

— defun undoFromFile —

```

(defun |undoFromFile| (n)
  (let (var1 prop vec x p p1 val)
    (declare (special |$InteractiveFrame| |$HiFiAccess|))
    (do ((tmp0 (caar |$InteractiveFrame|) (cdr tmp0)) (tmp1 nil))
      ((or (atom tmp0)
            (progn (setq tmp1 (car tmp0)) nil)
            (progn
              (progn
                (setq x (car tmp1))
                (setq var1 (cdr tmp1))
                tmp1)
              nil))
           nil)
      (seq
        (exit
          (do ((tmp2 var1 (cdr tmp2)) (p nil))
              ((or (atom tmp2) (progn (setq p (car tmp2)) nil)) nil)
          (seq
            (exit
              (progn
                (setq prop (car p))
                (setq val (cdr p))
                (when val
                  (progn
                    (when (null (eq x '%))
                      (|recordOldValue| x prop val))
                    (when |$HiFiAccess|
                      (|recordNewValue| x prop val))
                    (rplacd p nil))))))))))
      (do ((i 1 (1+ i)))
          ((> i n) nil)
          (setq vec
            (unwind-protect (cdr (|readHiFi| i)) (|disableHist|)))

```

```

(do ((tmp3 vec (cdr tmp3)) (p1 nil))
  ((or (atom tmp3) (progn (setq p1 (car tmp3)) nil)) nil)
  (setq x (car p1))
  (do ((tmp4 (cdr p1) (cdr tmp4)) (p2 nil))
    ((or (atom tmp4) (progn (setq p2 (car tmp4)) nil)) nil)
    (setq |$InteractiveFrame|
      (|putHist| x (car p2) (CDR p2) |$InteractiveFrame|))))
(setq val
  (and
    (setq p (assq '% vec))
    (setq p1 (assq '|value| (cdr p)))
    (cdr p1)))
(setq |$InteractiveFrame| (|putHist| '% '|value| val |$InteractiveFrame|))
(|updateHist|)))

```

defun saveHistory

```

[sayKeyedMsg p43]
[makeInputFilename p1169]
[histFileName p888]
[throwKeyedMsg p??]
[makeHistFileName p887]
[histInputFileName p888]
[writeInputLines p896]
[histFileErase p928]
[rdefiostream p??]
[spadrwrite0 p914]
[object2Identifier p??]
[rshut p??]
[namestring p1230]
[$seen p??]
[$HiFiAccess p1005]
[$useInternalHistoryTable p887]
[$internalHistoryTable p48]

```

— defun saveHistory —

```

(defun |saveHistory| (fn)
  (let (|$seen| savefile inputfile saveStr n rec val)
    (declare (special |$seen| |$HiFiAccess| |$useInternalHistoryTable|
      |$internalHistoryTable|))
    (setq |$seen| (make-hash-table :test #'eq))
    (cond
      ((null |$HiFiAccess|)

```



```

(|sayKeyedMsg|
  "The history facility is not on, so no information can be saved."
  nil)) ; the history file is not on
(and (null |$useInternalHistoryTable|)
      (null (makeInputFilename (|histFileName|))))
(|sayKeyedMsg| "No history information had been saved yet." nil))
(null fn)
(|throwKeyedMsg|
  "You must specify a file name to the history save command"
  nil))
(t
  (setq savefile (|makeHistFileName| fn))
  (setq inputfile (|histInputFileName| fn))
  (|writeInputLines| fn 1)
  (|histFileErase| savefile)
  (when |$useInternalHistoryTable|
    (setq saveStr
      (rdefiostream
        (cons '(mode . output)
              (cons (cons 'file savefile) nil))))
    (do ((tmp0 (reverse |$internalHistoryTable|) (cdr tmp0))
        (tmp1 nil))
        ((or (atom tmp0)
              (progn (setq tmp1 (car tmp0)) nil)
                     (progn
                      (progn
                        (setq n (car tmp1))
                        (setq rec (cdr tmp1))
                        tmp1)
                      nil))
             nil)
      (setq val (spadrwrite0 (|object2Identifier| n) rec saveStr))
      (when (eq val '|writifyFailed|)
        (|sayKeyedMsg|
          (format nil
            "Can't save the value of step number %1. You can re-generate ~
            this value by running the input file %2.")
            (list n inputfile))))
      (rshut saveStr))
    (|sayKeyedMsg| "The saved history file is %1 ."
      (cons (|namestring| savefile) nil))
    nil))))

```

[qcdr p??]
[qcar p??]
[identp p1237]
[throwKeyedMsg p??]
[makeHistFileName p887]
[putHist p900]
[makeInputFilename p1169]
[sayKeyedMsg p43]
[namestring p1230]
[clearSpad2Cmd p835]
[histFileName p888]
[histFileErase p928]
[\$fcopy p??]
[rkeyids p??]
[readHiFi p911]
[disableHist p913]
[updateInCoreHist p900]
[get p??]
[rempropI p??]
[clearCmdSortedCaches p837]
[\$options p73]
[\$internalHistoryTable p48]
[\$HiFiAccess p1005]
[\$e p315]
[\$useInternalHistoryTable p887]
[\$InteractiveFrame p39]
[\$oldHistoryFileName p886]

```
(defun |restoreHistory| (fn)
  (let (|$options| fnq restfile curfile l oldInternal vec line x a)
    (declare (special |$options| |$internalHistoryTable| |$HiFiAccess| |$e|
      |$useInternalHistoryTable| |$InteractiveFrame| |$oldHistoryFileName|))
    (cond
      ((null fn) (setq fnq |$oldHistoryFileName|))
      ((and (consp fn)
        (eq (qcdr fn) nil)
        (progn
          (setq fnq (qcar fn))
          t)
        (identp fnq)))
        (setq fnq fnq))
      (t (|throwKeyedMsg| "%1 is not a valid filename for the history file."
        (cons fnq nil)))))) ; invalid filename
```

```

(setq restfile (|makeHistFileName| fnq))
(if (null (makeInputFilename restfile))
    (|sayKeyedMsg|
     (format nil
              "History information cannot be restored from %1 because the file does ~
              not exist.")
      (cons (|namestring| restfile) nil)))
(progn
  (setq |$options| nil)
  (|clearSpad2Cmd| '(|all|))
  (setq curfile (|histFileName|))
  (|histFileErase| curfile)
  ($fcopy restfile curfile)
  (setq l (length (rkeyids curfile)))
  (setq |$HiFiAccess| t)
  (setq oldInternal |$useInternalHistoryTable|)
  (setq |$useInternalHistoryTable| nil)
  (when oldInternal (setq |$internalHistoryTable| nil))
  (do ((i 1 (1+ i)))
      ((> i l) nil)
    (setq vec (unwind-protect (|readHiFi| i) (|disableHist|)))
    (when oldInternal
      (setq |$internalHistoryTable|
        (cons (cons i vec) |$internalHistoryTable|)))
    (setq line (car vec))
    (dolist (p1 (cdr vec))
      (setq x (car p1))
      (do ((tmp1 (cdr p1) (cdr tmp1)) (p2 nil))
          ((or (atom tmp1) (progn (setq p2 (car tmp1)) nil)) nil)
        (setq |$InteractiveFrame|
          (|putHist| x
                    (car p2) (cdr p2) |$InteractiveFrame|)))
      (|updateInCoreHist|))
    (setq |$e| |$InteractiveFrame|)
    (do ((tmp2 (caar |$InteractiveFrame|) (cdr tmp2)) (tmp3 nil))
        ((or (atom tmp2)
              (progn
                (setq tmp3 (car tmp2))
                nil)
              (progn
                (progn
                  (setq a (car tmp3))
                  tmp3)
                nil))
              nil)
          (when (|get| a '|localModemap| |$InteractiveFrame|)
            (|rempropI| a '|localModemap|)
            (|rempropI| a '|localVars|)
            (|rempropI| a '|mapBody|)))
        (setq |$IOindex| (1+ l)))

```

```
(setq |$useInternalHistoryTable| oldInternal)
(|sayKeyedMsg|
  "The workspace has been successfully restored from the history file %1 ."
  (cons (|namestring| restfile) nil))
(|clearCmdSortedCaches|
  nil))))
```

defun setIOindex

[[\\$IOindex](#) [p39](#)]

— defun setIOindex —

```
(defun |setIOindex| (n)
  (declare (special |$IOindex|))
  (setq |$IOindex| n))
```

defun showInput

[[tab](#) [p??](#)]
 [[readHiFi](#) [p911](#)]
 [[disableHist](#) [p913](#)]
 [[sayMSG](#) [p45](#)]

— defun showInput —

```
(defun |showInput| (mini maxi)
  (let (vec 1)
    (do ((|ind| mini (+ |ind| 1)))
      ((> |ind| maxi) nil)
      (setq vec (unwind-protect (|readHiFi| |ind|) (|disableHist|))))
    (cond
      ((> 10 |ind|) (tab 2))
      ((> 100 |ind|) (tab 1))
      (t nil))
    (setq l (car vec))
    (if (stringp l)
      (|sayMSG| (list " [" |ind| "]" " (car vec)))
      (progn
        (|sayMSG| (list " [" |ind| "]" ""))
        (do ((tmp0 l (cdr tmp0)) (ln nil))
```

```
((or (atom tmp0) (progn (setq ln (car tmp0)) nil)) nil)
(|sayMSG| (list " " ln)))))))))
```

defun showInOut

```
[assq p1240]
[spadPrint p??]
[objValUnwrap p516]
[objMode p516]
[readHiFi p911]
[disableHist p913]
[sayMSG p45]
```

— defun showInOut —

```
(defun |showInOut| (mini maxi)
  (let (vec Alist triple)
    (do ((ind mini (+ ind 1)))
      (> ind maxi) nil)
    (setq vec (unwind-protect (|readHiFi| ind) (|disableHist|)))
    (|sayMSG| (cons (car vec) nil))
    (cond
      ((setq Alist (assq '% (cdr vec)))
       (setq triple (cdr (assq '|value| (cdr Alist))))
       (setq |$IOindex| ind)
       (|spadPrint| (|objValUnwrap| triple) (|objMode| triple)))))))
```

defun fetchOutput

```
[boot-equal p??]
[getI p??]
[throwKeyedMsg p??]
[readHiFi p911]
[disableHist p913]
[assq p1240]
```

— defun fetchOutput —

```
(defun |fetchOutput| (n)
  (let (vec Alist val)
```

```

(cond
  ((and (boot-equal n (- 1)) (setq val (|getI| '% '|value|)))
    val)
  (|$HiFiAccess|
    (setq n
      (cond
        ((minusp n) (+ |$IOindex| n))
        (t n)))
      (cond
        ((>= n |$IOindex|)
          (|throwKeyedMsg|
            "You have not reached step %1, and so its value cannot be supplied."
            (cons n nil)))
        ((> 1 n)
          (|throwKeyedMsg|
            "Cannot supply value for step %1b because 1 is the first step."
            (cons n nil))) ; only nonzero steps
        (t
          (setq vec (unwind-protect (|readHiFi| n) (|disableHist|)))
            (cond
              ((setq Alist (assq '% (cdr vec)))
                (cond
                  ((setq val (cdr (assq '|value| (cdr Alist))))
                    val)
                  (t
                     (|throwKeyedMsg| "Step %1 has no value." (cons n nil))))
                (t (|throwKeyedMsg| "Step %1 has no value." (cons n nil))))))
            (t (|throwKeyedMsg|
              "The history facility is not on, so you cannot use %%.
              nil)))) ; history not on

```

Read the history file using index n

```

[assoc p??]
[keyedSystemError p??]
[qcdr p??]
[rdefiostream p??]
[histFileName p888]
[spadrread p915]
[object2Identifier p??]
[rshut p??]
[$useInternalHistoryTable p887]
[$internalHistoryTable p48]

```

— defun readHiFi —

```
(defun |readHiFi| (n)
  "Read the history file using index n"
  (let (pair HiFi vec)
    (declare (special |$useInternalHistoryTable| |$internalHistoryTable|))
    (if |$useInternalHistoryTable|
      (progn
        (setq pair (|assoc| n |$internalHistoryTable|))
        (if (atom pair)
          (|keyedSystemError| "Missing element in internal history table." nil)
          (setq vec (qcdr pair))))
      (progn
        (setq HiFi
          (rdefiostream
            (cons
              '(mode . input)
              (cons
                (cons 'file (|histFileName|)) nil))))
        (setq vec (spadrread (|object2Identifier| n) HiFi))
        (rshut HiFi))
      vec))
```

Write information of the current step to history file

```
[rdefiostream p??]
[histFileName p888]
[spadrwrite p915]
[object2Identifier p??]
[rshut p??]
[$useInternalHistoryTable p887]
[$internalHistoryTable p48]
[$IOindex p39]
[$HistRecord p48]
[$currentLine p??]
```

— defun writeHiFi —

```
(defun |writeHiFi| ()
  "Writes information of the current step to history file"
  (let (HiFi)
    (declare (special |$useInternalHistoryTable| |$internalHistoryTable|
      |$IOindex| |$HistRecord| |$currentLine|))
    (if |$useInternalHistoryTable|
      (setq |$internalHistoryTable|
        (cons
          (cons |$IOindex|
```

```

      (cons |$currentLine| |$HistRecord|))
      |$internalHistoryTable|))
    (progn
      (setq HiFi
        (rdefiostream
          (cons
            '(mode . output)
            (cons (cons 'file (|histFileName|)) nil))))
      (spadrwrite (|object2Identifier| |$IOindex|)
        (cons |$currentLine| |$HistRecord|) HiFi)
      (rshut HiFi))))))

```

Disable history if an error occurred

[histFileErase p928]
 [histFileName p888]
 [\$HiFiAccess p1005]

— defun disableHist —

```

(defun |disableHist| ()
  "Disable history if an error occurred"
  (declare (special |$HiFiAccess|))
  (cond
    ((null |$HiFiAccess|)
     (|histFileErase| (|histFileName|)))
    (t nil)))

```

defun writeHistModesAndValues

[get p??]
 [putHist p900]
 [\$InteractiveFrame p39]

— defun writeHistModesAndValues —

```

(defun |writeHistModesAndValues| ()
  (let (a x)
    (declare (special |$InteractiveFrame|))
    (do ((tmp0 (caar |$InteractiveFrame|) (cdr tmp0)) (tmp1 nil))
        ((or (atom tmp0)

```



```

      (progn
        (setq tmp1 (car tmp0))
        nil)
      (progn
        (progn
          (setq a (car tmp1))
          tmp1)
        nil))
    nil)
  (cond
    ((setq x (|get| a '|value| |$InteractiveFrame|))
     (|putHist| a '|value| x |$InteractiveFrame|))
    ((setq x (|get| a '|mode| |$InteractiveFrame|))
     (|putHist| a '|mode| x |$InteractiveFrame|))))))

```

Lisplib output transformations

Some types of objects cannot be saved by LISP/VM in lislibs. These functions transform an object to a writable form and back.

defun spadrwrite0

[safeWritify [p917](#)]
 [rwrite [p914](#)]

— defun spadrwrite0 —

```

(defun spadrwrite0 (vec item stream)
  (let (val)
    (setq val (|safeWritify| item))
    (if (eq val '|writifyFailed|)
        val
        (progn
          (|rwrite| vec val stream)
          item))))

```

defun Random write to a stream

[rwrite [p914](#)]
 [pname [p1235](#)]
 [identp [p1237](#)]

— defun rwrite —

```
(defun |rwrite| (key val stream)
  (when (identp key) (setq key (pname key)))
  (rwrite key val stream))
```

—————

defun spadrwrite

[spadrwrite0 p914]
[throwKeyedMsg p??]

— defun spadrwrite —

```
(defun spadrwrite (vec item stream)
  (let (val)
    (setq val (spadrwrite0 vec item stream))
    (if (eq val '|writifyFailed|)
        (|throwKeyedMsg| "The value specified cannot be saved to a file." nil)
        item)))
```

—————

defun spadrread

[dewritify p925]
[rread p915]

— defun spadrread —

```
(defun spadrread (vec stream)
  (|dewritify| (|rread| vec stream nil)))
```

—————

defun Random read a key from a stream

RREAD takes erroval to return if key is missing

[rread p915]
[identp p1237]

[pname p1235]

— defun rread —

```
(defun |rread| (key rstream errorval)
  (when (identp key) (setq key (pname key)))
  (rread key rstream errorval))
```

—————

defun unwritable?

[vecp p??]
[placep p1297]

— defun unwritable? —

```
(defun |unwritable?| (ob)
  (cond
    ((or (consp ob) (simple-vector-p ob)) nil)
    ((or (compiled-function-p ob) (hash-table-p ob)) t)
    ((or (placep ob) (readtablep ob)) t)
    ((floatp ob) t)
    (t nil)))
```

—————

defun writifyComplain

Create a full isomorphic object to be saved in a lisplib. Note that `dewritify(writify(x))` preserves `UEQUALity` of hashtables. `HASHTABLEs` go both ways. `READTABLEs` cannot presently be transformed back. [sayKeyedMsg p43]

[\$writifyComplained p??]

— defun writifyComplain —

```
(defun |writifyComplain| (s)
  (declare (special |$writifyComplained|))
  (unless |$writifyComplained|
    (setq |$writifyComplained| t)
    (|sayKeyedMsg|
     (format nil
              "A value containing a %1 is being saved in a history file or a ~
              compiled input file INLIB. This type is not yet usable in other ~
```

```

    history operations. You might want to issue )history )off")
    (list s)))) ; cannot save value

```

defun safeWritify

```

[writifyTag p??]
[writify p920]

```

— defun safeWritify —

```

(defun |safeWritify| (ob)
  (catch '|writifyTag| (|writify| ob)))

```

defun writify,writifyInner

```

[writifyTag p??]
[seq p??]
[exit p??]
[hget p1234]
[qcar p??]
[qcdr p??]
[spadClosure? p921]
[writify,writifyInner p917]
[hput p1234]
[qrplaca p??]
[qrplacd p??]
[vecp p??]
[isDomainOrPackage p107]
[mkEvalable p1115]
[devaluate p??]
[qvmaxindex p??]
[qsetvelt p??]
[qvelt p??]
[constructor? p??]
[hkeys p1235]
[hashtable-class p??]
[placep p1297]
[boot-equal p??]
[$seen p??]

```

[\[\\$NonNullStream p921\]](#)

[\[\\$NullStream p922\]](#)

— defun writify,writifyInner —

```
(defun |writify,writifyInner| (ob)
  (prog (e name tmp1 tmp2 tmp3 x qcar qcdr d n keys nob)
    (declare (special |$seen| |$NonNullStream| |$NullStream|))
    (return
      (seq
        (when (null ob) (exit nil))
        (when (setq e (hget |$seen| ob)) (exit e))
        (when (consp ob)
          (exit
            (seq
              (setq qcar (qcar ob))
              (setq qcdr (qcdr ob))
              (when (setq name (|spadClosure?| ob))
                (exit
                  (seq
                    (setq d (|writify,writifyInner| (qcdr ob)))
                    (setq nob
                      (cons 'writified!!
                        (cons 'spadclosure
                          (cons d (cons name nil))))))
                    (hput |$seen| ob nob)
                    (hput |$seen| nob nob)
                    (exit nob))))
                (when
                  (and
                    (and (consp ob)
                      (eq (qcar ob) 'lambda-closure)
                      (progn
                        (setq tmp1 (qcdr ob))
                        (and (consp tmp1)
                          (progn
                            (setq tmp2 (qcdr tmp1))
                            (and
                              (consp tmp2)
                              (progn
                                (setq tmp3 (qcdr tmp2))
                                (and (consp tmp3)
                                  (progn
                                    (setq x (qcar tmp3))
                                    t)))))))))) x)
                  (exit
                    (throw '|writifyTag| '|writifyFailed|))))
                (setq nob (cons qcar qcdr))
                (hput |$seen| ob nob)
                (hput |$seen| nob nob))
          )
        )
      )
    )
  )
```

```

    (setq qcar (|writify,writifyInner| qcar))
    (setq qcdr (|writify,writifyInner| qcdr))
    (qrplaca nob qcar)
    (qrplacd nob qcdr)
    (exit nob)))
(when (simple-vector-p ob)
  (exit
   (seq
    (when (|isDomainOrPackage| ob)
      (setq d (|mkEvalable| (|devaluate| ob)))
      (setq nob (list 'writified!! 'devaluated (|writify,writifyInner| d)))
      (hput |$seen| ob nob)
      (hput |$seen| nob nob)
      (exit nob))
      (setq n (qvmaxindex ob))
      (setq nob (make-array (1+ n)))
      (hput |$seen| ob nob)
      (hput |$seen| nob nob)
      (do ((i 0 (≠ i)))
          ((> i n) nil)
        (qsetvelt nob i (|writify,writifyInner| (qvelt ob i))))
      (exit nob))))
  (when (eq ob 'writified!!)
    (exit
     (cons 'writified!! (cons 'self nil))))
  (when (|constructor?| ob)
    (exit ob))
  (when (compiled-function-p ob)
    (exit
     (throw '|writifyTag| '|writifyFailed|)))
  (when (hash-table-p ob)
    (setq nob (cons 'writified!! nil))
    (hput |$seen| ob nob)
    (hput |$seen| nob nob)
    (setq keys (hkeys ob))
    (qrplacd nob
     (cons
      'hashtable
      (cons
       (hashtable-class ob)
       (cons
        (|writify,writifyInner| keys)
        (cons
         (prog (tmp0)
              (setq tmp0 nil)
              (return
               (do ((tmp1 keys (cdr tmp1)) (k nil))
                   ((or (atom tmp1)
                        (progn
                         (setq k (car tmp1))
                         (return k))
                     (return k))))))))))))))

```

```

                                nil))
                                (nreverse0 tmp0))
                                (setq tmp0
                                  (cons (|writify,writifyInner| (hget ob k)) tmp0))))))
                                nil))))))
  (exit nob))
(when (placep ob)
  (setq nob (cons 'writified!! (cons 'place nil)))
  (hput |$seen| ob nob)
  (hput |$seen| nob nob)
  (exit nob))
(when (readtablep ob)
  (exit
   (throw '|writifyTag| '|writifyFailed|)))
(when (stringp ob)
  (exit
   (seq
    (when (eq ob |$NullStream|)
      (exit
       (cons 'writified!! (cons 'nullstream nil))))))
    (when (eq ob |$NonNullStream|)
      (exit
       (cons 'writified!! (cons 'nonnullstream nil))))))
    (exit ob))))))
(when (floatp ob)
  (exit
   (seq
    (when (boot-equal ob (read-from-string (princ-to-string ob)))
      (exit ob))
    (exit
     (cons 'writified!!
      (cons 'float
       (cons ob
        (multiple-value-list (integer-decode-float ob))))))))))
  (exit ob))))))

```

defun writify

```

[ScanOrPairVec p926]
[function p??]
[writify,writifyInner p917]
[$seen p??]
[$writifyComplained p??]

```

— defun writify —

```
(defun |writify| (ob)
  (let (|$seen| |$writifyComplained|)
    (declare (special |$seen| |$writifyComplained|))
    (if (null (|ScanOrPairVec| #'|unwritable?| ob))
        ob
        (progn
          (setq |$seen| (make-hash-table :test #'eq))
          (setq |$writifyComplained| nil)
          (|writify,writifyInner| ob))))))
```

defun spadClosure?

```
[qcar p??]
[bpname p??]
[qcdr p??]
[vecp p??]
```

— defun spadClosure? —

```
(defun |spadClosure?| (ob)
  (let (fun name vec)
    (setq fun (qcar ob))
    (if (null (setq name (bpname fun)))
        nil
        (progn
          (setq vec (qcdr ob))
          (if (null (simple-vector-p vec))
              nil
              name))))))
```

defvar \$NonNullStream

— initvars —

```
(defvar |$NonNullStream| "NonNullStream")
```

defvar \$NullStream

— initvars —

```
(defvar |$NullStream| "NullStream")
```

—————

defun dewritify,dewritifyInner

```
[seq p??]
[exit p??]
[hget p1234]
[intp p??]
[gensymmer p??]
[error p??]
[poundsign p??]
[hput p1234]
[dewritify,dewritifyInner p922]
[concat p1237]
[vmread p1297]
[make-instream p1167]
[qcar p??]
[qcdr p??]
[qrplaca p??]
[qrplacd p??]
[vecp p??]
[qvmaxindex p??]
[qsetvelt p??]
[qvelt p??]
[$seen p??]
[$NullStream p922]
[$NonNullStream p921]
```

— defun dewritify,dewritifyInner —

```
(defun |dewritify,dewritifyInner| (ob)
  (prog (e type oname f vec name tmp1 signif expon sign fval qcar qcdr n nob)
    (declare (special |$seen| |$NullStream| |$NonNullStream|))
    (return
      (seq
        (when (null ob)
          (exit nil))
        (when (setq e (hget |$seen| ob))
```

```

(exit e))
(when (and (consp ob) (eq (car ob) 'writified!!))
  (exit
    (seq
      (setq type (elt ob 1))
      (when (eq type 'self)
        (exit 'writified!!))
      (when (eq type 'bpi)
        (exit
          (seq
            (setq oname (elt ob 2))
            (setq f
              (seq
                (when (integerp oname) (exit (eval (gensymmer oname))))
                (exit (symbol-function oname))))
            (when (null (compiled-function-p f))
              (exit (|error| "A required BPI does not exist.")))
            (when (and (> (|#| ob) 3) (not (equal (sxhash f) (elt ob 3))))
              (exit (|error| "A required BPI has been redefined.")))
            (hput |$seen| ob f)
            (exit f))))))
      (when (eq type 'hashtable)
        (exit
          (seq
            (setq nob (make-hash-table :test #'equal))
            (hput |$seen| ob nob)
            (hput |$seen| nob nob)
            (do ((tmp0 (elt ob 3) (cdr tmp0))
                (k nil)
                (tmp1 (elt ob 4) (cdr tmp1))
                (e nil))
              ((or (atom tmp0)
                  (progn
                     (setq k (car tmp0))
                     nil)
                  (atom tmp1)
                  (progn
                     (setq e (car tmp1))
                     nil))
                 nil)
              (seq
                (exit
                  (hput nob (|dewritify,dewritifyInner| k)
                    (|dewritify,dewritifyInner| e))))
                (exit nob))))))
            (when (eq type 'devaluated)
              (exit
                (seq
                  (setq nob (eval (|dewritify,dewritifyInner| (elt ob 2))))
                  (hput |$seen| ob nob)

```

```

        (hput |$seen| nob nob)
        (exit nob)))
(when (eq type 'spadclosure)
  (exit
    (seq
      (setq vec (|dewritify,dewritifyInner| (elt ob 2)))
      (setq name (ELT ob 3))
      (when (null (fboundp name))
        (exit
          (|error|
            (concat "undefined function: " (symbol-name name))))))
      (setq nob (cons (symbol-function name) vec))
      (hput |$seen| ob nob)
      (hput |$seen| nob nob)
      (exit nob))))
(when (eq type 'place)
  (exit
    (seq
      (setq nob (vmread (make-instream nil)))
      (hput |$seen| ob nob)
      (hput |$seen| nob nob)
      (exit nob))))
(when (eq type 'readtable)
  (exit (|error| "Cannot de-writify a read table.")))
(when (eq type 'nullstream)
  (exit |$NullStream|))
(when (eq type 'nonnullstream)
  (exit |$NonnullStream|))
(when (eq type 'float)
  (exit
    (seq
      (progn
        (setq tmp1 (cddr ob))
        (setq fval (car tmp1))
        (setq signif (cadr tmp1))
        (setq expon (caddr tmp1))
        (setq sign (caddr tmp1))
        tmp1)
      (setq fval (scale-float (float signif fval) expon))
      (when (minusp sign)
        (exit (- fval)))
      (exit fval))))
  (exit (|error| "Unknown type to de-writify."))))
(when (consp ob)
  (exit
    (seq
      (setq qcar (qcar ob))
      (setq qcdr (qcdr ob))
      (setq nob (cons qcar qcdr))
      (hput |$seen| ob nob)

```

```

(hput |$seen| nob nob)
(qrplaca nob (|dewritify,dewritifyInner| qcar))
(qrplacd nob (|dewritify,dewritifyInner| qcdr))
(exit nob)))
(when (simple-vector-p ob)
  (exit
    (seq
      (setq n (qvmaxindex ob))
      (setq nob (make-array (1+ n)))
      (hput |$seen| ob nob)
      (hput |$seen| nob nob)
      (do ((i 0 (1+ i)))
          ((> i n) nil)
        (seq
          (exit
            (qsetvelt nob i
              (|dewritify,dewritifyInner| (qvelt ob i))))))
      (exit nob))))
  (exit ob))))

```

defun dewritify

[ScanOrPairVec p926]
 [function p??]
 [dewritify,dewritifyInner p922]
 [\$seen p??]

— defun dewritify —

```

(defun |dewritify| (ob)
  (let (|$seen|)
    (declare (special |$seen|))
    (if (null (|ScanOrPairVec| #'(lambda (a) (eq a 'writified!)) ob))
        ob
        (progn
          (setq |$seen| (make-hash-table :test #'eq))
          (|dewritify,dewritifyInner| ob))))))

```

defun ScanOrPairVec,ScanOrInner

[ScanOrPairVecAnswer p??]
 [hget p1234]

```
[hput p1234]
[ScanOrPairVec,ScanOrInner p925]
[qcar p??]
[qcdr p??]
[vecp p??]
[$seen p??]
```

— defun ScanOrPairVec,ScanOrInner —

```
(defun |ScanOrPairVec,ScanOrInner| (f ob)
  (declare (special |$seen|))
  (when (hget |$seen| ob) nil)
  (when (consp ob)
    (hput |$seen| ob t)
    (|ScanOrPairVec,ScanOrInner| f (qcar ob))
    (|ScanOrPairVec,ScanOrInner| f (qcdr ob)))
  (when (simple-vector-p ob)
    (hput |$seen| ob t)
    (do ((tmp0 (- (|#| ob) 1)) (i 0 (1+ i)))
        ((> i tmp0) nil)
        (|ScanOrPairVec,ScanOrInner| f (elt ob i))))
  (when (funcall f ob) (throw '|ScanOrPairVecAnswer| t))
  nil)
```

defun ScanOrPairVec

```
[ScanOrPairVecAnswer p??]
[ScanOrPairVec,ScanOrInner p925]
[$seen p??]
```

— defun ScanOrPairVec —

```
(defun |ScanOrPairVec| (f ob)
  (let (|$seen|)
    (declare (special |$seen|))
    (setq |$seen| (make-hash-table :test #'eq))
    (catch '|ScanOrPairVecAnswer| (|ScanOrPairVec,ScanOrInner| f ob))))
```

defun gensymInt

```
[gensymp p??]
[error p??]
[pname p1235]
[charDigitVal p927]
```

— defun gensymInt —

```
(defun |gensymInt| (g)
  (let (p n)
    (if (null (gensymp g))
        (|error| "Need a GENSYM")
        (progn
          (setq p (pname g))
          (setq n 0)
          (do ((tmp0 (- (|#| p) 1)) (i 2 (1+ i)))
              ((> i tmp0) nil)
              (setq n (+ (* 10 n) (|charDigitVal| (elt p i))))
              n))))))
```

defun charDigitVal

```
[error p??]
```

— defun charDigitVal —

```
(defun |charDigitVal| (c)
  (let (digits n)
    (setq digits "0123456789")
    (setq n (- 1))
    (do ((tmp0 (- (|#| digits) 1)) (i 0 (1+ i)))
        ((or (> i tmp0) (null (minusp n))) nil)
        (if (char= c (elt digits i))
            (setq n i)
            nil))
    (if (minusp n)
        (|error| "Character is not a digit")
        n)))
```

defun histFileErase

— defun histFileErase —

```
(defun |histFileErase| (file)
  (when (probe-file file) (delete-file file)))
```

—————

26.24 `)include` Command

`include` man page

— `include.help` —

User Level Required: `interpreter`

Command Syntax:

```
)include filename
```

Command Description:

The `)include` command can be used in `.input` files to place the contents of another file inline with the current file. The path can be an absolute or relative pathname.

`defun ncloopInclude1`

[`ncloopIncFileName` p929]

[`ncloopInclude` p930]

— `defun ncloopInclude1` —

```
(defun |ncloopInclude1| (name n)
  (let (a)
    (if (setq a (|ncloopIncFileName| name))
        (|ncloopInclude| a n)
        n)))
```

Returns the first non-blank substring of the given string

[`incFileName` p930]

[`concat` p1237]

— `defun ncloopIncFileName` —

```
(defun |ncloopIncFileName| (string)
```



```
"Returns the first non-blank substring of the given string"
(let (fn)
  (unless (setq fn (|incFileName| string))
    (write-line (concat string " not found")))
  fn))
```

Open the include file and read it in

The `ncloopInclude0` function is part of the parser and lives in `int-top.boot`.

[`ncloopInclude0` p365]

— **defun ncloopInclude** —

```
(defun |ncloopInclude| (name n)
  "Open the include file and read it in"
  (with-open-file (st name) (|ncloopInclude0| st name n)))
```

Return the include filename

Given a string we return the first token from the string which is the first non-blank substring.

[`incBiteOff` p930]

— **defun incFileName** —

```
(defun |incFileName| (x)
  "Return the include filename"
  (car (|incBiteOff| x)))
```

Return the next token

Takes a sequence and returns the a list of the first token and the remaining string characters. If there are no remaining string characters the second string is of length 0. Effectively it "bites off" the first token in the string. If the string only 0 or more blanks it returns nil.

— **defun incBiteOff** —

```
(defun |incBiteOff| (x)
```

```
"Return the next token"
(let (blank nonblank)
  (setq x (string x))
  (when (setq nonblank (position #\space x :test-not #'char=))
    (setq blank (position #\space x :start nonblank))
    (if blank
      (list (subseq x nonblank blank) (subseq x blank))
      (list (subseq x nonblank) ""))))
```

26.25)library Command

library man page

— library.help —

```
=====
A.14. )library
=====
```

User Level Required: interpreter

Command Syntax:

-)library libName1 [libName2 ...]
-)library)dir dirName
-)library)only objName1 [objlib2 ...]
-)library)noexpose

Command Description:

This command replaces the)load system command that was available in AXIOM releases before version 2.0. The)library command makes available to AXIOM the compiled objects in the libraries listed.

For example, if you)compile dopler.spad in your home directory, issue)library dopler to have AXIOM look at the library, determine the category and domain constructors present, update the internal database with various properties of the constructors, and arrange for the constructors to be automatically loaded when needed. If the)noexpose option has not been given, the constructors will be exposed (that is, available) in the current frame.

If you compiled a file you will have an NRLIB present, for example, DOPLER.NRLIB, where DOPLER is a constructor abbreviation. The command)library DOPLER will then do the analysis and database updates as above.

To tell the system about all libraries in a directory, use)library)dir dirName where dirName is an explicit directory. You may specify '.' as the directory, which means the current directory from which you started the system or the one you set via the)cd command. The directory name is required.

You may only want to tell the system about particular constructors within a library. In this case, use the)only option. The command)library dopler)only Test1 will only cause the Test1 constructor to be analyzed, autoloading, etc..

Finally, each constructor in a library are usually automatically exposed when the)library command is used. Use the)noexpose option if you not want them

exposed. At a later time you can use `)set expose add constructor` to expose any hidden constructors.

Note for AXIOM beta testers: At various times this command was called `)local` and `)with` before the name `)library` became the official name.

Also See:

- o `)cd`
- o `)compile`
- o `)frame`
- o `)set`

¹⁰ “`cd`” (26.11 p 832) “`frame`” (3.5 p 25) “`set`” (26.51 p 1081)

26.26)license Command

license man page

— license.help —

```
=====
A.15. )license
=====
```

Command Syntax:

```
- )license
```

Command Description:

This command displays the Axiom license.

Also See:

```
o )trademark
```

defun license

```
[obey p??]
[concat p1237]
[getenvirom p322]
```

— defun license —

```
(defun |license| (l)
  (declare (ignore l))
  (obey (concat "cat " (getenvirom "AXIOM") "/doc/spadhelp/copyright.help")))
```

26.27 `)lisp` Command

`lisp` man page

— `lisp.help` —

```
=====
A.15.  )lisp
=====
```

User Level Required: `development`

Command Syntax:

```
-  )lisp [lispExpression]
```

Command Description:

This command is used by AXIOM system developers to have single expressions evaluated by the Lisp system on which AXIOM is built. The `lispExpression` is read by the Lisp reader and evaluated. If this expression is not complete (unbalanced parentheses, say), the reader will wait until a complete expression is entered.

Since this command is only useful for evaluating single expressions, the `)fin` command may be used to drop out of AXIOM into Lisp.

Also See:

- o `)system`
- o `)boot`
- o `)fin`

[11](#)

This command is in the list of `$noParseCommands` [26.1](#) which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` [26.2](#)

¹¹ “system” ([26.56](#) p 1108) “boot” ([26.4](#) p 826) “fin” ([26.20](#) p 876)

26.28)ltrace Command

ltrace man page

— ltrace.help —

```
=====
A.17. )ltrace
=====
```

User Level Required: development

Command Syntax:

This command has the same arguments as options as the)trace command.

Command Description:

This command is used by AXIOM system developers to trace Lisp functions.
It is not supported for general use.

Also See:

- o)lisp
- o)trace

[12](#)

defun The top level)ltrace function

[trace p[78](#)]

— defun ltrace —

```
(defun |ltrace| (arg) (|trace| arg))
```

¹² “lisp” ([26.27 p 935](#)) “trace” ([7.4 p 78](#))

26.29)pquit Command

pquit man page

— pquit.help —

```
=====
A.18. )pquit
=====
```

User Level Required: interpreter

Command Syntax:

-)pquit

Command Description:

This command is used to terminate AXIOM and return to the operating system. Other than by redoing all your computations or by using the)history)restore command to try to restore your working environment, you cannot return to AXIOM in the same state.

)pquit differs from the)quit in that it always asks for confirmation that you want to terminate AXIOM (the ‘p’ is for ‘protected’). When you enter the)pquit command, AXIOM responds

Please enter y or yes if you really want to leave the interactive
environment and return to the operating system:

If you respond with y or yes, you will see the message

You are now leaving the AXIOM interactive environment.
Issue the command axiom to the operating system to start a new session.

and AXIOM will terminate and return you to the operating system (or the environment from which you invoked the system). If you responded with something other than y or yes, then the message

You have chosen to remain in the AXIOM interactive environment.

will be displayed and, indeed, AXIOM would still be running.

Also See:

- o)fin
- o)history
- o)close
- o)quit


```
o )system
```

13

The top level pquit command

[pquitSpad2Cmd p938]

— defun pquit —

```
(defun |pquit| ()
  "The top level pquit command"
  (|pquitSpad2Cmd|))
```

The top level pquit command handler

[quitSpad2Cmd p940]

[\$quitCommandType p1073]

— defun pquitSpad2Cmd —

```
(defun |pquitSpad2Cmd| ()
  "The top level pquit command handler"
  (let ((|$quitCommandType| ' |protected|))
    (declare (special |$quitCommandType|))
    (|quitSpad2Cmd|)))
```

This command is in the list of `$noParseCommands` 26.1 which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` 26.2

¹³ “fin” (26.20 p 876) “history” (26.23 p 889) “close” (26.13 p 845) “quit” (26.30 p 940) “system” (26.56 p 1108)

26.30)quit Command

quit man page

— quit.help —

```
=====
A.19. )quit
=====
```

User Level Required: interpreter

Command Syntax:

-)quit
-)set quit protected | unprotected

Command Description:

This command is used to terminate AXIOM and return to the operating system. Other than by redoing all your computations or by using the)history)restore command to try to restore your working environment, you cannot return to AXIOM in the same state.

)quit differs from the)pquit in that it asks for confirmation only if the command

)set quit protected

has been issued. Otherwise,)quit will make AXIOM terminate and return you to the operating system (or the environment from which you invoked the system).

The default setting is)set quit protected so that)quit and)pquit behave in the same way. If you do issue

)set quit unprotected

we suggest that you do not (somehow) assign)quit to be executed when you press, say, a function key.

Also See:

- o)fin
- o)history
- o)close
- o)pquit
- o)system

14

The top level quit command

[quitSpad2Cmd p940]

— defun quit —

```
(defun |quit| ()
  "The top level quit command"
  (|quitSpad2Cmd|))
```

The top level quit command handler

[upcase p1272]
 [queryUserKeyedMsg p??]
 [string2id-n p??]
 [leaveScratchpad p941]
 [sayKeyedMsg p43]
 [tersyscommand p792]
 [\$quitCommandType p1073]

— defun quitSpad2Cmd —

```
(defun |quitSpad2Cmd| ()
  "The top level quit command handler"
  (declare (special |$quitCommandType|))
  (if (eq |$quitCommandType| '|protected|)
    (let (x)
      (setq x
        (upcase
          (|queryUserKeyedMsg|
            (format nil
              "Please enter y or yes if you really want to leave the interactive ~
              environment and return to the operating system:")
            nil)))
      (when (member (string2id-n x 1) '(y yes)) (|leaveScratchpad|))
      (|sayKeyedMsg|
        "You have chosen to remain in the Axiom interactive environment." nil)
      (tersyscommand)))
```

¹⁴ “fin” (26.20 p 876) “history” (26.23 p 889) “close” (26.13 p 845) “pquit” (26.29 p 938) “system” (26.56 p 1108)

```
(|leaveScratchpad|))
```

Leave the Axiom interpreter

— `defun leaveScratchpad` —

```
(defun |leaveScratchpad| ()  
  "Leave the Axiom interpreter"  
  (bye))
```

This command is in the list of `$noParseCommands` [26.1](#) which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` [26.2](#)

26.31)read Command

read man page

— read.help —

```
=====
A.20. )read
=====
```

User Level Required: interpreter

Command Syntax:

-)read [fileName]
-)read [fileName] []quiet] []ifthere]

Command Description:

This command is used to read .input files into AXIOM. The command

```
)read matrix.input
```

will read the contents of the file matrix.input into AXIOM. The ‘.input’ file extension is optional. See the AXIOM User Guide index for more information about .input files.

This command remembers the previous file you edited, read or compiled. If you do not specify a file name, the previous file will be read.

The)ifthere option checks to see whether the .input file exists. If it does not, the)read command does nothing. If you do not use this option and the file does not exist, you are asked to give the name of an existing .input file.

The)quiet option suppresses output while the file is being read.

Also See:

- o)compile
- o)edit
- o)history

defun The)read command

[readSpad2Cmd p943]

— defun read —

```
(defun |read| (arg) (|readSpad2Cmd| arg))
```

—————

defun Implement the)read command

```
[selectOptionLC p819]
[optionError p789]
[pathname p1232]
[pathnameTypeId p1231]
[makePathname p1233]
[pathnameName p1231]
[mergePathnames p1231]
[findfile p??]
[throwKeyedMsg p??]
[namestring p1230]
[upcase p1272]
[member p1238]
[/read p944]
[$InteractiveMode p315]
[$findfile p??]
[$UserLevel p1080]
[$options p73]
[/editfile p849]
```

— defun readSpad2Cmd —

```
(defun |readSpad2Cmd| (arg)
  (prog (|$InteractiveMode| fullopt ifthere quiet ef devFTs fileTypes
        ll ft upft fs)
    (declare (special |$InteractiveMode| $findfile |$UserLevel| |$options|
                      /editfile))
    (setq |$InteractiveMode| t)
    (dolist (opt |$options|)
      (setq fullopt
        (|selectOptionLC| (car opt) '(|quiet| |test| |ifthere|) '|optionError|))
      (cond
        ((eq fullopt '|ifthere|) (setq ifthere t))
        ((eq fullopt '|quiet|) (setq quiet t))))
    (setq ef (|pathname| /editfile))
```

```

(when (eq (|pathnameTypeId| ef) 'spad)
  (setq ef (|makePathname| (|pathnameName| ef) "*" "*")))
(if arg
  (setq arg (|mergePathnames| (|pathname| arg) ef))
  (setq arg ef))
(setq devFTs '("input" "INPUT" "boot" "BOOT" "lisp" "LISP"))
(setq fileTypes
  (cond
    ((eq |$UserLevel| '|interpreter|) '("input" "INPUT"))
    ((eq |$UserLevel| '|compiler|) '("input" "INPUT"))
    (t devFTs)))
(setq ll ($findfile arg fileTypes))
(unless ll
  (if ifthere
    (return nil)
    (|throwKeyedMsg| "The file %1 is needed but does not exist."
      (list (|namestring| arg)))))
(setq ll (|pathname| ll))
(setq ft (|pathnameType| ll))
(setq upft (upcase ft))
(cond
  ((null (|member| upft fileTypes))
    (setq fs (|namestring| arg))
    (if (|member| upft devFTs)
      (|throwKeyedMsg|
        (format nil
          "You cannot )read the file %1 because your user-level is not is ~
          not high enough. For more information about your user-level, issue ~
          )set userlevel.")
        (list fs))
      (|throwKeyedMsg|
        (format nil
          "You cannot )read the file %1 because it is not suitable for ~
          reading by Axiom. Note that files with file extension .spad ~
          can now only be compiled with the )compile system command.")
        (list fs))))
  (t
    (setq /editfile ll)
    (when (string= upft "BOOT") (setq |$InteractiveMode| nil))
    (/read ll quiet))))

```

defun /read

```

[/read /rf (vol9)]
[/read /rq (vol9)]
[/editfile p849]

```

— defun /read —

```
(defun /read (l q)
  (declare (special /editfile))
  (setq /editfile l)
  (cond
    (q (/rq))
    (t (/rf)) )
  (flag |boot-NewKEY| 'key)
  (|terminateSystemCommand|)
  (|spadPrompt|))
```

—————

26.32)regress Command

regress man page

— regress.help —

```
=====
A.18. )regress
=====
```

User Level Required: interpreter

Command Syntax:

```
- )regress fileName
```

Command Description:

The regress command will run the regress function that was compiled as part of the lisp image build process. This function expects an input filename, possibly containing a path prefix.

If the filename contains a period then we consider it a fully formed filename, otherwise we append ‘.output’, which is the default file extension.

```
)regress matrix
)regress matrix.output
)regress /path/to/file/matrix
)regress /path/to/file/matrix.output
```

will test the contents of the file matrix.output.

The idea behind regression testing is to check that the results we currently get match the results we used to get. In order to do that we create input files with a special comment format that contains the prior results. These are easy to create as all you need to do is run the Axiom function, capture the results, and turn them input specially formed comments using the -- comment.

A regression file caches the result of an Axiom function so we can automate the testing process. It is a file of many tests, each with their own output.

The regression file format uses the Axiom -- comment syntax to keep a copy of the expected output from an Axiom command. This expected output is compared character by character against the actual output.

The regression file is broken into numbered blocks, delimited by a --S for the beginning and a --E for the end. The total number of blocks is also given so missing or failed tests also raise an error.

There are 4 special kinds of -- comments in regression files:

```
--S n of M      this is test n of M tests in this file
--E n           this marks the end of test n
--R any output  this marks the actual expected output line
--I any output  this line is compared but ignored
```

A regression test file looks like:

```
)set break resume
)spool foo.output
)set message type off
)clear all

--S 1 of 3
2+3
--R                               this is the exact Axiom output
--R  (1)  5
--E 1

--S 2 of 3
2+3
--R                               this should fail to match
--R  (2)  7
--E 2

--S 3 of 3
2+3
--R                               this fails to match but we
--I  (3)  7                               use --I to ignore this line
--E 3
```

We can now run this file with

```
)read foo.input
```

Note that when this file is run it will create a spool file called "foo.output" because of the lines:

```
)spool foo.output
)spool
```

The "foo.output" file contains the console image of the result. It will look like:

```
Starts dribbling to foo.output (2012/2/28, 12:25:7).
```

```

)set message type off
)clear all

--S 1 of 3
2+3

(1) 5
--R
--R (1) 5
--E 1

--S 2 of 3
2+3

(2) 5
--R
--R (2) 7
--E 2

--S 3 of 3
2+3

(3) 5
--R
--I (3) 7
--E 3

)spool

```

This "foo.output" file can now be checked using the `)regress` command.

When we run the `)regress foo.output` we see;

```

testing foo
passed foo 1 of 3
MISMATCH
expected:" (2) 7"
got:" (2) 5"
FAILED foo 2 of 2
passed foo 3 of 3
regression result FAILED 1 of 3 stanzas file foo

```

Tests either pass or fail. A passing test generates the message:

```
passed foo 1 of 3
```

A failing test will give a reversed printout of the expected vs actual output as well as a FAILED message, as in:

```
MISMATCH
```

```

expected:"    (2)  7"
got:"      (2)  5"
FAILED foo  2 of 3

```

The last line of output is a summary:

```

regression result FAILED 1 of 3 stanzas file foo

```

— defun regress command —

```

(defun |regress| (arg)
  (let (|$InteractiveMode| namestring dot1 outfile (extension "output"))
    (declare (special |$InteractiveMode|))
    (setq |$InteractiveMode| t)
    (setq namestring (symbol-name (car arg)))
    (setq dot1 (position #\. namestring))
    (unless dot1
      (setq outfile (concatenate 'string (subseq namestring 0) "." extension)))
    (if (probe-file outfile)
        (regress outfile)
        (format t (concatenate 'string outfile "% file not found")))))

```

The regress function details

This is the regression test mechanism. The input files have been rewritten to have a standard structure. This fixed format identifies the tests within a file. Each test is run and any mismatch between the actual and expected results is reported.

In order to regression test axiom results we created a standard file format. This format has 3 kinds of markers:

- “-S” marker which must have a integer test number
- “-R” marker lines, one per expected output from axiom
- “-E” marker which has an integer matching the preceeding “-S”
- “-I” marker ignores the line, useful for gensyms and random

Because these markers use Axiom’s standard comment prefix they are valid lines in input files and are ignored by the “)read” command. They are simply copied to the output file.

This allows us to include the expected output in the output file so we can compare what Axiom computes with what it should compute.

To create these regression files all you need to do is create an input file and run it through Axiom. Then, for each test case in the file you mark it up by putting a “-S number” **before** the Axiom input line. You put “-R” prefixes on each line of Axiom output, including the blank lines. Then you put a “-E number” line after the last output line, usually the **Type:** line. This newly marked-up input file is now a regression test.

To actually run the regression test you simply include the marked up the input file in the **src/input** subdirectory. This file will automatically be run at build time and any failing tests will be marked. This code will ignore any input that does not contain proper regression markups.

Ideally the regression test files should be pamphlet files that explain the content and purpose of each regression test case.

Thus you run the marked-up input file **foo.input** and spool the result to **foo.output** and then run the lisp function

```
(regress ‘‘foo.output’’)
```

If the file does not contain proper regression markups it is ignored. Comments or any other commands in the file that are not surrounded by “-S” and “-E” boundaries are ignored.

defvar *all-tests-ran*

This variable is used to check whether all of the tests actually ran. This is needed to see if the execution ended early.

— **initvars** —

```
(defvar *all-tests-ran* nil "true implies that all tests ran")
```

—————

defun Scan a spool output file for failures

This function takes an output file which has been created by the Axiom **)spool** command and looks for regression test markups. Each regression test is checked against the actual result and any failures are marked.

```
[getspoolname p951]
[findnexttest p952]
[testpassed p953]
[*all-tests-ran* p950]
```

— **defun regress** —

```
(defun regress (infile)
```

```

(let (name comment test (count 0) (passed 0) (failed 0))
(declare (special *all-tests-ran*))
(setq *all-tests-ran* nil)
(with-open-file (stream infile :direction :input)
  (setq name (getspoolname stream))
  (when name
    (format t "testing ~a~%" name)
    (loop
      (setq *ok* nil)
      (multiple-value-setq (comment test) (findnexttest stream))
      (unless comment (return))
      (setq count (+ count 1))
      (if (testpassed test)
        (progn
          (setq passed (+ passed 1))
          (format t "passed ~a ~a~%" name comment))
        (progn
          (setq failed (+ failed 1))
          (format t "FAILED ~a ~a~%" name comment))))))
  (if (= failed 0)
    (format t "regression result passed ~a of ~a stanzas ~Tfile ~a~%"
      passed count name)
    (format t "regression result FAILED ~a of ~a stanzas ~Tfile ~a~%"
      failed count name))
  (unless *all-tests-ran*
    (format t "regression result FAILED early exit in file ~a?~%" name))))))

```

defun Parse test name from the spool command

We need to parse out the name of the test. The “)spool” command writes a line into the output file containing the name of the test. We parse out the name of the test from this line.

— defun getspoolname 0 —

```

(defun getspoolname (stream)
  (let (line point)
    (setq line (read-line stream))
    (setq point (position #\. line))
    (if (or (null point)
      (< (length line) 30)
      (not (string= (subseq line (+ point 1) (+ point 7)) "output"))))
      nil
      (subseq line 20 point))))

```

defun Find the next -S marker

We need to break the file into separate test cases. This routine looks for the “-S” line which indicates a test is starting. It collects up input lines until it encounters the “-E” line marking the end of the test case. These lines are returned as a list of strings.

[testnumberp p952]

— **defun findnexttest** —

```
(defun findnexttest (stream)
  (let (teststart result)
    (do ((line (read-line stream nil 'done) (read-line stream nil 'done)))
        ((or (eq line 'done) (endedp line))
         (values (if line teststart) result)))
    (if teststart
        (push line result)
        (setq teststart (testnumberp line))))))
```

—————

defun Parse out the test number from -S lines

The “-S” line has a test number on the line. We parse out the test number for printing.
[startp p955]

— **defun testnumberp** —

```
(defun testnumberp (oneline)
  (when (startp oneline) (subseq oneline 3)))
```

—————

defvar *ok*

We can mark a test as always ok by putting the word “ok” anywhere on the start line. The regress function resets this value. The startp function checks the -S line for the word “ok”. If found, it sets this value to true which causes a failing test to be considered as passed.

— **initvars** —

```
(defvar *ok* nil "did we mark this test as always ok?")
```

—————

defun Compare the computed and expected results

This routine takes the test input, passes it to `split` to clean up and break into two lists, and then compares the resulting lists element by element, complaining about any mismatches. The result is either true if everything passes or false if a mismatch occurs.

A test line can also be considered at passing if the expected line is the string “ignore”.

The `ok` variable allows us to mark failing tests as “ok” because we expect the test might fail due to random values or testing known bugs against expected output. We filter these tests marked “ok” so they do not count as “real” failures.

[split p953]
[*ok* p952]

— defun testpassed —

```
(defun testpassed (test)
  (let (answer expected (passed t) mismatchedLines)
    (declare (special *ok*))
    (multiple-value-setq (answer expected) (split test))
    (dotimes (i (length answer))
      (unless
        (or (string= (first expected) "ignore")
            (string= (first expected) (first answer)))
        (unless *ok* (setq passed nil))
        (push (cons (first expected) (first answer)) mismatchedLines))
      (pop answer)
      (pop expected))
    (when mismatchedLines
      (dolist (pair mismatchedLines)
        (format t "expected:~s~%      got:~s~%" (car pair) (cdr pair))))
    passed))
```

—————

defun Split the calculated and expect results into lists

We have a list containing all of the lines in a test. The input is of the form:

```
("--R                                                    Type: List Integer"
 "--R   (1)  [1,4,2,- 6,0,3,5,4,2,3]"
 "--R"
 "--R "
 "                                                    Type: List Integer"
 "   (1)  [1,4,2,- 6,0,3,5,4,2,3]"
 ""
 " "
 "1 := [1,4,2,-6,0,3,5,4,2,3]")
```


It removes the “-R” prefix from the result strings and generates two hopefully equal-length lists, thus:

```
(  
  "                                Type: List Integer"  
  "  (1)  [1,4,2,- 6,0,3,5,4,2,3]"  
  ""  
  " ")  
(  
  "                                Type: List Integer"  
  "  (1)  [1,4,2,- 6,0,3,5,4,2,3]"  
  ""  
  " "))
```

Thus the first line is the start line, the second line is the Axiom input line, followed by the Axiom output. Then we have the lines marked “-R” which are the expected result. We split these into two separate lists and throw away the lines that are the start and end lines.

Once we have classified all of the lines we need to throw away the input lines. By assumption there will be more answer lines than expected lines because the input lines are included. And given the way we process the file these input lines are on the top of the answer stack. Since the number of answer lines should equal the number of expected lines we pop the stack until the numbers are equal.

Each element of the answer list should be `string=` to the corresponding element of the result list.

If the input line starts with “-I” we push the string “ignore”. This is useful for handling random results or gensym symbols.

```
[startp p955]  
[endedp p955]  
[ignorep p956]  
[resultp p956]
```

— defun split —

```
(defun split (test)  
  (let (answer (acnt 0) expected (ecnt 0))  
    (dolist (oneline test)  
      (cond  
        ((startp oneline))  
        ((endedp oneline))  
        ((ignorep oneline)  
         (setq ecnt (+ ecnt 1))  
         (push "ignore" expected))  
        ((resultp oneline)  
         (setq ecnt (+ ecnt 1))  
         (push (subseq oneline 3) expected))  
        (t  
         (setq acnt (+ acnt 1))  
         (push oneline answer))))  
    (dotimes (i (- acnt ecnt)) (pop answer))
```

```
(values (nreverse answer) (nreverse expected))))
```

defun Returns true on -S lines

This test returns true if we have a “start” line. That is, a line with a “-S” prefix.

The `*all-tests-ran*` variable is true if the start line is of the form “-S N of M” and $N=M$, that is, it checks that all tests were performed since this should only occur on the last start line. This will detect “premature exit” in processing.

If a test is failing because of random input values or we want the test to fail but not to count toward failing values then put the string “ok” somewhere on the “-S” line as in:

```
--S 29 of 42 fails due to random values but that is ok
```

```
[lastcount p956]
```

```
[*ok* p952]
```

— defun startp —

```
(defun startp (oneline)
  (let (result)
    (declare (special *ok*))
    (when
      (setq result
        (and (>= (length oneline) 3) (string= (subseq oneline 0 3) "--S")))
      (setq *ok* (search "ok" oneline))
      (setq *all-tests-ran* (lastcount oneline)))
    result))
```

defun Returns true on -E lines

This test returns true if we have a “ended” line. That is, a line with a “-E” prefix.

— defun endedp 0 —

```
(defun endedp (oneline)
  (and (>= (length oneline) 3) (string= (subseq oneline 0 3) "--E")))
```

defun Returns true on -R lines

This test returns true if we have a “results” line. That is, a line with a “-R” prefix.

— defun resultp 0 —

```
(defun resultp (oneline)
  (and (>= (length oneline) 3) (string= (subseq oneline 0 3) "--R")))
```

—————

defun Returns true on -I lines

This test returns true if we have an “ignore” line. That is, a line with a “-I” prefix.

— defun ignorep 0 —

```
(defun ignorep (oneline)
  (and (>= (length oneline) 3) (string= (subseq oneline 0 3) "--I")))
```

—————

defun Check the last -S line ran

If the “-S” line has the format “-S n of m” we return true if n=m, false otherwise. Thus,

```
--S          => nil
--S 1 of 4"   => nil
--S 10 of 40" => nil
--S 4 of 4"   => t
--S 40 of 40" => t
--S 1 of a"   => nil
```

This is used as a final end check to make sure that all of the tests actually ran rather than having the regression test exit early and quietly. This will be false on all but the last test and will be false if the “-S” line does not contain the optional count marker. It is not required but is highly recommended.

— defun lastcount 0 —

```
(defun lastcount (oneline)
  (let ((n :done) (m :done) next somemore isof)
    (when (and (>= (length oneline) 3) (string= (subseq oneline 0 3) "--S"))
      (setq somemore (string-trim " " (subseq oneline 3)))
      (when somemore
        (multiple-value-setq (n next) (read-from-string somemore nil :done))
```

```
(when (integerp n)
  (setq somemore (string-trim " " (subseq somemore next)))
  (multiple-value-setq (isof next) (read-from-string somemore nil :done))
  (when (string= isof "0F")
    (setq somemore (string-trim " " (subseq somemore next)))
    (multiple-value-setq (m next) (read-from-string somemore nil :done))))))
(and (integerp m) (integerp n) (= m n)))
```

26.33)savesystem Command

savesystem man page

— savesystem.help —

```
=====
A.8. )savesystem
=====
```

User Level Required: interpreter

Command Syntax:

```
- )savesystem filename
```

Command Description:

This command is used to save an AXIOM image to disk. This creates an executable file which, when started, has everything loaded into it that was there when the image was saved. Thus, after executing commands which cause the loading of some packages, the command:

```
)savesystem /tmp/savesys
```

will create an image that can be restarted with the UNIX command:

```
axiom -ws /tmp/savesys
```

This new system will not need to reload the packages and domains that were already loaded when the system was saved.

There is currently a restriction that only systems started with the command "AXIOMsys" may be saved.

```
axiom
(1) -> t1:=4
(1) -> )savesystem foo
```

and Axiom exits. Then do

```
./foo
(1) -> t1
4
```

—————

defvar *ThisIsARunningSystem*

When a user does

```
)savesystem foo
```

we set this variable to true. This is tested in the restart function, which is called when the system starts, to prevent losing user information.

— **initvars** —

```
(defvar *ThisIsARunningSystem* nil "Are we restarting a running system?")
```

defun The)savesystem command

[helpSpad2Cmd p881]

[spad-save p1175]

— **defun savesystem** —

```
(defun |savesystem| (arg)
  (if (or (not (eql (|#| arg) 1)) (null (symbolp (car arg))))
      (|helpSpad2Cmd| '(|savesystem|))
      (progn
        (setq *ThisIsARunningSystem* t)
        (spad-save (symbol-name (car arg))))))
```

26.34)set Command

set man page

— set.help —

```
=====
A.21. )set
=====
```

User Level Required: interpreter

Command Syntax:

-)set
-)set label1 [... labelN]
-)set label1 [... labelN] newValue

Command Description:

The)set command is used to view or set system variables that control what messages are displayed, the type of output desired, the status of the history facility, the way AXIOM user functions are cached, and so on. Since this collection is very large, we will not discuss them here. Rather, we will show how the facility is used. We urge you to explore the)set options to familiarize yourself with how you can modify your AXIOM working environment. There is a HyperDoc version of this same facility available from the main HyperDoc menu. Click [\[here\]](#) to go to it.

The)set command is command-driven with a menu display. It is tree-structured. To see all top-level nodes, issue)set by itself.

)set

Variables with values have them displayed near the right margin. Subtrees of selections have ‘...’ displayed in the value field. For example, there are many kinds of messages, so issue)set message to see the choices.

)set message

The current setting for the variable that displays whether computation times are displayed is visible in the menu displayed by the last command. To see more information, issue

)set message time

This shows that time printing is on now. To turn it off, issue

```
)set message time off
```

As noted above, not all settings have so many qualifiers. For example, to change the)quit command to being unprotected (that is, you will not be prompted for verification), you need only issue

```
)set quit unprotected
```

Also See:

- o)quit

16

Overview

This section contains tree of information used to initialize the)set command in the interpreter. The current list is:

Variable	Description	Current Value

compile	Library compiler options	...
breakmode	execute break processing on error	break
expose	control interpreter constructor exposure	...
functions	some interpreter function options	...
fortran	view and set options for FORTRAN output	...
kernel	library functions built into the kernel for efficiency	...
hyperdoc	options in using HyperDoc	...
help	view and set some help options	...
history	save workspace values in a history file	on
messages	show messages for various system features	...
naglink	options for NAGLink	...
output	view and set some output options	...
quit	protected or unprotected quit	unprotected
streams	set some options for working with streams	...
system	set some system development variables	...
userlevel	operation access level of system user	development

Variables with current values of ... have further sub-options. For example, issue)set system to see what the options are for system. For more information, issue)help set .

¹⁶ “quit” (26.30 p 940)

Initialize the set variables

The argument `settree` is initially the `$setOption` variable. The fourth element is a union-style switch symbol. The fifth element is usually a variable to set. The sixth element is a subtree to recurse for the `TREE` switch. The seventh element is usually the default value. For more detailed explanations see the list structure section [26.34](#). [`sayMSG` p45]

[literals p??]

[`translateYesNo2TrueFalse` p967]

[`tree` p??]

[`initializeSetVariables` p962]

— `defun initializeSetVariables` —

```
(defun |initializeSetVariables| (settree)
  "Initialize the set variables"
  (dolist (setdata settree)
    (case (fourth setdata)
      (function
        (if (canFuncall? (fifth setdata))
            (funcall (fifth setdata) '|%initialize%|)
            (|sayMSG| (concatenate 'string "      Function not implemented. "
                                   (package-name *package*) ":" (string (fifth setdata))))))
      (integer (set (fifth setdata) (seventh setdata)))
      (string (set (fifth setdata) (seventh setdata)))
      (literals
        (set (fifth setdata) (|translateYesNo2TrueFalse| (seventh setdata))))
      (tree (|initializeSetVariables| (sixth setdata))))))
```

—————→

Reset the workspace variables

[`copy` p??]

[`initializeSetVariables` p962]

[`/editfile` p849]

[`/sourcefiles` p??]

[`/pretty` p??]

[\$spaceList p74]

[\$countList p68]

[\$timerList p74]

[\$sourceFiles p??]

[\$existingFiles p??]

[\$functionTable p838]

[\$boot p826]

[\$compileMapFlag p??]

[\$echoLineStack p??]

```

[$operationNameList p??]
[$slamFlag p??]
[$CommandSynonymAlist p817]
[$InitialCommandSynonymAlist p816]
[$UserAbbreviationsAlist p??]
[$msgAlist p42]
[$msgDatabase p??]
[$msgDatabaseName p199]
[$dependeeClosureAlist p??]
[$IOindex p39]
[$coerceIntByMapCounter p??]
[$e p315]
[$env p315]
[$setOptions p??]

```

— **defun resetWorkspaceVariables** —

```

(defun |resetWorkspaceVariables| ()
  "Reset the workspace variables"
  (declare (special |$countList| /editfile /sourcefiles |$sourceFiles| /pretty
    |$spaceList| |$timerList| |$existingFiles| |$functionTable| $boot
    |$compileMapFlag| |$echoLineStack| |$operationNameList| |$slamFlag| | |
    |$CommandSynonymAlist| |$InitialCommandSynonymAlist|
    |$UserAbbreviationsAlist| |$msgAlist| |$msgDatabase| |$msgDatabaseName|
    |$dependeeClosureAlist| |$IOindex| |$coerceIntByMapCounter| |$e| |$env|
    |$setOptions|))
  (setq |$countList| nil)
  (setq /editfile nil)
  (setq /sourcefiles nil)
  (setq |$sourceFiles| nil)
  (setq /pretty nil)
  (setq |$spaceList| nil)
  (setq |$timerList| nil)
  (setq |$existingFiles| (make-hash-table :test #'equal))
  (setq |$functionTable| nil)
  (setq $boot nil)
  (setq |$compileMapFlag| nil)
  (setq |$echoLineStack| nil)
  (setq |$operationNameList| nil)
  (setq |$slamFlag| nil)
  (setq |$CommandSynonymAlist| (copy |$InitialCommandSynonymAlist|))
  (setq |$UserAbbreviationsAlist| nil)
  (setq |$msgAlist| nil)
  (setq |$msgDatabase| nil)
  (setq |$msgDatabaseName| nil)
  (setq |$dependeeClosureAlist| nil)
  (setq |$IOindex| 1)
  (setq |$coerceIntByMapCounter| 0)

```

```
(setq |$e| (cons (cons nil nil) nil))
(setq |$env| (cons (cons nil nil) nil))
(|initializeSetVariables| |$setOptions|))
```

Display the set option information

```
[displaySetVariableSettings p965]
[concat p1237]
[object2String p??]
[specialChar p1166]
[sayBrightly p??]
[bright p??]
[sayMSG p45]
[boot-equal p??]
[sayMessage p??]
[eval p??]
[literals p??]
[translateTrueFalse2YesNo p967]
[$linelength p1051]
```

— defun displaySetOptionInformation —

```
(defun |displaySetOptionInformation| (arg setdata)
  "Display the set option information"
  (let (current)
    (declare (special $linelength))
    (cond
      ((eq (fourth setdata) 'tree)
        (|displaySetVariableSettings| (sixth setdata) (first setdata)))
      (t
        (format t "~v,,,'-:@<~a~>~%" (- $linelength 2)
          (concat " The " (|object2String| arg) " Option "))
        (|sayBrightly|
          '(|%l| ,@( |bright| "Description:") ,(second setdata)))
        (case (fourth setdata)
          (function
            (terpri)
            (if (canFuncall? (fifth setdata))
              (funcall (fifth setdata) '|%describe%|)
              (|sayMSG| " Function not implemented.")))
          (integer
            (|sayMessage|
              '(" The" ,@( |bright| arg) "option"
                " may be followed by an integer in the range"
```

```

,@(|bright| (elt (sixth setdata) 0)) "to"
| %l| ,@(|bright| (elt (sixth setdata) 1)) "inclusive."
" The current setting is" ,@(|bright| (|eval| (fifth setdata))))))
(string
  (|sayMessage|
    ('(" The" ,@(|bright| arg) "option"
      " is followed by a string enclosed in double quote marks."
      '| %l| " The current setting is"
      ,@(|bright| (list '| "| (|eval| (fifth setdata)) '| "|))))))
  (literals
    (|sayMessage|
      ('(" The" ,@(|bright| arg) "option"
        " may be followed by any one of the following:"))
    (setq current
      (|translateTrueFalse2YesNo| (|eval| (fifth setdata))))
    (dolist (name (sixth setdata))
      (if (boot-equal name current)
        (|sayBrightly| ('( " ->" ,@(|bright| (|object2String| name))))
        (|sayBrightly| (list " " (|object2String| name))))))
    (|sayMessage| " The current setting is indicated."))))))

```

Display the set variable settings

```

[concat p1237]
[object2String p??]
[sayBrightly p??]
[say p??]
[fillerSpaces p310]
[specialChar p1166]
[concat p1237]
[satisfiesUserLevel p791]
[poundsign p??]
[eval p??]
[bright p??]
[literals p??]
[translateTrueFalse2YesNo p967]
[tree p??]
[$linelength p1051]

```

— defun displaySetVariableSettings —

```

(defun |displaySetVariableSettings| (settree label)
  "Display the set variable settings"
  (let (setoption opt subtree subname)

```

```

(declare (special $linelength))
(if (eq label '|')
    (setq label ")set")
    (setq label (concat " " (|object2String| label) " ")))
(format t "~v:@<~a~>~%" (- $linelength 2)
    (concat " Current Values of" label " Variables "))
(terpri)
(|sayBrightly|
  (list "Variable      " "Description
        "Current Value" ))
  (say (|fillerSpaces| $linelength (|specialChar| '|hbar|)))
  (setq subtree nil)
  (dolist (setdata settree)
    (when (|satisfiesUserLevel| (third setdata))
      (setq setoption (|object2String| (first setdata)))
      (setq setoption
        (concat setoption
          (|fillerSpaces| (- 13 (|#| setoption)) " "
            (second setdata)))
        (setq setoption
          (concat setoption
            (|fillerSpaces| (- 55 (|#| setoption)) " ")))
        (case (fourth setdata)
          (function
            (setq opt
              (if (canFuncall? (fifth setdata))
                  (funcall (fifth setdata) '|%display%|)
                  "unimplemented"))
            (cond
              ((consp opt)
               (setq opt
                 (do ((t2 opt (cdr t2)) t1 (o nil))
                     ((or (atom t2) (progn (setq o (car t2)) nil)) t1)
                 (setq t1 (append t1 (cons o (cons " " nil)))))))
              (|sayBrightly| (|concat| setoption opt)))
            (string
              (setq opt (|object2String| (|eval| (fifth setdata))))
              (|sayBrightly| '(',setoption ,@(|bright| opt))))
            (integer
              (setq opt (|object2String| (|eval| (fifth setdata))))
              (|sayBrightly| '(',setoption ,@(|bright| opt))))
            (literals
              (setq opt (|object2String|
                (|translateTrueFalse2YesNo| (|eval| (fifth setdata)))))
              (|sayBrightly| '(',setoption ,@(|bright| opt))))
            (TREE
              (|sayBrightly| '(',setoption ,@(|bright| "..."))
              (setq subtree t)
              (setq subname (|object2String| (first setdata))))))
    (terpri)

```

```
(when subtree
  (|sayBrightly|
    '("Variables with current values of" ,@(|bright| "...")
      "have further sub-options. For example,")
  (|sayBrightly|
    '("issue" ,@(|bright| ")set ") ,subname
      " to see what the options are for" ,@(|bright| subname) "."
      |%l| "For more information, issue" ,@(|bright| ")help set") "."))))
```

Translate options values to t or nil

[member p1238]

— `defun translateYesNo2TrueFalse` —

```
(defun |translateYesNo2TrueFalse| (x)
  "Translate options values to t or nil"
  (cond
    ((|member| x '(|yes| |on|)) t)
    ((|member| x '(|no| |off|)) nil)
    (t x)))
```

Translate t or nil to option values

— `defun translateTrueFalse2YesNo` —

```
(defun |translateTrueFalse2YesNo| (x)
  "Translate t or nil to option values"
  (cond
    ((eq x t) '|on|)
    ((null x) '|off|)
    (t x)))
```

The list structure

The structure of each list item consists of 7 items. Consider this example:

```
(userlevel
  "operation access level of system user"
  interpreter
  LITERALS
  $UserLevel
  (interpreter compiler development)
  development)
```

The list contains (the names in bold are accessor names that can be found in **property.lisp.pamphlet**. Look for "setName".):

1 *Name* the keyword the user will see. In this example the user would say ")set output userlevel".

2 *Label* the message the user will see. In this example the user would see "operation access level of system user".

3 *Level* the level where the command will be accepted. There are three levels: interpreter, compiler, development. These commands are restricted to keep the user from causing damage.

4 *Type* a symbol, one of **FUNCTION**, **INTEGER**, **STRING**, **LITERALS**, **FILENAME** or **TREE**.

5 *Var*

FUNCTION is the function to call

INTEGER is the variable holding the current user setting.

STRING is the variable holding the current user setting.

LITERALS variable which holds the current user setting.

FILENAME is the variable that holds the current user setting.

TREE

6 *Leaf*

FUNCTION is the list of all possible values

INTEGER is the range of possible values

STRING is a list of all possible values

LITERALS is a list of all of the possible values

FILENAME is the function to check the filename

TREE

7 *Def* is the default value

FUNCTION is the default setting

INTEGER is the default setting

STRING is the default setting
 LITERALS is the default setting
 FILENAME is the default value
 TREE

26.35 set breakmode

----- The breakmode Option -----

Description: execute break processing on error

The breakmode option may be followed by any one of the following:

```
nobreak
-> break
  query
  resume
  fastlinks
  quit
```

The current setting is indicated.

defvar \$BreakMode

— initvars —

```
(defvar |$BreakMode| ' |nobreak| "execute break processing on error")
```

— breakmode —

```
(|breakmode|
 "execute break processing on error"
 |interpreter|
 LITERALS
 |$BreakMode|
 (|nobreak| |break| |query| |resume| |fastlinks| |quit|)
 |nobreak|) ; needed to avoid possible startup looping
```

26.36 set debug

Current Values of debug Variables

Variable	Description	Current Value
lambdtype	Show type information for #1 syntax	off
dalymode	Interpret leading open paren as lisp	off

— debug —

```
(|debug|
  "debug options"
  |interpreter|
  TREE
  |novar|
  (
    \getchunk{debuglambdtype}
    \getchunk{debugdalymode}
  ))
```

—————→

set debug lambdtype

----- The lambdtype Option -----

Description: Show type information for #1 syntax

defvar \$lambdtype

— initvars —

```
(defvar $lambdtype nil "show type information for #1 syntax")
```

—————→

— debuglambdtype —

```
(|lambdtype|
  "show type information for #1 syntax"
```

```
|interpreter|
LITERALS
$lamdbatype
(|on| |off|)
|off|)
```

26.37 set compiler

Current Values of compiler Variables

Variable	Description	Current Value
output	library in which to place compiled code	
input	controls libraries from which to load compiled code	

— compile —

```
(|compiler|
 "Library compiler options"
 |interpreter|
 TREE
 |novar|
 (
 \getchunk{compileoutput}
 \getchunk{compileinput}
 ))
```

set compiler output

----- The output Option -----

Description: library in which to place compiled code

— compileoutput —

```
(|output|
 "library in which to place compiled code "
 |interpreter|
```

```

FUNCTION
|setOutputLibrary|
NIL
|htSetOutputLibrary|
)

```

The set output command handler

```

[poundsign p??]
[describeOutputLibraryArgs p972]
[filep p??]
[openOutputLibrary p973]
[$outputLibraryName p??]

```

— defun setOutputLibrary —

```

(defun |setOutputLibrary| (arg)
  "The set output command handler"
  (let (fn)
    (declare (special |$outputLibraryName|))
    (cond
      ((eq arg '|%initialize%|) (setq |$outputLibraryName| nil))
      ((eq arg '|%display%|) (or |$outputLibraryName| "user.lib"))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?)) (/= (|#| arg) 1))
      (|describeOutputLibraryArgs|))
    (t
     (when (probe-file (setq fn (princ-to-string (car arg))))
       (setq fn (truename fn)))
     (|openOutputLibrary| (setq |$outputLibraryName| fn))))))

```

Describe the set output library arguments

```

[sayBrightly p??]

```

— defun describeOutputLibraryArgs —

```

(defun |describeOutputLibraryArgs| ()
  "Describe the set output library arguments"
  (|sayBrightly| (list
    " )set compile output library is used to tell the compiler where to place"
    '|%l| "compiled code generated by the library compiler. By default it goes"
    '|%l| "in a file called user.lib in the current directory.")))

```

defvar output-library

— initvars —

```
(defvar output-library nil)
```

Open the output library

The input-libraries and output-library are now truename based.

[dropInputLibrary p976]
 [output-library p973]
 [input-libraries p975]

— defun openOutputLibrary —

```
(defun |openOutputLibrary| (lib)
  "Open the output library"
  (declare (special output-library input-libraries))
  (|dropInputLibrary| lib)
  (setq output-library (truename lib))
  (push output-library input-libraries))
```

set compiler input

----- The input Option -----

Description: controls libraries from which to load compiled code

```
)set compile input add library is used to tell AXIOM to add
library to the front of the path which determines where
compiled code is loaded from.
)set compile input drop library is used to tell AXIOM to remove
library from this path.
```

— compileinput —

```
(|input|
 "controls libraries from which to load compiled code"
 |interpreter|
 FUNCTION
 |setInputLibrary|
 NIL
 |htSetInputLibrary|)
```

The set input library command handler

The input-libraries is now maintained as a list of truenames.

```
[describeInputLibraryArgs p975]
[qcar p??]
[qcdr p??]
[selectOptionLC p819]
[addInputLibrary p975]
[dropInputLibrary p976]
[setInputLibrary p974]
[input-libraries p975]
```

— defun setInputLibrary —

```
(defun |setInputLibrary| (arg)
 "The set input library command handler"
 (declare (special input-libraries))
 (let (tmp1 filename act)
 (cond
 ((eq arg '|%initialize%|) t)
 ((eq arg '|%display%|) (mapcar #'namestring input-libraries))
 ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
 (|describeInputLibraryArgs|))
 ((and (consp arg)
 (progn
 (setq act (qcar arg))
 (setq tmp1 (qcdr arg))
 (and (consp tmp1)
 (eq (qcdr tmp1) nil)
 (progn (setq filename (qcar tmp1)) t))))
 (setq act (|selectOptionLC| act '(|add| |drop|) nil)))
 (cond
 ((eq act '|add|)
 (|addInputLibrary| (truename (princ-to-string filename))))
```

```
((eq act '|drop|)
  (|dropInputLibrary| (truename (princ-to-string filename))))))
(t (|setInputLibrary| nil))))
```

Describe the set input library arguments

[sayBrightly p??]

— defun describeInputLibraryArgs —

```
(defun |describeInputLibraryArgs| ()
  "Describe the set input library arguments"
  (|sayBrightly| (list
    " )set compile input add library "
    "is used to tell AXIOM to add library to"
    '|%1| " the front of the path used to find compile code."
    '|%1|
    " )set compile input drop library is used to tell AXIOM to remove library"
    '|%1| " from this path.")))
```

Add the input library to the list

The input-libraries variable is now maintained as a list of truenames. [dropInputLibrary p976]
 [input-libraries p975]

— defun addInputLibrary —

```
(defun |addInputLibrary| (lib)
  "Add the input library to the list"
  (declare (special input-libraries))
  (|dropInputLibrary| lib)
  (push (truename lib) input-libraries))
```

defvar input-libraries

— initvars —

```
(defvar input-libraries nil)
```

Drop an input library from the list

[input-libraries [p975](#)]

— defun dropInputLibrary —

```
(defun |dropInputLibrary| (lib)
  "Drop an input library from the list"
  (declare (special input-libraries))
  (setq input-libraries (delete (truename lib) input-libraries :test #'equal)))
```

26.38 set debug dalymode

The `$dalymode` variable is used in a case statement in `intloopReadConsole`. This variable can be set to any non-nil value. When not nil the interpreter will send any line that begins with an “(” to be sent to the underlying lisp. This is useful for debugging Axiom. The normal value of this variable is NIL.

This variable was created as an alternative to prefixing every lisp command with `)lisp`. When doing a lot of debugging this is tedious and error prone. This variable was created to shortcut that process. Clearly it breaks some semantics of the language accepted by the interpreter as parens are used for grouping expressions.

----- The dalymode Option -----

Description: Interpret leading open paren as lisp

defvar dalymode

— initvars —

```
(defvar $dalymode nil "Interpret leading open paren as lisp")
```

— debugdalymode —

```
(|dalymode|
  "Interpret leading open paren as lisp"
  |interpreter|
  LITERALS
  $dalymode
  (|on| |off|)
  |off|)
```

—————

26.39 set expose

----- The expose Option -----

Description: control interpreter constructor exposure

The following groups are explicitly exposed in the current frame (called initial):

```
        basic
categories
        naglink
        anna
```

The following constructors are explicitly exposed in the current frame:

there are no explicitly exposed constructors

The following constructors are explicitly hidden in the current frame:

there are no explicitly hidden constructors

When)set expose is followed by no arguments, the information you now see is displayed. When followed by the initialize argument, the exposure group data in the file interp.exposed is read and is then available. The arguments add and drop are used to add or drop exposure groups or explicit constructors from the local frame exposure data. Issue

```
        )set expose add    or    )set expose drop
for more information.
```

— expose —

```
(|expose|
```



```
"control interpreter constructor exposure"
|interpreter|
FUNCTION
|setExpose|
NIL
|htSetExpose|)
```

functions

Current Values of functions Variables

Variable	Description	Current Value
cache	number of function results to cache	0
compile	compile, don't just define function bodies	off
recurrence	specially compile recurrence relations	on

— functions —

```
(|functions|
"some interpreter function options"
|interpreter|
TREE
|novar|
(
\getchunk{functionscache}
\getchunk{functionscompile}
\getchunk{functionsrecurrence}
))
```

functions cache

----- The cache Option -----

Description: number of function results to cache

)set functions cache is used to tell AXIOM how many values computed by interpreter functions should be saved. This can save quite a bit of time in recursive functions, though one must consider that the cached values will take up (perhaps

valuable) room in the workspace.

The value given after cache must either be the word all or a positive integer. This may be followed by any number of function names whose cache sizes you wish to so set. If no functions are given, the default cache size is set.

Examples:)set fun cache all
)set fun cache 10 f g Legendre

In general, functions will cache no returned values.

— functionscache —

```
(|cache|
 "number of function results to cache"
 |interpreter|
 FUNCTION
 |setFunctionsCache|
 NIL
 |htSetCache|)
```

defvar \$cacheAlist

— initvars —

```
(defvar |$cacheAlist| nil)
```

The top level set functions cache handler

```
[object2String p??]
[describeSetFunctionsCache p982]
[sayAllCacheCounts p982]
[sayMessage p??]
[bright p??]
[terminateSystemCommand p792]
[countCache p980]
[$options p73]
[$cacheCount p??]
```

`[cacheAlist p979]`

— `defun setFunctionsCache` —

```
(defun |setFunctionsCache| (arg)
  "The top level set functions cache handler"
  (let (|options| n)
    (declare (special |options| |cacheCount| |cacheAlist|))
    (cond
      ((eq arg '|%initialize%|)
       (setq |cacheCount| 0)
       (setq |cacheAlist| nil))
      ((eq arg '|%display%|)
       (if (null |cacheAlist|)
           (|object2String| |cacheCount|)
           "..."))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
       (|describeSetFunctionsCache|)
       (terpri)
       (|sayAllCacheCounts|))
      (t
       (setq n (car arg))
       (cond
         ((and (not (eq n '|all|)) (or (null (integerp n)) (minusp n)))
          (|sayMessage|
           '("Your value of" ,@(|bright| n) "is invalid because ..."))
          (|describeSetFunctionsCache|)
          (|terminateSystemCommand|))
         (t
          (when (cdr arg) (list (cons '|vars| (cdr arg)))))
          (|countCache| n)))))))
```

—————→

Display a particular cache count

`[qcdr p??]`
`[qcar p??]`
`[identp p1237]`
`[sayKeyedMsg p43]`
`[insertAlist p981]`
`[internl p??]`
`[sayCacheCount p983]`
`[optionError p789]`
`[options p73]`
`[cacheAlist p979]`
`[cacheCount p??]`

— defun countCache —

```

(defun |countCache| (n)
  "Display a particular cache count"
  (let (tmp1 1 cachecountname)
    (declare (special |$options| |$cacheAlist| |$cacheCount|))
    (cond
      (|$options|
        (cond
          ((and (consp |$options|)
                (eq (qcdr |$options|) nil)
                (progn
                  (setq tmp1 (qcar |$options|))
                  (and (consp tmp1)
                      (eq (qcar tmp1) '|vars|)
                      (progn (setq 1 (qcdr tmp1)) t))))))
          (dolist (x 1)
            (if (null (identp x))
                (|sayKeyedMsg| "%1 is not a valid function name." (list x))
                (progn
                  (setq |$cacheAlist| (|insertAlist| x n |$cacheAlist|))
                  (setq cachecountname (intern1 x ";COUNT"))
                  (set cachecountname n)
                  (|sayCacheCount| x n))))))
      (t (|optionError| (caar |$options|) nil))))
  (|sayCacheCount| nil (setq |$cacheCount| n))))

```

defun insertAlist

```

[rplac p??]
[?order p??]

```

— defun insertAlist —

```

(defun |insertAlist| (a b z)
  (labels (
    (fn (a b z)
      (cond
        ((null (cdr z)) (rplac (cdr z) (list (cons a b))))
        ((equal a (elt (elt z 1) 0)) (rplac (cdr (elt z 1)) b))
        ((?order (elt (elt z 1) 0) a) (rplac (cdr z) (cons (cons a b) (cdr z))))
        (t (fn a b (cdr z))))))
    (cond

```

```
((null z) (list (cons a b)))
((equal a (elt (elt z 0) 0)) (rplac (cdar z) b) z)
((?order (elt (elt z 0) 0) a) (cons (cons a b) z))
(t (fn a b z) z)))
```

Describe the set functions cache

[sayBrightly p??]

— defun describeSetFunctionsCache —

```
(defun |describeSetFunctionsCache| ()
  "Describe the set functions cache"
  (|sayBrightly| (list
    " )set functions cache "
    "is used to tell AXIOM how many"
    '|%1| " values computed by interpreter functions should be saved. This"
    '|%1| " can save quite a bit of time in recursive functions, though one"
    '|%1| " must consider that the cached values will take up (perhaps"
    '|%1| " valuable) room in the workspace."
    '|%1|
    '|%1| " The value given after "
    "cache must either be the word all or a positive integer."
    '|%1| " This may be followed by any number of function names whose cache"
    '|%1| " sizes you wish to so set. If no functions are given, the default"
    '|%1| " cache size is set."
    '|%1|
    '|%1| " Examples:"
    '|%1| " )set fun cache all          )set fun cache 10 f g Legendre"))))
```

Display all cache counts

[sayCacheCount p983]

[\$cacheCount p??]

[\$cacheAlist p979]

— defun sayAllCacheCounts —

```
(defun |sayAllCacheCounts| ()
  "Display all cache counts"
  (let (x n)
```

```
(declare (special |$cacheCount| |$cacheAlist|))
(|sayCacheCount| nil |$cacheCount|)
(when |$cacheAlist|
  (do ((t0 |$cacheAlist| (cdr t0)) (t1 nil))
      ((or (atom t0)
            (progn (setq t1 (car t0)) nil)
            (progn
              (progn (setq x (car t1)) (setq n (cdr t1)) t1)
              nil))
         nil)
    (when (not (equal n |$cacheCount|)) (|sayCacheCount| x n))))))
```

Describe the cache counts

```
[bright p??]
[linearFormatName p??]
[sayBrightly p??]
```

— defun sayCacheCount —

```
(defun |sayCacheCount| (fn n)
  "Describe the cache counts"
  (let (prefix phrase)
    (setq prefix
      (cond
        (fn (cons 'function (|bright| (|linearFormatName| fn))))
        ((eql n 0) (list '|interpreter functions|))
        (t (list '|In general, interpreter functions|))))
    (cond
      ((eql n 0)
        (cond
          (fn
            (|sayBrightly|
              '(" Caching for " ,prefix "is turned off"))
            (t
              (|sayBrightly| " In general, functions will cache no returned values."
                )))))
      (t
        (setq phrase
          (cond
            ((eq n '|all|) '(',@(|bright| '|all|) |values.|))
            ((eql n 1) (list '| only the last value.|))
            (t '(| the last| ,@(|bright| n) |values.|))))
        (|sayBrightly|
          '(" " ,@prefix "will cache" ,@phrase))))))
```

functions compile

----- The compile Option -----

Description: compile, don't just define function bodies

The compile option may be followed by any one of the following:

-> on
 off

The current setting is indicated.

```
\def\dollar{compileDontDefineFunctions}
\begin{chunk}{initvars}
(defvar |$compileDontDefineFunctions| t
  "compile, don't just define function bodies")

\end{chunk}
\begin{chunk}{functionscompile}
  (|compile|
    "compile, don't just define function bodies"
    |interpreter|
    LITERALS
    |$compileDontDefineFunctions|
    (|on| |off|)
    |on|)
\end{chunk}
\subsection{functions recurrence}
\begin{verbatim}
```

----- The recurrence Option -----

Description: specially compile recurrence relations

The recurrence option may be followed by any one of the following:

-> on
 off

The current setting is indicated.

defvar \$compileRecurrence

— initvars —

```
(defvar |$compileRecurrence| t "specially compile recurrence relations")
```

—————

— functionsrecurrence —

```
(|recurrence|
 "specially compile recurrence relations"
 |interpreter|
 LITERALS
 |$compileRecurrence|
 (|on| |off|)
 |on|)
```

—————

26.40 set fortran

Current Values of fortran Variables

Variable	Description	Current Value

ints2floats	where sensible, coerce integers to reals	on
fortindent	the number of characters indented	6
fortlength	the number of characters on a line	72
typedecs	print type and dimension lines	on
defaulttype	default generic type for FORTRAN object	REAL
precision	precision of generated FORTRAN objects	double
intrinsic	whether to use INTRINSIC FORTRAN functions	off
explength	character limit for FORTRAN expressions	1320
segment	split long FORTRAN expressions	on
optlevel	FORTAN optimisation level	0
startindex	starting index for FORTRAN arrays	1
calling	options for external FORTRAN calls	...

Variables with current values of ... have further sub-options.
 For example, issue)set calling to see what the options are for calling.
 For more information, issue)help set .

— fortran —


```

(|fortran|
  "view and set options for FORTRAN output"
  |interpreter|
  TREE
  |novar|
  (
\getchunk{fortranints2floats}
\getchunk{fortranfortindent}
\getchunk{fortranfortlength}
\getchunk{fortrantypedecs}
\getchunk{fortrandefaulttype}
\getchunk{fortranprecision}
\getchunk{fortranintrinsic}
\getchunk{fortranexplength}
\getchunk{fortransegment}
\getchunk{fortranoptlevel}
\getchunk{fortranstartindex}
\getchunk{fortrancalling}
  ))

```

set ints2floats

----- The ints2floats Option -----

Description: where sensible, coerce integers to reals

The ints2floats option may be followed by any one of the following:

```

-> on
    off

```

The current setting is indicated.

defvar \$fortInts2Floats

— initvars —

```
(defvar |$fortInts2Floats| t "where sensible, coerce integers to reals")
```

— **fortranints2floats** —

```
(|ints2floats|
 "where sensible, coerce integers to reals"
 |interpreter|
 LITERALS
 |$fortInts2Floats|
 (|on| |off|)
 |on|)
```

set fortindent

----- The fortindent Option -----

Description: the number of characters indented

The fortindent option may be followed by an integer in the range 0 to inclusive. The current setting is 6

defvar \$fortIndent— **initvars** —

```
(defvar |$fortIndent| 6 "the number of characters indented")
```

— **fortranfortindent** —

```
(|fortindent|
 "the number of characters indented"
 |interpreter|
 INTEGER
 |$fortIndent|
 (0 NIL)
 6)
```

set fortlength

----- The fortlength Option -----

Description: the number of characters on a line

The fortlength option may be followed by an integer in the range 1 to inclusive. The current setting is 72

defvar \$fortLength

— initvars —

```
(defvar |$fortLength| 72 "the number of characters on a line")
```

— fortranfortlength —

```
(|fortlength|
 "the number of characters on a line"
 |interpreter|
 INTEGER
 |$fortLength|
 (1 NIL)
 72)
```

set typedecs

----- The typedecs Option -----

Description: print type and dimension lines

The typedecs option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$printFortranDecs

— initvars —

```
(defvar |$printFortranDecs| t "print type and dimension lines")
```

— fortrantypedecs —

```
(|typedecs|
 "print type and dimension lines"
 |interpreter|
 LITERALS
 |$printFortranDecs|
 (|on| |off|)
 |on|)
```

set defaulttype

----- The defaulttype Option -----

Description: default generic type for FORTRAN object

The defaulttype option may be followed by any one of the following:

```
-> REAL
    INTEGER
    COMPLEX
    LOGICAL
    CHARACTER
```

The current setting is indicated.

defvar \$defaultFortranType

— initvars —

```
(defvar |$defaultFortranType| 'real "default generic type for FORTRAN object")
```

— **fortrandefaulttype** —

```
(|defaulttype|
 "default generic type for FORTRAN object"
 |interpreter|
 LITERALS
 |$defaultFortranType|
 (REAL INTEGER COMPLEX LOGICAL CHARACTER)
 REAL)
```

set precision

----- The precision Option -----

Description: precision of generated FORTRAN objects

The precision option may be followed by any one of the following:

```
single
-> double
```

The current setting is indicated.

defvar \$fortranPrecision

— **initvars** —

```
(defvar |$fortranPrecision| 'double| "precision of generated FORTRAN objects")
```

— **fortranprecision** —

```
(|precision|
 "precision of generated FORTRAN objects"
 |interpreter|
 LITERALS
 |$fortranPrecision|
```

```
(|single| |double|)
|double|)
```

set intrinsic

----- The intrinsic Option -----

Description: whether to use INTRINSIC FORTRAN functions

The intrinsic option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$useIntrinsicFunctions

— initvars —

```
(defvar |$useIntrinsicFunctions| nil
  "whether to use INTRINSIC FORTRAN functions")
```

— fortranintrinsic —

```
(|intrinsic|
  "whether to use INTRINSIC FORTRAN functions"
  |interpreter|
  LITERALS
  |$useIntrinsicFunctions|
  (|on| |off|)
  |off|)
```

set explength

----- The explength Option -----

Description: character limit for FORTRAN expressions

The `explength` option may be followed by an integer in the range 0 to inclusive. The current setting is 1320

defvar \$maximumFortranExpressionLength

— initvars —

```
(defvar |$maximumFortranExpressionLength| 1320
  "character limit for FORTRAN expressions")
```

— fortranexplength —

```
(|explength|
  "character limit for FORTRAN expressions"
  |interpreter|
  INTEGER
  |$maximumFortranExpressionLength|
  (0 NIL)
  1320)
```

set segment

----- The segment Option -----

Description: split long FORTRAN expressions

The `segment` option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$fortranSegment

— initvars —

```
(defvar |$fortranSegment| t "split long FORTRAN expressions")
```

— fortransegment —

```
(|segment|
 "split long FORTRAN expressions"
 |interpreter|
 LITERALS
 |$fortranSegment|
 (|on| |off|)
 |on|)
```

set optlevel

----- The optlevel Option -----

Description: FORTRAN optimisation level

The optlevel option may be followed by an integer in the range 0 to 2 inclusive. The current setting is 0

defvar \$fortranOptimizationLevel

— initvars —

```
(defvar |$fortranOptimizationLevel| 0 "FORTRAN optimisation level")
```

— fortranoptlevel —

```
(|optlevel|
 "FORTRAN optimisation level")
```



```
|interpreter|
INTEGER
|$fortranOptimizationLevel|
(0 2)
0)
```

set startindex

----- The startindex Option -----

Description: starting index for FORTRAN arrays

The startindex option may be followed by an integer in the range 0 to 1 inclusive. The current setting is 1

defvar \$fortranArrayStartingIndex

— initvars —

```
(defvar |$fortranArrayStartingIndex| 1 "starting index for FORTRAN arrays")
```

— fortranstartindex —

```
(|startindex|
"starting index for FORTRAN arrays"
|interpreter|
INTEGER
|$fortranArrayStartingIndex|
(0 1)
1)
```

set calling

Current Values of calling Variables

Variable	Description	Current Value

```

tempfile      set location of temporary data files      /tmp/
directory     set location of generated FORTRAN files   ./
linker        linker arguments (e.g. libraries to search) -lxf

```

— fortrancalling —

```

(|calling|
"options for external FORTRAN calls"
|interpreter|
TREE
|novar|
(
\getchunk{callingtempfile}
\getchunk{callingdirectory}
\getchunk{callinglinker}
)
)

```

set tempfile

----- The tempfile Option -----

Description: set location of temporary data files

)set fortran calling tempfile is used to tell AXIOM where to place intermediate FORTRAN data files . This must be the name of a valid existing directory to which you have permission to write (including the final slash).

Syntax:

```
)set fortran calling tempfile DIRECTORYNAME
```

The current setting is /tmp/

defvar \$fortranTmpDir

— initvars —

```
(defvar |$fortranTmpDir| "/tmp/" "set location of temporary data files ")
```

— callingtempfile —

```
(|tempfile|
  "set location of temporary data files "
  |interpreter|
  FUNCTION
  |setFortTmpDir|
  (("enter directory name for which you have write-permission "
    DIRECTORY
    |$fortranTmpDir|
    |chkDirectory|
    "/tmp/"))
  NIL)
```

The top level set fortran calling tempfile handler

```
[pname p1235]
[describeSetFortTmpDir p997]
[validateOutputDirectory p997]
[sayBrightly p??]
[bright p??]
[$fortranTmpDir p995]
```

— defun setFortTmpDir —

```
(defun |setFortTmpDir| (arg)
  "The top level set fortran calling tempfile handler"
  (let (mode)
    (declare (special |$fortranTmpDir|))
    (cond
      ((eq arg '|%initialize%|) (setq |$fortranTmpDir| "/tmp/"))
      ((eq arg '|%display%|)
        (if (stringp |$fortranTmpDir|)
          |$fortranTmpDir|
          (pname |$fortranTmpDir|)))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
        (|describeSetFortTmpDir|))
      ((null (setq mode (|validateOutputDirectory| arg)))
        (|sayBrightly|
          (" Sorry, but your argument(s)" ,@(|bright| arg)
            "is(are) not valid." |%l|))
        (|describeSetFortTmpDir|))
      (t (setq |$fortranTmpDir| mode))))))
```

Validate the output directory

— defun validateOutputDirectory —

```
(defun |validateOutputDirectory| (x)
  "Validate the output directory"
  (let ((dirname (car x)))
    (when (and (pathname-directory dirname) (null (probe-file dirname)))
      dirname)))
```

Describe the set fortran calling tempfile

```
[sayBrightly p??]
[$fortranTmpDir p995]
```

— defun describeSetFortTmpDir —

```
(defun |describeSetFortTmpDir| ()
  "Describe the set fortran calling tempfile"
  (declare (special |$fortranTmpDir|))
  (|sayBrightly| (list
    " )set fortran calling tempfile"
    " is used to tell AXIOM where"
    '|%1| " to place intermediate FORTRAN data files . This must be the "'
    '|%1| " name of a valid existing directory to which you have permission "'
    '|%1| " to write (including the final slash)."'
    '|%1|
    '|%1| " Syntax:"
    '|%1| " )set fortran calling tempfile DIRECTORYNAME"
    '|%1|
    '|%1| " The current setting is "'
    |$fortranTmpDir|
  )))
```

directory

----- The directory Option -----

Description: set location of generated FORTRAN files

)set fortran calling directory is used to tell AXIOM where to place generated FORTRAN files. This must be the name of a valid existing directory to which you have permission to write (including the final slash).

Syntax:

)set fortran calling directory DIRECTORYNAME

The current setting is ./

defvar \$fortranDirectory

— initvars —

```
(defvar |$fortranDirectory| "./" "set location of generated FORTRAN files ")
```

—————

— callingdirectory —

```
(|directory|
 "set location of generated FORTRAN files "
 |interpreter|
 FUNCTION
 |setFortDir|
 (("enter directory name for which you have write-permission "
  DIRECTORY
  |$fortranDirectory|
  |chkDirectory|
  "./")
  NIL)
```

—————

defun setFortDir

```
[pname p1235]
[describeSetFortDir p999]
[validateOutputDirectory p997]
[sayBrightly p??]
[bright p??]
```

[[\\$fortranDirectory](#) p998]

— defun setFortDir —

```
(defun |setFortDir| (arg)
  (declare (special |$fortranDirectory|))
  (let (mode)
    (COND
      ((eq arg '|%initialize%|) (setq |$fortranDirectory| ". /"))
      ((eq arg '|%display%|)
       (if (stringp |$fortranDirectory|
           |$fortranDirectory|
           (pname |$fortranDirectory|)))
        (or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
        (|describeSetFortDir|))
      ((null (setq mode (|validateOutputDirectory| arg)))
       (|sayBrightly|
        (" Sorry, but your argument(s)" ,@( |bright| arg)
         "is(are) not valid." |%l|))
       (|describeSetFortDir|))
      (t (setq |$fortranDirectory| mode))))))
```

defun describeSetFortDir

[[sayBrightly](#) p??]

[[\\$fortranDirectory](#) p998]

— defun describeSetFortDir —

```
(defun |describeSetFortDir| ()
  (declare (special |$fortranDirectory|))
  (|sayBrightly| (list
    " )set fortran calling directory"
    " is used to tell AXIOM where"
    '|%l| " to place generated FORTRAN files. This must be the name "'
    '|%l| " of a valid existing directory to which you have permission "'
    '|%l| " to write (including the final slash)."'
    '|%l|
    '|%l| " Syntax:"
    '|%l| " )set fortran calling directory DIRECTORYNAME"'
    '|%l|
    '|%l| " The current setting is "'
    |$fortranDirectory|
  )))
```

linker

----- The linker Option -----

Description: linker arguments (e.g. libraries to search)

)set fortran calling linkerargs is used to pass arguments to the linker when using mkFort to create functions which call Fortran code. For example, it might give a list of libraries to be searched, and their locations. The string is passed verbatim, so must be the correct syntax for the particular linker being used.

Example:)set fortran calling linker "-lxlif"

The current setting is -lxlif

defvar \$fortranLibraries

— initvars —

```
(defvar |$fortranLibraries| "-lxlif"
  "linker arguments (e.g. libraries to search)")
```

— callinglinker —

```
(|linker|
  "linker arguments (e.g. libraries to search) "
  |interpreter|
  FUNCTION
  |setLinkerArgs|
  (("enter linker arguments "
    STRING
    |$fortranLibraries|
    |chkDirectory|
    "-lxlif"))
  NIL
)
```

defun setLinkerArgs

```
[object2String p??]
[describeSetLinkerArgs p1001]
[$fortranLibraries p1000]
```

— defun setLinkerArgs —

```
(defun |setLinkerArgs| (arg)
  (declare (special |$fortranLibraries|))
  (cond
    ((eq arg '|%initialize%|) (setq |$fortranLibraries| "-lxlif"))
    ((eq arg '|%display%|) (|object2String| |$fortranLibraries|))
    ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
     (|describeSetLinkerArgs|))
    ((and (listp arg) (stringp (car arg)))
     (setq |$fortranLibraries| (car arg)))
    (t (|describeSetLinkerArgs|))))
```

—————

defun describeSetLinkerArgs

```
[sayBrightly p??]
[$fortranLibraries p1000]
```

— defun describeSetLinkerArgs —

```
(defun |describeSetLinkerArgs| ()
  (declare (special |$fortranLibraries|))
  (|sayBrightly| (list
    " )set fortran calling linkerargs"
    " is used to pass arguments to the linker"
    '|%l| " when using "'
    "mkFort to create functions which call Fortran code."
    '|%l| " For example, it might give a list of libraries to be searched,"
    '|%l| " and their locations."
    '|%l| " The string is passed verbatim, so must be the correct syntax for"
    '|%l| " the particular linker being used."
    '|%l|
    '|%l| " Example: )set fortran calling linker \"-lxlif\""'
    '|%l|
    '|%l| " The current setting is "'
    |$fortranLibraries|
  )))
```

—————

26.41 set hyperdoc

Current Values of hyperdoc Variables

Variable	Description	Current Value
fullscreen	use full screen for this facility	off
mathwidth	screen width for history output	120

— hyperdoc —

```
(|hyperdoc|
  "options in using HyperDoc"
  |interpreter|
  TREE
  |novar|
  (
    \getchunk{hyperdocfullscreen}
    \getchunk{hyperdocmathwidth}
  ))
```

fullscreen

----- The fullscreen Option -----

Description: use full screen for this facility

The fullscreen option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$fullScreenSysVars

— initvars —

```
(defvar |$fullScreenSysVars| nil "use full screen for this facility")
```

— hyperdocfullscreen —

```
(|fullscreen|
 "use full screen for this facility"
 |interpreter|
 LITERALS
 |$fullScreenSysVars|
 (|on| |off|)
 |off|)
```

mathwidth

----- The mathwidth Option -----

Description: screen width for history output

The mathwidth option may be followed by an integer in the range 0 to inclusive. The current setting is 120

defvar \$historyDisplayWidth

— initvars —

```
(defvar |$historyDisplayWidth| 120 "screen width for history output")
```

— hyperdocmathwidth —

```
(|mathwidth|
 "screen width for history output"
 |interpreter|
 INTEGER
 |$historyDisplayWidth|
 (0 NIL)
 120)
```

26.42 set help

Current Values of help Variables

Variable	Description	Current Value

fullscreen	use fullscreen facility, if possible	on

— help —

```
(|help|
 "view and set some help options"
 |interpreter|
 TREE
 |novar|
 (
 \getchunk{helpfullscreen}
 ))
```

—————

fullscreen

----- The fullscreen Option -----

Description: use fullscreen facility, if possible

The fullscreen option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$useFullScreenHelp

— initvars —

```
(defvar |$useFullScreenHelp| t "use fullscreen facility, if possible")
```

— helpfullscreen —

```
(|fullscreen|
 "use fullscreen facility, if possible"
 |interpreter|
 LITERALS
 |$useFullScreenHelp|
 (|on| |off|)
 |on|)
```

26.43 set history

----- The history Option -----

Description: save workspace values in a history file

The history option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$HiFiAccess

— initvars —

```
(defvar |$HiFiAccess| t "save workspace values in a history file")
```

— history —

```
(|history|
 "save workspace values in a history file"
 |interpreter|
 LITERALS)
```

```

|$HiFiAccess|
(|on| |off|)
|on|)

```

26.44 set messages

Current Values of messages Variables

Variable	Description	Current Value
autoload	print file auto-load messages	off
bottomup	display bottom up modemap selection	off
coercion	display datatype coercion messages	off
dropmap	display old map defn when replaced	off
expose	warning for unexposed functions	off
file	print msgs also to SPADMSG LISTING	off
frame	display messages about frames	off
highlighting	use highlighting in system messages	off
instant	present instantiation summary	off
insteach	present instantiation info	off
interponly	say when function code is interpreted	on
number	display message number with message	off
prompt	set type of input prompt to display	step
selection	display function selection msgs	off
set	show)set setting after assignment	off
startup	display messages on start-up	off
summary	print statistics after computation	off
testing	print system testing header	off
time	print timings after computation	off
type	print type after computation	on
void	print Void value when it occurs	off
any	print the internal type of objects of domain Any	on
naglink	show NAGLink messages	on

— messages —

```

(|messages|
 "show messages for various system features"
 |interpreter|
 TREE
 |novar|
 (
 \getchunk{messagesany}
 \getchunk{messagesautoload}

```

```

\getchunk{messagesbottomup}
\getchunk{messagescoercion}
\getchunk{messagesdropmap}
\getchunk{messagesexpose}
\getchunk{messagesfile}
\getchunk{messagesframe}
\getchunk{messageshighlighting}
\getchunk{messagesinstant}
\getchunk{messagesinsteach}
\getchunk{messagesinterponly}
\getchunk{messagesnaglink}
\getchunk{messagesnumber}
\getchunk{messagesprompt}
\getchunk{messagesselecction}
\getchunk{messagesset}
\getchunk{messagesstartup}
\getchunk{messagessummary}
\getchunk{messagestesting}
\getchunk{messagestime}
\getchunk{messagestype}
\getchunk{messagesvoid}
))

```

set message any

----- The any Option -----

Description: print the internal type of objects of domain Any

The any option may be followed by any one of the following:

```

-> on
    off

```

The current setting is indicated.

defvar \$printAnyIfTrue

— initvars —

```

(defvar |$printAnyIfTrue| t
  "print the internal type of objects of domain Any")

```

— messagesany —

```
(|any|
 "print the internal type of objects of domain Any"
 |interpreter|
 LITERALS
 |$printAnyIfTrue|
 (|on| |off|)
 |on|)
```

set message autoload

----- The autoload Option -----

Description: print file auto-load messages

defvar \$printLoadMsgs

— initvars —

```
(defvar |$printLoadMsgs| nil "print file auto-load messages")
```

— messagesautoload —

```
(|autoload|
 "print file auto-load messages"
 |interpreter|
 LITERALS
 |$printLoadMsgs|
 (|on| |off|)
 |on|)
```

set message bottomup

----- The bottomup Option -----

Description: display bottom up modemap selection

The bottomup option may be followed by any one of the following:

on
-> off

The current setting is indicated.

defvar \$reportBottomUpFlag

— initvars —

```
(defvar |$reportBottomUpFlag| nil "display bottom up modemap selection")
```

— messagesbottomup —

```
(|bottomup|
 "display bottom up modemap selection"
 |development|
 LITERALS
 |$reportBottomUpFlag|
 (|on| |off|)
 |off|)
```

set message coercion

----- The coercion Option -----

Description: display datatype coercion messages

The coercion option may be followed by any one of the following:


```

    on
-> off

```

The current setting is indicated.

defvar \$reportCoerceIfTrue

— initvars —

```
(defvar |$reportCoerceIfTrue| nil "display datatype coercion messages")
```

—————

— messagescoercion —

```

(|coercion|
 "display datatype coercion messages"
 |development|
 LITERALS
 |$reportCoerceIfTrue|
 (|on| |off|)
 |off|)

```

—————

set message dropmap

----- The dropmap Option -----

Description: display old map defn when replaced

The dropmap option may be followed by any one of the following:

```

    on
-> off

```

The current setting is indicated.

defvar \$displayDroppedMap

— initvars —

```
(defvar |$displayDroppedMap| nil "display old map defn when replaced")
```

— messagesdropmap —

```
(|dropmap|
  "display old map defn when replaced"
  |interpreter|
  LITERALS
  |$displayDroppedMap|
  (|on| |off|)
  |off|)
```

set message expose

----- The expose Option -----

Description: warning for unexposed functions

The expose option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$giveExposureWarning

— initvars —

```
(defvar |$giveExposureWarning| nil "warning for unexposed functions")
```

— messagesexpose —

```
(|expose|
 "warning for unexposed functions"
 |interpreter|
 LITERALS
 |$giveExposureWarning|
 (|on| |off|)
 |off|)
```

—————

set message file

----- The file Option -----

Description: print msgs also to SPADMSG LISTING

The file option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$printMsgsToFile

— initvars —

```
(defvar |$printMsgsToFile| nil "print msgs also to SPADMSG LISTING")
```

—————

— messagesfile —

```
(|file|
 "print msgs also to SPADMSG LISTING"
 |development|
 LITERALS
 |$printMsgsToFile|
 (|on| |off|)
```

```
|off|)
```

set message frame

----- The frame Option -----

Description: display messages about frames

The frame option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$frameMessages

— initvars —

```
(defvar |$frameMessages| nil "display messages about frames")
```

— messagesframe —

```
(|frame|
 "display messages about frames"
 |interpreter|
 LITERALS
 |$frameMessages|
 (|on| |off|)
 |off|)
```

set message highlighting

----- The highlighting Option -----

Description: use highlighting in system messages

The highlighting option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$highlightAllowed

— initvars —

```
(defvar |$highlightAllowed| nil "use highlighting in system messages")
```

—————

— messageshighlighting —

```
(|highlighting|
 "use highlighting in system messages"
 |interpreter|
 LITERALS
 |$highlightAllowed|
 (|on| |off|)
 |off|)
```

—————

set message instant

----- The instant Option -----

Description: present instantiation summary

The instant option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$reportInstantiations

— initvars —

```
(defvar |$reportInstantiations| nil "present instantiation summary")
```

— messagesinstant —

```
(|instant|
 "present instantiation summary"
 |development|
 LITERALS
 |$reportInstantiations|
 (|on| |off|)
 |off|)
```

set message insteach

----- The insteach Option -----

Description: present instantiation info

The insteach option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$reportEachInstantiation—

— initvars —

```
(defvar |$reportEachInstantiation| nil "present instantiation info")
```

— messagesinstead —

```
(|instead|
 "present instantiation info"
 |development|
 LITERALS
 |$reportEachInstantiation|
 (|on| |off|)
 |off|)
```

—————

set message interponly

----- The interponly Option -----

Description: say when function code is interpreted

The interponly option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$reportInterpOnly

— initvars —

```
(defvar |$reportInterpOnly| t "say when function code is interpreted")
```

—————

— messagesinterponly —

```
(|interponly|
 "say when function code is interpreted"
 |interpreter|
 LITERALS
 |$reportInterpOnly|
 (|on| |off|)
```

|on|)

set message naglink

----- The naglink Option -----

Description: show NAGLink messages

The naglink option may be followed by any one of the following:

-> on
 off

The current setting is indicated.

defvar \$nagMessages

— initvars —

```
(defvar |$nagMessages| t "show NAGLink messages")
```

— messagesnaglink —

```
(|naglink|
 "show NAGLink messages"
 |interpreter|
 LITERALS
 |$nagMessages|
 (|on| |off|)
 |on|)
```

set message number

----- The number Option -----

Description: display message number with message

The number option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$displayMsgNumber

— initvars —

```
(defvar |$displayMsgNumber| nil "display message number with message")
```

—————

— messagesnumber —

```
(|number|
 "display message number with message"
 |interpreter|
 LITERALS
 |$displayMsgNumber|
 (|on| |off|)
 |off|)
```

—————

set message prompt

----- The prompt Option -----

Description: set type of input prompt to display

The prompt option may be followed by any one of the following:

```
none
frame
plain
-> step
```

verbose

The current setting is indicated.

defvar \$inputPromptType

— initvars —

```
(defvar |$inputPromptType| ' |step| "set type of input prompt to display")
```

—————

— messagesprompt —

```
(|prompt|
 "set type of input prompt to display"
 |interpreter|
 LITERALS
 |$inputPromptType|
 (|none| |frame| |plain| |step| |verbose|)
 |step|)
```

—————

set message selection

----- The selection Option -----

Description: display function selection msgs

The selection option may be followed by any one of the following:

on
-> off

The current setting is indicated.

TPDHERE: This is a duplicate of)set mes bot on because both use the \$reportBottomUpFlag flag

— messagesselection —

```
(|selection|
  "display function selection msgs"
  |interpreter|
  LITERALS
  |$reportBottomUpFlag|
  (|on| |off|)
  |off|)
```

set

----- The set Option -----

Description: show)set setting after assignment

The set option may be followed by any one of the following:

```
    on
-> off
```

The current setting is indicated.

defvar \$displaySetValue

— initvars —

```
(defvar |$displaySetValue| nil "show )set setting after assignment")
```

— messageset —

```
(|set|
  "show )set setting after assignment"
  |interpreter|
  LITERALS
  |$displaySetValue|
  (|on| |off|)
  |off|)
```

set message startup

----- The startup Option -----

Description: display messages on start-up

The startup option may be followed by any one of the following:

on
-> off

The current setting is indicated.

defvar \$displayStartMsgs

— initvars —

```
(defvar |$displayStartMsgs| t "display messages on start-up")
```

— messagesstartup —

```
(|startup|
 "display messages on start-up"
 |interpreter|
 LITERALS
 |$displayStartMsgs|
 (|on| |off|)
 |on|)
```

set message summary

----- The summary Option -----

Description: print statistics after computation

The summary option may be followed by any one of the following:

```

    on
-> off

```

The current setting is indicated.

defvar \$printStatsSummaryIfTrue

— initvars —

```

(defvar |$printStatsSummaryIfTrue| nil
  "print statistics after computation")

```

—————

— messagessummary —

```

(|summary|
  "print statistics after computation"
  |interpreter|
  LITERALS
  |$printStatsSummaryIfTrue|
  (|on| |off|)
  |off|)

```

—————

set message testing

----- The testing Option -----

Description: print system testing header

The testing option may be followed by any one of the following:

```

    on
-> off

```

The current setting is indicated.

defvar \$testingSystem

— initvars —

```
(defvar |$testingSystem| nil "print system testing header")
```

— messagestesting —

```
(|testing|
 "print system testing header"
 |development|
 LITERALS
 |$testingSystem|
 (|on| |off|)
 |off|)
```

set message time

----- The time Option -----

Description: print timings after computation

The time option may be followed by any one of the following:

```
on
-> off
long
```

The current setting is indicated.

defvar \$printTimeIfTrue

— initvars —

```
(defvar |$printTimeIfTrue| nil "print timings after computation")
```

— **messagetime** —

```
(|time|
 "print timings after computation"
 |interpreter|
 LITERALS
 |$printTimeIfTrue|
 (|on| |off| |long|)
 |off|)
```

set message type

----- The type Option -----

Description: print type after computation

The type option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$printTypeIfTrue

— **initvars** —

```
(defvar |$printTypeIfTrue| t "print type after computation")
```

— **messagetype** —

```
(|type|
 "print type after computation"
 |interpreter|
 LITERALS
 |$printTypeIfTrue|
 (|on| |off|)
 |on|)
```

set message void

----- The void Option -----

Description: print Void value when it occurs

The void option may be followed by any one of the following:

on
-> off

The current setting is indicated.

defvar \$printVoidIfTrue

— initvars —

```
(defvar |$printVoidIfTrue| nil "print Void value when it occurs")
```

— messagesvoid —

```
(|void|
 "print Void value when it occurs"
 |interpreter|
 LITERALS
 |$printVoidIfTrue|
 (|on| |off|)
 |off|)
```

26.45 set naglink

Current Values of naglink Variables

Variable	Description	Current Value

host	internet address of host for NAGLink	localhost
persistence	number of (fortran) functions to remember	1
messages	show NAGLink messages	on
double	enforce DOUBLE PRECISION ASPs	on

— naglink —

```
(|naglink|
  "options for NAGLink"
  |interpreter|
  TREE
  |novar|
  (
    \getchunk{naglinkhost}
    \getchunk{naglinkpersistence}
    \getchunk{naglinkmessages}
    \getchunk{naglinkdouble}
  ))
```

—————

set naglink host

----- The host Option -----

Description: internet address of host for NAGLink

)set naglink host is used to tell AXIOM which host to contact for a NAGLink request. An Internet address should be supplied. The host specified must be running the NAGLink daemon.

The current setting is localhost

defvar \$nagHost

— initvars —

```
(defvar |$nagHost| "localhost" "internet address of host for NAGLink")
```

—————

— naglinkhost —

```
(|host|
  "internet address of host for NAGLink "
  |interpreter|
  FUNCTION
  |setNagHost|
  (("enter host name"
    DIRECTORY
    |$nagHost|
    |chkDirectory|
    "localhost"))
  NIL)
```

defun setNagHost

```
[object2String p??]
[describeSetNagHost p1027]
[$nagHost p1026]
```

— defun setNagHost —

```
(defun |setNagHost| (arg)
  (declare (special |$nagHost|))
  (cond
    ((eq arg '|%initialize%|) (setq |$nagHost| "localhost"))
    ((eq arg '|%display%|) (|object2String| |$nagHost|))
    ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
     (|describeSetNagHost|))
    (t (setq |$nagHost| (|object2String| arg)))))
```

defun describeSetNagHost

```
[sayBrightly p??]
[$nagHost p1026]
```

— defun describeSetNagHost —

```
(defun |describeSetNagHost| ()
  (declare (special |$nagHost|))
  (|sayBrightly| (list
    " )set naglink host "
    "is used to tell AXIOM which host to contact for"
```

```
'|%l| " a NAGLink request. An Internet address should be supplied. The host"
'|%l| " specified must be running the NAGLink daemon."
'|%l|
'|%l| " The current setting is "
|$nagHost|
)))
```

set naglink persistence

----- The persistence Option -----

Description: number of (fortran) functions to remember

)set naglink persistence is used to tell the nagd daemon how many ASP source and object files to keep around in case you reuse them. This helps to avoid needless recompilations. The number specified should be a non-negative integer.

The current setting is 1

defvar \$fortPersistence

— initvars —

```
(defvar |$fortPersistence| 1 "number of (fortran) functions to remember")
```

— naglinkpersistence —

```
(|persistence|
 "number of (fortran) functions to remember "
 |interpreter|
 FUNCTION
 |setFortPers|
 ("Requested remote storage (for asps):"
 INTEGER
 |$fortPersistence|
 (0 NIL)
 10))
 NIL)
```

defun setFortPers

```
[describeFortPersistence p1029]
[sendMessage p??]
[bright p??]
[terminateSystemCommand p792]
[$fortPersistence p1028]
```

— **defun setFortPers** —

```
(defun |setFortPers| (arg)
  (let (n)
    (declare (special |$fortPersistence|))
    (cond
      ((eq arg '|%initialize%|) (setq |$fortPersistence| 1))
      ((eq arg '|%display%|) |$fortPersistence|)
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
       (|describeFortPersistence|))
      (t
       (setq n (car arg))
       (cond
         ((or (null (integerp n)) (minusp n))
          (|sendMessage|
           '("Your value of" ,@( |bright| n) "is invalid because ..."))
          (|describeFortPersistence|)
          (|terminateSystemCommand|))
         (t (setq |$fortPersistence| (car arg)))))))
```

defun describeFortPersistence

```
[sayBrightly p??]
[$fortPersistence p1028]
```

— **defun describeFortPersistence** —

```
(defun |describeFortPersistence| ()
  (declare (special |$fortPersistence|))
  (|sayBrightly| (list
    " )set naglink persistence "
    "is used to tell the "
    '|nagd|
    '| daemon how many ASP|
```

```
'|%1|
" source and object files to keep around in case you reuse them. This helps"
'|%1| " to avoid needless recompilations. The number specified should be a "
'|%1| " non-negative integer."
'|%1|
'|%1| " The current setting is "
|$fortPersistence|
)))
```

set naglink messages

----- The messages Option -----

Description: show NAGLink messages

The messages option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

TPDHERE: this is the same as)set nag mes on

— **naglinkmessages** —

```
(|messages|
 "show NAGLink messages"
 |interpreter|
 LITERALS
|$nagMessages|
(|on| |off|)
 |on|)
```

set naglink double

----- The double Option -----

Description: enforce DOUBLE PRECISION ASPs

The double option may be followed by any one of the following:

```
-> on
    off
```

The current setting is indicated.

defvar \$nagEnforceDouble

— initvars —

```
(defvar |$nagEnforceDouble| t "enforce DOUBLE PRECISION ASPs")
```

— naglinkdouble —

```
(|double|
 "enforce DOUBLE PRECISION ASPs"
 |interpreter|
 LITERALS
 |$nagEnforceDouble|
 (|on| |off|)
 |on|)
```

26.46 set output

The result of the **)set output** command is:

Variable	Description	Current Value
abbreviate	abbreviate type names	off
algebra	display output in algebraic form	On:CONSOLE
characters	choose special output character set	plain
fortran	create output in FORTRAN format	Off:CONSOLE
fraction	how fractions are formatted	vertical
html	create output in HTML style	Off:CONSOLE
length	line length of output displays	77
mathml	create output in MathML style	Off:CONSOLE
openmath	create output in OpenMath style	Off:CONSOLE

<code>script</code>	display output in SCRIPT formula format	Off:CONSOLE
<code>scripts</code>	show subscripts,... linearly	off
<code>showeditor</code>	view output of)show in editor	off
<code>tex</code>	create output in TeX style	Off:CONSOLE

Since the output option has a bunch of sub-options each suboption is defined within the output structure.

— **output** —

```
(|output|
  "view and set some output options"
  |interpreter|
  TREE
  |novar|
  (
    \getchunk{outputabbreviate}
    \getchunk{outputalgebra}
    \getchunk{outputcharacters}
    \getchunk{outputfortran}
    \getchunk{outputfraction}
    \getchunk{outputhtml}
    \getchunk{outputlength}
    \getchunk{outputmathml}
    \getchunk{outputopenmath}
    \getchunk{outputscript}
    \getchunk{outputscripts}
    \getchunk{outputshoweditor}
    \getchunk{outputtex}
  ))
```

set output abbreviate

----- The abbreviate Option -----

Description: abbreviate type names

The abbreviate option may be followed by any one of the following:

on
-> off

The current setting is indicated.

defvar \$abbreviateTypes

— initvars —

```
(defvar |$abbreviateTypes| nil "abbreviate type names")
```

— outputabbreviate —

```
(|abbreviate|
 "abbreviate type names"
 |interpreter|
 LITERALS
 |$abbreviateTypes|
 (|on| |off|)
 |off|)
```

set output algebra

----- The algebra Option -----

Description: display output in algebraic form

)set output algebra is used to tell AXIOM to turn algebra-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output algebra <arg>

where arg can be one of

on	turn algebra printing on (default state)
off	turn algebra printing off
console	send algebra output to screen (default state)
fp<.fe>	send algebra output to file with file prefix fp and file extension .fe. If not given, .fe defaults to .spout.

If you wish to send the output to a file, you may need to issue this command twice: once with on and once with the file name. For example, to send algebra output to the file polymer.spout, issue the two commands


```

)set output algebra on
)set output algebra polymer

```

The output is placed in the directory from which you invoked AXIOM or the one you set with the `)cd` system command.
 The current setting is: On:CONSOLE

defvar \$algebraFormat

— initvars —

```
(defvar |$algebraFormat| t "display output in algebraic form ")
```

—————

defvar \$algebraOutputFile

— initvars —

```
(defvar |$algebraOutputFile| "CONSOLE"
  "where algebra printing goes (enter {\em console} or a pathname)?")
```

—————

— outputalgebra —

```
(|algebra|
  "display output in algebraic form "
  |interpreter|
  FUNCTION
  |setOutputAlgebra|
  (("display output in algebraic form "
    LITERALS
    |$algebraFormat|
    (|off| |on|)
    |on|)
    (break $algebraFormat)
    ("where algebra printing goes (enter {\em console} or a pathname)?"
     FILENAME
     |$algebraOutputFile|
     |chkOutputFileName|
     "console"))
  NIL)
```

defvar \$algebraOutputStream

— initvars —

```
(defvar |$algebraOutputStream| *standard-output*)
```

defun setOutputAlgebra

```
[defiostream p1168]
[concat p1237]
[describeSetOutputAlgebra p1037]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$algebraOutputStream p1035]
[$algebraOutputFile p1034]
[$filep p??]
[$algebraFormat p1034]
```

— defun setOutputAlgebra —

```
(defun |setOutputAlgebra| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$algebraOutputStream| |$algebraOutputFile| $filep
      |$algebraFormat|))
    (cond
      ((eq arg '|%initialize%|)
       (setq |$algebraOutputStream|
         (defiostream '((mode . output) (device . console)) 255 0))
       (setq |$algebraOutputFile| "CONSOLE")
       (setq |$algebraFormat| t))
```



```

                                (progn
                                  (setq fm (qcar tmp2))
                                  t))))))
(when (setq ptype (|pathnameType| fn))
  (setq fn (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
  (setq ft ptype))
(unless fm (setq fm 'a))
(setq filename ($filep fn ft fm))
(cond
  ((null filename)
   (|sayKeyedMsg|
    "It is not possible to open or create a file called %1 %2 %3 ."
    (list fn ft fm)))
  ((setq teststream (make-outstream filename 255 0))
   (shut |$algebraOutputStream|)
   (setq |$algebraOutputStream| teststream)
   (setq |$algebraOutputFile| (|object2String| filename))
   (|sayKeyedMsg|
    "%1 output will be written to file %2 ."
    (list "Algebra" |$algebraOutputFile|)))
  (t (|sayKeyedMsg|
      "It is not possible to open or create a file called %1 %2 %3 ."
      (list fn ft fm)))))
(t
 (|sayKeyedMsg| "Your argument list is not valid." nil)
 (|describeSetOutputAlgebra|))))))

```

defun describeSetOutputAlgebra

```

[|sayBrightly| p??]
[|setOutputAlgebra| p1035]

```

— defun describeSetOutputAlgebra —

```

(defun |describeSetOutputAlgebra| ()
  (|sayBrightly| (list
    " )set output algebra "
    "is used to tell AXIOM to turn algebra-style output"
    '|%1| "printing on and off, and where to place the output. By default, the"
    '|%1| "destination for the output is the screen but printing is turned off."
    '|%1|
    '|%1| "Syntax:   )set output algebra <arg>"
    '|%1| "   where arg can be one of"
    '|%1| "   on      turn algebra printing on (default state)"
    '|%1| "   off     turn algebra printing off"
  ))

```

```
'|%l| " console      send algebra output to screen (default state)"
'|%l| " fp<.fe>      send algebra output to file with file prefix fp"
'|%l|
"                and file extension .fe. If not given, .fe defaults to .spout."
'|%l|
'|%l|
"If you wish to send the output to a file, you may need to issue this command"
'|%l| "twice: once with"
" on and once with the file name. For example, to send"
'|%l| "algebra output to the file polymer.spout, issue the two commands"
'|%l|
'|%l| " )set output algebra on"
'|%l| " )set output algebra polymer"
'|%l|
'|%l| "The output is placed in the directory from which you invoked AXIOM or"
'|%l| "the one you set with the )cd system command."
'|%l| "The current setting is: "
(|setOutputAlgebra| '|%display%|)
))
```

set output characters

----- The characters Option -----

Description: choose special output character set

The characters option may be followed by any one of the following:

```
default
-> plain
```

The current setting is indicated. This option determines the special characters used for algebraic output. This is what the current choice of special characters looks like:

ulc is shown as +	urc is shown as +
llc is shown as +	lrc is shown as +
vbar is shown as	hbar is shown as -
quad is shown as ?	lbrk is shown as [
rbrk is shown as]	lbrc is shown as {
rbrc is shown as }	ttee is shown as +
btee is shown as +	rtee is shown as +
ltee is shown as +	ctee is shown as +
bslash is shown as \	

— **outputcharacters** —

```
(|characters|
 "choose special output character set "
 |interpreter|
 FUNCTION
 |setOutputCharacters|
 NIL
 |htSetOutputCharacters|)
```

defun setOutputCharacters

```
[sayMessage p??]
[bright p??]
[sayBrightly p??]
[concat p1237]
[pname p1235]
[specialChar p1166]
[sayAsManyPerLineAsPossible p??]
[qcdr p??]
[qcar p??]
[downcase p1272]
[setOutputCharacters p1039]
[$specialCharacters p1165]
[$plainRTspecialCharacters p1164]
[$RTspecialCharacters p1164]
[$specialCharacterAlist p1165]
```

— **defun setOutputCharacters** —

```
(defun |setOutputCharacters| (arg)
 (let (current char s l fn)
 (declare (special |$specialCharacters| |$plainRTspecialCharacters|
 |$RTspecialCharacters| |$specialCharacterAlist|))
 (if (eq arg '|%initialize%|)
 (setq |$specialCharacters| |$plainRTspecialCharacters|)
 (progn
 (setq current
 (cond
 ((eq |$specialCharacters| |$RTspecialCharacters|) "default")
 ((eq |$specialCharacters| |$plainRTspecialCharacters|) "plain")
 (t "unknown"))))
 (cond
```

```

((eq arg '|%display%|) current)
((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
 (format t
  " The characters option may be followed by any one ~
    of the following:~%~%"
  (dolist (name '("default" "plain"))
    (if (string= (string current) name)
      (|sayBrightly| '(" ->" ,@(|bright| name)))
      (|sayBrightly| (list " " name))))
  (terpri)
  (format t
   " The current setting is indicated within the list. ~
     This option determines ~% the special characters used ~
     for algebraic output. This is what the~% current choice of ~
     special characters looks like:~%"
   (do ((t1 |$specialCharacterAlist| (CDR t1)) (t2 nil))
     ((or (atom t1)
          (progn (setq t2 (car t1)) nil)
          (progn (progn (setq char (car t2)) t2) nil)) nil)
    (setq s
     (concat " " (pname char) " is shown as "
      (pname (|specialChar| char))))
    (setq l (cons s l)))
   (|sayAsManyPerLineAsPossible| (reverse l)))
  (and (consp arg)
    (eq (qcdr arg) NIL)
    (progn (setq fn (qcar arg)) t)
    (setq fn (downcase fn))))
  (cond
   ((eq fn '|default|)
    (setq |$specialCharacters| |$RTspecialCharacters|))
   ((eq fn '|plain|)
    (setq |$specialCharacters| |$plainRTspecialCharacters|))
   (t (|setOutputCharacters| nil))))
  (t (|setOutputCharacters| nil))))))

```

set output fortran

----- The fortran Option -----

Description: create output in FORTRAN format

)set output fortran is used to tell AXIOM to turn FORTRAN-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Also See: `)set fortran`

Syntax: `)set output fortran <arg>`
 where `arg` can be one of
`on` turn FORTRAN printing on
`off` turn FORTRAN printing off (default state)
`console` send FORTRAN output to screen (default state)
`fp<.fe>` send FORTRAN output to file with file prefix
`fp` and file extension `.fe`. If not given,
`.fe` defaults to `.sfort`.

If you wish to send the output to a file, you must issue this command twice: once with `on` and once with the file name. For example, to send FORTRAN output to the file `polymer.sfort`, issue the two commands

```
)set output fortran on
)set output fortran polymer
```

The output is placed in the directory from which you invoked AXIOM or the one you set with the `)cd` system command. The current setting is: `Off:CONSOLE`

defvar \$fortranFormat

— initvars —

```
(defvar |$fortranFormat| nil "create output in FORTRAN format ")
```

—————

defvar \$fortranOutputFile

— initvars —

```
(defvar |$fortranOutputFile| "CONSOLE"
  "where FORTRAN output goes (enter {\em console} or a a pathname)")
```

—————

— outputfortran —


```
(|fortran|
  "create output in FORTRAN format "
  |interpreter|
  FUNCTION
  |setOutputFortran|
  ("create output in FORTRAN format "
    LITERALS
    |$fortranFormat|
    (|off| |on|)
    |off|)
  (|break| |$fortranFormat|)
  ("where FORTRAN output goes (enter {\em console} or a a pathname)"
    FILENAME
    |$fortranOutputFile|
    |chkOutputFileName|
    "console"))
  NIL)
```

defun setOutputFortran

```
[defiostream p1168]
[concat p1237]
[describeSetOutputFortran p1045]
[upcase p1272]
[qcdr p??]
[qcar p??]
[member p1238]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[makeStream p1169]
[object2String p??]
[$fortranOutputStream p??]
[$fortranOutputFile p1041]
[$filep p??]
[$fortranFormat p1041]
```

— defun setOutputFortran —

```
(defun |setOutputFortran| (arg)
  (let (label APPEND quiet tmp1 tmp2 ptype fn ft fm filename teststream)
```

```

(declare (special |$fortranOutputStream| |$fortranOutputFile| $filep
  |$fortranFormat|))
(cond
  ((eq arg '|%initialize%|)
    (setq |$fortranOutputStream|
      (defiostream '((mode . output) (device . console)) 255 0))
    (setq |$fortranOutputFile| "CONSOLE")
    (setq |$fortranFormat| nil))
  ((eq arg '|%display%|)
    (if |$fortranFormat|
      (setq label "On:")
      (setq label "Off:"))
    (concat label |$fortranOutputFile|))
  ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
    (|describeSetOutputFortran|))
  (t
    (DO ()
      ((null (and (listp arg)
        (|member| (upcase (car arg)) '(append quiet))))
        nil)
      (cond
        ((eq (upcase (car arg)) 'append) (setq append t))
        ((eq (upcase (car arg)) 'quiet) (setq quiet t))
        (t nil))
      (setq arg (cdr arg)))
    (cond
      ((and (consp arg)
        (eq (qcdr arg) nil)
        (progn (setq fn (qcar arg)) t)
        (|member| fn '(Y N YE YES NO O ON OF OFF CONSOLE
          |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|)))
        'ok|)
      (t (setq arg (list fn '|sfort|))))
    (cond
      ((and (consp arg) (eq (qcdr arg) nil) (progn (setq fn (qcar arg)) t))
        (cond
          ((|member| (upcase fn) '(y n ye o of))
            (|sayKeyedMsg|
              (format nil
                "To toggle %1 printing on and off, specify %1 )set output %2 ~
                yes/no/on/off %1 Yes, no, on and off cannot be abbreviated.")
              '(fortran |fortran|)))
            ((|member| (upcase fn) '(no off)) (setq |$fortranFormat| nil))
            ((|member| (upcase fn) '(yes on)) (setq |$fortranFormat| t))
            ((eq (upcase fn) 'console)
              (shut |$fortranOutputStream|)
              (setq |$fortranOutputStream|
                (defiostream '((mode . output) (device . console)) 255 0))
              (setq |$fortranOutputFile| "CONSOLE"))))
          ((or

```

```

    (and (consp arg)
      (progn
        (setq fn (qcar arg))
        (setq tmp1 (qcdr arg))
        (and (consp tmp1)
          (eq (qcdr tmp1) nil)
          (progn (setq ft (qcar tmp1)) t))))
    (and (consp arg)
      (progn
        (setq fn (qcar arg))
        (setq tmp1 (qcdr arg))
        (and (consp tmp1)
          (progn
            (setq ft (qcar tmp1))
            (setq tmp2 (qcdr tmp1))
            (and (consp tmp2)
              (eq (qcdr tmp2) nil)
              (progn (setq fm (qcar tmp2)) t))))))
    (when (setq ptype (|pathnameType| fn))
      (setq fn (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
      (setq ft ptype))
    (unless fm (setq fm 'a))
    (setq filename ($filep fn ft fm))
    (cond
      ((null filename)
        (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 ."
          (list fn ft fm)))
      ((setq teststream (|makeStream| append filename 255 0))
        (SHUT |$fortranOutputStream|)
        (setq |$fortranOutputStream| teststream)
        (setq |$fortranOutputFile| (|object2String| filename))
        (unless quiet
          (|sayKeyedMsg|
            "%1 output will be written to file %2 ."
            (list 'fortran |$fortranOutputFile|))))
      ((null quiet)
        (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 ."
          (list fn ft fm)))
      (t nil)))
    (t
      (unless quiet (|sayKeyedMsg| "Your argument list is not valid." nil))
      (|describeSetOutputFortran|))))))

```

defun describeSetOutputFortran

```
[sayBrightly p??]
[setOutputFortran p1042]
```

— defun describeSetOutputFortran —

```
(defun |describeSetOutputFortran| ()
  (|sayBrightly| (list
    " )set output fortran"
    " is used to tell AXIOM to turn FORTRAN-style output "
    '|%1| "printing on and off, and where to place the output. By default, the"
    '|%1| "destination for the output is the screen but printing is turned off."
    '|%1|
    '|%1| "Also See: )set fortran"
    '|%1|
    '|%1| "Syntax: )set output fortran <arg>"
    '|%1| " where arg can be one of"
    '|%1| " on turn FORTRAN printing on"
    '|%1| " off turn FORTRAN printing off (default state)"
    '|%1| " console send FORTRAN output to screen (default state)"
    '|%1|
    " fp<.fe> send FORTRAN output to file with file prefix fp and file"
    '|%1| " extension .fe. If not given, .fe defaults to .sfort."
    '|%1|
    '|%1| "If you wish to send the output to a file, you must issue this command"
    '|%1| "twice: once with"
    " on and once with the file name. For example, to send "
    '|%1| "FORTRAN output to the file polymer.sfort, issue the two commands"
    '|%1|
    '|%1| " )set output fortran on"
    '|%1| " )set output fortran polymer"
    '|%1|
    '|%1| "The output is placed in the directory from which you invoked AXIOM or"
    '|%1| "the one you set with the )cd system command."
    '|%1| "The current setting is: "
    (|setOutputFortran| '%display%)
  )))
```

set output fraction

----- The fraction Option -----

Description: how fractions are formatted

The fraction option may be followed by any one of the following:

```
-> vertical
    horizontal
```

The current setting is indicated.

defvar \$fractionDisplayType

— initvars —

```
(defvar |$fractionDisplayType| '|vertical| "how fractions are formatted")
```

— outputfraction —

```
(|fraction|
 "how fractions are formatted"
 |interpreter|
 LITERALS
 |$fractionDisplayType|
 (|vertical| |horizontal|)
 |vertical|)
```

set output html

----- The html Option -----

Description: create output in html style

)set output html is used to tell AXIOM to turn html-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output html <arg>

where arg can be one of

```
on          turn html printing on
off         turn html printing off (default state)
console     send html output to screen (default state)
```

```
fp<.fe>      send html output to file with file prefix fp
              and file extension .fe. If not given,
              .fe defaults to .html.
```

If you wish to send the output to a file, you must issue this command twice: once with `on` and once with the file name. For example, to send MathML output to the file `polymer.html`, issue the two commands

```
)set output html on
)set output html polymer
```

The output is placed in the directory from which you invoked Axiom or the one you set with the `)cd` system command. The current setting is: `Off:CONSOLE`

defvar \$htmlFormat

— initvars —

```
(defvar |$htmlFormat| nil "create output in HTML format ")
```

—————

defvar \$htmlOutputFile

— initvars —

```
(defvar |$htmlOutputFile| "CONSOLE"
  "where HTML output goes (enter {\em console} or a pathname)")
```

—————

— outputhtml —

```
(|html|
  "create output in HTML style "
  |interpreter|
  FUNCTION
  |setOutputHtml|
  (("create output in HTML format "
    LITERALS
    |$htmlFormat|
```

```

(|off| |on|)
|off|)
(|break| |$htmlFormat|)
("where HTML output goes (enter {\em console} or a pathname)"
 FILENAME
 |$htmlOutputFile|
 |chkOutputFileName|
 "console"))
NIL)

```

defun setOutputHtml

```

[defiostream p1168]
[concat p1237]
[describeSetOutputHtml p1050]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$htmlOutputStream p??]
[$htmlOutputFile p1047]
[$htmlFormat p1047]
[$filep p??]

```

— defun setOutputHtml —

```

(defun |setOutputHtml| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$htmlOutputStream| |$htmlOutputFile| |$htmlFormat|
                      $filep))
    (cond
      ((eq arg '|%initialize%|)
       (setq |$htmlOutputStream|
             (defiostream '((mode . output) (device . console)) 255 0))
       (setq |$htmlOutputFile| "CONSOLE")
       (setq |$htmlFormat| nil))
    )
  )

```

```

((eq arg '|%display%|)
 (if |$htmlFormat|
  (setq label "On:")
  (setq label "Off:"))
 (concat label |$htmlOutputFile|))
((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
 (|describeSetOutputHtml|))
(t
 (cond
  ((and (consp arg)
        (eq (qcdr arg) nil)
        (progn (setq fn (qcar arg)) t)
        (|member| fn '(y n ye yes no o on of off console
                      |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|)))
   '|ok|)
  (t (setq arg (list fn '|smml|)))))
(cond
 ((and (consp arg)
        (eq (qcdr arg) nil)
        (progn (setq fn (qcar arg)) t))
  (cond
   ((|member| (upcase fn) '(y n ye o of))
    (|sayKeyedMsg|
     (format nil
              "To toggle %1 printing on and off, specify %1 )set output %2 ~
              yes/no/on/off %1 Yes, no, on and off cannot be abbreviated.")
            '|HTML| |html|)))
   ((|member| (upcase fn) '(no off)) (setq |$htmlFormat| nil))
   ((|member| (upcase fn) '(yes on)) (setq |$htmlFormat| t))
   ((eq (upcase fn) 'console)
    (shut |$htmlOutputStream|)
    (setq |$htmlOutputStream|
          (defiostream '((mode . output) (device . console)) 255 0))
    (setq |$htmlOutputFile| "CONSOLE")))))
((or
 (and (consp arg)
      (progn
       (setq fn (qcar arg))
       (setq tmp1 (qcdr arg))
       (and (consp tmp1)
            (eq (qcdr tmp1) nil)
            (progn (setq ft (qcar tmp1)) t))))
 (and (consp arg)
      (progn (setq fn (qcar arg))
              (setq tmp1 (qcdr arg))
              (and (consp tmp1)
                   (progn
                    (setq ft (qcar tmp1))
                    (setq tmp2 (qcdr tmp1))
                    (and (consp tmp2)

```



```

                                (eq (qcdr tmp2) nil)
                                (progn
                                  (setq fm (qcar tmp2))
                                  t))))))
(when (setq ptype (|pathnameType| fn))
  (setq fn
    (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
  (setq ft ptype))
(unless fm (setq fm 'a))
(setq filename ($filep fn ft fm))
(cond
  ((null filename)
   (|sayKeyedMsg|
    "It is not possible to open or create a file called %1 %2 %3 ."
    (list fn ft fm)))
  ((setq teststream (make-outstream filename 255 0))
   (shut |$htmlOutputStream|)
   (setq |$htmlOutputStream| teststream)
   (setq |$htmlOutputFile| (|object2String| filename))
   (|sayKeyedMsg|
    "%1 output will be written to file %2 ."
    (list "HTML" |$htmlOutputFile|)))
  (t (|sayKeyedMsg|
      "It is not possible to open or create a file called %1 %2 %3 ."
      (list fn ft fm)))))
(t
  (|sayKeyedMsg| "Your argument list is not valid." nil)
  (|describeSetOutputHtml|))))))

```

defun describeSetOutputHtml

[sayBrightly p??]
 [setOutputHtml p1048]

— defun describeSetOutputHtml —

```

(defun |describeSetOutputHtml| ()
  (|sayBrightly| (LIST
    " )set output html "
    "is used to tell AXIOM to turn HTML-style output"
    '|%1| "printing on and off, and where to place the output.  By default, the"
    '|%1| "destination for the output is the screen but printing is turned off."
    '|%1|
    '|%1| "Syntax:   )set output html <arg>"
    '|%1| "      where arg can be one of"

```

```
'|%l| " on          turn HTML printing on"
'|%l| " off         turn HTML printing off (default state)"
'|%l| " console     send HTML output to screen (default state)"
'|%l| " fp<.fe>      send HTML output to file with file prefix fp and file"
'|%l| "              extension .fe. If not given, .fe defaults to .stex."
'|%l|
'|%l| "If you wish to send the output to a file, you must issue this command"
'|%l| "twice: once with "
"on and once with the file name. For example, to send"
'|%l| "HTML output to the file polymer.smml, issue the two commands"
'|%l|
'|%l| " )set output html on"
'|%l| " )set output html polymer"
'|%l|
'|%l| "The output is placed in the directory from which you invoked AXIOM or"
'|%l| "the one you set with the )cd system command."
'|%l| "The current setting is: "
(|setOutputHtml| '%display%)
))
```

set output length

----- The length Option -----

Description: line length of output displays

The length option may be followed by an integer in the range
10 to 245 inclusive. The current setting is 77

defvar \$margin

— initvars —

```
(defvar $margin 3)
```

defvar \$linelength

— initvars —

```
(defvar $linelength 77 "line length of output displays")
```

— outputlength —

```
(|length|
 "line length of output displays"
 |interpreter|
 INTEGER
 $LINELENGTH
 (10 245)
 77)
```

set output mathml

----- The mathml Option -----

Description: create output in MathML style

)set output mathml is used to tell AXIOM to turn MathML-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output mathml <arg>
 where arg can be one of
 on turn MathML printing on
 off turn MathML printing off (default state)
 console send MathML output to screen (default state)
 fp<.fe> send MathML output to file with file prefix fp
 and file extension .fe. If not given,
 .fe defaults to .smml.

If you wish to send the output to a file, you must issue this command twice: once with on and once with the file name. For example, to send MathML output to the file polymer.smml, issue the two commands

```
)set output mathml on
)set output mathml polymer
```

The output is placed in the directory from which you invoked AXIOM or the one you set with the)cd system command.

The current setting is: Off:CONSOLE

defvar \$mathmlFormat

— initvars —

```
(defvar |$mathmlFormat| nil "create output in MathML format ")
```

—————

defvar \$mathmlOutputFile

— initvars —

```
(defvar |$mathmlOutputFile| "CONSOLE"
  "where MathML output goes (enter {\em console} or a pathname)")
```

—————

— outputmathml —

```
(|mathml|
  "create output in MathML style "
  |interpreter|
  FUNCTION
  |setOutputMathml|
  (("create output in MathML format "
    LITERALS
    |$mathmlFormat|
    (|off| |on|)
    |off|)
  (|break| |$mathmlFormat|)
  ("where MathML output goes (enter {\em console} or a pathname)"
   FILENAME
   |$mathmlOutputFile|
   |chkOutputFileName|
   "console"))
  NIL)
```

—————

defun setOutputMathml

```

[defiostream p1168]
[concat p1237]
[describeSetOutputMathml p1056]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$mathmlOutputStream p??]
[$mathmlOutputFile p1053]
[$mathmlFormat p1053]
[$filep p??]

```

— **defun setOutputMathml** —

```

(defun |setOutputMathml| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$mathmlOutputStream| |$mathmlOutputFile| |$mathmlFormat|
                      $filep))
    (cond
      ((eq arg '|%initialize%|)
       (setq |$mathmlOutputStream|
             (defiostream '((mode . output) (device . console)) 255 0))
       (setq |$mathmlOutputFile| "CONSOLE")
       (setq |$mathmlFormat| nil))
      ((eq arg '|%display%|)
       (if |$mathmlFormat|
         (setq label "On:")
         (setq label "Off:"))
       (concat label |$mathmlOutputFile|))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '??))
       (|describeSetOutputMathml|))
      (t
       (cond
         ((and (consp arg)
              (eq (qcdr arg) nil)
              (progn (setq fn (qcar arg)) t)
              (|member| fn '(y n ye yes no o on of off console)
                        |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|))))

```

```

      'ok|)
    (t (setq arg (list fn '|smml|))))
  (cond
    ((and (consp arg)
          (eq (qcdr arg) nil)
          (progn (setq fn (qcar arg)) t))
     (cond
       ((|member| (upcase fn) '(y n ye o of))
        (|sayKeyedMsg|
         (format nil
                  "To toggle %1 printing on and off, specify %1 )set output %2 ~
                  yes/no/on/off %1 Yes, no, on and off cannot be abbreviated.")
         '|MathML| |mathml|)))
       ((|member| (upcase fn) '(no off)) (setq |$mathmlFormat| nil))
       ((|member| (upcase fn) '(yes on)) (setq |$mathmlFormat| t))
       ((eq (upcase fn) 'console)
        (shut |$mathmlOutputStream|)
        (setq |$mathmlOutputStream|
              (defiostream '((mode . output) (device . console)) 255 0))
        (setq |$mathmlOutputFile| "CONSOLE")))))
    ((or
      (and (consp arg)
           (progn
            (setq fn (qcar arg))
            (setq tmp1 (qcdr arg))
            (and (consp tmp1)
                  (eq (qcdr tmp1) nil)
                  (progn (setq ft (qcar tmp1)) t))))
      (and (consp arg)
           (progn (setq fn (qcar arg))
                  (setq tmp1 (qcdr arg))
                  (and (consp tmp1)
                       (progn
                        (setq ft (qcar tmp1))
                        (setq tmp2 (qcdr tmp1))
                        (and (consp tmp2)
                              (eq (qcdr tmp2) nil)
                              (progn
                               (setq fm (qcar tmp2))
                               t))))))))
      (when (setq ptype (|pathnameType| fn))
        (setq fn
              (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
        (setq ft ptype))
      (unless fm (setq fm 'a))
      (setq filename ($filep fn ft fm))
      (cond
        ((null filename)
         (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 .")

```

```

        (list fn ft fm)))
      ((setq teststream (make-outstream filename 255 0))
       (shut |$mathmlOutputStream|)
       (setq |$mathmlOutputStream| teststream)
       (setq |$mathmlOutputFile| (|object2String| filename))
       (|sayKeyedMsg|
        "%1 output will be written to file %2 ."
        (list "MathML" |$mathmlOutputFile|)))
      (t (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 ."
          (list fn ft fm))))))
    (t
     (|sayKeyedMsg| "Your argument list is not valid." nil)
     (|describeSetOutputMathml|))))))

```

defun describeSetOutputMathml

```

[sayBrightly p??]
[setOutputMathml p1054]

```

— defun describeSetOutputMathml —

```

(defun |describeSetOutputMathml| ()
  (|sayBrightly| (LIST
    " )set output mathml "
    "is used to tell AXIOM to turn MathML-style output"
    '|%1| "printing on and off, and where to place the output. By default, the"
    '|%1| "destination for the output is the screen but printing is turned off."
    '|%1|
    '|%1| "Syntax:   )set output mathml <arg>"
    '|%1| "   where arg can be one of"
    '|%1| "   on      turn MathML printing on"
    '|%1| "   off     turn MathML printing off (default state)"
    '|%1| "   console  send MathML output to screen (default state)"
    '|%1| "   fp<.fe>   send MathML output to file with file prefix fp and file"
    '|%1| "               extension .fe. If not given, .fe defaults to .stex."
    '|%1|
    '|%1| "If you wish to send the output to a file, you must issue this command"
    '|%1| "twice: once with"
    " on and once with the file name. For example, to send"
    '|%1| "MathML output to the file polymer.smml, issue the two commands"
    '|%1|
    '|%1| "   )set output mathml on"
    '|%1| "   )set output mathml polymer"
    '|%1|

```

```
'|%l| "The output is placed in the directory from which you invoked AXIOM or"
'|%l| "the one you set with the )cd system command."
'|%l| "The current setting is: "
(|setOutputMathml| '|%display%|)
)))
```

set output openmath

----- The openmath Option -----

Description: create output in OpenMath style

)set output tex is used to tell AXIOM to turn OpenMath output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output tex <arg>
 where arg can be one of
 on turn OpenMath printing on
 off turn OpenMath printing off (default state)
 console send OpenMath output to screen (default state)
 fp<.fe> send OpenMath output to file with file prefix fp
 and file extension .fe. If not given,
 .fe defaults to .sopen.

If you wish to send the output to a file, you must issue this command twice: once with on and once with the file name. For example, to send OpenMath output to the file polymer.sopen, issue the two commands

```
)set output openmath on
)set output openmath polymer
```

The output is placed in the directory from which you invoked AXIOM or the one you set with the)cd system command.
 The current setting is: Off:CONSOLE

defvar \$openMathFormat

— initvars —

```
(defvar |$openMathFormat| nil "create output in OpenMath format ")
```

defvar \$openMathOutputFile

— initvars —

```
(defvar |$openMathOutputFile| "CONSOLE"
  "where TeX output goes (enter {\em console} or a pathname)")
```

— outputopenmath —

```
(|openmath|
 "create output in OpenMath style "
 |interpreter|
 FUNCTION
 |setOutputOpenMath|
 ("create output in OpenMath format "
  LITERALS
  |$openMathFormat|
  (|off| |on|)
  |off|)
 (|break| |$openMathFormat|)
 ("where TeX output goes (enter {\em console} or a pathname)"
  FILENAME
  |$openMathOutputFile|
  |chkOutputFileName|
  "console"))
NIL)
```

defun setOutputOpenMath

```
[defiostream p1168]
[concat p1237]
[describeSetOutputOpenMath p1061]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
```

```
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$openMathOutputStream p??]
[$openMathFormat p1057]
[$filep p??]
[$openMathOutputFile p1058]
```

— defun setOutputOpenMath —

```
(defun |setOutputOpenMath| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$openMathOutputStream| |$openMathFormat| $filep
      |$openMathOutputFile|))
    (cond
      ((eq arg '|%initialize%|)
        (setq |$openMathOutputStream|
          (defiostream '((mode . output) (device . console)) 255 0))
        (setq |$openMathOutputFile| "CONSOLE")
        (setq |$openMathFormat| NIL))
      ((eq arg '|%display%|)
        (if |$openMathFormat|
          (setq label "On:")
          (setq label "Off:"))
        (concat label |$openMathOutputFile|))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
        (|describeSetOutputOpenMath|))
      (t
        (cond
          ((and (consp arg)
            (eq (qcdr arg) nil)
            (progn (setq fn (qcar arg)) t)
            (|member| fn '(y n ye yes no o on of off console
              |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|))))
            '|ok|)
          (t (setq arg (list fn '|som|))))
        (cond
          ((and (consp arg)
            (eq (qcdr arg) nil)
            (progn (setq fn (qcar arg)) t))
            (cond
              ((|member| (upcase fn) '(y n ye o of))
                (|sayKeyedMsg|
                  (format nil
                    "To toggle %1 printing on and off, specify %1 )set output %2 ~
```

```

        yes/no/on/off %1 Yes, no, on and off cannot be abbreviated.")
      '(|OpenMath| |openmath|)))
    ((|member| (upcase fn) '(no off)) (setq |$openMathFormat| nil))
    ((|member| (upcase fn) '(yes on)) (setq |$openMathFormat| t))
    ((eq (upcase fn) 'console)
     (shut |$openMathOutputStream|)
     (setq |$openMathOutputStream|
      (defiostream '((mode . output) (device . console)) 255 0))
     (setq |$openMathOutputFile| "CONSOLE"))))
  ((or
   (and (consp arg)
        (progn (setq fn (qcar arg))
                 (setq tmp1 (qcdr arg))
                 (and (consp tmp1)
                      (eq (qcdr tmp1) nil)
                      (progn (setq ft (qcar tmp1)) t))))
   (and (consp arg)
        (progn
         (setq fn (qcar arg))
         (setq tmp1 (qcdr arg))
         (and (consp tmp1)
              (progn (setq ft (qcar tmp1))
                     (setq tmp2 (qcdr tmp1))
                     (and (consp tmp2)
                          (eq (qcdr tmp2) nil)
                          (progn (setq fm (qcar tmp2)) t)))))))
   (when (setq ptype (|pathnameType| fn))
     (setq fn (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
     (setq ft ptype))
   (unless fm (setq fm 'a))
   (setq filename ($filep fn ft fm))
   (cond
    ((null filename)
     (|sayKeyedMsg|
      "It is not possible to open or create a file called %1 %2 %3 ."
      (list fn ft fm)))
    ((setq teststream (make-outstream filename 255 0))
     (shut |$openMathOutputStream|)
     (setq |$openMathOutputStream| teststream)
     (setq |$openMathOutputFile| (|object2String| filename))
     (|sayKeyedMsg|
      "%1 output will be written to file %2 ."
      (list "OpenMath" |$openMathOutputFile|)))
    (t
     (|sayKeyedMsg|
      "It is not possible to open or create a file called %1 %2 %3 ."
      (list fn ft fm)))))
  (t
   (|sayKeyedMsg| "Your argument list is not valid." nil)
   (|describeSetOutputOpenMath|))))))

```

defun describeSetOutputOpenMath

[sayBrightly p??]
 [setOutputOpenMath p1058]

— defun describeSetOutputOpenMath —

```
(defun |describeSetOutputOpenMath| ()
  (|sayBrightly| (list
    " )set output openmath "
    "is used to tell AXIOM to turn OpenMath output"
    '|%1| "printing on and off, and where to place the output.  By default, the"
    '|%1| "destination for the output is the screen but printing is turned off."
    '|%1|
    '|%1| "Syntax:  )set output openmath <arg>"
    '|%1| "   where arg can be one of"
    '|%1| "   on           turn OpenMath printing on"
    '|%1| "   off          turn OpenMath printing off (default state)"
    '|%1| "   console      send OpenMath output to screen (default state)"
    '|%1|
    "   fp<.fe>      send OpenMath output to file with file prefix fp and file"
    '|%1| "                   extension .fe. If not given, .fe defaults to .som."
    '|%1|
    '|%1| "If you wish to send the output to a file, you must issue this command"
    '|%1| "twice: once with"
    "   on and once with the file name. For example, to send"
    '|%1| "OpenMath output to the file polymer.som, issue the two commands"
    '|%1|
    '|%1| "   )set output openmath on"
    '|%1| "   )set output openmath polymer"
    '|%1|
    '|%1| "The output is placed in the directory from which you invoked AXIOM or"
    '|%1| "the one you set with the )cd system command."
    '|%1| "The current setting is: "
    (|setOutputOpenMath| '|%display%|)
  )))
```

set output script

----- The script Option -----

Description: display output in SCRIPT formula format

)set output script is used to tell AXIOM to turn IBM Script formula-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output script <arg>
 where arg can be one of
 on turn IBM Script formula printing on
 off turn IBM Script formula printing off
 (default state)
 console send IBM Script formula output to screen
 (default state)
 fp<.fe> send IBM Script formula output to file with file
 prefix fp and file extension .fe. If not given,
 .fe defaults to .sform.

If you wish to send the output to a file, you must issue this command twice: once with on and once with the file name. For example, to send IBM Script formula output to the file polymer.sform, issue the two commands

```
)set output script on
)set output script polymer
```

The output is placed in the directory from which you invoked AXIOM or the one you set with the)cd system command. The current setting is: Off:CONSOLE

defvar \$formulaFormat

— initvars —

```
(defvar |$formulaFormat| nil "display output in SCRIPT format")
```

—

defvar \$formulaOutputFile

— initvars —

```
(defvar |$formulaOutputFile| "CONSOLE")
```

"where script output goes (enter {\em console} or a a pathname)")

— **outputscript** —

```
(|script|
  "display output in SCRIPT formula format "
  |interpreter|
  FUNCTION
  |setOutputFormula|
  (("display output in SCRIPT format "
    LITERALS
    |$formulaFormat|
    (|off| |on|)
    |off|)
  (|break| |$formulaFormat|)
  ("where script output goes (enter {\em console} or a a pathname)"
   FILENAME
   |$formulaOutputFile|
   |chkOutputFileName|
   "console"))
  NIL)
```

defun setOutputFormula

```
[defiostream p1168]
[concat p1237]
[describeSetOutputFormula p1065]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$formulaOutputStream p??]
[$formulaOutputFile p1062]
```

`[$filep p??]`

`[$formulaFormat p1062]`

— `defun setOutputFormula` —

```
(defun |setOutputFormula| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$formulaOutputStream| |$formulaOutputFile| $filep
      |$formulaFormat|))
    (cond
      ((eq arg '|%initialize%|)
        (setq |$formulaOutputStream|
          (defiostream '((mode . output) (device . console)) 255 0))
        (setq |$formulaOutputFile| "CONSOLE")
        (setq |$formulaFormat| nil))
      ((eq arg '|%display%|)
        (if |$formulaFormat|
          (setq label "On:")
          (setq label "Off:"))
        (concat label |$formulaOutputFile|))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
        (|describeSetOutputFormula|))
      (t
        (cond
          ((and (consp arg)
            (eq (qcdr arg) nil)
            (progn (setq fn (qcar arg)) t)
            (|member| fn '(y n ye yes no o on of off console
              |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|)))
            'ok|)
          (t (setq arg (list fn '|sform|))))
        (cond
          ((and (consp arg)
            (eq (qcdr arg) nil)
            (progn (setq fn (qcar arg)) t))
            (cond
              ((|member| (upcase fn) '(y n ye o of))
                (|sayKeyedMsg|
                  (format nil
                    "To toggle %1 printing on and off, specify %1 )set output %2 ~
                    yes/no/on/off %1 Yes, no, on and off cannot be abbreviated.")
                  '(|script| |script|)))
              ((|member| (upcase fn) '(no off)) (setq |$formulaFormat| nil))
              ((|member| (upcase fn) '(yes on)) (setq |$formulaFormat| t))
              ((eq (upcase fn) 'console)
                (SHUT |$formulaOutputStream|
                  (setq |$formulaOutputStream|
                    (defiostream '((mode . output) (device . console)) 255 0))
                    (setq |$formulaOutputFile| "CONSOLE"))))
              ((or
```

```

    (and (consp arg)
      (progn (setq fn (qcar arg))
        (setq tmp1 (qcdr arg))
        (and (consp tmp1)
          (eq (qcdr tmp1) nil)
          (progn (setq ft (qcar tmp1)) t))))
    (and (consp arg)
      (progn (setq fn (qcar arg))
        (setq tmp1 (qcdr arg))
        (and (consp tmp1)
          (progn (setq ft (qcar tmp1))
            (setq tmp2 (qcdr tmp1))
            (and (consp tmp2)
              (eq (qcdr tmp2) nil)
              (progn
                (setq fm (qcar tmp2)) t)))))))
    (if (setq ptype (|pathnameType| fn))
      (setq fn (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
      (setq ft ptype))
    (unless fm (setq fm 'a))
    (setq filename ($filep fn ft fm))
    (cond
      ((null filename)
        (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 ."
          (list fn ft fm)))
      ((setq teststream (make-outstream filename 255 0))
        (shut |$formulaOutputStream|)
        (setq |$formulaOutputStream| teststream)
        (setq |$formulaOutputFile| (|object2String| filename))
        (|sayKeyedMsg|
          "%1 output will be written to file %2 ."
          (list "IBM Script formula" |$formulaOutputFile| )))
      (t
        (|sayKeyedMsg|
          "It is not possible to open or create a file called %1 %2 %3 ."
          (list fn ft fm)))))
    (t
      (|sayKeyedMsg| "Your argument list is not valid." nil)
      (|describeSetOutputFormula|))))))

```

defun describeSetOutputFormula

[sayBrightly p??]
 [setOutputFormula p1063]

— defun describeSetOutputFormula —

```
(defun |describeSetOutputFormula| ()
  (|sayBrightly| (list
    " )set output script "
    "is used to tell AXIOM to turn IBM Script formula-style"
    '|%l|'
    "output printing on and off, and where to place the output. By default, the"
    '|%l|' "destination for the output is the screen but printing is turned off."
    '|%l|'
    '|%l|' "Syntax: )set output script <arg>"
    '|%l|' " where arg can be one of"
    '|%l|' " on turn IBM Script formula printing on"
    '|%l|' " off turn IBM Script formula printing off (default state)"
    '|%l|' " console send IBM Script formula output to screen (default state)"
    '|%l|'
    " fp<.fe> send IBM Script formula output to file with file prefix fp"
    '|%l|'
    " and file extension .fe. If not given, .fe defaults to .sform."
    '|%l|'
    '|%l|' "If you wish to send the output to a file, you must issue this command"
    '|%l|' "twice: once with"
    " on and once with the file name. For example, to send "
    '|%l|' "IBM Script formula output to the file polymer.sform,"
    " issue the two commands"
    '|%l|'
    '|%l|' " )set output script on"
    '|%l|' " )set output script polymer"
    '|%l|'
    '|%l|' "The output is placed in the directory from which you invoked AXIOM or"
    '|%l|' "the one you set with the )cd system command."
    '|%l|' "The current setting is: "
    (|setOutputFormula| '|%display%|')
  )))
```

set output scripts

----- The scripts Option -----

Description: show subscripts,... linearly

The scripts option may be followed by any one of the following:

```
yes
no
```

The current setting is indicated.

defvar \$linearFormatScripts

— initvars —

```
(defvar |$linearFormatScripts| nil "show subscripts,... linearly")
```

—————

— outputscripts —

```
(|scripts|
 "show subscripts,... linearly"
 |interpreter|
 LITERALS
 |$linearFormatScripts|
 (|on| |off|)
 |off|)
```

—————

set output showeditor

----- The showeditor Option -----

Description: view output of)show in editor

The showeditor option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$useEditorForShowOutput

— initvars —

```
(defvar |$useEditorForShowOutput| nil "view output of )show in editor")
```

— outputshoweditor —

```
(|showeditor|
 "view output of )show in editor"
 |interpreter|
 LITERALS
 |$useEditorForShowOutput|
 (|on| |off|)
 |off|)
```

set output tex

----- The tex Option -----

Description: create output in TeX style

)set output tex is used to tell AXIOM to turn TeX-style output printing on and off, and where to place the output. By default, the destination for the output is the screen but printing is turned off.

Syntax:)set output tex <arg>
 where arg can be one of
 on turn TeX printing on
 off turn TeX printing off (default state)
 console send TeX output to screen (default state)
 fp<.fe> send TeX output to file with file prefix fp
 and file extension .fe. If not given,
 .fe defaults to .stex.

If you wish to send the output to a file, you must issue this command twice: once with on and once with the file name. For example, to send TeX output to the file polymer.stex, issue the two commands

```
      )set output tex on
      )set output tex polymer
```

The output is placed in the directory from which you invoked AXIOM or the one you set with the)cd system command. The current setting is: Off:CONSOLE

defvar \$texFormat

— initvars —

```
(defvar |$texFormat| nil "create output in TeX format ")
```

—————

defvar \$texOutputFile

— initvars —

```
(defvar |$texOutputFile| "CONSOLE"
  "where TeX output goes (enter {\em console} or a pathname)")
```

—————

— outputtex —

```
(|tex|
  "create output in TeX style "
  |interpreter|
  FUNCTION
  |setOutputTex|
  ("create output in TeX format "
    LITERALS
    |$texFormat|
    (|off| |on|)
    |off|)
  (|break| |$texFormat|)
  ("where TeX output goes (enter {\em console} or a pathname)"
    FILENAME
    |$texOutputFile|
    |chkOutputFileName|
    "console"))
NIL)
```

—————

defun setOutputTex

```
[defiostream p1168]
[concat p1237]
```

```

[describeSetOutputTex p1072]
[qcdr p??]
[qcar p??]
[member p1238]
[upcase p1272]
[sayKeyedMsg p43]
[shut p1168]
[pathnameType p1231]
[pathnameDirectory p1232]
[pathnameName p1231]
[$filep p??]
[make-outstream p1167]
[object2String p??]
[$texOutputStream p??]
[$texOutputFile p1069]
[$texFormat p1069]
[$filep p??]

```

— defun setOutputTex —

```

(defun |setOutputTex| (arg)
  (let (label tmp1 tmp2 ptype fn ft fm filename teststream)
    (declare (special |$texOutputStream| |$texOutputFile| |$texFormat| $filep))
    (cond
      ((eq arg '|%initialize%|)
       (setq |$texOutputStream|
              (defiostream '((mode . output) (device . console)) 255 0))
       (setq |$texOutputFile| "CONSOLE")
       (setq |$texFormat| nil))
      ((eq arg '|%display%|)
       (if |$texFormat|
           (setq label "On:")
           (setq label "Off:"))
       (concat label |$texOutputFile|))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
       (|describeSetOutputTex|))
      (t
       (cond
         ((and (consp arg)
                  (eq (qcdr arg) nil)
                  (progn (setq fn (qcar arg)) t)
                  (|member| fn '(y n ye yes no o on of off console
                                |y| |n| |ye| |yes| |no| |o| |on| |of| |off| |console|)))
          '|ok|)
         (t (setq arg (list fn '|stex| nil))))
       (cond
         ((and (consp arg)
                  (eq (qcdr arg) nil)

```

```

      (progn (setq fn (qcar arg)) t))
(cond
  ((|member| (upcase fn) '(y n ye o of))
    (|sayKeyedMsg|
      (format nil
        "To toggle %1 printing on and off, specify %1 )set output %2 ~
        yes/no/on/off %1 Yes, no, on and off cannot be abbreviated."
        '(|TeX| |tex|)))
    ((|member| (upcase fn) '(no off)) (setq |$texFormat| nil))
    ((|member| (upcase fn) '(yes on)) (setq |$texFormat| t))
    ((eq (upcase fn) 'console)
      (shut |$texOutputStream|)
      (setq |$texOutputStream|
        (defiostream '((mode . output) (device . console)) 255 0))
      (setq |$texOutputFile| "CONSOLE"))))
((or
  (and (consp arg)
    (progn (setq fn (qcar arg))
      (setq tmp1 (qcdr arg))
      (and (consp tmp1)
        (eq (qcdr tmp1) nil)
        (progn (setq ft (qcar tmp1)) t))))
  (and (consp arg)
    (progn (setq fn (qcar arg))
      (setq tmp1 (qcdr arg))
      (and (consp tmp1)
        (progn (setq ft (qcar tmp1))
          (setq tmp2 (qcdr tmp1))
          (and (consp tmp2)
            (eq (qcdr tmp2) nil)
            (progn (setq fm (qcar tmp2)) t)))))))
  (when (setq ptype (|pathnameType| fn))
    (setq fn (concat (|pathnameDirectory| fn) (|pathnameName| fn)))
    (setq ft ptype))
  (unless fm (setq fm 'A))
  (setq filename ($filep fn ft fm))
  (cond
    ((null filename)
      (|sayKeyedMsg|
        "It is not possible to open or create a file called %1 %2 %3 ."
        (list fn ft fm)))
    ((setq teststream (make-outstream filename 255 0))
      (shut |$texOutputStream|)
      (setq |$texOutputStream| teststream)
      (setq |$texOutputFile| (|object2String| filename))
      (|sayKeyedMsg|
        "%1 output will be written to file %2 ."
        (list "TeX" |$texOutputFile|)))
    (t (|sayKeyedMsg|
      "It is not possible to open or create a file called %1 %2 %3 ."

```

```

        (list fn ft fm )))))
(t
  (|sayKeyedMsg| "Your argument list is not valid." nil)
  (|describeSetOutputTex|))))))

```

defun describeSetOutputTex

```

[|sayBrightly| p??]
[|setOutputTex| p1069]

```

— defun describeSetOutputTex —

```

(defun |describeSetOutputTex| ()
  (|sayBrightly| (list
    " )set output tex "
    "is used to tell AXIOM to turn TeX-style output"
    '|%l| "printing on and off, and where to place the output.  By default, the"
    '|%l| "destination for the output is the screen but printing is turned off."
    '|%l|
    '|%l| "Syntax:  )set output tex <arg>"
    '|%l| "      where arg can be one of"
    '|%l| " on          turn TeX printing on"
    '|%l| " off         turn TeX printing off (default state)"
    '|%l| " console     send TeX output to screen (default state)"
    '|%l| " fp<.fe>      send TeX output to file with file prefix fp and file"
    '|%l| "                extension .fe. If not given, .fe defaults to .stex."
    '|%l|
    '|%l| "If you wish to send the output to a file, you must issue this command"
    '|%l| "twice: once with"
    " on and once with the file name. For example, to send"
    '|%l| "TeX output to the file polymer.stex, issue the two commands"
    '|%l|
    '|%l| " )set output tex on"
    '|%l| " )set output tex polymer"
    '|%l|
    '|%l| "The output is placed in the directory from which you invoked AXIOM or"
    '|%l| "the one you set with the )cd system command."
    '|%l| "The current setting is: "
    (|setOutputTex| '|%display%|)
  )))

```

26.47 quit

----- The quit Option -----

Description: protected or unprotected quit

The quit option may be followed by any one of the following:

```
protected
-> unprotected
```

The current setting is indicated.

defvar \$quitCommandType

— initvars —

```
(defvar |$quitCommandType| '|unprotected| "protected or unprotected quit")
```

— quit —

```
(|quit|
 "protected or unprotected quit"
 |interpreter|
 LITERALS
 |$quitCommandType|
 (|protected| |unprotected|)
 |unprotected|)
```

26.48 streams

Current Values of streams Variables

Variable	Description	Current Value

calculate	specify number of elements to calculate	10
showall	display all stream elements computed	off

— streams —

```
(|streams|
  "set some options for working with streams"
  |interpreter|
  TREE
  |novar|
  (
    \getchunk{streamscalculate}
    \getchunk{streamsshowall}
  ))
```

—————

set streams calculate

----- The calculate Option -----

Description: specify number of elements to calculate

)set streams calculate is used to tell AXIOM how many elements of a stream to calculate when a computation uses the stream. The value given after calculate must either be the word all or a positive integer.

The current setting is 10 .

defvar \$streamCount

— initvars —

```
(defvar |$streamCount| 10
  "number of initial stream elements you want calculated")
```

—————

— streamscalculate —

```
(|calculate|
  "specify number of elements to calculate "
  |interpreter|
  FUNCTION
  |setStreamsCalculate|
```

```

    ("number of initial stream elements you want calculated "
     INTEGER
     |$streamCount|
     (0 NIL)
     10))
  NIL)

```

defun setStreamsCalculate

```

[object2String p??]
[describeSetStreamsCalculate p1075]
[sayMessage p??]
[bright p??]
[terminateSystemCommand p792]
[$streamCount p1074]

```

— defun setStreamsCalculate —

```

(defun |setStreamsCalculate| (arg)
  (let (n)
    (declare (special |$streamCount|))
    (cond
      ((eq arg '|%initialize%|) (setq |$streamCount| 10))
      ((eq arg '|%display%|) (|object2String| |$streamCount|))
      ((or (null arg) (eq arg '|%describe%|) (eq (car arg) '?))
       (|describeSetStreamsCalculate|))
      (t
       (setq n (car arg))
       (cond
         ((and (not (eq n '|all|)) (or (null (integerp n)) (minusp n)))
          (|sayMessage|
           '("Your value of" ,@(|bright| n) "is invalid because ..."))
          (|describeSetStreamsCalculate|)
          (|terminateSystemCommand|))
         (t (setq |$streamCount| n)))))))

```

defun describeSetStreamsCalculate

```

[sayKeyedMsg p43]
[$streamCount p1074]

```

— defun describeSetStreamsCalculate —

```
(defun |describeSetStreamsCalculate| ()
  (declare (special |$streamCount|))
  (|sayKeyedMsg|
   (format nil
    ")set streams calculate is used to tell Axiom how many elements of a ~
    stream to calculate when a computation uses the stream. The value ~
    given after calculate must either be the word all or a positive ~
    integer.  %l %l The current setting is %l .")
    (list |$streamCount|)))
```

—————

set streams showall

----- The showall Option -----

Description: display all stream elements computed

The showall option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$streamsShowAll

— initvars —

```
(defvar |$streamsShowAll| nil "display all stream elements computed")
```

—————

— streamsshowall —

```
(|showall|
 "display all stream elements computed"
 |interpreter|
 LITERALS
 |$streamsShowAll|
```

```
(|on| |off|)
|off|)
```

26.49 set system

Current Values of system Variables

Variable	Description	Current Value
functioncode	show gen. LISP for functions when compiled	off
optimization	show optimized LISP code	off
prettyprint	prettyprint BOOT func's as they compile	off

— system —

```
(|system|
 "set some system development variables"
 |development|
 TREE
 |novar|
 (
 \getchunk{systemfunctioncode}
 \getchunk{systemoptimization}
 \getchunk{systemprettyprint}
 ))
```

set system functioncode

----- The functioncode Option -----

Description: show gen. LISP for functions when compiled

The functioncode option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$reportCompilation

— initvars —

```
(defvar |$reportCompilation| nil "show gen. LISP for functions when compiled")
```

—————

— systemfunctioncode —

```
(|functioncode|
 "show gen. LISP for functions when compiled"
 |development|
 LITERALS
 |$reportCompilation|
 (|on| |off|)
 |off|)
```

—————

set system optimization

----- The optimization Option -----

Description: show optimized LISP code

The optimization option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$reportOptimization

— initvars —

```
(defvar |$reportOptimization| nil "show optimized LISP code")
```

—————

— systemoptimization —

```
(|optimization|
 "show optimized LISP code"
 |development|
 LITERALS
 |$reportOptimization|
 (|on| |off|)
 |off|)
```

set system prettyprint

----- The prettyprint Option -----

Description: prettyprint BOOT func's as they compile

The prettyprint option may be followed by any one of the following:

```
on
-> off
```

The current setting is indicated.

defvar \$prettyprint

— initvars —

```
(defvar $prettyprint t "prettyprint BOOT func's as they compile")
```

— systemprettyprint —

```
(|prettyprint|
 "prettyprint BOOT func's as they compile"
 |development|
 LITERALS
 $prettyprint
 (|on| |off|)
```

```
|on|)
```

26.50 set userlevel

----- The userlevel Option -----

Description: operation access level of system user

The userlevel option may be followed by any one of the following:

```
    interpreter
    compiler
-> development
```

The current setting is indicated.

defvar \$UserLevel

— initvars —

```
(defvar |$UserLevel| '|development| "operation access level of system user")
```

— userlevel —

```
(|userlevel|
 "operation access level of system user"
 |interpreter|
 LITERALS
 |$UserLevel|
 (|interpreter| |compiler| |development|)
 |development|)
```

— initvars —

```
(defvar |$setOptions| '(
\getchunk{breakmode}
\getchunk{compile}
\getchunk{debug}
\getchunk{expose}
\getchunk{functions}
\getchunk{fortran}
\getchunk{kernel}
\getchunk{hyperdoc}
\getchunk{help}
\getchunk{history}
\getchunk{messages}
\getchunk{naglink}
\getchunk{output}
\getchunk{quit}
\getchunk{streams}
\getchunk{system}
\getchunk{userlevel}
))
```

defvar \$setOptionNames

— initvars —

```
(defvar |$setOptionNames| (mapcar #'car |$setOptions|))
```

— postvars —

```
(eval-when (eval load)
  (|initializeSetVariables| |$setOptions|))
```

26.51 Set code

defun set

```
[set1 p1082]
|$setOptions p??]
```


— **defun set** —

```
(defun |set| (l)
  (declare (special |$setOptions|))
  (|set1| l |$setOptions|))
```

—————→

defun set1

This function will be called with the top level arguments to `)set`. For instance, given the command

```
)set break break
```

this function gets

```
(set1 (|break| |break|) ....)
```

and given the command

```
)set mes auto off
```

this function gets

```
(set1 (|mes| |auto| |off|) ....)
```

which, because “message” is a TREE, generates the recursive call:

```
(set1 (|auto| |off|) <the message subtree>)
```

The “autoload” subtree contains a FUNCTION called `printLoadMessages`, which gets called with `%describe%`

```
[displaySetVariableSettings p965]
[seq p??]
[exit p??]
[selectOption p819]
[downcase p1272]
[lassoc p??]
[satisfiesUserLevel p791]
[sayKeyedMsg p43]
[poundsign p??]
[displaySetOptionInformation p964]
[sayMSG p45]
```

```

[sayMessage p??]
[bright p??]
[object2String p??]
[translateYesNo2TrueFalse p967]
[use-fast-links p??]
[literals p??]
[tree p??]
[set1 p1082]
[$setOptionNames p1081]
[$UserLevel p1080]
[$displaySetValue p1020]

```

— defun set1 —

```

(defun |set1| (l settree)
  (let (|$setOptionNames| arg setdata st setfunarg num upperlimit arg2)
    (declare (special |$setOptionNames| |$UserLevel| |$displaySetValue|))
    (cond
      ((null l) (|displaySetVariableSettings| settree '||))
      (t
       (setq |$setOptionNames|
              (do ((t1 settree (cdr t1)) t0 (x nil))
                  ((or (atom t1) (progn (setq x (car t1)) nil)) (nreverse0 t0))
              (seq
               (exit
                (setq t0 (cons (elt x 0) t0))))))
       (setq arg
              (|selectOption| (downcase (car l)) |$setOptionNames| '|optionError|))
       (setq setdata (cons arg (lassoc arg settree)))
       (cond
          ((null (|satisfiesUserLevel| (third setdata)))
           (|sayKeyedMsg|
            "Your %1 is ambiguous. The following are abbreviated by %2 : "
            (list |$UserLevel| "set option" nil)))
          ((eq 1 (|#| l)) (|displaySetOptionInformation| arg setdata))
          (t
           (setq st (fourth setdata))
           (case (fourth setdata)
             (function
              (setq setfunarg
                     (if (eq (elt l 1) 'default)
                         '|%initialize%|
                         (ifcdr l)))
              (if (canFuncall? (fifth setdata))
                  (funcall (fifth setdata) setfunarg)
                  (|sayMSG| (concatenate 'string "    Function not implemented. "
                                           (string (fifth setdata)))))
              (when |$displaySetValue|
                (|displaySetOptionInformation| arg setdata)))

```

```

NIL)
(string
  (setq arg2 (elt 1 1))
  (cond
    ((eq arg2 'default) (set (fifth setdata) (seventh setdata)))
    (arg2 (set (fifth setdata) arg2))
    (t nil))
  (when (or |$displaySetValue| (null arg2))
    (|displaySetOptionInformation| arg setdata))
  NIL)
(integer
  (setq arg2
    (progn
      (setq num (elt 1 1))
      (cond
        ((and (integerp num)
              (>= num (elt (sixth setdata) 0))
              (or (null (setq upperlimit (elt (sixth setdata) 1)))
                  (<= num upperlimit)))
          num)
        (t
         (|selectOption|
          (elt 1 1)
          (cons '|default| (sixth setdata)) nil))))))
  (cond
    ((eq arg2 'default) (set (fifth setdata) (seventh setdata)))
    (arg2 (set (fifth setdata) arg2))
    (t nil))
  (cond
    ((or |$displaySetValue| (null arg2))
     (|displaySetOptionInformation| arg setdata))
  (cond
    ((null arg2)
     (|sayMessage|
      (" Your value" ,@( |bright| (|object2String| (elt 1 1)))
       "is not among the valid choices.")))
    (t nil)))
(literals
  (cond
    ((setq arg2
      (|selectOption| (elt 1 1)
      (cons '|default| (sixth setdata)) nil))
     (cond
       ((eq arg2 'default)
        (set (fifth setdata)
              (|translateYesNo2TrueFalse| (seventh setdata))))
       (t
        (cond ((eq arg2 '|nobreak|)
                 #+:GCL (si::use-fast-links t)))
              (cond

```

```

      ((eq arg2 '|fastlinks|)
       #+:GCL (si::use-fast-links nil)
       (setq arg2 '|break|)))
    (set (fifth setdata) (|translateYesNo2TrueFalse| arg2))))))
  (when (or |$displaySetValue| (null arg2))
    (|displaySetOptionInformation| arg setdata))
  (cond
   ((null arg2)
    (|sayMessage|
     (cons " Your value"
            (append (|bright| (|object2String| (elt 1 1)))
                    (cons "is not among the valid choices." nil)))))
    (t nil)))
  (tree (|set1| (ifcdr 1) (sixth setdata)) nil)
  (t
   (|sayMessage|
    '("Cannot handle set tree node type" ,@(|bright| st) |yet|))
    nil))))))

```

26.52)show Command

show man page

— show.help —

```
=====
A.22. )show
=====
```

User Level Required: interpreter

Command Syntax:

-)show nameOrAbbrev
-)show nameOrAbbrev)operations
-)show nameOrAbbrev)attributes

Command Description:

This command displays information about AXIOM domain, package and category constructors. If no options are given, the)operations option is assumed. For example,

```
)show POLY
)show POLY )operations
)show Polynomial
)show Polynomial )operations
```

each display basic information about the Polynomial domain constructor and then provide a listing of operations. Since Polynomial requires a Ring (for example, Integer) as argument, the above commands all refer to a unspecified ring R. In the list of operations, \$ means Polynomial(R).

The basic information displayed includes the signature of the constructor (the name and arguments), the constructor abbreviation, the exposure status of the constructor, and the name of the library source file for the constructor.

If operation information about a specific domain is wanted, the full or abbreviated domain name may be used. For example,

```
)show POLY INT
)show POLY INT )operations
)show Polynomial Integer
)show Polynomial Integer )operations
```

are among the combinations that will display the operations exported by the domain Polynomial(Integer) (as opposed to the general domain constructor

Polynomial). Attributes may be listed by using the)attributes option.

Also See:

- o)display
- o)set
- o)what

17

defun The)show command

[showSpad2Cmd p1087]

— defun show —

```
(defun |show| (arg) (|showSpad2Cmd| arg))
```

defun The internal)show command

[member p1238]
 [helpSpad2Cmd p881]
 [sayKeyedMsg p43]
 [qcar p??]
 [reportOperations p1089]
 [\$showOptions p??]
 [\$e p315]
 [\$env p315]
 [\$InteractiveFrame p39]
 [\$options p73]

— defun showSpad2Cmd —

```
(defun |showSpad2Cmd| (arg)
  (let (|$showOptions| |$e| |$env| constr)
    (declare (special |$showOptions| |$e| |$env| |$InteractiveFrame| |$options|))
    (if (equal arg (list nil))
      (|helpSpad2Cmd| '(|show|))
      (progn
        (setq |$showOptions| '(|attributes| |operations|))
```

¹⁷ “display” (26.18 p 864) “set” (26.51 p 1081) “what” (26.61 p 1131)

```

(unless |$options| (setq |$options| '(|operations|)))
(setq |$e| |$InteractiveFrame|)
(setq |$env| |$InteractiveFrame|)
(cond
  ((and (consp arg) (eq (qcdr arg) nil) (progn (setq constr (qcar arg)) t))
    (cond
      ((|member| constr '(|Union| |Record| |Mapping|))
        (cond
          ((eq constr '|Record|)
            (|sayKeyedMsg|
              (format nil
                "Record(a:A,...,b:B) %l Record takes any number of ~
                selector-domain pairs as arguments: %i %l a, a selector, an ~
                element of domain Symbol %l A, a domain of category ~
                SetCategory %l ... %l b, a selector, an element of domain ~
                Symbol %l B, a domain of category SetCategory %u %l ~
                This constructor is a primitive in Axiom. ~
                The selectors a,...,b of a Record type must be distinct. %l %l ~
                In order for more information to be displayed about %1 , ~
                you must give it specific arguments. For example: %2 %l ~
                You can also use the HyperDoc Browser.")
              (list constr ")show Record(a: Integer, b: String)" )))
          ((eq constr '|Mapping|)
            (|sayKeyedMsg|
              (format nil
                "Mapping(T, S, ...) %l Mapping takes any number of arguments ~
                of the form: %i %l T, a domain of category SetCategory %l ~
                S, a domain of category SetCategory %l ... %u %l ~
                Mapping(T, S, ...) denotes the class of objects which are ~
                mappings from a source domain (S, ...) into a target domain T. ~
                The Mapping constructor can take any number of arguments. ~
                All but the first argument is regarded as part of a source ~
                tuple for the mapping. For example, Mapping(T, A, B) denotes ~
                the class of mappings from (A, B) into T. %l ~
                This constructor is a primitive in Axiom. ~
                For more information, use the HyperDoc Browser.")
              nil)))
        (t
          (|sayKeyedMsg|
            (format nil
              "Tagged union: Union(a:A, ..., b:B) %l Union takes any number ~
              of 'tag'-domain pairs of arguments: %i %l a, a tag, an ~
              element of domain Symbol %l A, a domain of category ~
              SetCategory %l ... %l b, a tag, an element of domain ~
              Symbol %l B, a domain of category SetCategory %u %l ~
              This constructor is a primitive in Axiom. ~
              In this tagged Union, tags a, ..., b must be distinct. %l %l ~
              In order for more information to be displayed about %1 , ~
              you must give it specific arguments. For example: %2 %l ~
              You can also use the HyperDoc Browser.")
            ))
          ))
      ))
    ))

```

```

      (list constr ")show Union(a: Integer, b: String)" )))
    (|sayKeyedMsg|
      (format nil
        "Untagged union: Union(A, ..., B) %l Union takes any number ~
        of domain arguments: %i %l A, a domain of category ~
        SetCategory %l ... %l B, a domain of category SetCategory %u %l ~
        In this untagged form of Union, domains A, ..., B must be ~
        distinct. In order for more information to be displayed about ~
        %l , you must give it specific arguments. For example: %2 %l ~
        You can also use the HyperDoc Browser.")
        (list constr ")show Union(Integer, String)" ))))
    ((and (consp constr) (eq (qcar constr) '|Mapping|))
      (|sayKeyedMsg|
        (format nil
          "Mapping(T, S, ...) %l Mapping takes any number of arguments ~
          of the form: %i %l T, a domain of category SetCategory %l ~
          S, a domain of category SetCategory %l ... %u %l ~
          Mapping(T, S, ...) denotes the class of objects which are ~
          mappings from a source domain (S, ...) into a target domain T. ~
          The Mapping constructor can take any number of arguments. ~
          All but the first argument is regarded as part of a source ~
          tuple for the mapping. For example, Mapping(T, A, B) denotes ~
          the class of mappings from (A, B) into T. %l ~
          This constructor is a primitive in Axiom. ~
          For more information, use the HyperDoc Browser."
          nil))
        (t (|reportOperations| constr constr))))
    (t (|reportOperations| arg arg))))))

```

defun reportOperations

```

[sayBrightly p??]
[bright p??]
[sayKeyedMsg p43]
[qcar p??]
[isNameOfType p??]
[isDomainValuedVariable p1145]
[reportOpsFromUnitDirectly0 p1095]
[opOf p??]
[unabbrev p??]
[reportOpsFromLisplib0 p1091]
[evaluateType p1118]
[mkAtree p??]
[removeZeroOneDestructively p??]
[isType p??]

```



```

[$env p315]
[$eval p??]
[$genValue p346]
[$quadSymbol p??]
[$doNotAddEmptyModeIfTrue p69]

```

— defun reportOperations —

```

(defun |reportOperations| (oldArg u)
  (let (|$env| |$eval| |$genValue| |$doNotAddEmptyModeIfTrue|
        tmp1 v unitForm tree unitFormp)
    (declare (special |$env| |$eval| |$genValue| |$quadSymbol|
                      |$doNotAddEmptyModeIfTrue|))
    (setq |$env| (list (list nil)))
    (setq |$eval| t)
    (setq |$genValue| t)
    (when u
      (setq |$doNotAddEmptyModeIfTrue| t)
      (cond
        ((equal u |$quadSymbol|)
         (|sayBrightly|
          (cons " mode denotes" (append (|bright| "any") (list '|type|)))))
        ((eq u '|%)
         (|sayKeyedMsg|
          (format nil
                   "The )show system command is used to display information about ~
                   types or partial types. For example, )show Integer will show ~
                   information about Integer.")
          nil)
         (|sayKeyedMsg|
          (format nil
                   "%1 %% is a special variable holding the result of the last ~
                   computation. Issue )display properties %% to see this value.")
          nil)))
    ((and (null (and (consp u) (eq (qcar u) '|Record|)))
          (null (and (consp u) (eq (qcar u) '|Union|)))
          (null (|isNameOfType| u))
          (null (and (consp u)
                     (eq (qcar u) '|typeOf|)
                     (progn
                      (setq tmp1 (qcdr u))
                      (and (consp tmp1) (eq (qcdr tmp1) nil))))))
      (when (atom oldArg) (setq oldArg (list oldArg)))
      (|sayKeyedMsg|
       (format nil
                "The )show system command is used to display information about ~
                types or partial types. For example, )show Integer will show ~
                information about Integer.")
       nil)

```

```

(dolist (op oldArg)
  (|sayKeyedMsg|
   (format nil
    "%1 %1 is not the name of a known type constructor. If you want ~
    to see information about any operations named %1, issue ~
    %ceon )display operations %1 %ceoff")
    (list (|opOf| op))))))
((setq v (|isDomainValuedVariable| u)) (|reportOpsFromUnitDirectly0| v))
(t
 (if (atom u)
   (setq unitForm (|opOf| (|unabbrev| u)))
   (setq unitForm (|unabbrev| u)))
 (if (atom unitForm)
   (|reportOpsFromLisplib0| unitForm u)
   (progn
    (setq unitFormp (|evaluateType| unitForm))
    (setq tree (|mkAtree| (|removeZeroOneDestructively| unitForm)))
    (if (setq unitFormp (|isType| tree))
      (|reportOpsFromUnitDirectly0| unitFormp)
      (|sayKeyedMsg|
       (format nil
        "It is not known what %1p is, so no information about it can be ~
        displayed.")
        (list unitForm))))))))))

```

defun reportOpsFromLisplib0

[\[reportOpsFromLisplib1 p1092\]](#)
[\[reportOpsFromLisplib p1092\]](#)
[\[\\$useEditorForShowOutput p1067\]](#)

— defun reportOpsFromLisplib0 —

```

(defun |reportOpsFromLisplib0| (unitForm u)
  (declare (special |$useEditorForShowOutput|))
  (if |$useEditorForShowOutput|
    (|reportOpsFromLisplib1| unitForm u)
    (|reportOpsFromLisplib| unitForm u)))

```

defun reportOpsFromLisplib1

```
[pathname p1232]
[erase p??]
[defiostream p1168]
[sayShowWarning p1100]
[reportOpsFromLisplib p1092]
[shut p1168]
[editFile p873]
[$sayBrightlyStream p??]
[$erase p??]
```

— defun reportOpsFromLisplib1 —

```
(defun |reportOpsFromLisplib1| (unitForm u)
  (let (|$sayBrightlyStream| showFile)
    (declare (special |$sayBrightlyStream| $erase))
    (setq showFile (|pathname| (list 'show 'listing 'a)))
    ($erase showFile)
    (setq |$sayBrightlyStream|
      (defiostream '((file ,showFile) (mode . output)) 255 0))
    (|sayShowWarning|)
    (|reportOpsFromLisplib| unitForm u)
    (shut |$sayBrightlyStream|)
    (|editFile| showFile)))
```

defun reportOpsFromLisplib

```
[constructor? p??]
[sayKeyedMsg p43]
[getConstructorSignature p??]
[ifcdr p??]
[getdatabase p1196]
[eqsubstlist p??]
[nreverse0 p??]
[sayBrightly p??]
[concat p1237]
[bright p??]
[form2StringWithWhere p??]
[isExposedConstructor p1094]
[concat p1237]
[namestring p1230]
[selectOptionLC p819]
```

```
[dc1 p??]
[specialChar p1166]
[remdup p??]
[msort p??]
[form2String p??]
[say2PerLine p??]
[formatAttribute p??]
[displayOperationsFromLisplib p1095]
[$linelength p1051]
[$showOptions p??]
[$options p73]
[$FormalMapVariableList p17]
```

— defun reportOpsFromLisplib —

```
(defun |reportOpsFromLisplib| (op u)
  (let (fn s typ nArgs argList functorForm argml tmp1 functorFormWithDecl
        verb sourceFile opt attList)
    (declare (special $linelength |$showOptions| |$options|
                      |$FormalMapVariableList|))
    (if (null (setq fn (|constructor?| op)))
        (|sayKeyedMsg| "%1 is unknown, so no information is available." (list u))
        (progn
          (setq argml (when (setq s (|getConstructorSignature| op)) (ifcdr s)))
          (setq typ (getdatabase op 'constructorkind))
          (setq nArgs (|#| argml))
          (setq argList (ifcdr (getdatabase op 'constructorform)))
          (setq functorForm (cons op argList))
          (setq argml (eqsubstlist argList |$FormalMapVariableList| argml))
          (mapcar #'(lambda (a m) (push (list ' |:| a m) tmp1)) argList argml)
          (setq functorFormWithDecl (cons op (nreverse0 tmp1)))
          (|sayBrightly|
            (|concat| (|bright| (|form2StringWithWhere| functorFormWithDecl))
                      "is a" (|bright| typ) "constructor"))
          (|sayBrightly|
            (cons " Abbreviation for"
                  (append (|bright| op) (cons "is" (|bright| fn))))))
        (if (|isExposedConstructor| op)
            (setq verb "is")
            (setq verb "is not"))
        (|sayBrightly|
          (cons " This constructor"
                (append (|bright| verb) (list "exposed in this frame."))))
        (setq sourceFile (getdatabase op 'sourcefile))
        (|sayBrightly|
          (cons " Issue"
                (append (|bright| (concat ")edit " (|namestring| sourceFile)))
                        (cons "to see algebra source code for"
                              (append (|bright| fn) (list ' |%1|)))))))
```

```

;TPDHERE -- add domain doc to output
;   (setq domaindoc (car (cdadar (getdatabase op 'documentation))))
;   (|sayBrightly| (cleanupLine domaindoc))
;   (terpri)
(dolist (item |$options|)
  (setq opt (|selectOptionLC| (car item) |$showOptions| '|optionError|))
  (cond
    ((eq opt '|layout|) (|dc1| fn))
    ((eq opt '|views|)
     (|sayBrightly|
      (cons "To get" (append (|bright| "views"
        (list "you must give parameters of constructor")))))
     ((eq opt '|attributes|)
      (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " Attributes ")
      (|sayBrightly| ""))
      (setq attList
        (remdup
         (msort
          (mapcar #'(lambda (x) (caar x))
                   (reverse (getdatabase op 'attributes))))))
      (if (null attList)
          (|sayBrightly|
           (|concat| (|form2String| functorForm)
                     '|has no attributes.| '|%1|))
          (|say2PerLine| (mapcar #'|formatAttribute| attList))))
    ((eq opt '|operations|)
     (|displayOperationsFromLisplib| functorForm))))))

```

defun isExposedConstructor

```

[getalist p??]
[$localExposureData p166]
[$globalExposureGroupAlist p167]

```

— defun isExposedConstructor —

```

(defun |isExposedConstructor| (name)
  (let (x found)
    (declare (special |$globalExposureGroupAlist| |$localExposureData|))
    (cond
      ((member name '(|Union| |Record| |Mapping|)) t)
      ((member name (elt |$localExposureData| 2)) nil)
      ((member name (elt |$localExposureData| 1)) t)
      (t
       (loop for g in (elt |$localExposureData| 0)

```

```

when (not found)
do
  (setq x (getalist |$globalExposureGroupAlist| g))
  (when (and x (getalist x name)) (setq found t))
found)))

```

defun displayOperationsFromLisplib

```

[getdatabase p1196]
[specialChar p1166]
[remdup p??]
[msort p??]
[eqsubstlist p??]
[formatOperationAlistEntry p??]
[say2PerLine p??]
[$FormalMapVariableList p17]
[$linelength p1051]

```

— defun displayOperationsFromLisplib —

```

(defun |displayOperationsFromLisplib| (form)
  (let (name arg1 kind opList opl ops)
    (declare (special |$FormalMapVariableList| $linelength))
    (setq name (car form))
    (setq arg1 (cdr form))
    (setq kind (getdatabase name 'constructorkind))
    (setq opList (getdatabase name 'operationalist))
    (when opList
      (format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " Operations ")
      (setq opl
        (remdup (msort (eqsubstlist arg1 |$FormalMapVariableList| opList)))))
    (setq ops nil)
    (dolist (x opl)
      (setq ops (append ops (|formatOperationAlistEntry| x))))
    (|say2PerLine| ops))))

```

defun reportOpsFromUnitDirectly0

```

[reportOpsFromUnitDirectly1 p1100]
[reportOpsFromUnitDirectly p1096]

```

`[$useEditorForShowOutput p1067]`

— **defun reportOpsFromUnitDirectly0** —

```
(defun |reportOpsFromUnitDirectly0| (D)
  (declare (special |$useEditorForShowOutput|))
  (if |$useEditorForShowOutput|
    (|reportOpsFromUnitDirectly1| D)
    (|reportOpsFromUnitDirectly| D)))
```

—

defun reportOpsFromUnitDirectly

```
[member p1238]
[qcar p??]
[evalDomain p1115]
[getdatabase p1196]
[sayBrightly p??]
[concat p1237]
[formatOpType p??]
[isExposedConstructor p1094]
[bright p??]
[sayBrightly p??]
[concat p1237]
[namestring p1230]
[selectOptionLC p819]
[specialChar p1166]
[remdup p??]
[msort p??]
[formatAttribute p??]
[getl p1240]
[systemErrorHere p??]
[nreverse0 p??]
[getOplistForConstructorForm p1098]
[say2PerLine p??]
[formatOperation p??]
[$commentedOps p??]
[$CategoryFrame p??]
[$linelength p1051]
[$options p73]
[$showOptions p??]
```

— **defun reportOpsFromUnitDirectly** —

```

(defun |reportOpsFromUnitDirectly| (unitForm)
  (let (|$commentedOps| isRecordOrUnion unit top kind abb sourceFile verb opt
        attList constructorFunction tmp1 funlist a sigList tmp2)
    (declare (special |$commentedOps| |$CategoryFrame| $linelength |$options|
                      |$showOptions|))
    (setq isRecordOrUnion
          (and (consp unitForm)
               (progn (setq a (qcar unitForm)) t)
               (|member| a '(|Record| |Union|))))
    (setq unit (|evalDomain| unitForm))
    (setq top (car unitForm))
    (setq kind (getdatabase top 'constructorkind))
    (|sayBrightly|
     (|concat| " " (|formatOpType| unitForm)
               " is a " kind " constructor."))
    (unless isRecordOrUnion
      (setq abb (getdatabase top 'abbreviation))
      (setq sourceFile (getdatabase top 'sourcefile))
      (|sayBrightly|
       (cons " Abbreviation for"
             (append (|bright| top) (cons "is" (|bright| abb)))))
      (if (|isExposedConstructor| top)
          (setq verb "is")
          (setq verb "is not"))
      (|sayBrightly|
       (cons " This constructor"
             (append (|bright| verb) (list "exposed in this frame." ))))
      (|sayBrightly|
       (cons " Issue"
             (append (|bright| (concat ")edit " (|namestring| sourceFile))
                     (cons "to see algebra source code for"
                           (append (|bright| abb) (list '|%l|)))))))
    (dolist (item |$options|)
      (setq opt (|selectOptionLC| (car item) |$showOptions| '|optionError|))
      (cond
        ((eq opt '|attributes|)
         (format t "~v,,, '-:@<~a~>~%" (- $linelength 2) " Attributes ")
         (if isRecordOrUnion
             (|sayBrightly| " Records and Unions have no attributes.")
             (progn
              (|sayBrightly| "")
              (setq attList
                    (remdup
                     (msort
                      (mapcar #'(lambda (unit2) (car unit2)) (reverse (elt unit 2))))))
              (|say2PerLine|
               (mapcar #'|formatAttribute| attList))
              nil)))
        ((eq opt '|operations|)
         (setq |$commentedOps| 0)

```



```

;      --new form is (<op> <signature> <slotNumber> <condition> <kind>)
(format t "~v,,,'-:@<~a~>~%" (- $linelength 2) " Operations ")
(|sayBrightly| "")
(cond
  (isRecordOrUnion
    (setq constructorFunction (get1 top '|makeFunctionList|))
    (unless constructorFunction
      (|systemErrorHere| "reportOpsFromUnitDirectly"))
    (setq tmp1
      (funcall constructorFunction '$ unitForm |$CategoryFrame|))
    (setq funlist (car tmp1))
    (setq sigList
      (remdup
        (msort
          (dolist (fun funlist (nreverse0 tmp2))
            (push '(((, (caar fun) ,(cadar fun)) t ,(caddar fun) 0 1)))
              tmp2))))))
    (t
      (setq sigList
        (remdup (msort (|getOplistForConstructorForm| unitForm))))))
  (|say2PerLine|
    (mapcar #'(lambda (x) (|formatOperation| x unit)) sigList))
  (unless (= |$commentedOps| 0)
    (|sayBrightly|
      (list "Functions that are not yet implemented are preceded by"
        (|bright| "--"))))
    (|sayBrightly| ""))))
nil))

```

defun getOplistForConstructorForm

The new form is an op-Alist which has entries

```
(<op> . signature-Alist)
```

where signature-Alist has entries

```
(<signature> . item)
```

where item has form (|slotNumber_i |condition_i |kind_i)

```
(<slotNumber> <condition> <kind>)
```

where |kind_i = ELT — CONST — Subsumed — (XLAM..) ..

```
<kind> = ELT | CONST | Subsumed | (XLAM..) ..
```

— defun getOplistForConstructorForm —

```
(defun |getOplistForConstructorForm| (form)
  (let (argl pairlis opAlist op signatureAlist result)
    (declare (special |$FormalMapVariableList|))
    (setq op (car form))
    (setq argl (cdr form))
    (setq pairlis
      (loop for fv in |$FormalMapVariableList|
            for arg in argl
            collect (cons fv arg)))
    (setq opAlist (|getOperationAlistFromLisplib| op))
    (loop for item in opAlist do
      (setq op (car item))
      (setq signatureAlist (cdr item))
      (setq result
        (append result
          (|getOplistWithUniqueSignatures| op pairlis signatureAlist))))
    result))
```

—————

defun getOplistWithUniqueSignatures

— defun getOplistWithUniqueSignatures —

```
(defun |getOplistWithUniqueSignatures| (op pairlis signatureAlist)
  (let (sig slotNumber pred kind alist)
    (loop for item in signatureAlist
      when (not (eq (fourth item) '|Subsumed|))
      do
        (setq sig (first item))
        (setq slotNumber (second item))
        (setq pred (third item))
        (setq kind (fourth item))
        (setq alist
          (|insertAlist|
            (sublis pairlis (list op sig))
            (sublis pairlis (list pred (list kind nil slotNumber)))
            alist)))
    alist))
```

—————

defun reportOpsFromUnitDirectly1

```
[pathname p1232]
[erase p??]
[defiostream p1168]
[sayShowWarning p1100]
[reportOpsFromUnitDirectly p1096]
[shut p1168]
[editFile p873]
[$sayBrightlyStream p??]
[$erase p??]
```

— defun reportOpsFromUnitDirectly1 —

```
(defun |reportOpsFromUnitDirectly1| (D)
  (let (|$sayBrightlyStream| showFile)
    (declare (special |$sayBrightlyStream| $erase))
    (setq showFile (|pathname| (list 'show 'listing 'a)))
    ($erase showFile)
    (setq |$sayBrightlyStream|
      (defiostream '((file ,showFile) (mode . output)) 255 0))
    (|sayShowWarning|)
    (|reportOpsFromUnitDirectly| D)
    (shut |$sayBrightlyStream|)
    (|editFile| showFile)))
```

defun sayShowWarning

```
[sayBrightly p??]
```

— defun sayShowWarning —

```
(defun |sayShowWarning| ()
  (|sayBrightly|
    "Warning: this is a temporary file and will be deleted the next")
  (|sayBrightly|
    "      time you use )show. Rename it and FILE if you wish to")
  (|sayBrightly| "      save the contents.")
  (|sayBrightly| ""))
```

26.53)spool Command

spool man page

— spool.help —

```
=====
A.23. )spool
=====
```

User Level Required: interpreter

Command Syntax:

-)spool [fileName]
-)spool

Command Description:

This command is used to save (spool) all AXIOM input and output into a file, called a spool file. You can only have one spool file active at a time. To start spool, issue this command with a filename. For example,

```
)spool integrate.out
```

To stop spooling, issue)spool with no filename.

If the filename is qualified with a directory, then the output will be placed in that directory. If no directory information is given, the spool file will be placed in the current directory. The current directory is the directory from which you started AXIOM or is the directory you specified using the)cd command.

Also See:

- o)cd

26.54)summary Command

summary man page

— summary.help —

```

)credits      : list the people who have contributed to Axiom

)help <command> gives more information
)quit        : exit AXIOM

)abbreviation : query, set and remove abbreviations for constructors
)browse       : start an Axiom http server on 127.0.0.1 port 8085
)cd           : set working directory
)clear        : remove declarations, definitions or values
)close        : throw away an interpreter client and workspace
)compile      : invoke constructor compiler
)copyright    : show copyright and trademark information
)describe     : show database information for a category, domain, or package
)display      : display Library operations and objects in your workspace
)edit         : edit a file
)fin          : drop into lisp, use (restart) to return to the session
)frame        : manage interpreter workspaces
)history      : manage aspects of interactive session
)include      : insert a file into a .input file
)library      : introduce new constructors
)license      : display the Axiom license file
)lisp         : evaluate a LISP expression
)ltrace       : trace functions
)pquit        : ask if you really want to exit Axiom
)quit         : exit Axiom
)read         : execute AXIOM commands from a file
)regress      : regression test an output spool file
)savesystem   : save LISP image to a file
)set          : view and set system variables
)show         : show constructor information
)spool        : log input and output to a file
)synonym      : define an abbreviation for system commands
)system       : issue shell commands
)tangle       : extract chunks from a literate program to an input file
)trace        : trace execution of functions
)trademark    : declare that Axiom is a trademark of this software effort
)undo         : restore workspace to earlier state
)what         : search for various things by name

```

defun summary

```
[obey p??]
[concat p1237]
[getenvIRON p322]
```

— defun summary —

```
(defun |summary| (l)
  (declare (ignore l))
  (obey (concat "cat " (getenvIRON "AXIOM") "/doc/spadhelp/summary.help")))
```

—————

26.55)synonym Command

synonym man page

— synonym.help —

```
=====
A.24. )synonym
=====
```

User Level Required: interpreter

Command Syntax:

-)synonym
-)synonym synonym fullCommand
-)what synonyms

Command Description:

This command is used to create short synonyms for system command expressions. For example, the following synonyms might simplify commands you often use.

```
)synonym save      history )save
)synonym restore   history )restore
)synonym mail      system mail
)synonym ls        system ls
)synonym fortran   set output fortran
```

Once defined, synonyms can be used in place of the longer command expressions. Thus

```
)fortran on
```

is the same as the longer

```
)set fortran output on
```

To list all defined synonyms, issue either of

```
)synonyms
)what synonyms
```

To list, say, all synonyms that contain the substring ‘ap’, issue

```
)what synonyms ap
```

Also See:

```
o )set
o )what
```

19

defun The)synonym command

[synonymSpad2Cmd p1105]

— defun synonym —

```
(defun |synonym| (&rest ignore)
  (declare (ignore ignore))
  (|synonymSpad2Cmd|))
```

defun The)synonym command implementation

[getSystemCommandLine p1106]
 [printSynonyms p813]
 [processSynonymLine p1107]
 [putalist p??]
 [terminateSystemCommand p792]
 [\$CommandSynonymAlist p817]

— defun synonymSpad2Cmd —

```
(defun |synonymSpad2Cmd| ()
  (let (line pair)
    (declare (special |$CommandSynonymAlist|))
    (setq line (|getSystemCommandLine|))
    (if (string= line "")
        (|printSynonyms| nil)
        (progn
          (setq pair (|processSynonymLine| line))
          (if |$CommandSynonymAlist|
              (putalist |$CommandSynonymAlist| (car pair) (cdr pair))
              (setq |$CommandSynonymAlist| (cons pair nil))))
        (|terminateSystemCommand|))))
```

¹⁹ “set” (26.51 p 1081) “what” (26.61 p 1131)

defun Return a sublist of applicable synonyms

The argument is a list of synonyms, and this returns a sublist of applicable synonyms at the current user level. [string2id-n p??]

[selectOptionLC p819]
 [commandsForUserLevel p788]
 [\$systemCommands p783]
 [\$UserLevel p1080]

— defun synonymsForUserLevel —

```
(defun |synonymsForUserLevel| (arg)
  (let (cmd nl)
    (declare (special |$systemCommands| |$UserLevel|))
    (if (eq |$UserLevel| '|development|)
      arg
      (dolist (syn (reverse arg))
        (setq cmd (string2id-n (cdr syn) 1))
        (when (|selectOptionLC| cmd (|commandsForUserLevel| |$systemCommands|) nil)
          (push syn nl))))
      nl))
```

defun Get the system command from the input line

[strpos p1236]
 [substring p324]
 [\$currentLine p??]

— defun getSystemCommandLine —

```
(defun |getSystemCommandLine| ()
  (let (p line)
    (declare (special |$currentLine|))
    (setq p (strpos ")" |$currentLine| 0 nil))
    (if p
      (setq line (substring |$currentLine| p nil))
      (setq line |$currentLine|))
    (string-left-trim '(\space) line)))
```

defun Remove system keyword

[maxindex p??]

— defun processSynonymLine,removeKeyFromLine —

```

(defun |processSynonymLine,removeKeyFromLine| (line)
  (prog (mx)
    (return
      (seq
        (setq line (string-left-trim " " line))
        (setq mx (maxindex line))
        (exit
          (do ((i 0 (1+ i)))
              ((> i mx) nil)
            (seq
              (exit
                (if (char= (elt line i) #\space)
                  (exit
                    (return
                     (do ((j (1+ i) (1+ j)))
                         ((> j mx) nil)
                       (seq
                        (exit
                          (if (char\= (elt line j) #\space)
                            (exit
                              (return
                               (substring line j nil))))))))))))))
                (return
                  (substring line j nil))))))))))))))))))

```

—————

defun processSynonymLine

[processSynonymLine,removeKeyFromLine p1107]

— defun processSynonymLine —

```

(defun |processSynonymLine| (line)
  (cons
    (string2id-n line 1)
    (|processSynonymLine,removeKeyFromLine| line)))

```

—————

This command is in the list of `$noParseCommands` 26.1 which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` 26.2

26.56 `)system` Command

system man page

— system.help —

```
=====
A.25. )system
=====
```

User Level Required: interpreter

Command Syntax:

```
- )system cmdExpression
```

Command Description:

This command may be used to issue commands to the operating system while remaining in AXIOM. The `cmdExpression` is passed to the operating system for execution.

To get an operating system shell, issue, for example, `)system sh`. When you enter the key combination, Ctrl-D (pressing and holding the Ctrl key and then pressing the D key) the shell will terminate and you will return to AXIOM. We do not recommend this way of creating a shell because Lisp may field some interrupts instead of the shell. If possible, use a shell running in another window.

If you execute programs that misbehave you may not be able to return to AXIOM. If this happens, you may have no other choice than to restart AXIOM and restore the environment via `)history )restore`, if possible.

Also See:

```
o )boot
o )fin
o )lisp
o )pquit
o )quit
```

20

This command is in the list of `$noParseCommands` [26.1](#) which means that its arguments are passed verbatim. This will eventually result in a call to the function `handleNoParseCommands` [26.2](#)

²⁰ “boot” ([26.4 p 826](#)) “fin” ([26.20 p 876](#)) “lisp” ([26.27 p 935](#)) “pquit” ([26.29 p 938](#)) “quit” ([26.30 p 940](#))

26.57)tangle Command

tangle man page

— tangle.help —

```
=====
A.19. )tangle
=====
```

User Level Required: interpreter

Command Syntax:

```
- )tangle [fileName]
```

Command Description:

This command is used to tangle pamphlet files.

```
)tangle matrix.input.pamphlet
```

will tangle the contents of the file matrix.input.pamphlet into matrix.input. The ‘.input.pamphlet’ is optional.

— defun tangle —

```
(defun |tangle| (arg)
  (let (|$InteractiveMode| namestring dot1 dot2 outfile
        (chunkname "*" (extension "input"))
        (declare (special |$InteractiveMode| |$Options|))
        (setq |$InteractiveMode| t)
        (setq namestring (symbol-name (car arg)))
        (setq dot1 (position #\. namestring))
        (if dot1
            (setq outfile
                  (concatenate 'string (subseq namestring 0 dot1) "." extension))
            (setq outfile
                  (concatenate 'string (subseq namestring 0) "." extension)))
        (setq dot2 (position #\. namestring :from-end t))
        (cond
         ((and (numberp dot1) (numberp dot2) (< dot1 dot2)))
         ((and (numberp dot1) (numberp dot2) (= dot1 dot2))
          (setq namestring (concatenate 'string namestring ".pamphlet"))))
```

```
(t
  (setq namestring (concatenate 'string namestring ".input.pamphlet")))
(if (probe-file namestring)
  (progn
    (tangle namestring chunkname outfile)
    (format t (concatenate 'string outfile " created from " namestring "%"))
    (format t (concatenate 'string namestring " file not found~%")))))
```

26.58 *)trademark* Command

trademark man page

— *trademark.help* —

```
=====
A.15. )trademark
=====
```

Command Syntax:

```
- )trademark
```

Command Description:

This command displays the Axiom trademark information.

Also See:

o *)license*

26.59)undo Command

undo man page

— undo.help —

```
=====
A.27. )undo
=====
```

User Level Required: interpreter

Command Syntax:

-)undo
-)undo integer
-)undo integer [option]
-)undo)redo

where option is one of

-)after
-)before

Command Description:

This command is used to restore the state of the user environment to an earlier point in the interactive session. The argument of an)undo is an integer which must designate some step number in the interactive session.

```
)undo n
)undo n )after
```

These commands return the state of the interactive environment to that immediately after step n. If n is a positive number, then n refers to step number n. If n is a negative number, it refers to the nth previous command (that is, undoes the effects of the last -n commands).

A)clear all resets the)undo facility. Otherwise, an)undo undoes the effect of)clear with options properties, value, and mode, and that of a previous undo. If any such system commands are given between steps n and n + 1 (n > 0), their effect is undone for)undo m for any 0 < m ≤ n .

The command)undo is equivalent to)undo -1 (it undoes the effect of the previous user expression). The command)undo 0 undoes any of the above system commands issued since the last user expression.

```
)undo n )before
```

This command returns the state of the interactive environment to that immediately before step *n*. Any `)undo` or `)clear` system commands given before step *n* will not be undone.

`)undo )redo`

This command reads the file `redo.input.` created by the last `)undo` command. This file consists of all user input lines, excluding those backtracked over due to a previous `)undo`.

The command `)history )write` will eliminate the “undone” command lines of your program.

Also See:
o `)history`

21

26.60 Evaluation

Some Antique Comments About the Interpreter

EVAL BOOT contains the top level interface to the Scratchhpad-II interpreter. The Entry point into the interpreter from the parser is `processInteractive`.

The type analysis algorithm is contained in the file `BOTMUP BOOT`, and `MODSEL boot`, the map handling routines are in `MAP BOOT` and `NEWMAP BOOT`, and the interactive coerce routines are in `COERCE BOOT` and `COERCEFN BOOT`.

Conventions: All spad values in the interpreter are passed around in triples. These are lists of three items:

`[value,mode,environment]`

The value may be wrapped (this is a pair whose CAR is the atom `WRAPPED` and whose CDR is the value), which indicates that it is a real value, or unwrapped in which case it needs to be `EVAL`ed to produce the proper value. The mode is the type of value, and should always be completely specified (not contain `$EmptyMode`). The environment is always empty, and is included for historical reasons.

Modemaps: Modemaps are descriptions of compiled Spad function which the interpreter uses to perform type analysis. They consist of patterns of types for the arguments, and conditions the types must satisfy for the function to apply. For each function name there is a list of modemaps in file `modemap DATABASE` for each distinct function with that name.

²¹ “history” (26.23 p 889)

This is the signature pattern for the modemap, it is of the form:

Environments: Environments associate properties with atoms. Some common properties are:

- **modeSet:** During interpretation we build a modeSet property for each node in the expression. This is (in theory) a list of all the types possible for the node. In the current implementation these modeSets always contain a single type.
- **value:** Value properties are always triples. This is where the values of variables are stored. We also build value properties for internal nodes during the bottom up phase.
- **mode:** This is the declared type of an identifier.

- `$genValue`: if true then evaluate generated code, otherwise leave code unevaluated. If `$genValue` is false then we are compiling.
- `$op`: name of the top level operator (unused except in map printing)
- `$mapList`: list of maps being type analyzed, used in recursive map type analysis.
- `$compilingLoop`: true when compiling a loop body, used to control nesting level of interp-only loop CATCH points

- `$interpOnly`: true when in interpret only mode, used to call alternate forms of `COLLECT` and `REPEAT`.
- `$inCOLLECT`: true when compiling a `COLLECT`, used only for hacked stream compiler.
- `$StreamFrame`: used in printing streams, it is the environment where local stream variables are stored
- `$declaredMode`: Weak type propagation for symbols, set in `upCOERCE` and `upLET`. This variable is used to determine the alternate polynomial types of Symbols.
- `$localVars`: list of local variables in a map body
- `$MapArgumentTypeList`: hack for stream compilation

defun evalDomain

```
[sayMSG p45]
[concat p1237]
[prefix2String p??]
[startTimingProcess p??]
[eval p??]
[mkEvalable p1115]
[stopTimingProcess p??]
[$evalDomain p1115]
```

— defun evalDomain —

```
(defun |evalDomain| (form)
  (let (result)
    (declare (special |$evalDomain|))
    (when |$evalDomain|
      (|sayMSG| (|concat| "   instantiating " (|prefix2String| form))))
    (|startTimingProcess| '|instantiation|)
    (setq result (|eval| (|mkEvalable| form)))
    (|stopTimingProcess| '|instantiation|)
    result))
```

defun mkEvalable

```
[qcar p??]
[qcdr p??]
[mkEvalable p1115]
[devaluate p??]
```

[\[mkEvaluableRecord p1118\]](#)
[\[mkEvaluableUnion p1117\]](#)
[\[mkEvaluableMapping p1118\]](#)
[\[loadIfNecessary p??\]](#)
[\[getdatabase p1196\]](#)
[\[mkq p??\]](#)
[\[constructor? p??\]](#)
[\[fbpip p??\]](#)
[\[bpiname p??\]](#)
[\[\\$Integer p711\]](#)
[\[\\$EmptyMode p708\]](#)

— defun mkEvaluable —

```

(defun |mkEvaluable| (form)
  (let (op argl kind cosig)
    (declare (special |$Integer| |$EmptyMode|))
    (cond
      ((consp form)
       (setq op (qcar form))
       (setq argl (qcdr form))
       (cond
         ((eq op 'quote) form)
         ((eq op 'wrapped) (|mkEvaluable| (|devaluate| argl)))
         ((eq op '|Record|) (|mkEvaluableRecord| form))
         ((eq op '|Union|) (|mkEvaluableUnion| form))
         ((eq op '|Mapping|) (|mkEvaluableMapping| form))
         ((eq op '|Enumeration|) form)
         (t
          (|loadIfNecessary| op)
          (setq kind (getdatabase op 'constructorkind))
          (cond
            ((setq cosig (getdatabase op 'cosig))
             (cons op
              (loop for x in argl for typeFlag in (rest cosig)
                collect
                  (cond
                    (typeFlag
                     (cond
                       ((eq kind '|category|) (mkq x))
                       ((simple-vector-p x) (mkq x))
                     (t
                      (|loadIfNecessary| x)
                      (|mkEvaluable| x))))
                    ((and (consp x) (eq (qcar x) 'quote)) x)
                    ((and (consp x) (eq (qcar x) '|#|) (consp (qcdr x))
                     (eq (qcdr (qcdr x)) nil))
                     (list 'size (mkq (qcar (qcdr x)))))
                    (t (mkq x)))))))

```

```

(t
  (cons op
    (loop for x in arg1
      collect (|mkEvaluable| x))))))
((equal form |$EmptyModel| |$Integer|)
 ((and (identp form) (|constructor?| form)) (list form))
 ((fbpip form) (bpiname form))
 (t form)))

```

defun mkEvaluableUnion

[mkEvaluable p1115]

— defun mkEvaluableUnion —

```

(defun |mkEvaluableUnion| (form)
  (cond
    ((|isTaggedUnion| form)
     (cons
      (car form)
      (loop for item in (rest form)
        collect (list '|:| (second item) (|mkEvaluable| (third item))))))
    (t
     (cons (car form)
      (loop for d in (rest form)
        collect (|mkEvaluable| d))))))

```

defun isTaggedUnion

— defun isTaggedUnion —

```

(defun |isTaggedUnion| (u)
  (and (eq (car u) '|Union|) (eq (caadr u) '|:|)))

```

defun mkEvalableRecord

[mkEvalable p1115]

— defun mkEvalableRecord —

```
(defun |mkEvalableRecord| (form)
  (cons
    (car form)
    (loop for item in (rest form)
      collect (list (quote |:|) (second item) (|mkEvalable| (third item))))))
```

defun mkEvalableMapping

[mkEvalable p1115]

— defun mkEvalableMapping —

```
(defun |mkEvalableMapping| (form)
  (cons
    (car form)
    (loop for d in (rest form)
      collect (|mkEvalable| d))))
```

defun evaluateType

Takes a parsed, unabbreviated type and evaluates it, replacing type valued variables with their values, and calling `bottomUp` on non-type valued arguemnts to the constructor and finally checking to see whether the type satisfies the conditions of its `modemap`.

[isDomainValuedVariable p1145]

[qcar p??]

[qcdr p??]

[mkAtree p??]

[bottomUp p??]

[objVal p516]

[getValue p??]

[evaluateSignature p1123]

[member p1238]

[evaluateType p1118]

```
[constructor? p??]
[throwEvalTypeMsg p1122]
[$EmptyMode p708]
[$expandSegments p??]
```

— defun `evaluateType` —

```
(defun |evaluateType| (form)
  (let (|$expandSegments| domain formp op argl)
    (declare (special |$expandSegments| |$EmptyMode|))
    (cond
      ((setq domain (|isDomainValuedVariable| form)) domain)
      ((equal form |$EmptyMode|) form)
      ((eq form '?) |$EmptyMode|)
      ((stringp form) form)
      ((eq form '$) form)
      (t
       (setq |$expandSegments| nil)
       (cond
         ((and (consp form) (eq (qcar form) '|typeOf|) (consp (qcdr form))
                  (eq (qcdr (qcdr form)) nil))
          (setq formp (|mkAtree| form))
          (|bottomUp| formp)
          (|objVal| (|getValue| formp)))
         ((consp form)
          (setq op (qcar form))
          (setq argl (qcdr form))
          (cond
            ((eq op 'category)
             (cond
               ((consp argl)
                (cons op
                      (cons (qcar argl)
                            (loop for s in (qcdr argl)
                                collect (|evaluateSignature| s))))
                (t form)))
              ((|member| op '(|Join| |Mapping|))
               (cons op
                     (loop for arg in argl
                         collect (|evaluateType| arg))))
              ((eq op '|Union|)
               (cond
                 ((and argl (consp (car argl)) (consp (qcdr (car argl)))
                          (consp (qcdr (qcdr (car argl))))
                          (eq (qcdr (qcdr (qcdr (car argl)))) nil)
                          (|member| (qcar (car argl)) '(|:| |Declare|)))
                  (cons op
                        (loop for item in argl
                            collect
```

```

      (list '|:| (second item) (|evaluateType| (third item))))))
    (t
      (cons op
        (loop for arg in arg1
          collect (|evaluateType| arg))))))
  ((eq op '|Record|)
    (cons op
      (loop for item in arg1
        collect
          (list '|:| (second item) (|evaluateType| (third item))))))
    ((eq op '|Enumeration|) form)
    (t (|evaluateType1| form))))
  ((|constructor?| form)
    (if (atom form)
      (|evaluateType| (list form))
      (|throwEvalTypeMsg|
        (format nil
          "Although %1 is the name of a constructor, a full type must be ~
          specified in the context you have used it. Issue )show %2 ~
          for more information.")
        (list form form))))
    (t (|throwEvalTypeMsg| "%1p is not a valid type." (list form))))))

```

defun Eval args passed to a constructor

Evaluates the arguments passed to a constructor.

```

[constructor? p??]
[getConstructorSignature p??]
[throwEvalTypeMsg p1122]
[replaceSharps p1144]
[categoryForm? p??]
[evaluateType p1118]
[evalCategory p1145]
[getdatabase p1196]
[mkAtree p??]
[putTarget p??]
[bottumUp p??]
[qcar p??]
[qcdr p??]
[getAndEvalConstructorArgument p1144]
[coerceOrRetract p771]
[objValUnwrap p516]
[throwKeyedMsgCannotCoerceWithValue p??]
[makeOrdinal p1123]

```

— defun evaluateType1 —

```
(defun |evaluateType| (form)
  (let (op argl sig ml xp tree tmp1 m1 z1 zt zv v typeList (argnum 0))
    (declare (special |$quadSymbol| |$EmptyMode|))
    (setq op (car form))
    (setq argl (cdr form))
    (cond
      ((|constructor?| op)
       (cond
         ((null (setq sig (|getConstructorSignature| form)))
          (|throwEvalTypeMsg|
            "You cannot now use %1p in the context you have it." (list form)))
        (t
         (setq ml (cdr sig))
         (setq ml (|replaceSharps| ml form))
         (cond
           ((not (eql (|#| argl) (|#| ml)))
            (|throwEvalTypeMsg|
              (format nil
                "Although %1 is the name of a constructor, a full type must be ~
specified in the context you have used it. Issue )show %2 ~
for more information.")
              (list form form)))
          (t
           (loop for x in argl for m in ml
             do
               (setq typeList
                 (cons
                  (cond
                    ((|categoryForm?| m)
                     (setq m (|evaluateType| (subst x '$ m)))
                     (if (|valCategory| (setq xp (|evaluateType| x)) m)
                         xp
                        (|throwEvalTypeMsg| "%1p is not a valid type."
                          (list form))))
                  (t
                   (setq m (|evaluateType| m))
                   (cond
                     ((and (eq (getdatabase (|opOf| m) 'constructorkind) '|domain|)
                      (setq tree (|mkAtree| x))
                      (|putTarget| tree m)
                      (progn
                       (setq tmp1 (|bottomUp| tree))
                       (and (consp tmp1)
                            (eq (qcdr tmp1) nil))))
                     (setq m1 (qcar tmp1))))
                 typeList))))
```



```

      (setq z1 (|getAndEvalConstructorArgument| tree))
      (setq zt (car z1))
      (setq zv (cdr z1))
      (if (setq v (|coerceOrRetract| z1 m))
          (|objValUnwrap| v)
          (|throwKeyedMsgCannotCoerceWithValue| zv zt m)))
    (t
     (when (equal x |$EmptyMode|) (setq x |$quadSymbol|))
     (|throwEvalTypeMsg|
      "Cannot convert the %1 argument of %3p to the type %2p ."
      (list (|makeOrdinal| (incf argnum)) m form))))))
  typeList)))
  (cons op (nreverse typeList))))))
  (t (|throwEvalTypeMsg|
      "Category, domain or package constructor %1 is not available."
      (list op))))))

```

defvar \$noEvalTypeMsg

— initvars —

```
(defvar |$noEvalTypeMsg| nil)
```

defun throwEvalTypeMsg

```

[spadThrow p??]
[throwKeyedMsg p??]
[$noEvalTypeMsg p1122]

```

— defun throwEvalTypeMsg —

```

(defun |throwEvalTypeMsg| (msg args)
  (declare (special |$noEvalTypeMsg|))
  (if |$noEvalTypeMsg|
      (|spadThrow|)
      (|throwKeyedMsg| msg args)))

```

defun makeOrdinal

— defun makeOrdinal —

```
(defun |makeOrdinal| (i)
  (elt '(|first| |second| |third| |fourth| |fifth| |sixth| |seventh|
        |eighth| |ninth| |tenth|)
    (1- i)))
```

—————

defun evaluateSignature

Calls evaluateType on a signature.

[evaluateType p1118]

— defun evaluateSignature —

```
(defun |evaluateSignature| (sig)
  (cond
    ((and (consp sig) (eq (qcar sig) 'signature) (consp (qcdr sig))
          (consp (qcdr (qcdr sig))) (eq (qcdr (qcdr (qcdr sig))) nil))
      (cons 'signature (cons (qcar (qcdr sig))
                             (list
                              (loop for z in (qcar (qcdr (qcdr sig)))
                                    collect (if (eq z '$) z (|evaluateType| z)))))))
      (t sig)))
```

—————

defun recordFrame

```
[diffAlist p1125]
[seq p??]
[exit p??]
[$frameRecord p51]
[$InteractiveFrame p39]
[$previousBindings p52]
```

— defun recordFrame —

```
(defun recordFrame (systemNormal)
  (prog (currentAlist delta)
```

```

(declare (special |$frameRecord| |$InteractiveFrame| |$previousBindings|))
(return
  (seq
    (setq currentAlist (ifcar |$frameRecord|))
    (setq delta (|diffAlist| (caar |$InteractiveFrame|) |$previousBindings|))
    (cond
      ((eq systemNormal 'system)
        (cond
          ((null delta)
            (return nil))
          (t
            (setq delta (cons '|systemCommand| delta))))))
    (setq |$frameRecord| (cons delta |$frameRecord|))
    ; copy all but the individual properties
    ; note that this loop makes no sense. In boot it read:
    ; [cons(first x, [cons(first y,rest y) for y in rest x]) for x
    ; in caar $InteractiveFrame
    ; ... but cons(first y, rest y) == y
    (setq |$previousBindings|
      (prog (tmp0)
        (setq tmp0 nil)
        (return
          (do ((tmp1 (caar |$InteractiveFrame|) (cdr tmp1)) (x nil))
            ((or (atom tmp1)
              (progn (setq x (car tmp1)) nil))
              (nreverse0 tmp0))
          (seq
            (exit
              (setq tmp0
                (cons
                  (cons
                    (car x)
                    (prog (tmp2)
                      (setq tmp2 nil)
                      (return
                        (do ((tmp3 (cdr x) (cdr tmp3)) (y nil))
                          ((or (atom tmp3)
                            (progn (setq y (car tmp3)) nil))
                            (nreverse0 tmp2))
                        (seq
                          (exit
                            (setq tmp2 (cons (cons (car y) (cdr y)) tmp2))))))))
                        tmp0))))))))
            (first |$frameRecord|))))))

```

defun diffAlist

```

diffAlist(new,old) ==
--record only those properties which are different
for (pair := [name,:proplist]) in new repeat
  -- name has an entry both in new and old world
  -- (1) if the old world had no proplist for that variable, then
  --   record NIL as the value of each new property
  -- (2) if the old world does have a proplist for that variable, then
  --   a) for each property with a value: give the old value
  --   b) for each property missing:      give NIL as the old value
oldPair := ASSQ(name,old) =>
  null (oldProplist := CDR oldPair) =>
    --record old values of new properties as NIL
    acc := [ [name,:[ [prop] for [prop,.] in proplist] ],:acc]
  deltas := nil
  for (propval := [prop,:val]) in proplist repeat
    null (oldPropval := ASSOC(prop,oldProplist)) => --missing property
      deltas := [ [prop],:deltas]
      EQ(CDR oldPropval,val) => 'skip
      deltas := [oldPropval,:deltas]
    deltas => acc := [ [name,:NREVERSE deltas],:acc]
  acc := [ [name,:[ [prop] for [prop,.] in proplist] ],:acc]
--record properties absent on new list (say, from a )cl all)
for (oldPair := [name,:r]) in old repeat
  r and null LASSQ(name,new) =>
    acc := [oldPair,:acc]
  -- name has an entry both in new and old world
  -- (1) if the new world has no proplist for that variable
  --   (a) if the old world does, record the old proplist
  --   (b) if the old world does not, record nothing
  -- (2) if the new world has a proplist for that variable, it has
  --   been handled by the first loop.
res := NREVERSE acc
if BOUNDP '$reportundo and $reportundo then reportUndo res
res

```

```

[assq p1240]
[tmp1 p??]
[seq p??]
[exit p??]
[assoc p??]
[lassq p??]
[reportUndo p59]

```

— defun diffAlist —

```

(defun |diffAlist| (new old)
  (prog (proplist oldPair oldProplist val oldPropval deltas prop name r acc res)

```

```

(return
  (seq
    (progn
      (do ((tmp0 new (cdr tmp0)) (pair nil))
        ((or (atom tmp0)
              (progn (setq pair (car tmp0)) nil)
              (progn
                (progn
                  (setq name (car pair))
                  (setq proplist (cdr pair))
                  pair)
                nil)))
          nil)
      nil)
    (seq
      (exit
        (cond
          ((setq oldPair (assq name old))
            (cond
              ((null (setq oldProplist (cdr oldPair)))
                (setq acc
                  (cons
                    (cons
                      name
                      (prog (tmp1)
                        (setq tmp1 nil)
                        (return
                          (do ((tmp2 proplist (cdr tmp2)) (tmp3 nil))
                            ((or (atom tmp2)
                                  (progn (setq tmp3 (car tmp2)) nil)
                                  (progn
                                    (progn (setq prop (car tmp3)) tmp3)
                                    nil)))
                              (nreverse0 tmp1)))
                        (seq
                          (exit
                            (setq tmp1 (cons (cons prop nil) tmp1))))))
                          acc)))
              (t
                (setq deltas nil)
                (do ((tmp4 proplist (cdr tmp4)) (|propval| nil))
                  ((or (atom tmp4)
                        (progn (setq |propval| (car tmp4)) nil)
                        (progn
                          (progn
                            (setq prop (car |propval|))
                            (setq val (cdr |propval|))
                            |propval|)
                          nil)))
                    nil)
                (seq
                  (seq

```

```

        (exit
         (cond
          ((null (setq oldPropval (|assoc| prop oldProplist)))
           (setq deltas (cons (cons prop nil) deltas)))
          ((eq (cdr oldPropval) val) '|skip|)
          (t (setq deltas (cons oldPropval deltas))))))
        (when deltas
         (setq acc
          (cons (cons name (nreverse deltas)) acc))))))
  (t
   (setq acc
    (cons
     (cons
      name
      (prog (tmp5)
             (setq tmp5 nil)
             (return
              (do ((tmp6 proplist (cdr tmp6)) (tmp7 nil))
                  ((or (atom tmp6)
                       (progn (setq tmp7 (CAR tmp6)) nil)
                              (progn
                               (progn (setq prop (CAR tmp7)) tmp7)
                               nil))
                   (nreverse0 tmp5))
              (seq
               (exit
                (setq tmp5 (cons (cons prop nil) tmp5))))))
              acc))))))
   (seq
    (do ((tmp8 old (cdr tmp8)) (oldPair nil))
        ((or (atom tmp8)
              (progn (setq oldPair (car tmp8)) nil)
                    (progn
                     (progn
                      (setq name (car oldPair))
                      (setq r (cdr oldPair))
                      oldPair)
                     nil))
            nil)
        nil)
    (seq
     (exit
      (cond
       ((and r (null (lassq name new)))
        (exit
         (setq acc (cons oldPair acc))))))
     (setq res (nreverse acc))
     (cond
      ((and (boundp '|$reportundo|) |$reportundo|)
       (|reportUndo| res)))
     (exit res))))))

```

defun clearFrame

```
[clearCmdAll p839]  
[$frameRecord p51]  
[$previousBindings p52]
```

— **defun clearFrame** —

```
(defun |clearFrame| ()  
  (declare (special |$frameRecord| |$previousBindings|))  
  (|clearCmdAll|)  
  (setq |$frameRecord| nil)  
  (setq |$previousBindings| nil))
```

26.61)what Command

what man page

— what.help —

```
=====
A.28. )what
=====
```

User Level Required: interpreter

Command Syntax:

```
- )what categories pattern1 [pattern2 ...]
- )what commands   pattern1 [pattern2 ...]
- )what domains    pattern1 [pattern2 ...]
- )what operations pattern1 [pattern2 ...]
- )what packages   pattern1 [pattern2 ...]
- )what synonym    pattern1 [pattern2 ...]
- )what things     pattern1 [pattern2 ...]
- )apropos         pattern1 [pattern2 ...]
```

Command Description:

This command is used to display lists of things in the system. The patterns are all strings and, if present, restrict the contents of the lists. Only those items that contain one or more of the strings as substrings are displayed. For example,

```
)what synonym
```

displays all command synonyms,

```
)what synonym ver
```

displays all command synonyms containing the substring ‘ver’,

```
)what synonym ver pr
```

displays all command synonyms containing the substring ‘ver’ or the substring ‘pr’. Output similar to the following will be displayed

```
----- System Command Synonyms -----
```

user-defined synonyms satisfying patterns:

```
ver pr
```



```

)apr ..... )what things
)apropos ..... )what things
)prompt ..... )set message prompt
)version ..... )lisp *yearweek*

```

Several other things can be listed with the `)what` command:

```

categories displays a list of category constructors.
commands displays a list of system commands available at your
  user-level. Your user-level is set via the )set userlevel command. To get
  a description of a particular command, such as ‘( )what’, issue )help
  what.
domains displays a list of domain constructors.
operations displays a list of operations in the system library.
  It is recommended that you qualify this command with one or more
  patterns, as there are thousands of operations available. For example,
  say you are looking for functions that involve computation of
  eigenvalues. To find their names, try )what operations eig. A rather
  large list of operations is loaded into the workspace when this command
  is first issued. This list will be deleted when you clear the workspace
  via )clear all or )clear completely. It will be re-created if it is
  needed again.
packages displays a list of package constructors.
synonym lists system command synonyms.
things displays all of the above types for items containing
  the pattern strings as substrings. The command synonym )apropos is
  equivalent to )what things.

```

Also See:

- o)display
- o)set
- o)show

22

defvar \$whatOptions

— initvars —

```

(defvar |$whatOptions| '(|operations| |categories| |domains| |packages|
  |commands| |synonyms| |things|))

```

²² “display” (26.18 p 864) “set” (26.51 p 1081) “show” (26.52 p 1087)

defun what

[whatSpad2Cmd p1131]

— defun what —

```
(defun |what| (l)
  (|whatSpad2Cmd| l))
```

defun whatSpad2Cmd,fixpat[qcar p??]
[downcase p1272]

— defun whatSpad2Cmd,fixpat —

```
(defun |whatSpad2Cmd,fixpat| (x)
  (let (xp)
    (if (and (consp x) (progn (setq xp (qcar x)) t))
        (downcase xp)
        (downcase x))))
```

defun whatSpad2Cmd[reportWhatOptions p1133]
[selectOptionLC p819]
[sayKeyedMsg p43]
[seq p??]
[exit p??]
[whatSpad2Cmd,fixpat p1131]
[whatSpad2Cmd p1131]
[filterAndFormatConstructors p1136]
[whatCommands p1133]
[apropos p1137]
[printSynonyms p813]
[\$e p315]
[\$whatOptions p1130]

— defun whatSpad2Cmd —

```
(defun |whatSpad2Cmd| (arg)
  (prog (|$e| |key0| key args)
    (declare (special |$e| |$whatOptions|))
    (return
      (seq
        (progn
          (setq |$e| |$EmptyEnvironment|)
          (cond
            ((null arg) (|reportWhatOptions|))
            (t
              (setq |key0| (car arg))
              (setq args (cdr arg))
              (setq key (|selectOptionLC| |key0| |$whatOptions| nil))
              (cond
                ((null key)
                  (|sayKeyedMsg|
                    (format nil
                      "Your argument is not valid for the )what system command. %l %l ~
                      Use the )show system command to display the operations for a ~
                      constructor. Use the )display operations system command to see ~
                      information about an operation. These may be abbreviated to ~
                      )sh and )d op, respectively.")
                    nil))
                (t
                  (setq args
                    (prog (t0)
                      (setq t0 nil)
                      (return
                        (do ((t1 args (cdr t1)) (p nil))
                          ((or (atom t1)
                              (progn (setq p (car t1)) nil))
                            (nreverse0 t0))
                        (seq
                          (exit
                            (setq t0 (cons (|whatSpad2Cmd,fixpat| p) t0)))))))
                  (seq
                    (cond
                      ((eq key '|things|)
                        (do ((t2 |$whatOptions| (cdr t2)) (opt nil))
                          ((or (atom t2) (progn (setq opt (CAR t2)) nil)) nil)
                        (seq
                          (exit
                            (cond
                              ((null (member opt '|things|))
                                (exit (|whatSpad2Cmd| (cons opt args)))))))
                      ((eq key '|categories|)
                        (|filterAndFormatConstructors| '|category| "Categories" args))
```

```

((eq key '|commands|) (|whatCommands| args))
((eq key '|domains|)
  (|filterAndFormatConstructors| '|domain| "Domains" args))
((eq key '|operations|)
  (|apropos| args))
((eq key '|packages|)
  (|filterAndFormatConstructors| '|package| "Packages" args))
(t
  (cond ((eq key '|synonyms|)
    (|printSynonyms| args)))))))))

```

defun Show keywords for)what command

[sayBrightly p??]
 [\$whatOptions p1130]

— defun reportWhatOptions —

```

(defun |reportWhatOptions| ()
  (let (optlist)
    (declare (special |$whatOptions|))
    (setq optlist
      (reduce #'append
        (mapcar #'(lambda (x) '(|%l| " " ,x)) |$whatOptions|)))
    (|sayBrightly|
      '(" )what" "argument keywords are" ,@optlist
        |%l| " or abbreviations thereof." |%l| |%l| " Issue" ")what ?"
        "for more information.")))

```

defun The)what commands implementation

[concat p1237]
 [specialChar p1166]
 [filterListOfStrings p1134]
 [commandsForUserLevel p788]
 [sayMessage p??]
 [blankList p??]
 [sayAsManyPerLineAsPossible p??]
 [say p??]
 [sayKeyedMsg p43]

[[\\$systemCommands](#) p783]

[[\\$linelength](#) p1051]

[[\\$UserLevel](#) p1080]

— **defun whatCommands** —

```
(defun |whatCommands| (patterns)
  (let (label ell)
    (declare (special |$systemCommands| $linelength |$UserLevel|))
    (setq label
      (concat '|System Commands for User Level: |
        (princ-to-string |$UserLevel|)))
    (format t "~v,,,'-:@< ~a ~>~%" (- $linelength 2) label)
    (setq ell
      (|filterListOfStrings| patterns
        (mapcar #'princ-to-string (|commandsForUserLevel| |$systemCommands|))))
    (when patterns
      (if ell
        (|sayMessage|
          ("System commands at this level matching patterns:" |%1| " "
            ,@(append (|blankList| patterns) (list nil))))
        (|sayMessage|
          ("No system commands at this level matching patterns:" |%1| " "
            ,@(append (|blankList| patterns) (list nil))))))
    (when ell
      (|sayAsManyPerLineAsPossible| ell)
      (say " "))
    (unless patterns
      (|sayKeyedMsg|
        (format nil
          "For more information about individual commands, use the )help ~
          system command followed by the command name or the command name ~
          followed by a question mark. Some commands (such as )lisp ) may ~
          require the )help lisp format. For example, issue )help help or ~
          )help %x1 ? to find out more about the help command itself.")
        nil))))
```

defun Find all names contained in a pattern

Names and patterns are lists of strings. This returns a list of strings in names that contains any of the strings in the patterns [[satisfiesRegularExpressions](#) p1135]

— **defun filterListOfStrings** —

```
(defun |filterListOfStrings| (patterns names)
```

```
(let (result)
  (if (or (null patterns) (null names))
      names
      (dolist (name (reverse names) result)
        (when (|satisfiesRegularExpressions| name patterns)
          (push name result))))))
```

defun Find function of names contained in pattern

The argument names and patterns are lists of strings. The argument fn is something like CAR or CADR This returns a list of strings in names that contains any of the strings in patterns.

[satisfiesRegularExpressions p1135]

— defun filterListOfStringsWithFn —

```
(defun |filterListOfStringsWithFn| (patterns names fn)
  (let (result)
    (if (or (null patterns) (null names))
        names
        (dolist (name (reverse names) result)
          (when (|satisfiesRegularExpressions| (funcall fn name) patterns)
            (push name result))))))
```

defun satisfiesRegularExpressions

[strpos p1236]

— defun satisfiesRegularExpressions —

```
(defun |satisfiesRegularExpressions| (name patterns)
  (let ((dname (downcase (copy name))))
    (dolist (pattern patterns)
      (when (strpos pattern dname 0 "@")
        (return-from nil t)))))
```

defun filterAndFormatConstructors

```

[sayMessage p??]
[blankList p??]
[pp2Cols p??]
[specialChar p1166]
[filterListOfStringsWithFn p1135]
[whatConstructors p1137]
[function p??]
[$linelength p1051]

```

— **defun filterAndFormatConstructors** —

```

(defun |filterAndFormatConstructors| (constrType label patterns)
  (prog (1)
    (declare (special $linelength))
    (return
      (progn
        (format t "~v,,,'-:@< ~a ~>~%" (- $linelength 2) label)
        (setq l
          (|filterListOfStringsWithFn| patterns
            (|whatConstructors| constrType)
            #'cdr))
        (cond (patterns)
          (cond
            ((null l)
              (|sayMessage|
                (cons " No "
                  (cons label
                    (cons " with names matching patterns:"
                      (cons '|%1|
                        (cons " "
                          (append (|blankList| patterns)
                                (cons " " nil))))))))))
            (t
              (|sayMessage|
                (cons label
                  (cons " with names matching patterns:"
                    (cons '|%1|
                      (cons " "
                        (append (|blankList| patterns)
                              (cons " " nil))))))))))))
          (cond (1 (|pp2Cols| 1))))))

```

defun whatConstructors

```

[boot-equal p??]
[getdatabase p1196]
[seq p??]
[msort p??]
[exit p??]

— defun whatConstructors —

(defun |whatConstructors| (constrType)
  (prog nil
    (return
      (seq
        (msort
          (prog (t0)
            (setq t0 nil)
            (return
              (do ((t1 (|allConstructors|) (cdr t1)) (|con| nil))
                ((or (atom t1) (progn (setq |con| (car t1)) nil)) (nreverse0 t0))
              (seq
                (exit
                  (cond
                    ((equal (getdatabase |con| 'constructorkind) constrType)
                     (setq t0
                       (cons
                        (cons
                          (getdatabase |con| 'abbreviation)
                          (string |con|))
                        t0))))))))))))))

```

Display all operation names containing the fragment

Argument *l* is a list of operation name fragments. This displays all operation names containing these fragments.

```

[allOperations p1219]
[filterListOfStrings p1134]
[seq p??]
[exit p??]
[downcase p1272]
[sayMessage p??]
[sayAsManyPerLineAsPossible p??]
[msort p??]
[sayKeyedMsg p43]

```


— defun apropos —

```
(defun |apropos| (arg)
  "Display all operation names containing the fragment"
  (prog (ops)
    (return
      (seq
        (progn
          (setq ops
            (cond
              ((null arg) (|allOperations|))
              (t
               (|filterListOfStrings|
                (prog (t0)
                  (setq t0 nil)
                  (return
                     (do ((t1 arg (cdr t1)) (p nil))
                       ((or (atom t1) (progn (setq p (car t1)) nil))
                        (nreverse0 t0))
                     (seq (exit (setq t0 (cons (downcase (princ-to-string p)) t0))))))
                (|allOperations|))))))
          (cond
            (ops
             (|sayMessage| "Operations whose names satisfy the above pattern(s):")
             (|sayAsManyPerLineAsPossible| (msort ops))
             (|sayKeyedMsg|
              (format nil
                "%1 To get more information about an operation such as %1, issue ~
the command )display op %1")
              (cons (car ops) nil)))
            (t
             (|sayMessage| "  There are no operations containing those patterns")
             nil))))))
```

26.62)workfiles Command

workfiles man page

defun workfiles

[workfilesSpad2Cmd p1139]

— defun workfiles —

```
(defun |workfiles| (l)
  (|workfilesSpad2Cmd| l))
```

defun workfilesSpad2Cmd

```
[throwKeyedMsg p??]
[selectOptionLC p819]
[pathname p1232]
[delete p??]
[makeInputFilename p1169]
[sayKeyedMsg p43]
[namestring p1230]
[updateSourceFiles p874]
[say p??]
[specialChar p1166]
[sortby p??]
[sayBrightly p??]
[$options p73]
[$sourceFiles p??]
[$linelength p1051]
```

— defun workfilesSpad2Cmd —

```
(defun |workfilesSpad2Cmd| (args)
  (let (deleteflag type flist type1 fl)
    (declare (special |$options| |$sourceFiles| $linelength))
    (cond
      (args
       (|throwKeyedMsg|
        (format nil
          "Arguments are not allowed for the )workfiles system command. ~
          The )boot, )lisp, )meta and )delete options may be used with this ~
          command, however. Issue )help workfiles for more information.")
```

```

nil))
(t
  (setq deleteflag nil)
  (do ((t0 |$options| (cdr t0)) (t1 nil))
      ((or (atom t0)
            (progn (setq t1 (car t0)) nil)
            (progn (progn (setq type (car t1)) t1) nil))
        nil)
    (setq type1
      (|selectOptionLC| type '(|boot| |lisp| |meta| |delete|) nil))
    (cond
      ((null type1)
        (|throwKeyedMsg|
          (format nil
            "%1 is not an allowable option for the )workfiles system command. ~
            The )boot, )lisp, )meta and )delete options may be used with this ~
            command, however. Issue )help workfiles for more information.")
          (cons type nil)))
      ((eq type1 '|delete|) (setq deleteflag t))))
  (do ((t2 |$options| (cdr t2)) (t3 nil))
      ((or (atom t2)
            (progn (setq t3 (CAR t2)) nil)
            (progn
              (progn
                (setq type (car t3))
                (setq flist (cdr t3)) t3)
              nil))
        nil)
    (setq type1 (|selectOptionLC| type '(|boot| |lisp| |meta| |delete|) nil))
    (unless (eq type1 '|delete|)
      (dolist (file flist)
        (setq fl (|pathname| (list file type1 "*")))
        (cond
          (deleteflag
            (setq |$sourceFiles| (|delete| fl |$sourceFiles|)))
          ((null (makeInputFilename fl))
            (|sayKeyedMsg|
              (format nil
                "The file %1 will not be added to the list of working source ~
                files because the file does not exist.")
                (list (|namestring| fl))))
            (t (|updateSourceFiles| fl))))))
    (say " ")
    (format t "~v,,,'-:@<~a~>~%" (- $linelength 2)
      " User-specified work files ")
    (say " ")
    (if (null |$sourceFiles|)
      (say " no files specified")
      (progn
        (setq |$sourceFiles| (sortby '|pathnameType| |$sourceFiles|)))

```

```
(do ((t5 |$sourceFiles| (cdr t5)) (f1 nil))
    ((or (atom t5) (progn (setq f1 (car t5)) nil)) nil)
    (|sayBrightly| (list " " (|namestring| f1))))))
```

Chapter 27

Handlers for Special Forms

This file contains the functions which do type analysis and evaluation of special functions in the interpreter. Special functions are ones which are not defined in the algebra code, such as assignment, construct, COLLECT and declaration.

Operators which require special handlers all have a LISP “up” property which is the name of the special handler, which is always the word “up” followed by the operator name. If an operator has this “up” property the handler is called automatically from bottomUp instead of general modemap selection.

The up handlers are usually split into two pieces, the first is the up function itself, which performs the type analysis, and an “eval” function, which generates (and executes, if required) the code for the function.

The up functions always take a single argument, which is the entire attributed tree for the operation, and return the modeSet of the node, which is a singleton list containing the type computed for the node.

The eval functions can take any arguments deemed necessary. Actual evaluation is done if `$genValue` is true, otherwise code is generated.

(See the function analyzeMap for other things that may affect what is generated in these functions.)

These functions are required to do two things:

1. do a putValue on the operator vector with the computed value of the node, which is a triple. This is usually done in the eval functions.
2. do a putModeSet on the operator vector with a list of the computed type of the node. This is usually done in the up functions.

There are several special modes used in these functions:

1. Void is the mode that should be used for all statements that do not otherwise return values, such as declarations, loops, IF-THEN's without ELSE's, etc..

2. `$NoValueMode` and `$ThrowAwayMode` used to be used in situations where `Void` is now used, and are being phased out completely.

defun getAndEvalConstructorArgument

```
[getValue p??]
[objMode p516]
[isWrapped p1661]
[objVal p516]
[isLocalVar p??]
[compFailure p??]
[mkObjWrap p515]
[timedEVALFUN p??]
```

— defun getAndEvalConstructorArgument —

```
(defun |getAndEvalConstructorArgument| (tree)
  (let (triple)
    (setq triple (|getValue| tree))
    (cond
      ((eq (|objMode| triple) '(|Domain|)) triple)
      ((|isWrapped| (|objVal| triple)) triple)
      ((|isLocalVar| (|objVal| triple))
       (|compFailure| " Local variable or parameter used in type"))
      (t
       (mkObjWrap (|timedEVALFUN| (|objVal| triple)) (|objMode| triple))))))
```

—————

defun replaceSharps

Replaces all sharps in `x` by the arguments of domain `d`. Replaces all replaces the triangle variables.

```
[subCopy p??]
[$TriangleVariableList p??]
[$FormalMapVariableList p17]
```

— defun replaceSharps —

```
(defun |replaceSharps| (x d)
  (let (sl)
    (declare (special |$TriangleVariableList| |$FormalMapVariableList|))
    (loop for e in (rest d) for var in |$FormalMapVariableList|
      do (setq sl (cons (cons var e) sl)))
    (setq x (|subCopy| x sl))
```

```
(setq sl nil)
(loop for e in (rest d) for var in |$TriangleVariableList|
  do (setq sl (cons (cons var e) sl)))
(|subCopy| x sl)))
```

defun isDomainValuedVariable

Returns the value of form if form is a variable with a type value.

```
[identp p1237]
[get p??]
[member p1238]
[objMode p516]
[objValUnwrap p516]
[$e p315]
[$env p315]
[$InteractiveFrame p39]
```

— defun isDomainValuedVariable —

```
(defun |isDomainValuedVariable| (form)
  (let (val)
    (declare (special |$e| |$env| |$InteractiveFrame|))
    (when (and (identp form)
               (setq val
                     (or (|get| form '|value| |$InteractiveFrame|)
                         (and (consp |$env|) (|get| form '|value| |$env|))
                         (and (consp |$e|) (|get| form '|value| |$e|))))
          (|member| (|objMode| val) '((|Domain|) (|SubDomain| (|Domain|)))))
      (|objValUnwrap| val))))
```

defun evalCategory

```
[ofCategory p717]
[isPartialMode p718]
```

— defun evalCategory —

```
(defun |evalCategory| (d c)
  (or (|isPartialMode| d) (|ofCategory| d c)))
```


Chapter 28

Handling input files

defun Handle .axiom.input file

[/editfile p849]

— defun readSpadProfileIfThere —

```
(defun |readSpadProfileIfThere| ()  
  (let ((file (list '|.axiom| '|input|)))  
    (declare (special /editfile))  
    (when (makeInputFilename file) (setq /editfile file) (/rq))))
```

—————

defvar boot-line-stack

— initvars —

```
(defvar boot-line-stack nil "List of lines returned from preparse")
```

—————

defvar in-stream

— initvars —

```
(defvar in-stream t "Current input stream.")
```

defvar out-stream

— initvars —

```
(defvar out-stream t "Current output stream.")
```

defvar file-closed

— initvars —

```
(defvar file-closed nil "Way to stop EOF tests for console input.")
```

defvar echo-meta

— initvars —

```
(defvar echo-meta nil "T if you want a listing of what has been read.")
```

defvar \$noSubsumption

— initvars —

```
(defvar |$noSubsumption| t)
```

defvar \$envHashTable

The `$envHashTable` variable is a hashtable that optimizes lookups in the environment, which normally involve search. This gets populated in the `addBinding` function.

— **initvars** —

```
(defvar |$envHashTable| nil)
```

defun Dynamically add bindings to the environment

```
[getProplist p1150]
[addBindingInteractive p1153]
[hput p1234]
[$InteractiveMode p315]
[$envHashTable p1149]
```

— **defun addBinding** —

```
(defun |addBinding| (var proplist e)
  (let (tailContour tailEnv tmp1 curContour lx)
    (declare (special |$InteractiveMode| |$envHashTable|))
    (setq curContour (caar e))
    (setq tailContour (cdar e))
    (setq tailEnv (cdr e))
    (cond
      ((eq proplist (|getProplist| var e)) e)
      (t
       (when |$envHashTable|
         (do ((prop proplist (cdr prop)) (u nil))
             ((or (atom prop)
                  (progn (setq u (car prop)) nil))
              nil)
          (hput |$envHashTable| (list var (car u)) t)))
        (cond
          (|$InteractiveMode| (|addBindingInteractive| var proplist e))
          (t
           (when (and (consp curContour)
                      (progn
                        (setq tmp1 (qcar curContour))
                        (and (consp tmp1) (equal (qcar tmp1) var))))
             (setq curContour (cdr curContour)))
           (setq lx (cons var proplist))
           (cons (cons (cons lx curContour) tailContour) tailEnv)))))))
```

defun Fetch a property list for a symbol from CategoryFrame

```
[getProplist p1150]
[search p1150]
[$CategoryFrame p??]

— defun getProplist —

(defun |getProplist| (x e)
  (let (u pl)
    (declare (special |$CategoryFrame|))
    (cond
      ((null (atom x)) (|getProplist| (car x) e))
      ((setq u (|search| x e)) u)
      ((setq pl (|search| x |$CategoryFrame|)) pl))))
```

defun Search for a binding in the environment list

```
[searchCurrentEnv p1150]
[searchTailEnv p1151]

— defun search —

(defun |search| (x e)
  (let ((curEnv (car e)) (tailEnv (cdr e)))
    (or (|searchCurrentEnv| x curEnv) (|searchTailEnv| x tailEnv))))
```

defun Search for a binding in the current environment

```
searchCurrentEnv(x,currentEnv) ==
  for contour in currentEnv repeat
    if u:= ASSQ(x,contour) then return (signal:= u)
  KDR signal
```

```
[assq p1240]
```

— defun searchCurrentEnv —

```
(defun |searchCurrentEnv| (x currentEnv)
  (prog (u signal)
```

```

(return
  (seq
    (progn
      (do ((thisenv currentEnv (cdr thisenv)) (contour nil))
          ((or (atom thisenv) (progn (setq contour (car thisenv)) nil)) nil)
      (seq
        (exit
          (cond
            ((setq u (assq x contour)) (return (setq signal u)))
            (t nil))))))
    (ifcdr signal))))))

```

—————

defun searchTailEnv

```

;searchTailEnv(x,e) ==
;  for env in e repeat
;    signal:=
;      for contour in env repeat
;        if (u:= ASSQ(x,contour)) and ASSQ("FLUID",u) then return (signal:= u)
;      if signal then return signal
;  KDR signal

```

[assq p1240]

— defun searchTailEnv —

```

(defun |searchTailEnv| (x e)
  (prog (u signal)
    (return
      (seq
        (progn
          (do ((thise e (cdr thise)) (env nil))
              ((or (atom thise) (progn (setq env (car thise)) nil)) nil)
          (seq
            (exit
              (setq signal
                (progn
                  (do ((cone env (cdr cone)) (contour nil))
                      ((or (atom cone) (progn (setq contour (car cone)) nil)) nil)
                  (seq
                    (exit
                      (cond
                        ((and (setq u (assq x contour)) (assq 'fluid u))
                          (return (setq signal u)))
                        (t nil))))))
                (t nil))))))
          (t nil))))))

```

```
(cond
  (signal (return signal))
  (t nil))))))
(ifcdr signal))))))
```

—————

Chapter 29

File Parsing

defun Bind a variable in the interactive environment

[assq p1240]

— defun addBindingInteractive —

```
(defun |addBindingInteractive| (var proplist e)
  (let ((curContour (caar e)) u)
    (cond
      ((setq u (assq var curContour)) (rplacd u proplist) e)
      (t (rplac (caar e) (cons (cons var proplist) curContour)) e))))
```

—————

defvar line-handler

— initvars —

```
(defparameter line-handler 'next-META-line "Who grabs lines for us.")
```

—————

defvar \$spad-errors

— initvars —


```
(defvar $spad_errors (vector 0 0 0))
```

defvar xtokenreader

— initvars —

```
(defvar xtokenreader 'spadtok)
```

defun Initialize the spad reader

```
[next-lines-clear p1158]
[ioclear p1158]
[$spad-errors p1153]
[spaderrorstream p??]
[*standard-output* p??]
[xtokenreader p1154]
[line-handler p1153]
[meta-error-handler p??]
[file-closed p1148]
[boot-line-stack p1147]
```

— defun init-boot/spad-reader —

```
(defun init-boot/spad-reader ()
  (declare (special $spad_errors spaderrorstream *standard-output*
                  xtokenreader line-handler meta-error-handler file-closed
                  boot-line-stack))
  (setq $spad_errors (vector 0 0 0))
  (setq spaderrorstream *standard-output*)
  (setq xtokenreader 'get-BOOT-token)
  (setq line-Handler 'next-BOOT-line)
  (setq meta-error-handler 'spad-syntax-error)
  (setq file-closed nil)
  (next-lines-clear)
  (ioclear))
```

defun spad-syntax-error

```
[bumperrorcount p??]
[consoleinputp p??]
[spad-long-error p1155]
[spad-short-error p1156]
[ioclear p1158]
[debugmode p??]
[spad-reader p??]
```

— defun spad-syntax-error —

```
(defun spad-syntax-error (&rest byebye)
  "Print syntax error indication, underline character, scrub line."
  (declare (special debugmode byebye))
  (bumperrorcount '|syntax|)
  (cond ((and (eq debugmode 'yes) (not(consoleinputp in-stream)))
    (spad-long-error))
    ((spad-short-error)))
  (ioclear)
  (throw 'spad_reader nil))
```

—————

defun spad-long-error

```
[spad-error-loc p1156]
[iostat p1156]
[out-stream p1148]
[spaderrorstream p??]
```

— defun spad-long-error —

```
(defun spad-long-error ()
  (declare (special spaderrorstream))
  (spad-error-loc spaderrorstream)
  (iostat)
  (unless (equal out-stream spaderrorstream)
    (spad-error-loc out-stream)
    (terpri out-stream)))
```

—————

defun spad-short-error

```
[line-past-end-p p??]
[line-print p??]
[$current-line p??]
```

— defun spad-short-error —

```
(defun spad-short-error ()
  (if (line-past-end-p Current-Line)
      (format t "~&The current line is empty.~%" )
      (progn
        (format t "~&The current line is:~%~%" )
        (line-print current-line))))
```

—————

defun spad-error-loc

— defun spad-error-loc —

```
(defun spad-error-loc (str)
  (format str "***** Boot Syntax Error detected *****"))
```

—————

defun iostat

```
[line-past-end-p p??]
[line-print p??]
[token-stack-show p1157]
[next-lines-show p1157]
[$boot p826]
[$spad p310]
[$current-line p??]
```

— defun iostat —

```
(defun iostat ()
  "Tell me what the current state of the parsing world is."
  (declare (special $boot $spad))
  (if (line-past-end-p Current-Line)
      (format t "~&The current line is empty.~%" )
```

```

(progn
  (format t "~&The current line is:~%~%")
  (line-print current-line)))
(if (or $boot $spad) (next-lines-show))
(token-stack-show)
nil)

```

defun next-lines-show

[boot-line-stack p[1147](#)]

— defun next-lines-show —

```

(defun next-lines-show ()
  (declare (special boot-line-stack))
  (and boot-line-stack (format t "Currently preparsed lines are:~%~%")
    (mapcar #'(lambda (line)
      (format t "~&~5D> ~A~%" (car line) (cdr line)))
      boot-line-stack))

```

defun token-stack-show

[token-type p??]
 [valid-tokens p??]
 [current-token p??]
 [next-token p??]
 [prior-token p??]

— defun token-stack-show —

```

(defun token-stack-show ()
  (if (= valid-tokens 0)
    (format t "~%There are no valid tokens.~%")
    (format t "~%The number of valid tokens is ~S.~%" valid-tokens))
  (when (> valid-tokens 0)
    (format t "The current token is~%")
    (describe current-token))
  (when (> valid-tokens 1)
    (format t "The next token is~%")
    (describe next-token))
  (when (token-type prior-token)

```

```
(format t "The prior token was~%")
(describe prior-token))
```

defun ioclear

The IO state manipulation routines assume that

- one I/O stream pair is in effect at any moment
- there is a current line
- there is a current token and a next token
- there is a reduction stack

```
[line-clear p??]
[reduce-stack-clear p??]
[current-fragment p??]
[current-line p??]
[$boot p826]
[$spad p310]
```

— defun ioclear —

```
(defun ioclear (&optional (in t) (out t))
  (declare (special current-fragment current-line $boot $spad)
    (ignore in out))
  (setq current-fragment nil)
  (line-clear current-line)
  (token-install nil nil current-token nil)
  (token-install nil nil next-token nil)
  (token-install nil nil prior-token nil)
  (reduce-stack-clear)
  (if (or $boot $spad) (next-lines-clear))
  nil)
```

defun Set boot-line-stack to nil

```
[boot-line-stack p1147]
```

— defun next-lines-clear —

```
(defun next-lines-clear ()  
  (setq boot-line-stack nil))
```

Chapter 30

Handling output

30.1 Special Character Tables

`defvar $defaultSpecialCharacters`

— initvars —

```
(defvar |$defaultSpecialCharacters| (list
  (int-char 28)    ; upper left corner
  (int-char 27)    ; upper right corner
  (int-char 30)    ; lower left corner
  (int-char 31)    ; lower right corner
  (int-char 79)    ; vertical bar
  (int-char 45)    ; horizontal bar
  (int-char 144)   ; APL quad
  (int-char 173)   ; left bracket
  (int-char 189)   ; right bracket
  (int-char 192)   ; left brace
  (int-char 208)   ; right brace
  (int-char 59)    ; top    box tee
  (int-char 62)    ; bottom box tee
  (int-char 63)    ; right  box tee
  (int-char 61)    ; left   box tee
  (int-char 44)    ; center box tee
  (int-char 224))) ; back slash
```

defvar \$plainSpecialCharacters0

— initvars —

```
(defvar |$plainSpecialCharacters0| (list
  (int-char 78)      ; upper left corner  (+)
  (int-char 78)      ; upper right corner (+)
  (int-char 78)      ; lower left corner  (+)
  (int-char 78)      ; lower right corner (+)
  (int-char 79)      ; vertical bar
  (int-char 96)      ; horizontal bar      (-)
  (int-char 111)     ; APL quad            (?)
  (int-char 173)     ; left bracket
  (int-char 189)     ; right bracket
  (int-char 192)     ; left brace
  (int-char 208)     ; right brace
  (int-char 78)      ; top    box tee      (+)
  (int-char 78)      ; bottom box tee      (+)
  (int-char 78)      ; right  box tee      (+)
  (int-char 78)      ; left   box tee      (+)
  (int-char 78)      ; center box tee      (+)
  (int-char 224))) ; back slash
```

—————

defvar \$plainSpecialCharacters1

— initvars —

```
(defvar |$plainSpecialCharacters1| (list
  (int-char 107)     ; upper left corner  (,)
  (int-char 107)     ; upper right corner (,)
  (int-char 125)     ; lower left corner  (')
  (int-char 125)     ; lower right corner (')
  (int-char 79)      ; vertical bar
  (int-char 96)      ; horizontal bar      (-)
  (int-char 111)     ; APL quad            (?)
  (int-char 173)     ; left bracket
  (int-char 189)     ; right bracket
  (int-char 192)     ; left brace
  (int-char 208)     ; right brace
  (int-char 78)      ; top    box tee      (+)
  (int-char 78)      ; bottom box tee      (+)
  (int-char 78)      ; right  box tee      (+)
  (int-char 78)      ; left   box tee      (+)
```

```
(int-char 78)      ; center box tee      (+)
(int-char 224)))   ; back slash
```

defvar \$plainSpecialCharacters2

— initvars —

```
(defvar |$plainSpecialCharacters2| (list
  (int-char 79)      ; upper left corner  (|)
  (int-char 79)      ; upper right corner (|)
  (int-char 79)      ; lower left corner  (|)
  (int-char 79)      ; lower right corner (|)
  (int-char 79)      ; vertical bar
  (int-char 96)      ; horizontal bar      (-)
  (int-char 111)     ; APL quad           (?)
  (int-char 173)     ; left bracket
  (int-char 189)     ; right bracket
  (int-char 192)     ; left brace
  (int-char 208)     ; right brace
  (int-char 78)      ; top    box tee      (+)
  (int-char 78)      ; bottom box tee      (+)
  (int-char 78)      ; right  box tee      (+)
  (int-char 78)      ; left   box tee      (+)
  (int-char 78)      ; center box tee      (+)
  (int-char 224)))   ; back slash
```

defvar \$plainSpecialCharacters3

— initvars —

```
(defvar |$plainSpecialCharacters3| (list
  (int-char 96)      ; upper left corner  (-)
  (int-char 96)      ; upper right corner (-)
  (int-char 96)      ; lower left corner  (-)
  (int-char 96)      ; lower right corner (-)
  (int-char 79)      ; vertical bar
  (int-char 96)      ; horizontal bar      (-)
  (int-char 111)     ; APL quad           (?)
  (int-char 173)     ; left bracket
```

```

(int-char 189)    ; right bracket
(int-char 192)    ; left brace
(int-char 208)    ; right brace
(int-char 78)     ; top    box tee      (+)
(int-char 78)     ; bottom box tee      (+)
(int-char 78)     ; right  box tee      (+)
(int-char 78)     ; left   box tee      (+)
(int-char 78)     ; center box tee      (+)
(int-char 224)))  ; back slash

```

defvar \$plainRTspecialCharacters

— initvars —

```

(defvar |$plainRTspecialCharacters| (list
  (QUOTE +)      ; upper left corner  (+)
  (QUOTE +)      ; upper right corner (+)
  (QUOTE +)      ; lower left corner  (+)
  (QUOTE +)      ; lower right corner (+)
  (QUOTE |\\|)   ; vertical bar
  (QUOTE -)      ; horizontal bar      (-)
  (QUOTE ?)      ; APL quad            (?)
  (QUOTE [)      ; left bracket
  (QUOTE ])      ; right bracket
  (QUOTE {)      ; left brace
  (QUOTE })      ; right brace
  (QUOTE +)      ; top    box tee      (+)
  (QUOTE +)      ; bottom box tee      (+)
  (QUOTE +)      ; right  box tee      (+)
  (QUOTE +)      ; left   box tee      (+)
  (QUOTE +)      ; center box tee      (+)
  (QUOTE |\\|))) ; back slash

```

defvar \$RTspecialCharacters

— initvars —

```

(defvar |$RTspecialCharacters| (list
  (intern (string (code-char 218))) ;-- upper left corner  (+)

```

```

(intern (string (code-char 191))) ;-- upper right corner (+)
(intern (string (code-char 192))) ;-- lower left corner (+)
(intern (string (code-char 217))) ;-- lower right corner (+)
(intern (string (code-char 179))) ;-- vertical bar
(intern (string (code-char 196))) ;-- horizontal bar      (-)
(list (code-char #x1d) (code-char #xe2))
                                ;-- APL quad             (?)
(QUOTE [])                     ;-- left bracket
(QUOTE ])                     ;-- right bracket
(QUOTE {)                     ;-- left brace
(QUOTE })                     ;-- right brace
(intern (string (code-char 194))) ;-- top box tee         (+)
(intern (string (code-char 193))) ;-- bottom box tee      (+)
(intern (string (code-char 180))) ;-- right box tee       (+)
(intern (string (code-char 195))) ;-- left box tee        (+)
(intern (string (code-char 197))) ;-- center box tee      (+)
(QUOTE \\|))                 ;-- back slash

```

defvar \$specialCharacters

— initvars —

```
(defvar |$specialCharacters| |$RTspecialCharacters|)
```

defvar \$specialCharacterAlist

— initvars —

```

(defvar |$specialCharacterAlist|
  '([|ulc| . 0)
    (|urc| . 1)
    (|llc| . 2)
    (|lrc| . 3)
    (|vbar| . 4)
    (|hbar| . 5)
    (|quad| . 6)
    (|lbrk| . 7)
    (|rbrk| . 8)
    (|lbrc| . 9)

```

```
(|rbrcl| . 10)
(|ttee| . 11)
(|btee| . 12)
(|rtee| . 13)
(|ltee| . 14)
(|ctee| . 15)
(|bslash| . 16)))
```

defun Look up a special character code for a symbol

This function looks up a symbol in `$specialCharacterAlist`, gets the index into the EBCDIC table, and returns the appropriate character. **TPDHERE: Make this more international, not EBCDIC.**

```
[ifcdr p??]
[assq p1240]
[$specialCharacters p1165]
[$specialCharacterAlist p1165]
```

— defun specialChar —

```
(defun |specialChar| (symbol)
  (let (code)
    (declare (special |$specialCharacters| |$specialCharacterAlist|))
    (if (setq code (ifcdr (assq symbol |$specialCharacterAlist|)))
        (elt |$specialCharacters| code)
        "?")))
```

Chapter 31

Stream and File Handling

defun make-instream

[makeInputFilename p1169]

— defun make-instream —

```
(defun make-instream (filespec &optional (recnum 0))
  (declare (ignore recnum))
  (cond ((numberp filespec) (make-synonym-stream '*terminal-io*))
        ((null filespec) (error "not handled yet"))
        (t (open (makeInputFilename filespec)
                  :direction :input :if-does-not-exist nil))))
```

—————

defun make-outstream

[make-filename p??]

— defun make-outstream —

```
(defun make-outstream (filespec &optional (width nil) (recnum 0))
  (declare (ignore width) (ignore recnum))
  (cond ((numberp filespec) (make-synonym-stream '*terminal-io*))
        ((null filespec) (error "not handled yet"))
        (t (open (make-filename filespec) :direction :output))))
```

—————

defun make-appendstream

[make-filename p??]

— defun make-appendstream —

```

(defun make-appendstream (filespec &optional (width nil) (recnum 0))
  "fortran support"
  (declare (ignore width) (ignore recnum))
  (cond
    ((numberp filespec) (make-synonym-stream '*terminal-io*))
    ((null filespec) (error "make-appendstream: not handled yet"))
    ('else (open (make-filename filespec) :direction :output
                  :if-exists :append :if-does-not-exist :create))))

```

defun defiostream

— defun defiostream —

```

(defun defiostream (stream-alist buffer-size char-position)
  (declare (ignore buffer-size))
  (let ((mode (or (cdr (assoc 'mode stream-alist)) 'input))
        (filename (cdr (assoc 'file stream-alist)))
        (dev (cdr (assoc 'device stream-alist))))
    (if (eq dev 'console) (make-synonym-stream '*terminal-io*)
        (let ((strm (case mode
                      ((output o) (open (make-filename filename)
                                           :direction :output))
                      ((input i) (open (makeInputFilename filename)
                                           :direction :input)))))
          (if (and (numberp char-position) (> char-position 0))
              (file-position strm char-position)
              strm))))

```

defun shut

[shut is-console (vol9)]

— defun shut —

```
(defun shut (st)
  (if (is-console st)
      st
      (if (streamp st) (close st) -1)))
```

defun eofp

— defun eofp —

```
(defun eofp (stream) (null (peek-char nil stream nil nil)))
```

defun makeStream

```
[make-appendstream p1168]
[make-outstream p1167]
```

— defun makeStream —

```
(defun |makeStream| (append filename i j)
  (if append
      (make-appendstream filename i j)
      (make-outstream filename i j)))
```

defun Construct a new input file name

— defun makeInputFilename —

```
(defun makeInputFilename (filearg &optional (filetype nil))
  (let*
    ((filename (make-filename filearg filetype))
     (dirname (pathname-directory filename))
     (ft (pathname-type filename))
     (dirs (getDirectoryList ft))
     (newfn nil))
    (if (or (null dirname) (eqcar dirname :relative))
```



```
(dolist (dir dirs (probeName filename))
  (when (probe-file (setq newfn (concatenate 'string dir filename)))
    (return newfn)))
(probeName filename))))
```

defun getDirectoryList

[[\\$current-directory](#) p333]
 [[\\$UserLevel](#) p1080]
 [[\\$library-directory-list](#) p199]
 [[\\$directory-list](#) p198]

— defun getDirectoryList —

```
(defun getDirectoryList (ft &aux (cd (namestring $current-directory)))
  (declare (special $current-directory |$UserLevel| $library-directory-list
    $directory-list))
  (if (member ft '("nrlib" "daase" "exposed")) :test #'string=)
  (if (eq |$UserLevel| '|development|)
    (cons cd $library-directory-list)
    $library-directory-list)
  (adjoin cd
    (adjoin (namestring (user-homedir-pathname)) $directory-list
      :test #'string=)
    :test #'string=)))
```

defun probeName

Sometimes we are given a file and sometimes we are given the name of an Axiom KAF (Keyed-Access File). KAF files are actually directories with a single file called “index.kaf”. We check for the latter case and return the directory name as the filename, per Axiom convention.

— defun probeName —

```
(defun probeName (file)
  (when (or (probe-file file)
    (probe-file (concatenate 'string (namestring file) "/index.kaf")))
    (namestring file)))
```

defun makeFullNamestring

— defun makeFullNamestring —

```
(defun makeFullNamestring (filearg &optional (filetype nil))
  (namestring (merge-pathnames (make-filename filearg filetype))))
```

—————

defun Replace a file by erase and rename

[makeFullNamestring [p1171](#)]

— defun replaceFile —

```
(defun replaceFile (filespec1 filespec2)
  ($erase (setq filespec1 (makeFullNamestring filespec1)))
  (rename-file (makeFullNamestring filespec2) filespec1))
```

—————

Chapter 32

The Spad Server Mechanism

defun openserver

This is a cover function for the C code used for communication interface.

— **defun** openserver —

```
(defun openserver (name)
  (open_server name))
```

—————

Chapter 33

Axiom Build-time Functions

defun spad-save

The **spad-save** function is just a cover function for more lisp system specific save functions. There is no standard name for saving a lisp image so we make one and conditionalize it at compile time.

This function is passed the name of an image that will be saved. The saved image contains all of the loaded functions.

This is used in the src/interp/Makefile.pamphlet in three places:

- creating depsys, an image for compiling axiom.

Some of the Common Lisp code we compile uses macros which are assumed to be available at compile time. The **DEPSYS** image is created to contain the compile time environment and saved. We pipe compile commands into this environment to compile from Common Lisp to machine dependent code.

```
DEPSYS=${OBJ}/${SYS}/bin/depsys
```

- creating savesys, an image for running axiom.

Once we've compile all of the Common Lisp files we fire up a clean lisp image called **LOADSYS**, load all of the final executable code and save it out as **SAVESYS**. The **SAVESYS** image is copied to the `${MNT}/${SYS}/bin` subdirectory and becomes the axiom executable image.

```
LOADSYS= ${OBJ}/${SYS}/bin/lisp
SAVESYS= ${OBJ}/${SYS}/bin/interpsys
AXIOMSYS= ${MNT}/${SYS}/bin/AXIOMsys
```

- creating debugsys, an image with all interpreted functions loaded.

Occasionally we need to really get into the system internals. The best way to do this is to run almost all of the lisp code interpreted rather than compiled (note that `cfuncs.lisp` and `sockio.lisp` still need to be loaded in compiled form as they depend on the loader to link with lisp internals). This image is nothing more than a load of the file `src/interp/debugsys.lisp.pamphlet`. If you need to make test modifications you can add code to that file and it will show up here.

```
DEBUGSYS=${OBJ}/${SYS}/bin/debugsys
```

```
[save-system p??]  
[$SpadServer p201]  
[$openServerIfTrue p199]
```

— defun spad-save —

```
(defun user::spad-save (save-file)
  (declare (special |$SpadServer| $openServerIfTrue))
  (setq |$SpadServer| nil)
  (setq $openServerIfTrue t)
  #+:AKCL
  (system::save-system save-file)
  #+:allegro
  (if (fboundp 'boot::restart)
      (excl::dumplisp :name save-file :restart-function #'boot::restart)
      (excl::dumplisp :name save-file))
  #+:Lucid
  (if (fboundp 'boot::restart)
      (sys::disksave save-file :restart-function #'boot::restart)
      (sys::disksave save-file))
  #+:CCL
  (preserve)
)
```

Chapter 34

Exposure Groups

Exposure groups are a way of controlling the namespace available to the user. Certain algebra files are only useful for internal purposes but they contain functions have common names (like “map”). In order to separate the user visible functions from the internal functions the algebra files are collected into “exposure groups”. These large groups are grouped into sets in the variable `$globalExposureGroupAlist`.

Exposure group information is kept in the local frame. For more information “The Frame Mechanism” [3.1](#) on page [18](#).

Chapter 35

Databases

35.1 Database structure

In order to understand this program you need to understand some details of the structure of the databases it reads. Axiom has 5 databases, the `interp.daase`, `operation.daase`, `category.daase`, and `browse.daase`.

kaf File Format

This documentation refers to kaf files which are random access files. `nrllib` files are kaf files (look for `nrllib/index.kaf`) The format of a random access file is

```
byte-offset-of-key-table
first-entry
second-entry
...
last-entry
((key1 . first-entry-byte-address)
 (key2 . second-entry-byte-address)
 ...
 (keyN . last-entry-byte-address))
```

The key table is a standard lisp alist.

To open a database you fetch the first number, seek to that location, and `(read)` which returns the key-data alist. To look up data you index into the key-data alist, find the `ith-entry-byte-address`, seek to that address, and `(read)`.

For instance, see `src/share/algebra/users.daase/index.kaf`

One existing optimization is that if the data is a simple thing like a symbol then the `nth-entry-byte-address` is replaced by immediate data.

Another existing one is a compression algorithm applied to the data so that the very long names don't take up so much space. We could probably remove the compression algorithm as 64k is no longer considered 'huge'. The database-abbreviation routine handles this on read and write-compress handles this on write.

Indeed, a faster optimization is to simply read the whole database into the image before it is saved. The system would be easier to understand and the interpreter would be faster.

The fastest optimization is to fix the time stamp mechanism which is currently broken. Making this work requires a small bit of coordination at 'make' time which I forgot to implement.

Database Files

Database files are very similar to kaf files except that there is an optimization (currently broken) which makes the first item a pair of two numbers. The first number in the pair is the offset of the key-value table, the second is a time stamp. If the time stamp in the database matches the time stamp in the image the database is not needed (since the internal hash tables already contain all of the information). When the database is built the time stamp is saved in both the gcl image and the database.

Regarding the 'ancestors field in a category: At database build time there exists a `*ancestors-hash*` hash table that gets filled with CATEGORY (not domain) ancestor information. This later provides the information that goes into `interp.daase`. This `*ancestors-hash*` does not exist at normal runtime (it can be made by a call to `genCategoryTable`). Note that the ancestor information in `*ancestors-hash*` (and hence `interp.daase`) involves #1, #2, etc instead of R, Coef, etc. The latter thingies appear in all `.nrlib/index.kaf` files. So we need to be careful when we `)lib` categories and update the ancestor info.

This file contains the code to build, open and access the `.daase` files. This file contains the code to `)library` `nrlibs` and `asy` files

There is a major issue about the data that resides in these databases. the fundamental problem is that the system requires more information to build the databases than it needs to run the interpreter. in particular, `modemap.daase` is constructed using properties like "modemaps" but the interpreter will never ask for this information.

So, the design is as follows:

- the `modemap.daase` needs to be built. this is done by doing a `)library` on ALL of the `nrlib` files that are going into the system. this will bring in "modemap" information and add it to the `*modemaps-hash*` hashtable.
- database build proceeds, accessing the "modemap" property from the hashtables. once this completes this information is never used again.
- the `interp.daase` database is built. this contains only the information necessary to run the interpreter. note that during the running of the interpreter users can extend the system by do a `)library` on a new `nrlib` file. this will cause fields such as "modemap" to be read and hashed.

Each constructor (e.g. LIST) had one library directory (e.g. LIST.nrlib). This directory contained a random access file called the index.kaf file. These files contain runtime information such as the operationAlist and the ConstructorModemap. At system build time we merge all of these .nrlib/index.kaf files into one database, INTERP.daase. Requests to get information from this database are cached so that multiple references do not cause additional disk i/o.

This database is left open at all times as it is used frequently by the interpreter. one minor complication is that newly compiled files need to override information that exists in this database.

The design calls for constructing a random read (kaf format) file that is accessed by functions that cache their results. when the database is opened the list of constructor-index pairs is hashed by constructor name. a request for information about a constructor causes the information to replace the index in the hash table. since the index is a number and the data is a non-numeric sexpr there is no source of confusion about when the data needs to be read.

The format of this new database is as follows:

```
first entry:
  an integer giving the byte offset to the constructor alist
  at the bottom of the file
second and subsequent entries (one per constructor)
  (operationAlist)
  (constructorModemap)
  ....
last entry: (pointed at by the first entry)
  an alist of (constructor . index) e.g.
  ( (PI offset-of-operationAlist offset-of-constructorModemap)
    (NMI offset-of-operationAlist offset-of-constructorModemap)
    ....)
This list is read at open time and hashed by the car of each item.
```

The system has been changed to use the property list of the symbols rather than hash tables. since we already hashed once to get the symbol we need only an offset to get the property list. this also has the advantage that eq hash tables no longer need to be moved during garbage collection.

There are 3 potential speedups that could be done.

- the best would be to use the value cell of the symbol rather than the property list but i'm unable to determine all uses of the value cell at the present time.
- a second speedup is to guarantee that the property list is a single item, namely the database structure. this removes an assoc but leaves one open to breaking the system if someone adds something to the property list. this was not done because of the danger mentioned.
- a third speedup is to make the getdatabase call go away, either by making it a macro or eliding it entirely. this was not done because we want to keep the flexibility of changing the database forms.

The new design does not use hash tables. the database structure contains an entry for each item that used to be in a hash table. initially the structure contains file-position pointers and these are replaced by real data when they are first looked up. the database structure is kept on the property list of the constructor, thus, (get '—DenavitHartenbergMatrix— 'database) will return the database structure object.

Each operation has a property on its symbol name called 'operation which is a list of all of the signatures of operations with that name.

defstruct database

— initvars —

```
(defstruct database
  abbreviation          ; interp.
  ancestors             ; interp.
  constructor           ; interp.
  constructorcategory   ; interp.
  constructorkind       ; interp.
  constructormodemap    ; interp.
  cosig                 ; interp.
  defaultdomain        ; interp.
  modemaps             ; interp.
  niladic              ; interp.
  object               ; interp.
  operationalist        ; interp.
  documentation         ; browse.
  constructorform       ; browse.
  attributes           ; browse.
  predicates            ; browse.
  sourcefile           ; browse.
  parents              ; browse.
  users                ; browse.
  dependents           ; browse.
  spare                ; superstition
) ; database structure
```

—————

defvar *defaultdomain-list*

There are only a small number of domains that have default domains. rather than keep this slot in every domain we maintain a list here.

— initvars —

```
(defvar *defaultdomain-list* '(
```

```
(|MultisetAggregate| |Multiset|)
(|FunctionSpace| |Expression|)
(|AlgebraicallyClosedFunctionSpace| |Expression|)
(|ThreeSpaceCategory| |ThreeSpace|)
(|DequeueAggregate| |Dequeue|)
(|ComplexCategory| |Complex|)
(|LazyStreamAggregate| |Stream|)
(|AssociationListAggregate| |AssociationList|)
(|QuaternionCategory| |Quaternion|)
(|PriorityQueueAggregate| |Heap|)
(|PointCategory| |Point|)
(|PlottableSpaceCurveCategory| |Plot3D|)
(|PermutationCategory| |Permutation|)
(|StringCategory| |String|)
(|FileNameCategory| |FileName|)
(|OctonionCategory| |Octonion|)))
```

defvar *operation-hash*

— **initvars** —

```
(defvar *operation-hash* nil "given an operation name, what are its modemaps?")
```

defvar *hasCategory-hash*

This hash table is used to answer the question “does domain x have category y?”. this is answered by constructing a pair of (x . y) and doing an equal hash into this table.

— **initvars** —

```
(defvar *hasCategory-hash* nil "answers x has y category questions")
```

defvar *miss*

This variable is used for debugging. If a hash table lookup fails and this variable is non-nil then a message is printed.

— **initvars** —

```
(defvar *miss* nil "print out cache misses on getdatabase calls")
```

Note that constructorcategory information need only be kept for items of type category. this will be fixed in the next iteration when the need for the various caches are reviewed

Note that the *modemaps-hash* information does not need to be kept for system files. these are precomputed and kept in modemap.daase however, for user-defined files these are needed. Currently these are added to the database for 2 reasons; there is a still-unresolved issue of user database extensions and this information is used during database build time

Database streams

This are the streams for the databases. They are always open. There is an optimization for speeding up system startup. If the database is opened and the `..stream-stamp*` variable matches the position information in the database then the database is NOT read in and is assumed to match the in-core version

```
defvar *interp-stream*
```

— initvars —

```
(defvar *interp-stream* nil "an open stream to the interpreter database")
```

```
defvar *interp-stream-stamp*
```

— initvars —

```
(defvar *interp-stream-stamp* 0 " *interp-stream* (position . time)")
```

```
defvar *operation-stream*
```

This is indexed by operation, not constructor

— initvars —

```
(defvar *operation-stream* nil "the stream to operation.daase")
```

defvar *operation-stream-stamp*

— initvars —

```
(defvar *operation-stream-stamp* 0 "*operation-stream* (position . time)")
```

defvar *browse-stream*

— initvars —

```
(defvar *browse-stream* nil "an open stream to the browser database")
```

defvar *browse-stream-stamp*

— initvars —

```
(defvar *browse-stream-stamp* 0 "*browse-stream* (position . time)")
```

defvar *category-stream*

This is indexed by (domain . category)

— initvars —

```
(defvar *category-stream* nil "an open stream to the category table")
```

```
defvar *category-stream-stamp*
```

```
— initvars —
```

```
(defvar *category-stream-stamp* 0 "category-stream* (position . time)")
```

```
—————
```

```
defvar *allconstructors*
```

```
— initvars —
```

```
(defvar *allconstructors* nil "a list of all the constructors in the system")
```

```
—————
```

```
defvar *allOperations*
```

```
— initvars —
```

```
(defvar *allOperations* nil "a list of all the operations in the system")
```

```
—————
```

```
defun Reset all hash tables before saving system
```

```
[interpopen p1189]
[operationopen p1193]
[browseopen p1191]
[categoryopen p1192]
[initial-getdatabase p1187]
[*sourcefiles* p1204]
[*interp-stream* p1184]
[*operation-stream* p1184]
[*category-stream* p1185]
[*browse-stream* p1185]
[*category-stream-stamp* p1186]
[*operation-stream-stamp* p1185]
[*interp-stream-stamp* p1184]
```

[*allconstructors* p1186]
 [*operation-hash* p1183]
 [*hascategory-hash* p??]

— defun resethashtables —

```
(defun resethashtables ()
  "set all -hash* to clean values. used to clean up core before saving system"
  (declare (special *sourcefiles* *interp-stream* *operation-stream*
                    *category-stream* *browse-stream* *category-stream-stamp*
                    *operation-stream-stamp* *interp-stream-stamp*
                    *allconstructors* *operation-hash* *hascategory-hash*))
  (setq *hascategory-hash* (make-hash-table :test #'equal))
  (setq *operation-hash* (make-hash-table))
  (setq *allconstructors* nil)
  (setq *sourcefiles* nil)
  (setq *interp-stream-stamp* '(0 . 0))
  (interpopen)
  (setq *operation-stream-stamp* '(0 . 0))
  (operationopen)
  (setq *browse-stream-stamp* '(0 . 0))
  (browseopen)
  (setq *category-stream-stamp* '(0 . 0))
  (categoryopen) ;note: this depends on constructorform in browse.daase
  (initial-getdatabase)
  (close *interp-stream*)
  (close *operation-stream*)
  (close *category-stream*)
  (close *browse-stream*)
  (gbc t))
```

—

defun Preload algebra into saved system

[getdatabase p1196]
 [getenvirom p322]

— defun initial-getdatabase —

```
(defun initial-getdatabase ()
  "fetch data we want in the saved system"
  (let (hascategory constructormodemapAndoperationalist operation constr)
    (format t "Initial getdatabase~%")
    (setq hascategory '(
      (|Equation| . |Ring|)
      (|Expression| . |CoercibleTo|) (|Expression| . |CommutativeRing|)
```

```

(|Expression| . |IntegralDomain|) (|Expression| . |Ring|)
(|Float| . |RetractableTo|)
(|Fraction| . |Algebra|) (|Fraction| . |CoercibleTo|)
(|Fraction| . |OrderedSet|) (|Fraction| . |RetractableTo|)
(|Integer| . |Algebra|) (|Integer| . |CoercibleTo|)
(|Integer| . |ConvertibleTo|) (|Integer| . |LinearlyExplicitRingOver|)
(|Integer| . |RetractableTo|)
(|List| . |CoercibleTo|) (|List| . |FiniteLinearAggregate|)
(|List| . |OrderedSet|)
(|Polynomial| . |CoercibleTo|) (|Polynomial| . |CommutativeRing|)
(|Polynomial| . |ConvertibleTo|) (|Polynomial| . |OrderedSet|)
(|Polynomial| . |RetractableTo|)
(|Symbol| . |CoercibleTo|) (|Symbol| . |ConvertibleTo|)
(|Variable| . |CoercibleTo|)))
(dolist (pair hascategory) (getdatabase pair 'hascategory))
(setq constructormodemapAndoperationalist '(
|BasicOperator| |Boolean|
|CardinalNumber| |Color| |Complex|
|Database|
|Equation| |EquationFunctions2| |Expression|
|Float| |Fraction| |FractionFunctions2|
|Integer| |IntegralDomain|
|Kernel|
|List|
|Matrix| |MappingPackage1|
|Operator| |OutputForm|
|NonNegativeInteger|
|ParametricPlaneCurve| |ParametricSpaceCurve| |Point| |Polynomial|
|PolynomialFunctions2| |PositiveInteger|
|Ring|
|SetCategory| |SegmentBinding| |SegmentBindingFunctions2| |DoubleFloat|
|SparseMultivariatePolynomial| |SparseUnivariatePolynomial| |Segment|
|String| |Symbol|
|UniversalSegment|
|Variable| |Vector|))
(dolist (con constructormodemapAndoperationalist)
  (getdatabase con 'constructormodemap)
  (getdatabase con 'operationalist))
(setq operation '(
|+| |-| |*| | / | |**| |coerce| |convert| |elt| |equation|
|float| |sin| |cos| |map| |SEGMENT|))
(dolist (op operation) (getdatabase op 'operation))
(setq constr '( ;these are sorted least-to-most freq. delete early ones first
|Factored| |SparseUnivariatePolynomialFunctions2| |TableAggregate&| | |
|RetractableTo&| |RecursiveAggregate&| |UserDefinedPartialOrdering|
|None| |UnivariatePolynomialCategoryFunctions2| |IntegerPrimesPackage|
|SetCategory&| |IndexedExponents| |QuotientFieldCategory&| |Polynomial|
|EltableAggregate&| |PartialDifferentialRing&| |Set|
|UnivariatePolynomialCategory&| |FlexibleArray|
|SparseMultivariatePolynomial| |PolynomialCategory&|

```

```

|DifferentialExtension&| |IndexedFlexibleArray| |AbelianMonoidRing&| | | | |
|FiniteAbelianMonoidRing&| |DivisionRing&| |FullyLinearlyExplicitRingOver&|
|IndexedVector| |IndexedOneDimensionalArray| |LocalAlgebra| |Localize|
|Boolean| |Field&| |Vector| |IndexedDirectProductObject| |Aggregate&|
|PolynomialRing| |FreeModule| |IndexedDirectProductAbelianGroup|
|IndexedDirectProductAbelianMonoid| |SingletonAsOrderedSet|
|SparseUnivariatePolynomial| |Fraction| |Collection&| |HomogeneousAggregate&|
|RepeatedSquaring| |IntegerNumberSystem&| |AbelianSemiGroup&|
|AssociationList| |OrderedRing&| |SemiGroup&| |Symbol|
|UniqueFactorizationDomain&| |EuclideanDomain&| |IndexedAggregate&|
|GcdDomain&| |IntegralDomain&| |DifferentialRing&| |Monoid&| |Reference|
|UnaryRecursiveAggregate&| |OrderedSet&| |AbelianGroup&| |Algebra&|
|Module&| |Ring&| |StringAggregate&| |AbelianMonoid&|
|ExtensibleLinearAggregate&| |PositiveInteger| |StreamAggregate&|
|IndexedString| |IndexedList| |ListAggregate&| |LinearAggregate&|
|Character| |String| |NonNegativeInteger| |SingleInteger|
|OneDimensionalArrayAggregate&| |FiniteLinearAggregate&| |PrimitiveArray|
|Integer| |List| |OutputForm|))
(dolist (con constr)
  (let ((c (concatenate 'string
                        (getenv "AXIOM") "/algebra/"
                        (string (getdatabase con 'abbreviation)) ".o"))))
    (format t "  preloading ~a.." c)
    (if (probe-file c)
        (progn
          (put con 'loaded c)
          (load c)
          (format t "loaded.~%")
          (format t "skipped.~%"))
        (format t "~%"))))

```

defun Open the interp database

Format of an entry in interp.daase:

```

(constructor-name
 operationalist
 constructormodemap
 modemaps          -- this should not be needed. eliminate it.
 object            -- the name of the object file to load for this con.
 constructorcategory -- note that this info is the cadar of the
                    constructormodemap for domains and packages so it is stored
                    as NIL for them. it is valid for categories.
 niladic           -- t or nil directly
 unused
 cosig             -- kept directly

```

```

        constructorkind      -- kept directly
        defaultdomain       -- a short list, for %i
        ancestors           -- used to compute new category updates
    )

[make-database p??]
[DaaseName p1209]
[$spadroot p201]
[*allconstructors* p1186]
[*interp-stream* p1184]
[*interp-stream-stamp* p1184]

```

— defun interpopen —

```

(defun interpopen ()
  "open the interpreter database and hash the keys"
  (declare (special $spadroot *allconstructors* *interp-stream*
                    *interp-stream-stamp*))
  (let (constructors pos stamp dbstruct)
    (setq *interp-stream* (open (DaaseName "interp.daase" nil)))
    (setq stamp (read *interp-stream*))
    (unless (equal stamp *interp-stream-stamp*)
      (format t "  Re-reading interp.daase")
      (setq *interp-stream-stamp* stamp)
      (setq pos (car stamp))
      (file-position *interp-stream* pos)
      (setq constructors (read *interp-stream*))
      (dolist (item constructors)
        (setq *allconstructors* (adjoin (first item) *allconstructors*))
        (setq dbstruct (make-database))
        (setf (get (car item) 'database) dbstruct)
        (setf (database-operationalist dbstruct) (second item))
        (setf (database-constructormodemap dbstruct) (third item))
        (setf (database-modemaps dbstruct) (fourth item))
        (setf (database-object dbstruct) (fifth item))
        (setf (database-constructorcategory dbstruct) (sixth item))
        (setf (database-niladic dbstruct) (seventh item))
        (setf (database-abbreviation dbstruct) (eighth item))
        (setf (get (eighth item) 'abbreviationfor) (first item)) ;invert
        (setf (database-cosig dbstruct) (ninth item))
        (setf (database-constructorkind dbstruct) (tenth item))
        (setf (database-ancestors dbstruct) (nth 11 item))))
    (format t "~&"))

```

This is an initialization function for the constructor database it sets up 2 hash tables, opens the database and hashes the index values.

There is a slight asymmetry in this code. The sourcefile information for system files is only the filename and extension. For user files it contains the full pathname. when the database is first opened the sourcefile slot contains system names. The lookup function has to prefix the “\$spadroot” information if the directory-namestring is null (we don’t know the real root at database build time).

An object-hash table is set up to look up nrlib and ao information. this slot is empty until a user does a)library call. We remember the location of the nrlib or ao file for the users local library at that time. A NIL result from this probe means that the library is in the system-specified place. When we get into multiple library locations this will also contain system files.

defun Open the browse database

Format of an entry in browse.daase:

```
( constructorname
  sourcefile
  constructorform
  documentation
  attributes
  predicates
)
```

```
[$spadroot p201]
[*allconstructors* p1186]
[*browse-stream* p1185]
[*browse-stream-stamp* p1185]
```

— defun browseopen —

```
(defun browseopen ()
  "open the constructor database and hash the keys"
  (declare (special $spadroot *allconstructors* *browse-stream*
    *browse-stream-stamp*))
  (let (constructors pos stamp dbstruct)
    (setq *browse-stream* (open (DaaseName "browse.daase" nil)))
    (setq stamp (read *browse-stream*))
    (unless (equal stamp *browse-stream-stamp*)
      (format t " Re-reading browse.daase")
      (setq *browse-stream-stamp* stamp)
      (setq pos (car stamp))
      (file-position *browse-stream* pos)
      (setq constructors (read *browse-stream*))
      (dolist (item constructors)
        (unless (setq dbstruct (get (car item) 'database))
          (format t "browseopen:~%")
          (format t "the browse database contains a constructor ~a~%" item))
```

```

(format t "that is not in the interp.daase file. we cannot~%")
(format t "get the database structure for this constructor and~%")
(warn "will create a new one~%")
(setf (get (car item) 'database) (setq dbstruct (make-database)))
(setq *allconstructors* (adjoin item *allconstructors*))
(setf (database-sourcefile dbstruct) (second item))
(setf (database-constructorform dbstruct) (third item))
(setf (database-documentation dbstruct) (fourth item))
(setf (database-attributes dbstruct) (fifth item))
(setf (database-predicates dbstruct) (sixth item))
(setf (database-parents dbstruct) (seventh item)))
(format t "~&"))

```

defun Open the category database

```

[$spadroot p201]
[*hasCategory-hash* p1183]
[*category-stream* p1185]
[*category-stream-stamp* p1186]

```

— defun categoryopen —

```

(defun categoryopen ()
  "open category.daase and hash the keys"
  (declare (special $spadroot *hasCategory-hash* *category-stream*
                    *category-stream-stamp*))
  (let (pos keys stamp)
    (setq *category-stream* (open (DaaseName "category.daase" nil)))
    (setq stamp (read *category-stream*))
    (unless (equal stamp *category-stream-stamp*)
      (format t "  Re-reading category.daase")
      (setq *category-stream-stamp* stamp)
      (setq pos (car stamp))
      (file-position *category-stream* pos)
      (setq keys (read *category-stream*))
      (setq *hasCategory-hash* (make-hash-table :test #'equal))
      (dolist (item keys)
        (setf (gethash (first item) *hasCategory-hash*) (second item))))
    (format t "~&"))

```

defun Open the operations database

[[\\$spadroot](#) [p201](#)]
 [[*operation-hash*](#) [p1183](#)]
 [[*operation-stream*](#) [p1184](#)]
 [[*operation-stream-stamp*](#) [p1185](#)]

— **defun operationopen** —

```
(defun operationopen ()
  "read operation database and hash the keys"
  (declare (special $spadroot *operation-hash* *operation-stream*
                    *operation-stream-stamp*))
  (let (operations pos stamp)
    (setq *operation-stream* (open (DaaseName "operation.daase" nil)))
    (setq stamp (read *operation-stream*))
    (unless (equal stamp *operation-stream-stamp*)
      (format t "  Re-reading operation.daase")
      (setq *operation-stream-stamp* stamp)
      (setq pos (car stamp))
      (file-position *operation-stream* pos)
      (setq operations (read *operation-stream*))
      (dolist (item operations)
        (setf (gethash (car item) *operation-hash*) (cdr item))))
    (format t "~&")))
```

—

defun Add operations from newly compiled code

[[getdatabase](#) [p1196](#)]
 [[*operation-hash*](#) [p1183](#)]

— **defun addoperations** —

```
(defun addoperations (constructor oldmaps)
  "add ops from a )library domain to *operation-hash*"
  (declare (special *operation-hash*))
  (dolist (map oldmaps) ; out with the old
    (let (oldop op)
      (setq op (car map))
      (setq oldop (getdatabase op 'operation))
      (setq oldop (lisp::delete (cdr map) oldop :test #'equal))
      (setf (gethash op *operation-hash*) oldop)))
  (dolist (map (getdatabase constructor 'modemaps)) ; in with the new
    (let (op newmap)
      (setq op (car map))
```



```
(setq newmap (getdatabase op 'operation))
(setf (gethash op *operation-hash*) (cons (cdr map) newmap))))
```

defun Show all database attributes of a constructor

[getdatabase p1196]

— defun showdatabase —

```
(defun showdatabase (constructor)
  (format t "~&~a: ~a~%" 'constructorkind
    (getdatabase constructor 'constructorkind))
  (format t "~&~a: ~a~%" 'cosig
    (getdatabase constructor 'cosig))
  (format t "~&~a: ~a~%" 'operation
    (getdatabase constructor 'operation))
  (format t "~&~a: ~%" 'constructormodemap)
  (pprint (getdatabase constructor 'constructormodemap))
  (format t "~&~a: ~%" 'constructorcategory)
  (pprint (getdatabase constructor 'constructorcategory))
  (format t "~&~a: ~%" 'operationalist)
  (pprint (getdatabase constructor 'operationalist))
  (format t "~&~a: ~%" 'modemaps)
  (pprint (getdatabase constructor 'modemaps))
  (format t "~&~a: ~a~%" 'hascategory
    (getdatabase constructor 'hascategory))
  (format t "~&~a: ~a~%" 'object
    (getdatabase constructor 'object))
  (format t "~&~a: ~a~%" 'niladic
    (getdatabase constructor 'niladic))
  (format t "~&~a: ~a~%" 'abbreviation
    (getdatabase constructor 'abbreviation))
  (format t "~&~a: ~a~%" 'constructor?
    (getdatabase constructor 'constructor?))
  (format t "~&~a: ~a~%" 'constructor
    (getdatabase constructor 'constructor))
  (format t "~&~a: ~a~%" 'defaultdomain
    (getdatabase constructor 'defaultdomain))
  (format t "~&~a: ~a~%" 'ancestors
    (getdatabase constructor 'ancestors))
  (format t "~&~a: ~a~%" 'sourcefile
    (getdatabase constructor 'sourcefile))
  (format t "~&~a: ~a~%" 'constructorform
    (getdatabase constructor 'constructorform))
  (format t "~&~a: ~a~%" 'constructorargs
    (getdatabase constructor 'constructorargs)))
```

```
(format t "~&~a: ~a~%" 'attributes
  (getdatabase constructor 'attributes))
(format t "~&~a: ~%" 'predicates)
  (pprint (getdatabase constructor 'predicates))
(format t "~&~a: ~a~%" 'documentation
  (getdatabase constructor 'documentation))
(format t "~&~a: ~a~%" 'parents
  (getdatabase constructor 'parents)))
```

defun Set a value for a constructor key in the database

[make-database p??]

— defun setdatabase —

```
(defun setdatabase (constructor key value)
  (let (struct)
    (when (symbolp constructor)
      (unless (setq struct (get constructor 'database))
        (setq struct (make-database))
        (setf (get constructor 'database) struct))
      (case key
        (abbreviation
          (setf (database-abbreviation struct) value)
          (when (symbolp value)
            (setf (get value 'abbreviationfor) constructor))))
        (constructorkind
          (setf (database-constructorkind struct) value))))))
```

defun Delete a value for a constructor key in the database

— defun deldatabase —

```
(defun deldatabase (constructor key)
  (when (symbolp constructor)
    (case key
      (abbreviation
        (setf (get constructor 'abbreviationfor) nil))))))
```

defun Get constructor information for a database key

```
[warn p??]
[$spadroot p201]
[*miss* p1183]
[*hascategory-hash* p??]
[*operation-hash* p1183]
[*browse-stream* p1185]
[*defaultdomain-list* p1182]
[*interp-stream* p1184]
[*category-stream* p1185]
[*hasCategory-hash* p1183]
[*operation-stream* p1184]
```

— **defun** getdatabase —

```
(defun getdatabase (constructor key)
  (declare (special $spadroot) (special *miss*))
  (when (eq *miss* t) (format t "getdatabase call: ~20a ~a~%" constructor key))
  (let (data table stream ignore struct)
    (declare (ignore ignore)
      (special *hascategory-hash* *operation-hash*
        *browse-stream* *defaultdomain-list* *interp-stream*
        *category-stream* *hasCategory-hash* *operation-stream*))
    (when (or (symbolp constructor)
      (and (eq key 'hascategory) (consp constructor))))
    (case key
      ; note that abbreviation, constructorkind and cosig are heavy hitters
      ; thus they occur first in the list of things to check
      (abbreviation
        (setq stream *interp-stream*)
        (when (setq struct (get constructor 'database))
          (setq data (database-abbreviation struct)))))
      (constructorkind
        (setq stream *interp-stream*)
        (when (setq struct (get constructor 'database))
          (setq data (database-constructorkind struct)))))
      (cosig
        (setq stream *interp-stream*)
        (when (setq struct (get constructor 'database))
          (setq data (database-cosig struct)))))
      (operation
        (setq stream *operation-stream*)
        (setq data (gethash constructor *operation-hash*)))
      (constructormodemap
        (setq stream *interp-stream*)
        (when (setq struct (get constructor 'database))
          (setq data (database-constructormodemap struct)))))
      (constructorcategory
```

```

(setq stream *interp-stream*)
(when (setq struct (get constructor 'database))
  (setq data (database-constructcategory struct))
  (when (null data) ;domain or package then subfield of constructormodemap
    (setq data (cadar (getdatabase constructor 'constructormodemap))))))
(operationalist
  (setq stream *interp-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-operationalist struct))))
(modemaps
  (setq stream *interp-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-modemaps struct))))
(hascategory
  (setq table *hasCategory-hash*)
  (setq stream *category-stream*)
  (setq data (gethash constructor table)))
(object
  (setq stream *interp-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-object struct))))
(niladic
  (setq stream *interp-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-niladic struct))))
(constructor?
  (when (setq struct (get constructor 'database))
    (setq data (when (database-operationalist struct) t))))
(superdomain ; only 2 superdomains in the world
  (case constructor
    (|NonNegativeInteger|
      (setq data '(|Integer|) (IF (< |#1| 0) |false| |true|))))
    (|PositiveInteger|
      (setq data '(|NonNegativeInteger|) (< 0 |#1|)))))
(constructor
  (when (setq data (get constructor 'abbreviationfor))))
(defaultdomain
  (setq data (cadar (assoc constructor *defaultdomain-list*)))))
(ancestors
  (setq stream *interp-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-ancestors struct))))
(sourcefile
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-sourcefile struct))))
(constructorform
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-constructorform struct))))

```

```

(constructorargs
  (setq data (cdr (getdatabase constructor 'constructorform))))
(attributes
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-attributes struct))))
(predicates
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-predicates struct))))
(documentation
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-documentation struct))))
(parents
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-parents struct))))
(users
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-users struct))))
(dependents
  (setq stream *browse-stream*)
  (when (setq struct (get constructor 'database))
    (setq data (database-dependents struct))))
(otherwise (warn "~%(GETDATABASE ~a ~a) failed~%" constructor key)))
(when (numberp data) ;fetch the real data
  (when *miss* (format t "getdatabase miss: ~20a ~a~%" constructor key))
  (file-position stream data)
  (setq data (read stream))
  (case key ; cache the result of the database read
    (operation (setf (gethash constructor *operation-hash*) data))
    (hascategory (setf (gethash constructor *hascategory-hash*) data))
    (constructorkind (setf (database-constructorkind struct) data))
    (cosig (setf (database-cosig struct) data))
    (constructormodemap (setf (database-constructormodemap struct) data))
    (constructorcategory (setf (database-constructorcategory struct) data))
    (operationalist (setf (database-operationalist struct) data))
    (modemaps (setf (database-modemaps struct) data))
    (object (setf (database-object struct) data))
    (niladic (setf (database-niladic struct) data))
    (abbreviation (setf (database-abbreviation struct) data))
    (constructor (setf (database-constructor struct) data))
    (ancestors (setf (database-ancestors struct) data))
    (constructorform (setf (database-constructorform struct) data))
    (attributes (setf (database-attributes struct) data))
    (predicates (setf (database-predicates struct) data))
    (documentation (setf (database-documentation struct) data))
    (parents (setf (database-parents struct) data))

```

```

(users          (setf (database-users struct) data))
(dependents     (setf (database-dependents struct) data))
(sourcefile     (setf (database-sourcefile struct) data)))
(case key ; fixup the special cases
  (sourcefile
    (when (and data (string= (directory-namestring data) ""))
      (string= (pathname-type data) "spad"))
    (setq data
      (concatenate 'string $spadroot "/../src/algebra/" data))))
  (object      ; fix up system object pathname
    (if (consp data)
      (setq data
        (if (string= (directory-namestring (car data)) "")
          (concatenate 'string $spadroot "/algebra/" (car data) ".o")
          (car data)))
      (when (and data (string= (directory-namestring data) ""))
        (setq data (concatenate 'string $spadroot "/algebra/" data ".o"))))))
  data))

```

defun The `)library` top level command

```

[localdatabase p1200]
[extendLocalLibdb p??]
[serverReadLine is-console (vol9)]
[tersyscommand p792]
[$newConlist p??]
[$options p73]

```

— defun library —

```

(defun |library| (args)
  (let (original-directory)
    (declare (special |$options| |$newConlist|))
    (setq original-directory (get-current-directory))
    (setq |$newConlist| nil)
    (localdatabase args |$options|)
    (|extendLocalLibdb| |$newConlist|)
    (system::chdir original-directory)
    (tersyscommand)))

```

defun Read a local filename and update the hash tables

The localdatabase function tries to find files in the order of: nrllib/index.kaf [sayKeyedMsg p43]

```
[localnrllib p1201]
[$forceDatabaseUpdate p??]
[$ConstructorCache p??]
[*index-filename* p??]
```

— defun localdatabase —

```
(defun localdatabase (filelist options &optional (make-database? nil))
  "read a local filename and update the hash tables"
  (labels (
    (processOptions (options)
      (let (only dir noexpose)
        (when (setq only (assoc '|only| options))
          (setq options (lisp::delete only options :test #'equal))
          (setq only (cdr only)))
        (when (setq dir (assoc '|dir| options))
          (setq options (lisp::delete dir options :test #'equal))
          (setq dir (second dir))
          (when (null dir)
            (|sayKeyedMsg|
              "Ignoring )dir because an explicit directory was not given after )dir."
              nil)))
        (when (setq noexpose (assoc '|noexpose| options))
          (setq options (lisp::delete noexpose options :test #'equal))
          (setq noexpose 't) )
        (when options
          (format t " Ignoring unknown )library option: ~a~%" options))
        (values only dir noexpose)))
    (processDir (dirarg thisdir)
      (let (allfiles)
        (declare (special vmlisp::*index-filename*))
        (system:chdir (string dirarg))
        (setq allfiles (directory "*"))
        (system:chdir thisdir)
        (mapcan #'(lambda (f)
          (when (string-equal (pathname-type f) "nrllib")
            (list (concatenate 'string (namestring f) "/"
                                vmlisp::*index-filename*)))) allfiles))))
    (let (thisdir nrllibs object only dir key (|$forceDatabaseUpdate| t) noexpose)
      (declare (special |$forceDatabaseUpdate| vmlisp::*index-filename*
                        |$ConstructorCache|))
      (setq thisdir (namestring (truename ".")))
      (setq noexpose nil)
      (multiple-value-setq (only dir noexpose) (processOptions options))
      ;don't force exposure during database build
```

```

(if make-database? (setq noexpose t))
(when dir (setq nrlibs (processDir dir thisdir)))
(dolist (file filelist)
  (let ((filename (pathname-name file))
        (namedir (directory-namestring file)))
    (unless namedir (setq thisdir (concatenate 'string thisdir "/"))))
    (cond
      ((setq file (probe-file
                  (concatenate 'string namedir filename ".nrlib/"
                               vmlisp::*index-filename*)))
        (push (namestring file) nrlibs))
      ('else (format t "  library cannot find the file ~a.~%" filename))))))
(dolist (file (nreverse nrlibs))
  (setq key (pathname-name (first (last (pathname-directory file)))))
  (setq object (concatenate 'string (directory-namestring file) "code"))
  (localnrlib key file object make-database? noexpose))
(clrhash |$ConstructorCache|)))

```

defun Update the database from an nrlib index.kaf file

```

[getdatabase p1196]
[make-database p??]
[addoperations p1193]
[sublislis p??]
[updateDatabase p1203]
[installConstructor p??]
[updateCategoryTable p??]
[categoryForm? p??]
[setExposeAddConstr p160]
[startTimingProcess p??]
[loadLibNoUpdate p1223]
[sayKeyedMsg p43]
[$FormalMapVariableList p17]
[*allOperations* p1186]
[*allconstructors* p1186]

```

— defun localnrlib —

```

(defun localnrlib (key nrlib object make-database? noexpose)
  "given a string pathname of an index.kaf and the object update the database"
  (labels (
    (fetchdata (alist in index)
      (let (pos)
        (setq pos (third (assoc index alist :test #'string=))))

```



```

    (when pos
      (file-position in pos)
      (read in))))
(let (alist kind (systemdir? nil) pos constructorform oldmaps abbrev dbstruct)
  (declare (special *allOperations* *allConstructors*
    |$FormalMapVariableList|))
  (with-open-file (in nrlib)
    (file-position in (read in))
    (setq alist (read in))
    (setq pos (third (assoc "constructorForm" alist :test #'string=)))
    (file-position in pos)
    (setq constructorform (read in))
    (setq key (car constructorform))
    (setq oldmaps (getdatabase key 'modemaps))
    (setq dbstruct (make-database))
    (setq *allConstructors* (adjoin key *allConstructors*))
    (setf (get key 'database) dbstruct) ; store the struct, side-effect it...
    (setf (database-constructorform dbstruct) constructorform)
    (setq *allOperations* nil) ; force this to recompute
    (setf (database-object dbstruct) object)
    (setq abbrev
      (intern (pathname-name (first (last (pathname-directory object))))))
    (setf (database-abbreviation dbstruct) abbrev)
    (setf (get abbrev 'abbreviationfor) key)
    (setf (database-operationalist dbstruct) nil)
    (setf (database-operationalist dbstruct)
      (fetchdata alist in "operationAlist"))
    (setf (database-constructormodemap dbstruct)
      (fetchdata alist in "constructorModemap"))
    (setf (database-modemaps dbstruct) (fetchdata alist in "modemaps"))
    (setf (database-sourcefile dbstruct) (fetchdata alist in "sourceFile"))
    (when make-database?
      (setf (database-sourcefile dbstruct)
        (file-namestring (database-sourcefile dbstruct))))
    (setf (database-constructorkind dbstruct)
      (setq kind (fetchdata alist in "constructorKind")))
    (setf (database-constructorkind dbstruct)
      (fetchdata alist in "constructorCategory"))
    (setf (database-documentation dbstruct)
      (fetchdata alist in "documentation"))
    (setf (database-attributes dbstruct)
      (fetchdata alist in "attributes"))
    (setf (database-predicates dbstruct)
      (fetchdata alist in "predicates"))
    (setf (database-niladic dbstruct)
      (when (fetchdata alist in "NILADIC") t))
    (addoperations key oldmaps)
    (unless make-database?
      (if (eq kind '|category|)
        (setf (database-ancestors dbstruct)

```

```

(sublislis |$FormalMapVariableList|
  (cdr constructorform) (fetchdata alist in "ancestors"))))
(|updateDatabase| key key systemdir?) ;makes many hashtables???
(|installConstructor| key kind) ;used to be key cname ...
(|updateCategoryTable| key kind)
(if |$InteractiveMode| (setq |$CategoryFrame| |$EmptyEnvironment|)))
(setf (database-cosig dbstruct)
  (cons nil (mapcar #'|categoryForm?|
    (cddar (database-constructormodemap dbstruct)))))
(remprop key 'loaded)
(if (null noexpose) (|setExposeAddConstr| (cons key nil)))
(setf (symbol-function key) ; sets the autoload property for cname
  #'(lambda (&rest args)
    (unless (get key 'loaded)
      (|startTimingProcess| '|load|)
      (|loadLibNoUpdate| key key object)) ; used to be cname key
    (apply key args)))
(|sayKeyedMsg| "%1 will be automatically loaded when needed from %2"
  (list key object))))

```

defun updateDatabase

For now in NRUNTIME do database update only if forced [constructor? p??]

```

[clearClams p??]
[clearAllSlams p??]
[$forceDatabaseUpdate p??]

```

— defun updateDatabase —

```

(defun |updateDatabase| (fname cname systemdirp)
  (declare (ignore fname))
  (declare (special |$forceDatabaseUpdate|))
  (when |$forceDatabaseUpdate|
    (when (|constructor?| cname)
      (|clearClams|)
      (|clearAllSlams| nil)
      (when (get1 cname 'loaded) (|clearConstructorCaches|)))
    (when (or |$forceDatabaseUpdate| (null systemdirp))
      (|clearClams|)
      (|clearAllSlams| nil))))

```

```
defvar *sourcefiles*
```

```
— initvars —
```

```
(defvar *sourcefiles* nil)
```

```
—————
```

defun Make new databases

Making new databases consists of:

1. reset all of the system hash tables
2. set up Union, Record and Mapping
3. map `)library` across all of the system files (fills the databases)
4. loading some normally autoloaded files
5. making some database entries that are computed (like ancestors)
6. writing out the databases
7. write out 'warm' data to be loaded into the image at build time

Note that this process should be done in a clean image followed by a rebuild of the system image to include the new index pointers (e.g. `*interp-stream-stamp*`)

The system will work without a rebuild but it needs to re-read the databases on startup. Rebuilding the system will cache the information into the image and the databases are opened but not read, saving considerable startup time. Also note that the order the databases are written out is critical. The `interp.daase` depends on prior computations and has to be written out last.

The `build-name-to-pamphlet-hash` builds a hash table whose key-value is:

- `abbreviation` — `>` pamphlet file name
- `abbreviation-line` — `>` pamphlet file position
- `constructor` — `>` pamphlet file name
- `constructor-line` — `>` pamphlet file position

is the symbol of the constructor name and whose value is the name of the source file without any path information. We hash the constructor abbreviation to pamphlet file name.

```

[localdatabase p1200]
[getenvirom p322]
[browserAutoloadOnceTrigger p??]
[mkTopicHashTable p??]
[buildLibdb p??]
[dbSplitLibdb p??]
[mkUsersHashTable p??]
[saveUsersHashTable p1208]
[mkDependentsHashTable p??]
[saveDependentsHashTable p1208]
[write-browsedb p1215]
[write-operationdb p1217]
[write-categorydb p1216]
[allConstructors p1218]
[categoryForm? p??]
[domainsOf p??]
[getConstructorForm p??]
[write-interpdb p1213]
[write-warmdata p1218]
[$constructorList p??]
[*sourcefiles* p1204]
[*allconstructors* p1186]
[*operation-hash* p1183]

```

— defun make-databases —

```

(defun make-databases (ext dirlist)
  (labels (
    (build-name-to-pamphlet-hash (dir)
      (let ((ht (make-hash-table)) (eof '(done)) point mark abbrev name file ns)
        (dolist (fn (directory dir))
          (when (and (string= (pathname-type fn) "pamphlet")
                      (or (string= (pathname-name fn) "bookvol10.2") ; category
                          (string= (pathname-name fn) "bookvol10.3") ; domain
                          (string= (pathname-name fn) "bookvol10.4") ; package
                          (string= (pathname-name fn) "bookvol10.5"))) ; numerics
            (with-open-file (f fn)
              (do ((ln (read-line f nil eof) (read-line f nil eof))
                  (line 0 (incf line)))
                  ((eq ln eof))
                (when (and (setq mark (search ")abb" ln)) (= mark 0))
                  (setq mark (position #\space ln :from-end t))
                  (setq name (intern (string-trim '(\space) (subseq ln mark))))
                  (cond
                    ((setq mark (search "domain" ln)) (setq mark (+ mark 7)))
                    ((setq mark (search "package" ln)) (setq mark (+ mark 8)))
                    ((setq mark (search "category" ln)) (setq mark (+ mark 9)))
                    (setq point (position #\space ln :start (+ mark 1))))

```

```

    (setq abbrev
      (intern (string-trim '(#\space) (subseq ln mark point))))
    (setq ns (namestring fn))
    (setq mark (position #\/ ns :from-end t))
    (setq file (subseq ns (+ mark 1)))
    (setf (gethash abbrev ht) file)
    (setf (gethash (format nil "~a-line" abbrev) ht) line)
    (setf (gethash name ht) file)
    (setf (gethash (format nil "~a-line" name) ht) line))))))
  ht))
;; these are types which have no library object associated with them.
;; we store some constructed data to make them perform like library
;; objects, the *operationalist-hash* key entry is used by allConstructors
(withSpecialConstructors ()
  (declare (special *allconstructors*))
  ; note: if item is not in *operationalist-hash* it will not be written
  ; Category
  (setf (get '|Category| 'database)
    (make-database :operationalist nil :niladic t))
  (push '|Category| *allconstructors*)
  ; UNION
  (setf (get '|Union| 'database)
    (make-database :operationalist nil :constructorkind '|domain|))
  (push '|Union| *allconstructors*)
  ; RECORD
  (setf (get '|Record| 'database)
    (make-database :operationalist nil :constructorkind '|domain|))
  (push '|Record| *allconstructors*)
  ; MAPPING
  (setf (get '|Mapping| 'database)
    (make-database :operationalist nil :constructorkind '|domain|))
  (push '|Mapping| *allconstructors*)
  ; ENUMERATION
  (setf (get '|Enumeration| 'database)
    (make-database :operationalist nil :constructorkind '|domain|))
  (push '|Enumeration| *allconstructors*)
  )
  (final-name (root)
    (format nil "~a.daase~a" root ext))
  )
)
(let (d)
  (declare (special |$constructorList| *sourcefiles*
    *allconstructors* *operation-hash*))
  (do-symbols (symbol)
    (when (get symbol 'database)
      (setf (get symbol 'database) nil)))
  (setq *hascategory-hash* (make-hash-table :test #'equal))
  (setq *operation-hash* (make-hash-table))
  (setq *allconstructors* nil)
  (withSpecialConstructors)

```

```

(localdatabase nil
  (list (list '|dir| (namestring (truename "./*")) ))
  'make-database)
(dolist (dir dirlist)
  (localdatabase nil
    (list (list '|dir| (namestring (truename (format nil "~a" dir))))
    'make-database)))
;browse.daase
(load (concatenate 'string (getenv "AXIOM") "/autoload/topics")) ;; hack
(|browserAutoloadOnceTrigger|)
(|mkTopicHashTable|)
(setq |$constructorList| nil) ;; affects buildLibdb
(setq *sourcefiles* (build-name-to-pamphlet-hash
  (concatenate 'string (getenv "AXIOM") "/../../*books/*pamphlet")))
(|buildLibdb|)
(|dbSplitLibdb|)
; (|dbAugmentConstructorDataTable|)
(|mkUsersHashTable|)
(|saveUsersHashTable|)
(|mkDependentsHashTable|)
(|saveDependentsHashTable|)
; (|buildGloss|)
(write-browsedb)
(write-operationdb)
; note: genCategoryTable creates a new *hascategory-hash* table
; this smashes the existing table and regenerates it.
; write-categorydb does getdatabase calls to write the new information
(write-categorydb)
(dolist (con (|allConstructors|))
  (let (dbstruct)
    (when (setq dbstruct (get con 'database))
      (setf (database-cosig dbstruct)
        (cons nil (mapcar #'|categoryForm?|
          (cddar (database-constructormodemap dbstruct))))))
    (when (and (|categoryForm?| con)
      (= (length (setq d (|domainsOf| (list con) NIL NIL))) 1))
      (setq d (caar d))
      (when (= (length d) (length (|getConstructorForm| con)))
        (format t " ~a has a default domain of ~a~%" con (car d))
        (setf (database-defaultdomain dbstruct) (car d))))))
    ; note: genCategoryTable creates *ancestors-hash*. write-interpdb
    ; does gethash calls into it rather than doing a getdatabase call.
  (write-interpdb)
  (write-warmdata)
  (when (probe-file (final-name "interp"))
    (delete-file (final-name "interp")))
  (rename-file "interp.build" (final-name "interp"))
  (when (probe-file (final-name "operation"))
    (delete-file (final-name "operation")))
  (rename-file "operation.build" (final-name "operation")))

```

```

    (when (probe-file (final-name "browse"))
      (delete-file (final-name "browse")))
    (rename-file "browse.build"
      (final-name "browse"))
    (when (probe-file (final-name "category"))
      (delete-file (final-name "category")))
    (rename-file "category.build"
      (final-name "category"))))

```

defun saveDependentsHashTable

```

[erase p??]
[writeLib1 p??]
[msort p??]
[hkeys p1235]
[rwrite p914]
[hget p1234]
[rshut p??]
[$depTb p??]
[$erase p??]

```

— defun saveDependentsHashTable —

```

(defun |saveDependentsHashTable| ()
  (let (stream)
    (declare (special |$depTb| $erase))
    ($erase '|dependents| 'database '|a|)
    (setq stream (|writeLib1| '|dependents| 'database '|a|))
    (dolist (k (msort (hkeys |$depTb|)))
      (|rwrite| k (hget |$depTb| k) stream))
    (rshut stream)))

```

defun saveUsersHashTable

```

[erase p??]
[writeLib1 p??]
[msort p??]
[hkeys p1235]
[rwrite p914]
[hget p1234]

```

```
[rshut p??]
[$erase p??]
[$usersTb p??]
```

— defun saveUsersHashTable —

```
(defun |saveUsersHashTable| ()
  (let (stream)
    (declare (special |$usersTb| $erase))
    ($erase '|users| 'database '|a|)
    (setq stream (|writeLib1| '|users| 'database '|a|))
    (dolist (k (msort (hkeys |$usersTb|)))
      (|rwrite| k (HGET |$usersTb| k) stream))
    (rshut stream)))
```

—————

defun Construct the proper database full pathname

```
[getenvIRON p322]
[$spadroot p201]
```

— defun DaaseName —

```
(defun DaaseName (name erase?)
  (let (daase filename)
    (declare (special $spadroot))
    (if (setq daase (getenvIRON "DAASE"))
      (progn
        (setq filename (concatenate 'string daase "/algebra/" name))
        (format t " Using local database ~a." filename))
      (setq filename (concatenate 'string $spadroot "/algebra/" name)))
    (when erase? (system::system (concatenate 'string "rm -f " filename)))
    filename))
```

—————

Building the interp.daase from hash tables

format of an entry in interp.daase:

```
(constructor-name
  operationalist
  constructormodemap
  modemaps          -- this should not be needed. eliminate it.
  object            -- the name of the object file to load for this con.
```



```

    constructorcategory -- note that this info is the cadar of the
                          constructormodemap for domains and packages so it is stored
                          as NIL for them. it is valid for categories.
    niladic              -- t or nil directly
    unused
    cosig                -- kept directly
    constructorkind      -- kept directly
    defaultdomain        -- a short list, for %i
    ancestors            -- used to compute new category updates
  )

```

Here I'll try to outline the interp database write procedure

```

(defun write-interpdb ()
  "build interp.daase from hash tables"
  (declare (special $spadroot *ancestors-hash*))
  (let (opalistpos modemapspos cmodemappos master masterpos obj *print-pretty*
        concategory categorypos kind niladic cosig abbrev defaultdomain
        ancestors ancestorspos out)
    (declare (special *print-pretty*))
    (print "building interp.daase")

; 1. We open the file we're going to create

    (setq out (open "interp.build" :direction :output))

; 2. We reserve some space at the top of the file for the key-time pair
;    We will overwrite these spaces just before we close the file.

    (princ "                                " out)

; 3. Make sure we write it out
    (finish-output out)

; 4. For every constructor in the system we write the parts:

    (dolist (constructor (|allConstructors|))
      (let (struct)

; 4a. Each constructor has a property list. A property list is a list
;     of (key . value) pairs. The property we want is called 'database
;     so there is a ('database . something) in the property list

        (setq struct (get constructor 'database))

; 5 We write the "operationsalist"
; 5a. We remember the current file position before we write
;     We need this information so we can seek to this position on read

        (setq opalistpos (file-position out))

```

```

; 5b. We get the "operationalist" and write it out

      (print (database-operationalist struct) out)

; 5c. We make sure it was written

      (finish-output out)

; 6 We write the "constructormodemap"
; 6a. We remember the current file position before we write

      (setq cmodemappos (file-position out))

; 6b. We get the "constructormodemap" and write it out

      (print (database-constructormodemap struct) out)

; 6c. We make sure it was written

      (finish-output out)

; 7. We write the "modemaps"
; 7a. We remember the current file position before we write

      (setq modemapspos (file-position out))

; 7b. We get the "modemaps" and write it out

      (print (database-modemaps struct) out)

; 7c. We make sure it was written

      (finish-output out)

; 8. We remember source file pathnames in the obj variable

      (setq obj
        (pathname-name
          (first (last (pathname-directory (database-object struct))))))

; 9. We write the "constructorcategory", if it is a category, else nil
; 9a. Get the constructorcategory

      (setq concategory (database-constructorcategory struct))

; 9b. If we have any data we write it out, else we don't write it
;      Note that if there is no data then the byte index for the
;      constructorcategory will not be a number but will be nil.

```

```

      (if concategory ; if category then write data else write nil
        (progn
          (setq categorypos (file-position out))
          (print concategory out)
          (finish-output out))
        (setq categorypos nil))

; 10. We get a set of properties which are kept as "immediate" data
;     This means that the key table will hold this data directly
;     rather than as a byte index into the file.
; 10a. niladic data

      (setq niladic (database-niladic struct))

; 10b. abbreviation data (e.g. POLY for polynomial)

      (setq abbrev (database-abbreviation struct))

; 10c. cosig data

      (setq cosig (database-cosig struct))

; 10d. kind data

      (setq kind (database-constructorkind struct))

; 10e. defaultdomain data

      (setq defaultdomain (database-defaultdomain struct))

; 11. The ancestor data might exist. If it does we fetch it
;     and write it out. If it does not we place
;     and immediate value of nil in the key-value table

      (setq ancestors (gethash constructor *ancestors-hash*)) ;cattable.boot
      (if ancestors
        (progn
          (setq ancestorpos (file-position out))
          (print ancestors out)
          (finish-output out))
        (setq ancestorpos nil))

; 12. "master" is an alist. Each element of the alist has the name of
;     the constructor and all of the above attributes. When the loop
;     finishes we will have constructed all of the data for the key-value
;     table

      (push (list constructor opalistpos cmodemappos modemappos
        obj categorypos niladic abbrev cosig kind defaultdomain
        ancestorpos) master)))

```

```

; 13. The loop is done, we make sure all of the data is written

  (finish-output out)

; 14. We remember where the key-value table will be written in the file

  (setq masterpos (file-position out))

; 15. We print the key-value table

  (print master out)

; 16. We make sure we write the table

  (finish-output out)

; 17. We go to the top of the file

  (file-position out 0)

; 18. We write out the (master-byte-position . universal-time) pair
;     Note that if the universal-time value matches the value of
;     *interp-stream-stamp* then there is no reason to read the
;     interp database because all of the data is already cached in
;     the image. This happens if you build a database and immediatly
;     save the image. The saved image already has the data since we
;     just wrote it out. If the *interp-stream-stamp* and the database
;     time stamp differ we "reread" the database on startup. Actually
;     we just open the database and fetch as needed. You can see fetches
;     by setting the *miss* variable non-nil.

  (print (cons masterpos (get-universal-time)) out)

; 19. We make sure we write it.

  (finish-output out)

; 20 And we are done

  (close out)))

```

defun Write the interp database

```

[$spadroot p201]
[*ancestors-hash* p??]
[*print-pretty* p??]

```

— defun write-interpdb —

```

(defun write-interpdb ()
  "build interp.daase from hash tables"
  (declare (special $spadroot *ancestors-hash*))
  (let (opalistpos modemapspos cmodemappos master masterpos obj *print-pretty*
        concategory categorypos kind niladic cosig abbrev defaultdomain
        ancestors ancestorspos out)
    (declare (special *print-pretty*))
    (print "building interp.daase")
    (setq out (open "interp.build" :direction :output))
    (princ " " out)
    (finish-output out)
    (dolist (constructor (|allConstructors|))
      (let (struct)
        (setq struct (get constructor 'database))
        (setq opalistpos (file-position out))
        (print (database-operationalist struct) out)
        (finish-output out)
        (setq cmodemappos (file-position out))
        (print (database-constructormodemap struct) out)
        (finish-output out)
        (setq modemapspos (file-position out))
        (print (database-modemaps struct) out)
        (finish-output out)
        (setq obj
          (pathname-name
            (first (last (pathname-directory (database-object struct))))))
        (setq concategory (database-constructcategory struct))
        (if concategory ; if category then write data else write nil
          (progn
            (setq categorypos (file-position out))
            (print concategory out)
            (finish-output out)
            (setq categorypos nil))
          (setq niladic (database-niladic struct))
          (setq abbrev (database-abbreviation struct))
          (setq cosig (database-cosig struct))
          (setq kind (database-constructorkind struct))
          (setq defaultdomain (database-defaultdomain struct))
          (setq ancestors (gethash constructor *ancestors-hash*)) ;cattable.boot
          (if ancestors
            (progn
              (setq ancestorspos (file-position out))
              (print ancestors out)
              (finish-output out)
              (setq ancestorspos nil))
            (push (list constructor opalistpos cmodemappos modemapspos
                        obj categorypos niladic abbrev cosig kind defaultdomain
                        ancestorspos) master)))

```

```

(finish-output out)
(setq masterpos (file-position out))
(print master out)
(finish-output out)
(file-position out 0)
(print (cons masterpos (get-universal-time)) out)
(finish-output out)
(close out)))

```

Building the browse.daase from hash tables

format of an entry in browse.daase:

```

( constructorname
  sourcefile
  constructorform
  documentation
  attributes
  predicates
)

```

This is essentially the same overall process as write-interpdb.

We reserve some space for the (key-table-byte-position . timestamp)

We loop across the list of constructors dumping the data and remembering the byte positions in a key-value pair table.

We dump the final key-value pair table, write the byte position and time stamp at the top of the file and close the file.

defun Write the browse database

```

[allConstructors p1218]
[$spadroot p201]
[*sourcefiles* p1204]
[*print-pretty* p??]

```

— defun write-browsedb —

```

(defun write-browsedb ()
  "make browse.daase from hash tables"
  (declare (special $spadroot *sourcefiles*))
  (let (master masterpos src formpos docpos attpos predpos *print-pretty* out)
    (declare (special *print-pretty*))
    (print "building browse.daase")

```

```

(setq out (open "browse.build" :direction :output))
(princ " " out)
(finish-output out)
(dolist (constructor (|allConstructors|))
  (let (struct)
    (setq struct (get constructor 'database))
    ; sourcefile is small. store the string directly
    (setq src (gethash constructor *sourcefiles*))
    (setq formpos (file-position out))
    (print (database-constructorform struct) out)
    (finish-output out)
    (setq docpos (file-position out))
    (print (database-documentation struct) out)
    (finish-output out)
    (setq attpos (file-position out))
    (print (database-attributes struct) out)
    (finish-output out)
    (setq predpos (file-position out))
    (print (database-predicates struct) out)
    (finish-output out)
    (push (list constructor src formpos docpos attpos predpos) master)))
(finish-output out)
(setq masterpos (file-position out))
(print master out)
(finish-output out)
(file-position out 0)
(print (cons masterpos (get-universal-time)) out)
(finish-output out)
(close out)))

```

Building the category.daase from hash tables

This is a single table of category hash table information, dumped in the database format.

defun Write the category database

```

[genCategoryTable p??]
[*print-pretty* p??]
[*hasCategory-hash* p1183]

```

— defun write-categorydb —

```

(defun write-categorydb ()
  "make category.daase from scratch. contains the *hasCategory-hash* table"

```

```

(let (out master pos *print-pretty*)
  (declare (special *print-pretty* *hasCategory-hash*))
  (print "building category.daase")
  (|genCategoryTable|)
  (setq out (open "category.build" :direction :output))
  (princ " " out)
  (finish-output out)
  (maphash #'(lambda (key value)
    (if (or (null value) (eq value t))
      (setq pos value)
      (progn
        (setq pos (file-position out))
        (print value out)
        (finish-output out)))
    (push (list key pos) master))
    *hasCategory-hash*)
  (setq pos (file-position out))
  (print master out)
  (finish-output out)
  (file-position out 0)
  (print (cons pos (get-universal-time)) out)
  (finish-output out)
  (close out)))

```

Building the operation.daase from hash tables

This is a single table of operations hash table information, dumped in the database format.

defun Write the operations database

[*operation-hash* p1183]

— defun write-operationdb —

```

(defun write-operationdb ()
  (let (pos master out)
    (declare (special leaves *operation-hash*))
    (setq out (open "operation.build" :direction :output))
    (princ " " out)
    (finish-output out)
    (maphash #'(lambda (key value)
      (setq pos (file-position out))
      (print value out)
      (finish-output out))
      leaves)
    (close out)))

```



```

(push (cons key pos) master))
*operation-hash*)
(finish-output out)
(setq pos (file-position out))
(print master out)
(file-position out 0)
(print (cons pos (get-universal-time)) out)
(finish-output out)
(close out)))

```

Database support operations

defun Data preloaded into the image at build time

[[\\$topicHash p??](#)]

— defun write-warmdata —

```

(defun write-warmdata ()
  "write out information to be loaded into the image at build time"
  (declare (special |$topicHash|))
  (with-open-file (out "warm.data" :direction :output)
    (format out "(in-package \"BOOT\")~%" )
    (format out "(setq |$topicHash| (make-hash-table))~%" )
    (maphash #'(lambda (k v)
      (format out "(setf (gethash '|~a| |$topicHash|) ~a)~%" k v)) |$topicHash|)))

```

defun Return all constructors

[[*allconstructors*](#) [p1186](#)]

— defun allConstructors —

```

(defun |allConstructors| ()
  (declare (special *allconstructors*))
  *allconstructors*)

```

defun Return all operations

[*allOperations* p1186]
 [*operation-hash* p1183]

— defun allOperations —

```
(defun |allOperations| ()
  (declare (special *allOperations* *operation-hash*))
  (unless *allOperations*
    (maphash #'(lambda (k v) (declare (ignore v)) (push k *allOperations*))
              *operation-hash*))
  *allOperations*)
```

—————

Chapter 36

System Statistics

defun statisticsInitialization

[gbc-time p??]

— defun statisticsInitialization —

```
(defun |statisticsInitialization| ()  
  "initialize the garbage collection timer"  
  #+:akcl (system:gbc-time 0)  
  nil)
```

—————

36.1 Lisp Library Handling

defun loadLib

```
[startTimingProcess p??]  
[getdatabase p1196]  
[isSystemDirectory p1223]  
[pathnameDirectory p1232]  
[loadLibNoUpdate p1223]  
[sayKeyedMsg p43]  
[namestring p1230]  
[clearConstructorCache p??]  
[updateDatabase p1203]  
[installConstructor p??]  
[updateCategoryTable p??]
```

```

[categoryForm? p??]
[remprop p??]
[stopTimingProcess p??]
[$InteractiveMode p315]
[$printLoadMsgs p1008]
[$forceDatabaseUpdate p??]
[$CategoryFrame p??]

```

— defun loadLib —

```

(defun |loadLib| (cname)
  (let (fullLibName systemdir? update? kind u sig coSig)
    (declare (special |$CategoryFrame| |$InteractiveMode| |$printLoadMsgs|
                      |$forceDatabaseUpdate|))
    (|startTimingProcess| '|load|)
    (when (setq fullLibName (getdatabase cname 'object))
      (setq systemdir? (|isSystemDirectory| (|pathnameDirectory| fullLibName)))
      (setq update? (or |$forceDatabaseUpdate| (null systemdir?)))
      (cond
        ((null update?) (|loadLibNoUpdate| cname cname fullLibName))
        (t
         (setq kind (getdatabase cname 'constructorkind))
         (when |$printLoadMsgs|
           (|sayKeyedMsg| "Loading %1 for %2 %3"
            (list (|namestring| fullLibName) kind cname)))
         (load fullLibName)
         (|clearConstructorCache| cname)
         (|updateDatabase| cname cname systemdir?)
         (|installConstructor| cname kind)
         (setq u (getdatabase cname 'constructormodemap))
         (|updateCategoryTable| cname kind)
         (setq coSig
          (when u
           (setq sig (cdar u))
           (cons nil (loop for x in (cdr sig) collect (|categoryForm?| x))))))
        (if (null (cdr (getdatabase cname 'constructorform)))
          (setf (get cname 'niladic) t)
          (remprop cname 'niladic))
        (setf (get cname 'loaded) fullLibName)
        (when |$InteractiveMode| (setq |$CategoryFrame| (list (list nil))))
        (|stopTimingProcess| '|load|)
        t))))))

```

defun isSystemDirectory

```
[function p??]
[$spadroot p201]
```

— defun isSystemDirectory —

```
(defun |isSystemDirectory| (dir)
  (declare (special $spadroot))
  (every #'char= $spadroot dir))
```

—————

defun loadLibNoUpdate

```
[getdatabase p1196]
[sayKeyedMsg p43]
[toplevel p??]
[clearConstructorCache p??]
[installConstructor p??]
[stopTimingProcess p??]
[$printLoadMsgs p1008]
[$InteractiveMode p315]
[$CategoryFrame p??]
```

— defun loadLibNoUpdate —

```
(defun |loadLibNoUpdate| (cname libName fullLibName)
  (declare (ignore libName))
  (let (kind)
    (declare (special |$CategoryFrame| |$InteractiveMode| |$printLoadMsgs|))
    (setq kind (getdatabase cname 'constructorkind))
    (when |$printLoadMsgs|
      (|sayKeyedMsg| "Loading %1 for %2 %3"
        (list (|namestring| fullLibName) kind cname)))
    (cond
      ((equal (catch 'versioncheck (load fullLibName)) (- 1))
        (princ "    wrong library version...recompile ")
        (princ fullLibName)
        (terpri)
        (toplevel))
      (t
        (|clearConstructorCache| cname)
        (|installConstructor| cname kind)
        (setf (get cname 'loaded) fullLibName)
        (when |$InteractiveMode| (setq |$CategoryFrame| (list (list nil))))))
```

```
(|stopTimingProcess| 'load|'))
t))
```

defun loadFunctor

```
[loadFunctor p1224]
[loadLibIfNotLoaded p??]
```

— **defun loadFunctor** —

```
(defun |loadFunctor| (u)
  (cond
    ((null (atom u)) (|loadFunctor| (car u)))
    (t
     (|loadLibIfNotLoaded| u)
     u)))
```

Chapter 37

Special Lisp Functions

defun compiledLookup

[isDomain p??]
[NRTevalDomain p1229]

— defun compiledLookup —

```
(defun |compiledLookup| (op sig dollar)
  (setq dollar (|NRTevalDomain| dollar))
  (|basicLookup| op sig dollar dollar))
```

—————

defmacro hashCode?

— defmacro hashCode? 0 —

```
(defmacro |hashCode?| (x)
  `(integerp ,x))
```

—————

defun basicLookup

[spadcall p??]
[hashCode? p1225]


```

[opIsHasCat p??]
[HasCategory p??]
[hashType p??]
[hashString p??]
[error p??]
[vecp p??]
[isNewWorldDomain p??]
[oldCompLookup p1229]
[lookupInDomainVector p1227]
[$hashSeg p??]
[$hashOpSet p??]
[$hashOpApply p??]
[$hashOp0 p??]
[$hashOp1 p??]

```

— defun basicLookup —

```

(defun |basicLookup| (op sig domain dollar)
  (let (hashPercent box dispatch lookupFun hashSig val boxval)
    (declare (special |$hashSeg| |$hashOpSet| |$hashOpApply| |$hashOp0|
                      |$hashOp1|))
    (cond
      ((simple-vector-p domain)
       (if (|isNewWorldDomain| domain)
           (|oldCompLookup| op sig domain dollar)
           (|lookupInDomainVector| op sig domain dollar)))
      (t
       (setq hashPercent
              (if (simple-vector-p dollar)
                  (|hashType| (elt dollar 0) 0)
                  (|hashType| dollar 0)))
       (setq box (cons nil nil))
       (cond
         ((null (simple-vector-p (setq dispatch (car domain))))
          (|error| '|bad domain format|))
         (t
          (setq lookupFun (elt dispatch 3))
          (cond
            ((eq1 (elt dispatch 0) 0)
             (setq hashSig
                    (cond
                     ((|hashCode?| sig) sig)
                     ((|opIsHasCat| op) (|hashType| sig hashPercent))
                     (t (|hashType| (cons '|Mapping| sig) hashPercent))))
              (when (symbolp op)
                (cond
                 ((eq op '|Zero|) (setq op |$hashOp0|))
                 ((eq op '|One|) (setq op |$hashOp1|))

```

```

      ((eq op '|elt|)      (setq op |$hashOpApply|))
      ((eq op '|setelt|)  (setq op |$hashOpSet|))
      (t                  (setq op (|hashString| (symbol-name op)))))
    (cond
      ((setq val
        (car
          (spadcall (cdr domain) dollar op hashSig box nil lookupFun)))
        val)
      ((|hashCode?| sig) nil)
      ((or (> (|#| sig) 1) (|opIsHasCat| op)) nil)
      ((setq boxval
        (spadcall (cdr dollar) dollar op
          (|hashType| (car sig) hashPercent)
          box nil lookupFun))
        (cons #'identity (car boxval)))
      (t nil)))
    ((|opIsHasCat| op) (|HasCategory| domain sig))
    (t
      (when (|hashCode?| op)
        (cond
          ((eq op |$hashOp1|)      (setq op '|One|))
          ((eq op |$hashOp0|)      (setq op '|Zero|))
          ((eq op |$hashOpApply|)  (setq op '|elt|))
          ((eq op |$hashOpSet|)    (setq op '|setelt|))
          ((eq op |$hashSeg|)      (setq op '|segment|)))
        (cond
          ((and (|hashCode?| sig) (eq sig hashPercent))
            (spadcall
              (car (spadcall (cdr dollar) dollar op '($) box nil lookupFun))))
          (t
            (car
              (spadcall (cdr dollar) dollar op sig box nil lookupFun))))))))))

```

defun lookupInDomainVector

[basicLookupCheckDefaults p1228]

[spadcall p??]

— defun lookupInDomainVector —

```

(defun |lookupInDomainVector| (op sig domain dollar)
  (if (consp domain)
    (|basicLookupCheckDefaults| op sig domain domain)
    (spadcall op sig dollar (elt domain 1))))

```

defun basicLookupCheckDefaults

```
[vecp p??]
[error p??]
[hashType p??]
[hashCode? p1225]
[hashString p??]
[spadcall p??]
[$lookupDefaults p??]
```

— defun basicLookupCheckDefaults —

```
(defun |basicLookupCheckDefaults| (op sig domain dollar)
  (declare (ignore domain))
  (let (box dispatch lookupFun hashPercent hashSig)
    (declare (special |$lookupDefaults|))
    (setq box (cons nil nil))
    (cond
      ((null (simple-vector-p (setq dispatch (car dollar))))
        (|error| '|bad domain format|))
      (t
        (setq lookupFun (elt dispatch 3))
        (cond
          ((eql (elt dispatch 0) 0)
            (setq hashPercent
              (if (simple-vector-p dollar)
                (|hashType| (elt dollar 0) 0)
                (|hashType| dollar 0)))
            (setq hashSig
              (if (|hashCode?| sig)
                sig
                (|hashType| (cons '|Mapping| sig) hashPercent)))
            (when (symbolp op) (setq op (|hashString| (symbol-name op))))
            (car (spadcall (cdr dollar) dollar op hashSig
                          box (null |$lookupDefaults|) lookupFun)))
          (t
            (car (spadcall (cdr dollar) dollar op sig box
                          (null |$lookupDefaults|) lookupFun)))))))
```

defun oldCompLookup

[lookupInDomainVector p1227]
 [\$lookupDefaults p??]

— defun oldCompLookup —

```
(defun |oldCompLookup| (op sig domvec dollar)
  (let (|$lookupDefaults| u)
    (declare (special |$lookupDefaults|))
    (setq |$lookupDefaults| nil)
    (cond
      ((setq u (|lookupInDomainVector| op sig domvec dollar))
       u)
      (t
       (setq |$lookupDefaults| t)
       (|lookupInDomainVector| op sig domvec dollar))))))
```

—————

defun NRTevalDomain

[qcar p??]
 [eval p??]
 [evalDomain p1115]

— defun NRTevalDomain —

```
(defun |NRTevalDomain| (form)
  (if (and (consp form) (eq (qcar form) 'setelt))
      (|eval| form)
      (|evalDomain| form)))
```

—————

37.1 Axiom control structure macros

Axiom used various control structures in the boot code which are not available in Common Lisp. We write some macros here to make the boot to lisp translations easier to read.

defun put

— defun put —

```
(defun put (sym ind val) (setf (get sym ind) val))
```

defmacro while

While the condition is true, repeat the body. When the condition is false, return t.

— **defmacro while** —

```
(defmacro while (condition &rest body)
  '(loop (if (not ,condition) (return t)) ,@body))
```

defmacro whileWithResult

While the condition is true, repeat the body. When the condition is false, return the result form's value.

— **defmacro whileWithResult** —

```
(defmacro whileWithResult (condition result &rest body)
  '(loop (if (not ,condition) ,@result) ,@body))
```

37.2 Filename Handling

This code implements the Common Lisp pathname functions for Lisp/VM. On VM, a filename is 3-list consisting of the filename, filetype and filemode. We also UPCASE everything.

defun namestring

```
[pathname p1232]
```

— **defun namestring** —

```
(defun |namestring| (arg)
  (namestring (|pathname| arg)))
```

defun pathnameName

[pathname p¹²³²]

— defun pathnameName —

```
(defun |pathnameName| (arg)
  (pathname-name (|pathname| arg)))
```

—————

defun pathnameType

[pathname p¹²³²]

— defun pathnameType —

```
(defun |pathnameType| (arg)
  (pathname-type (|pathname| arg)))
```

—————

defun pathnameTypeId

[upcase p¹²⁷²]

[object2Identifier p??]

[pathnameType p¹²³¹]

— defun pathnameTypeId —

```
(defun |pathnameTypeId| (arg)
  (upcase (|object2Identifier| (|pathnameType| arg))))
```

—————

defun mergePathnames

[pathnameName p¹²³¹]

[pathnameType p¹²³¹]

[pathnameDirectory p¹²³²]

— defun mergePathnames —

```
(defun |mergePathnames| (a b)
  (let (fn ft fm)
    (cond
      ((string= (setq fn (|pathnameName| a)) "*") b)
      ((not (equal fn (|pathnameName| b))) a)
      ((string= (setq ft (|pathnameType| a)) "*") b)
      ((not (equal ft (|pathnameType| b))) a)
      ((equal (setq fm (|pathnameDirectory| a)) (list "*" )) b)
      (t a))))
```

defun pathnameDirectory

[pathname p¹²³²]

— defun pathnameDirectory —

```
(defun |pathnameDirectory| (arg)
  (namestring (make-pathname :directory (pathname-directory (|pathname| arg)))))
```

defun Axiom pathnames

[pathname p¹²³²]

[make-filename p??]

— defun pathname —

```
(defun |pathname| (p)
  (cond
    ((null p) p)
    ((pathnamep p) p)
    ((null (consp p)) (pathname p))
    (t
     (when (> (|#| p) 2) (setq p (cons (elt p 0) (cons (elt p 1) nil))))
     (pathname (apply #'make-filename p)))))
```

defun makePathname

```
[pathname p1232]
[object2String p??]
```

— defun makePathname —

```
(defun |makePathname| (name type dir)
  (declare (ignore dir))
  (|pathname| (list (|object2String| name) (|object2String| type))))
```

—————

defun Delete a file

```
[erase p??]
[pathname p1232]
[$erase p??]
```

— defun deleteFile —

```
(defun |deleteFile| (arg)
  (declare (special $erase))
  ($erase (|pathname| arg)))
```

—————

defun wrap

```
[lotsof p1234]
[wrap p1233]
```

— defun wrap —

```
(defun wrap (list-of-items wrapper)
  (prog nil
    (cond
      ((or (not (consp list-of-items)) (not wrapper))
        (return list-of-items))
      ((not (consp wrapper))
        (setq wrapper (lotsof wrapper))))
    (return
      (cons
        (if (first wrapper)
```



```

      ‘(,(first wrapper) ,(first list-of-items))
      (first list-of-items))
      (wrap (cdr list-of-items) (cdr wrapper))))))

```

defun lotsof

— defun lotsof —

```

(defun lotsof (&rest items)
  (setq items (copy-list items))
  (nconc items items))

```

defmacro startsId?

— defmacro startsId? —

```

(defmacro |startsId?| (x)
  ‘(or (alpha-char-p ,x) (member ,x ‘(#\? #\% #\!) :test #'char=)))

```

defun hput

— defun hput —

```

(defun hput (table key value)
  (setf (gethash key table) value))

```

defmacro hget

— defmacro hget —

```
(defmacro HGET (table key &rest default)
  `(gethash ,key ,table ,@default))
```

defun hkeys

— defun hkeys —

```
(defun hkeys (table)
  (let (keys)
    (maphash
     #'(lambda (key val) (declare (ignore val)) (push key keys)) table)
    keys))
```

defun digitp

[digitp p1235]

— defun digitp —

```
(defun digitp (x)
  (or (and (symbolp x) (digitp (symbol-name x)))
      (and (characterp x) (digit-char-p x))
      (and (stringp x) (= (length x) 1) (digit-char-p (char x 0)))))
```

defun pname

Note it is important that PNAME returns nil not an error for non-symbols

— defun pname 0 —

```
(defun pname (x)
  (cond ((symbolp x) (symbol-name x))
        ((characterp x) (string x))
        (t nil)))
```

defun size

— defun size —

```
(defun size (l)
  (cond
    ((vectorp l) (length l))
    ((consp l)  (list-length l))
    (t 0)))
```

defun strpos

— defun strpos —

```
(defun strpos (what in start dontcare)
  (setq what (string what) in (string in))
  (if dontcare
    (progn
      (setq dontcare (character dontcare))
      (search what in :start2 start
                :test #'(lambda (x y) (or (eql x dontcare) (eql x y)))))
    (if (= start 0)
      (search what in)
      (search what in :start2 start)))))
```

defun strposl

Note that this assumes “table” is a string.

— defun strposl —

```
(defun strposl (table cvec sint item)
  (setq cvec (string cvec))
  (if (not item)
    (position table cvec :test #'(lambda (x y) (position y x)) :start sint)
    (position table cvec :test-not #'(lambda (x y) (position y x)) :start sint)))
```

defmacro identp— **defmacro identp 0** —

```
(defmacro identp (x)
  (if (atom x)
      '(and ,x (symbolp ,x))
      (let ((xx (gensym)))
        '(let ((,xx ,x))
            (and ,xx (symbolp ,xx)))))))
```

defun concat

[string-concatenate p??]

— **defun concat 0** —

```
(defun concat (a b &rest l)
  (if (bit-vector-p a)
      (if l
          (apply #'concatenate 'bit-vector a b l)
          (concatenate 'bit-vector a b))
      (if l
          (apply #'system:string-concatenate a b l)
          (system:string-concatenate a b))))
```

This function was called `|functionp|` which is a lower-case version of the common lisp function called `functionp`. Camm Maguire found a bug related to this ambiguity so this was renamed.

defun canFuncall?— **defun canFuncall?** —

```
(defun canFuncall? (fn)
  (if (identp fn)
      (and (fboundp fn) (not (macro-function fn)))
      (functionp fn)))
```

```
;; ----- NEW DEFINITION (override in msgdb.boot.pamphlet)
```

defun brightprint

```
[messageprint p1239]
```

— defun brightprint —

```
(defun brightprint (x)
  (messageprint x))
```

```
;; ----- NEW DEFINITION (override in msgdb.boot.pamphlet)
```

defun brightprint-0

```
[messageprint-1 p1239]
```

— defun brightprint-0 —

```
(defun brightprint-0 (x)
  (messageprint-1 x))
```

defun member

— defun member 0 —

```
(defun |member| (item sequence)
  (cond
    ((symbolp item) (member item sequence :test #'eq))
    ((stringp item) (member item sequence :test #'equal))
    ((and (atom item) (not (arrayp item))) (member item sequence))
    (t (member item sequence :test #'equalp))))
```

defun messageprint

— defun messageprint —

```
(defun messageprint (x)
  (mapc #'messageprint-1 x))
```

—

defun messageprint-1

```
[identp p1237]
[messageprint-1 p1239]
[messageprint-2 p1239]
```

— defun messageprint-1 —

```
(defun messageprint-1 (x)
  (cond
    ((or (eq x '|%l|) (equal x "%l")) (terpri))
    ((stringp x) (princ x))
    ((identp x) (princ x))
    ((atom x) (princ x))
    ((princ "(")
     (messageprint-1 (car x))
     (messageprint-2 (cdr x))
     (princ ")"))))
```

—

defun messageprint-2

```
[messageprint-1 p1239]
[messageprint-2 p1239]
```

— defun messageprint-2 —

```
(defun messageprint-2 (x)
  (if (atom x)
      (unless x (progn (princ " . ") (messageprint-1 x)))
      (progn (princ " ") (messageprint-1 (car x)) (messageprint-2 (cdr x)))))
```

—

defun sayBrightly1

```
[brightprint-0 p1238]
[brightprint p1238]
```

— defun sayBrightly1 —

```
(defun sayBrightly1 (x *standard-output*)
  (if (atom x)
      (progn (brightprint-0 x) (terpri) (force-output))
      (progn (brightprint x) (terpri) (force-output))))
```

—————

defmacro assq

TPDHERE: This could probably be replaced by the default assoc using eql

— defmacro assq —

```
(defmacro assq (a b)
  '(assoc ,a ,b :test #'eq))
```

—————

defun A version of GET that works with lists

— defun getl 0 —

```
(defun getl (op prop)
  (when (and op (symbolp op)) (get op prop)))
```

—————

Chapter 38

Record, Union, Mapping, and Enumeration

— postvars —

```
(eval-when (eval load)
  (mapcar #'(lambda (alist)
    (setf (get (first alist) '|makeFunctionList|) (second alist)))
    '((|Record| |mkRecordFunList|)
      (|Union| |mkUnionFunList|)
      (|Mapping| |mkMappingFunList|)
      (|Enumeration| |mkEnumerationFunList|))))
```

—————

Chapter 39

Numeric Function Support

defmacro fracpart

fracpart : or rational float $\rightarrow$ or rational float
— defmacro fracpart 0 —

```
(defmacro fracpart (x)
  '(cadr (multiple-value-list (floor ,x))))
```

—————

defun list to complex conversion

— defun s-to-c 0 —

```
(defun s-to-c (c)
  (complex (car c) (cdr c)))
```

—————

defun complex to list conversion

— defun c-to-s 0 —

```
(defun c-to-s (c)
  (cons (realpart c) (imagpart c)))
```

defun complex to real conversion

— defun c-to-r —

```
(defun c-to-r (c)
  (let ((r (realpart c)) (i (imagpart c)))
    (if (or (zerop i) (< (abs i) (* 1.0E-10 (abs r))))
        r
        (|error| "Result is not real.)))
```

defmacro FloatError

— defmacro FloatError 0 —

```
(defmacro FloatError (formatstring arg)
  `(error (format nil ,formatstring ,arg)))
```

defun Rational approximation to $\Gamma(x)$

[phiRatapprox p1244]

— defun lnrgammaRatapprox —

```
(defun lnrgammaRatapprox (x)
  "(x-.5)*log(x) - x + log(sqrt(2.0*Pi)) + phiRatapprox(x)"
  (+ (+ (- (* (- x 0.5) (log x)) x)
        (log (sqrt (* 2.0 Pi)))))
     (phiRatapprox x)))
```

defun phiRatapprox

[horner p1247]

— defun phiRatapprox —

```
(defun phiRatapprox (x)
  (let ((arg (/ 1 (* x x))))
    (/
      (horner
        '(0.066662907040200753 0.64507302912899211
          0.67082783834332138 0.12398282342474939) arg)
      (* x
        (horner
          '(1.0 7.9966911236636431 8.0995271894897574 1.4877938810969931) arg))))))
```

defun Log approximation to $\Gamma(x)$

[lnrgammaRatapprox p1244]
 [gammaRatapprox p1246]

— defun lnrgamma —

```
(defun lnrgamma (x)
  (if (< 20 x)
      (lnrgammaRatapprox x)
      (log (gammaRatapprox x))))
```

defun Stirling's approximation to $\Gamma(x)$

[lnrgamma p1245]

— defun gammaStirling —

```
(defun gammaStirling (x)
  (exp (lnrgamma x)))
```

defun rgammaImpl

This code implements the real $\Gamma(x)$ function used in DoubleFloatSpecialFunctions. [FloatError p1244]
 [gammaStirling p1245]

[gammaRatapprox p1246]

— defun rgammaImpl —

```
(defun rgammaImpl (x)
  (when (complexp x)
    (FloatError "Gamma not implemented for complex value ~D" x))
  (if (zerop (- x 1.0))
      1.0
      (if (< 20 x)
          (gammaStirling x)
          (gammaRatapprox x)))))
```

—————

defun gammaRatapprox

[gammaRatapprox p1246]

[FloatError p1244]

[gammaRatkernel p1247]

— defun gammaRatapprox —

```
(defun gammaRatapprox (x)
  (let (restx intpartx lx prod reducedarg a n result)
    (cond
      ((and (not (< x 2)) (not (< 3 x)))
       (setq result (gammaRatkernel x)))
      ((< 3 x)
       (setq n (- (floor x) 2))
       (setq a (- (- x n) 2))
       (setq reducedarg (+ 2 a))
       (setq prod
        (reduce #'* (loop for i from 0 to (- n 1) collect (+ reducedarg i))))
       (setq result (* prod (gammaRatapprox reducedarg))))
      ((and (< x 2) (< 0 x))
       (setq n (- 2 (floor x)))
       (setq a (- x (floor x)))
       (setq reducedarg (+ 2 a))
       (setq prod (reduce #'* (loop for i from 0 to (- n 1) collect (+ x i))))
       (setq result (/ (gammaRatapprox reducedarg) prod)))
      (t
       (setq lx (multiple-value-list (floor x)))
       (setq intpartx (+ (car lx) 1))
       (setq restx (cadr lx))
       (cond
         ((zerop restx)
```

```

(FloatError "Gamma undefined for non-positive integers: ~D" x)
(setq result (/ 1.0 (complex 0.0)))
(t
 (setq result
  (/ Pi
   (* (* (gammaRatapprox (- 1.0 x))
        (expt (- 1.0) (+ intpartx 1)))
      (sin (* restx Pi))))))
result))

```

defun gammaRatkernel

[horner p1247]

— defun gammaRatkernel —

```

(defun gammaRatkernel (x)
  (/
   (horner
    '(0.77807958561330059 6.1260674503360839 48.95434622790993
      222.11239616801177 893.58180452374972 2077.4597938941874
      3786.0105034825724)
    (- x 2))
   (horner
    '(1 -13.400414785781347 50.788475328895402 83.550058667919771
      -867.23098753110298 476.79386050368794 3786.0105034825719)
    (- x 2.0))))

```

defun Horner's rule of polynomial evaluation

— defun horner 0 —

```

(defun horner (l x)
  (let ((result 0))
    (loop for el in l do (setq result (+ (* x result) el)))
    result))

```

defun Complex implementation of $\Gamma(z)$

[rgammaImpl p1245]
 [clngammaImpl p1248]

— defun cgammaImpl —

```
(defun cgammaImpl (z)
  (if (zerop (imagpart z))
      (rgammaImpl (realpart z))
      (exp (clngammaImpl (realpart z) (imagpart z) z))))
```

—————

Compute the conjugate of gamma which is the gamma of the conjugate. Map the 2nd and the 4th quadrants to first and third quadrants.

defun Compute the conjugate of $\Gamma(z)$

[clngammacase1 p1248]
 [clngammacase23 p1250]

— defun clngammaImpl —

```
(defun clngammaImpl (real imag z)
  (cond
    ((< real 0.0)
     (if (< 0.0 imag)
         (conjugate (clngammacase1 real (- imag) (complex real (- imag))))
         (clngammacase1 real imag z)))
    ((< imag 0.0)
     (conjugate (clngammacase23 real (- imag) (complex real (- imag)))))
    (t
     (clngammacase23 real imag z))))
```

—————

defun $\Gamma(z)$ negative real branch

[PiMinusLogSinPi p1249]
 [clngammaImpl p1248]

— defun clngammacase1 —

```
(defun clngammacase1 (real imag z)
```

```
(- (PiMinusLogSinPi real imag z)
   (cloggammaImpl (- 1.0 real) (- imag) (- 1.0 z))))
```

defun PiMinusLogSinPi

[cgammaG p1249]
[logH p1249]

— defun PiMinusLogSinPi —

```
(defun PiMinusLogSinPi (real imag z)
  (- (cgammaG real imag) (logH real imag z)))
```

defun cgammaG

— defun cgammaG 0 —

```
(defun cgammaG (real imag)
  (- (+ (log (* 2 Pi)) (* Pi imag))
      (* (* (complex 0.0 1.0) Pi) (- real 0.5)))))
```

defun logH

— defun logH 0 —

```
(defun logH (real imag z)
  (declare (ignore z))
  (let (part1 part2 twopiz2 z1bar)
    (setq z1bar (cadr (multiple-value-list (floor real))))
    (setq twopiz2 (* 2.0 (* Pi imag)))
    (setq part2
      (* (exp twopiz2)
         (+ (* 2.0 (expt (sin (* Pi z1bar)) 2))
            (* (sin (* 2.0 (* Pi z1bar))) (complex 0.0 1.0)))))
    ;--- part1 is another way of saying 1 - exp(2*Pi*z1bar)
```



```
(setq part1 (- (* (tanh (* Pi imag)) (+ 1.0 (exp twopiz2)))))
(log (+ part1 part2)))
```

defun $\Gamma(z)$ positive real branch

```
[cgammat p1250]
[clngammacase2 p1250]
[clngammacase3 p1251]
```

— defun clngammacase23 —

```
(defun clngammacase23 (real imag z)
  (let ((tz2 (cgammat imag)))
    (if (< real tz2)
        (clngammacase2 real imag tz2 z)
        (clngammacase3 z))))
```

defun cgammat

The cgammat is auxiliary "t" function [kuki72a, kuki72b]

— defun cgammat 0 —

```
(defun cgammat (x)
  (max 0.10000000000000001 (min 10.0 (- (* 10.0 (sqrt 2.0)) (abs x)))))
```

defun $\Gamma(z)$ case 2

```
[cgammaBernsum p1252]
[cgammaAdjust p1251]
[logS p1251]
```

— defun clngammacase2 —

```
(defun clngammacase2 (real imag tz2 z)
  (let (zpn n)
    (setq n (float (ceiling (- tz2 real)))))
```

```
(setq zpn (+ z n))
(- (+ (- (* (- z 0.5) (log zpn)) zpn) (cgammaBernsum zpn))
   (cgammaAdjust (logS real imag z n zpn))))
```

defun logS

— defun logS 0 —

```
(defun logS (real imag z n zpn)
  (let ((sum 0.0))
    (dotimes (k n)
      (if (< (+ real k) (- 5.0 (* 0.6000000000000009 imag)))
          (setq sum (+ sum (log (/ (+ z k) zpn)))))
      (setq sum (+ sum (log (- 1.0 (/ (- n k) zpn)))))
    sum))
```

defun Adjust logS if imaginary part is negative

The logS result should have its imaginary part adjusted by 2π if it is negative. [[kuki72a](#), [kuki72b](#)]

— defun cgammaAdjust 0 —

```
(defun cgammaAdjust (z)
  (if (< (imagpart z) 0.0)
      (+ z (complex 0.0 (* 2.0 Pi)))
      z))
```

defun $\Gamma(z)$ case 3

[cgammaBernsum p[1252](#)]

— defun clngammacase3 —

```
(defun clngammacase3 (z)
  (+ (- (* (- z 0.5) (log z)) z) (cgammaBernsum z)))
```

defun cgammaBernsum

— defun cgammaBernsum 0 —

```
(defun cgammaBernsum (z)
  (let (l zsquaredinv zterm sum)
    (setq sum (/ (log (* 2.0 Pi)) 2.0))
    (setq zterm z)
    (setq zsquaredinv (/ 1.0 (* z z)))
    (setq l
      (list 0.083333333333333315 (- 0.002777777777777779)
            7.9365079365079376E-4 (- 5.9523809523809529E-4)
            8.4175084175084182E-4 (- 0.0019175269175269176)
            0.0064102564102564109))
    (loop for el in l do
      (setq zterm (* zterm zsquaredinv))
      (setq sum (+ sum (* el zterm))))
    sum))
```

—————→

defun Bessell

[Bessell p1252]
 [FloatError p1244]
 [bessellback p1253]
 [bessellcheb p1254]

— defun Bessell —

```
(defun Bessell (v z)
  (let ((b1 15.0))
    (cond
      ((and (zerop z) (floatp v) (not (< v 0.0)))
       (if (zerop v) 1.0 0.0))
      ; Transformations for negative integer orders
      ((and (floatp v) (zerop (fracpart v)) (minusp v))
       (Bessell (- v) z))
      ; Halfplane transformations for Re(z)<0
      ((< (realpart z) 0.0)
       (* (Bessell v (- z)) (expt (- 1.0) v)))
      ; Conjugation for complex order and real argument
      ((and (< (realpart v) 0.0) (null (zerop (imagpart v))) (floatp z))
       (conjugate (Bessell (conjugate v) z)))
      ; We now know that Re(z)>= 0.0 (asymptotic argument case)
      ((< b1 (abs z))
```

```

      (FloatError "Besseli not implemented for ~S" (list v z)))
    ((< b1 (abs v))
      (FloatError "Besseli not implemented for ~S" (list v z)))
    ; case of small argument and order
    ((not (< (realpart v) 0.0))
      (besselIback v z))
    ((< (realpart v) 0.0)
      (besselIcheb z v 50))
    (t
      (FloatError "Besseli not implemented for ~S" (list v z))))))

```

defun besselIback

[BesselIBackRecur p1253]
 [cgammaImpl p1248]

— defun besselIback —

```

(defun besselIback (v z)
  (let (result vp1 tv n m lm rpv ipv)
    (setq ipv (imagpart v))
    (setq rpv (realpart v))
    (setq lm (multiple-value-list (floor rpv)))
    (setq m (car lm)) ; floor of real part of v
    (setq n (* 2 (max 20 (+ m 10)))) ; how large the back recurrence should be
    (setq tv (+ (cadr lm) (- v rpv))) ; fractional part of real part of v
    ; plus imaginary part of v
    (setq vp1 (+ tv 1.0))
    (setq result (BesselIBackRecur v m tv z "I" n))
    (setq result (* (/ result (cgammaImpl vp1)) (expt (/ z 2.0) tv)))))

```

defun Backward recurrence for Bessel functions

Backward recurrence for Bessel functions. Luke (1975), p. 247. works for $-\pi < \arg z \leq \pi$
 and $-\pi < \arg v \leq \pi$

— defun BesselIBackRecur 0 —

```

(defun BesselIBackRecur (largev argm v z type n)
  (declare (ignore largev))
  (let (v1 pn ct1 xm m m1 val w m2 z2 start zero two one)
    (setq one 1.0)

```

```

(setq two 2.0)
(setq zero 0.0)
(setq start (expt 10.0 (- 40)))
(setq z2 (/ two z))
(setq m2 (+ n 3))
(setq w (make-array (+ m2 1)))
(setf (aref w m2) zero)
(if (string= type "I")
    (setq val one)
    (setq val (- one)))
(setq m1 (+ n 2))
(setf (aref w m1) start)
(setq m (+ n 1))
(setq xm (float m))
(setq ct1 (* z2 (+ xm v)))
; initialize
(loop for m from (+ n 1) downto 1 by 1 do
  (setf (aref w m) (+ (* (aref w (+ m 1)) ct1) (* val (aref w (+ m 2))))
  (setq ct1 (- ct1 z2)))
(setq m (+ 1 (floor (/ n 2))))
(setq m2 (- (+ m m) 1))
(cond
  ((eql v 0)
   (setq pn (aref w (+ m2 2)))
   (loop for m2 from (- (* 2 m) 1) downto 3 by 2 do
     (setq pn (- (aref w m2) (* val pn))))
   (setq pn (- (aref w 1) (* val (+ pn pn)))))
  ('T (setq v1 (- v one)) (setq xm (float m))
   (setq ct1 (+ (+ v xm) xm))
   (setq pn (* ct1 (aref w (+ m2 2))))
   (loop for m2 from (- (+ m m) 1) downto 3 by 2 do
     (setq ct1 (- ct1 two))
     (setq pn (- (* ct1 (aref w m2)) (* (/ (* val pn) xm) (+ v1 xm))))
     (setq xm (- xm one)))
   (setq pn (- (aref w 1) (* val pn)))))
(setq m1 (+ n 2))
(loop for m from 1 to m1 do
  (setf (aref w m) (/ (aref w m) pn)))
(aref w (+ argm 1)))

```

defun Compute n terms of the chebychev series for f01

[chebf01coefmake p1255]
 [chebstarevalarr p1256]
 [cgammaImpl p1248]

— defun bessellCheb —

```
(defun bessellCheb (z v n)
  (let (result arr sum tmp1 vp1 w arg)
    (setq arg (/ (* z z) 4.0))
    (setq w (* 2.0 arg))
    (setq vp1 (+ v 1.0))
    (setq tmp1 (chebf01coefmake vp1 w n))
    (setq sum (car tmp1))
    (setq arr (cadr tmp1))
    (setq result
      (* (/ (chebstarevalarr arr (/ arg w) n) (cgammaImpl vp1))
        (expt (/ z 2.0) v))))))
```

—————

defun chebf01coefmake

Program transcribed from Fortran,, p. 80 Luke 1977 [Luke77]

where c is a parameter to 0F1, w is a scale factor so that $0 < z/w < 1$, n such that $n + 2$ coefficients will be produced stored in an array indexed from 0 to $n + 1$. The *arr* array will be used to store the Cheb. series coefficients

— defun chebf01coefmake 0 —

```
(defun chebf01coefmake (c w n)
  (let (p sum rho divfac c1 x1 arr ncount z1 a1 a2 a3 n2 n1 start four)
    (setq four 4.0)
    (setq start (expt 10.0 (- 200)))
    (setq n1 (+ n 1))
    (setq n2 (+ n 2))
    (setq a3 0.0)
    (setq a2 0.0)
    (setq a1 start) ; arbitrary starting value
    (setq z1 (/ four w))
    (setq ncount n1)
    (setq arr (make-array n2))
    (setf (aref arr ncount) start) ; start off
    (setq x1 n2)
    (setq c1 (- 1.0 c))
    (loop for ncount from n downto 0 do
      (setq divfac (/ 1.0 x1))
      (setq x1 (- x1 1.0))
      (setf (aref arr ncount)
        (* x1
```

```

      (- (+ (* (+ divfac (* z1 (- x1 c1))) a1)
            (* (+ (/ 1.0 x1) (* z1 (+ (+ x1 c1) 1.0))) a2))
        (* divfac a3))))
    (setq a3 a2)
    (setq a2 a1)
    (setq a1 (aref arr ncount)))
  (setf (aref arr 0) (/ (aref arr 0) 2.0))
  ; compute scale factor
  (setq rho (aref arr 0))
  (setq sum rho)
  (setq p 1.0)
  (loop for i from 1 to n1 do
    (setq rho (- rho (* p (aref arr i))))
    (setq sum (+ sum (aref arr i)))
    (setq p (- p)))
  (loop for l from 0 to n1 do
    (setf (aref arr l) (/ (aref arr l) rho)))
  (setq sum (/ sum rho))
  (list sum arr))

```

defun chebstarevalarr

Evaluation of the sum($C(n)*T^{(n,x)}$)

— defun chebstarevalarr 0 —

```

(defun chebstarevalarr (coefarr x n)
  (let (c y temp b)
    (setq b 0)
    (setq temp 0)
    (setq y (* 2 (- (* 2 x) 1)))
    (loop for i from (+ n 1) downto 0 do
      (setq c b)
      (setq b temp)
      (setq temp (+ (- (* y b) c) (aref coefarr i))))
    (- temp (/ (* y b) 2))))

```

defun lngamma

[clngamma p1278]

— defun lngamma —

```
(defun lncgamma (z)
  (clngammaImpl (realpart z) (imagpart z) z))
```

defun rPsiImpl

[FloatError p1244]
 [cotdiffeval p1257]
 [rPsiImpl p1257]
 [rPsiW p1258]
 [rgammaImpl p1245]

— defun rPsiImpl —

```
(defun rPsiImpl (n x)
  (let (skipit sign m)
    (cond
      ((not (< 0.0 x))
        (cond
          ((zerop (fracpart x))
            (FloatError "singularity encountered at ~D" x))
          (t
            (setq m (mod n 2))
            (setq sign (expt (- 1) m))
            (if (equal (fracpart x) 0.5)
              (setq skipit 1)
              (setq skipit 0))
            (* sign
              (+ (* (expt Pi (+ n 1))
                (cotdiffeval n (* Pi (- x)) skipit))
                (rPsiImpl n (- 1.0 x)))))))
      ((eql n 0) (- (rPsiW n x)))
      (t
        (* (* (rgammaImpl (float (+ n 1))) (rPsiW n x))
          (expt (- 1) (mod (+ n 1) 2)))))))
```

defun cotdiffeval

Code for computation of derivatives of $\cot(z)$, necessary for polygamma reflection formula. If you want to compute n^{th} derivatives of $\cot(\pi * x)$, you have to multiply the result of cotdiffeval by π^n .

Set *skip* = 1 if arg *z* is known to be an exact multiple of $\pi/2$

— **defun cotdiffeval 0** —

```
(defun cotdiffeval (n z skipit)
  (let (s sq v t2 t1 m a)
    (setq a (make-array (+ n 2)))
    (setf (aref a 0) 0.0)
    (setf (aref a 1) 1.0)
    (loop for i from 2 to n do
      (setf (aref a i) 0.0))
    (loop for r from 1 to n do
      (setq m (mod (+ r 1) 2))
      (loop for k from m to (+ z 1) by 2 do
        (if (< k 1)
          (setq t1 0)
          (setq t1 (- (* (aref a (- k 1)) (- k 1)))))
        (if (< r k)
          (setq t2 0)
          (setq t2 (- (* (aref a (+ k 1)) (+ k 1)))))
        (setf (aref a k) (+ t1 t2))))
    ; evaluate d^N/dX^N cot(z) via Horner-like rule
    (setq v (cot z))
    (setq sq (* v v))
    (setq s (aref a (+ n 1)))
    (loop for i from (- n 1) downto 0 by 2 do
      (setq s (+ (* s sq) (aref a i))))
    (setq m (mod n 2))
    (when (eql m 0) (setq s (* s v)))
    (if (eql skipit 1)
      (if (eql m 0) 0 (aref a 0))
      s)))
```

—

defun Amos' w function

Amos' w function, with $w(0,x)$ picked to be $-\psi(x)$ for $x > 0$ [PsiAsymptoticOrder p1259]
 [PsiAsymptotic p1260]
 [PsiBack p1259]

— **defun rPsiW** —

```
(defun rPsiW (n x)
  (let (bign fln c a xmin beta alpha nd result)
    (when (or (not (< 0 x)) (minusp n))
      (error "rPsiW not implemented for negative n or non-positive x"))
    (setq nd 6) ; magic number for number of digits in a word?
```

```

(setq alpha (+ 3.5 (* 0.40000000000000002 nd)))
(setq beta
  (+ (+ 0.20999999999999999 (* 8.6770000000000001E-6 (- nd 3)))
    (* 6.0380000000000002E-8 (expt (- nd 3) 2))))
(setq xmin (float (+ (floor (+ alpha (* beta n))) 1)))
(when (< 0 n)
  (setq a
    (min 0 (* (/ 1.0 (float n)) (log (/ double-float-epsilon (min 1.0 x))))))
  (setq c (exp a))
  (if (not (< (abs a) 0.001))
    (setq fln (* (/ x c) (- 1.0 c)))
    (setq fln (- (/ (* x a) c))))
  (setq bign (+ (floor fln) 1))
  ; Amos says to use alternative series for large order if ordinary
  ; backwards recurrence too expensive
  (when (and (< bign 15) (< (+ 7.0 x) xmin))
    (setq result (PsiAsymptoticOrder n x bign)))
  (when (and (not result) (not (< x xmin)))
    (setq result (PsiAsymptotic n x)))
  (unless result
    ; ordinary case -- use backwards recursion
    (setq result (PsiBack n x xmin)))
  result))

```

defun PsiAsymptoticOrder

— defun PsiAsymptoticOrder 0 —

```

(defun PsiAsymptoticOrder (n x nterms)
  (loop for k from 0 to nterms
    sum (/ 1.0 (expt (+ x (float k)) (+ n 1)))))

```

defun PsiBack

[PsiIntpart p1261]
 [PsiAsymptotic p1260]

— defun PsiBack —

```

(defun PsiBack (n x xmin)

```

```
(let (result x0 xintpart)
  (setq xintpart (PsiIntpart x))
  (setq x0 (- x xintpart)) ; frac part of x
  (setq result (PsiAsymptotic n (+ (+ x0 xmin) 1.0)))
  ; Why not decrement from x? See Amos p. 498
  (loop for k from xmin downto xintpart by 1 do
    (setq result (+ result (/ 1.0 (expt (+ x0 (float k)) (+ n 1))))))
  result))
```

defvar PsiAsymptoticBern

The n^{th} derivatives of $\ln \Gamma(x)$ for real x , $n = 0, 1, \dots$

— initvars —

```
(defvar |$PsiAsymptoticBern|
  (vector 0.0 0.16666666666666669 (- 0.03333333333333333)
          0.023809523809523812 (- 0.03333333333333333)
          0.07575757575757576 (- 0.2531135531135531)
          1.1666666666666672 (- 7.0921568627450986)
          54.971177944862163 (- 529.1242424242424)
          6192.123188405797 (- 86580.253113553103)
          1425517.166666667 (- 2.729823106781609E7)
          6.015808739006424E8 (- 1.5116315767092161E10)
          4.2961464306116675E11 (- 1.371165520508833E13)
          4.8833231897359325E14 (- 1.9296579341940072E16)
          8.4169304757368269E17 (- 4.0338071854059463E19)))
```

defun PsiAsymptotic

[[\\$PsiAsymptoticBern](#) p1260]
 [[rgammaImpl](#) p1245]
 [[PsiEps](#) p1261]

— defun PsiAsymptotic —

```
(defun PsiAsymptotic (n x)
  (let (sum factterm xterm xsq xnp1 xn)
    (declare (special |$PsiAsymptoticBern|))
    (setq xn (expt x n))
    (setq xnp1 (* xn x))
    (setq xsq (* x x))
```

```

(setq xterm xsq)
(setq factterm (/ (/ (rgammaImpl (+ n 2)) 2.0) (rgammaImpl (float (+ n 1)))))
; initialize to 1/n!
(setq sum (/ (* (aref |$PsiAsymptoticBern| 1) factterm) xterm))
(loop for k from 2 to 22 do
  (setq xterm (* xterm xsq))
  (cond
    ((eql n 0) (setq factterm (/ 1.0 (float (* 2 k)))))
    ((eql n 1) (setq factterm 1))
    (t
     (setq factterm
       (/ (* factterm (float (- (+ (* 2 k) n) 1)))
          (float (- (+ (* 2 k) n) 2)))
          (* (float (* 2 k)) (float (- (* 2 k) 1)))))))
    (setq sum (+ sum (/ (* (aref |$PsiAsymptoticBern| k) factterm) xterm))))
(+ (+ (PsiEps n x) (/ 1.0 (* 2.0 xnp1))) (* (/ 1.0 xn) sum)))

```

defun PsiEps

— defun PsiEps 0 —

```

(defun PsiEps (n x)
  (if (eql n 0)
      (- (log x))
      (/ 1.0 (* (float n) (expt x n)))))

```

defun PsiIntpart

— defun PsiIntpart 0 —

```

(defun PsiIntpart (x)
  (if (minusp x)
      (- (PsiIntpart (- x)))
      (floor x)))

```

defun cPsiImpl— **defun cPsiImpl** —

```

(defun cPsiImpl (n z)
  (prog (result m bound nterms conjresult y x)
    (return
      (progn
        (setq x (realpart z))
        (setq y (imagpart z))
        (cond ((zerop y) (return (rPsiImpl n x))) ; call real function if real
              (when (< y 0.0) ; if imagpart(z) negative, take conjugate of conjugate
                (setq conjresult (cPsiImpl n (complex x (- y))))
                (return (complex (realpart conjresult) (- (imagpart conjresult)))))
              (t (return (complex (realpart conjresult) (- (imagpart conjresult))))))
        (setq nterms 22)
        (setq bound 10.0)
        (cond
          ((< x 0.0)
            (FloatError "Psi implementation can't compute at ~S " (list n z)))
          ((and (< 0.0 x) (< bound (abs z)))
            (return (PsiXotic n (PsiAsymptotic n z))))
          (t ; use recursion formula
            (setq m (ceiling (+ (- (sqrt (- (* bound bound) (* y y))) x) 1.0)))
            (setq result (complex 0.0 0.0))
            (loop for k from 0 to (- m 1) do
              (setq result (+ result (/ 1.0 (expt (+ z (float k)) (+ n 1))))))
            (return (PsiXotic n (+ result (PsiAsymptotic n (+ z m))))))))))

```

defun PsiXotic

The n^{th} derivatives of $\ln \Gamma$ for complex z , $n = 0, 1, \dots$ requires files `rpsi` (and dependents), floaterrors currently defined only in right half plane until reflection formula works

— **defun PsiXotic** —

```

(defun PsiXotic (n result)
  (* (* (rgammaImpl (float (+ n 1))) (expt (- 1) (mod (+ n 1) 2))) result))

```

defun BesselJ

BesselJ works for complex and real values of v and z

— **defun BesselJ** —

```

(defun BesselJ (v z)
  (let (arr sum t1 vp1 w arg rz rv n b2 b1)
    (setq b1 10) ; Ad hoc boundaries for approximation
    (setq b2 10)
    (setq n 50) ; number of terms in Chebychev series.
    (cond
      ; tests for negative integer order
      ((or (and (floatp v) (zerop (fracpart v)) (minuspl v))
            (and (complexp v) (zerop (imagpart v)) (zerop (fracpart (realpart v)))
                  (< (realpart v) 0.0))))
        ; odd or even according to v (9.1.5 A&S)
        ;  $J_{-n}(z) = (-1)^n J_n(z)$ 
        (* (BesselJ (- v) z) (expt (- 1.0) v)))
      ((or (and (floatp z) (minusp z)) (and (complexp z) (< (realpart z) 0.0)))
        ; negative argument (9.1.35 A&S)
        ;  $J_{\nu}(z e^{m\pi i}) = e^{m\nu\pi i} J_{\nu}(z)$ 
        (* (BesselJ v (- z)) (expt (- 1.0) v)))
      ((and (zerop z)
            (or (and (floatp v) (not (< v 0.0)))
                (and (complexp v) (zerop (imagpart v))
                    (not (< (realpart v) 0.0)))))
        ; zero arg, pos. real order
        (if (zerop v) 1.0 0.0))
      (t
        (setq rv (abs v))
        (setq rz (abs z))
        (cond
          ((and (< b1 rz) (< (* b2 rv) rz)) ; asymptotic argument
            (BesselJAsympt v z))
          ((and (< b1 rv) (< (* b2 rz) rv)) ; asymptotic order
            (BesselJAsymptOrder v z))
          ((and (< rz b1) (< rv b1)) ; small order and argument
            (setq arg (- (/ (* z z) 4.0)))
            (setq w (* 2.0 arg))
            (setq vp1 (+ v 1.0))
            (setq t1 (chebf01coefmake vp1 w n))
            (setq sum (car t1))
            (setq arr (cadr t1))
            ; if we get NaNs then half n
            ((lambda ()
              (loop
                (cond
                  ((= sum sum) (return nil))
                  (t
                   (setq n (floor (/ n 2)))
                   (setq t1 (chebf01coefmake vp1 w n))
                   (setq sum (car t1))
                   (setq arr (cadr t1))
                   t1))))))
            ; now n is safe, can we increase it (we know that 2*n is bad)?

```

```

(* (/ (chebstarevalarr arr (/ arg w) n) (cgammaImpl vp1))
   (expt (/ z 2.0) v)))
(t (BesselJRecur v z))))))

```

Asymptotic functions for large values of z . See Luke [Luke69a] p. 204 where **mu** is $4v^2$, **zsqr** is z^2 , and **zfth** is z^4

defun Asymptotic series for BesselJ

Asymptotic series only works when $|v| < |z|$.

— defun BesselJAsympt —

```

(defun BesselJAsympt (v z)
  (let (zfth zsqr mu)
    (setq mu (* (* 4.0 v) v))
    (setq zsqr (* z z))
    (setq zfth (* zsqr zsqr))
    (* (* (sqrt (/ 2.0 (* Pi z))) (exp (BesselasymptA mu zsqr zfth)))
       (cos (- (- (BesselasymptB mu z zsqr zfth)
                  (/ (* Pi v) 2.0))
              (/ Pi 4.0))))))

```

defun BesselasymptA

— defun BesselasymptA 0 —

```

(defun BesselasymptA (mu zsqr zfth)
  (* (/ (- mu 1) (* 16.0 zsqr))
     (+ (+ 1 (/ (- mu 13.0) (* 8.0 zsqr)))
        (/ (+ (- (* mu mu) (* 53.0 mu)) 412.0)
           (* 48.0 zfth)))))

```

defun BesselasymptB

— defun BesselasymptB 0 —

```
(defun BesselasympdB (mu z zsqr zfth)
  (let ((musqr (* mu mu)))
    (+ z
      (* (/ (- mu 1.0) (* 8.0 z))
        (+ (+ (+ 1.0 (/ (- mu 25.0) (* 48.0 zsqr)))
              (/ (+ (- musqr (* 114.0 mu)) 1073.0) (* 640.0 zfth)))
          (/ (- (+ (- (* (* 5.0 mu) musqr) (* 1535.0 musqr))
                (* 54703.0 mu))
              375733.0)
          (* (* 128.0 zsqr) zfth)))))))
```

defun BesselJRecur

— defun BesselJRecur —

```
(defun BesselJRecur (v z)
  (let (w m so)
    ; boost order. Numerical.Recipes. suggest so:=v+sqrt(n.s.f.^2*v)
    (setq so (* 15.0 z))
    ; reduce order until non-zero
    (loop while (not (zerop (abs (BesselJAsymptOrder so z)))) do
      (setq so (/ so 2.0)))
    (when (< (abs so) (abs z))
      (setq so (+ v (* 18.0 (sqrt v)))))
    (setq m (+ (floor (abs (- so v))) 1))
    (setq w (make-array m))
    (setf (aref w (- m 1)) (BesselJAsymptOrder (- (+ v m) 1) z))
    (setf (aref w (- m 2)) (BesselJAsymptOrder (- (+ v m) 2) z))
    (loop for i from (- m 3) downto 0 by 1 do
      (setf (aref w i)
        (- (/ (* (* 2.0 (+ (+ v i) 1.0)) (aref w (+ i 1))) z)
          (aref w (+ i 2)))))
    (aref w 0)))
```

defun BesselJAsymptOrder

Asymptotic formula for BesselJ when order is large comes from Debye (1909). See Olver, Asymptotics and Special Functions, p. 134.

Expansion good for $0 \leq \text{phase}(v) < \pi$

A&S recommend “uniform expansion” with complicated coefficients and Airy function.

Debye's Formula is in 9.3.7,9.3.9,9.3.10 of A&S

AXIOM recurrence for u_k

```
f(0)==1::EXPR INT
f(n)== (t^2)*(1-t^2)*D(f(n-1),t)/2 + (1/8)*integrate( (1-5*t^2)*f(n-1),t)
```

— defun BesselJAsymptOrder —

```
(defun BesselJAsymptOrder (v z)
  (let (ca8 ca4 ca2 ca tanhalpha alpha sechalpha)
    (setq sechalpha (/ z v))
    (setq alpha (acosh (/ 1.0 sechalpha)))
    (setq tanhalpha (sqrt (- 1.0 (* sechalpha sechalpha))))
    (setq ca (/ 1.0 tanhalpha))
    (setq ca2 (* ca ca))
    (setq ca4 (* ca2 ca2))
    (setq ca8 (* ca4 ca4))
    (* (/ (exp (- (* v (- alpha tanhalpha))))
          (sqrt (* (* (2.0 Pi) v) tanhalpha)))
       (+ (+ (+ (+ (+ 1.0 (/ (* (horner '(-5.0 3.0) ca2) ca)
                                (* v 24.0)))
              (/ (* (horner '(385.0 -462.0 81.0) ca2) ca2)
                  (* (* 1152.0 v) v)))
            (/ (* (* (horner
                      '(-425425.0 765765.0 -369603.0 30375.0)
                      ca2)
                  ca2)
              ca)
            (* (* (* 414720.0 v) v) v)))
       (/ (* (horner
              '(1.85910725E8 -4.4618574E8 3.4992243E8
                -9.4121676E7 4465125.0)
              ca2)
          ca4)
          (* (* (* (* 3.981312E7 v) v) v) v)))
       (/ (* (* (horner
              '(-1.88699385875E11 5.66098157625E11
                -6.1413587235E11 2.84499769554E11
                -4.9286948607E10 1.519035525E9)
              ca2)
          ca4)
          ca)
          (* (* (* (* (* 6.68860416E9 v) v) v) v) v)))
       (/ (* (* (horner
              '(1.023694168371875E15 -3.6852990061387505E15
                5.104696716244125E15 -3.36903206826186E15
                1.050760774457901E15 -1.2757729835475E14
                2.757049477875E12)
```

Where:

c parameter to 0F1, possibly complex

z argument to 0F1

w scale factor so that $0 < \frac{z}{w} < 1$

$n, n + 2$ coefficients will be produced stored in an array indexed from 0 to $n + 1$.

Program transcribed from Fortran., p. 80 [Luke77]

— defun chebf01 —

```
(defun chebf01 (c z)
  (let (cc temp b p sum rho divfac c1 x1 arr
        ncount z1 a1 a2 a3 n2 n1 start four w n)
    (setq n 75) ; ad hoc decision
    (setq w (* 2.0 z))
    ; arr will be used to store the Cheb. series coefficients
    (setq four 4.0)
    (setq start (expt 10.0 -200))
    (setq n1 (+ n 1))
    (setq n2 (+ n 2))
    (setq a3 0.0)
    (setq a2 0.0)
    (setq a1 start) ; arbitrary starting value
    (setq z1 (/ four w))
    (setq ncount n1)
    (setq arr (make-array n2))
    (setf (aref arr ncount) start) ; start off
    (setq x1 n2)
    (setq c1 (- 1.0 c))
    (loop for ncount from n downto 0 by 1 do
      (setq divfac (/ 1.0 x1))
      (setq x1 (- x1 1.0))
      (setf (aref arr ncount)
        (* x1 (- (+ (* (+ divfac (* z1 (- x1 c1))) a1)
                    (* (+ (/ 1.0 x1) (* z1 (+ (+ x1 c1) 1.0))) a2))
            (* divfac a3))))
      (setq a3 a2)
      (setq a2 a1)
```

```
(setq a1 (aref arr ncount)))
(setf (aref arr 0) (/ (aref arr 0) 2.0))
; compute scale factor
(setq rho (aref arr 0))
(setq sum rho)
(setq p 1.0)
(loop for i from 1 to n1 do
  (setq rho (- rho (* p (aref arr i))))
  (setq sum (+ sum (aref arr i)))
  (setq p (- p)))
(loop for m from 0 to n1 do
  (setf (aref arr m) (/ (aref arr m) rho)))
(setq sum (/ sum rho))
; Now evaluate array at argument
(setq b 0.0)
(setq temp 0.0)
(loop for i from (+ n 1) downto 0 by 1 do
  (setq cc b)
  (setq b temp)
  (setq temp (+ (- cc) (aref arr i))))
temp))
```

Chapter 40

Common Lisp Algebra Support

These functions are called directly from the algebra source code. They fall into two basic categories, one are the functions that are raw Common Lisp calls and the other are Axiom specific functions or macros.

Raw function calls are used where there is an alignment of the Axiom type and the underlying representation in Common Lisp. These form the support pillars upon which Axiom rests. For instance, the 'EQ' function is called to support the Axiom equivalent 'eq?' function.

Macros are used to add type information in order to make low level operations faster. An example is the use of macros in DoubleFloat to add Common Lisp type information. Since DoubleFloat is machine arithmetic we give the compiler explicit type information so it can generate fast code.

Functions are used to do manipulations which are Common Lisp operations but the Axiom semantics are not the same. Because Axiom was originally written in Maclisp, then VMLisp, and then Common Lisp some of these old semantics survive.

40.1 AlgebraicFunction

defun retract

```
[objMode p516]  
[objVal p516]  
[isWrapped p1661]  
[qcar p??]  
[retract1 p??]  
[mkObj p514]  
[$EmptyMode p708]
```

— defun retract —

```

(defun |retract| (object)
  (labels (
    (retract1 (object)
      (let (type val underDomain objectp)
        (declare (special |$SingleInteger| |$Integer| |$NonNegativeInteger|
                          |$PositiveInteger|))
        (setq type (|objMode| object))
        (cond
          ((stringp type) '|failed|)
          (t
           (setq val (|objVal| object))
           (cond
            ((equal type |$PositiveInteger|) (mkObj val |$NonNegativeInteger|))
            ((equal type |$NonNegativeInteger|) (mkObj val |$Integer|))
            ((and (equal type |$Integer|) (typep (|unwrap| val) 'fixnum))
             (mkObj val |$SingleInteger|))
            (t
             (cond
              ((or (eql 1 (|#| type))
                   (and (consp type) (eq (qcar type) '|Union|))
                   (and (consp type) (eq (qcar type) '|FunctionCalled|)
                        (and (consp (qcdr type)) (eq (qcddr type) nil)))
                   (and (consp type) (eq (qcar type) '|OrderedVariableList|)
                        (and (consp (qcdr type)) (eq (qcddr type) nil)))
                   (and (consp type) (eq (qcar type) '|Variable|)
                        (and (consp (qcdr type)) (eq (qcddr type) nil))))
              (if (setq objectp (|retract2Specialization| object))
                  objectp
                  '|failed|))
             (null (setq underDomain (|underDomainOf| type))
                  '|failed|)
             ; try to retract the "coefficients", e.g. P RN -> P I or M RN -> M I
             (t
              (setq objectp (|retractUnderDomain| object type underDomain))
              (cond
               ((not (eq objectp '|failed|)) objectp)
               ; see if we can use the retract functions
               ((setq objectp (|coerceRetract| object underDomain)) objectp)
               ; see if we have a special case here
               ((setq objectp (|retract2Specialization| object)) objectp)
              (t '|failed|))))))))))
  (let (type val ans)
    (declare (special |$EmptyMode|))
    (setq type (|objMode| object))
    (cond
      ((stringp type) '|failed|)
      ((equal type |$EmptyMode|) '|failed|)
      (t
       (setq val (|objVal| object))
       (cond

```

```
((and (null (|isWrapped| val))
      (null (and (consp val) (eq (qcar val) 'map))))
  '|failed|)
(t
 (cond
  ((eq (setq ans (retract1 (mkObj val type))) '|failed|)
   ans)
  (t
   (mkObj (|objVal| ans) (|objMode| ans)))))))))
```

40.2 Any

defun spad2BootCoerce

— defun spad2BootCoerce —

```
(defun |spad2BootCoerce| (x source target)
  (let (xp)
    (cond
     ((null (|isValidType| source))
      (|throwKeyedMsg| "%1p is not a valid type." (list source)))
     ((null (|isValidType| target))
      (|throwKeyedMsg| "%1p is not a valid type." (list target)))
     ((setq xp (|coerceInteractive| (mkObjWrap x source) target))
      (|objValUnwrap| xp))
     (t
      (|throwKeyedMsgCannotCoerceWithValue| (|wrap| x) source target)))))
```

40.3 ApplicationProgramInterface

defun Report what domains get instantiated

[[\\$reportinstantiations](#) p1271]

— defun reportinstantiations —

```
(defun reportinstantiations (b)
  (setq |$reportInstantiations| b))
```

40.4 Boolean

defun The Boolean = function support

— **defun** BooleanEquality 0 —

```
(defun |BooleanEquality| (x y) (if x y (null y)))
```

40.5 Char

defun **upcase**

[identp p¹²³⁷]
[upcase p¹²⁷²]

— **defun** **upcase** —

```
(defun upcase (l)
  (cond ((stringp l) (string-upcase l))
        ((identp l) (intern (string-upcase (symbol-name l))))
        ((characterp l) (char-upcase l))
        ((atom l) l)
        (t (mapcar #'upcase l))))
```

defun **downcase**

[identp p¹²³⁷]
[downcase p¹²⁷²]

— **defun** **downcase** —

```
(defun downcase (l)
  (cond ((stringp l) (string-downcase l))
        ((identp l) (intern (string-downcase (symbol-name l))))
        ((characterp l) (char-downcase l))
        (t (mapcar #'downcase l))))
```

```
((atom 1) 1)
(t (mapcar #'downcase 1))))
```

40.6 ComplexDoubleFloatMatrix

defmacro make-cdouble-matrix

ComplexDoubleFloatMatrix function support

— **defmacro make-cdouble-matrix** 0 —

```
(defmacro make-cdouble-matrix (n m)
  '(make-array (list ,n (* 2 ,m)) :element-type 'double-float))
```

defmacro cdaref2

ComplexDoubleFloatMatrix function support

— **defmacro cdaref2** 0 —

```
(defmacro cdaref2 (ov oi oj)
  (let ((v (gensym))
        (i (gensym))
        (j (gensym)))
    '(let ((,v ,ov)
          (,i ,oi)
          (,j ,oj))
      (cons
        (aref (the (simple-array double-float (* *)) ,v) ,i (* 2 ,j))
        (aref (the (simple-array double-float (* *)) ,v)
              ,i (+ (* 2 ,j) 1))))))
```

defmacro cdsetaref2

ComplexDoubleFloatMatrix function support

— **defmacro cdsetaref2** 0 —


```

(defmacro cdsetaref2 (ov oi oj os)
  (let ((v (gensym))
        (i (gensym))
        (j (gensym))
        (s (gensym)))
    `(let ((,v ,ov)
          (,i ,oi)
          (,j ,oj)
          (,s ,os))
      (setf (aref (the (simple-array double-float (* *)) ,v) ,i (* 2 ,j))
            (car ,s))
      (setf (aref (the (simple-array double-float (* *)) ,v)
                  ,i (+ (* 2 ,j) 1))
            (cdr ,s))
      ,s)))

```

defmacro cdanrows

ComplexDoubleFloatMatrix function support

— **defmacro cdanrows 0** —

```

(defmacro cdanrows (v)
  `(array-dimension (the (simple-array double-float (* *)) ,v) 0))

```

defmacro cdancols

ComplexDoubleFloatMatrix function support

— **defmacro cdancols 0** —

```

(defmacro cdancols (v)
  `(truncate
    (array-dimension (the (simple-array double-float (* *)) ,v) 1) 2))

```

40.7 ComplexDoubleFloat Vector

Complex Double Float Vectors are simple arrays of lisp double-floats made available at the Spad language level. Note that these vectors are 0 based whereas other Spad language

vectors are 1-based. Complex array is implemented as an array of doubles. Each complex number occupies two positions in the real array.

defmacro make-cdouble-vector

ComplexDoubleFloatVector Qnew function support
— **defmacro make-cdouble-vector 0** —

```
(defmacro make-cdouble-vector (n)
  '(make-array (list (* 2 ,n)) :element-type 'double-float))
```

defmacro cdelt

ComplexDoubleFloatVector Qelt1 function support
— **defmacro cdelt 0** —

```
(defmacro CDELTA(ov oi)
  (let ((v (gensym))
        (i (gensym)))
    '(let ((,v ,ov)
          (,i ,oi))
      (cons
        (aref (the (simple-array double-float (*)) ,v) (* 2 ,i))
        (aref (the (simple-array double-float (*)) ,v) (+ (* 2 ,i) 1))))))
```

defmacro cdsetelt

ComplexDoubleFloatVector Qsetelt1 function support
— **defmacro cdsetelt 0** —

```
(defmacro cdsetelt(ov oi os)
  (let ((v (gensym))
        (i (gensym))
        (s (gensym)))
    '(let ((,v ,ov)
          (,i ,oi)
          (,s ,os))
      (setf (aref (the (simple-array double-float (*)) ,v) (* 2 ,i))
            (car ,s))
      (setf (aref (the (simple-array double-float (*)) ,v) (+ (* 2 ,i) 1))
```

```
(cdr ,s))
,s)))
```

defmacro cdlen

ComplexDoubleFloatVector Qsize function support

— **defmacro cdlen 0** —

```
(defmacro cdlen(v)
  '(truncate (length (the (simple-array double-float (*)) ,v)) 2))
```

40.8 Database

defun Database elt function support

[basicMatch? p??]

— **defun stringMatches?** —

```
(defun |stringMatches?| (pattern subject)
  (when (integerp (|basicMatch?| pattern subject)) t))
```

40.9 DirectProduct

defun vec2list

— **defun vec2list 0** —

```
(defun vec2list (vec)
  (coerce vec 'list))
```

40.10 DoubleFloat

These macros wrap their arguments with strong type information in order to optimize doublefloat computations. They are used directly in the DoubleFloat domain (see Volume 10.3).

defmacro DFLessThan

Compute a strongly typed doublefloat comparison See Steele Common Lisp 1990 p293

— **defmacro DFLessThan 0** —

```
(defmacro DFLessThan (x y)
  '(< (the double-float ,x) (the double-float ,y)))
```

40.11 DoubleFloatSpecialFunctions

defun Real Gamma $\Gamma(x)$

[rgammaImpl p1245]

— **defun rgamma** —

```
(defun rgamma (x)
  (rgammaImpl x))
```

defun Complex Gamma $\Gamma(z)$

The cgamma(z) function is the Γ function for complex arguments implemented by Bruce Char, April-May, 1990.

Our text for complex gamma is H. Kuki's paper "Complex Gamma Function with Error Control" [kuki72a]

It uses the reflection formula and the basic $z + 1$ recurrence to transform the argument into something that Stirling's asymptotic formula can handle.

However along the way it does a few tricky things to reduce problems due to round-off/cancellation error for particular values.

Small float implementation of Gamma function. Valid for real arguments. Maximum error (relative) seems to be 2-4 ulps for real x $2 < x < 9$, and up to ten ulps for larger x up to overflow. See Hart and Cheney. (Bruce Char, April, 1990).

[cgammaImpl p1248]
 [c-to-s p1243]
 [s-to-c p1243]

— defun cgamma —

```
(defun cgamma (z)
  (c-to-s (cgammaImpl (s-to-c z)) ))
```

—————

defun The complex logGamma function

[lncgamma p1256]
 [c-to-s p1243]
 [s-to-c p1243]

— defun clngamma —

```
(defun clngamma (z)
  (c-to-s (lncgamma (s-to-c z)) ))
```

—————

defun The real logGamma function

[lnrgamma p1245]

— defun rlngamma —

```
(defun rlngamma (x)
  (lnrgamma x))
```

—————

defun The real Psi function

[rPsiImpl p1257]

— defun rpsi —

```
(defun rpsi (n x)
  (rPsiImpl n x))
```

defun The complex Psi function

```
[c-to-s p1243]
[cPsi p??]
[s-to-c p1243]
```

— defun cpsi —

```
(defun cpsi (n z)
  (c-to-s (cPsiImpl n (s-to-c z)) ))
```

defun The real BesselJ function

```
[c-to-r p1244]
[BesselJ p1262]
```

— defun rbesselj —

```
(defun rbesselj (n x)
  (c-to-r (BesselJ n x)) ))
```

defun The complex BesselJ function

```
[c-to-s p1243]
[BesselJ p1262]
[s-to-c p1243]
```

— defun cbesselj —

```
(defun cbesselj (v z)
  (c-to-s (BesselJ (s-to-c v) (s-to-c z)) ))
```

defun The real BesselI function

[c-to-r p1244]
 [BesselI p1252]

— defun rbesseli —

```
(defun rbesseli (n x)
  (c-to-r (BesselI n x)) ))
```

—————

defun The complex BesselI function

[c-to-s p1243]
 [BesselI p1252]
 [s-to-c p1243]

— defun cbesseli —

```
(defun cbesseli (v z)
  (c-to-s (BesselI (s-to-c v) (s-to-c z)) ))
```

—————

defun The complex hypergeometric function

[c-to-s p1243]
 [chebf01 p1267]
 [s-to-c p1243]

— defun chyper0f1 —

```
(defun chyper0f1 (a z)
  (c-to-s (chebf01 (s-to-c a) (s-to-c z)) ))
```

—————

defmacro DFUnaryMinus

Compute a strongly typed unary doublefloat minus See Steele Common Lisp 1990 p295

— defmacro DFUnaryMinus 0 —

```
(defmacro DFUnaryMinus (x)
  '(the double-float (- (the double-float ,x))))
```

defmacro DFMinusp

Compute a strongly typed unary doublefloat test for negative See Steele Common Lisp 1990 p292

— **defmacro DFMinusp 0** —

```
(defmacro DFMinusp (x)
  '(minusp (the double-float ,x)))
```

defmacro DFZerop

Compute a strongly typed unary doublefloat test for zero See Steele Common Lisp 1990 p292

— **defmacro DFZerop 0** —

```
(defmacro DFZerop (x)
  '(zerop (the double-float ,x)))
```

defmacro DFAdd

Compute a strongly typed doublefloat addition See Steele Common Lisp 1990 p295

— **defmacro DFAdd 0** —

```
(defmacro DFAdd (x y)
  '(the double-float (+ (the double-float ,x) (the double-float ,y))))
```

defmacro DFSubtract

Compute a strongly typed doublefloat subtraction See Steele Common Lisp 1990 p295

— **defmacro DFSubtract 0** —


```
(defmacro DFSubtract (x y)
  '(the double-float (- (the double-float ,x) (the double-float ,y))))
```

defmacro DFMultiply

Compute a strongly typed doublefloat multiplication See Steele Common Lisp 1990 p296

— **defmacro DFMultiply 0** —

```
(defmacro DFMultiply (x y)
  '(the double-float (* (the double-float ,x) (the double-float ,y))))
```

defmacro DFIntegerMultiply

Compute a strongly typed doublefloat multiplication by an integer. See Steele Common Lisp 1990 p296

— **defmacro DFIntegerMultiply 0** —

```
(defmacro DFIntegerMultiply (i y)
  '(the double-float (* (the integer ,i) (the double-float ,y))))
```

defmacro DFMax

Choose the maximum of two doublefloats. See Steele Common Lisp 1990 p294

— **defmacro DFMax 0** —

```
(defmacro DFMax (x y)
  '(the double-float (max (the double-float ,x) (the double-float ,y))))
```

defmacro DFMin

Choose the minimum of two doublefloats. See Steele Common Lisp 1990 p294

— **defmacro DFMin 0** —

```
(defmacro DfMin (x y)
  '(the double-float (min (the double-float ,x) (the double-float ,y))))
```

defmacro DFEql

Compare two doublefloats for equality, where equality is eq, or numbers of the same type with the same value. See Steele Common Lisp 1990 p105

— **defmacro DFEql 0** —

```
(defmacro DFEql (x y)
  '(eq (the double-float ,x) (the double-float ,y)))
```

defmacro DFDivide

Divide a doublefloat by a doublefloat See Steele Common Lisp 1990 p296

— **defmacro DFDivide 0** —

```
(defmacro DFDivide (x y)
  '(the double-float (/ (the double-float ,x) (the double-float ,y))))
```

defmacro DFIntegerDivide

Divide a doublefloat by an integer See Steele Common Lisp 1990 p296

— **defmacro DFIntegerDivide 0** —

```
(defmacro DFIntegerDivide (x i)
  '(the double-float (/ (the double-float ,x) (the integer ,i))))
```

defmacro DFSqrt

Compute the doublefloat square root of x . The result will be complex if the argument is negative. See Steele Common Lisp 1990 p302

— **defmacro DFSqrt 0** —

```
(defmacro DFSqrt (x)
  '(sqrt (the double-float ,x)))
```

defmacro DFLogE

Compute the doublefloat log of x with the base e . The result will be complex if the argument is negative. See Steele Common Lisp 1990 p301

— **defmacro DFLogE 0** —

```
(defmacro DFLogE (x)
  '(log (the double-float ,x)))
```

defmacro DFLog

Compute the doublefloat log of x with a given base b . The result will be complex if x is negative. See Steele Common Lisp 1990 p301

— **defmacro DFLog 0** —

```
(defmacro DFLog (x b)
  '(log (the double-float ,x) (the fixnum ,b)))
```

defmacro DFIntegerExpt

Compute the doublefloat expt of x with a given integer power i See Steele Common Lisp 1990 p300

— **defmacro DFIntegerExpt 0** —

```
(defmacro DFIntegerExpt (x i)
  '(the double-float (expt (the double-float ,x) (the integer ,i))))
```

defmacro DFExpt

Compute the doublefloat expt of x with a given power p . The result could be complex if the base is negative and the power is not an integer. See Steele Common Lisp 1990 p300

— **defmacro DFExpt 0** —

```
(defmacro DFExpt (x p)
  '(expt (the double-float ,x) (the double-float ,p)))
```

defmacro DFExp

Compute the doublefloat exp with power e See Steele Common Lisp 1990 p300

— **defmacro DFExp 0** —

```
(defmacro DFExp (x)
  '(the double-float (exp (the double-float ,x))))
```

defmacro DFSin

Compute a strongly typed doublefloat sin See Steele Common Lisp 1990 p304

— **defmacro DFSin 0** —

```
(defmacro DFSin (x)
  '(the double-float (sin (the double-float ,x))))
```

defmacro DFCos

Compute a strongly typed doublefloat cos See Steele Common Lisp 1990 p304

— **defmacro DFCos 0** —

```
(defmacro DFCos (x)
  '(the double-float (cos (the double-float ,x))))
```

defmacro DFTan

Compute a strongly typed doublefloat tan See Steele Common Lisp 1990 p304

— **defmacro DFTan 0** —

```
(defmacro DFTan (x)
  '(the double-float (tan (the double-float ,x))))
```

defmacro DFAsin

Compute a strongly typed doublefloat asin. The result is complex if the absolute value of the argument is greater than 1. See Steele Common Lisp 1990 p305

— **defmacro DFAsin 0** —

```
(defmacro DFAsin (x)
  '(asin (the double-float ,x)))
```

defmacro DFAcos

Compute a strongly typed doublefloat acos. The result is complex if the absolute value of the argument is greater than 1. See Steele Common Lisp 1990 p305

— **defmacro DFAcos 0** —

```
(defmacro DFAcos (x)
  '(acos (the double-float ,x)))
```

defmacro DFAtan

Compute a strongly typed doublefloat atan See Steele Common Lisp 1990 p305

— **defmacro DFAtan 0** —

```
(defmacro DFAtan (x)
  '(the double-float (atan (the double-float ,x))))
```

defmacro DFAtan2

Compute a strongly typed doublefloat atan with 2 arguments

$y = 0$	$x > 0$	Positive x-axis	0
$y > 0$	$x > 0$	Quadrant I	$0 < \text{result} < \pi/2$
$y > 0$	$x = 0$	Positive y-axis	$\pi/2$
$y > 0$	$x < 0$	Quadrant II	$\pi/2 < \text{result} < \pi$
$y = 0$	$x < 0$	Negative x-axis	π
$y < 0$	$x < 0$	Quadrant III	$-\pi < \text{result} < -\pi/2$
$y < 0$	$x = 0$	Negative y-axis	$-\pi/2$
$y < 0$	$x > 0$	Quadrant IV	$-\pi/2 < \text{result} < 0$
$y = 0$	$x = 0$	Origin	error

See Steele Common Lisp 1990 p306

— **defmacro DFAtan2 0** —

```
(defmacro DFAtan2 (y x)
  '(the double-float (atan (the double-float ,x) (the double-float ,y))))
```

defmacro DFSinh

Compute a strongly typed doublefloat sinh

$$(e^z - e^{-z})/2$$

See Steele Common Lisp 1990 p308

— **defmacro DFSinh 0** —

```
(defmacro DFSinh (x)
  '(the double-float (sinh (the double-float ,x))))
```

defmacro DFCosh

Compute a strongly typed doublefloat cosh

$$(e^z + e^{-z})/2$$

See Steele Common Lisp 1990 p308

— **defmacro DFCosh 0** —

```
(defmacro DFCosh (x)
  '(the double-float (cosh (the double-float ,x))))
```

defmacro DFTanh

Compute a strongly typed doublefloat tanh

$$(e^z - e^{-z}) / (e^z + e^{-z})$$

See Steele Common Lisp 1990 p308

— **defmacro DFTanh 0** —

```
(defmacro DFTanh (x)
  '(the double-float (tanh (the double-float ,x))))
```

defmacro DFAsinh

Compute the inverse hyperbolic sin.

$$\log \left(z + \sqrt{1 + z^2} \right)$$

See Steele Common Lisp 1990 p308

— **defmacro DFAsinh 0** —

```
(defmacro DFAsinh (x)
  '(the double-float (asinh (the double-float ,x))))
```

defmacro DFAcosh

Compute the inverse hyperbolic cos. Note that the acosh function will return a complex result if the argument is less than 1.

$$\log \left(z + (z + 1) \sqrt{(z - 1) / (z + 1)} \right)$$

See Steele Common Lisp 1990 p308

— **defmacro DFAcosh 0** —

```
(defmacro DFAcosh (x)
  '(acosh (the double-float ,x)))
```

defmacro DFAtanh

Compute the inverse hyperbolic tan. Note that the acosh function will return a complex result if the argument is greater than 1.

$$\log \left((1+z)\sqrt{1/(1-z^2)} \right)$$

See Steele Common Lisp 1990 p308

— **defmacro DFAtanh 0** —

```
(defmacro DFAtanh (x)
  '(atanh (the double-float ,x)))
```

defun Machine specific float numerator

This is used in the DoubleFloat integerDecode function

— **defun integer-decode-float-numerator 0** —

```
(defun integer-decode-float-numerator (x)
  (integer-decode-float x))
```

defun Machine specific float denominator

This is used in the DoubleFloat integerDecode function

— **defun integer-decode-float-denominator 0** —

```
(defun integer-decode-float-denominator (x)
  (multiple-value-bind (mantissa exponent sign) (integer-decode-float x)
    (declare (ignore mantissa sign)) (expt 2 (abs exponent)))))
```

defun Machine specific float sign

This is used in the DoubleFloat integerDecode function

— **defun integer-decode-float-sign 0** —


```
(defun integer-decode-float-sign (x)
  (multiple-value-bind (mantissa exponent sign) (integer-decode-float x)
    (declare (ignore mantissa exponent)) sign))
```

defun Machine specific float bit length

This is used in the DoubleFloat integerDecode function

— **defun integer-decode-float-exponent 0** —

```
(defun integer-decode-float-exponent (x)
  (multiple-value-bind (mantissa exponent sign) (integer-decode-float x)
    (declare (ignore mantissa sign)) exponent))
```

defun Decode floating-point values

This function is used by DoubleFloat to implement the “mantissa” and “exponent” functions.

— **defun manexp 0** —

```
(defun manexp (u)
  (multiple-value-bind (f e s)
    (decode-float u)
    (cons (* s f) e)))
```

defun The cotangent routine

The cotangent function is defined as

$$\cot(z) = \frac{1}{\tan(z)}$$

— **defun cot 0** —

```
(defun cot (a)
  (if (or (> a 1000.0) (< a -1000.0))
    (/ (cos a) (sin a))
    (/ 1.0 (tan a))))
```

defun The inverse cotangent function

The inverse cotangent (arc-cotangent) function is defined as

$$\operatorname{acot}(z) = \cot^{-1}(z) = \tan^{-1}\left(\frac{1}{z}\right)$$

See Steele Common Lisp 1990 pp305-307

— **defun acot 0** —

```
(defun acot (a)
  (if (> a 0.0)
    (if (> a 1.0)
      (atan (/ 1.0 a))
      (- (/ pi 2.0) (atan a)))
    (if (< a -1.0)
      (- pi (atan (/ -1.0 a)))
      (+ (/ pi 2.0) (atan (- a))))))
```

defun The secant function

$$\sec(x) = \frac{1}{\cos(x)}$$

— **defun sec 0** —

```
(defun sec (x) (/ 1 (cos x)))
```

defun The inverse secant function

$$\operatorname{asec}(x) = \operatorname{acos}\left(\frac{1}{x}\right)$$

— **defun asec 0** —

```
(defun asec (x) (acos (/ 1 x)))
```

defun The cosecant function

$$\csc(x) = \frac{1}{\sin(x)}$$

— defun csc 0 —

```
(defun csc (x) (/ 1 (sin x)))
```

defun The inverse cosecant function

$$\operatorname{acsc}(x) = \frac{1}{\operatorname{asin}(x)}$$

— defun acsc 0 —

```
(defun acsc (x) (asin (/ 1 x)))
```

defun The hyperbolic cosecant function

$$\operatorname{csch}(x) = \frac{1}{\sinh(x)}$$

— defun csch 0 —

```
(defun csch (x) (/ 1 (sinh x)))
```

defun The hyperbolic cotangent function

$$\operatorname{coth}(x) = \cosh(x)\operatorname{csch}(x)$$

— defun coth 0 —

```
(defun coth (x) (* (cosh x) (csch x)))
```

defun The hyperbolic secant function

$$\operatorname{sech}(x) = \frac{1}{\cosh(x)}$$

— defun sech 0 —

```
(defun sech (x) (/ 1 (cosh x)))
```

—————

defun The inverse hyperbolic cosecant function

$$\operatorname{acsch}(x) = \operatorname{asinh}\left(\frac{1}{x}\right)$$

— defun acsch 0 —

```
(defun acsch (x) (asinh (/ 1 x)))
```

—————

defun The inverse hyperbolic cotangent function

$$\operatorname{acoth}(x) = \operatorname{atanh}\left(\frac{1}{x}\right)$$

— defun acoth 0 —

```
(defun acoth (x) (atanh (/ 1 x)))
```

—————

defun The inverse hyperbolic secant function

$$\operatorname{asech}(x) = \operatorname{acosh}\left(\frac{1}{x}\right)$$

— defun asech 0 —

```
(defun asech (x) (acosh (/ 1 x)))
```

—————

40.12 DoubleFloatMatrix

defmacro make-double-matrix

DoubleFloatMatrix qnew function support

— **defmacro make-double-matrix 0** —

```
(defmacro make-double-matrix (n m)
  '(make-array (list ,n ,m) :element-type 'double-float))
```

defmacro make-double-matrix1

DoubleFloatMatrix new function support

— **defmacro make-double-matrix1 0** —

```
(defmacro make-double-matrix1 (n m s)
  '(make-array (list ,n ,m) :element-type 'double-float :initial-element ,s))
```

defmacro daref2

DoubleFloatMatrix qelt function support

— **defmacro daref2 0** —

```
(defmacro daref2 (v i j)
  '(aref (the (simple-array double-float (* *)) ,v) ,i ,j))
```

defmacro dsetaref2

DoubleFloatMatrix qsetelt! function support

— **defmacro dsetaref2 0** —

```
(defmacro dsetaref2 (v i j s)
  '(setf (aref (the (simple-array double-float (* *)) ,v) ,i ,j) ,s))
```

defmacro danrows

DoubleFloatMatrix nrow function support

— **defmacro danrows 0** —

```
(defmacro danrows (v)
  '(array-dimension (the (simple-array double-float (* *)) ,v) 0))
```

defmacro dancols

DoubleFloatMatrix ncol function support

— **defmacro dancols 0** —

```
(defmacro dancols (v)
  '(array-dimension (the (simple-array double-float (* *)) ,v) 1))
```

40.13 DoubleFloatVector

Double Float Vectors are simple arrays of lisp double-floats made available at the Spad language level. Note that these vectors are 0 based whereas other Spad language vectors are 1-based.

defmacro dlen

DoubleFloatVector Qsize function support

— **defmacro dlen 0** —

```
(defmacro dlen (v)
  '(length (the (simple-array double-float (*)) ,v)))
```

defmacro make-double-vector

DoubleFloatVector Qnew function support

— **defmacro make-double-vector 0** —

```
(defmacro make-double-vector (n)
  '(make-array (list ,n) :element-type 'double-float))
```

defmacro make-double-vector1

DoubleFloatVector Qnew1 function support

— **defmacro make-double-vector1 0** —

```
(defmacro make-double-vector1 (n s)
  '(make-array (list ,n) :element-type 'double-float :initial-element ,s))
```

defmacro delt

DoubleFloatVector Qelt1 function support

— **defmacro delt 0** —

```
(defmacro delt (v i)
  '(aref (the (simple-array double-float (*)) ,v) ,i))
```

defmacro dsetelt

DoubleFloatVector Qsetelt1 function support

— **defmacro dsetelt 0** —

```
(defmacro dsetelt (v i s)
  '(setf (aref (the (simple-array double-float (*)) ,v) ,i) ,s))
```

40.14 File

defvar *read-place-holder*

— **initvars** —

```
(defvar *read-place-holder* (make-symbol "%.EOF")
  "default value returned by read and read-line at end-of-file")
```

defun placep

[**read-place-holder** p1296]

— defun placep 0 —

```
(defun placep (item)
  (declare (special *read-place-holder*))
  (eq item *read-place-holder*))
```

defun vmread

— defun vmread 0 —

```
(defun vmread (&optional (st *standard-input*) (eofval *read-place-holder*))
  (read st nil eofval))
```

40.15 FileName

defun FileName filename function implementation

[StringToDir p1298]

— defun fnameMake —

```
(defun |fnameMake| (d n e)
  (if (string= e "") (setq e nil))
  (make-pathname :directory (|StringToDir| d) :name n :type e))
```

defun FileName filename support function

[lastc p??]

— defun StringToDir —

```

(defun |StringToDir| (s)
  (cond
    ((string= s "/" ) '(:root))
    ((string= s "") nil)
    (t
     (let ((lastc (aref s (- (length s) 1))))
       (if (char= lastc #\/)
           (pathname-directory (concat s "name.type"))
           (pathname-directory (concat s "/name.type")) ))) ))

```

defun FileName directory function implementation

[DirToString p1298]

— defun fnameDirectory —

```

(defun |fnameDirectory| (f)
  (|DirToString| (pathname-directory f)))

```

defun FileName directory function support

For example, “/” “/u/smwatt” “../src”

— defun DirToString 0 —

```

(defun |DirToString| (d)
  (cond
    ((equal d '(:root)) "/")
    ((null d) "")
    ('t (string-right-trim "/" (namestring (make-pathname :directory d))))))

```

defun FileName name function implementation

— defun fNameName 0 —

```
(defun |fNameName| (f)
  (let ((s (pathname-name f)))
    (if s s "")))
```

defun FileName extension function implementation

— defun fNameType 0 —

```
(defun |fNameType| (f)
  (let ((s (pathname-type f)))
    (if s s "")))
```

defun FileName exists? function implementation

— defun fNameExists? 0 —

```
(defun |fNameExists?| (f)
  (if (probe-file (namestring f)) 't nil))
```

defun FileName readable? function implementation

— defun fNameReadable? 0 —

```
(defun |fNameReadable?| (f)
  (let ((s (open f :direction :input :if-does-not-exist nil)))
    (cond (s (close s) t) (t nil))))
```

defun FileName writeable? function implementation

```
[myWritable? p??]
```

— defun fnameWritable? —

```
(defun |fnameWritable?| (f)
  (|myWritable?| (namestring f)) )
```

defun FileName writeable? function support

```
[error p??]
```

```
[fnameExists? p1299]
```

```
[fnameDirectory p1298]
```

```
[writeablep p??]
```

— defun myWritable? —

```
(defun |myWritable?| (s)
  (if (not (stringp s)) (|error| "'myWritable?' requires a string arg."))
  (if (string= s "") (setq s "."))
  (if (not (|fnameExists?| s)) (setq s (|fnameDirectory| s)))
  (if (string= s "") (setq s "."))
  (if (> (|writeablep| s) 0) 't nil) )
```

defun FileName new function implementation

```
[fnameMake p1297]
```

— defun fnameNew —

```
(defun |fnameNew| (d n e)
  (if (not (|myWritable?| d))
      nil
      (do ((fn))
          (nil)
          (setq fn (|fnameMake| d (string (gensym n)) e))
          (if (not (probe-file (namestring fn)))
              (return-from |fnameNew| fn) ) ) ) )
```

40.16 IndexedBits

defmacro truth-to-bit

IndexedBits new function support

— **defmacro truth-to-bit** —

```
(defmacro truth-to-bit (x) '(cond (,x 1) ('else 0)))
```

—————

defun IndexedBits new function support

— **defun bvec-make-full 0** —

```
(defun bvec-make-full (n x)
  (make-array (list n) :element-type 'bit :initial-element x))
```

—————

defmacro bit-to-truth

IndexedBits elt function support

— **defmacro bit-to-truth 0** —

```
(defmacro bit-to-truth (b) '(eq ,b 1))
```

—————

defmacro bvec-elt

IndexedBits elt function support

— **defmacro bvec-elt 0** —

```
(defmacro bvec-elt (bv i) '(sbit ,bv ,i))
```

—————

defmacro bvec-setelt

IndexedBits setelt function support

— **defmacro bvec-setelt** —

```
(defmacro bvec-setelt (bv i x) '(setf (sbit ,bv ,i) ,x))
```

defmacro bvec-size

IndexedBits length function support

— **defmacro bvec-size** —

```
(defmacro bvec-size (bv) '(size ,bv))
```

defun IndexedBits concat function support— **defun bvec-concat 0** —

```
(defun bvec-concat (bv1 bv2) (concatenate '(vector bit) bv1 bv2))
```

defun IndexedBits copy function support— **defun bvec-copy 0** —

```
(defun bvec-copy (bv) (copy-seq bv))
```

defun IndexedBits = function support— **defun bvec-equal 0** —

```
(defun bvec-equal (bv1 bv2) (equal bv1 bv2))
```

defun IndexedBits < function support

— defun bvec-greater 0 —

```
(defun bvec-greater (bv1 bv2)
  (let ((pos (mismatch bv1 bv2)))
    (cond ((or (null pos) (>= pos (length bv1))) nil)
          ((< pos (length bv2)) (> (bit bv1 pos) (bit bv2 pos)))
          ((find 1 bv1 :start pos) t)
          (t nil))))
```

defun IndexedBits And function support

— defun bvec-and 0 —

```
(defun bvec-and (bv1 bv2) (bit-and bv1 bv2))
```

defun IndexedBits Or function support

— defun bvec-or 0 —

```
(defun bvec-or (bv1 bv2) (bit-ior bv1 bv2))
```

defun IndexedBits xor function support

— defun bvec-xor 0 —

```
(defun bvec-xor (bv1 bv2) (bit-xor  bv1 bv2))
```

defun IndexedBits nand function support

— defun bvec-nand 0 —

```
(defun bvec-nand (bv1 bv2) (bit-nand bv1 bv2))
```

defun IndexedBits nor function support

— defun bvec-nor 0 —

```
(defun bvec-nor (bv1 bv2) (bit-nor  bv1 bv2))
```

defun IndexedBits not function support

— defun bvec-not 0 —

```
(defun bvec-not (bv) (bit-not  bv))
```

40.17 IndexCard

defun IndexCard origin function support

```
[dbPart p??]  
[charPosition p??]  
[substring p324]
```

— defun alqlGetOrigin —

```
(defun |alqlGetOrigin| (x)
  (let (field k)
    (setq field (|dbPart| x 5 1))
    (setq k (|charPosition| #\ ( field 2))
    (substring field 1 (1- k))))
```

defun IndexCard origin function support

```
[dbPart p??]
[charPosition p??]
[substring p324]
```

— defun alqlGetParams —

```
(defun |alqlGetParams| (x)
  (let (field k)
    (setq field (|dbPart| x 5 1))
    (setq k (|charPosition| #\ ( field 2))
    (substring field k nil)))
```

defun IndexCard elt function support

```
[dbPart p??]
[substring p324]
```

— defun alqlGetKindString —

```
(defun |alqlGetKindString| (x)
  (if (or (char= (elt x 0) #\a) (char= (elt x 0) #\o))
    (substring (|dbPart| x 5 1) 0 1)
    (substring x 0 1)))
```

40.18 IndexedString

defun qenum

This is also used in bookvol10.3 in the CHAR domain.

— defun qenum 0 —

```
(defun qenum (cvec ind)
  (char-code (char cvec ind)))
```

—————

— defmacro qcsize 0 —

```
(defmacro qcsize (x)
  '(the fixnum (length (the simple-string ,x))))
```

—————

— defun qeset 0 —

```
(defun qeset (cvec ind charnum)
  (setf (char cvec ind) (code-char charnum)))
```

—————

— defmacro qsgreaterp 0 —

```
(defmacro qsgreaterp (a b)
  '(> (the fixnum ,a) (the fixnum ,b)))
```

—————

40.19 InputForm

defun called by interpret function

— defun mkObjFn 0 —

```
(defun |mkObjFn| (val mode)
  (cons mode val))
```

defun called by interpret function

— defun objValFn 0 —

```
(defun |objValFn| (obj)
  (cdr obj))
```

defun called by interpret function

— defun objModeFn 0 —

```
(defun |objModeFn| (obj)
  (car obj))
```

defun unparseInputForm

This fixes bug 7217. The default title generation is bogus. This is called from the unparse function in InputForm, bookvol10.3 Given a form, *u*, we try to recover the input line that created it.

```
[$InteractiveMode p315]
[$formatSigAsTex p??]
```

— defun unparseInputForm —

```
(defun |unparseInputForm| (u)
  (let (|$formatSigAsTex| |$InteractiveMode|)
    (declare (special |$formatSigAsTex| |$InteractiveMode|))
    (setq |$formatSigAsTex| 1)
    (setq |$InteractiveMode| nil)
    (|form2StringLocal| u)))
```

40.20 Integer

defun Integer divide function support

Note that this is defined as a SPADReplace function in Integer so that algebra code that uses the Integer divide function actually inlines a call to this code. The Integer domain contains the line:

```
(PUT (QUOTE |INT;divide;2$R;44|) (QUOTE |SPADreplace|) (QUOTE DIVIDE2))
```

— defun divide2 0 —

```
(defun divide2 (x y)
  (multiple-value-call #'cons (truncate x y)))
```

defun Integer quo function support

Note that this is defined as a SPADReplace function in Integer so that algebra code that uses the Integer quo function actually inlines a call to this code. The Integer domain contains the line:

```
(PUT (QUOTE |INT;rem;3$;46|) (QUOTE |SPADreplace|) (QUOTE REMAINDER2))
```

Because these are identical except for name we make the symbol-functions equivalent. This was done in the original code for efficiency.

— defun remainder2 0 —

```
(setf (symbol-function 'remainder2) #'rem)
```

defun Integer quo function support

Note that this is defined as a SPADReplace function in Integer so that algebra code that uses the Integer quo function actually inlines a call to this code. The Integer domain contains the line:

```
(PUT (QUOTE |INT;quo;3$;45|) (QUOTE |SPADreplace|) (QUOTE QUOTIENT2))
```

— defun quotient2 0 —

```
(defun quotient2 (x y)
  (values (truncate x y)))
```

defun Integer random function support

This is used for calls to random with no arguments. If an argument is supplied to random then the common lisp random function is called directly. This could be lifted up into the spad code.

— defun random 0 —

```
(defun |random| () (random (expt 2 26)))
```

40.21 KeyedAccessFile

defun KeyedAccessFile defstream function support

This is a simpler interface to RDEFIOSTREAM [rdefiostream p??]

— defun rdefinstream —

```
(defun rdefinstream (&rest fn)
  ;; following line prevents rdefiostream from adding a default filetype
  (unless (rest fn) (setq fn (list (pathname (car fn)))))
  (rdefiostream (list (cons 'file fn) '(mode . input))))
```

defun KeyedAccessFile defstream function support

[rdefiostream p??]

— defun rdefoutstream —

```
(defun rdefoutstream (&rest fn)
  ;; following line prevents rdefiostream from adding a default filetype
  (unless (rest fn) (setq fn (list (pathname (car fn)))))
  (rdefiostream (list (cons 'FILE fn) '(mode . OUTPUT))))
```

40.22 NumberFormats

defun ncParseFromString

— **defun ncParseFromString** —

```
(defun |ncParseFromString| (s)
  (|zeroOneTran| (catch 'SPAD_READER (|parseFromString| s))))
```

40.23 OperationsQuery

defun OperationQuery getDatabase function support

This function, called as `getBrowseDatabase(arg)` returns a list of appropriate entries in the browser database. The legal values for `arg` are

- “o” (operations)
- “k” (constructors)
- “d” (domains)
- “c” (categories)
- “p” (packages)

```
[member p1238]
[|grepConstruct| p??]
[|$includeUnexposed?| p??]
```

— **defun getBrowseDatabase** —

```
(defun |getBrowseDatabase| (kind)
  (let (|$includeUnexposed?|)
    (declare (special |$includeUnexposed?|))
    (setq |$includeUnexposed?| t)
    (when (|member| kind '("o" "k" "c" "d" "p"))
      (|grepConstruct| "*" (intern kind)))))
```

40.24 ParametricLinearEquations

defun algCoerceInteractive

— **defun algCoerceInteractive** —

```
(defun |algCoerceInteractive| (p source target)
  (let (|$useConvertForCoercions| u)
    (declare (special |$useConvertForCoercions|))
    (setq |$useConvertForCoercions| t)
    (setq source (|devaluate| source))
    (setq target (|devaluate| target))
    (setq u (|coerceInteractive| (mkObjWrap p source) target))
    (if u
      (|objValUnwrap| u)
      (|error| (list "can't convert" p "of mode" source "to mode" target)))))
```

40.25 Plot3d

We catch numeric errors and throw a different failure than normal. The `trapNumericErrors` macro will return a pair of the the form `Union(type-of-form, "failed")`. This pair is tested for eq-ness so it has to be unique. It lives in the defvar `$numericFailure`. The old value of the `$BreakMode` variable is saved in a defvar named `$oldBreakMode`.

defvar \$numericFailure

This is a failed union branch which is the value returned for numeric failure.

— **initvars** —

```
(defvar |$numericFailure| (cons 1 "failed"))
```

defvar \$oldBreakMode

— **initvars** —

```
(defvar |$oldBreakMode| nil "the old value of the $BreakMode variable")
```

defmacro trapNumericErrors

The following macro evaluates form returning Union(type-of form, "failed"). It is used in the myTrap local function in Plot3d.

— defmacro trapNumericErrors —

```
(defmacro |trapNumericErrors| (form)
  '(let ((|$oldBreakMode| |$BreakMode|) (|$BreakMode| 'trapNumerics|) (val))
    (declare (special |$BreakMode| |$numericFailure| |$oldBreakMode|))
    (setq val (catch 'trapNumerics| ,form))
    (if (eq val |$numericFailure|) val (cons 0 val))))
```

40.26 SingleInteger

defun qsquotient

— defun qsquotient 0 —

```
(defun qsquotient (a b)
  (the fixnum (truncate (the fixnum a) (the fixnum b))))
```

defun qsremainder

— defun qsremainder 0 —

```
(defun qsremainder (a b)
  (the fixnum (rem (the fixnum a) (the fixnum b))))
```

defmacro qsdifference

— defmacro qsdifference 0 —

```
(defmacro qsdifference (x y)
  '(the fixnum (- (the fixnum ,x) (the fixnum ,y))))
```

defmacro qslessp

This is also used in IndexedString

— defmacro qslessp 0 —

```
(defmacro qslessp (a b)
  '(< (the fixnum ,a) (the fixnum ,b)))
```

defmacro qsadd1

— defmacro qsadd1 0 —

```
(defmacro qsadd1 (x)
  '(the fixnum (1+ (the fixnum ,x))))
```

defmacro qssub1

— defmacro qssub1 0 —

```
(defmacro qssub1 (x)
  '(the fixnum (1- (the fixnum ,x))))
```

defmacro qsminus

— defmacro qsminus 0 —


```
(defmacro qsminus (x)
  '(the fixnum (minus (the fixnum ,x))))
```

defmacro qsplus

— defmacro qsplus 0 —

```
(defmacro qsplus (x y)
  '(the fixnum (+ (the fixnum ,x) (the fixnum ,y))))
```

defmacro qstimes

— defmacro qstimes 0 —

```
(defmacro qstimes (x y)
  '(the fixnum (* (the fixnum ,x) (the fixnum ,y))))
```

defmacro qsabsval

— defmacro qsabsval 0 —

```
(defmacro qsabsval (x)
  '(the fixnum (abs (the fixnum ,x))))
```

defmacro qsoddp

— defmacro qsoddp 0 —

```
(defmacro qsoddp (x)
  '(oddp (the fixnum ,x)))
```

defmacro qzerop

— defmacro qzerop 0 —

```
(defmacro qzerop (x)
  '(zerop (the fixnum ,x)))
```

defmacro qsmax

— defmacro qsmax 0 —

```
(defmacro qsmax (x y)
  '(the fixnum (max (the fixnum ,x) (the fixnum ,y))))
```

defmacro qsmmin

— defmacro qsmmin 0 —

```
(defmacro qsmmin (x y)
  '(the fixnum (min (the fixnum ,x) (the fixnum ,y))))
```

40.27 Table

defun Table InnerTable support

We look inside the Key domain given to Table and find if there is an equality predicate associated with the domain. If found then Table will use a HashTable representation, otherwise it will use an AssociationList representation.

```
[knownEqualPred p??]
[compiledLookup p1225]
[Boolean p709]
[bpiname p??]
[knownEqualPred p??]
```

— defun hashable —

```
(defun |hashable| (dom)
  (labels (
    (|knownEqualPred| (dom)
      (let ((fun (|compiledLookup| '= '(|Boolean|) $ $) dom)))
      (if fun
        (get (bpiname (car fun)) '|SPADreplace|)
        nil))))
    (member (|knownEqualPred| dom) '(eq eql equal))))
```

—————

40.28 U8Vector

defmacro qvlenU8

— defmacro qvlenU8 —

```
(defmacro qvlenU8 (v)
  '(length (the (simple-array (unsigned-byte 8) (*)) ,v)))
```

—————

defmacro eltU8

— defmacro eltU8 —

```
(defmacro eltU8 (v i)
  '(aref (the (simple-array (unsigned-byte 8) (*)) ,v) ,i))
```

—————

defmacro seteltU8

— defmacro seteltU8 —

```
(defmacro seteltU8 (v i s)
  '(setf (aref (the (simple-array (unsigned-byte 8) (*)) ,v) ,i) ,s))
```

defun getRefvU8

— defun getRefvU8 —

```
(defun getRefvU8 (n x)
  (make-array n :initial-element x :element-type '(unsigned-byte 8)))
```

40.29 U16Vector**defmacro qvlenU16**

— defmacro qvlenU16 —

```
(defmacro qvlenU16 (v)
  '(length (the (simple-array (unsigned-byte 16) (*)) ,v)))
```

defmacro eltU16

— defmacro eltU16 —

```
(defmacro eltU16 (v i)
  '(aref (the (simple-array (unsigned-byte 16) (*)) ,v) ,i))
```

defmacro seteltU16

— defmacro seteltU16 —

```
(defmacro seteltU16 (v i s)
  '(setf (aref (the (simple-array (unsigned-byte 16) (*)) ,v) ,i), s))
```

—————

defun getRefvU16

— defun getRefvU16 —

```
(defun getRefvU16 (n x)
  (make-array n :initial-element x :element-type '(unsigned-byte 16)))
```

—————

40.30 U32Vector**defmacro qvlenU32**

— defmacro qvlenU32 —

```
(defmacro qvlenU32 (v)
  '(length (the (simple-array (unsigned-byte 32) (*)) ,v)))
```

—————

defmacro eltU32

— defmacro eltU32 —

```
(defmacro eltU32 (v i)
  '(aref (the (simple-array (unsigned-byte 32) (*)) ,v) ,i))
```

—————

defmacro seteltU32

— defmacro seteltU32 —

```
(defmacro seteltU32 (v i s)
  '(setf (aref (the (simple-array (unsigned-byte 32) (*)) ,v) ,i), s))
```

—————

defun getRefvU32

— defun getRefvU32 —

```
(defun getRefvU32 (n x)
  (make-array n :initial-element x :element-type '(unsigned-byte 32)))
```

—————

40.31 U8Matrix**defmacro aref2U8**

— defmacro aref2U8 —

```
(defmacro aref2U8 (v i j)
  '(aref (the (simple-array (unsigned-byte 8) (* *)) ,v) ,i ,j))
```

—————

defmacro setAref2U8

— defmacro setAref2U8 —

```
(defmacro setAref2U8 (v i j s)
  '(setf (aref (the (simple-array (unsigned-byte 8) (* *)) ,v) ,i ,j), s))
```

—————

defmacro anrowsU8

— defmacro anrowsU8 —

```
(defmacro anrowsU8 (v)
  '(array-dimension (the (simple-array (unsigned-byte 8) (* *)) ,v) 0))
```

—————

defmacro ancolsU8

— defmacro ancolsU8 —

```
(defmacro ancolsU8 (v)
  '(array-dimension (the (simple-array (unsigned-byte 8) (* *)) ,v) 1))
```

—————

defmacro makeMatrixU8

— defmacro makeMatrixU8 —

```
(defmacro makeMatrixU8 (n m)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 8)
               :initial-element 0))
```

—————

defmacro makeMatrix1U8

— defmacro makeMatrix1U8 —

```
(defmacro makeMatrix1U8 (n m s)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 8)
               :initial-element ,s))
```

—————

40.32 U16Matrix

defmacro aref2U16

— defmacro aref2U16 —

```
(defmacro aref2U16 (v i j)
  '(aref (the (simple-array (unsigned-byte 16) (* *)) ,v) ,i ,j))
```

—————

defmacro setAref2U16

— defmacro setAref2U16 —

```
(defmacro setAref2U16 (v i j s)
  '(setf (aref (the (simple-array (unsigned-byte 16) (* *)) ,v) ,i ,j), s))
```

—————

defmacro anrowsU16

— defmacro anrowsU16 —

```
(defmacro anrowsU16 (v)
  '(array-dimension (the (simple-array (unsigned-byte 16) (* *)) ,v) 0))
```

—————

defmacro ancolsU16

— defmacro ancolsU16 —

```
(defmacro ancolsU16 (v)
  '(array-dimension (the (simple-array (unsigned-byte 16) (* *)) ,v) 1))
```

—————

defmacro makeMatrixU16

— defmacro makeMatrixU16 —

```
(defmacro makeMatrixU16 (n m)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 16)
               :initial-element 0))
```

defmacro makeMatrix1U16

— defmacro makeMatrix1U16 —

```
(defmacro makeMatrix1U16 (n m s)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 16)
               :initial-element ,s))
```

40.33 U32Matrix**defmacro aref2U32**

— defmacro aref2U32 —

```
(defmacro aref2U32 (v i j)
  '(aref (the (simple-array (unsigned-byte 32) (* *)) ,v) ,i ,j))
```

defmacro setAref2U32

— defmacro setAref2U32 —

```
(defmacro setAref2U32 (v i j s)
  '(setf (aref (the (simple-array (unsigned-byte 32) (* *)) ,v) ,i ,j), s))
```

defmacro anrowsU32

— defmacro anrowsU32 —

```
(defmacro anrowsU32 (v)
  '(array-dimension (the (simple-array (unsigned-byte 32) (* *)) ,v) 0))
```

defmacro ancolsU32

— defmacro ancolsU32 —

```
(defmacro ancolsU32 (v)
  '(array-dimension (the (simple-array (unsigned-byte 32) (* *)) ,v) 1))
```

defmacro makeMatrixU32

— defmacro makeMatrixU32 —

```
(defmacro makeMatrixU32 (n m)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 32)
               :initial-element 0))
```

defmacro makeMatrix1U32

— defmacro makeMatrix1U32 —

```
(defmacro makeMatrix1U32 (n m s)
  '(make-array (list ,n ,m) :element-type '(unsigned-byte 32)
               :initial-element ,s))
```

40.34 U32VectorPolynomialOperations

defmacro qsMulAdd6432

— defmacro qsMulAdd6432 —

```
(defmacro qsMulAdd6432 (x y z)
  '(the (unsigned-byte 64)
    (+ (the (unsigned-byte 64)
      (* (the (unsigned-byte 32) ,x)
        (the (unsigned-byte 32) ,y)))
      (the (unsigned-byte 64) ,z))))
```

—————

defmacro qsMulMod32

— defmacro qsMulMod32 —

```
(defmacro qsMulMod32 (x y)
  '(the (unsigned-byte 64)
    (* (the (unsigned-byte 32) ,x)
      (the (unsigned-byte 32) ,y))))
```

—————

defmacro qsMod6432

— defmacro qsMod6432 —

```
(defmacro qsMod6432 (x p)
  '(the (unsigned-byte 32)
    (rem (the (unsigned-byte 64) ,x) (the (unsigned-byte 32) ,p))))
```

—————

defmacro qsMulAddMod6432

— defmacro qsMulAddMod6432 —

```
(defmacro qsMulAddMod6432 (x y z p)
  '(qsMod6432 (qsMulAdd6432 ,x ,y ,z) ,p))
```

defmacro qsMul6432

— defmacro qsMul6432 —

```
(defmacro qsMul6432 (x y)
  '(the (unsigned-byte 64)
    (* (the (unsigned-byte 32) ,x)
       (the (unsigned-byte 32) ,y))))
```

defmacro qsDot26432

— defmacro qsDot26432 —

```
(defmacro qsDot26432 (a1 b1 a2 b2)
  '(qsMulAdd6432 ,a1 ,b1 (qsMul6432 ,a2 ,b2)))
```

defmacro qsDot2Mod6432

— defmacro qsDot2Mod6432 —

```
(defmacro qsDot2Mod6432 (a1 b1 a2 b2 p)
  '(qsMod6432 (qsDot26432 ,a1 ,b1 ,a2 ,b2) ,p))
```

40.35 Void

defun voidValue

— **defun voidValue** —

```
(defun |voidValue| () "()")
```

Chapter 41

OpenMath

41.1 A Technical Overview

OpenMath[Dewa] is a standard for representing mathematical data in as unambiguous a way as possible. It can be used to exchange mathematical objects between software packages or via email, or as a persistent data format in a database. It is tightly focussed on representing semantic information and is not intended to be used directly for presentation, although tools exist to facilitate this.

The original motivation for OpenMath came from the Computer Algebra community. Computer Algebra packages were getting bigger and more unwieldy, and it seemed reasonable to adopt a generic "plug and play" architecture to allow specialised programs to be used from general purpose environments. There were plenty of mechanisms for connecting software components together, but no common format for representing the underlying data objects. It quickly became clear that any standard had to be vendor-neutral and that objects encoded in OpenMath should not be too verbose. This has led to the design outlined below.

In 1998, the Worldwide Web Consortium (W3C) produced its first recommendation for the Extensible Markup Language (XML), intended to be a universal format for representing structured information on the worldwide web. It was swiftly followed by the first MathML recommendation which is an XML application oriented mainly towards the presentation (i.e. the rendering) of mathematical expressions.

The formal definition of OpenMath is contained within The OpenMath Standard and its accompanying documents, and the reader is referred there for more details.

The OpenMath Architecture

The OpenMath representation of a mathematical structure is referred to as an OpenMath object. This is an abstract structure which is represented concretely via an OpenMath encoding. These encoded objects are what an OpenMath application would read and write,

and in practice the OpenMath objects themselves almost never exist, except on paper. The advantage of this is that OpenMath is not tied to any one underlying mechanism: in the past we have used functional, SGML and binary encodings. The current favourite is XML, as described below, and we will tend to use XML notation when describing OpenMath objects (even though strictly speaking the XML representation is an encoding). OpenMath Objects

Formally, an OpenMath object is a labelled tree whose leaves are the basic OpenMath objects integers, IEEE double precision floats, unicode strings, byte arrays, variables or symbols. Of these, symbols are the most interesting since they consist of a name and a reference to a definition in an external document called a content dictionary (or CD). Using XML notation where the element name OMS indicates an OpenMath symbol, the following:

```
<OMS name="sin" cd="transc1"/>
```

represents the usual sine function, as defined in the CD "transc1". A basic OpenMath object is an OpenMath object, although its XML representation will be:

```
<OMOBJ>
  <OMS name="sin" cd="transc1"/>
</OMOBJ>
```

OpenMath objects can be built up recursively in a number of ways. The simplest is function application, for example the expression $\sin(x)$ can be represented by the XML:

```
<OMOBJ>
  <OMA>
    <OMS name="sin" cd="transc1"/>
    <OMV name="x"/>
  </OMA>
</OMOBJ>
```

where OMV introduces a variable and OMA is the application element. Another straightforward method is attribution which as the name suggests can be used to add additional information (for example "the AXIOM command which generated me was ...") to an object without altering its fundamental meaning. More interesting are binding objects which are used to represent an expression containing bound variables, for example:

```
<OMOBJ>
  <OMA>
    <OMS cd="calculus1" name="int"/>
    <OMS cd="transc1" name="sin"/>
  </OMA>
</OMOBJ>
```

represents the integral of the sin function, but the encoding:

```
<OMOBJ>
  <OMA>
```

```

<OMS cd="calculus1" name="int"/>
<OMBIND>
  <OMS cd="fns1" name="lambda"/>
  <OMBVAR> <OMV name="x"/> </OMBVAR>
  <OMA>
    <OMS name="sin" cd="transc1"/>
    <OMV name="x"/>
  </OMA>
</OMBIND>
</OMA>
</OMOBJ>

```

represents $\sin(x)dx$. This may appear overly complicated but it is useful, for example when searching in a database for expressions which match $\sin(y)dy$. The definition of a symbol in the CD specifies whether or not it may be used to bind variables, which is why

```
<OMS cd="calculus1" name="int"/>
```

cannot be used as a binding symbol.

The final kind of OpenMath object is an error which is built up from a symbol describing the error and a sequence of OpenMath objects. For example:

```

<OMOBJ>
  <OME>
    <OMS name="unexpected_symbol" cd="error1">
    <OMS name="sine" cd="transc1">
  </OME>
</OMOBJ>

```

represents the error which might be generated when an application sees a symbol it doesn't recognise from a CD it thought it knew about.

OpenMath Encodings

We have already seen some examples of the XML encoding, but it is by no means the only encoding. In the past there was a functional encoding (which looked like Lisp) and an SGML encoding which evolved into the current XML. Both of these are now obsolete, but there is still a binary encoding described in the standard, which is much more compact than the XML one.

In fact the XML encoding is not 100% XML. When XML was in its infancy the developers of OpenMath realised that it might become significant and decided to add some XML-like features to the SGML encoding so that an OpenMath object could be encoded as valid XML. Thus it is currently the case that any well-formed OpenMath object encoded using the XML encoding as described in the standard is a valid XML document. However, if one uses standard XML tools to generate an OpenMath object in the XML encoding from the DTD given in chapter 4 of the standard, it is possible that the result will not be valid OpenMath,

although in practice this is highly unlikely. To cover all the possibilities allowed by XML would make it much more complicated to write an application to read any OpenMath object from scratch. Whether to adopt XML completely remains a hot topic of debate within the OpenMath community!

Generally speaking, it is not intended that the existing encodings should be readable by a human user or writable by hand. It is desirable that they be compact and it is also desirable that they be linear, but neither of these is a requirement. It is a property of encodings that it is possible to convert between them with no loss of information.

Content Dictionaries

Content Dictionaries (or CDs for short) are the most important, and the most interesting, aspect of OpenMath because they define the meaning of the objects being transmitted. A CD is a collection of related symbols and their definitions, encoded in an XML format. Defining the meaning of a symbol is not a trivial task, and even referring to well-known references can be fraught with pitfalls. Formal definitions and properties can be very useful but time-consuming to produce and verbose, not to mention difficult to get right. A symbol definition in an OpenMath CD consists of the following pieces of information:

- the symbol name;
- a description in plain text;
- optionally, a set of this symbol's properties in plain text
(Commented Mathematical Properties, or CMPs);
- optionally, a set of this symbol's properties encoded in OpenMath
(Formal Mathematical Properties, or FMPs);
- optionally, one or more examples of its use (encoded in OpenMath).

In practice the CMPs and FMPs can come as pairs, and often serve in the place of examples.

A very simple instance of a CD definition is:

```
<CDDefinition>
<Name> log </Name>

<Description>
This symbol represents a binary log function; the first argument is
the base, to which the second argument is log'ed.
It is defined in Abramowitz and Stegun, Handbook of Mathematical
Functions, section 4.1
</Description>
<CMP>
  a^b = c implies log_a c = b
</CMP>
<FMP>
  <OMOBJ>
    <OMA>
      <OMS cd="logic1" name="implies"/>
    </OMA>
  </OMOBJ>
</FMP>
</CDDefinition>
```

```

    <OMS cd="relation1" name="eq"/>

    <OMA>
      <OMS cd="arith1" name="power"/>
      <OMV name="a"/>
      <OMV name="b"/>
    </OMA>
    <OMV name="c"/>
  </OMA>
</OMA>
<OMS cd="relation1" name="eq"/>

  <OMA>
    <OMS cd="transc1" name="log"/>
    <OMV name="a"/>
    <OMV name="c"/>
  </OMA>
  <OMV name="b"/>
</OMA>
</OMA>
</OMOBJ>

</FMP>

<Example>
log 100 to base 10 (which is 2).
<OMOBJ>
  <OMA>
    <OMS cd="transc1" name="log"/>
    <OMF dec="10"/>
    <OMF dec="100"/>
  </OMA>
</OMOBJ>
</Example>

</CDDefinition>

```

Another example would be to print the list

[1, 1/2]

as

```

<OMOBJ>
  <OMA>
    <OMS cd="list1" name="list"/>
    <OMI>1</OMI>
  </OMA>
  <OMS cd="nums1" name="rational"/>
  <OMI>1</OMI>

```

```

    <OMI>2</OMI>
  </OMA>
</OMA>
</OMOBJ>

```

This provides a symbol to represent the log function by giving a pointer to a standard reference book. It provides the property that:

$$a^b = c \rightarrow \log_a(c) = b$$

both as plain text and as OpenMath, and also gives an example of how the symbol is used.

CDs usually consist of related symbols and collections of related CDs can be grouped together, for convenience, as CD Groups. One very important CD Group is that corresponding to the content part of MathML.

It is possible to associate extra information with CDs, in particular type information. Since there are many type systems available, each of which has its own strengths and advocates, the OpenMath community does not mandate any single system. Simple signatures can be encoded using the Simple Type System, while more formal definitions are possible using the Extended Calculus of Constructors. Other associated information can include style sheets for rendering OpenMath symbols in MathML, and mathematical definitions to be used by formal logic systems.

Given the evolutionary nature of mathematics, it is clear that the set of CDs should be forever growing and never complete. Currently there are CDs for high-school mathematics, linear algebra, polynomials and group theory to name a few, and new contributions are always welcome. There is no requirement that applications use the standard set of CDs and it is often very useful to design a "private" CD for a specific purpose.

OpenMath in Action

There is no definitive way in which OpenMath should be used, as the protocol has been designed to be as flexible as possible. Nevertheless many OpenMath applications share common characteristics which we shall discuss here.

Suppose that we wish to have two applications communicating by sending OpenMath objects to each other, e.g. a client program and a computational server. It is unlikely that the internal data structures used by the applications will be OpenMath, and so translation between the internal representations and OpenMath (almost certainly OpenMath encodings rather than objects) will have to take place. The piece of software which does this is usually referred to as a phrase-book.

It is possible to write a generic phrase-book which can handle any piece of OpenMath, but applications where this makes sense are few and far between. In practice an OpenMath phrase book will usually only handle a fixed set of CDs (and hence a fixed set of symbols). What "handle" means will vary from case to case: a computer algebra system will usually try and evaluate its input and return a result or an error, while a typesetter will print its input according to some rendering rules and not return anything. OpenMath carefully avoids

defining what the “right” behaviour is in a given circumstance, and leaves that up to the phrase-book writer. Indeed it is quite possible that a piece of software could have multiple phrase-books associated with it for different purposes. OpenMath symbols should not be regarded as verbs since they are used to construct objects rather than to send commands, and the presence of both nouns and verbs in a CD (e.g. “integral” and “integrate”) is strongly discouraged.

Writing a phrase-book may be non-trivial, and requires an understanding of the semantics of the underlying software. An OpenMath object may not map directly into a private object and vice-versa, for example in some systems a rational number might have to be represented by a float, or a sparse matrix by a dense one.

The OpenMath standard includes a section on compliance, which describes the behaviour of an OpenMath application when certain errors occur. It also insists that all compliant software has the capability to use the XML encoding, to guarantee a degree of interoperability. This is an area where the standard is expected to evolve as more OpenMath applications become available.

41.2 Technical Details

This chapter describes the Axiom implementation of the OpenMath project[Dalm97] at INRIA. The code enables the exchange of OpenMath objects between two processes and more generally the input and output of OpenMath objects. First we describe the library API and then we implement the functions used by Axiom.

41.3 The Structure of the API

The library and its API are logically structured in four parts:

- Functions that deal with *devices*, the abstraction from which OpenMath objects are read and written to.
- Functions that read from and write to OpenMath devices. These functions use a simple model that read and write tokens.
- Functions that create I/O structures to be used by devices, so that, for example, an OpenMath object can be read from a file or a socket. This part is extensible by the user.
- Functions that deal with interprocess communication.

41.4 OpenMath Expressions

Expressions

The library understands the following kinds of basic OpenMath expressions:

- integers
- double precision floating-point numbers (64 bits, following IEEE 754)
- byte arrays
- character strings
- symbols
- variables

and the four kinds of constructions:

- applications $e_0(e_1, \dots e_n)$
- errors $s(e_1, \dots e_n)$
- binders $e_1, (v_1, \dots v_n), e_2$
- attributed expressions $[s_1 e_1, \dots s_n e_n]e$

where e_i are OpenMath expressions, v_i are OpenMath variables and s and s_i are OpenMath symbols.

Symbols

Symbols are constructed from a content dictionary (abbreviated as CD in the sequel) and a name. A content dictionary is identified by its name. The API permits the creation of any symbol in any content dictionary: there is nothing that prevents creating symbols that do not belong to a known CD.

Encoding and Decoding OpenMath Expressions

An OpenMath object is encoded as a sequence of bytes that is read and written sequentially. The library views this sequence as a stream of tokens. Expressions are linearized in a way that looks like Lisp with typed parenthesis. For example, the linearization of the application of S to $E_1 \dots E_n$ is:

- indicating that this is an application (a “begin application” token)
- linearizing S

- linearizing $E_1, \dots, E_n$
- indicating that all arguments have been given (an “end application” token)

The other constructions are linearized the same way (each one with its own begin and end tokens). Note that there is no explicit arity indication so that we don’t have to introduce a special mechanism when we don’t know beforehand how many arguments there are.

To give attributes to an expression, the attributes and their associated values are put before the expression. To give the attributes a_i with values v_i (where a_i are symbols and v_i are OpenMath expressions) to an expression E the process is:

- put a “begin attributed expression” token
- put a “begin attribute pairs” token
- put the symbol a_1 followed by the linearization of v_1 etc
- put an “end attribute pairs” token
- linearize E
- put an “end attributed expression” token

Decoding is done by first querying the type of the next OpenMath token and then invoking the right function to get this particular kind of token.

41.5 Big Integers

The library supports big integers that can potentially be given in various formats. The `OMBigIntType` describes the different possible formats.

```
typedef enum OMBigIntType {
    OMBIunknown = 0, /* this is base 10, digits in normal order */
    OMBIbase10      /* this is base 16, digits in normal order (MSB) */
    OMBigIntBase16
} OMBigIntType;
```

41.6 Functions Dealing with OpenMath Devices

OpenMath expressions are read and written through *devices*. Basically, an OpenMath device has an associated encoding and an I/O method. There are basically two encodings defined and implemented. The first one is a human readable and writable one that can be used for example as the encoding for sending OpenMath objects via e-mail or storing OpenMath objects to files. This encoding is SGML compatible in the sense that it can be used to represent OpenMath objects in SGML texts. I has an XML variant. The second encoding

is a binary one that can be used when compactness and speed of encoding and decoding is important. The encodings are defined by the `OMencodingType` type which is an enumerated type defined as

```
typedef enum OMencodingType
{
    OMencodingUnknown,
    OMencodingBinary,
    OMencodingSGML,
    OMencodingXML} OMencodingType;
```

`OMencodingUnknown` is to be used when creating a device that does not know which kind of encoding will be used. It must be used only for input devices.

A device is created with the following function, given an encoding and an appropriate I/O method:

- `OMdev OMmakeDevice(OMencodingType encoding, OMIO IO)`

Devices are closed with the following function

- `void OMcloseDevice(OMdev dev)`

Whether a device could be used both for reading and writing is entirely dependent on its I/O method.

The user can define its own I/O method as a function returning an `OMIO` object. This could enable him, for example, to use an existing transport protocol to exchange OpenMath expressions or to implement cut-and-paste of OpenMath expression by writing I/O structures that input and output to strings. The I/O section describes the available I/O structures in the library.

An `OMdev` object is a pointer to a structure that contains a lot of state. Almost all functions taking an `OMdev` object modify it. Likewise, an `OMIO` object carries a lot of state.

41.7 Functions to Write OpenMath Expressions to Devices

Beginning and Ending Objects

The following two functions mark the beginning and end of an OpenMath object.

- `OMstatus OMputObject(OMdev dev)`
- `OMstatus OMputEndObject(OMdev dev)`

These functions should be called before and after an OpenMath object is constructed in a device. In particular, the `OMputEndObject` function insures that the object has been completely written if any buffering was used.

Writing Basic Objects

Basic OpenMath objects are written using these functions:

- `OMstatus OMputInt32(OMdev dev, int n)`
- `OMstatus OMputBigInt(OMdev dev, const char *data, int len, int sign, OMbigIntType format)`
- `OMstatus OMputFloat64(OMdev dev, double *f)`
- `OMstatus OMputByteArray(OMdev dev, const char *data, int len)`
- `OMstatus OMputString(OMdev dev, const char *s)`
- `OMstatus OMputVar(OMdev dev, const char *name)`
- `OMstatus OMputSymbol(OMdev dev, const char *cd, const char *name)`

The `char *` arguments of `OMputString`, `OMputVar` and `OMputSymbol` are null-terminated strings. There are other functions that accept non null-terminated arrays of characters with their length. These are

- `OMstatus OMputStringN(OMdev dev, const char *str, int len)`
- `OMstatus OMputVarN(OMdev dev, const char *var, int len)`
- `OMstatus OMputSymbolN(OMdev dev, const char *cd, int clen, const char *name, int nlen)`

The format for the `data` argument of the `OMputBigInt` function is given by `format`. When `format` is `OMBIbase10`, it is the sequence of character of its base 10 representation without sign (most significant digit first). The sign of the big integer is given by the `sign` argument that should be an integer greater or equal to zero for a positive integer and less than zero for a negative one. For example, the following line outputs the value of $20!$ to `dev`:

```
OMputBigInt(dev, "265252859812191058636308480000000", 33, 1, OMBIbase10);
```

Writing Structured Objects

The following functions are used to mark the beginning and end of the structured objects. They should be called in nested pairs, correctly bracketed:

- `OMstatus OMputApp(OMdev dev)`
- `OMstatus OMputEndApp(OMdev dev)`
- `OMstatus OMputAttr(OMdev dev)`
- `OMstatus OMputEndAttr(OMdev dev)`

- `OMstatus OMputBind(OMdev dev)`
- `OMstatus OMputEndBind(OMdev dev)`
- `OMstatus OMputBVar(OMdev dev)`
- `OMstatus OMputEndBVar(OMdev dev)`
- `OMstatus OMputAtp(OMdev dev)`
- `OMstatus OMputEndAtp(OMdev dev)`
- `OMstatus OMputError(OMdev dev)`
- `OMstatus OMputEndError(OMdev dev)`

Here is an example showing how to use these functions to output $\sin x + y$, where x and y are represented as variables and $\sin$ is the symbol whose name is `sin` in the `Basic` content dictionary. This can be done using the following sequence:

```
OMputObject(dev);
OMputApp(dev);
  OMputSymbol(dev, "Basic", "sin");
  OMputApp(dev)
    OMputSymbol(dev, "Basic", "+");
    OMputVar(dev, "x");
    OMputVar(dev, "y");
  OMputEndApp(dev);
OMputEndApp(dev);
OMputEndObject(dev);
```

41.8 Functions to Extract OpenMath Expressions from Devices

Testing the type of the current token

The first step in decoding an expression from a device is to call the `OMgetType` function

- `OMstatus OMgetType(OMdev dev, OMtokenType *type)`

so that the correct function can be called to recover the current token.

`OMgetType` returns via its `type` argument an `OMtokenType` object indicating the type of the next object to be read from the device. `OMtokenType` is an enumerated type defined as

```
typedef enum OMtokenType {
  OMtokenUnknown, /* error catching trick */
  OMtokenInt32,
```

```

OMtokenBigInt,
OMtokenFloat64,
OMtokenByteArray,
OMtokenVar,
OMtokenString,
OMtokenSymbol,
OMtokenComment,
OMtokenApp,    OMtokenEndApp,
OMtokenAttr,   OMtokenEndAttr,
OMtokenAtp,    OMtokenEndAtp,
OMtokenError,  OMtokenEndError,
OMtokenObject, OMtokenEndObject,
OMtokenBind,   OMtokenEndBind,
OMtokenBVar,   OMtokenEndBVar,
} OMTokenType;

```

Note that the type of the current token can be tested multiple times. Two successive calls to `OMgetType` will always return the same result if no other `OMget...` function was called in between.

Extracting the current token

The following functions are used to read the basic OpenMath objects from devices:

- `OMstatus OMgetInt32(OMdev dev, int *i)`
- `OMstatus OMgetFloat64(OMdev dev, double *d)`
- `OMstatus OMgetBigInt(OMdev dev, char **data, int *len, int *sign, OMbigIntType *fmt)`
- `OMstatus OMgetBigIntN(OMdev dev, char *data, int len, int *sign, OMbigIntType *fmt)`
- `OMstatus OMgetByteArray(OMdev dev, char **data, int *len)`
- `OMstatus OMgetByteArrayN(OMdev dev, char *data, int len)`
- `OMstatus OMgetString(OMdev dev, char **str)`
- `OMstatus OMgetStringN(OMdev dev, char *str, int len)`
- `OMstatus OMgetVar(OMdev dev, char **var)`
- `OMstatus OMgetVarN(OMdev dev, char *var, int len)`
- `OMstatus OMgetSymbol(OMdev dev, char **cd, char **name)`
- `OMstatus OMgetSymbolN(OMdev dev, char *cd, int clen, char *name, int nlen)`

The functions that return variable size data exist in two versions. A simple version that does the necessary memory allocation itself (using `OMmalloc`) and a version (suffixed with `N`) that lets the user do the allocation itself. The size of the needed area can be determined with the following function:

- `int OMgetLength(OMdev dev)` returns the length of the next object.

that works for big integers, byte arrays, strings and variables. For symbols, the following function returns both the length of the content dictionary name and the length of the symbol name:

- `OMstatus OMgetSymbolLength(OMdev dev, int *klen, int *nlen)`

When the current token does not carry any data i.e. when `OMgetType` returns a marker, i.e. one of:

- `OMtokenApp,`
- `OMtokenEndApp,`
- `OMtokenAttr,`
- `OM tokenEndAttr,`
- `OMtokenAtp,`
- `OMtokenEndAtp,`
- `OMtokenError,`
- `OMtokenEndError,`
- `OMtokenObject,`
- `OMtokenEndObject,`
- `OMtokenBind,`
- `OMtokenEndBind,`
- `OMtokenBVar`
- `OMtokenEndBVar`

it is necessary to call the correct function to remove the marker. The available functions are

- `OMstatus OMgetObject(OMdev dev)`
- `OMstatus OMgetEndObject(OMdev dev)`
- `OMstatus OMgetApp(OMdev dev)`

- OMstatus OMgetEndApp(OMdev dev)
- OMstatus OMgetAttr(OMdev dev)
- OMstatus OMgetEndAttr(OMdev dev)
- OMstatus OMgetAtp(OMdev dev)
- OMstatus OMgetEndAtp(OMdev dev)
- OMstatus OMgetBind(OMdev dev)
- OMstatus OMgetEndBind(OMdev dev)
- OMstatus OMgetBVar(OMdev dev)
- OMstatus OMgetEndBVar(OMdev dev)
- OMstatus OMgetError(OMdev dev)
- OMstatus OMgetEndError(OMdev dev)

All the previous functions return OMsuccess when they succeed. When they return something else, there has been a problem such as calling the wrong function (OMgetApp when there is not a “beginning of application” mark) or a system error.

The sequence of calls to read an expression is thus completely similar (if we omit the calls to OMgetType) to the sequence of calls to write the expression. For example, the previous expression $(\sin x + y)$ can be recovered via the sequence:

```
OMgetObject(dev);
OMgetApp(dev);
  OMgetSymbol(dev, ...);
  OMgetApp(dev);
    OMgetSymbol(dev, ...);
    OMgetVar(dev, ...);
    OMgetVar(dev, ...);
  OMgetEndApp(dev);
OMgetEndApp(dev);
OMgetEndObject(dev);
```

OMgetInt32(OMdev dev, int *i) returns the integer through its i argument.

OMgetBigInt(OMdev dev, char **data, int *len, int *sign, OMbigIntType *fmt)

returns the data corresponding to the big integer in data, its length in len, its sign in sign and its format in fmt.

OMgetBigIntN(OMdev dev, char *data, int len, int *sign, OMbigIntType *fmt)

copies the data corresponding to the big integer in **data** buffer that should be (at least) **len** characters long. The sign and format are returned in the **sign** and **fmt** arguments.

OMgetByteArray(OMdev dev, char **data, int *len) returns the byte array through its **data** argument. Its length is returned via the **len** argument.

OMgetByteArrayN(OMdev dev, char *data, int len) copies the byte array in the **data** buffer that should be (at least) **len** characters long.

OMgetString(OMdev dev, char **str) returns the string through its **str** argument.

OMgetStringN(OMdev dev, char *str, int len) copies the string in the **str** buffer whose length should be (at least) **len**. If **len** is greater than the actual length of the string, a null character is added at the end of **str**.

OMgetVar(OMdev dev, char **var) returns the name of the variable (as a null-terminated string) in its **var** argument

OMgetVarN(OMdev dev, char *var, int len) copies the name of the variable in the **var** buffer, whose length should be (at least) **len**. If **len** is greater than the actual length of the variable name, a null character is added at the end of **var**.

OMgetSymbol(OMdev dev, char **cd, char **name) returns the content dictionary and the name of the symbol through the **cd** and **name** arguments.

OMgetSymbolN(OMdev dev, char *cd, int clen, char *name, int nlen) copies the content dictionary and the name of the symbols in the **cd** and **name** buffers. **cd** should be at least **clen** character long and **name** should be at least **nlen** long. When there is enough room (based on **clen** or **nlen**) a null character is added after the last character of the name (**cd** or **name**).

41.9 Comments in the SGML/XML Encodings

The library can also output and read comments (SGML/XML comments) with the following functions:

- **OMstatus OMputComment**(OMdev dev, char *comment)
- **OMstatus OMputCommentN**(OMdev dev, char *comment, int len)
- **OMstatus OMgetComment**(OMdev dev, char **comment)
- **OMstatus OMgetCommentN**(OMdev dev, char *comment, int len)

By default, comments are silently ignored by the library when reading OpenMath objects (and writing them using the binary encoding). The function

- **OMbool OMignoreComment**(OMdev dev, OMbool set)

changes this behaviour. When called with **OMfalse**, comments are passed to the application: the **OMgetType** function will return **OMtokenComment** when the current token is a comment and the **OMgetComment** or **OMgetCommentN** functions should be used to get the comments. When **OMignoreComment** is called with **OMtrue**, comments are ignored.

41.10 I/O Functions for Devices

We provide four functions that produce `OMIO` objects for devices. These functions provide I/O through the `stdio` library (on `FILE` object), file descriptors and character strings.

- `OMIO OMmakeIOFile(FILE *f)` associates the device with the file pointer `f`.
- `OMIO OMmakeIOfd(int fd)` associates the device with the file descriptor `fd`.
- `OMIO OMmakeIOHandle(HANDLE handle)` associates the device with a file handle *Windows specific version of `OMmakeIOfd().fd`.
- `OMIO OMmakeIOString(char **s)` associates the device with a string.

For example, the following code opens a device that reads from standard input:

```
dev = OMmakeDevice(OMencodingSGML, OMmakeIOFile(stdin));
```

The `OMmakeIOString` builds an input device that reads from a string or an output device that writes to a string. For input, `s` must point to a character string (null terminated). For output, `s` will point to a string allocated by the library (note that the string `s` points to can be reallocated by the library).

41.11 Communications

A communication layer can be put above the device layer. In fact, the I/O structure in a device provides all the necessary support to use any transmission or communication means. This library directly provides some connection-oriented, client-server facilities (based on TCP).

A set of functions are used to set up connections. Connections are described by the `OMconn` type. An `OMconn` is a (pointer to a) structure with two user-accessible fields `in` and `out`. `in` is a pointer to a device to be used for input. `out` is pointer to a device to be used for output. These devices use the binary encoding.

An `OMconn` object is made with the following function:

- `OMconn OMmakeConn(int timeout)`

where `timeout` is a timeout for the connection, expressed in milliseconds.

- `OMdev OMconnIn(OMconn conn)` returns the input device associated with the connection.
- `OMdev OMconnOut(OMconn conn)` returns the output device associated with the connection.

Functions to Initiate an OMconn

The functions we provide can be divided in two classes. The first one simply establishes an interprocess communication using IP addresses. The second one provides functions that can be used to launch a server. The addresses used are then generated by the library.

Simple Connections Functions

The following functions allow a client OpenMath application to contact an OpenMath server at a specified address:

- `OMstatus OMconnTCP(OMconn conn, char *machine, int port)`
- `OMstatus OMconnUnix(OMconn conn, char *file)`

These functions first physically establish the connection. Then, they enter negotiation with the server (they send the first message). When they return, the negotiation is finished and the devices in the `conn` argument are ready.

On the server side, the following functions provide bindings at specified addresses and take care of the negotiation:

- `OMstatus OMbindTCP(OMconn conn, int port)`
- `OMstatus OMbindUnix(OMconn conn, char *file)`

All four the previous functions block until the connection is established (and negotiation is over) or the timeout of the `conn` argument is reached.

The following function returns the file descriptor associated with a device. This is intended to be used when there is a need to poll the device (through the `select` or `poll` system calls).

- `OMdeviceFd(OMdev dev)`

Functions that Launch Servers

These functions provide the same functionalities for launching a server that were provided in the ASAP library.

In this model, the client calls `OMlaunch` with a machine name `mach` and a string `cmd` that is executed via `rsh` on machine `mach` as a shell command line. This command is supposed to launch the server program. The command is executed in an environment (in the UNIX sense) where some variables are associated with an address on the machine that runs the client. The server can then connect to the client with the `OMserveClient` function.

If the machine name is `localhost`, the command is started on the same machine (without calling `rsh`).

- `OMstatus OMlaunchEnv(OMconn conn, char *machine, char *command, char *env)`

- `OMstatus OMlaunch(OMconn conn, char *machine, char *command)`
- `OMstatus OMserveClient(OMconn conn)`

The environment variables sent to the server (launched program) are `OM_CALLER_UNIX_SOCKET` (when a local connection is required) and `OM_CALLER_MACHINE` and `OM_CALLER_PORT` (for internet connections).

The `OMlaunchEnv` function enables the command to be run with a particular environment (in the UNIX sense). For example to run a `plot` server on the `kama` machine, we could use a piece of code such as

```
conn = OMmakeConn(2000);
OMlaunchEnv(conn, "kama", "plot", "DISPLAY=rati:0 PATH=/users/bin");
```

Termination

- `OMstatus OMconnClose(OMconn conn)`

41.12 Parameters

The library internally uses three functions that can be supplied by the user.

- `extern void *(*OMmalloc) (size_t size)`
- `extern void *(*OMrealloc) (void *ptr, size_t size)`
- `extern void (*OMfree) (void *ptr)`
- `OMmalloc` is used for all memory allocations in the library. The default value is the `malloc` function.
- `OMfree` is used for deallocations. The default value is the `free` function.
- `OMfatal` is invoked when a fatal error is detected in the library (for example when memory allocation failed or when an inconsistency is detected in the library code data structures). The default value just does an `exit`.

`OMfatal` is declared as `extern void (*OMfatal)(OMstatus status)`. All memory allocations and deallocations in the library are done through the `OMmalloc` and `OMfree` functions.

41.13 Miscellaneous Functions and Variables

- `char *OMstatusToString(OMstatus status)` make a status into a human readable string.

- `char *OMtokenTypeToString(OMtokenType ttype)` makes a `tokenType` into a human readable string.
- `OMencodingType OMgetDeviceEncoding(OMdev dev)` returns the encoding actually used by the device.
- `char *OMlibDynamicInfo(void)`
- `extern const char *OMlibVersion` is the version of the library.
- `extern const char *OMlibInfo` contains some textual information about the library.

41.14 The OM.h header file

```
#ifndef __OM_h__
#define __OM_h__

/*
 *
 *          All types used through API.
 */

/* These types are anonymized by the mean of a generic pointer.
 * You should not allocate or dereference objects of these types.
 * API (hopefully) provides you with all needed methods.
 * If you find any that are not included, please refer to
 * us rather than using private structures.
 * ie: If you need to do something like
 *   malloc(sizeof(OMdevStruct));
 * or
 *   OMdevStruct * pDev;
 *   pDev->anyField = something;
 * this probably means we need to discuss your problem.
 */

/* A device is an abstraction for put/get of OpenMath tokens */
typedef struct OMdevStruct *OMdev;

/* IO is a device field, (the physical IO channel) */
typedef struct OMIOStruct *OMIO;

/* Error status that may be returned
 */
typedef enum OMstatus {
    /* Last call was successful. */
    OMsuccess = 0,
    /* Last call failed for some undetermined reason. */
    OMfailed = 1,
    /* Last call failed for memory reasons. */

```

```

OMnoMem,
/* Last call failed during some system call. */
OMerrorSys,
/* Last call to some OMget* function failed due to an unexpected EOF
   on input IO. */
OMemptyIO,
/* Last call to some OMget* function failed because there is no more
   token on device. */
OMnoMoreToken,
/* Last call to some OMget* function timedout. */
OMtimedoutRead,
/* Last call to some OMget* function failed due to malformed input.
   (this error covers all low level lexical or syntactic problems). */
OMmalformedInput,
/* Last call to OMbindTCP failed because address is already in use
   (EADDRINUSE). */
OMaddrInUse,
/* Last call to OMconnTCP failed to set connection. */
OMconnectFailed,
/* Last call triggered some not (yet) implemented code in this lib. */
OMnotImplemented,
/* Last call caused some internal trouble. */
OMinternalError
} OMstatus;

/* All OpenMath token kinds are identified by one of these types.
 * Values given in this enum have been chosen to:
 * - avoid conflicts with specific XML characters
 *   to help automatic detection of encoding type.
 * (no: '\t'(9) '\r'(13) '\n'(10) '<'(60) or ' '(32))
 * - keep some bits (3) available for special encodings purpose
 * (eg: sharing or big len flags in binary encoding)
 */
typedef enum OMtokenType {
  OMtokenUnknown = 0, /* error catching trick */
  OMtokenInt32 = 1,
  OMtokenBigInt = 2,
  OMtokenFloat64 = 3,
  OMtokenByteArray = 4,
  OMtokenVar = 5,
  OMtokenString = 6,
  OMtokenWCString = 7,
  OMtokenSymbol = 8,
  OMtokenComment = 15,
  OMtokenApp = 16, OMtokenEndApp = 17,
  OMtokenAttr = 18, OMtokenEndAttr = 19,
  OMtokenAtp = 20, OMtokenEndAtp = 21,
  OMtokenError = 22, OMtokenEndError = 23,
  OMtokenObject = 24, OMtokenEndObject = 25,
  OMtokenBind = 26, OMtokenEndBind = 27,

```

```

    OMtokenBVar = 28,    OMtokenEndBVar = 29
} OMtokenType;

typedef enum OMbigIntType {
    OMbigIntUnknown = 0,
    /* this is base 10, digits in normal order (MSB) */
    OMbigIntBase10,
    /* this is base 16, digits in normal order (MSB) */
    OMbigIntBase16
} OMbigIntType;

/* Encodings should not be "user visible"
 * We thus refer to encoding as "symbolic constants" from this enum type. */
typedef enum OMencodingType {
    /* You may set an input stream to "unknown encoding".
     * By doing this, you let library auto detect the
     * encoding type of the device during first token input.*/
    OMencodingUnknown = 0,
    /* Binary encoding, more compact than XML one. */
    OMencodingBinary,
    /* XML-like encoding, human readable. */
    OMencodingXML,
} OMencodingType;

/* This is a portable equivalent to wchar_t for unicode strings */
typedef unsigned short OMUCS2;

/* Replacment for lacking C bools */
typedef unsigned char OMbool;
#define OMfalse (0)
#define OMtrue  (1)

/*
 *
 *                      Some global variables
 */

/* Version of this lib (eg: "1.0") */
extern const char *OMlibVersion;

/* Some textual information about this lib (eg: "debug is on" */
extern const char *OMlibInfo;

/* These pointers allow you to redefine memory managment functions
   used in lib. */
extern void *(*OMmalloc) (size_t size);
extern void *(*OMrealloc) (void *ptr, size_t size);
extern void (*OMfree) (void *ptr);

/* If set, this function will be called by OMfatal, thus you may use it for
   error handling (by default it is set to exit()) */

```

```

extern void (*OMfatal) (OMstatus status);

/* for C++ includes */
#ifdef __cplusplus
#define OMbeginPrototypes    extern "C" {
#define OMendPrototypes      }
#else /*__cplusplus */
#define OMbeginPrototypes
#define OMendPrototypes
#endif /*__cplusplus */

/*
 *                      Prototypes of OpenMath API
 */

/* Prototypes that are spread along all headers are repeated here.
 * - This should ease the API users.
 *   (docs are fine but source is always the ultimate help)
 * - This allow a cleaner embedding of library
 *   (no need to install all .h! just take this one and the .a)
 */
OMbeginPrototypes
#ifndef OM_DEV
/* this part is automatically updated, do NOT edit below */
/** Prototypes */
/* OMPut* functions.
 *   They all take a device <dev> to put token to.
 *   Some of them need more parameters to define the token content.
 *   They are thoroughly documented in OpenMath Specification shipped
 *   with the library.
 * return: a status that reflect the operation success.
 */
extern OMstatus OMputInt32(OMdev dev, int n);
extern OMstatus OMputFloat64(OMdev dev, double *d);
extern OMstatus OMputBigInt(OMdev dev, const char *data, int len,
                           int sign, OMbigIntType format);
extern OMstatus OMputByteArray(OMdev dev, const char *data, int len);
/* OMputString*
 *   If you want to output plain 8bits C like strings there is no need
 *   to use the OMputWCString* functions. This one is more efficient
 *   (faster and more compact output for some encodings)
 */
extern OMstatus OMputString(OMdev dev, const char *str);
extern OMstatus OMputStringN(OMdev dev, const char *str, int len);
/* OMputWCString
 *   If you are using wide char strings you need to output them
 *   with that function rather than with OMputString.
 *   (It takes endianness into account)

```

```

*/
extern OMstatus OMputWCString(OMdev dev, const OMUCS2 * wcstr);
extern OMstatus OMputVar(OMdev dev, const char *var);
extern OMstatus OMputVarN(OMdev dev, const char *var, int len);
extern OMstatus OMputSymbol(OMdev dev, const char *cd, const char *name);
extern OMstatus OMputSymbolN(OMdev dev, const char *cd, int clen,
                             const char *name, int nlen);

extern OMstatus OMputApp(OMdev dev);
extern OMstatus OMputEndApp(OMdev dev);
extern OMstatus OMputAttr(OMdev dev);
extern OMstatus OMputEndAttr(OMdev dev);
extern OMstatus OMputAtp(OMdev dev);
extern OMstatus OMputEndAtp(OMdev dev);
extern OMstatus OMputBind(OMdev dev);
extern OMstatus OMputEndBind(OMdev dev);
extern OMstatus OMputBVar(OMdev dev);
extern OMstatus OMputEndBVar(OMdev dev);
extern OMstatus OMputObject(OMdev dev);
extern OMstatus OMputEndObject(OMdev dev);
extern OMstatus OMputError(OMdev dev);
extern OMstatus OMputEndError(OMdev dev);
extern OMstatus OMputComment(OMdev dev, const char *comment);
extern OMstatus OMputCommentN(OMdev dev, const char *comment, int len);
/* OMgetType
 * Get the type of the current token on device <dev>/
 * dev: device to look at.
 * type: where to store returned type.
 * return: 0 or some error code
 */
extern OMstatus OMgetType(OMdev dev, OMtokenType * type);
/* OMgetLength
 * Get the current token length.
 * dev: device to read from
 * len: where to put the token length
 * the last '\0' for string like tokens is not counted
 * (rem: for WCString it is the number of bytes not the number of
 * wide chars)
 * return: 0 or some error code
 */
extern OMstatus OMgetLength(OMdev dev, int *len);
/* OMgetSymbolLength
 * Get the current token (wich is assumed to be a symbol) lengths.
 * dev: device to read from
 * clen: where to put the cd length (not counting the last '\0')
 * nlen: where to put the name length (not counting the last '\0')
 * return: 0 or some error code
 */
extern OMstatus OMgetSymbolLength(OMdev dev, int *clen, int *nlen);
/* OMGet* functions.
 * They all take a device <dev> to get token from.

```

```

*   Some of them need more parameters to fill with the token content.
*   They are thoroughly documented in OpenMath Specification shipped with
*   the library.
* return: a status that reflect the operation success.
*/
extern OMstatus OMgetInt32(OMdev dev, int *i);
extern OMstatus OMgetFloat64(OMdev dev, double *d);
extern OMstatus OMgetBigInt(OMdev dev, char **data, int *len, int *sign,
                           OMbigIntType * format);
extern OMstatus OMgetBigIntN(OMdev dev, char *data, int len, int *sign,
                           OMbigIntType * format);
extern OMstatus OMgetByteArray(OMdev dev, char **data, int *len);
extern OMstatus OMgetByteArrayN(OMdev dev, char *data, int len);
/* OMgetString*
*   Beware! You are not supposed to use these functions unless you know
*   for sure you are reading plain 8bits strings.
*   Thus it is here only for speed/space consideration in very
*   specific applications.
*   If input is a 16 bit char string and you read it with these
*   functions you will lose the 8 most significant bits of each char.
*   You should rather refer to OMgetWCString* functions.
*/
extern OMstatus OMgetString(OMdev dev, char **str);
extern OMstatus OMgetStringN(OMdev dev, char *str, int len);
/* OMgetWCString*
*   These functions return 16 bits wide strings. (regardless input
*   was done in 8 or 16 bits mode).
*   Thus, most if not all applications should use these functions
*   preferably to OMgetString*.
*/
extern OMstatus OMgetWCString(OMdev dev, OMUCS2 ** wctr);
/* BEWARE: the <len> is supposed to be the length in bytes for the
* preallocated buffer <wctr> (not the length in number of wide chars)
*/
extern OMstatus OMgetWCStringN(OMdev dev, OMUCS2 * wctr, int len);
extern OMstatus OMgetVar(OMdev dev, char **var);
extern OMstatus OMgetVarN(OMdev dev, char *var, int len);
extern OMstatus OMgetSymbol(OMdev dev, char **cd, char **name);
extern OMstatus OMgetSymbolN(OMdev dev, char *cd, int clen, char *name,
                             int nlen);
extern OMstatus OMgetApp(OMdev dev);
extern OMstatus OMgetEndApp(OMdev dev);
extern OMstatus OMgetAttr(OMdev dev);
extern OMstatus OMgetEndAttr(OMdev dev);
extern OMstatus OMgetAtp(OMdev dev);
extern OMstatus OMgetEndAtp(OMdev dev);
extern OMstatus OMgetBind(OMdev dev);
extern OMstatus OMgetEndBind(OMdev dev);
extern OMstatus OMgetBVar(OMdev dev);
extern OMstatus OMgetEndBVar(OMdev dev);

```

```

extern OMstatus OMgetObject(OMdev dev);
extern OMstatus OMgetEndObject(OMdev dev);
extern OMstatus OMgetError(OMdev dev);
extern OMstatus OMgetEndError(OMdev dev);
extern OMstatus OMgetComment(OMdev dev, char **comment);
extern OMstatus OMgetCommentN(OMdev dev, char *comment, int len);
/* OMbeginObject
 * Must be called before every new OpenMath object put.
 * (Not before every token!)
 * dev: device where new object is to be put.
 * return: status describing operation success
 */
extern OMstatus OMbeginObject(OMdev dev);
/* OMendObject
 * Must be called after every OpenMath object put.
 * (Not after every token!)
 * dev: device where object has been put.
 * return: status describing operation success
 */
extern OMstatus OMendObject(OMdev dev);
/* OMignoreComment
 * Set behavior of a device concerning comments.
 * (Comments on an input device may safely be ignored.)
 * dev: device to modify
 * set: If set == OMtrue then device will ignore incoming comments
 *      If set == OMfalse then device will process incoming comments
 *      like other tokens.
 *      By default comments are ignored.
 *      Whatever is <set> value, output of comments is always done.
 * return: previous value
 */
extern OMbool OMignoreComment(OMdev dev, OMbool set);
/* OMtokenCount
 * Reports the number of tokens that have been in/output on a device
 * dev: device to examine
 * inTokenNb: where to store number of input tokens (if not NULL)
 * outTokenNb: where to store number of output tokens (if not NULL)
 */
extern void OMtokenCount(OMdev dev, int *inTokenNb, int *outTokenNb);
/* OMgetDeviceEncoding
 * Get the current encoding used by a device
 * dev: device to examine
 * return: current encoding
 */
extern OMencodingType OMgetDeviceEncoding(OMdev dev);
/* OMsetDeviceEncoding
 * Set the encoding that will be used on a device
 * BEWARE: changing encoding on a device that has already been used
 * for IO is unsafe.
 * but setting encoding on a new device is safe.

```

```

*   (in some occasions, it is not easy to know which encoding to
*   use at device creation)
* dev: device to modify
* encoding: encoding to use
*/
extern void OMsetDeviceEncoding(OMdev dev, OMencodingType encoding);
/* OMmakeDevice
*   Create a device from a low level IO
*   Warning: "IO" should be a "generated" (new) structure as it contains some
*   state that is private to the device. It is very dangerous for two devices
*   to share the same "IO" structure.
* encoding: encoding scheme used by device
* IO: low level I/O support for device
* return: a newly allocated device
*/
extern OMdev OMmakeDevice(OMencodingType encoding, OMIO IO);
/* OMcloseDevice
*   Close a device previously created with OMmakeDevice
*   (embedded IO is closed too)
* dev: device to close
*/
extern void OMcloseDevice(OMdev dev);
/* OMmakeIOFd
*   Create a low level IO object from a file descriptor.
*   (May be used on socket for instance.)
* fd: file descriptor to wrap into the OpenMath IO object.
* return: a newly allocated IO object.
*/
extern OMIO OMmakeIOFd(int fd);
/* OMmakeIOFile
*   Create a low level IO object from a FILE*.
*   (May be used on stdin for instance.)
* fd: FILE* to wrap into the OpenMath IO object.
* return: a newly allocated IO object.
*/
extern OMIO OMmakeIOFile(FILE * f);
/* OMmakeIOString
*   Create a low level IO object from a string (NUL terminator is not needed).
*   (May be used for copy/paste for instance.)
* s: pointer to string to use into the OpenMath IO object.
*   - In case of input device the string must be NUL terminated.
*   - In case of output device string may be reallocated
*     to fit size of outgoing objects.
* return: a newly allocated IO object.
*/
extern OMIO OMmakeIOString(char **s);
/* OMstatusToString
*   Convert a status to a human readable string that explain its meaning
* status: status to explain
* return: corresponding string

```



```

*/
extern char *OMstatusToString(OMstatus status);
/* OMtokenTypeToString
 * Convert a tokenType to a human readable string
 * ttype: type to convert
 * return: corresponding string
 */
extern char *OMtokenTypeToString(OMtokenType ttype);
/* OMsetVerbosityLevel
 * When using API some infos may be logged.
 * This set the required verbosity level.
 * level: level of verbosity.
 * 0 means nothing is nether printed
 * 1 everything is printed (default)
 * 2,... less verbose
 * return: last verbosity level
 */
extern int OMsetVerbosityLevel(int level);
/* OMsetVerbosityOutput
 * When using API some infos may be logged.
 * This set the destination for logs.
 * logFile: where to output logs (default is stderr)
 * return: last output
 */
extern FILE *OMsetVerbosityOutput(FILE * logFile);
/* OMLibDynamicInfo
 * Gather some informations about lib that can't be statically determined.
 * Complete them with some relevant static information too.
 * return: a newly allocated string
 */
extern char *OMlibDynamicInfo(void);
/**** End Prototypes */
/* end of automatically updated part */

#ifdef WIN32
#include "windows.h"

/* OMmakeIOHandle
 * Create a low level IO object from a widows handle.
 * handle: windows handle to wrap into the OpenMath IO object.
 * return: a newly allocated IO object.
 */
extern OMIO OMmakeIOHandle(HANDLE handle);
extern void OMfreeIOHandle(OMIO io);
#endif

#else /* OM_DEV */
/* The prototypes above are in fact collected from all these .h files */
#include "OMbase.h"
#include "OMdev.h"

```

```
#include "OMdevFd.h"
#include "OMdevFile.h"
#include "OMdevString.h"
#include "OMdevHandle.h"
#include "OMencBin.h"
#include "OMencXml.h"
#include "OMmisc.h"
#include "OMutf7.h"
#endif /* OM_DEV */
```

```
OMendPrototypes
```

```
#endif /* __OM_h__ */
```

41.15 Axiom OpenMath stub functions

These stub functions will eventually be expanded to handle OpenMath. See the OpenMath-Device domain in Volume 10.3. Note that the argument list for the Spad functions does not always match the argument list specified in the OpenMath specification.

There are 4 known OpenMath encodings which are set up in the OpenMathEncoding domain in Volume 10.3.

- Unknown
- Binary
- XML
- SGML

Axiom specific functions

This is used in OpenMathPackage in Volume 10.4.

```
(read OMdev)           -> LispObject
(listCDs)              -> List(String)
(listSymbols)          -> List(String)
(supportsCD cd)        -> Boolean
(supportsSymbol cd name) -> Boolean
```

defun om-Read

Read an OpenMath object from dev.
— defun om-Read —

```
(defun om-Read (dev)
  (declare (ignore dev)))
```

defun om-listCDs

Lists all of the CDs supported by Axiom.

— **defun om-listCDs** —

```
(defun om-listCDs ())
```

defun om-listSymbols

Lists all the symbols in CD

— **defun om-listSymbols** —

```
(defun om-listSymbols ())
```

defun om-supportsCD

Return true if Axiom supports this CD.

— **defun om-supportsCD** —

```
(defun om-supportsCD (cd)
  (declare (ignore cd)))
```

defun om-supportsSymbol

— **defun om-supportsSymbol** —

```
(defun om-supportsSymbol (cd name)
  (declare (ignore cd name)))
```

Lisp conversion functions

The lisp conversion functions are:

(toDev	LispObject)	-> OMdev
(fromDev	OMdev)	-> LispObject
(toStatus	LispObject)	-> LispObject
(fromStatus	OMstatus)	-> LispObject
(toEncodingType	LispObject)	-> OMencodingType
(fromEncodingType	OMencodingType)	-> LispObject
(toBigNumStr	LispObject)	-> char *
(fromBigNumStr	char *,int,int, OMbigIntType)	-> LispObject
(toConn	LispObject)	-> OMconn
(fromConn	OMconn)	-> LispObject
(toCString	LispObject)	-> char **
(fromCString	char **)	-> LispObject
(lispStringFromCString	LispObject)	-> LispObject
(cStringFromLispString	LispObject)	-> LispObject

defun om-setDevEncoding

This sets the encoding used for reading or writing OpenMath objects to or from dev to enc.

— defun om-setDevEncoding —

```
(defun om-setDevEncoding (dev enc)
  (declare (ignore dev enc)))
```

Device manipulation functions

(openFileDev	LispObject, ints, ...)	-> LispObject
(openStrDev	LispObject, LispObject, LispObject)	-> LispObject
(closeDev	LispObject, LispObject)	-> LispObject

defun om-openFileDev

This opens file fname for reading or writing OpenMath objects. The mode can be “r” for read, “w” for write, or “a” for append.

— defun om-openFileDev —

```
(defun om-openFileDev (fname fmode enc)
  (declare (ignore fname fmode enc)))
```

defun om-openStringDev

This opens the string str for reading and writing OpenMath objects in encoding enc.

— **defun om-openStringDev** —

```
(defun om-openStringDev (str enc)
  (declare (ignore str enc)))
```

—————

defun om-closeDev

This closes dev, flushing output if necessary.

— **defun om-closeDev** —

```
(defun om-closeDev (dev)
  (declare (ignore dev)))
```

—————

Connection manipulation functions

These are covered in the OpenMathConnection domain in Volume 10.3.

```
(makeConn      LispObject, LispObject) -> LispObject
(closeConn     LispObject, LispObject) -> LispObject
(getConnInDev  LispObject, LispObject) -> LispObject
(getConnOutDev LispObject, LispObject) -> LispObject
```

defun om-makeConn

— **defun om-makeConn** —

```
(defun om-makeConn (conn)
  (declare (ignore conn)))
```

—————

defun om-closeConn

— **defun om-closeConn** —

```
(defun om-closeConn (conn)
  (declare (ignore conn)))
```

defun om-getConnInDev

— defun om-getConnInDev —

```
(defun om-getConnInDev (conn)
  (declare (ignore conn)))
```

defun om-getConnOutDev

— defun om-getConnOutDev —

```
(defun om-getConnOutDev (conn)
  (declare (ignore conn)))
```

Client/Server functions

These are covered in the OpenMathConnection domain in Volume 10.3. See OMconn.h

```
(bindTCP    LispObject, LispObject, LispObject) -> LispObject
(connectTCP LispObject, int, ...)               -> LispObject
```

defun om-bindTCP

— defun om-bindTCP —

```
(defun om-bindTCP (conn port)
  (declare (ignore conn port)))
```

defun om-connectTCP

— defun om-connectTCP —

```
(defun om-connectTCP (conn host port)
  (declare (ignore conn host port)))
```

—————

Device input/output functions

Most of these functions are in the OpenMathDevice domain in Volume 10.3. The only exception seems to be the om-stringPtrToString and om-stringToStringPtr functions which are called in the domains that export primitives. Currently these are:

- Complex (10.3)
- DoubleFloat (10.3)
- Float (10.3)
- Fraction (10.3)
- Integer (10.3)
- List (10.3)
- SingleInteger (10.3)
- String (10.3)
- Symbol (10.3)
- ExpressionToOpenMath (10.4)
- OpenMathPackage (10.4)

Note that putSymbol2 is not implemented.

(getApp	LispObject, LispObject)	-> LispObject
(getAtp	LispObject, LispObject)	-> LispObject
(getAttr	LispObject, LispObject)	-> LispObject
(getBind	LispObject, LispObject)	-> LispObject
(getBVar	LispObject, LispObject)	-> LispObject
(getByteArray	LispObject, LispObject)	-> LispObject
(getEndApp	LispObject, LispObject)	-> LispObject
(getEndAtp	LispObject, LispObject)	-> LispObject
(getEndAttr	LispObject, LispObject)	-> LispObject

```

(getEndBind      LispObject, LispObject) -> LispObject
(getEndBVar      LispObject, LispObject) -> LispObject
(getEndError     LispObject, LispObject) -> LispObject
(getEndObject    LispObject, LispObject) -> LispObject
(getError        LispObject, LispObject) -> LispObject
(getFloat        LispObject, LispObject) -> LispObject
(getInt          LispObject, LispObject) -> LispObject
(getObject       LispObject, LispObject) -> LispObject
(getString       LispObject, LispObject) -> LispObject
(getSymbol       LispObject, LispObject) -> LispObject
(getType         LispObject, LispObject) -> LispObject
(getVar          LispObject, LispObject) -> LispObject
(putApp          LispObject, LispObject) -> LispObject
(putAtp          LispObject, LispObject) -> LispObject
(putAttr        LispObject, LispObject) -> LispObject
(putBind        LispObject, LispObject) -> LispObject
(putBVar        LispObject, LispObject) -> LispObject
(putByteArray    LispObject, LispObject, LispObject) -> LispObject
(putEndApp       LispObject, LispObject) -> LispObject
(putEndAtp       LispObject, LispObject) -> LispObject
(putEndAttr      LispObject, LispObject) -> LispObject
(putEndBind      LispObject, LispObject) -> LispObject
(putEndBVar      LispObject, LispObject) -> LispObject
(putEndError     LispObject, LispObject) -> LispObject
(putEndObject    LispObject, LispObject) -> LispObject
(putError        LispObject, LispObject) -> LispObject
(putFloat        LispObject, LispObject, LispObject) -> LispObject
(putInt          LispObject, LispObject, LispObject) -> LispObject
(putObject       LispObject, LispObject) -> LispObject
(putString       LispObject, LispObject, LispObject) -> LispObject
(putSymbol       LispObject, LispObject, LispObject) -> LispObject
(putSymbol2      LispObject, int nargs, ...) -> LispObject
(putVar          LispObject, LispObject, LispObject) -> LispObject
(stringPtrToString LispObject, LispObject) -> LispObject
(stringToStringPtr LispObject, LispObject) -> LispObject

```

defun om-getApp

Reads a begin application token from dev.

— **defun om-getApp** —

```

(defun om-getApp (dev)
  (declare (ignore dev)))

```

—————

defun om-getAtp

Reads a begin attribute pair token from dev.

— **defun om-getAtp** —

```
(defun om-getAtp (dev)
  (declare (ignore dev)))
```

—————

defun om-getAttr

Reads a begin attribute token from dev

— **defun om-getAttr** —

```
(defun om-getAttr (dev)
  (declare (ignore dev)))
```

—————

defun om-getBind

Reads a begin binder token from dev.

— **defun om-getBind** —

```
(defun om-getBind (dev)
  (declare (ignore dev)))
```

—————

defun om-getBVar

Reads a begin bound variable list token from dev.

— **defun om-getBVar** —

```
(defun om-getBVar (dev)
  (declare (ignore dev)))
```

—————

defun om-getByteArray

Reads a byte array from dev.

— **defun om-getByteArray** —

```
(defun om-getByteArray (dev))
```

—————

defun om-getEndApp

Reads an end application token from dev

— **defun om-getEndApp** —

```
(defun om-getEndApp (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndAtp

Reads an end attribute pair token from dev.

— **defun om-getEndAtp** —

```
(defun om-getEndAtp (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndAttr

Reads an end attribute token from dev.

— **defun om-getEndAttr** —

```
(defun om-getEndAttr (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndBind

Reads an end binder token from dev.

— **defun om-getEndBind** —

```
(defun om-getEndBind (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndBVar

Reads an end bound variable list token from dev.

— **defun om-getEndBVar** —

```
(defun om-getEndBVar (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndError

Reads an end error token from dev.

— **defun om-getEndError** —

```
(defun om-getEndError (dev)
  (declare (ignore dev)))
```

—————

defun om-getEndObject

Reads an end object token from dev.

— **defun om-getEndObject** —

```
(defun om-getEndObject (dev)
  (declare (ignore dev)))
```

—————

defun om-getError

Reads a begin error token from dev.

— **defun om-getError** —

```
(defun om-getError (dev)
  (declare (ignore dev)))
```

—————

defun om-getFloat

Reads a float from dev.

— **defun om-getFloat** —

```
(defun om-getFloat (dev)
  (declare (ignore dev)))
```

—————

defun om-getInt

Reads an integer from dev.

— **defun om-getInt** —

```
(defun om-getInt (dev)
  (declare (ignore dev)))
```

—————

defun om-getObject

Reads a begin object token from dev.

— **defun om-getObject** —

```
(defun om-getObject (dev)
  (declare (ignore dev)))
```

—————

defun om-getString

Reads a string from dev.

— **defun om-getString** —

```
(defun om-getString (dev)
  (declare (ignore dev)))
```

—————

defun om-getSymbol

Reads a symbol from dev.

— **defun om-getSymbol** —

```
(defun om-getSymbol (dev)
  (declare (ignore dev)))
```

—————

defun om-getType

Returns the type of the next object on dev.

— **defun om-getType** —

```
(defun om-getType (dev)
  (declare (ignore dev)))
```

—————

defun om-getVar

Reads a variable from dev.

— **defun om-getVar** —

```
(defun om-getVar (dev)
  (declare (ignore dev)))
```

—————

defun om-putApp

Writes a begin application token to dev.

— **defun om-putApp** —

```
(defun om-putApp (dev)
  (declare (ignore dev)))
```

—————

defun om-putAtp

This writea a begin application pair token to dev.

— **defun om-putAtp** —

```
(defun om-putAtp (dev)
  (declare (ignore dev)))
```

—————

defun om-putAttr

This writes a begin attribute token to dev.

— **defun om-putAttr** —

```
(defun om-putAttr (dev)
  (declare (ignore dev)))
```

—————

defun om-putBind

This writes a begin binder token to dev.

— **defun om-putBind** —

```
(defun om-putBind (dev)
  (declare (ignore dev)))
```

—————

defun om-putBVar

This writes a begin bound variable list token to dev.

— **defun om-putBVar** —

```
(defun om-putBVar (dev)
  (declare (ignore dev)))
```

—————

defun om-putByteArray

This writes a byte array to dev.

— **defun om-putByteArray** —

```
(defun om-putByteArray (dev b)
  (declare (ignore dev b)))
```

—————

defun om-putEndApp

This writes an end application token to dev.

— **defun om-putEndApp** —

```
(defun om-putEndApp (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndAtp

This writes an end attribute pair to dev.

— **defun om-putEndAtp** —

```
(defun om-putEndAtp (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndAttr

This writes an end attribute token to dev.

— **defun om-putEndAttr** —

```
(defun om-putEndAttr (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndBind

This writes an end binder token to dev.

— **defun om-putEndBind** —

```
(defun om-putEndBind (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndBVar

This writes and end bound variable list token to dev

— **defun om-putEndBVar** —

```
(defun om-putEndBVar (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndError

This writes an end error token to dev

— **defun om-putEndError** —

```
(defun om-putEndError (dev)
  (declare (ignore dev)))
```

—————

defun om-putEndObject

This writes an end object token to dev.

— **defun om-putEndObject** —

```
(defun om-putEndObject (dev)
  (declare (ignore dev)))
```

—————

defun om-putError

This writes a begin error token to dev.

— **defun om-putError** —

```
(defun om-putError (dev)
  (declare (ignore dev)))
```

—————

defun om-putFloat

This writes the float f to dev.

— **defun om-putFloat** —

```
(defun om-putFloat (dev f)
  (declare (ignore dev f)))
```

—————

defun om-putInt

This writes the integer i to dev

— **defun om-putInt** —

```
(defun om-putInt (dev i)
  (declare (ignore dev i)))
```

—————

defun om-putObject

This writes a begin object token to dev.

— **defun om-putObject** —

```
(defun om-putObject (dev)
  (declare (ignore dev)))
```

—————

defun om-putString

This writes the string s to dev.

— **defun om-putString** —

```
(defun om-putString (dev s)
  (declare (ignore dev s)))
```

—————

defun om-putSymbol

This writes the symbol nm using semantics from cd to dev.

— **defun om-putSymbol** —

```
(defun om-putSymbol (dev cd nm)
  (declare (ignore dev cd nm)))
```

—————

defun om-putVar

This writes the variable v to dev.

— **defun om-putVar** —

```
(defun om-putVar (dev v)
  (declare (ignore dev v)))
```

—————

defun om-stringToStringPtr

This is used in the SingleInteger domain in Volume 10.3. This is supposed to return the string from its address? It would appear to be a nop in lisp.

— **defun om-stringToStringPtr** —

```
(defun om-stringToStringPtr (str)
  (declare (ignore str)))
```

—————

defun om-stringPtrToString

This is used in the SingleInteger domain in Volume 10.3. This is supposed to return the string address from a string? It would appear to be a nop in lisp.

— **defun om-stringPtrToString** —

```
(defun om-stringPtrToString (str)
  (declare (ignore str)))
```

—————

Chapter 42

NRLIB code.lisp support code

defun makeByteWordVec2

— defun makeByteWordVec2 0 —

```
(defun |makeByteWordVec2| (maxelement initialvalue)
  (let ((n (cond ((null initialvalue) 7) ('t maxelement))))
    (make-array (length initialvalue)
      :element-type (list 'mod (1+ n))
      :initial-contents initialvalue)))
```

—————

defmacro spadConstant

— defmacro spadConstant 0 —

```
(defmacro |spadConstant| (dollar n)
  '(spadcall (svref ,dollar (the fixnum ,n))))
```

—————

Chapter 43

Monitoring execution

MONITOR

This file contains a set of function for monitoring the execution of the functions in a file. It constructs a hash table that contains the function name as the key and monitor-data structures as the value

The technique is to use a :cond parameter on trace to call the monitor-incr function to incr the count every time a function is called

```
*monitor-table*                                HASH TABLE
  is the monitor table containing the hash entries
*monitor-nrlibs*                               LIST of STRING
  list of nrlib filenames that are monitored
*monitor-domains*                             LIST of STRING
  list of domains to monitor-report (default is all exposed domains)
monitor-data                                  STRUCTURE
  is the defstruct name of records in the table
  name is the first field and is the name of the monitored function
  count contains a count of times the function was called
  monitorp is a flag that skips counting if nil, counts otherwise
  sourcefile is the name of the file that contains the source code
```

***** SETUP, SHUTDOWN *****

```
monitor-inittable ()                          FUNCTION
  creates the hashtable and sets *monitor-table*
  note that it is called every time this file is loaded
monitor-end ()                                FUNCTION
  unhooks all of the trace hooks
```

***** TRACE, UNTRACE *****

```

monitor-add (name &optional sourcefile)      FUNCTION
  sets up the trace and adds the function to the table
monitor-delete (fn)                          FUNCTION
  untraces a function and removes it from the table
monitor-enable (&optional fn)                FUNCTION
  starts tracing for all (or optionally one) functions that
  are in the table
monitor-disable (&optional fn)               FUNCTION
  stops tracing for all (or optionally one) functions that
  are in the table

```

***** COUNTING, RECORDING *****

```

monitor-reset (&optional fn)                FUNCTION
  reset the table count for the table (or optionally, for a function)
monitor-incr (fn)                            FUNCTION
  increments the count information for a function
  it is called by trace to increment the count
monitor-decr (fn)                            FUNCTION
  decrements the count information for a function
monitor-info (fn)                            FUNCTION
  returns the monitor-data structure for a function

```

***** FILE IO *****

```

monitor-write (items file)                  FUNCTION
  writes a list of symbols or structures to a file
monitor-file (file)                         FUNCTION
  will read a file, scan for defuns, monitor each defun
  NOTE: monitor-file assumes that the file has been loaded

```

***** RESULTS *****

```

monitor-results ()                          FUNCTION
  returns a list of the monitor-data structures
monitor-unttested ()                        FUNCTION
  returns a list of files that have zero counts
monitor-tested (&optional delete)           FUNCTION
  returns a list of files that have nonzero counts
  optionally calling monitor-delete on those functions

```

***** CHECKPOINT/RESTORE *****

```

monitor-checkpoint (file)                   FUNCTION
  save the *monitor-table* in a loadable form
monitor-restore (file)                      FUNCTION
  restore a checkpointed file so that everything is monitored

```

***** ALGEBRA *****

```

monitor-autoload ()                         FUNCTION
  traces autoload of algebra to monitor corresponding source files

```

NOTE: this requires the /spad/int/algebra directory

monitor-dirname (args) FUNCTION
 expects a list of 1 libstream (loadvol's arglist) and monitors the source
 this is a function called by monitor-autoload

monitor-nrllib (nrllib) FUNCTION
 takes an nrllib name as a string (eg POLY) and returns a list of
 monitor-data structures from that source file

monitor-report () FUNCTION
 generate a report of the monitored activity for domains in
 monitor-domains

monitor-spadfile (name) FUNCTION
 given a spad file, report all nrllibs it creates
 this adds each nrllib name to *monitor-domains* but does not
 trace the functions from those domains

monitor-percent () FUNCTION
 ratio of (functions executed)/(functions traced)

monitor-apropos (str) FUNCTION
 given a string, find all monitored symbols containing the string
 the search is case-insensitive. returns a list of monitor-data items

for example:

suppose we have a file "/u/daly/testmon.lisp" that contains:

```
(defun foo1 () (print 'foo1))
(defun foo2 () (print 'foo2))
(defun foo3 () (foo1) (foo2) (print 'foo3))
(defun foo4 () (print 'foo4))
```

an example session is:

```
; FIRST WE LOAD THE FILE (WHICH INITIS *monitor-table*)
```

```
>(load "/u/daly/monitor.lisp")
Loading /u/daly/monitor.lisp
Finished loading /u/daly/monitor.lisp
T
```

```
; SECOND WE LOAD THE TESTMON FILE
```

```
>(load "/u/daly/testmon.lisp")
T
```

```
; THIRD WE MONITOR THE FILE
```

```
>(monitor-file "/u/daly/testmon.lisp")
monitoring "/u/daly/testmon.lisp"
NIL
```

```
; FOURTH WE CALL A FUNCTION FROM THE FILE (BUMP ITS COUNT)
```

```
>(foo1)
```

```
F001
```

```
F001
```



```

; AND ANOTHER FUNCTION (BUMP ITS COUNT)
>(foo2)

FOO2
FOO2

; AND A THIRD FUNCTION THAT CALLS THE OTHER TWO (BUMP ALL THREE)
>(foo3)

FOO1
FOO2
FOO3
FOO3

; CHECK THAT THE RESULTS ARE CORRECT

>(monitor-results)
(#S(MONITOR-DATA NAME FOO1 COUNT 2 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME FOO2 COUNT 2 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME FOO3 COUNT 1 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))
 #S(MONITOR-DATA NAME FOO4 COUNT 0 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))

; STOP COUNTING CALLS TO FOO2

>(monitor-disable 'foo2)
NIL

; INVOKE FOO2 THRU FOO3

>(foo3)

FOO1
FOO2
FOO3
FOO3

; NOTICE THAT FOO1 AND FOO3 WERE BUMPED BUT NOT FOO2
>(monitor-results)
(#S(MONITOR-DATA NAME FOO1 COUNT 3 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME FOO2 COUNT 2 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME FOO3 COUNT 2 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))
 #S(MONITOR-DATA NAME FOO4 COUNT 0 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))

```

```

"/u/daly/testmon.lisp"))

; TEMPORARILY STOP ALL MONITORING

>(monitor-disable)
NIL

; CHECK THAT NOTHING CHANGES

>(foo3)

F001
F002
F003
F003

; NO COUNT HAS CHANGED

>(monitor-results)
(#S(MONITOR-DATA NAME F001 COUNT 3 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F002 COUNT 2 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F003 COUNT 2 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp"))
 #S(MONITOR-DATA NAME F004 COUNT 0 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))

; MONITOR ONLY CALLS TO F001

>(monitor-enable 'foo1)
T

; F003 CALLS F001

>(foo3)

F001
F002
F003
F003

; F001 HAS CHANGED BUT NOT F002 OR F003

>(monitor-results)
(#S(MONITOR-DATA NAME F001 COUNT 4 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F002 COUNT 2 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F003 COUNT 2 MONITORP NIL SOURCEFILE
    "/u/daly/testmon.lisp"))

```

```

        "/u/daly/testmon.lisp"))
#S(MONITOR-DATA NAME F004 COUNT 0 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))

; MONITOR EVERYBODY

>(monitor-enable)
NIL

; CHECK THAT EVERYBODY CHANGES

>(foo3)

F001
F002
F003
F003

; EVERYBODY WAS BUMPED

>(monitor-results)
(#S(MONITOR-DATA NAME F001 COUNT 5 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F002 COUNT 3 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F003 COUNT 3 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))
#S(MONITOR-DATA NAME F004 COUNT 0 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp"))

; WHAT FUNCTIONS WERE TESTED?

>(monitor-tested)
(F001 F002 F003)

; WHAT FUNCTIONS WERE NOT TESTED?

>(monitor-untested)
(F004)

; UNTRACE THE WHOLE WORLD, MONITORING CANNOT RESTART

>(monitor-end)
NIL

; CHECK THE RESULTS

>(monitor-results)
(#S(MONITOR-DATA NAME F001 COUNT 5 MONITORP T SOURCEFILE
    "/u/daly/testmon.lisp")

```

```

#S(MONITOR-DATA NAME F002 COUNT 3 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp")
#S(MONITOR-DATA NAME F003 COUNT 3 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp"))
#S(MONITOR-DATA NAME F004 COUNT 0 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp"))

; CHECK THAT THE FUNCTIONS STILL WORK

>(foo3)

F001
F002
F003
F003

; CHECK THAT MONITORING IS NOT OCCURING

>(monitor-results)
(#S(MONITOR-DATA NAME F001 COUNT 5 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F002 COUNT 3 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp")
 #S(MONITOR-DATA NAME F003 COUNT 3 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp"))
 #S(MONITOR-DATA NAME F004 COUNT 0 MONITORP T SOURCEFILE
   "/u/daly/testmon.lisp"))

```

defvar *monitor-domains*

— initvars —

```
(defvar *monitor-domains* nil "a list of domains to report")
```

—————

defvar *monitor-nrlibs*

— initvars —

```
(defvar *monitor-nrlibs* nil "a list of nrlibs that have been traced")
```

—————

defvar *monitor-table*

— initvars —

```
(defvar *monitor-table* nil "a table of all of the monitored data")
```

—————

— postvars —

```
(eval-when (eval load)
  (unless *monitor-table* (monitor-inittable)))
```

—————

defstruct monitor-data

— initvars —

```
(defstruct monitor-data name count monitorp sourcefile)
```

—————

defstruct libstream

— initvars —

```
(defstruct libstream mode dirname (indextable nil) (indexstream nil))
```

—————

defun Initialize the monitor statistics hashtable

[*monitor-table* p1382]

— defun monitor-inittable 0 —

```
(defun monitor-inittable ()
  "initialize the monitor statistics hashtable"
  (declare (special *monitor-table*))
  (setq *monitor-table* (make-hash-table)))
```

defun End the monitoring process, we cannot restart

[*monitor-table* p1382]

— defun monitor-end 0 —

```
(defun monitor-end ()
  "End the monitoring process. we cannot restart"
  (declare (special *monitor-table*))
  (maphash
    #'(lambda (key value)
        (declare (ignore value))
        (eval '(untrace ,key)))
    *monitor-table*))
```

defun Return a list of the monitor-data structures

[*monitor-table* p1382]

— defun monitor-results 0 —

```
(defun monitor-results ()
  "return a list of the monitor-data structures"
  (let (result)
    (declare (special *monitor-table*))
    (maphash
      #'(lambda (key value)
          (declare (ignore key))
          (push value result))
      *monitor-table*)
    (mapcar #'(lambda (x) (pprint x))
      (sort result #'string-lessp :key #'monitor-data-name))))
```

defun Add a function to be monitored

```
[monitor-delete p1384]
[make-monitor-data p??]
[*monitor-table* p1382]

— defun monitor-add 0 —

(defun monitor-add (name &optional sourcefile)
  "add a function to be monitored"
  (declare (special *monitor-table*))
  (unless (fboundp name) (load sourcefile))
  (when (gethash name *monitor-table*)
    (monitor-delete name))
  (eval '(trace (,name :cond (progn (monitor-incr ',name) nil))))
  (setf (gethash name *monitor-table*)
    (make-monitor-data
      :name name :count 0 :monitortp t :sourcefile sourcefile)))
```

defun Remove a function being monitored

```
[*monitor-table* p1382]

— defun monitor-delete 0 —

(defun monitor-delete (fn)
  "Remove a function being monitored"
  (declare (special *monitor-table*))
  (eval '(untrace ,fn))
  (remhash fn *monitor-table*))
```

defun Enable all (or optionally one) function for monitoring

```
[*monitor-table* p1382]

— defun monitor-enable 0 —

(defun monitor-enable (&optional fn)
  "enable all (or optionally one) function for monitoring"
  (declare (special *monitor-table*))
  (if fn
```

```
(progn
  (eval '(trace (,fn :cond (progn (monitor-incr ',fn) nil))))
  (setf (monitor-data-monitorp (gethash fn *monitor-table*)) t))
(maphash
 #'(lambda (key value)
   (declare (ignore value))
   (eval '(trace (,key :cond (progn (monitor-incr ',key) nil))))
   (setf (monitor-data-monitorp (gethash key *monitor-table*)) t))
 *monitor-table*))
```

defun Disable all (optionally one) function for monitoring

[*monitor-table* p1382]

— defun monitor-disable 0 —

```
(defun monitor-disable (&optional fn)
  "disable all (optionally one) function for monitoring"
  (declare (special *monitor-table*))
  (if fn
    (progn
      (eval '(untrace ,fn))
      (setf (monitor-data-monitorp (gethash fn *monitor-table*)) nil))
    (maphash
     #'(lambda (key value)
       (declare (ignore value))
       (eval '(untrace ,key))
       (setf (monitor-data-monitorp (gethash key *monitor-table*)) nil))
     *monitor-table*)))
```

defun Reset the table count for the table (or a function)

[*monitor-table* p1382]

— defun monitor-reset 0 —

```
(defun monitor-reset (&optional fn)
  "reset the table count for the table (or a function)"
  (declare (special *monitor-table*))
  (if fn
    (setf (monitor-data-count (gethash fn *monitor-table*)) 0)
```



```
(maphash
  #'(lambda (key value)
    (declare (ignore value))
    (setf (monitor-data-count (gethash key *monitor-table*)) 0))
  *monitor-table*))
```

defun Incr the count of fn by 1

[*monitor-table* p1382]

— defun monitor-incr 0 —

```
(defun monitor-incr (fn)
  "incr the count of fn by 1"
  (let (data)
    (declare (special *monitor-table*))
    (setq data (gethash fn *monitor-table*))
    (if data
      (incf (monitor-data-count data)) ;; change table entry by side-effect
      (warn "~s is monitored but not in table..do (untrace ~s)~%" fn fn))))
```

defun Decr the count of fn by 1

[*monitor-table* p1382]

— defun monitor-decr 0 —

```
(defun monitor-decr (fn)
  "decr the count of fn by 1"
  (let (data)
    (declare (special *monitor-table*))
    (setq data (gethash fn *monitor-table*))
    (if data
      (decf (monitor-data-count data)) ;; change table entry by side-effect
      (warn "~s is monitored but not in table..do (untrace ~s)~%" fn fn))))
```

defun Return the monitor information for a function

[*monitor-table* p1382]

— defun monitor-info 0 —

```
(defun monitor-info (fn)
  "return the monitor information for a function"
  (declare (special *monitor-table*))
  (gethash fn *monitor-table*))
```

—————

defun Hang a monitor call on all of the defuns in a file

[done p??]
[done p??]
[monitor-add p1384]

— defun monitor-file 0 —

```
(defun monitor-file (file)
  "hang a monitor call on all of the defuns in a file"
  (let (expr (package "BOOT"))
    (format t "monitoring ~s~%" file)
    (with-open-file (in file)
      (catch 'done
        (loop
          (setq expr (read in nil 'done))
          (when (eq expr 'done) (throw 'done nil))
          (if (and (consp expr) (eq (car expr) 'in-package))
              (if (and (consp (second expr)) (eq (first (second expr)) 'quote))
                  (setq package (string (second (second expr))))
                  (setq package (second expr)))
              (when (and (consp expr) (eq (car expr) 'defun))
                (monitor-add (intern (string (second expr)) package) file))))))))))
```

—————

defun Return a list of the functions with zero count fields

[*monitor-table* p1382]

— defun monitor-untested 0 —

```
(defun monitor-untested ()
  "return a list of the functions with zero count fields"
  (let (result)
    (declare (special *monitor-table*))
    (maphash
      #'(lambda (key value)
        (if (and (monitor-data-monitorp value) (= (monitor-data-count value) 0))
            (push key result)))
      *monitor-table*)
    (sort result #'string-lessp)))
```

defun Return a list of functions with non-zero counts

[monitor-delete p1384]
 [*monitor-table* p1382]

— defun monitor-tested 0 —

```
(defun monitor-tested (&optional delete)
  "return a list of functions with non-zero counts, optionally deleting them"
  (let (result)
    (declare (special *monitor-table*))
    (maphash
      #'(lambda (key value)
        (when (and (monitor-data-monitorp value)
                   (> (monitor-data-count value) 0))
          (when delete (monitor-delete key))
          (push key result)))
      *monitor-table*)
    (sort result #'string-lessp)))
```

defun Write out a list of symbols or structures to a file

— defun monitor-write 0 —

```
(defun monitor-write (items file)
  "write out a list of symbols or structures to a file"
  (with-open-file (out file :direction :output)
    (dolist (item items)
      (if (symbolp item)
```

```
(format out "~s~%" item)
(format out "~s~50t~s~100t~s~%"
  (monitor-data-sourcefile item)
  (monitor-data-name item)
  (monitor-data-count item))))))
```

defun Save the **monitor-table** in loadable form

```
[*monitor-table* p1382]
[*print-package* p??]
```

— defun monitor-checkpoint 0 —

```
(defun monitor-checkpoint (file)
  "save the *monitor-table* in loadable form"
  (let ((*print-package* t))
    (declare (special *print-package* *monitor-table*))
    (with-open-file (out file :direction :output)
      (format out "(in-package \"BOOT\")~%" )
      (format out "(monitor-inittable)~%" )
      (dolist (data (monitor-results))
        (format out "(monitor-add '~s ~s)~%"
          (monitor-data-name data)
          (monitor-data-sourcefile data))
        (format out "(setf (gethash '~s *monitor-table*)
          (make-monitor-data :name '~s :count ~s :monitorp ~s
            :sourcefile ~s))~%"
          (monitor-data-name data)
          (monitor-data-name data)
          (monitor-data-count data)
          (monitor-data-monitorp data)
          (monitor-data-sourcefile data))))))
```

defun restore a checkpointed file

— defun monitor-restore 0 —

```
(defun monitor-restore (file)
  "restore a checkpointed file"
  (load file))
```

defun Printing help documentation

— defun monitor-help 0 —

```
(defun monitor-help ()
  (format t "%~%
;;; MONITOR
;;;
;;; This file contains a set of function for monitoring the execution
;;; of the functions in a file. It constructs a hash table that contains
;;; the function name as the key and monitor-data structures as the value
;;;
;;; The technique is to use a :cond parameter on trace to call the
;;; monitor-incr function to incr the count every time a function is called
;;;
;;; *monitor-table*                                HASH TABLE
;;;   is the monitor table containing the hash entries
;;; *monitor-nrlibs*                                LIST of STRING
;;;   list of nrllib filenames that are monitored
;;; *monitor-domains*                                LIST of STRING
;;;   list of domains to monitor-report (default is all exposed domains)
;;; monitor-data                                    STRUCTURE
;;;   is the defstruct name of records in the table
;;;   name is the first field and is the name of the monitored function
;;;   count contains a count of times the function was called
;;;   monitorp is a flag that skips counting if nil, counts otherwise
;;;   sourcefile is the name of the file that contains the source code
;;;
;;; ***** SETUP, SHUTDOWN *****
;;;
;;; monitor-inittable ()                            FUNCTION
;;;   creates the hashtable and sets *monitor-table*
;;;   note that it is called every time this file is loaded
;;; monitor-end ()                                  FUNCTION
;;;   unhooks all of the trace hooks
;;;
;;; ***** TRACE, UNTRACE *****
;;;
;;; monitor-add (name &optional sourcefile)          FUNCTION
;;;   sets up the trace and adds the function to the table
;;; monitor-delete (fn)                              FUNCTION
;;;   untraces a function and removes it from the table
;;; monitor-enable (&optional fn)                    FUNCTION
;;;   starts tracing for all (or optionally one) functions that
;;;   are in the table
;;; monitor-disable (&optional fn)                    FUNCTION
```

```

;;; stops tracing for all (or optionally one) functions that
;;; are in the table
;;;
;;; ***** COUNTING, RECORDING *****
;;;
;;; monitor-reset (&optional fn)                FUNCTION
;;; reset the table count for the table (or optionally, for a function)
;;; monitor-incr (fn)                            FUNCTION
;;; increments the count information for a function
;;; it is called by trace to increment the count
;;; monitor-decr (fn)                            FUNCTION
;;; decrements the count information for a function
;;; monitor-info (fn)                            FUNCTION
;;; returns the monitor-data structure for a function
;;;
;;; ***** FILE IO *****
;;;
;;; monitor-write (items file)                   FUNCTION
;;; writes a list of symbols or structures to a file
;;; monitor-file (file)                         FUNCTION
;;; will read a file, scan for defuns, monitor each defun
;;; NOTE: monitor-file assumes that the file has been loaded
;;;
;;; ***** RESULTS *****
;;;
;;; monitor-results ()                           FUNCTION
;;; returns a list of the monitor-data structures
;;; monitor-untested ()                         FUNCTION
;;; returns a list of files that have zero counts
;;; monitor-tested (&optional delete)          FUNCTION
;;; returns a list of files that have nonzero counts
;;; optionally calling monitor-delete on those functions
;;;
;;; ***** CHECKPOINT/RESTORE *****
;;;
;;; monitor-checkpoint (file)                   FUNCTION
;;; save the *monitor-table* in a loadable form
;;; monitor-restore (file)                     FUNCTION
;;; restore a checkpointed file so that everything is monitored
;;;
;;; ***** ALGEBRA *****
;;;
;;; monitor-autoload ()                         FUNCTION
;;; traces autoload of algebra to monitor corresponding source files
;;; NOTE: this requires the /spad/int/algebra directory
;;; monitor-dirname (args)                     FUNCTION
;;; expects a list of 1 libstream (loadvol's arglist) and monitors the source
;;; this is a function called by monitor-autoload
;;; monitor-nrllib (nrllib)                   FUNCTION
;;; takes an nrllib name as a string (eg POLY) and returns a list of

```

```

;;; monitor-data structures from that source file
;;; monitor-report () FUNCTION
;;; generate a report of the monitored activity for domains in
;;; *monitor-domains*
;;; monitor-spadfile (name) FUNCTION
;;; given a spad file, report all nrlibs it creates
;;; this adds each nrlib name to *monitor-domains* but does not
;;; trace the functions from those domains
;;; monitor-percent () FUNCTION
;;; ratio of (functions executed)/(functions traced)
;;; monitor-apropos (str) FUNCTION
;;; given a string, find all monitored symbols containing the string
;;; the search is case-insensitive. returns a list of monitor-data items
") nil)

```

Monitoring algebra files

defun Monitoring algebra code.lsp files

[*monitor-nrlibs* p1381]

— defun monitor-dirname 0 —

```

(defun monitor-dirname (args)
  "expects a list of 1 libstream (loadvol's arglist) and monitors the source"
  (let (name)
    (declare (special *monitor-nrlibs*))
    (setq name (libstream-dirname (car args)))
    (setq name (file-namestring name))
    (setq name (concatenate 'string "/spad/int/algebra/" name "/code.lsp"))
    (when (probe-file name)
      (push name *monitor-nrlibs*)
      (monitor-file name))))

```

defun Monitor autoloaded files

— defun monitor-autoload 0 —

```

(defun monitor-autoload ()

```

```
"traces autoload of algebra to monitor corresponding source files"
(trace (vmlisp::loadvol
      :entrycond nil
      :exitcond (progn (monitor-dirname system::arglist) nil))))
```

defun Monitor an nrlib

[*monitor-table* p1382]

— defun monitor-nrlib 0 —

```
(defun monitor-nrlib (nrlib)
  "takes an nrlib name as a string (eg POLY) and returns a list of
  monitor-data structures from that source file"
  (let (result)
    (declare (special *monitor-table*))
    (maphash
      #'(lambda (k v)
          (declare (ignore k))
          (when (string= nrlib
                        (pathname-name (car (last
                                           (pathname-directory (monitor-data-sourcefile v))))))
              (push v result)))
      *monitor-table*)
    result))
```

defun Given a monitor-data item, extract the nrlib name

— defun monitor-libname 0 —

```
(defun monitor-libname (item)
  "given a monitor-data item, extract the nrlib name"
  (pathname-name (car (last
                       (pathname-directory (monitor-data-sourcefile item))))))
```

defun Is this an exposed algebra function?

— defun monitor-exposedp 0 —

```
(defun monitor-exposedp (fn)
  "exposed functions have more than 1 semicolon. given a symbol, count them"
  (> (count #\; (symbol-name fn)) 1))
```

—————

defun Monitor exposed domains

TPDHERE: note that the file `interp.exposed` no longer exists. The exposure information is now in this book. This needs to work off the internal exposure list, not the file.

```
[done p??]
[done p??]
[*monitor-domains* p1381]
```

— defun monitor-readinterp 0 —

```
(defun monitor-readinterp ()
  "read interp.exposed to initialize *monitor-domains* to exposed domains.
  this is the default action. adding or deleting domains from the list
  will change the report results"
  (let (skip expr name)
    (declare (special *monitor-domains*))
    (setq *monitor-domains* nil)
    (with-open-file (in "/spad/src/algebra/interp.exposed")
      (read-line in)
      (read-line in)
      (read-line in)
      (read-line in)
      (catch 'done
        (loop
          (setq expr (read-line in nil "done"))
          (when (string= expr "done") (throw 'done nil))
          (cond
            ((string= expr "basic") (setq skip nil))
            ((string= expr "categories") (setq skip t))
            ((string= expr "hidden") (setq skip t))
            ((string= expr "defaults") (setq skip nil)))
          (when (and (not skip) (> (length expr) 58))
            (setq name (subseq expr 58 (length expr)))
            (setq name (string-right-trim '(#\space) name)))
```

```
(when (> (length name) 0)
  (push name *monitor-domains*)))))))))
```

defun Generate a report of the monitored domains

```
[monitor-readinterp p1394]
[*monitor-domains* p1381]
```

— defun monitor-report 0 —

```
(defun monitor-report ()
  "generate a report of the monitored activity for domains in *monitor-domains*"
  (let (nrlibs nonzero total)
    (declare (special *monitor-domains*))
    (unless *monitor-domains* (monitor-readinterp))
    (setq nonzero 0)
    (setq total 0)
    (maphash
     #'(lambda (k v)
         (declare (ignore k))
         (let (nextlib point)
           (when (> (monitor-data-count v) 0) (incf nonzero))
           (incf total)
           (setq nextlib (monitor-libname v))
           (setq point (member nextlib nrlibs :test #'string= :key #'car))
           (if point
               (setf (cdr (first point)) (cons v (cdr (first point))))
               (push (cons nextlib (list v)) nrlibs))))
      *monitor-table*)
    (format t "~d of ~d (~d percent) tested~%" nonzero total
            (round (/ (* 100.0 nonzero) total)))
    (setq nrlibs (sort nrlibs #'string< :key #'car))
    (dolist (pair nrlibs)
      (let ((exposedcount 0) (testcount 0))
        (when (member (car pair) *monitor-domains* :test #'string=)
          (format t "for library ~s~%" (car pair))
          (dolist (item (sort (cdr pair) #'> :key #'monitor-data-count))
            (when (monitor-exposedp (monitor-data-name item))
              (incf exposedcount)
              (when (> (monitor-data-count item) 0) (incf testcount))
              (format t "~5d ~s~%"
                      (monitor-data-count item)
                      (monitor-data-name item))))
          (if (= exposedcount testcount)
              (format t "~a has all exposed functions tested~%" (car pair))
```

```
(format t "Daly bug:~a has untested exposed functions~%" (car pair))))))
nil))
```

defun Parse an)abbrev expression for the domain name

— defun monitor-parse 0 —

```
(defun monitor-parse (expr)
  (let (point1 point2)
    (setq point1 (position #\space expr :test #'char=))
    (setq point1 (position #\space expr :start point1 :test-not #'char=))
    (setq point1 (position #\space expr :start point1 :test #'char=))
    (setq point1 (position #\space expr :start point1 :test-not #'char=))
    (setq point2 (position #\space expr :start point1 :test #'char=))
    (subseq expr point1 point2)))
```

defun Given a spad file, report all nrlibs it creates

```
[done p??]
[done p??]
[monitor-parse p1396]
[*monitor-domains* p1381]
```

— defun monitor-spadfile 0 —

```
(defun monitor-spadfile (name)
  "given a spad file, report all nrlibs it creates"
  (let (expr)
    (declare (special *monitor-domains*))
    (with-open-file (in name)
      (catch 'done
        (loop
          (setq expr (read-line in nil 'done))
          (when (eq expr 'done) (throw 'done nil))
          (when (and (> (length expr) 4) (string= (subseq expr 0 4) ")abb"))
            (setq *monitor-domains*
                  (adjoin (monitor-parse expr) *monitor-domains* :test #'string=)))))))
```

defun Print percent of functions tested

[*monitor-table* p1382]

— defun monitor-percent 0 —

```
(defun monitor-percent ()
  "Print percent of functions tested"
  (let (nonzero total)
    (declare (special *monitor-table*))
    (setq nonzero 0)
    (setq total 0)
    (maphash
     #'(lambda (k v)
         (declare (ignore k))
         (when (> (monitor-data-count v) 0) (incf nonzero))
         (incf total)))
     *monitor-table*)
    (format t "~d of ~d (~d percent) tested~%" nonzero total
            (round (/ (* 100.0 nonzero) total)))))
```

defun Find all monitored symbols containing the string

[*monitor-table* p1382]

— defun monitor-afropos 0 —

```
(defun monitor-afropos (str)
  "given a string, find all monitored symbols containing the string
   the search is case-insensitive. returns a list of monitor-data items"
  (let (result)
    (maphash
     #'(lambda (k v)
         (when
          (search (string-upcase str)
                  (string-upcase (symbol-name k))
                  :test #'string=)
          (push v result))))
     *monitor-table*)
    result))
```

Chapter 44

HyperDoc

Hyperdoc works by building up a page “description” which consists of a list of items. Each item is itself a list whose first element is a tag. The `htMakePage1` routine walks the list of items, dispatching on the tag (called `itemType`), to create the final page.

Pages are constructed with a latex-like syntax. The valid syntax values are in the `primitive-HtCommands` list.

Each page is an 8 part list, of which the description is the last item. See `HTPage Layout` for more details.

Pages can also be constructed by code. For example, the page found by

```
Basic Commands -> Matrix
```

is constructed by a call to `bcReadMatrix`. This routine sets the page title `htInitPage`, sets up the EXIT button handler `htpSetProperty`, constructs a page descriptions which is passes to `htMakePage`, and then calls `htShowPage` to display the result.

44.1 Hyperdoc macro handling and `util.ht`

All of the macros used in hyperdoc are in this hash table. User-defined macros are read from the file `doc/util.ht`

```
defvar $htMacroTable
```

```
— initvars —
```

```
(defvar |$htMacroTable| (make-hash-table :test #'equal))
```

These are the primitive hyperdoc commands. They are directly implemented. The **build-HtMacroTable** function adds these to the **\$HtMacroTable** at startup.

defvar \$primitiveHtCommands

— initvars —

```
(defvar |$primitiveHtCommands|
  '((("\\ContinueButton" . 1)
    ("\\andexample" . 1)
    ("\\autobutt" . 0)
    ("\\autobuttons" . 0)
    ("\\begin" . 1)
    ("\\beginscroll" . 0)
    ("\\bound" . 1)
    ("\\fbox" . 1)
    ("\\centerline" . 1)
    ("\\downlink" . 2)
    ("\\em" . 0)
    ("\\end" . 1)
    ("\\endscroll" . 0)
    ("\\example" . 1)
    ("\\free" . 1)
    ("\\graphpaste" . 1)
    ("\\helppage" . 1)
    ("\\htbmdir" . 0)
    ("\\htbmfile" . 1)
    ("\\indent" . 1)
    ("\\inputbitmap" . 1)
    ("\\inputstring" . 3)
    ("\\item" . 0)
    ("\\keyword" . 1)
    ("\\link" . 2)
    ("\\lispdownlink" . 2)
    ("\\lispmemolink" . 2)
    ("\\lispwindowlink" . 2)
    ("\\menudownlink" . 2)
    ("\\menuitemstyle" . 1)
    ("\\menulink" . 2)
    ("\\menulispdownlink" . 2)
    ("\\menulispmemolink" . 2)
    ("\\menulispwindowlink" . 2)
    ("\\menumemolink" . 2)
    ("\\menuwindowlink" . 2)
    ("\\newline" . 0)
    ("\\radioboxes" . 3))
```

```
( "\\space" . 1)
( "\\spadcommand" . 1)
( "\\stringvalue" . 1)
( "\\tab" . 1)
( "\\table" . 1)
( "\\vspace" . 1)
( "\\windowlink" . 2)))
```

defvar \$newPage

— initvars —

```
(defvar |$newPage| nil "The new page being built")
```

defun Build the table of hyperdoc macros

Hash user-defined macros from **doc/util.ht** into **htMacroTable**. Hash primitive hyperdoc macros into **htMacroTable**. [buildHtMacroTable util.ht (vol7.1)]

```
[getHtMacroItem p1402]
[sayBrightly p??]
[concat p1237]
[getenvirom p322]
[hput p1234]
[$htMacroTable p1399]
[$primitiveHtCommands p1400]
```

— defun buildHtMacroTable —

```
(defun |buildHtMacroTable| ()
  (let (fn)
    (declare (special |$htMacroTable| |$primitiveHtCommands|))
    (setq fn (concat (getenvirom "AXIOM") "/doc/util.ht"))
    (cond
      ((probe-file fn)
        (with-open-file (instream fn)
          (loop
            for line = (read-line instream nil :eof)
            until (eq line :eof)
            do
```



```

      (when
        (multiple-value-bind (command numOfArgs) (|getHtMacroItem| line)
          (hput |$htMacroTable| command numOfArgs))))
    (dolist (pair |$primitiveHtCommands|)
      (hput |$htMacroTable| (car pair) (cdr pair))))
    (t (|sayBrightly| "Warning: macro table not found")))
  |$htMacroTable|))

```

defun Get new command name and number of args

This processes **newcommand** lines read from **doc/util.ht**. An example newcommand looks like

```
\newcommand{\menulink}[2]          {\menudownlink{#1}{#2}}
```

This function returns a pair whose CAR is the new command name and whose CDR is the number of arguments. If there are zero arguments the brackets and number will not appear.

However brackets can appear in the new command so we need to fix the original code to handle this new case. We set up a wall starting after the first closing brace.

```
\newcommand{\beginmenu}           {\beginitems[\MenuDotBitmap]}
```

```
[getHtMacroItem util.ht (vol7.1)]
```

getHtMacroItem : String → Values (String NonNegativeInteger)
 — defun getHtMacroItem —

```

(defun |getHtMacroItem| (line)
  (let (k command m i j wall digitString)
    (when (|stringPrefix?| "\\newcommand{" line)
      (setq k (position #\} line :start 11))
      (setq wall (position #\{ line :start k)) ; wall off the body of command
      (setq command (substring line 12 (- k 12)))
      (setq m (length line))
      (setq i (position #\[ line :start k))
      (if (and i (< i m) wall (< i wall))
        (progn
          ; brackets. parse number of args
          (setq j (position #\] line :start (+ i 1)))
          (setq digitString (substring line (+ i 1) (- (- j i) 1)))
          (when (every #'digitp digitString)
            (values command (parse-integer digitString))))
        (values command 0)))) ; no brackets

```

We populate the htMacroTable at load time.

— **postvars** —

```
(eval-when (eval load)
  (|buildHtMacroTable|))
```

defun Is the first string a prefix of the second?

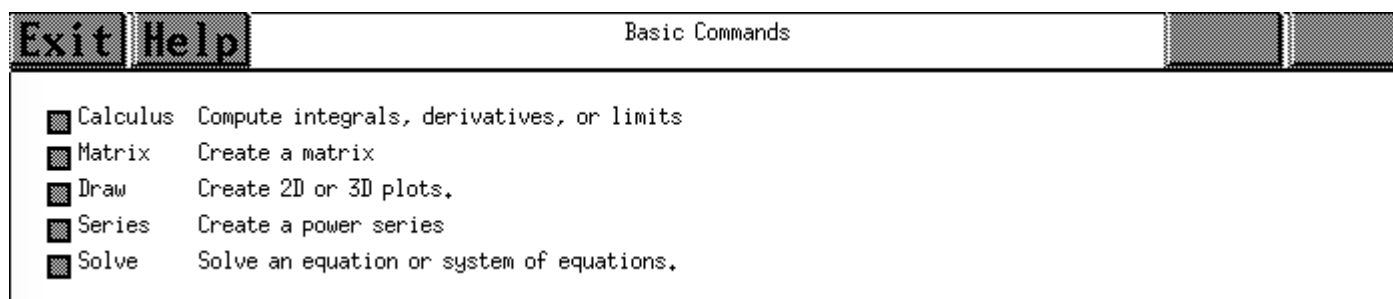
— **defun** stringPrefix? 0 —

```
(defun |stringPrefix?| (pref str)
  (let (lp)
    (cond
      ((null (and (stringp pref) (stringp str))) nil)
      ((eql (setq lp (length pref)) 0) t)
      ((> lp (length str)) nil)
      (t (every #'char= pref str)))))
```

Chapter 45

HyperDoc Basic Command support

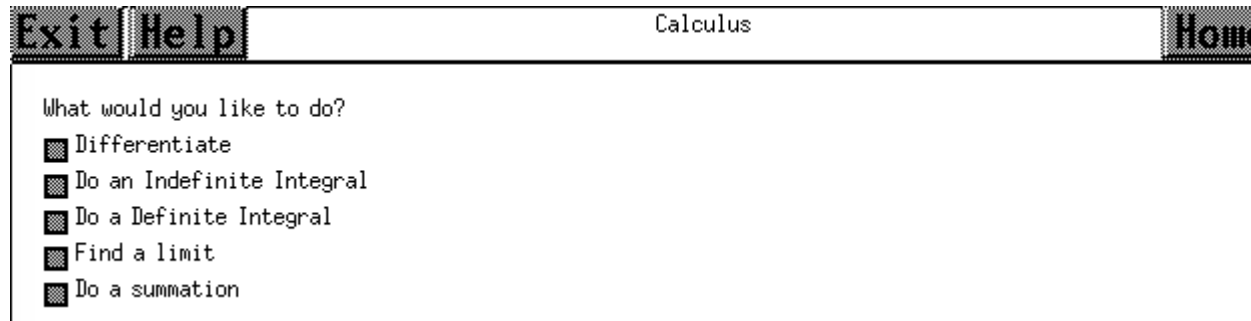
Basic Command



This is the root page of the basic commands dialog. The goal is to present examples of how to construct command lines which demonstrate using Axiom to solve problems.

45.1 Calculus

Calculus



defun Calculus - Differentiate

```
[htInitPage p1510]
[htMakePage p1513]
[htMakeDoneButton p1533]
[htShowPage p1512]
[$EmptyMode p708]
```

Calculus → Differentiate

☐ Enter the function you want to differentiate:

☐ List the variables you want to differentiate with respect to?

☐ List the number of times you want to differentiate with respect to each variable (leave blank if once for each)

Continue

Pressing the **Continue** button will call the function `bcDifferentiateGen` due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcDifferentiateGen|)
```

— defun `bcDifferentiate` —

```
(defun |bcDifferentiate| ()
```

```

(declare (special |$EmptyModel|))
(|htInitPage| '|Differentiate Basic Command| nil)
(|htMakePage|
  '((|domainConditions| (|isDomain| EM |$EmptyModel|)
    (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|))))
  (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| . "Enter the {\\em function} you want to differentiate:")
  (|text| . "\\newline\\tab{2} ")
  (|bcStrings| (55 "sin(x*y)" |diffand| EM))
  (|text| . "\\blankline") (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| .
    "\\newline List the {\\em variables} you want to differentiate with respect to?")
    (|text| . "\\newline\\tab{2} ")
    (|bcStrings| (55 "x y" |variables| S . |quoteString|))
    (|text| . "\\blankline") (|text| . "\\newline ")
    (|text| . "\\menuitemstyle{\\tab{2}}")
    (|text| .
      "\\newline List the number of {\\em times} you want to differentiate with respect to each variable (leave blank)?")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (55 "1 2" |times| S . |quoteString|))))
  (|htMakeDoneButton| "Continue" '|bcDifferentiateGen|)
  (|htShowPage|))

```

Pressing the **Continue** calls `bcDifferentiateGen`

defun bcDifferentiateGen

```

[htLabelInputString p1503]
[bcString2WordList p1495]
[bcwords2liststring p1495]
[length p??]
[bcError p1496]
[concat p1237]
[bcGen p1494]

```

— defun bcDifferentiateGen —

```

(defun |bcDifferentiateGen| (htPage)
  (let (mand varlist indexList varpart indexpart lastPart)
    (setq mand (|htLabelInputString| htPage '|diffand|))
    (setq varlist
      (|bcString2WordList| (|htLabelInputString| htPage '|variables|)))
    (setq indexList

```

```

(|bcString2WordList| (|httpLabelInputString| htPage '|times|)))
(setq varpart
  (if (> (|#| varlist) 1)
    (|bcwords2liststring| varlist)
    (car varlist)))
(setq indexpart
  (cond
    ((null indexList) nil)
    ((null (cdr indexList)) (car indexList))
    ((= (|#| indexList) (|#| varlist)) (|bcwords2liststring| indexList))
    (t (|bcError|
      "You must say how many times you want to differentiate with respect to each variable---or leave
      (setq lastPart (if indexpart (concat "," indexpart ")") ")")
      (|bcGen| (concat "differentiate(" mand "," varpart lastPart)))))

```

defun Calculus - Do an Indefinite Integral

```

[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]

```

Calculus → Indefinite Integral

Enter the function you would like to integrate:

1/(x**2 + 6)_

Enter the variable of integration:

x

Continue

Pressing the **Continue** button will call the function **bcIndefiniteIntegrateGen** due to this line:

```
(|doneButton| "Continue" |bcIndefiniteIntegrateGen|))
```

— defun bcIndefiniteIntegrate —

```
(defun |bcIndefiniteIntegrate| ()
```

```
(declare (special |$EmptyMode|))
(|htInitPage| '|Indefinite Integration Basic Command| nil)
(|htMakePage|
  '((|domainConditions| (|isDomain| EM |$EmptyMode|)
    (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|))))
  (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| . "Enter the {\\em function} you would like to integrate:")
  (|text| . "\\newline\\tab{2} ")
  (|bcStrings| (45 "1/(x**2 + 6)" |integrand| EM))
  (|text| . "\\blankline") (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| . "Enter the {\\em variable of integration}:")
  (|text| . "\\tab{37}") (|bcStrings| (10 |x| |symbol| SY))
  (|doneButton| "Continue" |bcIndefiniteIntegrateGen|)))
(|htShowPage|))
```

defun bcIndefiniteIntegrateGen

[[htLabelInputString](#) p1503]
 [concat p1237]
 [bcGen p1494]

— defun bcIndefiniteIntegrateGen —

```
(defun |bcIndefiniteIntegrateGen| (htPage)
  (let (integrand var)
    (setq integrand (|htLabelInputString| htPage '|integrand|))
    (setq var (|htLabelInputString| htPage '|symbol|))
    (|bcGen| (concat "integrate(" integrand "," var ")"))))
```

defun Calculus - Do a Definite Integral

[[htInitPage](#) p1510]
 [htMakePage p1513]
 [htShowPage p1512]
 [\$EmptyMode p708]

Calculus → Definite Integral

☐ Enter the function you would like to integrate:

☐ Enter the variable of integration:

☐ Enter lower limit:
☒ Minus infinity
☐ A finite point:

☐ Enter upper limit:
☒ Plus infinity
☐ A finite point:

Continue

Pressing the **Continue** button will call the function `bcDefiniteIntegrateGen` due to this line:

```
(|doneButton| "Continue" |bcDefiniteIntegrateGen|)
```

— defun `bcDefiniteIntegrate` —

```
(defun |bcDefiniteIntegrate| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| ' |Definite Integration Basic Command| NIL)
  (|htMakePage|
    ' ((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em function} you would like to integrate:")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (45 "1/(x**2 + 6)" |integrand| EM))
      (|text| . "\\blankline") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em variable of integration}:")
      (|text| . "\\tab{37}") (|bcStrings| (10 |x| |symbol| SY))
      (|text| . "\\blankline") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "\\newline Enter {\\em lower limit}:")
      (|radioButtons| |fromButton|
        (" " "Minus infinity" |minusInfinity|)
        (" "
```

```

      ((|text| . "A finite point:\\tab{15}")
       (|bcStrings| (10 0 |from| EM . |bcOptional|)))
      |fromPoint|))
(|text| . "\\blankline") (|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "\\indent{2}\\newline Enter {\\em upper limit}:")
(|radioButtons| |toButton|
 (" " "Plus infinity" |plusInfinity|)
 (" "
  ((|text| "A finite point:\\tab{15}")
   (|bcStrings| (10 |y| |to| EM . |bcOptional|)))
   |toPoint|))
(|doneButton| "Continue" |bcDefiniteIntegrateGen|)))
(|htShowPage|))

```

defun bcDefiniteIntegrateGen

```

[|htLabelInputString| p1503]
[|htButtonValue| p1501]
[|concat| p1237]
[|bcGen| p1494]

```

— defun bcDefiniteIntegrateGen —

```

(defun |bcDefiniteIntegrateGen| (htPage)
  (let (integrand var lowerLimit upperLimit varpart)
    (setq integrand (|htLabelInputString| htPage '|integrand|))
    (setq var (|htLabelInputString| htPage '|symbol|))
    (setq lowerLimit
      (if (eq (|htButtonValue| htPage '|fromButton|) '|fromPoint|)
          (|htLabelInputString| htPage '|from|)
          "%minusInfinity"))
    (setq upperLimit
      (if (eq (|htButtonValue| htPage '|toButton|) '|toPoint|)
          (|htLabelInputString| htPage '|to|)
          "%plusInfinity"))
    (setq varpart (concat var " = " lowerLimit ".." upperLimit))
    (|bcGen| (concat "integrate(" integrand "," varpart ")"))))

```

defun Calculus - Find a limit

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Calculus $\rightarrow$ Limit

What kind of limit do you want to compute?

■ A real limit? The limit as the variable approaches a real value along the real axis

■ A complex limit? The limit as the variable approaches a complex value along any path in the complex plane

— defun bcLimit —

```
(defun |bcLimit| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Limit Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| . "What kind of limit do you want to compute? ")
      (|text| . "\\blankline ")
      (|text| . "\\beginmenu")
      (|text| . "\\item ")
      (|bcLinks| ("\\menuitemstyle{A real limit?}" "" |bcRealLimit| |real|))
      (|text| . "\\indentrel{17}\\tab{0}")
      (|text| .
        "The limit as the variable approaches a {\\em real} value along the real axis")
      (|text| . "\\indentrel{-17}") (|text| . "\\item ")
      (|text| . "\\blankline ")
      (|bcLinks|
        ("\\menuitemstyle{A complex limit?}" "" |bcComplexLimit| |complex|))
      (|text| . "\\indentrel{17}\\tab{0}")
      (|text| .
        "The limit as the variable approaches a {\\em complex} value along any path in the complex plane")
      (|text| . "\\indentrel{-17}")
      (|text| . "\\endmenu"))))
  (|htShowPage|))
```

defun Calculus - Do a summation

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Calculus → Summation

Enter the function you would like to sum:

i^{**3}

Enter the summation index:

i

Enter the limits of the sum:

From: 1 To: n

Continue

Pressing the **Continue** button will call the function **bcSumGen** due to this line:

```
(|doneButton| "Continue" |bcSumGen|)
```

— defun bcSum —

```
(defun |bcSum| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| '|Sum Basic Command| NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em function} you would like to sum:")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (44 "i**3" |summand| EM))
      (|text| . "\\blankline ") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em summation index}:")
      (|text| . "\\tab{36}") (|bcStrings| (10 |i| |index| SY))
      (|text| . "\\blankline ") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the limits of the sum:"))
```

```
(|text| . "\\newline\\tab{10}{\\em From:}")
(|bcStrings| (10 1 |first| S))
(|text| . "\\tab{32}{\\em To:}") (|text| . "\\tab{36}")
(|bcStrings| (10 |n| |last| S))
(|doneButton| "Continue" |bcSumGen|)))
(|htShowPage|))
```

defun bcSumGen

```
[htpLabelInputString p1503]
[concat p1237]
[bcGen p1494]
```

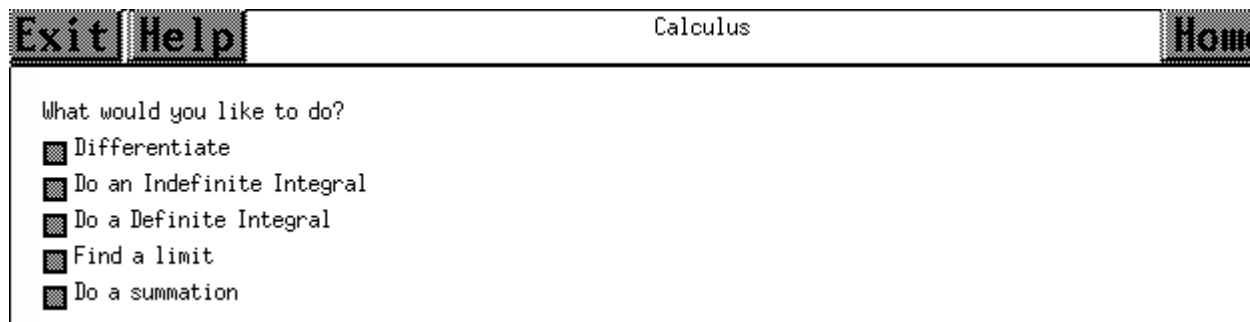
— defun bcSumGen —

```
(defun |bcSumGen| (htPage)
  (let (mand index car last)
    (setq mand (|htpLabelInputString| htPage '|summand|))
    (setq index (|htpLabelInputString| htPage '|index|))
    (setq car (|htpLabelInputString| htPage '|first|))
    (setq last (|htpLabelInputString| htPage '|last|))
    (|bcGen| (concat "sum(" mand "," index " = " car " .." last ")"))))
```

45.2 Matrix

defun Basic Commands - Matrix

Matrix



[bcReadMatrix p1475]

— defun bcMatrix —

```
(defun |bcMatrix| () (|bcReadMatrix| nil))
```

—————

45.3 Draw

defun Basic Commands - Draw

[htInitPage p1510]

[bcHt p1508]

[htShowPage p1512]

Draw Basic Command

What would you like to draw?

Two Dimensional Plots

A function of one variable $y = f(x)$

A parametrically defined curve $(x(t), y(t))$

A solution to a polynomial equation $p(x,y) = 0$

Three Dimensional Surfaces

A function of two variables $z = f(x,y)$

A parametrically defined tube $(x(t), y(t), z(t))$

A parameterically defined surface $(x(u,v), y(u,v), z(u,v))$

— defun bcDraw —

```
(defun |bcDraw| ()
  (|htInitPage| "Draw Basic Command" NIL)
  (|bcHt| "What would you like to draw?")
  (|bcHt| "\\newline\\centerline{{\\em Two Dimensional Plots}}\\newline")
  (|bcHt| "\\lispdownlink{A function of one variable}{{(|bcDraw2Dfun|)}}")
  (|bcHt| "\\space{2}y = f(x)\\newline")
  (|bcHt| "\\lispdownlink{A parametrically defined curve}{{(|bcDraw2Dpar|)}}")
  (|bcHt| "\\space{2}(x(t), y(t))\\newline")
  (|bcHt|
```

```
"\\lispdownlink{A solution to a polynomial equation}{(|bcDraw2DSolve|)}")
(|bcHt| "\\space{2} p(x,y) = 0\\newline")
(|bcHt| "\\vspace{1}\\newline ")
(|bcHt| "\\centerline{\\em Three Dimensional Surfaces}\\}\\newline\\newline")
(|bcHt| "\\lispdownlink{A function of two variables}{(|bcDraw3Dfun|)}")
(|bcHt| "\\space{2} z = f(x,y)\\newline")
(|bcHt| "\\lispdownlink{A parametrically defined tube}{(|bcDraw3Dpar|)}")
(|bcHt| "\\space{2}(x(t), y(t), z(t))\\newline")
(|bcHt| "\\lispdownlink{A parameterically defined surface}{(|bcDraw3Dpar1|)}")
(|bcHt| "\\space{2}(x(u,v), y(u,v), z(u,v))\\newline")
(|htShowPage|))
```

defun Draw - Function of one variable

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Draw Basic Command by Function

Drawing $y = f(x)$
 where y is the dependent variable and
 where x is the independent variable

What function f would you like to draw?

Enter dependent variable:

Enter independent variable and range:
 Variable: ranges from: to:

Optionally enter a title for your curve:

Continue

Pressing the **Continue** button will call the function `bcDraw2DfunGen` due to this line:

```
(|doneButton| "Continue" |bcDraw2DfunGen|)
```

— defun bcDraw2Dfun —

```
(defun |bcDraw2Dfun| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Draw Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|)))
      (|text| "\\centerline{Drawing {\\em y = f(x)}}\\newline "
        "\\centerline{where {\\em y} is the dependent variable and}\\newline "
        "\\centerline{where {\\em x} is the independent variable}\\vspace{1}\\newline "
        "\\menuitemstyle{}\\tab{2}What {\\em function} f would you like to draw?\\newline\\tab{2}")
        (|bcStrings| (55 "x*cos(x)" |function| EM))
        (|text| .
          "\\vspace{1}\\newline\\menuitemstyle{}\\tab{2}Enter {\\em dependent} variable:")
          (|bcStrings| (6 |y| |dependent| SY))
          (|text| . "\\newline\\vspace{1}\\newline ")
          (|text| .
            "\\menuitemstyle{}\\tab{2}Enter {\\em independent} variable and {\\em range}:\\newline\\tab{2} ")
            (|text| . "{\\em Variable:}") (|bcStrings| (6 |x| |ind| SY))
            (|text| . "ranges {\\em from:}")
            (|bcStrings| (9 0 |from1| F)) (|text| . "{\\em to:}")
            (|bcStrings| (9 30 |to1| F))
            (|text| "\\indent{0}\\vspace{1}\\newline\\menuitemstyle{}\\tab{2} "
              "Optionally enter a {\\em title} for your curve:")
              (|bcStrings| (15 "y = x*cos(x)" |title| S))
              (|text| . "\\indent{0}")
              (|doneButton| "Continue" |bcDraw2DfunGen|) (|text| . "{}"))
          (|htShowPage|))
```

—————

defun bcDraw2DfunGen

```
[|htLabelInputString| p1503]
[|concat| p1237]
[|bcFinish| p1493]
[|bcDrawIt2| p1473]
```

— defun bcDraw2DfunGen —

```
(defun |bcDraw2DfunGen| (htPage)
  (let (fun dep ind from1 to1 title titlePart)
    (setq fun (|htLabelInputString| htPage '|function|))
```



```

(setq dep  (|htLabelInputString| htPage '|dependent|))
(setq ind  (|htLabelInputString| htPage '|ind|))
(setq from1 (|htLabelInputString| htPage '|from1|))
(setq to1   (|htLabelInputString| htPage '|to1|))
(setq title (|htLabelInputString| htPage '|title|))
(cond
  ((not (string-equal title ""))
    (setq titlePart (concat "{}" "title ==\" title "\""))
    (|bcFinish| "draw" fun (|bcDrawIt2| ind from1 to1) titlePart))
  (t
    (|bcFinish| "draw" fun (|bcDrawIt2| ind from1 to1))))))

```

defun Draw - Parametrically defined curve

```

[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]

```

Draw Basic Command by Parameters

Drawing a parametrically defined curve:
 ($f_1(t)$, $f_2(t)$)
 in terms of two functions f_1 and f_2
 and an independent variable t

Enter the two functions:
 Function 1:
 Function 2:

Enter independent variable and range:
 Variable: ranges from: to:

Optionally enter a title for your curve:

Continue

Pressing the **Continue** button will call the function **bcDraw2DparGen** due to this line:

```
(|doneButton| "Continue" |bcDraw2DparGen|)
```

— defun bcDraw2Dpar —

```
(defun |bcDraw2Dpar| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Draw Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|)))
      (|text| "\\centerline{Drawing a parametrically defined curve:}\\newline "
        "\\centerline{{\\em ( f1(t), f2(t) )}}\\newline "
        "\\centerline{in terms of two functions {\\em f1} and {\\em f2}} "
        "\\centerline{and an independent variable {\\em t}}\\vspace{1}\\newline "
        "\\menuitemstyle{\\tab{2}Enter the two {\\em functions:}}")
      (|text| . "\\newline\\tab{2}{\\em Function 1:}")
      (|bcStrings| (44 "-9*sin(4*t/5)" |function1| EM))
      (|text| . "\\newline\\tab{2}{\\em Function 2:}")
      (|bcStrings| (44 "8*sin(t)" |function2| EM))
      (|text| .
        "\\vspace{1}\\newline\\menuitemstyle{\\tab{2}Enter {\\em independent} variable and range:\\newline\\tab{2}
        (|text| . "{\\em Variable:}") (|bcStrings| (6 |t| |ind| SY))
        (|text| . "ranges {\\em from:}")
        (|bcStrings| (9 "-5*\\%pi" |from1| F))
        (|text| . "{\\em to:}") (|bcStrings| (9 "5*\\%pi" |to1| F))
        (|text| "\\vspace{1}\\newline\\menuitemstyle{\\tab{2}"
          "Optionally enter a {\\em title} for your curve:")
        (|bcStrings| (15 "Lissajous" |title| S))
        (|text| . "\\indent{0}")
        (|doneButton| "Continue" |bcDraw2DparGen|)))
  (|htShowPage|))
```

—————

defun bcDraw2DparGen

```
[|htpLabelInputString| p1503]
[|concat| p1237]
[|bcFinish| p1493]
[|bcDrawIt2| p1473]
```

— defun bcDraw2DparGen —

```
(defun |bcDraw2DparGen| (htPage)
  (let (fun1 fun2 ind from1 to1 title curvePart titlePart)
    (setq fun1 (|htpLabelInputString| htPage '|function1|))
```

```

(setq fun2 (|htLabelInputString| htPage '|function2|))
(setq ind (|htLabelInputString| htPage '|ind|))
(setq from1 (|htLabelInputString| htPage '|from1|))
(setq to1 (|htLabelInputString| htPage '|to1|))
(setq title (|htLabelInputString| htPage '|title|))
(setq curvePart (concat "curve(" "{" fun1 "," "{" fun2 ")"))
(cond
  ((not (string-equal title ""))
    (setq titlePart (concat "{" "title ==\" title "\""))
    (|bcFinish| "draw" curvePart (|bcDrawIt2| ind from1 to1) titlePart))
  (t
    (|bcFinish| "draw" curvePart (|bcDrawIt2| ind from1 to1))))))

```

defun Draw - Solution to a polynomial equation

```

[htInitPage p1510]
[htMakePage p1513]
[htMakeDoneButton p1533]
[htShowPage p1512]
[$EmptyMode p708]

```

Draw Basic Command by Equation Solution

Plotting the solution to $p(x,y) = 0$, where p is a polynomial in two variables x and y

Enter the polynomial p:

$y^{**2}+7*x*y-(x^{**3}+16*x)$

Enter the variables:

Variable 1: x ranges from: -15 to: 10

Variable 2: y ranges from: -10 to: 50

Optionally enter a title for your curve:

Continue

Pressing the **Continue** button will call the function `bcDraw2DSolveGen` due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcDraw2DSolveGen|)
```

— defun bcDraw2DSolve —

```
(defun |bcDraw2DSolve| ()
  (declare (special |$EmptyModel|))
  (|htInitPage| "Draw Basic Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyModel|)
      (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|)))
      (|text| "\\centerline{Plotting the solution to {\\em p(x,y) = 0}, where} "
        "\\centerline{{\\em p} is a polynomial in two variables {\\em x} and {\\em y}}"}
        "\\vspace{1}\\newline\\menuitemstyle{\\tab{2}Enter the {\\em polynomial} p:"
          "\\newline\\tab{2}")
        (|bcStrings| (40 "y**2+7*x*y-(x**3+16*x)" |function| EM))
        (|text| .
          "\\vspace{1}\\newline\\menuitemstyle{\\tab{2}Enter the {\\em variables}:")
        (|text| . "\\newline\\tab{2}{\\em Variable 1:} ")
        (|bcStrings| (4 |x| |independent1| SY))
        (|text| . "ranges {\\em from:}")
        (|bcStrings| (9 -15 |from1| F)) (|text| . "{\\em to:}")
        (|bcStrings| (9 10 |to1| F))
        (|text| . "\\newline\\tab{2}{\\em Variable 2:} ")
        (|bcStrings| (4 |y| |independent2| SY))
        (|text| . "ranges {\\em from:}")
        (|bcStrings| (9 -10 |from2| F)) (|text| . "{\\em to:}")
        (|bcStrings| (9 50 |to2| F))
        (|text| "\\indent{0}\\vspace{1}\\newline\\menuitemstyle{\\tab{2} "
          "Optionally enter a {\\em title} for your curve:")
        (|bcStrings| (15 "" |title| S)) (|text| . "\\indent{0}"))))
  (|htMakeDoneButton| "Continue" ' |bcDraw2DSolveGen|)
  (|htShowPage|))
```

—————

defun bcDraw2DSolveGen

```
[htpLabelInputString p1503]
[concat p1237]
[bcFinish p1493]
```

— defun bcDraw2DSolveGen —

```
(defun |bcDraw2DSolveGen| (htPage)
  (let (fun ind1 from1 to1 ind2 from2 to2 title clipPart titlePart)
    (setq fun (|htpLabelInputString| htPage ' |function|))
```

```

(setq ind1 (|htLabelInputString| htPage '|independent1|))
(setq from1 (|htLabelInputString| htPage '|from1|))
(setq to1 (|htLabelInputString| htPage '|to1|))
(setq ind2 (|htLabelInputString| htPage '|independent2|))
(setq from2 (|htLabelInputString| htPage '|from2|))
(setq to2 (|htLabelInputString| htPage '|to2|))
(setq title (|htLabelInputString| htPage '|title|))
(setq clipPart (concat "{" "range==" "{"
                      from1 ".." to1 '|,{'|
                      from2 ".." to2 "|}"))

(cond
  ((not (string-equal title ""))
   (setq titlePart (concat "{" "title ==\" title "\""))
   (|bcFinish| "draw" (concat fun " = 0 ") ind1 ind2 clipPart titlePart))
  (t
   (|bcFinish| "draw" (concat fun " = 0 ") ind1 ind2 clipPart))))

```

defun Draw - Function of two variables

```

[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]

```

Draw Basic Command by 3D function

Drawing $z = f(x,y)$
 where z is the dependent variable and
 where x, y are the independent variables

What function f which you like to draw?
`exp(cos(x-y)-sin(x*y))-2`

Enter dependent variable: `z`

Enter independent variables and ranges:
 Variable: `x` ranges from: `-5` to: `5`
 Variable: `y` ranges from: `-5` to: `5`

Optionally enter a title for your surface:

Continue

Pressing the **Continue** button will call the function `bcDraw3DfunGen` due to this line:

```
(|doneButton| "Continue" |bcDraw3DfunGen|)
```

— defun `bcDraw3Dfun` —

```
(defun |bcDraw3Dfun| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Three Dimensional Draw Basic Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|)))
      (|text| "\\centerline{Drawing {\em z = f(x,y)}}\\newline "
        "\\centerline{where {\em z} is the dependent variable and}\\newline "
        "\\centerline{where {\em x, y} are the independent variables}\\vspace{1}\\newline\\menuitemstyle{\\tab{2}}
        "What {\em function} f which you like to draw?\\newline\\tab{2}}")
      (|bcStrings| (55 "exp(cos(x-y)-sin(x*y))-2" |function| EM))
      (|text| .
        "\\newline\\menuitemstyle{\\tab{2}}Enter {\em dependent} variable:")
      (|bcStrings| (6 |z| |dependent| SY))
      (|text| "\\vspace{1}\\newline\\menuitemstyle{\\tab{2}}
        "Enter {\em independent} variables and ranges:\\newline\\tab{2}} "
        "{\\em Variable:}")
      (|bcStrings| (6 |x| |independent1| SY))
      (|text| . "ranges {\em from:}")
      (|bcStrings| (9 -5 |from1| F)) (|text| . "{\\em to:}")
      (|bcStrings| (9 5 |to1| F))
```

```
(|text| . "\\newline\\tab{2}{\\em Variable:}")
(|bcStrings| (6 |y| |independent2| SY))
(|text| . "ranges {\\em from:}")
(|bcStrings| (9 -5 |from2| F)) (|text| . "{\\em to:}")
(|bcStrings| (9 5 |to2| F))
(|text| "\\indent{0}\\vspace{1}\\newline\\menuitemstyle{\\tab{2} "
  "Optionally enter a {\\em title} for your surface:")
(|bcStrings| (15 "" |title| S)) (|text| . "\\indent{0}")
(|doneButton| "Continue" |bcDraw3DfunGen|))
(|htShowPage|))
```

defun bcDraw3DfunGen

```
[|htLabelInputString| p1503]
[concat p1237]
[bcFinish p1493]
[bcDrawIt2 p1473]
```

— defun bcDraw3DfunGen —

```
(defun |bcDraw3DfunGen| (htPage)
  (let (fun dep ind1 from1 to1 ind2 from2 to2 title titlePart)
    (setq fun (|htLabelInputString| htPage '|function|))
    (setq dep (|htLabelInputString| htPage '|dependent|))
    (setq ind1 (|htLabelInputString| htPage '|independent1|))
    (setq from1 (|htLabelInputString| htPage '|from1|))
    (setq to1 (|htLabelInputString| htPage '|to1|))
    (setq ind2 (|htLabelInputString| htPage '|independent2|))
    (setq from2 (|htLabelInputString| htPage '|from2|))
    (setq to2 (|htLabelInputString| htPage '|to2|))
    (setq title (|htLabelInputString| htPage '|title|))
    (cond
      ((not (string-equal title ""))
        (setq titlePart (concat "{" "title ==\" title \"\""))
        (|bcFinish| "draw" fun
          (|bcDrawIt2| ind1 from1 to1)
          (|bcDrawIt2| ind2 from2 to2) titlePart))
      (t
        (|bcFinish| "draw" fun
          (|bcDrawIt2| ind1 from1 to1)
          (|bcDrawIt2| ind2 from2 to2))))))
```

defun Draw - Parametrically defined tube

[htInitPage p1510]
 [htMakePage p1513]
 [htShowPage p1512]
 [\$EmptyMode p708]

Draw Basic Command by 3D parameterized tube

Drawing a parametrically defined curve: (f1(t), f2(t), f3(t))
 in terms of three functions f1, f2, and f3
 and an independent variable t

■ Enter the three functions of the independent variable:

Function f1: `1.3*cos(2*t)*cos(4*t) + sin(4*t)*cos(t)`

Function f2: `1.3*sin(2*t)*cos(4*t) - sin(4*t)*sin(t)`

Function f3: `2.5*cos(4*t)`

■ Enter independent variable and range:

Variable: `t` ranges from: `0` to: `4*pi`

■ Optionally enter a title for your surface: `knot`

Continue

Pressing the **Continue** button will call the function `bcDraw3DparGen` due to this line:

```
(|doneButton| "Continue" |bcDraw3DparGen|)
```

— defun bcDraw3Dpar —

```
(defun |bcDraw3Dpar| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Draw Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|))))
    (|text| "\\centerline{Drawing a parametrically defined curve:"
      "{\\em ( f1(t), f2(t), f3(t) )}\\}\\newline "
      "\\centerline{in terms of three functions {\\em f1}, {\\em f2}, and {\\em f3}\\}\\newline "
      "\\centerline{and an independent variable {\\em t}\\}\\vspace{1}\\}\\newline\\menuitemstyle{\\}\\tab{2} "
```



```

      "Enter the three {\em functions} of the independent variable:")
(|text| . "\\\newline\\tab{2}{\\em Function f1:}")
(|bcStrings| (42 "1.3*cos(2*t)*cos(4*t) + sin(4*t)*cos(t)" |function1| EM))
(|text| . "\\\newline\\tab{2}{\\em Function f2:}")
(|bcStrings| (42 "1.3*sin(2*t)*cos(4*t) - sin(4*t)*sin(t)" |function2| EM))
(|text| . "\\\newline\\tab{2}{\\em Function f3:}")
(|bcStrings| (42 "2.5*cos(4*t)" |function3| EM))
(|text| .
"\\vspace{1}\\\\newline\\menuitemstyle{\\}\\tab{2}Enter {\em independent} variable and range:\\\\newl
(|text| . "{\em Variable:}") (|bcStrings| (6 |t| |ind| SY))
(|text| . "ranges {\em from:}")
(|bcStrings| (9 0 |from1| F)) (|text| "{\em to:}")
(|bcStrings| (9 "4*\\%pi" |to1| F))
(|text| "\\indent{0}\\vspace{1}\\\\newline\\menuitemstyle{\\}\\tab{2} "
      "Optionally enter a {\em title} for your surface:")
(|bcStrings| (15 "knot" |title| S)) (|text| . "\\indent{0}")
(|doneButton| "Continue" |bcDraw3DparGen|)))
(|htShowPage|))

```

defun bcDraw3DparGen

```

[httpLabelInputString p1503]
[concat p1237]
[bcFinish p1493]
[bcDrawIt2 p1473]

```

— defun bcDraw3DparGen —

```

(defun |bcDraw3DparGen| (htPage)
  (let (fun1 fun2 fun3 ind from1 to1 title curvePart tubePart titlePart)
    (setq fun1 (|httpLabelInputString| htPage '|function1|))
    (setq fun2 (|httpLabelInputString| htPage '|function2|))
    (setq fun3 (|httpLabelInputString| htPage '|function3|))
    (setq ind (|httpLabelInputString| htPage '|ind|))
    (setq from1 (|httpLabelInputString| htPage '|from1|))
    (setq to1 (|httpLabelInputString| htPage '|to1|))
    (setq title (|httpLabelInputString| htPage '|title|))
    (setq curvePart (concat "curve(" "{" fun1 ",{" fun2 ",{" fun3 ")"))
    (setq tubePart "{" tubeRadius==.25,{" tubePoints==16")
    (cond
      ((not (string-equal title ""))
        (setq titlePart (concat "{" "title ==\" title "\""))
        (|bcFinish| "draw" curvePart
          (|bcDrawIt2| ind from1 to1) tubePart titlePart))
      (t

```

```
(|bcFinish| "draw" curvePart
  (|bcDrawIt2| ind from1 to1) tubePart))))
```

—————

defun Draw - Parametrically defined surface

```
[htInitPage p1510]
[htMakePage p1513]
[htMakeDoneButton p1533]
[htShowPage p1512]
[$EmptyMode p708]
```

Draw Basic Command by 3D parameterized function

Drawing a parametrically defined surface:
 ($f_1(u,v)$, $f_2(u,v)$, $f_3(u,v)$)
 in terms of three functions f_1 , f_2 , and f_3
 and two independent variables u and v

```
■ Enter the three functions of the independent variables:
  Function f1: u*sin(v)
  Function f2: v*cos(u)
  Function f3: u*cos(v)
■ Enter independent variables and ranges:
  Variable 1: u ranges from: -%pi to: %pi
  Variable 2: v ranges from: -%pi/2 to: %pi/2
■ Optionally enter a title for your surface: surface
```

Continue

Pressing the **Continue** button will call the function **bcDraw3Dpar1Gen** due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcDraw3Dpar1Gen|)
```

— defun bcDraw3Dpar1 —

```
(defun |bcDraw3Dpar1| ()
  (declare (special |$EmptyMode|)))
```

```

(|htInitPage| "Draw Basic Command" NIL)
(|htMakePage|
  '((|domainConditions| (|isDomain| EM |$EmptyMode|)
    (|isDomain| F (|Float|)) (|isDomain| SY (|Symbol|)))
    (|text| "\\centerline{Drawing a parametrically defined surface:}\\newline "
      "\\centerline{{\\em ( f1(u,v), f2(u,v), f3(u,v) )}}\\newline "
      "\\centerline{in terms of three functions {\\em f1}, {\\em f2}, and {\\em f3}}\\newline "
      "\\centerline{and two independent variables {\\em u} and {\\em v}}\\vspace{1}\\newline\\menuitem
        "Enter the three {\\em functions} of the independent variables:")
      (|text| . "\\newline\\tab{2}")
      (|text| . "{\\em Function f1:}")
      (|bcStrings| (43 "u*sin(v)" |function1| EM))
      (|text| . "\\newline\\tab{2}")
      (|text| . "{\\em Function f2:}")
      (|bcStrings| (43 "v*cos(u)" |function2| EM))
      (|text| . "\\newline\\tab{2}")
      (|text| . "{\\em Function f3:}")
      (|bcStrings| (43 "u*cos(v)" |function3| EM))
      (|text| .
        "\\newline\\menuitemstyle{\\tab{2}Enter independent {\\em variables} and ranges:")
      (|text| . "\\newline\\tab{2}")
      (|text| . "{\\em Variable 1:}")
      (|bcStrings| (5 |u| |ind1| SY))
      (|text| . "ranges {\\em from:}")
      (|bcStrings| (9 "-\\%pi" |from1| F)) (|text| . "{\\em to:}")
      (|bcStrings| (9 "\\%pi" |to1| F))
      (|text| . "\\newline\\tab{2}")
      (|text| . "{\\em Variable 2:}")
      (|bcStrings| (5 |v| |ind2| SY))
      (|text| . "ranges {\\em from:}")
      (|bcStrings| (9 "-\\%pi/2" |from2| F))
      (|text| . "{\\em to:}") (|bcStrings| (9 "\\%pi/2" |to2| F))
      (|text| "\\indent{0}\\newline\\menuitemstyle{\\tab{2} "
        "Optionally enter a {\\em title} for your surface:")
      (|bcStrings| (15 "surface" |title| S))
      (|text| . "\\indent{0}"))
    (|htMakeDoneButton| "Continue" ' |bcDraw3Dpar1Gen|)
    (|htShowPage|))

```

defun bcDraw3Dpar1Gen

```

[htpLabelInputString p1503]
[bcDrawIt2 p1473]
[concat p1237]
[bcFinish p1493]

```

— defun bcDraw3Dpar1Gen —

```
(defun |bcDraw3Dpar1Gen| (htPage)
  (let (fun1 fun2 fun3 ind1 from1 to1 ind2 from2 to2
        title r1 r2 surfacePart titlePart)
    (setq fun1 (|htpLabelInputString| htPage '|function1|))
    (setq fun2 (|htpLabelInputString| htPage '|function2|))
    (setq fun3 (|htpLabelInputString| htPage '|function3|))
    (setq ind1 (|htpLabelInputString| htPage '|ind1|))
    (setq from1 (|htpLabelInputString| htPage '|from1|))
    (setq to1 (|htpLabelInputString| htPage '|to1|))
    (setq ind2 (|htpLabelInputString| htPage '|ind2|))
    (setq from2 (|htpLabelInputString| htPage '|from2|))
    (setq to2 (|htpLabelInputString| htPage '|to2|))
    (setq title (|htpLabelInputString| htPage '|title|))
    (setq r1 (|bcDrawIt2| ind1 from1 to1))
    (setq r2 (|bcDrawIt2| ind2 from2 to2))
    (setq surfacePart (concat "surface(" "{" fun1 ",{" fun2 ",{" fun3 ")"))
    (cond
      ((not (string= title ""))
        (setq titlePart (concat "{" "title ==\" title "\""))
        (|bcFinish| "draw" surfacePart r1 r2 titlePart))
      (t (|bcFinish| "draw" surfacePart r1 r2))))))
```

—————→

45.4 Series

defun Basic Commands - Series

[htInitPage p1510]
 [htMakePage p1513]
 [htShowPage p1512]
 [\$EmptyMode p708]

Matrix Basic Command

Create a series by:

- Expansion Expand a function in a series around a point
- Formula Give a formula for the i'th coefficient

— defun bcSeries —

```
(defun |bcSeries| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Series Basic Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| . "Create a series by: ") (|text| . "\\beginmenu")
      (|text| . "\\item ")
      (|bcLinks| ("\\menuitemstyle{Expansion}" "" |bcSeriesExpansion| nil))
      (|text| . "\\tab{11}Expand a function in a series around a point")
      (|text| . "\\item ")
      (|bcLinks| ("\\menuitemstyle{Formula}" "" |bcSeriesByFormula| nil))
      (|text| . "\\tab{11}Give a formula for the {\\em i}'th coefficient")
      (|text| . "\\endmenu"))))
  (|htShowPage|))
```

defun Series - Expansion

```
[htInitPage p1510]
[htMakePage p1513]
[htMakeDoneButton p1533]
[htShowPage p1512]
[$EmptyMode p708]
```

Series Basic Command expand around a point

Enter the function you want to expand in a power series

Enter the power series variable

Enter the point about which you want to expand

Continue

Pressing the **Continue** button will call the function `bcSeriesExpansionGen` due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcSeriesExpansionGen|)
```

— defun bcSeriesExpansion —

```
(defun |bcSeriesExpansion| (a b)
  (declare (ignore a b))
  (declare (special |$EmptyMode|))
  (|htInitPage| "Series Expansion Basic Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| EEM (|Expression| |$EmptyMode|))
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em function} you want to expand in a power series")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (55 "log(cot(x))" |function| EM))
      (|text| . "\\blankline ")
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em power series variable}")
      (|text| . "\\tab{49}")
      (|bcStrings| (8 |x| |variable| SY))
      (|text| . "\\blankline ")
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em point} about which you want to expand")
      (|text| . "\\tab{49}")
      (|bcStrings| (8 "\\pi/2" |point| EM))))
    (|htMakeDoneButton| "Continue" '|bcSeriesExpansionGen|)
    (|htShowPage|))
```

—————

defun bcSeriesExpansionGen

```
[|htLabelInputString| p1503]
[|concat| p1237]
[|bcFinish| p1493]
```

— defun bcSeriesExpansionGen —

```
(defun |bcSeriesExpansionGen| (htPage)
  (let (fun var point terms)
    (setq fun (|htLabelInputString| htPage '|function|))
    (setq var (|htLabelInputString| htPage '|variable|))
    (setq point (|htLabelInputString| htPage '|point|))
    (setq terms (|htLabelInputString| htPage '|numberOfTerms|))
```

```
(|bcFinish| "series" fun (concat var " = " point))))
```

defun Series - Formula

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
```

Series Basic Command series by formula

Select the kind of power series you want to create:

- Taylor Series
Series where the exponent ranges over the integers from a non-negative integer value to plus infinity by an arbitrary positive integer step size
- Laurent Series
Series where the exponent ranges from an arbitrary integer value to plus infinity by an arbitrary positive integer step size
- Puiseux Series
Series where the exponent ranges from an arbitrary rational value to plus infinity by an arbitrary positive rational number step size

— defun bcSeriesByFormula —

```
(defun |bcSeriesByFormula| (a b)
  (declare (ignore a b))
  (|htInitPage| "Power Series Basic Command" NIL)
  (|htMakePage|
    '((|text| . "Select the kind of power series you want to create:")
      (|text| . "\\beginmenu") (|text| . "\\item ")
      (|bcLinks|
        ("\\menuitemstyle{Taylor Series}" "" |bcTaylorSeries| |taylor|))
      (|text| .
        "\\newline Series where the exponent ranges over the integers from a {\\em non-negative integer}
        (|text| . "\\item ")
        (|bcLinks|
          ("\\menuitemstyle{Laurent Series}" "" |bcLaurentSeries| |laurent|))
        (|text| .
        "\\newline Series where the exponent ranges from an arbitrary {\\em integer} value to plus infin
        (|text| . "\\item ")
        (|bcLinks|
          ("\\menuitemstyle{Puisseux Series}" "" |bcPuisseuxSeries| |puiseux|))
        (|text| .
        "\\newline Series where the exponent ranges from an arbitrary {\\em rational value} to plus infi
```

```
(|text| . "\\endmenu"))
(|htShowPage|))
```

defun Series - Formula - Taylor Series

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Taylor Series Basic Command

Enter the formula for the general coefficient of the series

1/factorial(i)_

Enter the index variable for your formula

i

Enter the power series variable

x

Enter the point about which you want to expand

0

For Taylor Series, the exponent of the power series variable ranges from an initial value, an arbitrary non-negative integer, to plus infinity; the step size is any positive integer.

Enter the initial value of the index (an integer)

0

Enter the step size (a positive integer)

1

Continue

Pressing the **Continue** button will call the function **bcTaylorSeriesGen** due to this line:

```
(|doneButton| "Continue" |bcTaylorSeriesGen|)
```

— defun bcTaylorSeries —

```
(defun |bcTaylorSeries| (a b)
  (declare (ignore a b))
  (declare (special |$EmptyMode|))
  (|htInitPage| "Taylor Series Basic Command" NIL)
```



```
(|htMakePage|
'(|domainConditions| (|isDomain| EM |$EmptyMode|)
  (|isDomain| EEM (|Expression| |$EmptyMode|))
  (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the formula for the general coefficient of the series")
(|text| . "\\newline\\tab{2} ")
(|bcStrings| (55 "1/factorial(i)" |formula| EM))
(|text| . "\\blankline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\\em index variable} for your formula")
(|text| . "\\tab{49}") (|bcStrings| (8 |i| |index| SY))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\\em power series variable}")
(|text| . "\\tab{49}") (|bcStrings| (8 |x| |variable| SY))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\\em point} about which you want to expand")
(|text| . "\\tab{49}") (|bcStrings| (8 0 |point| EM))
(|text| . "\\blankline ")
(|text| .
"
For Taylor Series, the exponent of the power series variable ranges from an {\\em initial value}
(|text| . "\\blankline ") (|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\\em initial value} of the index (an integer)")
(|text| . "\\tab{49}") (|bcStrings| (8 "0" |min| I))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\\em step size} (a positive integer)")
(|text| . "\\tab{49}") (|bcStrings| (8 "1" |step| PI))
(|doneButton| "Continue" |bcTaylorSeriesGen|)))
(|htShowPage|))
```

defun bcTaylorSeriesGen

[bcSeriesGen p1435]

— defun bcTaylorSeriesGen —

```
(defun |bcTaylorSeriesGen| (htPage)
  (|bcSeriesGen| htPage))
```

defun bcSeriesGen

```
[htLabelInputString p1503]
[concat p1237]
[bcFinish p1493]
```

— defun bcSeriesGen —

```
(defun |bcSeriesGen| (htPage)
  (let (step min formula index var point varPart minPart)
    (setq step (|htLabelInputString| htPage '|step|))
    (setq min (|htLabelInputString| htPage '|min|))
    (setq formula (|htLabelInputString| htPage '|formula|))
    (setq index (|htLabelInputString| htPage '|index|))
    (setq var (|htLabelInputString| htPage '|variable|))
    (setq point (|htLabelInputString| htPage '|point|))
    (setq varPart (concat var " = " point))
    (setq minPart (concat min ".."))
    (|bcFinish| "series" (concat index " +-> " formula) varPart minPart step)))
```

defun Series - Formula - Laurent Series

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Laurent Series Basic Command

Enter the formula for the general coefficient of the series

$(-1)^{n-1}/(n+2)$

Enter the index variable for your formula

n

Enter the power series variable

x

Enter the point about which you want to expand

0

For Laurent Series, the exponent of the power series variable ranges from an initial value, an arbitrary integer value, to plus infinity; the step size is any positive integer.

Enter the initial value of the index (an integer)

-1

Enter the step size (a positive integer)

1

Continue

Pressing the **Continue** button will call the function **bcLaurentSeriesGen** due to this line:

```
(|doneButton| "Continue" |bcLaurentSeriesGen|)
```

— defun bcLaurentSeries —

```
(defun |bcLaurentSeries| (a b)
  (declare (special |$EmptyMode|) (ignore a b))
  (|htInitPage| "Laurent Series Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| EEM (|Expression| |$EmptyMode|))
      (|isDomain| S (|String|)) (|isDomain| I (|Integer|))
      (|isDomain| PI (|PositiveInteger|))
      (|isDomain| SY (|Symbol|)))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the formula for the general coefficient of the series")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (55 "(-1)**(n - 1)/(n + 2)" |formula| EM))
      (|text| . "\\vspace{1}\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em index variable} for your formula")
      (|text| . "\\tab{49}") (|bcStrings| (8 |n| |index| SY))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em power series variable}")
      (|text| . "\\tab{49}") (|bcStrings| (8 |x| |variable| SY))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em point} about which you want to expand")
      (|text| . "\\tab{49}")
      (|bcStrings| (8 0 |point| F))
      (|text| . "\\blankline")
      (|text| .
        "\\newline For Laurent Series, the exponent of the power series variable ranges from an {\\em in
          (|text| . "\\blankline")
          (|text| . "\\menuitemstyle{\\tab{2}}")
          (|text| . "Enter the {\\em initial value} of the index (an integer)")
          (|text| . "\\tab{49}") (|bcStrings| (8 "-1" |min| I))
          (|text| . "\\newline ")
          (|text| . "\\menuitemstyle{\\tab{2}}")
          (|text| . "Enter the {\\em step size} (a positive integer)")
          (|text| . "\\tab{49}") (|bcStrings| (8 "1" |step| PI))
          (|doneButton| "Continue" |bcLaurentSeriesGen|)))
      (|htShowPage|))
```

—

defun bcLaurentSeriesGen

[bcSeriesGen p1435]

— defun bcLaurentSeriesGen —

```
(defun |bcLaurentSeriesGen| (htPage)
  (|bcSeriesGen| htPage))
```

—————

defun Series - Formula - Puiseux Series

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Puiseux Series Basic Command

Enter the formula for the general coefficient of the series
 $(-1)^{((3*n - 4)/6)}/\text{factorial}(n - 1/3)$

Enter the index variable for your formula
 Enter the power series variable
 Enter the point about which you want to expand

For Puiseux Series, the exponent of the power series variable ranges from an initial value, an arbitrary rational number, to plus infinity; the step size is an any positive rational number.

Enter the initial value of index (a rational number)
 Enter the step size (a positive rational number)

Continue

Pressing the **Continue** button will call the function **bcPuisseuxSeriesGen** due to this line:

```
(|doneButton| "Continue" |bcPuisseuxSeriesGen|)
```

— defun bcPuisseuxSeries —

```
(defun |bcPuisseuxSeries| (a b)
  (declare (special |$EmptyMode|) (ignore a b))
  (|htInitPage| "Puisseux Series Basic Command" nil))
```

```
(|htMakePage|
'(|domainConditions| (|isDomain| EM |$EmptyMode|)
  (|isDomain| EEM (|Expression| |$EmptyMode|))
  (|isDomain| S (|String|)) (|isDomain| I (|Integer|))
  (|isDomain| PI (|PositiveInteger|))
  (|isDomain| RN (|Fraction| (|Integer|)))
  (|isDomain| SY (|Symbol|)))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| .
  "Enter the {\em formula} for the general coefficient of the series")
(|text| . "\\newline\\tab{2} ")
(|bcStrings| (55 "(-1)**((3*n - 4)/6)/factorial(n - 1/3)" |formula| EM))
(|text| . "\\vspace{1}\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\em index variable} for your formula")
(|text| . "\\tab{49}") (|bcStrings| (8 |n| |index| SY))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\em power series variable}")
(|text| . "\\tab{49}") (|bcStrings| (8 |x| |variable| SY))
(|text| . "\\newline ")
(|text| . "\\menuitemstyle{\\tab{2}}")
(|text| . "Enter the {\em point} about which you want to expand")
(|text| . "\\tab{49}") (|bcStrings| (8 0 |point| F))
(|text| . "\\blankline ")
(|text| .
  "For Puiseux Series, the exponent of the power series variable ranges from an {\em initial value}
  (|text| . "\\blankline ") (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| . "Enter the {\em initial value} of index (a rational number)")
  (|text| . "\\tab{51}") (|bcStrings| (6 "4/3" |min| RN))
  (|text| . "\\newline ")
  (|text| . "\\menuitemstyle{\\tab{2}}")
  (|text| . "Enter the {\em step size} (a positive rational number)")
  (|text| . "\\tab{51}") (|bcStrings| (6 "2" |step| RN))
  (|doneButton| "Continue" |bcPuisseuxSeriesGen|))
(|htShowPage|))
```

defun bcPuisseuxSeriesGen

[bcSeriesGen p1435]

— defun bcPuisseuxSeriesGen —

```
(defun |bcPuisseuxSeriesGen| (htPage)
```

```
(|bcSeriesGen| htPage))
```

defun Solve Basic Command

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Solve Basic Command

```
What do you want to solve?
■ A System Of Linear Equations
■ A System of Polynomial Equations
■ A Single Polynomial Equation
```

— defun bcSolve —

```
(defun |bcSolve| ()
  (|htInitPage| "Solve Basic Command" nil)
  (|htMakePage|
    '(|text| . "What do you want to solve? ")
    (|text| . "\\beginmenu") (|text| . "\\item ")
    (|bcLinks|
      ("\\menuitemstyle{A System Of Linear Equations}" ""
       |bcLinearSolve| |linear|))
    (|text| . "\\item ")
    (|bcLinks|
      ("\\menuitemstyle{A System of Polynomial Equations}" ""
       |bcSystemSolve| |polynomial|))
    (|text| . "\\item ")
    (|bcLinks|
      ("\\menuitemstyle{A Single Polynomial Equation}" ""
       |bcSolveSingle| |onePolynomial|))
    (|text| . "\\endmenu"))
  (|htShowPage|))
```

defun Solve - System of Linear Equations

[htInitPage p1510]
 [htMakePage p1513]
 [htShowPage p1512]
 [\$EmptyMode p708]

Linear Solve Basic Command

How do you want to enter the equations?

- ☒ Directly as equations
- ☒ In matrix form $AX = B$, where A is a matrix of coefficients and B is a vector

— defun bcLinearSolve —

```
(defun |bcLinearSolve| (p nn)
  (declare (ignore p nn))
  (|htInitPage| "Basic Solve Command" NIL)
  (|htMakePage|
    '(|text| . "How do you want to enter the equations?")
    (|text| . "\\beginmenu")
    (|text| . "\\item ")
    (|text| . "\\newline ")
    (|bcLinks|
      ("\\menuitemstyle{Directly as equations}" ""
        |bcLinearSolveEqns| |equations|))
    (|text| . "\\item ")
    (|text| . "\\newline ")
    (|bcLinks|
      ("\\menuitemstyle{In matrix form}" ""
        |bcLinearSolveMatrix| |matrix|))
    (|text| . "\\indentrel{16}\\tab{0}")
    (|text| .
      " \\spad{AX = B}, where \\spad{A} is a matrix of coefficients and \\spad{B} is a vector")
    (|text| . "\\indentrel{-16}\\item ")
    (|text| . "\\endmenu"))
  (|htShowPage|))
```

—————→

defun System of Linear Equations - Directly as equations

[htInitPage p1510]
 [htMakePage p1513]

[[htMakeDoneButton p1533](#)]
 [[htShowPage p1512](#)]
 [[\\$EmptyMode p708](#)]

Linear Solve Equations Basic Command

Enter the number of equations:



Pressing the **Continue** button will call the function **bcLinearSolveEqns1** due to this line:

```
(|htMakeDoneButton| "Continue" '|bcLinearSolveEqns1|)
```

— defun bcLinearSolveEqns —

```
(defun |bcLinearSolveEqns| (htPage p)
  (declare (ignore htPage p))
  (|htInitPage| "Basic Solve Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| PI (|PositiveInteger|))))
    (|inputStrings|
      ("Enter the {\em number} of equations:" "" 5 2
        |numberOfEquations| PI))))
  (|htMakeDoneButton| "Continue" '|bcLinearSolveEqns1|)
  (|htShowPage|))
```

—————

defun bcLinearSolveEqns1

[[httpSetProperty p1503](#)]
 [[bcInputEquations p1481](#)]

— defun bcLinearSolveEqns1 —

```
(defun |bcLinearSolveEqns1| (htPage)
  (|httpSetProperty| htPage '|systemType| '|linear|)
  (|httpSetProperty| htPage '|exitFunction| '|bcLinearSolveEqnsGen|)
  (|bcInputEquations| htPage '|exact|))
```

—————

defun System of Linear Equations - In matrix form

[bcReadMatrix p1475]

This routine is a trampoline. It calls **bcReadMatrix** passing the name of a call-back routine **bcLinearSolveMatrix1** to be called after the matrix has been read.

— defun bcLinearSolveMatrix —

```
(defun |bcLinearSolveMatrix| (htPage junk)
  (declare (ignore htPage junk))
  (|bcReadMatrix| ' |bcLinearSolveMatrix1|))
```

—————

defun System of Linear Equations - In matrix form direct

[htInitPage p1510]

[htpSetProperty p1503]

[htMakePage p1513]

[htShowPage p1512]

Matrix Basic Command

Enter the size of the matrix:

Number of rows:

Number of columns:

How would you like to enter the matrix?

☒ By entering individual entries

☐ By formula

This routine is called from several places to enter a matrix. The argument **bcReadMatrix** is the name of a function to call when the matrix has been entered. This value is set as an **exitFunction** in the page's association table.

— defun bcReadMatrix —

```
(defun |bcReadMatrix| (exitFunctionOrNil)
  (let (page)
    (setq page (|htInitPage| "Matrix Basic Command" nil))
    (|htpSetProperty| page ' |exitFunction| exitFunctionOrNil)
    (|htMakePage|
      ' ((|domainConditions| (|isDomain| PI (|PositiveInteger|)))
        (|text| . "Enter the size of the matrix:"))
```

```

(|inputStrings|
  ("Number of {\em rows}:\space{3}" " " 5 2 |rows| PI)
  ("Number of {\em columns}: " " " 5 2 |cols| PI))
(|text| . "\blankline ")
(|text| . "How would you like to enter the matrix?")
(|text| . "\beginmenu")
(|text| . "\item ")
(|bcLinks|
  ("\\menuitemstyle{By entering individual entries}" " "
   |bcInputExplicitMatrix| |explicit|))
(|text| . "\item ")
(|bcLinks|
  ("\\menuitemstyle{By formula}" " "
   |bcInputMatrixByFormula| |formula|))
(|text| . "\endmenu"))
(|htShowPage|))

```

defun Solve System of Linear Equations Individual

```

[objValUnwrap p516]
[htpLabelSpadValue p1504]
[htpLabelInputString p1503]
[parse-integer p??]
[length p??]
[nreverse0 p??]
[concat p1237]
[htInitPage p1510]
[htpPropertyList p1502]
[bcHt p1508]
[htMakePage p1513]
[htMakeDoneButton p1533]
[htpSetProperty p1503]
[htShowPage p1512]
[$EmptyMode p708]
[$bcParseOnly p1498]

```

Input Explicit Matrix

Enter the entries of the matrix:

Row 1, Column 1:	0
Row 1, Column 2:	0
Row 2, Column 1:	0
Row 2, Column 2:	0

Continue

Pressing the **Continue** button will call the function `bcGenExplicitMatrix` due to this line:

```
(|htMakeDoneButton| "Continue" '|bcGenExplicitMatrix|)
```

— `defun bcInputExplicitMatrix` —

```
(defun |bcInputExplicitMatrix| (htPage junk)
  (declare (ignore junk))
  (let (nrows ncols cond wrows wcols rowpart colpart prefix k name
        labelList page t1 t2)
    (declare (special |$EmptyMode| |$bcParseOnly|))
    (setq nrows
      (if (null |$bcParseOnly|)
          (|objValUnwrap| (|htLabelSpadValue| htPage '|rows|))
          (parse-integer (|htLabelInputString| htPage '|rows|))))
    (setq ncols
      (if (null |$bcParseOnly|)
          (|objValUnwrap| (|htLabelSpadValue| htPage '|cols|))
          (parse-integer (|htLabelInputString| htPage '|cols|))))
    (setq k 0)
    (setq wrows (|#| (princ-to-string nrows)))
    (setq wcols (|#| (princ-to-string ncols)))
    (setq labelList
      (do ((i 1 (1+ i))) ((> i nrows) t1)
        (setq t2 nil)
        (setq t1
          (append t1
            (do ((j 1 (1+ j))) ((> j ncols) (nreverse0 t2))
              (setq t2
                (cons
                  (progn
                    (setq rowpart (concat "{\\em Row" (|htStringPad| i wrows))
                    (setq colpart (concat ", Column" (|htStringPad| j wcols)
                                          ":\}\space{2}")
                    (setq prefix (concat rowpart colpart))
                    (setq name (intern (princ-to-string (setq k (1+ k)))))
```

```

      (list prefix "" 30 0 name 'P))
      t2))))))
(setq labelList
  (list
    (list '|domainConditions|
      '(|isDomain| P (|Polynomial| |$EmptyMode|))
      cond)
    (cons '|inputStrings| labelList)))
(setq page (|htInitPage| "Solve Basic Command" (|httpPropertyList| htPage)))
(|bcHt| "Enter the entries of the matrix:")
(|htMakePage| labelList)
(|htMakeDoneButton| "Continue" '|bcGenExplicitMatrix|)
(|httpSetProperty| page '|nrows| nrows)
(|httpSetProperty| page '|ncols| ncols)
(|htShowPage|))

```

defun System of Linear Equations In matrix form by formula

```

[htInitPage p1510]
[htMakePage p1513]
[htMakeDoneButton p1533]
[objValUnwrap p516]
[htpLabelSpadValue p1504]
[parse-integer p??]
[htpLabelInputString p1503]
[htpSetProperty p1503]
[htShowPage p1512]
[$bcParseOnly p1498]

```

Input Matrix By Formula

Enter the row variable:	i
Enter the column variable:	j
Enter the general formula for the entries:	1/(x - i - j - 1)
Continue	

Pressing the **Continue** button will call the function **bcInputMatrixByFormulaGen** due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcInputMatrixByFormulaGen|)
```

— defun bcInputMatrixByFormula —

```
(defun |bcInputMatrixByFormula| (htPage junk)
  (declare (ignore junk))
  (let (page nrows ncols)
    (declare (special |$bcParseOnly|))
    (setq page (|htInitPage| "Basic Matrix Command" (|httpPropertyList| htPage)))
    (|htMakePage|
      '((|domainConditions| (|isDomain| S (|Symbol|))
        (|isDomain| FE (|Expression| (|Integer|))))
        (|text| . "\\menuitemstyle{\\tab{2}}")
        (|text| . "Enter the {\\em row variable}: ")
        (|text| . "\\tab{36}") (|bcStrings| (6 |i| |rowVar| S))
        (|text| . "\\blankline ") (|text| . "\\newline ")
        (|text| . "\\menuitemstyle{\\tab{2}}")
        (|text| . "Enter the {\\em column variable}: ")
        (|text| . "\\tab{36}") (|bcStrings| (6 |j| |colVar| S))
        (|text| . "\\blankline ") (|text| . "\\newline ")
        (|text| . "\\menuitemstyle{\\tab{2}}")
        (|text| . "Enter the general {\\em formula} for the entries:")
        (|text| . "\\newline\\tab{2} ")
        (|bcStrings| (40 "1/(x - i - j - 1)" |formula| FE))))
    (|htMakeDoneButton| "Continue" ' |bcInputMatrixByFormulaGen|)
    (setq nrows
      (if (null |$bcParseOnly|)
        (|objValUnwrap| (|httpLabelSpadValue| htPage '|rows|))
        (parse-integer (|httpLabelInputString| htPage '|rows|))))
    (setq ncols
      (if (null |$bcParseOnly|)
        (|objValUnwrap| (|httpLabelSpadValue| htPage '|cols|))
        (parse-integer (|httpLabelInputString| htPage '|cols|))))
    (|httpSetProperty| page '|nrows| nrows)
    (|httpSetProperty| page '|ncols| ncols)
    (|htShowPage|)))
```

defun Solve - System of Polynomial Equations

[htInitPage p1510]

[htMakePage p1513]

[htMakeDoneButton p1533]
 [htShowPage p1512]

Solve Directly As Equations

Enter the number of equations:



Pressing the **Continue** button will call the function `bcSystemSolveEqns1` due to this line:

```
(|htMakeDoneButton| "Continue" ' |bcSystemSolveEqns1|)
```

— defun bcSystemSolve —

```
(defun |bcSystemSolve| (htPage p)
  (declare (ignore htPage p))
  (|htInitPage| "Basic Solve Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| PI (|PositiveInteger|)))
      (|inputStrings|
        ("Enter the {\em number} of equations:" "" 5 2
          |numberOfEquations| PI))))
  (|htMakeDoneButton| "Continue" ' |bcSystemSolveEqns1|)
  (|htShowPage|))
```

—————

defun bcSystemSolveEqns1

[htpSetProperty p1503]
 [bcInputEquations p1481]

— defun bcSystemSolveEqns1 —

```
(defun |bcSystemSolveEqns1| (htPage)
  (|htpSetProperty| htPage ' |systemType| ' |polynomial|)
  (|htpSetProperty| htPage ' |exitFunction| ' |bcInputSolveInfo|)
  (|bcInputEquations| htPage ' |exact|))
```

—————

defun bcInputSolveInfo

```
[htInitPage p1510]
[httpPropertyList p1502]
[httpSetProperty p1503]
[httpInputAreaList p??]
[htMakePage p1513]
[htShowPage p1512]
```

— defun bcInputSolveInfo —

```
(defun |bcInputSolveInfo| (htPage)
  (let (page)
    (setq page (|htInitPage| "Solve Basic Command" (|httpPropertyList| htPage)))
    (|httpSetProperty| page '|numberOfEquations|
      (|httpProperty| htPage '|numberOfEquations|))
    (|httpSetProperty| page '|inputArea| (|httpInputAreaList| htPage))
    (|htMakePage|
      '((|domainConditions| (|isDomain| PI (|PositiveInteger|)))
        (|text| . "What would you like?")
        (|text| . "\\beginmenu") (|text| . "\\item ")
        (|bcLinks|
          ("\\menuitemstyle{Exact Solutions}" "" |bcSolveEquations| |exact|))
        (|text| . "\\indentrel{18}\\tab{0} ")
        (|text| .
          "Solutions expressed in terms of {\\em roots} of irreducible polynomials")
        (|text| . "\\indentrel{-18}")
        (|text| . "\\item ")
        (|bcLinks|
          ("\\menuitemstyle{Numeric Solutions}" ""
            |bcSolveEquationsNumerically| |numeric|))
        (|text| . "\\indentrel{18}\\tab{0} ")
        (|text| .
          "Solutions expressed in terms of approximate real or complex {\\em numbers}")
        (|text| . "\\indentrel{-18}")
        (|text| . "\\item ")
        (|bcLinks|
          ("\\menuitemstyle{Radical Solutions}" "" |bcSolveEquations| |radical|))
        (|text| . "\\indentrel{18}\\tab{0} ")
        (|text| .
          "Solutions expressed in terms of {\\em radicals} if it is possible")
        (|text| . "\\indentrel{-18}")
        (|text| . "\\endmenu"))))
    (|htShowPage|)))
```

defun Solve - Single Polynomial Equation

[[httpSetProperty](#) p1503]
 [[bcInputEquations](#) p1481]

— **defun bcSolveSingle** —

```
(defun |bcSolveSingle| (htPage p)
  (declare (ignore p))
  (|httpSetProperty| htPage '|systemType| '|onePolynomial|)
  (|httpSetProperty| htPage '|exitFunction| '|bcInputSolveInfo|)
  (|bcInputEquations| htPage '|exact|))
```

—————

Chapter 46

HyperDoc Reference

46.1 Book

46.2 Topics

Numbers

Numbers - Integers Numbers - Fractions Numbers - Machine Floats Numbers - Real Numbers Numbers - Complex Numbers Numbers - Finite Fields Finite Fields - Modular Arithmetic and Prime Fields Finite Fields - Extensions of Finite Fields Extensions - Finite Field Extensions - FinitFieldCyclicGroup Extensions - FinitFieldNormalBasis Extensions - FinitFieldExtension Extensions - FinitFieldExtensionByPolynomial Extensions - FinitFieldCyclicGroupExtension Extensions - FinitFieldCyclicGroupExtensionByPolynomial Extensions - FinitFieldNormalBasisExtension Extensions - FinitFieldNormalBasisExtensionByPolynomial Finite Fields - Irreducible Modulus Polynomial Representations Finite Fields - Cyclic Group Representations Finite Fields - Normal Basis Representations Finite Fields - Conversion Operations for Finite Fields Finite Fields - Utility Operations for Finite Fields Numbers - Numeric Functions Numbers - Cardinal Numbers Numbers - Machine-size Integers Numbers - Roman Numerals Numbers - Continued Fractions Numbers - Partial Fractions Numbers - Quaternions Numbers - Octonions Numbers - Repeating Decimals Numbers - Repeating Binary Expansions Numbers - Repeating Hexadecimal Expansions Numbers - Expansions in other Bases

Polynomials

Polynomials - Basic Functions Polynomials - Substitutions Polynomials - Factorization Polynomials - GCDs and Friends Polynomials - Roots Polynomials -

Specific Types

Functions

Functions - Rational Functions Functions - Algebraic Equations Functions - Elementary Functions Functions - Special Functions Functions - Unknown Functions Functions - Simplification Functions - Pattern Matching Functions - Operator Algebra

Solving Equations

Solving Equations - Systems of Linear Equations Solving Equations - Single Polynomial Equation Solving Equations - Systems of Polynomial Equations Solving Equations - Differential Equations

Calculus

Calculus - Limits Calculus - Derivatives Calculus - Integrals Calculus - More Integrals Calculus - Laplace Calculus - Series Calculus - Differential Equations

Linear Algebra

Linear Algebra - Introduction Linear Algebra - Creating Matrices Linear Algebra - Operations on Matrices Linear Algebra - Eigenvalues and Eigenvectors Linear Algebra - Example: Determinant of a Hilbert Matrix Linear Algebra - Computing the Permanent Linear Algebra - Working with Vectors Linear Algebra - Working with Square Matrices Linear Algebra - Working with One-Dimensional Arrays Linear Algebra - Working with Two-Dimensional Arrays Linear Algebra - Conversion (Polynomials of Matrices)

Graphics

Graphics - Examples Examples - Assorted Examples Examples - Three Dimensional Graphics Examples - Functions of One Variable Examples - Parametric Curves Examples - Polar Coordinates Examples - Implicit Curves Examples - Lists of Points Graphics - 2D Graphics Graphics - 3D Graphics Graphics - Viewports

Algebra

Algebra - Number Theory Number Theory - Galois Groups Number Theory - Number Theory Functions Algebra - Group Theory Group Theory - Info on

Group Theory Group Theory - Info on Representation Theory Group Theory -
Representations of A_6

46.3 Language

46.4 Examples

46.5 Commands

46.6 Glossary

46.7 Hyperdoc

46.8 Search

Chapter 47

HyperDoc Topics

Chapter 48

HyperDoc Browse

Chapter 49

HyperDoc Examples

Chapter 50

HyperDoc Settings

Chapter 51

HyperDoc About

Chapter 52

HyperDoc What's New

Chapter 53

HyperDoc Support Functions

53.1 Handling page creation and deletion

defvar \$activePageList

— initvars —

```
(defvar |$activePageList| nil)
```

—————

defun httpMakeEmptyPage

[\$activePageList p1467]

— defun httpMakeEmptyPage 0 —

```
(defun |httpMakeEmptyPage| (&rest args)
  (let (name val (propList (car args)) (options (cdr args)))
    (declare (special |$activePageList|))
    (setq name (or (car options) (gentemp)))
    (setq |$activePageList| (cons name |$activePageList|))
    (set name (setq val (vector name nil nil nil nil nil propList nil)))
    val))
```

—————

defun httpDestroyPage

```
[member p1238]
[$activePageList p1467]

— defun httpDestroyPage —

(defun |httpDestroyPage| (pageName)
  (declare (special |$activePageList|))
  (when (|member| pageName |$activePageList|)
    (set pageName nil)
    (setq |$activePageList| (delete pageName |$activePageList|))))
```

53.2 Handling Axiom command execution

The `bcGen` function is called with a string which will be passed to the Axiom command line. For example, the path

```
Calculus -> Differentiate -> -> Continue -> Do it
```

will generate the Axiom command line

```
differentiate(sin(x*y),[x,y],[1,2])
```

defun Basic Command result page

```
[concat p1237]
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]

bcGen : Command → nil
— defun bcGen —

(defun |bcGen| (command)
  (let (string)
    (|htInitPage| "Basic Command" nil)
    (setq string
      (if (< (length command) 50)
        (concat "{\\centerline{\\tt " command " }}")
        (concat "{\\tt " command " }")))
    (|htMakePage|
      (list
```

```

'(|text|
  "{Here is the Axiom command you could have issued to compute this result:}"
  "\\vspace{2}\\newline "
  (cons '|text| string) ))
(|htMakeDoitButton| "Do It" command)
(|htShowPage|)))

```

The `htMakeDoitButton` displays the Axiom command to be executed on a page. Pushing the "Do it" invokes `doDoitButton`

defun htMakeDoitButton

```

doDoitButton : String,Command → nil
— defun htMakeDoitButton —

(defun |htMakeDoitButton| (label command)
  (declare (special |$curPage|))
  (cond
    ((equal label "Do It")
     (|bcHt|
      "\\newline\\vspace{1}\\centerline{\\lispcommand{\\DoItBitmap}{(|doDoitButton| ")")
      (t
       (|bcHt|
        (list "\\newline\\vspace{1}\\centerline{\\lispcommand{\\box{" label
          "}}{(|doDoitButton| ")"))
        (|bcHt| (|htpName| |$curPage|))
        (|bcHt| (list " \"" (|htEscapeString| command) "\""))
        (|bcHt| "}}}")
        (|bcHt| "\\vspace{2}{Select \\ \\UpButton{} \\ to go back one page.}")
        (|bcHt|
         "\\newline{Select \\ \\ExitButton{QuitPage} \\ to remove this window.}"))

```

The `doDoitButton` invokes `executeInterpreterCommand`. Why this intermediate function exists is unclear.

defun Execute a command from Hyperdoc

```

doDoitButton : Command → nil
— defun doDoitButton —

```

```
(defun |doDoitButton| (htPage command)
  (declare (ignore htPage))
  (|executeInterpreterCommand| command))
```

The `doDoitButton` function passes a valid Axiom **Command** (see Volume 13[[Book13](#)] for the definition of the “Command” type). This calls the Axiom command line to execute the command.

defun Execute a string in the interpreter

```
executeInterpreterCommand : Command → nil
— defun executeInterpreterCommand —

(defun |executeInterpreterCommand| (command)
  (princ command)
  (terpri)
  (|setCurrentLine| command)
  (catch 'spad_reader (|parseAndInterpret| command))
  (princ (mkprompt))
  (finish-output))
```

53.3 Functions creating pages

Most of the functions create a new page with a call to the function `htMakePage`. This function takes an association list which has several possible keys.

- **domainConditions** with tests such as `(isDomain S (String))` constraining the domains. The possible tests are
 - **isDomain**
- **text** which takes a string argument which may contain latex-like format strings.
 - a plain string
 - **beginmenu**
 - **blankline**
 - **centerline**
 - **em** with an argument to be emphasized

- **indent** sets the column
 - **indentrel** does a relative indent by a positive or negative amount
 - **inputStrings**
 - **item** occurs between a beginmenu and endmenu text
 - **lisplink** takes a string and a function to call
 - **lisplinks**
 - **menuitemstyle** takes a set of characters as an argument
 - **newline**
 - **space** with a numeric argument of the number of spaces
 - **tab** with a numeric argument indicating the tab column
 - **vspace** with the number of blank lines needed
- **bcStrings** which takes a list. The first element is the width of the input box, the second is the default contents, the third is the name of the variable to hold the contents, and the fourth is the domains allowed as input (see **domainConditions** above).
 - **bcLinks** which takes a list containing strings and function calls. It will link to another page by calling the page generation function for that page.
 - **doneButton** which takes 2 arguments, a label and a function to call.
 - **radioButtons** takes a button name and set of lists, each one creating a new radio button
 - **inputStrings**
 - **bcHt**

The `htMakeDoneButton` will put a button on the page with the given title and a function to call when pressed.

defun Basic Command Matrix by Formula generate

```
[htpProperty p1502]
[htpLabelInputString p1503]
[bcGen p1494]
[concat p1237]
```

— defun bcInputMatrixByFormulaGen —

```
(defun |bcInputMatrixByFormulaGen| (htPage)
  (let (fun formula rowVar colVar nrows ncols)
    (cond
      ((setq fun (|htpProperty| htPage '|exitFunction|))
       (funcall fun htPage))
```

```
(t
  (setq formula (|httpLabelInputString| htPage '|formula|))
  (setq rowVar (|httpLabelInputString| htPage '|rowVar|))
  (setq colVar (|httpLabelInputString| htPage '|colVar|))
  (setq nrows (|httpProperty| htPage '|nrows|))
  (setq ncols (|httpProperty| htPage '|ncols|))
  (|bcGen| (concat "matrix([" formula
                    " for " colVar
                    " in 1.." (princ-to-string ncols)
                    "]" for " rowVar
                    " in 1.." (princ-to-string nrows)
                    "])))))))
```

defun Basic Command generate explicit matrix

```
[httpSetProperty p1503]
[httpInputAreaAlist p1501]
[httpProperty p1502]
[bcGen p1494]
[bcMatrixGen p1472]
```

— defun bcGenExplicitMatrix —

```
(defun |bcGenExplicitMatrix| (htPage)
  (let (fun)
    (|httpSetProperty| htPage '|matrix| (|httpInputAreaAlist| htPage))
    (if (setq fun (|httpProperty| htPage '|exitFunction|))
        (funcall fun htPage)
        (|bcGen| (|bcMatrixGen| htPage)))))
```

defun Basic Command generate matrix

```
[httpProperty p1502]
[lassoc p??]
[concat p1237]
[bcwords2liststring p1495]
[systemError p??]
```

— defun bcMatrixGen —

```
(defun |bcMatrixGen| (htPage)
```

```

(let (nrows ncols formula rowVar colVar mat k matform matstring)
  (setq nrows (|httpProperty| htPage '|nrows|))
  (setq ncols (|httpProperty| htPage '|ncols|))
  (setq mat (|httpProperty| htPage '|matrix|))
  (cond
    ((setq formula (lassoc '|formula| mat))
     (setq formula (elt formula 0))
     (setq rowVar (elt (lassoc '|rowVar| mat) 0))
     (setq colVar (elt (lassoc '|colVar| mat) 0))
     (concat "matrix([" formula
              " for " colVar
              " in 1.." (princ-to-string ncols)
              "]" for " rowVar
              " in 1.." (princ-to-string nrows)
              "]))"
    ((setq mat (|httpProperty| htPage '|matrix|))
     (setq mat (reverse mat))
     (setq k (- 1))
     (setq matform
       (loop for i from 0 to (1- nrows)
             collect (loop for j from 0 to (1- ncols)
                           collect (elt (elt mat (incf k)) 1))))
     (setq matstring
       (|bcwords2liststring|
        (loop for t1 in matform collect (|bcwords2liststring| t1))))
     (concat "matrix(" matstring ")")
     (t (|systemError| nil))))))

```

;-Hypertext commands other than solve and matrix

defun Basic Command iteration

[bcMatrixGen p1472]

— defun bcDrawIt2 —

```

(defun |bcDrawIt2| (ind a b)
  (concat "{}" ind "=" a "{}.." b "{}"))

```

defun Sum Basic Command

[htInitPage p1510]
 [htMakePage p1513]
 [htShowPage p1512]
 [\$EmptyMode p708]

— defun bcProduct —

```
(defun |bcProduct| ()
  (declare (special |$EmptyMode|))
  (|htInitPage| "Product Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| SY (|Symbol|)))
      (|text| .
        "Enter the {\em function} you would like to compute the product of:")
      (|inputStrings| (" " " 45 "i**2" |mand| EM))
      (|text| . "\\vspace{1}\\newline")
      (|inputStrings|
        ("Enter the {\em index of the product}:" " " 5 |i| |index| SY))
      (|text| . "\\vspace{1}\\newline Enter the limits of the index:")
      (|inputStrings|
        ("\\newline{\em From:}" " " 10 "1" |first| EM)
        ("{\em To:}\\space{2}" " " 10 "n" |last| EM))
      (|doneButton| "Continue" |bcProductGen|)))
  (|htShowPage|))
```

—————

defun bcProductGen

[htpLabelInputString p1503]
 [concat p1237]
 [bcGen p1494]

— defun bcProductGen —

```
(defun |bcProductGen| (htPage)
  (let (mand index car last)
    (setq mand (|htpLabelInputString| htPage '|mand|))
    (setq index (|htpLabelInputString| htPage '|index|))
    (setq car (|htpLabelInputString| htPage '|first|))
    (setq last (|htpLabelInputString| htPage '|last|))
    (|bcGen| (concat "product(" mand "," index "," car "," last ")"))))
```

—————

defun Read Matrix

```
[htInitPage p1510]
[htpSetProperty p1503]
[htMakePage p1513]
[htShowPage p1512]
```

;-Hypertext commands other than solve and matrix

defun bcSeriesByFormulaGen

```
[bcNotReady p1496]
```

— defun bcSeriesByFormulaGen —

```
(defun |bcSeriesByFormulaGen| (htPage)
  (declare (ignore htPage))
  (|bcNotReady|))
```

—————

defun Real Limit Basic Command

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Real Limit Basic Command

☐ Enter the function you want to compute the limit of:

☐ Enter the name of the variable:

☐ Compute the limit at

☒ A finite point:

☐ Plus infinity

☐ Minus infinity

Pressing the **Continue** button will call the function `bcRealLimitGen` due to this line:

```
(|doneButton| "Continue" |bcRealLimitGen|)
```

— defun bcRealLimit —

```
(defun |bcRealLimit| (a b)
  (declare (special |$EmptyModel|) (ignore a b))
  (|htInitPage| "Real Limit Basic Command" NIL)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyModel|)
      (|isDomain| S (|String|)) (|isDomain| F (|Float|))
      (|isDomain| SY (|Symbol|))))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em function} you want to compute the limit of:")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (45 "x*sin(1/x)" |expression| EM))
      (|text| . "\\blankline") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the name of the {\\em variable}: ")
      (|text| . "\\tab{41}")
      (|bcStrings| (6 |x| |variable| SY))
      (|text| . "\\blankline") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Compute the limit at")
      (|radioButtons| |location|
        ("A finite point:"
          ((|text| . "\\tab{33}") (|bcStrings| (6 0 |point| F))) |finitePoint|)
        ("Plus infinity" "" |plusInfinity|)
        ("Minus infinity" "" |minusInfinity|))
      (|doneButton| "Continue" |bcRealLimitGen|)))
```

```
(|htShowPage|))
```

defun Real Limit Basic Command options

```
[htpButtonValue p1501]
[htpLabelInputString p1503]
[bcFinish p1493]
[htInitPage p1510]
[htMakePage p1513]
[htpSetProperty p1503]
[htShowPage p1512]
```

Real Limit Basic Command options

Compute the limit

- ☒ From both directions
- ☒ From the right
- ☒ From the left

— defun bcRealLimitGen —

```
(defun |bcRealLimitGen| (htPage)
  (let (|p| |fun| |var| |loc| |page|)
    (cond
      ((not (eq (setq |p| (|htpButtonValue| htPage '|location|)) '|finitePoint|))
        (setq |fun| (|htpLabelInputString| htPage '|expression|))
        (setq |var| (|htpLabelInputString| htPage '|variable|))
        (setq |loc|
          (if (eq |p| '|plusInfinity|) "%plusInfinity" "%minusInfinity"))
        (|bcFinish| "limit" |fun| (concat |var| " = " |loc|)))
      (t
        (setq |page| (|htInitPage| "Real Limit Basic Command" nil))
        (|htMakePage|
          '((|text| . "Compute the limit")
            (|lisplinks|
              ("\\menuitemstyle{From both directions}" "" |bcRealLimitGen1| |both|)
              ("\\menuitemstyle{From the right}" "" |bcRealLimitGen1| |right|)
              ("\\menuitemstyle{From the left}" "" |bcRealLimitGen1| |left|))))
        (|htpSetProperty| |page| '|fun|
          (|htpLabelInputString| htPage '|expression|))
```

```
(|httpSetProperty| |page| '|var|
  (|httpLabelInputString| htPage '|variable|))
(|httpSetProperty| |page| '|loc|
  (|httpLabelInputString| htPage '|point|))
(|htShowPage|))))))
```

defun bcRealLimitGen1

```
[httpProperty p1502]
[concat p1237]
[bcFinish p1493]
```

— defun bcRealLimitGen1 —

```
(defun |bcRealLimitGen1| (htPage key)
  (let (direction fun var loc varPart)
    (setq direction
      (cond
        ((eq key '|right|) "\"right\"")
        ((eq key '|left|) "\"left\"")
        (t nil)))
    (setq fun (|httpProperty| htPage '|fun|))
    (setq var (|httpProperty| htPage '|var|))
    (setq loc (|httpProperty| htPage '|loc|))
    (setq varPart (concat var " = " loc))
    (|bcFinish| "limit" fun varPart direction)))
```

defun Complex Limit Basic Command

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Complex Limit Basic Command

☐ Enter the function you want to compute the limit of:

☐ Enter the name of the variable:

☐ Compute the limit at

☒ A finite point:

Real part:

Complex part:

☐ Complex infinity

Continue

Pressing the **Continue** button will call the function `bcComplexLimitGen` due to this line:

```
(|doneButton| "Continue" |bcComplexLimitGen|)
```

— defun bcComplexLimit —

```
(defun |bcComplexLimit| (a b)
  (declare (special |$EmptyMode|) (ignore a b))
  (|htInitPage| "Complex Limit Basic Command" nil)
  (|htMakePage|
    '((|domainConditions| (|isDomain| EM |$EmptyMode|)
      (|isDomain| S (|String|)) (|isDomain| F (|Float|))
      (|isDomain| SY (|Symbol|)))
      (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the {\\em function} you want to compute the limit of:")
      (|text| . "\\newline\\tab{2} ")
      (|bcStrings| (40 "sin(a*x)/tan(b*x)" |expression| EM))
      (|text| . "\\blankline ") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Enter the name of the {\\em variable}: ")
      (|text| . "\\tab{37}") (|bcStrings| (5 |x| |variable| SY))
      (|text| . "\\blankline ") (|text| . "\\newline ")
      (|text| . "\\menuitemstyle{\\tab{2}}")
      (|text| . "Compute the limit at")
      (|radioButtons| |location|
        ("A finite point:"
          ((|text| . "\\newline\\space{0}Real part:\\space{3}")
            (|bcStrings| (20 0 |real| F))
            (|text| . "\\newline Complex part:")
            (|bcStrings| (20 0 |complex| F)))
```

```

      |finitePoint|)
      ("Complex infinity" "" |complexInfinity|))
      (|doneButton| "Continue" |bcComplexLimitGen|)))
(|htShowPage|))

```

defun bcComplexLimitGen

```

[htpLabelInputString p1503]
[htpButtonValue p1501]
[concat p1237]
[bcFinish p1493]

```

— defun bcComplexLimitGen —

```

(defun |bcComplexLimitGen| (htPage)
  (let (fun var p real comp complexPart loc varPart)
    (setq fun (|htpLabelInputString| htPage '|expression|))
    (setq var (|htpLabelInputString| htPage '|variable|))
    (setq loc
      (cond
        ((eq (setq p (|htpButtonValue| htPage '|location|)) '|finitePoint|)
          (setq real (|htpLabelInputString| htPage '|real|))
          (setq comp (|htpLabelInputString| htPage '|complex|))
          (setq complexPart
            (cond
              ((string= comp "0") "")
              ((string= comp "1") "%i")
              (t (concat comp "%i"))))
            (cond
              ((string= real "0") (if (string= complexPart "") '|0| complexPart))
              ((string= complexPart "") real)
              (t (concat real " + " complexPart))))
            (t "%infinity"))))
    (setq varPart (concat var " = " loc))
    (|bcFinish| "complexLimit" fun varPart)))

```

defvar \$systemType

— initvars —

```
(setq |$systemType| nil)
```

defvar \$numberOfEquations

— initvars —

```
(defvar |$numberOfEquations| 0)
```

defvar \$solutionMethod

— initvars —

```
(defvar |$solutionMethod| nil)
```

defun bcInputEquations

```
[concat p1237]
[bcMakeLinearEquations p1484]
[bcMakeEquations p1483]
[htProperty p??]
[parse-integer p??]
[objValUnwrap p516]
[htInitPage p1510]
[htpPropertyList p1502]
[htpSetProperty p1503]
[htSay p1509]
[htMakePage p1513]
[bcHt p1508]
[bcMakeUnknowns p1483]
[htMakeDoneButton p1533]
[htShowPage p1512]
[$EmptyMode p708]
[$bcParseOnly p1498]
```

— defun bcInputEquations —


```

(defun |bcInputEquations| (htPage solutionMethod)
(labels (
(f (i n linearp)
(let (spacer prefix lnam rnam var)
(setq spacer (cond ((> i 99) 0) ((> i 9) 1) (t 2)))
(setq prefix
(concat "\\newline\\tab{2}{\\em Equation " (princ-to-string i) ":}"))
(setq prefix (concat prefix "\\space{" (princ-to-string spacer) "}"))
(setq lnam (intern (concat "l" (princ-to-string i))))
(setq rnam (intern (concat "r" (princ-to-string i))))
(setq var (if linearp
(|bcMakeLinearEquations| i n)
(|bcMakeEquations| i n)))
(cons
(cons '|text| prefix)
(list (list '|bcStrings| (list 30 var lnam 'p))
'|text| . " = "
(list '|bcStrings| (list 5 '|0| rnam 'p))))))
(let (numEqs linearPred labelList equationPart page)
(declare (special |$EmptyMode| |$bcParseOnly|))
(setq numEqs
(cond
((eq (|httpProperty| htPage '|systemType|) '|onePolynomial|) 1)
(|$bcParseOnly|
(parse-integer (|httpLabelInputString| htPage '|numberOfEquations|)))
(t
(|objValUnwrap| (|httpLabelSpadValue| htPage '|numberOfEquations|))))
(setq linearPred (eq (|httpProperty| htPage '|systemType|) '|linear|))
(setq labelList
(cond
((eql numEqs 1)
'(|bcStrings| (42 "x^2+1" l1 p)) (|text| . " = "
(|bcStrings| (6 0 r1 P))))
(t
(loop for i from 1 to numEqs
append (f i numEqs linearPred))))
(setq equationPart
(cons '|domainConditions|
(|isDomain| P (|Polynomial| |$EmptyMode|))
(|isDomain| S (|String|))
(|isDomain| PI (|PositiveInteger|)))
labelList))
(setq page (|htInitPage| "Solve Basic Command" (|httpPropertyList| htPage)))
(|httpSetProperty| page '|numberOfEquations| numEqs)
(|httpSetProperty| page '|solutionMethod| solutionMethod)
(|htSay| "\\newline\\menuitemstyle{}\\tab{2}")
(|htSay| (if (eql numEqs 1)
"Enter the {\\em Equation}:"
"Enter the {\\em Equations}:"))
(|htSay| "\\newline\\tab{2}")

```

```
(|htMakePage| equationPart)
(|bcHt| "\\blankline ")
(|htSay| "\\newline\\menuitemstyle{\\tab{2}}")
(|htMakePage|
  (if (eq1 numEqs 1)
    '(|text| . "Enter the {\\em unknown} (leave blank if implied): ")
    (|text| . "\\tab{48}")
    (|bcStrings| (6 "x" unknowns S . |quoteString|)))
  (list
    '(|text| . "Enter the unknowns (leave blank if implied):")
    '(|text| . "\\tab{44}")
    (list ' |bcStrings|
      (list 10 (|bcMakeUnknowns| numEqs) ' |unknowns| 'p))))
(|htMakeDoneButton| "Continue" ' |bcInputEquationsEnd|)
(|htShowPage|)))
```

defun Create a variable string

— defun bcCreateVariableString —

```
(defun |bcCreateVariableString| (i)
  (format nil "x~a" i))
```

defun bcMakeUnknowns

— defun bcMakeUnknowns —

```
(defun |bcMakeUnknowns| (number)
  (format nil "~{~A~~~}"
    (loop for i from 1 to number collect (format nil "x~a " i))))
```

defun bcMakeEquations

```
[concat p1237]
[bcCreateVariableString p1483]
```

[nreverse0 p??]

— defun bcMakeEquations —

```
(defun |bcMakeEquations| (i number)
  (if (eql number 1)
      (concat (|bcCreateVariableString| 1) '|^2+1|))
      (progn
        (|bcCreateVariableString| i)
        (concat
          (concat (apply 'concat
            (let (t1)
              (do ((j 1 (1+ j))) (> j number) (nreverse0 t1))
              (setq t1 (cons (concat (|bcCreateVariableString| j) '+) t1))))
            '|1|)
          (concat '-2* (concat (|bcCreateVariableString| i) '|^2|))))))
```

defun bcMakeLinearEquations

[bcCreateVariableString p1483]
 [concat p1237]
 [nreverse0 p??]

— defun bcMakeLinearEquations —

```
(defun |bcMakeLinearEquations| (i number)
  (cond
    ((eql number 1) (|bcCreateVariableString| 1))
    ((eql number 2)
     (cond
       ((eql i 1)
        (concat (|bcCreateVariableString| 1)
                  (concat '+ (|bcCreateVariableString| 2))))
       (t
        (concat (|bcCreateVariableString| 1)
                  (concat '- (|bcCreateVariableString| 2))))))
    (t
     (concat
      (concat
       (apply 'concat
        (let (t1)
          (do ((j 1 (1+ j))) (> j number) (nreverse0 t1))
          (setq t1 (cons (concat (|bcCreateVariableString| j) '+) t1))))
        '|1|)
       (concat '-2* (|bcCreateVariableString| i))))))
```

defun bcInputEquationsEnd

If `exitFunction` is set, call it. [systemError p??]

— defun bcInputEquationsEnd —

```
(defun |bcInputEquationsEnd| (htPage)
  (let (fun)
    (if (setq fun (|httpProperty| htPage '|exitFunction|))
        (funcall fun htPage)
        (|systemError| nil))))
```

defun bcSolveEquationsNumerically

[htInitPage p1510]
 [htMakePage p1513]
 [htMakeDoneButton p1533]
 [htShowPage p1512]
 [httpPropertyList p1502]

— defun bcSolveEquationsNumerically —

```
(defun |bcSolveEquationsNumerically| (htPage p)
  (declare (ignore p))
  (|htInitPage| "Solve Basic Command" (|httpPropertyList| htPage))
  (|htMakePage|
    '(|text| . "What would you like?")
    (|radioButtons| |choice|
      ("Real roots expressed as rational numbers" "" |rr|)
      ("Real roots expressed as floats" "" |rf|)
      ("Complex roots expressed as rational numbers" "" |cr|)
      ("Complex roots expressed as floats" "" |cf|))
    (|text| . "\\vspace{1}\\newline")
    (|inputStrings| ("Enter the number of desired {\\em digits} of accuracy"
                     "" 5 20 |acc| PI))))
  (|htMakeDoneButton| "Continue" '|bcSolveNumerically1|)
  (|htShowPage|))
```

defun bcSolveNumerically1

[bcSolveEquations p1486]

— defun bcSolveNumerically1 —

```
(defun |bcSolveNumerically1| (htPage)
  (|bcSolveEquations| htPage '|numeric|))
```

—————

defun bcSolveEquations

[htpLabelInputString p1503]

[htpButtonValue p1501]

[member p1238]

[concat p1237]

[htpProperty p1502]

[bcString2WordList p1495]

[bcwords2liststring p1495]

[bcGenEquations p1492]

[bcFinish p1493]

— defun bcSolveEquations —

```
(defun |bcSolveEquations| (htPage solutionMethod)
  (let (digits kind accString alist varpart r varlist varString eqnString name)
    (when (eq solutionMethod '|numeric|)
      (setq digits (|htpLabelInputString| htPage '|acc|))
      (setq kind (|htpButtonValue| htPage '|choice|))
      (setq accString
        (if (|member| kind '(|rf| |cf|))
          (concat "1.e-" digits)
          (concat "1/10**" digits))))
      (setq alist (|htpProperty| htPage '|inputArea|))
      (setq varpart (cadar alist))
      (setq r (cdr alist))
      (setq varlist (|bcString2WordList| varpart))
      (setq varString
        (if (cdr varlist)
          (|bcwords2liststring| varlist)
          (car varlist)))
      (setq eqnString (|bcGenEquations| r))
      (cond
        ((eq solutionMethod '|numeric|)
         (setq name (if (|member| kind '(|rf| |rr|)) "solve" "complexSolve"))
```

```
(|bcFinish| name eqnString accString))
(t
  (setq name (if (eq solutionMethod '|radical|) "radicalSolve" "solve"))
  (|bcFinish| name eqnString varString accString))))
```

defun Linear Solve Basic Command options

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Linear Solve Basic Command options

```
The right side vector B is:
Zero;the system is homogeneous
Not zero;the system is not homogeneous
```

— defun bcLinearSolveMatrix1 —

```
(defun |bcLinearSolveMatrix1| (htPage)
  (let (page)
    (setq page
      (|htInitPage| "Linear Solve Basic Command" (|httpPropertyList| htPage)))
    (|httpSetProperty| page '|matrix| (|bcLinearExtractMatrix| htPage))
    (|htMakePage|
      '(|text| . "The right side vector B is:")
      (|lispLinks|
        ("Zero:" "the system is homogeneous" |bcLinearSolveMatrixHomo| |homo|)
        ("Not zero:" "the system is not homogeneous"
          |bcLinearSolveMatrixInhomo| |nothomo|))))
    (|htShowPage|)))
```

defun bcLinearExtractMatrix

```
[httpInputAreaAlist p1501]
```

— defun bcLinearExtractMatrix —

```
(defun |bcLinearExtractMatrix| (htPage)
  (reverse (|httpInputAreaAlist| htPage)))
```

defun Linear Solve Basic Command Inhomogeneous

```
[concat p1237]
[httpProperty p1502]
[htInitPage p1510]
[httpPropertyList p1502]
[httpSetProperty p1503]
[htMakePage p1513]
[htShowPage p1512]
[$EmptyMode p708]
```

Linear Solve Basic Command Inhomogeneous

Enter the right side vector B:

Coefficient 1: 0

Coefficient 2: 0

Do you want:

All the solutions?

A particular solution?

— defun bcLinearSolveMatrixInhomo —

```
(defun |bcLinearSolveMatrixInhomo| (htPage junk)
  (declare (ignore junk))
  (labels (
    (f (i)
      (let (spacer prefix name)
        (setq spacer (cond ((> i 99) 0) ((> i 9) 1) (t 2)))
        (setq prefix (concat "{\\em Coefficient " (princ-to-string i) ":}"))
        (unless (eql spacer 0)
          (setq prefix (concat prefix "\\space{" (princ-to-string spacer) "}"))))
        (setq name (intern (concat "c" (princ-to-string i))))
        (list prefix '|| 30 0 name 'p )))))
    (let (nrows ncols labelList page)
      (declare (special |$EmptyMode|))
      (setq nrows (|httpProperty| htPage '|nrows|))
```

```

(setq ncols (|httpProperty| htPage '|ncols|))
(setq labelList (loop for i from 1 to ncols collect (f i)))
(setq page
  (|htInitPage| "Linear Solve Basic Command" (|httpPropertyList| htPage)))
(|httpSetProperty| page '|matrix| (|httpProperty| htPage '|matrix|))
(|httpSetProperty| page '|nrows| nrows)
(|httpSetProperty| page '|ncols| ncols)
(|htMakePage|
  (list
    '(|domainConditions| (|isDomain| P (|Polynomial| |$EmptyMode|)))
    '(|text| . "Enter the right side vector B:")
    (cons
      (cons '|inputStrings| labelList)
      (list
        '(|text| . "\\vspace{1}\\newline Do you want:")
        '(|lispLinks|
          ("All the solutions?" "" |bcLinearSolveMatrixInhomoGen| |all|)
          ("A particular solution?" ""
           |bcLinearSolveMatrixInhomoGen| |particular|))
        )))
    (|htShowPage|))))

```

defun bcLinearSolveMatrixInhomoGen

[bcLinearMatrixGen p1490]

— defun bcLinearSolveMatrixInhomoGen —

```

(defun |bcLinearSolveMatrixInhomoGen| (htPage key)
  (|bcLinearMatrixGen| htPage key))

```

defun bcLinearSolveMatrixHomo

[bcLinearMatrixGen p1490]

— defun bcLinearSolveMatrixHomo —

```

(defun |bcLinearSolveMatrixHomo| (htPage key)
  (declare (ignore key))
  (|bcLinearMatrixGen| htPage '|homo|))

```

defun bcLinearMatrixGen

```
[bcMatrixGen p1472]
[bcFinish p1493]
[httpInputAreaAlist p1501]
[bcVectorGen p1495]
[bcMkFunction p1493]
[bcGen p1494]
[concat p1237]
```

— defun bcLinearMatrixGen —

```
(defun |bcLinearMatrixGen| (htPage key)
  (let (matform vector vecform form)
    (setq matform (|bcMatrixGen| htPage))
    (cond
      ((eq key '|homo|)
        (|bcFinish| "nullSpace" matform))
      (t
        (setq vector
          (loop for x in (reverse (|httpInputAreaAlist| htPage))
            collect (elt x 1)))
          (setq vecform (|bcVectorGen| vector))
          (setq form (|bcMkFunction| "solve" matform (cons vecform nil)))
          (|bcGen| (if (eq key '|particular|)
            (concat form ".particular")
            form))))))
```

defun linearFinalRequest

```
[sayBrightly p??]
[bcQueryInteger p??]
[explainLinear p1491]
```

— defun linearFinalRequest —

```
(defun |linearFinalRequest| (nhh mat vect)
  (declare (ignore mat vect))
  (let (tt)
    (|sayBrightly| "Do you want more information on the meaning of the output")
    (|sayBrightly| " (1) no ")
    (|sayBrightly| " (2) yes ")
```

```
(setq tt (|bcQueryInteger| 1 2 t))
(cond
  ((eq1 tt 1) (|sayBrightly| "Bye Bye"))
  ((eq1 tt 2) (|explainLinear| nhh))))
```

defun explainLinear

```
[systemError p??]
```

— defun explainLinear —

```
(defun |explainLinear| (flag)
  (cond
    ((eq flag '|notHomogeneous|)
     '("solve returns a particular solution and a basis for"
       "the vector space of solutions for the homogeneous part."
       "The particular solution is \"failed\" if one cannot be found.))
    ((eq flag '|homogeneous|)
     '("solve returns a basis for"
       "the vector space of solutions for the homogeneous part"))
    (t (|systemError| nil))))
```

defun finalExactRequest

```
[bcQueryInteger p??]
[|sayBrightly| p??]
[moreExactSolution p??]
[explainExact p??]
```

— defun finalExactRequest —

```
(defun |finalExactRequest| (equations unknowns)
  (let (tt)
    (|sayBrightly| "Do you like:")
    (|sayBrightly| " (1) the solutions how they are displayed")
    (|sayBrightly| " (2) to get ???")
    (|sayBrightly| " (3) more information on the meaning of the output")
    (setq tt (|bcQueryInteger| 1 3 t))
    (cond
      ((eq1 tt 1) (|sayBrightly| "Bye Bye"))
      ((eq1 tt 2) (|moreExactSolution| equations unknowns))
```

```
((eq1 tt 3) (|explainExact| equations unknowns))))))
```

defun bcLinearSolveEqnsGen

```
[httpInputAreaAlist p1501]
[httpLabelInputString p1503]
[bcString2WordList p1495]
[bcwords2liststring p1495]
[bcGenEquations p1492]
[bcFinish p1493]
```

— defun bcLinearSolveEqnsGen —

```
(defun |bcLinearSolveEqnsGen| (htPage)
  (let (vars varlist varString alist eqnString)
    (setq alist (|httpInputAreaAlist| htPage))
    (when (setq vars (|httpLabelInputString| htPage '|unknowns|))
      (setq varlist (|bcString2WordList| vars))
      (setq varString
        (if (cdr varlist) (|bcwords2liststring| varlist) (car varlist)))
      (setq alist (cdr alist)))
    (setq eqnString (|bcGenEquations| alist))
    (|bcFinish| "solve" eqnString varString)))
```

defun bcGenEquations

```
[concat p1237]
[bcwords2liststring p1495]
```

— defun bcGenEquations —

```
(defun |bcGenEquations| (alist)
  (let (right left y eqnlist)
    (setq y alist)
    (loop while y do
      (setq right (elt (car y) 1))
      (setq y (cdr y))
      (setq left (elt (car y) 1))
      (setq y (cdr y))
      (setq eqnlist (cons (concat left " = " right) eqnlist)))
    (if (cdr eqnlist)
```

```
(|bcwords2liststring| eqnlist)
(car eqnlist)))
```

defun Output the final formula

— defun bcFinish —

```
(defun |bcFinish| (&rest t1)
  (|bcGen| (|bcMkFunction| (car t1) (cadr t1) (cddr t1))))
```

defun convert arguments into function call syntax

Convert verb—(bcMkFunction "test" "arg1" '("arg2" "arg3"))— to "test(arg1,arg2,arg3)"

— defun bcMkFunction —

```
(defun |bcMkFunction| (name arg args)
  (let (str)
    (setq str
      (let ((result ""))
        (concatenate 'string arg
          (dolist (i args result)
            (when i
              (setq result (concatenate 'string result
                (concatenate 'string "," i)))))))
      (concatenate 'string name "(" str ")"))))
```

defun bcString2HyString2

— defun bcString2HyString2 —

```
(defun |bcString2HyString2| (s)
  (if (and (stringp s) (char= (elt s 0) #\"))
    (concatenate 'string "\\\" s "\\\"")
    s))
```

defun bcString2HyString

— defun bcString2HyString —

```
(defun |bcString2HyString| (s) s)
```

defun find a character position in a string

— defun bcFindString —

```
(defun |bcFindString| (s i n char)
  (position char s :start i :end n))
```

defun Basic Command result page – NAG version

```
[concat p1237]
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
```

Except for the banner the `bcGen` and `linkGen` functions are identical. We no longer care so we just call `bcGen`.

— defun linkGen —

```
(defun |linkGen| (command)
  (|bcGen| command))
```

defun bcOptional

— defun bcOptional —

```
(defun |bcOptional| (s)
  (if (string-equal s "") "2" s))
```

defun create a vertical space on a page

[bcHt p1508]

— defun bcvspace —

```
(defun |bcvspace| ()
  (|bcHt| "\\vspace{1}\\newline "))
```

—————

defun break a string into words

— defun bcString2WordList —

```
(defun |bcString2WordList| (string)
  (loop for i = 0 then (1+ j)
        as j = (position #\space string :start i)
        collect (subseq string i j)
        while j))
```

—————

defun format words into a string

[concat p1237]

— defun bcwords2liststring —

```
(defun |bcwords2liststring| (words)
  (format nil "[~{~A~~, ~}" words))
```

—————

defun format a vector

[bcwords2liststring p1495]

— defun bcVectorGen —

```
(defun |bcVectorGen| (vec)
  (|bcwords2liststring| vec))
```

defun format an error message

```
[sayBrightlyNT p??]
[sayBrightly p??]
```

— defun bcError —

```
(defun |bcError| (string)
  (|sayBrightlyNT| "NOTE: ")
  (|sayBrightly| string))
```

defun format intervals

```
[concat p1237]
```

— defun bcDrawIt —

```
(defun |bcDrawIt| (ind a b)
  (concat ind "=" a ".." b))
```

defun Basic Command page not ready

```
[htInitPage p1510]
[htMakePage p1513]
[htShowPage p1512]
```

— defun bcNotReady —

```
(defun |bcNotReady| (htPage)
  (declare (ignore htPage))
  (|htInitPage| "Basic Command" NIL)
  (|htMakePage|
    '((|text| . "{\\centerline{\\em This facility will soon be available}}"))))
  (|htShowPage|))
```

defun pad a string with blanks

[concat p1237]

— defun htStringPad —

```
(defun |htStringPad| (n w)
  (let (s ws)
    (setq s (princ-to-string n))
    (setq ws (|#| s))
    (concat "\\space{" (princ-to-string (1+ (- w ws))) "}" s)))
```

—————

defun construct a name string

Given ("one" "two" "three") generate "(one,two,three)"

— defun stringList2String —

```
(defun |stringList2String| (x)
  (let (str)
    (cond
      ((null x) "()")
      (t
       (setq str
              (let ((result ""))
                (concatenate 'string (car x)
                              (dolist (i (cdr x) result)
                                (setq result (concatenate 'string result
                                                          (concatenate 'string "," i))))))
              (concatenate 'string "(" str ")")))))
```

—————

defun construct a name string

[concat p1237]

— defun htMkName —

```
(defun |htMkName| (s n)
  (concat s (princ-to-string n)))
```

—————

;;; ht-util merge

defvar \$bcParseOnly

— initvars —

```
(defvar |$bcParseOnly| t)
```

defvar \$htLineList

— initvars —

```
(defvar |$htLineList| nil)
```

defvar \$curpage

— initvars —

```
(defvar |$curPage| nil)
```

HTPage Layout

This is a list with the fields

1. name
2. Domain Conditions
3. Domain Variable Alist
4. Domain Pvar Subst List
5. Radio Button Alist
6. Input Area Alist
7. Property List
8. Description

defun htpName

— defun htpName —

```
(defun |htpName| (htPage) (elt htPage 0))
```

—————

defun htpSetName

— defun htpSetName —

```
(defun |htpSetName| (htPage val) (setf (elt htPage 0) val))
```

—————

defun htpDomainConditions

— defun htpDomainConditions —

```
(defun |htpDomainConditions| (htPage) (elt htPage 1))
```

—————

defun htpSetDomainConditions

— defun htpSetDomainConditions —

```
(defun |htpSetDomainConditions| (htPage val)  
  (setf (elt htPage 1) val))
```

—————

defun htpDomainVariableAlist

— defun htpDomainVariableAlist —

```
(defun |httpDomainVariableAlist| (htPage) (elt htPage 2))
```

defun httpSetDomainVariableAlist

— defun httpSetDomainVariableAlist —

```
(defun |httpSetDomainVariableAlist| (htPage val)
  (setf (elt htPage 2) val))
```

defun httpDomainPvarSubstList

— defun httpDomainPvarSubstList —

```
(defun |httpDomainPvarSubstList| (htPage) (elt htPage 3))
```

defun httpSetDomainPvarSubstList

— defun httpSetDomainPvarSubstList —

```
(defun |httpSetDomainPvarSubstList| (htPage val)
  (setf (elt htPage 3) val))
```

defun httpRadioButtonAlist

— defun httpRadioButtonAlist —

```
(defun |httpRadioButtonAlist| (htPage) (elt htPage 4))
```

defun httpButtonValue

— defun httpButtonValue —

```
(defun |httpButtonValue| (htPage groupName)
  (prog ()
    (return
      (SEQ (DO ((G166092
                  (LASSOC groupName
                        (|httpRadioButtonAlist| htPage))
                  (CDR G166092))
                (|buttonName| nil))
            ((OR (ATOM G166092)
                  (progn (setq |buttonName| (car G166092)) nil))
              NIL)
            (SEQ (EXIT (COND
                        ((BOOT-EQUAL
                          (|stripSpaces|
                            (|httpLabelInputString| htPage
                              |buttonName|))
                          "t")
                        (EXIT (RETURN |buttonName|))))))))))
```

defun httpSetRadioButtonAlist

— defun httpSetRadioButtonAlist —

```
(defun |httpSetRadioButtonAlist| (htPage val)
  (setf (elt htPage 4) val))
```

defun httpInputAreaAlist

— defun httpInputAreaAlist —

```
(defun |httpInputAreaAlist| (htPage)
  (elt htPage 5))
```

defun httpSetInputAreaAlist

— defun httpSetInputAreaAlist —

```
(defun |httpSetInputAreaAlist| (htPage val)
  (setf (elt htPage 5) val))
```

—————

defun httpAddInputAreaProp

— defun httpAddInputAreaProp —

```
(defun |httpAddInputAreaProp| (htPage label prop)
  (setf (elt htPage 5)
    (cons
      (cons label (cons nil (cons nil (cons nil prop))))
      (elt htPage 5))))
```

—————

defun httpPropertyList

— defun httpPropertyList —

```
(defun |httpPropertyList| (htPage)
  (elt htPage 6))
```

—————

defun httpProperty

— defun httpProperty —

```
(defun |httpProperty| (htPage propName)
  (lassoc propName (elt htPage 6)))
```

—————

defun httpSetProperty

— defun httpSetProperty —

```
(defun |httpSetProperty| (htPage propName val)
  (let (pair)
    (setq pair (|assoc| propName (elt htPage 6)))
    (cond
      (pair (rplacd pair val))
      (t (setf (elt htPage 6) (cons (cons propName val) (elt htPage 6)))))))
```

—————

defun httpLabelInputString

— defun httpLabelInputString —

```
(defun |httpLabelInputString| (htPage label)
  (let (props s)
    (setq props (|assoc| label (|httpInputAreaAlist| htPage)))
    (when (and props (stringp (setq s (elt props 0))))
      (if (equal s "") s (|trimString| s)))))
```

—————

defun httpLabelFilteredInputString

— defun httpLabelFilteredInputString —

```
(defun |httpLabelFilteredInputString| (htPage label)
  (let (props)
    (setq props (|assoc| label (|httpInputAreaAlist| htPage)))
    (when props
      (cond
        ((and (> (|#| props) 5) (elt props 6))
         (funcall (symbol-function (elt props 6)) (elt props 0)))
        (t (|replacePercentByDollar| (elt props 0)))))))
```

—————

defun replacePercentByDollar,fn

— defun replacePercentByDollar,fn —

```
(defun |replacePercentByDollar,fn| (s i n)
  (let (m)
    (cond
      ((> i n) "")
      ((> (setq m (|charPosition| #\% s i)) n) (substring s i nil))
      (t (concat (substring s i (- m i)) "$"
                  (|replacePercentByDollar,fn| s (1+ m) n))))))
```

defun replacePercentByDollar

— defun replacePercentByDollar —

```
(defun |replacePercentByDollar| (s)
  (|replacePercentByDollar,fn| s 0 (maxindex s)))
```

defun httpSetLabelInputString

— defun httpSetLabelInputString —

```
(defun |httpSetLabelInputString| (htPage label val)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (setf (elt props 0) (princ-to-string val)))))
```

defun httpLabelSpadValue

— defun httpLabelSpadValue —

```
(defun |httpLabelSpadValue| (htPage label)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (elt props 1))))
```

defun httpSetLabelSpadValue

— defun httpSetLabelSpadValue —

```
(defun |httpSetLabelSpadValue| (htPage label val)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (setf (elt props 1) val))))
```

defun httpLabelErrorMsg

— defun httpLabelErrorMsg —

```
(defun |httpLabelErrorMsg| (htPage label)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (elt props 2))))
```

defun httpSetLabelErrorMsg

— defun httpSetLabelErrorMsg —

```
(defun |httpSetLabelErrorMsg| (htPage label val)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (setf (elt props 2) val))))
```

defun httpLabelType

— defun httpLabelType —

```
(defun |httpLabelType| (htPage label)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (elt props 3))))
```

—————

defun httpLabelDefault

— defun httpLabelDefault —

```
(defun |httpLabelDefault| (htPage label)
  (let (msg props)
    (cond
      ((setq msg (|httpLabelInputString| htPage label))
        (cond
          ((equal msg "t") 1)
          ((equal msg "nil") 0)
          (t msg)))
      (t
        (setq props (lassoc label (|httpInputAreaAlist| htPage)))
        (when props (elt props 4))))))
```

—————

defun httpLabelSpadType

— defun httpLabelSpadType —

```
(defun |httpLabelSpadType| (htPage label)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (elt props 5))))
```

—————

defun httpLabelFilter

— defun httpLabelFilter —

```
(defun |httpLabelFilter| (htPage label)
  (let (props)
    (setq props (lassoc label (|httpInputAreaAlist| htPage)))
    (when props (elt props 6))))
```

defun httpPageDescription

— defun httpPageDescription —

```
(defun |httpPageDescription| (htPage)
  (elt htPage 7))
```

defun httpSetPageDescription

— defun httpSetPageDescription —

```
(defun |httpSetPageDescription| (htPage pageDescription)
  (setf (elt htPage 7) pageDescription))
```

defun httpAddToPageDescription

— defun httpAddToPageDescription 0 —

```
(defun |httpAddToPageDescription| (htPage pageDescrip)
  (let (newDescript)
    (setq newDescript
      (if (stringp pageDescrip)
          (cons pageDescrip (elt htPage 7))
          (nconc (nreverse (copy-list pageDescrip)) (elt htPage 7)))))
```

```
(setf (elt htPage 7) newDescript)))
```

defun issue a single hypertext line or group of lines

```
[mapStringize p1509]
[basicStringize p1510]
[$htLineList p1498]
[$newPage p1401]
```

— defun iht —

```
(defun |iht| (line)
  (declare (special |$htLineList| |$newPage|))
  (cond
    (|$newPage| nil)
    ((consp line)
     (setq |$htLineList|
           (nconc (nreverse (|mapStringize| (copy-list line))) |$htLineList|)))
    (t
     (setq |$htLineList| (cons (|basicStringize| line) |$htLineList|))))
```

defun bcHt

```
[htpAddToPageDescription p1507]
[mapStringize p1509]
[basicStringize p1510]
[$newPage p1401]
[$htLineList p1498]
[$curPage p??]
```

— defun bcHt —

```
(defun |bcHt| (line)
  (let (text)
    (declare (special |$newPage| |$htLineList| |$curPage|))
    (cond
      (|$newPage|
       (setq text
             (cond
              ((consp line) (list (cons '|text| line))))
```

```

      ((stringp line) line)
      (t (list (cons '|text| (list line))))))
    (|httpAddToPageDescription| |$curPage| text))
  ((consp line)
   (setq |$htLineList|
    (nconc (nreverse (|mapStringize| (copy-list line))) |$htLineList|)))
  (t
   (setq |$htLineList| (cons (|basicStringize| line) |$htLineList|))))))

```

defun htSay

[bcHt p1508]

— defun htSay —

```

(defun |htSay| (&rest args)
  (|bcHt| (car args))
  (loop for y in (cdr args) do (|bcHt| y)))

```

defun bcIssueHt

— defun bcIssueHt —

```

(defun |bcIssueHt| (line)
  (cond ((consp line) (|htMakePage1| line)) (t (|iht| line))))

```

defun mapStringize

[basicStringize p1510]

[mapStringize p1509]

— defun mapStringize —

```

(defun |mapStringize| (z)
  (cond
   ((atom z) z)

```

```
(t (rplaca z (|basicStringize| (car z)))
  (rplacd z (|mapStringize| (cdr z))) z)))
```

defun basicStringize

— defun basicStringize 0 —

```
(defun |basicStringize| (s)
  (cond
    ((stringp s)
     (cond
       ((equal s "\\$") "\\%")
       ((equal s "{\\em $}") "{\\em \\%}")
       (t s)))
    ((eq s '$) "\\%")
    (t (princ-to-string s))))
```

defun stringize

— defun stringize 0 —

```
(defun |stringize| (s)
  (if (stringp s) s (princ-to-string s)))
```

defun htInitPage

This function creates a page with the given title. For example, a call might be:

```
(|htInitePage| |Differentiate Basic Command| nil)
```

which creates a blank page with the given title.

```
[htInitPageNoScroll p1511]
[htSayStandard p1511]
[$curPage p??]
```

— defun htInitPage —

```
(defun |htInitPage| (title propList)
  (declare (special |$curPage|))
  (|htInitPageNoScroll| propList title)
  (|htSayStandard| "\\beginscroll ")
  |$curPage|)
```

defun htInitPageNoScroll

```
[htpMakeEmptyPage p1467]
[htpName p1499]
[htSayStandard p1511]
[htSay p1509]
[$atLeastOneUnexposed p??]
[$curPage p??]
[$newPage p1401]
[$htLineList p1498]
```

— defun htInitPageNoScroll —

```
(defun |htInitPageNoScroll| (&rest args)
  (let (title (propList (car args)) (options (cdr args)))
    (declare (special |$atLeastOneUnexposed| |$curPage| |$newPage| |$htLineList|))
    ; reset every time a new page is initialized
    (setq |$atLeastOneUnexposed| nil)
    (setq title (car options))
    (setq |$curPage| (|htpMakeEmptyPage| propList))
    (setq |$newPage| t)
    (setq |$htLineList| nil)
    (when title
      (|htSayStandard| (list "\\begin{page}" (|htpName| |$curPage|) "}" ))
      (|htSay| title)
      (|htSayStandard| "}" ))
    |$curPage|))
```

defun htSayStandard

```
[htSayBind p1512]
```

— defun htSayStandard 0 —

```
(defun |htSayStandard| (&rest args)
```

```
(|htSayBind| (car args) (cdr args)))
```

defun htSayBind

[bcHt p1508]

— defun htSayBind —

```
(defun |htSayBind| (x options)
  (|bcHt| x)
  (loop for y in options do (|bcHt| y)))
```

defun htAddHeading

— defun htAddHeading —

```
(defun |htAddHeading| (title)
  (declare (special |$curPage|))
  (|htNewPage| title)
  |$curPage|)
```

defun htShowPage

— defun htShowPage —

```
(defun |htShowPage| ()
  (|htSayStandard| "\\endscroll")
  (|htShowPageNoScroll|))
```

defun show the page which has been computed

Until this point in page processing the page has been represented by a list of pages elements in reverse order. This function reverses the list so they appear in page display order.

— **defun htShowPageNoScroll** —

```
(defun |htShowPageNoScroll| ()
  (let (line)
    (declare (special |$htLineList| |$curPage| |$newPage|))
    (|htSayStandard| "\\autobuttons")
    (|httpSetPageDescription| |$curPage|
      (nreverse (|httpPageDescription| |$curPage|)))
    (setq |$newPage| nil)
    (setq |$htLineList| nil)
    (|htMakePage| (|httpPageDescription| |$curPage|))
    (setq line (apply #'concat (nreverse |$htLineList|)))
    (|issueHT| line)
    (|endHTPage|)))
```

defun make a page given the description in itemList

— **defun htMakePage** —

```
(defun |htMakePage| (itemList)
  (declare (special |$curPage| |$newPage|))
  (progn
    (cond
      (|$newPage| (|httpAddToPageDescription| |$curPage| itemList)))
    (|htMakePage1| itemList)))
```

defun htMakePage1

Make a page given the description in itemList

— **defun htMakePage1** —

```
(defun |htMakePage1| (itemList)
  (let (itemType items)
    (loop for u in itemList do
      (setq itemType '|text|)
      (setq items
```



```

(cond
  ((stringp u) u)
  ((atom u) (princ-to-string u))
  ((stringp (car u)) u)
  ((and (consp u) (eq (car u) '|text|)) (cdr u))
  (t
   (setq itemType (car u)) ; look up the tag for the next description
   (cdr u))))
(cond
  ((eq itemType '|text|) (|ht| items))
  ((eq itemType '|lispLinks|) (|htLispLinks| items))
  ((eq itemType '|lispmemoLinks|) (|htLispMemoLinks| items))
  ((eq itemType '|bcLinks|) (|htBcLinks| items))
  ((eq itemType '|bcLinksNS|) (|htBcLinks| items t))
  ((eq itemType '|bcLispLinks|) (|htBcLispLinks| items))
  ((eq itemType '|radioButtons|) (|htRadioButtons| items))
  ((eq itemType '|bcRadioButtons|) (|htBcRadioButtons| items))
  ((eq itemType '|inputStrings|) (|htInputStrings| items))
  ((eq itemType '|domainConditions|) (|htProcessDomainConditions| items))
  ((eq itemType '|bcStrings|) (|htProcessBcStrings| items))
  ((eq itemType '|toggleButtons|) (|htProcessToggleButtons| items))
  ((eq itemType '|bcButtons|) (|htProcessBcButtons| items))
  ((eq itemType '|doneButton|) (|htProcessDoneButton| items))
  ((eq itemType '|doitButton|) (|htProcessDoitButton| items))
  (t (|systemError| (list "unknown itemType" itemType)))))

```

defun htMakeErrorPage

— defun htMakeErrorPage —

```

(defun |htMakeErrorPage| (htPage)
  (prog (line)
    (declare (special |$curPage| |$htLineList| |$newPage|))
    (return
     (progn
      (setq |$newPage| nil)
      (setq |$htLineList| nil)
      (setq |$curPage| htPage)
      (|htMakePage| (|httpPageDescription| htPage))
      (setq line (apply #'CONCAT (NREVERSE |$htLineList|)))
      (|issueHT| line)
      (|endHTPage|))))))

```

— defun htQuote —

— defun htProcessToggleButtons —

```
(defun |htProcessToggleButtons| (buttons)
  (prog (message info defaultValue buttonName)
    (declare (special |$curPage|))
    (return
      (SEQ (progn
        (|iht| "\\newline\\indent{5}\\beginitems ")
        (DO ((G166302 buttons (CDR G166302))
            (G166286 nil))
          ((OR (ATOM G166302)
              (progn (setq G166286 (car G166302)) nil)
              (progn
                (progn
                  (setq message (car G166286))
                  (setq info (CADR G166286))
                  (setq defaultValue (CADDR G166286))
                  (setq buttonName (CADDRR G166286))
                  G166286)
                nil))
            nil)
          (SEQ (EXIT (progn
              (cond
                ((NULL (LASSOC buttonName
                  (|httpInputAreaAlist| |$curPage|)))
                  (|setUpDefault| buttonName
                    (cons '|button|
                      (cons defaultValue nil))))))
                (|iht| (cons
                  "\\item{\\em\\inputbox["
                    (cons
                      (|httpLabelDefault| |$curPage|
```

```

        buttonName)
      (cons "]" {
        (cons buttonName
          (cons
            "]" {\htbmfile{pick}}{\htbmfile{unpick}}\space{"
              nil))))))
      (|bcIssueHt| message)
      (|iht| "\space{")
      (|bcIssueHt| info))))
    (|iht| "\enditems\indent{0} "))))

```

defun htProcessBcButtons

— defun htProcessBcButtons —

```

(defun |htProcessBcButtons| (buttons)
  (prog (defaultValue buttonName k)
    (declare (special |$curPage|))
    (return
      (SEQ (DO ((G166328 buttons (CDR G166328)) (G166317 nil))
        ((OR (ATOM G166328)
          (progn (setq G166317 (car G166328)) nil)
          (progn
            (progn
              (setq defaultValue (car G166317))
              (setq buttonName (CADR G166317))
              G166317)
            nil))
          nil)
        (SEQ (EXIT (progn
          (cond
            ((NULL (LASSOC buttonName
              (|httpInputAreaAlist| |$curPage|)))
              (|setUpDefault| buttonName
                (cons '|button|
                  (cons defaultValue nil))))))
            (setq k
              (|httpLabelDefault| |$curPage|
                buttonName))
            (cond
              ((EQL k 0)
                (|iht| (cons "\off{"
                  (cons buttonName
                    (cons "]" nil))))))
              ((EQL k 1)

```

```

(|iht| (cons "\\on{"
  (cons buttonName
    (cons "}" nil))))))
(t
  (|iht| (cons "\\inputbox["
    (cons
      (|httpLabelDefault| |$curPage|
        buttonName)
      (cons "]"
        (cons buttonName
          (cons
            "{\\htbmfile{pick}}{\\htbmfile{unpick}}}"
            nil))))))))))

```

defun htProcessBcStrings

— defun htProcessBcStrings —

```

(defun |htProcessBcStrings| (strings)
  (PROG (numChars default stringName spadType filter mess2)
    (declare (special |$curPage|))
    (return
      (SEQ (DO ((g2 strings (CDR g2)) (G166343 nil))
        ((or (atom g2)
          (progn (setq G166343 (CAR g2)) nil)
          (progn
            (progn
              (setq numChars (car G166343))
              (setq default (cadr G166343))
              (setq stringName (caddr G166343))
              (setq spadType (caddr G166343))
              (setq filter (caddr G166343))
              G166343)
            nil))
          nil)
        (SEQ (EXIT (progn
          (setq mess2 "")
          (cond
            ((null (LASSOC stringName
              (|httpInputAreaAlist| |$curPage|)))
              (|setUpDefault| stringName
                (cons '|string|
                  (cons default
                    (cons spadType
                      (cons filter nil))))))))
          nil))))

```

```

(cond
  ((|httpLabelErrorMsg| |$curPage|
    stringName)
   (|iht| (cons
            "\\centerline{\\em "
            (cons
              (|httpLabelErrorMsg| |$curPage|
                stringName)
              (cons "}" nil))))
    (setq mess2
      (concat mess2 (|bcSadFaces|)))
    (|httpSetLabelErrorMsg| |$curPage|
      stringName nil)))
  (|iht| (cons "\\inputstring{"
              (cons stringName
                (cons "{"
                  (cons numChars
                    (cons "{"
                      (cons
                        (|httpLabelDefault|
                          |$curPage| stringName)
                        (cons "}" "
                          (cons mess2 nil))))))))))))))

```

defun bcSadFaces

— defun bcSadFaces —

```

(defun |bcSadFaces| ()
  "\\space{1}{\\em\\htbitmap{error}\\htbitmap{error}\\htbitmap{error}}")

```

defun htLispLinks

— defun htLispLinks —

```

(defun |htLispLinks| (&REST G166422 &AUX option links)
  (setq links (car G166422))
  (setq option (cdr G166422))
  (prog (t1 options indent message info func value call)

```

```

(return
  (SEQ (progn
    (setq t1 (|beforeAfter| ' |options| links))
    (setq links (car t1))
    (setq options (cadr t1))
    (setq indent (or (LASSOC ' |indent| options) 5))
    (|iht| "\\newline\\indent{")
    (|iht| (|stringize| indent))
    (|iht| "}\\beginitems")
    (DO ((G166403 links (CDR G166403)) (G166387 nil))
      ((or (atom G166403)
        (progn (setq G166387 (car G166403)) nil)
        (progn
          (progn
            (setq message (car G166387))
            (setq info (cadr G166387))
            (setq func (caddr G166387))
            (setq value (cdddd G166387))
            G166387)
          nil))
        nil)
      (SEQ (EXIT (progn
        (|iht| "\\item[")
        (setq call
          (cond
            ((IFCAR option)
              "\\lispmemolink")
            (t
              "\\lispdwnlink"))
          (|htMakeButton| call message
            (|mkCurryFun| func value))
          (|iht| (cons "]"\\space{}" nil))
          (|bcIssueHt| info))))
        (|iht| "\\enditems\\indent{0} "))))))

```

defun htLispMemoLinks

— defun htLispMemoLinks —

```
(defun |htLispMemoLinks| (links) (|htLispLinks| links t))
```

— defun htBcLinks —

— defun htBcLispLinks —

```
(defun |htBcLispLinks| (links)
  (prog (t1 options message info func value)
    (return
      (SEQ (progn
              (setq t1 (|beforeAfter| ' |options| links))
              (setq links (car t1))
              (setq options (cadr t1))
              (DO ((G166487 links (cdr G166487)) (G166474 nil))
                    ((or (atom G166487)
                          (progn (setq G166474 (car G166487)) nil)
                          (progn
```

```

      (progn
        (setq message (car G166474))
        (setq info (cadr G166474))
        (setq func (caddr G166474))
        (setq value (cddddr G166474))
        G166474)
      nil))
    nil)
  (SEQ (EXIT (progn
    (|htMakeButton| "\\lisplink"
      message
      (|mkCurryFun| func value))
    (|bcIssueHt| info)))))))))

```

defun beforeAfter

— defun beforeAfter —

```

(defun |beforeAfter| (x u)
  (prog (y r)
    (return
      (SEQ (cons (prog (G166514)
        (setq G166514 nil)
        (return
          (DO ((G166504 u (CDR G166504)))
            ((or (atom G166504)
              (progn
                (progn
                  (setq y (car G166504))
                  (setq r (cdr G166504))
                  G166504)
                nil)
              (null (NEQUAL x y)))
            (NREVERSEO G166514))
          (SEQ (EXIT (setq G166514 (cons y G166514))))))
        (cons r nil))))))

```

defun mkCurryFun

— defun mkCurryFun —


```

(defun |mkCurryFun| (fun val)
  (prog (name code)
    (return
      (progn
        (setq name (gentemp))
        (setq code
          (cons 'defun
            (cons name
              (cons '(arg)
                (cons
                  (cons 'apply
                    (cons (mkq fun)
                      (cons
                        (cons 'cons
                          (cons 'arg
                            (cons (mkq val) nil)))
                        nil)))
                  nil))))))
        (eval code)
        name))))

```

defun htRadioButtons

— defun htRadioButtons —

```

(defun |htRadioButtons| (G166546)
  (prog (groupName buttons boxesName message info buttonName defaultValue)
    (declare (special |$curPage|))
    (return
      (SEQ (progn
        (setq groupName (car G166546))
        (setq buttons (cdr G166546))
        (|httpSetRadioButtonAlist| |$curPage|
          (cons (cons groupName (|buttonNames| buttons))
            (|htRadioButtonAlist| |$curPage|)))
        (setq boxesName (gentemp))
        (|iht| (cons "\\newline\\indent{5}\\radioboxes{"
          (cons boxesName
            (cons
              "{\\htbmf{pick}}{\\htbmf{unpick}}\\beginitems "
              nil))))
        (setq defaultValue "1")
        (DO ((G166568 buttons (cdr G166568))
          (G166540 nil))
          ((or (atom G166568)

```

```

      (progn (setq G166540 (car G166568)) nil)
      (progn
        (progn
          (setq message (car G166540))
          (setq info (cadr G166540))
          (setq buttonName (caddr G166540))
          G166540)
        nil))
      nil)
    (SEQ (EXIT (progn
      (cond
        ((null (LASSOC buttonName
          (|httpInputAreaAlist| |$curPage|)))
          (|setUpDefault| buttonName
            (cons '|button|
              (cons defaultValue nil))))
        (setq defaultValue
          "0"))
      (|iht| (cons "\\item{\\em\\radiobox["
        (cons
          (|httpLabelDefault| |$curPage|
            buttonName)
          (cons "]"{"
            (cons buttonName
              (cons "}{"
                (cons boxesName
                  (cons
                    "\\space{"
                    nil))))))
          (|bcIssueHt| message)
          (|iht| "\\space{"))
          (|bcIssueHt| info))))
      (|iht| "\\enditems\\indent{0} "))))))

```

defun htBcRadioButtons

— defun htBcRadioButtons —

```

(defun |htBcRadioButtons| (G166594)
  (prog (groupName buttons boxesName message info buttonName defaultValue)
    (declare (special |$curPage|))
    (return
      (SEQ (progn
        (setq groupName (car G166594))
        (setq buttons (cdr G166594))

```

```

(|httpSetRadioButtonAlist| |$curPage|
  (cons (cons groupName (|buttonNames| buttons))
        (|httpRadioButtonAlist| |$curPage|)))
(setq boxesName (gentemp))
(|iht| (cons "\\radioboxes{"
  (cons boxesName
    (cons "}{"\\htbmf{pick}}{"\\htbmf{unpick}} "
      nil))))
(setq defaultValue "1")
(DO ((G166616 buttons (cdr G166616))
     (G166588 nil))
  ((or (atom G166616)
       (progn (setq G166588 (car G166616)) nil)
       (progn
         (progn
          (setq message (car G166588))
          (setq info (cadr G166588))
          (setq buttonName (caddr G166588))
          G166588)
         nil))
       nil)
  (SEQ (EXIT (progn
    (cond
      ((null (LASSOC buttonName
        (|httpInputAreaAlist| |$curPage|)))
        (|setUpDefault| buttonName
          (cons '|button|
            (cons defaultValue nil)))
        (setq defaultValue
          "0"))
      (t)
        (|ht| (cons
          "\\em\\radiobox["
          (cons
            (|httpLabelDefault| |$curPage|
              buttonName)
            (cons "]"
              (cons buttonName
                (cons "{"
                  (cons boxesName
                    (cons "]" nil))))))))
          (|bcIssueHt| message)
          (|iht| "\\space{ }")
          (|bcIssueHt| info))))))))

```

defun setUpDefault

— defun setUpDefault —

```
(defun |setUpDefault| (name props)
  (declare (special |$curPage|))
  (|htpAddInputAreaProp| |$curPage| name props))
```

defun buttonNames

— defun buttonNames —

```
(defun |buttonNames| (buttons)
  (prog (buttonName)
    (return
      (SEQ (prog (G166645)
        (setq G166645 nil)
        (return
          (DO ((G166651 buttons (cdr G166651))
            (G166637 nil))
            ((or (atom G166651)
              (progn (setq G166637 (car G166651)) nil)
              (progn
                (progn
                  (setq buttonName (caddr G166637))
                  G166637)
                nil))
              (NREVERSEO G166645))
            (SEQ (EXIT (setq G166645
              (cons buttonName G166645))))))))))))))
```

defun htInputStrings

— defun htInputStrings —

```
(defun |htInputStrings| (strings)
  (prog (mess1 numChars default stringName spadType filter mess2)
    (declare (special |$curPage|))
```

```

(return
  (SEQ (progn
    (|iht| "\\newline\\indent{5}\\beginitems ")
    (DO ((G166685 strings (cdr G166685))
        (G166665 nil))
      ((or (atom G166685)
        (progn (setq G166665 (car G166685)) nil)
        (progn
          (progn
            (setq mess1 (car G166665))
            (setq mess2 (cadr G166665))
            (setq numChars (caddr G166665))
            (setq default (caddr G166665))
            (setq stringName
              (car (cddddr G166665)))
            (setq spadType
              (cadr (cddddr G166665)))
            (setq filter (cddr (cddddr G166665)))
            G166665)
          nil))
        nil)
      (SEQ (EXIT (progn
        (cond
          ((null (LASSOC stringName
            (|httpInputAreaAlist| |$curPage|)))
            (|setUpDefault| stringName
              (cons '|string|
                (cons default
                  (cons spadType
                    (cons filter nil)))))))
          (cond
            ((|httpLabelErrorMsg| |$curPage|
              stringName)
              (|iht| (cons "\\centerline{\\em "
                (cons
                  (|httpLabelErrorMsg| |$curPage|
                    stringName)
                  (cons "}" nil))))
              (setq mess2
                (CONCAT mess2 (|bcSadFaces|)))
              (|httpSetLabelErrorMsg| |$curPage|
                stringName nil)))
            (|iht| "\\item ")
            (|bcIssueHt| mess1)
            (|iht| (cons "\\inputstring{"
              (cons stringName
                (cons "}"
                  (cons numChars
                    (cons "}"
                      (cons

```

```

(|httpLabelDefault| |$curPage|
  stringName)
  (cons "}" " nil)))))))))
(|bcIssueHt| mess2))))))
(|iht| "\\enditems\\indent{0}\\newline "))))))

```

defun htProcessDomainConditions

— defun htProcessDomainConditions —

```

(defun |htProcessDomainConditions| (condList)
  (declare (special |$curPage|))
  (|httpSetDomainConditions| |$curPage| (|renamePatternVariables| condList))
  (|httpSetDomainVariableAList| |$curPage| (|computeDomainVariableAList|)))

```

defun renamePatternVariables

— defun renamePatternVariables —

```

(defun |renamePatternVariables| (condList)
  (declare (special |$curPage| |$PatternVariableList|))
  (progn
    (|httpSetDomainPvarSubstList| |$curPage|
      (|renamePatternVariables1| condList nil
        |$PatternVariableList|))
    (|substFromAList| condList (|httpDomainPvarSubstList| |$curPage|))))

```

defun renamePatternVariables1

— defun renamePatternVariables1 —

```

(defun |renamePatternVariables1| (condList substList patVars)
  (prog (restConds pattern t2 pv t3 cond nsubst)
    (declare (special |$EmptyMode|))

```

```

(return
  (cond
    ((null condList) substList)
    (t (setq cond (car condList))
      (setq restConds (cdr condList))
      (cond
        ((or (and (consp cond) (eq (qcar cond) '|isDomain|)
          (progn
            (setq t2 (qcdr cond))
            (and (consp t2)
              (progn
                (setq pv (qcar t2))
                (setq t3 (qcdr t2))
                (and (consp t3)
                  (eq (qcdr t3) nil)
                  (progn
                    (setq pattern
                      (qcar t3))
                    t)))))))
          (and (consp cond) (eq (qcar cond) '|ofCategory|)
            (progn
              (setq t2 (qcdr cond))
              (and (consp t2)
                (progn
                  (setq pv (qcar t2))
                  (setq t3 (qcdr t2))
                  (and (consp t3)
                    (eq (qcdr t3) nil)
                    (progn
                      (setq pattern
                        (qcar t3))
                      t)))))))
          (and (consp cond) (eq (qcar cond) '|Satisfies|)
            (progn
              (setq t2 (qcdr cond))
              (and (consp t2)
                (progn
                  (setq pv (qcar t2))
                  (setq t3 (qcdr t2))
                  (and (consp t3)
                    (eq (qcdr t3) nil)
                    (progn
                      (setq cond (qcar t3))
                      t)))))))
          (cond
            ((equal pv |$EmptyModel|)
              (setq nsubst substList))
            (t
              (setq nsubst
                (cons (cons pv (car patVars)) substList))))))
  )

```

```
(|renamePatternVariables1| restConds nsubst
  (cdr patVars)))
(t substList))))))
```

defun substFromAlist

— defun substFromAlist —

```
(defun |substFromAlist| (z substAlist)
  (prog (pvar replace)
    (return
      (SEQ (progn
        (DO ((G166792 substAlist (cdr G166792))
              (G166783 nil))
          ((or (atom G166792)
              (progn (setq G166783 (car G166792)) nil)
              (progn
                (progn
                  (setq pvar (car G166783))
                  (setq replace (cdr G166783))
                  G166783)
                nil))
              nil)
          (SEQ (EXIT (setq z (subst replace pvar z :test #'equal))))
            z))))))
```

defun computeDomainVariableAlist

— defun computeDomainVariableAlist —

```
(defun |computeDomainVariableAlist| ()
  (prog (pvar)
    (declare (special |$curPage|))
    (return
      (SEQ (prog (G166813)
        (setq G166813 nil)
        (return
          (DO ((G166819 (|httpDomainPvarSubstList| |$curPage|)
                  (cdr G166819))
```



```

(G166805 NIL))
((or (atom G166819)
     (progn (setq G166805 (car G166819)) nil)
     (progn
      (progn
       (setq pvar (cdr G166805))
       G166805)
      NIL))
     (NREVERSEO G166813))
 (SEQ (EXIT (setq G166813
                  (cons (cons pvar
                              (|pvarCondList| pvar))
                        G166813)))))))))

```

defun pvarCondList

— defun pvarCondList —

```

(defun |pvarCondList| (pvar)
  (declare (special |$curPage|))
  (NREVERSE
   (|pvarCondList1| (cons pvar nil) nil
                     (|httpDomainConditions| |$curPage|))))

```

defun pvarCondList1

— defun pvarCondList1 —

```

(defun |pvarCondList1| (pvarList activeConds condList)
  (prog (cond restConds t2 pv t3 pattern)
    (return
     (cond
      ((null condList) activeConds)
      (t (setq cond (car condList))
          (setq restConds (cdr condList))
          (cond
           ((and (consp cond)
                 (progn
                  (setq t2 (qcdr cond))

```

```

      (and (consp t2)
        (progn
          (setq pv (qcar t2))
          (setq t3 (qcdr t2))
          (and (consp t3)
            (eq (qcdr t3) nil)
            (progn
              (setq pattern (qcar t3))
              t))))))
      (|member| pv pvarList))
    (|pvarCondList1|
      (NCONC pvarList (|pvarsOfPattern| pattern))
      (cons cond activeConds) restConds))
    (t (|pvarCondList1| pvarList activeConds restConds))))))

```

defun pvarsOfPattern

— defun pvarsOfPattern —

```

(defun |pvarsOfPattern| (pattern)
  (prog ()
    (declare (special |$PatternVariableList|))
    (return
      (SEQ (cond
        ((null (listp pattern)) nil)
        (t
          (prog (G166869)
            (setq G166869 nil)
            (return
              (DO ((G166875 (cdr pattern) (cdr G166875))
                (pvar nil))
                ((or (atom G166875)
                  (progn (setq pvar (car G166875)) nil))
                (NREVERSEO G166869))
              (SEQ (EXIT (cond
                ((|member| pvar
                  |$PatternVariableList|)
                  (setq G166869
                    (cons pvar G166869))))))))))))))

```

defun htMakeTemplates,substLabel

— defun htMakeTemplates,substLabel —

```
(defun |htMakeTemplates,substLabel| (i template)
  (SEQ (if (consp template)
    (EXIT (intern (CONCAT (car template) (princ-to-string i)
      (cdr template)))))
    (EXIT template)))
```

defun htMakeTemplates

— defun htMakeTemplates —

```
(defun |htMakeTemplates| (templateList numLabels)
  (prog ()
    (return
      (SEQ (progn
        (setq templateList
          (prog (G166895)
            (setq G166895 nil)
            (return
              (DO ((G166900 templateList
                (CDR G166900))
                (template nil))
                ((or (atom G166900)
                  (progn
                    (setq template (car G166900))
                    nil))
                (NREVERSE0 G166895))
              (SEQ (EXIT (setq G166895
                (cons
                  (|templateParts| template)
                  G166895))))))))))
        (prog (G166910)
          (setq G166910 nil)
          (return
            (DO ((i 1 (1+ i)))
              ((qsgreaterp i numLabels)
                (NREVERSE0 G166910))
              (SEQ (EXIT (setq G166910
                (cons
                  (prog (G166922)
```

```

(setq G166922 nil)
(return
  (DO
    ((G166927 templateList
      (CDR G166927))
     (template nil))
    ((or (atom G166927)
      (progn
        (setq template
          (car G166927))
        nil))
      (NREVERSEO G166922))
    (SEQ
      (EXIT
        (setq G166922
          (cons
            (|htMakeTemplates,substLabel|
              i template)
            G166922)))))))))
G166910)))))))))

```

defun templateParts

— defun templateParts —

```

(defun |templateParts| (template)
  (prog (i)
    (return
      (cond
        ((null (stringp template)) template)
        (t (setq i (SEARCH "%1" template))
          (cond
            ((null i) template)
            (t
              (cons (SUBSEQ template 0 i)
                (SUBSEQ template (+ i 2))))))))))

```

defun htMakeDoneButton

— defun htMakeDoneButton —

```
(defun |htMakeDoneButton| (message func)
  (progn
    (|bcHt| "\\newline\\vspace{1}\\centerline{")
    (cond
      ((equal message "Continue")
        (|bchtMakeButton| "\\lispdownlink"
          '|\\ContinueBitmap| func))
      (t
        (|bchtMakeButton| "\\lispdownlink"
          (CONCAT "\\box{" message "}")
          func)))
    (|bcHt| "}" ")))
```

defun htProcessDoneButton

— defun htProcessDoneButton —

```
(defun |htProcessDoneButton| (arg)
  (let (label func)
    (setq label (car arg))
    (setq func (cadr arg))
    (|iht| "\\newline\\vspace{1}\\centerline{")
    (cond
      ((equal label "Continue")
        (|htMakeButton| "\\lispdownlink" '|\\ContinueBitmap| func))
      ((equal label "Push to enter names")
        (|htMakeButton| "\\lispdownlink" "\\ControlBitmap{clicktaset}" func))
      (t
        (|htMakeButton| "\\lispdownlink" (concat "\\box{" label "}") func)))
    (|iht| "}" ")))
```

defun htMakeButton

— defun htMakeButton —

```
(defun |htMakeButton| (&rest arg)
  (let (options func message htCommand)
    (setq htCommand (car arg))
    (setq message (cadr arg))
```

```

(setq func (caddr arg))
(setq options (cddddr arg))
(prog (skipStateInfo? id type)
(declare (special |$curPage|))
(return
  (SEQ (progn
    (setq skipStateInfo? (ifcar options))
    (|iht| (cons htCommand (cons "{" nil)))
    (|bcIssueHt| message)
    (cond
      (skipStateInfo?
        (|iht| (cons "}{(|htDoneButton| '|"
          (cons func
            (cons "| "
              (cons (|httpName| |$curPage|)
                (cons "}" nil))))))))
      (t
        (|iht| (cons "}{(|htDoneButton| '|"
          (cons func
            (cons "| (progn " nil))))
          (DO ((G166977 (|httpInputAreaAlist| |$curPage|)
            (CDR G166977))
              (G166965 nil))
            ((OR (ATOM G166977)
              (progn (setq G166965 (car G166977)) nil)
              (progn
                (progn
                  (setq id (car G166965))
                  (setq type (car (cddddr G166965)))
                  G166965)
                nil))
              nil))
            (SEQ (EXIT (progn
              (|iht| (cons "(|httpSetLabelInputString| "
                (cons (|httpName| |$curPage|)
                  (cons "'|"
                    (cons id
                      (cons "| " nil))))))
              (cond
                ((eq type '|string|)
                  (|iht| (cons "\"\\stringvalue{"
                    (cons id
                      (cons "}\\\""
                        nil))))))
                (t
                  (|iht| (cons
                    "\"\\boxvalue{"
                    (cons id
                      (cons "}\\\""
                        nil))))))
              nil))))
            nil))))
          nil)
    (SEQ (EXIT (progn
      (|iht| (cons "(|httpSetLabelInputString| "
        (cons (|httpName| |$curPage|)
          (cons "'|"
            (cons id
              (cons "| " nil))))))
      (cond
        ((eq type '|string|)
          (|iht| (cons "\"\\stringvalue{"
            (cons id
              (cons "}\\\""
                nil))))))
        (t
          (|iht| (cons
            "\"\\boxvalue{"
            (cons id
              (cons "}\\\""
                nil))))))
      nil))))
    nil))))
  )

```

```

(|iht| " " "))))
(|iht| (cons (|htpName| |$curPage|)
              (cons "}") nil)))))))))

```

defun bchtMakeButton

— defun bchtMakeButton —

```

(defun |bchtMakeButton| (htCommand message func)
  (prog (id type)
    (declare (special |$curPage|))
    (return
      (SEQ (progn
        (|bcHt| (cons htCommand
                      (cons "{"
                          (cons message
                              (cons "}{(|htDoneButton| '|"
                                  (cons func
                                      (cons "| (progn "
                                          nil)))))))
          (DO ((G167004 (|htpInputAreaAlist| |$curPage|)
                  (cdr G167004))
              (G166992 nil))
            ((or (atom G167004)
                 (progn (setq G166992 (car G167004)) nil)
                 (progn
                   (progn
                     (setq id (car G166992))
                     (setq type (car (cddddr G166992)))
                     G166992)
                   nil))
              nil)
          (SEQ (EXIT (progn
            (|bcHt| (cons "(|htpSetLabelInputString| "
                          (cons (|htpName| |$curPage|)
                              (cons "'|"
                                  (cons id
                                      (cons "| " nil))))))
              (cond
                ((eq type '|string|)
                 (|bcHt| (cons
                           "\"\\stringValue{"
                           (cons id
                               (cons "}" nil))))
                  (t

```

```

(|bcHt| (cons
  "\\boxvalue{"
  (cons id
    (cons "}\" nil))))))
(|bcHt| " ) "))))
(|bcHt| (cons (|htpName| |$curPage|
  (cons ")} \" nil))))))

```

defun htProcessDoitButton

— defun htProcessDoitButton —

```

(defun |htProcessDoitButton| (arg)
  (let (label command func fun)
    (setq label (car arg))
    (setq command (cadr arg))
    (setq func (caddr arg))
    (setq fun (|mkCurryFun| func (cons command nil)))
    (|iht| "\\newline\\vspace{1}\\centerline{")
    (|htMakeButton| "\\lispcommand" (concat "\\box{" label "}") fun)
    (|iht| "} ")
    (|iht| "\\vspace{2}{Select \\ \\UpButton{} \\ to go back one page.}")
    (|iht|
      "\\newline{Select \\ \\ExitButton{QuitPage} \\ to remove this window.}"))

```

defun htDoneButton

— defun htDoneButton —

```

(defun |htDoneButton| (func htPage)
  (cond
    ((|typeCheckInputAreas| htPage) (|htMakeErrorPage| htPage))
    ((null (fboundp func))
      (|systemError| (cons "unknown function" (cons func nil))))
    (t (funcall (symbol-function func) htPage))))

```

— defun typeCheckInputAreas —

```
(defun |typeCheckInputAreas| (htPage)
  (prog (inputAlist stringName t2 t3 t4 t5 t6 t7 spadType t8 filter
    condList string val errorCondition)
    (declare (special |$bcParseOnly|))
    (return
      (SEQ (progn
        (setq inputAlist nil)
        (setq errorCondition nil)
        (DO ((G167160 (|httpInputAreaAlist| htPage)
          (cdr G167160))
          (entry nil))
          ((or (atom G167160)
            (progn (setq entry (car G167160)) nil))
            nil)
          (SEQ (EXIT (cond
            ((and (consp entry)
              (progn
                (setq stringName
                  (QCAR entry))
                (setq t2 (QCDR entry))
                (and (consp t2)
                  (progn
                    (setq t3
                      (QCDR t2))
                    (and (consp t3)
                      (progn
                        (setq t4
                          (QCDR t3))
                        (and (consp t4)
                          (progn
                            (setq t5
                              (QCDR t4))
                            (and (consp t5)
                              (eq (QCAR t5)
                                '|string|)
                              (progn
                                (setq t6
                                  (QCDR t5))
                                (and (consp t6)
                                  (progn
                                    (setq t7
                                      (QCDR t6))
                                    (and
                                      (consp t7)
                                      (progn
```

```

                                (setq
                                  spadType
                                  (QCAR t7))
                                (setq
                                  t8
                                  (QCDR t7))
                                (and
                                  (consp
                                    t8)
                                  (eq
                                    (QCDR
                                      t8)
                                    nil)
                                  (progn
                                    (setq
                                      filter
                                      (QCAR
                                        t8))
                                    t)))))))))
(progn
  (setq condList
    (LASSOC
      (LASSOC spadType
        (|httpDomainPvarSubstList|
          htPage))
      (|httpDomainVariableAlist|
        htPage)))
  (setq string
    (|httpLabelFilteredInputString|
      htPage stringName))
  (cond
    (|$bcParseOnly|
      (cond
        ((null
          (|ncParseFromString| string))
          (|httpSetLabelErrorMsg| htPage
            "Syntax Error"
            "Syntax Error"))
        (t nil)))
    (t
      (setq val
        (|checkCondition|
          (|httpLabelInputString|
            htPage stringName)
          string condList))
      (cond
        ((stringp val)
          (setq errorCondition t)
          (|httpSetLabelErrorMsg| htPage
            stringName val))

```

```

(t
  (|httpSetLabelSpadValue| htPage
    stringName val)))))))))
errorCondition))))))

```

defun checkCondition

— defun checkCondition —

```

(defun |checkCondition| (s1 string condList)
  (prog (pred t2 t3 pvar t4 pattern val type data newType)
    (return
      (cond
        ((and (consp condList) (eq (QCDR condList) nil)
          (progn
            (setq t2 (qcar condList))
            (and (consp t2)
              (eq (QCAR t2) '|Satisfies|)
              (progn
                (setq t3 (QCDR t2))
                (and (consp t3)
                  (progn
                    (setq pvar (QCAR t3))
                    (setq t4 (QCDR t3))
                    (AND (consp t4)
                      (eq (QCDR t4) nil)
                      (progn
                        (setq pred (QCAR t4))
                        t))))))))))
          (setq val (funcall pred string))
          (cond
            ((stringp val) val)
            (t (cons '(|String|) (|wrap| s1))))))
        ((null (and (consp condList) (eq (qcdr condList) nil)
          (progn
            (setq t2 (qcar condList))
            (and (consp t2)
              (eq (qcar t2) '|isDomain|)
              (progn
                (setq t3 (QCDR t2))
                (and (consp t3)
                  (progn
                    (setq pvar (QCAR t3))
                    (setq t4
                      (QCDR t3))

```

```

                                (and (consp t4)
                                    (eq (QCDR t4) nil)
                                    (progn
                                     (setq pattern
                                      (QCAR t4))
                                     t)))))))))
(|systemError|
 "currently invalid domain condition"))
(equal pattern '(|String|))
(cons '(|String|) (|wrap| s1)))
(t (setq val (|parseAndEval| string))
 (cond
  ((stringp val)
   (cond
    ((equal val "Syntax Error ")
     "Error: Syntax Error ")
    (t (|condErrorMsg| pattern))))
  (t (setq type (car val))
      (setq data (cdr val))
      (setq newType
               (catch 'spad_reader
                    (|resolveTM| type pattern)))
      (cond
       ((null newType) (|condErrorMsg| pattern))
       (t (|coerceInt| val newType))))))))))

```

defun condErrorMsg

— defun condErrorMsg —

```

(defun |condErrorMsg| (type)
  (prog (typeString)
    (return
     (progn
      (setq typeString (|form2String| type))
      (cond
       ((consp typeString)
        (setq typeString
                 (apply #'CONCAT typeString))))
      (CONCAT "Error: Could not make your input into a "
              typeString))))))

```

defun parseAndEval

— defun parseAndEval —

```
(defun |parseAndEval| (string)
  (prog (|$InteractiveMode| $boot $spad |$e| |$QuietCommand|)
    (declare (special |$InteractiveMode| $boot $spad |$e|
                    |$QuietCommand|))
    (return
      (progn
        (setq |$InteractiveMode| t)
        (setq $boot nil)
        (setq $spad t)
        (setq |$e| |$InteractiveFrame|)
        (setq |$QuietCommand| t)
        (|parseAndEval1| string))))))
```

defun parseAndEval1

— defun parseAndEval1 —

```
(defun |parseAndEval1| (string)
  (let (v syntaxError pform val)
    (setq syntaxError nil)
    (setq pform
      (progn
        (setq v
          (|applyWithOutputToString| '|ncParseFromString| (cons string nil)))
        (cond
          ((car v) (car v))
          (t (setq syntaxError t) (cdr v))))))
    (cond
      (syntaxError "Syntax Error ")
      (pform
        (setq val
          (|applyWithOutputToString| '|processInteractive|
            (cons pform (list nil))))
        (cond
          ((car val) (car val))
          (t "Type Analysis Error")))
      (t nil))))
```

defun oldParseString

— defun oldParseString —

```

(defun |oldParseString| (string)
  (prog (tree)
    (return
      (progn
        (setq tree
          (|applyWithOutputToString| '|string2SpadTree|
            (cons string nil)))
        (cond
          ((car tree)
            (|parseTransform| (postTransform (car tree))))
          (t (cdr tree)))))))

```

defun makeSpadCommand

— defun makeSpadCommand —

```

(defun |makeSpadCommand| (&rest a1)
  (let (opForm lastArg argList z)
    (setq z a1)
    (setq opForm (concat (car z) "("))
    (setq lastArg (|last| z))
    (setq z (cdr z))
    (setq argList nil)
    (do ((g1 z (cdr g1)) (arg nil))
      ((or (atom g1)
          (progn (setq arg (car g1)) nil)
          (null (nequal arg lastArg))
          nil)
        (setq argList (cons (concat arg ", ") argList)))
      (setq argList (nreverse (cons lastArg argList)))
      (concat opForm (apply #'concat argList) ")"))))

```

defun htMakeInputList

— defun htMakeInputList —

```

(defun |htMakeInputList| (stringList)
  (prog (lastArg argList)
    (return
      (SEQ (progn
              (setq lastArg (|last| stringList))
              (setq argList nil)
              (DO ((G167328 stringList (cdr G167328)) (arg nil))
                  ((or (atom G167328)
                       (progn (setq arg (car G167328)) nil)
                       (null (NEQUAL arg lastArg))))
                nil)
            (SEQ (EXIT (setq argList
                              (cons
                               (CONCAT arg ", ")
                               argList))))))
              (setq argList (NREVERSE (cons lastArg argList)))
              (|bracketString| (apply #'CONCAT argList))))))

```

defun bracketString

— defun bracketString —

```

(defun |bracketString| (string)
  (CONCAT "[" string "]"))

```

defun quoteString

— defun quoteString —

```

(defun |quoteString| (string)
  (concat "\"" string "\""))

```

defvar \$funnyQuote

— initvars —

```
(defvar |$funnyQuote| #\Rubout)
```

defvar \$funnyBacks

— initvars —

```
(defvar |$funnyBacks| #\\200)
```

defun htEscapeString

— defun htEscapeString —

```
(defun |htEscapeString| (str)
  (declare (special |$funnyBacks| |$funnyQuote|))
  (setq str (substitute |$funnyQuote| #" str))
  (substitute |$funnyBacks| #\\ str))
```

defun htsv

— defun htsv —

```
(defun |htsv| ()
  (|startHTPage| 50)
  (|htSetVars|))
```

defun htSetVars

— defun htSetVars —


```
(elt setData 1)))
      maxWidth2)))))))))
(setq maxWidth1 (max 9 maxWidth1))
(setq maxWidth2 (max 41 maxWidth2))
(setq tabset1 (PRINC-TO-STRING maxWidth1))
(setq tabset2
  (PRINC-TO-STRING
    (-
      (+ maxWidth2 maxWidth1) 1)))
(|htSay| "\\tab{2}\\newline Variable\\tab{"
  (PRINC-TO-STRING
    (+ maxWidth1
      (quotient maxWidth2 3)))
    "}Description\\tab{"
    (PRINC-TO-STRING
      (+ (+ maxWidth2 maxWidth1) 2))
      "}Value\\newline\\beginitems ")
(DO ((G167392 (reverse okList) (CDR G167392))
     (setData nil))
   ((or (atom G167392)
        (progn
          (setq setData (car G167392))
          nil))
    nil)
  (SEQ (EXIT (progn
    (|htSay| "\\item")
    (setq label
      (CONCAT "\\menuitemstyle{"
        (elt setData 0)
        "}))
    (setq links
      (cons label
        (cons
          (cons
            (cons '|text|
              (cons
                "\\tab{"
                (cons tabset1
                  (cons "]"
                    (cons
                      (elt setData 1)
                      (cons "\\tab{"
                        (cons tabset2
                          (cons "}{"\\em "
                            (cons
                              (|htShowSetTreeValue|
                                setData)
                                (cons
                                  "]"
                                  nil))))))))))))))
```

```

nil)
(cons '|htShowSetPage|
  (cons (elt setData 0)
    nil))))))
(|htMakePage|
  (cons
    (cons '|bcLispLinks|
      (cons links
        (cons '|options|
          (cons '(|indent| . 0) nil))))
      nil))))))
(|htSay| "\\enditems") (|htShowPage|))))))

```

defun htShowCount

— defun htShowCount —

```

(defun |htShowCount| (s)
  (prog (m i count)
    (return
      (SEQ (progn
        (setq m (|#| s))
        (cond
          ((> 8 m) (- m 1))
          (t (setq i 0) (setq count 0)
            (DO () ((NULL (> (- m 7) i)) nil)
              (SEQ (EXIT (cond
                ((and (equal (elt s i) #\{)
                  (equal (elt s (1+ i)) #\\)
                  (equal (elt s (+ i 2)) #\e)
                  (equal (elt s (+ i 3)) #\m))
                (setq i (+ i 6)))
                (t (setq i (1+ i))
                  (setq count (1+ count))))))))
              (+ count (- m i))))))))))

```

defun htShowSetTreeValue

— defun htShowSetTreeValue —

```
(defun |htShowSetTreeValue| (setData)
  (let (st)
    (setq st (elt setData 3))
    (cond
      ((eq st 'function)
        (|object2String| (funcall (elt setData 4) '|%display%|)))
      ((eq st 'integer)
        (|object2String| (|eval| (elt setData 4))))
      ((eq st 'string)
        (|object2String| (|eval| (elt setData 4))))
      ((eq st 'literals)
        (|object2String| (|translateTrueFalse2YesNo| (|eval| (elt setData 4)))))
      ((eq st 'tree) "...")
      (t (|systemError|))))
```

defun mkSetTitle

— defun mkSetTitle —

```
(defun |mkSetTitle| ()
  (declare (special |$path|))
  (concat "Command {\em }set " (|listOfStrings2String| |$path|) "}"))
```

defun listOfStrings2String

— defun listOfStrings2String —

```
(defun |listOfStrings2String| (u)
  (cond
    ((null u) "")
    (t (concat (|listOfStrings2String| (cdr u)) " " (|stringize| (car u))))))
```

defun htShowSetPage

— defun htShowSetPage —

```

(defun |htShowSetPage| (htPage branch)
  (let (setTree setData st)
    (declare (special |$path|))
    (setq setTree (|httpProperty| htPage '|setTree|))
    (setq |$path| (cons branch (take (- (lastatom setTree)) |$path|)))
    (setq setData (|assoc| branch setTree))
    (cond
      ((null setData) (|systemError| "No Set Data"))
      (t (setq st (elt setData 3))
         (cond
           ((eq st 'function) (|htShowFunctionPage| htPage setData))
           ((eq st 'integer) (|htShowIntegerPage| htPage setData))
           ((eq st 'literals) (|htShowLiteralsPage| htPage setData))
           ((eq st 'tree) (|htShowSetTree| (elt setData 5)))
           ((eq st 'string) (|htSetNotAvailable| htPage ")set compiler"))
           (t (|systemError| "Unknown data type"))))))))

```

defun htShowLiteralsPage

— defun htShowLiteralsPage —

```

(defun |htShowLiteralsPage| (htPage setData)
  (|htSetLiterals| htPage (elt setData 0) (elt setData 1)
    (elt setData 4) (elt setData 5) '|htSetLiteral|))

```

defun htSetLiterals

— defun htSetLiterals —

```

(defun |htSetLiterals| (htPage name message variable values functionToCall)
  (prog (page links)
    (return
      (SEQ (progn
              (setq page
                (|htInitPage| "Set Command"
                  (|httpPropertyList| htPage)))
              (|httpSetProperty| page '|variable| variable)
              (|bcHt| (cons "\\centerline{Set {\\em "
                (cons name

```

```

                                (cons "}}\\newline" nil))))
(|bcHt| (cons "{\\em Description: } "
              (cons message
                    (cons "\\newline\\vspace{1} "
                        nil))))))
(|bcHt| "Select one of the following: \\newline\\tab{3} ")
(setq links
  (prog (g2)
    (setq g2 nil)
    (return
      (DO ((G167465 values (cdr G167465))
          (opt nil))
        ((or (atom G167465)
              (progn
                (setq opt (car G167465))
                nil))
          (NREVERSE0 g2))
        (SEQ (EXIT (setq g2
                        (cons
                          (cons
                            (CONCAT ""
                              (PRINC-TO-STRING opt))
                            (cons "\\newline\\tab{3}"
                                (cons functionToCall
                                    (cons opt nil))))
                          g2)))))))
(|htMakePage| (cons (cons '|bcLispLinks| links) nil))
(|bcHt|
  (cons
    '|\\indent{0}\\newline\\vspace{1} The current setting is: {\\em |
      (cons (|translateTrueFalse2YesNo|
              (eval variable))
            (cons "}" " nil))))
(|htShowPage|))))))

```

defun htSetLiteral

— defun htSetLiteral —

```

(defun |htSetLiteral| (htPage val)
  (|htInitPage| "Set Command" nil)
  (set (|httpProperty| htPage '|variable|) (|translateYesNo2TrueFalse| val))
  (|htKill| htPage val))

```

defun htShowIntegerPage

— defun htShowIntegerPage —

```
(defun |htShowIntegerPage| (htPage setData)
  (prog (page message t1)
    (declare (special |$htFinal| |$htInitial|))
    (return
      (progn
        (setq page
          (|htInitPage| (|mkSetTitle|
            (|httpPropertyList| htPage)))
          (|httpSetProperty| page '|variable| (elt setData 4))
          (|bcHt| (list "\\centerline{Set {\\em " (elt setData 0) "}}\\newline"))
          (setq message (elt setData 1))
          (|bcHt| (list "{\\em Description: } " message "\\newline\\vspace{1} "))
          (setq t1 (elt setData 5))
          (setq |$htInitial| (car t1))
          (setq |$htFinal| (cadr t1))
          (cond
            ((equal |$htFinal| (+ |$htInitial| 1))
              (|bcHt| "Enter the integer {\\em ")
              (|bcHt| (|stringize| |$htInitial|))
              (|bcHt| "} or {\\em ")
              (|bcHt| (|stringize| |$htFinal|))
              (|bcHt| "}:"))
            ((null |$htFinal|)
              (|bcHt| "Enter an integer greater than {\\em ")
              (|bcHt| (|stringize| (- |$htInitial| 1)))
              (|bcHt| "}:"))
            (t (|bcHt| "Enter an integer between {\\em ")
              (|bcHt| (|stringize| |$htInitial|))
              (|bcHt| "} and {\\em ")
              (|bcHt| (|stringize| |$htFinal|))
              (|bcHt| "}:"))))
          (|htMakePage|
            (cons '(|domainConditions| (|Satisfies| S chkRange))
              (cons (cons '|bcStrings|
                (list (list 5 (|eval| (elt setData 4)) '|value| 'S)))
                  nil)))
          (|htSetvarDoneButton| "Select to Set Value" '|htSetInteger|)
          (|htShowPage|))))))
```

defun htSetInteger

— defun htSetInteger —

```
(defun |htSetInteger| (htPage)
  (prog (val)
    (return
      (progn
        (|htInitPage| (|mkSetTitle|) nil)
        (setq val
          (|chkRange| (|httpLabelInputString| htPage '|value|)))
        (cond
          ((null (integerp val))
           (|errorPage| htPage
            (cons "Value Error"
                  (cons nil
                        (cons "\\vspace{3}\\centerline{\\em "
                              (cons val
                                    (cons
                                      (cons
                                        "\\vspace{2}\\newline\\centerline{Click on \\UpBitmap{} to re-enter value}"
                                        nil)))))))))
          (t (set (|httpProperty| htPage '|variable|) val)
              (|htKill| htPage val)))))))
```

—————

defun htShowFunctionPage

— defun htShowFunctionPage —

```
(defun |htShowFunctionPage| (htPage setData)
  (prog (fn)
    (return
      (cond
        ((setq fn (elt setData 6)) (funcall fn htPage))
        (t (|httpSetProperty| htPage '|setData| setData)
           (|httpSetProperty| htPage '|parts| (elt setData 5))
           (|htShowFunctionPageContinued| htPage))))))
```

—————


```

                                (cons 'S nil))))))
                                nil))
                                nil))))
(|htSetvarDoneButton| "Select To Set Value" '|htSetFunCommand|)
(|htShowPage|))))))

```

defun htSetvarDoneButton

— defun htSetvarDoneButton —

```

(defun |htSetvarDoneButton| (message func)
  (progn
    (|bcHt| "\\newline\\vspace{1}\\centerline{")
    (cond
      ((OR (equal message "Select to Set Value")
           (equal message "Select to Set Values"))
        (|bchtMakeButton| "\\lisplink"
                          "\\ControlBitmap{clicktoreset}" func))
      (t
        (|bchtMakeButton| "\\lisplink"
                          (CONCAT "\\fbox{" message "}")
                          func)))
    (|bcHt| "}" ")))

```

defun htFunctionSetLiteral

— defun htFunctionSetLiteral —

```

(defun |htFunctionSetLiteral| (htPage val)
  (progn
    (|htInitPage| "Set Command" nil)
    (set (|httpProperty| htPage '|variable|)
         (|translateYesNo2TrueFalse| val))
    (|htSetFunCommandContinue| htPage val)))

```

defun htSetFunCommand

— defun htSetFunCommand —

```
(defun |htSetFunCommand| (htPage)
  (let (variable checker value)
    (setq variable (|httpProperty| htPage '|variable|))
    (setq checker (|httpProperty| htPage '|checker|))
    (setq value (|htCheck| checker (|httpLabelInputString| htPage '|value|)))
    (set variable value)
    (|htSetFunCommandContinue| htPage value)))
```

defun htSetFunCommandContinue

— defun htSetFunCommandContinue —

```
(defun |htSetFunCommandContinue| (htPage value)
  (let (parts continue)
    (setq parts (|httpProperty| htPage '|parts|))
    (setq continue
      (cond
        ((null parts) nil)
        ((and (consp parts)
              (consp (qcar parts)) (eq (qcaar parts) '|break|)
              (consp (qcdr parts)) (eq (qcddar parts) nil))
         (|eval| (qcadar parts)))
        (t t)))
    (cond
      (continue
       (|httpSetProperty| htPage '|parts| (qcdr parts))
       (|htShowFunctionPageContinued| htPage))
      (t (|htKill| htPage value)))))
```

defun htKill

— defun htKill —

```
(defun |htKill| (htPage value)
```

```

(declare (ignore htPage))
(prog (string)
(declare (special |$path|))
(return
  (progn
    (|htInitPage| "System Command" nil)
    (setq string
      (CONCAT "{\\em }set "
        (|listOfStrings2String|
          (cons value |$path|))
        "}"))
    (|htMakePage|
      (cons '(|text| "{Here is the AXIOM system command you could have issued:}"
        "\\vspace{2}\\newline\\centerline{\\tt"
          (cons (cons '(|text| string) nil)))
        (|htMakePage| '(|text| . "}\\vspace{1}\\newline\\rm"))
        (|htSay| "\\vspace{2}{Select \\ \\UpButton{} \\ to go back.}")
        (|htSay| "\\newline{Select \\ \\ExitButton{QuitPage} \\ to remove this window.}")
        (|htProcessDoitButton|
          (cons "Press to Remove Page"
            (cons "" (cons '(|htDoNothing| nil))))
          (|htShowPage|))))))

```

defun htSetNotAvailable

— defun htSetNotAvailable —

```

(defun |htSetNotAvailable| (htPage whatToType)
  (let (page string)
    (setq page
      (|htInitPage| "Unavailable Set Command" (|httpPropertyList| htPage))
      (|htInitPage| "Unavailable System Command" nil)
      (setq string (concat "{\\em " whatToType "}"))
      (|htMakePage|
        (cons '(|text| "\\vspace{1}\\newline"
          "{Sorry, but this system command is not available through HyperDoc. Please directly issue this command in a
            "\\vspace{2}\\newline\\centerline{\\tt"
              (cons (cons '(|text| string) nil)))
            (|htMakePage| '(|text| . "}\\vspace{1}\\newline"))
            (|htProcessDoitButton| (list "Press to Remove Page" "" '(|htDoNothing| ))
            (|htShowPage|))))

```

defun htDoNothing

— defun htDoNothing —

```
(defun |htDoNothing| (htPage command)
  (declare (ignore htPage command))
  nil)
```

—————

defun htCheck

— defun htCheck —

```
(defun |htCheck| (checker value)
  (cond
    ((consp checker) (|htCheckList| checker (|parseWord| value)))
    (t (funcall checker value))))
```

—————

defun parseWord

— defun parseWord —

```
(defun |parseWord| (x)
  (prog ()
    (return
      (SEQ (cond
        ((stringp x)
          (cond
            ((prog (G167588)
              (setq G167588 t)
              (return
                (DO ((G167594 nil (null G167588))
                  (G167595 (maxindex x))
                  (i 0 (1+ i)))
                  ((OR G167594 (QSGREATERP i G167595))
                   G167588)
                (SEQ (EXIT (setq G167588
                  (AND G167588
                    (digitp (elt x i))))))))))
```

```

      (parse-integer x))
    (t (intern x))))
  (t x))))))

```

defun htCheckList

— defun htCheckList —

```

(defun |htCheckList| (checker value)
  (prog (n t2 m)
    (return
      (progn
        (cond
          ((|member| value '(|y| |ye| |yes| Y YE YES))
           (setq value '|yes|)))
        (cond
          ((|member| value '(|n| |no| N NO)) (setq value '|no|)))
        (cond
          ((and (consp checker)
                (progn
                  (setq n (qcar checker))
                  (setq t2 (qcdr checker))
                  (and (consp t2) (eq (QCDR t2) nil)
                      (progn (setq m (QCAR t2)) t)))
                (integerp n))
           (cond
            ((eql m (1+ n))
             (cond ((|member| value checker) value) (t n)))
            (null m)
            (cond
             ((and (integerp value) (>= value n)) value)
             (t n)))
           ((integerp m)
            (cond
             ((and (integerp value) (>= value n)
                  (<= value m))
              value)
             (t n))))))
        ((|member| value checker) value)
        (t (car checker))))))

```

defun translateYesNoToTrueFalse

— defun translateYesNoToTrueFalse —

```
(defun |translateYesNoToTrueFalse| (x)
  (cond
    ((eq x '|yes|) t)
    ((eq x '|no|) nil)
    (t x)))
```

—————

defun chkNameList

— defun chkNameList —

```
(defun |chkNameList| (x)
  (prog (u parsedNames)
    (return
      (SEQ (progn
        (setq u (|bcString2ListWords| x))
        (setq parsedNames
          (prog (G167635)
            (setq G167635 nil)
            (return
              (DO ((G167640 u (CDR G167640))
                (x nil))
                ((or (atom G167640)
                  (progn
                    (setq x (car G167640))
                    nil))
                  (NREVERSEO G167635))
                (SEQ (EXIT (setq G167635
                  (cons (|ncParseFromString| x)
                    G167635))))))))))
          (cond
            ((prog (G167646)
              (setq G167646 t)
              (return
                (DO ((G167652 nil (NULL G167646))
                  (G167653 parsedNames (CDR G167653))
                  (x nil))
                  ((OR G167652 (ATOM G167653)
                    (progn (setq x (car G167653)) nil))
                    G167646))
```

```

      (SEQ (EXIT (setq G167646
        (AND G167646 (identp x))))))
    parsedNames)
  (t
    "Please enter a list of identifiers separated by blanks")))))))

```

defun chkPosInteger

— defun chkPosInteger —

```

(defun |chkPosInteger| (s)
  (prog (u)
    (return
      (cond
        ((and (setq u (|parseOnly| s)) (integerp u) (> u 0))
         u)
        (t "Please enter a positive integer")))))

```

defun chkOutputFileName

— defun chkOutputFileName —

```

(defun |chkOutputFileName| (s)
  (cond
    ((|member| (|bcString2WordList| s) '(CONSOLE |console|))
     '|console|)
    (t (|chkDirectory| s))))

```

defun chkDirectory

— defun chkDirectory —

```

(defun |chkDirectory| (s) s)

```

defun chkNonNegativeInteger

— defun chkNonNegativeInteger —

```
(defun |chkNonNegativeInteger| (s)
  (prog (u)
    (return
      (cond
        ((and (setq u (|ncParseFromString| s)) (integerp u)
              (>= u 0))
         u)
        (t "Please enter a non-negative integer")))))
```

—————

defun chkRange

— defun chkRange —

```
(defun |chkRange| (s)
  (prog (u)
    (declare (special |$htFinal| |$htInitial|))
    (return
      (cond
        ((and (setq u (|ncParseFromString| s)) (integerp u)
              (>= u |$htInitial|)
              (or (null |$htFinal|) (<= u |$htFinal|)))
         u)
        ((null |$htFinal|)
         (CONCAT "Please enter an integer greater than "
                  (|stringize| (- |$htInitial| 1)))))
        (t
         (CONCAT "Please enter an integer between "
                  (|stringize| |$htInitial|) " and "
                  (|stringize| |$htFinal|)))))))
```

—————

defun chkAllNonNegativeInteger

— defun chkAllNonNegativeInteger —

```
(defun |chkAllNonNegativeInteger| (s)
  (prog (u)
    (return
      (or (and (setq u (|ncParseFromString| s))
        (|member| u '(|a| |al| |all| A AL ALL)) 'ALL)
        (|chkNonNegativeInteger| s)
        "Please enter {\\em all} or a non-negative integer"))))
```

defun htMakePathKey,fn

— defun htMakePathKey,fn —

```
(defun |htMakePathKey,fn| (a b)
  (SEQ (if (null b) (EXIT a))
    (EXIT (|htMakePathKey,fn|
      (CONCAT a "." (PNAME (car b)))
      (cdr b)))))
```

defun htMakePathKey

— defun htMakePathKey —

```
(defun |htMakePathKey| (path)
  (cond
    ((null path) (|systemError| "path is not set"))
    (t
      (intern (|htMakePathKey,fn| (PNAME (car path)) (cdr path))))))
```

defun htMarkTree

— defun htMarkTree —

```
(defun |htMarkTree| (tree n)
  (SEQ (progn
```

```

(rplacd (last tree) n)
(SEQ (DO ((G167706 tree (cdr G167706)) (branch nil))
      ((OR (ATOM G167706)
            (progn (setq branch (car G167706)) nil))
       nil)
      (SEQ (EXIT (cond
                    ((eq (elt branch 3) 'tree)
                     (EXIT (|htMarkTree| (elt branch 5)
                                           (1+ n))))))))))

```

defun htSetHistory

— defun htSetHistory —

```

(defun |htSetHistory| (htPage)
  (let (msg data)
    (setq msg
      '|when the history facility is on (yes), results of computations are saved in memory|)
    (setq data
      (list '|history| msg '|history| 'literals '|$HiFiAccess|
              '|on| |off| |yes| |no|)))
    (|htShowLiteralsPage| htPage data)))

```

defun htSetOutputLibrary

— defun htSetOutputLibrary —

```

(defun |htSetOutputLibrary| (htPage)
  (|htSetNotAvailable| htPage ")set compiler output"))

```

defun htSetInputLibrary

— defun htSetInputLibrary —

```
(defun |htSetInputLibrary| (htPage)
  (|htSetNotAvailable| htPage ")set compiler input"))
```

defun htSetExpose

— defun htSetExpose —

```
(defun |htSetExpose| (htPage)
  (|htSetNotAvailable| htPage ")set expose"))
```

defun htSetOutputCharacters

— defun htSetOutputCharacters —

```
(defun |htSetOutputCharacters| (htPage)
  (|htSetNotAvailable| htPage ")set output characters"))
```

defun htSetLinkerArgs

— defun htSetLinkerArgs —

```
(defun |htSetLinkerArgs| (htPage)
  (|htSetNotAvailable| htPage ")set fortran calling linker"))
```

defun htSetCache

— defun htSetCache —

```

(defun |htSetCache| (&REST arg &AUX options htPage)
  (declare (special |$valueList| |$path|))
  (setq htPage (car arg))
  (setq options (cdr arg))
  (setq |$path| '(|functions| |cache|))
  (setq htPage (|htInitPage| (|mkSetTitle|) nil))
  (setq |$valueList| nil)
  (|htMakePage|
   '(|text|
    "Use this system command to cause the AXIOM interpreter to 'remember' "
    "past values of interpreter functions. "
    "To remember a past value of a function, the interpreter "
    "sets up a {\em cache} for that function based on argument values. "
    "When a value is cached for a given argument value, its value is gotten "
    "from the cache and not recomputed. Caching can often save much "
    "computing time, particularly with recursive functions or functions that "
    "are expensive to compute and that are called repeatedly "
    "with the same argument." "\vspace{1}\newline "
    (|domainConditions| (|Satisfies| S chkNameList))
    |text|
    "Enter below a list of interpreter functions you would like specially cached. "
    "Use the name {\em all} to give a default setting for all "
    "interpreter functions. " "\vspace{1}\newline "
    "Enter {\em all} or a list of names (separate names by blanks):")
    (|inputStrings| (" " " 60 "all" names S))
    (|doneButton| "Push to enter names" |htCacheAddChoice|)))
  (|htShowPage|))

```

defun htCacheAddChoice

— defun htCacheAddChoice —

```

(defun |htCacheAddChoice| (htPage)
  (prog (names page)
    (declare (special |$valueList|))
    (return
     (SEQ (progn
           (setq names
                (|bcString2WordList|
                 (|htLabelInputString| htPage '|names|)))
           (setq |$valueList|
                (cons (|listOfStrings2String| names)
                      |$valueList|))
           (cond
            ((null names) (|htCacheAddQuery|))

```

```

((null (cdr names)) (|htCacheOne| names))
(t (setq page (|htInitPage| (|mkSetTitle|) nil))
  (|htSetProperty| page '|names| names)
  (|htMakePage|
    '((|domainConditions|
      (|Satisfies| ALLPI chkAllPositiveInteger))
      (|text|
        "For each function, enter below a {\em cache length}, a positive integer. "
        "This number tells how many past values will "
        "be cached. "
        "A cache length of {\em 0} means the function won't be cached. "
        "To cache all past values, "
        "enter {\em all}."
        "\vspace{1}\newline "
        "For each function name, enter {\em all} or a positive integer:"))
      (DO ((i 1 (QSADD1 i))
          (G167755 names (CDR G167755)) (name nil))
        ((or (atom G167755)
            (progn (setq name (car G167755)) nil))
          nil)
        (SEQ (EXIT (|htMakePage|
                    (cons (cons '|inputStrings|
                                (cons
                                  (cons
                                    (CONCAT "Function {\em "
                                      name
                                      "} will cache"
                                    (cons "values"
                                      (cons 5
                                        (cons 10
                                          (cons
                                            (|htMakeLabel|
                                              "c" i)
                                            (cons 'ALLPI nil))))))
                                  nil))
                                nil))))
                    (|htSetvarDoneButton| "Select to Set Values" '|htCacheSet|)
                    (|htShowPage|))))))))

```

defun htMakeLabel

— defun htMakeLabel —

```

(defun |htMakeLabel| (prefix i)
  (intern (concat prefix (|stringize| i))))

```

defun htCacheSet

— defun htCacheSet —

```

(defun |htCacheSet| (htPage)
  (prog (names num n name val)
    (declare (special |$cacheCount| |$cacheAlist|))
    (return
      (SEQ (progn
        (setq names (|htProperty| htPage '|names|))
        (DO ((i 1 (QSADD1 i))
              (G167785 names (CDR G167785)) (name nil))
          ((or (atom G167785)
                (progn (setq name (car G167785)) nil))
           nil)
          (SEQ (EXIT (progn
                        (setq num
                          (|chkAllNonNegativeInteger|
                           (|htLabelInputString| htPage
                            (|htMakeLabel| "c"
                             i))))
                        (setq |$cacheAlist|
                          (ADDASSOC (intern name) num
                                    |$cacheAlist|))))))
          (cond
            ((setq n (LASSOC '|all| |$cacheAlist|))
             (setq |$cacheCount| n)
             (setq |$cacheAlist|
               (|deleteAssoc| '|all| |$cacheAlist|))))
            (|htInitPage| "Cache Summary" nil)
            (|bcHt| "In general, interpreter functions ")
            (|bcHt| (cond
                      ((EQL |$cacheCount| 0)
                       '|will {\em not} be cached.|)
                      (t (|bcHt| "cache ")
                        (|htAllOrNum| |$cacheCount|)
                        "| values.")))
            (|bcHt| "\\vspace{1}\\newline ")
            (cond
              (|$cacheAlist|
               (DO ((G167801 |$cacheAlist| (cdr G167801))
                     (G167774 nil))
                   ((or (atom G167801)
                        (progn

```

```

        (setq G167774 (car G167801))
        nil)
      (progn
        (progn
          (setq name (car G167774))
          (setq val (CDR G167774))
          G167774)
        nil))
      nil)
    (SEQ (EXIT (cond
      ((NEQUAL val |$cacheCount|)
        (progn
          (|bcHt| "\\newline function {\\em "
            (|bcHt| (|stringize| name))
            (|bcHt| "} will cache "
              (|htAllOrNum| val)
              (|bcHt| "} values"))))))))
      (|htProcessDoitButton|
        (cons "Press to Remove Page"
          (cons "" (cons ' |htDoNothing| nil))))
      (|htShowPage|))))))

```

defun htAllOrNum

— defun htAllOrNum —

```

(defun |htAllOrNum| (val)
  (|bcHt| (cond
    ((eq val ' |all|) "{\\em all")
    ((eq val 0) "{\\em no")
    (t
      (CONCAT "the last {\\em "
        (|stringize| val))))))

```

defun htCacheOne

— defun htCacheOne —

```

(defun |htCacheOne| (names)

```



```

(prog (page)
  (return
    (progn
      (setq page (|htInitPage| (|mkSetTitle|) nil))
      (|httpSetProperty| page '|names| names)
      (|htMakePage|
        '((|domainConditions|
          (|Satisfies| ALLPI |chkAllPositiveInteger|))
          (|text| "Enter below a {\em cache length}, a positive integer. "
            "This number tells how many past values will "
            "be cached. To cache all past values, "
            "enter {\em all}." "\vspace{1}\newline ")
          (|inputStrings|
            ("Enter {\em all} or a positive integer:" "" 5 10
              |c1| ALLPI))))
        (|htSetvarDoneButton| "Select to Set Value"
          '|htCacheSet|)
        (|htShowPage|))))))

```

defvar \$historyDisplayWidth

— initvars —

```
(defvar |$historyDisplayWidth| 120)
```

defvar \$newline

— initvars —

```
(defvar |$newline| #\Newline)
```

defun downlink

— defun downlink —

```
(defun |downlink| (page)
  (|htInitPage| "Bridge" nil)
  (|htSay| "\\replacepage{" page "}")
  (|htShowPage|))
```

defun dbNonEmptyPattern

— defun dbNonEmptyPattern —

```
(defun |dbNonEmptyPattern| (pattern)
  (cond
    ((null pattern) "*")
    (t (setq pattern (PRINC-TO-STRING pattern))
      (cond ((> (|#| pattern) 0) pattern) (t "*")))))
```

defun htSystemVariables,gn

— defun htSystemVariables,gn —

```
(defun |htSystemVariables,gn| (t1 al)
  (let (class key options)
    (declare (special |$heading| |$levels|))
    (setq class (caddr t1))
    (setq key (caddr t1))
    (setq options (cadr (cddddr t1)))
    (cond
      ((null (member class |$levels|)) al)
      ((or (or (eq key 'literals) (eq key 'integer))
           (eq key 'string))
        (cons (cons |$heading| t1) al))
      ((eq key 'tree)
        (|htSystemVariables,fn| options al nil))
      ((eq key 'function)
        (cons (cons |$heading| t1) al))
      (t (|systemError| key)))))
```

defun htSystemVariables,fn

— defun htSystemVariables,fn —

```
(defun |htSystemVariables,fn| (t1 al firstTime)
  (declare (special |$heading|))
  (SEQ (if (atom t1) (EXIT al))
    (if firstTime (setq |$heading| (|opOf| (car t1))) nil)
    (EXIT (|htSystemVariables,fn| (cdr t1)
      (|htSystemVariables,gn| (car t1) al) firstTime))))
```

—————→

defun htSystemVariables,displayOptions

— defun htSystemVariables,displayOptions —

```
(defun |htSystemVariables,displayOptions| (name class variable val options)
  (SEQ (if (eq class 'integer)
    (EXIT (SEQ (|htMakePage|
      (cons (cons '|bcLispLinks|
        (cons
          (cons
            (cons '|text|
              (cons (elt options 0)
                (cons "-"
                  (cons
                    (or (elt options 1)
                      ""))
                    nil))))
          nil)
        (cons ""
          (cons
            '|htSetSystemVariableKind|
            (cons
              (cons variable
                (cons name
                  (cons 'parse-integer nil)))
              nil))))
          nil))
      nil))
    (|htMakePage|
      '((|domainConditions|
        (|isDomain| INT (|Integer|))))))
```

```

      (EXIT (|htMakePage|
(list
  (cons '|bcStrings| (list (list 5 (princ-to-string val) name 'int))))))
(if (eq class 'string)
  (EXIT (|htSay| "{\\em " val
    "\\space{1}")
  (EXIT (DO ((G167913 options (cdr G167913)) (x nil))
    ((or (atom G167913)
      (progn (setq x (car G167913)) nil))
    nil)
    (SEQ (if (or (or (equal val x)
      (and (eq val t)
        (eq x '|on|)))
      (and (null val) (eq x '|off|)))
      (EXIT (|htSay| "{\\em " x
        "\\space{1}")
      (EXIT (|htMakePage|
        (cons (cons '|bcLispLinks|
          (cons
            (cons x
              (cons " "
                (cons '|htSetSystemVariable|
                  (cons
                    (cons variable
                      (cons x nil))
                    nil))))
            nil))
          nil))
        nil))))))

```

defun htSystemVariables,functionTail

— defun htSystemVariables,functionTail —

```

(defun |htSystemVariables,functionTail| (name class var valuesOrFunction)
  (prog (val)
    (return
      (SEQ (setq val (|eval| var))
        (if (atom valuesOrFunction)
          (EXIT (SEQ (|htMakePage|
            '((|domainConditions|
              (|isDomain| STR (|String|))))
            (|htMakePage|
              (cons (cons '|bcLinks|
                (cons
                  (cons "reset"

```

```

                                (cons ""
                                (cons
                                  ' |htSetSystemVariableKind|
                                  (cons
                                    (cons var
                                      (cons name (cons nil nil)))
                                    nil))))
                                nil))
                                nil))
                                (EXIT (|htMakePage|
                                (cons
                                  (cons ' |bcStrings|
                                  (cons
                                    (cons 30
                                      (cons (PRINC-TO-STRING val)
                                        (cons name
                                          (cons valuesOrFunction nil))))
                                    nil))
                                  nil))))))
                                (EXIT (|htSystemVariables,displayOptions| name class
                                var val valuesOrFunction))))))

```

defun htSystemVariables

— defun htSystemVariables —

```

(defun |htSystemVariables| ()
  (prog (|$levels| |$heading| classlevel table heading name
        message key variable options func lastHeading
        t1 msg class var valuesOrFunction val)
    (DECLARE (SPECIAL |$levels| |$heading| |$setOptions| |$UserLevel|
                  |$fullScreenSysVars|))
    (return
      (SEQ (cond
        ((null |$fullScreenSysVars|) (|htSetVars|))
        (t (setq classlevel |$UserLevel|)
          (setq |$levels| '(|compiler| |development| |interpreter|))
          (setq |$heading| nil)
          (DO () ((NULL (NEQUAL classlevel (car |$levels|))) nil)
            (SEQ (EXIT (setq |$levels| (cdr |$levels|))))))
          (setq table
            (NREVERSE
              (|htSystemVariables,fn| |$setOptions| nil
                t)))
          (|htInitPage| "System Variables" nil)

```

```

(|htSay| "\\beginmenu")
(setq lastHeading nil)
(DO ((G167961 table (cdr G167961)) (G167879 nil))
  ((or (atom G167961)
    (progn (setq G167879 (car G167961)) nil)
    (progn
      (progn
        (setq heading (car G167879))
        (setq name (cadr G167879))
        (setq message (caddr G167879))
        (setq key (car (cddddr G167879)))
        (setq variable (cadr (cddddr G167879)))
        (setq options (caddr (cddddr G167879)))
        (setq func (caddrr (cddddr G167879)))
        G167879)
      nil))
    nil)
  (SEQ (EXIT (progn
    (|htSay| "\\newline\\item ")
    (cond
      ((equal heading lastHeading)
        (|htSay| "\\tab{8}"))
      (t
        (|htSay| heading
          "\\tab{8}")
        (setq lastHeading heading)))
    (|htSay| "{\\em " name
      '}\\tab{22}| message)
    (|htSay| "\\tab{80}")
    (cond
      ((eq key 'function)
        (cond
          ((null options)
            (|htMakePage|
              (cons
                (cons '|bcLinks|
                  (cons
                    (cons "reset"
                      (cons ""
                        (cons func (cons nil nil))))
                    nil))
                nil)))
          (t
            (setq t1 (car options))
            (setq msg (car t1))
            (setq class (cadr t1))
            (setq var (caddr t1))
            (setq valuesOrFunction (caddrr t1))
            (|htSystemVariables,functionTail|
              name class var valuesOrFunction)
          ))
      ))
    ))
  ))

```

```

(DO
  ((G167971 (cdr options)
    (cdr G167971))
   (option nil))
  ((or (atom G167971)
    (progn
      (setq option (car G167971))
      nil))
   nil)
  (SEQ
   (EXIT
    (cond
      ((and (consp option)
        (eq (QCAR option)
          '|break|))
       '|break|)
      ('|skip|)
      (t
       (setq msg (car option))
       (setq class (cadr option))
       (setq var (caddr option))
       (setq valuesOrFunction
        (caddr option))
       (|htSay| "\\newline\\tab{22}"
        msg
        "\\tab{80}")
       (|htSystemVariables,functionTail|
        name class var
        valuesOrFunction))))))
  (t (setq val (|eval| variable))
    (|htSystemVariables,displayOptions|
     name key variable val options))))))
(|htSay| "\\endmenu") (|htShowPage|))))))

```

defun htSetSystemVariableKind

— defun htSetSystemVariableKind —

```

(defun |htSetSystemVariableKind| (htPage arg)
  (let (variable name fun value)
    (setq variable (car arg))
    (setq name (cadr arg))
    (setq fun (caddr arg))
    (setq value (|htLabelInputString| htPage name))
    (when (and (stringp value) fun) (setq value (funcall fun value)))
    (set variable value)
  )
)

```

```
(|htSystemVariables|)))
```

defun htSetSystemVariable

— defun htSetSystemVariable —

```
(defun |htSetSystemVariable| (htPage arg)
  (declare (ignore htPage))
  (let (name value)
    (setq name (car arg))
    (setq value (cadr arg))
    (setq value
      (cond
        ((eq value '|on|) t)
        ((eq value '|off|) nil)
        (t value)))
    (set name value)
    (|htSystemVariables|)))
```

defun htGloss

— defun htGloss —

```
(defun |htGloss| (pattern)
  (|htGlossPage| nil
    (or (|dbNonEmptyPattern| pattern) "*" ) t))
```

defun htGlossPage

— defun htGlossPage —

```
(defun |htGlossPage| (htPage pattern tryAgain?)
  (prog (|$wildCard| |$key| filter grepForm results defstream
    lines heading k tick)
```



```

(declare (special |$wildCard| |$key| |$tick|))
(return
  (SEQ (progn
    (setq |$wildCard| #\*)
    (cond
      ((equal pattern "*")
        (|downlink| '|GlossaryPage|))
      (t (setq filter (|pmTransFilter| pattern))
        (setq grepForm (|mkGrepPattern| filter '|none|))
        (setq |$key| '|none|)
        (setq results (|applyGrep| grepForm '|gloss|))
        (setq defstream
          (make-instream
            (CONCAT (getenv "AXIOM")
              "/algebra/glossdef.text"))))
      (setq lines
        (|gatherGlossLines| results defstream))
      (setq heading
        (cond
          ((equal pattern "")
            "Glossary")
          ((null lines)
            (cons "No glossary items match {\em "
              (cons pattern
                (cons "}" nil))))
          (t
            (cons "Glossary items matching {\em "
              (cons pattern
                (cons "}" nil))))))
      (cond
        ((null lines)
          (cond
            ((and tryAgain? (> (|#| pattern) 0))
              (cond
                ((equal
                  (elt pattern
                    (setq k (MAXINDEX pattern)))
                  #\s)
                  (|htGlossPage| htPage
                    (SUBSTRING pattern 0 k) t))
                ((upper-case-p (elt pattern 0))
                  (|htGlossPage| htPage (downcase pattern)
                    nil))
                (t
                  (|errorPage| htPage
                    (cons "Sorry"
                      (cons nil
                        (cons
                          (cons "\\centerline{"
                            (append heading

```

```

                                (cons "}" nil)))
                                nil))))))
(t
  (|errorPage| htPage
    (cons "Sorry"
      (cons nil
        (cons
          (cons
            "\\centerline{"
            (append heading
              (cons "}" nil)))
            nil))))))
  (t (|htInitPageNoScroll| nil heading)
    (|htSay| "\\beginscroll\\beginmenu")
    (DO ((G168058 lines (cdr G168058))
        (line nil))
      ((or (atom G168058)
        (progn (setq line (car G168058)) nil))
        nil)
      (SEQ (EXIT (progn
        (setq tick
          (|charPosition| |$tick|
            line 1))
          (|htSay|
            "\\item{\\em \\menuitemstyle{}}\\tab{0}{\\em "
              (|escapeString|
                (SUBSTRING line 0 tick))
              "}"
              (SUBSTRING line
                (1+ tick) nil))))))
        (|htSay| "\\endmenu ")
        (|htSay| "\\endscroll\\newline ")
        (|htMakePage|
          (cons (cons '|bcLinks|
            (cons
              (cons "Search"
                (cons ""
                  (cons '|htGlossSearch|
                    (cons nil nil))))
                nil))
            nil))
          nil))
        (|htSay| " for glossary entry matching ")
        (|htMakePage|
          (cons (cons '|bcStrings|
            (cons
              (cons 24
                (cons "*"
                  (cons '|filter| (cons 'em nil))))
                nil))
            nil))
          nil))
    nil))

```

```
(|htShowPageNoScroll|)))))))))
```

defun gatherGlossLines

— defun gatherGlossLines —

```
(defun |gatherGlossLines| (results defstream)
  (prog (n keyAndTick byteAddress line k pointer def x
        j nextPointer xtralines acc)
    (declare (special |$tick|))
    (return
      (SEQ (progn
        (setq acc nil)
        (DO ((G168098 results (cdr G168098))
              (keyline nil))
              ((or (atom G168098)
                    (progn (setq keyline (car G168098)) nil))
              nil)
          (SEQ (EXIT (progn
            (setq n
              (|charPosition| |$tick| keyline
                0))
            (setq keyAndTick
              (SUBSTRING keyline 0
                (1+ n)))
            (setq byteAddress
              (|string2Integer|
                (SUBSTRING keyline (1+ n)
                  nil)))
            (file-position defstream byteAddress)
            (setq line (readline defstream))
            (setq k
              (|charPosition| |$tick| line 1))
            (setq pointer
              (SUBSTRING line 0 k))
            (setq def
              (SUBSTRING line (1+ k)
                nil))
            (setq xtralines nil)
            (DO ()
              ((null (and (null (eofp defstream))
                (setq x
                  (readline defstream))
                (setq j
                  (|charPosition| |$tick| x 1))
```

```

                                (setq nextPointer
                                (SUBSTRING x 0 j))
                                (equal nextPointer
                                pointer)))
    nil)
  (SEQ (EXIT
        (setq xtralines
              (cons
                (SUBSTRING x (1+ j) nil)
                xtralines))))))
  (setq acc
    (cons
      (CONCAT keyAndTick def
        (prog (G168110)
          (setq G168110 "")
          (return
            (DO
              ((G168115
                (NREVERSE xtralines)
                (CDR G168115))
              (G168081 nil))
              ((OR (ATOM G168115)
                (progn
                  (setq G168081
                    (car G168115))
                  nil))
                G168110)
              (SEQ
                (EXIT
                  (setq G168110
                    (CONCAT G168110
                      G168081))))))))
        acc))))))
  (reverse acc))))))

```

defun htGlossSearch

— defun htGlossSearch —

```

(defun |htGlossSearch| (htPage junk)
  (declare (ignore junk))
  (|htGloss| (|httpLabelInputString| htPage '|filter|)))

```

defun htGreekSearch

— defun htGreekSearch —

```

(defun |htGreekSearch| (filter)
  (prog (ss s names matches nonmatches)
    (return
      (SEQ (progn
        (setq ss (|dbNonEmptyPattern| filter))
        (setq s (|pmTransFilter| ss))
        (cond
          ((and (consp s) (eq (QCAR s) '|error|))
            (|bcErrorPage| s))
          ((null s)
            (|errorPage| nil
              (cons (cons "Missing search string"
                        nil)
                    (cons nil
                        (cons
                          "\vspace{2}\\\centerline{To select one of the greek letters:}\\\newline "
                          (cons
                            "\\centerline{{\\em first} enter a search key into the input area}\\\newline "
                            (cons
                              "\\centerline{{\\em then } move the mouse cursor to the work {\\em search} and click}"
                              nil)))))))
            (t (setq filter (|patternCheck| s))
              (setq names
                '(|alpha| |beta| |gamma| |delta| |epsilon|
                  |zeta| |eta| |theta| |iota| |kappa|
                  |lambda| |mu| |nu| |pi|))
              (DO ((G168149 names (CDR G168149)) (x nil))
                ((or (atom G168149)
                  (progn (setq x (car G168149)) nil))
                nil)
              (cond
                ((|superMatch?| filter (PNAME x))
                  (setq matches
                    (cons x matches)))
                (t
                  (setq nonmatches
                    (cons x nonmatches))))))
              (setq matches (nreverse matches))
              (setq nonmatches (nreverse nonmatches))
              (|htInitPage| "Greek Names" nil)
              (cond
                ((null matches)
                  (|htInitPage|
                    (cons "Greek names matching search string {\\em "
                        (cons ss (cons "}" nil))))

```

```

        nil)
(|htSay| '|\vspace{2}\centerline{Sorry, but no greek letters match your search string}\centerline{\em
        ss
        '}}\centerline{Click on the up-arrow to try again}|)
(|htShowPage|))
(t
  (|htInitPage|
    (cons "Greek letters matching search string {\em "
      (cons ss (cons "}" nil)))
    nil)
  (cond
    (nonmatches
      (|htSay|
        "The greek letters that {\em match} your search string {\em "
          ss "}:"))
    (t
      (|htSay| "Your search string {\em "
        ss
        '} matches all of the greek letters:|))
      (|htSay| "{\em \table{"
        (DO ((G168158 matches (CDR G168158))
          (x nil))
          ((or (atom G168158)
            (progn (setq x (car G168158)) nil))
          nil)
          (SEQ (EXIT (|htSay| "{" x
            "})))))
        (|htSay| "}}\vspace{1}")
        (cond
          (nonmatches
            (|htSay|
              "The greek letters that {\em do not match} your search string:{\em \table{"
                (DO ((G168167 nonmatches (CDR G168167))
                  (x nil))
                  ((or (atom G168167)
                    (progn
                      (setq x (car G168167))
                      nil))
                  nil)
                  (SEQ (EXIT (|htSay| "{" x
                    "})))))
                (|htSay| "}}"))
              (|htShowPage|)))))))))

```

defun htTextSearch

— defun htTextSearch —

```

(defun |htTextSearch| (filter)
  (prog (s lines matches nonmatches)
    (return
      (SEQ (progn
        (setq s
          (|pmTransFilter| (|dbNonEmptyPattern| filter)))
        (cond
          ((and (consp s) (eq (QCAR s) '|error|))
            (|bcErrorPage| s))
          ((null s)
            (|errorPage| nil
              (cons (cons "Missing search string"
                nil)
                (cons nil
                  (cons
                    "\\vspace{2}\\centerline{To select one of the lines of text:}\\newline "
                    (cons
                      "\\centerline{{\\em first} enter a search key into the input area}\\newline "
                      (cons
                        "\\centerline{{\\em then } move the mouse cursor to the work {{\\em search} and click}"
                        nil)))))))
            (t (setq filter s)
              (setq lines
                (cons
                  "{\\em Fruit flies} *like* a {\\em banana and califlower ears.}"
                  (cons
                    "{\\em Sneak Sears Silas with Savings Snatch}"
                    nil)))
              (DO ((G168191 lines (cdr G168191)) (x nil))
                ((or (atom G168191)
                  (progn (setq x (car G168191)) nil))
                  nil)
              (SEQ (EXIT (cond
                ((|superMatch?| filter x)
                  (setq matches
                    (cons x matches)))
                (t
                  (setq nonmatches
                    (cons x nonmatches))))))
                (setq matches (NREVERSE matches))
                (setq nonmatches (NREVERSE nonmatches))
                (|htInitPage| "Text Matches" nil)
              (cond
                ((null matches)
                  (|htInitPage|

```

```

      (cons "Lines matching search string {\em "
            (cons s (cons ")" nil)))
      nil)
    (|htSay|
      '\vspace{2}\centerline{Sorry, but no lines match your search string}\centerline{{\em |
        s
        '}}\centerline{Click on the up-arrow to try again}|)
    (|htShowPage|))
  (t
    (|htInitPage|
      (cons "Lines matching search string {\em "
            (cons s (cons ")" nil)))
      nil)
    (cond
      (nonmatches
        (|htSay| "The lines that {\em match} your search string {\em "
                  s "}.:"))
        (t
          (|htSay| "Your search string {\em "
                    s '} matches both lines:|)))
        (|htSay| "{\em \table{")
        (DO ((G168200 matches (CDR G168200))
              (x nil))
              ((or (atom G168200)
                    (progn (setq x (car G168200)) nil))
              nil)
          (SEQ (EXIT (|htSay| "{ " x
                               "}.:"))))
        (|htSay| "}}\vspace{1}")
        (cond
          (nonmatches
            (|htSay|
              "The line that {\em does not match} your search string:{\em \table{")
              (DO ((G168209 nonmatches (cdr G168209))
                    (x nil))
                    ((or (atom G168209)
                          (progn
                            (setq x (car G168209))
                            nil))
                    nil)
              (SEQ (EXIT (|htSay| "{ " x
                               "}.:"))))
              (|htSay| "}}"))
            (|htShowPage|))))))

```

defun htTutorialSearch

— defun htTutorialSearch —

```

(defun |htTutorialSearch| (pattern)
  (prog (s source target lines t1 name title)
    (return
      (SEQ (progn
        (setq s
          (or (|dbNonEmptyPattern| pattern)
              (return
                (|errorPage| nil
                  (cons "Empty search key"
                      (cons nil
                          (cons
                            "\vspace{3}\centerline{You must enter some search string"
                            nil)))))))
          (setq s (|mkUnixPattern| s))
          (setq source "$AXIOM/doc/hypertext/pages/ht.db")
          (setq target "/tmp/temp.text.$SPADNUM")
          (OBEY (CONCAT "$AXIOM/lib/hthits"
                        " \" s \" "
                        source " > " target))
          (setq lines (|dbReadLines| 'temp))
          (|htInitPageNoScroll| nil
            (cons "Tutorial Pages mentioning {\em "
                  (cons pattern (cons ")" nil))))
          (|htSay| "\beginscroll\table{")
          (DO ((G168241 lines (cdr G168241)) (line nil))
              ((or (atom G168241)
                   (progn (setq line (car G168241)) nil))
               nil)
            (SEQ (EXIT (progn
                          (setq t1 (|dbParts| line 3 0))
                          (setq name (car t1))
                          (setq title (cadr t1))
                          (|htSay| (cons "{\downlink{"
                                          (cons title
                                              (cons "{"
                                                  (cons name
                                                      (cons "}" nil))))))))
                          (|htSay| "}")
                          (|htShowPage|))))))

```

defun mkUnixPattern

— defun mkUnixPattern —

```

(defun |mkUnixPattern| (s)
  (prog (starPositions k u)
    (declare (special |$wild|))
    (return
      (SEQ (progn
        (setq u (|mkUpDownPattern| s))
        (setq starPositions
          (reverse (prog (G168264)
            (setq G168264 nil)
            (return
              (DO
                ((G168270
                  (+ (- 1)
                    (MAXINDEX u)))
                (i 1 (QSADD1 i)))
                ((QSGREATERP i G168270)
                  (NREVERSE0 G168264))
                (SEQ
                  (EXIT
                    (cond
                      ((equal (elt u i)
                        |$wild|)
                      (setq G168264
                        (cons i G168264))))))))))
              (DO ((G168277 starPositions (cdr G168277))
                (i nil))
                ((or (atom G168277)
                  (progn (setq i (car G168277)) nil))
                  nil)
                (SEQ (EXIT (setq u
                  (CONCAT (SUBSTRING u 0 i)
                    ".*"
                    (SUBSTRING u (1+ i) nil))))))
                (cond
                  ((NEQUAL (elt u 0) |$wild|)
                   (setq u (CONCAT "[^a-zA-Z]" u)))
                  (t (setq u (SUBSTRING u 1 nil))))
                (cond
                  ((NEQUAL (elt u (setq k (MAXINDEX u))) |$wild|)
                   (setq u (CONCAT u "[^a-zA-Z]")))
                  (t (setq u (SUBSTRING u 0 k))))
                u))))))

```

Chapter 54

Browser Support Code

54.1 Pages Initiated from HyperDoc Pages

Search routines

defun dKind

— defun dbKind 0 —

```
(defun |dbKind| (line)
  (elt line 0))
```

—————

defun checkFilter

[trimString p??]

— defun checkFilter —

```
(defun |checkFilter| (filter)
  (setq filter (princ-to-string filter))
  (if (string= filter "")
      "*"
      (|trimString| filter)))
```

—————

```
;concatWithBlanks r == ; r is [head,:tail] =i ; tail =i CONCAT(head,"" ",concatWithBlanks
tail) ; head ; """
```

defun Concatenate words with blanks

— defun concatWithBlanks 0 —

```
(defun |concatWithBlanks| (r)
  (if (consp r)
      (format nil "~{~a~^ ~}" r)
      ""))
```

defun Make constructor names lowercase

```
[hget p1234]
[ifcar p??]
[conLowerCaseConTran p1590]
[$lowerCaseConTb p??]
```

— defun conLowerCaseConTran —

```
(defun |conLowerCaseConTran| (x)
  (declare (special |$lowerCaseConTb|))
  (cond
    ((identp x) (or (ifcar (hget |$lowerCaseConTb| x)) x))
    ((atom x) x)
    (t (loop for y in x collect (|conLowerCaseConTran| y)))))
```

defun string2Constructor

```
[downcase p1272]
[hget p1234]
[ifcar p??]
[$lowerCaseConTb p??]
```

— defun string2Constructor —

```
(defun |string2Constructor| (x)
```

```
(declare (special |$lowerCaseConTb|))
(cond
  ((null (stringp x)) x)
  (t (or (ifcar (hget |$lowerCaseConTb| (intern (downcase x)))) x))))
```

defvar dbDelimiters

— initvars —

```
(defvar |$dbDelimiters| (list #\space #\(\ #\)) )
```

defun String to words respecting delimiters

This breaks a string into words respecting delimiters, so if

```
$dbDelimiters = ( #\space #\(\ #\))
```

then

```
(|dbString2Words| "now is (the) time")
Value = ("now" "is" #\(\ "the" #\)) "time")
```

[dbWordFrom p1592]

— defun dbString2Words —

```
(defun |dbString2Words| (z)
  (loop
    with i = 0
    with pair = nil
    do (setq pair (|dbWordFrom| z i))
    while (and (consp pair) (= (length pair) 2)) ; dbWordFrom(1,i) is [w,i]
    do (setq i (second pair))
    collect (first pair)))
```

defun Next word respecting delimiters

This returns the next word or the next delimiter. So given

```
$dbDelimiters = ( #\space #\( #\ ) )
(|dbWordFrom| "now is (the) time")

(|dbWordFrom| b 0)  Value = ("now" 3)
(|dbWordFrom| b 3)  Value = ("is" 6)
(|dbWordFrom| b 6)  Value = (#\ ( 8)
(|dbWordFrom| b 8)  Value = ("the" 11)
(|dbWordFrom| b 11) Value = (#\ ) 12)
(|dbWordFrom| b 12) Value = ("time" 17)
(|dbWordFrom| b 17) Value = NIL
```

```
[maxindex p??]
[member p1238]
[concat p1237]
[$dbDelimiters p1591]
```

— **defun dbWordFrom** —

```
(defun |dbWordFrom| (z i)
  (let (maxIndex c ch buf k g1)
    (declare (special |$dbDelimiters|))
    (setq maxIndex (maxindex z))
    (loop while (and (>= maxIndex i) (char= (elt z i) #\space)) do (incf i))
    (if (and (>= maxIndex i) (|member| (elt z i) |$dbDelimiters|))
        (list (elt z i) (+ i 1))
        (progn
          (setq k
            (do ((g2 nil g1) (j i (+ j 1)))
                ((or g2 (> j maxIndex)) g1)
              (unless (|member| (elt z j) |$dbDelimiters|) (setq g1 (or g1 j)))))
          (when k
            (setq buf "")
            (do ()
                ((null (and (<= k maxIndex)
                          (null (|member| (setq c (elt z k)) |$dbDelimiters|))))
              nil)
            (setq ch (if (char= c #\_) (elt z (setq k (+ 1 k))) c))
            (setq buf (concat buf ch))
            (setq k (+ k 1)))
            (list buf k))))))
```

This creates a page for any cat, dom, package, default package

```
constructors Cname\#\E\sig \args \abb \comments (C is C, D, P, X)
```

There are 8 parts of an htPage:

1. kind
2. name
3. nargs
4. xflag
5. sig
6. args
7. abbrev
8. comments

```
[dbXParts p??]
[mkConform p1628]
[opOf p??]
[capitalize p??]
[ncParseFromString p1310]
[dbSourceFile p??]
[dbConformGenUnder p??]
[concat p1237]
[isExposedConstructor p1094]
[htInitPageNoScroll p1511]
[htAddHeading p1512]
[htSayStandard p1511]
[htpSetProperty p1503]
[dbShowConsDoc1 p1646]
[addParameterTemplates p??]
[htSay p1509]
[htSayStandard p1511]
[kPageContextMenu p??]
[htShowPageNoScroll p1513]
[$atLeastOneUnexposed p??]
[$conformsAreDomains p??]
```

— defun kPage —

```
(defun |kPage| (&rest a1)
  (let (parts name nargs sig args form isFile kind
        conform conname capitalKind signature sourceFileName constrings
        emString heading page options line)
    (declare (special |$conformsAreDomains| |$atLeastOneUnexposed|)))
```



```

(setq line (car a1))
(setq options (cdr a1))
; constructors Cname\#\E\sig \args \abb \comments (C is C, D, P, X)
(setq parts (|dbXParts| line 7 1))
(setq kind (first parts))
(setq name (second parts))
(setq nargs (third parts))
(setq sig (fifth parts))
(setq args (sixth parts))
(setq form (ifcar options))
(setq isFile (null kind))
(setq kind (or kind "package"))
(rplaca parts kind)
(setq conform (|mkConform| kind name args))
(setq conname (|opOf| conform))
(setq capitalKind (|capitalize| kind))
(setq signature (|ncParseFromString| sig))
(setq sourceFileName (|dbSourceFile| (intern name)))
(setq constrings
  (if (ifcdr form)
    (|dbConformGenUnder| form)
    (list (concat name args))))
(setq emString (cons "{\sf " (append constrings (list "}"))))
(setq heading (cons capitalKind (cons " " emString)))
(unless (|isExposedConstructor| conname)
  (setq heading (cons "Unexposed " heading)))
(setq page (|htInitPageNoScroll| NIL))
(|htAddHeading| heading)
(|htSayStandard| '||\beginscroll |)
(|httpSetProperty| page 'isFile t)
(|httpSetProperty| page 'parts parts)
(|httpSetProperty| page 'heading heading)
(|httpSetProperty| page 'kind kind)
(|httpSetProperty| page 'conform conform)
(|httpSetProperty| page 'signature signature)
; what follows is stuff from kiPage with domain = nil
(setq |$conformsAreDomains| nil)
(|dbShowConsDoc1| page conform nil)
(when (and (nequal kind 'category) (> nargs 0))
  (|addParameterTemplates| page conform))
(when |$atLeastOneUnexposed|
  (|htSay| "\\\newline{\\em *} = unexposed"))
(|htSayStandard| '||\endscroll |)
(|kPageContextMenu| page)
(|htShowPageNoScroll|))

```

defun Hyperdoc category search

[constructorSearch p1597]

— defun cSearch —

```
(defun |cSearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|c| "category"))
```

—————

defun Hyperdoc default domain search

[constructorSearch p1597]

— defun xSearch —

```
(defun |xSearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|x| "default package"))
```

—————

defun Hyperdoc domain search

[constructorSearch p1597]

— defun dSearch —

```
(defun |dSearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|d| "domain"))
```

—————

defun Hyperdoc package search

[constructorSearch p1597]

— defun pSearch —

```
(defun |pSearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|p| "package"))
```

—————

defun Hyperdoc constructor search

[constructorSearch p1597]

— defun kSearch —

```
(defun |kSearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|k| "constructor"))
```

—————

defun Hyperdoc default constructor search

[constructorSearch p1597]

— defun ySearch —

```
(defun |ySearch| (filter)
  (|constructorSearch| (|checkFilter| filter) '|y| "constructor"))
```

—————

defun Read libdb.text at file-position n

— defun dbRead 0 —

```
(defun |dbRead| (n)
  (with-open-file
    (instream (concat (getenv "AXIOM") "/algebra/libdb.text"))
    (file-position instream n)
    (read-line instream)))
```

—————

defun String trim with newlines removed

— defun libdbTrim 0 —

```
(defun |libdbTrim| (s)
  (string-trim '(#\space #\tab #\newline) (substitute #\space #\newline s)))
```

—————

defun Hyperdoc common constructor search

```
[dbKind p??]
[conSpecialString? p1598]
[conPage p1599]
[lassoc p??]
[downcase p1272]
[downlink p1570]
[kPage p??]
[htInitPage p1510]
[htpSetProperty p1503]
[dbName p??]
[htQuery p??]
[htShowPage p1512]
[grepSearchQuery p??]
[constructorSearchGrep p??]
[$lowerCaseConTb p??]
```

— **defun constructorSearch** —

```
(defun |constructorSearch| (filter key kind)
  (let (parse pageName name u line newkind page message)
    (declare (special |$lowerCaseConTb|))
    (cond
      ((null filter) nil)
      ((setq parse (|conSpecialString?| filter)) (|conPage| parse))
      ((setq pageName
        (lassoc (downcase filter)
          '(("union" . |DomainUnion|)
            ("record" . |DomainRecord|)
            ("mapping" . |DomainMapping|)
            ("enumeration" . |DomainEnumeration|))))
        (|downlink| pageName))
      (t
        (setq name (if (stringp filter) (intern filter) filter))
        (when (setq u (hget |$lowerCaseConTb| name))
          (setq filter (princ-to-string (car u))))
        (cond
          ((setq line (|conPageFastPath| (downcase filter)))
            (setq newkind
              (case (|dbKind| line)
                (#\p "package")
                (#\d "domain")
                (#\c "category")))
              (cond
                ((or (equal kind "constructor") (equal kind newkind))
                  (|kPage| line))
                (t
```

```

(setq page (|htInitPage| "Query Page" nil))
(|htpSetProperty| page '|line| line)
(setq message
  (list "{\\em " (|dbName| line) "} is not a {\\em " kind
        "} but a {\\em " newkind
        "}. Would you like to view it?\\vspace{1}" ))
(|htQuery| message '|grepConstructorSearch| 't)
(|htShowPage|)))
((equal filter "*")
  (|grepSearchQuery| kind
    (list filter key kind '|constructorSearchGrep| )))
(t (|constructorSearchGrep| filter key kind))))))

```

defun conSpecialString?

```

[ifcar p??]
[string2Words p??]
[ncParseFromString p1310]
[member p1238]
[conLowerCaseConTran p1590]
[contained p??]
[kisValidType p1626]
[concat p1237]
[dbString2Words p1591]
[string2Constructor p1590]
[conSpecialString? p1598]

```

— defun conSpecialString? —

```

(defun |conSpecialString?| (&REST a1 &AUX options filter)
  (let (secondTime t1 words parse form u)
    (setq filter (car a1))
    (setq options (cdr a1))
    (setq secondtime (ifcar options))
    (setq t1 (|string2Words| filter))
    (setq parse
      (cond
        ((and (consp t1) (not (qcdr t1))) ; t1 is [s]
          (setq words (|ncParseFromString| (qcar t1)))
          ((every #'(lambda (x) (null (|member| x '("and" "or" "not")))) words)
            (|ncParseFromString| filter))))
        (t
          (null parse) nil)
        (t
          (setq form (|conLowerCaseConTran| parse))

```

```
(cond
  ((or (member (ifcar form) '(|and| |or| |not|)) (contained '* form)) nil)
  ((equal filter "Mapping") nil)
  ((setq u (|kisValidType| form)) u)
  (secondTime nil)
  (t
   (setq u
    (reduce #'concat
      (loop for x in (|dbString2Words| filter)
        collect (|string2Constructor| x))))
    (|conSpecialString?| u t))))))
```

Page construction

defun conPage

```
[form2HtString p??]
[downcase p1272]
[lassq p??]
[downlink p1570]
[conPageFastPath p1600]
[kPage p??]
[ySearch p1596]
[$conArgstrings p??]
```

— defun conPage —

```
(defun |conPage| (a &rest b)
  (let (|$conArgstrings| form da pageName line)
    (declare (special |$conArgstrings|))
    ; the next 4 lines allow e.g. MATRIX INT ==> Matrix Integer (see kPage)
    (setq form (if (atom a) (cons a b) a))
    (setq |$conArgstrings| (loop for x in (ifcdr a) collect (|form2HtString| x)))
    (cond ((null (atom a)) (setq a (car a))))
    (setq da (downcase a))
    (cond
      ((setq pageName
        (lassq da
          '(|type| . |CategoryType|)
          (|union| . |DomainUnion|)
          (|record| . |DomainRecord|)
          (|mapping| . |DomainMapping|)
          (|enumeration| . |DomainEnumeration|))))
        (|downlink| pageName))
      ((setq line (|conPageFastPath| da)) ; lower case name of cons?
```

```

(|kPage| line form))
((setq line (|conPageFastPath| (upcase a))) ; upper case an abbrev?
(|kPage| line form))
(t (|ySearch| a)))) ; slow search (include default packages)

```

defun gets line quickly for constructor name or abbreviation

```

[length p??]
[charPosition p??]
[lassq p??]
[dbRead p1596]
[conPageConEntry p1600]
[$lowerCaseConTb p??]

```

— defun conPageFastPath —

```

(defun |conPageFastPath| (x)
  (let (s name entry lineNumber)
    (declare (special |$lowerCaseConTb|))
    (setq s (princ-to-string x))
    (unless (> (|#| s) (|charPosition| #\* s 0)) ; quit if name has * in it
      (setq name (cond ((stringp x) (intern x)) (t x)))
      (setq entry (hget |$lowerCaseConTb| name))
      (when entry
        ;'dbLineNumbers property is set by function dbAugmentConstructorDataTable
        (if (setq lineNumber (lassq '|dbLineNumber| (cddr entry)))
            (|dbRead| lineNumber)
            (|conPageConEntry| (car entry)))))))

```

defun conPageConEntry

```

[conPageConEntry buildLibdbConEntry (vol9)]
[$conname p??]
[$conform p??]
[$exposed? p??]
[$doc p??]
[$kind p??]

```

— defun conPageConEntry —

```
(defun |conPageConEntry| (entry)
  (let (|$conname| |$conform| |$exposed?| |$doc| |$kind|)
    (declare (special |$conname| |$conform| |$exposed?| |$doc| |$kind|))
    (setq |$conname| nil)
    (setq |$conform| nil)
    (setq |$exposed?| nil)
    (setq |$doc| nil)
    (setq |$kind| nil)
    (|buildLibdbConEntry| entry)))
```

defun kdPageInfo

```
[htSay p1509]
[nequal p??]
[bcHt p1508]
[htSayStandard p1511]
[kPageArgs p??]
[length p??]
[extractFileNameFromPath p??]
[subseq p??]
[getdatabase p1196]
[htSay p1509]
[htMakePage p1513]
```

— defun kdPageInfo —

```
(defun |kdPageInfo| (name abbrev nargs conform signature file?)
  (let (sourceFileName filename)
    (|htSay| '|{\sf | name }"|)
    (when (nequal abbrev name) (|bcHt| (list '| has abbreviation | abbrev)))
    (when file? (|bcHt| (list " is a source file.")))
    (cond
      ((eql nargs 0)
       (when (nequal abbrev name) (|bcHt| ".")))
      (t
       (when (nequal abbrev name) (|bcHt| " and"))
       (|bcHt|
        (if (eql nargs 1)
            " takes one argument:"
            (list '| takes | (princ-to-string nargs) '| arguments:|))))))
    (|htSayStandard| "\\indentrel{2}")
    (when (> nargs 0) (|kPageArgs| conform signature))
    (|htSayStandard| "\\indentrel{-2}")
    (when (char= (elt name (1- (|#| name))) #\&)
      (setq name (subseq name 0 (1- (|#| name))))))
```



```

(setq sourceFileName (getdatabase (intern name) 'sourcefile))
(setq filename (|extractFileNameFromPath| sourceFileName))
(when (nequal filename "")
  (|htSayStandard| "\\newline{ }")
  (|htSay| "The source code for the constructor is found in "))
(|htMakePage|
  (list (list '|text| "\\unixcommand{" filename "}-{\\$AXIOM/lib/SPAEDIT "
    sourceFileName " " name "}")
    (when (nequal nargs 0) (|htSay| ".")))))

```

defun kArgPage

```

[httpProperty p1502]
[getConstructorModemap p??]
[position p??]
[sublisFormal p??]
[mkDomTypeForm p1603]
[domainDescendantsOf p1603]
[httpSetProperty p1503]
[dbShowCons p1641]

```

— defun kArgPage —

```

(defun |kArgPage| (htPage arg)
  (let (conform op args domname source n typeForm domTypeForm descendants rank)
    (setq conform (|httpProperty| htPage '|conform|))
    (setq op (car conform))
    (setq args (cdr conform))
    (setq domname (|httpProperty| htPage '|domname|))
    (setq source (cddar (|getConstructorModemap| op)))
    (setq n (|position| arg args))
    (setq typeForm (|sublisFormal| args (elt source n)))
    (setq domTypeForm (|mkDomTypeForm| typeForm conform domname))
    (setq descendants (|domainDescendantsOf| typeForm domTypeForm))
    (|httpSetProperty| htPage '|cAlist| descendants)
    (setq rank
      (unless (> n 4) (elt '(|First| |Second| |Third| |Fourth| |Fifth|) n)))
    (|httpSetProperty| htPage '|rank| rank)
    (|httpSetProperty| htPage '|thing| "argument")
    (|dbShowCons| htPage '|names|)))

```

defun mkDomTypeForm

```
[sublislis p??]
[mkDomTypeForm p1603]
[hasIndent p??]
```

— defun mkDomTypeForm —

```
(defun |mkDomTypeForm| (typeForm conform domname)
  (cond
    (domname (sublislis (cdr domname) (cdr conform) typeForm))
    ((and (consp typeForm) (eq (qcar typeForm) '|Join|))
     (cons '|Join|
           (loop for t1 in (qcdr typeForm) collect
                 (|mkDomTypeForm| t1 conform domname))))
    ((null (|hasIdent| typeForm)) typeForm)))
```

defun domainDescendantsOf

```
[systemError p??]
[simpHasPred p??]
[quickAnd p??]
[domainsOf p??]
[ifcdr p??]
[qcar p??]
[qcdr p??]
[assoc p??]
[listSort p??]
[function p??]
[delete p??]
```

— defun domainDescendantsOf —

```
(defun |domainDescendantsOf| (conform domform)
  (labels (
    (catScreen (r alist)
      (let (t1 item pred pred1 npred)
        (dolist (x r)
          (unless (and (consp x) (member (qcar x) '(attribute signature)))
            (|systemError| x))
          (setq alist
                (dolist (anitem alist (nreverse0 t1))
                  (setq item (car anitem))
                  (setq pred (cdr anitem))
```

```

      (when (and
        (setq pred1 (|simpHasPred| (list '|has| item x)))
        (setq npred (|quickAnd| pred1 pred)))
        (setq t1 (cons (cons item npred) t1))))))
    alist))
; keep only those domains that appear in ALL parts of Join
(jfn (arg domlist)
  (let (y r item pred u keepList alist)
    (setq y (car arg))
    (setq r (cdr arg))
    (setq alist (|domainsOf| y (ifcar domlist)))
    (dolist (x r)
      (setq domlist (ifcdr domlist))
      (when (and (consp x) (eq (qcar x) 'category) (consp (qcdr x)))
        (setq alist (catScreen (cddr x) alist)))
      (setq keepList nil)
      (dolist (dom (|domainsOf| x (ifcar domlist)))
        (setq item (car dom))
        (setq pred (cdr dom))
        (when (setq u (|assoc| item alist))
          (setq keepList
            (cons (cons item (|quickAnd| (cdr u) pred)) keepList))))
      (setq alist keepList))
    (dolist (pair alist)
      (rplacd pair (|simpHasPred| (cdr pair)))
      (|listSort| #'glesseqp alist)))
  (if (consp conform)
    (cond
      ((eq (qcar conform) '|Join|)
        (jfn
          (|delete| '|Type| object) (qcdr conform))
          (|delete| '|Type| object) (ifcdr domform)))
      ((eq (qcar conform) 'category) nil)
      (t (|domainsOf| conform domform)))
    (|domainsOf| conform domform)))

```

54.2 Branches of Constructor Page

defun kiPage

[httpProperty p1502]
 [mkConform p1628]
 [kDomainName p1624]
 [errorPage p??]

```
[capitalize p??]
[htInitPage p1510]
[htCopyPropList p??]
[dbShowConsDoc1 p1646]
[htShowPage p1512]
[$conformsAreDomains p??]
```

— **defun kiPage** —

```
(defun |kiPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs args conform domname heading page)
    (declare (special |$conformsAreDomains|))
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq args (sixth lt1))
    (setq conform (|mkConform| kind name args))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (setq heading
          (list "Description of " (|capitalize| kind) " {\sf " name args "}"))
        (setq page (|htInitPage| heading (|htCopyPropList| htPage)))
        (setq |$conformsAreDomains| domname)
        (|dbShowConsDoc1| htPage conform nil)
        (|htShowPage|))))))
```

—————

defun kePage

```
[httpProperty p1502]
[concat p1237]
[kDomainName p1624]
[errorPage p??]
[httpSetProperty p1503]
[mkConform p1628]
[opOf p??]
[capitalize p??]
[form2HtString p??]
[sublisFormal p??]
[ifcdr p??]
```

```

[getConstructorExports p??]
[simpHasPred p??]
[pluralSay p??]
[length p??]
[htInitPage p1510]
[htCopyProplist p??]
[htSayStandard p1511]
[htpSetProperty p1503]
[htMakePage p1513]
[menuButton p??]
[htSay p1509]
[bcConPredTable p??]
[htBigSkip p??]
[kePageDisplay p1608]
[kePageOpAlist p1607]
[htSowPage p??]
[$conformsAreDomains p??]

```

— defun kePage —

```

(defun |kePage| (htPage junk)
  (declare (ignore junk))
  (let (|$conformsAreDomains| lt1 kind name nargs args constring domname
        conform conname heading data conlist attrlist oplist prefix page)
    (declare (special |$conformsAreDomains|))
    (setq lt1 (|htpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq args (sixth lt1))
    (setq constring (concat name args))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (|htpSetProperty| htPage '|domname| domname)
        (setq |$conformsAreDomains| domname)
        (setq conform (|mkConform| kind name args))
        (setq conname (|opOf| conform))
        (setq heading
          (list (|capitalize| kind) " {\sf "
            (if domname (|form2HtString| domname nil t) constring) "}" ))
        (setq data
          (|sublisFormal|
            (or (ifcdr domname) (cdr conform))
            (|getConstructorExports| (or domname conform) t)))
        (setq conlist (car data))

```

```

(setq attrlist (cadr data))
(setq oplist (cddr data))
(when domname
  (dolist (x conlist) (rplac (cdr x) (|simpHasPred| (cdr x))))
  (dolist (x attrlist) (rplac (cddr x) (|simpHasPred| (cddr x))))
  (dolist (x oplist) (rplac (cddr x) (|simpHasPred| (cddr x))))
  (setq prefix
    (|pluralSay| (+ (+ (|#| conlist) (|#| attrlist)) (|#| oplist))
      "Export" "Exports"))
  (setq page
    (|htInitPage| (append prefix (cons " of " heading))
      (|htCopyPropList| htPage)))
  (|htSayStandard| "\\beginmenu ")
  (|htpSetProperty| page '|data| data)
  (when conlist
    (|htMakePage|
      (list
        (list '|bcLinks|
          (list (|menuButton|) "" '|dbShowCons1| conlist '|names|))))
    (|htSayStandard| "\\tab{2}")
    (|htSay| "All attributes and operations from:")
    (|bcConPredTable| conlist (|opOf| conform) (cdr conform)))
  (when attrlist
    (when conlist (|htBigSkip|))
    (|kePageDisplay| page "attribute" (|kePageOpAlist| attrlist)))
  (when oplist
    (when (or conlist attrlist) (|htBigSkip|))
    (|kePageDisplay| page "operation" (|kePageOpAlist| oplist)))
  (|htSayStandard| " \\endmenu ")
  (|htShowPage|))))

```

defun kePageOpAlist

```

[lassoc p??]
[insertAlist p981]
[zeroOneConvert p??]

```

— defun kePageOpAlist —

```

(defun |kePageOpAlist| (oplist)
  (let (op sig pred u opAlist)
    (dolist (item oplist)
      (setq op (car item))
      (setq sig (cadr item))
      (setq pred (cddr item))

```

```

(setq u (lassoc op opAlist))
(setq opAlist
  (|insertAlist| (|zeroOneConvert| op)
                 (cons (list sig pred) u)
                 opAlist)))
opAlist))

```

defun kePageDisplay

```

[length p??]
[htpSetProperty p1503]
[htMakePage p1513]
[menuButton p??]
[htSayStandard p1511]
[htSay p1509]
[pluralize p??]
[dbGatherData p??]
[dbSowOpItems p??]

```

— defun kePageDisplay —

```

(defun |kePageDisplay| (htPage which opAlist)
  (let (count total expandProperty data)
    (setq count (|#| opAlist))
    (cond
      ((eql count 0) nil)
      (t
       (setq total
         (apply #'(loop for entry in opAlist collect (|#| (cdr entry))))))
       (if (string= which "operation")
           (|htpSetProperty| htPage '|opAlist| opAlist)
           (|htpSetProperty| htPage '|attrAlist| opAlist))
       (setq expandProperty
         (if (string= which "operation")
             '|expandOperations|
             '|expandAttributes|))
       (|htpSetProperty| htPage expandProperty '|lists|)
       (|htMakePage|
        (list
         (list '|bcLinks| (list (|menuButton|) "" '|dbShowOps| which '|names|))))
         (|htSayStandard| "\\tab{2}")
         (unless (= count total)
           (if (eql count 1)
               (|htSay| "1 name for ")
               (|htSay| (princ-to-string count) " names for "))))

```

```
(if (> total 1)
  (|htSay| (princ-to-string total) " " (|pluralize| which)
    " are explicitly exported:")
  (|htSay| "1 " which " is explicitly exported:"))
(setq data (|dbGatherData| htPage opAlist which '|names|))
(|dbShowOpItems| which data nil))))
```

defun ksPage

```
[|httpProperty| p1502]
[kDomainName p1624]
[errorPage p??]
[form2HtString p??]
[|httpSetProperty| p1503]
[|htInitPageNoScroll| p1511]
[|htCopyPropList| p??]
[|htSay| p1509]
[|htSayStandard| p1511]
[|dbSearchOrder| p1610]
[|dbShowCons| p1641]
```

— defun ksPage —

```
(defun |ksPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs domname heading domain conform page u)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (setq heading
          (if (null domname)
            (|httpProperty| htPage '|heading|)
            (list "{\\sf " (|form2HtString| domname nil t) "}"))))
        (when domname
          (|httpSetProperty| htPage '|domname| domname)
          (|httpSetProperty| htPage '|heading| heading))
        (setq domain (unless (string= kind "category") (eval domname)))
        (setq conform (|httpProperty| htPage '|conform|))
        (setq page
```



```

(|htInitPageNoScroll| (|htCopyPropList| htPage)
  (cons "Search order for " heading)))
(|htSay| (concat
  "When an operation is not defined by the domain, the following "
  "domains are searched in order for a \"default definition\"))
(|htSayStandard| "\\beginscroll ")
(setq u (|dbSearchOrder| conform domname domain))
(|httpSetProperty| htPage '|cAlist| u)
(|httpSetProperty| htPage '|thing| "constructor")
(|dbShowCons| htPage '|names|))))

```

defun dbSearchOrder

```

[opOf p??]
[dbInfovec p??]
[getdatabase p1196]
[simpCatPredicate p??]
[sublislis p??]
[kTestPred p1639]
[devaluate p??]
[kFormatSlotDomain p??]
[dbSubConform p1640]
[dbAddChain p1641]
[$domain p??]
[$infovec p??]
[$predvec p??]

```

— defun dbSearchOrder —

```

(defun |dbSearchOrder| (conform domname |$domain|)
  (declare (special |$domain|))
  (let (|$infovec| name u catpredvec catinfo catvec p pred pak catform res
        catforms t1)
    (declare (special |$infovec| |$predvec|))
    (setq conform (or domname conform))
    (setq name (|opOf| conform))
    (setq |$infovec| (|dbInfovec| name))
    (when |$infovec|
      (setq u (elt |$infovec| 3))
      (setq |$predvec|
        (if |$domain| (elt |$domain| 3) (getdatabase name 'predicates)))
      (setq catpredvec (car u))
      (setq catinfo (cadr u))
      (setq catvec (caddr u)))

```

```

(setq catforms
  (dotimes (i (maxindex catvec) (nreverse0 t1))
    (cond
      ((progn
        (setq pred
          (|simpCatPredicate|
            (progn
              (setq p
                (sublislis (cdr conform) |$FormalMapVariableList|
                  (|kTestPred| (elt catpredvec i))))
              (if |$domain| (eval p) p))))
          (when (and domname (contained '$ pred))
            (setq pred (subst domname '$ pred :test #'equal)))
            (and (setq pak (elt catinfo i)) pred))
        (setq t1
          (cons
            (cons
              (cond
                ((and pak (null (identp pak)))
                  (|devaluate| pak))
                (t
                  (setq catform (|kFormatSlotDomain| (elt catvec i)))
                  (setq res (|dbSubConform| (cdr conform)
                    (cons pak (cons '$ (cdr catform)))))
                  (when domname (setq res (subst domname '$ res :test #'equal)))
                  res))
              pred)
            t1))))))
  (append (|dbAddChain| conform) catforms))))

```

defun kcPage

```

[httpProperty p1502]
[kDomainName p1624]
[qcar p??]
[errorPage p??]
[opOf p??]
[form2HtString p??]
[htInitPage p1510]
[htCopyPropList p??]
[httpSetProperty p1503]
[dbpHasDefaultCategory? p??]
[htSay p1509]
[brCon p??]
[htSayStandard p1511]

```

```

[htBeginMenu p??]
[htMakePage p1513]
[satBreak p??]
[nequal p??]
[hget p1234]
[hasNewInfoAlist p??]
[htEndMenu p??]
[htShowPage p1512]
[$defaultPackageNamesHT p??]

```

— defun kcPage —

```

(defun |kcPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs xpart domname conform conname heading page message)
    (declare (special |$defaultPackageNamesHT|))
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq xpart (fourth lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (setq conform (|httpProperty| htPage '|conform|))
        (setq conname (|opOf| conform))
        (setq heading
          (if (null domname)
              (|httpProperty| htPage '|heading|)
              (list "{\\sf " (|form2HtString| domname nil t) "}"))))
        (setq page
          (|htInitPage| (cons "Cross Reference for " heading)
            (|htCopyProplist| htPage)))
        (when domname
          (|httpSetProperty| htPage '|domname| domname)
          (|httpSetProperty| htPage '|heading| heading))
        (when (and (string= kind "category")
          (|dbpHasDefaultCategory?| xpart))
          (|htSay| "This category has default package ")
          (|bcCon| (concat name #\\&) ""))
        (|htSayStandard| "\\newline")
        (|htBeginMenu| 3)
        (|htSayStandard| "\\item ")
        (setq message
          (if (string= kind "category")
              (list "Categories it directly extends")
              (list "Categories the "

```

```
(if (string= kind "default package") "package" kind)
  " belongs to by assertion"))))
(|htMakePage|
(list
  (list '|bcLinks|
    (list "\\menuitemstyle{Parents}"
      (list (list '|text| "\\tab{12}" message)) '|kcpPage| nil))))
(|satBreak|)
(setq message
  (if (string= kind "category")
    (list "All categories it is an extension of")
    (list "All categories the " kind " belongs to")))
(|htMakePage|
(list
  (list '|bcLinks|
    (list "\\menuitemstyle{Ancestors}"
      (list (list '|text| "\\tab{12}" message)) '|kcaPage| nil))))
(when (string= kind "category")
  (|satBreak|)
  (|htMakePage|
  (list
    (list '|bcLinks|
      (list "\\menuitemstyle{Children}"
        (list (list '|text| "\\tab{12}"
          "Categories which directly extend this category"))))))
  (|satBreak|)
  (|htMakePage|
  (list
    (list '|bcLinks|
      (list "\\menuitemstyle{Descendants}"
        (list (list '|text| "\\tab{12}"
          "All categories which extend this category"))))))
  (|satBreak|)
  (setq message "Constructors mentioning this as an argument type")
  (|htMakePage|
  (list
    (list '|bcLinks|
      (list "\\menuitemstyle{Dependents}"
        (list (list '|text| "\\tab{12}" message)) '|kcdPage| nil))))
  (when (nequal kind "category")
    (|satBreak|)
    (|htMakePage|
    (list
      (list '|bcLinks|
        (list "\\menuitemstyle{Lineage}"
          "\\tab{12}Constructor hierarchy used for operation lookup"
          '|ksPage| nil))))
    (when (string= kind "category")
      (|satBreak|)
      (|htMakePage|
```

```

(list
  (list '|bcLinks|
    (list "\\menuitemstyle{Domains}"
      (list (list '|text| "\\tab{12}"
        "All domains which are of this category"))
        '|kcdPage| nil))))))
(unless (string= kind "category")
  (|satBreak|)
  (|htMakePage|
    (list
      (list '|bcLinks|
        (list "\\menuitemstyle{Clients}" "\\tab{12}Constructors"
          '|kcuPage| nil))))))
    (if (hget |$defaultPackageNamesHT| conname)
      (|htSay| " which {\\em may use} this default package")
      (|htSay| " which {\\em use} this " kind)))
    (when (or (nequal kind "category") (|dbpHasDefaultCategory?| xpart))
      (|satBreak|)
      (setq message
        (if (string= kind "category")
          (list "Constructors {\\em used by} its default package")
          (list "Constructors {\\em used by} the " kind)))
      (|htMakePage|
        (list
          (list '|bcLinks|
            (list "\\menuitemstyle{Benefactors}"
              (list (list '|text| "\\tab{12}" message) '|kcnPage| nil))))))
        (when (|hasNewInfoAlist| conname)
          (|satBreak|)
          (setq message (list "Cross reference for capsule implementation"))
          (|htMakePage|
            (list
              (list '|bcLinks|
                (list "\\menuitemstyle{CapsuleInfo}"
                  (list (list '|text| "\\tab{12}" message) '|kciPage| nil))))))
            (|htEndMenu| 3)
            (|htShowPage|))))))

```

defun kcpPage

```

[httpProperty p1502]
[kDomainName p1624]
[errorPage p??]
[qcar p??]
[form2HtString p??]

```

```
[httpSetProperty p1503]
[opOf p??]
[htInitPage p1510]
[htCopyProplist p??]
[parentsOf p??]
[sublislis p??]
[dbShowCons p1641]
```

— defun kcpPage —

```
(defun |kcpPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs domname heading conform conname page parents choice)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
     ((and (consp domname) (eq (qcar domname) '|error|))
      (|errorPage| htPage domname))
     (t
      (setq heading
        (if (null domname)
            (|httpProperty| htPage '|heading|)
            (list "{\\sf " (|form2HtString| domname nil t) "}"))))
      (when domname
        (|httpSetProperty| htPage '|domname| domname)
        (|httpSetProperty| htPage '|heading| heading))
      (setq conform (|httpProperty| htPage '|conform|))
      (setq conname (|opOf| conform))
      (setq page
        (|htInitPage| (cons "Parents of " heading) (|htCopyProplist| htPage)))
      (setq parents (|parentsOf| conname))
      (when domname
        (setq parents (sublislis (cdr domname) (cdr conform) parents)))
      (|httpSetProperty| htPage '|cAlist| parents)
      (|httpSetProperty| htPage '|thing| "parent")
      (setq choice (if domname '|parameters| '|names|))
      (|dbShowCons| htPage choice))))))
```

—————

defun reduceAlistForDomain

```
[sublislis p??]
[simpHasPred p??]
```

[nreverse0 p??]

— defun reduceAlistForDomain —

```
(defun |reduceAlistForDomain| (alist domform conform)
  (let (pred result)
    (setq alist (sublislis (cdr domform) (cdr conform) alist))
    (dolist (pair alist)
      (rplacd pair (|simpHasPred| (cdr pair) domform)))
    (dolist (pair alist (nreverse0 result))
      (setq pred (cdr pair))
      (when pred (setq result (cons pair result))))))
```

—————

defun kcaPage

[kcaPage1 p1617]
[ancestorsOf p??]

— defun kcaPage —

```
(defun |kcaPage| (htPage junk)
  (declare (ignore junk))
  (|kcaPage1| htPage "category" " an "
    "ancestor" #'|ancestorsOf| nil))
```

—————

defun kcdPage

[kcaPage1 p1617]
[descendantsOf p??]

— defun kcdPage —

```
(defun |kcdPage| (htPage junk)
  (declare (ignore junk))
  (|kcaPage1| htPage "category" " a "
    "descendant" #'|descendantsOf| t))
```

—————

defun kcdoPage

[kcdoPage p1617]
[domainsOf p??]

— defun kcdoPage —

```
(defun |kcdoPage| (htPage junk)
  (declare (ignore junk))
  (|kcaPage1| htPage "domain" " a "
    "descendant" #'|domainsOf| nil))
```

defun kcaPage1

[httpProperty p1502]
[kDomainName p1624]
[errorPage p??]
[form2HtString p??]
[httpSetProperty p1503]
[opOf p??]
[augmentHasArgs p1619]
[listSort p??]
[function p??]
[dbShowCons p1641]

— defun kcaPage1 —

```
(defun |kcaPage1| (htPage kind article whichever fn isCatDescendants?)
  (declare (ignore article))
  (let (lt1 name nargs domname heading conform conname ancestors choice)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (setq heading
          (if (null domname)
              (|httpProperty| htPage '|heading|)
              (list "{\\sf " (|form2HtString| domname nil t) "}"))))
        (when (and domname (null isCatDescendants?))
```



```

      (|httpSetProperty| htPage '|domname| domname)
      (|httpSetProperty| htPage '|heading| heading))
    (setq conform (|httpProperty| htPage '|conform|))
    (setq conname (|opOf| conform))
    (setq ancestors (FUNCALL fn conform domname))
    (unless (string= whichever "ancestor")
      (setq ancestors (|augmentHasArgs| ancestors conform)))
    (setq ancestors (|listSort| #'glesseq ancestors))
    (|httpSetProperty| htPage '|cAlist| ancestors)
    (|httpSetProperty| htPage '|thing| whichever)
    (setq choice '|names|)
    (|dbShowCons| htPage choice))))

```

defun kccPage

```

[httpProperty p1502]
[kDomainName p1624]
[qcar p??]
[errorPage p??]
[form2HtString p??]
[httpSetProperty p1503]
[opOf p??]
[htInitPage p1510]
[htCopyProplist p??]
[augmentHasArgs p1619]
[childrenOf p??]
[reduceAlistForDomain p1615]
[dbShowCons p1641]

```

— defun kccPage —

```

(defun |kccPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs domname heading conform conname page children)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq nargs (third lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
      ((and (consp domname) (eq (qcar domname) '|error|))
        (|errorPage| htPage domname))
      (t
        (setq heading

```

```

(if (null domname)
  (|httpProperty| htPage '|heading|)
  (list "{\\sf " (|form2HtString| domname nil t) "}"))))
(when domname
  (|httpSetProperty| htPage '|domname| domname)
  (|httpSetProperty| htPage '|heading| heading))
(setq conform (|httpProperty| htPage '|conform|))
(setq conname (|opOf| conform))
(setq page
  (|htInitPage| (cons "Children of " heading) (|htCopyProplist| htPage)))
(setq children (|augmentHasArgs| (|childrenOf| conform) conform))
(when domname
  (setq children (|reduceAlistForDomain| children domname conform)))
(|httpSetProperty| htPage '|cAlist| children)
(|httpSetProperty| htPage '|thing| "child")
(|dbShowCons| htPage '|names|))))

```

defun augmentHasArgs

```

[opOf p??]
[length p??]
[nreverse0 p??]
[extractHasArgs p??]
[getConstructorForm p??]

```

— defun augmentHasArgs —

```

(defun |augmentHasArgs| (alist conform)
  (let (conname args n name p result pred)
    (setq conname (|opOf| conform))
    (setq args (ifcdr conform))
    (cond
      (args
        (setq n (|#| args))
        (dolist (item alist (nreverse0 result))
          (setq name (car item))
          (setq p (cdr item))
          (setq pred
            (if (consp (|extractHasArgs| p))
              p
              (|quickAnd| p
                (cons '|hasArgs|
                  (take n (ifcdr (|getConstructorForm| (|opOf| name))))))))
          (setq result (cons (cons name pred) result))))
      (t alist))))

```

defun kcdePage

```
[httpProperty p1502]
[concat p1237]
[nequal p??]
[ncParseFromString p1310]
[opOf p??]
[getDependentsOfConstructor p1620]
[getConstructorForm p??]
[httpSetProperty p1503]
[dbShowCons p1641]
```

— defun kcdePage —

```
(defun |kcdePage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name args conname constring conform pakname domlist cAlist)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq args (sixth lt1))
    (setq conname (intern name))
    (setq constring (concat name args))
    (setq conform
      (if (nequal kind "default package")
          (|ncParseFromString| constring)
          (cons (intern name) (cdr (|ncParseFromString| (concat #\d args))))))
    (setq pakname (|opOf| conform))
    (setq domlist (|getDependentsOfConstructor| pakname))
    (setq cAlist
      (loop for x in domList collect (cons (|getConstructorForm| x) t)))
    (|httpSetProperty| htPage '|cAlist| cAlist)
    (|httpSetProperty| htPage '|thing| "dependent")
    (|dbShowCons| htPage '|names|)))
```

defun getDependentsOfConstructor

```
[readLibPathFast p??]
[pathname p1232]
```

```
[rread p915]
[rshut p??]
```

— defun getDependentsOfConstructor —

```
(defun |getDependentsOfConstructor| (con)
  (let (stream val)
    (setq stream
      (|readLibPathFast| (|pathname| (list ' |dependents| 'database ' |a|))))
    (setq val (|rread| con stream nil))
    (rshut stream)
    val))
```

defun kcuPage

```
[httpProperty p1502]
[concat p1237]
[nequal p??]
[ncParseFromString p1310]
[opOf p??]
[getUsersOfConstructor p1622]
[getConstructorForm p??]
[httpSetProperty p1503]
[dbShowCons p1641]
```

— defun kcuPage —

```
(defun |kcuPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name args constring conform pakname domlist cAlist)
    (setq lt1 (|httpProperty| htPage ' |parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq args (sixth lt1))
    (setq constring (concat name args))
    (setq conform
      (if (nequal kind "default package")
          (|ncParseFromString| constring)
          (cons (intern name)
                (cdr (|ncParseFromString| (concat #\d args))))))
    (setq pakname
      (if (string= kind "category")
          (intern (concat name #\&))
          (|opOf| conform)))
```

```
(setq domlist (|getUsersOfConstructor| pakname))
(setq cAlist
  (loop for x in domlist collect (cons (|getConstructorForm| x) t)))
(|httpSetProperty| htPage '|cAlist| cAlist)
(|httpSetProperty| htPage '|thing| "user")
(|dbShowCons| htPage '|names|)))
```

defun getUsersOfConstructor

```
[readLibPathFast p??]
[pathname p1232]
[rread p915]
[rshut p??]
```

— defun getUsersOfConstructor —

```
(defun |getUsersOfConstructor| (con)
  (let (stream val)
    (setq stream (|readLibPathFast| (|pathname| (list '|users| 'database '|a|))))
    (setq val (|rread| con stream nil))
    (rshut stream)
    val))
```

defun kcnPage

```
[kDomainName p1624]
[qcar p??]
[errorPage p??]
[httpProperty p1502]
[form2HtString p??]
[httpSetProperty p1503]
[concat p1237]
[pname p1235]
[opOf p??]
[getImports p??]
[sublislis p??]
[dbShowCons p1641]
```

— defun kcnPage —

```

(defun |kcnPage| (htPage junk)
  (declare (ignore junk))
  (let (lt1 kind name nargs domname heading conform pakname domlist cAlist
        conname)
    (setq lt1 (|httpProperty| htPage '|parts|))
    (setq kind (first lt1))
    (setq name (second lt1))
    (setq conname (intern name))
    (setq nargs (third lt1))
    (setq domname (|kDomainName| htPage kind name nargs))
    (cond
     ((and (consp domname) (eq (qcar domname) '|error|))
      (|errorPage| htPage domname))
     (t
      (setq heading
        (if (null domname)
          (|httpProperty| htPage '|heading|)
          (list "{\\sf " (|form2HtString| domname nil t) "}"))))
      (if domname
        (|httpSetProperty| htPage '|domname| domname)
        (|httpSetProperty| htPage '|heading| heading))
      (setq conform (|httpProperty| htPage '|conform|))
      (setq pakname
        (if (string= kind "category")
          (intern (concat (pname conname) #\&))
          (|opOf| conform)))
      (setq domlist (|getImports| pakname))
      (when domname
        (setq domlist
          (sublislis (cons domname (cdr domname))
                     (cons '$ (cdr conform)) domlist)))
      (setq cAlist (loop for x in domList collect (cons x t)))
      (|httpSetProperty| htPage '|cAlist| cAlist)
      (|httpSetProperty| htPage '|thing| "benefactor")
      (|dbShowCons| htPage '|names|))))))

```

defun koPageInputAreaUnchanged?

```

[|httpLabelInputString| p1503]
[concat p1237]
[|httpProperty| p1502]

```

— defun koPageInputAreaUnchanged? —

```

(defun |koPageInputAreaUnchanged?| (htPage nargs)

```

```
(equal
(loop for i from 1 to nargs
  collect
    (|httpLabelInputString| htPage (intern (concat "*" (princ-to-string i)))))
(|httpProperty| htPage '|inputAreaList|)))
```

defun kDomainName

```
[httpSetProperty p1503]
[httpLabelInputString p1503]
[getdatabase p1196]
[kArgumentCheck p1625]
[concat p1237]
[unabbrev p??]
[mkConform p1628]
[kisValidType p1626]
[dbMkEvaluable p1625]
[spad-reader p??]
[$PatternVariableList p17]
```

— defun kDomainName —

```
(defun |kDomainName| (htPage kind name nargs)
  (let (inputAreaList conname args n argTailPart argString typeForm
        evaluatedTypeForm)
    (|httpSetProperty| htPage '|domname| nil)
    (setq inputAreaList
      (loop for i from 1 to nargs for var in |$PatternVariableList|
        collect (|httpLabelInputString| htPage var)))
    (|httpSetProperty| htPage '|inputAreaList| inputAreaList)
    (setq conname (intern name))
    (setq args
      (loop for x in inputAreaList
        for domain? in (cdr (getdatabase conname 'cosig))
        collect (or (|kArgumentCheck| domain? x) nil)))
    (when (some #'identity (loop for x in args collect (null x)))
      (cond
        ((> (setq n (apply #'+ (loop for x in args collect (if x 1 0)))) 0)
         (list '|error| nil "\\centerline{You gave values for only {\em "
           n " } of the {\em " (|#| args) "}}"
           "\\centerline{parameters of {\sf " name
           "}}\\vspace{1}\\centerline{Please enter either {\em all} or "
           "{\em none} of the type parameters}")
          nil)
        (t
```

```

(setq argString
  (cond
    ((null args) "()")
    (t
     (setq argTailPart
       (apply #'concat
         (loop for x in (ifcdr args) collect (concat (cons "," x))))
       (apply #'concat (list "(" (car args) argTailPart "))))))
(setq typeForm
  (or (catch 'spad_reader (|unabbrev| (|mkConform| kind name argString)))
      (list 'error| 'invalidType| (concat name argString))))
(if (null (setq evaluatedTypeForm (|kisValidType| typeForm)))
    (list 'error| 'invalidType| (concat name argString))
    (|dbMkEvalable| evaluatedTypeForm))))))

```

defun kArgumentCheck

[conSpecialString? p1598]
 [opOf p??]
 [form2String p??]

— defun kArgumentCheck —

```

(defun |kArgumentCheck| (domain? s)
  (let (form)
    (cond
      ((string= s "") nil)
      ((and domain? (setq form (|conSpecialString?| s)))
       (if (null (ifcdr form))
           (list (princ-to-string (|opOf| form)))
           (|form2String| form)))
      (t (list s)))))

```

defun dbMkEvalable

[getdatabase p1196]
 [mkEvalable p1115]

— defun dbMkEvalable —

```

(defun |dbMkEvalable| (form)

```



```
(let (op kind)
  (setq op (car form))
  (setq kind (getdatabase op 'constructorkind))
  (if (eq kind 'category|)
      form
      (|mkEvalable| form))))
```

defun topLevelInterpEval

```
[processInteractive p342]
[$ProcessInteractiveValue p344]
[$noEvalTypeMsg p1122]
```

— defun topLevelInterpEval —

```
(defun |topLevelInterpEval| (x)
  (let (|ProcessInteractiveValue| |noEvalTypeMsg|)
    (declare (special |ProcessInteractiveValue| |noEvalTypeMsg|))
    (setq |ProcessInteractiveValue| t)
    (setq |noEvalTypeMsg| t)
    (|processInteractive| x nil)))
```

defun kisValidType

```
[processInteractive p342]
[member p1238]
[kCheckArgumentNumbers p1627]
[$ProcessInteractiveValue p344]
[$noEvalTypeMsg p1122]
[spad-reader p??]
```

— defun kisValidType —

```
(defun |kisValidType| (typeForm)
  (let (|ProcessInteractiveValue| |noEvalTypeMsg| it1)
    (declare (special |ProcessInteractiveValue| |noEvalTypeMsg|))
    (setq |ProcessInteractiveValue| t)
    (setq |noEvalTypeMsg| t)
    (setq it1 (catch 'spad_reader (|processInteractive| typeForm nil)))
    (when (and (consp it1) (consp (qcar it1)))
```

```
(|member| (caar it1) '(domain |SubDomain|)))
(and (|kCheckArgumentNumbers| (qcdr it1)) (qcdr it1)))
```

defun kCheckArgumentNumbers

[getdatabase p1196]
[kCheckArgumentNumber p??]

— defun kCheckArgumentNumbers —

```
(defun |kCheckArgumentNumbers| (tt)
  (let (conname args cosig)
    (setq conname (car tt))
    (setq args (cdr tt))
    (setq cosig (ifcdr (getdatabase conname 'cosig)))
    (every #'identity
      (loop for domain? in cosig for x in args
        collect (if domain? (|kCheckArgumentNumbers| x) t)))))
```

defun parseNoMacroFromString

[next p330]
[function p??]
[ncloopParse p329]
[lineoftoks p403]
[incString p331]
[StreamNull p623]
[pf2Sex p597]

— defun parseNoMacroFromString —

```
(defun |parseNoMacroFromString| (s)
  (setq s
    (|next| #'|ncloopParse|
      (|next| #'|lineoftoks|
        (|incString| s))))
  (if (|StreamNull| s)
    nil
    (|pf2Sex| (cadar s))))
```

defun mkConform

```
[nequal p??]
[parseNoMacroFromString p1627]
[sayBrightlyNT p??]
[pp p??]
[systemError p??]
[ncParseFromString p1310]
[concat p1237]
```

— defun mkConform —

```
(defun |mkConform| (kind name argString)
  (let (form parse)
    (cond
      ((nequal kind "default package")
       (setq form (concat name argString))
       (setq parse (|parseNoMacroFromString| form))
       (cond
         ((null parse)
          (|sayBrightlyNT| "Won't parse: ")
          (|pp| form)
          (|systemError| "Keywords in argument list?"))
         ((atom parse) (cons parse nil))
         (t parse)))
      (t
       (cons (intern name) (cdr (|ncParseFromString| (concat #\d argString))))))))
```

—————

54.3 Operation Page for a Domain Form from Scratch**defun conOpPage**

```
[dbCompositeWithMap p1630]
[htpProperty p1502]
[conOpPage1 p1629]
[dbExtractUnderlyingDomain p1631]
```

— defun conOpPage —

```
(defun |conOpPage| (htPage conform)
  (declare (ignore conform))
  (let (updown domname)
    (setq updown (|dbCompositeWithMap| htPage))
```

```
(cond
  ((string= updown "DOWN")
    (setq domname (|httpProperty| htPage '|domname|))
    (|conOpPage1| (|dbExtractUnderlyingDomain| domname)
      (list (cons '|updomain| domname))))
  (t
    (setq domname (|httpProperty| htPage '|updomain|))
    (|conOpPage1| domname nil))))
```

defun conOpPage1

```
[ifcar p??]
[opOf p??]
[dbSpecialOperations p1652]
[conPageFastPath p1600]
[dbXParts p??]
[concat p1237]
[mkConform p1628]
[capitalize p??]
[ncParseFromString p1310]
[dbSourceFile p??]
[isExposedConstructor p1094]
[htInitPage p1510]
[httpSetProperty p1503]
[lassoc p??]
[ifcdr p??]
[koPage p1631]
[$Primitives p??]
```

— defun conOpPage1 —

```
(defun |conOpPage1| (&rest arg)
  (let (bindingsAlist conname domname line parts name sig args isFile kind
        constring capitalKind signature sourceFileName emString heading page
        selectedOperation options conform)
    (declare (special |$Primitives|))
    (setq conform (car arg))
    (setq options (cdr arg))
    (setq bindingsAlist (ifcar options))
    (setq conname (|opOf| conform))
    (cond
      ((member conname |$Primitives|) (|dbSpecialOperations| conname))
      (t
        (setq domname (unless (atom conform) conform))
```

```

(setq line (|conPageFastPath| conname))
(setq parts (|dbXParts| line 7 1))
(setq kind (first parts))
(setq name (second parts))
(setq sig (fifth parts))
(setq args (sixth parts))
(setq isFile (null kind))
(setq kind (or kind "package"))
(rplaca parts kind)
(setq constring (concat name args))
(setq conform (|mkConform| kind name args))
(setq capitalKind (|capitalize| kind))
(setq signature (|ncParseFromString| sig))
(setq sourceFileName (|dbSourceFile| (intern name)))
(setq emString (list "{\\sf " constring "}"))
(setq heading (cons capitalKind (cons " " emString)))
(unless (|isExposedConstructor| conname)
  (setq heading (cons "Unexposed " heading)))
(setq page (|htInitPage| heading nil))
(|httpSetProperty| page '|isFile| t)
(|httpSetProperty| page '|fromConOpPage1| t)
(|httpSetProperty| page '|parts| parts)
(|httpSetProperty| page '|heading| heading)
(|httpSetProperty| page '|kind| kind)
(|httpSetProperty| page '|domname| domname)
(|httpSetProperty| page '|conform| conform)
(|httpSetProperty| page '|signature| signature)
(when
  (setq selectedOperation (lassoc '|selectedOperation| (ifcdr options)))
  (|httpSetProperty| page '|selectedOperation| selectedOperation))
(loop for item in bindingsAlist
  collect (|httpSetProperty| page (car item) (cdr item)))
(|koPage| page "operation"))))

```

defun dbCompositeWithMap

[[httpProperty](#) p1502]
 [[dbExtractUnderlyingDomain](#) p1631]

— defun dbCompositeWithMap —

```

(defun |dbCompositeWithMap| (htPage)
  (let (domain opAlist)
    (cond
      ((|httpProperty| htPage '|updomain|) "UP")

```

```
(t
  (setq domain (|httpProperty| htPage '|domname|))
  (cond
    ((null domain) nil)
    (t
     (setq opAlist (|httpProperty| htPage '|opAlist|))
     (when
      (|dbExtractUnderlyingDomain| (|httpProperty| htPage '|domname|))
      "DOWN"))))))
```

defun dbExtractUnderlyingDomain

[isValidType p??]

— defun dbExtractUnderlyingDomain —

```
(defun |dbExtractUnderlyingDomain| (domain)
  (some #'identity
    (loop for x in (ifcdr domain) when (|isValidType| x) collect x)))
```

54.4 Operation Page from Main Page

defun koPage

```
[httpProperty p1502]
[concat p1237]
[koPageInputAreaUnchanged? p1623]
[kDomainName p1624]
[errorPage p??]
[form2HtString p??]
[capitalize p??]
[httpSetProperty p1503]
[koPageAux p1632]
```

— defun koPage —

```
(defun |koPage| (htPage which)
  (let (lt1 kind name nargs args constring conname u IT1 domname headingString
        heading)
```

```

(setq lt1 (|httpProperty| htPage '|parts|))
(setq kind (first lt1))
(setq name (second lt1))
(setq nargs (third lt1))
(setq args (sixth lt1))
(setq constring (concat name args))
(setq conname (intern name))
(setq IT1 (setq u (|httpProperty| htPage '|domname|)))
(setq domname
  (cond
    ((and (consp IT1) (equal (qcar IT1) conname)
      (or (eq (|httpProperty| htPage '|fromConOpPage1|) t)
        (|koPageInputAreaUnchanged?| htPage nargs)))
      u)
    (t (|kDomainName| htPage kind name nargs))))
(cond
  ((and (consp domname) (eq (qcar domname) '|error|))
    (|errorPage| htPage domname))
  (t
    (|httpSetProperty| htPage '|domname| domname)
    (setq headingString (if domname (|form2HtString| domname nil t) constring))
    (setq heading (list (|capitalize| kind) " {\sf " headingString "}" ))
    (|httpSetProperty| htPage '|which| which)
    (|httpSetProperty| htPage '|heading| heading)
    (|koPageAux| htPage which domname heading))))

```

defun koPageFromKKPage

[koPageAux p1632]
 [httpProperty p1502]

— defun koPageFromKKPage —

```

(defun |koPageFromKKPage| (htPage ao)
  (|koPageAux| htPage ao (|httpProperty| htPage '|domname|)
    (|httpProperty| htPage '|heading|)))

```

defun koPageAux

[httpSetProperty p1503]
 [koAttrs p??]

```
[koOps p??]
[assoc p??]
[systemError p??]
[dbShowOperationsFromConform p??]
```

— defun koPageAux —

```
(defun |koPageAux| (htPage which domname heading)
  (let (conform selectedOperation opAlist)
    (|httpSetProperty| htPage '|which| which)
    (setq domname (|httpProperty| htPage '|domname|))
    (setq conform (|httpProperty| htPage '|conform|))
    (setq heading (|httpProperty| htPage '|heading|))
    (setq opAlist
      (cond
        ((string= which "attribute") (|koAttrs| conform domname))
        ((string= which "general operation") (|koOps| conform domname t))
        (t (|koOps| conform domname))))
    (cond
      ((setq selectedOperation (|httpProperty| htPage '|selectedOperation|))
       (setq opAlist
         (list (or (|assoc| selectedOperation opAlist) (|systemError|))))))
      (|dbShowOperationsFromConform| htPage which opAlist)))
```

—————

defun koPageAux1

```
[httpProperty p1502]
[dbShowOperationsFromConform p??]
```

— defun koPageAux1 —

```
(defun |koPageAux1| (htPage opAlist)
  (let (which)
    (setq which (|httpProperty| htPage '|which|))
    (|dbShowOperationsFromConform| htPage which opAlist)))
```

—————

defun koaPageFilterByName

```
[httpLabelInputString p1503]
[koaPageFilterByCategory p??]
[pmTransFilter p??]
```



```
[httpProperty p1502]
[dbGetInputString p??]
[superMatch? p??]
[downcase p1272]
[httpSetProperty p1503]
```

— defun koaPageFilterByName —

```
(defun |koaPageFilterByName| (htPage functionToCall)
  (let (filter which opAlist)
    (cond
      ((string= (|httpLabelInputString| htPage '|filter|) "")
        (|koaPageFilterByCategory| htPage functionToCall))
      (t
        (setq filter (|pmTransFilter| (|dbGetInputString| htPage)))
        (setq which (|httpProperty| htPage '|which|))
        (setq opAlist
          (loop for x in (|httpProperty| htPage '|opAlist|)
            when (|superMatch?| filter (downcase (princ-to-string (car x))))
            collect x))
        (|httpSetProperty| htPage '|opAlist| opAlist)
        (funcall functionToCall htPage nil))))))
```

—————

54.5 Get Constructor Documentation

defun dbConstructorDoc,hn

```
[length p??]
[sublislis p??]
[$FormalMapVariableList p17]
[$sig p??]
[$args p??]
```

— defun dbConstructorDoc,hn —

```
(defun |dbConstructorDoc,hn| (sig)
  (declare (special |$sig| |$args|))
  (and (equal (|#| |$sig|) (|#| sig))
    (equal |$sig| (sublislis |$args| |$FormalMapVariableList| sig))))
```

—————

defun dbConstructorDoc,gn

[dbConstructorDoc,hn p1634]
 [\$op p??]

— defun dbConstructorDoc,gn —

```
(defun |dbConstructorDoc,gn| (arg)
  (let (op alist)
    (declare (special |$op|))
    (setq op (car arg))
    (setq alist (cdr arg))
    (and |$op|
      (some #'identity
        (loop for item in alist when (|dbConstructorDoc,hn| (car item))
          collect (or (cdr item) '(""))))))))
```

—————

defun dbConstructorDoc

[dbConstructorDoc,fn p??]
 [\$sig p??]
 [\$op p??]

— defun dbConstructorDoc —

```
(defun |dbConstructorDoc| (conform |$op| |$sig|)
  (declare (special |$op| |$sig|))
  (|dbConstructorDoc,fn| conform))
```

—————

defun dbDocTable

[hget p1234]
 [make-hashtable p??]
 [originsInOrder p1636]
 [dbAddDocTable p1636]
 [\$docTable p??]
 [\$docTableHash p??]

— defun dbDocTable —

```
(defun |dbDocTable| (conform)
  (let (|$docTable| table)
    (declare (special |$docTable| |$docTableHash|))
    (cond
      ((setq table (hget |$docTableHash| conform))
       table)
      (t
       (setq |$docTable| (make-hashtable 'id))
       (loop for x in (|originsInOrder| conform) do (|dbAddDocTable| x))
       (|dbAddDocTable| conform)
       (hput |$docTableHash| conform |$docTable|)
       |$docTable|))))
```

defun originsInOrder

```
[getdatabase p1196]
[assocleft p??]
[ancestorsOf p??]
[parentsOf p??]
[originsInOrder p1636]
[insert p??]
```

— defun originsInOrder —

```
(defun |originsInOrder| (conform)
  (let (con argl acc)
    (setq con (car conform))
    (setq argl (cdr conform))
    (cond
      ((eq (getdatabase con 'constructorkind) '|category|)
       (assocleft (|ancestorsOf| conform nil)))
      (t
       (setq acc (assocleft (|parentsOf| con)))
       (loop for x in acc do
         (loop for y in (|originsInOrder| x) do
           (setq acc (|insert| y acc))))
       acc))))
```

defun dbAddDocTable

```
[opOf p??]
[getConstructorForm p??]
```

```
[sublislis p??]
[getdatabase p1196]
[hput p1234]
[hget p1234]
[$docTable p??]
```

— defun dbAddDocTable —

```
(defun |dbAddDocTable| (conform)
  (let (conname storedArgs op alist op1 sig doc tmp)
    (declare (special |$docTable|))
    (setq conname (|opOf| conform))
    (setq storedArgs (cdr (|getConstructorForm| conname)))
    (setq tmp (sublislis (cons '$ (cdr conform)) (cons '% storedArgs
                                                         (getdatabase (|opOf| conform) 'documentation)))
    (loop for item in tmp do
      (setq op (car item))
      (setq alist (cdr item))
      (setq op1
        (cond
          ((eq op '(|Zero|)) 0)
          ((eq op '(|One|)) 1)
          (t op)))
      (loop for item1 in alist do
        (setq sig (first item1))
        (setq doc (second item1))
        (hput |$docTable| op1 (cons (cons conform alist)
                                     (hget |$docTable| op1)))))))
```

—————

defun dbGetDocTable,hn

```
[sublislis p??]
[qcdr p??]
[qcar p??]
[$which p??]
[$conform p??]
[$sig p??]
[$FormalMapVariableList p17]
```

— defun dbGetDocTable,hn —

```
(defun |dbGetDocTable,hn| (arg)
  (let (sig doc alteredSig pred)
    (declare (special |$which| |$conform| |$sig| |$FormalMapVariableList|))
```

```

(setq sig (car arg))
(setq doc (cdr arg))
(if (string= |$which| "attribute")
    (and (consp sig) (eq (qcar sig) '|attribute|) (equal (qcdr sig) |$sig|)
        doc)
    (progn
      (setq pred
        (and
          (eql (|#| |$sig|) (|#| sig))
          (setq alteredSig
            (sublislis (ifcdr |$conform|) |$FormalMapVariableList| sig))
            (equal alteredSig |$sig|)))
      (when (and pred doc
        (and (consp doc) (eq (qcar doc) '|constant|)) (qcdr doc) doc)
        '("))))))

```

defun dbGetDocTable,gn

```

[lastatom p??]
[dbGetDocTable,hn p1637]
[$conform p??]

```

— defun dbGetDocTable,gn —

```

(defun |dbGetDocTable,gn| (u)
  (let (code p comments)
    (declare (special |$conform|))
    (setq |$conform| (car u))
    (when (atom |$conform|) (setq |$conform| (list |$conform|)))
    (setq code (lastatom u))
    (setq comments
      (some #'identity
        (loop for entry in (cdr u)
          when (setq p (|dbGetDocTable,hn| entry))
            collect p)))
    (when comments (cons |$conform| (cons (car comments) code)))))

```

defun dbGetDocTable

```

[string2Integer p??]
[dbConstructorDoc p1635]

```

```
[qcdr p??]
[hget p1234]
[dbGetDocTable,gn p1638]
[$sig p??]
[$which p??]
[$conform p??]
[$op p??]
```

— defun dbGetDocTable —

```
(defun |dbGetDocTable| (op |$sig| docTable |$which| aux)
  (declare (special |$sig| |$which|))
  (let (doc origin s)
    (declare (special |$conform| |$op|))
    (when (and (null (integerp op)) (digitp (elt (setq s (princ-to-string op)) 0)))
      (setq op (|string2Integer| s)))
    (cond
      ((and (consp aux) (consp (qcar aux)))
       (setq doc (|dbConstructorDoc| (car aux) |$op| |$sig|))
       (setq origin (if (qcdr aux) (cons '|ifp| aux) (car aux)))
       (cons origin doc))
      (t
       (some #'identity
        (loop for x in (hget docTable op)
              collect (|dbGetDocTable,gn| x)))))))
```

defun kTestPred

```
[testBitVector p??]
[simpHasPred p??]
[$predvec p??]
[$domain p??]
```

— defun kTestPred —

```
(defun |kTestPred| (n)
  (declare (special |$predvec| |$domain|))
  (cond
    ((eql n 0) t)
    (|$domain| (|testBitVector| |$predvec| n))
    (t (|simpHasPred| (elt |$predvec| (1- n))))))
```

defun dbAddChainDomain

```
[dbInfovec p??]
[dbSubConform p1640]
[kFormatSlotDomain p??]
[devaluate p??]
[$infovec p??]
```

— defun dbAddChainDomain —

```
(defun |dbAddChainDomain| (conform)
  (let (name args template form)
    (declare (special |$infovec|))
    (setq name (car conform))
    (setq args (cdr conform))
    (setq |$infovec| (|dbInfovec| name))
    (when |$infovec|
      (setq template (elt |$infovec| 0))
      (when (setq form (elt template 5))
        (|dbSubConform| args (|kFormatSlotDomain| (|devaluate| form)))))))
```

—————

defun dbSubConform

```
[position p??]
[dbSubConform p1640]
[$FormalMapVariableList p17]
```

— defun dbSubConform —

```
(defun |dbSubConform| (args u)
  (let (n y)
    (declare (special |$FormalMapVariableList|))
    (cond
      ((atom u)
        (if (>= (setq n (|position| u |$FormalMapVariableList|)) 0)
          (elt args n)
          u))
      ((and (consp u) (eq (car u) '|local|) (consp (cdr u)) (eq (caddr u) nil))
        (setq y (cadr u))
        (|dbSubConform| args y))
      (t
        (loop for x in u collect (|dbSubConform| args x))))))
```

—————

defun dbAddChain

[dbAddChainDomain p1640]
 [dbAddChain p1641]

— defun dbAddChain —

```
(defun |dbAddChain| (conform)
  (let (u)
    (when (setq u (|dbAddChainDomain| conform))
      (unless (atom u)
        (cons (cons u t) (|dbAddChain| u))))))
```

—————

54.6 Constructor Page Menu**defun dbShowCons**

[htpProperty p1502]
 [pmTransFilter p??]
 [ifcar p??]
 [dbGetInputString p??]
 [bcErrorPage p??]
 [constructor? p??]
 [superMatch? p??]
 [downcase p1272]
 [emptySearchPage p??]
 [htInitPageNoScroll p1511]
 [htCopyProplist p??]
 [htSetProperty p1503]
 [dbShowCons p1641]
 [member p1238]
 [dbShowCons1 p1643]
 [\$exposedOnlyIfTrue p??]

— defun dbShowCons —

```
(defun |dbShowCons| (&rest args)
  (let (cAlist filter abbrev? conname subject u options key htPage)
    (declare (special |$exposedOnlyIfTrue|))
    (setq htPage (first args))
    (setq key (second args))
    (setq options (cddr args))
    (setq cAlist (|htpProperty| htPage 'cAlist)))
```



```

(cond
  ((eq key '|filter|)
   (setq filter
    (|pmTransFilter| (or (ifcar options) (|dbGetInputString| htPage))))
  (cond
    ((and (consp filter) (eq (car filter) '|error|))
     (|bcErrorPage| filter))
    (t
     (setq abbrev? (eq (|httpProperty| htPage '|exclusion|) '|abbrs|))
     (setq u
      (loop for x in cAlist
        when (progn
          (setq conname (caar x))
          (setq subject (if abbrev? (|constructor?| conname) conname))
          (|superMatch?| filter (downcase (princ-to-string subject)))))
        collect x))
     (cond
      ((null u)
       (|emptySearchPage| "constructor" filter))
      (t
       (setq htPage (|htInitPageNoScroll| (|htCopyProplist| htPage)))
       (|httpSetProperty| htPage '|cAlist| u)
       (|dbShowCons| htPage (|httpProperty| htPage '|exclusion|))))))
  (t
   (when (member key '(|exposureOn| |exposureOff|))
    (setq |$exposedOnlyIfTrue| (eq key '|exposureOn|))
    (setq key (|httpProperty| htPage '|exclusion|))
    (|dbShowCons1| htPage cAlist key))))

```

defun conPageChoose

```

[getConstructorForm p??]
[dbShowCons1 p1643]

```

— defun conPageChoose —

```

(defun |conPageChoose| (conname)
  (let (cAlist)
    (setq cAlist (list (cons (|getConstructorForm| conname) t)))
    (|dbShowCons1| nil cAlist '|names|)))

```

defun dbShowCons1

```

[remdup p??]
[isExposedConstructor p1094]
[opOf p??]
[conPage p1599]
[htpProperty p1502]
[union p??]
[dbConstructorKind p??]
[htCopyProplist p??]
[htInitPageNoScroll p1511]
[dbConsHeading p1649]
[htSayStandard p1511]
[htpSetProperty p1503]
[bcNameConTable p??]
[bcAbbTable p??]
[getCDTEntry p??]
[getdatabase p1196]
[bcUnixTable p1650]
[listSort p??]
[function p??]
[qlesseqp p??]
[dbShowConsDoc p1645]
[isExposedConstructor p1094]
[dbShowConditions p1648]
[bcConTable p??]
[assocleft p??]
[dbShowConsKinds p??]
[dbConsExposureMessage p1645]
[dbPresentCons p??]
[htShowPageNoScroll p1513]
[$conformsAreDomains p??]
[$exposedOnlyIfTrue p??]

```

— **defun dbShowCons1** —

```

(defun |dbShowCons1| (htPage cAlist key)
  (let (|$conformsAreDomains| conlist kinds kind proplist page u flist result)
    (declare (special |$conformsAreDomains| |$exposedOnlyIfTrue|))
    (setq conlist
      (remdup
        (dolist (x cAlist result)
          (push
            (if |$exposedOnlyIfTrue|
              (|isExposedConstructor| (|opOf| (car x)))
              (car x))
            result))))))

```

```

(cond
  ((and (consp conlist) (eq (qcdr conlist) nil))
    (|conPage|
      (if (and htPage (|httpProperty| htPage '|domname|))
        (car conlist)
        (|opOf| (car conlist)))))
  (t
    (setq conlist (loop for x in conlist collect (|opOf| x)))
    (setq kinds
      (apply #'|union|
        (loop for x in conlist collect (|dbConstructorKind| x))))
    (setq kind
      (if (and (consp kinds) (eq (qcdr kinds) nil))
        (qcar kinds)
        '|constructor|))
    (setq proplist (when htPage (|htCopyProplist| htPage)))
    (setq page
      (|htInitPageNoScroll| proplist
        (|dbConsHeading| htPage conlist key kind)))
    (if (setq u (|httpProperty| page '|specialMessage|))
      (apply (car u) (cdr u)))
    (|htSayStandard| "\\beginscroll ")
    (|httpSetProperty| page '|cAlist| cAlist)
    (setq |$conformsAreDomains| (|httpProperty| page '|domname|))
    (cond
      ((eq key '|names|) (|bcNameConTable| conlist))
      ((eq key '|abbrs|)
        (|bcAbbTable|
          (loop for con in conlist collect (|getCDTEntry| con t))))
      ((eq key '|files|)
        (setq flist
          (loop for con in conlist collect (getdatabase con 'sourcefile)))
        (|bcUnixTable|
          (|listSort| #'glesseqp (remdup flist))))
      ((eq key '|documentation|) (|dbShowConsDoc| page conlist))
      (t
        (when |$exposedOnlyIfTrue|
          (setq cAlist
            (loop for x in cAlist
              when (|isExposedConstructor| (|opOf| (car x)))
                collect x)))
        (cond
          ((eq key '|conditions|) (|dbShowConditions| page cAlist kind))
          ((eq key '|parameters|)
            (|bcConTable| (remdup (assocleft cAlist))))
          ((eq key '|kinds|) (|dbShowConsKinds| cAlist))))
      (|dbConsExposureMessage|)
      (|htSayStandard| '|\\endscroll |)
      (|dbPresentCons| page kind key)
      (|htShowPageNoScroll|))))

```

defun dbConsExposureMessage

[htSay p1509]

[\$atLeastOneUnexposed p??]

— **defun dbConsExposureMessage** —

```
(defun |dbConsExposureMessage| ()
  (declare (special |$atLeastOneUnexposed|))
  (when |$atLeastOneUnexposed|
    (|htSay| "\newline{}-----\newline{}{\em *} = unexposed")))
```

defun dbShowConsKindsFilter

[httpSetProperty p1503]

[dbShowCons p1641]

[httpProperty p1502]

— **defun dbShowConsKindsFilter** —

```
(defun |dbShowConsKindsFilter| (htPage args)
  (|httpSetProperty| htPage '|cAlist| (second args))
  (|dbShowCons| htPage (|httpProperty| htPage '|exclusion|)))
```

defun dbShowConsDoc

[systemError p??]

[dbShowConsDoc1 p1646]

[getConstructorForm p??]

[opOf p??]

[httpProperty p1502]

[remdup p??]

— **defun dbShowConsDoc** —

```
(defun |dbShowConsDoc| (htPage conlist)
  (labels (
    (fn (cAlist x)
      (let ((index 0))
        (loop while (not (equal (caaar cAlist) x))
          do (setq index (1+ index))
              (setq cAlist (cdr cAlist))
              (unless cAlist (|systemError|)))
          index)))
    (let (cAlist)
      (cond
        ((null (cdr conlist))
         (|dbShowConsDoc1| htPage
          (|getConstructorForm| (|opOf| (car conlist))) nil))
        (t
         (setq cAlist (|httpProperty| htPage '|cAlist|))
         (loop for x in (remdup conlist) do
           (|dbShowConsDoc1| htPage
            (|getConstructorForm| x) (fn cAlist x))))))))
```

defun dbShowConsDoc1

```
[member p1238]
[httpProperty p1502]
[getl p1240]
[displayDomainOp p??]
[isExposedConstructor p1094]
[getConstructorDocumentation p1647]
[getConstructorSignature p??]
[getdatabase p1196]
[sublislis p??]
[sublisFormal p??]
[displayDomainOp p??]
[$TriangleVariableList p??]
[$Primitives p??]
```

— defun dbShowConsDoc1 —

```
(defun |dbShowConsDoc1| (htPage conform indexOrNil)
  (let (conargs conname lt1 exposeFlag doc signature sig)
    (declare (special |$TriangleVariableList| |$Primitives|))
    (setq conname (car conform))
    (setq conargs (cdr conform))
    (cond
```

```

(member conname |$Primitives|)
(setq conname (|httpProperty| htPage '|conname|))
(setq lt1 (getl conname '|documentation|))
(cond ((eq (caar lt1) '|constructor|) (caar lt1)))
(cond ((eq (caadar lt1) 'nil) (caadar lt1)))
(setq doc (car (cdadar lt1)))
(setq sig '((category domain) (|SetCategory|) (|SetCategory|)))
(|displayDomainOp| htPage "constructor"
  conform conname sig t doc indexOrNil '|dbSelectCon| nil nil))
(t
  (setq exposeFlag (|isExposedConstructor| conname))
  (setq doc (list (|getConstructorDocumentation| conname)))
  (setq signature (|getConstructorSignature| conname))
  (setq sig
    (if (eq (getdatabase conname 'constructorkind) '|category|)
      (sublislis conargs |$TriangleVariableList| signature)
      (|sublisFormal| conargs signature)))
  (|displayDomainOp| htPage "constructor" conform conname sig t doc
    indexOrNil '|dbSelectCon| (null exposeFlag) nil))))

```

defun getConstructorDocumentation

```

[lassoc p??]
[getdatabase p1196]
[qcar p??]
[qcaar p??]
[qcdar p??]
[qcadar p??]

```

— defun getConstructorDocumentation —

```

(defun |getConstructorDocumentation| (conname)
  (let (IT1)
    (setq IT1 (lassoc '|constructor| (getdatabase conname 'documentation)))
    (or
      (and (consp IT1) (consp (qcar IT1)) (null (qcaar IT1)) (consp (qcdar IT1))
        (qcadar IT1))
      "")))

```

defun dbSelectCon

```
[conPage p1599]
[opOf p??]
[httpProperty p1502]
```

— defun dbSelectCon —

```
(defun |dbSelectCon| (htPage which index)
  (declare (ignore which))
  (|conPage| (|opOf| (car (elt (|httpProperty| htPage '|cAlist|) index))))))
```

defun dbShowConditions

```
[httpProperty p1502]
[opOf p??]
[splitConTable p??]
[pluralize p??]
[length p??]
[dbSayItems p??]
[bcConPredTable p??]
[htSayHrule p??]
```

— defun dbShowConditions —

```
(defun |dbShowConditions| (htPage cAlist kind)
  (let (conform conname article whichever lt1 consNoPred consPred singular
        plural)
    (setq conform (|httpProperty| htPage '|conform|))
    (setq conname (|opOf| conform))
    (setq article (|httpProperty| htPage '|article|))
    (setq whichever (|httpProperty| htPage '|whichever|))
    (setq lt1 (|splitConTable| cAlist))
    (setq consNoPred (car lt1))
    (setq consPred (cdr lt1))
    (setq singular (list kind " is"))
    (setq plural (list (|pluralize| (princ-to-string kind)) " are"))
    (|dbSayItems| (|#| consNoPred) singular plural " unconditional")
    (|bcConPredTable| consNoPred conname)
    (|htSayHrule|)
    (|dbSayItems| (|#| consPred) singular plural " conditional")
    (|bcConPredTable| consPred conname)))
```

defun dbConsHeading

```
[htpProperty p1502]
[length p??]
[remdup p??]
[form2HtString p??]
[capitalize p??]
[pluralize p??]
[member p1238]
[nequal p??]
[$exposedOnlyIfTrue p??]
```

— defun dbConsHeading —

```
(defun |dbConsHeading| (htPage conlist view kind)
  (let (thing place count rank modifier exposureWord firstWord prefix
        placepart connective heading)
    (declare (special |$exposedOnlyIfTrue|))
    (setq thing (or (and htPage (|htpProperty| htPage '|thing|)) "constructor"))
    (setq place
      (when htPage
        (or (|htpProperty| htPage '|domname|) (|htpProperty| htPage '|conform|))))
    (setq count (|#| (remdup conlist)))
    (cond
      ((string= thing "benefactor")
       (list (princ-to-string count) " Constructors Used by "
             (|form2HtString| place nil t) ))
      (t
       (setq modifier
         (cond
           ((string= thing "argument")
            (setq rank (and htPage (|htpProperty| htPage '|rank|)))
            (list " Possible " rank " "))
           ((eq kind '|constructor|)
            (list " "))
           (t
            (cons " " (|capitalize| (princ-to-string kind)) " "))))
       (setq exposureWord (when |$exposedOnlyIfTrue| '(" Exposed ")))
       (setq prefix
         (cond
           ((eql count 1)
            (cons (princ-to-string count)
                  (append modifier (list (|capitalize| thing))))))
           (t
            (setq firstWord (if (eql count 0) "No " (princ-to-string count)))
            (cons firstWord
                  (append exposureWord
                        (append modifier
                              (list (|capitalize| (|pluralize| thing)))))))))))
```



```

(setq placepart
  (when place (list " of {\em " (|form2HtString| place nil t) '))))
(setq heading (append prefix placepart))
(setq connective
  (if (|member| view '(|abbrs| |files| |kinds|)) " as " " with "))
(cond
  ((and (nequal count 0)
        (|member| view '(|abbrs| |files| |parameters| |conditions|))))
  (setq heading
    (append heading
      (list " viewed" connective "{\em " (princ-to-string view) "}")))))
heading)))

```

defun dbShowConstructorLines

```

[getConstructorForm p??]
[intern p??]
[dbName p??]
[dbShowCons1 p1643]
[listSort p??]
[function p??]
[glesseq p??]

```

— defun dbShowConstructorLines —

```

(defun |dbShowConstructorLines| (lines)
  (let (cAlist)
    (setq cAlist
      (loop for line in lines
        collect (cons (|getConstructorForm| (|intern| (|dbName| line))) t)))
    (|dbShowCons1| nil (|listSort| #'glesseq cAlist) '|names|)))

```

defun bcUnixTable

```

[htSay p1509]
[htBeginTable p??]
[namestring p1230]
[findfile p??]
[htMakePage p1513]
[htEndTable p??]

```

```
[firstTime p??]
```

— defun bcUnixTable —

```
(defun |bcUnixTable| (u)
  (declare (special firstTime))
  (let (filename)
    (|htSay| "\\newline")
    (|htBeginTable|)
    (setq firstTime t)
    (loop for x in u do
      (|htSay| "{")
      (setq filename (namestring ($findfile (princ-to-string x) "SPAD")))
      (|htMakePage|
        (list
          (list '|text| "\\unixcommand{" (pathname-name x)
            "-{" $AXIOM/lib/SPAEDIT " filename "}" ")))
      (|htSay| "}"))
    (|htEndTable|)))
```

Special Code for Union, Mapping, and Record

defun dbSpecialDescription

```
[getConstructorForm p??]
[form2HtString p??]
[htInitPage p1510]
[httpSetProperty p1503]
[dbShowConsDoc1 p1646]
[htShowPage p1512]
[$conformsAreDomains p??]
```

— defun dbSpecialDescription —

```
(defun |dbSpecialDescription| (conname)
  (let (conform heading page)
    (declare (special |$conformsAreDomains|))
    (setq conform (|getConstructorForm| conname))
    (setq heading
      (list "Description of Domain {" $sf " (" |form2HtString| conform) "}))
    (setq page (|htInitPage| heading nil))
    (|httpSetProperty| page '|conname| conname)
    (setq |$conformsAreDomains| nil)
    (|dbShowConsDoc1| page conform nil))
```

```
(|htShowPage|)))
```

defun dbSpecialOperations

```
[htInitPage p1510]
[getConstructorForm p??]
[dbSpecialExpandIfNecessary p1653]
[getl p1240]
[form2HtString p??]
[httpSetProperty p1503]
[dbShowOp1 p??]
```

— defun dbSpecialOperations —

```
(defun |dbSpecialOperations| (conname)
  (let (page conform opAlist fromHeading)
    (setq page (|htInitPage| nil nil))
    (setq conform (|getConstructorForm| conname))
    (setq opAlist
      (|dbSpecialExpandIfNecessary| conform
        (cdr (getl conname '|documentation|))))
    (setq fromHeading (list " from domain {\sf " (|form2HtString| conform) "}"))
    (|httpSetProperty| page '|fromHeading| fromHeading)
    (|httpSetProperty| page '|conform| conform)
    (|httpSetProperty| page '|opAlist| opAlist)
    (|httpSetProperty| page '|noUsage| t)
    (|httpSetProperty| page '|condition?| '|no|)
    (|dbShowOp1| page opAlist "operation" '|names|)))
```

defun dbSpecialExports

```
[getConstructorForm p??]
[htInitPage p1510]
[form2HtString p??]
[dbSpecialExpandIfNecessary p1653]
[getl p1240]
[kePageDisplay p1608]
[htShowPage p1512]
```

— defun dbSpecialExports —

```
(defun |dbSpecialExports| (conname)
  (let (conform page opAlist)
    (setq conform (|getConstructorForm| conname))
    (setq page
      (|htInitPage| (list "Exports of {\sf " (|form2HtString| conform) ")") nil))
    (setq opAlist
      (|dbSpecialExpandIfNecessary| conform
        (cdr (getl conname '|documentation|))))
    (|kePageDisplay| page "operation" opAlist)
    (|htShowPage|)))
```

defun dbSpecialExpandIfNecessary

```
[qcar p??]
[qcdar p??]
[qcadar p??]
[qcdr p??]
```

— defun dbSpecialExpandIfNecessary —

```
(defun |dbSpecialExpandIfNecessary| (conform opAlist)
  (if (and (consp opAlist) (consp (qcar opAlist)) (consp (qcdar opAlist))
    (consp (qcadar opAlist)) (cdr (qcdr (qcadar opAlist)))))
    opAlist
    (dolist (item opAlist)
      (dolist (pair (cdr item))
        (rplacd pair (list t conform t (second pair))))))
    opAlist)
```

— initvars —

```
(defvar message1 (concatenate 'string
  "{\sf Record(a:A,b:B)} is used to create the class of pairs of objects "
  "made up of a value of type {\em A} selected by the symbol {\em a} and "
  "a value of type {\em B} selected by the symbol {\em b}. "
  "In general, the {\sf Record} constructor can take any number of arguments "
  "and thus can be used to create aggregates of heterogeneous components of "
  "arbitrary size selectable by name. "
  "{\sf Record} is a primitive domain of Axiom which cannot be "
  "defined in the Axiom language."))
```

— postvars —

```
(eval-when (eval load)
  (put '|Record| '|documentation|
    (subst message1 'message
      '(((|constructor| (nil message))
        (= (((|Boolean|) $ $)
          "\spad{r = s} tests for equality of two records \spad{r} and \spad{s}"))
        (|coerce| (((|OutputForm|) $)
          "\spad{coerce(r)} returns an representation of \spad{r} as an output form")
          (($ (|List| (|Any|)))
            , (concatenate 'string
              "\spad{coerce(u)}, where \spad{u} is the list \spad{[x,y]} for \spad{x} "
              "of type \spad{A} and \spad{y} of type \spad{B}, returns the record "
              "\spad{[a:x,b:y]}"))
              (|elt| ((A $ "a")
                , (concatenate 'string
                  "\spad{r . a} returns the value stored in record \spad{r} under "
                  "selector \spad{a}."))
                  ((B $ "b")
                    , (concatenate 'string
                      "\spad{r . b} returns the value stored in record \spad{r} "
                      "under selector \spad{b}."))))
              (|setelt| ((A $ "a" A)
                , (concatenate 'string
                  "\spad{r . a := x} destructively replaces the value stored in "
                  "record \spad{r} under selector \spad{a} by the value of \spad{x}. "
                  "Error: if \spad{r} has not been previously assigned a value."))
                  ((B $ "b" B)
                    , (concatenate 'string
                      "\spad{r . b := y} destructively replaces the value stored in "
                      "record \spad{r} under selector \spad{b} by the value of \spad{y}. "
                      "Error: if \spad{r} has not been previously assigned a value."))))
              :test #'equal)))
```

— initvars —

```
(defvar message2 (concatenate 'string
  "{\sf Union(A,B)} denotes the class of objects which are either "
  "members of domain {\em A} or of domain {\em B}. The {\sf Union} "
  "constructor can take any number of arguments. "
  "For an alternate form of {\sf Union} with \"tags\", see "
  "\downlink{Union(a:A,b:B)}{DomainUnion}. {\sf Union} is a primitive "
```

"domain of Axiom which cannot be defined in the Axiom language."))

— postvars —

```
(eval-when (eval load)
  (put '|UntaggedUnion| '|documentation|
    (subst message2 'message
      '((|constructor| (nil message))
        (= (((|Boolean|) $ $)
          , (concatenate 'string
            "\\spad{u = v} tests if two objects of the union are equal, "
            "that is, u and v are hold objects of same branch which are equal.))))
        (|case| (((|Boolean|) $ "A")
          , (concatenate 'string
            "\\spad{u case A} tests if \\spad{u} is of the type \\spad{A} "
            "branch of the union.)))
        (((|Boolean|) $ "B")
          , (concatenate 'string
            "\\spad{u case B} tests if \\spad{u} is of the \\spad{B} branch "
            "of the union.)))
        (|coerce| ((A $)
          , (concatenate 'string
            "\\spad{coerce(u)} returns \\spad{x} of type \\spad{A} if "
            "\\spad{x} is of the \\spad{A} branch of the union. "
            "Error: if \\spad{u} is of the \\spad{B} branch of the union.)))
        ((B $)
          , (concatenate 'string
            "\\spad{coerce(u)} returns \\spad{x} of type \\spad{B} if "
            "\\spad{x} is of the \\spad{B} branch of the union. "
            "Error: if \\spad{u} is of the \\spad{A} branch of the union.)))
        (($ A)
          , (concatenate 'string
            "\\spad{coerce(x)}, where \\spad{x} has type \\spad{A}, "
            "returns \\spad{x} as a union type.)))
        (($ B)
          , (concatenate 'string
            "\\spad{coerce(y)}, where \\spad{y} has type \\spad{B}, "
            "returns \\spad{y} as a union type.))))
    :test #'equal)))
```

— initvars —

```
(defvar message3 (concatenate 'string
  "{\\sf Union(a:A,b:B)} denotes the class of objects which are either "
  "members of domain {\\em A} or of domain {\\em B}. "
  "The symbols {\\em a} and {\\em b} are called \"tags\" and are used to "
  "identify the two \"branches\" of the union. "
  "The {\\sf Union} constructor can take any number of arguments and has an "
  "alternate form without {\\em tags} "
  "(see \\downlink{Union(A,B)}{UntaggedUnion}). "
  "This tagged {\\sf Union} type is necessary, for example, to disambiguate "
  "two branches of a union where {\\em A} and {\\em B} denote the same type. "
  "{\\sf Union} is a primitive domain of Axiom which cannot be "
  "defined in the Axiom language.))
```

— postvars —

```
(eval-when (eval load)
  (put '|Union| '|documentation|
    (subst message3 'message
      '(|constructor| (NIL MESSAGE))
        (= (((|Boolean|) $ $)
          ,(concatenate 'string
            "\\spad{u = v} tests if two objects of the union are equal, that "
            "is, \\spad{u} and \\spad{v} are objects of same branch which are equal.))))
        (|case| (((|Boolean|) $ "A")
          "\\spad{u case a} tests if \\spad{u} is of branch \\spad{a} of the union."
          (((|Boolean|) $ "B")
            "\\spad{u case b} tests if \\spad{u} is of branch \\spad{b} of the union."
            (|coerce| ((A $)
              ,(concatenate 'string
                "\\spad{coerce(u)} returns \\spad{x} of type \\spad{A} if "
                "\\spad{x} is of branch \\spad{a} of the union. "
                "Error: if \\spad{u} is of branch \\spad{b} of the union.))
                ((B $)
                  ,(concatenate 'string
                    "\\spad{coerce(u)} returns \\spad{x} of type \\spad{B} if "
                    "\\spad{x} is of branch \\spad{b} branch of the union. "
                    "Error: if \\spad{u} is of the \\spad{a} branch of the union.))
                    (($ A)
                      ,(concatenate 'string
                        "\\spad{coerce(x)}, where \\spad{x} has type \\spad{A}, returns "
                        "\\spad{x} as a union type.))
                        (($ B)
                          ,(concatenate 'string
                            "\\spad{coerce(y)}, where \\spad{y} has type \\spad{B}, returns "
                            "\\spad{y} as a union type.))))))
        :test #'equal)))
```

— initvars —

```
(defvar message4 (concatenate 'string
"{\\sf Mapping(T,S,...)} denotes the class of objects which are mappings from "
"a source domain ({\\em S,...}) into a target domain {\\em T}. The "
"{\\sf Mapping} constructor can take any number of arguments."
" All but the first argument is regarded as part of a source tuple for the "
"mapping. For example, {\\sf Mapping(T,A,B)} denotes the class of mappings "
"from {\\em (A,B)} into {\\em T}. "
"{\\sf Mapping} is a primitive domain of Axiom which cannot be defined in "
"the Axiom language."))
```

— postvars —

```
(eval-when (eval load)
(put '|Mapping| '|documentation|
(subst message4 'message
'(|constructor| (NIL MESSAGE))
(= (((|Boolean|) $ $)
"\\spad{u = v} tests if mapping objects are equal.")))
:test #'equal)))
```

— initvars —

```
(defvar message5 (concatenate 'string
"{\\em Enumeration(a1, a2 ,..., aN)} creates an object which is exactly one "
"of the N symbols {\\em a1}, {\\em a2}, ..., or {\\em aN}, N > 0. "
" The {\\em Enumeration} can constructor can take any number of symbols as "
"arguments."))
```

— postvars —


```

(eval-when (eval load)
  (put '|Enumeration| '|documentation|
    (subst message5 'message
      '(((|constructor| (nil message))
        (= (((|Boolean|) $ $)
          ,(concatenate 'string
            "\\spad{e = f} tests for equality of two enumerations \\spad{e} "
            "and \\spad{f}"))))
        (^= (((|Boolean|) $ $)
          ,(concatenate 'string
            "\\spad{e ^= f} tests that two enumerations \\spad{e} and "
            "\\spad{f} are not equal"))))
        (|coerce| (((|OutputForm|) $)
          ,(concatenate 'string
            "\\spad{coerce(e)} returns a representation of enumeration "
            "\\spad{r} as an output form"))
          (($ (|Symbol|))
            ,(concatenate 'string
              "\\spad{coerce(s)} converts a symbol \\spad{s} into an "
              "enumeration which has \\spad{s} as a member symbol"))))
        :test #'equal)))

```

defun lefts

[hkeys p1235]
 [hascategory-hash p??]

— defun lefts —

```

(defun |lefts| (u)
  (let (keys)
    (setq keys (hkeys *hascategory-hash*))
    (loop for x in keys when (equal (cdr x) u) collect x)))

```

Build Library Database (libdb.text,...)

defun dbMkForm

— defun dbMkForm —

```
(defun |dbMkForm| (x)
  (or (and (atom x) (cons x nil)) x))
```

defun libConstructorSig

```
[getdatabase p1196]
[take p??]
[length p??]
[sublislis p??]
[form2LispString p??]
[ncParseFromString p1310]
[sayBrightly p??]
[$TriangleVariableList p??]
```

— defun libConstructorSig —

```
(defun |libConstructorSig| (arg)
  (labels (
    (fn (x)
      (cond
        ((atom x) x)
        ((and (consp x) (eq (qcar x) '|Join|) (consp (qcdr x)))
         (list '|Join| (fn (qcadr x)) '|etc|))
        ((and (consp x) (eq (qcar x) 'category))
         '|etc|)
        (t
         (loop for y in x collect (fn y))))))
    (g (x u i)
      "does x appear in any but i-th element of u?"
      (some #'identity
        (loop for y in u for j from 1
          when (not (= i j))
            collect (contained x y)))))
    (let (conname argl formals keys sig sigpart)
      (declare (special |$TriangleVariableList|))
      (setq conname (car arg))
      (setq argl (cdr arg))
      (setq sig (cdar (getdatabase conname 'constructormodemap)))
      (setq formals (take (|#| argl) |$FormalMapVariableList|))
      (setq sig (sublislis formals |$TriangleVariableList| sig))
      (setq keys
        (loop for f in formals for i from 1
          collect (g f sig i)))
      (setq sig
        (fn (sublislis argl |$FormalMapVariableList| sig)))
```

```
(setq sig (cons (car sig)
  (loop for a in arg1 for s in (cdr sig) for k in keys
    collect (if k (list #\: a s) s))))
(setq sigpart (|form2LispString| (cons '|Mapping| sig)))
(unless (|ncParseFromString| sigpart)
  (|sayBrightly| (list "Won't parse: " sigpart)))
sigpart)))
```

Chapter 55

Utility functions

55.1 Utility functions

defun Delete an alist pair given the key

— defun delasc 0 —

```
(defun delasc (key alist)
  (remove key alist :key #'car))
```

—————

defun readline

— defun readline —

```
(defun readline (t1)
  (if t1
    (|read-line| t1)
    (|read-line| *STANDARD-INPUT*)))
```

—————

defun isWrapped

This was proven by ACL2 to accept any input and return either T or NIL. Note that ACL2 does not support FLOATP.

— defun isWrapped 0 ACL2 —

```
(defun isWrapped (x)
  (or (and (consp x) (eq (car x) 'wrapped))
      (acl2-numberp x)
      (stringp x)))

;==>
;(OR (EQUAL (ISWRAPPED X) T) (EQUAL (ISWRAPPED X) NIL))
```

—————

isWrapped : $t \rightarrow (\text{or } t \text{ nil})$

— defun isWrapped :proven —

```
(defun |isWrapped| (x)
  (or (and (consp x) (eq (qcar x) 'wrapped))
      (numberp x)
      (floatp x)
      (stringp x)))
```

—————

Chapter 56

The Proofs

— acl2 —

`\getchunk{defun isWrapped 0 ACL2}`

—————

Chapter 57

Signatures

22 type : FrameName → Symbol
22 frameName : Frame → FrameName
22 frameInteractive : Frame → Interactive
22 frameIOIndex : Frame → IOIndex
23 frameHiFiAccess : Frame → HiFiAccess
23 frameHistList : Frame → HistList
23 type : HistListLen → NonNegativeInteger
23 frameHistListLen : Frame → HistListLen
24 frameHistListAct : Frame → HistListAct
24 frameHistRecord : Frame → HistRecord
24 frameHistoryTable : Frame → HistoryTable
25 frameExposureData : Frame → ExposureData
25 enum : FrameArgs → (nil,drop,import,last,names,new,next)
25 frameSpad2Cmd : FrameArgs → nil
27 emptyInterpreterFrame : Symbol → Frame
27 emptyInterpreterFrame : Symbol → Frame
28 frameNames : nil → List Symbol
28 displayFrameNames : nil → nil
29 createCurrentInterpreterFrame : nil → Frame
29 updateFromCurrentInterpreterFrame : nil → nil
31 updateCurrentInterpreterFrame : nil → nil
31 frameEnvironment : FrameName → nil


```

31  findFrameInRing : FrameName → Union(Frame,nil)
32  changeToNamedInterpreterFrame : FrameName → nil
32  nextInterpreterFrame : nil → nil
33  previousInterpreterFrame : nil → nil
37  closeInterpreterFrame : FrameName → nil
88  embed2 : Symbol,Function,Function → Symbol
100 removeOption : Option → List Option
108 flattenOperationAlist : OperationAlist → OperationAlist
124 /untrace-reduce : Union(Atom,List) → Atom
126 transTraceItem : traceArgument → symbol
305 set-restart-hook : Void → 'restart
321 intloopReadConsole : (String Integer) → Throw
327 intloopPrefix? : String → Union(String,NIL)
329 intloopProcessString : (String,StepNo) → StepNo
330 next : (Function,Delay) → Delay
330 next1 : Delay → ParsePair
331 incString : String → Function
334 setCurrentLine : String → List(String)
334 mkprompt : Void → String
336 serverReadLine : Stream → String
356 intloopProcess : (StepNo,Boolean,Delay) → StepNo
366 incRenumber : Delay → Delay
366 incZip : (Function,Delay,Delay) → Delay
366 incZip1 : Delay → ParsePair
367 incIgen : Integer → Delay
369 incLude : (Int,List(String),Int,List(String),List(Int)) → Delay
394 incCommand? : String → Boolean
397 Delay : (Function,List(Any)) → Delay
623 StreamNull : Delay → Union(T,NIL)
1243 fracpart : or rational float → or rational float
1402 getHtMacroItem : String → Values (String NonNegativeInteger)
1468 bcGen : Command → nil
1469 doDoitButton : String,Command → nil
1469 doDoitButton : Command → nil

```

1470 executeInterpreterCommand : Command $\rightarrow$ nil

1662 isWrapped : t $\rightarrow$ (or t nil)

Chapter 58

The Interpreter

— Interpreter —

```
(setq *print-array* nil)
(setq *print-circle* nil)
(setq *print-pretty* nil)

(in-package "BOOT")
\getchunk{initvars}

;;; level 0 macros

\getchunk{defmacro bit-to-truth 0}
\getchunk{defmacro bvec-elt 0}
\getchunk{defmacro cdancols 0}
\getchunk{defmacro cdanrows 0}
\getchunk{defmacro cdaref2 0}
\getchunk{defmacro cdelt 0}
\getchunk{defmacro cdlen 0}
\getchunk{defmacro cdsetaref2 0}
\getchunk{defmacro cdsetelt 0}
\getchunk{defmacro dancols 0}
\getchunk{defmacro danrows 0}
\getchunk{defmacro daref2 0}
\getchunk{defmacro delt 0}
\getchunk{defmacro DFAcos 0}
\getchunk{defmacro DFAcosh 0}
\getchunk{defmacro DFAdd 0}
\getchunk{defmacro DFAsin 0}
\getchunk{defmacro DFAsinh 0}
\getchunk{defmacro DFAtan 0}
\getchunk{defmacro DFAtan2 0}
```

```

\getchunk{defmacro DFAtanh 0}
\getchunk{defmacro DFCos 0}
\getchunk{defmacro DFCosh 0}
\getchunk{defmacro DFDivide 0}
\getchunk{defmacro DFEql 0}
\getchunk{defmacro DFExp 0}
\getchunk{defmacro DFExpt 0}
\getchunk{defmacro DFIntegerDivide 0}
\getchunk{defmacro DFIntegerExpt 0}
\getchunk{defmacro DFIntegerMultiply 0}
\getchunk{defmacro DFLessThan 0}
\getchunk{defmacro DFLog 0}
\getchunk{defmacro DFLogE 0}
\getchunk{defmacro DFMax 0}
\getchunk{defmacro DFMin 0}
\getchunk{defmacro DFMinusp 0}
\getchunk{defmacro DFMultiply 0}
\getchunk{defmacro DFSin 0}
\getchunk{defmacro DFSinh 0}
\getchunk{defmacro DFSqrt 0}
\getchunk{defmacro DFSubtract 0}
\getchunk{defmacro DFTan 0}
\getchunk{defmacro DFTanh 0}
\getchunk{defmacro DFUnaryMinus 0}
\getchunk{defmacro DFZerop 0}
\getchunk{defmacro dlen 0}
\getchunk{defmacro dsetaref2 0}
\getchunk{defmacro dsetelt 0}
\getchunk{defmacro idChar? 0}
\getchunk{defmacro identp 0}
\getchunk{defmacro FloatError 0}
\getchunk{defmacro fracpart 0}
\getchunk{defmacro frameExposureData 0}
\getchunk{defmacro frameHiFiAccess 0}
\getchunk{defmacro frameHistListAct 0}
\getchunk{defmacro frameHistList 0}
\getchunk{defmacro frameHistListLen 0}
\getchunk{defmacro frameHistoryTable 0}
\getchunk{defmacro frameHistRecord 0}
\getchunk{defmacro frameInteractive 0}
\getchunk{defmacro frameIOIndex 0}
\getchunk{defmacro frameName 0}
\getchunk{defmacro frameNames 0}
\getchunk{defmacro getMsgArgL 0}
\getchunk{defmacro getMsgKey 0}
\getchunk{defmacro getMsgPosTag0b 0}
\getchunk{defmacro getMsgPrefix 0}
\getchunk{defmacro getMsgPrefix? 0}
\getchunk{defmacro getMsgTag 0}
\getchunk{defmacro getMsgTag? 0}

```

```

\getchunk{defmacro getMsgText 0}
\getchunk{defmacro hashCode? 0}
\getchunk{defmacro make-cdouble-matrix 0}
\getchunk{defmacro make-cdouble-vector 0}
\getchunk{defmacro make-double-matrix 0}
\getchunk{defmacro make-double-matrix1 0}
\getchunk{defmacro make-double-vector 0}
\getchunk{defmacro make-double-vector1 0}
\getchunk{defmacro qcsiz 0}
\getchunk{defmacro qsabsval 0}
\getchunk{defmacro qsadd1 0}
\getchunk{defmacro qsdifference 0}
\getchunk{defmacro qsgreaterp 0}
\getchunk{defmacro qslessp 0}
\getchunk{defmacro qsmax 0}
\getchunk{defmacro qsmin 0}
\getchunk{defmacro qsminus 0}
\getchunk{defmacro qsoddp 0}
\getchunk{defmacro qsplus 0}
\getchunk{defmacro qssub1 0}
\getchunk{defmacro qstimes 0}
\getchunk{defmacro qszerop 0}
\getchunk{defmacro setMsgPrefix 0}
\getchunk{defmacro setMsgText 0}
\getchunk{defmacro spadConstant 0}

```

```
;;; above level 0 macros
```

```

\getchunk{defmacro ancolsU8}
\getchunk{defmacro ancolsU16}
\getchunk{defmacro ancolsU32}
\getchunk{defmacro anrowsU8}
\getchunk{defmacro anrowsU16}
\getchunk{defmacro anrowsU32}
\getchunk{defmacro aref2U8}
\getchunk{defmacro aref2U16}
\getchunk{defmacro aref2U32}
\getchunk{defmacro assq}
\getchunk{defmacro bvec-setelt}
\getchunk{defmacro bvec-size}
\getchunk{defmacro eltU8}
\getchunk{defmacro eltU16}
\getchunk{defmacro eltU32}
\getchunk{defmacro funfind}
\getchunk{defmacro hget}
\getchunk{defmacro leader?}
\getchunk{defmacro line?}
\getchunk{defmacro makeMatrixU8}
\getchunk{defmacro makeMatrix1U8}
\getchunk{defmacro makeMatrixU16}

```

```

\getchunk{defmacro makeMatrix1U16}
\getchunk{defmacro makeMatrixU32}
\getchunk{defmacro makeMatrix1U32}
\getchunk{defmacro mkObj}
\getchunk{defmacro mkObjCode}
\getchunk{defmacro mkObjWrap}
\getchunk{defmacro objCodeVal}
\getchunk{defmacro objCodeMode}
\getchunk{defmacro objMode}
\getchunk{defmacro objSetMode}
\getchunk{defmacro objSetVal}
\getchunk{defmacro objVal}
\getchunk{defmacro objValUnwrap}
\getchunk{defmacro qsDot26432}
\getchunk{defmacro qsDot2Mod6432}
\getchunk{defmacro qsMod6432}
\getchunk{defmacro qsMulAdd6432}
\getchunk{defmacro qsMulAddMod6432}
\getchunk{defmacro qsMul6432}
\getchunk{defmacro qsMulMod32}
\getchunk{defmacro qvlenU8}
\getchunk{defmacro qvlenU16}
\getchunk{defmacro qvlenU32}
\getchunk{defmacro Rest}
\getchunk{defmacro startsId?}
\getchunk{defmacro setAref2U8}
\getchunk{defmacro setAref2U16}
\getchunk{defmacro setAref2U32}
\getchunk{defmacro seteltU8}
\getchunk{defmacro seteltU16}
\getchunk{defmacro seteltU32}
\getchunk{defmacro toScreen?}
\getchunk{defmacro trapNumericErrors}
\getchunk{defmacro truth-to-bit}
\getchunk{defmacro while}
\getchunk{defmacro whileWithResult}

```

```

;;; layer 0 (all common lisp)

```

```

\getchunk{defun acot 0}
\getchunk{defun acoth 0}
\getchunk{defun acsc 0}
\getchunk{defun acsch 0}
\getchunk{defun asec 0}
\getchunk{defun asech 0}
\getchunk{defun axiomVersion 0}

\getchunk{defun basicStringize 0}
\getchunk{defun BesselasympA 0}
\getchunk{defun BesselasympB 0}

```

```

\getchunk{defun BesselIBackRecur 0}
\getchunk{defun BooleanEquality 0}
\getchunk{defun bvec-and 0}
\getchunk{defun bvec-concat 0}
\getchunk{defun bvec-copy 0}
\getchunk{defun bvec-equal 0}
\getchunk{defun bvec-greater 0}
\getchunk{defun bvec-make-full 0}
\getchunk{defun bvec-nand 0}
\getchunk{defun bvec-nor 0}
\getchunk{defun bvec-not 0}
\getchunk{defun bvec-or 0}
\getchunk{defun bvec-xor 0}

\getchunk{defun cgammaAdjust 0}
\getchunk{defun cgammaBernsum 0}
\getchunk{defun cgammaG 0}
\getchunk{defun cgammaT 0}
\getchunk{defun chebf01coefmake 0}
\getchunk{defun chebstarevalarr 0}
\getchunk{defun cleanupLine 0}
\getchunk{defun clearMacroTable 0}
\getchunk{defun concat 0}
\getchunk{defun concatWithBlanks 0}
\getchunk{defun cot 0}
\getchunk{defun cotdiffeval 0}
\getchunk{defun coth 0}
\getchunk{defun createCurrentInterpreterFrame 0}
\getchunk{defun credits 0}
\getchunk{defun csc 0}
\getchunk{defun csch 0}
\getchunk{defun c-to-s 0}

\getchunk{defun dbKind 0}
\getchunk{defun dbRead 0}
\getchunk{defun delasc 0}
\getchunk{defun Delay 0}
\getchunk{defun desiredMsg 0}
\getchunk{defun DirToString 0}
\getchunk{defun displayFrameNames 0}
\getchunk{defun divide2 0}
\getchunk{defun dqAppend 0}
\getchunk{defun dqToList 0}
\getchunk{defun dqUnit 0}

\getchunk{defun embed2 0}
\getchunk{defun emptyInterpreterFrame 0}
\getchunk{defun endedp 0}
\getchunk{defun evalSharpOne 0}

```



```

\getchunk{defun fin 0}
\getchunk{defun findFrameInRing 0}
\getchunk{defun flatten 0}
\getchunk{defun flattenOperationAlist 0}
\getchunk{defun fnameExists? 0}
\getchunk{defun fnameName 0}
\getchunk{defun fnameReadable? 0}
\getchunk{defun fnameType 0}
\getchunk{defun frameNames 0}
\getchunk{defun From 0}
\getchunk{defun FromTo 0}

\getchunk{defun get-current-directory 0}
\getchunk{defun getenviron 0}
\getchunk{defun getl 0}
\getchunk{defun getLinePos 0}
\getchunk{defun getLineText 0}
\getchunk{defun getMsgKey? 0}
\getchunk{defun getParserMacroNames 0}
\getchunk{defun getPreStL 0}
\getchunk{defun getspoolname 0}

\getchunk{defun hasCorrectTarget 0}
\getchunk{defun hasOptArgs? 0}
\getchunk{defun horner 0}
\getchunk{defun httpAddToPageDescription 0}
\getchunk{defun httpMakeEmptyPage 0}
\getchunk{defun htSayStandard 0}

\getchunk{defun ignorep 0}
\getchunk{defun incActive? 0}
\getchunk{defun incCommand? 0}
\getchunk{defun incDrop 0}
\getchunk{defun incHandleMessage 0}
\getchunk{defun inclmsgConsole 0}
\getchunk{defun inclmsgFinSkipped 0}
\getchunk{defun inclmsgPrematureEOF 0}
\getchunk{defun inclmsgCmdBug 0}
\getchunk{defun inclmsgIfBug 0}
\getchunk{defun incPrefix? 0}
\getchunk{defun init-memory-config 0}
\getchunk{defun insertPos 0}
\getchunk{defun integer-decode-float-denominator 0}
\getchunk{defun integer-decode-float-exponent 0}
\getchunk{defun integer-decode-float-sign 0}
\getchunk{defun integer-decode-float-numerator 0}
\getchunk{defun intloopPrefix? 0}
\getchunk{defun isIntegerString 0}
\getchunk{defun isWrapped :proven}

```

```

\getchunk{defun keyword 0}
\getchunk{defun keyword? 0}

\getchunk{defun lastcount 0}
\getchunk{defun lfcomment 0}
\getchunk{defun lferror 0}
\getchunk{defun lffloat 0}
\getchunk{defun lfid 0}
\getchunk{defun lfinteger 0}
\getchunk{defun lfnegcomment 0}
\getchunk{defun lfrinteger 0}
\getchunk{defun lfspaces 0}
\getchunk{defun lfstring 0}
\getchunk{defun libdbTrim 0}
\getchunk{defun limitedPrint1 0}
\getchunk{defun lnCreate 0}
\getchunk{defun lnExtraBlanks 0}
\getchunk{defun lnFileName? 0}
\getchunk{defun lnGlobalNum 0}
\getchunk{defun lnImmediate? 0}
\getchunk{defun lnLocalNum 0}
\getchunk{defun lnPlaceOfOrigin 0}
\getchunk{defun lnSetGlobalNum 0}
\getchunk{defun lnString 0}
\getchunk{defun logH 0}
\getchunk{defun logS 0}

\getchunk{defun mac0Define 0}
\getchunk{defun mac0InfiniteExpansion,name 0}
\getchunk{defun make-absolute-filename 0}
\getchunk{defun makeByteWordVec2 0}
\getchunk{defun makeInitialModemapFrame 0}
\getchunk{defun manexp 0}
\getchunk{defun markUnique 0}
\getchunk{defun member 0}
\getchunk{defun mkObjFn 0}
\getchunk{defun monitor-add 0}
\getchunk{defun monitor-apropos 0}
\getchunk{defun monitor-autoload 0}
\getchunk{defun monitor-checkpoint 0}
\getchunk{defun monitor-decr 0}
\getchunk{defun monitor-delete 0}
\getchunk{defun monitor-dirname 0}
\getchunk{defun monitor-disable 0}
\getchunk{defun monitor-enable 0}
\getchunk{defun monitor-end 0}
\getchunk{defun monitor-exposedp 0}
\getchunk{defun monitor-file 0}
\getchunk{defun monitor-help 0}
\getchunk{defun monitor-incr 0}

```

```

\getchunk{defun monitor-info 0}
\getchunk{defun monitor-inittable 0}
\getchunk{defun monitor-libname 0}
\getchunk{defun monitor-nrlib 0}
\getchunk{defun monitor-parse 0}
\getchunk{defun monitor-percent 0}
\getchunk{defun monitor-readinterp 0}
\getchunk{defun monitor-report 0}
\getchunk{defun monitor-reset 0}
\getchunk{defun monitor-restore 0}
\getchunk{defun monitor-results 0}
\getchunk{defun monitor-spadfile 0}
\getchunk{defun monitor-tested 0}
\getchunk{defun monitor-untested 0}
\getchunk{defun monitor-write 0}

\getchunk{defun ncError 0}
\getchunk{defun ncloopEscaped 0}
\getchunk{defun ncloopPrefix? 0}
\getchunk{defun ncloopPrintLines 0}
\getchunk{defun nonBlank 0}
\getchunk{defun npAnyNo 0}
\getchunk{defun npboot 0}
\getchunk{defun npEqPeek 0}
\getchunk{defun nplisp 0}
\getchunk{defun npPop1 0}
\getchunk{defun npPop2 0}
\getchunk{defun npPop3 0}
\getchunk{defun npPush 0}

\getchunk{defun objEnv 0}
\getchunk{defun objModeFn 0}
\getchunk{defun objValFn 0}
\getchunk{defun opTran 0}

\getchunk{defun pfAndLeft 0}
\getchunk{defun pfAndRight 0}
\getchunk{defun pfAppend 0}
\getchunk{defun pfApplicationArg 0}
\getchunk{defun pfApplicationOp 0}
\getchunk{defun pfAssignLhsItems 0}
\getchunk{defun pf0AssignLhsItems 0}
\getchunk{defun pfAssignRhs 0}
\getchunk{defun pfBreakFrom 0}
\getchunk{defun pfCoercetoExpr 0}
\getchunk{defun pfCoercetoType 0}
\getchunk{defun pfCollectBody 0}
\getchunk{defun pfCollectIterators 0}
\getchunk{defun pfDefinitionLhsItems 0}
\getchunk{defun pfDefinitionRhs 0}

```

```

\getchunk{defun pfDoBody 0}
\getchunk{defun pfExitCond 0}
\getchunk{defun pfExitExpr 0}
\getchunk{defun pfFirst 0}
\getchunk{defun pfFreeItems 0}
\getchunk{defun pfForinLhs 0}
\getchunk{defun pfForinWhole 0}
\getchunk{defun pfFromdomDomain 0}
\getchunk{defun pfFromdomWhat 0}
\getchunk{defun pfIfCond 0}
\getchunk{defun pfIfElse 0}
\getchunk{defun pfIfThen 0}
\getchunk{defun pfLambdaArgs 0}
\getchunk{defun pfLambdaBody 0}
\getchunk{defun pfLambdaRets 0}
\getchunk{defun pfLiteral? 0}
\getchunk{defun pfLocalItems 0}
\getchunk{defun pfLoopIterators 0}
\getchunk{defun pfMacroLhs 0}
\getchunk{defun pfMacroRhs 0}
\getchunk{defun pfMLambdaArgs 0}
\getchunk{defun pfMLambdaBody 0}
\getchunk{defun pfNotArg 0}
\getchunk{defun pfNovalueExpr 0}
\getchunk{defun pfOrLeft 0}
\getchunk{defun pfOrRight 0}
\getchunk{defun pfParts 0}
\getchunk{defun pfPile 0}
\getchunk{defun pfPretendExpr 0}
\getchunk{defun pfPretendType 0}
\getchunk{defun pfRestrictExpr 0}
\getchunk{defun pfRestrictType 0}
\getchunk{defun pfReturnExpr 0}
\getchunk{defun pfRuleLhsItems 0}
\getchunk{defun pfRuleRhs 0}
\getchunk{defun pfSecond 0}
\getchunk{defun pfSequenceArgs 0}
\getchunk{defun pfSuchthatCond 0}
\getchunk{defun pfTaggedExpr 0}
\getchunk{defun pfTaggedTag 0}
\getchunk{defun pfTree 0}
\getchunk{defun pfTypedId 0}
\getchunk{defun pfTypedType 0}
\getchunk{defun pfTupleParts 0}
\getchunk{defun pfWhereContext 0}
\getchunk{defun pfWhereExpr 0}
\getchunk{defun pfWhileCond 0}
\getchunk{defun placep 0}
\getchunk{defun pmDontQuote? 0}
\getchunk{defun pname 0}

```

```

\getchunk{defun poCharPosn 0}
\getchunk{defun poGetLineObject 0}
\getchunk{defun poNopos? 0}
\getchunk{defun poNoPosition 0}
\getchunk{defun poNoPosition? 0}
\getchunk{defun printAsTeX 0}
\getchunk{defun PsiAsymptoticOrder 0}
\getchunk{defun PsiEps 0}
\getchunk{defun PsiIntpart 0}

\getchunk{defun qenum 0}
\getchunk{defun qeset 0}
\getchunk{defun qsquotient 0}
\getchunk{defun qsremainder 0}
\getchunk{defun quotient2 0}

\getchunk{defun random 0}
\getchunk{defun rdigit? 0}
\getchunk{defun reclaim 0}
\getchunk{defun remainder2 0}
\getchunk{defun remLine 0}
\getchunk{defun removeOption 0}
\getchunk{defun rep 0}
\getchunk{defun resetStackLimits 0}
\getchunk{defun resultp 0}

\getchunk{defun sameUnionBranch 0}
\getchunk{defun satisfiesUserLevel 0}
\getchunk{defun scanCloser? 0}
\getchunk{defun sec 0}
\getchunk{defun sech 0}
\getchunk{defun setCurrentLine 0}
\getchunk{defun set-restart-hook 0}
\getchunk{defun showMsgPos? 0}
\getchunk{defun smallEnoughCount 0}
\getchunk{defun startsComment? 0}
\getchunk{defun s-to-c 0}
\getchunk{defun StreamNull 0}
\getchunk{defun stringize 0}
\getchunk{defun stringPrefix? 0}
\getchunk{defun stripLisp 0}
\getchunk{defun stripSpaces 0}
\getchunk{defun substring 0}

\getchunk{defun theid 0}
\getchunk{defun thefname 0}
\getchunk{defun theorigin 0}
\getchunk{defun tokPart 0}
\getchunk{defun To 0}
\getchunk{defun Top? 0}

```

```

\getchunk{defun trademark 0}

\getchunk{defun /untrace-reduce 0}

\getchunk{defun vec2list 0}
\getchunk{defun vmread 0}

\getchunk{defun zeroOneTran 0}

;;; above level 0

\getchunk{defun abbQuery}
\getchunk{defun abbreviations}
\getchunk{defun abbreviationsSpad2Cmd}
\getchunk{defun absolutelyCanCoerceByCheating}
\getchunk{defun addBinding}
\getchunk{defun addBindingInteractive}
\getchunk{defun addInputLibrary}
\getchunk{defun addNewInterpreterFrame}
\getchunk{defun addoperations}
\getchunk{defun addTraceItem}
\getchunk{defun algCoerceInteractive}
\getchunk{defun algEqual}
\getchunk{defun allConstructors}
\getchunk{defun allOperations}
\getchunk{defun alqlGetOrigin}
\getchunk{defun alqlGetParams}
\getchunk{defun alqlGetKindString}
\getchunk{defun alreadyOpened?}
\getchunk{defun apropos}
\getchunk{defun assertCond}
\getchunk{defun augmentHasArgs}
\getchunk{defun augmentTraceNames}

\getchunk{defun basicLookup}
\getchunk{defun basicLookupCheckDefaults}
\getchunk{defun bcComplexLimit}
\getchunk{defun bcComplexLimitGen}
\getchunk{defun bcCreateVariableString}
\getchunk{defun bcDefiniteIntegrate}
\getchunk{defun bcDefiniteIntegrateGen}
\getchunk{defun bcDifferentiate}
\getchunk{defun bcDifferentiateGen}
\getchunk{defun bcDraw}
\getchunk{defun bcDrawIt}
\getchunk{defun bcDrawIt2}
\getchunk{defun bcDraw2Dfun}
\getchunk{defun bcDraw2DfunGen}
\getchunk{defun bcDraw2Dpar}
\getchunk{defun bcDraw2DparGen}

```

```

\getchunk{defun bcDraw2DSolve}
\getchunk{defun bcDraw2DSolveGen}
\getchunk{defun bcDraw3Dfun}
\getchunk{defun bcDraw3DfunGen}
\getchunk{defun bcDraw3Dpar}
\getchunk{defun bcDraw3DparGen}
\getchunk{defun bcDraw3Dpar1}
\getchunk{defun bcDraw3Dpar1Gen}
\getchunk{defun bcError}
\getchunk{defun bcFindString}
\getchunk{defun bcFinish}
\getchunk{defun bcGen}
\getchunk{defun bcGenEquations}
\getchunk{defun bcGenExplicitMatrix}
\getchunk{defun bcHt}
\getchunk{defun bcHtMakeButton}
\getchunk{defun bcIndefiniteIntegrate}
\getchunk{defun bcIndefiniteIntegrateGen}
\getchunk{defun bcInputEquations}
\getchunk{defun bcInputEquationsEnd}
\getchunk{defun bcInputExplicitMatrix}
\getchunk{defun bcInputMatrixByFormula}
\getchunk{defun bcInputMatrixByFormulaGen}
\getchunk{defun bcInputSolveInfo}
\getchunk{defun bcIssueHt}
\getchunk{defun bcLaurentSeries}
\getchunk{defun bcLaurentSeriesGen}
\getchunk{defun bcLimit}
\getchunk{defun bcLinearExtractMatrix}
\getchunk{defun bcLinearMatrixGen}
\getchunk{defun bcLinearSolve}
\getchunk{defun bcLinearSolveEqns}
\getchunk{defun bcLinearSolveEqns1}
\getchunk{defun bcLinearSolveEqnsGen}
\getchunk{defun bcLinearSolveMatrix}
\getchunk{defun bcLinearSolveMatrix1}
\getchunk{defun bcLinearSolveMatrixHomo}
\getchunk{defun bcLinearSolveMatrixInhomo}
\getchunk{defun bcLinearSolveMatrixInhomoGen}
\getchunk{defun bcMatrix}
\getchunk{defun bcMatrixGen}
\getchunk{defun bcMakeEquations}
\getchunk{defun bcMakeLinearEquations}
\getchunk{defun bcMakeUnknowns}
\getchunk{defun bcMkFunction}
\getchunk{defun bcNotReady}
\getchunk{defun bcOptional}
\getchunk{defun bcProduct}
\getchunk{defun bcProductGen}
\getchunk{defun bcPuisseuxSeries}

```

```

\getchunk{defun bcPuisseuxSeriesGen}
\getchunk{defun bcReadMatrix}
\getchunk{defun bcRealLimit}
\getchunk{defun bcRealLimitGen}
\getchunk{defun bcRealLimitGen1}
\getchunk{defun bcSadFaces}
\getchunk{defun bcSeries}
\getchunk{defun bcSeriesByFormula}
\getchunk{defun bcSeriesByFormulaGen}
\getchunk{defun bcSeriesExpansion}
\getchunk{defun bcSeriesExpansionGen}
\getchunk{defun bcSeriesGen}
\getchunk{defun bcSolve}
\getchunk{defun bcSolveEquations}
\getchunk{defun bcSolveEquationsNumerically}
\getchunk{defun bcSolveNumerically1}
\getchunk{defun bcSolveSingle}
\getchunk{defun bcString2HyString}
\getchunk{defun bcString2HyString2}
\getchunk{defun bcString2WordList}
\getchunk{defun bcSystemSolveEqns1}
\getchunk{defun bcSum}
\getchunk{defun bcSumGen}
\getchunk{defun bcSystemSolve}
\getchunk{defun bcTaylorSeries}
\getchunk{defun bcTaylorSeriesGen}
\getchunk{defun bcUnixTable}
\getchunk{defun bcVectorGen}
\getchunk{defun bcvspace}
\getchunk{defun bcwords2liststring}
\getchunk{defun beforeAfter}
\getchunk{defun BesselI}
\getchunk{defun besseliBack}
\getchunk{defun besseliCheb}
\getchunk{defun BesselJ}
\getchunk{defun BesselJAsympt}
\getchunk{defun BesselJAsymptOrder}
\getchunk{defun BesselJRecur}
\getchunk{defun bptrace}
\getchunk{defun bracketString}
\getchunk{defun break}
\getchunk{defun breaklet}
\getchunk{defun brightprint}
\getchunk{defun brightprint-0}
\getchunk{defun browse}
\getchunk{defun browseopen}
\getchunk{defun buildHtMacroTable}
\getchunk{defun buttonNames}

\getchunk{defun canFuncall?}

```



```

\getchunk{defun categoryopen}
\getchunk{defun catchCoerceFailure}
\getchunk{defun cbesseli}
\getchunk{defun cbesselj}
\getchunk{defun cgamma}
\getchunk{defun cgammaImpl}
\getchunk{defun changeHistListLen}
\getchunk{defun changeToNamedInterpreterFrame}
\getchunk{defun charDigitVal}
\getchunk{defun chebf01}
\getchunk{defun checkCondition}
\getchunk{defun checkFilter}
\getchunk{defun chkAllNonNegativeInteger}
\getchunk{defun chkDirectory}
\getchunk{defun chkNameList}
\getchunk{defun chkNonNegativeInteger}
\getchunk{defun chkOutputFileName}
\getchunk{defun chkPosInteger}
\getchunk{defun chkRange}
\getchunk{defun chyper0f1}
\getchunk{defun cleanline}
\getchunk{defun clear}
\getchunk{defun clearCmdAll}
\getchunk{defun clearCmdCompletely}
\getchunk{defun clearCmdExcept}
\getchunk{defun clearCmdParts}
\getchunk{defun clearCmdSortedCaches}
\getchunk{defun clearFrame}
\getchunk{defun clearParserMacro}
\getchunk{defun clearSpad2Cmd}
\getchunk{defun clngamma}
\getchunk{defun clngammacase1}
\getchunk{defun clngammacase2}
\getchunk{defun clngammacase3}
\getchunk{defun clngammacase23}
\getchunk{defun clngammaImpl}
\getchunk{defun close}
\getchunk{defun closeInterpreterFrame}
\getchunk{defun cmpnote}
\getchunk{defun coerceBranch2Union}
\getchunk{defun coerceByFunction}
\getchunk{defun coerceByTable}
\getchunk{defun coerceCommutateTest}
\getchunk{defun coerceConvertMmSelection}
\getchunk{defun coerceImmediateSubDomain}
\getchunk{defun coerceInt}
\getchunk{defun coerceInt0}
\getchunk{defun coerceInt1}
\getchunk{defun coerceIntX}
\getchunk{defun coerceIntAlgebraicConstant}

```

```

\getchunk{defun coerceIntByMap}
\getchunk{defun coerceIntByMapInner}
\getchunk{defun coerceIntCommute}
\getchunk{defun coerceInteractive}
\getchunk{defun coerceIntFromUnion}
\getchunk{defun coerceIntPermute}
\getchunk{defun coerceIntSpecial}
\getchunk{defun coerceIntTableOrFunction}
\getchunk{defun coerceIntTest}
\getchunk{defun coerceIntTower}
\getchunk{defun coerceInt2Union}
\getchunk{defun coerceOrRetract}
\getchunk{defun coerceOrThrowFailure}
\getchunk{defun coerceRetract}
\getchunk{defun coerceSpadArgs2E}
\getchunk{defun coerceSpadFunValue2E}
\getchunk{defun coerceSubDomain}
\getchunk{defun coerceTraceArgs2E}
\getchunk{defun coerceTraceFunValue2E}
\getchunk{defun coerceUnion2Branch}
\getchunk{defun coercionFailure}
\getchunk{defun commandAmbiguityError}
\getchunk{defun commandError}
\getchunk{defun commandErrorIfAmbiguous}
\getchunk{defun commandErrorMessage}
\getchunk{defun commandsForUserLevel}
\getchunk{defun commandUserLevelError}
\getchunk{defun compareposns}
\getchunk{defun compareTypeLists}
\getchunk{defun compileBoot}
\getchunk{defun compiledLookup}
\getchunk{defun compiledLookupCheck}
\getchunk{defun computeDomainVariableAlist}
\getchunk{defun computeTTTranspositions}
\getchunk{defun condErrorMsg}
\getchunk{defun conLowerCaseConTran}
\getchunk{defun conOpPage}
\getchunk{defun conOpPage1}
\getchunk{defun conPage}
\getchunk{defun conPageChoose}
\getchunk{defun conPageConEntry}
\getchunk{defun conPageFastPath}
\getchunk{defun conSpecialString?}
\getchunk{defun constoken}
\getchunk{defun constructorSearch}
\getchunk{defun constructSubst}
\getchunk{defun containsVars}
\getchunk{defun containsVars1}
\getchunk{defun copyright}
\getchunk{defun countCache}

```

```

\getchunk{defun cpsi}
\getchunk{defun cPsiImpl}
\getchunk{defun cSearch}
\getchunk{defun c-to-r}

\getchunk{defun DaaseName}
\getchunk{defun dbAddChain}
\getchunk{defun dbAddChainDomain}
\getchunk{defun dbAddDocTable}
\getchunk{defun dbCompositeWithMap}
\getchunk{defun dbConsExposureMessage}
\getchunk{defun dbConsHeading}
\getchunk{defun dbConstructorDoc}
\getchunk{defun dbConstructorDoc,hn}
\getchunk{defun dbConstructorDoc,gn}
\getchunk{defun dbDocTable}
\getchunk{defun dbExtractUnderlyingDomain}
\getchunk{defun dbGetDocTable}
\getchunk{defun dbGetDocTable,gn}
\getchunk{defun dbGetDocTable,hn}
\getchunk{defun dbMkEvalable}
\getchunk{defun dbMkForm}
\getchunk{defun dbNonEmptyPattern}
\getchunk{defun dbSearchOrder}
\getchunk{defun dbSelectCon}
\getchunk{defun dbShowConditions}
\getchunk{defun dbShowCons}
\getchunk{defun dbShowCons1}
\getchunk{defun dbShowConsDoc}
\getchunk{defun dbShowConsDoc1}
\getchunk{defun dbShowConsKindsFilter}
\getchunk{defun dbShowConstructorLines}
\getchunk{defun dbSpecialDescription}
\getchunk{defun dbSpecialExpandIfNecessary}
\getchunk{defun dbSpecialExports}
\getchunk{defun dbSpecialOperations}
\getchunk{defun dbString2Words}
\getchunk{defun dbSubConform}
\getchunk{defun dbWordFrom}
\getchunk{defun decideHowMuch}
\getchunk{defun decomposeTypeIntoTower}
\getchunk{defun defaultTargetFE}
\getchunk{defun defiostream}
\getchunk{defun deldatabase}
\getchunk{defun deleteFile}
\getchunk{defun describe}
\getchunk{defun describeFortPersistence}
\getchunk{defun describeInputLibraryArgs}
\getchunk{defun describeOutputLibraryArgs}
\getchunk{defun describeSetFortDir}

```

```

\getchunk{defun describeSetFortTmpDir}
\getchunk{defun describeSetFunctionsCache}
\getchunk{defun describeSetLinkerArgs}
\getchunk{defun describeSetNagHost}
\getchunk{defun describeSetOutputAlgebra}
\getchunk{defun describeSetOutputFormula}
\getchunk{defun describeSetOutputFortran}
\getchunk{defun describeSetOutputHtml}
\getchunk{defun describeSetOutputMathml}
\getchunk{defun describeSetOutputOpenMath}
\getchunk{defun describeSetOutputTex}
\getchunk{defun describeSetStreamsCalculate}
\getchunk{defun describeSpad2Cmd}
\getchunk{defun dewritify}
\getchunk{defun dewritify,dewritifyInner}
\getchunk{defun diffAlist}
\getchunk{defun digit?}
\getchunk{defun digitp}
\getchunk{defun disableHist}
\getchunk{defun display}
\getchunk{defun displayCondition}
\getchunk{defun displayExposedConstructors}
\getchunk{defun displayExposedGroups}
\getchunk{defun displayHiddenConstructors}
\getchunk{defun displayMacro}
\getchunk{defun displayMacros}
\getchunk{defun displayMode}
\getchunk{defun displayModemap}
\getchunk{defun displayOperations}
\getchunk{defun displayOperationsFromLisplib}
\getchunk{defun displayParserMacro}
\getchunk{defun displayProperties}
\getchunk{defun displayProperties,sayFunctionDeps}
\getchunk{defun displaySetOptionInformation}
\getchunk{defun displaySetVariableSettings}
\getchunk{defun displaySpad2Cmd}
\getchunk{defun displayType}
\getchunk{defun displayValue}
\getchunk{defun displayWorkspaceNames}
\getchunk{defun doDoitButton}
\getchunk{defun domainDescendantsOf}
\getchunk{defun domainToGenvar}
\getchunk{defun domArg}
\getchunk{defun domArg2}
\getchunk{defun doSystemCommand}
\getchunk{defun downcase}
\getchunk{defun downlink}
\getchunk{defun dqConcat}
\getchunk{defun dropInputLibrary}
\getchunk{defun dSearch}

```

```

\getchunk{defun dumbTokenize}

\getchunk{defun edit}
\getchunk{defun editFile}
\getchunk{defun editSpad2Cmd}
\getchunk{defun Else?}
\getchunk{defun Elseif?}
\getchunk{defun embeddedFunction}
\getchunk{defun enPile}
\getchunk{defun eofp}
\getchunk{defun eqpileTree}
\getchunk{defun erMsgCompare}
\getchunk{defun erMsgSep}
\getchunk{defun erMsgSort}
\getchunk{defun evalCategory}
\getchunk{defun evalDomain}
\getchunk{defun evaluateSignature}
\getchunk{defun evaluateType}
\getchunk{defun evaluateType1}
\getchunk{defun executeInterpreterCommand}
\getchunk{defun ExecuteInterpSystemCommand}
\getchunk{defun executeQuietCommand}
\getchunk{defun explainLinear}

\getchunk{defun fetchOutput}
\getchunk{defun fillerSpaces}
\getchunk{defun filterAndFormatConstructors}
\getchunk{defun filterListOfStrings}
\getchunk{defun filterListOfStringsWithFn}
\getchunk{defun finalExactRequest}
\getchunk{defun findnexttest}
\getchunk{defun firstTokPosn}
\getchunk{defun fixObjectForPrinting}
\getchunk{defun flatBvList}
\getchunk{defun float2Sex}
\getchunk{defun fnameDirectory}
\getchunk{defun fnameMake}
\getchunk{defun fnameNew}
\getchunk{defun fnameWritable?}
\getchunk{defun frame}
\getchunk{defun frameEnvironment}
\getchunk{defun frameSpad2Cmd}
\getchunk{defun funfind,LAM}

\getchunk{defun gammaRatapprox}
\getchunk{defun gammaRatkernel}
\getchunk{defun gammaStirling}
\getchunk{defun gatherGlossLines}
\getchunk{defun genDomainTraceName}
\getchunk{defun gensymInt}

```

```

\getchunk{defun getAliasIfTracedMapParameter}
\getchunk{defun getAndEvalConstructorArgument}
\getchunk{defun getAndSay}
\getchunk{defun getBpiNameIfTracedMap}
\getchunk{defun getBrowseDatabase}
\getchunk{defun getConstantFromDomain}
\getchunk{defun getConstructorDocumentation}
\getchunk{defun getdatabase}
\getchunk{defun getDependentsOfConstructor}
\getchunk{defun getDirectoryList}
\getchunk{defun getFirstWord}
\getchunk{defun getHtMacroItem}
\getchunk{defun getMapSig}
\getchunk{defun getMapSubNames}
\getchunk{defun getMsgCatAttr}
\getchunk{defun getMsgFTTag?}
\getchunk{defun getMsgInfoFromKey}
\getchunk{defun getMsgPos}
\getchunk{defun getMsgPos2}
\getchunk{defun getMsgToWhere}
\getchunk{defun getOperationAlistFromLisplib}
\getchunk{defun getOplistForConstructorForm}
\getchunk{defun getOplistWithUniqueSignatures}
\getchunk{defun getOption}
\getchunk{defun getPosStL}
\getchunk{defun getPreviousMapSubNames}
\getchunk{defun getProplist}
\getchunk{defun getRefvU8}
\getchunk{defun getRefvU16}
\getchunk{defun getRefvU32}
\getchunk{defun getStFromMsg}
\getchunk{defun getSubDomainPredicate}
\getchunk{defun getSystemCommandLine}
\getchunk{defun getTraceOption}
\getchunk{defun getTraceOption,hn}
\getchunk{defun getTraceOptions}
\getchunk{defun getUsersOfConstructor}
\getchunk{defun getWorkspaceNames}

\getchunk{defun handleNoParseCommands}
\getchunk{defun handleParsedSystemCommands}
\getchunk{defun handleTokenSizeSystemCommands}
\getchunk{defun hasAtt}
\getchunk{defun hasAttSig}
\getchunk{defun hasCatExpression}
\getchunk{defun hasCate}
\getchunk{defun hasCateSpecial}
\getchunk{defun hasCateSpecialNew}
\getchunk{defun hasCate1}
\getchunk{defun hasCaty}

```

```
\getchunk{defun hasCaty1}  
\getchunk{defun hashable}  
\getchunk{defun hasOption}  
\getchunk{defun hasPair}  
\getchunk{defun hasSharpVar}  
\getchunk{defun hasSig}  
\getchunk{defun hasSigAnd}  
\getchunk{defun hasSigOr}  
\getchunk{defun help}  
\getchunk{defun helpSpad2Cmd}  
\getchunk{defun histFileErase}  
\getchunk{defun histFileName}  
\getchunk{defun histInputFileName}  
\getchunk{defun history}  
\getchunk{defun historySpad2Cmd}  
\getchunk{defun hkeys}  
\getchunk{defun hput}  
\getchunk{defun htAddHeading}  
\getchunk{defun htAllOrNum}  
\getchunk{defun htBcLinks}  
\getchunk{defun htBcLispLinks}  
\getchunk{defun htBcRadioButtons}  
\getchunk{defun htCacheAddChoice}  
\getchunk{defun htCacheOne}  
\getchunk{defun htCacheSet}  
\getchunk{defun htCheckList}  
\getchunk{defun htCheck}  
\getchunk{defun htDoneButton}  
\getchunk{defun htDoNothing}  
\getchunk{defun htEscapeString}  
\getchunk{defun htFunctionSetLiteral}  
\getchunk{defun htGlossPage}  
\getchunk{defun htGlossSearch}  
\getchunk{defun htGloss}  
\getchunk{defun htGreekSearch}  
\getchunk{defun htInitPage}  
\getchunk{defun htInitPageNoScroll}  
\getchunk{defun htInputStrings}  
\getchunk{defun htKill}  
\getchunk{defun htLispLinks}  
\getchunk{defun htLispMemoLinks}  
\getchunk{defun htMakeButton}  
\getchunk{defun htMakeDoitButton}  
\getchunk{defun htMakeDoneButton}  
\getchunk{defun htMakeErrorPage}  
\getchunk{defun htMakeInputList}  
\getchunk{defun htMakeLabel}  
\getchunk{defun htMakePage}  
\getchunk{defun htMakePage1}  
\getchunk{defun htMakePathKey,fn}
```

```

\getchunk{defun htMakePathKey}
\getchunk{defun htMakeTemplates,substLabel}
\getchunk{defun htMakeTemplates}
\getchunk{defun htMarkTree}
\getchunk{defun htMkName}
\getchunk{defun httpAddInputAreaProp}
\getchunk{defun httpButtonValue}
\getchunk{defun httpDestroyPage}
\getchunk{defun httpDomainConditions}
\getchunk{defun httpDomainPvarSubstList}
\getchunk{defun httpDomainVariableAlist}
\getchunk{defun httpInputAreaAlist}
\getchunk{defun httpLabelDefault}
\getchunk{defun httpLabelErrorMsg}
\getchunk{defun httpLabelFilteredInputString}
\getchunk{defun httpLabelFilter}
\getchunk{defun httpLabelInputString}
\getchunk{defun httpLabelSpadType}
\getchunk{defun httpLabelSpadValue}
\getchunk{defun httpLabelType}
\getchunk{defun httpName}
\getchunk{defun httpPageDescription}
\getchunk{defun httpRadioButtonAlist}
\getchunk{defun httpProperty}
\getchunk{defun httpPropertyList}
\getchunk{defun htProcessBcButtons}
\getchunk{defun htProcessBcStrings}
\getchunk{defun htProcessDoitButton}
\getchunk{defun htProcessDomainConditions}
\getchunk{defun htProcessDoneButton}
\getchunk{defun htProcessToggleButtons}
\getchunk{defun httpSetDomainConditions}
\getchunk{defun httpSetDomainPvarSubstList}
\getchunk{defun httpSetDomainVariableAlist}
\getchunk{defun httpSetInputAreaAlist}
\getchunk{defun httpSetLabelErrorMsg}
\getchunk{defun httpSetLabelInputString}
\getchunk{defun httpSetLabelSpadValue}
\getchunk{defun httpSetName}
\getchunk{defun httpSetPageDescription}
\getchunk{defun httpSetProperty}
\getchunk{defun httpSetRadioButtonAlist}
\getchunk{defun htQuote}
\getchunk{defun htRadioButtons}
\getchunk{defun htSay}
\getchunk{defun htSayBind}
\getchunk{defun htSetCache}
\getchunk{defun htSetExpose}
\getchunk{defun htSetFunCommandContinue}
\getchunk{defun htSetFunCommand}

```



```

\getchunk{defun htSetHistory}
\getchunk{defun htSetInputLibrary}
\getchunk{defun htSetInteger}
\getchunk{defun htSetLinkerArgs}
\getchunk{defun htSetLiterals}
\getchunk{defun htSetLiteral}
\getchunk{defun htSetNotAvailable}
\getchunk{defun htSetOutputCharacters}
\getchunk{defun htSetOutputLibrary}
\getchunk{defun htSetSystemVariableKind}
\getchunk{defun htSetSystemVariable}
\getchunk{defun htSetVars}
\getchunk{defun htSetvarDoneButton}
\getchunk{defun htShowCount}
\getchunk{defun htShowFunctionPageContinued}
\getchunk{defun htShowFunctionPage}
\getchunk{defun htShowIntegerPage}
\getchunk{defun htShowLiteralsPage}
\getchunk{defun htShowPage}
\getchunk{defun htShowPageNoScroll}
\getchunk{defun htShowSetPage}
\getchunk{defun htShowSetTreeValue}
\getchunk{defun htShowSetTree}
\getchunk{defun htStringPad}
\getchunk{defun htstv}
\getchunk{defun htSystemVariables,displayOptions}
\getchunk{defun htSystemVariables,fn}
\getchunk{defun htSystemVariables,functionTail}
\getchunk{defun htSystemVariables,gn}
\getchunk{defun htSystemVariables}
\getchunk{defun htTextSearch}
\getchunk{defun htTutorialSearch}

\getchunk{defun If?}
\getchunk{defun ifCond}
\getchunk{defun iht}
\getchunk{defun importFromFrame}
\getchunk{defun incAppend}
\getchunk{defun incAppend1}
\getchunk{defun incBiteOff}
\getchunk{defun incClassify}
\getchunk{defun incCommandTail}
\getchunk{defun incConsoleInput}
\getchunk{defun incFileInput}
\getchunk{defun incFileName}
\getchunk{defun incIgen}
\getchunk{defun incIgen1}
\getchunk{defun inclFname}
\getchunk{defun inclLine}
\getchunk{defun inclLine1}

```

```

\getchunk{defun inclmsgCannotRead}
\getchunk{defun inclmsgFileCycle}
\getchunk{defun inclmsgPrematureFin}
\getchunk{defun include}
\getchunk{defun include1}
\getchunk{defun inclmsgConActive}
\getchunk{defun inclmsgConStill}
\getchunk{defun inclmsgIfSyntax}
\getchunk{defun inclmsgNoSuchFile}
\getchunk{defun inclmsgSay}
\getchunk{defun incNConsoles}
\getchunk{defun incRenumber}
\getchunk{defun incRenumberItem}
\getchunk{defun incRenumberLine}
\getchunk{defun incRgen}
\getchunk{defun incRgen1}
\getchunk{defun incStream}
\getchunk{defun incString}
\getchunk{defun incZip}
\getchunk{defun incZip1}
\getchunk{defun init-boot/spad-reader}
\getchunk{defun initHist}
\getchunk{defun initHistList}
\getchunk{defun initial-getdatabase}
\getchunk{defun initializeInterpreterFrameRing}
\getchunk{defun initializeSetVariables}
\getchunk{defun initImPr}
\getchunk{defun initroot}
\getchunk{defun initToWhere}
\getchunk{defun insertAlist}
\getchunk{defun insertpile}
\getchunk{defun InterpExecuteSpadSystemCommand}
\getchunk{defun interpFunctionDepAlists}
\getchunk{defun interpopen}
\getchunk{defun interpret}
\getchunk{defun interpret1}
\getchunk{defun interpret2}
\getchunk{defun interpretTopLevel}
\getchunk{defun intInterpretPform}
\getchunk{defun intloop}
\getchunk{defun intloopEchoParse}
\getchunk{defun intloopInclude}
\getchunk{defun intloopInclude0}
\getchunk{defun intnplisp}
\getchunk{defun intloopProcess}
\getchunk{defun intloopProcessString}
\getchunk{defun intloopReadConsole}
\getchunk{defun intloopSpadProcess}
\getchunk{defun intloopSpadProcess,interp}
\getchunk{defun intProcessSynonyms}

```

```

\getchunk{defun ioclear}
\getchunk{defun iostat}
\getchunk{defun isDomainOrPackage}
\getchunk{defun isDomainValuedVariable}
\getchunk{defun isEqualOrSubDomain}
\getchunk{defun isExposedConstructor}
\getchunk{defun isGenvar}
\getchunk{defun isInterpOnlyMap}
\getchunk{defun isListOfIdentifiers}
\getchunk{defun isListOfIdentifiersOrStrings}
\getchunk{defun isPartialMode}
\getchunk{defun isPatternVar}
\getchunk{defun isSharpVar}
\getchunk{defun isSharpVarWithNum}
\getchunk{defun isSubForRedundantMapName}
\getchunk{defun isSystemDirectory}
\getchunk{defun isTaggedUnion}
\getchunk{defun isUncompiledMap}

\getchunk{defun justifyMyType}

\getchunk{defun kArgPage}
\getchunk{defun kArgumentCheck}
\getchunk{defun kcaPage}
\getchunk{defun kcaPage1}
\getchunk{defun kccPage}
\getchunk{defun kcdePage}
\getchunk{defun kcdPage}
\getchunk{defun kcdoPage}
\getchunk{defun kCheckArgumentNumbers}
\getchunk{defun kcnPage}
\getchunk{defun kcPage}
\getchunk{defun kcpPage}
\getchunk{defun kDomainName}
\getchunk{defun kdPageInfo}
\getchunk{defun KeepPart?}
\getchunk{defun kePage}
\getchunk{defun kePageDisplay}
\getchunk{defun kePageOpAlist}
\getchunk{defun kiPage}
\getchunk{defun kisValidType}
\getchunk{defun koaPageFilterByName}
\getchunk{defun koPage}
\getchunk{defun koPageAux}
\getchunk{defun koPageAux1}
\getchunk{defun koPageFromKKPage}
\getchunk{defun koPageInputAreaUnchanged?}
\getchunk{defun ksPage}
\getchunk{defun kcuPage}
\getchunk{defun kPage}

```

```

\getchunk{defun kSearch}
\getchunk{defun kTestPred}

\getchunk{defun lassocSub}
\getchunk{defun lastTokPosn}
\getchunk{defun leaveScratchpad}
\getchunk{defun lefts}
\getchunk{defun letPrint}
\getchunk{defun letPrint2}
\getchunk{defun letPrint3}
\getchunk{defun lfkey}
\getchunk{defun libConstructorSig}
\getchunk{defun library}
\getchunk{defun license}
\getchunk{defun linearFinalRequest}
\getchunk{defun lineoftoks}
\getchunk{defun linkGen}
\getchunk{defun listConstructorAbbreviations}
\getchunk{defun listDecideHowMuch}
\getchunk{defun listOfStrings2String}
\getchunk{defun listOutputter}
\getchunk{defun lngamma}
\getchunk{defun lnFileName}
\getchunk{defun load}
\getchunk{defun loadFunctor}
\getchunk{defun loadLib}
\getchunk{defun loadLibNoUpdate}
\getchunk{defun localdatabase}
\getchunk{defun localnrllib}
\getchunk{defun lookupInDomainVector}
\getchunk{defun loopIters2Sex}
\getchunk{defun lotsof}
\getchunk{defun lngamma}
\getchunk{defun lngammaRatapprox}
\getchunk{defun ltrace}

\getchunk{defun macApplication}
\getchunk{defun macExpand}
\getchunk{defun macId}
\getchunk{defun macLambda}
\getchunk{defun macLambda,mac}
\getchunk{defun macLambdaParameterHandling}
\getchunk{defun macMacro}
\getchunk{defun macSubstituteId}
\getchunk{defun macSubstituteOuter}
\getchunk{defun macroExpanded}
\getchunk{defun macWhere}
\getchunk{defun macWhere,mac}
\getchunk{defun macOExpandBody}
\getchunk{defun macOGet}

```

```
\getchunk{defun macOGetName}  
\getchunk{defun macOInfiniteExpansion}  
\getchunk{defun macOMLambdaApply}  
\getchunk{defun macOSubstituteOuter}  
\getchunk{defun make-appendstream}  
\getchunk{defun make-databases}  
\getchunk{defun makeFullNameString}  
\getchunk{defun makeHistFileName}  
\getchunk{defun makeInputFilename}  
\getchunk{defun make-instream}  
\getchunk{defun makeLeaderMsg}  
\getchunk{defun makeMsgFromLine}  
\getchunk{defun makeOrdinal}  
\getchunk{defun make-outstream}  
\getchunk{defun makePathname}  
\getchunk{defun makeSpadCommand}  
\getchunk{defun makeStream}  
\getchunk{defun mapLetPrint}  
\getchunk{defun mapStringize}  
\getchunk{defun mergePathnames}  
\getchunk{defun messageprint}  
\getchunk{defun messageprint-1}  
\getchunk{defun messageprint-2}  
\getchunk{defun mkConform}  
\getchunk{defun mkCurryFun}  
\getchunk{defun mkDomPvar}  
\getchunk{defun mkDomTypeForm}  
\getchunk{defun mkEvalable}  
\getchunk{defun mkEvalableMapping}  
\getchunk{defun mkEvalableRecord}  
\getchunk{defun mkEvalableUnion}  
\getchunk{defun mkLineList}  
\getchunk{defun mkprompt}  
\getchunk{defun mkSetTitle}  
\getchunk{defun mkUnixPattern}  
\getchunk{defun monitorEvalAfter}  
\getchunk{defun monitorEvalBefore}  
\getchunk{defun monitorEvalTran}  
\getchunk{defun monitorEvalTran1}  
\getchunk{defun monitorGetValue}  
\getchunk{defun monitorPrint}  
\getchunk{defun monitorPrintArg}  
\getchunk{defun monitorPrintArgs}  
\getchunk{defun monitorPrintRest}  
\getchunk{defun monitorPrintValue}  
\getchunk{defun monitorX}  
\getchunk{defun monitorEnter}  
\getchunk{defun monitorExit}  
\getchunk{defun monitorXX}  
\getchunk{defun msgCreate}
```

```

\getchunk{defun msgImPr?}
\getchunk{defun msgNoRep?}
\getchunk{defun msgOutputter}
\getchunk{defun msgText}
\getchunk{defun myWritable?}

\getchunk{defun namestring}
\getchunk{defun ncAlist}
\getchunk{defun ncBug}
\getchunk{defun ncConversationPhase}
\getchunk{defun ncConversationPhase,wrapup}
\getchunk{defun ncEltQ}
\getchunk{defun ncHardError}
\getchunk{defun ncIntLoop}
\getchunk{defun ncloopCommand}
\getchunk{defun ncloopDQlines}
\getchunk{defun ncloopIncFileName}
\getchunk{defun ncloopInclude}
\getchunk{defun ncloopInclude0}
\getchunk{defun ncloopInclude1}
\getchunk{defun ncloopParse}
\getchunk{defun ncParseFromString}
\getchunk{defun ncPutQ}
\getchunk{defun ncSoftError}
\getchunk{defun ncTag}
\getchunk{defun ncTopLevel}
\getchunk{defun newHelpSpad2Cmd}
\getchunk{defun next}
\getchunk{defun next1}
\getchunk{defun nextInterpreterFrame}
\getchunk{defun nextline}
\getchunk{defun next-lines-clear}
\getchunk{defun next-lines-show}
\getchunk{defun npAdd}
\getchunk{defun npADD}
\getchunk{defun npAmpersand}
\getchunk{defun npAmpersandFrom}
\getchunk{defun npAndOr}
\getchunk{defun npAngleBared}
\getchunk{defun npApplication}
\getchunk{defun npApplication2}
\getchunk{defun npArith}
\getchunk{defun npAssign}
\getchunk{defun npAssignment}
\getchunk{defun npAssignVariable}
\getchunk{defun npAtom1}
\getchunk{defun npAtom2}
\getchunk{defun npBacksetElse}
\getchunk{defun npBackTrack}
\getchunk{defun npBDefinition}

```

```

\getchunk{defun npBFileDefinition}
\getchunk{defun npBraced}
\getchunk{defun npBracked}
\getchunk{defun npBracketed}
\getchunk{defun npBreak}
\getchunk{defun npBy}
\getchunk{defun npCategory}
\getchunk{defun npCategoryL}
\getchunk{defun npCoerceTo}
\getchunk{defun npColon}
\getchunk{defun npColonQuery}
\getchunk{defun npComma}
\getchunk{defun npCommaBackSet}
\getchunk{defun npCompMissing}
\getchunk{defun npConditional}
\getchunk{defun npConditionalStatement}
\getchunk{defun npConstTok}
\getchunk{defun npDDInfKey}
\getchunk{defun npDecl}
\getchunk{defun npDef}
\getchunk{defun npDefaultDecl}
\getchunk{defun npDefaultItem}
\getchunk{defun npDefaultItemList}
\getchunk{defun npDefaultValue}
\getchunk{defun npDefinition}
\getchunk{defun npDefinitionItem}
\getchunk{defun npDefinitionlist}
\getchunk{defun npDefinitionOrStatement}
\getchunk{defun npDefn}
\getchunk{defun npDefTail}
\getchunk{defun npDiscrim}
\getchunk{defun npDisjand}
\getchunk{defun npDollar}
\getchunk{defun npDotted}
\getchunk{defun npElse}
\getchunk{defun npEncAp}
\getchunk{defun npEncl}
\getchunk{defun npEnclosed}
\getchunk{defun npEqKey}
\getchunk{defun npExit}
\getchunk{defun npExpress}
\getchunk{defun npExpress1}
\getchunk{defun npExport}
\getchunk{defun npFirstTok}
\getchunk{defun npFix}
\getchunk{defun npForIn}
\getchunk{defun npFree}
\getchunk{defun npFromdom}
\getchunk{defun npFromdom1}
\getchunk{defun npGives}

```

```

\getchunk{defun npId}
\getchunk{defun npImport}
\getchunk{defun npInfGeneric}
\getchunk{defun npInfixOp}
\getchunk{defun npInfixOperator}
\getchunk{defun npInfKey}
\getchunk{defun npInline}
\getchunk{defun npInterval}
\getchunk{defun npItem}
\getchunk{defun npItem1}
\getchunk{defun npIterate}
\getchunk{defun npIterator}
\getchunk{defun npIterators}
\getchunk{defun npLambda}
\getchunk{defun npLeftAssoc}
\getchunk{defun npLet}
\getchunk{defun npLetQualified}
\getchunk{defun npList}
\getchunk{defun npListAndRecover}
\getchunk{defun npListing}
\getchunk{defun npListofFun}
\getchunk{defun npLocal}
\getchunk{defun npLocalDecl}
\getchunk{defun npLocalItem}
\getchunk{defun npLocalItemList}
\getchunk{defun npLogical}
\getchunk{defun npLoop}
\getchunk{defun npMacro}
\getchunk{defun npMatch}
\getchunk{defun npMdef}
\getchunk{defun npMDEF}
\getchunk{defun npMDEFinition}
\getchunk{defun npMissing}
\getchunk{defun npMissingMate}
\getchunk{defun npMoveTo}
\getchunk{defun npName}
\getchunk{defun npNext}
\getchunk{defun npNull}
\getchunk{defun npParened}
\getchunk{defun npParenthesize}
\getchunk{defun npParenthesized}
\getchunk{defun npParse}
\getchunk{defun npPDefinition}
\getchunk{defun npPileBracketed}
\getchunk{defun npPileDefinitionlist}
\getchunk{defun npPileExit}
\getchunk{defun npPower}
\getchunk{defun npPP}
\getchunk{defun npPPf}
\getchunk{defun npPPff}

```



```
\getchunk{defun npPPg}  
\getchunk{defun npPrefixColon}  
\getchunk{defun npPretend}  
\getchunk{defun npPrimary}  
\getchunk{defun npPrimary1}  
\getchunk{defun npPrimary2}  
\getchunk{defun npProcessSynonym}  
\getchunk{defun npProduct}  
\getchunk{defun npPushId}  
\getchunk{defun npRelation}  
\getchunk{defun npRemainder}  
\getchunk{defun npQualDef}  
\getchunk{defun npQualified}  
\getchunk{defun npQualifiedDefinition}  
\getchunk{defun npQualType}  
\getchunk{defun npQualTypelist}  
\getchunk{defun npQuiver}  
\getchunk{defun npRecoverTrap}  
\getchunk{defun npRestore}  
\getchunk{defun npRestrict}  
\getchunk{defun npReturn}  
\getchunk{defun npRightAssoc}  
\getchunk{defun npRule}  
\getchunk{defun npSCategory}  
\getchunk{defun npSDefaultItem}  
\getchunk{defun npSegment}  
\getchunk{defun npSelector}  
\getchunk{defun npSemiBackSet}  
\getchunk{defun npSemiListing}  
\getchunk{defun npSigDecl}  
\getchunk{defun npSigItem}  
\getchunk{defun npSigItemList}  
\getchunk{defun npSignature}  
\getchunk{defun npSignatureDefinee}  
\getchunk{defun npSingleRule}  
\getchunk{defun npSLocalItem}  
\getchunk{defun npSQualTypelist}  
\getchunk{defun npStatement}  
\getchunk{defun npSuch}  
\getchunk{defun npSuchThat}  
\getchunk{defun npSum}  
\getchunk{defun npsynonym}  
\getchunk{defun npSymbolVariable}  
\getchunk{defun npSynthetic}  
\getchunk{defun npsystem}  
\getchunk{defun npState}  
\getchunk{defun npTagged}  
\getchunk{defun npTerm}  
\getchunk{defun npTrap}  
\getchunk{defun npTrapForm}
```

```

\getchunk{defun npTuple}
\getchunk{defun npType}
\getchunk{defun npTypedForm}
\getchunk{defun npTypedForm1}
\getchunk{defun npTypeStyle}
\getchunk{defun npTypified}
\getchunk{defun npTyping}
\getchunk{defun npTypeVariable}
\getchunk{defun npTypeVariablelist}
\getchunk{defun npVariable}
\getchunk{defun npVariablelist}
\getchunk{defun npVariableName}
\getchunk{defun npVoid}
\getchunk{defun npWConditional}
\getchunk{defun npWhile}
\getchunk{defun npWith}
\getchunk{defun npZeroOrMore}
\getchunk{defun NRTevalDomain}

\getchunk{defun ofCategory}
\getchunk{defun oldCompLookup}
\getchunk{defun oldHistFileName}
\getchunk{defun oldParseString}
\getchunk{defun om-bindTCP}
\getchunk{defun om-closeConn}
\getchunk{defun om-closeDev}
\getchunk{defun om-connectTCP}
\getchunk{defun om-getApp}
\getchunk{defun om-getAtp}
\getchunk{defun om-getAttr}
\getchunk{defun om-getBind}
\getchunk{defun om-getBVar}
\getchunk{defun om-getConnInDev}
\getchunk{defun om-getConnOutDev}
\getchunk{defun om-getEndApp}
\getchunk{defun om-getEndAtp}
\getchunk{defun om-getEndAttr}
\getchunk{defun om-getEndBind}
\getchunk{defun om-getEndBVar}
\getchunk{defun om-getEndError}
\getchunk{defun om-getEndObject}
\getchunk{defun om-getError}
\getchunk{defun om-getFloat}
\getchunk{defun om-getInt}
\getchunk{defun om-getObject}
\getchunk{defun om-getString}
\getchunk{defun om-getSymbol}
\getchunk{defun om-getType}
\getchunk{defun om-getVar}
\getchunk{defun om-listCDs}

```

```

\getchunk{defun om-listSymbols}
\getchunk{defun om-makeConn}
\getchunk{defun om-openFileDev}
\getchunk{defun om-openStringDev}
\getchunk{defun om-putApp}
\getchunk{defun om-putAtp}
\getchunk{defun om-putAttr}
\getchunk{defun om-putBind}
\getchunk{defun om-putBVar}
\getchunk{defun om-putByteArray}
\getchunk{defun om-putEndApp}
\getchunk{defun om-putEndAtp}
\getchunk{defun om-putEndAttr}
\getchunk{defun om-putEndBind}
\getchunk{defun om-putEndBVar}
\getchunk{defun om-putEndError}
\getchunk{defun om-putEndObject}
\getchunk{defun om-putError}
\getchunk{defun om-putFloat}
\getchunk{defun om-putInt}
\getchunk{defun om-putObject}
\getchunk{defun om-putString}
\getchunk{defun om-putSymbol}
\getchunk{defun om-putVar}
\getchunk{defun om-Read}
\getchunk{defun om-setDevEncoding}
\getchunk{defun om-stringPtrToString}
\getchunk{defun om-stringToStringPtr}
\getchunk{defun om-supportsCD}
\getchunk{defun om-supportsSymbol}
\getchunk{defun openOutputLibrary}
\getchunk{defun openserver}
\getchunk{defun operationopen}
\getchunk{defun optionError}
\getchunk{defun /options}
\getchunk{defun optionUserLevelError}
\getchunk{defun orderBySlotNumber}
\getchunk{defun originsInOrder}

\getchunk{defun parseAndEval}
\getchunk{defun parseAndEval1}
\getchunk{defun parseAndInterpret}
\getchunk{defun parseFromString}
\getchunk{defun parseNoMacroFromString}
\getchunk{defun parseSystemCmd}
\getchunk{defun parseWord}
\getchunk{defun pathname}
\getchunk{defun pathnameDirectory}
\getchunk{defun pathnameName}
\getchunk{defun pathnameType}

```

```

\getchunk{defun pathnameTypeId}
\getchunk{defun patternVarsOf}
\getchunk{defun patternVarsOf1}
\getchunk{defun permuteToOrder}
\getchunk{defun pcounters}
\getchunk{defun pfAbSynOp}
\getchunk{defun pfAbSynOp?}
\getchunk{defun pfAdd}
\getchunk{defun pfAnd}
\getchunk{defun pfAnd?}
\getchunk{defun pfApplication}
\getchunk{defun pfApplication?}
\getchunk{defun pfApplication2Sex}
\getchunk{defun pfAssign}
\getchunk{defun pfAssign?}
\getchunk{defun pfAttribute}
\getchunk{defun pfBrace}
\getchunk{defun pfBraceBar}
\getchunk{defun pfBracket}
\getchunk{defun pfBracketBar}
\getchunk{defun pfBreak}
\getchunk{defun pfBreak?}
\getchunk{defun pfCharPosn}
\getchunk{defun pfCheckArg}
\getchunk{defun pfCheckMacroOut}
\getchunk{defun pfCheckId}
\getchunk{defun pfCheckItOut}
\getchunk{defun pfCoerceto}
\getchunk{defun pfCoerceto?}
\getchunk{defun pfCollect}
\getchunk{defun pfCollect?}
\getchunk{defun pfCollect1?}
\getchunk{defun pfCollectArgTran}
\getchunk{defun pfCollectVariable1}
\getchunk{defun pfCollect2Sex}
\getchunk{defun pfCopyWithPos}
\getchunk{defun pfDefinition}
\getchunk{defun pfDefinition?}
\getchunk{defun pfDefinition2Sex}
\getchunk{defun pfDo}
\getchunk{defun pfDo?}
\getchunk{defun pfDocument}
\getchunk{defun pfEnSequence}
\getchunk{defun pfExit}
\getchunk{defun pfExit?}
\getchunk{defun pfExport}
\getchunk{defun pfExpression}
\getchunk{defun pfFileName}
\getchunk{defun pfFix}
\getchunk{defun pfFlattenApp}

```

```

\getchunk{defun pfFree}
\getchunk{defun pfFree?}
\getchunk{defun pfForin}
\getchunk{defun pfForin?}
\getchunk{defun pfFromDom}
\getchunk{defun pfFromdom}
\getchunk{defun pfFromdom?}
\getchunk{defun pfGlobalLinePosn}
\getchunk{defun pfHide}
\getchunk{defun pfId}
\getchunk{defun pfId?}
\getchunk{defun pfIdPos}
\getchunk{defun pfIdSymbol}
\getchunk{defun pfIf}
\getchunk{defun pfIf?}
\getchunk{defun pfIfThenOnly}
\getchunk{defun pfImport}
\getchunk{defun pfInline}
\getchunk{defun pfInfApplication}
\getchunk{defun pfIterate}
\getchunk{defun pfIterate?}
\getchunk{defun pfLam}
\getchunk{defun pfLambda}
\getchunk{defun pfLambdaTran}
\getchunk{defun pfLambda?}
\getchunk{defun pfLambda2Sex}
\getchunk{defun pfLeaf}
\getchunk{defun pfLeaf?}
\getchunk{defun pfLeafPosition}
\getchunk{defun pfLeafToken}
\getchunk{defun pfLhsRule2Sex}
\getchunk{defun pfLinePosn}
\getchunk{defun pfListOf}
\getchunk{defun pfLiteralClass}
\getchunk{defun pfLiteralString}
\getchunk{defun pfLiteral2Sex}
\getchunk{defun pfLocal}
\getchunk{defun pfLocal?}
\getchunk{defun pfLoop}
\getchunk{defun pfLoop1}
\getchunk{defun pfLoop?}
\getchunk{defun pfLp}
\getchunk{defun pfMacro}
\getchunk{defun pfMacro?}
\getchunk{defun pfMapParts}
\getchunk{defun pfMLambda}
\getchunk{defun pfMLambda?}
\getchunk{defun pfname}
\getchunk{defun pfNoPosition}
\getchunk{defun pfNoPosition?}

```

```

\getchunk{defun pfNot?}
\getchunk{defun pfNothing}
\getchunk{defun pfNothing?}
\getchunk{defun pfNoValue}
\getchunk{defun pfNoValue?}
\getchunk{defun pfOp2Sex}
\getchunk{defun pfOr}
\getchunk{defun pfOr?}
\getchunk{defun pfParen}
\getchunk{defun pfPretend}
\getchunk{defun pfPretend?}
\getchunk{defun pfPushBody}
\getchunk{defun pfPushMacroBody}
\getchunk{defun pfQualType}
\getchunk{defun pfRestrict}
\getchunk{defun pfRestrict?}
\getchunk{defun pfRetractTo}
\getchunk{defun pfReturn}
\getchunk{defun pfReturn?}
\getchunk{defun pfReturnNoName}
\getchunk{defun pfReturnTyped}
\getchunk{defun pfRhsRule2Sex}
\getchunk{defun pfRule}
\getchunk{defun pfRule?}
\getchunk{defun pfRule2Sex}
\getchunk{defun pfSequence}
\getchunk{defun pfSequence?}
\getchunk{defun pfSequenceToList}
\getchunk{defun pfSequence2Sex}
\getchunk{defun pfSequence2Sex0}
\getchunk{defun pfSexpr}
\getchunk{defun pfSexpr,strip}
\getchunk{defun pfSourcePosition}
\getchunk{defun pfSourceStok}
\getchunk{defun pfSpread}
\getchunk{defun pfSuch}
\getchunk{defun pfSuchthat}
\getchunk{defun pfSuchthat?}
\getchunk{defun pfSuchThat2Sex}
\getchunk{defun pfSymb}
\getchunk{defun pfSymbol}
\getchunk{defun pfSymbol?}
\getchunk{defun pfSymbolSymbol}
\getchunk{defun pfTagged}
\getchunk{defun pfTagged?}
\getchunk{defun pfTaggedToTyped}
\getchunk{defun pfTaggedToTyped1}
\getchunk{defun pfTransformArg}
\getchunk{defun pfTuple}
\getchunk{defun pfTupleListOf}

```

```

\getchunk{defun pfTweakIf}
\getchunk{defun pfTyped}
\getchunk{defun pfTyped?}
\getchunk{defun pfTyping}
\getchunk{defun pfTuple?}
\getchunk{defun pfUnSequence}
\getchunk{defun pfWDec}
\getchunk{defun pfWDeclare}
\getchunk{defun pfWhere}
\getchunk{defun pfWhere?}
\getchunk{defun pfWhile}
\getchunk{defun pfWhile?}
\getchunk{defun pfWith}
\getchunk{defun pfWrong}
\getchunk{defun pfWrong?}
\getchunk{defun pf0ApplicationArgs}
\getchunk{defun pf0DefinitionLhsItems}
\getchunk{defun pf0FlattenSyntacticTuple}
\getchunk{defun pf0ForinLhs}
\getchunk{defun pf0FreeItems}
\getchunk{defun pf0LambdaArgs}
\getchunk{defun pf0LocalItems}
\getchunk{defun pf0LoopIterators}
\getchunk{defun pf0MLambdaArgs}
\getchunk{defun pf0SequenceArgs}
\getchunk{defun pf0TupleParts}
\getchunk{defun pf0WhereContext}
\getchunk{defun pf2Sex}
\getchunk{defun pf2Sex1}
\getchunk{defun phMacro}
\getchunk{defun phParse}
\getchunk{defun phInterpret}
\getchunk{defun phIntReportMsgs}
\getchunk{defun phiRatapprox}
\getchunk{defun pileCforest}
\getchunk{defun pileColumn}
\getchunk{defun pileCtree}
\getchunk{defun pileForest}
\getchunk{defun pileForest1}
\getchunk{defun pileForests}
\getchunk{defun pilePlusComment}
\getchunk{defun pilePlusComments}
\getchunk{defun pileTree}
\getchunk{defun PiMinusLogSinPi}
\getchunk{defun poFileName}
\getchunk{defun poGlobalLinePosn}
\getchunk{defun poLinePosn}
\getchunk{defun poPosImmediate?}
\getchunk{defun porigin}
\getchunk{defun posend}

```

```

\getchunk{defun posPointers}
\getchunk{defun ppos}
\getchunk{defun pquit}
\getchunk{defun pquitSpad2Cmd}
\getchunk{defun previousInterpreterFrame}
\getchunk{defun printLabelledList}
\getchunk{defun printStatisticsSummary}
\getchunk{defun printStorage}
\getchunk{defun printSynonyms}
\getchunk{defun printTypeAndTime}
\getchunk{defun probeName}
\getchunk{defun processChPosesForOneLine}
\getchunk{defun processInteractive}
\getchunk{defun processInteractive1}
\getchunk{defun processKeyedError}
\getchunk{defun processMsgList}
\getchunk{defun protectedEVAL}
\getchunk{defun processSynonymLine}
\getchunk{defun processSynonymLine,removeKeyFromLine}
\getchunk{defun processSynonyms}
\getchunk{defun prTraceNames}
\getchunk{defun prTraceNames,fn}
\getchunk{defun pSearch}
\getchunk{defun PsiAsymptotic}
\getchunk{defun PsiBack}
\getchunk{defun PsiXotic}
\getchunk{defun pspacers}
\getchunk{defun ptimers}
\getchunk{defun put}
\getchunk{defun putFTText}
\getchunk{defun punctuation?}
\getchunk{defun putDatabaseStuff}
\getchunk{defun putHist}
\getchunk{defun pvarCondList1}
\getchunk{defun pvarCondList}
\getchunk{defun pvarPredTran}
\getchunk{defun pvarsOfPattern}

\getchunk{defun queryClients}
\getchunk{defun queueUpErrors}
\getchunk{defun quit}
\getchunk{defun quitSpad2Cmd}
\getchunk{defun quoteString}

\getchunk{defun rassocSub}
\getchunk{defun rbesseli}
\getchunk{defun rbesselj}
\getchunk{defun rdefinstream}
\getchunk{defun rdefoutstream}
\getchunk{defun read}

```



```

\getchunk{defun /read}
\getchunk{defun readHiFi}
\getchunk{defun readline}
\getchunk{defun readSpadProfileIfThere}
\getchunk{defun readSpad2Cmd}
\getchunk{defun reassembleTowerIntoType}
\getchunk{defun recordAndPrint}
\getchunk{defun recordFrame}
\getchunk{defun recordNewValue}
\getchunk{defun recordNewValue0}
\getchunk{defun recordOldValue}
\getchunk{defun recordOldValue0}
\getchunk{defun reduceAlistForDomain}
\getchunk{defun redundant}
\getchunk{defun regress command}
\getchunk{defun regress}
\getchunk{defun remFile}
\getchunk{defun remover}
\getchunk{defun removeTracedMapSigs}
\getchunk{defun removeUndoLines}
\getchunk{defun renamePatternVariables1}
\getchunk{defun renamePatternVariables}
\getchunk{defun replaceFile}
\getchunk{defun replacePercentByDollar,fn}
\getchunk{defun replacePercentByDollar}
\getchunk{defun replaceSharps}
\getchunk{defun reportA0}
\getchunk{defun reportinstantiations}
\getchunk{defun reportOperations}
\getchunk{defun reportOpsFromLisplib}
\getchunk{defun reportOpsFromLisplib0}
\getchunk{defun reportOpsFromLisplib1}
\getchunk{defun reportOpsFromUnitDirectly}
\getchunk{defun reportOpsFromUnitDirectly0}
\getchunk{defun reportOpsFromUnitDirectly1}
\getchunk{defun reportSpadTrace}
\getchunk{defun reportUndo}
\getchunk{defun reportWhatOptions}
\getchunk{defun reroot}
\getchunk{defun resetCounters}
\getchunk{defun resethashtables}
\getchunk{defun resetInCoreHist}
\getchunk{defun resetSpacers}
\getchunk{defun resetTimers}
\getchunk{defun resetWorkspaceVariables}
\getchunk{defun restart}
\getchunk{defun restart0}
\getchunk{defun restoreHistory}
\getchunk{defun retract}
\getchunk{defun retractByFunction}

```

```

\getchunk{defun retractUnderDomain}
\getchunk{defun retract2Specialization}
\getchunk{defun rgamma}
\getchunk{defun rgammaImpl}
\getchunk{defun rlngamma}
\getchunk{defun rpsi}
\getchunk{defun rPsiImpl}
\getchunk{defun rPsiW}
\getchunk{defun rread}
\getchunk{defun ruleLhsTran}
\getchunk{defun rulePredicateTran}
\getchunk{defun runspad}
\getchunk{defun rwrite}

\getchunk{defun safeWritify}
\getchunk{defun sameMsg?}
\getchunk{defun satisfiesRegularExpressions}
\getchunk{defun saveHistory}
\getchunk{defun saveMapSig}
\getchunk{defun savesystem}
\getchunk{defun saveDependentsHashTable}
\getchunk{defun saveUsersHashTable}
\getchunk{defun sayAllCacheCounts}
\getchunk{defun sayBrightly1}
\getchunk{defun sayCacheCount}
\getchunk{defun sayExample}
\getchunk{defun sayKeyedMsg}
\getchunk{defun sayKeyedMsgLocal}
\getchunk{defun sayMSG}
\getchunk{defun sayMSG2File}
\getchunk{defun sayShowWarning}
\getchunk{defun scanCheckRadix}
\getchunk{defun scanComment}
\getchunk{defun scanDictCons}
\getchunk{defun scanError}
\getchunk{defun scanEsc}
\getchunk{defun scanEscape}
\getchunk{defun scanExponent}
\getchunk{defun scanIgnoreLine}
\getchunk{defun scanInsert}
\getchunk{defun scanKeyTr}
\getchunk{defun scanNegComment}
\getchunk{defun scanNumber}
\getchunk{defun ScanOrPairVec}
\getchunk{defun ScanOrPairVec,ScanOrInner}
\getchunk{defun scanPossFloat}
\getchunk{defun scanPunct}
\getchunk{defun scanPunCons}
\getchunk{defun scanS}
\getchunk{defun scanSpace}

```

```
\getchunk{defun scanString}  
\getchunk{defun scanKeyTableCons}  
\getchunk{defun scanToken}  
\getchunk{defun scanTransform}  
\getchunk{defun scanW}  
\getchunk{defun scanWord}  
\getchunk{defun search}  
\getchunk{defun searchCurrentEnv}  
\getchunk{defun searchTailEnv}  
\getchunk{defun segmentKeyedMsg}  
\getchunk{defun selectOption}  
\getchunk{defun selectOptionLC}  
\getchunk{defun separatePiles}  
\getchunk{defun serverReadLine}  
\getchunk{defun set}  
\getchunk{defun set1}  
\getchunk{defun setdatabase}  
\getchunk{defun setExpose}  
\getchunk{defun setExposeAdd}  
\getchunk{defun setExposeAddConstr}  
\getchunk{defun setExposeAddGroup}  
\getchunk{defun setExposeDrop}  
\getchunk{defun setExposeDropConstr}  
\getchunk{defun setExposeDropGroup}  
\getchunk{defun setFortDir}  
\getchunk{defun setFortPers}  
\getchunk{defun setFortTmpDir}  
\getchunk{defun setFunctionsCache}  
\getchunk{defun setHistoryCore}  
\getchunk{defun setInputLibrary}  
\getchunk{defun setIOindex}  
\getchunk{defun setLinkerArgs}  
\getchunk{defun setMsgCatlessAttr}  
\getchunk{defun setMsgForcedAttr}  
\getchunk{defun setMsgForcedAttrList}  
\getchunk{defun setMsgUnforcedAttr}  
\getchunk{defun setMsgUnforcedAttrList}  
\getchunk{defun setNagHost}  
\getchunk{defun setOutputAlgebra}  
\getchunk{defun setOutputCharacters}  
\getchunk{defun setOutputFormula}  
\getchunk{defun setOutputFortran}  
\getchunk{defun setOutputLibrary}  
\getchunk{defun setOutputHtml}  
\getchunk{defun setOutputMathml}  
\getchunk{defun setOutputOpenMath}  
\getchunk{defun setOutputTex}  
\getchunk{defun setStreamsCalculate}  
\getchunk{defun setUpDefault}  
\getchunk{defun shortenForPrinting}
```

```

\getchunk{defun show}
\getchunk{defun showdatabase}
\getchunk{defun showHistory}
\getchunk{defun showInOut}
\getchunk{defun showInput}
\getchunk{defun showSpad2Cmd}
\getchunk{defun shut}
\getchunk{defun size}
\getchunk{defun SkipEnd?}
\getchunk{defun SkipPart?}
\getchunk{defun Skipping?}
\getchunk{defun smallEnough}
\getchunk{defun spad}
\getchunk{defun spadClosure?}
\getchunk{defun spad-error-loc}
\getchunk{defun SpadInterpretStream}
\getchunk{defun spad-long-error}
\getchunk{defun spadReply}
\getchunk{defun spadrread}
\getchunk{defun spadrwrite}
\getchunk{defun spadrwrite0}
\getchunk{defun spad-save}
\getchunk{defun spad-short-error}
\getchunk{defun spadStartupMsgs}
\getchunk{defun spad-syntax-error}
\getchunk{defun spadTrace}
\getchunk{defun spadTraceAlias}
\getchunk{defun spadTrace,g}
\getchunk{defun spadTrace,isTraceable}
\getchunk{defun spadUntrace}
\getchunk{defun spad2BootCoerce}
\getchunk{defun specialChar}
\getchunk{defun spleI}
\getchunk{defun spleI1}
\getchunk{defun split}
\getchunk{defun splitIntoOptionBlocks}
\getchunk{defun stackTraceOptionError}
\getchunk{defun startp}
\getchunk{defun startsNegComment?}
\getchunk{defun statisticsInitialization}
\getchunk{defun streamChop}
\getchunk{defun stringList2String}
\getchunk{defun stringMatches?}
\getchunk{defun string2Constructor}
\getchunk{defun StringToDir}
\getchunk{defun stripUnionTags}
\getchunk{defun strpos}
\getchunk{defun strpos1}
\getchunk{defun stupidIsSpadFunction}
\getchunk{defun subMatch}

```

```

\getchunk{defun substFromAlist}
\getchunk{defun substringMatch}
\getchunk{defun subTypes}
\getchunk{defun summary}
\getchunk{defun syGeneralErrorHere}
\getchunk{defun syIgnoredFromTo}
\getchunk{defun synonym}
\getchunk{defun synonymsForUserLevel}
\getchunk{defun synonymSpad2Cmd}
\getchunk{defun sySpecificErrorAtToken}
\getchunk{defun sySpecificErrorHere}
\getchunk{defun systemCommand}

\getchunk{defun ?t}
\getchunk{defun tabbing}
\getchunk{defun templateParts}
\getchunk{defun tangle}
\getchunk{defun terminateSystemCommand}
\getchunk{defun tersyscommand}
\getchunk{defun testnumberp}
\getchunk{defun testpassed}
\getchunk{defun thisPosIsEqual}
\getchunk{defun thisPosIsLess}
\getchunk{defun throwEvalTypeMsg}
\getchunk{defun toFile?}
\getchunk{defun tokConstruct}
\getchunk{defun token-stack-show}
\getchunk{defun tokPosn}
\getchunk{defun tokTran}
\getchunk{defun tokType}
\getchunk{defun topLevelInterpEval}
\getchunk{defun trace}
\getchunk{defun trace1}
\getchunk{defun traceDomainConstructor}
\getchunk{defun traceDomainLocalOps}
\getchunk{defun tracelet}
\getchunk{defun traceOptionError}
\getchunk{defun /tracereply}
\getchunk{defun traceReply}
\getchunk{defun trace2}
\getchunk{defun trace3}
\getchunk{defun traceSpad2Cmd}
\getchunk{defun translateTrueFalse2YesNo}
\getchunk{defun translateYesNo2TrueFalse}
\getchunk{defun translateYesNoToTrueFalse}
\getchunk{defun transOnlyOption}
\getchunk{defun transTraceItem}
\getchunk{defun truncList}
\getchunk{defun typeCheckInputAreas}

```

```

\getchunk{defun unAbbreviateKeyword}
\getchunk{defun undo}
\getchunk{defun undoChanges}
\getchunk{defun undoCount}
\getchunk{defun undoFromFile}
\getchunk{defun undoInCore}
\getchunk{defun undoLocalModemapHack}
\getchunk{defun undoSingleStep}
\getchunk{defun undoSteps}
\getchunk{defun unescapeStringsInForm}
\getchunk{defun unifyStruct}
\getchunk{defun unifyStructVar}
\getchunk{defun unparseInputForm}
\getchunk{defun untrace}
\getchunk{defun /untrace-0}
\getchunk{defun /untrace-1}
\getchunk{defun /untrace-2}
\getchunk{defun untraceDomainConstructor}
\getchunk{defun untraceDomainConstructor,keepTraced?}
\getchunk{defun untraceDomainLocalOps}
\getchunk{defun untraceMapSubNames}
\getchunk{defun unwritable?}
\getchunk{defun upcase}
\getchunk{defun updateCurrentInterpreterFrame}
\getchunk{defun updateDatabase}
\getchunk{defun updateFromCurrentInterpreterFrame}
\getchunk{defun updateHist}
\getchunk{defun updateInCoreHist}
\getchunk{defun updateSourceFiles}
\getchunk{defun userLevelErrorMessage}

\getchunk{defun validateOutputDirectory}
\getchunk{defun valueArgsEqual?}
\getchunk{defun var}
\getchunk{defun voidValue}

\getchunk{defun what}
\getchunk{defun whatCommands}
\getchunk{defun whatConstructors}
\getchunk{defun whatSpad2Cmd}
\getchunk{defun whatSpad2Cmd,fixpat}
\getchunk{defun whichCat}
\getchunk{defun with}
\getchunk{defun workfiles}
\getchunk{defun workfilesSpad2Cmd}
\getchunk{defun wrap}
\getchunk{defun write-browsedb}
\getchunk{defun write-categorydb}
\getchunk{defun writeHiFi}
\getchunk{defun writeHistModesAndValues}

```

```
\getchunk{defun writeInputLines}
\getchunk{defun write-interpdb}
\getchunk{defun write-operationdb}
\getchunk{defun write-warmdata}
\getchunk{defun writify}
\getchunk{defun writifyComplain}
\getchunk{defun writify,writifyInner}

\getchunk{defun xlCannotRead}
\getchunk{defun xlCmdBug}
\getchunk{defun xlConActive}
\getchunk{defun xlConsole}
\getchunk{defun xlConStill}
\getchunk{defun xlFileCycle}
\getchunk{defun xlIfBug}
\getchunk{defun xlIfSyntax}
\getchunk{defun xlMsg}
\getchunk{defun xlNoSuchFile}
\getchunk{defun xlOK}
\getchunk{defun xlOK1}
\getchunk{defun xlPrematureEOF}
\getchunk{defun xlPrematureFin}
\getchunk{defun xlSay}
\getchunk{defun xlSkip}
\getchunk{defun xlSkippingFin}
\getchunk{defun xSearch}

\getchunk{defun yesanswer}
\getchunk{defun ySearch}

\getchunk{postvars}
```

Chapter 59

The Global Variables

59.1 Star Global Variables

NAME	SET	USE
<code>*eof*</code>	<code>ncTopLevel</code>	
<code>*features*</code>		restart
<code>*package*</code>		restart
<code>*standard-input*</code>		<code>ncIntLoop</code>
<code>*standard-output*</code>		<code>ncIntLoop</code>
<code>*top-level-hook*</code>	<code>set-restart-hook</code>	

`*eof*`

The `*eof*` variable is set to `NIL` in `ncTopLevel`.

`*features*`

The `*features*` variable from common lisp is tested for the presence of the `:unix` keyword. Apparently this controls the use of Saturn, a previous Axiom frontend. The Saturn frontend was never released as open source and so this test and the associated variables are probably not used.

`*package*`

The `*package*` variable, from common lisp, is set in restart to the `BOOT` package where the interpreter lives.

standard-input

The ***standard-input*** common lisp variable is used to set the `curinstream` variable in `ncIntLoop`.

This variable is an argument to `serverReadLine` in the `intloopReadConsole` function.

standard-output

The ***standard-output*** common lisp variable is used to set the `curoutstream` variable in `ncIntLoop`.

top-level-hook

The ***top-level-hook*** common lisp variable contains the name of a function to invoke when an image is started. In our case it is called `restart`. This is the entry point to the Axiom interpreter.

59.2 Dollar Global Variables

NAME	SET	USE
\$boot	ncTopLevel	
coerceFailure		runspad
curinstream	ncIntLoop	
curoutstream	ncIntLoop	
\$currentLine	restart	removeUndoLines
\$dalymode		intloopReadConsole
\$displayStartMsgs		restart
\$e	ncTopLevel	
\$erMsgToss	SpadInterpretStream	
	SpadInterpretStream	
\$frameRecord	initvars	
	clearFrame	
	undoSteps	undoSteps
	recordFrame	recordFrame
\$HiFiAccess	initHist	historySpad2Cmd
	historySpad2Cmd	
		setHistoryCore
\$HistList	initHist	
\$HistListAct	initHist	
\$HistListLen	initHistList	
\$HistRecord	initHistList	
\$historyDirectory		makeHistFileName
		makeHistFileName
\$historyFileType	initvars	histInputFileName
\$InteractiveFrame	restart	ncTopLevel
	undo	recordFrame
	undoSteps	undoSteps
		reportUndo
\$internalHistoryTable	initvars	
\$interpreterFrameName	initializeInterpreterFrameRing	
\$interpreterFrameRing	initializeInterpreterFrameRing	
\$intRestart		intloop
\$intTopLevel	intloop	
\$IOindex	restart	historySpad2Cmd
	removeUndoLines	undoCount
\$genValue	bookvol5	i-toplev
		i-analy
		i-syscmd
		i-spec1
		i-spec2
		i-map
\$lastPos	SpadInterpretStream	
\$libQuiet	SpadInterpretStream	
\$msgDatabaseName	reroot *	
\$ncMsgList	SpadInterpretStream	
\$newcompErrorCount	SpadInterpretStream	
\$newspad	ncTopLevel	
\$nopus		SpadInterpretStream
\$okToExecuteMachineCode	SpadInterpretStream	
\$oldHistoryFileName	initvars	oldHistFileName
\$options		history
	historySpad2Cmd	historySpad2Cmd
		undo

\$boot

The `$boot` variable is set to `NIL` in `ncTopLevel`.

coerceFailure

The `coerceFailure` symbol is a catch tag used in `runspad` to catch an exit from `ncTopLevel`.

\$currentLine

The `$currentLine` line is set to `NIL` in `restart`. It is used in `removeUndoLines` in the undo mechanism.

\$displayStartMsgs

The `$displayStartMsgs` variable is used in `restart` but is not set so this is likely a bug.

\$erMsgToss

The `$erMsgToss` variable is set to `NIL` in `SpadInterpretStream`.

\$frameRecord

`$frameRecord = [delta1, delta2, ... ]` where `delta(i)` contains changes in the “backwards” direction. Each `delta(i)` has the form `((var . proplist)...)` where `proplist` denotes an ordinary `proplist`. For example, an entry of the form `((x (value) (mode (Integer))))...` indicates that to undo 1 step, `x`’s value is cleared and its mode should be set to `(Integer)`.

A `delta(i)` of the form `(systemCommand . delta)` is a special `delta` indicating changes due to system commands executed between the last command and the current command. By recording these `deltas` separately, it is possible to undo to either `BEFORE` or `AFTER` the command. These special `delta(i)`s are given **ONLY** when a system command is given which alters the environment.

`recordFrame('system)` is called before a command is executed, and `recordFrame('normal)` is called after (see `processInteractive1`). If no changes are found for former, no special entry is given.

This is part of the undo mechanism.

\$intRestart

The `$intRestart` variable is used in `intloop` but has no value. This is probably a bug. While the variable’s value is unchanged the system will continually reenter the `SpadInterpretStream`

function.

\$intTopLevel

The `$intTopLevel` is a catch tag. Throwing to this tags which is caught in the `intloop` will restart the `SpadInterpretStream` function.

\$IOindex

The `$IOindex` index variable is set to 1 in `restart`. This variable is used in the `historySpad2Cmd` function in the history mechanism. It is set in the `removeUndoLines` function in the undo mechanism.

This is used in the undo mechanism in function `undoCount` to compute the number of undos. You can't undo more actions then have already happened.

\$lastPos

The `$lastPos` variable is set in `SpadInterpretStream` to the value of the `$npos` variable. Since `$npos` appears to have no value this is likely a bug.

\$libQuiet

The `$libQuiet` variable is set to the third argument of the `SpadInterpretStream` function. This is passed from `intloop` with the value of `T`. This variable appears to be intended to control the printing of library loading messages which would need to be suppressed if input was coming from a file.

\$msgDatabaseName

The `$msgDatabaseName` is set to `NIL` in `reroot`.

\$ncMsgList

The `$ncMsgList` is set to `NIL` in `SpadInterpretStream`.

\$newcompErrorCount

The `$newcompErrorCount` is set to 0 in `SpadInterpretStream`.

\$nopus

The `$nopus` variable is used in `SpadInterpretStream` but does not appear to have a value and is likely a bug.

\$oldHistoryFileName

The `$oldHistoryFileName` is set at load time by a call to `initvars` to a value of “last”. It is part of the history mechanism. It is used in the function `oldHistFileName` and `restoreHistory`.

\$okToExecuteMachineCode

The `$okToExecuteMachineCode` is set to T in `SpadInterpretStream`.

\$options

The `$options` variable is tested by the history function. If it is NIL then output the message

```
You have not used the correct syntax for the history command.  
Issue )help history for more information.
```

The `$options` variable is tested in the `historySpad2Cmd` function. It appears to record the options that were given to a `spad` command on the input line. The function `selectOptionLC` appears to take a list off options to scan.

This variable is not yet set and is probably a bug.

\$previousBindings

The `$previousBindings` is a copy of the CAAR `$InteractiveFrame`. This is used to compute the `delta(i)s` stored in `$frameRecord`. This is part of the undo mechanism.

\$reportundo

The `$reportundo` variable is used in `diffAlist`. It was not normally bound but has been set to T in `initvars`. If the variable is set to T then we call `reportUndo`.

It is part of the undo mechanism.

\$spad

The `$spad` variable is set to T in `ncTopLevel`.

\$SpadServer

If an open server is not requested then this variable is T. It has no value before this time (and is thus a bug).

\$SpadServerName

The `$SpadServerName` is passed to the `openServer` function, if the function exists.

\$systemCommandFunction

The `$systemCommandFunction` is set in `SpadInterpretStream` to point to the function `InterpExecuteSpadSystemCommand`.

top_level

The `top_level` symbol is a catch tag used in `runspad` to catch an exit from `ncTopLevel`.

\$quitTag

The `$quitTag` is used as a variable in a catch block. It appears that it can be thrown somewhere below `ncTopLevel`.

\$useInternalHistoryTable

The `$useInternalHistoryTable` variable is set at load time by a call to `initvars` to a value of `NIL`. It is part of the history mechanism.

```

22 type : FrameName → Symbol
22 frameName : Frame → FrameName
22 frameInteractive : Frame → Interactive
22 frameIOIndex : Frame → IOIndex
23 frameHiFiAccess : Frame → HiFiAccess
23 frameHistList : Frame → HistList
23 type : HistListLen → NonNegativeInteger
23 frameHistListLen : Frame → HistListLen
24 frameHistListAct : Frame → HistListAct
24 frameHistRecord : Frame → HistRecord
24 frameHistoryTable : Frame → HistoryTable
25 frameExposureData : Frame → ExposureData
25 enum : FrameArgs → (nil,drop,import,last,names,new,next)
25 frameSpad2Cmd : FrameArgs → nil
27 emptyInterpreterFrame : Symbol → Frame
27 emptyInterpreterFrame : Symbol → Frame
28 frameNames : nil → List Symbol
28 displayFrameNames : nil → nil
29 createCurrentInterpreterFrame : nil → Frame
29 updateFromCurrentInterpreterFrame : nil → nil
31 updateCurrentInterpreterFrame : nil → nil
31 frameEnvironment : FrameName → nil
31 findFrameInRing : FrameName → Union(Frame,nil)
32 changeToNamedInterpreterFrame : FrameName → nil
32 nextInterpreterFrame : nil → nil
33 previousInterpreterFrame : nil → nil
37 closeInterpreterFrame : FrameName → nil
88 embed2 : Symbol,Function,Function → Symbol
100 removeOption : Option → List Option
108 flattenOperationAlist : OperationAlist → OperationAlist
124 /untrace-reduce : Union(Atom,List) → Atom
126 transTraceItem : traceArgument → symbol
305 set-restart-hook : Void → 'restart
321 intloopReadConsole : (String Integer) → Throw

```



```

327 intloopPrefix? : String → Union(String,NIL)
329 intloopProcessString : (String,StepNo) → StepNo
330 next : (Function,Delay) → Delay
330 next1 : Delay → ParsePair
331 incString : String → Function
334 setCurrentLine : String → List(String)
334 mkprompt : Void → String
336 serverReadLine : Stream → String
356 intloopProcess : (StepNo,Boolean,Delay) → StepNo
366 incRenumber : Delay → Delay
366 incZip : (Function,Delay,Delay) → Delay
366 incZip1 : Delay → ParsePair
367 incIgen : Integer → Delay
369 incLude : (Int,List(String),Int,List(String),List(Int)) → Delay
394 incCommand? : String → Boolean
397 Delay : (Function,List(Any)) → Delay
623 StreamNull : Delay → Union(T,NIL)
1243 fracpart : or rational float → or rational float
1402 getHtMacroItem : String → Values (String NonNegativeInteger)
1468 bcGen : Command → nil
1469 doDoitButton : String,Command → nil
1469 doDoitButton : Command → nil
1470 executeInterpreterCommand : Command → nil
1662 isWrapped : t → (or t nil)

```

Bibliography

[Book13] Axiom Authors. *Volume 13: Proving Axiom Correct*. Axiom Project, 2016.

Link: <http://axiom-developer.org/axiom-website/bookvol13.pdf>

[Burg74] William H. Burge. Stream processing functions. *IBM Journal of Research and Development*, 19:12–25, January 1974.

Abstract: One principle of structured programming is that a program should be separated into meaningful independent subprograms, which are then combined so that the relation of the parts to the whole can be clearly established. This paper describes several alternative ways to compose programs. The main method used is to permit the programmer to denote by an expression the sequence of values taken on by a variable. The sequence is represented by a function called a stream, which is a functional analog of a coroutine. The conventional while and for loops of structured programming may be composed by a technique of stream processing (analogous to list processing), which results in more structured programs than the originals. This technique makes it possible to structure a program in a natural way into its logically separate parts, which can then be considered independently.

[Dalm97] Stéphane Dalmas, Marc Gaëtano, and Stephen Watt. An openmath 1.0 implementation. In *Proc. 1997 Int. Symp. on Symbolic and Algebraic Computation*, ISSAC’97, pages 241–248. ACM, New York, NY USA, 1997, 0-89791-875-4.

Link: <http://doi.acm.org/10.1145/258726.258794>

[Dewa] Mike Dewar. Openmath: An overview.

Link: <http://www.sigsam.org/bulletin/articles/132/paper1.pdf>

[Fort85] A. Fortenbacher, Richard Jenks, Michael Lucks, Robert Sutor, Barry Trager, and Stephen Watt. An overview of the scratchpad ii language and system. Research report, IBM, 1985.

- [Jenk81] Richard D. Jenks and Barry M. Trager. A language for computational algebra. In *Proc. Symp. on Symbolic and Algebraic Manipulation*, SYMSAC 1981, 1981.

Abstract: This paper reports ongoing research at the IBM Research Center on the development of a language with extensible parameterized types and generic operators for computational algebra. The language provides an abstract data type mechanism for defining algorithms which work in as general a setting as possible. The language is based on the notions of domains and categories. Domains represent algebraic structures. Categories designate collections of domains having common operations with stated mathematical properties. Domains and categories are computed objects which may be dynamically assigned to variables, passed as arguments, and returned by functions. Although the language has been carefully tailored for the application of algebraic computation, it actually provides a very general abstract data type mechanism. Our notion of a category to group domains with common properties appears novel among programming languages (cf. image functor of RUSSELL) and leads to a very powerful notion of abstract algorithms missing from other work on data types known to the authors.

Comment: IBM Research Report 8930

- [Jenk86] Richard D. Jenks, Robert S. Sutor, and Stephen M. Watt. Scratchpad ii: An abstract datatype system for mathematical computation. Research Report RC 12327 (#55257), IBM Research, 1986.

Abstract: Scratchpad II is an abstract datatype language and system that is under development in the Computer Algebra Group, Mathematical Sciences Department, at the IBM Thomas J. Watson Research Center. Some features of APL that made computation particularly elegant have been borrowed. Many different kinds of computational objects and data structures are provided. Facilities for computation include symbolic integration, differentiation, factorization, solution of equations and linear algebra. Code economy and modularity is achieved by having polymorphic packages of functions that may create datatypes. The use of categories makes these facilities as general as possible.

Link: <http://www.csd.uwo.ca/~watt/pub/reprints/1987-ima-spadadt.pdf>

- [Lisk79] Barbara Liskov, Russ Atkinson, Toby Bloom, Eliot Moss, Craig Schaffert, Bob Scheifler, and Alan Snyder. Clu reference manual. Technical report, Massachusetts Institute of Technology, 1979.
- [Luke69a] Yudell L. Luke. *The Special Functions and their Approximations*, volume 1. Academic Press, 1969. **Algebra:**
(p??) package DFSFUN DoubleFloatSpecialFunctions

- [Luke77] Yudell L. Luke. *Algorithms for the Computation of Mathematical Functions*. Academic Press, 1977, 978-0124599406.
- [Scha86] C. Schaffert and T. Cooper. An introduction to trellis/owl. *SIGPLAN Notices*, 21(11):9–16, 1986.
- [Swee86] Moss E. Sweedler. Typing in scratchpad ii. Scratchpad II Newsletter 2, IBM Research, January 1986.
- [Watt87] Stephen M. Watt and Richard D. Jenks. Abstract datatypes, multiple views and multiple inheritance in scratchpad ii, 1987.

Abstract: Scratchpad II is an abstract datatype language developed at Yorktown Heights for the implementation of a new computer algebra system. It provides packages of polymorphic functions and parameterized, abstract datatypes with operator overloading and multiple inheritance. To express the intricate inter-relationships between the datatypes necessary for the description of mathematical objects, a number of techniques based on the notion of *category* have been used. Categories are used to enforce relationships between type parameters and to provide the mechanism for multiple inheritance. They also allow the language to be statically type checked and the generation of efficient code. This paper describes the role of categories in Scratchpad II.

Link: <https://cs.uwaterloo.ca/~smatt/pub/reprints/1987-itl-spadvies.pdf>

- [kuki72a] Hirondo Kuki. Complex gamma function with error control. *Communications of the ACM*, 15(4):262–267, 1972.

Abstract: An algorithm to compute the gamma function and the loggamma function of a complex variable is presented. The standard algorithm is modified in several respects to insure the continuity of the function value and to reduce accumulation of round-off errors. In addition to computation of function values, this algorithm includes an object-time estimation of round-off errors. Experimental data with regard to the effectiveness of this error control are presented. A Fortran program for the algorithm appears in the algorithms section of this issue.

- [kuki72b] Hirondo Kuki. Algorithm 421: Complex gamma function with error control. *Communications of the ACM*, 15(4):271–272, 1972.

Abstract: This Fortran program computes either the gamma function or the loggamma function of a complex variable in double precision. In addition, it provides an error estimate of the computed answer. The calling sequences are: CALL CDLGAM (X, W, E, 0) for the loggamma, and CALL CDLGAM (X, W, E, 1) for the gamma, where Z is the double precision argument, W is the answer of the same type, and E is a single

precision real variable. Before the call, the value of E is an estimate of the error in Z , and after the call, it is an estimate of the error in W .

Index

- `*ThisIsARunningSystem*`, 959
 - defvar, 959
- `*all-tests-ran*`, 950
 - usedby regress, 950
 - defvar, 950
- `*allOperations*`, 1186
 - usedby allOperations, 1219
 - usedby localnrlib, 1201
 - defvar, 1186
- `*allconstructors*`, 1186
 - usedby allConstructors, 1218
 - usedby browseopen, 1191
 - usedby interpopen, 1190
 - usedby localnrlib, 1201
 - usedby make-databases, 1205
 - usedby resethashtables, 1187
 - defvar, 1186
- `*ancestors-hash*`
 - usedby write-interpdb, 1213
- `*browse-stream*`, 1185
 - usedby browseopen, 1191
 - usedby getdatabase, 1196
 - usedby resethashtables, 1186
 - defvar, 1185
- `*browse-stream-stamp*`, 1185
 - usedby browseopen, 1191
 - defvar, 1185
- `*build-version*`
 - usedby axiomVersion, 816
 - usedby spadStartUpMsgs, 309
- `*category-stream*`, 1185
 - usedby categoryopen, 1192
 - usedby getdatabase, 1196
 - usedby resethashtables, 1186
 - defvar, 1185
- `*category-stream-stamp*`, 1186
 - usedby categoryopen, 1192
 - usedby resethashtables, 1186
 - defvar, 1186
- `*defaultdomain-list*`, 1182
 - usedby getdatabase, 1196
 - defvar, 1182
- `*eof*`, 314
 - usedby ncTopLevel, 316
 - usedby serverReadLine, 337
 - defvar, 314
- `*hasCategory-hash*`, 1183
 - usedby categoryopen, 1192
 - usedby getdatabase, 1196
 - usedby write-categorydb, 1216
 - defvar, 1183
- `*hascategory-hash*`
 - usedby getdatabase, 1196
 - usedby resethashtables, 1187
- `*index-filename*`
 - usedby localdatabase, 1200
- `*interp-stream*`, 1184
 - usedby getdatabase, 1196
 - usedby interpopen, 1190
 - usedby resethashtables, 1186
 - defvar, 1184
- `*interp-stream-stamp*`, 1184
 - usedby interpopen, 1190
 - usedby resethashtables, 1187
 - defvar, 1184
- `*miss*`, 1183
 - usedby getdatabase, 1196
 - defvar, 1183
- `*monitor-domains*`, 1381
 - usedby monitor-readinterp, 1394
 - usedby monitor-report, 1395
 - usedby monitor-spadfile, 1396
 - defvar, 1381
- `*monitor-nrlibs*`, 1381

- usedby monitor-dirname, 1392
- defvar, 1381
- *monitor-table*, 1382
 - usedby monitor-add, 1384
 - usedby monitor-apropos, 1397
 - usedby monitor-checkpoint, 1389
 - usedby monitor-decr, 1386
 - usedby monitor-delete, 1384
 - usedby monitor-disable, 1385
 - usedby monitor-enable, 1384
 - usedby monitor-end, 1383
 - usedby monitor-incr, 1386
 - usedby monitor-info, 1387
 - usedby monitor-inittable, 1382
 - usedby monitor-nrlib, 1393
 - usedby monitor-percent, 1397
 - usedby monitor-reset, 1385
 - usedby monitor-results, 1383
 - usedby monitor-tested, 1388
 - usedby monitor-untested, 1387
 - defvar, 1382
- *ok*, 952
 - usedby startp, 955
 - usedby testpassed, 953
 - defvar, 952
- *operation-hash*, 1183
 - usedby addoperations, 1193
 - usedby allOperations, 1219
 - usedby getdatabase, 1196
 - usedby make-databases, 1205
 - usedby operationopen, 1193
 - usedby resethashtables, 1187
 - usedby write-operationdb, 1217
 - defvar, 1183
- *operation-stream*, 1184
 - usedby getdatabase, 1196
 - usedby operationopen, 1193
 - usedby resethashtables, 1186
 - defvar, 1184
- *operation-stream-stamp*, 1185
 - usedby operationopen, 1193
 - usedby resethashtables, 1186
 - defvar, 1185
- *print-length*
 - usedby limitedPrint1, 153
- *print-level*
 - usedby limitedPrint1, 153
- *print-package*
 - usedby monitor-checkpoint, 1389
- *print-pretty*
 - usedby write-browsedb, 1215
 - usedby write-categorydb, 1216
 - usedby write-interpdb, 1213
- *read-place-holder*, 1296
 - usedby placep, 1297
 - defvar, 1296
- *sourcefiles*, 1204
 - usedby make-databases, 1205
 - usedby resethashtables, 1186
 - usedby write-browsedb, 1215
 - defvar, 1204
- *standard-output*
 - usedby init-boot/spad-reader, 1154
- *whitespace*, 314
 - usedby assertCond, 389
 - defvar, 314
- *yearweek*
 - usedby axiomVersion, 816
 - usedby spadStartUpMsgs, 309
- /D,1
 - calledby compileBoot, 115
- /breakcondition
 - usedby break, 156
- /editfile, 849
 - usedby /read, 945
 - usedby editSpad2Cmd, 872
 - usedby readSpad2Cmd, 943
 - usedby readSpadProfileIfThere, 1147
 - usedby resetWorkspaceVariables, 962
 - defvar, 849
- /options, 96
 - calledby /options, 96
 - calledby /untrace-0, 123
 - calls /options, 96
 - calls isFunctor, 96
 - defun, 96
- /pretty
 - usedby resetWorkspaceVariables, 962
- /read, 944
 - calledby readSpad2Cmd, 943
 - calls /rf[9], 944
 - calls /rq[9], 944

- uses /editfile, 945
 - defun, 944
- /rf[9]
 - called by /read, 944
- /rq[9]
 - called by /read, 944
- /sourcefiles
 - used by resetWorkspaceVariables, 962
- /trace,0
 - called by trace1, 79
- /tracereply, 142
 - called by /untrace-1, 124
 - calls devaluate, 142
 - calls exit, 142
 - calls isDomainOrPackage, 142
 - calls qcar, 142
 - calls seq, 142
 - uses \$traceNames, 142
 - defun, 142
- /untrace,0
 - called by untraceDomainConstructor,keepTraced, 138
 - called by untrace, 123
- /untrace,2
 - called by untraceMapSubNames, 105
- /untrace-0, 123
 - calls /options, 123
 - calls /untrace-1, 123
 - calls truncList, 123
 - defun, 123
- /untrace-1, 124
 - called by /untrace-0, 123
 - calls /tracereply, 124
 - calls /untrace-2, 124
 - calls /untrace-reduce, 124
 - uses \$traceNames, 124
 - defun, 124
- /untrace-2, 124
 - called by /untrace-1, 124
 - calls concat, 125
 - calls devaluate, 125
 - calls eqcar, 124
 - calls isDomainOrPackage, 124
 - calls isDomain, 124
 - calls isFunctor, 124
 - calls isGenvar, 125
 - calls isSubForRedundantMapName, 124
 - calls rassocSub, 125
 - calls sayBrightly, 124
 - calls spadUntrace, 124
 - calls unembed, 124
 - calls untraceDomainConstructor, 124
 - uses \$mapSubNameAlist, 125
 - uses \$mathTraceList, 125
 - uses \$timerList, 125
 - uses \$traceNames, 125
 - uses \$traceNoisely, 125
 - defun, 124
- /untrace-reduce, 124
 - called by /untrace-1, 124
 - defun, 124
- ?order
 - called by insertAlist, 981
- ?t, 146
 - called by trace1, 79
 - calls bright, 146
 - calls devaluate, 147
 - calls get, 146
 - calls isDomainOrPackage, 146
 - calls isDomain, 147
 - calls isgenvar, 146
 - calls qcar, 146
 - calls qcdr, 146
 - calls rassocSub, 146
 - calls reportSpadTrace, 147
 - calls sayBrightly, 147
 - calls sayMSG, 146
 - calls take, 147
 - uses \$InteractiveFrame, 147
 - uses \$mapSubNameAlist, 147
 - uses \$traceNames, 147
 - defun, 146
- \$FINDFILE
 - called by \$editSpad2Cmd, 872
- \$erase
 - called by \$addNewInterpreterFrame, 34
 - called by \$closeInterpreterFrame, 37
- \$fcopy
 - called by \$restoreHistory, 907
- \$filep
 - called by \$setOutputAlgebra, 1035
 - called by \$setOutputFormula, 1063

- calledby \$setOutputFortran, 1042
- calledby \$setOutputHtml, 1048
- calledby \$setOutputMathml, 1054
- calledby \$setOutputOpenMath, 1059
- calledby \$setOutputTex, 1070
- \$nagMessages, 1030
 - defvar, 1030
- \$replace
 - calledby \$initHist, 888
- \$reportBottomUpFlag, 1019
 - defvar, 1019
- \$systemCommandFunction
 - calledby \$intloopProcess, 356
 - calledby \$ncloopCommand, 818
- \$AnonymousFunction, 708
 - usedby coerceInt1, 744
 - defvar, 708
- \$Any, 708
 - usedby coerceInt0, 742
 - usedby coerceInt1, 744
 - usedby retract2Specialization, 772
 - defvar, 708
- \$BFtag, 708
 - defvar, 708
- \$BigFloat, 710
 - defvar, 710
- \$Boolean, 709
 - usedby algEqual, 765
 - usedby getSubDomainPredicate, 770
 - defvar, 709
- \$BreakMode, 969
 - usedby letPrint2, 110
 - usedby letPrint3, 112
 - defvar, 969
- \$CallInterp
 - usedby serverReadLine, 337
- \$CatOfCatDatabase
 - usedby clearCmdCompletely, 838
- \$CategoryFrame
 - local def loadLibNoUpdate, 1223
 - local def loadLib, 1222
 - usedby getProplist, 1150
 - usedby reportOpsFromUnitDirectly, 1096
 - usedby systemCommand, 788
- \$Category, 709
 - defvar, 709
- \$CloseClient
 - usedby close, 845
- \$CoerceTable
 - usedby coerceIntTableOrFunction, 759
 - usedby coerceIntTest, 752
- \$Coerce
 - local ref unifyStructVar, 727
 - usedby processInteractive, 342
- \$CommandSynonymAlist, 817
 - usedby npProcessSynonym, 813
 - usedby printSynonyms, 813
 - usedby processSynonyms, 325
 - usedby resetWorkspaceVariables, 963
 - usedby synonymSpad2Cmd, 1105
 - defvar, 817
- \$ComplexInteger, 711
 - local ref hasCateSpecialNew, 734
 - defvar, 711
- \$ConstructorCache
 - usedby clearCmdSortedCaches, 837
 - usedby localDatabase, 1200
 - usedby traceDomainConstructor, 137
- \$CreateFrameAnswer
 - usedby serverReadLine, 337
- \$CreateFrame
 - usedby serverReadLine, 337
- \$DomOfCatDatabase
 - usedby clearCmdCompletely, 838
- \$Domain, 709
 - defvar, 709
- \$DoubleFloat, 710, 715
 - defvar, 710, 715
- \$EchoLines
 - usedby intloopEchoParse, 361
- \$EmptyEnvironment
 - usedby describeSpad2Cmd, 858
 - usedby displaySpad2Cmd, 864
- \$EmptyMode, 708
 - local ref evaluateType1, 1121
 - local ref evaluateType, 1119
 - local ref hasCate, 731
 - local ref isPartialMode, 718
 - local ref mkEvalable, 1116
 - local ref retract, 1269
 - usedby bcComplexLimit, 1478
 - usedby bcDefiniteIntegrate, 1409

- usedby bcDifferentiate, [1406](#)
- usedby bcDraw2DSolve, [1420](#)
- usedby bcDraw2Dfun, [1416](#)
- usedby bcDraw2Dpar, [1418](#)
- usedby bcDraw3Dfun, [1422](#)
- usedby bcDraw3Dpar1, [1427](#)
- usedby bcDraw3Dpar, [1425](#)
- usedby bcIndefiniteIntegrate, [1408](#)
- usedby bcInputEquations, [1481](#)
- usedby bcInputExplicitMatrix, [1443](#)
- usedby bcLaurentSeries, [1435](#)
- usedby bcLimit, [1412](#)
- usedby bcLinearSolveEqns, [1441](#)
- usedby bcLinearSolveMatrix1, [1487](#)
- usedby bcLinearSolveMatrixInhomo, [1488](#)
- usedby bcLinearSolve, [1440](#)
- usedby bcProduct, [1474](#)
- usedby bcPuisseuxSeries, [1437](#)
- usedby bcRealLimit, [1475](#)
- usedby bcSeriesExpansion, [1430](#)
- usedby bcSeries, [1429](#)
- usedby bcSolve, [1439](#)
- usedby bcSum, [1413](#)
- usedby bcTaylorSeries, [1433](#)
- usedby coerceInt1, [744](#)
- usedby coerceInteractive, [740](#)
- usedby displayValue, [799](#)
- usedby interpret2, [348](#)
- usedby recordAndPrint, [350](#)
- defvar, [708](#)
- `$EndOfOutput`
 - usedby serverReadLine, [337](#)
- `$EndServerSession`, [335](#)
 - usedby serverReadLine, [337](#)
 - defvar, [335](#)
- `$EndSession`
 - usedby serverReadLine, [337](#)
- `$Exit`, [709](#)
 - defvar, [709](#)
- `$Expression`, [709](#)
 - defvar, [709](#)
- `$Float`, [710](#)
 - defvar, [710](#)
- `$FontTable`, [711](#)
 - defvar, [711](#)
- `$FormalMapVariableList`, [17](#)
- local ref domArg, [720](#)
- local ref mkDomPvar, [731](#)
- local ref replaceSharps, [1144](#)
- usedby dbConstructorDoc,hn, [1634](#)
- usedby dbGetDocTable,hn, [1637](#)
- usedby dbSubConform, [1640](#)
- usedby displayOperationsFromLisplib, [1095](#)
- usedby localnrLib, [1201](#)
- usedby reportOpsFromLisplib, [1093](#)
- defvar, [17](#)
- `$FunctionalExpression`, [714](#)
 - local ref defaultTargetFE, [736](#)
 - defvar, [714](#)
- `$HTCompanionWindowID`, [344](#)
 - usedby recordAndPrint, [350](#)
 - defvar, [344](#)
- `$HiFiAccess`, [47](#), [1005](#)
 - usedby createCurrentInterpreterFrame, [29](#)
 - usedby disableHist, [913](#)
 - usedby emptyInterpreterFrame, [27](#)
 - usedby historySpad2Cmd, [890](#)
 - usedby initHist, [888](#)
 - usedby putHist, [900](#)
 - usedby restoreHistory, [907](#)
 - usedby saveHistory, [905](#)
 - usedby setHistoryCore, [894](#)
 - usedby undoFromFile, [904](#)
 - usedby undoInCore, [902](#)
 - usedby updateFromCurrentInterpreterFrame, [30](#)
 - usedby updateHist, [899](#)
 - usedby writeInputLines, [897](#)
 - defvar, [47](#), [1005](#)
- `$HistListAct`, [48](#)
 - usedby changeHistListLen, [898](#)
 - usedby createCurrentInterpreterFrame, [29](#)
 - usedby emptyInterpreterFrame, [27](#)
 - usedby initHistList, [889](#)
 - usedby resetInCoreHist, [898](#)
 - usedby updateFromCurrentInterpreterFrame, [30](#)
 - usedby updateInCoreHist, [900](#)
 - defvar, [48](#)
- `$HistListLen`, [48](#)
 - usedby changeHistListLen, [898](#)
 - usedby createCurrentInterpreterFrame, [29](#)

- usedby emptyInterpreterFrame, 27
- usedby initHistList, 889
- usedby resetInCoreHist, 898
- usedby undoInCore, 902
- usedby updateFromCurrentInterpreterFrame, 30
- usedby updateInCoreHist, 900
- defvar, 48
- \$HistList, 47
 - usedby changeHistListLen, 898
 - usedby createCurrentInterpreterFrame, 29
 - usedby emptyInterpreterFrame, 27
 - usedby initHistList, 889
 - usedby recordOldValue0, 902
 - usedby resetInCoreHist, 898
 - usedby undoChanges, 903
 - usedby undoInCore, 902
 - usedby updateFromCurrentInterpreterFrame, 30
 - usedby updateInCoreHist, 900
 - defvar, 47
- \$HistRecord, 48
 - usedby createCurrentInterpreterFrame, 29
 - usedby emptyInterpreterFrame, 27
 - usedby initHistList, 889
 - usedby recordNewValue0, 901
 - usedby updateFromCurrentInterpreterFrame, 30
 - usedby updateHist, 899
 - usedby writeHiFi, 912
 - defvar, 48
- \$IOindex, 39
 - usedby createCurrentInterpreterFrame, 29
 - usedby historySpad2Cmd, 890
 - usedby mkprompt, 334
 - usedby removeUndoLines, 57
 - usedby resetWorkspaceVariables, 963
 - usedby restart, 307
 - usedby setHistoryCore, 894
 - usedby setIOindex, 909
 - usedby undoCount, 61
 - usedby undoInCore, 902
 - usedby undoSteps, 54
 - usedby updateFromCurrentInterpreterFrame, 30
 - usedby updateHist, 899
- usedby writeHiFi, 912
- usedby writeInputLines, 897
- defvar, 39
- \$InitialCommandSynonymAlist, 816
- usedby resetWorkspaceVariables, 963
- defvar, 816
- \$InitialModemapFrame, 198
- usedby makeInitialModemapFrame, 328
- defvar, 198
- \$Integer, 711
 - local ref defaultTargetFE, 736
 - local ref hasCateSpecialNew, 734
 - local ref hasCateSpecial, 733
 - local ref mkEvalable, 1116
 - usedby absolutelyCanCoerceByCheating, 770
 - usedby coerceInt1, 744
 - usedby coerceRetract, 777
 - usedby retract2Specialization, 772
 - defvar, 711
- \$InteractiveFrame, 39
 - local ref isDomainValuedVariable, 1145
 - usedby ?t, 147
 - usedby augmentTraceNames, 78
 - usedby clearCmdAll, 839
 - usedby clearCmdParts, 841
 - usedby createCurrentInterpreterFrame, 29
 - usedby getAliasIfTracedMapParameter, 140
 - usedby getBpiNameIfTracedMap, 140
 - usedby getMapSig, 117
 - usedby getMapSubNames, 131
 - usedby getPreviousMapSubNames, 103
 - usedby getSubDomainPredicate, 770
 - usedby getWorkspaceNames, 795
 - usedby interpFunctionDepAlists, 805
 - usedby isInterpOnlyMap, 105
 - usedby isUncompiledMap, 104
 - usedby ncTopLevel, 316
 - usedby parseAndInterpret, 340
 - usedby processInteractive1, 345
 - usedby recordFrame, 1123
 - usedby reportundo, 60
 - usedby restart, 307
 - usedby restoreHistory, 907
 - usedby showSpad2Cmd, 1087

- usedby undoChanges, 903
- usedby undoFromFile, 904
- usedby undoInCore, 902
- usedby undoSteps, 54
- usedby undo, 52
- usedby updateFromCurrentInterpreterFrame, 30
- usedby writeHistModesAndValues, 913
- defvar, 39
- \$InteractiveMode, 315**
 - local def unparseInputForm, 1307
 - local ref loadLibNoUpdate, 1223
 - local ref loadLib, 1222
 - usedby addBinding, 1149
 - usedby ncTopLevel, 316
 - usedby parseAndInterpret, 340
 - usedby readSpad2Cmd, 943
 - defvar, 315
- \$JoinOfCatDatabase**
 - usedby clearCmdCompletely, 838
- \$JoinOfDomDatabase**
 - usedby clearCmdCompletely, 838
- \$KillLispSystem**
 - usedby serverReadLine, 336
- \$LispCommand**
 - usedby serverReadLine, 337
- \$MenuServer**
 - usedby executeQuietCommand, 339
 - usedby serverReadLine, 337
- \$Mode, 711**
 - defvar, 711
- \$NeedToSignalSessionManager, 336**
 - usedby intloopSpadProcess, 357
 - usedby serverReadLine, 337
 - defvar, 336
- \$NegativeInteger, 711**
 - defvar, 711
- \$NoValueMode**
 - usedby coerceInteractive, 740
- \$NonNegativeInteger, 712**
 - usedby coerceInt1, 744
 - usedby retract2Specialization, 772
 - defvar, 712
- \$NonNullStream, 921**
 - usedby dewritify,dewritifyInner, 922
 - usedby writify,writifyInner, 918
 - defvar, 921
- \$NonPositiveInteger, 712**
 - defvar, 712
- \$NonSmanSession**
 - usedby serverReadLine, 336
- \$NullStream, 922**
 - usedby dewritify,dewritifyInner, 922
 - usedby writify,writifyInner, 918
 - defvar, 922
- \$OutputForm, 73, 710**
 - usedby coerceByTable, 760
 - usedby coerceInt0, 742
 - usedby coerceInt1, 744
 - usedby coerceInteractive, 740
 - usedby coerceRetract, 777
 - usedby coerceSpadArgs2E, 129
 - usedby coerceSpadFunValue2E, 130
 - usedby coerceTraceArgs2E, 127
 - usedby coerceTraceFunValue2E, 130
 - defvar, 73, 710
- \$PatternVariableList, 17**
 - usedby kDomainName, 1624
 - defvar, 17
- \$PositiveInteger, 712**
 - usedby coerceInt1, 744
 - usedby retract2Specialization, 772
 - defvar, 712
- \$Primitives**
 - usedby conOpPage1, 1629
 - usedby dbShowConsDoc1, 1646
- \$PrintCompilerMessageIfTrue, 310**
 - usedby spad, 310
 - defvar, 310
- \$ProcessInteractiveValue, 344**
 - usedby kisValidType, 1626
 - usedby processInteractive1, 345
 - usedby processInteractive, 343
 - usedby topLevelInterpEval, 1626
 - defvar, 344
- \$PsiAsymptoticBern**
 - usedby PsiAsymptotic, 1260
- \$QueryClients**
 - usedby queryClients, 845
- \$QuickLet, 73**
 - usedby breaklet, 155
 - usedby tracelet, 154

- defvar, 73
- \$QuietCommand, 339
 - usedby executeQuietCommand, 339
 - usedby pf2Sex1, 600
 - usedby pf2Sex, 597
 - usedby recordAndPrint, 350
 - defvar, 339
- \$QuietSpadCommand
 - usedby serverReadLine, 336
- \$QuotientField, 713
 - local ref hasCateSpecial, 733
 - usedby coerceInt1, 744
 - usedby retract2Specialization, 772
 - defvar, 713
- \$RTspecialCharacters, 1164
 - usedby setOutputCharacters, 1039
 - defvar, 1164
- \$RationalNumber, 712
 - local ref defaultTargetFE, 736
 - local ref hasCateSpecialNew, 734
 - defvar, 712
- \$SessionManager
 - usedby close, 845
 - usedby queryClients, 845
 - usedby serverReadLine, 337
- \$SingleFloat, 714
 - defvar, 714
- \$SingleInteger, 715
 - usedby absolutelyCanCoerceByCheating, 770
 - usedby coerceInt1, 744
 - usedby coerceRetract, 777
 - defvar, 715
- \$SmallInteger, 714
 - defvar, 714
- \$SpadCommand
 - usedby serverReadLine, 336
- \$SpadServerName, 202
 - usedby restart, 307
 - defvar, 202
- \$SpadServer, 201
 - usedby close, 845
 - usedby restart, 307
 - usedby serverReadLine, 337
 - usedby spad-save, 1176
 - defvar, 201
- \$StreamFrame
 - usedby processInteractive, 342
- \$StringCategory, 713
 - defvar, 713
- \$String, 713
 - usedby coerceInt1, 744
 - defvar, 713
- \$Subst
 - local def ofCategory, 717
 - local ref hasCate, 732
 - local ref unifyStructVar, 727
- \$SwitchFrames
 - usedby serverReadLine, 337
- \$Symbol, 713
 - local ref defaultTargetFE, 736
 - usedby coerceInt1, 744
 - usedby coerceRetract, 777
 - usedby retract2Specialization, 772
 - defvar, 713
- \$ThrowAwayMode, 347
 - usedby interpret2, 348
 - defvar, 347
- \$TraceFlag, 75
 - usedby monitorEnter, 148
 - usedby monitorExit, 151
 - defvar, 75
- \$TriangleVariableList
 - local ref replaceSharps, 1144
 - usedby dbShowConsDoc1, 1646
 - usedby libConstructorSig, 1659
- \$UserAbbreviationsAlist
 - usedby resetWorkspaceVariables, 963
- \$UserLevel, 1080
 - usedby getDirectoryList, 1170
 - usedby readSpad2Cmd, 943
 - usedby satisfiesUserLevel, 791
 - usedby set1, 1083
 - usedby synonymsForUserLevel, 1106
 - usedby userLevelErrorMessage, 791
 - usedby whatCommands, 1134
 - defvar, 1080
- \$Void, 713
 - usedby clearCmdSortedCaches, 837
 - usedby coerceInt1, 744
 - usedby recordAndPrint, 350
 - defvar, 713

- \$abbreviateTypes, 1033
 - defvar, 1033
- \$activePageList, 1467
 - usedby httpDestroyPage, 1468
 - usedby httpMakeEmptyPage, 1467
 - defvar, 1467
- \$algebraFormat, 1034
 - usedby setOutputAlgebra, 1035
 - defvar, 1034
- \$algebraOutputFile, 1034
 - usedby setOutputAlgebra, 1035
 - defvar, 1034
- \$algebraOutputStream, 1035
 - usedby recordAndPrint, 350
 - usedby sayMSG, 45
 - usedby setOutputAlgebra, 1035
 - defvar, 1035
- \$analyzingMapList
 - usedby processInteractive, 342
- \$ans
 - usedby nplisp, 812
- \$args
 - usedby dbConstructorDoc,hn, 1634
- \$atLeastOneUnexposed
 - usedby dbConsExposureMessage, 1645
 - usedby htInitPageNoScroll, 1511
 - usedby kPage, 1593
- \$attrCats, 654
 - usedby whichCat, 655
 - defvar, 654
- \$attributeDb
 - usedby clearCmdCompletely, 838
- \$bcParseOnly, 1498
 - usedby bcInputEquations, 1481
 - usedby bcInputExplicitMatrix, 1443
 - usedby bcInputMatrixByFormula, 1445
 - defvar, 1498
- \$boot, 315
 - usedby ioclear, 1158
 - usedby iostat, 1156
 - usedby ncTopLevel, 316
 - usedby parseAndInterpret, 340
 - usedby resetWorkspaceVariables, 962
 - defvar, 315
- \$breakCondition, 68
 - usedby monitorXX, 90
- defvar, 68
- \$cacheAList, 979
 - usedby countCache, 980
 - usedby sayAllCacheCounts, 982
 - usedby setFunctionsCache, 980
 - defvar, 979
- \$cacheCount
 - usedby countCache, 981
 - usedby sayAllCacheCounts, 982
 - usedby setFunctionsCache, 980
- \$clearExcept, 835
 - usedby clearSpad2Cmd, 835
 - defvar, 835
- \$clearOptions, 834
 - usedby clearCmdExcept, 840
 - usedby clearCmdParts, 841
 - usedby clearSpad2Cmd, 835
 - defvar, 834
- \$coerceConvertMmSelection;AL
 - usedby coerceConvertMmSelection, 753
- \$coerceFailure
 - usedby catchCoerceFailure, 760
 - usedby coerceByTable, 760
 - usedby coerceIntCommute, 757
 - usedby coerceRetract, 777
- \$coerceIntByMapCounter
 - usedby resetWorkspaceVariables, 963
- \$collectOutput
 - usedby printStatisticsSummary, 351
 - usedby printStorage, 351
 - usedby printTypeAndTime, 352
 - usedby recordAndPrint, 350
- \$commentedOps
 - usedby reportOpsFromUnitDirectly, 1096
- \$compErrorMessageStack
 - usedby processInteractive, 342
- \$compileMapFlag
 - usedby resetWorkspaceVariables, 962
- \$compileRecurrence, 985
 - defvar, 985
- \$compilingLoop
 - usedby processInteractive, 342
- \$compilingMap, 341
 - usedby coerceInteractive, 740
 - usedby processInteractive, 342
 - defvar, 341

- \$conArgstrings
 - usedby conPage, 1599
- \$conformrsAreDomains
 - usedby kePage, 1606
- \$conformsAreDomains
 - usedby dbShowCons1, 1643
 - usedby dbSpecialDescription, 1651
 - usedby kPage, 1593
 - usedby kiPage, 1605
- \$conform
 - usedby conPageConEntry, 1600
 - usedby dbGetDocTable,gn, 1638
 - usedby dbGetDocTable,hn, 1637
 - usedby dbGetDocTable, 1639
- \$conname
 - usedby conPageConEntry, 1600
- \$constructorList
 - usedby make-databases, 1205
- \$constructors, 68
 - usedby addTraceItem, 146
 - usedby traceReply, 95
 - defvar, 68
- \$countList, 68
 - usedby pcounters, 99
 - usedby resetCounters, 98
 - usedby resetWorkspaceVariables, 962
 - usedby trace3, 84
 - defvar, 68
- \$curPage
 - usedby bcHt, 1508
 - usedby htInitPageNoScroll, 1511
 - usedby htInitPage, 1510
- \$curpage, 1498
 - defvar, 1498
- \$current-directory, 197, 333
 - usedby getDirectoryList, 1170
 - usedby reroot, 333
 - usedby restart, 307
 - defvar, 197, 333
- \$current-line
 - usedby iostat, 1156
 - usedby spad-short-error, 1156
- \$currentCarrier
 - usedby intloopSpadProcess, 357
- \$currentFrameNum, 335
 - usedby close, 845
- usedby serverReadLine, 337
 - defvar, 335
- \$currentLine
 - usedby ExecuteInterpSystemCommand, 324
 - usedby clearCmdAll, 839
 - usedby getSystemCommandLine, 1106
 - usedby intnplisp, 328
 - usedby removeUndoLines, 57
 - usedby restart, 307
 - usedby setCurrentLine, 334
 - usedby unAbbreviateKeyword, 809
 - usedby updateHist, 899
 - usedby writeHiFi, 912
- \$dalymode
 - usedby intloopReadConsole, 321
- \$dbDelimiters
 - usedby dbWordFrom, 1592
- \$declaredMode
 - usedby coerceConvertMmSelection, 753
 - usedby processInteractive, 342
- \$defaultFortVar
 - usedby processInteractive, 343
- \$defaultFortranType, 989
 - defvar, 989
- \$defaultFunctionTargets, 714
 - defvar, 714
- \$defaultPackageNamesHT
 - usedby kcPage, 1612
- \$defaultSpecialCharacters, 1161
 - defvar, 1161
- \$depTb
 - local ref saveDependentsHashTable, 1208
- \$dependeeAlist
 - usedby displayProperties,sayFunctionDeps, 798
 - usedby displayProperties, 801
 - usedby interpFunctionDepAlists, 805
- \$dependeeClosureAlist
 - usedby resetWorkspaceVariables, 963
- \$dependentAlist
 - usedby displayProperties,sayFunctionDeps, 798
 - usedby displayProperties, 801
 - usedby interpFunctionDepAlists, 805
- \$depthAlist, 69

- usedby monitorXX, 90
 - usedby monitorX, 90
 - defvar, 69
- \$describeOptions, 857
 - usedby describeSpad2Cmd, 858
 - defvar, 857
- \$directory-list, 198
 - usedby getDirectoryList, 1170
 - usedby reroot, 333
 - defvar, 198
- \$displayDroppedMap, 1011
 - defvar, 1011
- \$displayMsgNumber, 1018
 - usedby sayKeyedMsgLocal, 44
 - defvar, 1018
- \$displayOptions, 863
 - usedby displaySpad2Cmd, 864
 - defvar, 863
- \$displaySetValue, 1020
 - usedby set1, 1083
 - defvar, 1020
- \$displayStartMsgs, 1021
 - usedby restart, 307
 - defvar, 1021
- \$doNotAddEmptyModeIfTrue, 69
 - usedby domainToGenvar, 101
 - usedby reportOperations, 1090
 - usedby transTraceItem, 126
 - defvar, 69
- \$docTableHash
 - usedby dbDocTable, 1635
- \$docTable
 - usedby dbAddDocTable, 1637
 - usedby dbDocTable, 1635
- \$doc
 - usedby conPageConEntry, 1600
- \$dollar
 - usedby retractByFunction, 778
- \$domPvar, 341
 - local def hasAtt, 722
 - local def hasCatel, 725
 - local def hasCaty1, 730
 - local def hasSig, 721
 - local ref domArg2, 720
 - local ref hasCaty, 719
 - local ref unifyStructVar, 727
 - usedby processInteractive, 343
 - defvar, 341
- \$domainTraceNameAssoc, 69
 - usedby genDomainTraceName, 122
 - defvar, 69
- \$domains, 69
 - usedby addTraceItem, 146
 - usedby traceReply, 95
 - defvar, 69
- \$domain
 - usedby dbSearchOrder, 1610
 - usedby kTestPred, 1639
- \$echoLineStack
 - usedby resetWorkspaceVariables, 963
- \$embeddedFunctions, 69
 - defvar, 69
- \$envHashTable, 1149
 - usedby addBinding, 1149
 - defvar, 1149
- \$env, 315
 - local ref isDomainValuedVariable, 1145
 - usedby getSubDomainPredicate, 769
 - usedby interpret, 346
 - usedby reportOperations, 1090
 - usedby resetWorkspaceVariables, 963
 - usedby showSpad2Cmd, 1087
 - defvar, 315
- \$erMsgToss
 - usedby SpadInterpretStream, 320
 - usedby initImPr, 657
 - usedby phIntReportMsgs, 359
 - usedby showMsgPos?, 648
- \$erase
 - local ref saveDependentsHashTable, 1208
 - local ref saveUsersHashTable, 1209
 - usedby deleteFile, 1233
 - usedby reportOpsFromLisplib1, 1092
 - usedby reportOpsFromUnitDirectly1, 1100
- \$evalDomain
 - local ref evalDomain, 1115
- \$evalTimePrint
 - usedby showHistory, 893
- \$eval
 - usedby interpret1, 347
 - usedby interpret, 346
 - usedby reportOperations, 1090

- \$existingFiles
 - usedby clearCmdCompletely, 838
 - usedby resetWorkspaceVariables, 962
- \$expandSegments
 - local def evaluateType, 1119
- \$exposed?
 - usedby conPageConEntry, 1600
- \$exposedOnlyIfTrue
 - usedby dbConsHeading, 1649
 - usedby dbShowCons1, 1643
 - usedby dbShowCons, 1641
- \$e, 315
 - local ref isDomainValuedVariable, 1145
 - usedby clearCmdParts, 841
 - usedby coerceInt1, 744
 - usedby describeSpad2Cmd, 858
 - usedby displaySpad2Cmd, 864
 - usedby interpFunctionDepAlists, 805
 - usedby ncTopLevel, 316
 - usedby parseAndInterpret, 340
 - usedby processInteractive1, 345
 - usedby recordAndPrint, 349
 - usedby resetWorkspaceVariables, 963
 - usedby restoreHistory, 907
 - usedby retract2Specialization, 772
 - usedby showSpad2Cmd, 1087
 - usedby systemCommand, 788
 - usedby whatSpad2Cmd, 1131
 - defvar, 315
- \$filep
 - usedby setOutputAlgebra, 1035
 - usedby setOutputFormula, 1064
 - usedby setOutputFortran, 1042
 - usedby setOutputHtml, 1048
 - usedby setOutputMathml, 1054
 - usedby setOutputOpenMath, 1059
 - usedby setOutputTex, 1070
- \$findfile
 - usedby readSpad2Cmd, 943
- \$floatok
 - usedby lineoftoks, 403
 - usedby scanKeyTr, 411
 - usedby scanNumber, 423
 - usedby scanSpace, 420
 - usedby scanString, 421
 - usedby scanWord, 417
- \$fn
 - usedby SpadInterpretStream, 320
 - usedby toFile?, 653
- \$forceDatabaseUpdate
 - local ref loadLib, 1222
 - local ref updateDatabase, 1203
 - usedby localdatabase, 1200
- \$formatSigAsTex
 - local def unparseInputForm, 1307
- \$formulaFormat, 1062
 - usedby setOutputFormula, 1064
 - defvar, 1062
- \$formulaOutputFile, 1062
 - usedby setOutputFormula, 1064
 - defvar, 1062
- \$formulaOutputStream
 - usedby setOutputFormula, 1063
- \$fortIndent, 987
 - defvar, 987
- \$fortInts2Floats, 986
 - defvar, 986
- \$fortLength, 988
 - defvar, 988
- \$fortPersistence, 1028
 - usedby describeFortPersistence, 1029
 - usedby setFortPers, 1029
 - defvar, 1028
- \$fortVar
 - usedby processInteractive, 342
- \$fortranArrayStartingIndex, 994
 - defvar, 994
- \$fortranDirectory, 998
 - usedby describeSetFortDir, 999
 - usedby setFortDir, 999
 - defvar, 998
- \$fortranFormat, 1041
 - usedby setOutputFortran, 1042
 - defvar, 1041
- \$fortranLibraries, 1000
 - usedby describeSetLinkerArgs, 1001
 - usedby setLinkerArgs, 1001
 - defvar, 1000
- \$fortranOptimizationLevel, 993
 - defvar, 993
- \$fortranOutputFile, 1041
 - usedby setOutputFortran, 1042

- defvar, 1041
- \$fortranOutputStream
 - usedby setOutputFortran, 1042
- \$fortranPrecision, 990
 - defvar, 990
- \$fortranSegment, 993
 - defvar, 993
- \$fortranTmpDir, 995
 - usedby describeSetFortTmpDir, 997
 - usedby setFortTmpDir, 996
 - defvar, 995
- \$fractionDisplayType, 1046
 - defvar, 1046
- \$frameAlist, 335
 - usedby serverReadLine, 337
 - defvar, 335
- \$frameMessages, 1013
 - usedby clearCmdAll, 839
 - usedby displayProperties, 801
 - usedby updateFromCurrentInterpreterFrame, 30
 - defvar, 1013
- \$frameNumber, 335
 - usedby serverReadLine, 337
 - defvar, 335
- \$frameRecord, 51
 - usedby clearCmdAll, 839
 - usedby clearFrame, 1128
 - usedby recordFrame, 1123
 - usedby undoSteps, 54
 - defvar, 51
- \$freeVars
 - usedby processInteractive, 342
- \$fromSpadTrace, 70
 - usedby spadTrace, 133
 - usedby trace3, 84
 - defvar, 70
- \$fullScreenSysVars, 1002
 - defvar, 1002
- \$functionTable, 838
 - usedby clearCmdCompletely, 838
 - usedby resetWorkspaceVariables, 962
 - defvar, 838
- \$funnyBacks, 1545
 - usedby unescapeStringsInForm, 354
 - defvar, 1545
- \$funnyQuote, 1544
 - usedby unescapeStringsInForm, 354
 - defvar, 1544
- \$f
 - usedby lineoftoks, 403
 - usedby nextline, 404
- \$genValue, 346
 - usedby coerceInt0, 742
 - usedby coerceInt1, 744
 - usedby interpret1, 347
 - usedby interpret, 346
 - usedby reportOperations, 1090
 - defvar, 346
- \$giveExposureWarning, 1011
 - defvar, 1011
- \$globalExposureGroupAlist, 167
 - local ref isExposedConstructor, 1094
 - usedby setExposeAddGroup, 157
 - usedby setExposeDropGroup, 163
 - defvar, 167
- \$hashOp0
 - local ref basicLookup, 1226
- \$hashOp1
 - local ref basicLookup, 1226
- \$hashOpApply
 - local ref basicLookup, 1226
- \$hashOpSet
 - local ref basicLookup, 1226
- \$hashSeg
 - local ref basicLookup, 1226
- \$highlightAllowed, 1014
 - defvar, 1014
- \$historyDirectory, 887
 - usedby histInputFileName, 888
 - defvar, 887
- \$historyDisplayWidth, 1003, 1570
 - defvar, 1003, 1570
- \$historyFileType, 49, 886
 - defvar, 49, 886
- \$hope
 - local def hasCate, 732
 - local def ofCategory, 717
 - local def unifyStructVar, 727
- \$htLineList, 1498
 - usedby bcHt, 1508
 - usedby htInitPageNoScroll, 1511

- usedby iht, 1508
 - defvar, 1498
- \$htMacroTable, 1399
 - usedby buildHtMacroTable, 1401
 - defvar, 1399
- \$htmlFormat, 1047
 - usedby setOutputHtml, 1048
 - defvar, 1047
- \$htmlOutputFile, 1047
 - usedby setOutputHtml, 1048
 - defvar, 1047
- \$htmlOutputStream
 - usedby setOutputHtml, 1048
- \$imPrGuys, 648
 - defvar, 648
- \$imPrTagGuys, 657
 - usedby initImPr, 657
 - defvar, 657
- \$inRetract
 - usedby processInteractive, 343
- \$inclAssertions
 - usedby SpadInterpretStream, 320
 - usedby assertCond, 389
 - usedby ifCond, 382
- \$includeUnexposed?
 - usedby getBrowseDatabase, 1310
- \$infovec
 - usedby dbAddChainDomain, 1640
 - usedby dbSearchOrder, 1610
- \$inputPromptType, 1019
 - usedby mkprompt, 334
 - defvar, 1019
- \$inputStream
 - usedby npFirstTok, 435
 - usedby npListAndRecover, 482
 - usedby npMoveTo, 483
 - usedby npNext, 438
 - usedby npParse, 433
 - usedby npRestore, 444
 - usedby npState, 504
- \$insideApplication
 - usedby pf2Sex, 597
 - usedby pfApplication2Sex, 604
 - usedby pfDefinition2Sex, 613
- \$insideCoerceInteractive
 - usedby coerceInteractive, 740
- \$insideRule
 - usedby pf2Sex1, 600
 - usedby pf2Sex, 597
 - usedby pfApplication2Sex, 604
 - usedby pfLhsRule2Sex, 617
 - usedby pfLiteral2Sex, 602
 - usedby pfOp2Sex, 606
 - usedby pfRhsRule2Sex, 617
- \$insideSEQ
 - usedby pf2Sex1, 600
 - usedby pf2Sex, 597
 - usedby pfSequence2Sex, 608
- \$instantCanCoerceCount
 - usedby processInteractive, 342
- \$instantCoerceCount
 - usedby processInteractive, 342
- \$instantMmCondCount
 - usedby processInteractive, 342
- \$instantRecord, 342
 - usedby processInteractive, 343
 - defvar, 342
- \$intCoerceFailure, 323
 - usedby InterpExecuteSpadSystemCommand, 323
 - usedby intloopSpadProcess, 357
 - defvar, 323
- \$intRestart, 317
 - usedby intloop, 318
 - defvar, 317
- \$intSpadReader, 323
 - usedby InterpExecuteSpadSystemCommand, 323
 - usedby intloopSpadProcess, 357
 - defvar, 323
- \$intTopLevel, 317
 - usedby intloop, 317
 - defvar, 317
- \$internalHistoryTable, 48
 - usedby clearCmdAll, 839
 - usedby createCurrentInterpreterFrame, 29
 - usedby readHiFi, 911
 - usedby restoreHistory, 907
 - usedby saveHistory, 905
 - usedby setHistoryCore, 894
 - usedby updateFromCurrentInterpreterFrame, 30

- usedby writeHiFi, [912](#)
 - defvar, [48](#)
- \$interpOnly, [341](#)
 - usedby processInteractive, [342](#)
 - defvar, [341](#)
- \$interpreterFrameName, [38](#)
 - usedby clearCmdAll, [839](#)
 - usedby closeInterpreterFrame, [37](#)
 - usedby createCurrentInterpreterFrame, [29](#)
 - usedby displayExposedGroups, [165](#)
 - usedby displayProperties, [801](#)
 - usedby histFileName, [888](#)
 - usedby histInputFileName, [888](#)
 - usedby initializeInterpreterFrameRing, [27](#)
 - usedby mkprompt, [334](#)
 - usedby setExposeAddConstr, [161](#)
 - usedby setExposeAddGroup, [157](#)
 - usedby setExposeDropConstr, [164](#)
 - usedby setExposeDropGroup, [163](#)
 - usedby updateFromCurrentInterpreterFrame, [30](#)
 - defvar, [38](#)
- \$interpreterFrameRing, [38](#)
 - usedby addNewInterpreterFrame, [34](#)
 - usedby changeToNamedInterpreterFrame, [32](#)
 - usedby closeInterpreterFrame, [37](#)
 - usedby displayFrameNames, [28](#)
 - usedby findFrameInRing, [32](#)
 - usedby frameNames, [28](#)
 - usedby importFromFrame, [35](#)
 - usedby initializeInterpreterFrameRing, [27](#)
 - usedby nextInterpreterFrame, [33](#)
 - usedby previousInterpreterFrame, [33](#)
 - usedby updateCurrentInterpreterFrame, [31](#)
 - usedby updateFromCurrentInterpreterFrame, [30](#)
 - defvar, [38](#)
- \$interpreterTimedClasses
 - usedby printStorage, [351](#)
 - usedby printTypeAndTime, [352](#)
 - usedby processInteractive, [343](#)
- \$interpreterTimedNames
 - usedby printStorage, [351](#)
 - usedby printTypeAndTime, [352](#)
- usedby processInteractive, [343](#)
- \$kind
 - usedby conPageConEntry, [1600](#)
- \$lambdtype, [970](#)
 - defvar, [970](#)
- \$lastLineInSEQ
 - usedby processInteractive, [342](#)
- \$lastPos
 - usedby SpadInterpretStream, [320](#)
 - usedby getPosStL, [646](#)
- \$lastUntraced, [70](#)
 - usedby getMapSubNames, [131](#)
 - usedby trace1, [80](#)
 - usedby untraceMapSubNames, [105](#)
 - usedby untrace, [123](#)
 - defvar, [70](#)
- \$letAssoc, [70](#)
 - usedby breaklet, [155](#)
 - usedby letPrint2, [110](#)
 - usedby letPrint3, [112](#)
 - usedby letPrint, [108](#)
 - usedby spadTrace, [133](#)
 - usedby spadUntrace, [143](#)
 - usedby tracelet, [154](#)
 - defvar, [70](#)
- \$libQuiet
 - usedby SpadInterpretStream, [320](#)
- \$library-directory-list, [199](#)
 - usedby getDirectoryList, [1170](#)
 - usedby reroot, [333](#)
 - defvar, [199](#)
- \$linearFormatScripts, [1067](#)
 - defvar, [1067](#)
- \$linelength, [1051](#)
 - usedby displayOperationsFromLisplib, [1095](#)
 - usedby displaySetOptionInformation, [964](#)
 - usedby displaySetVariableSettings, [965](#)
 - usedby filterAndFormatConstructors, [1136](#)
 - usedby justifyMyType, [354](#)
 - usedby msgOutputter, [644](#)
 - usedby msgText, [354](#)
 - usedby printSynonyms, [814](#)
 - usedby reportOpsFromLisplib, [1093](#)
 - usedby reportOpsFromUnitDirectly, [1096](#)
 - usedby sayKeyedMsgLocal, [44](#)
 - usedby setExposeAddConstr, [161](#)

- usedby setExposeAddGroup, 157
- usedby setExposeAdd, 160
- usedby setExposeDropConstr, 164
- usedby setExposeDropGroup, 163
- usedby setExposeDrop, 162
- usedby spadStartUpMsgs, 309
- usedby traceReply, 95
- usedby whatCommands, 1134
- usedby workfilesSpad2Cmd, 1139
- defvar, 1051
- \$linepos
 - usedby lineoftoks, 403
 - usedby nextline, 404
 - usedby scanCheckRadix, 425
 - usedby scanError, 426
 - usedby scanS, 422
 - usedby scanToken, 406
- \$lines
 - usedby intloopEchoParse, 361
 - usedby intloopInclude0, 355
 - usedby ncloopInclude0, 365
- \$ln
 - usedby lineoftoks, 403
 - usedby nextline, 404
 - usedby scanComment, 407
 - usedby scanError, 426
 - usedby scanEsc, 415
 - usedby scanExponent, 418
 - usedby scanNegComment, 408
 - usedby scanNumber, 423
 - usedby scanPossFloat, 412
 - usedby scanPunct, 409
 - usedby scanSpace, 420
 - usedby scanS, 422
 - usedby scanToken, 406
 - usedby scanW, 419
 - usedby spleI1, 414
 - usedby startsComment?, 407
 - usedby startsNegComment?, 408
- \$localExposureDataDefault, 167
 - usedby clearCmdCompletely, 838
 - usedby emptyInterpreterFrame, 27
 - defvar, 167
- \$localExposureData, 166
 - local ref isExposedConstructor, 1094
 - usedby clearCmdCompletely, 838
 - usedby createCurrentInterpreterFrame, 29
 - usedby displayExposedConstructors, 165
 - usedby displayExposedGroups, 165
 - usedby displayHiddenConstructors, 166
 - usedby setExposeAddConstr, 161
 - usedby setExposeAddGroup, 157
 - usedby setExposeDropConstr, 164
 - usedby setExposeDropGroup, 163
 - usedby updateFromCurrentInterpreterFrame, 30
 - defvar, 166
- \$localVars
 - usedby processInteractive, 342
- \$lookupDefaults
 - local def oldCompLookup, 1229
 - local ref basicLookupCheckDefaults, 1228
 - usedby clearCmdSortedCaches, 837
- \$lowerCaseConTb
 - usedby conLowerCaseConTran, 1590
 - usedby conPageFastPath, 1600
 - usedby constructorSearch, 1597
 - usedby string2Constructor, 1590
- \$macActive
 - usedby mac0ExpandBody, 521
 - usedby mac0MLambdaApply, 520
 - usedby macId, 523
 - usedby macroExpanded, 518
- \$mapList
 - usedby processInteractive, 342
- \$mapName
 - usedby coerceInteractive, 740
- \$mapSubNameAlist, 70
 - usedby /untrace-2, 125
 - usedby ?t, 147
 - usedby isSubForRedundantMapName, 105
 - usedby monitorXX, 90
 - usedby saveMapSig, 117
 - usedby trace3, 84
 - usedby traceSpad2Cmd, 78
 - usedby untraceMapSubNames, 105
 - usedby untrace, 123
 - defvar, 70
- \$margin, 1051
 - usedby msgText, 354
 - usedby sayKeyedMsgLocal, 44
 - defvar, 1051

- `$mathTraceList`, 71
 - usedby `/untrace-2`, 125
 - usedby `coerceTraceArgs2E`, 127
 - usedby `coerceTraceFunValue2E`, 130
 - usedby `trace3`, 84
 - defvar, 71
- `$mathTrace`, 71
 - usedby `monitorEnter`, 148
 - usedby `monitorExit`, 151
 - usedby `monitorPrintArgs`, 148
 - usedby `monitorPrintRest`, 150
 - usedby `monitorPrintValue`, 152
 - usedby `monitorXX`, 90
 - usedby `prinmathor0`, 151
 - defvar, 71
- `$mathmlFormat`, 1053
 - usedby `setOutputMathml`, 1054
 - defvar, 1053
- `$mathmlOutputFile`, 1053
 - usedby `setOutputMathml`, 1054
 - defvar, 1053
- `$mathmlOutputStream`
 - usedby `setOutputMathml`, 1054
- `$maximumFortranExpressionLength`, 992
 - defvar, 992
- `$minivectorCode`
 - usedby `processInteractive`, 342
- `$minivectorNames`, 341
 - usedby `processInteractive`, 342
 - defvar, 341
- `$minivector`
 - usedby `processInteractive`, 342
- `$mkTestFlag`, 349
 - usedby `recordAndPrint`, 350
 - defvar, 349
- `$mkTestInputStack`
 - usedby `updateHist`, 899
- `$mkTestOutputType`
 - usedby `recordAndPrint`, 349
- `$monitorArgs`, 71
 - usedby `monitorGetValue`, 94
 - usedby `monitorXX`, 90
 - defvar, 71
- `$monitorCaller`, 71
 - usedby `monitorEnter`, 148
 - usedby `monitorGetValue`, 94
 - usedby `monitorXX`, 90
 - defvar, 71
- `$monitorDepth`, 71
 - usedby `monitorEnter`, 148
 - usedby `monitorPrintRest`, 150
 - usedby `monitorXX`, 90
 - usedby `monitorX`, 90
 - usedby `prinmathor0`, 151
 - defvar, 71
- `$monitorFunDepth`, 72
 - usedby `monitorEnter`, 148
 - usedby `monitorExit`, 151
 - usedby `monitorXX`, 90
 - defvar, 72
- `$monitorName`, 72
 - usedby `monitorGetValue`, 94
 - usedby `monitorXX`, 90
 - defvar, 72
- `$monitorPretty`, 72
 - usedby `monitorPrintRest`, 150
 - usedby `monitorPrintValue`, 152
 - usedby `monitorPrint`, 150
 - defvar, 72
- `$monitorValue`, 72
 - usedby `monitorExit`, 151
 - usedby `monitorGetValue`, 94
 - usedby `monitorXX`, 90
 - defvar, 72
- `$msgAlist`, 42
 - usedby `resetWorkspaceVariables`, 963
 - usedby `spadStartUpMsgs`, 309
 - defvar, 42
- `$msgDatabaseName`, 199
 - usedby `getMsgInfoFromKey`, 656
 - usedby `resetWorkspaceVariables`, 963
 - defvar, 199
- `$msgDatabase`
 - usedby `resetWorkspaceVariables`, 963
- `$msgdbNoBlanksAfterGroup`, 43
 - defvar, 43
- `$msgdbNoBlanksBeforeGroup`, 43
 - defvar, 43
- `$msgdbPrims`, 42
 - usedby `fixObjectForPrinting`, 796
 - defvar, 42
- `$msgdbPunct`, 43

- defvar, 43
- \$multiVarPredicateList
 - usedby pfRule2Sex, 616
 - usedby ruleLhsTran, 620
 - usedby rulePredicateTran, 618
- \$nagEnforceDouble, 1031
 - defvar, 1031
- \$nagHost, 1026
 - usedby describeSetNagHost, 1027
 - usedby setNagHost, 1027
 - defvar, 1026
- \$nagMessages, 1017
 - defvar, 1017
- \$ncMsgList, 318
 - usedby SpadInterpretStream, 319
 - usedby intloopSpadProcess, 357
 - usedby ncConversationPhase, wrapup, 361
 - usedby ncConversationPhase, 360
 - usedby processKeyedError, 643
 - defvar, 318
- \$newConlist
 - usedby library, 1199
- \$newPage, 1401
 - usedby bcHt, 1508
 - usedby htInitPageNoScroll, 1511
 - usedby iht, 1508
 - defvar, 1401
- \$newcompErrorCount, 319
 - usedby SpadInterpretStream, 320
 - usedby ncBug, 658
 - usedby ncHardError, 642
 - usedby ncSoftError, 642
 - defvar, 319
- \$newline, 1570
 - defvar, 1570
- \$newspad, 316
 - usedby ncTopLevel, 316
 - defvar, 316
- \$noEvalTypeMsg, 1122
 - local ref throwEvalTypeMsg, 1122
 - usedby kisValidType, 1626
 - usedby topLevelInterpEval, 1626
 - defvar, 1122
- \$noParseCommands, 784
 - usedby doSystemCommand, 786
 - defvar, 784
- \$noRepList
 - usedby processMsgList, 659
 - usedby redundant, 664
- \$noSubsumption, 1148
 - defvar, 1148
- \$nopos, 319
 - usedby SpadInterpretStream, 320
 - usedby makeLeaderMsg, 667
 - usedby ncBug, 658
 - usedby pfSourcePosition, 536
 - usedby poNoPosition, 704
 - defvar, 319
- \$npPParg
 - usedby npPPff, 503
- \$npTokToNames, 497
 - usedby npId, 497
 - defvar, 497
- \$numberOfEquations, 1481
 - defvar, 1481
- \$numericFailure, 1311
 - defvar, 1311
- \$n
 - usedby lineoftoks, 403
 - usedby nextline, 404
 - usedby scanCheckRadix, 425
 - usedby scanComment, 407
 - usedby scanError, 426
 - usedby scanEscape, 426
 - usedby scanEsc, 415
 - usedby scanExponent, 418
 - usedby scanNegComment, 408
 - usedby scanNumber, 423
 - usedby scanPossFloat, 412
 - usedby scanPunct, 409
 - usedby scanSpace, 420
 - usedby scanString, 421
 - usedby scanS, 422
 - usedby scanToken, 406
 - usedby scanW, 419
 - usedby spleI1, 414
 - usedby startsComment?, 407
 - usedby startsNegComment?, 408
- \$okToExecuteMachineCode
 - usedby SpadInterpretStream, 320
- \$oldBreakMode, 1311
 - defvar, 1311

- \$oldHistoryFileName, 886
 - usedby oldHistFileName, 887
 - usedby restoreHistory, 907
 - defvar, 886
- \$oldline, 789
 - usedby commandErrorIfAmbiguous, 810
 - usedby commandErrorMessage, 790
 - defvar, 789
- \$opSysName
 - usedby spadStartUpMsgs, 309
- \$openMathFormat, 1057
 - usedby setOutputOpenMath, 1059
 - defvar, 1057
- \$openMathOutputFile, 1058
 - usedby setOutputOpenMath, 1059
 - defvar, 1058
- \$openMathOutputStream
 - usedby setOutputOpenMath, 1059
- \$openServerIfTrue, 199
 - usedby restart, 307
 - usedby spad-save, 1176
 - defvar, 199
- \$operationNameList
 - usedby resetWorkspaceVariables, 963
- \$optionAlist, 73
 - usedby trace1, 80
 - defvar, 73
- \$options, 73
 - usedby abbreviationsSpad2Cmd, 823
 - usedby clearSpad2Cmd, 835
 - usedby close, 845
 - usedby countCache, 980
 - usedby frameSpad2Cmd, 26
 - usedby historySpad2Cmd, 890
 - usedby history, 889
 - usedby library, 1199
 - usedby readSpad2Cmd, 943
 - usedby reportOpsFromLisplib, 1093
 - usedby reportOpsFromUnitDirectly, 1096
 - usedby restoreHistory, 907
 - usedby setFunctionsCache, 979
 - usedby showSpad2Cmd, 1087
 - usedby systemCommand, 788
 - usedby trace1, 80
 - usedby traceSpad2Cmd, 78
 - usedby undo, 52
 - usedby workfilesSpad2Cmd, 1139
 - defvar, 73
- \$op
 - usedby dbConstructorDoc,gn, 1635
 - usedby dbConstructorDoc, 1635
 - usedby dbGetDocTable, 1639
 - usedby displayMacro, 794
 - usedby displayType, 800
 - usedby displayValue, 799
 - usedby processInteractive, 342
- \$outputLibraryName
 - usedby setOutputLibrary, 972
- \$outputLines
 - usedby printTypeAndTime, 352
- \$outputList
 - usedby processMsgList, 659
 - usedby queueUpErrors, 662
- \$outputMode
 - usedby recordAndPrint, 349, 350
- \$packages, 73
 - usedby addTraceItem, 146
 - usedby traceReply, 95
 - defvar, 73
- \$pfMacros, 392
 - usedby clearMacroTable, 840
 - usedby clearParserMacro, 793
 - usedby displayParserMacro, 804
 - usedby getParserMacroNames, 793
 - usedby mac0Define, 526
 - usedby mac0GetName, 522
 - usedby mac0Get, 524
 - usedby mac0MLambdaApply, 520
 - usedby macApplication, 519
 - usedby macLambda,mac, 525
 - usedby macLambda, 525
 - usedby macWhere,mac, 525
 - usedby macWhere, 524
 - defvar, 392
- \$plainRTspecialCharacters, 1164
 - usedby setOutputCharacters, 1039
 - defvar, 1164
- \$plainSpecialCharacters0, 1162
 - defvar, 1162
- \$plainSpecialCharacters1, 1162
 - defvar, 1162
- \$plainSpecialCharacters2, 1163

- defvar, 1163
- \$plainSpecialCharacters3, 1163
 - defvar, 1163
- \$posActive
 - usedby mac0ExpandBody, 521
 - usedby mac0MLambdaApply, 520
 - usedby macId, 523
 - usedby macroExpanded, 518
- \$preLength, 645
 - usedby getPreStL, 645
 - usedby makeLeaderMsg, 667
 - usedby makeMsgFromLine, 661
 - usedby processChPosesForOneLine, 666
 - usedby tabbing, 652
 - defvar, 645
- \$predicateList
 - usedby pfRule2Sex, 616
 - usedby pfSuchThat2Sex, 605
 - usedby ruleLhsTran, 620
- \$predvec
 - usedby dbSearchOrder, 1610
 - usedby kTestPred, 1639
- \$prettyprint, 1079
 - defvar, 1079
- \$prevCarrier
 - usedby intloopSpadProcess, 357
- \$previousBindings, 52
 - usedby clearCmdAll, 839
 - usedby clearFrame, 1128
 - usedby recordFrame, 1123
 - defvar, 52
- \$primitiveHtCommands, 1400
 - usedby buildHtMacroTable, 1401
 - defvar, 1400
- \$printAnyIfTrue, 1007
 - usedby recordAndPrint, 350
 - defvar, 1007
- \$printFortranDecs, 989
 - defvar, 989
- \$printLoadMsgs, 1008
 - local ref loadLibNoUpdate, 1223
 - local ref loadLib, 1222
 - usedby restart, 307
 - defvar, 1008
- \$printMsgsToFile, 1012
 - usedby sayKeyedMsgLocal, 44
- defvar, 1012
- \$printStatisticsSummaryIfTrue, 1022
 - usedby recordAndPrint, 350
 - defvar, 1022
- \$printStorageIfTrue
 - usedby recordAndPrint, 350
- \$printTimeIfTrue, 1023
 - usedby printTypeAndTime, 352
 - usedby recordAndPrint, 350
 - defvar, 1023
- \$printTimeSum
 - usedby showHistory, 893
- \$printTypeIfTrue, 1024
 - usedby printTypeAndTime, 352
 - usedby recordAndPrint, 350
 - defvar, 1024
- \$printVoidIfTrue, 1025
 - usedby recordAndPrint, 350
 - defvar, 1025
- \$squadSymbol
 - local ref evaluateType1, 1121
 - usedby reportOperations, 1090
- \$quitCommandType, 1073
 - usedby pquitSpad2Cmd, 938
 - usedby quitSpad2Cmd, 940
 - defvar, 1073
- \$quitTag, 311
 - usedby runspad, 311
 - defvar, 311
- \$quotedOpList
 - usedby pfOp2Sex, 606
 - usedby pfRule2Sex, 616
- \$relative-directory-list, 200
 - usedby reroot, 333
 - defvar, 200
- \$relative-library-directory-list, 200
 - usedby reroot, 333
 - defvar, 200
- \$repGuys, 665
 - defvar, 665
- \$reportBottomUpFlag, 1009
 - usedby coerceConvertMmSelection, 752
 - usedby retractByFunction, 778
 - defvar, 1009
- \$reportCoerceIfTrue, 1010
 - defvar, 1010

- \$reportCompilation, 1078
 - defvar, 1078
- \$reportInstantiations, 1015
 - usedby processInteractive, 343
 - defvar, 1015
- \$reportInterpOnly, 1016
 - defvar, 1016
- \$reportOptimization, 1078
 - defvar, 1078
- \$reportSpadTrace
 - usedby spadTrace, 133
- \$reportSpadtrace, 74
 - defvar, 74
- \$reportinstantiations
 - usedby reportinstantiations, 1271
- \$reportundo, 52
 - defvar, 52
- \$runTestFlag, 349
 - usedby recordAndPrint, 349
 - defvar, 349
- \$r
 - usedby lineoftoks, 403
 - usedby nextline, 404
 - usedby scanEsc, 415
- \$sayBrightlyStream
 - usedby reportOpsFromLisplib1, 1092
 - usedby reportOpsFromUnitDirectly1, 1100
- \$seen
 - usedby ScanOrPairVec,ScanOrInner, 926
 - usedby ScanOrPairVec, 926
 - usedby dewritify,dewritifyInner, 922
 - usedby dewritify, 925
 - usedby saveHistory, 905
 - usedby writify,writifyInner, 918
 - usedby writify, 920
- \$setOptionNames, 1081
 - usedby set1, 1083
 - defvar, 1081
- \$setOptions
 - usedby resetWorkspaceVariables, 963
 - usedby set, 1082
- \$showOptions
 - usedby reportOpsFromLisplib, 1093
 - usedby reportOpsFromUnitDirectly, 1096
 - usedby showSpad2Cmd, 1087
- \$sig
 - usedby dbConstructorDoc,hn, 1634
 - usedby dbConstructorDoc, 1635
 - usedby dbGetDocTable,hn, 1637
 - usedby dbGetDocTable, 1639
- \$slamFlag
 - usedby resetWorkspaceVariables, 963
- \$sockBufferLength, 336
 - usedby serverReadLine, 337
 - defvar, 336
- \$solutionMethod, 1481
 - defvar, 1481
- \$sourceFiles
 - usedby resetWorkspaceVariables, 962
 - usedby updateSourceFiles, 874
 - usedby workfilesSpad2Cmd, 1139
- \$spaceList, 74
 - usedby pspacers, 99
 - usedby resetSpacers, 97
 - usedby resetWorkspaceVariables, 962
 - defvar, 74
- \$spad-errors, 1153
 - usedby init-boot/spad-reader, 1154
 - defvar, 1153
- \$spadroot, 201
 - local ref isSystemDirectory, 1223
 - usedby DaaseName, 1209
 - usedby browseopen, 1191
 - usedby categoryopen, 1192
 - usedby getdatabase, 1196
 - usedby initroot, 327
 - usedby interpopen, 1190
 - usedby make-absolute-filename, 328
 - usedby operationopen, 1193
 - usedby reroot, 333
 - usedby write-browsedb, 1215
 - usedby write-interpdb, 1213
 - defvar, 201
- \$spad
 - usedby ioclear, 1158
 - usedby iostat, 1156
 - usedby ncTopLevel, 316
 - usedby parseAndInterpret, 340
- \$specialCharacterAlist, 1165
 - usedby setOutputCharacters, 1039
 - usedby specialChar, 1166
 - defvar, 1165

- \$specialCharacters, 1165
 - usedby setOutputCharacters, 1039
 - usedby specialChar, 1166
 - defvar, 1165
- \$stack
 - usedby npListAndRecover, 482
 - usedby npListofFun, 513
 - usedby npList, 447
 - usedby npParse, 433
 - usedby npPop1, 436
 - usedby npPop2, 436
 - usedby npPop3, 436
 - usedby npPushId, 501
 - usedby npPush, 435
 - usedby npRestore, 444
 - usedby npState, 504
 - usedby npZeroOrMore, 469
- \$stepNo
 - usedby intloopSpadProcess, 357
- \$stok
 - usedby npConstTok, 477
 - usedby npDDInfKey, 500
 - usedby npDollar, 476
 - usedby npEnclosed, 504
 - usedby npEqKey, 437
 - usedby npEqPeek, 444
 - usedby npFirstTok, 435
 - usedby npId, 497
 - usedby npInfKey, 501
 - usedby npInfixOperator, 452
 - usedby npInfixOp, 453
 - usedby npMissing, 444
 - usedby npParenthesize, 507
 - usedby npParse, 433
 - usedby npPrefixColon, 454
 - usedby npPushId, 501
 - usedby npRecoverTrap, 483
 - usedby npTrap, 505
 - usedby sySpecificErrorHere, 485
- \$streamCount, 74, 1074
 - usedby coerceSpadArgs2E, 128
 - usedby coerceSpadFunValue2E, 130
 - usedby describeSetStreamsCalculate, 1076
 - usedby setStreamsCalculate, 1075
 - defvar, 74, 1074
- \$streamsShowAll, 1076
 - defvar, 1076
- \$superHash
 - usedby getSubDomainPredicate, 769
- \$syscommands, 784
 - usedby newHelpSpad2Cmd, 881
 - usedby systemCommand, 788
 - usedby unAbbreviateKeyword, 809
 - defvar, 784
- \$systemCommandFunction
 - usedby SpadInterpretStream, 319
 - usedby intloopProcess, 356
 - usedby ncloopCommand, 818
- \$systemCommands, 783
 - usedby synonymsForUserLevel, 1106
 - usedby systemCommand, 788
 - usedby unAbbreviateKeyword, 809
 - usedby whatCommands, 1134
 - defvar, 783
- \$systemType, 1480
 - defvar, 1480
- \$sz
 - usedby lineoftoks, 403
 - usedby nextline, 404
 - usedby scanComment, 407
 - usedby scanEsc, 415
 - usedby scanExponent, 418
 - usedby scanNegComment, 408
 - usedby scanNumber, 423
 - usedby scanPossFloat, 412
 - usedby scanS, 422
 - usedby scanW, 419
 - usedby splel1, 414
 - usedby startsComment?, 407
 - usedby startsNegComment?, 408
- \$testingErrorPrefix, 42
 - defvar, 42
- \$testingSystem, 1023
 - defvar, 1023
- \$texFormat, 1069
 - usedby setOutputTex, 1070
 - defvar, 1069
- \$texOutputFile, 1069
 - usedby setOutputTex, 1070
 - defvar, 1069
- \$texOutputStream
 - usedby printAsTeX, 353

- usedby setOutputTex, 1070
- \$timeGlobalName
 - usedby processInteractive, 342
- \$timedNameStack
 - usedby interpretTopLevel, 345
- \$timerList, 74
 - usedby /untrace-2, 125
 - usedby ptimers, 98
 - usedby resetTimers, 97
 - usedby resetWorkspaceVariables, 962
 - usedby trace3, 84
 - defvar, 74
- \$timerTicksPerSecond
 - usedby ptimers, 98
- \$toWhereGuys, 653
 - defvar, 653
- \$tokenCommands, 815
 - usedby doSystemCommand, 786
 - defvar, 815
- \$topicHash
 - usedby write-warmdata, 1218
- \$traceDomains, 75
 - usedby trace3, 84
 - defvar, 75
- \$traceErrorStack, 75
 - usedby getTraceOptions, 116
 - usedby stackTraceOptionError, 100
 - defvar, 75
- \$traceNames, 76
 - usedby /tracereply, 142
 - usedby /untrace-1, 124
 - usedby /untrace-2, 125
 - usedby ?t, 147
 - usedby clearCmdAll, 839
 - usedby getBpiNameIfTracedMap, 141
 - usedby getMapSubNames, 131
 - usedby prTraceNames, 145
 - usedby spadReply, 114
 - usedby spadTrace, 133
 - usedby spadUntrace, 143
 - usedby trace3, 84
 - usedby traceReply, 95
 - usedby untraceDomainConstructor, 139
 - usedby untraceMapSubNames, 105
 - usedby untrace, 123
 - defvar, 76
- \$traceNoisely, 76
 - usedby /untrace-2, 125
 - usedby reportSpadTrace, 141
 - usedby spadTrace, 133
 - usedby trace1, 80
 - usedby trace3, 84
 - defvar, 76
- \$traceOptionList, 76
 - usedby getTraceOption, 118
 - defvar, 76
- \$traceSize, 77
 - defvar, 77
- \$traceStream, 77
 - usedby monitorEnter, 148
 - usedby monitorExit, 151
 - usedby monitorPrintArgs, 148
 - usedby monitorPrintArg, 149
 - usedby monitorPrintRest, 150
 - usedby monitorPrintValue, 152
 - defvar, 77
- \$tracedMapSignatures, 75
 - usedby coerceTraceArgs2E, 128
 - usedby coerceTraceFunValue2E, 130
 - usedby removeTracedMapSigs, 127
 - usedby saveMapSig, 117
 - defvar, 75
- \$tracedModemap
 - usedby spadTrace, 133
- \$tracedSpadModemap, 77
 - usedby coerceSpadArgs2E, 129
 - usedby coerceSpadFunValue2E, 130
 - usedby monitorXX, 90
 - defvar, 77
- \$traceletFunctions, 76
 - usedby breaklet, 155
 - usedby tracelet, 154
 - defvar, 76
- \$traceletflag, 75
 - usedby tracelet, 154
 - defvar, 75
- \$ttok
 - usedby npEqKey, 437
 - usedby npEqPeek, 444
 - usedby npFirstTok, 435
 - usedby npId, 497
 - usedby npInfKey, 501

- usedby npInfixOp, 453
 - usedby npParse, 433
 - usedby npPushId, 501
- \$underbar, 896
 - defvar, 896
- \$useBFasDefault
 - usedby float2Sex, 603
- \$useCoerceOrCroak
 - usedby coerceInt1, 744
- \$useConvertForCoercions
 - usedby coerceIntTest, 752
- \$useEditorForShowOutput, 1067
 - usedby reportOpsFromLisplib0, 1091
 - usedby reportOpsFromUnitDirectly0, 1096
 - defvar, 1067
- \$useFullScreenHelp, 1004
 - usedby newHelpSpad2Cmd, 881
 - defvar, 1004
- \$useInternalHistoryTable, 887
 - usedby clearCmdAll, 839
 - usedby initHist, 888
 - usedby readHiFi, 911
 - usedby restoreHistory, 907
 - usedby saveHistory, 905
 - usedby setHistoryCore, 894
 - usedby writeHiFi, 912
 - defvar, 887
- \$useIntrinsicFunctions, 991
 - defvar, 991
- \$usersTb
 - local ref saveUsersHashTable, 1209
- \$variableNumberAlist
 - usedby clearCmdAll, 839
- \$whatOptions, 1130
 - usedby reportWhatOptions, 1133
 - usedby whatSpad2Cmd, 1132
 - defvar, 1130
- \$whereCacheList
 - usedby processInteractive, 342
- \$which
 - usedby dbGetDocTable,hn, 1637
 - usedby dbGetDocTable, 1639
- \$writifyComplained
 - usedby writifyComplain, 916
 - usedby writify, 920
- \$xdatabase
 - usedby clearCmdCompletely, 838
- abbQuery, 865
 - calledby abbreviationsSpad2Cmd, 823
 - calledby displaySpad2Cmd, 864
 - calls getdatabase, 865
 - calls sayKeyedMsg, 865
 - defun, 865
- abbreviate
 - calledby traceReply, 95
- abbreviation?
 - calledby abbreviationsSpad2Cmd, 823
 - calledby newHelpSpad2Cmd, 881
- abbreviations, 821, 822
 - calls abbreviationsSpad2Cmd, 822
 - defun, 822
 - manpage, 821
- abbreviationsSpad2Cmd, 822
 - calledby abbreviations, 822
 - calls abbQuery, 823
 - calls abbreviation?, 823
 - calls deldatabase, 823
 - calls exit, 823
 - calls helpSpad2Cmd, 823
 - calls listConstructorAbbreviations, 822
 - calls mkUserConstructorAbbreviation, 823
 - calls opOf, 823
 - calls qcar, 823
 - calls qcdr, 823
 - calls sayKeyedMsg, 823
 - calls selectOptionLC, 823
 - calls seq, 823
 - calls setdatabase, 823
 - calls size, 823
 - uses \$options, 823
 - defun, 822
- absolutelyCanCoerceByCheating, 770
 - calledby absolutelyCanCoerceByCheating, 770
 - calledby coerceInt1, 743
 - calls absolutelyCanCoerceByCheating, 770
 - calls deconstructT, 770
 - calls nequal, 770
 - uses \$Integer, 770
 - uses \$SingleInteger, 770
 - defun, 770

- acot, 1291
 - defun, 1291
- acoth, 1293
 - defun, 1293
- acsc, 1292
 - defun, 1292
- acsch, 1293
 - defun, 1293
- addassoc
 - calledby saveMapSig, 117
 - calledby trace1, 79
- addBinding, 1149
 - calls addBindingInteractive, 1149
 - calls getProplist, 1149
 - calls hput, 1149
 - uses \$InteractiveMode, 1149
 - uses \$envHashTable, 1149
 - defun, 1149
- addBindingInteractive, 1153
 - calledby addBinding, 1149
 - calls assq, 1153
 - defun, 1153
- addConsDB
 - calledby getOperationAlistFromLisplib, 134
- addInputLibrary, 975
 - calledby setInputLibrary, 974
 - calls dropInputLibrary, 975
 - uses input-libraries, 975
 - defun, 975
- addNewInterpreterFrame, 33
 - calledby frameSpad2Cmd, 26
 - calledby serverReadLine, 336
 - calls \$erase, 34
 - calls boot-equal, 33
 - calls emptyInterpreterFrame, 34
 - calls framename, 33
 - calls histFileName, 34
 - calls initHistList, 34
 - calls throwKeyedMsg, 33
 - calls updateCurrentInterpreterFrame, 33
 - calls updateFromCurrentInterpreterFrame, 34
 - uses \$interpreterFrameRing, 34
 - defun, 33
- addoperations, 1193
 - calledby localnrlib, 1201
 - calls getdatabase, 1193
 - uses *operation-hash*, 1193
 - defun, 1193
- addParameterTemplates
 - calledby kPage, 1593
- addTraceItem, 146
 - calledby traceReply, 94
 - calls constructor?, 146
 - calls devaluate, 146
 - calls isDomainOrPackage, 146
 - calls isDomain, 146
 - uses \$constructors, 146
 - uses \$domains, 146
 - uses \$packages, 146
 - defun, 146
- adjoinEqual
 - calledby trace3, 84
- aldorTrace
 - calledby spadTrace, 132
- algCoerceInteractive, 1311
 - defun, 1311
- algEqual, 764
 - calledby valueArgsEqual?, 764
 - calls compiledLookupCheck, 765
 - calls evalDomain, 765
 - calls spadcall, 765
 - uses \$Boolean, 765
 - defun, 764
- allConstructors, 1218
 - calledby make-databases, 1205
 - calledby write-browsedb, 1215
 - uses *allconstructors*, 1218
 - defun, 1218
- allocate
 - calledby init-memory-config, 326
- allocate-contiguous-pages
 - calledby init-memory-config, 326
- allocate-relocatable-pages
 - calledby init-memory-config, 326
- allOperations, 1219
 - calledby apropos, 1137
 - uses *allOperations*, 1219
 - uses *operation-hash*, 1219
 - defun, 1219
- alqlGetKindString, 1305
 - calls dbPart, 1305

- calls substring, 1305
- defun, 1305
- alqlGetOrigin, 1304
 - calls charPosition, 1304
 - calls dbPart, 1304
 - calls substring, 1304
 - defun, 1304
- alqlGetParams, 1305
 - calls charPosition, 1305
 - calls dbPart, 1305
 - calls substring, 1305
 - defun, 1305
- alreadyOpened?, 653
 - calledby msgOutputter, 644
 - calls msgImPr?, 653
 - defun, 653
- ancestorsOf
 - calledby kcaPage, 1616
 - calledby originsInOrder, 1636
- ancolsU16, 1321
 - defmacro, 1321
- ancolsU32, 1323
 - defmacro, 1323
- ancolsU8, 1320
 - defmacro, 1320
- anrowsU16, 1321
 - defmacro, 1321
- anrowsU32, 1323
 - defmacro, 1323
- anrowsU8, 1320
 - defmacro, 1320
- apropos, 1137
 - calledby whatSpad2Cmd, 1131
 - calls allOperations, 1137
 - calls downcase, 1137
 - calls exit, 1137
 - calls filterListOfStrings, 1137
 - calls msort, 1137
 - calls sayAsManyPerLineAsPossible, 1137
 - calls sayKeyedMsg, 1138
 - calls sayMessage, 1137
 - calls seq, 1137
 - defun, 1137
- aref2U16, 1321
 - defmacro, 1321
- aref2U32, 1322
 - defmacro, 1322
- aref2U8, 1319
 - defmacro, 1319
- as-insert
 - calledby spadTrace, 132
- asec, 1291
 - defun, 1291
- asech, 1293
 - defun, 1293
- assertCond, 389
 - calledby incLude1, 375
 - calls MakeSymbol, 389
 - calls incCommandTail, 389
 - uses *whitespace*, 389
 - uses \$inclAssertions, 389
 - defun, 389
- assignment, 673
 - syntax, 673
- assoc
 - calledby breaklet, 155
 - calledby clearCmdParts, 841
 - calledby clearParserMacro, 793
 - calledby diffAlist, 1125
 - calledby domainDescendantsOf, 1603
 - calledby getOption, 113
 - calledby koPageAux, 1633
 - calledby readHiFi, 911
 - calledby spadTrace, 132
 - calledby spadUntrace, 143
- assocleft
 - calledby clearCmdParts, 841
 - calledby dbShowCons1, 1643
 - calledby isSubForRedundantMapName, 105
 - calledby originsInOrder, 1636
- assocright
 - calledby orderBySlotNumber, 114
 - calledby untraceMapSubNames, 105
- assq, 1240
 - calledby addBindingInteractive, 1153
 - calledby coerceIntTableOrFunction, 759
 - calledby coerceIntTest, 752
 - calledby diffAlist, 1125
 - calledby fetchOutput, 910
 - calledby hasSig, 721
 - calledby recordNewValue0, 901
 - calledby recordOldValue0, 901

- calledby searchCurrentEnv, 1150
 - calledby searchTailEnv, 1151
 - calledby showInOut, 910
 - calledby specialChar, 1166
 - calledby undoFromFile, 904
 - calledby undoInCore, 902
 - calledby undoSingleStep, 55
 - defmacro, 1240
- asTupleAsList
 - calledby coerceInt1, 743
- augmentHasArgs, 1619
 - calledby kcaPage1, 1617
 - calledby kccPage, 1618
 - calls extractHasArgs, 1619
 - calls getConstructorForm, 1619
 - calls length, 1619
 - calls nreverse0, 1619
 - calls opOf, 1619
 - defun, 1619
- augmentSub
 - calledby hasCateSpecialNew, 734
 - calledby hasCateSpecial, 733
 - calledby hasCaty, 718
 - calledby unifyStructVar, 727
- augmentTraceNames, 78
 - calledby traceSpad2Cmd, 78
 - calls get, 78
 - uses \$InteractiveFrame, 78
 - defun, 78
- AxiomServer
 - calledby browse, 830
- axiomVersion, 816
 - uses *build-version*, 816
 - uses *yearweek*, 816
 - defun, 816
- AXSERV;axServer;IMV;2
 - calledby browse, 830
- basicLookup, 1225
 - calls HasCategory, 1226
 - calls error, 1226
 - calls hashCode?, 1226
 - calls hashString, 1226
 - calls hashType, 1226
 - calls isNewWorldDomain, 1226
 - calls lookupInDomainVector, 1226
 - calls oldCompLookup, 1226
 - calls opIsHasCat, 1226
 - calls spadcall, 1225
 - calls vecp, 1226
 - local ref \$hashOp0, 1226
 - local ref \$hashOp1, 1226
 - local ref \$hashOpApply, 1226
 - local ref \$hashOpSet, 1226
 - local ref \$hashSeg, 1226
 - defun, 1225
- basicLookupCheckDefaults, 1228
 - calledby lookupInDomainVector, 1227
 - calls error, 1228
 - calls hashCode?, 1228
 - calls hashString, 1228
 - calls hashType, 1228
 - calls spadcall, 1228
 - calls vecp, 1228
 - local ref \$lookupDefaults, 1228
 - defun, 1228
- basicMatch?
 - calledby stringMatches?, 1276
- basicStringize, 1510
 - calledby bcHt, 1508
 - calledby iht, 1508
 - calledby mapStringize, 1509
 - defun, 1510
- bcAbbTable
 - calledby dbShowCons1, 1643
- bcComplexLimit, 1478
 - calls htInitPage, 1478
 - calls htMakePage, 1478
 - calls htShowPage, 1478
 - uses \$EmptyMode, 1478
 - defun, 1478
- bcComplexLimitGen, 1480
 - calls bcFinish, 1480
 - calls concat, 1480
 - calls httpButtonValue, 1480
 - calls httpLabelInputString, 1480
 - defun, 1480
- bcConPredTable
 - calledby dbShowConditions, 1648
 - calledby kePage, 1606
- bcConTable
 - calledby dbShowCons1, 1643

- bcCreateVariableString, 1483
 - calledby bcMakeEquations, 1484
 - calledby bcMakeLinearEquations, 1484
 - defun, 1483
- bcDefiniteIntegrate, 1409
 - calls htInitPage, 1409
 - calls htMakePage, 1409
 - calls htShowPage, 1409
 - uses \$EmptyMode, 1409
 - defun, 1409
- bcDefiniteIntegrateGen, 1411
 - calls bcGen, 1411
 - calls concat, 1411
 - calls httpButtonValue, 1411
 - calls httpLabelInputString, 1411
 - defun, 1411
- bcDifferentiate, 1406
 - calls htInitPage, 1406
 - calls htMakeDoneButton, 1406
 - calls htMakePage, 1406
 - calls htShowPage, 1406
 - uses \$EmptyMode, 1406
 - defun, 1406
- bcDifferentiateGen, 1407
 - calls bcError, 1407
 - calls bcGen, 1407
 - calls bcString2WordList, 1407
 - calls bcwords2liststring, 1407
 - calls concat, 1407
 - calls httpLabelInputString, 1407
 - calls length, 1407
 - defun, 1407
- bcDraw, 1415
 - calls bcHt, 1415
 - calls htInitPage, 1415
 - calls htShowPage, 1415
 - defun, 1415
- bcDraw2Dfun, 1416
 - calls htInitPage, 1416
 - calls htMakePage, 1416
 - calls htShowPage, 1416
 - uses \$EmptyMode, 1416
 - defun, 1416
- bcDraw2DfunGen, 1417
 - calls bcDrawIt2, 1417
 - calls bcFinish, 1417
 - calls concat, 1417
 - calls httpLabelInputString, 1417
 - defun, 1417
- bcDraw2Dpar, 1418
 - calls htInitPage, 1418
 - calls htMakePage, 1418
 - calls htShowPage, 1418
 - uses \$EmptyMode, 1418
 - defun, 1418
- bcDraw2DparGen, 1419
 - calls bcDrawIt2, 1419
 - calls bcFinish, 1419
 - calls concat, 1419
 - calls httpLabelInputString, 1419
 - defun, 1419
- bcDraw2DSolve, 1420
 - calls htInitPage, 1420
 - calls htMakeDoneButton, 1420
 - calls htMakePage, 1420
 - calls htShowPage, 1420
 - uses \$EmptyMode, 1420
 - defun, 1420
- bcDraw2DSolveGen, 1421
 - calls bcFinish, 1421
 - calls concat, 1421
 - calls httpLabelInputString, 1421
 - defun, 1421
- bcDraw3Dfun, 1422
 - calls htInitPage, 1422
 - calls htMakePage, 1422
 - calls htShowPage, 1422
 - uses \$EmptyMode, 1422
 - defun, 1422
- bcDraw3DfunGen, 1424
 - calls bcDrawIt2, 1424
 - calls bcFinish, 1424
 - calls concat, 1424
 - calls httpLabelInputString, 1424
 - defun, 1424
- bcDraw3Dpar, 1425
 - calls htInitPage, 1425
 - calls htMakePage, 1425
 - calls htShowPage, 1425
 - uses \$EmptyMode, 1425
 - defun, 1425
- bcDraw3Dpar1, 1427

- calls htInitPage, [1427](#)
- calls htMakeDoneButton, [1427](#)
- calls htMakePage, [1427](#)
- calls htShowPage, [1427](#)
- uses \$EmptyMode, [1427](#)
- defun, [1427](#)
- bcDraw3Dpar1Gen, [1428](#)
 - calls bcDrawIt2, [1428](#)
 - calls bcFinish, [1429](#)
 - calls concat, [1428](#)
 - calls httpLabelInputString, [1428](#)
 - defun, [1428](#)
- bcDraw3DparGen, [1426](#)
 - calls bcDrawIt2, [1426](#)
 - calls bcFinish, [1426](#)
 - calls concat, [1426](#)
 - calls httpLabelInputString, [1426](#)
 - defun, [1426](#)
- bcDrawIt, [1496](#)
 - calls concat, [1496](#)
 - defun, [1496](#)
- bcDrawIt2, [1473](#)
 - calledby bcDraw2DfunGen, [1417](#)
 - calledby bcDraw2DparGen, [1419](#)
 - calledby bcDraw3DfunGen, [1424](#)
 - calledby bcDraw3Dpar1Gen, [1428](#)
 - calledby bcDraw3DparGen, [1426](#)
 - calls bcMatrixGen, [1473](#)
 - defun, [1473](#)
- bcError, [1496](#)
 - calledby bcDifferentiateGen, [1407](#)
 - calls sayBrightlyNT, [1496](#)
 - calls sayBrightly, [1496](#)
 - defun, [1496](#)
- bcErrorPage
 - calledby dbShowCons, [1641](#)
- bcFindString, [1494](#)
 - defun, [1494](#)
- bcFinish, [1493](#)
 - calledby bcComplexLimitGen, [1480](#)
 - calledby bcDraw2DSolveGen, [1421](#)
 - calledby bcDraw2DfunGen, [1417](#)
 - calledby bcDraw2DparGen, [1419](#)
 - calledby bcDraw3DfunGen, [1424](#)
 - calledby bcDraw3Dpar1Gen, [1429](#)
 - calledby bcDraw3DparGen, [1426](#)
 - calledby bcLinearMatrixGen, [1490](#)
 - calledby bcLinearSolveEqnsGen, [1492](#)
 - calledby bcRealLimitGen1, [1478](#)
 - calledby bcRealLimitGen, [1477](#)
 - calledby bcSeriesExpansionGen, [1431](#)
 - calledby bcSeriesGen, [1435](#)
 - calledby bcSolveEquations, [1486](#)
 - defun, [1493](#)
- bcGen, [1468](#), [1494](#)
 - calledby bcDefiniteIntegrateGen, [1411](#)
 - calledby bcDifferentiateGen, [1407](#)
 - calledby bcGenExplicitMatrix, [1472](#)
 - calledby bcIndefiniteIntegrateGen, [1409](#)
 - calledby bcInputMatrixByFormulaGen, [1471](#)
 - calledby bcLinearMatrixGen, [1490](#)
 - calledby bcProductGen, [1474](#)
 - calledby bcSumGen, [1414](#)
 - calls concat, [1468](#), [1494](#)
 - calls htInitPage, [1468](#), [1494](#)
 - calls htMakePage, [1468](#), [1494](#)
 - calls htShowPage, [1468](#), [1494](#)
 - defun, [1468](#), [1494](#)
- bcGenEquations, [1492](#)
 - calledby bcLinearSolveEqnsGen, [1492](#)
 - calledby bcSolveEquations, [1486](#)
 - calls bcwords2liststring, [1492](#)
 - calls concat, [1492](#)
 - defun, [1492](#)
- bcGenExplicitMatrix, [1472](#)
 - calls bcGen, [1472](#)
 - calls bcMatrixGen, [1472](#)
 - calls httpInputAreaAlist, [1472](#)
 - calls httpProperty, [1472](#)
 - calls httpSetProperty, [1472](#)
 - defun, [1472](#)
- bcHt, [1508](#)
 - calledby bcDraw, [1415](#)
 - calledby bcInputEquations, [1481](#)
 - calledby bcInputExplicitMatrix, [1443](#)
 - calledby bcvspace, [1495](#)
 - calledby htSayBind, [1512](#)
 - calledby htSay, [1509](#)
 - calledby kdPageInfo, [1601](#)
 - calls basicStringize, [1508](#)
 - calls httpAddToPageDescription, [1508](#)
 - calls mapStringize, [1508](#)

- uses \$curPage, 1508
 - uses \$htLineList, 1508
 - uses \$newPage, 1508
 - defun, 1508
- bchtMakeButton, 1536
 - defun, 1536
- bcIndefiniteIntegrate, 1408
 - calls htInitPage, 1408
 - calls htMakePage, 1408
 - calls htShowPage, 1408
 - uses \$EmptyMode, 1408
 - defun, 1408
- bcIndefiniteIntegrateGen, 1409
 - calls bcGen, 1409
 - calls concat, 1409
 - calls httpLabelInputString, 1409
 - defun, 1409
- bcInputEquations, 1481
 - calledby bcLinearSolveEqns1, 1441
 - calledby bcSolveSingle, 1449
 - calledby bcSystemSolveEqns1, 1447
 - calls bcHt, 1481
 - calls bcMakeEquations, 1481
 - calls bcMakeLinearEquations, 1481
 - calls bcMakeUnknowns, 1481
 - calls concat, 1481
 - calls htInitPage, 1481
 - calls htMakeDoneButton, 1481
 - calls htMakePage, 1481
 - calls htProperty, 1481
 - calls htSay, 1481
 - calls htShowPage, 1481
 - calls httpPropertyList, 1481
 - calls httpSetProperty, 1481
 - calls objValUnwrap, 1481
 - calls parse-integer, 1481
 - uses \$EmptyMode, 1481
 - uses \$bcParseOnly, 1481
 - defun, 1481
- bcInputEquationsEnd, 1485
 - calls systemError, 1485
 - defun, 1485
- bcInputExplicitMatrix, 1443
 - calls bcHt, 1443
 - calls concat, 1443
 - calls htInitPage, 1443
 - calls htMakeDoneButton, 1443
 - calls htMakePage, 1443
 - calls htShowPage, 1443
 - calls httpLabelInputString, 1443
 - calls httpLabelSpadValue, 1443
 - calls httpPropertyList, 1443
 - calls httpSetProperty, 1443
 - calls length, 1443
 - calls nreverse0, 1443
 - calls objValUnwrap, 1443
 - calls parse-integer, 1443
 - uses \$EmptyMode, 1443
 - uses \$bcParseOnly, 1443
 - defun, 1443
- bcInputMatrixByFormula
 - calls htInitPage, 1445
 - calls htMakeDoneButton, 1445
 - calls htMakePage, 1445
 - calls htShowPage, 1445
 - calls httpLabelInputString, 1445
 - calls httpLabelSpadValue, 1445
 - calls httpSetProperty, 1445
 - calls objValUnwrap, 1445
 - calls parse-integer, 1445
 - uses \$bcParseOnly, 1445
- bcInputMatrixByFormulaGen, 1471
 - calls bcGen, 1471
 - calls concat, 1471
 - calls httpLabelInputString, 1471
 - calls htProperty, 1471
 - defun, 1471
- bcInputSolveInfo, 1448
 - calls htInitPage, 1448
 - calls htMakePage, 1448
 - calls htShowPage, 1448
 - calls httpInputAreaList, 1448
 - calls httpPropertyList, 1448
 - calls httpSetProperty, 1448
 - defun, 1448
- bcIssueHt, 1509
 - defun, 1509
- bcLaurentSeries, 1435
 - calls htInitPage, 1435
 - calls htMakePage, 1435
 - calls htShowPage, 1435
 - uses \$EmptyMode, 1435

- defun, [1435](#)
- bcLaurentSeriesGen, [1437](#)
 - calls bcSeriesGen, [1437](#)
 - defun, [1437](#)
- bcLimit, [1412](#)
 - calls htInitPage, [1412](#)
 - calls htMakePage, [1412](#)
 - calls htShowPage, [1412](#)
 - uses \$EmptyMode, [1412](#)
 - defun, [1412](#)
- bcLinearExtractMatrix, [1487](#)
 - calls httpInputAreaAlist, [1487](#)
 - defun, [1487](#)
- bcLinearMatrixGen, [1490](#)
 - calledby bcLinearSolveMatrixHomo, [1489](#)
 - calledby bcLinearSolveMatrixInhomoGen, [1489](#)
 - calls bcFinish, [1490](#)
 - calls bcGen, [1490](#)
 - calls bcMatrixGen, [1490](#)
 - calls bcMkFunction, [1490](#)
 - calls bcVectorGen, [1490](#)
 - calls concat, [1490](#)
 - calls httpInputAreaAlist, [1490](#)
 - defun, [1490](#)
- bcLinearSolve, [1440](#)
 - calls htInitPage, [1440](#)
 - calls htMakePage, [1440](#)
 - calls htShowPage, [1440](#)
 - uses \$EmptyMode, [1440](#)
 - defun, [1440](#)
- bcLinearSolveEqns, [1440](#)
 - calls htInitPage, [1440](#)
 - calls htMakeDoneButton, [1441](#)
 - calls htMakePage, [1441](#)
 - calls htShowPage, [1441](#)
 - uses \$EmptyMode, [1441](#)
 - defun, [1440](#)
- bcLinearSolveEqns1, [1441](#)
 - calls bcInputEquations, [1441](#)
 - calls httpSetProperty, [1441](#)
 - defun, [1441](#)
- bcLinearSolveEqnsGen, [1492](#)
 - calls bcFinish, [1492](#)
 - calls bcGenEquations, [1492](#)
 - calls bcString2WordList, [1492](#)
 - calls bcwords2liststring, [1492](#)
 - calls httpInputAreaAlist, [1492](#)
 - calls httpLabelInputString, [1492](#)
 - defun, [1492](#)
- bcLinearSolveMatrix, [1442](#)
 - calls bcReadMatrix, [1442](#)
 - defun, [1442](#)
- bcLinearSolveMatrix1, [1487](#)
 - calls htInitPage, [1487](#)
 - calls htMakePage, [1487](#)
 - calls htShowPage, [1487](#)
 - uses \$EmptyMode, [1487](#)
 - defun, [1487](#)
- bcLinearSolveMatrixHomo, [1489](#)
 - calls bcLinearMatrixGen, [1489](#)
 - defun, [1489](#)
- bcLinearSolveMatrixInhomo, [1488](#)
 - calls concat, [1488](#)
 - calls htInitPage, [1488](#)
 - calls htMakePage, [1488](#)
 - calls htShowPage, [1488](#)
 - calls httpPropertyList, [1488](#)
 - calls httpProperty, [1488](#)
 - calls httpSetProperty, [1488](#)
 - uses \$EmptyMode, [1488](#)
 - defun, [1488](#)
- bcLinearSolveMatrixInhomoGen, [1489](#)
 - calls bcLinearMatrixGen, [1489](#)
 - defun, [1489](#)
- bcMakeEquations, [1483](#)
 - calledby bcInputEquations, [1481](#)
 - calls bcCreateVariableString, [1484](#)
 - calls concat, [1483](#)
 - calls nreverse0, [1484](#)
 - defun, [1483](#)
- bcMakeLinearEquations, [1484](#)
 - calledby bcInputEquations, [1481](#)
 - calls bcCreateVariableString, [1484](#)
 - calls concat, [1484](#)
 - calls nreverse0, [1484](#)
 - defun, [1484](#)
- bcMakeUnknowns, [1483](#)
 - calledby bcInputEquations, [1481](#)
 - defun, [1483](#)
- bcMatrix, [1414](#)
 - calls bcReadMatrix, [1415](#)

- defun, 1414
- bcMatrixGen, 1472
 - calledby bcDrawIt2, 1473
 - calledby bcGenExplicitMatrix, 1472
 - calledby bcLinearMatrixGen, 1490
 - calls bcwords2liststring, 1472
 - calls concat, 1472
 - calls httpProperty, 1472
 - calls lassoc, 1472
 - calls systemError, 1472
 - defun, 1472
- bcMkFunction, 1493
 - calledby bcLinearMatrixGen, 1490
 - defun, 1493
- bcNameConTable
 - calledby dbShowCons1, 1643
- bcNotReady, 1496
 - calledby bcSeriesByFormulaGen, 1475
 - calls htInitPage, 1496
 - calls htMakePage, 1496
 - calls htShowPage, 1496
 - defun, 1496
- bcOptional, 1494
 - defun, 1494
- bcProduct, 1474
 - calls htInitPage, 1474
 - calls htMakePage, 1474
 - calls htShowPage, 1474
 - uses \$EmptyMode, 1474
 - defun, 1474
- bcProductGen, 1474
 - calls bcGen, 1474
 - calls concat, 1474
 - calls httpLabelInputString, 1474
 - defun, 1474
- bcPuisseuxSeries, 1437
 - calls htInitPage, 1437
 - calls htMakePage, 1437
 - calls htShowPage, 1437
 - uses \$EmptyMode, 1437
 - defun, 1437
- bcPuisseuxSeriesGen, 1438
 - calls bcSeriesGen, 1438
 - defun, 1438
- bcQueryInteger
 - calledby finalExactRequest, 1491
- calledby linearFinalRequest, 1490
- bcReadMatrix, 1442, 1475
 - calledby bcLinearSolveMatrix, 1442
 - calledby bcMatrix, 1415
 - calls htInitPage, 1442, 1475
 - calls htMakePage, 1442, 1475
 - calls htShowPage, 1442, 1475
 - calls httpSetProperty, 1442, 1475
 - defun, 1442, 1475
- bcRealLimit, 1475
 - calls htInitPage, 1475
 - calls htMakePage, 1475
 - calls htShowPage, 1475
 - uses \$EmptyMode, 1475
 - defun, 1475
- bcRealLimitGen, 1477
 - calls bcFinish, 1477
 - calls htInitPage, 1477
 - calls htMakePage, 1477
 - calls htShowPage, 1477
 - calls httpButtonValue, 1477
 - calls httpLabelInputString, 1477
 - calls httpSetProperty, 1477
 - defun, 1477
- bcRealLimitGen1, 1478
 - calls bcFinish, 1478
 - calls concat, 1478
 - calls httpProperty, 1478
 - defun, 1478
- bcSadFaces, 1518
 - defun, 1518
- bcSeries, 1429
 - calls htInitPage, 1429
 - calls htMakePage, 1429
 - calls htShowPage, 1429
 - uses \$EmptyMode, 1429
 - defun, 1429
- bcSeriesByFormula, 1432
 - calls htInitPage, 1432
 - calls htMakePage, 1432
 - calls htShowPage, 1432
 - defun, 1432
- bcSeriesByFormulaGen, 1475
 - calls bcNotReady, 1475
 - defun, 1475
- bcSeriesExpansion, 1430

- calls htInitPage, 1430
- calls htMakeDoneButton, 1430
- calls htMakePage, 1430
- calls htShowPage, 1430
- uses \$EmptyMode, 1430
- defun, 1430
- bcSeriesExpansionGen, 1431
 - calls bcFinish, 1431
 - calls concat, 1431
 - calls httpLabelInputString, 1431
 - defun, 1431
- bcSeriesGen, 1435
 - calledby bcLaurentSeriesGen, 1437
 - calledby bcPuisseuxSeriesGen, 1438
 - calledby bcTaylorSeriesGen, 1434
 - calls bcFinish, 1435
 - calls concat, 1435
 - calls httpLabelInputString, 1435
 - defun, 1435
- bcSolve, 1439
 - calls htInitPage, 1439
 - calls htMakePage, 1439
 - calls htShowPage, 1439
 - uses \$EmptyMode, 1439
 - defun, 1439
- bcSolveEquations, 1486
 - calledby bcSolveNumerically1, 1486
 - calls bcFinish, 1486
 - calls bcGenEquations, 1486
 - calls bcString2WordList, 1486
 - calls bcwords2liststring, 1486
 - calls concat, 1486
 - calls httpButtonValue, 1486
 - calls httpLabelInputString, 1486
 - calls httpProperty, 1486
 - calls member, 1486
 - defun, 1486
- bcSolveEquationsNumerically, 1485
 - calls htInitPage, 1485
 - calls htMakeDoneButton, 1485
 - calls htMakePage, 1485
 - calls htShowPage, 1485
 - calls httpPropertyList, 1485
 - defun, 1485
- bcSolveNumerically1, 1486
 - calls bcSolveEquations, 1486
 - defun, 1486
- bcSolveSingle, 1449
 - calls bcInputEquations, 1449
 - calls httpSetProperty, 1449
 - defun, 1449
- bcString2HyString, 1494
 - defun, 1494
- bcString2HyString2, 1493
 - defun, 1493
- bcString2WordList, 1495
 - calledby bcDifferentiateGen, 1407
 - calledby bcLinearSolveEqnsGen, 1492
 - calledby bcSolveEquations, 1486
 - defun, 1495
- bcSum, 1413
 - calls htInitPage, 1413
 - calls htMakePage, 1413
 - calls htShowPage, 1413
 - uses \$EmptyMode, 1413
 - defun, 1413
- bcSumGen, 1414
 - calls bcGen, 1414
 - calls concat, 1414
 - calls httpLabelInputString, 1414
 - defun, 1414
- bcSystemSolve, 1446
 - calls htInitPage, 1446
 - calls htMakeDoneButton, 1447
 - calls htMakePage, 1447
 - calls htShowPage, 1447
 - defun, 1446
- bcSystemSolveEqns1, 1447
 - calls bcInputEquations, 1447
 - calls httpSetProperty, 1447
 - defun, 1447
- bcTaylorSeries, 1433
 - calls htInitPage, 1433
 - calls htMakePage, 1433
 - calls htShowPage, 1433
 - uses \$EmptyMode, 1433
 - defun, 1433
- bcTaylorSeriesGen, 1434
 - calls bcSeriesGen, 1434
 - defun, 1434
- bcUnixTable, 1650
 - calledby dbShowCons1, 1643

- calls findfile, 1650
- calls htBeginTable, 1650
- calls htEndTable, 1651
- calls htMakePage, 1650
- calls htSay, 1650
- calls namestring, 1650
- uses firstTime, 1651
- defun, 1650
- bcVectorGen, 1495
 - calledby bcLinearMatrixGen, 1490
 - calls bcwords2liststring, 1495
 - defun, 1495
- bcvspace, 1495
 - calls bcHt, 1495
 - defun, 1495
- bcwords2liststring, 1495
 - calledby bcDifferentiateGen, 1407
 - calledby bcGenEquations, 1492
 - calledby bcLinearSolveEqnsGen, 1492
 - calledby bcMatrixGen, 1472
 - calledby bcSolveEquations, 1486
 - calledby bcVectorGen, 1495
 - calls concat, 1495
 - defun, 1495
- beforeAfter, 1521
 - defun, 1521
- BesselasymptA, 1264
 - defun, 1264
- BesselasymptB, 1264
 - defun, 1264
- BesselI, 1252
 - calledby BesselI, 1252
 - calledby cbesseli, 1280
 - calledby rbesseli, 1280
 - calls BesselI, 1252
 - calls FloatError, 1252
 - calls besseliBack, 1252
 - calls besseliCheb, 1252
 - defun, 1252
- besseliBack, 1253
 - calledby BesselI, 1252
 - calls BesseliBackRecur, 1253
 - calls cgammaImpl, 1253
 - defun, 1253
- BesseliBackRecur, 1253
 - calledby besseliBack, 1253
- defun, 1253
- besseliCheb, 1254
 - calledby BesselI, 1252
 - calls cgammaImpl, 1255
 - calls chebf01coefmake, 1254
 - calls chebstarevalarr, 1254
 - defun, 1254
- BesselJ, 1262
 - calledby cbesselj, 1279
 - calledby rbesselj, 1279
 - defun, 1262
- BesselJAsympt, 1264
 - defun, 1264
- BesselJAsymptOrder, 1265
 - defun, 1265
- BesselJRecur, 1265
 - defun, 1265
- bit-to-truth, 1301
 - defmacro, 1301
- blankList
 - calledby filterAndFormatConstructors, 1136
 - calledby printLabelledList, 814
 - calledby whatCommands, 1133
- blocks, 676
 - syntax, 676
- Boolean
 - calledby hashable, 1316
- BooleanEquality, 1272
 - defun, 1272
- boot, 826
 - manpage, 826
- boot-equal
 - calledby addNewInterpreterFrame, 33
 - calledby clearCmdParts, 841
 - calledby displaySetOptionInformation, 964
 - calledby fetchOutput, 910
 - calledby findFrameInRing, 31
 - calledby getMapSig, 117
 - calledby importFromFrame, 34
 - calledby setHistoryCore, 894
 - calledby undoChanges, 903
 - calledby untraceDomainConstructor,keepTraced?, 138
 - calledby whatConstructors, 1137
 - calledby writify,writifyInner, 917
- boot-line-stack, 1147

- usedby init-boot/spad-reader, 1154
 - usedby next-lines-clear, 1158
 - usedby next-lines-show, 1157
 - defvar, 1147
- bottomUp
 - calledby coerceInt1, 743
 - calledby evaluateType, 1118
 - calledby interpret1, 347
- bottumUp
 - calledby evaluateType1, 1120
- bpiname
 - calledby breaklet, 155
 - calledby hashable, 1316
 - calledby mkEvalable, 1116
 - calledby spadClosure?, 921
 - calledby spadTrace,isTraceable, 132
 - calledby spadTrace, 132
 - calledby spadUntrace, 143
 - calledby tracelet, 154
- bpitrace, 136
 - calledby spadTrace, 133
 - calls trace3, 136
 - defun, 136
- bpiuntrace
 - calledby spadUntrace, 143
- bracketString, 1544
 - defun, 1544
- brCon
 - calledby kcPage, 1611
- break, 156
 - calledby letPrint2, 110
 - calledby letPrint3, 111
 - calledby letPrint, 108
 - calledby monitorXX, 90
 - calls MONITOR,EVALTRAN, 156
 - calls sayBrightly, 156
 - uses /breakcondition, 156
 - defun, 156
- breaklet, 155
 - calledby trace3, 84
 - calls assoc, 155
 - calls bpiname, 155
 - calls compileBoot, 155
 - calls delete, 155
 - calls gensymp, 155
 - calls lassoc, 155
 - calls setletprintflag, 155
 - calls stupidIsSpadFunction, 155
 - calls union, 155
 - uses \$QuickLet, 155
 - uses \$letAssoc, 155
 - uses \$traceletFunctions, 155
 - defun, 155
- bright
 - calledby ?t, 146
 - calledby commandAmbiguityError, 792
 - calledby displayCondition, 804
 - calledby displayFrameNames, 28
 - calledby displayMacro, 794
 - calledby displayModemap, 806
 - calledby displayMode, 806
 - calledby displayProperties,sayFunctionDeps, 798
 - calledby displayProperties, 801
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby letPrint2, 110
 - calledby letPrint3, 112
 - calledby letPrint, 108
 - calledby pcounters, 99
 - calledby pspacers, 98
 - calledby ptimers, 98
 - calledby reportOperations, 1089
 - calledby reportOpsFromLisplib, 1092
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby sayCacheCount, 983
 - calledby set1, 1083
 - calledby setFortDir, 999
 - calledby setFortPers, 1029
 - calledby setFortTmpDir, 996
 - calledby setFunctionsCache, 979
 - calledby setOutputCharacters, 1039
 - calledby setStreamsCalculate, 1075
 - calledby showHistory, 892
 - calledby spadUntrace, 143
- brightprint, 1238
 - calledby saybrightly1, 1240
 - calls messageprint, 1238
 - defun, 1238
- brightprint-0, 1238
 - calledby saybrightly1, 1240
 - calls messageprint-1, 1238

- defun, 1238
- browse, 827
 - calls AXSERV;axServer;IMV;2, 830
 - calls AxiomServer, 830
 - calls loadLib, 830
 - calls set, 830
 - manpage, 827
- browseopen, 1191
 - calledby resethashtables, 1186
 - calledby restart0, 309
 - uses *allconstructors*, 1191
 - uses *browse-stream*, 1191
 - uses *browse-stream-stamp*, 1191
 - uses \$spadroot, 1191
 - defun, 1191
- browserAutoloadOnceTrigger
 - calledby make-databases, 1205
- bubbleConstructor
 - calledby coerceIntTower, 750
- buildHtMacroTable, 1401
 - calls concat, 1401
 - calls getHtMacroItem, 1401
 - calls getenviron, 1401
 - calls hput, 1401
 - calls sayBrightly, 1401
 - calls util.ht[7.1], 1401
 - uses \$htMacroTable, 1401
 - uses \$primitiveHtCommands, 1401
 - defun, 1401
- buildLibdb
 - calledby make-databases, 1205
- buildLibdbConEntry[9]
 - called by conPageConEntry, 1600
- bumperrorcount
 - calledby spad-syntax-error, 1155
- buttonNames, 1525
 - defun, 1525
- bvec-and, 1303
 - defun, 1303
- bvec-concat, 1302
 - defun, 1302
- bvec-copy, 1302
 - defun, 1302
- bvec-elt, 1301
 - defmacro, 1301
- bvec-equal, 1302
 - defun, 1302
- bvec-greater, 1303
 - defun, 1303
- bvec-make-full, 1301
 - defun, 1301
- bvec-nand, 1304
 - defun, 1304
- bvec-nor, 1304
 - defun, 1304
- bvec-not, 1304
 - defun, 1304
- bvec-or, 1303
 - defun, 1303
- bvec-setelt, 1302
 - defmacro, 1302
- bvec-size, 1302
 - defmacro, 1302
- bvec-xor, 1303
 - defun, 1303
- c-to-r, 1244
 - calledby rbesseli, 1280
 - calledby rbesselj, 1279
 - defun, 1244
- c-to-s, 1243
 - calledby cbesseli, 1280
 - calledby cbesselj, 1279
 - calledby cgamma, 1278
 - calledby chyper0f1, 1280
 - calledby clngamma, 1278
 - calledby cpsi, 1279
 - defun, 1243
- CallerName
 - calledby processKeyedError, 643
- canCoerce
 - calledby unifyStructVar, 727
- canCoerceFrom
 - calledby hasCateSpecial, 733
- canFuncall?, 1237
 - calledby coerceRetract, 777
 - defun, 1237
- capitalize
 - calledby dbConsHeading, 1649
 - calledby kPage, 1593
 - calledby kePage, 1605
 - calledby kiPage, 1605

- calledby koPage, 1631
- captialize
 - calledby conOpPage1, 1629
- Catch
 - calledby intloopSpadProcess, 357
- CatchAsCan
 - calledby intloopSpadProcess, 357
- catchCoerceFailure, 760
 - calls throwKeyedMsgCannotCoerceWith-Value, 760
 - calls unwrap, 760
 - calls wrap, 760
 - uses \$coerceFailure, 760
 - catches, 760
 - defun, 760
- catches
 - catchCoerceFailure, 760
 - coerceByTable, 760
 - coerceInt1, 744
 - coerceIntByMap, 761
 - coerceIntCommute, 757
 - coerceRetract, 777
 - executeQuietCommand, 339
 - InterpExecuteSpadSystemCommand, 323
 - interpretTopLevel, 345
 - intloop, 317
 - intloopSpadProcess, 357
 - kDomainName, 1624
 - kisValidType, 1626
 - letPrint2, 110
 - letPrint3, 111
 - monitor-file, 1387
 - monitor-readinterp, 1394
 - monitor-spadfile, 1396
 - npListAndRecover, 482
 - npParse, 433
 - runspad, 311
 - safeWritify, 917
 - ScanOrPairVec, 926
 - serverReadLine, 336
- categoryForm?
 - calledby evaluateType1, 1120
 - calledby loadLib, 1222
 - calledby localnrlib, 1201
 - calledby make-databases, 1205
- categoryopen, 1192
 - calledby resethashables, 1186
 - calledby restart0, 309
 - uses *category-stream*, 1192
 - uses *category-stream-stamp*, 1192
 - uses *hasCategory-hash*, 1192
 - uses \$spadroot, 1192
 - defun, 1192
- cbesseli, 1280
 - calls BesselI, 1280
 - calls c-to-s, 1280
 - calls s-to-c, 1280
 - defun, 1280
- cbesselj, 1279
 - calls BesselJ, 1279
 - calls c-to-s, 1279
 - calls s-to-c, 1279
 - defun, 1279
- cd, 832
 - manpage, 832
- cdancols, 1274
 - defmacro, 1274
- cdanrows, 1274
 - defmacro, 1274
- cdaref2, 1273
 - defmacro, 1273
- cdelt, 1275
 - defmacro, 1275
- cdlen, 1276
 - defmacro, 1276
- cdsetaref2, 1273
 - defmacro, 1273
- cdsetelt, 1275
 - defmacro, 1275
- cgamma, 1277
 - calls c-to-s, 1278
 - calls cgammaImpl, 1278
 - calls s-to-c, 1278
 - defun, 1277
- cgammaAdjust, 1251
 - calledby clngammacase2, 1250
 - defun, 1251
- cgammaBernsum, 1252
 - calledby clngammacase2, 1250
 - calledby clngammacase3, 1251
 - defun, 1252
- cgammaG, 1249

- calledby PiMinusLogSinPi, 1249
- defun, 1249
- cgammaImpl, 1248
 - calledby bessellback, 1253
 - calledby bessellcheb, 1255
 - calledby cgamma, 1278
 - calls clnggammaImpl, 1248
 - calls rgammaImpl, 1248
 - defun, 1248
- cgamma, 1250
 - calledby clnggammacase23, 1250
 - defun, 1250
- changeHistListLen, 898
 - calledby historySpad2Cmd, 890
 - calls sayKeyedMsg, 898
 - uses \$HistListAct, 898
 - uses \$HistListLen, 898
 - uses \$HistList, 898
 - defun, 898
- changeToNamedInterpreterFrame, 32
 - calledby serverReadLine, 336
 - calls findFrameInRing, 32
 - calls updateCurrentInterpreterFrame, 32
 - calls updateFromCurrentInterpreterFrame, 32
 - uses \$interpreterFrameRing, 32
 - defun, 32
- char
 - calledby makeMsgFromLine, 661
- charDigitVal, 927
 - calledby gensymInt, 927
 - calls error, 927
 - defun, 927
- charPosition
 - calledby alqlGetOrigin, 1304
 - calledby alqlGetParams, 1305
 - calledby conPageFastPath, 1600
 - calledby removeUndoLines, 57
- chebf01, 1267
 - calledby chyper0f1, 1280
 - defun, 1267
- chebf01coefmake, 1255
 - calledby bessellcheb, 1254
 - defun, 1255
- chebstarevalarr, 1256
 - calledby bessellcheb, 1254
 - defun, 1256
- checkCondition, 1540
 - defun, 1540
- checkFilter, 1589
 - calls trimString, 1589
 - defun, 1589
- childrenOf
 - calledby kccPage, 1618
- chkAllNonNegativeInteger, 1562
 - defun, 1562
- chkDirectory, 1561
 - defun, 1561
- chkNameList, 1560
 - defun, 1560
- chkNonNegativeInteger, 1562
 - defun, 1562
- chkOutputFileName, 1561
 - defun, 1561
- chkPosInteger, 1561
 - defun, 1561
- chkRange, 1562
 - defun, 1562
- chyper0f1, 1280
 - calls c-to-s, 1280
 - calls chebf01, 1280
 - calls s-to-c, 1280
 - defun, 1280
- cleanline, 859
 - calledby describeSpad2Cmd, 858
 - defun, 859
- cleanupLine, 869
 - defun, 869
- cleanupline
 - calledby sayexample, 869
- clear, 833, 835
 - calls clearSpad2Cmd, 835
 - defun, 835
 - manpage, 833
- clearAllSlams
 - calledby updateDatabase, 1203
- clearClams
 - calledby clearCmdCompletely, 838
 - calledby setExposeAddConstr, 161
 - calledby setExposeAddGroup, 157
 - calledby setExposeDropConstr, 164
 - calledby setExposeDropGroup, 162

- calledby updateDatabase, 1203
- clearCmdAll, 839
 - calledby clearCmdCompletely, 838
 - calledby clearFrame, 1128
 - calledby clearSpad2Cmd, 835
 - calls clearCmdSortedCaches, 839
 - calls clearMacroTable, 839
 - calls deleteFile, 839
 - calls histFileName, 839
 - calls resetInCoreHist, 839
 - calls sayKeyedMsg, 839
 - calls untraceMapSubNames, 839
 - calls updateCurrentInterpreterFrame, 839
 - uses \$InteractiveFrame, 839
 - uses \$currentLine, 839
 - uses \$frameMessages, 839
 - uses \$frameRecord, 839
 - uses \$internalHistoryTable, 839
 - uses \$interpreterFrameName, 839
 - uses \$previousBindings, 839
 - uses \$traceNames, 839
 - uses \$useInternalHistoryTable, 839
 - uses \$variableNumberAlist, 839
 - defun, 839
- clearCmdCompletely, 838
 - calledby clearSpad2Cmd, 835
 - calls clearClams, 838
 - calls clearCmdAll, 838
 - calls clearConstructorCaches, 838
 - calls reclaim, 838
 - calls sayKeyedMsg, 838
 - uses \$CatOfCatDatabase, 838
 - uses \$DomOfCatDatabase, 838
 - uses \$JoinOfCatDatabase, 838
 - uses \$JoinOfDomDatabase, 838
 - uses \$attributeDb, 838
 - uses \$existingFiles, 838
 - uses \$functionTable, 838
 - uses \$localExposureDataDefault, 838
 - uses \$localExposureData, 838
 - uses \$xdatabase, 838
 - defun, 838
- clearCmdExcept, 840
 - calledby clearSpad2Cmd, 835
 - calls clearCmdParts, 840
 - calls object2String, 840
 - calls stringPrefix?, 840
 - uses \$clearOptions, 840
 - defun, 840
- clearCmdParts, 841
 - calledby clearCmdExcept, 840
 - calledby clearSpad2Cmd, 835
 - calledby importFromFrame, 34
 - calls assocleft, 841
 - calls assoc, 841
 - calls boot-equal, 841
 - calls clearDependencies, 841
 - calls clearParserMacro, 841
 - calls deleteAssoc, 841
 - calls exit, 841
 - calls fixObjectForPrinting, 841
 - calls getInterpMacroNames, 841
 - calls getParserMacroNames, 841
 - calls get, 841
 - calls isMap, 841
 - calls member, 841
 - calls modes, 841
 - calls pname, 841
 - calls recordNewValue, 841
 - calls recordOldValue, 841
 - calls remdup, 841
 - calls sayKeyedMsg, 841
 - calls sayMessage, 841
 - calls selectOptionLC, 841
 - calls seq, 841
 - calls types, 841
 - calls untraceMapSubNames, 841
 - calls values, 841
 - uses \$InteractiveFrame, 841
 - uses \$clearOptions, 841
 - uses \$e, 841
 - defun, 841
- clearCmdSortedCaches, 837
 - calledby clearCmdAll, 839
 - calledby clearSpad2Cmd, 835
 - calledby restoreHistory, 907
 - calls compiledLookupCheck, 837
 - calls spadcall, 837
 - uses \$ConstructorCache, 837
 - uses \$Void, 837
 - uses \$lookupDefaults, 837
 - defun, 837

- clearConstructorCache
 - calledby loadLibNoUpdate, 1223
 - calledby loadLib, 1221
- clearConstructorCaches
 - calledby clearCmdCompletely, 838
- clearDependencies
 - calledby clearCmdParts, 841
- clearDependentMaps
 - calledby coerceInteractive, 740
- clearFrame, 1128
 - calls clearCmdAll, 1128
 - uses \$frameRecord, 1128
 - uses \$previousBindings, 1128
 - defun, 1128
- clearMacroTable, 840
 - calledby clearCmdAll, 839
 - uses \$pfMacros, 840
 - defun, 840
- clearParserMacro, 793
 - calledby clearCmdParts, 841
 - calls assoc, 793
 - calls ifcdr, 793
 - calls remalist, 793
 - uses \$pfMacros, 793
 - defun, 793
- clearSpad2Cmd, 835
 - calledby clear, 835
 - calledby historySpad2Cmd, 890
 - calledby restoreHistory, 907
 - calls clearCmdAll, 835
 - calls clearCmdCompletely, 835
 - calls clearCmdExcept, 835
 - calls clearCmdParts, 835
 - calls clearCmdSortedCaches, 835
 - calls sayKeyedMsg, 835
 - calls selectOptionLC, 835
 - calls updateCurrentInterpreterFrame, 835
 - uses \$clearExcept, 835
 - uses \$clearOptions, 835
 - uses \$options, 835
 - defun, 835
- clef, 678
 - syntax, 678
- clngamma, 1278
 - calledby lnggamma, 1256
 - calls c-to-s, 1278
 - calls lnggamma, 1278
 - calls s-to-c, 1278
 - defun, 1278
- clngammacase1, 1248
 - calledby clngammaImpl, 1248
 - calls PiMinusLogSinPi, 1248
 - calls clngammaImpl, 1248
 - defun, 1248
- clngammacase2, 1250
 - calledby clngammacase23, 1250
 - calls cgammaAdjust, 1250
 - calls cgammaBernsum, 1250
 - calls logS, 1250
 - defun, 1250
- clngammacase23, 1250
 - calledby clngammaImpl, 1248
 - calls cgamma, 1250
 - calls clngammacase2, 1250
 - calls clngammacase3, 1250
 - defun, 1250
- clngammacase3, 1251
 - calledby clngammacase23, 1250
 - calls cgammaBernsum, 1251
 - defun, 1251
- clngammaImpl, 1248
 - calledby cgammaImpl, 1248
 - calledby clngammacase1, 1248
 - calls clngammacase1, 1248
 - calls clngammacase23, 1248
 - defun, 1248
- close, 844, 845
 - calls closeInterpreterFrame, 845
 - calls queryClients, 845
 - calls queryUserKeyedMsg, 845
 - calls selectOptionLC, 845
 - calls sockSendInt, 845
 - calls string2id-n, 845
 - calls throwKeyedMsg, 845
 - calls upcase, 845
 - uses \$CloseClient, 845
 - uses \$SessionManager, 845
 - uses \$SpadServer, 845
 - uses \$currentFrameNum, 845
 - uses \$options, 845
 - defun, 845
 - manpage, 844

- closeInterpreterFrame, 37
 - calledby close, 845
 - calledby frameSpad2Cmd, 26
 - calls \$erase, 37
 - calls frameName, 37
 - calls makeHistFileName, 37
 - calls throwKeyedMsg, 37
 - calls updateFromCurrentInterpreterFrame, coerceConvertMmSelection, 37
 - uses \$interpreterFrameName, 37
 - uses \$interpreterFrameRing, 37
 - defun, 37
- clrhash
 - calledby processInteractive, 342
- cmpnote, 319
 - defun, 319
- cnstructSubst
 - calledby hasSig, 721
- coerceBranch2Union, 766
 - calls keyedSystemError, 766
 - calls mkObjWrap, 766
 - calls mkObj, 766
 - calls mkPredList, 766
 - calls objMode, 766
 - calls objVal, 766
 - calls orderUnionEntries, 766
 - calls position, 766
 - calls removeQuote, 766
 - calls stripUnionTags, 766
 - calls unwrap, 766
 - defun, 766
- coerceByFunction, 748
 - calledby coerceInt1, 743
 - calledby coerceIntTableOrFunction, 759
 - defun, 748
- coerceByTable, 759
 - calledby coerceIntTableOrFunction, 759
 - calls isTotalCoerce, 759
 - calls isWrapped, 759
 - calls mkObjWrap, 759
 - calls mkObj, 759
 - calls mkq, 759
 - calls unwrap, 759
 - uses \$OutputForm, 760
 - uses \$coerceFailure, 760
 - catches, 760
- defun, 759
- coerceCommuteTest, 758
 - calledby coerceIntCommute, 757
 - calls deconstructT, 758
 - calls isLegitimateMode, 758
 - calls underDomainOf, 758
 - defun, 758
- coerceConvertMmSelection, 752
 - calledby coerceIntTest, 752
 - calls selectMms1, 752
 - uses \$coerceConvertMmSelection;AL, 753
 - uses \$declaredMode, 753
 - uses \$reportBottomUpFlag, 752
 - defun, 752
- coerceConvertMmSelection;AL, 752
 - defvar, 752
- coerceImmediateSubDomain, 769
 - calledby coerceSubDomain, 769
 - calls getSubDomainPredicate, 769
 - defun, 769
- coerceInt, 741
 - calledby coerceInt0, 742
 - calledby coerceInt1, 743
 - calledby coerceIntPermute, 754
 - calledby coerceIntSpecial, 761
 - calledby coerceIntX, 768
 - calledby coerceInt, 741
 - calledby retract2Specialization, 772
 - calledby retractUnderDomain, 777
 - calledby valueArgsEqual?, 764
 - calls coerceInt1, 741
 - calls coerceInt, 741
 - calls getMinimalVarMode, 741
 - calls objMode, 741
 - calls objVal, 741
 - calls unwrap, 741
 - defun, 741
- coerceInt0, 742
 - calledby coerceInt0, 742
 - calledby coerceInteractive, 740
 - calls coerceInt0, 742
 - calls coerceInt, 742
 - calls conCoerceFrom, 742
 - calls intCodeGenCOERCE, 742
 - calls isWrapped, 742
 - calls mkObj, 742

- calls objMode, 742
- calls objSetMode, 742
- calls objVal, 742
- calls unwrap, 742
- uses \$Any, 742
- uses \$OutputForm, 742
- uses \$genValue, 742
- defun, 742
- coerceInt1, 743
 - calledby coerceInt1, 743
 - calledby coerceInt, 741
 - calls NRTcompileEvalForm, 743
 - calls absolutelyCanCoerceByCheating, 743
 - calls asTupleAsList, 743
 - calls bottomUp, 743
 - calls coerceByFunction, 743
 - calls coerceInt1, 743
 - calls coerceInt2Union, 743
 - calls coerceIntAlgebraicConstant, 743
 - calls coerceIntFromUnion, 743
 - calls coerceIntTower, 743
 - calls coerceIntX, 743
 - calls coerceInt, 743
 - calls coerceRetract, 743
 - calls coerceSubDomain, 743
 - calls compareTypeLists, 743
 - calls deconstructT, 743
 - calls evalDomain, 743
 - calls getFunctionFromDomain, 743
 - calls getValue, 743
 - calls isEqualOrSubDomain, 743
 - calls isSubDomain, 743
 - calls mkAtreeNode, 743
 - calls mkAtree, 743
 - calls mkObjWrap, 743
 - calls mkObj, 743
 - calls nequal, 743
 - calls nreverse0, 744
 - calls objMode, 744
 - calls objVal, 744
 - calls selectLocalMms, 744
 - calls selectMms1, 744
 - calls transferPropsToNode, 744
 - calls unwrap, 744
 - uses \$AnonymousFunction, 744
 - uses \$Any, 744
 - uses \$EmptyMode, 744
 - uses \$Integer, 744
 - uses \$NonNegativeInteger, 744
 - uses \$OutputForm, 744
 - uses \$PositiveInteger, 744
 - uses \$QuotientField, 744
 - uses \$SingleInteger, 744
 - uses \$String, 744
 - uses \$Symbol, 744
 - uses \$Void, 744
 - uses \$e, 744
 - uses \$genValue, 744
 - uses \$useCoerceOrCroak, 744
 - catches, 744
 - defun, 743
- coerceInt2Union, 765
 - calledby coerceInt1, 743
 - defun, 765
- coerceIntAlgebraicConstant, 766
 - calledby coerceInt1, 743
 - calls getConstantFromDomain, 767
 - calls mkObjWrap, 767
 - calls objMode, 766
 - calls objValUnwrap, 766
 - calls ofCategory, 767
 - defun, 766
- coerceIntByMap, 761
 - calledby coerceIntTower, 750
 - calls compiledLookup, 761
 - calls deconstructT, 761
 - calls evalDomain, 761
 - calls isSubDomain, 761
 - calls length, 761
 - calls member, 761
 - calls mkObjWrap, 761
 - calls nequal, 761
 - calls objMode, 761
 - calls objVal, 761
 - calls sayFunctionSelectionResult, 761
 - calls sayFunctionSelection, 761
 - calls selectMms1, 761
 - calls timedEvaluate, 761
 - calls underDomainOf, 761
 - calls valueArgsEqual?, 761
 - calls wrapped2Quote, 761
 - catches, 761

- defun, 761
- coerceIntByMapInner, 763
 - calls coerceOrThrowFailure, 763
 - defun, 763
- coerceIntCommute, 757
 - calledby coerceIntTower, 750
 - calls coerceCommuteTest, 757
 - calls concat, 757
 - calls getl, 757
 - calls mkObjWrap, 757
 - calls objMode, 757
 - calls objValUnwrap, 757
 - calls underDomainOf, 757
 - uses \$coerceFailure, 757
 - catches, 757
 - defun, 757
- coerceInteractive, 740
 - calledby coerceOrRetract, 771
 - calledby coerceSpadArgs2E, 128
 - calledby coerceSpadFunValue2E, 130
 - calledby coerceTraceArgs2E, 127
 - calledby coerceTraceFunValue2E, 129
 - calledby interpret2, 348
 - calls clearDependentMaps, 740
 - calls coerceInt0, 740
 - calls mkObjWrap, 740
 - calls mkObj, 740
 - calls objMode, 740
 - calls objVal, 740
 - calls startTimingProcess, 740
 - calls stopTimingProcess, 740
 - calls throwKeyedMsg, 740
 - uses \$EmptyMode, 740
 - uses \$NoValueMode, 740
 - uses \$OutputForm, 740
 - uses \$compilingMap, 740
 - uses \$insideCoerceInteractive, 740
 - uses \$mapName, 740
 - defun, 740
- coerceIntFromUnion, 765
 - calledby coerceInt1, 743
 - defun, 765
- coerceIntPermute, 754
 - calledby coerceIntTower, 750
 - calls coerceInt, 754
 - calls computeTTTranspositions, 754
 - calls member, 754
 - calls objMode, 754
 - defun, 754
- coerceIntSpecial, 760
 - calledby coerceIntTower, 750
 - calls coerceInt, 761
 - calls objMode, 760
 - defun, 760
- coerceIntTableOrFunction, 759
 - calledby coerceIntTower, 750
 - calls assq, 759
 - calls coerceByFunction, 759
 - calls coerceByTable, 759
 - calls isLegitimateMode, 759
 - calls isValidType, 759
 - calls objMode, 759
 - calls objVal, 759
 - uses \$CoerceTable, 759
 - defun, 759
- coerceIntTest, 752
 - calledby coerceIntTower, 750
 - calls assq, 752
 - calls coerceConvertMmSelection, 752
 - uses \$CoerceTable, 752
 - uses \$useConvertForCoercions, 752
 - defun, 752
- coerceIntTower, 750
 - calledby coerceInt1, 743
 - calls bubbleConstructor, 750
 - calls coerceIntByMap, 750
 - calls coerceIntCommute, 750
 - calls coerceIntPermute, 750
 - calls coerceIntSpecial, 750
 - calls coerceIntTableOrFunction, 750
 - calls coerceIntTest, 750
 - calls constructT, 750
 - calls deconstructT, 750
 - calls isValidType, 750
 - calls last, 750
 - calls replaceLast, 750
 - defun, 750
- coerceIntX, 768
 - calledby coerceInt1, 743
 - calls coerceInt, 768
 - calls mkObjWrap, 768
 - calls underDomainOf, 768

- calls unwrap, 768
- defun, 768
- coerceOrRetract, 771
 - calledby coerceOrThrowFailure, 763
 - calledby evaluateType1, 1120
 - calls coerceInteractive, 771
 - calls retract, 771
 - defun, 771
- coerceOrThrowFailure, 763
 - calledby coerceIntByMapInner, 763
 - calls coerceOrRetract, 763
 - calls coercionFailure, 763
 - calls mkObjWrap, 763
 - calls objValUnwrap, 763
 - defun, 763
- coerceRetract, 777
 - calledby coerceInt1, 743
 - calls canFuncall?, 777
 - calls getl, 777
 - calls isEqualOrSubDomain, 777
 - calls mkObjWrap, 777
 - calls objMode, 777
 - calls objValUnwrap, 777
 - calls retractByFunction, 777
 - uses \$Integer, 777
 - uses \$OutputForm, 777
 - uses \$SingleInteger, 777
 - uses \$Symbol, 777
 - uses \$coerceFailure, 777
 - catches, 777
 - defun, 777
- coerceSpadArgs2E, 128
 - calledby coerceTraceArgs2E, 127
 - calls coerceInteractive, 128
 - calls exit, 128
 - calls mkObjWrap, 128
 - calls objValUnwrap, 128
 - calls seq, 128
 - uses \$OutputForm, 129
 - uses \$streamCount, 128
 - uses \$tracedSpadModemap, 129
 - defun, 128
- coerceSpadFunValue2E, 130
 - calledby coerceTraceFunValue2E, 129
 - calls coerceInteractive, 130
 - calls mkObjWrap, 130
- calls objValUnwrap, 130
 - uses \$OutputForm, 130
 - uses \$streamCount, 130
 - uses \$tracedSpadModemap, 130
 - defun, 130
- coerceSubDomain, 769
 - calledby coerceInt1, 743
 - calledby coerceSubDomain, 769
 - calls coerceImmediateSubDomain, 769
 - calls coerceSubDomain, 769
 - calls getdatabase, 769
 - defun, 769
- coerceTraceArgs2E, 127
 - calledby monitorEnter, 147
 - calls coerceInteractive, 127
 - calls coerceSpadArgs2E, 127
 - calls mkObjWrap, 127
 - calls objValUnwrap, 127
 - calls pname, 127
 - calls spadsysnamep, 127
 - uses \$OutputForm, 127
 - uses \$mathTraceList, 127
 - uses \$tracedMapSignatures, 128
 - defun, 127
- coerceTraceFunValue2E, 129
 - calledby monitorExit, 151
 - calls coerceInteractive, 129
 - calls coerceSpadFunValue2E, 129
 - calls lassoc, 129
 - calls mkObjWrap, 129
 - calls objValUnwrap, 129
 - calls pname, 129
 - calls spadsysnamep, 129
 - uses \$OutputForm, 130
 - uses \$mathTraceList, 130
 - uses \$tracedMapSignatures, 130
 - defun, 129
- coerceUnion2Branch, 775
 - calledby retract2Specialization, 772
 - calledby retractByFunction, 778
 - calls evalSharpOne, 775
 - calls mkObj, 775
 - calls mkPredList, 775
 - calls objMode, 775
 - calls objValUnwrap, 775
 - calls objVal, 775

- calls orderUnionEntries, 775
 - calls stripUnionTags, 775
 - defun, 775
- coercionFailure, 763
 - calledby coerceOrThrowFailure, 763
 - defun, 763
 - throws, 763
- collection, 679
 - syntax, 679
- commandAmbiguityError, 792
 - calledby commandErrorIfAmbiguous, 810
 - calledby commandErrorMessage, 789
 - calledby traceOptionError, 121
 - calledby userLevelErrorMessage, 790
 - calls bright, 792
 - calls sayKeyedMsg, 792
 - calls sayMSG, 792
 - calls terminateSystemCommand, 792
 - defun, 792
- commandError, 789
 - calls commandErrorMessage, 789
 - defun, 789
- commandErrorIfAmbiguous, 810
 - calls commandAmbiguityError, 810
 - uses \$oldline, 810
 - uses line, 810
 - defun, 810
- commandErrorMessage, 789
 - calledby commandError, 789
 - calledby optionError, 789
 - calls commandAmbiguityError, 789
 - calls sayKeyedMsg, 790
 - calls terminateSystemCommand, 790
 - uses \$oldline, 790
 - uses line, 790
 - defun, 789
- commandsForUserLevel, 788
 - calledby synonymsForUserLevel, 1106
 - calledby systemCommand, 788
 - calledby unAbbreviateKeyword, 809
 - calledby whatCommands, 1133
 - calls satisfiesUserLevel, 788
 - defun, 788
- commandUserLevelError, 790
 - calls userLevelErrorMessage, 790
 - defun, 790
- compareposns, 660
 - calledby erMsgCompare, 660
 - calls poCharPosn, 660
 - calls poGlobalLinePosn, 660
 - defun, 660
- compareTypeLists, 768
 - calledby coerceInt1, 743
 - defun, 768
- compFailure
 - calledby getAndEvalConstructorArgument, 1144
- compile, 847
 - manpage, 847
- compileBoot, 115
 - calledby breaklet, 155
 - calledby tracelet, 154
 - calls /D,1, 115
 - defun, 115
- compiledLookup, 1225
 - calledby coerceIntByMap, 761
 - calledby compiledLookupCheck, 837
 - calledby hashable, 1316
 - calledby retractByFunction, 778
 - calls NRTevalDomain, 1225
 - calls isDomain, 1225
 - defun, 1225
- compiledLookupCheck, 837
 - calledby algEqual, 765
 - calledby clearCmdSortedCaches, 837
 - calledby getConstantFromDomain, 767
 - calls compiledLookup, 837
 - calls formatSignature, 837
 - calls keyedSystemError, 837
 - defun, 837
- computeDomainVariableAlist, 1529
 - defun, 1529
- computeTTTranspositions, 754
 - calledby coerceIntPermute, 754
 - calls decomposeTypeIntoTower, 754
 - calls length, 754
 - calls list2vec, 754
 - calls member, 754
 - calls msort, 754
 - calls nequal, 754
 - calls permuteToOrder, 754
 - calls reassembleTowerIntoType, 754

- calls remdup, 754
- calls vec2list, 754
- defun, 754
- concat, 1237
 - calledby /untrace-2, 125
 - calledby bcComplexLimitGen, 1480
 - calledby bcDefiniteIntegrateGen, 1411
 - calledby bcDifferentiateGen, 1407
 - calledby bcDraw2DSolveGen, 1421
 - calledby bcDraw2DfunGen, 1417
 - calledby bcDraw2DparGen, 1419
 - calledby bcDraw3DfunGen, 1424
 - calledby bcDraw3Dpar1Gen, 1428
 - calledby bcDraw3DparGen, 1426
 - calledby bcDrawIt, 1496
 - calledby bcGenEquations, 1492
 - calledby bcGen, 1468, 1494
 - calledby bcIndefiniteIntegrateGen, 1409
 - calledby bcInputEquations, 1481
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormulaGen, 1471
 - calledby bcLinearMatrixGen, 1490
 - calledby bcLinearSolveMatrixInhomo, 1488
 - calledby bcMakeEquations, 1483
 - calledby bcMakeLinearEquations, 1484
 - calledby bcMatrixGen, 1472
 - calledby bcProductGen, 1474
 - calledby bcRealLimitGen1, 1478
 - calledby bcSeriesExpansionGen, 1431
 - calledby bcSeriesGen, 1435
 - calledby bcSolveEquations, 1486
 - calledby bcSumGen, 1414
 - calledby bcwords2liststring, 1495
 - calledby buildHtMacroTable, 1401
 - calledby coerceIntCommute, 757
 - calledby conOpPage1, 1629
 - calledby conSpecialString?, 1598
 - calledby copyright, 855
 - calledby dbWordFrom, 1592
 - calledby dewritify,dewritifyInner, 922
 - calledby displayCondition, 804
 - calledby displayMacro, 794
 - calledby displayModemap, 806
 - calledby displayMode, 806
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby displayType, 800
 - calledby displayValue, 799
 - calledby doSystemCommand, 786
 - calledby editFile, 873
 - calledby evalDomain, 1115
 - calledby fixObjectForPrinting, 796
 - calledby getTraceOption, 118
 - calledby handleNoParseCommands, 810
 - calledby htMkName, 1497
 - calledby htStringPad, 1497
 - calledby incLude1, 374
 - calledby inclmsgIfSyntax, 390
 - calledby intloopReadConsole, 321
 - calledby kDomainName, 1624
 - calledby kPage, 1593
 - calledby kcodePage, 1620
 - calledby kcnPage, 1622
 - calledby kcuPage, 1621
 - calledby kePage, 1605
 - calledby koPageInputAreaUnchanged?, 1623
 - calledby koPage, 1631
 - calledby lfloat, 419
 - calledby lfrinteger, 425
 - calledby license, 934
 - calledby makeMsgFromLine, 661
 - calledby mkConform, 1628
 - calledby mkprompt, 334
 - calledby ncloopIncFileName, 929
 - calledby newHelpSpad2Cmd, 881
 - calledby pcounters, 99
 - calledby printLabelledList, 814
 - calledby processChPosesForOneLine, 666
 - calledby processSynonyms, 325
 - calledby pspacers, 98
 - calledby ptimers, 98
 - calledby removeUndoLines, 57
 - calledby reportOpsFromLisplib, 1092
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby reportUndo, 60
 - calledby resetCounters, 98
 - calledby resetSpacers, 97
 - calledby resetTimers, 97
 - calledby scanExponent, 418
 - calledby scanNumber, 423
 - calledby scanS, 422
 - calledby scanW, 419

- calledby setOutputAlgebra, 1035
- calledby setOutputCharacters, 1039
- calledby setOutputFormula, 1063
- calledby setOutputFortran, 1042
- calledby setOutputHtml, 1048
- calledby setOutputMathml, 1054
- calledby setOutputOpenMath, 1058
- calledby setOutputTex, 1070
- calledby showHistory, 892
- calledby splell, 414
- calledby summary, 1103
- calledby traceDomainConstructor, 137
- calledby traceReply, 94
- calledby undoCount, 61
- calledby untraceDomainConstructor, 139
- calledby whatCommands, 1133
- calledby writeInputLines, 896
- calledby xlskip, 382
- calls string-concatenate, 1237
- defun, 1237
- concatWithBlanks, 1590
 - defun, 1590
- conCoerceFrom
 - calledby coerceInt0, 742
- condErrorMsg, 1541
 - defun, 1541
- conLowerCaseConTran, 1590
 - calledby conLowerCaseConTran, 1590
 - calledby conSpecialString?, 1598
 - calls conLowerCaseConTran, 1590
 - calls hget, 1590
 - calls ifcar, 1590
 - uses \$lowerCaseConTb, 1590
 - defun, 1590
- conOpPage, 1628
 - calls conOpPage1, 1628
 - calls dbCompositeWithMap, 1628
 - calls dbExtractUnderlyingDomain, 1628
 - calls httpProperty, 1628
 - defun, 1628
- conOpPage1, 1629
 - calledby conOpPage, 1628
 - calls captialize, 1629
 - calls conPageFastPath, 1629
 - calls concat, 1629
 - calls dbSourceFile, 1629
 - calls dbSpecialOperations, 1629
 - calls dbXParts, 1629
 - calls htInitPage, 1629
 - calls httpSetProperty, 1629
 - calls ifcar, 1629
 - calls ifcdr, 1629
 - calls isExposedConstructor, 1629
 - calls koPage, 1629
 - calls lassoc, 1629
 - calls mkConform, 1629
 - calls ncParseFromString, 1629
 - calls opOf, 1629
 - uses \$Primitives, 1629
 - defun, 1629
- conPage, 1599
 - calledby constructorSearch, 1597
 - calledby dbSelectCon, 1648
 - calledby dbShowCons1, 1643
 - calls conPageFastPath, 1599
 - calls downcase, 1599
 - calls downlink, 1599
 - calls form2HtString, 1599
 - calls kPage, 1599
 - calls lassq, 1599
 - calls ySearch, 1599
 - uses \$conArgstrings, 1599
 - defun, 1599
- conPageChoose, 1642
 - calls dbShowCons1, 1642
 - calls getConstructorForm, 1642
 - defun, 1642
- conPageConEntry, 1600
 - calledby conPageFastPath, 1600
 - calls buildLLbdbConEntry[9], 1600
 - uses \$conform, 1600
 - uses \$conname, 1600
 - uses \$doc, 1600
 - uses \$exposed?, 1600
 - uses \$kind, 1600
 - defun, 1600
- conPageFastPath, 1600
 - calledby conOpPage1, 1629
 - calledby conPage, 1599
 - calls charPosition, 1600
 - calls conPageConEntry, 1600
 - calls dbRead, 1600

- calls lassq, 1600
 - calls length, 1600
 - uses \$lowerCaseConTb, 1600
 - defun, 1600
- consoleinputp
 - calledby spad-syntax-error, 1155
- conSpecialString?, 1598
 - calledby conSpecialString?, 1598
 - calledby constructorSearch, 1597
 - calledby kArgumentCheck, 1625
 - calls conLowerCaseConTran, 1598
 - calls conSpecialString?, 1598
 - calls concat, 1598
 - calls contained, 1598
 - calls dbString2Words, 1598
 - calls ifcar, 1598
 - calls kisValidType, 1598
 - calls member, 1598
 - calls ncParseFromString, 1598
 - calls string2Constructor, 1598
 - calls string2Words, 1598
 - defun, 1598
- constoken, 405
 - calledby lineoftoks, 403
 - calledby scanToken, 406
 - calls ncPutQ, 405
 - defun, 405
- constructor?
 - calledby addTraceItem, 146
 - calledby dbShowCons, 1641
 - calledby evaluateType1, 1120
 - calledby evaluateType, 1119
 - calledby hasSig, 721
 - calledby mkEvalable, 1116
 - calledby reportOpsFromLisplib, 1092
 - calledby transTraceItem, 126
 - calledby unifyStructVar, 727
 - calledby updateDatabase, 1203
 - calledby writify,writifyInner, 917
- constructorSearch, 1597
 - calledby cSearch, 1595
 - calledby dSearch, 1595
 - calledby kSearch, 1596
 - calledby pSearch, 1595
 - calledby ySearch, 1596
 - calls conPage, 1597
 - calls conSpecialString?, 1597
 - calls constructorSearchGrep, 1597
 - calls dbKind, 1597
 - calls dbName, 1597
 - calls downcase, 1597
 - calls downlink, 1597
 - calls grepSearchQuery, 1597
 - calls htInitPage, 1597
 - calls htQuery, 1597
 - calls htShowPage, 1597
 - calls httpSetProperty, 1597
 - calls kPage, 1597
 - calls lassoc, 1597
 - uses \$lowerCaseConTb, 1597
 - defun, 1597
- constructorSearchGrep
 - calledby constructorSearch, 1597
- constructSubst, 732
 - calledby hasAtt, 722
 - calledby hasCaty, 718
 - calledby spadTrace, 132
 - calls internl, 732
 - defun, 732
- constructT
 - calledby coerceIntTower, 750
 - calledby retractUnderDomain, 777
- contained
 - calledby conSpecialString?, 1598
 - calledby isPartialMode, 718
 - calledby unifyStructVar, 727
- containsVariables
 - calledby hasCate, 731
- containsVars, 729
 - calledby hasCateSpecial, 733
 - calledby unifyStructVar, 727
 - calls containsVars1, 729
 - calls isPatternVar, 729
 - defun, 729
- containsVars1, 729
 - calledby containsVars1, 729
 - calledby containsVars, 729
 - calls containsVars1, 729
 - calls isPatternVar, 729
 - defun, 729
- copy
 - calledby makeInitialModemapFrame, 328

- calledby resetWorkspaceVariables, 962
- calledby undoSteps, 54
- calledby untrace, 122
- copyright, 850, 855
 - calls concat, 855
 - calls getenv, 855
 - calls obey, 855
 - defun, 855
 - manpage, 850
- cot, 1290
 - defun, 1290
- cotdiffeval, 1257
 - calledby rPsiImpl, 1257
 - defun, 1257
- coth, 1292
 - defun, 1292
- countCache, 980
 - calledby setFunctionsCache, 979
 - calls identp, 980
 - calls insertAlist, 980
 - calls internl, 980
 - calls optionError, 980
 - calls qcar, 980
 - calls qcdr, 980
 - calls sayCacheCount, 980
 - calls sayKeyedMsg, 980
 - uses \$cacheAlist, 980
 - uses \$cacheCount, 981
 - uses \$options, 980
 - defun, 980
- cPsi
 - calledby cpsi, 1279
- cpsi, 1279
 - calls c-to-s, 1279
 - calls cPsi, 1279
 - calls s-to-c, 1279
 - defun, 1279
- cPsiImpl, 1262
 - defun, 1262
- createCurrentInterpreterFrame, 29
 - calledby updateCurrentInterpreterFrame, 31
 - uses \$HiFiAccess, 29
 - uses \$HistListAct, 29
 - uses \$HistListLen, 29
 - uses \$HistList, 29
 - uses \$HistRecord, 29
 - uses \$IOindex, 29
 - uses \$InteractiveFrame, 29
 - uses \$internalHistoryTable, 29
 - uses \$interpreterFrameName, 29
 - uses \$localExposureData, 29
 - defun, 29
- creditlist, 195
 - defvar, 195
- credits, 856
 - usedby credits, 856
 - uses credits, 856
 - defun, 856
 - manpage, 856
- csc, 1292
 - defun, 1292
- csch, 1292
 - defun, 1292
- cSearch, 1595
 - calls constructorSearch, 1595
 - defun, 1595
- curinstream, 313
 - usedby ncIntLoop, 317
 - defvar, 313
- curoutstream, 313
 - usedby ncIntLoop, 317
 - defvar, 313
- current-fragment
 - usedby ioclear, 1158
- current-line
 - usedby ioclear, 1158
- current-token
 - usedby token-stack-show, 1157
- currenttime
 - calledby mkprompt, 334
- DaaseName, 1209
 - calledby interopen, 1190
 - calls getenv, 1209
 - uses \$spadroot, 1209
 - defun, 1209
- dalymode, 976
 - defvar, 976
- dancols, 1295
 - defmacro, 1295
- danrows, 1295

- defmacro, 1295
- daref2, 1294
 - defmacro, 1294
- database, 1182
 - defstruct, 1182
- dbAddChain, 1641
 - calledby dbAddChain, 1641
 - calledby dbSearchOrder, 1610
 - calls dbAddChainDomain, 1641
 - calls dbAddChain, 1641
 - defun, 1641
- dbAddChainDomain, 1640
 - calledby dbAddChain, 1641
 - calls dbInfovec, 1640
 - calls dbSubConform, 1640
 - calls devaluate, 1640
 - calls kFormatSlotDomain, 1640
 - uses \$infovec, 1640
 - defun, 1640
- dbAddDocTable, 1636
 - calledby dbDocTable, 1635
 - calls getConstructorForm, 1637
 - calls getdatabase, 1637
 - calls hget, 1637
 - calls hput, 1637
 - calls opOf, 1636
 - calls sublislis, 1637
 - uses \$docTable, 1637
 - defun, 1636
- dbCompositeWithMap, 1630
 - calledby conOpPage, 1628
 - calls dbExtractUnderlyingDomain, 1630
 - calls httpProperty, 1630
 - defun, 1630
- dbConformGenUnder
 - calledby kPage, 1593
- dbConsExposureMessage, 1645
 - calledby dbShowCons1, 1643
 - calls htSay, 1645
 - uses \$atLeastOneUnexposed, 1645
 - defun, 1645
- dbConsHeading, 1649
 - calledby dbShowCons1, 1643
 - calls capitalize, 1649
 - calls form2HtString, 1649
 - calls httpProperty, 1649
 - calls length, 1649
 - calls member, 1649
 - calls nequal, 1649
 - calls pluralize, 1649
 - calls remdup, 1649
 - uses \$exposedOnlyIfTrue, 1649
 - defun, 1649
- dbConstructorDoc, 1635
 - calledby dbGetDocTable, 1639
 - calls dbConstructorDoc,fn, 1635
 - uses \$op, 1635
 - uses \$sig, 1635
 - defun, 1635
- dbConstructorDoc,fn
 - calledby dbConstructorDoc, 1635
- dbConstructorDoc,gn, 1635
 - calls dbConstructorDoc,hn, 1635
 - uses \$op, 1635
 - defun, 1635
- dbConstructorDoc,hn, 1634
 - calledby dbConstructorDoc,gn, 1635
 - calls length, 1634
 - calls sublislis, 1634
 - uses \$FormalMapVariableList, 1634
 - uses \$args, 1634
 - uses \$sig, 1634
 - defun, 1634
- dbConstructorKind
 - calledby dbShowCons1, 1643
- dbDelimiters, 1591
 - defvar, 1591
- dbDocTable, 1635
 - calls dbAddDocTable, 1635
 - calls hget, 1635
 - calls make-hashtable, 1635
 - calls originsInOrder, 1635
 - uses \$docTableHash, 1635
 - uses \$docTable, 1635
 - defun, 1635
- dbExtractUnderlyingDomain, 1631
 - calledby conOpPage, 1628
 - calledby dbCompositeWithMap, 1630
 - calls isValidType, 1631
 - defun, 1631
- dbGatherData
 - calledby kePageDisplay, 1608

- dbGetDocTable, 1638
 - calls dbConstructorDoc, 1639
 - calls dbGetDocTable,gn, 1639
 - calls hget, 1639
 - calls qcdr, 1639
 - calls string2Integer, 1638
 - uses \$conform, 1639
 - uses \$op, 1639
 - uses \$sig, 1639
 - uses \$which, 1639
 - defun, 1638
- dbGetDocTable,gn, 1638
 - calledby dbGetDocTable, 1639
 - calls dbGetDocTable,hn, 1638
 - calls lastatom, 1638
 - uses \$conform, 1638
 - defun, 1638
- dbGetDocTable,hn, 1637
 - calledby dbGetDocTable,gn, 1638
 - calls qcar, 1637
 - calls qcdr, 1637
 - calls sublislis, 1637
 - uses \$FormalMapVariableList, 1637
 - uses \$conform, 1637
 - uses \$sig, 1637
 - uses \$which, 1637
 - defun, 1637
- dbGetInputString
 - calledby dbShowCons, 1641
 - calledby koaPageFilterByName, 1634
- dbInfovec
 - calledby dbAddChainDomain, 1640
 - calledby dbSearchOrder, 1610
- dbKind
 - calledby constructorSearch, 1597
- dbMkEvalable, 1625
 - calledby kDomainName, 1624
 - calls getdatabase, 1625
 - calls mkEvalable, 1625
 - defun, 1625
- dbMkForm, 1658
 - defun, 1658
- dbName
 - calledby constructorSearch, 1597
 - calledby dbShowConstructorLines, 1650
- dbNonEmptyPattern, 1571
 - defun, 1571
- dbPart
 - calledby alqlGetKindString, 1305
 - calledby alqlGetOrigin, 1304
 - calledby alqlGetParams, 1305
- dbpHasDefaultCategory?
 - calledby kcPage, 1611
- dbPresentCons
 - calledby dbShowCons1, 1643
- dbRead, 1596
 - calledby conPageFastPath, 1600
 - defun, 1596
- dbSayItems
 - calledby dbShowConditions, 1648
- dbSearchOrder, 1610
 - calledby ksPage, 1609
 - calls dbAddChain, 1610
 - calls dbInfovec, 1610
 - calls dbSubConform, 1610
 - calls devaluate, 1610
 - calls getdatabase, 1610
 - calls kFormatSlotDomain, 1610
 - calls kTestPred, 1610
 - calls opOf, 1610
 - calls simpCatPredicate, 1610
 - calls sublislis, 1610
 - uses \$domain, 1610
 - uses \$infovec, 1610
 - uses \$predvec, 1610
 - defun, 1610
- dbSelectCon, 1648
 - calls conPage, 1648
 - calls httpProperty, 1648
 - calls opOf, 1648
 - defun, 1648
- dbShowConditions, 1648
 - calledby dbShowCons1, 1643
 - calls bcConPredTable, 1648
 - calls dbSayItems, 1648
 - calls htSayHrule, 1648
 - calls httpProperty, 1648
 - calls length, 1648
 - calls opOf, 1648
 - calls pluralize, 1648
 - calls splitConTable, 1648
 - defun, 1648

- dbShowCons, 1641
 - calledby dbShowConsKindsFilter, 1645
 - calledby dbShowCons, 1641
 - calledby kArgPage, 1602
 - calledby kcaPage1, 1617
 - calledby kccPage, 1618
 - calledby kcdePage, 1620
 - calledby kcnPage, 1622
 - calledby kcpPage, 1615
 - calledby kcuPage, 1621
 - calledby ksPage, 1609
 - calls bcErrorPage, 1641
 - calls constructor?, 1641
 - calls dbGetInputString, 1641
 - calls dbShowCons1, 1641
 - calls dbShowCons, 1641
 - calls downcase, 1641
 - calls emptySearchPage, 1641
 - calls htCopyProplist, 1641
 - calls htInitPageNoScroll, 1641
 - calls httpProperty, 1641
 - calls httpSetProperty, 1641
 - calls ifcar, 1641
 - calls member, 1641
 - calls pmTransFilter, 1641
 - calls superMatch?, 1641
 - uses \$exposedOnlyIfTrue, 1641
 - defun, 1641
- dbShowCons1, 1643
 - calledby conPageChoose, 1642
 - calledby dbShowConstructorLines, 1650
 - calledby dbShowCons, 1641
 - calls assocleft, 1643
 - calls bcAbbTable, 1643
 - calls bcConTable, 1643
 - calls bcNameConTable, 1643
 - calls bcUnixTable, 1643
 - calls conPage, 1643
 - calls dbConsExposureMessage, 1643
 - calls dbConsHeading, 1643
 - calls dbConstructorKind, 1643
 - calls dbPresentCons, 1643
 - calls dbShowConditions, 1643
 - calls dbShowConsDoc, 1643
 - calls dbShowConsKinds, 1643
 - calls function, 1643
 - calls getCDTEntry, 1643
 - calls getdatabase, 1643
 - calls htCopyProplist, 1643
 - calls htInitPageNoScroll, 1643
 - calls htSayStandard, 1643
 - calls htShowPageNoScroll, 1643
 - calls httpProperty, 1643
 - calls httpSetProperty, 1643
 - calls isExposedConstructor, 1643
 - calls listSort, 1643
 - calls opOf, 1643
 - calls qlesseqp, 1643
 - calls remdup, 1643
 - calls union, 1643
 - uses \$conformsAreDomains, 1643
 - uses \$exposedOnlyIfTrue, 1643
 - defun, 1643
- dbShowConsDoc, 1645
 - calledby dbShowCons1, 1643
 - calls dbShowConsDoc1, 1645
 - calls getConstructorForm, 1645
 - calls httpProperty, 1645
 - calls opOf, 1645
 - calls remdup, 1645
 - calls systemError, 1645
 - defun, 1645
- dbShowConsDoc1, 1646
 - calledby dbShowConsDoc, 1645
 - calledby dbSpecialDescription, 1651
 - calledby kPage, 1593
 - calledby kiPage, 1605
 - calls displayDomainOp, 1646
 - calls getConstructorDocumentation, 1646
 - calls getConstructorSignature, 1646
 - calls getdatabase, 1646
 - calls getl, 1646
 - calls httpProperty, 1646
 - calls isExposedConstructor, 1646
 - calls member, 1646
 - calls sublisFormal, 1646
 - calls sublislis, 1646
 - uses \$Primitives, 1646
 - uses \$TriangleVariableList, 1646
 - defun, 1646
- dbShowConsKinds
 - calledby dbShowCons1, 1643

- dbShowConsKindsFilter, 1645
 - calls dbShowCons, 1645
 - calls httpProperty, 1645
 - calls httpSetProperty, 1645
 - defun, 1645
- dbShowConstructorLines, 1650
 - calls dbName, 1650
 - calls dbShowCons1, 1650
 - calls function, 1650
 - calls getConstructorForm, 1650
 - calls glesseqp, 1650
 - calls intern, 1650
 - calls listSort, 1650
 - defun, 1650
- dbShowOp1
 - calledby dbSpecialOperations, 1652
- dbShowOperationsFromConform
 - calledby koPageAux1, 1633
 - calledby koPageAux, 1633
- dbSourceFile
 - calledby conOpPage1, 1629
 - calledby kPage, 1593
- dbSowOpItems
 - calledby kePageDisplay, 1608
- dbSpecialDescription, 1651
 - calls dbShowConsDoc1, 1651
 - calls form2HtString, 1651
 - calls getConstructorForm, 1651
 - calls htInitPage, 1651
 - calls htShowPage, 1651
 - calls httpSetProperty, 1651
 - uses \$conformsAreDomains, 1651
 - defun, 1651
- dbSpecialExpandIfNecessary, 1653
 - calledby dbSpecialExports, 1652
 - calledby dbSpecialOperations, 1652
 - calls qcadar, 1653
 - calls qcar, 1653
 - calls qcdr, 1653
 - calls qcdr, 1653
 - defun, 1653
- dbSpecialExports, 1652
 - calls dbSpecialExpandIfNecessary, 1652
 - calls form2HtString, 1652
 - calls getConstructorForm, 1652
 - calls getl, 1652
 - calls htInitPage, 1652
 - calls htShowPage, 1652
 - calls kePageDisplay, 1652
 - defun, 1652
- dbSpecialOperations, 1652
 - calledby conOpPage1, 1629
 - calls dbShowOp1, 1652
 - calls dbSpecialExpandIfNecessary, 1652
 - calls form2HtString, 1652
 - calls getConstructorForm, 1652
 - calls getl, 1652
 - calls htInitPage, 1652
 - calls httpSetProperty, 1652
 - defun, 1652
- dbSplitLibdb
 - calledby make-databases, 1205
- dbString2Words, 1591
 - calledby conSpecialString?, 1598
 - calls dbWordFrom, 1591
 - defun, 1591
- dbSubConform, 1640
 - calledby dbAddChainDomain, 1640
 - calledby dbSearchOrder, 1610
 - calledby dbSubConform, 1640
 - calls dbSubConform, 1640
 - calls position, 1640
 - uses \$FormalMapVariableList, 1640
 - defun, 1640
- dbWordFrom, 1592
 - calledby dbString2Words, 1591
 - calls concat, 1592
 - calls maxindex, 1592
 - calls member, 1592
 - uses \$dbDelimiters, 1592
 - defun, 1592
- dbXParts
 - calledby conOpPage1, 1629
 - calledby kPage, 1593
- dc1
 - calledby reportOpsFromLisplib, 1093
- debugmode
 - usedby spad-syntax-error, 1155
- decideHowMuch, 649
 - calledby getPosStL, 646
 - calls poFileName, 649
 - calls poLinePosn, 650

- calls poNopos?, 649
 - calls poPosImmediate?, 649
 - defun, 649
- decomposeTypeIntoTower, 756
 - calledby computeTTTranspositions, 754
 - calledby decomposeTypeIntoTower, 756
 - calls decomposeTypeIntoTower, 756
 - calls deconstructT, 756
 - defun, 756
- deconstructT
 - calledby absolutelyCanCoerceByCheating, 770
 - calledby coerceCommuteTest, 758
 - calledby coerceInt1, 743
 - calledby coerceIntByMap, 761
 - calledby coerceIntTower, 750
 - calledby decomposeTypeIntoTower, 756
 - calledby retractUnderDomain, 777
- defaultTargetFE, 735
 - calledby defaultTargetFE, 736
 - calledby hasCateSpecialNew, 734
 - calls defaultTargetFE, 736
 - calls ifcar, 736
 - calls isEqualOrSubDomain, 735
 - local ref \$FunctionalExpression, 736
 - local ref \$Integer, 736
 - local ref \$RationalNumber, 736
 - local ref \$Symbol, 736
 - defun, 735
- defiostream, 1168
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calledby sayMSG2File, 44
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1058
 - calledby setOutputTex, 1069
 - calledby writeInputLines, 896
 - defun, 1168
- defmacro
 - ancolsU16, 1321
 - ancolsU32, 1323
 - ancolsU8, 1320
 - anrowsU16, 1321
 - anrowsU32, 1323
 - anrowsU8, 1320
 - aref2U16, 1321
 - aref2U32, 1322
 - aref2U8, 1319
 - assq, 1240
 - bit-to-truth, 1301
 - bvec-elt, 1301
 - bvec-setelt, 1302
 - bvec-size, 1302
 - cdancols, 1274
 - cdanrows, 1274
 - cdaref2, 1273
 - cdelt, 1275
 - cdlen, 1276
 - cdsetaref2, 1273
 - cdsetelt, 1275
 - dancols, 1295
 - danrows, 1295
 - daref2, 1294
 - delt, 1296
 - DFAcos, 1286
 - DFAcosh, 1288
 - DFAdd, 1281
 - DFAsin, 1286
 - DFAsinh, 1288
 - DFAtan, 1286
 - DFAtan2, 1286
 - DFAtanh, 1289
 - DFCos, 1285
 - DFCosh, 1287
 - DFDivide, 1283
 - DFEqI, 1283
 - DFExp, 1285
 - DFExpt, 1284
 - DFIntegerDivide, 1283
 - DFIntegerExpt, 1284
 - DFIntegerMultiply, 1282
 - DFLessThan, 1277
 - DFLog, 1284
 - DFLogE, 1284
 - DFMax, 1282
 - DFMin, 1282
 - DFMinusp, 1281
 - DFMultiply, 1282

- DFSin, [1285](#)
- DFSinh, [1287](#)
- DFSqrt, [1283](#)
- DFSubtract, [1281](#)
- DFTan, [1285](#)
- DFTanh, [1288](#)
- DFUnaryMinus, [1280](#)
- DFZerop, [1281](#)
- dlen, [1295](#)
- dsetaref2, [1294](#)
- dsetelt, [1296](#)
- eltU16, [1317](#)
- eltU32, [1318](#)
- eltU8, [1316](#)
- FloatError, [1244](#)
- fracpart, [1243](#)
- frameExposureData, [25](#)
- frameHiFiAccess, [23](#)
- frameHistList, [23](#)
- frameHistListAct, [24](#)
- frameHistListLen, [23](#)
- frameHistoryTable, [24](#)
- frameHistRecord, [24](#)
- frameInteractive, [22](#)
- frameIOIndex, [23](#)
- frameName, [22](#)
- funfind, [106](#)
- getMsgArgL, [639](#)
- getMsgKey, [638](#)
- getMsgPosTagOb, [638](#)
- getMsgPrefix, [639](#)
- getMsgPrefix?, [640](#)
- getMsgTag, [640](#)
- getMsgTag?, [640](#)
- getMsgText, [639](#)
- hashCode?, [1225](#)
- hget, [1234](#)
- idChar?, [419](#)
- identp, [1237](#)
- leader?, [641](#)
- line?, [641](#)
- make-cdouble-matrix, [1273](#)
- make-cdouble-vector, [1275](#)
- make-double-matrix, [1294](#)
- make-double-matrix1, [1294](#)
- make-double-vector, [1295](#)
- make-double-vector1, [1296](#)
- makeMatrix1U16, [1322](#)
- makeMatrix1U32, [1323](#)
- makeMatrix1U8, [1320](#)
- makeMatrixU16, [1322](#)
- makeMatrixU32, [1323](#)
- makeMatrixU8, [1320](#)
- mkObj, [514](#)
- mkObjCode, [515](#)
- mkObjWrap, [515](#)
- objCodeMode, [517](#)
- objCodeVal, [517](#)
- objMode, [516](#)
- objSetMode, [516](#)
- objSetVal, [515](#)
- objVal, [516](#)
- objValUnwrap, [516](#)
- qsabsval, [1314](#)
- qsadd1, [1313](#)
- qsdifference, [1312](#)
- qsDot26432, [1325](#)
- qsDot2Mod6432, [1325](#)
- qslessp, [1313](#)
- qsmax, [1315](#)
- qsmin, [1315](#)
- qsminus, [1313](#)
- qsMod6432, [1324](#)
- qsMul6432, [1325](#)
- qsMulAdd6432, [1324](#)
- qsMulAddMod6432, [1324](#)
- qsMulMod32, [1324](#)
- qsoddp, [1314](#)
- qsplus, [1314](#)
- qssub1, [1313](#)
- qstimes, [1314](#)
- qszerop, [1315](#)
- qvlenU16, [1317](#)
- qvlenU32, [1318](#)
- qvlenU8, [1316](#)
- Rest, [369](#)
- setAref2U16, [1321](#)
- setAref2U32, [1322](#)
- setAref2U8, [1319](#)
- seteltU16, [1318](#)
- seteltU32, [1319](#)
- seteltU8, [1317](#)

- setMsgPrefix, 639
- setMsgText, 640
- spadConstant, 1373
- startsId?, 1234
- toScreen?, 641
- trapNumericErrors, 1312
- truth-to-bit, 1301
- while, 1230
- whileWithResult, 1230
- defstruct
 - database, 1182
 - libstream, 1382
 - monitor-data, 1382
- defun
 - /options, 96
 - /read, 944
 - /tracereply, 142
 - /untrace-0, 123
 - /untrace-1, 124
 - /untrace-2, 124
 - /untrace-reduce, 124
 - ?t, 146
 - abbQuery, 865
 - abbreviations, 822
 - abbreviationsSpad2Cmd, 822
 - absolutelyCanCoerceByCheating, 770
 - acot, 1291
 - acoth, 1293
 - acsc, 1292
 - acsch, 1293
 - addBinding, 1149
 - addBindingInteractive, 1153
 - addInputLibrary, 975
 - addNewInterpreterFrame, 33
 - addoperations, 1193
 - addTraceItem, 146
 - algCoerceInteractive, 1311
 - algEqual, 764
 - allConstructors, 1218
 - allOperations, 1219
 - alqlGetKindString, 1305
 - alqlGetOrigin, 1304
 - alqlGetParams, 1305
 - alreadyOpened?, 653
 - apropos, 1137
 - asec, 1291
 - asech, 1293
 - assertCond, 389
 - augmentHasArgs, 1619
 - augmentTraceNames, 78
 - axiomVersion, 816
 - basicLookup, 1225
 - basicLookupCheckDefaults, 1228
 - basicStringize, 1510
 - bcComplexLimit, 1478
 - bcComplexLimitGen, 1480
 - bcCreateVariableString, 1483
 - bcDefiniteIntegrate, 1409
 - bcDefiniteIntegrateGen, 1411
 - bcDifferentiate, 1406
 - bcDifferentiateGen, 1407
 - bcDraw, 1415
 - bcDraw2Dfun, 1416
 - bcDraw2DfunGen, 1417
 - bcDraw2Dpar, 1418
 - bcDraw2DparGen, 1419
 - bcDraw2DSolve, 1420
 - bcDraw2DSolveGen, 1421
 - bcDraw3Dfun, 1422
 - bcDraw3DfunGen, 1424
 - bcDraw3Dpar, 1425
 - bcDraw3Dpar1, 1427
 - bcDraw3Dpar1Gen, 1428
 - bcDraw3DparGen, 1426
 - bcDrawIt, 1496
 - bcDrawIt2, 1473
 - bcError, 1496
 - bcFindString, 1494
 - bcFinish, 1493
 - bcGen, 1468, 1494
 - bcGenEquations, 1492
 - bcGenExplicitMatrix, 1472
 - bcHt, 1508
 - bchtMakeButton, 1536
 - bcIndefiniteIntegrate, 1408
 - bcIndefiniteIntegrateGen, 1409
 - bcInputEquations, 1481
 - bcInputEquationsEnd, 1485
 - bcInputExplicitMatrix, 1443
 - bcInputMatrixByFormulaGen, 1471
 - bcInputSolveInfo, 1448
 - bcIssueHt, 1509

- bcLaurentSeries, [1435](#)
- bcLaurentSeriesGen, [1437](#)
- bcLimit, [1412](#)
- bcLinearExtractMatrix, [1487](#)
- bcLinearMatrixGen, [1490](#)
- bcLinearSolve, [1440](#)
- bcLinearSolveEqns, [1440](#)
- bcLinearSolveEqns1, [1441](#)
- bcLinearSolveEqnsGen, [1492](#)
- bcLinearSolveMatrix, [1442](#)
- bcLinearSolveMatrix1, [1487](#)
- bcLinearSolveMatrixHomo, [1489](#)
- bcLinearSolveMatrixInhomo, [1488](#)
- bcLinearSolveMatrixInhomoGen, [1489](#)
- bcMakeEquations, [1483](#)
- bcMakeLinearEquations, [1484](#)
- bcMakeUnknowns, [1483](#)
- bcMatrix, [1414](#)
- bcMatrixGen, [1472](#)
- bcMkFunction, [1493](#)
- bcNotReady, [1496](#)
- bcOptional, [1494](#)
- bcProduct, [1474](#)
- bcProductGen, [1474](#)
- bcPuisseuxSeries, [1437](#)
- bcPuisseuxSeriesGen, [1438](#)
- bcReadMatrix, [1442](#), [1475](#)
- bcRealLimit, [1475](#)
- bcRealLimitGen, [1477](#)
- bcRealLimitGen1, [1478](#)
- bcSadFaces, [1518](#)
- bcSeries, [1429](#)
- bcSeriesByFormula, [1432](#)
- bcSeriesByFormulaGen, [1475](#)
- bcSeriesExpansion, [1430](#)
- bcSeriesExpansionGen, [1431](#)
- bcSeriesGen, [1435](#)
- bcSolve, [1439](#)
- bcSolveEquations, [1486](#)
- bcSolveEquationsNumerically, [1485](#)
- bcSolveNumerically1, [1486](#)
- bcSolveSingle, [1449](#)
- bcString2HyString, [1494](#)
- bcString2HyString2, [1493](#)
- bcString2WordList, [1495](#)
- bcSum, [1413](#)
- bcSumGen, [1414](#)
- bcSystemSolve, [1446](#)
- bcSystemSolveEqns1, [1447](#)
- bcTaylorSeries, [1433](#)
- bcTaylorSeriesGen, [1434](#)
- bcUnixTable, [1650](#)
- bcVectorGen, [1495](#)
- bcvspace, [1495](#)
- bcwords2liststring, [1495](#)
- beforeAfter, [1521](#)
- BesselasympA, [1264](#)
- BesselasympB, [1264](#)
- BesselI, [1252](#)
- besselIback, [1253](#)
- BesselIBackRecur, [1253](#)
- besselIcheb, [1254](#)
- BesselJ, [1262](#)
- BesselJAsympt, [1264](#)
- BesselJAsymptOrder, [1265](#)
- BesselJRecur, [1265](#)
- BooleanEquality, [1272](#)
- bpitrace, [136](#)
- bracketString, [1544](#)
- break, [156](#)
- breaklet, [155](#)
- brightprint, [1238](#)
- brightprint-0, [1238](#)
- browseopen, [1191](#)
- buildHtMacroTable, [1401](#)
- buttonNames, [1525](#)
- bvec-and, [1303](#)
- bvec-concat, [1302](#)
- bvec-copy, [1302](#)
- bvec-equal, [1302](#)
- bvec-greater, [1303](#)
- bvec-make-full, [1301](#)
- bvec-nand, [1304](#)
- bvec-nor, [1304](#)
- bvec-not, [1304](#)
- bvec-or, [1303](#)
- bvec-xor, [1303](#)
- c-to-r, [1244](#)
- c-to-s, [1243](#)
- canFuncall?, [1237](#)
- catchCoerceFailure, [760](#)
- categoryopen, [1192](#)

- cbesseli, 1280
- cbesselj, 1279
- cgamma, 1277
- cgammaAdjust, 1251
- cgammaBernsum, 1252
- cgammaG, 1249
- cgammaImpl, 1248
- cgammaMat, 1250
- changeHistListLen, 898
- changeToNamedInterpreterFrame, 32
- charDigitVal, 927
- chebf01, 1267
- chebf01coefmake, 1255
- chebstarevalarr, 1256
- checkCondition, 1540
- checkFilter, 1589
- chkAllNonNegativeInteger, 1562
- chkDirectory, 1561
- chkNameList, 1560
- chkNonNegativeInteger, 1562
- chkOutputFileName, 1561
- chkPosInteger, 1561
- chkRange, 1562
- chyper0f1, 1280
- cleanline, 859
- cleanupLine, 869
- clear, 835
- clearCmdAll, 839
- clearCmdCompletely, 838
- clearCmdExcept, 840
- clearCmdParts, 841
- clearCmdSortedCaches, 837
- clearFrame, 1128
- clearMacroTable, 840
- clearParserMacro, 793
- clearSpad2Cmd, 835
- clngamma, 1278
- clngammacase1, 1248
- clngammacase2, 1250
- clngammacase23, 1250
- clngammacase3, 1251
- clngammaImpl, 1248
- close, 845
- closeInterpreterFrame, 37
- cmpnote, 319
- coerceBranch2Union, 766
- coerceByFunction, 748
- coerceByTable, 759
- coerceCommuteTest, 758
- coerceConvertMmSelection, 752
- coerceImmediateSubDomain, 769
- coerceInt, 741
- coerceInt0, 742
- coerceInt1, 743
- coerceInt2Union, 765
- coerceIntAlgebraicConstant, 766
- coerceIntByMap, 761
- coerceIntByMapInner, 763
- coerceIntCommute, 757
- coerceInteractive, 740
- coerceIntFromUnion, 765
- coerceIntPermute, 754
- coerceIntSpecial, 760
- coerceIntTableOrFunction, 759
- coerceIntTest, 752
- coerceIntTower, 750
- coerceIntX, 768
- coerceOrRetract, 771
- coerceOrThrowFailure, 763
- coerceRetract, 777
- coerceSpadArgs2E, 128
- coerceSpadFunValue2E, 130
- coerceSubDomain, 769
- coerceTraceArgs2E, 127
- coerceTraceFunValue2E, 129
- coerceUnion2Branch, 775
- coercionFailure, 763
- commandAmbiguityError, 792
- commandError, 789
- commandErrorIfAmbiguous, 810
- commandErrorMessage, 789
- commandsForUserLevel, 788
- commandUserLevelError, 790
- compareposns, 660
- compareTypeLists, 768
- compileBoot, 115
- compiledLookup, 1225
- compiledLookupCheck, 837
- computeDomainVariableAlist, 1529
- computeTTTranspositions, 754
- concat, 1237
- concatWithBlanks, 1590

- condErrorMsg, 1541
- conLowerCaseConTran, 1590
- conOpPage, 1628
- conOpPage1, 1629
- conPage, 1599
- conPageChoose, 1642
- conPageConEntry, 1600
- conPageFastPath, 1600
- conSpecialString?, 1598
- constoken, 405
- constructorSearch, 1597
- constructSubst, 732
- containsVars, 729
- containsVars1, 729
- copyright, 855
- cot, 1290
- cotdiffeval, 1257
- coth, 1292
- countCache, 980
- cpsi, 1279
- cPsiImpl, 1262
- createCurrentInterpreterFrame, 29
- credits, 856
- csc, 1292
- csch, 1292
- cSearch, 1595
- DaaseName, 1209
- dbAddChain, 1641
- dbAddChainDomain, 1640
- dbAddDocTable, 1636
- dbCompositeWithMap, 1630
- dbConsExposureMessage, 1645
- dbConsHeading, 1649
- dbConstructorDoc, 1635
- dbConstructorDoc,gn, 1635
- dbConstructorDoc,hn, 1634
- dbDocTable, 1635
- dbExtractUnderlyingDomain, 1631
- dbGetDocTable, 1638
- dbGetDocTable,gn, 1638
- dbGetDocTable,hn, 1637
- dbMkEvalable, 1625
- dbMkForm, 1658
- dbNonEmptyPattern, 1571
- dbRead, 1596
- dbSearchOrder, 1610
- dbSelectCon, 1648
- dbShowConditions, 1648
- dbShowCons, 1641
- dbShowCons1, 1643
- dbShowConsDoc, 1645
- dbShowConsDoc1, 1646
- dbShowConsKindsFilter, 1645
- dbShowConstructorLines, 1650
- dbSpecialDescription, 1651
- dbSpecialExpandIfNecessary, 1653
- dbSpecialExports, 1652
- dbSpecialOperations, 1652
- dbString2Words, 1591
- dbSubConform, 1640
- dbWordFrom, 1592
- decideHowMuch, 649
- decomposeTypeIntoTower, 756
- defaultTargetFE, 735
- defiostream, 1168
- delasc, 1661
- Delay, 397
- deldatabase, 1195
- deleteFile, 1233
- describe, 858
- describeFortPersistence, 1029
- describeInputLibraryArgs, 975
- describeOutputLibraryArgs, 972
- describeSetFortDir, 999
- describeSetFortTmpDir, 997
- describeSetFunctionsCache, 982
- describeSetLinkerArgs, 1001
- describeSetNagHost, 1027
- describeSetOutputAlgebra, 1037
- describeSetOutputFormula, 1065
- describeSetOutputFortran, 1045
- describeSetOutputHtml, 1050
- describeSetOutputMathml, 1056
- describeSetOutputOpenMath, 1061
- describeSetOutputTex, 1072
- describeSetStreamsCalculate, 1075
- describeSpad2Cmd, 858
- desiredMsg, 643
- dewritify, 925
- dewritify,dewritifyInner, 922
- diffAlist, 1125
- digit?, 413

- digitp, 1235
- DirToString, 1298
- disableHist, 913
- display, 864
- displayCondition, 804
- displayExposedConstructors, 165
- displayExposedGroups, 165
- displayFrameNames, 28
- displayHiddenConstructors, 166
- displayMacro, 794
- displayMacros, 867
- displayMode, 806
- displayModemap, 806
- displayOperations, 866
- displayOperationsFromLisplib, 1095
- displayParserMacro, 804
- displayProperties, 801
- displayProperties,sayFunctionDeps, 796
- displaySetOptionInformation, 964
- displaySetVariableSettings, 965
- displayType, 800
- displayValue, 799
- displayWorkspaceNames, 794
- divide2, 1308
- dKind, 1589
- doDoitButton, 1469
- domainDescendantsOf, 1603
- domainToGenvar, 100
- domArg, 720
- domArg2, 720
- doSystemCommand, 786
- downcase, 1272
- downlink, 1570
- dqAppend, 634
- dqConcat, 633
- dqToList, 634
- dqUnit, 633
- dropInputLibrary, 976
- dSearch, 1595
- dumbTokenize, 807
- edit, 872
- editFile, 873
- editSpad2Cmd, 872
- Else?, 373
- Elseif?, 372
- embed2, 88
- embeddedFunction, 88
- emptyInterpreterFrame, 27
- endedp, 955
- enPile, 630
- eofp, 1169
- eqpileTree, 629
- erMsgCompare, 660
- erMsgSep, 660
- erMsgSort, 659
- evalCategory, 1145
- evalDomain, 1115
- evalSharpOne, 776
- evaluateSignature, 1123
- evaluateType, 1118
- evaluateType1, 1120
- executeInterpreterCommand, 1470
- ExecuteInterpSystemCommand, 324
- executeQuietCommand, 339
- explainLinear, 1491
- fetchOutput, 910
- fillerSpaces, 310
- filterAndFormatConstructors, 1136
- filterListOfStrings, 1134
- filterListOfStringsWithFn, 1135
- fin, 876
- finalExactRequest, 1491
- findFrameInRing, 31
- findnexttest, 952
- firstTokPosn, 631
- fixObjectForPrinting, 795
- flatBvList, 89
- flatten, 861
- flattenOperationAlist, 107
- float2Sex, 603
- fnameDirectory, 1298
- fnameExists?, 1299
- fnameMake, 1297
- fnameName, 1299
- fnameNew, 1300
- fnameReadable?, 1299
- fnameType, 1299
- fnameWritable?, 1300
- frame, 25
- frameEnvironment, 31
- frameNames, 28
- frameSpad2Cmd, 25

- From, 670
- FromTo, 670
- funfind,LAM, 106
- gammaRatapprox, 1246
- gammaRatkernel, 1247
- gammaStirling, 1245
- gatherGlossLines, 1580
- genDomainTraceName, 122
- gensymInt, 927
- get-current-directory, 328
- getAliasIfTracedMapParameter, 140
- getAndEvalConstructorArgument, 1144
- getAndSay, 800
- getBpiNameIfTracedMap, 140
- getBrowseDatabase, 1310
- getConstantFromDomain, 767
- getConstructorDocumentation, 1647
- getdatabase, 1196
- getDependentsOfConstructor, 1620
- getDirectoryList, 1170
- getenviron, 322
- getFirstWord, 809
- getHtMacroItem, 1402
- getl, 1240
- getLinePos, 662
- getLineText, 662
- getMapSig, 117
- getMapSubNames, 130
- getMsgCatAttr, 648
- getMsgFTTag?, 649
- getMsgInfoFromKey, 656
- getMsgKey?, 652
- getMsgPos, 649
- getMsgPos2, 668
- getMsgToWhere, 653
- getOperationAlistFromLisplib, 134
- getOplistForConstructorForm, 1098
- getOplistWithUniqueSignatures, 1099
- getOption, 113
- getParserMacroNames, 793
- getPosStL, 646
- getPreStL, 645
- getPreviousMapSubNames, 103
- getProplist, 1150
- getRefvU16, 1318
- getRefvU32, 1319
- getRefvU8, 1317
- getspoolname, 951
- getStFromMsg, 644
- getSubDomainPredicate, 769
- getSystemCommandLine, 1106
- getTraceOption, 118
- getTraceOption,hn, 117
- getTraceOptions, 116
- getUsersOfConstructor, 1622
- getWorkspaceNames, 795
- handleNoParseCommands, 785
- handleParsedSystemCommands, 808
- handleTokenSizeSystemCommands, 787
- hasAtt, 722
- hasAttSig, 724
- hasCate, 731
- hasCate1, 725
- hasCateSpecial, 733
- hasCateSpecialNew, 734
- hasCatExpression, 725
- hasCaty, 718
- hasCaty1, 730
- hasCorrectTarget, 753
- hashable, 1315
- hasOptArgs?, 607
- hasOption, 791
- hasPair, 113
- hasSharpVar, 93
- hasSig, 721
- hasSigAnd, 723
- hasSigOr, 724
- help, 880
- helpSpad2Cmd, 881
- histFileErase, 928
- histFileName, 888
- histInputFileName, 888
- history, 889
- historySpad2Cmd, 890
- hkeys, 1235
- horner, 1247
- hput, 1234
- htAddHeading, 1512
- htAllOrNum, 1569
- htBcLinks, 1520
- htBcLispLinks, 1520
- htBcRadioButtons, 1523

- htCacheAddChoice, 1566
- htCacheOne, 1569
- htCacheSet, 1568
- htCheck, 1558
- htCheckList, 1559
- htDoneButton, 1537
- htDoNothing, 1558
- htEscapeString, 1545
- htFunctionSetLiteral, 1555
- htGloss, 1577
- htGlossPage, 1577
- htGlossSearch, 1581
- htGreekSearch, 1582
- htInitPage, 1510
- htInitPageNoScroll, 1511
- htInputStrings, 1525
- htKill, 1556
- htLispLinks, 1518
- htLispMemoLinks, 1519
- htMakeButton, 1534
- htMakeDoitButton, 1469
- htMakeDoneButton, 1533
- htMakeErrorPage, 1514
- htMakeInputList, 1543
- htMakeLabel, 1567
- htMakePage, 1513
- htMakePage1, 1513
- htMakePathKey, 1563
- htMakePathKey,fn, 1563
- htMakeTemplates, 1532
- htMakeTemplates,substLabel, 1532
- htMarkTree, 1563
- htMkName, 1497
- httpAddInputAreaProp, 1502
- httpAddToPageDescription, 1507
- httpButtonValue, 1501
- httpDestroyPage, 1468
- httpDomainConditions, 1499
- httpDomainPvarSubstList, 1500
- httpDomainVariableAlist, 1499
- httpInputAreaAlist, 1501
- httpLabelDefault, 1506
- httpLabelErrorMsg, 1505
- httpLabelFilter, 1507
- httpLabelFilteredInputString, 1503
- httpLabelInputString, 1503
- httpLabelSpadType, 1506
- httpLabelSpadValue, 1504
- httpLabelType, 1506
- httpMakeEmptyPage, 1467
- httpName, 1499
- httpPageDescription, 1507
- httpProperty, 1502
- httpPropertyList, 1502
- httpRadioButtonAlist, 1500
- htProcessBcButtons, 1516
- htProcessBcStrings, 1517
- htProcessDoitButton, 1537
- htProcessDomainConditions, 1527
- htProcessDoneButton, 1534
- htProcessToggleButtons, 1515
- httpSetDomainConditions, 1499
- httpSetDomainPvarSubstList, 1500
- httpSetDomainVariableAlist, 1500
- httpSetInputAreaAlist, 1502
- httpSetLabelErrorMsg, 1505
- httpSetLabelInputString, 1504
- httpSetLabelSpadValue, 1505
- httpSetName, 1499
- httpSetPageDescription, 1507
- httpSetProperty, 1503
- httpSetRadioButtonAlist, 1501
- htQuote, 1515
- htRadioButtons, 1522
- htSay, 1509
- htSayBind, 1512
- htSayStandard, 1511
- htSetCache, 1565
- htSetExpose, 1565
- htSetFunCommand, 1556
- htSetFunCommandContinue, 1556
- htSetHistory, 1564
- htSetInputLibrary, 1564
- htSetInteger, 1553
- htSetLinkerArgs, 1565
- htSetLiteral, 1551
- htSetLiterals, 1550
- htSetNotAvailable, 1557
- htSetOutputCharacters, 1565
- htSetOutputLibrary, 1564
- htSetSystemVariable, 1577
- htSetSystemVariableKind, 1576

- htSetvarDoneButton, 1555
- htSetVars, 1545
- htShowCount, 1548
- htShowFunctionPage, 1553
- htShowFunctionPageContinued, 1554
- htShowIntegerPage, 1552
- htShowLiteralsPage, 1550
- htShowPage, 1512
- htShowPageNoScroll, 1513
- htShowSetPage, 1549
- htShowSetTree, 1546
- htShowSetTreeValue, 1548
- htStringPad, 1497
- htsv, 1545
- htSystemVariables, 1574
- htSystemVariables,displayOptions, 1572
- htSystemVariables,fn, 1572
- htSystemVariables,functionTail, 1573
- htSystemVariables,gn, 1571
- htTextSearch, 1584
- htTutorialSearch, 1586
- If?, 372
- ifCond, 382
- ignorep, 956
- iht, 1508
- importFromFrame, 34
- incActive?, 396
- incAppend, 380
- incAppend1, 380
- incBiteOff, 930
- incClassify, 392
- incCommand?, 394
- incCommandTail, 395
- incConsoleInput, 396
- incDrop, 395
- incFileInput, 395
- incFileName, 930
- incHandleMessage, 368
- incIgen, 367
- incIgen1, 367
- inclFname, 395
- inclLine, 380
- inclLine1, 381
- inclmsgCannotRead, 385
- inclmsgCmdBug, 391
- inclmsgConActive, 386
- inclmsgConsole, 388
- inclmsgConStill, 387
- inclmsgFileCycle, 385
- inclmsgFinSkipped, 388
- inclmsgIfBug, 391
- inclmsgIfSyntax, 390
- inclmsgNoSuchFile, 384
- inclmsgPrematureEOF, 381
- inclmsgPrematureFin, 389
- inclmsgSay, 383
- incLude, 369
- incLude1, 374
- incNConsoles, 396
- incPrefix?, 394
- incRenum, 366
- incRenumItem, 368
- incRenumLine, 368
- incRgen, 397
- incRgen1, 398
- incStream, 366
- incString, 331
- incZip, 366
- incZip1, 367
- init-boot/spad-reader, 1154
- init-memory-config, 326
- initHist, 888
- initHistList, 889
- initial-getdatabase, 1187
- initializeInterpreterFrameRing, 27
- initializeSetVariables, 962
- initImPr, 657
- initroot, 327
- initToWhere, 658
- insertAlist, 981
- insertpile, 625
- insertPos, 669
- integer-decode-float-denominator, 1289
- integer-decode-float-exponent, 1290
- integer-decode-float-numerator, 1289
- integer-decode-float-sign, 1289
- InterpExecuteSpadSystemCommand, 323
- interpFunctionDepAlists, 805
- interpopen, 1189
- interpret, 346
- interpret1, 347
- interpret2, 348

- interpretTopLevel, 345
- intInterpretPform, 360
- intloop, 317
- intloopEchoParse, 361
- intloopInclude, 355
- intloopInclude0, 355
- intloopPrefix?, 327
- intloopProcess, 356
- intloopProcessString, 329
- intloopReadConsole, 320
- intloopSpadProcess, 357
- intloopSpadProcess,interp, 358
- intnplisp, 327
- intProcessSynonyms, 324
- ioclear, 1158
- iostat, 1156
- isDomainOrPackage, 107
- isDomainValuedVariable, 1145
- isEqualOrSubDomain, 736
- isExposedConstructor, 1094
- isGenvar, 126
- isIntegerString, 808
- isInterpOnlyMap, 104
- isListOfIdentifiers, 102
- isListOfIdentifiersOrStrings, 102
- isPartialMode, 718
- isPatternVar, 729
- isSharpVar, 110
- isSharpVarWithNum, 109
- isSubForRedundantMapName, 105
- isSystemDirectory, 1223
- isTaggedUnion, 1117
- isUncompiledMap, 104
- isWrapped, 1661
- justifyMyType, 354
- kArgPage, 1602
- kArgumentCheck, 1625
- kcaPage, 1616
- kcaPage1, 1617
- kccPage, 1618
- kcdePage, 1620
- kcdoPage, 1617
- kcdPage, 1616
- kCheckArgumentNumbers, 1627
- kcnPage, 1622
- kcPage, 1611
- kcpPage, 1614
- kcuPage, 1621
- kDomainName, 1624
- kdPageInfo, 1601
- KeepPart?, 373
- kePage, 1605
- kePageDisplay, 1608
- kePageOpAlist, 1607
- keyword, 412
- keyword?, 412
- kiPage, 1604
- kisValidType, 1626
- koaPageFilterByName, 1633
- koPage, 1631
- koPageAux, 1632
- koPageAux1, 1633
- koPageFromKKPage, 1632
- koPageInputAreaUnchanged?, 1623
- kSearch, 1596
- ksPage, 1609
- kTestPred, 1639
- lassocSub, 103
- lastcount, 956
- lastTokPosn, 631
- leaveScratchpad, 941
- lefts, 1658
- letPrint, 108
- letPrint2, 110
- letPrint3, 111
- lfcomment, 408
- lffloat, 419
- lfid, 406
- lfinteger, 425
- lfkey, 413
- lfnegcomment, 409
- lfrinteger, 425
- lfspace, 421
- lfstring, 421
- libConstructorSig, 1659
- libdbTrim, 1596
- library, 1199
- license, 934
- limitedPrint1, 153
- linearFinalRequest, 1490
- lineoftoks, 402

- [listConstructorAbbreviations](#), 824
- [listDecideHowMuch](#), 651
- [listOfStrings2String](#), 1549
- [listOutputter](#), 644
- [lnGamma](#), 1256
- [lnCreate](#), 635
- [lnExtraBlanks](#), 635
- [lnFileName](#), 637
- [lnFileName?](#), 637
- [lnGlobalNum](#), 636
- [lnImmediate?](#), 637
- [lnLocalNum](#), 636
- [lnPlaceOfOrigin](#), 636
- [lnGamma](#), 1245
- [lnGammaRatApprox](#), 1244
- [lnSetGlobalNum](#), 636
- [lnString](#), 635
- [loadFunctor](#), 1224
- [loadLib](#), 1221
- [loadLibNoUpdate](#), 1223
- [localdatabase](#), 1200
- [localnrLib](#), 1201
- [logH](#), 1249
- [logS](#), 1251
- [lookupInDomainVector](#), 1227
- [loopIters2Sex](#), 609
- [lotsof](#), 1234
- [ltrace](#), 936
- [mac0Define](#), 526
- [mac0ExpandBody](#), 521
- [mac0Get](#), 524
- [mac0GetName](#), 522
- [mac0InfiniteExpansion](#), 521
- [mac0InfiniteExpansion,name](#), 522
- [mac0MLambdaApply](#), 520
- [mac0SubstituteOuter](#), 527
- [macApplication](#), 519
- [macExpand](#), 518
- [macId](#), 523
- [macLambda](#), 525
- [macLambda,mac](#), 525
- [macLambdaParameterHandling](#), 527
- [macMacro](#), 525
- [macroExpanded](#), 518
- [macSubstituteId](#), 528
- [macSubstituteOuter](#), 527
- [macWhere](#), 524
- [macWhere,mac](#), 524
- [make-absolute-filename](#), 328
- [make-appendstream](#), 1168
- [make-databases](#), 1204
- [make-instream](#), 1167
- [make-outstream](#), 1167
- [makeByteWordVec2](#), 1373
- [makeFullNamestring](#), 1171
- [makeHistFileName](#), 887
- [makeInitialModemapFrame](#), 328
- [makeInputFilename](#), 1169
- [makeLeaderMsg](#), 667
- [makeMsgFromLine](#), 661
- [makeOrdinal](#), 1123
- [makePathname](#), 1233
- [makeSpadCommand](#), 1543
- [makeStream](#), 1169
- [manexp](#), 1290
- [mapLetPrint](#), 139
- [mapStringize](#), 1509
- [markUnique](#), 135
- [member](#), 1238
- [mergePathnames](#), 1231
- [messageprint](#), 1239
- [messageprint-1](#), 1239
- [messageprint-2](#), 1239
- [mkConform](#), 1628
- [mkCurryFun](#), 1521
- [mkDomPvar](#), 731
- [mkDomTypeForm](#), 1603
- [mkEvalable](#), 1115
- [mkEvalableMapping](#), 1118
- [mkEvalableRecord](#), 1118
- [mkEvalableUnion](#), 1117
- [mkLineList](#), 362
- [mkobjFn](#), 1306
- [mkprompt](#), 334
- [mkSetTitle](#), 1549
- [mkUnixPattern](#), 1587
- [monitor-add](#), 1384
- [monitor-apropos](#), 1397
- [monitor-autoload](#), 1392
- [monitor-checkpoint](#), 1389
- [monitor-decr](#), 1386
- [monitor-delete](#), 1384

- monitor-dirname, 1392
- monitor-disable, 1385
- monitor-enable, 1384
- monitor-end, 1383
- monitor-exposedp, 1394
- monitor-file, 1387
- monitor-help, 1390
- monitor-incr, 1386
- monitor-info, 1387
- monitor-inittable, 1382
- monitor-libname, 1393
- monitor-nrlib, 1393
- monitor-parse, 1396
- monitor-percent, 1397
- monitor-readinterp, 1394
- monitor-report, 1395
- monitor-reset, 1385
- monitor-restore, 1389
- monitor-results, 1383
- monitor-spadfile, 1396
- monitor-tested, 1388
- monitor-untested, 1387
- monitor-write, 1388
- monitorEnter, 147
- monitorEvalAfter, 92
- monitorEvalBefore, 92
- monitorEvalTran, 93
- monitorEvalTran1, 93
- monitorExit, 151
- monitorGetValue, 94
- monitorPrint, 150
- monitorPrintArg, 149
- monitorPrintArgs, 148
- monitorPrintRest, 150
- monitorPrintValue, 152
- monitorX, 90
- monitorXX, 90
- msgCreate, 637
- msgImPr?, 648
- msgNoRep?, 665
- msgOutputter, 643
- msgText, 354
- myWritable?, 1300
- namestring, 1230
- ncAlist, 705
- ncBug, 658
- ncConversationPhase, 360
- ncConversationPhase,wrapup, 361
- ncEltQ, 706
- ncError, 361
- ncHardError, 642
- ncIntLoop, 317
- ncloopCommand, 818
- ncloopDQlines, 364
- ncloopEscaped, 329
- ncloopIncFileName, 929
- ncloopInclude, 930
- ncloopInclude0, 365
- ncloopInclude1, 929
- ncloopParse, 329
- ncloopPrefix?, 818
- ncloopPrintLines, 362
- ncParseFromString, 1310
- ncPutQ, 706
- ncSoftError, 642
- ncTag, 705
- ncTopLevel, 316
- newHelpSpad2Cmd, 881
- next, 330
- next-lines-clear, 1158
- next-lines-show, 1157
- next1, 330
- nextInterpreterFrame, 32
- nextline, 404
- nonBlank, 363
- npADD, 450
- npAdd, 451
- npAmpersand, 496
- npAmpersandFrom, 495
- npAndOr, 474
- npAngleBared, 479
- npAnyNo, 455
- npApplication, 454
- npApplication2, 455
- npArith, 493
- npAssign, 509
- npAssignment, 509
- npAssignVariable, 510
- npAtom1, 475
- npAtom2, 452
- npBacksetElse, 489
- npBackTrack, 440

- npBDefinition, [477](#)
- npboot, [812](#)
- npBPileDefinition, [480](#)
- npBraced, [479](#)
- npBracketed, [478](#)
- npBracketed, [478](#)
- npBreak, [467](#)
- npBy, [492](#)
- npCategory, [445](#)
- npCategoryL, [445](#)
- npCoerceTo, [512](#)
- npColon, [510](#)
- npColonQuery, [512](#)
- npComma, [438](#)
- npCommaBackSet, [438](#)
- npCompMissing, [443](#)
- npConditional, [488](#)
- npConditionalStatement, [472](#)
- npConstTok, [476](#)
- npDDInfKey, [500](#)
- npDecl, [507](#)
- npDef, [480](#)
- npDefaultDecl, [462](#)
- npDefaultItem, [462](#)
- npDefaultItemList, [461](#)
- npDefaultValue, [487](#)
- npDefinition, [459](#)
- npDefinitionItem, [460](#)
- npDefinitionlist, [485](#)
- npDefinitionOrStatement, [440](#)
- npDefn, [479](#)
- npDefTail, [487](#)
- npDiscrim, [490](#)
- npDisjand, [490](#)
- npDollar, [476](#)
- npDotted, [454](#)
- npElse, [488](#)
- npEncAp, [474](#)
- npEncl, [475](#)
- npEnclosed, [504](#)
- npEqKey, [437](#)
- npEqPeek, [444](#)
- npExit, [508](#)
- npExport, [463](#)
- npExpress, [471](#)
- npExpress1, [472](#)
- npFirstTok, [435](#)
- npFix, [458](#)
- npForIn, [470](#)
- npFree, [466](#)
- npFromdom, [495](#)
- npFromdom1, [496](#)
- npGives, [440](#)
- npId, [497](#)
- npImport, [473](#)
- npInfGeneric, [500](#)
- npInfixOp, [453](#)
- npInfixOperator, [452](#)
- npInfKey, [501](#)
- npInline, [466](#)
- npInterval, [492](#)
- npItem, [434](#)
- npItem1, [434](#)
- npIterate, [467](#)
- npIterator, [468](#)
- npIterators, [468](#)
- npLambda, [441](#)
- npLeftAssoc, [499](#)
- npLet, [458](#)
- npLetQualified, [459](#)
- npLisp, [812](#)
- npList, [447](#)
- npListAndRecover, [482](#)
- npListing, [447](#)
- npListofFun, [513](#)
- npLocal, [465](#)
- npLocalDecl, [465](#)
- npLocalItem, [465](#)
- npLocalItemList, [464](#)
- npLogical, [489](#)
- npLoop, [467](#)
- npMacro, [456](#)
- npMatch, [442](#)
- npMDEF, [457](#)
- npMdef, [457](#)
- npMDEFinition, [458](#)
- npMissing, [443](#)
- npMissingMate, [508](#)
- npMoveTo, [483](#)
- npName, [496](#)
- npNext, [437](#)
- npNull, [623](#)

- npParened, [478](#)
- npParenthesize, [507](#)
- npParenthesized, [507](#)
- npParse, [433](#)
- npPDefinition, [476](#)
- npPileBracketed, [481](#)
- npPileDefinitionlist, [481](#)
- npPileExit, [508](#)
- npPop1, [436](#)
- npPop2, [436](#)
- npPop3, [436](#)
- npPower, [495](#)
- npPP, [502](#)
- npPPf, [503](#)
- npPPff, [503](#)
- npPPg, [503](#)
- npPrefixColon, [453](#)
- npPretend, [512](#)
- npPrimary, [450](#)
- npPrimary1, [456](#)
- npPrimary2, [450](#)
- npProcessSynonym, [813](#)
- npProduct, [494](#)
- npPush, [435](#)
- npPushId, [501](#)
- npQualDef, [437](#)
- npQualified, [439](#)
- npQualifiedDefinition, [439](#)
- npQualType, [474](#)
- npQualTypelist, [473](#)
- npQuiver, [490](#)
- npRecoverTrap, [483](#)
- npRelation, [491](#)
- npRemainder, [494](#)
- npRestore, [444](#)
- npRestrict, [513](#)
- npReturn, [470](#)
- npRightAssoc, [498](#)
- npRule, [486](#)
- npSCategory, [445](#)
- npSDefaultItem, [461](#)
- npSegment, [493](#)
- npSelector, [455](#)
- npSemiBackSet, [486](#)
- npSemiListing, [485](#)
- npSigDecl, [449](#)
- npSigItem, [448](#)
- npSigItemList, [446](#)
- npSignature, [446](#)
- npSignatureDefinee, [449](#)
- npSingleRule, [486](#)
- npSLocalItem, [464](#)
- npSQualTypelist, [473](#)
- npState, [504](#)
- npStatement, [463](#)
- npSuch, [442](#)
- npSuchThat, [469](#)
- npSum, [493](#)
- npSymbolVariable, [498](#)
- npsynonym, [813](#)
- npSynthetic, [491](#)
- npsystem, [812](#)
- npTagged, [510](#)
- npTerm, [493](#)
- npTrap, [505](#)
- npTrapForm, [505](#)
- npTuple, [438](#)
- npType, [441](#)
- npTypedForm, [513](#)
- npTypedForm1, [511](#)
- npTypeStyle, [511](#)
- npTypeVariable, [448](#)
- npTypeVariablelist, [449](#)
- npTypified, [511](#)
- npTyping, [460](#)
- npVariable, [505](#)
- npVariablelist, [506](#)
- npVariableName, [506](#)
- npVoid, [471](#)
- npWConditional, [487](#)
- npWhile, [470](#)
- npWith, [442](#)
- npZeroOrMore, [469](#)
- NRTEvalDomain, [1229](#)
- objEnv, [517](#)
- objModeFn, [1307](#)
- objValFn, [1307](#)
- ofCategory, [717](#)
- oldCompLookup, [1229](#)
- oldHistFileName, [887](#)
- oldParseString, [1543](#)
- om-bindTCP, [1359](#)

- om-closeConn, 1358
- om-closeDev, 1358
- om-connectTCP, 1360
- om-getApp, 1361
- om-getAtp, 1362
- om-getAttr, 1362
- om-getBind, 1362
- om-getBVar, 1362
- om-getByteArray, 1363
- om-getConnInDev, 1359
- om-getConnOutDev, 1359
- om-getEndApp, 1363
- om-getEndAtp, 1363
- om-getEndAttr, 1363
- om-getEndBind, 1364
- om-getEndBVar, 1364
- om-getEndError, 1364
- om-getEndObject, 1364
- om-getError, 1365
- om-getFloat, 1365
- om-getInt, 1365
- om-getObject, 1365
- om-getString, 1366
- om-getSymbol, 1366
- om-getType, 1366
- om-getVar, 1366
- om-listCDs, 1356
- om-listSymbols, 1356
- om-makeConn, 1358
- om-openFileDev, 1357
- om-openStringDev, 1358
- om-putApp, 1367
- om-putAtp, 1367
- om-putAttr, 1367
- om-putBind, 1367
- om-putBVar, 1368
- om-putByteArray, 1368
- om-putEndApp, 1368
- om-putEndAtp, 1368
- om-putEndAttr, 1369
- om-putEndBind, 1369
- om-putEndBVar, 1369
- om-putEndError, 1369
- om-putEndObject, 1370
- om-putError, 1370
- om-putFloat, 1370
- om-putInt, 1370
- om-putObject, 1371
- om-putString, 1371
- om-putSymbol, 1371
- om-putVar, 1371
- om-Read, 1355
- om-setDevEncoding, 1357
- om-stringPtrToString, 1372
- om-stringToStringPtr, 1372
- om-supportsCD, 1356
- om-supportsSymbol, 1356
- openOutputLibrary, 973
- openserver, 1173
- operationopen, 1193
- optionError, 789
- optionUserLevelError, 790
- opTran, 621
- orderBySlotNumber, 114
- originsInOrder, 1636
- parseAndEval, 1542
- parseAndEval1, 1542
- parseAndInterpret, 340
- parseFromString, 340
- parseNoMacroFromString, 1627
- parseSystemCmd, 808
- parseWord, 1558
- pathname, 1232
- pathnameDirectory, 1232
- pathnameName, 1231
- pathnameType, 1231
- pathnameTypeId, 1231
- patternVarsOf, 619
- patternVarsOf1, 619
- pcounters, 99
- permuteToOrder, 756
- pf0ApplicationArgs, 535
- pf0AssignLhsItems, 554
- pf0DefinitionLhsItems, 560
- pf0FlattenSyntacticTuple, 535
- pf0ForinLhs, 565
- pf0FreeItems, 564
- pf0LambdaArgs, 572
- pf0LocalItems, 573
- pf0LoopIterators, 574
- pf0MLambdaArgs, 576
- pf0SequenceArgs, 585

- pf0TupleParts, 591
- pf0WhereContext, 593
- pf2Sex, 597
- pf2Sex1, 598
- pfAbSynOp, 702
- pfAbSynOp?, 702
- pfAdd, 550
- pfAnd, 551
- pfAnd?, 552
- pfAndLeft, 552
- pfAndRight, 553
- pfAppend, 553
- pfApplication, 551
- pfApplication2Sex, 603
- pfApplication?, 553
- pfApplicationArg, 552
- pfApplicationOp, 552
- pfAssign, 553
- pfAssign?, 554
- pfAssignLhsItems, 554
- pfAssignRhs, 554
- pfAttribute, 551
- pfBrace, 555
- pfBraceBar, 555
- pfBracket, 555
- pfBracketBar, 555
- pfBreak, 556
- pfBreak?, 556
- pfBreakFrom, 556
- pfCharPosn, 533
- pfCheckArg, 539
- pfCheckId, 539
- pfCheckItOut, 537
- pfCheckMacroOut, 538
- pfCoerceto, 557
- pfCoerceto?, 557
- pfCoercetoExpr, 557
- pfCoercetoType, 557
- pfCollect, 558
- pfCollect1?, 540
- pfCollect2Sex, 612
- pfCollect?, 558
- pfCollectArgTran, 615
- pfCollectBody, 558
- pfCollectIterators, 558
- pfCollectVariable1, 540
- pfCopyWithPos, 534
- pfDefinition, 559
- pfDefinition2Sex, 613
- pfDefinition?, 559
- pfDefinitionLhsItems, 559
- pfDefinitionRhs, 559
- pfDo, 560
- pfDo?, 560
- pfDoBody, 560
- pfDocument, 544
- pfEnSequence, 561
- pfExit, 561
- pfExit?, 561
- pfExitCond, 561
- pfExitExpr, 562
- pfExport, 562
- pfExpression, 562
- pfFileName, 534
- pfFirst, 562
- pfFix, 563
- pfFlattenApp, 539
- pfForin, 564
- pfForin?, 564
- pfForinLhs, 565
- pfForinWhole, 565
- pfFree, 563
- pfFree?, 563
- pfFreeItems, 564
- pfFromDom, 565
- pfFromdom, 566
- pfFromdom?, 566
- pfFromdomDomain, 567
- pfFromdomWhat, 566
- pfGlobalLinePosn, 533
- pfHide, 567
- pfId, 544
- pfId?, 544
- pfIdPos, 544
- pfIdSymbol, 545
- pfIf, 567
- pfIf?, 567
- pfIfCond, 568
- pfIfElse, 568
- pfIfThen, 568
- pfIfThenOnly, 568
- pfImport, 569

- pfInfApplication, 569
- pfInline, 570
- pfIterate, 569
- pfIterate?, 569
- pfLam, 570
- pfLambda, 571
- pfLambda2Sex, 616
- pfLambda?, 571
- pfLambdaArgs, 572
- pfLambdaBody, 571
- pfLambdaRets, 571
- pfLambdaTran, 614
- pfLeaf, 545
- pfLeaf?, 545
- pfLeafPosition, 546
- pfLeafToken, 546
- pfLhsRule2Sex, 617
- pfLinePosn, 533
- pfListOf, 543
- pfLiteral2Sex, 602
- pfLiteral?, 546
- pfLiteralClass, 546
- pfLiteralString, 547
- pfLocal, 572
- pfLocal?, 572
- pfLocalItems, 573
- pfLoop, 573
- pfLoop1, 573
- pfLoop?, 574
- pfLoopIterators, 574
- pfLp, 574
- pfMacro, 575
- pfMacro?, 575
- pfMacroLhs, 575
- pfMacroRhs, 575
- pfMapParts, 534
- pfMLambda, 576
- pfMLambda?, 576
- pfMLambdaArgs, 576
- pfMLambdaBody, 577
- pfname, 384
- pfNoPosition, 704
- pfNoPosition?, 702
- pfNot?, 577
- pfNotArg, 577
- pfNothing, 543
- pfNothing?, 543
- pfNovalue, 577
- pfNovalue?, 578
- pfNovalueExpr, 578
- pfOp2Sex, 606
- pfOr, 578
- pfOr?, 578
- pfOrLeft, 579
- pfOrRight, 579
- pfParen, 579
- pfParts, 547
- pfPile, 547
- pfPretend, 579
- pfPretend?, 580
- pfPretendExpr, 580
- pfPretendType, 580
- pfPushBody, 547
- pfPushMacroBody, 541
- pfQualType, 580
- pfRestrict, 581
- pfRestrict?, 581
- pfRestrictExpr, 581
- pfRestrictType, 581
- pfRetractTo, 582
- pfReturn, 582
- pfReturn?, 582
- pfReturnExpr, 582
- pfReturnNoName, 583
- pfReturnTyped, 583
- pfRhsRule2Sex, 617
- pfRule, 583
- pfRule2Sex, 616
- pfRule?, 584
- pfRuleLhsItems, 584
- pfRuleRhs, 584
- pfSecond, 584
- pfSequence, 585
- pfSequence2Sex, 608
- pfSequence2Sex0, 608
- pfSequence?, 585
- pfSequenceArgs, 585
- pfSequenceToList, 536
- pfSexpr, 548
- pfSexpr,strip, 548
- pfSourcePosition, 536
- pfSourceStok, 541

- pfSpread, 537
- pfSuch, 542
- pfSuchthat, 586
- pfSuchThat2Sex, 605
- pfSuchthat?, 586
- pfSuchthatCond, 586
- pfSymb, 549
- pfSymbol, 549
- pfSymbol?, 550
- pfSymbolSymbol, 550
- pfTagged, 586
- pfTagged?, 587
- pfTaggedExpr, 587
- pfTaggedTag, 587
- pfTaggedToTyped, 587
- pfTaggedToTyped1, 542
- pfTransformArg, 542
- pfTree, 550
- pfTuple, 590
- pfTuple?, 590
- pfTupleListOf, 590
- pfTupleParts, 591
- pfTweakIf, 588
- pfTyped, 588
- pfTyped?, 589
- pfTypedId, 589
- pfTypedType, 589
- pfTyping, 589
- pfUnSequence, 591
- pfWDec, 591
- pfWDeclare, 592
- pfWhere, 592
- pfWhere?, 592
- pfWhereContext, 593
- pfWhereExpr, 593
- pfWhile, 593
- pfWhile?, 594
- pfWhileCond, 594
- pfWith, 594
- pfWrong, 594
- pfWrong?, 595
- phInterpret, 359
- phIntReportMsgs, 358
- phiRatapprox, 1244
- phMacro, 517
- phParse, 358
- pileCforest, 630
- pileColumn, 627
- pileCtree, 630
- pileForest, 628
- pileForest1, 628
- pileForests, 627
- pilePlusComment, 626
- pilePlusComments, 626
- pileTree, 627
- PiMinusLogSinPi, 1249
- placep, 1297
- pmDontQuote?, 607
- pname, 1235
- poCharPosn, 667
- poFileName, 650
- poGetLineObject, 651
- poGlobalLinePosn, 364
- poLinePosn, 651
- poNopos?, 650
- poNoPosition, 704
- poNoPosition?, 703
- poPosImmediate?, 650
- porigin, 381
- posend, 420
- posPointers, 668
- ppos, 647
- pquit, 938
- pquitSpad2Cmd, 938
- previousInterpreterFrame, 33
- primmathor0, 151
- printAsTeX, 353
- printLabelledList, 814
- printStatisticsSummary, 351
- printStorage, 351
- printSynonyms, 813
- printTypeAndTime, 351
- probeName, 1170
- processChPosesForOneLine, 666
- processInteractive, 342
- processInteractive1, 344
- processKeyedError, 643
- processMsgList, 659
- processSynonymLine, 1107
- processSynonymLine,removeKeyFromLine,
1107
- processSynonyms, 325

- protectedEVAL, [339](#)
- prTraceNames, [145](#)
- prTraceNames,fn, [145](#)
- pSearch, [1595](#)
- PsiAsymptotic, [1260](#)
- PsiAsymptoticOrder, [1259](#)
- PsiBack, [1259](#)
- PsiEps, [1261](#)
- PsiIntpart, [1261](#)
- PsiXotic, [1262](#)
- pspacers, [98](#)
- ptimers, [98](#)
- punctuation?, [409](#)
- put, [1229](#)
- putDatabaseStuff, [655](#)
- putFTText, [669](#)
- putHist, [900](#)
- pvarCondList, [1530](#)
- pvarCondList1, [1530](#)
- pvarPredTran, [620](#)
- pvarsOfPattern, [1531](#)
- qenum, [1306](#)
- qsquotient, [1312](#)
- qsremainder, [1312](#)
- queryClients, [845](#)
- queueUpErrors, [662](#)
- quit, [940](#)
- quitSpad2Cmd, [940](#)
- quoteString, [1544](#)
- quotient2, [1308](#)
- random, [1309](#)
- rassocSub, [104](#)
- rbesseli, [1280](#)
- rbesselj, [1279](#)
- rdefinstream, [1309](#)
- rdefoutstream, [1309](#)
- rdigit?, [424](#)
- read, [943](#)
- readHiFi, [911](#)
- readline, [1661](#)
- readSpad2Cmd, [943](#)
- readSpadProfileIfThere, [1147](#)
- reassembleTowerIntoType, [757](#)
- reclaim, [331](#)
- recordAndPrint, [349](#)
- recordFrame, [1123](#)
- recordNewValue, [901](#)
- recordNewValue0, [901](#)
- recordOldValue, [901](#)
- recordOldValue0, [902](#)
- reduceAlistForDomain, [1615](#)
- redundant, [664](#)
- regress, [950](#)
- remainder2, [1308](#)
- remFile, [647](#)
- remLine, [652](#)
- removeOption, [100](#)
- remover, [115](#)
- removeTracedMapSigs, [127](#)
- removeUndoLines, [57](#)
- renamePatternVariables, [1527](#)
- renamePatternVariables1, [1527](#)
- rep, [661](#)
- replaceFile, [1171](#)
- replacePercentByDollar, [1504](#)
- replacePercentByDollar,fn, [1504](#)
- replaceSharps, [1144](#)
- reportinstantiations, [1271](#)
- reportOperations, [1089](#)
- reportOpsFromLisplib, [1092](#)
- reportOpsFromLisplib0, [1091](#)
- reportOpsFromLisplib1, [1092](#)
- reportOpsFromUnitDirectly, [1096](#)
- reportOpsFromUnitDirectly0, [1095](#)
- reportOpsFromUnitDirectly1, [1100](#)
- reportSpadTrace, [141](#)
- reportUndo, [59](#)
- reportWhatOptions, [1133](#)
- reroot, [332](#)
- resetCounters, [98](#)
- resethashtables, [1186](#)
- resetInCoreHist, [898](#)
- resetSpacers, [97](#)
- resetStackLimits, [312](#)
- resetTimers, [97](#)
- resetWorkspaceVariables, [962](#)
- restart0, [309](#)
- restoreHistory, [907](#)
- resultp, [956](#)
- retract, [1269](#)
- retract2Specialization, [772](#)
- retractByFunction, [778](#)

- retractUnderDomain, 777
- rgamma, 1277
- rgammaImpl, 1245
- rlngamma, 1278
- rpsi, 1278
- rPsiImpl, 1257
- rPsiW, 1258
- rread, 915
- ruleLhsTran, 620
- rulePredicateTran, 617
- runspad, 311
- rwrite, 914
- s-to-c, 1243
- safeWritify, 917
- sameMsg?, 666
- sameUnionBranch, 353
- satisfiesRegularExpressions, 1135
- satisfiesUserLevel, 791
- saveDependentsHashTable, 1208
- saveHistory, 905
- saveMapSig, 117
- savesystem, 959
- saveUsersHashTable, 1208
- sayAllCacheCounts, 982
- sayBrightly1, 1240
- sayCacheCount, 983
- sayExample, 868
- sayKeyedMsg, 43
- sayKeyedMsgLocal, 44
- sayMSG, 45
- sayMSG2File, 44
- sayShowWarning, 1100
- scanCheckRadix, 425
- scanCloser?, 417
- scanComment, 407
- scanDictCons, 428
- scanError, 426
- scanEsc, 414
- scanEscape, 426
- scanExponent, 417
- scanIgnoreLine, 404
- scanInsert, 429
- scanKeyTableCons, 427
- scanKeyTr, 411
- scanNegComment, 408
- scanNumber, 423
- ScanOrPairVec, 926
- ScanOrPairVec,ScanOrInner, 925
- scanPossFloat, 412
- scanPunCons, 430
- scanPunct, 409
- scanS, 422
- scanSpace, 420
- scanString, 421
- scanToken, 405
- scanTransform, 423
- scanW, 419
- scanWord, 417
- search, 1150
- searchCurrentEnv, 1150
- searchTailEnv, 1151
- sec, 1291
- sech, 1293
- segmentKeyedMsg, 44
- selectOption, 819
- selectOptionLC, 819
- separatePiles, 631
- serverReadLine, 336
- set, 1081
- set-restart-hook, 305
- set1, 1082
- setCurrentLine, 333
- setdatabase, 1195
- setExpose, 158
- setExposeAdd, 159
- setExposeAddConstr, 160
- setExposeAddGroup, 157
- setExposeDrop, 161
- setExposeDropConstr, 164
- setExposeDropGroup, 162
- setFortDir, 998
- setFortPers, 1029
- setFortTmpDir, 996
- setFunctionsCache, 979
- setHistoryCore, 894
- setInputLibrary, 974
- setIOindex, 909
- setLinkerArgs, 1001
- setMsgCatlessAttr, 655
- setMsgForcedAttr, 654
- setMsgForcedAttrList, 654
- setMsgUnforcedAttr, 657

- setMsgUnforcedAttrList, 656
- setNagHost, 1027
- setOutputAlgebra, 1035
- setOutputCharacters, 1039
- setOutputFormula, 1063
- setOutputFortran, 1042
- setOutputHtml, 1048
- setOutputLibrary, 972
- setOutputMathml, 1054
- setOutputOpenMath, 1058
- setOutputTex, 1069
- setStreamsCalculate, 1075
- setUpDefault, 1525
- shortenForPrinting, 113
- show, 1087
- showdatabase, 1194
- showHistory, 892
- showInOut, 910
- showInput, 909
- showMsgPos?, 648
- showSpad2Cmd, 1087
- shut, 1168
- size, 1236
- SkipEnd?, 373
- SkipPart?, 373
- Skipping?, 374
- smallEnough, 153
- smallEnoughCount, 153
- Solve - bcInputMatrixByFormula, 1445
- spad, 310
- spad-error-loc, 1156
- spad-long-error, 1155
- spad-save, 1175
- spad-short-error, 1156
- spad-syntax-error, 1155
- spad2BootCoerce, 1271
- spadClosure?, 921
- SpadInterpretStream, 318
- spadReply, 114
- spadrread, 915
- spadrwrite, 915
- spadrwrite0, 914
- spadStartUpMsgs, 309
- spadTrace, 132
- spadTrace,g, 131
- spadTrace,isTraceable, 131
- spadTraceAlias, 141
- spadUntrace, 143
- specialChar, 1166
- spleI, 413
- spleI1, 414
- split, 953
- splitIntoOptionBlocks, 787
- stackTraceOptionError, 100
- startp, 955
- startsComment?, 407
- startsNegComment?, 408
- statisticsInitialization, 1221
- streamChop, 364
- StreamNull, 623
- string2Constructor, 1590
- stringize, 1510
- stringList2String, 1497
- stringMatches?, 1276
- stringPrefix?, 1403
- StringToDir, 1298
- stripLisp, 811
- stripSpaces, 811
- stripUnionTags, 776
- strpos, 1236
- strposl, 1236
- stupidIsSpadFunction, 115
- subMatch, 410
- substFromAlist, 1529
- substring, 324
- substringMatch, 410
- subTypes, 101
- summary, 1103
- syGeneralErrorHere, 484
- syIgnoredFromTo, 484
- synonym, 1105
- synonymsForUserLevel, 1106
- synonymSpad2Cmd, 1105
- sySpecificErrorAtToken, 485
- sySpecificErrorHere, 484
- systemCommand, 788
- tabbing, 652
- templateParts, 1533
- terminateSystemCommand, 792
- tersyscommand, 792
- testnumberp, 952
- testpassed, 953

- thefname, 384
- theid, 383
- theorigin, 381
- thisPosIsEqual, 664
- thisPosIsLess, 664
- throwEvalTypeMsg, 1122
- To, 670
- toFile?, 653
- tokConstruct, 701
- token-stack-show, 1157
- tokPart, 703
- tokPosn, 703
- tokTran, 807
- tokType, 703
- Top?, 372
- topLevelInterpEval, 1626
- trace, 78
- trace1, 79
- trace2, 82
- trace3, 83
- traceDomainConstructor, 137
- traceDomainLocalOps, 136
- tracelet, 154
- traceOptionError, 121
- traceReply, 94
- traceSpad2Cmd, 78
- trademark, 855
- translateTrueFalse2YesNo, 967
- translateYesNo2TrueFalse, 967
- translateYesNoToTrueFalse, 1560
- transOnlyOption, 99
- transTraceItem, 126
- trunclist, 97
- typeCheckInputAreas, 1538
- unAbbreviateKeyword, 809
- undo, 52
- undoChanges, 903
- undoCount, 61
- undoFromFile, 904
- undoInCore, 902
- undoLocalModemapHack, 56
- undoSingleStep, 54
- undoSteps, 53
- unescapeStringsInForm, 354
- unifyStruct, 726
- unifyStructVar, 727
- unparseInputForm, 1307
- untrace, 122
- untraceDomainConstructor, 139
- untraceDomainConstructor,keepTraced?, 138
- untraceDomainLocalOps, 136
- untraceMapSubNames, 105
- unwritable?, 916
- upcase, 1272
- updateCurrentInterpreterFrame, 31
- updateDatabase, 1203
- updateFromCurrentInterpreterFrame, 30
- updateHist, 899
- updateInCoreHist, 900
- updateSourceFiles, 874
- userLevelErrorMessage, 790
- validateOutputDirectory, 997
- valueArgsEqual?, 764
- varp, 89
- vec2list, 1276
- vmread, 1297
- voidValue, 1325
- what, 1131
- whatCommands, 1133
- whatConstructors, 1137
- whatSpad2Cmd, 1131
- whatSpad2Cmd,fixpat, 1131
- whichCat, 655
- workfiles, 1139
- workfilesSpad2Cmd, 1139
- wrap, 1233
- write-browsedb, 1215
- write-categorydb, 1216
- write-interpdb, 1213
- write-operationdb, 1217
- write-warmdata, 1218
- writeHiFi, 912
- writeHistModesAndValues, 913
- writeInputLines, 896
- writify, 920
- writify,writifyInner, 917
- writifyComplain, 916
- xlCannotRead, 384
- xlCmdBug, 391
- xlConActive, 386
- xlConsole, 387

- xlConStill, 387
- xlFileCycle, 385
- xlIfBug, 391
- xlIfSyntax, 390
- xlMsg, 379
- xlNoSuchFile, 383
- xlOK, 379
- xlOK1, 379
- xlPrematureEOF, 378
- xlPrematureFin, 389
- xlSay, 382
- xlSkip, 382
- xlSkippingFin, 388
- yesanswer, 866
- ySearch, 1596
- zeroOneTran, 360
- defvar
 - *ThisIsARunningSystem*, 959
 - *all-tests-ran*, 950
 - *allOperations*, 1186
 - *allconstructors*, 1186
 - *browse-stream*, 1185
 - *browse-stream-stamp*, 1185
 - *category-stream*, 1185
 - *category-stream-stamp*, 1186
 - *defaultdomain-list*, 1182
 - *eof*, 314
 - *hasCategory-hash*, 1183
 - *interp-stream*, 1184
 - *interp-stream-stamp*, 1184
 - *miss*, 1183
 - *monitor-domains*, 1381
 - *monitor-nrlibs*, 1381
 - *monitor-table*, 1382
 - *ok*, 952
 - *operation-hash*, 1183
 - *operation-stream*, 1184
 - *operation-stream-stamp*, 1185
 - *read-place-holder*, 1296
 - *sourcefiles*, 1204
 - *whitespace*, 314
 - /editfile, 849
 - \$nagMessages, 1030
 - \$reportBottomUpFlag, 1019
 - \$AnonymousFunction, 708
 - \$Any, 708
 - \$BFtag, 708
 - \$BigFloat, 710
 - \$Boolean, 709
 - \$BreakMode, 969
 - \$Category, 709
 - \$CommandSynonymAlist, 817
 - \$ComplexInteger, 711
 - \$Domain, 709
 - \$DoubleFloat, 710, 715
 - \$EmptyMode, 708
 - \$EndServerSession, 335
 - \$Exit, 709
 - \$Expression, 709
 - \$Float, 710
 - \$FontTable, 711
 - \$FormalMapVariableList, 17
 - \$FunctionalExpression, 714
 - \$HTCompanionWindowID, 344
 - \$HiFiAccess, 47, 1005
 - \$HistListAct, 48
 - \$HistListLen, 48
 - \$HistList, 47
 - \$HistRecord, 48
 - \$IOindex, 39
 - \$InitialCommandSynonymAlist, 816
 - \$InitialModemapFrame, 198
 - \$Integer, 711
 - \$InteractiveFrame, 39
 - \$InteractiveMode, 315
 - \$Mode, 711
 - \$NeedToSignalSessionManager, 336
 - \$NegativeInteger, 711
 - \$NonNegativeInteger, 712
 - \$NonNullStream, 921
 - \$NonPositiveInteger, 712
 - \$NullStream, 922
 - \$OutputForm, 73, 710
 - \$PatternVariableList, 17
 - \$PositiveInteger, 712
 - \$PrintCompilerMessageIfTrue, 310
 - \$ProcessInteractiveValue, 344
 - \$QuickLet, 73
 - \$QuietCommand, 339
 - \$QuotientField, 713
 - \$RTspecialCharacters, 1164
 - \$RationalNumber, 712

- \$SingleFloat, 714
- \$SingleInteger, 715
- \$SmallInteger, 714
- \$SpadServerName, 202
- \$SpadServer, 201
- \$StringCategory, 713
- \$String, 713
- \$Symbol, 713
- \$ThrowAwayMode, 347
- \$TraceFlag, 75
- \$UserLevel, 1080
- \$Void, 713
- \$abbreviateTypes, 1033
- \$activePageList, 1467
- \$algebraFormat, 1034
- \$algebraOutputFile, 1034
- \$algebraOutputStream, 1035
- \$attrCats, 654
- \$bcParseOnly, 1498
- \$boot, 315
- \$breakCondition, 68
- \$cacheAlist, 979
- \$clearExcept, 835
- \$clearOptions, 834
- \$compileRecurrence, 985
- \$compilingMap, 341
- \$constructors, 68
- \$countList, 68
- \$curpage, 1498
- \$current-directory, 197, 333
- \$currentFrameNum, 335
- \$defaultFortranType, 989
- \$defaultFunctionTargets, 714
- \$defaultSpecialCharacters, 1161
- \$depthAlist, 69
- \$describeOptions, 857
- \$directory-list, 198
- \$displayDroppedMap, 1011
- \$displayMsgNumber, 1018
- \$displayOptions, 863
- \$displaySetValue, 1020
- \$displayStartMsgs, 1021
- \$doNotAddEmptyModeIfTrue, 69
- \$domPvar, 341
- \$domainTraceNameAssoc, 69
- \$domains, 69
- \$embeddedFunctions, 69
- \$envHashTable, 1149
- \$env, 315
- \$e, 315
- \$formulaFormat, 1062
- \$formulaOutputFile, 1062
- \$fortIndent, 987
- \$fortInts2Floats, 986
- \$fortLength, 988
- \$fortPersistence, 1028
- \$fortranArrayStartingIndex, 994
- \$fortranDirectory, 998
- \$fortranFormat, 1041
- \$fortranLibraries, 1000
- \$fortranOptimizationLevel, 993
- \$fortranOutputFile, 1041
- \$fortranPrecision, 990
- \$fortranSegment, 993
- \$fortranTmpDir, 995
- \$fractionDisplayType, 1046
- \$frameAlist, 335
- \$frameMessages, 1013
- \$frameNumber, 335
- \$frameRecord, 51
- \$fromSpadTrace, 70
- \$fullScreenSysVars, 1002
- \$functionTable, 838
- \$funnyBacks, 1545
- \$funnyQuote, 1544
- \$genValue, 346
- \$giveExposureWarning, 1011
- \$globalExposureGroupAlist, 167
- \$highlightAllowed, 1014
- \$historyDirectory, 887
- \$historyDisplayWidth, 1003, 1570
- \$historyFileType, 49, 886
- \$htLineList, 1498
- \$htMacroTable, 1399
- \$htmlFormat, 1047
- \$htmlOutputFile, 1047
- \$imPrGuys, 648
- \$imPrTagGuys, 657
- \$inputPromptType, 1019
- \$instantRecord, 342
- \$intCoerceFailure, 323
- \$intRestart, 317

- `$intSpadReader`, 323
- `$intTopLevel`, 317
- `$internalHistoryTable`, 48
- `$interpOnly`, 341
- `$interpreterFrameName`, 38
- `$interpreterFrameRing`, 38
- `$lambdatatype`, 970
- `$lastUntraced`, 70
- `$letAssoc`, 70
- `$library-directory-list`, 199
- `$linearFormatScripts`, 1067
- `$linelength`, 1051
- `$localExposureDataDefault`, 167
- `$localExposureData`, 166
- `$mapSubNameAlist`, 70
- `$margin`, 1051
- `$mathTraceList`, 71
- `$mathTrace`, 71
- `$mathmlFormat`, 1053
- `$mathmlOutputFile`, 1053
- `$maximumFortranExpressionLength`, 992
- `$minivectorNames`, 341
- `$mkTestFlag`, 349
- `$monitorArgs`, 71
- `$monitorCaller`, 71
- `$monitorDepth`, 71
- `$monitorFunDepth`, 72
- `$monitorName`, 72
- `$monitorPretty`, 72
- `$monitorValue`, 72
- `$msgAlist`, 42
- `$msgDatabaseName`, 199
- `$msgdbNoBlanksAfterGroup`, 43
- `$msgdbNoBlanksBeforeGroup`, 43
- `$msgdbPrims`, 42
- `$msgdbPunct`, 43
- `$nagEnforceDouble`, 1031
- `$nagHost`, 1026
- `$nagMessages`, 1017
- `$ncMsgList`, 318
- `$newPage`, 1401
- `$newcompErrorCount`, 319
- `$newline`, 1570
- `$newspad`, 316
- `$noEvalTypeMsg`, 1122
- `$noParseCommands`, 784
- `$noSubsumption`, 1148
- `$nupos`, 319
- `$npTokToNames`, 497
- `$numberOfEquations`, 1481
- `$numericFailure`, 1311
- `$oldBreakMode`, 1311
- `$oldHistoryFileName`, 886
- `$oldline`, 789
- `$openMathFormat`, 1057
- `$openMathOutputFile`, 1058
- `$openServerIfTrue`, 199
- `$optionAlist`, 73
- `$options`, 73
- `$packages`, 73
- `$pfMacros`, 392
- `$plainRTspecialCharacters`, 1164
- `$plainSpecialCharacters0`, 1162
- `$plainSpecialCharacters1`, 1162
- `$plainSpecialCharacters2`, 1163
- `$plainSpecialCharacters3`, 1163
- `$preLength`, 645
- `$prettyprint`, 1079
- `$previousBindings`, 52
- `$primitiveHtCommands`, 1400
- `$printAnyIfTrue`, 1007
- `$printFortranDecs`, 989
- `$printLoadMsgs`, 1008
- `$printStatsToFile`, 1012
- `$printStatsSummaryIfTrue`, 1022
- `$printTimeIfTrue`, 1023
- `$printTypeIfTrue`, 1024
- `$printVoidIfTrue`, 1025
- `$quitCommandType`, 1073
- `$quitTag`, 311
- `$relative-directory-list`, 200
- `$relative-library-directory-list`, 200
- `$repGuys`, 665
- `$reportBottomUpFlag`, 1009
- `$reportCoerceIfTrue`, 1010
- `$reportCompilation`, 1078
- `$reportInstantiations`, 1015
- `$reportInterpOnly`, 1016
- `$reportOptimization`, 1078
- `$reportSpadtrace`, 74
- `$reportundo`, 52
- `$runTestFlag`, 349

- \$setOptionNames, 1081
- \$sockBufferLength, 336
- \$solutionMethod, 1481
- \$spaceList, 74
- \$spad-errors, 1153
- \$spadroot, 201
- \$specialCharacterAlist, 1165
- \$specialCharacters, 1165
- \$streamCount, 74, 1074
- \$streamsShowAll, 1076
- \$syscommands, 784
- \$systemCommands, 783
- \$systemType, 1480
- \$testingErrorPrefix, 42
- \$testingSystem, 1023
- \$texFormat, 1069
- \$texOutputFile, 1069
- \$timerList, 74
- \$toWhereGuys, 653
- \$tokenCommands, 815
- \$traceDomains, 75
- \$traceErrorStack, 75
- \$traceNames, 76
- \$traceNoisely, 76
- \$traceOptionList, 76
- \$traceSize, 77
- \$traceStream, 77
- \$tracedMapSignatures, 75
- \$tracedSpadModemap, 77
- \$traceletFunctions, 76
- \$traceletflag, 75
- \$underbar, 896
- \$useEditorForShowOutput, 1067
- \$useFullScreenHelp, 1004
- \$useInternalHistoryTable, 887
- \$useIntrinsicFunctions, 991
- \$whatOptions, 1130
- boot-line-stack, 1147
- coerceConvertMmSelection;AL, 752
- creditlist, 195
- curinstream, 313
- curoutstream, 313
- dalymode, 976
- dbDelimiters, 1591
- echo-meta, 1148
- ElseifKeepPart, 371
- ElseifSkipPart, 371
- ElseifSkipToEnd, 371
- ElseKeepPart, 371
- ElseSkipToEnd, 371
- errorinstream, 313
- erroroutstream, 314
- file-closed, 1148
- IfKeepPart, 370
- IfSkipPart, 370
- IfSkipToEnd, 370
- in-stream, 1147
- incCommands, 392
- infgeneric, 401
- input-libraries, 975
- line-handler, 1153
- localVars, 308
- npPParg, 502
- out-stream, 1148
- output-library, 973
- PsiAsymptoticBern, 1260
- scanCloser, 416
- scanDict, 428
- scanKeyTable, 427
- scanKeyWords, 399
- scanPun, 430
- StreamNil, 397
- Top, 370
- xtokenreader, 1154
- delasc, 1661
 - defun, 1661
- Delay, 397
 - calledby incAppend, 380
 - calledby incIgen, 367
 - calledby incLude, 369
 - calledby incRgen, 397
 - calledby incZip, 366
 - calledby next, 330
 - defun, 397
- deldatabase, 1195
 - calledby abbreviationsSpad2Cmd, 823
 - defun, 1195
- delete
 - calledby breaklet, 155
 - calledby domainDescendantsOf, 1603
 - calledby setExposeAddConstr, 160
 - calledby setExposeDropConstr, 164

- calledby setExposeDropGroup, 163
 - calledby trace1, 79
 - calledby tracelet, 154
 - calledby untraceDomainConstructor, 139
 - calledby workfilesSpad2Cmd, 1139
- deleteAssoc
 - calledby clearCmdParts, 841
- deleteFile, 1233
 - calledby clearCmdAll, 839
 - calls erase, 1233
 - calls pathname, 1233
 - uses \$erase, 1233
 - defun, 1233
- delt, 1296
 - defmacro, 1296
- descendantsOf
 - calledby kcdPage, 1616
- describe, 857, 858
 - calls describespad2cmd, 858
 - defun, 858
 - manpage, 857
- describeFortPersistence, 1029
 - calledby setFortPers, 1029
 - calls sayBrightly, 1029
 - uses \$fortPersistence, 1029
 - defun, 1029
- describeInputLibraryArgs, 975
 - calledby setInputLibrary, 974
 - calls sayBrightly, 975
 - defun, 975
- describeOutputLibraryArgs, 972
 - calledby setOutputLibrary, 972
 - calls sayBrightly, 972
 - defun, 972
- describeSetFortDir, 999
 - calledby setFortDir, 998
 - calls sayBrightly, 999
 - uses \$fortranDirectory, 999
 - defun, 999
- describeSetFortTmpDir, 997
 - calledby setFortTmpDir, 996
 - calls sayBrightly, 997
 - uses \$fortranTmpDir, 997
 - defun, 997
- describeSetFunctionsCache, 982
 - calledby setFunctionsCache, 979
 - calls sayBrightly, 982
 - defun, 982
- describeSetLinkerArgs, 1001
 - calledby setLinkerArgs, 1001
 - calls sayBrightly, 1001
 - uses \$fortranLibraries, 1001
 - defun, 1001
- describeSetNagHost, 1027
 - calledby setNagHost, 1027
 - calls sayBrightly, 1027
 - uses \$nagHost, 1027
 - defun, 1027
- describeSetOutputAlgebra, 1037
 - calledby setOutputAlgebra, 1035
 - calls sayBrightly, 1037
 - calls setOutputAlgebra, 1037
 - defun, 1037
- describeSetOutputFormula, 1065
 - calledby setOutputFormula, 1063
 - calls sayBrightly, 1065
 - calls setOutputFormula, 1066
 - defun, 1065
- describeSetOutputFortran, 1045
 - calledby setOutputFortran, 1042
 - calls sayBrightly, 1045
 - calls setOutputFortran, 1045
 - defun, 1045
- describeSetOutputHtml, 1050
 - calledby setOutputHtml, 1048
 - calls sayBrightly, 1050
 - calls setOutputHtml, 1050
 - defun, 1050
- describeSetOutputMathml, 1056
 - calledby setOutputMathml, 1054
 - calls sayBrightly, 1056
 - calls setOutputMathml, 1056
 - defun, 1056
- describeSetOutputOpenMath, 1061
 - calledby setOutputOpenMath, 1058
 - calls sayBrightly, 1061
 - calls setOutputOpenMath, 1061
 - defun, 1061
- describeSetOutputTex, 1072
 - calledby setOutputTex, 1070
 - calls sayBrightly, 1072
 - calls setOutputTex, 1072

- defun, 1072
- describeSetStreamsCalculate, 1075
 - calledby setStreamsCalculate, 1075
 - calls sayKeyedMsg, 1075
 - uses \$streamCount, 1076
 - defun, 1075
- describeSpad2Cmd, 858
 - calls cleanline, 858
 - calls flatten, 858
 - calls getdatabase, 858
 - calls sayMessage, 858
 - calls selectOptionLC, 858
 - uses \$EmptyEnvironment, 858
 - uses \$describeOptions, 858
 - uses \$e, 858
 - defun, 858
- describepad2cmd
 - calledby describe, 858
- desiredMsg, 643
 - calledby ncHardError, 642
 - calledby ncSoftError, 642
 - defun, 643
- devaluate
 - calledby /tracereply, 142
 - calledby /untrace-2, 125
 - calledby ?t, 147
 - calledby addTraceItem, 146
 - calledby dbAddChainDomain, 1640
 - calledby dbSearchOrder, 1610
 - calledby mkEvalable, 1116
 - calledby prTraceNames,fn, 145
 - calledby shortenForPrinting, 113
 - calledby spadUntrace, 143
 - calledby trace1, 79
 - calledby transTraceItem, 126
 - calledby untraceDomainConstructor,keepTraced, 138
 - calledby writify,writifyInner, 917
- dewritify, 925
 - calledby SPADRREAD, 915
 - calls ScanOrPairVec, 925
 - calls dewritify,dewritifyInner, 925
 - calls function, 925
 - uses \$seen, 925
 - defun, 925
- dewritify,dewritifyInner, 922
 - calledby dewritify,dewritifyInner, 922
 - calledby dewritify, 925
 - calls concat, 922
 - calls dewritify,dewritifyInner, 922
 - calls error, 922
 - calls exit, 922
 - calls gensymmer, 922
 - calls hget, 922
 - calls hput, 922
 - calls intp, 922
 - calls make-instream, 922
 - calls poundsign, 922
 - calls qcar, 922
 - calls qcdr, 922
 - calls qrplaca, 922
 - calls qrplacd, 922
 - calls qsetvelt, 922
 - calls qvelt, 922
 - calls qvmaxindex, 922
 - calls seq, 922
 - calls vecp, 922
 - calls vmread, 922
 - uses \$NonNullStream, 922
 - uses \$NullStream, 922
 - uses \$seen, 922
 - defun, 922
- DFAcos, 1286
 - defmacro, 1286
- DFAcosh, 1288
 - defmacro, 1288
- DFAdd, 1281
 - defmacro, 1281
- DFAsin, 1286
 - defmacro, 1286
- DFAsinh, 1288
 - defmacro, 1288
- DFAtan, 1286
 - defmacro, 1286
- DFAtan2, 1286
 - defmacro, 1286
- DFAtanh, 1289
 - defmacro, 1289
- DFCos, 1285
 - defmacro, 1285
- DFCosh, 1287
 - defmacro, 1287

- DFDivide, 1283
 - defmacro, 1283
- DFEqL, 1283
 - defmacro, 1283
- DFExp, 1285
 - defmacro, 1285
- DFExpt, 1284
 - defmacro, 1284
- DFIntegerDivide, 1283
 - defmacro, 1283
- DFIntegerExpt, 1284
 - defmacro, 1284
- DFIntegerMultiply, 1282
 - defmacro, 1282
- DFLessThan, 1277
 - defmacro, 1277
- DFLog, 1284
 - defmacro, 1284
- DFLogE, 1284
 - defmacro, 1284
- DFMax, 1282
 - defmacro, 1282
- DFMin, 1282
 - defmacro, 1282
- DFMinusp, 1281
 - defmacro, 1281
- DFMultiply, 1282
 - defmacro, 1282
- DFSin, 1285
 - defmacro, 1285
- DFSinh, 1287
 - defmacro, 1287
- DFSqrt, 1283
 - defmacro, 1283
- DFSubtract, 1281
 - defmacro, 1281
- DFTan, 1285
 - defmacro, 1285
- DFTanh, 1288
 - defmacro, 1288
- DFUnaryMinus, 1280
 - defmacro, 1280
- DFZerop, 1281
 - defmacro, 1281
- diffAlist, 1125
 - calledby recordFrame, 1123
- calls assoc, 1125
- calls assq, 1125
- calls exit, 1125
- calls lassq, 1125
- calls reportUndo, 1125
- calls seq, 1125
- calls tmp1, 1125
- defun, 1125
- dig2fix
 - calledby isSharpVarWithNum, 109
- digit?, 413
 - calledby scanExponent, 418
 - calls digitp, 413
 - defun, 413
- digitp, 1235
 - calledby digit?, 413
 - calledby digitp, 1235
 - calledby isGenvar, 126
 - calledby isSharpVarWithNum, 109
 - calls digitp, 1235
 - defun, 1235
- DirToString, 1298
 - calledby fnameDirectory, 1298
 - defun, 1298
- disableHist, 913
 - calledby fetchOutput, 910
 - calledby historySpad2Cmd, 890
 - calledby restoreHistory, 907
 - calledby setHistoryCore, 894
 - calledby showInOut, 910
 - calledby showInput, 909
 - calledby undoFromFile, 904
 - calledby undoInCore, 902
 - calledby updateHist, 899
 - calls histFileErase, 913
 - calls histFileName, 913
 - uses \$HiFiAccess, 913
 - defun, 913
- display, 862, 864
 - calls displayspad2cmd, 864
 - defun, 864
 - manpage, 862
- displayCondition, 804
 - calledby displayProperties, 801
 - calls bright, 804
 - calls concat, 804

- calls `pred2English`, 804
 - calls `sayBrightly`, 804
 - `defun`, 804
- `displayDomainOp`
 - calledby `dbShowConsDoc1`, 1646
- `displayExposedConstructors`, 165
 - calledby `setExposeAddConstr`, 161
 - calledby `setExposeAddGroup`, 157
 - calledby `setExposeAdd`, 159
 - calledby `setExposeDropConstr`, 164
 - calledby `setExposeDropGroup`, 162
 - calledby `setExpose`, 158
 - calls `sayKeyedMsg`, 165
 - uses `$localExposureData`, 165
 - `defun`, 165
- `displayExposedGroups`, 165
 - calledby `setExposeAddGroup`, 157
 - calledby `setExposeAdd`, 159
 - calledby `setExposeDropGroup`, 162
 - calledby `setExpose`, 158
 - calls `sayKeyedMsg`, 165
 - uses `$interpreterFrameName`, 165
 - uses `$localExposureData`, 165
 - `defun`, 165
- `displayFrameNames`, 28
 - calledby `frameSpad2Cmd`, 26
 - calls `bright`, 28
 - calls `framename`, 28
 - uses `$interpreterFrameRing`, 28
 - `defun`, 28
- `displayHiddenConstructors`, 166
 - calledby `setExposeAddGroup`, 157
 - calledby `setExposeDropConstr`, 164
 - calledby `setExposeDropGroup`, 162
 - calledby `setExposeDrop`, 161
 - calledby `setExpose`, 158
 - calls `sayKeyedMsg`, 166
 - uses `$localExposureData`, 166
 - `defun`, 166
- `displayMacro`, 794
 - calledby `displayMacros`, 867
 - calledby `displayProperties`, 801
 - calls `bright`, 794
 - calls `concat`, 794
 - calls `isInterpMacro`, 794
 - calls `mathprint`, 794
 - calls `object2String`, 794
 - calls `sayBrightly`, 794
 - uses `$op`, 794
 - `defun`, 794
- `displayMacros`, 867
 - calledby `displaySpad2Cmd`, 864
 - calls `displayMacro`, 867
 - calls `displayParserMacro`, 867
 - calls `exit`, 867
 - calls `getInterpMacroNames`, 867
 - calls `getParserMacroNames`, 867
 - calls `member`, 867
 - calls `remdup`, 867
 - calls `sayBrightly`, 867
 - calls `seq`, 867
 - `defun`, 867
- `displayMode`, 806
 - calledby `displayProperties`, 801
 - calls `bright`, 806
 - calls `concat`, 806
 - calls `fixObjectForPrinting`, 806
 - calls `prefix2String`, 806
 - calls `sayBrightly`, 806
 - `defun`, 806
- `displayModemap`, 806
 - calledby `displayProperties`, 801
 - calls `bright`, 806
 - calls `concat`, 806
 - calls `formatSignature`, 806
 - calls `sayBrightly`, 806
 - `defun`, 806
- `displayOperations`, 866
 - calledby `displaySpad2Cmd`, 864
 - calls `reportOpSymbol`, 866
 - calls `sayKeyedMsg`, 866
 - calls `yesanswer`, 866
 - `defun`, 866
- `displayOperationsFromLisplib`, 1095
 - calledby `reportOpsFromLisplib`, 1093
 - calls `eqsubstlist`, 1095
 - calls `formatOperationAlistEntry`, 1095
 - calls `getdatabase`, 1095
 - calls `msort`, 1095
 - calls `remdup`, 1095
 - calls `say2PerLine`, 1095
 - calls `specialChar`, 1095

- uses `$FormalMapVariableList`, 1095
 - uses `$linelength`, 1095
 - defun, 1095
- `displayParserMacro`, 804
 - calledby `displayMacros`, 867
 - calledby `displayProperties`, 801
 - calls `pfPrintSrcLines`, 804
 - uses `$pfMacros`, 804
 - defun, 804
- `displayProperties`, 801
 - calledby `displaySpad2Cmd`, 864
 - calls `bright`, 801
 - calls `displayCondition`, 801
 - calls `displayMacro`, 801
 - calls `displayModemap`, 801
 - calls `displayMode`, 801
 - calls `displayParserMacro`, 801
 - calls `displayProperties,sayFunctionDeps`, 801
 - calls `displayType`, 801
 - calls `displayValue`, 801
 - calls `exit`, 801
 - calls `fixObjectForPrinting`, 801
 - calls `getAndSay`, 801
 - calls `getIProplist`, 801
 - calls `getInterpMacroNames`, 801
 - calls `getI`, 801
 - calls `getParserMacroNames`, 801
 - calls `getWorkspaceNames`, 801
 - calls `interpFunctionDepAlists`, 801
 - calls `isInternalMapName`, 801
 - calls `isInterpMacro`, 801
 - calls `member`, 801
 - calls `msort`, 801
 - calls `prefix2String`, 801
 - calls `qcar`, 801
 - calls `qcdr`, 801
 - calls `remdup`, 801
 - calls `sayKeyedMsg`, 801
 - calls `sayMSG`, 801
 - calls `seq`, 801
 - calls `terminateSystemCommand`, 801
 - uses `$dependeeAlist`, 801
 - uses `$dependentAlist`, 801
 - uses `$frameMessages`, 801
 - uses `$interpreterFrameName`, 801
 - defun, 801
- `displayProperties,sayFunctionDeps`, 796
 - calledby `displayProperties`, 801
 - calls `bright`, 798
 - calls `exit`, 798
 - calls `getalist`, 798
 - calls `sayMSG`, 798
 - calls `seq`, 798
 - uses `$dependeeAlist`, 798
 - uses `$dependentAlist`, 798
 - defun, 796
- `displayRule`
 - calledby `displayValue`, 799
- `displaySetOptionInformation`, 964
 - calledby `set1`, 1082
 - calls `boot-equal`, 964
 - calls `bright`, 964
 - calls `concat`, 964
 - calls `displaySetVariableSettings`, 964
 - calls `eval`, 964
 - calls `literals`, 964
 - calls `object2String`, 964
 - calls `sayBrightly`, 964
 - calls `sayMSG`, 964
 - calls `sayMessage`, 964
 - calls `specialChar`, 964
 - calls `translateTrueFalse2YesNo`, 964
 - uses `$linelength`, 964
 - defun, 964
- `displaySetVariableSettings`, 965
 - calledby `displaySetOptionInformation`, 964
 - calledby `set1`, 1082
 - calls `bright`, 965
 - calls `concat`, 965
 - calls `eval`, 965
 - calls `fillerSpaces`, 965
 - calls `literals`, 965
 - calls `object2String`, 965
 - calls `poundsign`, 965
 - calls `satisfiesUserLevel`, 965
 - calls `sayBrightly`, 965
 - calls `say`, 965
 - calls `specialChar`, 965
 - calls `translateTrueFalse2YesNo`, 965
 - calls `tree`, 965
 - uses `$linelength`, 965

- defun, 965
- displaySpad2Cmd
 - calls abbQuery, 864
 - calls displayMacros, 864
 - calls displayOperations, 864
 - calls displayProperties, 864
 - calls displayWorkspaceNames, 864
 - calls listConstructorAbbreviations, 864
 - calls opOf, 864
 - calls sayMessage, 864
 - calls selectOptionLC, 864
 - uses \$EmptyEnvironment, 864
 - uses \$displayOptions, 864
 - uses \$e, 864
- displayspad2cmd
 - calledby display, 864
- displayType, 800
 - calledby displayProperties, 801
 - calls concat, 800
 - calls fixObjectForPrinting, 800
 - calls objMode, 800
 - calls pname, 800
 - calls prefix2String, 800
 - calls sayMSG, 800
 - uses \$op, 800
 - defun, 800
- displayValue, 799
 - calledby displayProperties, 801
 - calls concat, 799
 - calls displayRule, 799
 - calls fixObjectForPrinting, 799
 - calls form2String, 799
 - calls getdatabase, 799
 - calls mathprint, 799
 - calls objMode, 799
 - calls objValUnwrap, 799
 - calls outputFormat, 799
 - calls pname, 799
 - calls prefix2String, 799
 - calls sayMSG, 799
 - uses \$EmptyMode, 799
 - uses \$op, 799
 - defun, 799
- displayWorkspaceNames, 794
 - calledby displaySpad2Cmd, 864
 - calls getInterpMacroNames, 794
 - calls getParserMacroNames, 794
 - calls getWorkspaceNames, 795
 - calls msort, 795
 - calls sayAsManyPerLineAsPossible, 795
 - calls sayBrightly, 795
 - calls sayMessage, 794
 - calls setdifference, 795
 - defun, 794
- divide2, 1308
 - defun, 1308
- dKind, 1589
 - defun, 1589
- dlen, 1295
 - defmacro, 1295
- doDoitButton, 1469
 - defun, 1469
- domainDescendantsOf, 1603
 - calledby kArgPage, 1602
 - calls assoc, 1603
 - calls delete, 1603
 - calls domainsOf, 1603
 - calls function, 1603
 - calls ifcdr, 1603
 - calls listSort, 1603
 - calls qcar, 1603
 - calls qcdr, 1603
 - calls quickAnd, 1603
 - calls simpHasPred, 1603
 - calls systemError, 1603
 - defun, 1603
- domainsOf
 - calledby domainDescendantsOf, 1603
 - calledby kcdoPage, 1617
 - calledby make-databases, 1205
- domainToGenvar, 100
 - calledby getTraceOption,hn, 118
 - calledby transTraceItem, 126
 - calls evalDomain, 101
 - calls genDomainTraceName, 101
 - calls getdatabase, 101
 - calls opOf, 101
 - calls unabbrevAndLoad, 101
 - uses \$doNotAddEmptyModeIfTrue, 101
 - defun, 100
- domArg, 720
 - calledby hasCaty, 718

- calledby mkDomPvar, [731](#)
 - local ref \$FormalMapVariableList, [720](#)
 - defun, [720](#)
- domArg2, [720](#)
 - calledby hasCaty, [718](#)
 - calls isSharpVar, [720](#)
 - calls subCopy, [720](#)
 - local ref \$domPvar, [720](#)
 - defun, [720](#)
- done
 - calledby insertPos, [669](#)
- doSystemCommand, [786](#)
 - calledby ExecuteInterpSystemCommand, [324](#)
 - calls concat, [786](#)
 - calls expand-tabs, [786](#)
 - calls getFirstWord, [786](#)
 - calls handleNoParseCommands, [786](#)
 - calls handleParsedSystemCommands, [786](#)
 - calls handleTokenizeSystemCommands, [786](#)
 - calls member, [786](#)
 - calls processSynonyms, [786](#)
 - calls splitIntoOptionBlocks, [786](#)
 - calls substring, [786](#)
 - calls unAbbreviateKeyword, [786](#)
 - uses \$noParseCommands, [786](#)
 - uses \$tokenCommands, [786](#)
 - uses line, [786](#)
 - defun, [786](#)
- downcase, [1272](#)
 - calledby apropos, [1137](#)
 - calledby conPage, [1599](#)
 - calledby constructorSearch, [1597](#)
 - calledby dbShowCons, [1641](#)
 - calledby downcase, [1272](#)
 - calledby koaPageFilterByName, [1634](#)
 - calledby selectOptionLC, [819](#)
 - calledby set1, [1082](#)
 - calledby setOutputCharacters, [1039](#)
 - calledby string2Constructor, [1590](#)
 - calledby whatSpad2Cmd,fixpat, [1131](#)
 - calls downcase, [1272](#)
 - calls identp, [1272](#)
 - defun, [1272](#)
- downlink, [1570](#)
 - calledby conPage, [1599](#)
 - calledby constructorSearch, [1597](#)
 - defun, [1570](#)
- dqAppend, [634](#)
 - calledby dqConcat, [633](#)
 - calledby pileCtree, [630](#)
 - defun, [634](#)
- dqConcat, [633](#)
 - calledby dqConcat, [633](#)
 - calledby enPile, [630](#)
 - calledby separatePiles, [631](#)
 - calls dqAppend, [633](#)
 - calls dqConcat, [633](#)
 - defun, [633](#)
- dqToList, [634](#)
 - calledby intloopEchoParse, [361](#)
 - calledby nloopParse, [330](#)
 - defun, [634](#)
- dqUnit, [633](#)
 - calledby enPile, [630](#)
 - calledby lineoftoks, [403](#)
 - calledby scanToken, [406](#)
 - calledby separatePiles, [631](#)
 - defun, [633](#)
- drop[9]
 - called by frameSpad2Cmd, [26](#)
- dropInputLibrary, [976](#)
 - calledby addInputLibrary, [975](#)
 - calledby openOutputLibrary, [973](#)
 - calledby setInputLibrary, [974](#)
 - uses input-libraries, [976](#)
 - defun, [976](#)
- dSearch, [1595](#)
 - calls constructorSearch, [1595](#)
 - defun, [1595](#)
- dsetaref2, [1294](#)
 - defmacro, [1294](#)
- dsetelt, [1296](#)
 - defmacro, [1296](#)
- dumbTokenize, [807](#)
 - calledby handleParsedSystemCommands, [808](#)
 - calledby handleTokenizeSystemCommands, [787](#)
 - calledby parseSystemCmd, [808](#)
 - calls stripSpaces, [807](#)

- defun, 807
- echo-meta, 1148
 - defvar, 1148
- edit, 871, 872
 - calls editSpad2Cmd, 872
 - defun, 872
 - manpage, 871
- editFile, 873
 - calledby editSpad2Cmd, 872
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calls concat, 873
 - calls namestring, 873
 - calls obey, 873
 - calls pathname, 873
 - defun, 873
- editSpad2Cmd, 872
 - calledby edit, 872
 - calls \$FINDFILE, 872
 - calls editFile, 872
 - calls pathnameDirectory, 872
 - calls pathnameName, 872
 - calls pathnameType, 872
 - calls pathname, 872
 - calls updateSourceFiles, 872
 - uses /editfile, 872
 - defun, 872
- Else?, 373
 - calledby xIfSyntax, 390
 - calls quotient, 373
 - defun, 373
- Elseif?, 372
 - calledby incLude1, 375
 - calls quotient, 372
 - defun, 372
- ElseifKeepPart, 371
 - defvar, 371
- ElseifSkipPart, 371
 - defvar, 371
- ElseifSkipToEnd, 371
 - defvar, 371
- ElseKeepPart, 371
 - defvar, 371
- ElseSkipToEnd, 371
 - defvar, 371
- eltU16, 1317
 - defmacro, 1317
- eltU32, 1318
 - defmacro, 1318
- eltU8, 1316
 - defmacro, 1316
- embed
 - calledby traceDomainConstructor, 137
- embed2, 88
 - calledby trace3, 84
 - defun, 88
- embeddedFunction, 88
 - calledby trace3, 84
 - calls flatBvList, 88
 - defun, 88
- emptyInterpreterFrame, 27
 - calledby addNewInterpreterFrame, 34
 - calledby initializeInterpreterFrameRing, 27
 - uses \$HiFiAccess, 27
 - uses \$HistListAct, 27
 - uses \$HistListLen, 27
 - uses \$HistList, 27
 - uses \$HistRecord, 27
 - uses \$localExposureDataDefault, 27
 - defun, 27
- emptySearchPage
 - calledby dbShowCons, 1641
- enable-backtrace
 - calledby ncBug, 658
- endedp, 955
 - calledby split, 954
 - defun, 955
- enPile, 630
 - calledby pileCforest, 630
 - calls dqConcat, 630
 - calls dqUnit, 630
 - calls firstTokPosn, 630
 - calls lastTokPosn, 630
 - calls tokConstruct, 630
 - defun, 630
- entryWidth
 - calledby printLabelledList, 814
- eof
 - usedby fin, 876
- eofp, 1169

- defun, 1169
- eqcar
 - calledby /untrace-2, 124
 - calledby StreamNull, 623
 - calledby hasCateSpecialNew, 734
 - calledby hasCateSpecial, 733
 - calledby monitorPrintValue, 152
 - calledby pfAbSynOp?, 702
 - calledby poNoPosition?, 703
- eqpileTree, 629
 - calledby pileForest1, 628
 - calls npNull, 629
 - calls pileColumn, 629
 - calls pileForests, 629
 - defun, 629
- eqsubstlist
 - calledby displayOperationsFromLisplib, 1095
 - calledby reportOpsFromLisplib, 1092
- erase
 - calledby deleteFile, 1233
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1208
- erMsgCompare, 660
 - calls compareposns, 660
 - calls getMsgPos, 660
 - defun, 660
- erMsgSep, 660
 - calledby erMsgSort, 659
 - calls getMsgPos, 660
 - calls poNopos?, 660
 - defun, 660
- erMsgSort, 659
 - calledby processMsgList, 659
 - calls erMsgSep, 659
 - calls listSort, 659
 - defun, 659
- error
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1226
 - calledby charDigitVal, 927
 - calledby dewritify, dewritifyInner, 922
 - calledby gensymInt, 927
 - calledby myWritable?, 1300
- errorinstream, 313
 - defvar, 313
- erroroutstream, 314
 - defvar, 314
- errorPage
 - calledby kcPage, 1611
 - calledby kcaPage1, 1617
 - calledby kccPage, 1618
 - calledby kcnPage, 1622
 - calledby kcpPage, 1614
 - calledby kePage, 1605
 - calledby kiPage, 1605
 - calledby koPage, 1631
 - calledby ksPage, 1609
- eval
 - calledby NRTEvalDomain, 1229
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby evalDomain, 1115
- evalCategory, 1145
 - calledby evaluateType1, 1120
 - calls isPartialMode, 1145
 - calls ofCategory, 1145
 - defun, 1145
- evalDomain, 1115
 - calledby NRTEvalDomain, 1229
 - calledby algEqual, 765
 - calledby coerceInt1, 743
 - calledby coerceIntByMap, 761
 - calledby domainToGenvar, 101
 - calledby getConstantFromDomain, 767
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby retractByFunction, 778
 - calls concat, 1115
 - calls eval, 1115
 - calls mkEvalable, 1115
 - calls prefix2String, 1115
 - calls sayMSG, 1115
 - calls startTimingProcess, 1115
 - calls stopTimingProcess, 1115
 - local ref \$evalDomain, 1115
 - defun, 1115
- evalSharpOne, 776
 - calledby coerceUnion2Branch, 775
 - defun, 776
- evaluateSignature, 1123

- calledby evaluateType, 1118
- calls evaluateType, 1123
- defun, 1123
- evaluateType, 1118
 - calledby evaluateSignature, 1123
 - calledby evaluateType1, 1120
 - calledby evaluateType, 1119
 - calledby reportOperations, 1089
 - calls bottomUp, 1118
 - calls constructor?, 1119
 - calls evaluateSignature, 1118
 - calls evaluateType, 1119
 - calls getValue, 1118
 - calls isDomainValuedVariable, 1118
 - calls member, 1118
 - calls mkAtree, 1118
 - calls objVal, 1118
 - calls qcar, 1118
 - calls qcdr, 1118
 - calls throwEvalTypeMsg, 1119
 - local def \$expandSegments, 1119
 - local ref \$EmptyMode, 1119
 - defun, 1118
- evaluateType1, 1120
 - calls bottumUp, 1120
 - calls categoryForm?, 1120
 - calls coerceOrRetract, 1120
 - calls constructor?, 1120
 - calls evalCategory, 1120
 - calls evaluateType, 1120
 - calls getAndEvalConstructorArgument, 1120
 - calls getConstructorSignature, 1120
 - calls getdatabase, 1120
 - calls makeOrdinal, 1121
 - calls mkAtree, 1120
 - calls objValUnwrap, 1120
 - calls putTarget, 1120
 - calls qcar, 1120
 - calls qcdr, 1120
 - calls replaceSharps, 1120
 - calls throwEvalTypeMsg, 1120
 - calls throwKeyedMsgCannotCoerceWithValue, 1120
 - local ref \$EmptyMode, 1121
 - local ref \$quadSymbol, 1121
 - defun, 1120
- executeInterpreterCommand, 1470
 - defun, 1470
- ExecuteInterpSystemCommand, 324
 - calledby InterpExecuteSpadSystemCommand, 323
 - calls doSystemCommand, 324
 - calls intProcessSynonyms, 324
 - calls substring, 324
 - uses \$currentLine, 324
 - defun, 324
- executeQuietCommand, 339
 - calledby serverReadLine, 336
 - calls make-string, 339
 - calls parseAndInterpret, 339
 - calls sockGetString, 339
 - uses \$MenuServer, 339
 - uses \$QuietCommand, 339
 - catches, 339
 - defun, 339
- exit
 - calledby /tracereply, 142
 - calledby abbreviationsSpad2Cmd, 823
 - calledby apropos, 1137
 - calledby clearCmdParts, 841
 - calledby coerceSpadArgs2E, 128
 - calledby dewritify, dewritifyInner, 922
 - calledby diffAlist, 1125
 - calledby displayMacros, 867
 - calledby displayProperties, sayFunctionDeps, 798
 - calledby displayProperties, 801
 - calledby funfind, LAM, 106
 - calledby getAliasIfTracedMapParameter, 140
 - calledby getBpiNameIfTracedMap, 140
 - calledby getPreviousMapSubNames, 103
 - calledby getTraceOption, hn, 117
 - calledby getTraceOptions, 116
 - calledby getTraceOption, 118
 - calledby historySpad2Cmd, 890
 - calledby importFromFrame, 35
 - calledby isListOfIdentifiersOrStrings, 102
 - calledby isListOfIdentifiers, 102
 - calledby orderBySlotNumber, 114
 - calledby prTraceNames, fn, 145
 - calledby prTraceNames, 145

- calledby recordFrame, 1123
- calledby removeUndoLines, 57
- calledby reportUndo, 60
- calledby runspad, 311
- calledby set1, 1082
- calledby spadTrace,isTraceable, 131
- calledby spadTrace, 132
- calledby spadUntrace, 143
- calledby subTypes, 101
- calledby trace1, 79
- calledby traceDomainConstructor, 137
- calledby traceReply, 94
- calledby undoFromFile, 904
- calledby undoLocalModemapHack, 56
- calledby undoSingleStep, 55
- calledby untraceDomainConstructor,keepTraced, 138
- calledby untraceDomainConstructor, 139
- calledby whatConstructors, 1137
- calledby whatSpad2Cmd, 1131
- calledby writify,writifyInner, 917
- expand-tabs
 - calledby doSystemCommand, 786
 - calledby incLude1, 375
- explainExact
 - calledby finalExactRequest, 1491
- explainLinear, 1491
 - calledby linearFinalRequest, 1490
 - calls systemError, 1491
 - defun, 1491
- extendLocalLibdb
 - calledby library, 1199
- extractFileNameFromPath
 - calledby kdPageInfo, 1601
- extractHasArgs
 - calledby augmentHasArgs, 1619
- fbpip
 - calledby mkEvalable, 1116
- fetchOutput, 910
 - calls assq, 910
 - calls boot-equal, 910
 - calls disableHist, 910
 - calls getI, 910
 - calls readHiFi, 910
 - calls throwKeyedMsg, 910
- defun, 910
- file-closed, 1148
 - usedby init-boot/spad-reader, 1154
- defvar, 1148
- filep
 - calledby setOutputLibrary, 972
- fillerSpaces, 310
 - calledby displaySetVariableSettings, 965
 - calledby justifyMyType, 354
 - calledby printLabelledList, 814
 - calledby spadStartUpMsgs, 309
 - calls ifcar, 310
 - defun, 310
- filterAndFormatConstructors, 1136
 - calledby whatSpad2Cmd, 1131
 - calls blankList, 1136
 - calls filterListOfStringsWithFn, 1136
 - calls function, 1136
 - calls pp2Cols, 1136
 - calls sayMessage, 1136
 - calls specialChar, 1136
 - calls whatConstructors, 1136
 - uses \$linelength, 1136
 - defun, 1136
- filterListOfStrings, 1134
 - calledby apropos, 1137
 - calledby whatCommands, 1133
 - calls satisfiesRegularExpressions, 1134
 - defun, 1134
- filterListOfStringsWithFn, 1135
 - calledby filterAndFormatConstructors, 1136
 - calledby printSynonyms, 813
 - calls satisfiesRegularExpressions, 1135
 - defun, 1135
- fin, 876
 - uses eof, 876
 - defun, 876
 - manpage, 876
 - throws, 876
- finalExactRequest, 1491
 - calls bcQueryInteger, 1491
 - calls explainExact, 1491
 - calls moreExactSolution, 1491
 - calls sayBrightly, 1491
 - defun, 1491
- findfile

- calledby bcUnixTable, 1650
- calledby readSpad2Cmd, 943
- findFrameInRing, 31
 - calledby changeToNamedInterpreterFrame, 32
 - calls boot-equal, 31
 - calls frameName, 32
 - uses \$interpreterFrameRing, 32
 - defun, 31
- findFunctionInDomain
 - calledby retractByFunction, 778
- findnexttest, 952
 - calledby regress, 950
 - calls testnumberp, 952
 - defun, 952
- firstTime
 - usedby bcUnixTable, 1651
- firstTokPosn, 631
 - calledby enPile, 630
 - calls tokPosn, 631
 - defun, 631
- fixObjectForPrinting, 795
 - calledby clearCmdParts, 841
 - calledby displayMode, 806
 - calledby displayProperties, 801
 - calledby displayType, 800
 - calledby displayValue, 799
 - calls concat, 796
 - calls member, 796
 - calls object2Identifier, 796
 - calls pname, 796
 - uses \$msgdbPrims, 796
 - defun, 795
- flatBvList, 89
 - calledby embedFunction, 88
 - calledby flatBvList, 89
 - calls flatBvList, 89
 - calls nconc, 89
 - calls refvecp, 89
 - calls varp, 89
 - defun, 89
- flatten, 861
 - calledby describeSpad2Cmd, 858
 - defun, 861
- flattenOperationAlist, 107
 - calledby spadTrace, 132
- defun, 107
- float
 - calledby ptimers, 98
- float2Sex, 603
 - calledby pLiteral2Sex, 602
 - uses \$useBFasDefault, 603
 - defun, 603
- FloatError, 1244
 - calledby BesselI, 1252
 - calledby gammaRatapprox, 1246
 - calledby rPsiImpl, 1257
 - calledby rgammaImpl, 1245
 - defmacro, 1244
- flowSegmentedMsg
 - calledby msgOutputter, 644
 - calledby msgText, 354
 - calledby sayKeyedMsgLocal, 44
 - calledby traceReply, 94
- flung
 - usedby intloopSpadProcess, 357
- fnameDirectory, 1298
 - calledby myWritable?, 1300
 - calls DirToString, 1298
 - defun, 1298
- fnameExists?, 1299
 - calledby myWritable?, 1300
 - defun, 1299
- fnameMake, 1297
 - calledby fnameNew, 1300
 - calls StringToDir, 1297
 - defun, 1297
- fnameName, 1299
 - defun, 1299
- fnameNew, 1300
 - calls fnameMake, 1300
 - defun, 1300
- fnameReadable?, 1299
 - defun, 1299
- fnameType, 1299
 - defun, 1299
- fnameWritable?, 1300
 - calls myWritable?, 1300
 - defun, 1300
- for, 681
 - syntax, 681
- form2HtString

- calledby conPage, 1599
- calledby dbConsHeading, 1649
- calledby dbSpecialDescription, 1651
- calledby dbSpecialExports, 1652
- calledby dbSpecialOperations, 1652
- calledby kcPage, 1611
- calledby kcaPage1, 1617
- calledby kccPage, 1618
- calledby kcnPage, 1622
- calledby kcpPage, 1615
- calledby kePage, 1605
- calledby koPage, 1631
- calledby ksPage, 1609
- form2LispString
 - calledby libConstructorSig, 1659
- form2String
 - calledby displayValue, 799
 - calledby kArgumentCheck, 1625
 - calledby reportOpsFromLisplib, 1093
- form2StringWithWhere
 - calledby reportOpsFromLisplib, 1092
- formatAttribute
 - calledby reportOpsFromLisplib, 1093
 - calledby reportOpsFromUnitDirectly, 1096
- formatOperation
 - calledby reportOpsFromUnitDirectly, 1096
- formatOperationAlistEntry
 - calledby displayOperationsFromLisplib, 1095
- formatOpType
 - calledby reportOpsFromUnitDirectly, 1096
- formatSignature
 - calledby compiledLookupCheck, 837
 - calledby displayModemap, 806
- Fortenbacher, Albrecht, 14
- fracpart, 1243
 - defmacro, 1243
- frame, 18, 25
 - calls frameSpad2Cmd, 25
 - defun, 25
 - manpage, 18
- frameEnvironment, 31
 - calledby importFromFrame, 34
 - calls frameInteractive, 31
 - defun, 31
- frameExposureData, 25
 - defmacro, 25
- frameHiFiAccess, 23
 - defmacro, 23
- frameHistList, 23
 - defmacro, 23
- frameHistListAct, 24
 - defmacro, 24
- frameHistListLen, 23
 - defmacro, 23
- frameHistoryTable, 24
 - defmacro, 24
- frameHistRecord, 24
 - defmacro, 24
- frameInteractive, 22
 - calledby frameEnvironment, 31
 - defmacro, 22
- frameIOIndex, 23
 - defmacro, 23
- frameName, 22
 - calledby findFrameInRing, 32
 - defmacro, 22
- framename
 - calledby addNewInterpreterFrame, 33
 - calledby closeInterpreterFrame, 37
 - calledby displayFrameNames, 28
 - calledby importFromFrame, 34
- frameNames, 28
 - calledby importFromFrame, 34
 - uses \$interpreterFrameRing, 28
 - defun, 28
- frameSpad2Cmd, 25
 - calledby frame, 25
 - calls addNewInterpreterFrame, 26
 - calls closeInterpreterFrame, 26
 - calls displayFrameNames, 26
 - calls drop[9], 26
 - calls helpSpad2Cmd, 25
 - calls importFromFrame, 26
 - calls import, 26
 - calls last, 26
 - calls names, 26
 - calls new, 26
 - calls nextInterpreterFrame, 26
 - calls next, 26
 - calls object2Identifier, 26
 - calls previousInterpreterFrame, 26

- calls qcar, [26](#)
 - calls qcdr, [25](#)
 - calls selectOptionLC, [25](#)
 - calls throwKeyedMsg, [25](#)
 - uses \$options, [26](#)
 - defun, [25](#)
- From, [670](#)
 - calledby syIgnoredFromTo, [484](#)
 - defun, [670](#)
- FromTo, [670](#)
 - calledby syIgnoredFromTo, [484](#)
 - defun, [670](#)
- function
 - calledby dbShowCons1, [1643](#)
 - calledby dbShowConstructorLines, [1650](#)
 - calledby dewritify, [925](#)
 - calledby domainDescendantsOf, [1603](#)
 - calledby filterAndFormatConstructors, [1136](#)
 - calledby isSystemDirectory, [1223](#)
 - calledby kcaPage1, [1617](#)
 - calledby parseNoMacroFromString, [1627](#)
 - calledby writify, [920](#)
- funfind, [106](#)
 - defmacro, [106](#)
- funfind,LAM, [106](#)
 - calls SEQ, [106](#)
 - calls exit, [106](#)
 - calls isFunctor, [106](#)
 - calls qcar, [106](#)
 - defun, [106](#)
- gammaRatapprox, [1246](#)
 - calledby gammaRatapprox, [1246](#)
 - calledby lngamma, [1245](#)
 - calledby rgammaImpl, [1246](#)
 - calls FloatError, [1246](#)
 - calls gammaRatapprox, [1246](#)
 - calls gammaRatkernel, [1246](#)
 - defun, [1246](#)
- gammaRatkernel, [1247](#)
 - calledby gammaRatapprox, [1246](#)
 - calls horner, [1247](#)
 - defun, [1247](#)
- gammaStirling, [1245](#)
 - calledby rgammaImpl, [1246](#)
 - calls lngamma, [1245](#)
 - defun, [1245](#)
- gatherGlossLines, [1580](#)
 - defun, [1580](#)
- gbc-time
 - calledby statisticsInitialization, [1221](#)
- genCategoryTable
 - calledby write-categorydb, [1216](#)
- genDomainTraceName, [122](#)
 - calledby domainToGenvar, [101](#)
 - calls genvar, [122](#)
 - calls lassoc, [122](#)
 - uses \$domainTraceNameAssoc, [122](#)
 - defun, [122](#)
- gensymInt, [927](#)
 - calls charDigitVal, [927](#)
 - calls error, [927](#)
 - calls gensymp, [927](#)
 - calls pname, [927](#)
 - defun, [927](#)
- gensymmer
 - calledby dewritify,dewritifyInner, [922](#)
- gensymp
 - calledby breaklet, [155](#)
 - calledby gensymInt, [927](#)
 - calledby letPrint2, [110](#)
 - calledby letPrint3, [111](#)
 - calledby letPrint, [108](#)
 - calledby spadTrace,isTraceable, [131](#)
 - calledby tracelet, [154](#)
- genvar
 - calledby genDomainTraceName, [122](#)
- get
 - calledby ?t, [146](#)
 - calledby augmentTraceNames, [78](#)
 - calledby clearCmdParts, [841](#)
 - calledby getAliasIfTracedMapParameter, [140](#)
 - calledby getBpiNameIfTracedMap, [140](#)
 - calledby getMapSig, [117](#)
 - calledby getMapSubNames, [130](#)
 - calledby getPreviousMapSubNames, [103](#)
 - calledby importFromFrame, [35](#)
 - calledby isDomainValuedVariable, [1145](#)
 - calledby isInterpOnlyMap, [104](#)
 - calledby isUncompiledMap, [104](#)
 - calledby putHist, [900](#)

- calledby restoreHistory, 907
 - calledby retract2Specialization, 772
 - calledby transTraceItem, 126
 - calledby writeHistModesAndValues, 913
- get-current-directory, 328
 - calledby restart, 307
 - defun, 328
- getAliasIfTracedMapParameter, 140
 - calledby mapLetPrint, 139
 - calls exit, 140
 - calls get, 140
 - calls isSharpVarWithNum, 140
 - calls pname, 140
 - calls seq, 140
 - calls string2pint-n, 140
 - calls substring, 140
 - uses \$InteractiveFrame, 140
 - defun, 140
- getalist
 - calledby displayProperties,sayFunctionDeps, 798
 - calledby importFromFrame, 35
 - calledby interpFunctionDepAlists, 805
 - calledby isExposedConstructor, 1094
 - calledby setExposeAddGroup, 157
 - calledby setExposeDropGroup, 163
- getAndEvalConstructorArgument, 1144
 - calledby evaluateType1, 1120
 - calls compFailure, 1144
 - calls getValue, 1144
 - calls isLocalVar, 1144
 - calls isWrapped, 1144
 - calls mkObjWrap, 1144
 - calls objMode, 1144
 - calls objVal, 1144
 - calls timedEVALFUN, 1144
 - defun, 1144
- getAndSay, 800
 - calledby displayProperties, 801
 - calls getI, 800
 - calls sayMSG, 800
 - defun, 800
- getArgValue
 - calledby interpret1, 347
- getBpiNameIfTracedMap, 140
 - calledby mapLetPrint, 139
- calls exit, 140
 - calls get, 140
 - calls seq, 140
 - uses \$InteractiveFrame, 140
 - uses \$traceNames, 141
 - defun, 140
- getBrowseDatabase, 1310
 - calls grepConstruct, 1310
 - calls member, 1310
 - uses \$includeUnexposed?, 1310
 - defun, 1310
- getCDTEntry
 - calledby dbShowCons1, 1643
- getConstantFromDomain, 767
 - calledby coerceIntAlgebraicConstant, 767
 - calledby getConstantFromDomain, 767
 - calls compiledLookupCheck, 767
 - calls evalDomain, 767
 - calls getConstantFromDomain, 767
 - calls getOperationAlistFromLisplib, 767
 - calls isPartialMode, 767
 - calls lassoc, 767
 - calls opOf, 767
 - calls spadcall, 767
 - calls throwKeyedMsg, 767
 - defun, 767
- getConstructorDocumentation, 1647
 - calledby dbShowConsDoc1, 1646
 - calls getdatabase, 1647
 - calls lassoc, 1647
 - calls qcaar, 1647
 - calls qcadar, 1647
 - calls qcar, 1647
 - calls qcdar, 1647
 - defun, 1647
- getConstructorExports
 - calledby kePage, 1606
- getConstructorForm
 - calledby augmentHasArgs, 1619
 - calledby conPageChoose, 1642
 - calledby dbAddDocTable, 1637
 - calledby dbShowConsDoc, 1645
 - calledby dbShowConstructorLines, 1650
 - calledby dbSpecialDescription, 1651
 - calledby dbSpecialExports, 1652
 - calledby dbSpecialOperations, 1652

- calledby kcodePage, 1620
- calledby kcuPage, 1621
- calledby make-databases, 1205
- getConstructorModemap
 - calledby kArgPage, 1602
- getConstructorSignature
 - calledby dbShowConsDoc1, 1646
 - calledby evaluateType1, 1120
 - calledby reportOpsFromLisplib, 1092
 - calledby valueArgsEqual?, 764
- getdatabase, 1196
 - calledby abbQuery, 865
 - calledby addoperations, 1193
 - calledby coerceSubDomain, 769
 - calledby dbAddDocTable, 1637
 - calledby dbMkEvalable, 1625
 - calledby dbSearchOrder, 1610
 - calledby dbShowCons1, 1643
 - calledby dbShowConsDoc1, 1646
 - calledby describeSpad2Cmd, 858
 - calledby displayOperationsFromLisplib, 1095
 - calledby displayValue, 799
 - calledby domainToGenvar, 101
 - calledby evaluateType1, 1120
 - calledby getConstructorDocumentation, 1647
 - calledby getOperationAlistFromLisplib, 134
 - calledby hasAtt, 722
 - calledby initial-getdatabase, 1187
 - calledby kCheckArgumentNumbers, 1627
 - calledby kDomainName, 1624
 - calledby kdPageInfo, 1601
 - calledby libConstructorSig, 1659
 - calledby loadLibNoUpdate, 1223
 - calledby loadLib, 1221
 - calledby localnrlib, 1201
 - calledby mkEvalable, 1116
 - calledby originsInOrder, 1636
 - calledby reportOpsFromLisplib, 1092
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby setExposeAddConstr, 160
 - calledby setExposeDropConstr, 164
 - calledby showdatabase, 1194
 - calledby valueArgsEqual?, 764
 - calledby whatConstructors, 1137
- calls warn, 1196
- uses *browse-stream*, 1196
- uses *category-stream*, 1196
- uses *defaultdomain-list*, 1196
- uses *hasCategory-hash*, 1196
- uses *hascategory-hash*, 1196
- uses *interp-stream*, 1196
- uses *miss*, 1196
- uses *operation-hash*, 1196
- uses *operation-stream*, 1196
- uses \$spadroot, 1196
- defun, 1196
- getDependentsOfConstructor, 1620
 - calledby kcodePage, 1620
 - calls pathname, 1621
 - calls readLibPathFast, 1620
 - calls rread, 1621
 - calls rshut, 1621
 - defun, 1620
- getDirectoryList, 1170
 - uses \$UserLevel, 1170
 - uses \$current-directory, 1170
 - uses \$directory-list, 1170
 - uses \$library-directory-list, 1170
 - defun, 1170
- getenv
 - calledby getenviron, 322
- getenviron, 322
 - calledby DaaseName, 1209
 - calledby buildHtMacroTable, 1401
 - calledby copyright, 855
 - calledby initial-getdatabase, 1187
 - calledby initroot, 327
 - calledby license, 934
 - calledby make-databases, 1205
 - calledby summary, 1103
 - calls getenv, 322
 - defun, 322
- getErFromDbL
 - calledby getMsgInfoFromKey, 656
- getFirstWord, 809
 - calledby doSystemCommand, 786
 - calls stringSpaces, 809
 - calls subseq, 809
 - defun, 809
- getFlag

- calledby interpFunctionDepAlists, 805
- getFunctionFromDomain
 - calledby coerceInt1, 743
- getHtMacroItem, 1402
 - calledby buildHtMacroTable, 1401
 - calls util.ht[7.1], 1402
 - defun, 1402
- getI
 - calledby displayProperties, 801
 - calledby fetchOutput, 910
 - calledby getAndSay, 800
- getImports
 - calledby kcnPage, 1622
- getInfovec
 - calledby hasAtt, 722
- getInterpMacroNames
 - calledby clearCmdParts, 841
 - calledby displayMacros, 867
 - calledby displayProperties, 801
 - calledby displayWorkspaceNames, 794
- getIProplist
 - calledby displayProperties, 801
- getl, 1240
 - calledby coerceIntCommute, 757
 - calledby coerceRetract, 777
 - calledby dbShowConsDoc1, 1646
 - calledby dbSpecialExports, 1652
 - calledby dbSpecialOperations, 1652
 - calledby reportOpsFromUnitDirectly, 1096
 - defun, 1240
- getLinePos, 662
 - calledby makeMsgFromLine, 661
 - defun, 662
- getLineText, 662
 - calledby makeMsgFromLine, 661
 - defun, 662
- getMapSig, 117
 - calledby saveMapSig, 117
 - calls boot-equal, 117
 - calls get, 117
 - uses \$InteractiveFrame, 117
 - defun, 117
- getMapSubNames, 130
 - calledby traceSpad2Cmd, 78
 - calls getPreviousMapSubNames, 131
 - calls get, 130
- calls unionq, 131
- calls union, 131
- uses \$InteractiveFrame, 131
- uses \$lastUntraced, 131
- uses \$traceNames, 131
- defun, 130
- getMinimalVarMode
 - calledby coerceInt, 741
- getMsgArgL, 639
 - calledby getMsgInfoFromKey, 656
 - calledby sameMsg?, 666
 - defmacro, 639
- getMsgCatAttr, 648
 - calledby getMsgToWhere, 653
 - calledby initToWhere, 658
 - calledby msgImPr?, 648
 - calledby msgNoRep?, 665
 - calls ifcdr, 648
 - calls ncAlist, 649
 - calls qassq, 649
 - defun, 648
- getMsgFTTag
 - usedby posPointers, 668
- getMsgFTTag?, 649
 - calledby getMsgPos2, 668
 - calledby getMsgPos, 649
 - calledby processChPosesForOneLine, 666
 - calledby putFTText, 669
 - calls getMsgPosTagOb, 649
 - calls ifcar, 649
 - defun, 649
- getMsgInfoFromKey, 656
 - calledby putDatabaseStuff, 655
 - calls getErFromDbL, 656
 - calls getMsgArgL, 656
 - calls getMsgKey?, 656
 - calls getMsgKey, 656
 - calls removeAttributes, 656
 - calls segmentKeyedMsg, 656
 - calls substituteSegmentedMsg, 656
 - uses \$msgDatabaseName, 656
 - defun, 656
- getMsgKey, 638
 - calledby getMsgInfoFromKey, 656
 - calledby processKeyedError, 643
 - calledby sameMsg?, 666

- defmacro, 638
- getMsgKey?, 652
 - calledby getMsgInfoFromKey, 656
 - calledby getStFromMsg, 645
 - calls identp, 652
 - defun, 652
- getMsgPos, 649
 - calledby erMsgCompare, 660
 - calledby erMsgSep, 660
 - calledby getPosStL, 646
 - calledby posPointers, 668
 - calledby processChPosesForOneLine, 666
 - calledby processMsgList, 659
 - calledby putFTText, 669
 - calls getMsgFTTag?, 649
 - calls getMsgPosTagOb, 649
 - defun, 649
- getMsgPos2, 668
 - calledby posPointers, 668
 - calledby putFTText, 670
 - calls getMsgFTTag?, 668
 - calls getMsgPosTagOb, 669
 - calls ncBug, 669
 - defun, 668
- getMsgPosTagOb, 638
 - calledby getMsgFTTag?, 649
 - calledby getMsgPos2, 669
 - calledby getMsgPos, 649
 - defmacro, 638
- getMsgPrefix, 639
 - calledby processChPosesForOneLine, 666
 - defmacro, 639
- getMsgPrefix?, 640
 - calledby getStFromMsg, 644
 - calledby processKeyedError, 643
 - calledby tabbing, 652
 - defmacro, 640
- getMsgTag, 640
 - calledby getMsgTag?, 641
 - calledby getStFromMsg, 644
 - calledby initImPr, 657
 - calledby leader?, 641
 - calledby line?, 641
 - calls ncTag, 640
 - defmacro, 640
- getMsgTag?, 640
 - calledby processKeyedError, 643
 - calls getMsgTag, 641
 - calls ifcar, 640
 - defmacro, 640
- getMsgText, 639
 - calledby getStFromMsg, 644
 - calledby putFTText, 669
 - defmacro, 639
- getMsgToWhere, 653
 - calledby toFile?, 653
 - calledby toScreen?, 641
 - calls getMsgCatAttr, 653
 - defun, 653
- getOperationAlistFromLisplib, 134
 - calledby getConstantFromDomain, 767
 - calledby hasSig, 721
 - calledby spadTrace, 132
 - calls addConsDB, 134
 - calls getdatabase, 134
 - calls markUnique, 134
 - defun, 134
- getOplistForConstructorForm, 1098
 - calledby reportOpsFromUnitDirectly, 1096
 - defun, 1098
- getOplistWithUniqueSignatures, 1099
 - defun, 1099
- getOption, 113
 - calledby spadTrace, 132
 - calledby spadUntrace, 143
 - calledby traceDomainConstructor, 137
 - calls assoc, 113
 - defun, 113
- getParserMacroNames, 793
 - calledby clearCmdParts, 841
 - calledby displayMacros, 867
 - calledby displayProperties, 801
 - calledby displayWorkspaceNames, 794
 - uses \$pfMacros, 793
 - defun, 793
- getPosStL, 646
 - calledby getStFromMsg, 644
 - calls decideHowMuch, 646
 - calls getMsgPos, 646
 - calls listDecideHowMuch, 646
 - calls msgImPr?, 646
 - calls ppos, 646

- calls remFile, 646
- calls remLine, 646
- calls showMsgPos?, 646
- uses \$lastPos, 646
- defun, 646
- getPreStL, 645
 - calledby getStFromMsg, 644
 - calls size, 645
 - uses \$preLength, 645
 - defun, 645
- getPreviousMapSubNames, 103
 - calledby getMapSubNames, 131
 - calledby untraceMapSubNames, 105
 - calls exit, 103
 - calls get, 103
 - calls seq, 103
 - uses \$InteractiveFrame, 103
 - defun, 103
- getProplist, 1150
 - calledby addBinding, 1149
 - calledby getProplist, 1150
 - calls getProplist, 1150
 - calls search, 1150
 - uses \$CategoryFrame, 1150
 - defun, 1150
- getRefvU16, 1318
 - defun, 1318
- getRefvU32, 1319
 - defun, 1319
- getRefvU8, 1317
 - defun, 1317
- getspoolname, 951
 - calledby regress, 950
 - defun, 951
- getStFromMsg, 644
 - calledby msgOutputter, 643
 - calls getMsgKey?, 645
 - calls getMsgPrefix?, 644
 - calls getMsgTag, 644
 - calls getMsgText, 644
 - calls getPosStL, 644
 - calls getPreStL, 644
 - calls pname, 645
 - calls tabbing, 645
 - defun, 644
- getSubDomainPredicate, 769
 - calledby coerceImmediateSubDomain, 769
 - calls hput, 769
 - calls interpret, 769
 - calls mkAtree, 769
 - calls msubst, 769
 - calls removeZeroOne, 769
 - calls selectLocalMms, 769
 - calls transferPropsToNode, 769
 - uses \$Boolean, 770
 - uses \$InteractiveFrame, 770
 - uses \$env, 769
 - uses \$superHash, 769
 - defun, 769
- getSystemCommandLine, 1106
 - calledby synonymSpad2Cmd, 1105
 - calls strpos, 1106
 - calls substring, 1106
 - uses \$currentLine, 1106
 - defun, 1106
- getTraceOption, 118
 - calledby getTraceOptions, 116
 - calledby trace1, 79
 - calledby trace3, 84
 - calls concat, 118
 - calls exit, 118
 - calls getTraceOption,hn, 118
 - calls identp, 118
 - calls isListOfIdentifiersOrStrings, 118
 - calls isListOfIdentifiers, 118
 - calls object2String, 118
 - calls qcar, 118
 - calls qcdr, 118
 - calls selectOptionLC, 118
 - calls seq, 118
 - calls stackTraceOptionError, 118
 - calls throwKeyedMsg, 118
 - calls transOnlyOption, 118
 - uses \$traceOptionList, 118
 - defun, 118
- getTraceOption,hn, 117
 - calledby getTraceOption, 118
 - calls domainToGenvar, 118
 - calls exit, 117
 - calls isDomainOrPackage, 118
 - calls seq, 117
 - calls stackTraceOptionError, 118

- defun, 117
- getTraceOptions, 116
 - calledby trace1, 79
 - calls exit, 116
 - calls getTraceOption, 116
 - calls poundsign, 116
 - calls seq, 116
 - calls throwKeyedMsg, 116
 - calls throwListOfKeyedMsgs, 116
 - uses \$traceErrorStack, 116
 - defun, 116
- getUsersOfConstructor, 1622
 - calledby kcuPage, 1621
 - calls pathname, 1622
 - calls readLibPathFast, 1622
 - calls rread, 1622
 - calls rshut, 1622
 - defun, 1622
- getValue
 - calledby coerceInt1, 743
 - calledby evaluateType, 1118
 - calledby getAndEvalConstructorArgument, 1144
 - calledby interpret1, 347
- getWorkspaceNames, 795
 - calledby displayProperties, 801
 - calledby displayWorkspaceNames, 795
 - calls nmsort, 795
 - uses \$InteractiveFrame, 795
 - defun, 795
- glesseqp
 - calledby dbShowConstructorLines, 1650
- grepConstruct
 - calledby getBrowseDatabase, 1310
- grepSearchQuery
 - calledby constructorSearch, 1597
- handleNoParseCommands, 785
 - calledby doSystemCommand, 786
 - calls concat, 810
 - calls member, 810
 - calls npboot, 810
 - calls nplisp, 810
 - calls npsynonym, 810
 - calls npsystem, 810
 - calls sayKeyedMsg, 810
 - calls stripLisp, 810
 - calls stripSpaces, 810
 - defun, 785
- handleParsedSystemCommands, 808
 - calledby doSystemCommand, 786
 - calls dumbTokenize, 808
 - calls parseSystemCmd, 808
 - calls systemCommand, 808
 - calls tokTran, 808
 - defun, 808
- handleTokenizeSystemCommands, 787
 - calledby doSystemCommand, 786
 - calls dumbTokenize, 787
 - calls systemCommand, 787
 - calls tokTran, 787
 - defun, 787
- hasAtt, 722
 - calledby hasAttSig, 724
 - calledby hasCaty, 718
 - calls constructSubst, 722
 - calls getInfovec, 722
 - calls getdatabase, 722
 - calls hasCatExpression, 722
 - calls subCopy, 722
 - calls unifyStruct, 722
 - local def \$domPvar, 722
 - defun, 722
- hasAttSig, 724
 - calledby hasCaty, 718
 - calls hasAtt, 724
 - calls hasSig, 724
 - calls keyedSystemError, 724
 - defun, 724
- hasCat
 - calledby hasCaty, 718
- hasCate, 731
 - calledby hasCatExpression, 725
 - calledby hasCate1, 725
 - calledby hasCateSpecial, 733
 - calledby hasCaty1, 730
 - calledby hasSigAnd, 723
 - calledby hasSigOr, 724
 - calledby hasSig, 721
 - calls containsVariables, 731
 - calls hasCate1, 731
 - calls hasCateSpecial, 731

- calls hasCaty, 731
- calls isPatternVar, 731
- calls subCopy, 731
- local def \$hope, 732
- local ref \$EmptyMode, 731
- local ref \$Subst, 732
- defun, 731
- hasCate1, 725
 - calledby hasCate, 731
 - calls hasCate, 725
 - local def \$domPvar, 725
 - defun, 725
- HasCategory
 - calledby basicLookup, 1226
- hascategory-hash
 - usedby lefts, 1658
- hasCateSpecial, 733
 - calledby hasCate, 731
 - calls augmentSub, 733
 - calls canCoerceFrom, 733
 - calls containsVars, 733
 - calls eqcar, 733
 - calls hasCateSpecialNew, 733
 - calls hasCate, 733
 - calls hasCaty, 733
 - calls isSubDomain, 733
 - local ref \$Integer, 733
 - local ref \$QuotientField, 733
 - defun, 733
- hasCateSpecialNew, 734
 - calledby hasCateSpecial, 733
 - calls augmentSub, 734
 - calls defaultTargetFE, 734
 - calls eqcar, 734
 - calls hasCaty, 734
 - calls isEqualOrSubDomain, 734
 - calls member, 734
 - calls underDomainOf, 734
 - local ref \$ComplexInteger, 734
 - local ref \$Integer, 734
 - local ref \$RationalNumber, 734
 - defun, 734
- hasCatExpression, 725
 - calledby hasAtt, 722
 - calledby hasCatExpression, 725
 - calls hasCatExpression, 725
 - calls hasCate, 725
 - calls keyedSystemError, 725
 - defun, 725
- hasCaty, 718
 - calledby hasCateSpecialNew, 734
 - calledby hasCateSpecial, 733
 - calledby hasCate, 731
 - calledby ofCategory, 717
 - calls augmentSub, 718
 - calls constructSubst, 718
 - calls domArg2, 718
 - calls domArg, 718
 - calls hasAttSig, 718
 - calls hasAtt, 718
 - calls hasCaty1, 719
 - calls hasCat, 718
 - calls hasSig, 718
 - calls mkDomPvar, 718
 - calls opOf, 718
 - calls subCopy, 718
 - calls unifyStruct, 719
 - local ref \$domPvar, 719
 - defun, 718
- hasCaty1, 730
 - calledby hasCaty1, 730
 - calledby hasCaty, 719
 - calls hasCate, 730
 - calls hasCaty1, 730
 - calls keyedSystemError, 730
 - local def \$domPvar, 730
 - defun, 730
- hasCorrectTarget, 753
 - defun, 753
- hashable, 1315
 - calls Boolean, 1316
 - calls bpiname, 1316
 - calls compiledLookup, 1316
 - calls knownEqualPred, 1316
 - defun, 1315
- hashCode?, 1225
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1226
 - defmacro, 1225
- hashString
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1226

- hashtable-class
 - calledby writify,writifyInner, 917
- hashType
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1226
- hasIndent
 - calledby mkDomTypeForm, 1603
- hasNewInfoAlist
 - calledby kcPage, 1612
- hasOptArgs?, 607
 - calledby pfApplication2Sex, 604
 - defun, 607
- hasOption, 791
 - calledby trace1, 79
 - calls pname, 791
 - calls stringPrefix?, 791
 - defun, 791
- hasPair, 113
 - calledby hasPair, 113
 - calledby letPrint2, 110
 - calledby letPrint3, 111
 - calledby letPrint, 108
 - calls hasPair, 113
 - calls qcar, 113
 - calls qcdr, 113
 - defun, 113
- hasSharpVar, 93
 - calledby hasSharpVar, 93
 - calledby monitorEvalTran, 93
 - calls hasSharpVar, 93
 - calls isSharpVar, 93
 - defun, 93
- hasSig, 721
 - calledby hasAttSig, 724
 - calledby hasCaty, 718
 - calls assq, 721
 - calls cnstructSubst, 721
 - calls constructor?, 721
 - calls getOperationAlistFromLisplib, 721
 - calls hasCate, 721
 - calls hasSigAnd, 721
 - calls hasSigOr, 721
 - calls keyedSystemError, 721
 - calls subCopy, 721
 - calls unifyStruct, 721
 - local def \$domPvar, 721
 - defun, 721
- hasSigAnd, 723
 - calledby hasSigOr, 724
 - calledby hasSig, 721
 - calls hasCate, 723
 - calls keyedSystemError, 723
 - calls subCopy, 723
 - defun, 723
- hasSigOr, 724
 - calledby hasSig, 721
 - calls hasCate, 724
 - calls hasSigAnd, 724
 - calls keyedSystemError, 724
 - defun, 724
- help, 878, 880
 - calls helpSpad2Cmd, 880
 - defun, 880
 - manpage, 878
- helpSpad2Cmd, 881
 - calledby abbreviationsSpad2Cmd, 823
 - calledby frameSpad2Cmd, 25
 - calledby help, 880
 - calledby savesystem, 959
 - calledby showSpad2Cmd, 1087
 - calledby systemCommand, 788
 - calls newHelpSpad2Cmd, 881
 - calls sayKeyedMsg, 881
 - defun, 881
- hget, 1234
 - calledby ScanOrPairVec,ScanOrInner, 926
 - calledby conLowerCaseConTran, 1590
 - calledby dbAddDocTable, 1637
 - calledby dbDocTable, 1635
 - calledby dbGetDocTable, 1639
 - calledby dewritify,dewritifyInner, 922
 - calledby kcPage, 1612
 - calledby keyword?, 412
 - calledby keyword, 412
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1209
 - calledby string2Constructor, 1590
 - calledby writify,writifyInner, 917
 - defmacro, 1234
- histFileErase, 928
 - calledby disableHist, 913
 - calledby historySpad2Cmd, 890

- calledby initHist, 888
- calledby restoreHistory, 907
- calledby saveHistory, 905
- calledby setHistoryCore, 894
- calledby writeInputLines, 896
- defun, 928
- histFileName, 888
 - calledby addNewInterpreterFrame, 34
 - calledby clearCmdAll, 839
 - calledby disableHist, 913
 - calledby historySpad2Cmd, 890
 - calledby initHist, 888
 - calledby readHiFi, 911
 - calledby restoreHistory, 907
 - calledby saveHistory, 905
 - calledby setHistoryCore, 894
 - calledby writeHiFi, 912
 - calls makeHistFileName, 888
 - uses \$interpreterFrameName, 888
 - defun, 888
- histInputFileName, 888
 - calledby saveHistory, 905
 - calledby writeInputLines, 896
 - calls makePathname, 888
 - uses \$historyDirectory, 888
 - uses \$interpreterFrameName, 888
 - defun, 888
- history, 883, 889
 - calls historySpad2Cmd, 889
 - calls sayKeyedMsg, 889
 - uses \$options, 889
 - defun, 889
 - manpage, 883
- historySpad2Cmd, 890
 - calledby history, 889
 - calls changeHistListLen, 890
 - calls clearSpad2Cmd, 890
 - calls disableHist, 890
 - calls exit, 890
 - calls histFileErase, 890
 - calls histFileName, 890
 - calls initHistList, 890
 - calls member, 890
 - calls queryUserKeyedMsg, 890
 - calls resetInCoreHist, 890
 - calls restoreHistory, 890
 - calls saveHistory, 890
 - calls sayKeyedMsg, 890
 - calls selectOptionLC, 890
 - calls seq, 890
 - calls setHistoryCore, 890
 - calls showHistory, 890
 - calls string2id-n, 890
 - calls upcase, 890
 - calls writeInputLines, 890
 - uses \$HiFiAccess, 890
 - uses \$IOindex, 890
 - uses \$options, 890
 - defun, 890
- hkeys, 1235
 - calledby lefts, 1658
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1208
 - calledby scanDictCons, 428
 - calledby scanPunCons, 431
 - calledby writify,writifyInner, 917
 - defun, 1235
- horner, 1247
 - calledby gammaRatkernel, 1247
 - calledby phiRatapprox, 1244
 - defun, 1247
- hput, 1234
 - calledby ScanOrPairVec,ScanOrInner, 926
 - calledby addBinding, 1149
 - calledby buildHtMacroTable, 1401
 - calledby dbAddDocTable, 1637
 - calledby dewritify,dewritifyInner, 922
 - calledby getSubDomainPredicate, 769
 - calledby writify,writifyInner, 917
 - defun, 1234
- htAddHeading, 1512
 - calledby kPage, 1593
 - defun, 1512
- htAllOrNum, 1569
 - defun, 1569
- htBcLinks, 1520
 - defun, 1520
- htBcLispLinks, 1520
 - defun, 1520
- htBcRadioButtons, 1523
 - defun, 1523
- htBeginMenu

- calledby kcPage, [1612](#)
- htBeginTable
 - calledby bcUnixTable, [1650](#)
- htBigSkip
 - calledby kePage, [1606](#)
- htCacheAddChoice, [1566](#)
 - defun, [1566](#)
- htCacheOne, [1569](#)
 - defun, [1569](#)
- htCacheSet, [1568](#)
 - defun, [1568](#)
- htCheck, [1558](#)
 - defun, [1558](#)
- htCheckList, [1559](#)
 - defun, [1559](#)
- htCopyProplist
 - calledby dbShowCons1, [1643](#)
 - calledby dbShowCons, [1641](#)
 - calledby kcPage, [1611](#)
 - calledby kccPage, [1618](#)
 - calledby kcpPage, [1615](#)
 - calledby kePage, [1606](#)
 - calledby kiPage, [1605](#)
 - calledby ksPage, [1609](#)
- htDoneButton, [1537](#)
 - defun, [1537](#)
- htDoNothing, [1558](#)
 - defun, [1558](#)
- htEndMenu
 - calledby kcPage, [1612](#)
- htEndTable
 - calledby bcUnixTable, [1651](#)
- htEscapeString, [1545](#)
 - defun, [1545](#)
- htFunctionSetLiteral, [1555](#)
 - defun, [1555](#)
- htGloss, [1577](#)
 - defun, [1577](#)
- htGlossPage, [1577](#)
 - defun, [1577](#)
- htGlossSearch, [1581](#)
 - defun, [1581](#)
- htGreekSearch, [1582](#)
 - defun, [1582](#)
- htInitPage, [1510](#)
 - calledby bcComplexLimit, [1478](#)
- calledby bcDefiniteIntegrate, [1409](#)
- calledby bcDifferentiate, [1406](#)
- calledby bcDraw2DSolve, [1420](#)
- calledby bcDraw2Dfun, [1416](#)
- calledby bcDraw2Dpar, [1418](#)
- calledby bcDraw3Dfun, [1422](#)
- calledby bcDraw3Dpar1, [1427](#)
- calledby bcDraw3Dpar, [1425](#)
- calledby bcDraw, [1415](#)
- calledby bcGen, [1468](#), [1494](#)
- calledby bcIndefiniteIntegrate, [1408](#)
- calledby bcInputEquations, [1481](#)
- calledby bcInputExplicitMatrix, [1443](#)
- calledby bcInputMatrixByFormula, [1445](#)
- calledby bcInputSolveInfo, [1448](#)
- calledby bcLaurentSeries, [1435](#)
- calledby bcLimit, [1412](#)
- calledby bcLinearSolveEqns, [1440](#)
- calledby bcLinearSolveMatrix1, [1487](#)
- calledby bcLinearSolveMatrixInhomo, [1488](#)
- calledby bcLinearSolve, [1440](#)
- calledby bcNotReady, [1496](#)
- calledby bcProduct, [1474](#)
- calledby bcPuisseuxSeries, [1437](#)
- calledby bcReadMatrix, [1442](#), [1475](#)
- calledby bcRealLimitGen, [1477](#)
- calledby bcRealLimit, [1475](#)
- calledby bcSeriesByFormula, [1432](#)
- calledby bcSeriesExpansion, [1430](#)
- calledby bcSeries, [1429](#)
- calledby bcSolveEquationsNumerically, [1485](#)
- calledby bcSolve, [1439](#)
- calledby bcSum, [1413](#)
- calledby bcSystemSolve, [1446](#)
- calledby bcTaylorSeries, [1433](#)
- calledby conOpPage1, [1629](#)
- calledby constructorSearch, [1597](#)
- calledby dbSpecialDescription, [1651](#)
- calledby dbSpecialExports, [1652](#)
- calledby dbSpecialOperations, [1652](#)
- calledby kcPage, [1611](#)
- calledby kccPage, [1618](#)
- calledby kcpPage, [1615](#)
- calledby kePage, [1606](#)
- calledby kiPage, [1605](#)
- calls htInitPageNoScroll, [1510](#)

- calls htSayStandard, [1510](#)
 - uses \$curPage, [1510](#)
 - defun, [1510](#)
- htInitPageNoScroll, [1511](#)
 - calledby dbShowCons1, [1643](#)
 - calledby dbShowCons, [1641](#)
 - calledby htInitPage, [1510](#)
 - calledby kPage, [1593](#)
 - calledby ksPage, [1609](#)
 - calls htSayStandard, [1511](#)
 - calls htSay, [1511](#)
 - calls httpMakeEmptyPage, [1511](#)
 - calls httpName, [1511](#)
 - uses \$atLeastOneUnexposed, [1511](#)
 - uses \$curPage, [1511](#)
 - uses \$htLineList, [1511](#)
 - uses \$newPage, [1511](#)
 - defun, [1511](#)
- htInputStrings, [1525](#)
 - defun, [1525](#)
- htKill, [1556](#)
 - defun, [1556](#)
- htLispLinks, [1518](#)
 - defun, [1518](#)
- htLispMemoLinks, [1519](#)
 - defun, [1519](#)
- htMakeButton, [1534](#)
 - defun, [1534](#)
- htMakeDoitButton, [1469](#)
 - defun, [1469](#)
- htMakeDoneButton, [1533](#)
 - calledby bcDifferentiate, [1406](#)
 - calledby bcDraw2DSolve, [1420](#)
 - calledby bcDraw3Dpar1, [1427](#)
 - calledby bcInputEquations, [1481](#)
 - calledby bcInputExplicitMatrix, [1443](#)
 - calledby bcInputMatrixByFormula, [1445](#)
 - calledby bcLinearSolveEqns, [1441](#)
 - calledby bcSeriesExpansion, [1430](#)
 - calledby bcSolveEquationsNumerically, [1485](#)
 - calledby bcSystemSolve, [1447](#)
 - defun, [1533](#)
- htMakeErrorPage, [1514](#)
 - defun, [1514](#)
- htMakeInputList, [1543](#)
 - defun, [1543](#)
- htMakeLabel, [1567](#)
 - defun, [1567](#)
- htMakePage, [1513](#)
 - calledby bcComplexLimit, [1478](#)
 - calledby bcDefiniteIntegrate, [1409](#)
 - calledby bcDifferentiate, [1406](#)
 - calledby bcDraw2DSolve, [1420](#)
 - calledby bcDraw2Dfun, [1416](#)
 - calledby bcDraw2Dpar, [1418](#)
 - calledby bcDraw3Dfun, [1422](#)
 - calledby bcDraw3Dpar1, [1427](#)
 - calledby bcDraw3Dpar, [1425](#)
 - calledby bcGen, [1468](#), [1494](#)
 - calledby bcIndefiniteIntegrate, [1408](#)
 - calledby bcInputEquations, [1481](#)
 - calledby bcInputExplicitMatrix, [1443](#)
 - calledby bcInputMatrixByFormula, [1445](#)
 - calledby bcInputSolveInfo, [1448](#)
 - calledby bcLaurentSeries, [1435](#)
 - calledby bcLimit, [1412](#)
 - calledby bcLinearSolveEqns, [1441](#)
 - calledby bcLinearSolveMatrix1, [1487](#)
 - calledby bcLinearSolveMatrixInhomo, [1488](#)
 - calledby bcLinearSolve, [1440](#)
 - calledby bcNotReady, [1496](#)
 - calledby bcProduct, [1474](#)
 - calledby bcPuisseuxSeries, [1437](#)
 - calledby bcReadMatrix, [1442](#), [1475](#)
 - calledby bcRealLimitGen, [1477](#)
 - calledby bcRealLimit, [1475](#)
 - calledby bcSeriesByFormula, [1432](#)
 - calledby bcSeriesExpansion, [1430](#)
 - calledby bcSeries, [1429](#)
 - calledby bcSolveEquationsNumerically, [1485](#)
 - calledby bcSolve, [1439](#)
 - calledby bcSum, [1413](#)
 - calledby bcSystemSolve, [1447](#)
 - calledby bcTaylorSeries, [1433](#)
 - calledby bcUnixTable, [1650](#)
 - calledby kcPage, [1612](#)
 - calledby kdPageInfo, [1601](#)
 - calledby kePageDisplay, [1608](#)
 - calledby kePage, [1606](#)
 - defun, [1513](#)
- htMakePage1, [1513](#)
 - defun, [1513](#)

- htMakePathKey, 1563
 - defun, 1563
- htMakePathKey,fn, 1563
 - defun, 1563
- htMakeTemplates, 1532
 - defun, 1532
- htMakeTemplates,substLabel, 1532
 - defun, 1532
- htMarkTree, 1563
 - defun, 1563
- htMkName, 1497
 - calls concat, 1497
 - defun, 1497
- htpAddInputAreaProp, 1502
 - defun, 1502
- htpAddToPageDescription, 1507
 - calledby bcHt, 1508
 - defun, 1507
- htpButtonValue, 1501
 - calledby bcComplexLimitGen, 1480
 - calledby bcDefiniteIntegrateGen, 1411
 - calledby bcRealLimitGen, 1477
 - calledby bcSolveEquations, 1486
 - defun, 1501
- htpDestroyPage, 1468
 - calls member, 1468
 - uses \$activePageList, 1468
 - defun, 1468
- htpDomainConditions, 1499
 - defun, 1499
- htpDomainPvarSubstList, 1500
 - defun, 1500
- htpDomainVariableAlist, 1499
 - defun, 1499
- htpInputAreaAlist, 1501
 - calledby bcGenExplicitMatrix, 1472
 - calledby bcLinearExtractMatrix, 1487
 - calledby bcLinearMatrixGen, 1490
 - calledby bcLinearSolveEqnsGen, 1492
 - defun, 1501
- htpInputAreaList
 - calledby bcInputSolveInfo, 1448
- htpLabelDefault, 1506
 - defun, 1506
- htpLabelErrorMsg, 1505
 - defun, 1505
- htpLabelFilter, 1507
 - defun, 1507
- htpLabelFilteredInputString, 1503
 - defun, 1503
- htpLabelInputString, 1503
 - calledby bcComplexLimitGen, 1480
 - calledby bcDefiniteIntegrateGen, 1411
 - calledby bcDifferentiateGen, 1407
 - calledby bcDraw2DSolveGen, 1421
 - calledby bcDraw2DfunGen, 1417
 - calledby bcDraw2DparGen, 1419
 - calledby bcDraw3DfunGen, 1424
 - calledby bcDraw3Dpar1Gen, 1428
 - calledby bcDraw3DparGen, 1426
 - calledby bcIndefiniteIntegrateGen, 1409
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormulaGen, 1471
 - calledby bcInputMatrixByFormula, 1445
 - calledby bcLinearSolveEqnsGen, 1492
 - calledby bcProductGen, 1474
 - calledby bcRealLimitGen, 1477
 - calledby bcSeriesExpansionGen, 1431
 - calledby bcSeriesGen, 1435
 - calledby bcSolveEquations, 1486
 - calledby bcSumGen, 1414
 - calledby kDomainName, 1624
 - calledby koPageInputAreaUnchanged?, 1623
 - calledby koaPageFilterByName, 1633
 - defun, 1503
- htpLabelSpadType, 1506
 - defun, 1506
- htpLabelSpadValue, 1504
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormula, 1445
 - defun, 1504
- htpLabelType, 1506
 - defun, 1506
- htpMakeEmptyPage, 1467
 - calledby htInitPageNoScroll, 1511
 - uses \$activePageList, 1467
 - defun, 1467
- htpName, 1499
 - calledby htInitPageNoScroll, 1511
 - defun, 1499
- htpPageDescription, 1507
 - defun, 1507

- htpProperty, 1502
 - calledby bcGenExplicitMatrix, 1472
 - calledby bcInputMatrixByFormulaGen, 1471
 - calledby bcLinearSolveMatrixInhomo, 1488
 - calledby bcMatrixGen, 1472
 - calledby bcRealLimitGen1, 1478
 - calledby bcSolveEquations, 1486
 - calledby conOpPage, 1628
 - calledby dbCompositeWithMap, 1630
 - calledby dbConsHeading, 1649
 - calledby dbSelectCon, 1648
 - calledby dbShowConditions, 1648
 - calledby dbShowCons1, 1643
 - calledby dbShowConsDoc1, 1646
 - calledby dbShowConsDoc, 1645
 - calledby dbShowConsKindsFilter, 1645
 - calledby dbShowCons, 1641
 - calledby kArgPage, 1602
 - calledby kcPage, 1611
 - calledby kcaPage1, 1617
 - calledby kccPage, 1618
 - calledby kcdPage, 1620
 - calledby kcnPage, 1622
 - calledby kcpPage, 1614
 - calledby kcuPage, 1621
 - calledby kePage, 1605
 - calledby kiPage, 1604
 - calledby koPageAux1, 1633
 - calledby koPageFromKKPage, 1632
 - calledby koPageInputAreaUnchanged?, 1628
 - calledby koPage, 1631
 - calledby koaPageFilterByName, 1634
 - calledby ksPage, 1609
 - defun, 1502
- htpPropertyList, 1502
 - calledby bcInputEquations, 1481
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputSolveInfo, 1448
 - calledby bcLinearSolveMatrixInhomo, 1488
 - calledby bcSolveEquationsNumerically, 1485
 - defun, 1502
- htpRadioButtonAlist, 1500
 - defun, 1500
- htProcessBcButtons, 1516
 - defun, 1516
- htProcessBcStrings, 1517
 - defun, 1517
- htProcessDoitButton, 1537
 - defun, 1537
- htProcessDomainConditions, 1527
 - defun, 1527
- htProcessDoneButton, 1534
 - defun, 1534
- htProcessToggleButtons, 1515
 - defun, 1515
- htProperty
 - calledby bcInputEquations, 1481
- htpSetDomainConditions, 1499
 - defun, 1499
- htpSetDomainPvarSubstList, 1500
 - defun, 1500
- htpSetDomainVariableAlist, 1500
 - defun, 1500
- htpSetInputAreaAlist, 1502
 - defun, 1502
- htpSetLabelErrorMsg, 1505
 - defun, 1505
- htpSetLabelInputString, 1504
 - defun, 1504
- htpSetLabelSpadValue, 1505
 - defun, 1505
- htpSetName, 1499
 - defun, 1499
- htpSetPageDescription, 1507
 - defun, 1507
- htpSetProperty, 1503
 - calledby bcGenExplicitMatrix, 1472
 - calledby bcInputEquations, 1481
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormula, 1445
 - calledby bcInputSolveInfo, 1448
 - calledby bcLinearSolveEqns1, 1441
 - calledby bcLinearSolveMatrixInhomo, 1488
 - calledby bcReadMatrix, 1442, 1475
 - calledby bcRealLimitGen, 1477
 - calledby bcSolveSingle, 1449
 - calledby bcSystemSolveEqns1, 1447
 - calledby conOpPage1, 1629
 - calledby constructorSearch, 1597
 - calledby dbShowCons1, 1643
 - calledby dbShowConsKindsFilter, 1645
 - calledby dbShowCons, 1641

- calledby dbSpecialDescription, 1651
- calledby dbSpecialOperations, 1652
- calledby kArgPage, 1602
- calledby kDomainName, 1624
- calledby kPage, 1593
- calledby kcPage, 1611
- calledby kcaPage1, 1617
- calledby kccPage, 1618
- calledby kcdePage, 1620
- calledby kcnPage, 1622
- calledby kcpPage, 1615
- calledby kcuPage, 1621
- calledby kePageDisplay, 1608
- calledby kePage, 1605, 1606
- calledby koPageAux, 1632
- calledby koPage, 1631
- calledby koaPageFilterByName, 1634
- calledby ksPage, 1609
- defun, 1503
- htpSetRadioButtonAlist, 1501
 - defun, 1501
- htQuery
 - calledby constructorSearch, 1597
- htQuote, 1515
 - defun, 1515
- htRadioButtons, 1522
 - defun, 1522
- htSay, 1509
 - calledby bcInputEquations, 1481
 - calledby bcUnixTable, 1650
 - calledby dbConsExposureMessage, 1645
 - calledby htInitPageNoScroll, 1511
 - calledby kPage, 1593
 - calledby kcPage, 1611
 - calledby kdPageInfo, 1601
 - calledby kePageDisplay, 1608
 - calledby kePage, 1606
 - calledby ksPage, 1609
 - calls bcHt, 1509
 - defun, 1509
- htSayBind, 1512
 - calledby htSayStandard, 1511
 - calls bcHt, 1512
 - defun, 1512
- htSayHrule
 - calledby dbShowConditions, 1648
- htSayStandard, 1511
 - calledby dbShowCons1, 1643
 - calledby htInitPageNoScroll, 1511
 - calledby htInitPage, 1510
 - calledby kPage, 1593
 - calledby kcPage, 1612
 - calledby kdPageInfo, 1601
 - calledby kePageDisplay, 1608
 - calledby kePage, 1606
 - calledby ksPage, 1609
 - calls htSayBind, 1511
 - defun, 1511
- htSetCache, 1565
 - defun, 1565
- htSetExpose, 1565
 - defun, 1565
- htSetFunCommand, 1556
 - defun, 1556
- htSetFunCommandContinue, 1556
 - defun, 1556
- htSetHistory, 1564
 - defun, 1564
- htSetInputLibrary, 1564
 - defun, 1564
- htSetInteger, 1553
 - defun, 1553
- htSetLinkerArgs, 1565
 - defun, 1565
- htSetLiteral, 1551
 - defun, 1551
- htSetLiterals, 1550
 - defun, 1550
- htSetNotAvailable, 1557
 - defun, 1557
- htSetOutputCharacters, 1565
 - defun, 1565
- htSetOutputLibrary, 1564
 - defun, 1564
- htSetSystemVariable, 1577
 - defun, 1577
- htSetSystemVariableKind, 1576
 - defun, 1576
- htSetvarDoneButton, 1555
 - defun, 1555
- htSetVars, 1545
 - defun, 1545

- htShowCount, 1548
 - defun, 1548
- htShowFunctionPage, 1553
 - defun, 1553
- htShowFunctionPageContinued, 1554
 - defun, 1554
- htShowIntegerPage, 1552
 - defun, 1552
- htShowLiteralPage, 1550
 - defun, 1550
- htShowPage, 1512
 - calledby bcComplexLimit, 1478
 - calledby bcDefiniteIntegrate, 1409
 - calledby bcDifferentiate, 1406
 - calledby bcDraw2DSolve, 1420
 - calledby bcDraw2Dfun, 1416
 - calledby bcDraw2Dpar, 1418
 - calledby bcDraw3Dfun, 1422
 - calledby bcDraw3Dpar1, 1427
 - calledby bcDraw3Dpar, 1425
 - calledby bcDraw, 1415
 - calledby bcGen, 1468, 1494
 - calledby bcIndefiniteIntegrate, 1408
 - calledby bcInputEquations, 1481
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormula, 1445
 - calledby bcInputSolveInfo, 1448
 - calledby bcLaurentSeries, 1435
 - calledby bcLimit, 1412
 - calledby bcLinearSolveEqns, 1441
 - calledby bcLinearSolveMatrix1, 1487
 - calledby bcLinearSolveMatrixInhomo, 1488
 - calledby bcLinearSolve, 1440
 - calledby bcNotReady, 1496
 - calledby bcProduct, 1474
 - calledby bcPuisseuxSeries, 1437
 - calledby bcReadMatrix, 1442, 1475
 - calledby bcRealLimitGen, 1477
 - calledby bcRealLimit, 1475
 - calledby bcSeriesByFormula, 1432
 - calledby bcSeriesExpansion, 1430
 - calledby bcSeries, 1429
 - calledby bcSolveEquationsNumerically, 1489
 - calledby bcSolve, 1439
 - calledby bcSum, 1413
 - calledby bcSystemSolve, 1447
 - calledby bcTaylorSeries, 1433
 - calledby constructorSearch, 1597
 - calledby dbSpecialDescription, 1651
 - calledby dbSpecialExports, 1652
 - calledby kcPage, 1612
 - calledby kiPage, 1605
 - defun, 1512
- htShowPageNoScroll, 1513
 - calledby dbShowCons1, 1643
 - calledby kPage, 1593
 - defun, 1513
- htShowSetPage, 1549
 - defun, 1549
- htShowSetTree, 1546
 - defun, 1546
- htShowSetTreeValue, 1548
 - defun, 1548
- htSowPage
 - calledby kePage, 1606
- htStringPad, 1497
 - calls concat, 1497
 - defun, 1497
- htsv, 1545
 - defun, 1545
- htSystemVariables, 1574
 - defun, 1574
- htSystemVariables,displayOptions, 1572
 - defun, 1572
- htSystemVariables,fn, 1572
 - defun, 1572
- htSystemVariables,functionTail, 1573
 - defun, 1573
- htSystemVariables,gn, 1571
 - defun, 1571
- htTextSearch, 1584
 - defun, 1584
- htTutorialSearch, 1586
 - defun, 1586
- idChar?, 419
 - calledby scanW, 419
 - defmacro, 419
- identp, 1237
 - calledby countCache, 980
 - calledby downcase, 1272
 - calledby getMsgKey?, 652

- calledby getTraceOption, 118
- calledby isDomainValuedVariable, 1145
- calledby isGenvar, 126
- calledby isListOfIdentifiersOrStrings, 103
- calledby isListOfIdentifiers, 102
- calledby isSharpVar, 110
- calledby messageprint-1, 1239
- calledby ncAlist, 705
- calledby ncTag, 705
- calledby ofCategory, 717
- calledby restoreHistory, 907
- calledby rwrite, 915, 916
- calledby selectOption, 819
- calledby undo, 52
- calledby upcase, 1272
- defmacro, 1237
- if, 685
 - syntax, 685
- If?, 372
 - calledby incLude1, 375
 - calls quotient, 372
 - defun, 372
- ifcar
 - calledby conLowerCaseConTran, 1590
 - calledby conOpPage1, 1629
 - calledby conSpecialString?, 1598
 - calledby dbShowCons, 1641
 - calledby defaultTargetFE, 736
 - calledby fillerSpaces, 310
 - calledby getMsgFTTTag?, 649
 - calledby getMsgTag?, 640
 - calledby pfAbSynOp, 702
 - calledby pfExpression, 562
 - calledby pfLeaf, 545
 - calledby pfSymbol, 549
 - calledby pfSymb, 549
 - calledby posPointers, 668
 - calledby remLine, 647
 - calledby string2Constructor, 1590
 - calledby tokConstruct, 701
- ifcdr
 - calledby clearParserMacro, 793
 - calledby conOpPage1, 1629
 - calledby domainDescendantsOf, 1603
 - calledby getMsgCatAttr, 648
 - calledby kePage, 1606
 - calledby mac0Get, 524
 - calledby remFile, 647
 - calledby reportOpsFromLisplib, 1092
 - calledby setMsgCatlessAttr, 655
 - calledby specialChar, 1166
- ifCond, 382
 - calledby incLude1, 375
 - calls MakeSymbol, 382
 - calls incCommandTail, 382
 - uses \$inclAssertions, 382
 - defun, 382
- IfKeepPart, 370
 - defvar, 370
- IfSkipPart, 370
 - defvar, 370
- IfSkipToEnd, 370
 - defvar, 370
- ignorep, 956
 - calledby split, 954
 - defun, 956
- iht, 1508
 - calls basicStringize, 1508
 - calls mapStringize, 1508
 - uses \$htLineList, 1508
 - uses \$newPage, 1508
 - defun, 1508
- images
 - Restart, 306
- import
 - calledby frameSpad2Cmd, 26
- importFromFrame, 34
 - calledby frameSpad2Cmd, 26
 - calledby importFromFrame, 34
 - calls boot-equal, 34
 - calls clearCmdParts, 34
 - calls exit, 35
 - calls frameEnvironment, 34
 - calls frameNames, 34
 - calls framename, 34
 - calls getalist, 35
 - calls get, 35
 - calls importFromFrame, 34
 - calls member, 34
 - calls putHist, 35
 - calls queryUserKeyedMsg, 34
 - calls sayKeyedMsg, 34

- calls seq, 34
- calls string2id-n, 34
- calls throwKeyedMsg, 34
- calls upcase, 34
- uses \$interpreterFrameRing, 35
- defun, 34
- in-stream, 1147
 - usedby ncTopLevel, 316
 - usedby serverReadLine, 337
 - defvar, 1147
- incActive?, 396
 - calledby incLude1, 374
 - defun, 396
- incAppend, 380
 - calledby incAppend1, 380
 - calledby incLude1, 374
 - calledby next1, 330
 - calls Delay, 380
 - calls incAppend1, 380
 - defun, 380
- incAppend1, 380
 - calledby incAppend, 380
 - calls StreamNull, 380
 - calls incAppend, 380
 - defun, 380
- incBiteOff, 930
 - calledby incFileName, 930
 - defun, 930
- incClassify, 392
 - calledby incLude1, 375
 - calls incCommand?, 393
 - uses incCommands, 393
 - defun, 392
- incCommand?, 394
 - calledby incClassify, 393
 - defun, 394
- incCommands, 392
 - usedby incClassify, 393
 - defvar, 392
- incCommandTail, 395
 - calledby assertCond, 389
 - calledby ifCond, 382
 - calledby incLude1, 374
 - calledby inclFname, 395
 - calls incDrop, 395
 - defun, 395
- incConsoleInput, 396
 - calledby incLude1, 374
 - calls incRgen, 396
 - calls make-instream, 396
 - defun, 396
- incDrop, 395
 - calledby incCommandTail, 395
 - calls substring, 395
 - defun, 395
- incFileInput, 395
 - calledby incLude1, 374
 - calls incRgen, 395
 - calls make-instream, 395
 - defun, 395
- incFileName, 930
 - calledby inclFname, 395
 - calledby ncloopIncFileName, 929
 - calls incBiteOff, 930
 - defun, 930
- incHandleMessage, 368
 - calledby incRenumberLine, 368
 - calls ncBug, 368
 - calls ncSoftError, 368
 - defun, 368
- incIgen, 367
 - calledby incIgen1, 367
 - calledby incRenumber, 366
 - calls Delay, 367
 - calls incIgen1, 367
 - defun, 367
- incIgen1, 367
 - calledby incIgen, 367
 - calls incIgen, 367
 - defun, 367
- inclFname, 395
 - calledby incLude1, 374
 - calls incCommandTail, 395
 - calls incFileName, 395
 - defun, 395
- incLine, 380
 - calledby xlMsg, 379
 - calledby xlSkip, 382
 - calls incLine1, 380
 - defun, 380
- incLine1, 381
 - calledby incLine, 380

- calledby xlOK1, [379](#)
 - calls lnCreate, [381](#)
 - defun, [381](#)
- inclmsgCannotRead, [385](#)
 - calledby xlCannotRead, [384](#)
 - calls thefname, [385](#)
 - defun, [385](#)
- inclmsgCmdBug, [391](#)
 - calledby xlCmdBug, [391](#)
 - defun, [391](#)
- inclmsgConActive, [386](#)
 - calledby xlConActive, [386](#)
 - calls theid, [386](#)
 - defun, [386](#)
- inclmsgConsole, [388](#)
 - calledby xlConsole, [387](#)
 - defun, [388](#)
- inclmsgConStill, [387](#)
 - calledby xlConStill, [387](#)
 - calls theid, [387](#)
 - defun, [387](#)
- inclmsgFileCycle, [385](#)
 - calledby xlFileCycle, [385](#)
 - calls porigin, [385](#)
 - calls theid, [385](#)
 - defun, [385](#)
- inclmsgFinSkipped, [388](#)
 - calledby xlSkippingFin, [388](#)
 - defun, [388](#)
- inclmsgIfBug, [391](#)
 - calledby xlIfBug, [391](#)
 - defun, [391](#)
- inclmsgIfSyntax, [390](#)
 - calledby xlIfSyntax, [390](#)
 - calls concat, [390](#)
 - calls theid, [390](#)
 - calls theorigin, [390](#)
 - defun, [390](#)
- inclmsgNoSuchFile, [384](#)
 - calledby xlNoSuchFile, [383](#)
 - calls thefname, [384](#)
 - defun, [384](#)
- inclmsgPrematureEOF, [381](#)
 - calledby xlPrematureEOF, [378](#)
 - calls theorigin, [381](#)
 - defun, [381](#)
- inclmsgPrematureFin, [389](#)
 - calledby xlPrematureFin, [389](#)
 - calls theorigin, [389](#)
 - defun, [389](#)
- inclmsgSay, [383](#)
 - calledby xlSay, [383](#)
 - calls theid, [383](#)
 - defun, [383](#)
- incLude, [369](#)
 - calledby incLude1, [374](#)
 - calledby incStream, [366](#)
 - calledby incString, [331](#)
 - calls Delay, [369](#)
 - defun, [369](#)
- include, [929](#)
 - calls incLude1, [369](#)
 - manpage, [929](#)
- incLude1, [374](#)
 - calledby include, [369](#)
 - calls Elseif?, [375](#)
 - calls If?, [375](#)
 - calls KeepPart?, [375](#)
 - calls Rest, [374](#)
 - calls SkipEnd?, [375](#)
 - calls SkipPart?, [375](#)
 - calls Skipping?, [374](#)
 - calls StreamNull, [374](#)
 - calls Top?, [374](#)
 - calls assertCond, [375](#)
 - calls concat, [374](#)
 - calls expand-tabs, [375](#)
 - calls ifCond, [375](#)
 - calls incActive?, [374](#)
 - calls incAppend, [374](#)
 - calls incClassify, [375](#)
 - calls incCommandTail, [374](#)
 - calls incConsoleInput, [374](#)
 - calls incFileInput, [374](#)
 - calls incLude, [374](#)
 - calls incNConsoles, [375](#)
 - calls inclFname, [374](#)
 - calls xlCannotRead, [374](#)
 - calls xlCmdBug, [375](#)
 - calls xlConActive, [374](#)
 - calls xlConStill, [374](#)
 - calls xlConsole, [375](#)

- calls `xlFileCycle`, 374
- calls `xlIfBug`, 375
- calls `xlIfSyntax`, 375
- calls `xlNoSuchFile`, 374
- calls `xlOK1`, 374
- calls `xlOK`, 374
- calls `xlPrematureEOF`, 374
- calls `xlPrematureFin`, 375
- calls `xlSay`, 374
- calls `xlSkippingFin`, 375
- calls `xlSkip`, 374
- defun, 374
- `incNConsoles`, 396
 - calledby `incLude1`, 375
 - calledby `incNConsoles`, 396
 - calls `incNConsoles`, 396
 - defun, 396
- `incPrefix?`, 394
 - calledby `lineoftoks`, 403
 - calledby `scanIgnoreLine`, 404
 - defun, 394
- `incRenumber`, 366
 - calledby `incStream`, 366
 - calledby `incString`, 331
 - calls `incIgen`, 366
 - calls `incZip`, 366
 - defun, 366
- `incRenumberItem`, 368
 - calledby `incRenumberLine`, 368
 - calls `lnSetGlobalNum`, 368
 - defun, 368
- `incRenumberLine`, 368
 - calls `incHandleMessage`, 368
 - calls `incRenumberItem`, 368
 - defun, 368
- `incRgen`, 397
 - calledby `incConsoleInput`, 396
 - calledby `incFileInput`, 395
 - calledby `incRgen1`, 398
 - calledby `incStream`, 366
 - calls `Delay`, 397
 - calls `incRgen1`, 397
 - defun, 397
- `incRgen1`, 398
 - calledby `incRgen`, 397
 - calls `incRgen`, 398
- uses `StreamNil`, 398
- defun, 398
- `incStream`, 366
 - calledby `intloopInclude0`, 355
 - calledby `ncloopInclude0`, 365
 - calls `incLude`, 366
 - calls `incRenumber`, 366
 - calls `incRgen`, 366
 - uses `Top`, 366
 - defun, 366
- `incString`, 331
 - calledby `intloopProcessString`, 329
 - calledby `parseFromString`, 340
 - calledby `parseNoMacroFromString`, 1627
 - calls `incLude`, 331
 - calls `incRenumber`, 331
 - uses `Top`, 331
 - defun, 331
- `incZip`, 366
 - calledby `incRenumber`, 366
 - calledby `incZip1`, 367
 - calls `Delay`, 366
 - calls `incZip1`, 366
 - defun, 366
- `incZip1`, 367
 - calledby `incZip`, 366
 - calls `StreamNull`, 367
 - calls `incZip`, 367
 - defun, 367
- `infgeneric`, 401
 - defvar, 401
- `init-boot/spad-reader`, 1154
 - calls `ioclear`, 1154
 - calls `next-lines-clear`, 1154
 - uses `*standard-output*`, 1154
 - uses `$spad-errors`, 1154
 - uses `boot-line-stack`, 1154
 - uses `file-closed`, 1154
 - uses `line-handler`, 1154
 - uses `meta-error-handler`, 1154
 - uses `spaderrorstream`, 1154
 - uses `xtokenreader`, 1154
 - defun, 1154
- `init-memory-config`, 326
 - calledby `restart`, 307
 - calls `allocate-contiguous-pages`, 326

- calls allocate-relocatable-pages, 326
- calls allocate, 326
- calls set-hole-size, 326
- defun, 326
- initHist, 888
 - calledby restart, 307
 - calls \$replace, 888
 - calls histFileErase, 888
 - calls histFileName, 888
 - calls initHistList, 888
 - calls makeInputFilename, 888
 - calls oldHistFileName, 888
 - uses \$HiFiAccess, 888
 - uses \$useInternalHistoryTable, 888
 - defun, 888
- initHistList, 889
 - calledby addNewInterpreterFrame, 34
 - calledby historySpad2Cmd, 890
 - calledby initHist, 888
 - uses \$HistListAct, 889
 - uses \$HistListLen, 889
 - uses \$HistList, 889
 - uses \$HistRecord, 889
 - defun, 889
- initial-getdatabase, 1187
 - calledby resethashtables, 1186
 - calls getdatabase, 1187
 - calls getenviron, 1187
 - defun, 1187
- initializeInterpreterFrameRing, 27
 - calledby restart, 307
 - calls emptyInterpreterFrame, 27
 - calls updateFromCurrentInterpreterFrame, 27
 - uses \$interpreterFrameName, 27
 - uses \$interpreterFrameRing, 27
 - defun, 27
- initializeSetVariables, 962
 - calledby initializeSetVariables, 962
 - calledby resetWorkspaceVariables, 962
 - calls initializeSetVariables, 962
 - calls literals, 962
 - calls sayMSG, 962
 - calls translateYesNo2TrueFalse, 962
 - calls tree, 962
 - defun, 962
- initializeTimedNames
 - calledby processInteractive, 342
- initImPr, 657
 - calledby msgCreate, 638
 - calls getMsgTag, 657
 - calls setMsgUnforcedAttr, 657
 - uses \$erMsgToss, 657
 - uses \$imPrTagGuys, 657
 - defun, 657
- initroot, 327
 - calledby restart, 307
 - calls getenviron, 327
 - calls reroot, 327
 - uses \$spadroot, 327
 - defun, 327
- initToWhere, 658
 - calledby msgCreate, 638
 - calls getMsgCatAttr, 658
 - calls setMsgUnforcedAttr, 658
 - defun, 658
- input-libraries, 975
 - usedby addInputLibrary, 975
 - usedby dropInputLibrary, 976
 - usedby openOutputLibrary, 973
 - usedby setInputLibrary, 974
 - defvar, 975
- insert
 - calledby originsInOrder, 1636
 - calledby updateSourceFiles, 874
- insertAlist, 981
 - calledby countCache, 980
 - calledby kePageOpAlist, 1607
 - calls ?order, 981
 - calls rplac, 981
 - defun, 981
- insertpile, 625
 - calledby intloopInclude0, 355
 - calledby ncloopInclude0, 365
 - calls npNull, 625
 - calls pileCforest, 625
 - calls pilePlusComments, 625
 - calls pilePlusComment, 625
 - calls pileTree, 625
 - defun, 625
- insertPos, 669
 - calledby posPointers, 668

- calls done, 669
- defun, 669
- installConstructor
 - calledby loadLibNoUpdate, 1223
 - calledby loadLib, 1221
 - calledby localnrlib, 1201
- intCodeGenCOERCE
 - calledby coerceInt0, 742
- integer-decode-float-denominator, 1289
 - defun, 1289
- integer-decode-float-exponent, 1290
 - defun, 1290
- integer-decode-float-numerator, 1289
 - defun, 1289
- integer-decode-float-sign, 1289
 - defun, 1289
- InteractiveFrame, 78
- intern
 - calledby dbShowConstructorLines, 1650
- internl
 - calledby constructSubst, 732
 - calledby countCache, 980
 - calledby spadTraceAlias, 141
- InterpExecuteSpadSystemCommand, 323
 - calls ExecuteInterpSystemCommand, 323
 - uses \$intCoerceFailure, 323
 - uses \$intSpadReader, 323
 - catches, 323
 - defun, 323
- interpFunctionDepAlists, 805
 - calledby displayProperties, 801
 - calls getFlag, 805
 - calls getalist, 805
 - calls putalist, 805
 - uses \$InteractiveFrame, 805
 - uses \$dependeeAlist, 805
 - uses \$dependentAlist, 805
 - uses \$e, 805
 - defun, 805
- interpopen, 1189
 - calledby resethashables, 1186
 - calledby restart0, 309
 - calls DaaseName, 1190
 - calls make-database, 1190
 - uses *allconstructors*, 1190
 - uses *interp-stream*, 1190
 - uses *interp-stream-stamp*, 1190
 - uses \$spadroot, 1190
 - defun, 1189
- interpret, 346
 - calledby getSubDomainPredicate, 769
 - calledby interpretTopLevel, 345
 - calls interpret1, 346
 - uses \$env, 346
 - uses \$eval, 346
 - uses \$genValue, 346
 - defun, 346
- interpret1, 347
 - calledby interpret, 346
 - calls bottomUp, 347
 - calls getArgValue, 347
 - calls getValue, 347
 - calls interpret2, 347
 - calls keyedSystemError, 347
 - calls mkAtreeWithSrcPos, 347
 - calls mkObj, 347
 - calls putTarget, 347
 - uses \$eval, 347
 - uses \$genValue, 347
 - defun, 347
- interpret2, 348
 - calledby interpret1, 347
 - calls coerceInteractive, 348
 - calls member, 348
 - calls mkObj, 348
 - calls objMode, 348
 - calls objVal, 348
 - calls systemErrorHere, 348
 - calls throwKeyedMsgCannotCoerceWith-Value, 348
 - uses \$EmptyMode, 348
 - uses \$ThrowAwayMode, 348
 - defun, 348
- interpretTopLevel, 345
 - calledby interpretTopLevel, 345
 - calledby processInteractive1, 345
 - calls interpretTopLevel, 345
 - calls interpret, 345
 - calls peekTimedName, 345
 - calls stopTimingProcess, 345
 - uses \$timedNameStack, 345
 - catches, 345

- defun, [345](#)
- intInterpretPform, [360](#)
 - calledby phInterpret, [359](#)
 - calls pf2Sex, [360](#)
 - calls processInteractive, [360](#)
 - calls zeroOneTran, [360](#)
 - defun, [360](#)
- intloop, [317](#)
 - calledby ncIntLoop, [317](#)
 - calls SpadInterpretStream, [317](#)
 - calls resetStackLimits, [317](#)
 - uses \$intRestart, [318](#)
 - uses \$intTopLevel, [317](#)
 - catches, [317](#)
 - defun, [317](#)
- intloopEchoParse, [361](#)
 - calledby intloopInclude0, [355](#)
 - calls dqToList, [361](#)
 - calls mkLineList, [361](#)
 - calls nclloopDQlines, [361](#)
 - calls nclloopPrintLines, [361](#)
 - calls npParse, [361](#)
 - calls setCurrentLine, [361](#)
 - uses \$EchoLines, [361](#)
 - uses \$lines, [361](#)
 - defun, [361](#)
- intloopInclude, [355](#)
 - calledby SpadInterpretStream, [319](#)
 - calls intloopInclude0, [355](#)
 - defun, [355](#)
- intloopInclude0, [355](#)
 - calledby intloopInclude, [355](#)
 - calls incStream, [355](#)
 - calls insertpile, [355](#)
 - calls intloopEchoParse, [355](#)
 - calls intloopProcess, [355](#)
 - calls lineoftoks, [355](#)
 - calls next, [355](#)
 - uses \$lines, [355](#)
 - defun, [355](#)
- intloopPrefix?, [327](#)
 - calledby intloopReadConsole, [321](#)
 - defun, [327](#)
- intloopProcess, [356](#)
 - calledby intloopInclude0, [355](#)
 - calledby intloopProcessString, [329](#)
 - calledby intloopProcess, [356](#)
 - calls StreamNull, [356](#)
 - calls \$systemCommandFunction, [356](#)
 - calls intloopProcess, [356](#)
 - calls intloopSpadProcess, [356](#)
 - calls pfAbSynOp?, [356](#)
 - calls setCurrentLine, [356](#)
 - calls tokPart, [356](#)
 - uses \$systemCommandFunction, [356](#)
 - defun, [356](#)
- intloopProcessString, [329](#)
 - calledby intloopReadConsole, [321](#)
 - calls incString, [329](#)
 - calls intloopProcess, [329](#)
 - calls next, [329](#)
 - calls setCurrentLine, [329](#)
 - defun, [329](#)
- intloopReadConsole, [320](#)
 - calledby SpadInterpretStream, [319](#)
 - calledby intloopReadConsole, [321](#)
 - calls concat, [321](#)
 - calls intloopPrefix?, [321](#)
 - calls intloopProcessString, [321](#)
 - calls intloopReadConsole, [321](#)
 - calls intnplisp, [321](#)
 - calls leaveScratchpad, [321](#)
 - calls mkprompt, [321](#)
 - calls nclloopCommand, [321](#)
 - calls nclloopEscaped, [321](#)
 - calls serverReadLine, [321](#)
 - calls setCurrentLine, [321](#)
 - uses \$dalymode, [321](#)
 - defun, [320](#)
 - throws, [321](#)
- intloopSpadProcess, [357](#)
 - calledby intloopProcess, [356](#)
 - calls CatchAsCan, [357](#)
 - calls Catch, [357](#)
 - calls intloopSpadProcess,interp, [357](#)
 - calls ncPutQ, [357](#)
 - uses \$NeedToSignalSessionManager, [357](#)
 - uses \$currentCarrier, [357](#)
 - uses \$intCoerceFailure, [357](#)
 - uses \$intSpadReader, [357](#)
 - uses \$ncMsgList, [357](#)
 - uses \$prevCarrier, [357](#)

- uses \$stepNo, [357](#)
 - uses flung, [357](#)
 - catches, [357](#)
 - defun, [357](#)
- intloopSpadProcess,interp, [358](#)
 - calledby intloopSpadProcess, [357](#)
 - calls ncConversationPhase, [358](#)
 - calls ncEltQ, [358](#)
 - calls ncError, [358](#)
 - defun, [358](#)
- intnplisp, [327](#)
 - calledby intloopReadConsole, [321](#)
 - calls nplisp, [327](#)
 - uses \$currentLine, [328](#)
 - defun, [327](#)
- intp
 - calledby dewritify,dewritifyInner, [922](#)
- intProcessSynonyms, [324](#)
 - calledby ExecuteInterpSystemCommand, [324](#)
 - calls processSynonyms, [324](#)
 - uses line, [324](#)
 - defun, [324](#)
- ioclear, [1158](#)
 - calledby init-boot/spad-reader, [1154](#)
 - calledby spad-syntax-error, [1155](#)
 - calls line-clear, [1158](#)
 - calls reduce-stack-clear, [1158](#)
 - uses \$boot, [1158](#)
 - uses \$spad, [1158](#)
 - uses current-fragment, [1158](#)
 - uses current-line, [1158](#)
 - defun, [1158](#)
- iostat, [1156](#)
 - calledby spad-long-error, [1155](#)
 - calls line-past-end-p, [1156](#)
 - calls line-print, [1156](#)
 - calls next-lines-show, [1156](#)
 - calls token-stack-show, [1156](#)
 - uses \$boot, [1156](#)
 - uses \$current-line, [1156](#)
 - uses \$spad, [1156](#)
 - defun, [1156](#)
- is-console[9]
 - called by serverReadLine, [336](#), [1199](#)
 - called by shut, [1168](#)
- isDomain
 - calledby /untrace-2, [124](#)
 - calledby ?t, [147](#)
 - calledby addTraceItem, [146](#)
 - calledby compiledLookup, [1225](#)
- isDomainOrPackage, [107](#)
 - calledby /tracereply, [142](#)
 - calledby /untrace-2, [124](#)
 - calledby ?t, [146](#)
 - calledby addTraceItem, [146](#)
 - calledby getTraceOption,hn, [118](#)
 - calledby prTraceNames,fn, [145](#)
 - calledby shortenForPrinting, [113](#)
 - calledby spadReply, [114](#)
 - calledby spadTrace, [132](#)
 - calledby spadUntrace, [143](#)
 - calledby trace3, [84](#)
 - calledby traceReply, [94](#)
 - calledby untraceDomainConstructor,keepTraced?, [138](#)
 - calledby writify,writifyInner, [917](#)
 - calls isFunctor, [107](#)
 - calls opOf, [107](#)
 - calls poundsign, [107](#)
 - calls refvecp, [107](#)
 - defun, [107](#)
- isDomainValuedVariable, [1145](#)
 - calledby evaluateType, [1118](#)
 - calledby reportOperations, [1089](#)
 - calls get, [1145](#)
 - calls identp, [1145](#)
 - calls member, [1145](#)
 - calls objMode, [1145](#)
 - calls objValUnwrap, [1145](#)
 - local ref \$InteractiveFrame, [1145](#)
 - local ref \$env, [1145](#)
 - local ref \$e, [1145](#)
 - defun, [1145](#)
- isEqualOrSubDomain, [736](#)
 - calledby coerceInt1, [743](#)
 - calledby coerceRetract, [777](#)
 - calledby defaultTargetFE, [735](#)
 - calledby hasCateSpecialNew, [734](#)
 - calls isSubDomain, [736](#)
 - defun, [736](#)
- isExposedConstructor, [1094](#)

- calledby conOpPage1, 1629
- calledby dbShowCons1, 1643
- calledby dbShowConsDoc1, 1646
- calledby kPage, 1593
- calledby reportOpsFromLisplib, 1092
- calledby reportOpsFromUnitDirectly, 1096
- calls getalist, 1094
- local ref \$globalExposureGroupAlist, 1094
- local ref \$localExposureData, 1094
- defun, 1094
- isFunctor
 - calledby /options, 96
 - calledby /untrace-2, 124
 - calledby funfind,LAM, 106
 - calledby isDomainOrPackage, 107
 - calledby trace1, 79
 - calledby trace3, 83
 - calledby traceReply, 94
- isGenvar, 126
 - calledby /untrace-2, 125
 - calledby tracelet, 154
 - calls digitp, 126
 - calls identp, 126
 - calls size, 126
 - defun, 126
- isgenvar
 - calledby ?t, 146
 - calledby letPrint2, 110
 - calledby letPrint3, 111
 - calledby letPrint, 108
 - calledby traceReply, 94
- isIntegerString, 808
 - calledby tokTran, 807
 - defun, 808
- isInternalMapName
 - calledby displayProperties, 801
- isInterpMacro
 - calledby displayMacro, 794
 - calledby displayProperties, 801
- isInterpOnlyMap, 104
 - calledby trace3, 84
 - calls get, 104
 - uses \$InteractiveFrame, 105
 - defun, 104
- isLegitimateMode
 - calledby coerceCommuteTest, 758
- calledby coerceIntTableOrFunction, 759
- isListOfIdentifiers, 102
 - calledby getTraceOption, 118
 - calls exit, 102
 - calls identp, 102
 - calls seq, 102
 - defun, 102
- isListOfIdentifiersOrStrings, 102
 - calledby getTraceOption, 118
 - calls exit, 102
 - calls identp, 103
 - calls seq, 102
 - defun, 102
- isLocalVar
 - calledby getAndEvalConstructorArgument, 1144
- isMap
 - calledby clearCmdParts, 841
- isNameOfType
 - calledby reportOperations, 1089
- isNewWorldDomain
 - calledby basicLookup, 1226
- isPartialMode, 718
 - calledby evalCategory, 1145
 - calledby getConstantFromDomain, 767
 - calledby retract2Specialization, 772
 - calls contained, 718
 - local ref \$EmptyMode, 718
 - defun, 718
- isPatternVar, 729
 - calledby containsVars1, 729
 - calledby containsVars, 729
 - calledby hasCate, 731
 - calledby unifyStructVar, 727
 - calledby unifyStruct, 726
 - defun, 729
- isRectangularList
 - calledby retract2Specialization, 772
- isSharpVar, 110
 - calledby domArg2, 720
 - calledby hasSharpVar, 93
 - calledby isSharpVarWithNum, 109
 - calls identp, 110
 - defun, 110
- isSharpVarWithNum, 109

- calledby getAliasIfTracedMapParameter, 140
- calledby letPrint2, 110
- calledby letPrint3, 111
- calledby letPrint, 108
- calledby monitorEvalTran1, 93
- calls dig2fix, 109
- calls digitp, 109
- calls isSharpVar, 109
- calls pname, 109
- calls qcsize, 109
- defun, 109
- isSubDomain
 - calledby coerceInt1, 743
 - calledby coerceIntByMap, 761
 - calledby hasCateSpecial, 733
 - calledby isEqualOrSubDomain, 736
- isSubForRedundantMapName, 105
 - calledby /untrace-2, 124
 - calledby trace3, 83
 - calledby traceReply, 94
 - calls assocleft, 105
 - calls member, 105
 - calls rassocSub, 105
 - uses \$mapSubNameAlist, 105
 - defun, 105
- isSystemDirectory, 1223
 - calledby loadLib, 1221
 - calls function, 1223
 - local ref \$spadroot, 1223
 - defun, 1223
- isTaggedUnion, 1117
 - defun, 1117
- isTotalCoerce
 - calledby coerceByTable, 759
- isType
 - calledby reportOperations, 1090
- isUncompiledMap, 104
 - calledby trace3, 84
 - calls get, 104
 - uses \$InteractiveFrame, 104
 - defun, 104
- isValidType
 - calledby coerceIntTableOrFunction, 759
 - calledby coerceIntTower, 750
 - calledby dbExtractUnderlyingDomain, 1631
 - calledby coerceByTable, 759
 - calledby coerceInt0, 742
 - calledby getAndEvalConstructorArgument, 1144
 - calledby retract, 1269
 - defun, 1661
- iterate, 687
 - syntax, 687
- Jenks, Richard, 14
- justifyMyType, 354
 - calledby printTypeAndTime, 352
 - calls fillerSpaces, 354
 - uses \$linelength, 354
 - defun, 354
- kArgPage, 1602
 - calls dbShowCons, 1602
 - calls domainDescendantsOf, 1602
 - calls getConstructorModemap, 1602
 - calls httpProperty, 1602
 - calls httpSetProperty, 1602
 - calls mkDomTypeForm, 1602
 - calls position, 1602
 - calls sublisFormal, 1602
 - defun, 1602
- kArgumentCheck, 1625
 - calledby kDomainName, 1624
 - calls conSpecialString?, 1625
 - calls form2String, 1625
 - calls opOf, 1625
 - defun, 1625
- kcaPage, 1616
 - calls ancestorsOf, 1616
 - calls kcaPage1, 1616
 - defun, 1616
- kcaPage1, 1617
 - calledby kcaPage, 1616
 - calledby kcdPage, 1616
 - calls augmentHasArgs, 1617
 - calls dbShowCons, 1617
 - calls errorPage, 1617
 - calls form2HtString, 1617
 - calls function, 1617
 - calls httpProperty, 1617

- calls httpSetProperty, 1617
- calls kDomainName, 1617
- calls listSort, 1617
- calls opOf, 1617
- defun, 1617
- kccPage, 1618
 - calls augmentHasArgs, 1618
 - calls childrenOf, 1618
 - calls dbShowCons, 1618
 - calls errorPage, 1618
 - calls form2HtString, 1618
 - calls htCopyPropList, 1618
 - calls htInitPage, 1618
 - calls httpProperty, 1618
 - calls httpSetProperty, 1618
 - calls kDomainName, 1618
 - calls opOf, 1618
 - calls qcar, 1618
 - calls reduceAListForDomain, 1618
 - defun, 1618
- kcdePage, 1620
 - calls concat, 1620
 - calls dbShowCons, 1620
 - calls getConstructorForm, 1620
 - calls getDependentsOfConstructor, 1620
 - calls httpProperty, 1620
 - calls httpSetProperty, 1620
 - calls ncParseFromString, 1620
 - calls nequal, 1620
 - calls opOf, 1620
 - defun, 1620
- kcdoPage, 1617
 - calledby kcdoPage, 1617
 - calls domainsOf, 1617
 - calls kcdoPage, 1617
 - defun, 1617
- kcdPage, 1616
 - calls descendantsOf, 1616
 - calls kcaPage1, 1616
 - defun, 1616
- kCheckArgumentNumber
 - calledby kCheckArgumentNumbers, 1627
- kCheckArgumentNumbers, 1627
 - calledby kisValidType, 1626
 - calls getdatabase, 1627
 - calls kCheckArgumentNumber, 1627
 - defun, 1627
- kcnPage, 1622
 - calls concat, 1622
 - calls dbShowCons, 1622
 - calls errorPage, 1622
 - calls form2HtString, 1622
 - calls getImports, 1622
 - calls httpProperty, 1622
 - calls httpSetProperty, 1622
 - calls kDomainName, 1622
 - calls opOf, 1622
 - calls pname, 1622
 - calls qcar, 1622
 - calls sublislis, 1622
 - defun, 1622
- kcPage, 1611
 - calls brCon, 1611
 - calls dbpHasDefaultCategory?, 1611
 - calls errorPage, 1611
 - calls form2HtString, 1611
 - calls hasNewInfoAList, 1612
 - calls hget, 1612
 - calls htBeginMenu, 1612
 - calls htCopyPropList, 1611
 - calls htEndMenu, 1612
 - calls htInitPage, 1611
 - calls htMakePage, 1612
 - calls htSayStandard, 1612
 - calls htSay, 1611
 - calls htShowPage, 1612
 - calls httpProperty, 1611
 - calls httpSetProperty, 1611
 - calls kDomainName, 1611
 - calls nequal, 1612
 - calls opOf, 1611
 - calls qcar, 1611
 - calls satBreak, 1612
 - uses \$defaultPackageNameHT, 1612
 - defun, 1611
- kcpPage, 1614
 - calls dbShowCons, 1615
 - calls errorPage, 1614
 - calls form2HtString, 1615
 - calls htCopyPropList, 1615
 - calls htInitPage, 1615
 - calls httpProperty, 1614

- calls `httpSetProperty`, 1615
- calls `kDomainName`, 1614
- calls `opOf`, 1615
- calls `parentsOf`, 1615
- calls `qcar`, 1614
- calls `sublislis`, 1615
- defun, 1614
- `kcuPage`, 1621
 - calls `concat`, 1621
 - calls `dbShowCons`, 1621
 - calls `getConstructorForm`, 1621
 - calls `getUsersOfConstructor`, 1621
 - calls `httpProperty`, 1621
 - calls `httpSetProperty`, 1621
 - calls `ncParseFromString`, 1621
 - calls `nequal`, 1621
 - calls `opOf`, 1621
 - defun, 1621
- `kDomainName`, 1624
 - calledby `kcPage`, 1611
 - calledby `kcaPage1`, 1617
 - calledby `kccPage`, 1618
 - calledby `kcnPage`, 1622
 - calledby `kcpPage`, 1614
 - calledby `kePage`, 1605
 - calledby `kiPage`, 1604
 - calledby `koPage`, 1631
 - calledby `ksPage`, 1609
 - calls `concat`, 1624
 - calls `dbMkEvalable`, 1624
 - calls `getdatabase`, 1624
 - calls `httpLabelInputString`, 1624
 - calls `httpSetProperty`, 1624
 - calls `kArgumentCheck`, 1624
 - calls `kisValidType`, 1624
 - calls `mkConform`, 1624
 - calls `unabbrev`, 1624
 - uses `$PatternVariableList`, 1624
 - catches, 1624
 - defun, 1624
- `kdPageInfo`, 1601
 - calls `bcHt`, 1601
 - calls `extractFileNameFromPath`, 1601
 - calls `getdatabase`, 1601
 - calls `htMakePage`, 1601
 - calls `htSayStandard`, 1601
 - calls `htSay`, 1601
 - calls `kPageArgs`, 1601
 - calls `length`, 1601
 - calls `nequal`, 1601
 - calls `subseq`, 1601
 - defun, 1601
- `KeepPart?`, 373
 - calledby `Skipping?`, 374
 - calledby `incLude1`, 375
 - calls `remainder`, 373
 - defun, 373
- `kePage`, 1605
 - calls `bcConPredTable`, 1606
 - calls `capitalize`, 1605
 - calls `concat`, 1605
 - calls `errorPage`, 1605
 - calls `form2HtString`, 1605
 - calls `getConstructorExports`, 1606
 - calls `htBigSkip`, 1606
 - calls `htCopyPropList`, 1606
 - calls `htInitPage`, 1606
 - calls `htMakePage`, 1606
 - calls `htSayStandard`, 1606
 - calls `htSay`, 1606
 - calls `htSowPage`, 1606
 - calls `httpProperty`, 1605
 - calls `httpSetProperty`, 1605, 1606
 - calls `ifcdr`, 1606
 - calls `kDomainName`, 1605
 - calls `kePageDisplay`, 1606
 - calls `kePageOpAList`, 1606
 - calls `length`, 1606
 - calls `menuButton`, 1606
 - calls `mkConform`, 1605
 - calls `opOf`, 1605
 - calls `pluralSay`, 1606
 - calls `simpHasPred`, 1606
 - calls `sublisFormal`, 1605
 - uses `$conformersAreDomains`, 1606
 - defun, 1605
- `kePageDisplay`, 1608
 - calledby `dbSpecialExports`, 1652
 - calledby `kePage`, 1606
 - calls `dbGatherData`, 1608
 - calls `dbSowOpItems`, 1608
 - calls `htMakePage`, 1608

- calls htSayStandard, 1608
- calls htSay, 1608
- calls httpSetProperty, 1608
- calls length, 1608
- calls menuButton, 1608
- calls pluralize, 1608
- defun, 1608
- kePageOpAlist, 1607
 - calledby kePage, 1606
 - calls insertAlist, 1607
 - calls lassoc, 1607
 - calls zeroOneConvert, 1607
 - defun, 1607
- keyedSystemError
 - calledby coerceBranch2Union, 766
 - calledby compiledLookupCheck, 837
 - calledby hasAttSig, 724
 - calledby hasCatExpression, 725
 - calledby hasCaty1, 730
 - calledby hasSigAnd, 723
 - calledby hasSigOr, 724
 - calledby hasSig, 721
 - calledby interpret1, 347
 - calledby pf2Sex1, 599
 - calledby pfLiteral2Sex, 602
 - calledby readHiFi, 911
- keyword, 412
 - calledby lfkey, 413
 - calledby scanCloser?, 417
 - calledby scanKeyTr, 411
 - calls hget, 412
 - defun, 412
- keyword?, 412
 - calledby scanWord, 417
 - calls hget, 412
 - defun, 412
- kFormatSlotDomain
 - calledby dbAddChainDomain, 1640
 - calledby dbSearchOrder, 1610
- kiPage, 1604
 - calls capitalize, 1605
 - calls dbShowConsDoc1, 1605
 - calls errorPage, 1605
 - calls htCopyProplist, 1605
 - calls htInitPage, 1605
 - calls htShowPage, 1605
 - calls httpProperty, 1604
 - calls kDomainName, 1604
 - calls mkConform, 1604
 - uses \$conformsAreDomains, 1605
 - defun, 1604
- kisValidType, 1626
 - calledby conSpecialString?, 1598
 - calledby kDomainName, 1624
 - calls kCheckArgumentNumbers, 1626
 - calls member, 1626
 - calls processInteractive, 1626
 - uses \$ProcessInteractiveValue, 1626
 - uses \$noEvalTypeMsg, 1626
 - catches, 1626
 - defun, 1626
- knownEqualPred
 - calledby hashable, 1316
- koaPageFilterByCategory
 - calledby koaPageFilterByName, 1633
- koaPageFilterByName, 1633
 - calls dbGetInputString, 1634
 - calls downcase, 1634
 - calls httpLabelInputString, 1633
 - calls httpProperty, 1634
 - calls httpSetProperty, 1634
 - calls koaPageFilterByCategory, 1633
 - calls pmTransFilter, 1634
 - calls superMatch?, 1634
 - defun, 1633
- koAttrs
 - calledby koPageAux, 1633
- koOps
 - calledby koPageAux, 1633
- koPage, 1631
 - calledby conOpPage1, 1629
 - calls capitalize, 1631
 - calls concat, 1631
 - calls errorPage, 1631
 - calls form2HtString, 1631
 - calls httpProperty, 1631
 - calls httpSetProperty, 1631
 - calls kDomainName, 1631
 - calls koPageAux, 1631
 - calls koPageInputAreaUnchanged?, 1631
 - defun, 1631
- koPageAux, 1632

- calledby koPageFromKKPage, 1632
- calledby koPage, 1631
- calls assoc, 1633
- calls dbShowOperationsFromConform, 1633
- calls httpSetProperty, 1632
- calls koAttrs, 1633
- calls koOps, 1633
- calls systemError, 1633
- defun, 1632
- koPageAux1, 1633
 - calls dbShowOperationsFromConform, 1633
 - calls httpProperty, 1633
 - defun, 1633
- koPageFromKKPage, 1632
 - calls httpProperty, 1632
 - calls koPageAux, 1632
 - defun, 1632
- koPageInputAreaUnchanged?, 1623
 - calledby koPage, 1631
 - calls concat, 1623
 - calls httpLabelInputString, 1623
 - calls httpProperty, 1623
 - defun, 1623
- kPage
 - calledby conPage, 1599
 - calledby constructorSearch, 1597
 - calls addParameterTemplates, 1593
 - calls capitalize, 1593
 - calls concat, 1593
 - calls dbConformGenUnder, 1593
 - calls dbShowConsDoc1, 1593
 - calls dbSourceFile, 1593
 - calls dbXParts, 1593
 - calls htAddHeading, 1593
 - calls htInitPageNoScroll, 1593
 - calls htSayStandard, 1593
 - calls htSay, 1593
 - calls htShowPageNoScroll, 1593
 - calls httpSetProperty, 1593
 - calls isExposedConstructor, 1593
 - calls kPageContextMenu, 1593
 - calls mkConform, 1593
 - calls ncParseFromString, 1593
 - calls opOf, 1593
 - uses \$atLeastOneUnexposed, 1593
 - uses \$conformsAreDomains, 1593
- kPageArgs
 - calledby kdPageInfo, 1601
- kPageContextMenu
 - calledby kPage, 1593
- kSearch, 1596
 - calls constructorSearch, 1596
 - defun, 1596
- ksPage, 1609
 - calls dbSearchOrder, 1609
 - calls dbShowCons, 1609
 - calls errorPage, 1609
 - calls form2HtString, 1609
 - calls htCopyProplist, 1609
 - calls htInitPageNoScroll, 1609
 - calls htSayStandard, 1609
 - calls htSay, 1609
 - calls httpProperty, 1609
 - calls httpSetProperty, 1609
 - calls kDomainName, 1609
 - defun, 1609
- kTestPred, 1639
 - calledby dbSearchOrder, 1610
 - calls simpHasPred, 1639
 - calls testBitVector, 1639
 - uses \$domain, 1639
 - uses \$predvec, 1639
 - defun, 1639
- lassoc
 - calledby bcMatrixGen, 1472
 - calledby breaklet, 155
 - calledby coerceTraceFunValue2E, 129
 - calledby conOpPage1, 1629
 - calledby constructorSearch, 1597
 - calledby genDomainTraceName, 122
 - calledby getConstantFromDomain, 767
 - calledby getConstructorDocumentation, 1647
 - calledby kePageOpAlist, 1607
 - calledby letPrint2, 110
 - calledby letPrint3, 111
 - calledby letPrint, 108
 - calledby processSynonyms, 325
 - calledby reportUndo, 60
 - calledby serverReadLine, 336
 - calledby set1, 1082

- calledby subTypes, [101](#)
 - calledby trace1, [79](#)
 - calledby tracelet, [154](#)
 - calledby undoSingleStep, [55](#)
 - calledby unifyStructVar, [727](#)
- lassocSub, [103](#)
 - calledby untrace, [123](#)
 - calls lassq, [103](#)
 - defun, [103](#)
- lassq
 - calledby conPageFastPath, [1600](#)
 - calledby conPage, [1599](#)
 - calledby diffAlist, [1125](#)
 - calledby lassocSub, [103](#)
- last
 - calledby coerceIntTower, [750](#)
 - calledby frameSpad2Cmd, [26](#)
- lastatom
 - calledby dbGetDocTable,gn, [1638](#)
- lastc
 - calledby StringToDir, [1298](#)
- lastcount, [956](#)
 - calledby startp, [955](#)
 - defun, [956](#)
- lastTokPosn, [631](#)
 - calledby enPile, [630](#)
 - calledby separatePiles, [631](#)
 - calls tokPosn, [631](#)
 - defun, [631](#)
- leader?, [641](#)
 - calledby msgOutputter, [644](#)
 - calledby showMsgPos?, [648](#)
 - calls getMsgTag, [641](#)
 - defmacro, [641](#)
- leave, [688](#)
 - syntax, [688](#)
- leaveScratchpad, [941](#)
 - calledby intloopReadConsole, [321](#)
 - calledby quitSpad2Cmd, [940](#)
 - defun, [941](#)
- lefts, [1658](#)
 - calls hkeys, [1658](#)
 - uses hascategory-hash, [1658](#)
 - defun, [1658](#)
- length
 - calledby augmentHasArgs, [1619](#)
 - calledby bcDifferentiateGen, [1407](#)
 - calledby bcInputExplicitMatrix, [1443](#)
 - calledby coerceIntByMap, [761](#)
 - calledby computeTTTranspositions, [754](#)
 - calledby conPageFastPath, [1600](#)
 - calledby dbConsHeading, [1649](#)
 - calledby dbConstructorDoc,hn, [1634](#)
 - calledby dbShowConditions, [1648](#)
 - calledby kdPageInfo, [1601](#)
 - calledby kePageDisplay, [1608](#)
 - calledby kePage, [1606](#)
 - calledby libConstructorSig, [1659](#)
 - calledby mkDomPvar, [731](#)
- letPrint, [108](#)
 - calledby mapLetPrint, [139](#)
 - calls break, [108](#)
 - calls bright, [108](#)
 - calls gensymp, [108](#)
 - calls hasPair, [108](#)
 - calls isSharpVarWithNum, [108](#)
 - calls isgenvar, [108](#)
 - calls lassoc, [108](#)
 - calls pname, [108](#)
 - calls sayBrightlyNT, [108](#)
 - calls shortenForPrinting, [108](#)
 - uses \$letAssoc, [108](#)
 - defun, [108](#)
- letPrint2, [110](#)
 - calls break, [110](#)
 - calls bright, [110](#)
 - calls gensymp, [110](#)
 - calls hasPair, [110](#)
 - calls isSharpVarWithNum, [110](#)
 - calls isgenvar, [110](#)
 - calls lassoc, [110](#)
 - calls mathprint, [110](#)
 - calls pname, [110](#)
 - calls print, [110](#)
 - uses \$BreakMode, [110](#)
 - uses \$letAssoc, [110](#)
 - catches, [110](#)
 - defun, [110](#)
- letPrint3, [111](#)
 - calls break, [111](#)
 - calls bright, [112](#)
 - calls gensymp, [111](#)

- calls hasPair, [111](#)
- calls isSharpVarWithNum, [111](#)
- calls isgenvar, [111](#)
- calls lassoc, [111](#)
- calls mathprint, [111](#)
- calls pname, [111](#)
- calls print, [111](#)
- calls spadcall, [111](#)
- uses \$BreakMode, [112](#)
- uses \$letAssoc, [112](#)
- catches, [111](#)
- defun, [111](#)
- lfcomment, [408](#)
 - calledby scanComment, [407](#)
 - defun, [408](#)
- lfferror, [427](#)
 - calledby scanError, [426](#)
 - defun, [427](#)
- lffloat, [419](#)
 - calledby scanExponent, [417](#)
 - calls concat, [419](#)
 - defun, [419](#)
- lfid, [406](#)
 - calledby scanToken, [405](#)
 - calledby scanWord, [417](#)
 - defun, [406](#)
- lfinteger, [425](#)
 - calledby scanNumber, [423](#)
 - defun, [425](#)
- lfkey, [413](#)
 - calledby scanKeyTr, [411](#)
 - calledby scanPossFloat, [412](#)
 - calledby scanWord, [417](#)
 - calls keyword, [413](#)
 - defun, [413](#)
- lfnegcomment, [409](#)
 - calledby scanNegComment, [408](#)
 - defun, [409](#)
- lfrinteger, [425](#)
 - calledby scanNumber, [423](#)
 - calls concat, [425](#)
 - defun, [425](#)
- lfspace, [421](#)
 - calledby scanSpace, [420](#)
 - defun, [421](#)
- lfstring, [421](#)
 - calledby scanString, [421](#)
 - defun, [421](#)
- libConstructorSig, [1659](#)
 - calls form2LispString, [1659](#)
 - calls getdatabase, [1659](#)
 - calls length, [1659](#)
 - calls ncParseFromString, [1659](#)
 - calls sayBrightly, [1659](#)
 - calls sublis, [1659](#)
 - calls take, [1659](#)
 - uses \$TriangleVariableList, [1659](#)
 - defun, [1659](#)
- libdbTrim, [1596](#)
 - defun, [1596](#)
- library, [932](#), [1199](#)
 - calls extendLocalLibdb, [1199](#)
 - calls localdatabase, [1199](#)
 - calls tersyscommand, [1199](#)
 - uses \$newConlist, [1199](#)
 - uses \$options, [1199](#)
 - defun, [1199](#)
 - manpage, [932](#)
- libstream, [1382](#)
 - defstruct, [1382](#)
- license, [934](#)
 - calls concat, [934](#)
 - calls getenv, [934](#)
 - calls obey, [934](#)
 - defun, [934](#)
 - manpage, [934](#)
- limitedPrint1, [153](#)
 - calledby monitorPrintValue, [152](#)
 - calledby monitorPrint, [150](#)
 - uses *print-length*, [153](#)
 - uses *print-level*, [153](#)
 - defun, [153](#)
- line
 - usedby commandErrorIfAmbiguous, [810](#)
 - usedby commandErrorMessage, [790](#)
 - usedby doSystemCommand, [786](#)
 - usedby intProcessSynonyms, [324](#)
 - usedby processSynonyms, [325](#)
 - usedby unAbbreviateKeyword, [809](#)
- line-clear
 - calledby ioclear, [1158](#)
- line-handler, [1153](#)

- usedby init-boot/spad-reader, 1154
 - defvar, 1153
- line-past-end-p
 - calledby iostat, 1156
 - calledby spad-short-error, 1156
- line-print
 - calledby iostat, 1156
 - calledby spad-short-error, 1156
- line?, 641
 - calledby msgOutputter, 644
 - calls getMsgTag, 641
 - defmacro, 641
- linearFinalRequest, 1490
 - calls bcQueryInteger, 1490
 - calls explainLinear, 1490
 - calls sayBrightly, 1490
 - defun, 1490
- linearFormatName
 - calledby sayCacheCount, 983
- lineoftoks, 402
 - calledby intloopInclude0, 355
 - calledby nclloopInclude0, 365
 - calledby parseFromString, 340
 - calledby parseNoMacroFromString, 1627
 - calls constoken, 403
 - calls dqUnit, 403
 - calls incPrefix?, 403
 - calls nextline, 403
 - calls scanIgnoreLine, 403
 - calls substring, 403
 - uses \$floatok, 403
 - uses \$f, 403
 - uses \$linepos, 403
 - uses \$ln, 403
 - uses \$n, 403
 - uses \$r, 403
 - uses \$sz, 403
 - defun, 402
- lisp, 935
 - manpage, 935
- list2vec
 - calledby computeTTTranspositions, 754
- listConstructorAbbreviations, 824
 - calledby abbreviationsSpad2Cmd, 822
 - calledby displaySpad2Cmd, 864
 - calls queryUserKeyedMsg, 824
 - calls sayKeyedMsg, 824
 - calls string2id-n, 824
 - calls upcase, 824
 - calls whatSpad2Cmd, 824
 - defun, 824
- listDecideHowMuch, 651
 - calledby getPosStL, 646
 - calls poGlobalLinePosn, 651
 - calls poNopos?, 651
 - calls poPosImmediate?, 651
 - defun, 651
- ListMember?
 - calledby whichCat, 655
- listOfStrings2String, 1549
 - defun, 1549
- listOutputter, 644
 - calledby processMsgList, 659
 - calls msgOutputter, 644
 - defun, 644
- listSort
 - calledby dbShowCons1, 1643
 - calledby dbShowConstructorLines, 1650
 - calledby domainDescendantsOf, 1603
 - calledby erMsgSort, 659
 - calledby kcaPage1, 1617
- literals
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby initializeSetVariables, 962
 - calledby set1, 1083
- Incgamma, 1256
 - calledby clngamma, 1278
 - calls clngamma, 1256
 - defun, 1256
- InCreate, 635
 - calledby incLine1, 381
 - defun, 635
- InExtraBlanks, 635
 - calledby scanError, 426
 - calledby scanS, 422
 - calledby scanToken, 406
 - defun, 635
- InFileName, 637
 - calledby poFileName, 650
 - calls InFileName?, 637
 - calls ncBug, 637

- defun, [637](#)
- InFileName?, [637](#)
 - calledby InFileName, [637](#)
 - calledby InImmediate?, [637](#)
 - defun, [637](#)
- InGlobalNum, [636](#)
 - calledby poGlobalLinePosn, [364](#)
 - defun, [636](#)
- InImmediate?, [637](#)
 - calledby poPosImmediate?, [650](#)
 - calls InFileName?, [637](#)
 - defun, [637](#)
- InLocalNum, [636](#)
 - calledby poLinePosn, [651](#)
 - defun, [636](#)
- InPlaceOfOrigin, [636](#)
 - defun, [636](#)
- Inrgamma, [1245](#)
 - calledby gammaStirling, [1245](#)
 - calledby rlngamma, [1278](#)
 - calls gammaRatapprox, [1245](#)
 - calls InrgammaRatapprox, [1245](#)
 - defun, [1245](#)
- InrgammaRatapprox, [1244](#)
 - calledby Inrgamma, [1245](#)
 - calls phiRatapprox, [1244](#)
 - defun, [1244](#)
- InSetGlobalNum, [636](#)
 - calledby incRenumItem, [368](#)
 - defun, [636](#)
- InString, [635](#)
 - defun, [635](#)
- loadFunctor, [1224](#)
 - calledby loadFunctor, [1224](#)
 - calledby traceDomainConstructor, [137](#)
 - calls loadFunctor, [1224](#)
 - calls loadLibIfNotLoaded, [1224](#)
 - defun, [1224](#)
- loadIfNecessary
 - calledby mkEvalable, [1116](#)
- loadLib, [1221](#)
 - calledby browse, [830](#)
 - calls categoryForm?, [1222](#)
 - calls clearConstructorCache, [1221](#)
 - calls getdatabase, [1221](#)
 - calls installConstructor, [1221](#)
 - calls isSystemDirectory, [1221](#)
 - calls loadLibNoUpdate, [1221](#)
 - calls namestring, [1221](#)
 - calls pathnameDirectory, [1221](#)
 - calls remprop, [1222](#)
 - calls sayKeyedMsg, [1221](#)
 - calls startTimingProcess, [1221](#)
 - calls stopTimingProcess, [1222](#)
 - calls updateCategoryTable, [1222](#)
 - calls updateDatabase, [1221](#)
 - local def \$CategoryFrame, [1222](#)
 - local ref \$InteractiveMode, [1222](#)
 - local ref \$forceDatabaseUpdate, [1222](#)
 - local ref \$printLoadMsgs, [1222](#)
 - defun, [1221](#)
- loadLibIfNotLoaded
 - calledby loadFunctor, [1224](#)
- loadLibNoUpdate, [1223](#)
 - calledby loadLib, [1221](#)
 - calledby localnrllib, [1201](#)
 - calls clearConstructorCache, [1223](#)
 - calls getdatabase, [1223](#)
 - calls installConstructor, [1223](#)
 - calls sayKeyedMsg, [1223](#)
 - calls stopTimingProcess, [1223](#)
 - calls toplevel, [1223](#)
 - local def \$CategoryFrame, [1223](#)
 - local ref \$InteractiveMode, [1223](#)
 - local ref \$printLoadMsgs, [1223](#)
 - defun, [1223](#)
- localdatabase, [1200](#)
 - calledby library, [1199](#)
 - calledby make-databases, [1205](#)
 - calls localnrllib, [1200](#)
 - calls sayKeyedMsg, [1200](#)
 - uses *index-filename*, [1200](#)
 - uses \$ConstructorCache, [1200](#)
 - uses \$forceDatabaseUpdate, [1200](#)
 - defun, [1200](#)
- localModemap, [78](#)
- localnrllib, [1201](#)
 - calledby localdatabase, [1200](#)
 - calls addoperations, [1201](#)
 - calls categoryForm?, [1201](#)
 - calls getdatabase, [1201](#)
 - calls installConstructor, [1201](#)

- calls loadLibNoUpdate, 1201
- calls make-database, 1201
- calls sayKeyedMsg, 1201
- calls setExposeAddConstr, 1201
- calls startTimingProcess, 1201
- calls sublislis, 1201
- calls updateCategoryTable, 1201
- calls updateDatabase, 1201
- uses *allOperations*, 1201
- uses *allconstructors*, 1201
- uses \$FormalMapVariableList, 1201
- defun, 1201
- localVars, 308
 - defvar, 308
- logH, 1249
 - calledby PiMinusLogSinPi, 1249
 - defun, 1249
- logS, 1251
 - calledby clngammacase2, 1250
 - defun, 1251
- lookupInDomainVector, 1227
 - calledby basicLookup, 1226
 - calledby oldCompLookup, 1229
 - calls basicLookupCheckDefaults, 1227
 - calls spadcall, 1227
 - defun, 1227
- loopIters2Sex, 609
 - calledby pf2Sex1, 598
 - calledby pfCollect2Sex, 612
 - calls pf2Sex1, 610
 - defun, 609
- lotsof, 1234
 - calledby wrap, 1233
 - defun, 1234
- ltrace, 936
 - calls trace, 936
 - defun, 936
 - manpage, 936
- Lucks, Michael, 14
- mac0Define, 526
 - calledby mac0MLambdaApply, 520
 - calledby macMacro, 526
 - uses \$pfMacros, 526
 - defun, 526
- mac0ExpandBody, 521
 - calledby mac0MLambdaApply, 520
 - calledby macId, 523
 - calls mac0InfiniteExpansion, 521
 - calls macExpand, 521
 - calls pfSourcePosition, 521
 - uses \$macActive, 521
 - uses \$posActive, 521
 - defun, 521
- mac0Get, 524
 - calledby macId, 523
 - calls ifcdr, 524
 - uses \$pfMacros, 524
 - defun, 524
- mac0GetName, 522
 - calledby mac0InfiniteExpansion,name, 522
 - calls pfMLambdaBody, 522
 - uses \$pfMacros, 522
 - defun, 522
- mac0InfiniteExpansion, 521
 - calledby mac0ExpandBody, 521
 - calls mac0InfiniteExpansion,name, 521
 - calls ncSoftError, 521
 - calls pform, 521
 - defun, 521
- mac0InfiniteExpansion,name, 522
 - calledby mac0InfiniteExpansion, 521
 - calls mac0GetName, 522
 - calls pname, 522
 - defun, 522
- mac0MLambdaApply, 520
 - calledby macApplication, 519
 - calls mac0Define, 520
 - calls mac0ExpandBody, 520
 - calls ncHardError, 520
 - calls pf0MLambdaArgs, 520
 - calls pfId?, 520
 - calls pfMLambdaBody, 520
 - calls pfSourcePosition, 520
 - calls pform, 520
 - uses \$macActive, 520
 - uses \$pfMacros, 520
 - uses \$posActive, 520
 - defun, 520
- mac0SubstituteOuter, 527
 - calledby mac0SubstituteOuter, 527
 - calledby macSubstituteOuter, 527

- calls `mac0SubstituteOuter`, 527
- calls `macLambdaParameterHandling`, 527
- calls `macSubstituteId`, 527
- calls `pfId?`, 527
- calls `pfLambda?`, 527
- calls `pfLeaf?`, 527
- calls `pfParts`, 527
- defun, 527
- `macApplication`, 519
 - calledby `macExpand`, 519
 - calls `mac0MLambdaApply`, 519
 - calls `macExpand`, 519
 - calls `pf0ApplicationArgs`, 519
 - calls `pfApplicationOp`, 519
 - calls `pfMLambda?`, 519
 - calls `pfMapParts`, 519
 - uses `$pfMacros`, 519
 - defun, 519
- `macExpand`, 518
 - calledby `mac0ExpandBody`, 521
 - calledby `macApplication`, 519
 - calledby `macExpand`, 519
 - calledby `macLambda,mac`, 525
 - calledby `macWhere,mac`, 525
 - calledby `macroExpanded`, 518
 - calls `macApplication`, 519
 - calls `macExpand`, 519
 - calls `macId`, 519
 - calls `macLambda`, 518
 - calls `macMacro`, 519
 - calls `macWhere`, 518
 - calls `pfApplication?`, 519
 - calls `pfId?`, 519
 - calls `pfLambda?`, 518
 - calls `pfMacro?`, 519
 - calls `pfMapParts`, 519
 - calls `pfWhere?`, 518
 - defun, 518
- `macId`, 523
 - calledby `macExpand`, 519
 - calls `mac0ExpandBody`, 523
 - calls `mac0Get`, 523
 - calls `pfCopyWithPos`, 523
 - calls `pfIdSymbol`, 523
 - calls `pfSourcePosition`, 523
 - uses `$macActive`, 523
- uses `$posActive`, 523
- defun, 523
- `macLambda`, 525
 - calledby `macExpand`, 518
 - calls `macLambda,mac`, 525
 - uses `$pfMacros`, 525
 - defun, 525
- `macLambda,mac`, 525
 - calledby `macLambda`, 525
 - calls `macExpand`, 525
 - calls `pfMapParts`, 525
 - uses `$pfMacros`, 525
 - defun, 525
- `macLambdaParameterHandling`, 527
 - calledby `mac0SubstituteOuter`, 527
 - calledby `macLambdaParameterHandling`, 528
 - calledby `macSubstituteOuter`, 527
 - calls `macLambdaParameterHandling`, 528
 - calls `pf0LambdaArgs`, 528
 - calls `pf0MLambdaArgs`, 528
 - calls `pfAbSynOp`, 528
 - calls `pfIdSymbol`, 528
 - calls `pfLambda?`, 528
 - calls `pfLeaf?`, 527
 - calls `pfLeafPosition`, 528
 - calls `pfLeaf`, 528
 - calls `pfMLambda?`, 528
 - calls `pfParts`, 528
 - calls `pfTypedId`, 528
 - defun, 527
- `macMacro`, 525
 - calledby `macExpand`, 519
 - calls `mac0Define`, 526
 - calls `macSubstituteOuter`, 526
 - calls `ncSoftError`, 526
 - calls `pfId?`, 526
 - calls `pfIdSymbol`, 526
 - calls `pfMLambda?`, 526
 - calls `pfMacroLhs`, 525
 - calls `pfMacroRhs`, 526
 - calls `pfMacro`, 526
 - calls `pfNothing?`, 526
 - calls `pfNothing`, 526
 - calls `pfSourcePosition`, 526
 - calls `pform`, 526

- defun, 525
- macroExpanded, 518
 - calledby parseFromString, 340
 - calledby phMacro, 518
 - calls macExpand, 518
 - uses \$macActive, 518
 - uses \$posActive, 518
 - defun, 518
- macSubstituteId, 528
 - calledby mac0SubstituteOuter, 527
 - calls pfIdSymbol, 528
 - defun, 528
- macSubstituteOuter, 527
 - calledby macMacro, 526
 - calls mac0SubstituteOuter, 527
 - calls macLambdaParameterHandling, 527
 - defun, 527
- macWhere, 524
 - calledby macExpand, 518
 - calls macWhere,mac, 524
 - uses \$pfMacros, 524
 - defun, 524
- macWhere,mac, 524
 - calledby macWhere, 524
 - calls macExpand, 525
 - calls pfMapParts, 524
 - uses \$pfMacros, 525
 - defun, 524
- make-absolute-filename, 328
 - calledby reroot, 333
 - uses \$spadroot, 328
 - defun, 328
- make-appendstream, 1168
 - calledby makeStream, 1169
 - calls make-filename, 1168
 - defun, 1168
- make-cdouble-matrix, 1273
 - defmacro, 1273
- make-cdouble-vector, 1275
 - defmacro, 1275
- make-database
 - calledby interpopen, 1190
 - calledby localnrLib, 1201
 - calledby setdatabase, 1195
- make-databases, 1204
 - calls allConstructors, 1205
 - calls browserAutoloadOnceTrigger, 1205
 - calls buildLibdb, 1205
 - calls categoryForm?, 1205
 - calls dbSplitLibdb, 1205
 - calls domainsOf, 1205
 - calls getConstructorForm, 1205
 - calls getenvir, 1205
 - calls localdatabase, 1205
 - calls mkDependentsHashTable, 1205
 - calls mkTopicHashTable, 1205
 - calls mkUsersHashTable, 1205
 - calls saveDependentsHashTable, 1205
 - calls saveUsersHashTable, 1205
 - calls write-browsedb, 1205
 - calls write-categorydb, 1205
 - calls write-interpdb, 1205
 - calls write-operationdb, 1205
 - calls write-warndata, 1205
 - uses *allconstructors*, 1205
 - uses *operation-hash*, 1205
 - uses *sourcefiles*, 1205
 - uses \$constructorList, 1205
 - defun, 1204
- make-double-matrix, 1294
 - defmacro, 1294
- make-double-matrix1, 1294
 - defmacro, 1294
- make-double-vector, 1295
 - defmacro, 1295
- make-double-vector1, 1296
 - defmacro, 1296
- make-filename
 - calledby make-appendstream, 1168
 - calledby make-outstream, 1167
 - calledby pathname, 1232
- make-hashtable
 - calledby dbDocTable, 1635
- make-instream, 1167
 - calledby dewritify,dewritifyInner, 922
 - calledby incConsoleInput, 396
 - calledby incFileInput, 395
 - calledby newHelpSpad2Cmd, 881
 - calls makeInputFilename, 1167
 - defun, 1167
- make-monitor-data
 - calledby monitor-add, 1384

- make-outstream, 1167
 - calledby makeStream, 1169
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calls make-filename, 1167
 - defun, 1167
- make-string
 - calledby executeQuietCommand, 339
- makeByteWordVec2, 1373
 - defun, 1373
- makeFullNamestring, 1171
 - calledby makeStream, 1171
 - defun, 1171
- makeHistFileName, 887
 - calledby closeInterpreterFrame, 37
 - calledby histFileName, 888
 - calledby oldHistFileName, 887
 - calledby restoreHistory, 907
 - calledby saveHistory, 905
 - calls makePathname, 887
 - defun, 887
- makeInitialModemapFrame, 328
 - calledby restart, 307
 - calls copy, 328
 - uses \$InitialModemapFrame, 328
 - defun, 328
- makeInputFilename, 1169
 - calledby initHist, 888
 - calledby make-instream, 1167
 - calledby newHelpSpad2Cmd, 881
 - calledby restoreHistory, 907
 - calledby saveHistory, 905
 - calledby updateSourceFiles, 874
 - calledby workfilesSpad2Cmd, 1139
 - defun, 1169
- makeLeaderMsg, 667
 - calledby processChPosesForOneLine, 666
 - uses \$nopus, 667
 - uses \$preLength, 667
 - defun, 667
- makeLongSpaceString
 - calledby printStorage, 351
- makeLongTimeString
 - calledby printTypeAndTime, 352
- makeMatrix1U16, 1322
 - defmacro, 1322
- makeMatrix1U32, 1323
 - defmacro, 1323
- makeMatrix1U8, 1320
 - defmacro, 1320
- makeMatrixU16, 1322
 - defmacro, 1322
- makeMatrixU32, 1323
 - defmacro, 1323
- makeMatrixU8, 1320
 - defmacro, 1320
- makeMsgFromLine, 661
 - calledby processMsgList, 659
 - calls char, 661
 - calls concat, 661
 - calls getLinePos, 661
 - calls getLineText, 661
 - calls poGlobalLinePosn, 661
 - calls poLinePosn, 661
 - calls rep, 661
 - calls size, 661
 - uses \$preLength, 661
 - defun, 661
- makeOrdinal, 1123
 - calledby evaluateType1, 1121
 - defun, 1123
- makePathname, 1233
 - calledby histInputFileName, 888
 - calledby makeHistFileName, 887
 - calledby readSpad2Cmd, 943
 - calledby sayMSG2File, 44
 - calls object2String, 1233
 - calls pathname, 1233
 - defun, 1233
- makeSpadCommand, 1543
 - defun, 1543
- makeStream, 1169
 - calledby setOutputFortran, 1042
 - calls make-appendstream, 1169
 - calls make-outstream, 1169
 - calls makeFullNamestring, 1171
 - defun, 1169
- MakeSymbol

- calledby assertCond, 389
- calledby ifCond, 382
- manexp, 1290
 - defun, 1290
- mapage
 - abbreviations, 821
 - boot, 826
 - browse, 827
 - cd, 832
 - clear, 833
 - close, 844
 - compile, 847
 - copyright, 850
 - credits, 856
 - describe, 857
 - display, 862
 - edit, 871
 - fin, 876
 - frame, 18
 - help, 878
 - history, 883
 - include, 929
 - library, 932
 - license, 934
 - lisp, 935
 - ltrace, 936
 - pquit, 937
 - quit, 939
 - read, 942
 - regress, 946
 - savesystem, 958
 - set, 960
 - show, 1086
 - spool, 1101
 - summary, 1102
 - synonym, 1104
 - system, 1108
 - tangle, 1109
 - trademark, 1111
 - undo, 1112
 - what, 1129
 - workfiles, 1139
- mapLetPrint, 139
 - calls getAliasIfTracedMapParameter, 139
 - calls getBpiNameIfTracedMap, 139
 - calls letPrint, 139
 - defun, 139
- maprinSpecial
 - calledby prinmathor0, 151
- mapStringize, 1509
 - calledby bcHt, 1508
 - calledby iht, 1508
 - calledby mapStringize, 1509
 - calls basicStringize, 1509
 - calls mapStringize, 1509
 - defun, 1509
- markUnique, 135
 - calledby getOperationAlistFromLisplib, 134
 - defun, 135
- mathprint
 - calledby displayMacro, 794
 - calledby displayValue, 799
 - calledby letPrint2, 110
 - calledby letPrint3, 111
- maxindex
 - calledby dbWordFrom, 1592
 - calledby processSynonymLine,removeKeyFromLine, 1107
 - calledby removeUndoLines, 57
- member, 1238
 - calledby bcSolveEquations, 1486
 - calledby clearCmdParts, 841
 - calledby coerceIntByMap, 761
 - calledby coerceIntPermute, 754
 - calledby computeTTTranspositions, 754
 - calledby conSpecialString?, 1598
 - calledby dbConsHeading, 1649
 - calledby dbShowConsDoc1, 1646
 - calledby dbShowCons, 1641
 - calledby dbWordFrom, 1592
 - calledby displayMacros, 867
 - calledby displayProperties, 801
 - calledby doSystemCommand, 786
 - calledby evaluateType, 1118
 - calledby fixObjectForPrinting, 796
 - calledby getBrowseDatabase, 1310
 - calledby handleNoParseCommands, 810
 - calledby hasCateSpecialNew, 734
 - calledby historySpad2Cmd, 890
 - calledby httpDestroyPage, 1468
 - calledby importFromFrame, 34
 - calledby interpret2, 348

- calledby isDomainValuedVariable, 1145
- calledby isSubForRedundantMapName, 105
- calledby kisValidType, 1626
- calledby readSpad2Cmd, 943
- calledby reportOpsFromUnitDirectly, 1096
- calledby retract2Specialization, 772
- calledby selectOption, 819
- calledby setExposeAddConstr, 160
- calledby setExposeAddGroup, 157
- calledby setExposeDropConstr, 164
- calledby setExposeDropGroup, 162
- calledby setOutputAlgebra, 1035
- calledby setOutputFormula, 1063
- calledby setOutputFortran, 1042
- calledby setOutputHtml, 1048
- calledby setOutputMathml, 1054
- calledby setOutputOpenMath, 1058
- calledby setOutputTex, 1070
- calledby showSpad2Cmd, 1087
- calledby transTraceItem, 126
- calledby translateYesNo2TrueFalse, 967
- calledby updateSourceFiles, 874
- defun, 1238
- menuButton
 - calledby kePageDisplay, 1608
 - calledby kePage, 1606
- mergePathnames, 1231
 - calledby readSpad2Cmd, 943
 - calls pathnameDirectory, 1231
 - calls pathnameName, 1231
 - calls pathnameType, 1231
 - defun, 1231
- messageprint, 1239
 - calledby brightprint, 1238
 - defun, 1239
- messageprint-1, 1239
 - calledby brightprint-0, 1238
 - calledby messageprint-1, 1239
 - calledby messageprint-2, 1239
 - calls identp, 1239
 - calls messageprint-1, 1239
 - calls messageprint-2, 1239
 - defun, 1239
- messageprint-2, 1239
 - calledby messageprint-1, 1239
 - calledby messageprint-2, 1239
- calls messageprint-1, 1239
- calls messageprint-2, 1239
- defun, 1239
- meta-error-handler
 - usedby init-boot/spad-reader, 1154
- mkAtree
 - calledby coerceInt1, 743
 - calledby evaluateType1, 1120
 - calledby evaluateType, 1118
 - calledby getSubDomainPredicate, 769
 - calledby reportOperations, 1089
- mkAtreeNode
 - calledby coerceInt1, 743
- mkAtreeWithSrcPos
 - calledby interpret1, 347
- mkCompanionPage
 - calledby recordAndPrint, 349
- mkConform, 1628
 - calledby conOpPage1, 1629
 - calledby kDomainName, 1624
 - calledby kPage, 1593
 - calledby kePage, 1605
 - calledby kiPage, 1604
 - calls concat, 1628
 - calls ncParseFromString, 1628
 - calls nequal, 1628
 - calls parseNoMacroFromString, 1628
 - calls pp, 1628
 - calls sayBrightlyNT, 1628
 - calls systemError, 1628
 - defun, 1628
- mkCurryFun, 1521
 - defun, 1521
- mkDependentsHashTable
 - calledby make-databases, 1205
- mkDomPvar, 731
 - calledby hasCaty, 718
 - calls domArg, 731
 - calls length, 731
 - local ref \$FormalMapVariableList, 731
 - defun, 731
- mkDomTypeForm, 1603
 - calledby kArgPage, 1602
 - calledby mkDomTypeForm, 1603
 - calls hasIndent, 1603
 - calls mkDomTypeForm, 1603

- calls sublis, 1603
- defun, 1603
- mkEvalable, 1115
 - calledby dbMkEvalable, 1625
 - calledby evalDomain, 1115
 - calledby mkEvalableMapping, 1118
 - calledby mkEvalableRecord, 1118
 - calledby mkEvalableUnion, 1117
 - calledby mkEvalable, 1115
 - calledby writify, writifyInner, 917
 - calls bpiname, 1116
 - calls constructor?, 1116
 - calls devaluate, 1116
 - calls fbip, 1116
 - calls getdatabase, 1116
 - calls loadIfNecessary, 1116
 - calls mkEvalableMapping, 1116
 - calls mkEvalableRecord, 1116
 - calls mkEvalableUnion, 1116
 - calls mkEvalable, 1115
 - calls mkq, 1116
 - calls qcar, 1115
 - calls qcdr, 1115
 - local ref \$EmptyMode, 1116
 - local ref \$Integer, 1116
 - defun, 1115
- mkEvalableMapping, 1118
 - calledby mkEvalable, 1116
 - calls mkEvalable, 1118
 - defun, 1118
- mkEvalableRecord, 1118
 - calledby mkEvalable, 1116
 - calls mkEvalable, 1118
 - defun, 1118
- mkEvalableUnion, 1117
 - calledby mkEvalable, 1116
 - calls mkEvalable, 1117
 - defun, 1117
- mkLineList, 362
 - calledby intloopEchoParse, 361
 - defun, 362
- mkObj, 514
 - calledby coerceBranch2Union, 766
 - calledby coerceByTable, 759
 - calledby coerceInt0, 742
 - calledby coerceInt1, 743
 - calledby coerceInteractive, 740
 - calledby coerceUnion2Branch, 775
 - calledby interpret1, 347
 - calledby interpret2, 348
 - calledby retract2Specialization, 772
 - calledby retract, 1269
 - defmacro, 514
- mkObjCode, 515
 - defmacro, 515
- mkobjFn, 1306
 - defun, 1306
- mkObjWrap, 515
 - calledby coerceBranch2Union, 766
 - calledby coerceByTable, 759
 - calledby coerceInt1, 743
 - calledby coerceIntAlgebraicConstant, 767
 - calledby coerceIntByMap, 761
 - calledby coerceIntCommute, 757
 - calledby coerceIntX, 768
 - calledby coerceInteractive, 740
 - calledby coerceOrThrowFailure, 763
 - calledby coerceRetract, 777
 - calledby coerceSpadArgs2E, 128
 - calledby coerceSpadFunValue2E, 130
 - calledby coerceTraceArgs2E, 127
 - calledby coerceTraceFunValue2E, 129
 - calledby getAndEvalConstructorArgument, 1144
 - calledby printTypeAndTime, 352
 - calledby recordAndPrint, 349
 - calledby retract2Specialization, 772
 - calledby retractByFunction, 778
 - calledby valueArgsEqual?, 764
 - calls wrap, 515
 - defmacro, 515
- mkPredList
 - calledby coerceBranch2Union, 766
 - calledby coerceUnion2Branch, 775
- mkprompt, 334
 - calledby SpadInterpretStream, 319
 - calledby intloopReadConsole, 321
 - calledby serverReadLine, 336
 - calls concat, 334
 - calls currenttime, 334
 - calls substring, 334
 - uses \$IOindex, 334

- uses \$inputPromptType, 334
 - uses \$interpreterFrameName, 334
 - defun, 334
- mkq
 - calledby coerceByTable, 759
 - calledby mkEvalable, 1116
 - calledby monitorGetValue, 94
 - calledby monitorPrintArgs, 148
 - calledby monitorPrintValue, 152
 - calledby traceDomainConstructor, 137
- mkSetTitle, 1549
 - defun, 1549
- mkTopicHashTable
 - calledby make-databases, 1205
- mkUnixPattern, 1587
 - defun, 1587
- mkUserConstructorAbbreviation
 - calledby abbreviationsSpad2Cmd, 823
- mkUsersHashTable
 - calledby make-databases, 1205
- moan
 - calledby monitorXX, 90
- modes
 - calledby clearCmdParts, 841
- MONITOR,EVALTRAN
 - calledby break, 156
- monitor-add, 1384
 - calledby monitor-file, 1387
 - calls make-monitor-data, 1384
 - calls monitor-delete, 1384
 - uses *monitor-table*, 1384
 - defun, 1384
- monitor-a-propos, 1397
 - uses *monitor-table*, 1397
 - defun, 1397
- monitor-autoload, 1392
 - defun, 1392
- monitor-checkpoint, 1389
 - uses *monitor-table*, 1389
 - uses *print-package*, 1389
 - defun, 1389
- monitor-data, 1382
 - defstruct, 1382
- monitor-decr, 1386
 - uses *monitor-table*, 1386
 - defun, 1386
- monitor-delete, 1384
 - calledby monitor-add, 1384
 - calledby monitor-tested, 1388
 - uses *monitor-table*, 1384
 - defun, 1384
- monitor-dirname, 1392
 - uses *monitor-nrlibs*, 1392
 - defun, 1392
- monitor-disable, 1385
 - uses *monitor-table*, 1385
 - defun, 1385
- monitor-enable, 1384
 - uses *monitor-table*, 1384
 - defun, 1384
- monitor-end, 1383
 - uses *monitor-table*, 1383
 - defun, 1383
- monitor-exposedp, 1394
 - defun, 1394
- monitor-file, 1387
 - calls monitor-add, 1387
 - catches, 1387
 - defun, 1387
 - throws, 1387
- monitor-help, 1390
 - defun, 1390
- monitor-incr, 1386
 - uses *monitor-table*, 1386
 - defun, 1386
- monitor-info, 1387
 - uses *monitor-table*, 1387
 - defun, 1387
- monitor-inittable, 1382
 - uses *monitor-table*, 1382
 - defun, 1382
- monitor-libname, 1393
 - defun, 1393
- monitor-nrlib, 1393
 - uses *monitor-table*, 1393
 - defun, 1393
- monitor-parse, 1396
 - calledby monitor-spadfile, 1396
 - defun, 1396
- monitor-percent, 1397
 - uses *monitor-table*, 1397
 - defun, 1397

- monitor-readinterp, 1394
 - calledby monitor-report, 1395
 - uses *monitor-domains*, 1394
 - catches, 1394
 - defun, 1394
 - throws, 1394
- monitor-report, 1395
 - calls monitor-readinterp, 1395
 - uses *monitor-domains*, 1395
 - defun, 1395
- monitor-reset, 1385
 - uses *monitor-table*, 1385
 - defun, 1385
- monitor-restore, 1389
 - defun, 1389
- monitor-results, 1383
 - uses *monitor-table*, 1383
 - defun, 1383
- monitor-spadfile, 1396
 - calls monitor-parse, 1396
 - uses *monitor-domains*, 1396
 - catches, 1396
 - defun, 1396
 - throws, 1396
- monitor-tested, 1388
 - calls monitor-delete, 1388
 - uses *monitor-table*, 1388
 - defun, 1388
- monitor-untested, 1387
 - uses *monitor-table*, 1387
 - defun, 1387
- monitor-write, 1388
 - defun, 1388
- monitorBlanks
 - calledby monitorPrintRest, 150
- monitorEnter, 147
 - calledby monitorXX, 90
 - calls coerceTraceArgs2E, 147
 - calls monitorPrintArgs, 148
 - calls sayBrightlyNT2, 147
 - calls spadsysnamep, 147
 - uses \$TraceFlag, 148
 - uses \$mathTrace, 148
 - uses \$monitorCaller, 148
 - uses \$monitorDepth, 148
 - uses \$monitorFunDepth, 148
 - uses \$traceStream, 148
 - defun, 147
- monitorEvalAfter, 92
 - calledby monitorXX, 90
 - calls monitorEvalTran, 92
 - defun, 92
- monitorEvalBefore, 92
 - calledby monitorXX, 90
 - calls monitorEvalTran, 92
 - defun, 92
- monitorEvalTran, 93
 - calledby monitorEvalAfter, 92
 - calledby monitorEvalBefore, 92
 - calledby monitorXX, 90
 - calls hasSharpVar, 93
 - calls monitorEvalTran1, 93
 - defun, 93
- monitorEvalTran1, 93
 - calledby monitorEvalTran1, 93
 - calledby monitorEvalTran, 93
 - calls isSharpVarWithNum, 93
 - calls monitorEvalTran1, 93
 - calls monitorGetValue, 93
 - defun, 93
- monitorExit, 151
 - calledby monitorXX, 90
 - calls coerceTraceFunValue2E, 151
 - calls monitorPrintValue, 151
 - calls tab, 151
 - uses \$TraceFlag, 151
 - uses \$mathTrace, 151
 - uses \$monitorFunDepth, 151
 - uses \$monitorValue, 151
 - uses \$traceStream, 151
 - defun, 151
- monitorGetValue, 94
 - calledby monitorEvalTran1, 93
 - calls mkq, 94
 - calls spadThrowBrightly, 94
 - uses \$monitorArgs, 94
 - uses \$monitorCaller, 94
 - uses \$monitorName, 94
 - uses \$monitorValue, 94
 - defun, 94
- monitorPrint, 150
 - calledby monitorPrintArgs, 148

- calledby monitorPrintArg, 149
 - calls limitedPrint1, 150
 - calls prinmathor0, 150
 - calls smallEnough, 150
 - uses \$monitorPretty, 150
 - defun, 150
- monitorPrintArg, 149
 - calledby monitorPrintArgs, 148
 - calls monitorPrint, 149
 - uses \$traceStream, 149
 - defun, 149
- monitorPrintArgs, 148
 - calledby monitorEnter, 148
 - calls mkq, 148
 - calls monitorPrintArg, 148
 - calls monitorPrintRest, 148
 - calls monitorPrint, 148
 - calls prinmathor0, 148
 - uses \$mathTrace, 148
 - uses \$traceStream, 148
 - defun, 148
- monitorPrintRest, 150
 - calledby monitorPrintArgs, 148
 - calls monitorBlanks, 150
 - calls prinmathor0, 150
 - calls smallEnough, 150
 - uses \$mathTrace, 150
 - uses \$monitorDepth, 150
 - uses \$monitorPretty, 150
 - uses \$traceStream, 150
 - defun, 150
- monitorPrintValue, 152
 - calledby monitorExit, 151
 - calls eqcar, 152
 - calls limitedPrint1, 152
 - calls mkq, 152
 - calls prinmathor0, 152
 - calls smallEnough, 152
 - uses \$mathTrace, 152
 - uses \$monitorPretty, 152
 - uses \$traceStream, 152
 - defun, 152
- monitorX, 90
 - calls monitorXX, 90
 - uses \$depthAlist, 90
 - uses \$monitorDepth, 90
- defun, 90
- monitorXX, 90
 - calledby monitorX, 90
 - calls break, 90
 - calls moan, 90
 - calls monitorEnter, 90
 - calls monitorEvalAfter, 90
 - calls monitorEvalBefore, 90
 - calls monitorEvalTran, 90
 - calls monitorExit, 90
 - calls rassocSub, 90
 - calls startTimer, 90
 - calls stopTimer, 90
 - calls timerValue, 90
 - calls whocalled, 90
 - uses \$breakCondition, 90
 - uses \$depthAlist, 90
 - uses \$mapSubNameAlist, 90
 - uses \$mathTrace, 90
 - uses \$monitorArgs, 90
 - uses \$monitorCaller, 90
 - uses \$monitorDepth, 90
 - uses \$monitorFunDepth, 90
 - uses \$monitorName, 90
 - uses \$monitorValue, 90
 - uses \$tracedSpadModemap, 90
 - defun, 90
- moreExactSolution
 - calledby finalExactRequest, 1491
- Morrison, Scott C., 14
- msgCreate, 637
 - calledby ncBug, 658
 - calledby ncHardError, 642
 - calledby ncSoftError, 642
 - calls initImPr, 638
 - calls initToWhere, 638
 - calls putDatabaseStuff, 638
 - calls setMsgForcedAttrList, 638
 - defun, 637
- msgImPr?, 648
 - calledby alreadyOpened?, 653
 - calledby getPosStL, 646
 - calledby processKeyedError, 643
 - calledby showMsgPos?, 648
 - calls getMsgCatAttr, 648
 - defun, 648

- msgNoRep?, 665
 - calledby redundant, 664
 - calls getMsgCatAttr, 665
 - defun, 665
- msgOutputter, 643
 - calledby listOutputter, 644
 - calledby processKeyedError, 643
 - calls alreadyOpened?, 644
 - calls flowSegmentedMsg, 644
 - calls getStFromMsg, 643
 - calls leader?, 644
 - calls line?, 644
 - calls sayBrightly, 644
 - calls toFile?, 644
 - calls toScreen?, 644
 - uses \$linelength, 644
 - defun, 643
- msgText, 354
 - calledby printTypeAndTime, 352
 - calls flowSegmentedMsg, 354
 - calls segmentKeyedMsg, 354
 - calls substituteSegmentedMsg, 354
 - uses \$linelength, 354
 - uses \$margin, 354
 - defun, 354
- msort
 - calledby apropos, 1137
 - calledby computeTTTranspositions, 754
 - calledby displayOperationsFromLisplib, 1095
 - calledby displayProperties, 801
 - calledby displayWorkspaceNames, 795
 - calledby reportOpsFromLisplib, 1093
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1208
 - calledby setExposeAddConstr, 161
 - calledby setExposeAddGroup, 157
 - calledby setExposeDropConstr, 164
 - calledby whatConstructors, 1137
- msubst
 - calledby getSubDomainPredicate, 769
- myWritable?, 1300
 - calls error, 1300
 - calls fnameDirectory, 1300
 - calls fnameExists?, 1300
 - calls writeablep, 1300
 - defun, 1300
- myWritable?
 - calledby fnameWritable?, 1300
- names
 - calledby frameSpad2Cmd, 26
- namestring, 1230
 - calledby bcUnixTable, 1650
 - calledby editFile, 873
 - calledby loadLib, 1221
 - calledby newHelpSpad2Cmd, 881
 - calledby readSpad2Cmd, 943
 - calledby reportOpsFromLisplib, 1092
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby restoreHistory, 907
 - calledby saveHistory, 905
 - calledby setExposeAddGroup, 157
 - calledby setExpose, 159
 - calledby workfilesSpad2Cmd, 1139
 - calledby writeInputLines, 896
 - calls pathname, 1230
 - defun, 1230
- ncAbort
 - calledby ncBug, 658
- ncAlist, 705
 - calledby getMsgCatAttr, 649
 - calledby ncEltQ, 706
 - calledby ncPutQ, 707
 - calledby setMsgCatlessAttr, 655
 - calledby setMsgUnforcedAttr, 657
 - calledby tokPosn, 703
 - calls identp, 705
 - calls ncBug, 705
 - calls qcar, 705
 - calls qcdr, 706
 - defun, 705
- ncBug, 658
 - calledby getMsgPos2, 669
 - calledby incHandleMessage, 368
 - calledby lnFileName, 637
 - calledby ncAlist, 705
 - calledby ncEltQ, 706
 - calledby ncTag, 705
 - calledby poGlobalLinePosn, 364
 - calls enable-backtrace, 658

- calls msgCreate, 658
 - calls ncAbort, 658
 - calls processKeyedError, 658
 - uses \$newcompErrorCount, 658
 - uses \$nopus, 658
 - defun, 658
- ncConversationPhase, 360
 - calledby intloopSpadProcess,interp, 358
 - calls ncConversationPhase,wrapup, 360
 - uses \$ncMsgList, 360
 - defun, 360
- ncConversationPhase,wrapup, 361
 - calledby ncConversationPhase, 360
 - uses \$ncMsgList, 361
 - defun, 361
- ncEltQ, 706
 - calledby intloopSpadProcess,interp, 358
 - calledby phIntReportMsgs, 359
 - calledby phInterpret, 359
 - calledby phMacro, 518
 - calls ncAlist, 706
 - calls ncBug, 706
 - calls qassq, 706
 - defun, 706
- ncError, 361
 - calledby intloopSpadProcess,interp, 358
 - calledby ncHardError, 642
 - defun, 361
 - throws, 361
- ncHardError, 642
 - calledby mac0MLambdaApply, 520
 - calls desiredMsg, 642
 - calls msgCreate, 642
 - calls ncError, 642
 - calls processKeyedError, 642
 - uses \$newcompErrorCount, 642
 - defun, 642
- ncIntLoop, 317
 - calledby ncTopLevel, 316
 - calls intloop, 317
 - uses curinstream, 317
 - uses curoutstream, 317
 - defun, 317
- ncloopCommand, 818
 - calledby intloopReadConsole, 321
 - calls \$systemCommandFunction, 818
 - calls ncloopInclude1, 818
 - calls ncloopPrefix?, 818
 - uses \$systemCommandFunction, 818
 - defun, 818
- ncloopDQlines, 364
 - calledby intloopEchoParse, 361
 - calledby ncloopParse, 329
 - calls StreamNull, 364
 - calls poGlobalLinePosn, 364
 - calls streamChop, 364
 - calls tokPosn, 364
 - defun, 364
- ncloopEchoParse
 - calledby ncloopInclude0, 365
- ncloopEscaped, 329
 - calledby intloopReadConsole, 321
 - defun, 329
- ncloopIncFileName, 929
 - calledby ncloopInclude1, 929
 - calls concat, 929
 - calls incFileName, 929
 - defun, 929
- ncloopInclude, 930
 - calledby ncloopInclude1, 929
 - calls ncloopInclude0, 930
 - defun, 930
- ncloopInclude0, 365
 - calledby ncloopInclude, 930
 - calls incStream, 365
 - calls insertpile, 365
 - calls lineoftoks, 365
 - calls ncloopEchoParse, 365
 - calls ncloopProcess, 365
 - calls next, 365
 - uses \$lines, 365
 - defun, 365
- ncloopInclude1, 929
 - calledby ncloopCommand, 818
 - calls ncloopIncFileName, 929
 - calls ncloopInclude, 929
 - defun, 929
- ncloopParse, 329
 - calledby parseFromString, 340
 - calledby parseNoMacroFromString, 1627
 - calls dqToList, 330
 - calls ncloopDQlines, 329

- calls npParse, 330
 - defun, 329
- ncloopPrefix?, 818
 - calledby ncloopCommand, 818
 - calledby streamChop, 365
 - defun, 818
- ncloopPrintLines, 362
 - calledby intloopEchoParse, 361
 - defun, 362
- ncloopProcess
 - calledby ncloopInclude0, 365
- ncconc
 - calledby flatBvList, 89
- ncParseFromString, 1310
 - calledby conOpPage1, 1629
 - calledby conSpecialString?, 1598
 - calledby kPage, 1593
 - calledby kcodePage, 1620
 - calledby kcuPage, 1621
 - calledby libConstructorSig, 1659
 - calledby mkConform, 1628
 - defun, 1310
- ncPutQ, 706
 - calledby constoken, 405
 - calledby intloopSpadProcess, 357
 - calledby phIntReportMsgs, 359
 - calledby phInterpret, 359
 - calledby phMacro, 518
 - calledby phParse, 358
 - calledby setMsgCatlessAttr, 655
 - calledby setMsgForcedAttr, 654
 - calledby setMsgUnforcedAttr, 657
 - calledby tokConstruct, 701
 - calls ncAlist, 707
 - calls ncTag, 707
 - calls qassq, 707
 - defun, 706
- ncSoftError, 642
 - calledby incHandleMessage, 368
 - calledby mac0InfiniteExpansion, 521
 - calledby macMacro, 526
 - calledby npMissingMate, 508
 - calledby npMissing, 444
 - calledby npParse, 433
 - calledby npTrapForm, 505
 - calledby npTrap, 505
- calledby scanError, 426
 - calledby scanS, 422
 - calledby syIgnoredFromTo, 484
 - calledby sySpecificErrorAtToken, 485
 - calls desiredMsg, 642
 - calls msgCreate, 642
 - calls processKeyedError, 642
 - uses \$newcompErrorCount, 642
 - defun, 642
- ncTag, 705
 - calledby getMsgTag, 640
 - calledby ncPutQ, 707
 - calledby tokType, 703
 - calls identp, 705
 - calls ncBug, 705
 - calls qcar, 705
 - defun, 705
- ncTopLevel, 316
 - calledby runspad, 311
 - calls ncIntLoop, 316
 - uses *eof*, 316
 - uses \$InteractiveFrame, 316
 - uses \$InteractiveMode, 316
 - uses \$boot, 316
 - uses \$e, 316
 - uses \$newspad, 316
 - uses \$spad, 316
 - uses in-stream, 316
 - defun, 316
- nequal
 - calledby absolutelyCanCoerceByCheating, 770
 - calledby coerceInt1, 743
 - calledby coerceIntByMap, 761
 - calledby computeTTTranspositions, 754
 - calledby dbConsHeading, 1649
 - calledby kPage, 1612
 - calledby kcodePage, 1620
 - calledby kcuPage, 1621
 - calledby kdPageInfo, 1601
 - calledby mkConform, 1628
 - calledby retractUnderDomain, 777
- new
 - calledby frameSpad2Cmd, 26
- newHelpSpad2Cmd, 881
 - calledby helpSpad2Cmd, 881

- calls abbreviation?, 881
- calls concat, 881
- calls make-instream, 881
- calls makeInputFilename, 881
- calls namestring, 881
- calls obey, 881
- calls pname, 881
- calls poundsign, 881
- calls sayKeyedMsg, 881
- calls say, 881
- calls selectOptionLC, 881
- uses \$syscommands, 881
- uses \$useFullScreenHelp, 881
- defun, 881
- next, 330
 - calledby frameSpad2Cmd, 26
 - calledby intloopInclude0, 355
 - calledby intloopProcessString, 329
 - calledby nclloopInclude0, 365
 - calledby next1, 330
 - calledby parseFromString, 340
 - calledby parseNoMacroFromString, 1627
 - calls Delay, 330
 - calls next1, 330
 - defun, 330
- next-lines-clear, 1158
 - calledby init-boot/spad-reader, 1154
 - uses boot-line-stack, 1158
 - defun, 1158
- next-lines-show, 1157
 - calledby iostat, 1156
 - uses boot-line-stack, 1157
 - defun, 1157
- next-token
 - usedby token-stack-show, 1157
- next1, 330
 - calledby next, 330
 - calls StreamNull, 330
 - calls incAppend, 330
 - calls next, 330
 - defun, 330
- nextInterpreterFrame, 32
 - calledby frameSpad2Cmd, 26
 - calls updateFromCurrentInterpreterFrame, 32
 - uses \$interpreterFrameRing, 33
- defun, 32
- nextline, 404
 - calledby lineoftoks, 403
 - calledby scanEsc, 415
 - calls npNull, 404
 - calls strposl, 404
 - uses \$f, 404
 - uses \$linepos, 404
 - uses \$ln, 404
 - uses \$n, 404
 - uses \$r, 404
 - uses \$sz, 404
 - defun, 404
- nmsort
 - calledby getWorkspaceNames, 795
- nonBlank, 363
 - defun, 363
- npADD, 450
 - calledby npExpress1, 472
 - calls npAdd, 451
 - calls npPop1, 451
 - calls npPush, 451
 - calls npType, 450
 - defun, 450
- npAdd, 451
 - calledby npADD, 451
 - calledby npPrimary2, 450
 - calls npCompMissing, 451
 - calls npDefinitionOrStatement, 451
 - calls npEqKey, 451
 - calls npEqPeek, 451
 - calls npPop1, 451
 - calls npPop2, 451
 - calls npPush, 451
 - calls npRestore, 451
 - calls npState, 451
 - calls npTrap, 451
 - calls npVariable, 451
 - calls pfAdd, 451
 - calls pfNothing, 451
 - defun, 451
- npAmpersand, 496
 - calledby npAmpersandFrom, 495
 - calledby npAtom2, 452
 - calls npEqKey, 496
 - calls npName, 496

- calls npTrap, 496
- defun, 496
- npAmpersandFrom, 495
 - calledby npSynthetic, 491
 - calls npAmpersand, 495
 - calls npFromdom, 495
 - defun, 495
- npAndOr, 474
 - calledby npImport, 473
 - calledby npInline, 466
 - calledby npSuchThat, 469
 - calledby npVoid, 471
 - calledby npWhile, 470
 - calls npEqKey, 474
 - calls npPop1, 474
 - calls npPush, 474
 - calls npTrap, 474
 - defun, 474
- npAngleBared, 479
 - calledby npBracketed, 478
 - calls npEnclosed, 479
 - calls pfHide, 479
 - defun, 479
- npAnyNo, 455
 - calledby npColon, 510
 - calledby npEncAp, 474
 - calledby npTypified, 511
 - defun, 455
- npApplication, 454
 - calledby npFromdom1, 496
 - calledby npFromdom, 495
 - calledby npSCategory, 445
 - calledby npTypedForm, 513
 - calledby npTypified, 511
 - calls npApplication2, 454
 - calls npDotted, 454
 - calls npPop1, 454
 - calls npPop2, 454
 - calls npPrimary, 454
 - calls npPush, 454
 - calls pfApplication, 454
 - defun, 454
- npApplication2, 455
 - calledby npApplication2, 455
 - calledby npApplication, 454
 - calls npApplication2, 455
- calls npDotted, 455
- calls npPop1, 456
- calls npPop2, 456
- calls npPrimary1, 455
- calls npPush, 455
- calls pfApplication, 456
- defun, 455
- npArith, 493
 - calledby npInterval, 492
 - calls npLeftAssoc, 493
 - calls npSum, 493
 - defun, 493
- npAssign, 509
 - calledby npExit, 508
 - calledby npLoop, 467
 - calledby npPileExit, 508
 - calls npAssignment, 509
 - calls npBackTrack, 509
 - calls npMDEF, 509
 - defun, 509
- npAssignment, 509
 - calledby npAssign, 509
 - calls npAssignVariable, 509
 - calls npEqKey, 509
 - calls npGives, 509
 - calls npPop1, 509
 - calls npPop2, 509
 - calls npPush, 509
 - calls npTrap, 509
 - calls pfAssign, 509
 - defun, 509
- npAssignVariable, 510
 - calledby npAssignment, 509
 - calls npColon, 510
 - calls npPop1, 510
 - calls npPush, 510
 - calls pfListOf, 510
 - defun, 510
- npAtom1, 475
 - calledby npPrimary1, 456
 - calls npBDefinition, 475
 - calls npConstTok, 475
 - calls npDollar, 475
 - calls npFromdom, 475
 - calls npName, 475
 - calls npPDefinition, 475

- defun, [475](#)
- npAtom2, [452](#)
 - calledby npPrimary2, [450](#)
 - calls npAmpersand, [452](#)
 - calls npFromdom, [452](#)
 - calls npInfixOperator, [452](#)
 - calls npPrefixColon, [452](#)
 - defun, [452](#)
- npBacksetElse, [489](#)
 - calledby npElse, [489](#)
 - calls npEqKey, [489](#)
 - defun, [489](#)
- npBackTrack, [440](#)
 - calledby npAssign, [509](#)
 - calledby npDefinitionOrStatement, [440](#)
 - calledby npExit, [508](#)
 - calledby npGives, [440](#)
 - calledby npMDEF, [457](#)
 - calls npEqPeek, [440](#)
 - calls npRestore, [440](#)
 - calls npState, [440](#)
 - calls npTrap, [440](#)
 - defun, [440](#)
- npBDefinition, [477](#)
 - calledby npAtom1, [475](#)
 - calledby npEncl, [475](#)
 - calls npBracketed, [477](#)
 - calls npDefinitionlist, [477](#)
 - calls npPDefinition, [477](#)
 - defun, [477](#)
- npboot, [812](#)
 - calledby handleNoParseCommands, [810](#)
 - defun, [812](#)
- npBPileDefinition, [480](#)
 - calledby npPrimary1, [456](#)
 - calls npPileBracketed, [480](#)
 - calls npPileDefinitionlist, [480](#)
 - calls npPop1, [480](#)
 - calls npPush, [480](#)
 - calls pfListOf, [480](#)
 - calls pfSequence, [480](#)
 - defun, [480](#)
- npBraced, [479](#)
 - calledby npBracketed, [478](#)
 - calls npEnclosed, [479](#)
 - calls pfBraceBar, [479](#)
- calls pfBrace, [479](#)
- defun, [479](#)
- npBracketed, [478](#)
 - calledby npBracketed, [478](#)
 - calls npEnclosed, [478](#)
 - calls pfBracketBar, [478](#)
 - calls pfBracket, [478](#)
 - defun, [478](#)
- npBracketed, [478](#)
 - calledby npBDefinition, [477](#)
 - calls npAngleBared, [478](#)
 - calls npBraced, [478](#)
 - calls npBracketed, [478](#)
 - calls npParened, [478](#)
 - defun, [478](#)
- npBreak, [467](#)
 - calledby npStatement, [463](#)
 - calls npEqKey, [467](#)
 - calls npPush, [467](#)
 - calls pfBreak, [467](#)
 - calls pfNothing, [467](#)
 - defun, [467](#)
- npBy, [492](#)
 - calledby npForIn, [470](#)
 - calledby npSynthetic, [491](#)
 - calls npInterval, [492](#)
 - calls npLeftAssoc, [492](#)
 - defun, [492](#)
- npCategory, [445](#)
 - calledby npCategoryL, [445](#)
 - calls npPP, [445](#)
 - calls npSCategory, [445](#)
 - defun, [445](#)
- npCategoryL, [445](#)
 - calledby npSCategory, [445](#)
 - calledby npWith, [442](#)
 - calls npCategory, [445](#)
 - calls npPop1, [445](#)
 - calls npPush, [445](#)
 - calls pfUnSequence, [445](#)
 - defun, [445](#)
- npCoerceTo, [512](#)
 - calledby npTypeStyle, [511](#)
 - calls npTypedForm, [512](#)
 - calls pfCoerceto, [512](#)
 - defun, [512](#)

- npColon, 510
 - calledby npAssignVariable, 510
 - calledby npPower, 495
 - calls npAnyNo, 510
 - calls npTagged, 510
 - calls npTypified, 510
 - defun, 510
- npColonQuery, 512
 - calledby npTypeStyle, 512
 - calls npTypedForm, 512
 - calls pfRetractTo, 512
 - defun, 512
- npComma, 438
 - calledby npQualDef, 437
 - calls npQualifiedDefinition, 438
 - calls npTuple, 438
 - defun, 438
- npCommaBackSet, 438
 - calledby npTuple, 438
 - calls npEqKey, 438
 - defun, 438
- npCompMissing, 443
 - calledby npAdd, 451
 - calledby npForIn, 470
 - calledby npLetQualified, 459
 - calledby npLoop, 467
 - calledby npWith, 443
 - calls npEqKey, 443
 - calls npMissing, 443
 - defun, 443
- npConditional, 488
 - calledby npConditionalStatement, 472
 - calledby npWConditional, 487
 - calls npElse, 488
 - calls npEqKey, 488
 - calls npLogical, 488
 - calls npMissing, 488
 - calls npTrap, 488
 - defun, 488
- npConditionalStatement, 472
 - calledby npExpress1, 472
 - calls npConditional, 472
 - calls npQualifiedDefinition, 472
 - defun, 472
- npConstTok, 476
 - calledby npAtom1, 475
- calls npEqPeek, 476
- calls npNext, 476
- calls npPop1, 477
- calls npPrimary1, 477
- calls npPush, 476
- calls npRestore, 477
- calls npState, 477
- calls pfSymb, 477
- calls tokPosn, 477
- calls tokType, 476
- uses \$stok, 477
- defun, 476
- npDDInfKey, 500
 - calledby npInfGeneric, 500
 - calls npEqKey, 500
 - calls npInfKey, 500
 - calls npPop1, 500
 - calls npPush, 500
 - calls npRestore, 500
 - calls npState, 500
 - calls pfSymb, 500
 - calls tokConstruct, 500
 - calls tokPart, 500
 - calls tokPosn, 500
 - uses \$stok, 500
 - defun, 500
- npDecl, 507
 - calledby npVariableName, 506
 - calls npEqKey, 507
 - calls npPop1, 507
 - calls npPop2, 507
 - calls npPush, 507
 - calls npTrap, 507
 - calls npType, 507
 - calls pfTyped, 507
 - defun, 507
- npDef, 480
 - calledby npDefinitionItem, 460
 - calledby npDefinitionOrStatement, 440
 - calledby npDefn, 479
 - calledby npFix, 458
 - calls npDefTail, 480
 - calls npMatch, 480
 - calls npPop1, 480
 - calls npPush, 480
 - calls npTrap, 480

- calls pfCheckItOut, 480
- calls pfDefinition, 480
- calls pfPushBody, 480
- defun, 480
- npDefaultDecl, 462
 - calledby npDefaultItem, 462
 - calls npEqKey, 462
 - calls npPop1, 462
 - calls npPop2, 462
 - calls npPush, 462
 - calls npTrap, 462
 - calls npType, 462
 - calls pfParts, 462
 - calls pfSpread, 462
 - defun, 462
- npDefaultItem, 462
 - calledby npSDefaultItem, 461
 - calls npDefaultDecl, 462
 - calls npTrap, 462
 - calls npTypeVariable, 462
 - defun, 462
- npDefaultItemList, 461
 - calledby npTyping, 460
 - calls npPC, 461
 - calls npPop1, 461
 - calls npPush, 461
 - calls npSDefaultItem, 461
 - calls pfUnSequence, 461
 - defun, 461
- npDefaultValue, 487
 - calledby npSCategory, 445
 - calls npDefinitionOrStatement, 487
 - calls npEqKey, 487
 - calls npPop1, 487
 - calls npPush, 487
 - calls npTrap, 487
 - calls pfAdd, 487
 - calls pfNothing, 487
 - defun, 487
- npDefinition, 459
 - calledby npLetQualified, 459
 - calledby npQualified, 439
 - calls npDefinitionItem, 459
 - calls npPP, 459
 - calls npPop1, 459
 - calls npPush, 459
- calls pfSequenceToList, 459
- defun, 459
- npDefinitionItem, 460
 - calledby npDefinition, 459
 - calls npDefn, 460
 - calls npDef, 460
 - calls npEqPeek, 460
 - calls npImport, 460
 - calls npMacro, 460
 - calls npRestore, 460
 - calls npStatement, 460
 - calls npState, 460
 - calls npTrap, 460
 - calls npTyping, 460
 - defun, 460
- npDefinitionlist, 485
 - calledby npBDefinition, 477
 - calledby npPDefinition, 476
 - calledby npPileDefinitionlist, 481
 - calls npQualDef, 485
 - calls npSemiListing, 485
 - defun, 485
- npDefinitionOrStatement, 440
 - calledby npAdd, 451
 - calledby npDefTail, 487
 - calledby npDefaultValue, 487
 - calledby npLambda, 441
 - calledby npLet, 459
 - calledby npQualifiedDefinition, 439
 - calls npBackTrack, 440
 - calls npDef, 440
 - calls npGives, 440
 - defun, 440
- npDefn, 479
 - calledby npDefinitionItem, 460
 - calledby npPrimary1, 456
 - calls npDef, 479
 - calls npEqKey, 479
 - calls npPP, 479
 - defun, 479
- npDefTail, 487
 - calledby npDef, 480
 - calledby npMdef, 457
 - calledby npSingleRule, 486
 - calls npDefinitionOrStatement, 487
 - calls npEqKey, 487

- defun, [487](#)
- npDiscrim, [490](#)
 - calledby npDisjand, [490](#)
 - calls npLeftAssoc, [490](#)
 - calls npQuiver, [490](#)
 - defun, [490](#)
- npDisjand, [490](#)
 - calledby npLogical, [489](#)
 - calls npDiscrim, [490](#)
 - calls npLeftAssoc, [490](#)
 - defun, [490](#)
- npDollar, [476](#)
 - calledby npAtom1, [475](#)
 - calls npEqPeek, [476](#)
 - calls npNext, [476](#)
 - calls npPush, [476](#)
 - calls tokConstruct, [476](#)
 - calls tokPosn, [476](#)
 - uses \$stok, [476](#)
 - defun, [476](#)
- npDotted, [454](#)
 - calledby npApplication2, [455](#)
 - calledby npApplication, [454](#)
 - defun, [454](#)
- npElse, [488](#)
 - calledby npConditional, [488](#)
 - calls npBacksetElse, [489](#)
 - calls npPop1, [489](#)
 - calls npPop2, [489](#)
 - calls npPop3, [489](#)
 - calls npPush, [489](#)
 - calls npRestore, [489](#)
 - calls npState, [488](#)
 - calls npTrap, [489](#)
 - calls pIfThenOnly, [489](#)
 - calls pIf, [489](#)
 - defun, [488](#)
- npEncAp, [474](#)
 - calledby npPrimary1, [456](#)
 - calledby npPrimary2, [450](#)
 - calls npAnyNo, [474](#)
 - calls npEncl, [474](#)
 - calls npFromdom, [474](#)
 - defun, [474](#)
- npEncl, [475](#)
 - calledby npEncAp, [474](#)
- calls npBDefinition, [475](#)
- calls npPop1, [475](#)
- calls npPop2, [475](#)
- calls npPush, [475](#)
- calls pfApplication, [475](#)
- defun, [475](#)
- npEnclosed, [504](#)
 - calledby npAngleBared, [479](#)
 - calledby npBraced, [479](#)
 - calledby npBracked, [478](#)
 - calledby npParened, [478](#)
 - calls npEqKey, [504](#)
 - calls npMissingMate, [504](#)
 - calls npPop1, [504](#)
 - calls npPush, [504](#)
 - calls pfEnSequence, [504](#)
 - calls pfListOf, [504](#)
 - calls pfTuple, [504](#)
 - uses \$stok, [504](#)
 - defun, [504](#)
- npEqKey, [437](#)
 - calledby npAdd, [451](#)
 - calledby npAmpersand, [496](#)
 - calledby npAndOr, [474](#)
 - calledby npAssignment, [509](#)
 - calledby npBacksetElse, [489](#)
 - calledby npBreak, [467](#)
 - calledby npCommaBackSet, [438](#)
 - calledby npCompMissing, [443](#)
 - calledby npConditional, [488](#)
 - calledby npDDInfKey, [500](#)
 - calledby npDecl, [507](#)
 - calledby npDefTail, [487](#)
 - calledby npDefaultDecl, [462](#)
 - calledby npDefaultValue, [487](#)
 - calledby npDefn, [479](#)
 - calledby npEnclosed, [504](#)
 - calledby npExport, [463](#)
 - calledby npFix, [458](#)
 - calledby npForIn, [470](#)
 - calledby npFree, [466](#)
 - calledby npFromdom1, [496](#)
 - calledby npFromdom, [495](#)
 - calledby npInfGeneric, [500](#)
 - calledby npInfixOperator, [452](#)
 - calledby npItem1, [434](#)

- calledby npItem, 434
- calledby npIterate, 467
- calledby npLambda, 441
- calledby npLetQualified, 459
- calledby npListAndRecover, 482
- calledby npList, 447
- calledby npLocalDecl, 465
- calledby npLocal, 465
- calledby npLoop, 467
- calledby npMoveTo, 483
- calledby npParenthesize, 507
- calledby npPileBracketed, 481
- calledby npPileExit, 509
- calledby npQualified, 439
- calledby npReturn, 470, 471
- calledby npRule, 486
- calledby npSelector, 455
- calledby npSemiBackSet, 486
- calledby npSigDecl, 449
- calledby npSymbolVariable, 498
- calledby npTypedForm1, 511
- calledby npTypedForm, 513
- calledby npTyping, 460
- calledby npWith, 442
- calls npNext, 437
- uses \$stok, 437
- uses \$ttok, 437
- defun, 437
- npEqPeek, 444
 - calledby npAdd, 451
 - calledby npBackTrack, 440
 - calledby npConstTok, 476
 - calledby npDefinitionItem, 460
 - calledby npDollar, 476
 - calledby npInterval, 492
 - calledby npListAndRecover, 482
 - calledby npMoveTo, 483
 - calledby npPrefixColon, 453
 - calledby npSCategory, 445
 - calledby npSegment, 493
 - calledby npWith, 442
 - uses \$stok, 444
 - uses \$ttok, 444
 - defun, 444
- npExit, 508
 - calledby npGives, 440
 - calls npAssign, 508
 - calls npBackTrack, 508
 - calls npPileExit, 508
 - defun, 508
- npExport, 463
 - calledby npStatement, 463
 - calls npEqKey, 463
 - calls npLocalItemlist, 463
 - calls npPop1, 463
 - calls npPush, 463
 - calls npTrap, 463
 - calls pfExport, 463
 - defun, 463
- npExpress, 471
 - calledby npReturn, 471
 - calledby npStatement, 463
 - calls npExpress1, 471
 - calls npIterators, 472
 - calls npPop1, 472
 - calls npPop2, 472
 - calls npPush, 472
 - calls pfCollect, 472
 - calls pfListOf, 472
 - defun, 471
- npExpress1, 472
 - calledby npExpress, 471
 - calls npADD, 472
 - calls npConditionalStatement, 472
 - defun, 472
- npFirstTok, 435
 - calledby npNext, 438
 - calledby npParse, 433
 - calledby npRecoverTrap, 483
 - calledby npRestore, 444
 - calls tokConstruct, 435
 - calls tokPart, 435
 - calls tokPosn, 435
 - uses \$inputStream, 435
 - uses \$stok, 435
 - uses \$ttok, 435
 - defun, 435
- npFix, 458
 - calledby npPrimary1, 456
 - calls npDef, 458
 - calls npEqKey, 458
 - calls npPop1, 458

- calls npPush, 458
- calls pfFix, 458
- defun, 458
- npForIn, 470
 - calledby npIterators, 468
 - calledby npIterator, 468
 - calls npBy, 470
 - calls npCompMissing, 470
 - calls npEqKey, 470
 - calls npPop1, 470
 - calls npPop2, 470
 - calls npPush, 470
 - calls npTrap, 470
 - calls npVariable, 470
 - calls pfForin, 470
 - defun, 470
- npFree, 466
 - calledby npStatement, 463
 - calls npEqKey, 466
 - calls npLocalItemList, 466
 - calls npPop1, 466
 - calls npPush, 466
 - calls npTrap, 466
 - calls pfFree, 466
 - defun, 466
- npFromdom, 495
 - calledby npAmpersandFrom, 495
 - calledby npAtom1, 475
 - calledby npAtom2, 452
 - calledby npEncAp, 474
 - calledby npSegment, 493
 - calls npApplication, 495
 - calls npEqKey, 495
 - calls npFromdom1, 495
 - calls npPop1, 495
 - calls npPush, 495
 - calls npTrap, 495
 - calls pfFromDom, 495
 - defun, 495
- npFromdom1, 496
 - calledby npFromdom1, 496
 - calledby npFromdom, 495
 - calls npApplication, 496
 - calls npEqKey, 496
 - calls npFromdom1, 496
 - calls npPop1, 496
- calls npPush, 496
- calls npTrap, 496
- calls pfFromDom, 496
- defun, 496
- npGives, 440
 - calledby npAssignment, 509
 - calledby npDefinitionOrStatement, 440
 - calls npBackTrack, 440
 - calls npExit, 440
 - calls npLambda, 440
 - defun, 440
- npId, 497
 - calledby npName, 496
 - calledby npSymbolVariable, 498
 - calls npNext, 497
 - calls npPush, 497
 - calls tokConstruct, 497
 - calls tokPosn, 497
 - uses \$npTokToNames, 497
 - uses \$stok, 497
 - uses \$ttok, 497
 - defun, 497
- npImport, 473
 - calledby npDefinitionItem, 460
 - calledby npStatement, 463
 - calls npAndOr, 473
 - calls npQualTypelist, 473
 - calls pfImport, 473
 - defun, 473
- npInfGeneric, 500
 - calledby npLeftAssoc, 499
 - calledby npRightAssoc, 498
 - calledby npTerm, 493
 - calls npDDInfKey, 500
 - calls npEqKey, 500
 - defun, 500
- npInfixOp, 453
 - calledby npInfixOperator, 452
 - calls npPushId, 453
 - uses \$stok, 453
 - uses \$ttok, 453
 - defun, 453
- npInfixOperator, 452
 - calledby npAtom2, 452
 - calledby npSignatureDefinee, 449
 - calls npEqKey, 452

- calls npInfixOp, 452
- calls npPop1, 452
- calls npPush, 452
- calls npRestore, 452
- calls npState, 452
- calls pfSymb, 452
- calls tokConstruct, 452
- calls tokPart, 452
- calls tokPosn, 452
- uses \$stok, 452
- defun, 452
- npInfKey, 501
 - calledby npDDInfKey, 500
 - calls npPushId, 501
 - uses \$stok, 501
 - uses \$ttok, 501
 - defun, 501
- npInline, 466
 - calledby npStatement, 463
 - calls npAndOr, 466
 - calls npQualTypelist, 466
 - calls pfInline, 466
 - defun, 466
- npInterval, 492
 - calledby npBy, 492
 - calls npArith, 492
 - calls npEqPeek, 492
 - calls npPop1, 492
 - calls npPop2, 492
 - calls npPush, 492
 - calls npSegment, 492
 - calls pfApplication, 492
 - calls pfInfApplication, 492
 - defun, 492
- npItem, 434
 - calledby npParse, 433
 - calls npEqKey, 434
 - calls npItem1, 434
 - calls npPop1, 434
 - calls npPush, 434
 - calls npQualDef, 434
 - calls pfEnSequence, 434
 - calls pfNoValue, 434
 - defun, 434
- npItem1, 434
 - calledby npItem1, 435
 - calledby npItem, 434
 - calls npEqKey, 434
 - calls npItem1, 435
 - calls npPop1, 435
 - calls npQualDef, 434
 - defun, 434
- npIterate, 467
 - calledby npStatement, 463
 - calls npEqKey, 467
 - calls npPush, 467
 - calls pfIterate, 467
 - calls pfNothing, 467
 - defun, 467
- npIterator, 468
 - calledby npIterators, 468
 - calls npForIn, 468
 - calls npSuchThat, 468
 - calls npWhile, 469
 - defun, 468
- npIterators, 468
 - calledby npExpress, 472
 - calledby npIterators, 468
 - calledby npLoop, 467
 - calls npForIn, 468
 - calls npIterators, 468
 - calls npIterator, 468
 - calls npPop1, 468
 - calls npPop2, 468
 - calls npPush, 468
 - calls npWhile, 468
 - calls npZeroOrMore, 468
 - defun, 468
- npLambda, 441
 - calledby npGives, 440
 - calledby npLambda, 441
 - calls npDefinitionOrStatement, 441
 - calls npEqKey, 441
 - calls npLambda, 441
 - calls npPop1, 441
 - calls npPop2, 441
 - calls npPush, 441
 - calls npTrap, 441
 - calls npType, 441
 - calls npVariable, 441
 - calls pfLam, 441
 - calls pfReturnTyped, 441

- defun, [441](#)
- npLeftAssoc, [499](#)
 - calledby npArith, [493](#)
 - calledby npBy, [492](#)
 - calledby npDiscrim, [490](#)
 - calledby npDisjand, [490](#)
 - calledby npLogical, [489](#)
 - calledby npMatch, [442](#)
 - calledby npProduct, [494](#)
 - calledby npRelation, [491](#)
 - calledby npRemainder, [494](#)
 - calledby npSuch, [442](#)
 - calledby npSum, [493](#)
 - calls npInfGeneric, [499](#)
 - calls npPop1, [499](#)
 - calls npPop2, [499](#)
 - calls npPush, [499](#)
 - calls pfApplication, [499](#)
 - calls pfInfApplication, [499](#)
 - defun, [499](#)
- npLet, [458](#)
 - calledby npPrimary1, [456](#)
 - calls npDefinitionOrStatement, [459](#)
 - calls npLetQualified, [458](#)
 - defun, [458](#)
- npLetQualified, [459](#)
 - calledby npLet, [458](#)
 - calledby npQualified, [439](#)
 - calls npCompMissing, [459](#)
 - calls npDefinition, [459](#)
 - calls npEqKey, [459](#)
 - calls npPop1, [459](#)
 - calls npPop2, [459](#)
 - calls npPush, [459](#)
 - calls npTrap, [459](#)
 - calls pfWhere, [459](#)
 - defun, [459](#)
- npLisp, [812](#)
 - calledby handleNoParseCommands, [810](#)
 - calledby intnplisp, [327](#)
 - uses \$ans, [812](#)
 - defun, [812](#)
- npList, [447](#)
 - calledby npListing, [447](#)
 - calls npEqKey, [447](#)
 - calls npPop1, [447](#)
 - calls npPop2, [447](#)
 - calls npPop3, [447](#)
 - calls npPush, [447](#)
 - calls npTrap, [447](#)
 - uses \$stack, [447](#)
 - defun, [447](#)
- npListAndRecover, [482](#)
 - calledby npPPg, [503](#)
 - calledby npPileDefinitionlist, [481](#)
 - calls npEqKey, [482](#)
 - calls npEqPeek, [482](#)
 - calls npNext, [482](#)
 - calls npPop1, [482](#)
 - calls npPush, [482](#)
 - calls npRecoverTrap, [482](#)
 - calls syGeneralErrorHere, [482](#)
 - uses \$inputStream, [482](#)
 - uses \$stack, [482](#)
 - catches, [482](#)
 - defun, [482](#)
- npListing, [447](#)
 - calledby npSDefaultItem, [461](#)
 - calledby npSLocalItem, [464](#)
 - calledby npSQualTypelist, [473](#)
 - calledby npSigItemlist, [446](#)
 - calledby npTypeVariablelist, [449](#)
 - calledby npVariablelist, [506](#)
 - calls npList, [447](#)
 - calls pfListOf, [447](#)
 - defun, [447](#)
- npListofFun, [513](#)
 - calledby npSemiListing, [485](#)
 - calledby npTuple, [438](#)
 - calls npPop1, [513](#)
 - calls npPop2, [513](#)
 - calls npPop3, [513](#)
 - calls npPush, [513](#)
 - calls npTrap, [513](#)
 - uses \$stack, [513](#)
 - defun, [513](#)
- npLocal, [465](#)
 - calledby npStatement, [463](#)
 - calls npEqKey, [465](#)
 - calls npLocalItemlist, [465](#)
 - calls npPop1, [466](#)
 - calls npPush, [466](#)

- calls npTrap, 465
- calls pfLocal, 466
- defun, 465
- npLocalDecl, 465
 - calledby npLocalItem, 465
 - calls npEqKey, 465
 - calls npPop1, 465
 - calls npPop2, 465
 - calls npPush, 465
 - calls npTrap, 465
 - calls npType, 465
 - calls pfNothing, 465
 - calls pfParts, 465
 - calls pfSpread, 465
 - defun, 465
- npLocalItem, 465
 - calledby npSLocalItem, 464
 - calls npLocalDecl, 465
 - calls npTypeVariable, 465
 - defun, 465
- npLocalItemList, 464
 - calledby npExport, 463
 - calledby npFree, 466
 - calledby npLocal, 465
 - calls npPC, 464
 - calls npPop1, 464
 - calls npPush, 464
 - calls npSLocalItem, 464
 - calls pfUnSequence, 464
 - defun, 464
- npLogical, 489
 - calledby npConditional, 488
 - calledby npSuchThat, 469
 - calledby npSuch, 442
 - calledby npWhile, 470
 - calls npDisjand, 489
 - calls npLeftAssoc, 489
 - defun, 489
- npLoop, 467
 - calledby npStatement, 463
 - calls npAssign, 467
 - calls npCompMissing, 467
 - calls npEqKey, 467
 - calls npIterators, 467
 - calls npPop1, 467
 - calls npPop2, 467
 - calls npPush, 467
 - calls npTrap, 467
 - calls pfLoop1, 467
 - calls pfLp, 467
 - defun, 467
- npMacro, 456
 - calledby npDefinitionItem, 460
 - calledby npPrimary1, 456
 - calls npMdef, 457
 - calls npPP, 456
 - defun, 456
- npMatch, 442
 - calledby npDef, 480
 - calledby npType, 441
 - calls npLeftAssoc, 442
 - calls npSuch, 442
 - defun, 442
- npMDEF, 457
 - calledby npAssign, 509
 - calls npBackTrack, 457
 - calls npMDEFinition, 458
 - calls npStatement, 458
 - defun, 457
- npMdef, 457
 - calledby npMDEFinition, 458
 - calledby npMacro, 457
 - calls npDefTail, 457
 - calls npPop1, 457
 - calls npPush, 457
 - calls npQuiver, 457
 - calls npTrap, 457
 - calls pfCheckMacroOut, 457
 - calls pfMacro, 457
 - calls pfPushMacroBody, 457
 - defun, 457
- npMDEFinition, 458
 - calledby npMDEF, 458
 - calls npMdef, 458
 - calls npPP, 458
 - defun, 458
- npMissing, 443
 - calledby npCompMissing, 443
 - calledby npConditional, 488
 - calledby npMissingMate, 508
 - calledby npPileBracketed, 481
 - calls ncSoftError, 444

- calls pname, [444](#)
 - calls tokPosn, [444](#)
 - uses \$stok, [444](#)
 - defun, [443](#)
 - throws, [443](#)
- npMissingMate, [508](#)
 - calledby npEnclosed, [504](#)
 - calledby npParenthesize, [507](#)
 - calls ncSoftError, [508](#)
 - calls npMissing, [508](#)
 - calls tokPosn, [508](#)
 - defun, [508](#)
- npMoveTo, [483](#)
 - calledby npMoveTo, [483](#)
 - calledby npRecoverTrap, [483](#)
 - calls npEqKey, [483](#)
 - calls npEqPeek, [483](#)
 - calls npMoveTo, [483](#)
 - calls npNext, [483](#)
 - uses \$inputStream, [483](#)
 - defun, [483](#)
- npName, [496](#)
 - calledby npAmpersand, [496](#)
 - calledby npAtom1, [475](#)
 - calledby npReturn, [471](#)
 - calledby npSignatureDefinee, [449](#)
 - calledby npVariableName, [506](#)
 - calls npId, [496](#)
 - calls npSymbolVariable, [497](#)
 - defun, [496](#)
- npNext, [437](#)
 - calledby npConstTok, [476](#)
 - calledby npDollar, [476](#)
 - calledby npEqKey, [437](#)
 - calledby npId, [497](#)
 - calledby npListAndRecover, [482](#)
 - calledby npMoveTo, [483](#)
 - calledby npPrefixColon, [454](#)
 - calledby npPushId, [501](#)
 - calls npFirstTok, [438](#)
 - uses \$inputStream, [438](#)
 - defun, [437](#)
- npNull, [623](#)
 - calledby eqpileTree, [629](#)
 - calledby insertpile, [625](#)
 - calledby nextline, [404](#)
 - calledby pileForests, [628](#)
 - calledby pilePlusComments, [626](#)
 - calledby pileTree, [627](#)
 - calls StreamNull, [623](#)
 - defun, [623](#)
- npParened, [478](#)
 - calledby npBracketed, [478](#)
 - calledby npPP, [502](#)
 - calls npEnclosed, [478](#)
 - calls pfParen, [478](#)
 - defun, [478](#)
- npParenthesize, [507](#)
 - calledby npParenthesized, [507](#)
 - calls npEqKey, [507](#)
 - calls npMissingMate, [507](#)
 - calls npPush, [507](#)
 - uses \$stok, [507](#)
 - defun, [507](#)
- npParenthesized, [507](#)
 - calledby npPDefinition, [476](#)
 - calledby npTypeVariable, [448](#)
 - calledby npVariable, [505](#)
 - calls npParenthesize, [507](#)
 - defun, [507](#)
- npParse, [433](#)
 - calledby intloopEchoParse, [361](#)
 - calledby ncloopParse, [330](#)
 - calls ncSoftError, [433](#)
 - calls npFirstTok, [433](#)
 - calls npItem, [433](#)
 - calls pfDocument, [433](#)
 - calls pfListOf, [433](#)
 - calls pfWrong, [433](#)
 - calls tokPosn, [433](#)
 - uses \$inputStream, [433](#)
 - uses \$stack, [433](#)
 - uses \$stok, [433](#)
 - uses \$ttok, [433](#)
 - catches, [433](#)
 - defun, [433](#)
- npPC
 - calledby npDefaultItemList, [461](#)
 - calledby npLocalItemList, [464](#)
 - calledby npQualTypelist, [473](#)
- npPDefinition, [476](#)
 - calledby npAtom1, [475](#)

- calledby npBDefinition, 477
- calls npDefinitionlist, 476
- calls npParenthesized, 476
- calls npPop1, 476
- calls npPush, 476
- calls pfEnSequence, 476
- defun, 476
- npPileBracketed, 481
 - calledby npBPileDefinition, 480
 - calledby npPP, 502
 - calls npEqKey, 481
 - calls npMissing, 481
 - calls npPop1, 481
 - calls npPush, 481
 - calls pfNothing, 481
 - calls pfPile, 481
 - defun, 481
- npPileDefinitionlist, 481
 - calledby npBPileDefinition, 480
 - calls npDefinitionlist, 481
 - calls npListAndRecover, 481
 - calls npPop1, 481
 - calls npPush, 481
 - calls pfAppend, 481
 - defun, 481
- npPileExit, 508
 - calledby npExit, 508
 - calls npAssign, 508
 - calls npEqKey, 509
 - calls npPop1, 509
 - calls npPop2, 509
 - calls npPush, 509
 - calls npStatement, 509
 - calls pfExit, 509
 - defun, 508
- npPop1, 436
 - calledby npADD, 451
 - calledby npAdd, 451
 - calledby npAndOr, 474
 - calledby npApplication2, 456
 - calledby npApplication, 454
 - calledby npAssignVariable, 510
 - calledby npAssignment, 509
 - calledby npBPileDefinition, 480
 - calledby npCategoryL, 445
 - calledby npConstTok, 477
 - calledby npDDInfKey, 500
 - calledby npDecl, 507
 - calledby npDefaultDecl, 462
 - calledby npDefaultItemlist, 461
 - calledby npDefaultValue, 487
 - calledby npDefinition, 459
 - calledby npDef, 480
 - calledby npElse, 489
 - calledby npEnclosed, 504
 - calledby npEncl, 475
 - calledby npExport, 463
 - calledby npExpress, 472
 - calledby npFix, 458
 - calledby npForIn, 470
 - calledby npFree, 466
 - calledby npFromdom1, 496
 - calledby npFromdom, 495
 - calledby npInfixOperator, 452
 - calledby npInterval, 492
 - calledby npItem1, 435
 - calledby npItem, 434
 - calledby npIterators, 468
 - calledby npLambda, 441
 - calledby npLeftAssoc, 499
 - calledby npLetQualified, 459
 - calledby npListAndRecover, 482
 - calledby npListofFun, 513
 - calledby npList, 447
 - calledby npLocalDecl, 465
 - calledby npLocalItemlist, 464
 - calledby npLocal, 466
 - calledby npLoop, 467
 - calledby npMdef, 457
 - calledby npPDefinition, 476
 - calledby npPPPff, 503
 - calledby npPPg, 503
 - calledby npPP, 502
 - calledby npPileBracketed, 481
 - calledby npPileDefinitionlist, 481
 - calledby npPileExit, 509
 - calledby npQualDef, 437
 - calledby npQualTypelist, 473
 - calledby npQualType, 474
 - calledby npQualified, 439
 - calledby npReturn, 471
 - calledby npRightAssoc, 498

- calledby npSCategory, [445](#)
- calledby npSDefaultItem, [461](#)
- calledby npSLocalItem, [464](#)
- calledby npSQualTypelist, [473](#)
- calledby npSelector, [455](#)
- calledby npSigDecl, [449](#)
- calledby npSigItemList, [447](#)
- calledby npSignature, [446](#)
- calledby npSingleRule, [486](#)
- calledby npSymbolVariable, [498](#)
- calledby npSynthetic, [491](#)
- calledby npTerm, [494](#)
- calledby npTypeVariable, [448](#)
- calledby npTypedForm1, [511](#)
- calledby npTypedForm, [513](#)
- calledby npType, [441](#)
- calledby npTyping, [461](#)
- calledby npVariableName, [506](#)
- calledby npVariable, [506](#)
- calledby npWConditional, [488](#)
- calledby npWith, [443](#)
- calledby npZeroOrMore, [469](#)
- uses \$stack, [436](#)
- defun, [436](#)
- npPop2, [436](#)
 - calledby npAdd, [451](#)
 - calledby npApplication2, [456](#)
 - calledby npApplication, [454](#)
 - calledby npAssignment, [509](#)
 - calledby npDecl, [507](#)
 - calledby npDefaultDecl, [462](#)
 - calledby npElse, [489](#)
 - calledby npEncl, [475](#)
 - calledby npExpress, [472](#)
 - calledby npForIn, [470](#)
 - calledby npInterval, [492](#)
 - calledby npIterators, [468](#)
 - calledby npLambda, [441](#)
 - calledby npLeftAssoc, [499](#)
 - calledby npLetQualified, [459](#)
 - calledby npListofFun, [513](#)
 - calledby npList, [447](#)
 - calledby npLocalDecl, [465](#)
 - calledby npLoop, [467](#)
 - calledby npPileExit, [509](#)
 - calledby npReturn, [471](#)
 - calledby npRightAssoc, [498](#)
 - calledby npSelector, [455](#)
 - calledby npSigDecl, [449](#)
 - calledby npSingleRule, [486](#)
 - calledby npSynthetic, [491](#)
 - calledby npTerm, [494](#)
 - calledby npTypedForm1, [511](#)
 - calledby npTypedForm, [513](#)
 - calledby npWith, [443](#)
 - calledby npZeroOrMore, [469](#)
 - uses \$stack, [436](#)
 - defun, [436](#)
- npPop3, [436](#)
 - calledby npElse, [489](#)
 - calledby npListofFun, [513](#)
 - calledby npList, [447](#)
 - uses \$stack, [436](#)
 - defun, [436](#)
- npPower, [495](#)
 - calledby npProduct, [494](#)
 - calls npColon, [495](#)
 - calls npRightAssoc, [495](#)
 - defun, [495](#)
- npPP, [502](#)
 - calledby npCategory, [445](#)
 - calledby npDefinition, [459](#)
 - calledby npDefn, [479](#)
 - calledby npMDEFinition, [458](#)
 - calledby npMacro, [456](#)
 - calledby npRule, [486](#)
 - calls npPPf, [502](#)
 - calls npPPg, [502](#)
 - calls npParened, [502](#)
 - calls npPileBracketed, [502](#)
 - calls npPop1, [502](#)
 - calls npPush, [502](#)
 - calls pfEnSequence, [502](#)
 - uses npPParg, [502](#)
 - defun, [502](#)
- npPParg, [502](#)
 - usedby npPP, [502](#)
 - defvar, [502](#)
- npPPf, [503](#)
 - calledby npPPg, [503](#)
 - calledby npPP, [502](#)
 - calls npPPff, [503](#)

- calls npSemiListing, 503
- defun, 503
- npPPff, 503
 - calledby npPPf, 503
 - calls npPop1, 503
 - calls npPush, 503
 - uses \$npPParg, 503
 - defun, 503
- npPPg, 503
 - calledby npPP, 502
 - calls npListAndRecover, 503
 - calls npPPf, 503
 - calls npPop1, 503
 - calls npPush, 503
 - calls pfAppend, 503
 - defun, 503
- npPrefixColon, 453
 - calledby npAtom2, 452
 - calledby npSignatureDefinee, 449
 - calls npEqPeek, 453
 - calls npNext, 454
 - calls npPush, 453
 - calls tokConstruct, 453
 - calls tokPosn, 454
 - uses \$stok, 454
 - defun, 453
- npPretend, 512
 - calledby npTypeStyle, 512
 - calls npTypedForm, 512
 - calls pfPretend, 512
 - defun, 512
- npPrimary, 450
 - calledby npApplication, 454
 - calledby npSCategory, 445
 - calledby npSelector, 455
 - calls npPrimary1, 450
 - calls npPrimary2, 450
 - defun, 450
- npPrimary1, 456
 - calledby npApplication2, 455
 - calledby npConstTok, 477
 - calledby npPrimary, 450
 - calls npAtom1, 456
 - calls npBPileDefinition, 456
 - calls npDefn, 456
 - calls npEncAp, 456
 - calls npFix, 456
 - calls npLet, 456
 - calls npMacro, 456
 - calls npRule, 456
 - defun, 456
- npPrimary2, 450
 - calledby npPrimary, 450
 - calls npAdd, 450
 - calls npAtom2, 450
 - calls npEncAp, 450
 - calls npWith, 450
 - calls pfNothing, 450
 - defun, 450
- npProcessSynonym, 813
 - calledby npsynonym, 813
 - calls printSynonyms, 813
 - calls processSynonymLine, 813
 - calls putalist, 813
 - calls terminateSystemCommand, 813
 - uses \$CommandSynonymAlist, 813
 - defun, 813
- npProduct, 494
 - calledby npRemainder, 494
 - calls npLeftAssoc, 494
 - calls npPower, 494
 - defun, 494
- npPush, 435
 - calledby npADD, 451
 - calledby npAdd, 451
 - calledby npAndOr, 474
 - calledby npApplication2, 455
 - calledby npApplication, 454
 - calledby npAssignVariable, 510
 - calledby npAssignment, 509
 - calledby npBPileDefinition, 480
 - calledby npBreak, 467
 - calledby npCategoryL, 445
 - calledby npConstTok, 476
 - calledby npDDInfKey, 500
 - calledby npDecl, 507
 - calledby npDefaultDecl, 462
 - calledby npDefaultItemlist, 461
 - calledby npDefaultValue, 487
 - calledby npDefinition, 459
 - calledby npDef, 480
 - calledby npDollar, 476

- calledby npElse, [489](#)
- calledby npEnclosed, [504](#)
- calledby npEncl, [475](#)
- calledby npExport, [463](#)
- calledby npExpress, [472](#)
- calledby npFix, [458](#)
- calledby npForIn, [470](#)
- calledby npFree, [466](#)
- calledby npFromdom1, [496](#)
- calledby npFromdom, [495](#)
- calledby npId, [497](#)
- calledby npInfixOperator, [452](#)
- calledby npInterval, [492](#)
- calledby npItem, [434](#)
- calledby npIterate, [467](#)
- calledby npIterators, [468](#)
- calledby npLambda, [441](#)
- calledby npLeftAssoc, [499](#)
- calledby npLetQualified, [459](#)
- calledby npListAndRecover, [482](#)
- calledby npListofFun, [513](#)
- calledby npList, [447](#)
- calledby npLocalDecl, [465](#)
- calledby npLocalItemlist, [464](#)
- calledby npLocal, [466](#)
- calledby npLoop, [467](#)
- calledby npMdef, [457](#)
- calledby npPDefinition, [476](#)
- calledby npPPff, [503](#)
- calledby npPPg, [503](#)
- calledby npPP, [502](#)
- calledby npParenthesize, [507](#)
- calledby npPileBracketed, [481](#)
- calledby npPileDefinitionlist, [481](#)
- calledby npPileExit, [509](#)
- calledby npPrefixColon, [453](#)
- calledby npQualDef, [437](#)
- calledby npQualTypelist, [473](#)
- calledby npQualType, [474](#)
- calledby npQualified, [439](#)
- calledby npRecoverTrap, [483](#)
- calledby npReturn, [471](#)
- calledby npRightAssoc, [498](#)
- calledby npSCategory, [445](#)
- calledby npSDefaultItem, [461](#)
- calledby npSLocalItem, [464](#)
- calledby npSQualTypelist, [473](#)
- calledby npSelector, [455](#)
- calledby npSigDecl, [449](#)
- calledby npSigItemlist, [446](#)
- calledby npSignature, [446](#)
- calledby npSingleRule, [486](#)
- calledby npSymbolVariable, [498](#)
- calledby npSynthetic, [491](#)
- calledby npTerm, [494](#)
- calledby npTypeVariable, [448](#)
- calledby npTypedForm1, [511](#)
- calledby npTypedForm, [513](#)
- calledby npType, [441](#)
- calledby npTyping, [461](#)
- calledby npVariableName, [506](#)
- calledby npVariable, [506](#)
- calledby npWConditional, [488](#)
- calledby npWith, [443](#)
- calledby npZeroOrMore, [469](#)
- uses \$stack, [435](#)
- defun, [435](#)
- npPushId, [501](#)
 - calledby npInfKey, [501](#)
 - calledby npInfixOp, [453](#)
 - calledby npSegment, [493](#)
 - calls npNext, [501](#)
 - calls tokConstruct, [501](#)
 - calls tokPosn, [501](#)
 - uses \$stack, [501](#)
 - uses \$stok, [501](#)
 - uses \$ttok, [501](#)
 - defun, [501](#)
- npQualDef, [437](#)
 - calledby npDefinitionlist, [485](#)
 - calledby npItem1, [434](#)
 - calledby npItem, [434](#)
 - calls npComma, [437](#)
 - calls npPop1, [437](#)
 - calls npPush, [437](#)
 - defun, [437](#)
- npQualified, [439](#)
 - calledby npQualifiedDefinition, [439](#)
 - calls npDefinition, [439](#)
 - calls npEqKey, [439](#)
 - calls npLetQualified, [439](#)
 - calls npPop1, [439](#)

- calls npPush, 439
- calls npTrap, 439
- calls pfWhere, 439
- defun, 439
- npQualifiedDefinition, 439
 - calledby npComma, 438
 - calledby npConditionalStatement, 472
 - calls npDefinitionOrStatement, 439
 - calls npQualified, 439
 - defun, 439
- npQualType, 474
 - calledby npSQualTypelist, 473
 - calls npPop1, 474
 - calls npPush, 474
 - calls npType, 474
 - calls pfNothing, 474
 - calls pfQualType, 474
 - defun, 474
- npQualTypelist, 473
 - calledby npImport, 473
 - calledby npInline, 466
 - calls npPC, 473
 - calls npPop1, 473
 - calls npPush, 473
 - calls npSQualTypelist, 473
 - calls pfUnSequence, 473
 - defun, 473
- npQuiver, 490
 - calledby npDiscrim, 490
 - calledby npMdef, 457
 - calledby npSingleRule, 486
 - calls npRelation, 490
 - calls npRightAssoc, 490
 - defun, 490
- npRecoverTrap, 483
 - calledby npListAndRecover, 482
 - calls npFirstTok, 483
 - calls npMoveTo, 483
 - calls npPush, 483
 - calls pfDocument, 483
 - calls pfListOf, 483
 - calls pfWrong, 483
 - calls syIgnoredFromTo, 483
 - calls tokPosn, 483
 - uses \$stok, 483
 - defun, 483
- npRelation, 491
 - calledby npQuiver, 490
 - calls npLeftAssoc, 491
 - calls npSynthetic, 491
 - defun, 491
- npRemainder, 494
 - calledby npTerm, 493
 - calls npLeftAssoc, 494
 - calls npProduct, 494
 - defun, 494
- npRestore, 444
 - calledby npAdd, 451
 - calledby npBackTrack, 440
 - calledby npConstTok, 477
 - calledby npDDInfKey, 500
 - calledby npDefinitionItem, 460
 - calledby npElse, 489
 - calledby npInfixOperator, 452
 - calledby npRightAssoc, 498
 - calledby npSCategory, 445
 - calledby npSymbolVariable, 498
 - calledby npWith, 443
 - calls npFirstTok, 444
 - uses \$inputStream, 444
 - uses \$stack, 444
 - defun, 444
- npRestrict, 513
 - calledby npTypeStyle, 511
 - calls npTypedForm, 513
 - calls pfRestrict, 513
 - defun, 513
- npReturn, 470
 - calledby npStatement, 463
 - calls npEqKey, 470, 471
 - calls npExpress, 471
 - calls npName, 471
 - calls npPop1, 471
 - calls npPop2, 471
 - calls npPush, 471
 - calls npTrap, 471
 - calls pfNothing, 471
 - calls pfReturnNoName, 471
 - calls pfReturn, 471
 - defun, 470
- npRightAssoc, 498
 - calledby npPower, 495

- calledby npQuiver, [490](#)
- calledby npRightAssoc, [498](#)
- calls npInfGeneric, [498](#)
- calls npPop1, [498](#)
- calls npPop2, [498](#)
- calls npPush, [498](#)
- calls npRestore, [498](#)
- calls npRightAssoc, [498](#)
- calls npState, [498](#)
- calls pfApplication, [498](#)
- calls pfInfApplication, [498](#)
- defun, [498](#)
- npRule, [486](#)
 - calledby npPrimary1, [456](#)
 - calls npEqKey, [486](#)
 - calls npPP, [486](#)
 - calls npSingleRule, [486](#)
 - defun, [486](#)
- npSCategory, [445](#)
 - calledby npCategory, [445](#)
 - calls npApplication, [445](#)
 - calls npCategoryL, [445](#)
 - calls npDefaultValue, [445](#)
 - calls npEqPeek, [445](#)
 - calls npPop1, [445](#)
 - calls npPrimary, [445](#)
 - calls npPush, [445](#)
 - calls npRestore, [445](#)
 - calls npSignature, [445](#)
 - calls npState, [445](#)
 - calls npTrap, [446](#)
 - calls npWConditional, [445](#)
 - calls pfAttribute, [445](#)
 - defun, [445](#)
- npSDefaultItem, [461](#)
 - calledby npDefaultItemList, [461](#)
 - calls npDefaultItem, [461](#)
 - calls npListing, [461](#)
 - calls npPop1, [461](#)
 - calls npPush, [461](#)
 - calls pfAppend, [461](#)
 - calls pfParts, [461](#)
 - defun, [461](#)
- npSegment, [493](#)
 - calledby npInterval, [492](#)
 - calls npEqPeek, [493](#)
 - calls npFromdom, [493](#)
 - calls npPushId, [493](#)
 - defun, [493](#)
- npSelector, [455](#)
 - calls npEqKey, [455](#)
 - calls npPop1, [455](#)
 - calls npPop2, [455](#)
 - calls npPrimary, [455](#)
 - calls npPush, [455](#)
 - calls npTrap, [455](#)
 - calls pfApplication, [455](#)
 - defun, [455](#)
- npSemiBackSet, [486](#)
 - calledby npSemiListing, [485](#)
 - calls npEqKey, [486](#)
 - defun, [486](#)
- npSemiListing, [485](#)
 - calledby npDefinitionlist, [485](#)
 - calledby npPPf, [503](#)
 - calls npListofFun, [485](#)
 - calls npSemiBackSet, [485](#)
 - calls pfAppend, [485](#)
 - defun, [485](#)
- npSigDecl, [449](#)
 - calledby npSigItem, [448](#)
 - calls npEqKey, [449](#)
 - calls npPop1, [449](#)
 - calls npPop2, [449](#)
 - calls npPush, [449](#)
 - calls npTrap, [449](#)
 - calls npType, [449](#)
 - calls pfParts, [449](#)
 - calls pfSpread, [449](#)
 - defun, [449](#)
- npSigItem, [448](#)
 - calledby npSigItemList, [446](#)
 - calls npSigDecl, [448](#)
 - calls npTrap, [448](#)
 - calls npTypeVariable, [448](#)
 - defun, [448](#)
- npSigItemList, [446](#)
 - calledby npSignature, [446](#)
 - calls npListing, [446](#)
 - calls npPop1, [447](#)
 - calls npPush, [446](#)
 - calls npSigItem, [446](#)

- calls pfAppend, 447
- calls pfListOf, 447
- calls pfParts, 447
- defun, 446
- npSignature, 446
 - calledby npSCategory, 445
 - calls npPop1, 446
 - calls npPush, 446
 - calls npSigItemList, 446
 - calls pfNothing, 446
 - calls pfWDec, 446
 - defun, 446
- npSignatureDefinee, 449
 - calledby npTypeVariablelist, 449
 - calledby npTypeVariable, 448
 - calls npInfixOperator, 449
 - calls npName, 449
 - calls npPrefixColon, 449
 - defun, 449
- npSingleRule, 486
 - calledby npRule, 486
 - calls npDefTail, 486
 - calls npPop1, 486
 - calls npPop2, 486
 - calls npPush, 486
 - calls npQuiver, 486
 - calls npTrap, 486
 - calls pfRule, 486
 - defun, 486
- npSLocalItem, 464
 - calledby npLocalItemList, 464
 - calls npListing, 464
 - calls npLocalItem, 464
 - calls npPop1, 464
 - calls npPush, 464
 - calls pfAppend, 464
 - calls pfParts, 464
 - defun, 464
- npSQualTypelist, 473
 - calledby npQualTypelist, 473
 - calls npListing, 473
 - calls npPop1, 473
 - calls npPush, 473
 - calls npQualType, 473
 - calls pfParts, 473
 - defun, 473
- npState, 504
 - calledby npAdd, 451
 - calledby npBackTrack, 440
 - calledby npConstTok, 477
 - calledby npDDInfKey, 500
 - calledby npDefinitionItem, 460
 - calledby npElse, 488
 - calledby npInfixOperator, 452
 - calledby npRightAssoc, 498
 - calledby npSCategory, 445
 - calledby npSymbolVariable, 498
 - calledby npWith, 442
 - uses \$inputStream, 504
 - uses \$stack, 504
 - defun, 504
- npStatement, 463
 - calledby npDefinitionItem, 460
 - calledby npMDEF, 458
 - calledby npPileExit, 509
 - calledby npVoid, 471
 - calls npBreak, 463
 - calls npExport, 463
 - calls npExpress, 463
 - calls npFree, 463
 - calls npImport, 463
 - calls npInline, 463
 - calls npIterate, 463
 - calls npLocal, 463
 - calls npLoop, 463
 - calls npReturn, 463
 - calls npTyping, 463
 - calls npVoid, 463
 - defun, 463
- npSuch, 442
 - calledby npMatch, 442
 - calls npLeftAssoc, 442
 - calls npLogical, 442
 - defun, 442
- npSuchThat, 469
 - calledby npIterator, 468
 - calls npAndOr, 469
 - calls npLogical, 469
 - calls pfSuchthat, 469
 - defun, 469
- npSum, 493
 - calledby npArith, 493

- calls npLeftAssoc, 493
- calls npTerm, 493
- defun, 493
- npSymbolVariable, 498
 - calledby npName, 497
 - calls npEqKey, 498
 - calls npId, 498
 - calls npPop1, 498
 - calls npPush, 498
 - calls npRestore, 498
 - calls npState, 498
 - calls tokConstruct, 498
 - calls tokPart, 498
 - calls tokPosn, 498
 - defun, 498
- npsynonym, 813
 - calledby handleNoParseCommands, 810
 - calls npProcessSynonym, 813
 - defun, 813
- npSynthetic, 491
 - calledby npRelation, 491
 - calls npAmpersandFrom, 491
 - calls npBy, 491
 - calls npPop1, 491
 - calls npPop2, 491
 - calls npPush, 491
 - calls pfApplication, 491
 - calls pfInfApplication, 491
 - defun, 491
- npsystem, 812
 - calledby handleNoParseCommands, 810
 - calls sayKeyedMsg, 812
 - defun, 812
- npTagged, 510
 - calledby npColon, 510
 - calls npTypedForm1, 510
 - calls pfTagged, 510
 - defun, 510
- npTerm, 493
 - calledby npSum, 493
 - calls npInfGeneric, 493
 - calls npPop1, 494
 - calls npPop2, 494
 - calls npPush, 494
 - calls npRemainder, 493
 - calls pfApplication, 494
 - defun, 493
- npTrap, 505
 - calledby npAdd, 451
 - calledby npAmpersand, 496
 - calledby npAndOr, 474
 - calledby npAssignment, 509
 - calledby npBackTrack, 440
 - calledby npConditional, 488
 - calledby npDecl, 507
 - calledby npDefaultDecl, 462
 - calledby npDefaultItem, 462
 - calledby npDefaultValue, 487
 - calledby npDefinitionItem, 460
 - calledby npDef, 480
 - calledby npElse, 489
 - calledby npExport, 463
 - calledby npForIn, 470
 - calledby npFree, 466
 - calledby npFromdom1, 496
 - calledby npFromdom, 495
 - calledby npLambda, 441
 - calledby npLetQualified, 459
 - calledby npListofFun, 513
 - calledby npList, 447
 - calledby npLocalDecl, 465
 - calledby npLocal, 465
 - calledby npLoop, 467
 - calledby npMdef, 457
 - calledby npQualified, 439
 - calledby npReturn, 471
 - calledby npSCategory, 446
 - calledby npSelector, 455
 - calledby npSigDecl, 449
 - calledby npSigItem, 448
 - calledby npSingleRule, 486
 - calledby npTypedForm1, 511
 - calledby npTypedForm, 513
 - calledby npTyping, 460
 - calledby npWith, 442
 - calls ncSoftError, 505
 - uses \$stok, 505
 - defun, 505
 - throws, 505
- npTrapForm, 505
 - calledby pfCheckId, 539

- calledby pfCheckItOut, 537
- calledby pfCheckMacroOut, 538
- calls ncSoftError, 505
- calls pfSourceStok, 505
- calls syGeneralErrorHere, 505
- calls tokPosn, 505
- defun, 505
- throws, 505
- npTuple, 438
 - calledby npComma, 438
 - calls npCommaBackSet, 438
 - calls npListofFun, 438
 - calls pfTupleListOf, 438
 - defun, 438
- npType, 441
 - calledby npADD, 450
 - calledby npDecl, 507
 - calledby npDefaultDecl, 462
 - calledby npLambda, 441
 - calledby npLocalDecl, 465
 - calledby npQualType, 474
 - calledby npSigDecl, 449
 - calledby npTypedForm1, 511
 - calls npMatch, 441
 - calls npPop1, 441
 - calls npPush, 441
 - calls npWith, 441
 - defun, 441
- npTypedForm, 513
 - calledby npCoerceTo, 512
 - calledby npColonQuery, 512
 - calledby npPretend, 512
 - calledby npRestrict, 513
 - calls npApplication, 513
 - calls npEqKey, 513
 - calls npPop1, 513
 - calls npPop2, 513
 - calls npPush, 513
 - calls npTrap, 513
 - defun, 513
- npTypedForm1, 511
 - calledby npTagged, 510
 - calls npEqKey, 511
 - calls npPop1, 511
 - calls npPop2, 511
 - calls npPush, 511
- calls npTrap, 511
- calls npType, 511
- defun, 511
- npTypeStyle, 511
 - calledby npTypified, 511
 - calls npCoerceTo, 511
 - calls npColonQuery, 512
 - calls npPretend, 512
 - calls npRestrict, 511
 - defun, 511
- npTypeVariable, 448
 - calledby npDefaultItem, 462
 - calledby npLocalItem, 465
 - calledby npSigItem, 448
 - calls npParenthesized, 448
 - calls npPop1, 448
 - calls npPush, 448
 - calls npSignatureDefinee, 448
 - calls npTypeVariablelist, 448
 - calls pfListOf, 448
 - defun, 448
- npTypeVariablelist, 449
 - calledby npTypeVariable, 448
 - calls npListing, 449
 - calls npSignatureDefinee, 449
 - defun, 449
- npTypified, 511
 - calledby npColon, 510
 - calls npAnyNo, 511
 - calls npApplication, 511
 - calls npTypeStyle, 511
 - defun, 511
- npTyping, 460
 - calledby npDefinitionItem, 460
 - calledby npStatement, 463
 - calls npDefaultItemList, 460
 - calls npEqKey, 460
 - calls npPop1, 461
 - calls npPush, 461
 - calls npTrap, 460
 - calls pfTyping, 461
 - defun, 460
- npVariable, 505
 - calledby npAdd, 451
 - calledby npForIn, 470
 - calledby npLambda, 441

- calledby npWith, 443
- calls npParenthesized, 505
- calls npPop1, 506
- calls npPush, 506
- calls npVariableName, 506
- calls npVariablelist, 505
- calls pfListOf, 506
- defun, 505
- npVariablelist, 506
 - calledby npVariable, 505
 - calls npListing, 506
 - calls npVariableName, 506
 - defun, 506
- npVariableName, 506
 - calledby npVariablelist, 506
 - calledby npVariable, 506
 - calls npDecl, 506
 - calls npName, 506
 - calls npPop1, 506
 - calls npPush, 506
 - calls pfNothing, 506
 - calls pfTyped, 506
 - defun, 506
- npVoid, 471
 - calledby npStatement, 463
 - calls npAndOr, 471
 - calls npStatement, 471
 - calls pfNoval, 471
 - defun, 471
- npWConditional, 487
 - calledby npSCategory, 445
 - calls npConditional, 487
 - calls npPop1, 488
 - calls npPush, 488
 - calls pfTweakIf, 488
 - defun, 487
- npWhile, 470
 - calledby npIterators, 468
 - calledby npIterator, 469
 - calls npAndOr, 470
 - calls npLogical, 470
 - calls pfWhile, 470
 - defun, 470
- npWith, 442
 - calledby npPrimary2, 450
 - calledby npType, 441
- calls npCategoryL, 442
- calls npCompMissing, 443
- calls npEqKey, 442
- calls npEqPeek, 442
- calls npPop1, 443
- calls npPop2, 443
- calls npPush, 443
- calls npRestore, 443
- calls npState, 442
- calls npTrap, 442
- calls npVariable, 443
- calls pfNothing, 443
- calls pfWith, 443
- defun, 442
- npZeroOrMore, 469
 - calledby npIterators, 468
 - calls npPop1, 469
 - calls npPop2, 469
 - calls npPush, 469
 - uses \$stack, 469
 - defun, 469
- nreverse0
 - calledby augmentHasArgs, 1619
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcMakeEquations, 1484
 - calledby bcMakeLinearEquations, 1484
 - calledby coerceInt1, 744
 - calledby reduceAlistForDomain, 1616
 - calledby reportOpsFromLisplib, 1092
 - calledby reportOpsFromUnitDirectly, 1096
- NRTcompileEvalForm
 - calledby coerceInt1, 743
- NRTevalDomain, 1229
 - calledby compiledLookup, 1225
 - calls evalDomain, 1229
 - calls eval, 1229
 - calls qcar, 1229
 - defun, 1229
- nsbst
 - calledby ruleLhsTran, 620
 - calledby zeroOneTran, 360
- obey
 - calledby copyright, 855
 - calledby editFile, 873
 - calledby license, 934

- calledby newHelpSpad2Cmd, 881
- calledby summary, 1103
- objCodeMode, 517
 - defmacro, 517
- objCodeVal, 517
 - defmacro, 517
- object2Identifier
 - calledby fixObjectForPrinting, 796
 - calledby frameSpad2Cmd, 26
 - calledby pathnameTypeId, 1231
 - calledby readHiFi, 911
 - calledby saveHistory, 905
 - calledby selectOptionLC, 819
 - calledby setHistoryCore, 894
 - calledby writeHiFi, 912
- object2String
 - calledby clearCmdExcept, 840
 - calledby displayMacro, 794
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby getTraceOption, 118
 - calledby makePathname, 1233
 - calledby set1, 1083
 - calledby setExposeAddGroup, 157
 - calledby setFunctionsCache, 979
 - calledby setLinkerArgs, 1001
 - calledby setNagHost, 1027
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calledby setStreamsCalculate, 1075
- objEnv, 517
 - defun, 517
- objMode, 516
 - calledby coerceBranch2Union, 766
 - calledby coerceInt0, 742
 - calledby coerceInt1, 744
 - calledby coerceIntAlgebraicConstant, 766
 - calledby coerceIntByMap, 761
 - calledby coerceIntCommutate, 757
 - calledby coerceIntPermute, 754
 - calledby coerceIntSpecial, 760
 - calledby coerceIntTableOrFunction, 759
 - calledby coerceInteractive, 740
 - calledby coerceInt, 741
 - calledby coerceUnion2Branch, 775
 - calledby displayType, 800
 - calledby displayValue, 799
 - calledby getAndEvalConstructorArgument, 1144
 - calledby interpret2, 348
 - calledby isDomainValuedVariable, 1145
 - calledby printTypeAndTime, 352
 - calledby processInteractive1, 345
 - calledby retract2Specialization, 772
 - calledby retractByFunction, 778
 - calledby retract, 1269
 - calledby showInOut, 910
 - calledby transTraceItem, 126
 - defmacro, 516
- objModeFn, 1307
 - defun, 1307
- objSetMode, 516
 - calledby coerceInt0, 742
 - defmacro, 516
- objSetVal, 515
 - defmacro, 515
- objVal, 516
 - calledby coerceBranch2Union, 766
 - calledby coerceInt0, 742
 - calledby coerceInt1, 744
 - calledby coerceIntByMap, 761
 - calledby coerceIntTableOrFunction, 759
 - calledby coerceInteractive, 740
 - calledby coerceInt, 741
 - calledby coerceUnion2Branch, 775
 - calledby evaluateType, 1118
 - calledby getAndEvalConstructorArgument, 1144
 - calledby interpret2, 348
 - calledby retract2Specialization, 772
 - calledby retract, 1269
 - calledby transTraceItem, 126
 - defmacro, 516
- objValFn, 1307
 - defun, 1307
- objValUnwrap, 516

- calledby bcInputEquations, 1481
- calledby bcInputExplicitMatrix, 1443
- calledby bcInputMatrixByFormula, 1445
- calledby coerceIntAlgebraicConstant, 766
- calledby coerceIntCommute, 757
- calledby coerceOrThrowFailure, 763
- calledby coerceRetract, 777
- calledby coerceSpadArgs2E, 128
- calledby coerceSpadFunValue2E, 130
- calledby coerceTraceArgs2E, 127
- calledby coerceTraceFunValue2E, 129
- calledby coerceUnion2Branch, 775
- calledby displayValue, 799
- calledby evaluateType1, 1120
- calledby isDomainValuedVariable, 1145
- calledby processInteractive1, 345
- calledby retract2Specialization, 772
- calledby retractByFunction, 778
- calledby showInOut, 910
- calledby valueArgsEqual?, 764
- defmacro, 516
- ofCategory, 717
 - calledby coerceIntAlgebraicConstant, 767
 - calledby evalCategory, 1145
 - calledby ofCategory, 717
 - calls hasCaty, 717
 - calls identp, 717
 - calls ofCategory, 717
 - local def \$Subst, 717
 - local def \$hope, 717
 - defun, 717
- oldCompLookup, 1229
 - calledby basicLookup, 1226
 - calls lookupInDomainVector, 1229
 - local def \$lookupDefaults, 1229
 - defun, 1229
- oldHistFileName, 887
 - calledby initHist, 888
 - calls makeHistFileName, 887
 - uses \$oldHistoryFileName, 887
 - defun, 887
- oldParseString, 1543
 - defun, 1543
- om-bindTCP, 1359
 - defun, 1359
- om-closeConn, 1358
 - defun, 1358
- om-closeDev, 1358
 - defun, 1358
- om-connectTCP, 1360
 - defun, 1360
- om-getApp, 1361
 - defun, 1361
- om-getAtp, 1362
 - defun, 1362
- om-getAttr, 1362
 - defun, 1362
- om-getBind, 1362
 - defun, 1362
- om-getBVar, 1362
 - defun, 1362
- om-getByteArray, 1363
 - defun, 1363
- om-getConnInDev, 1359
 - defun, 1359
- om-getConnOutDev, 1359
 - defun, 1359
- om-getEndApp, 1363
 - defun, 1363
- om-getEndAtp, 1363
 - defun, 1363
- om-getEndAttr, 1363
 - defun, 1363
- om-getEndBind, 1364
 - defun, 1364
- om-getEndBVar, 1364
 - defun, 1364
- om-getEndError, 1364
 - defun, 1364
- om-getEndObject, 1364
 - defun, 1364
- om-getError, 1365
 - defun, 1365
- om-getFloat, 1365
 - defun, 1365
- om-getInt, 1365
 - defun, 1365
- om-getObject, 1365
 - defun, 1365
- om-getString, 1366
 - defun, 1366
- om-getSymbol, 1366
 - defun, 1366

- defun, 1366
- om-getType, 1366
 - defun, 1366
- om-getVar, 1366
 - defun, 1366
- om-listCDs, 1356
 - defun, 1356
- om-listSymbols, 1356
 - defun, 1356
- om-makeConn, 1358
 - defun, 1358
- om-openFileDev, 1357
 - defun, 1357
- om-openStringDev, 1358
 - defun, 1358
- om-putApp, 1367
 - defun, 1367
- om-putAtp, 1367
 - defun, 1367
- om-putAttr, 1367
 - defun, 1367
- om-putBind, 1367
 - defun, 1367
- om-putBVar, 1368
 - defun, 1368
- om-putByteArray, 1368
 - defun, 1368
- om-putEndApp, 1368
 - defun, 1368
- om-putEndAtp, 1368
 - defun, 1368
- om-putEndAttr, 1369
 - defun, 1369
- om-putEndBind, 1369
 - defun, 1369
- om-putEndBVar, 1369
 - defun, 1369
- om-putEndError, 1369
 - defun, 1369
- om-putEndObject, 1370
 - defun, 1370
- om-putError, 1370
 - defun, 1370
- om-putFloat, 1370
 - defun, 1370
- om-putInt, 1370
 - defun, 1370
- om-putObject, 1371
 - defun, 1371
- om-putString, 1371
 - defun, 1371
- om-putSymbol, 1371
 - defun, 1371
- om-putVar, 1371
 - defun, 1371
- om-Read, 1355
 - defun, 1355
- om-setDevEncoding, 1357
 - defun, 1357
- om-stringPtrToString, 1372
 - defun, 1372
- om-stringToStringPtr, 1372
 - defun, 1372
- om-supportsCD, 1356
 - defun, 1356
- om-supportsSymbol, 1356
 - defun, 1356
- openOutputLibrary, 973
 - calledby setOutputLibrary, 972
 - calls dropInputLibrary, 973
 - uses input-libraries, 973
 - uses output-library, 973
 - defun, 973
- openserver, 1173
 - calledby restart, 307
 - defun, 1173
- operationopen, 1193
 - calledby resethashtables, 1186
 - calledby restart0, 309
 - uses *operation-hash*, 1193
 - uses *operation-stream*, 1193
 - uses *operation-stream-stamp*, 1193
 - uses \$spadroot, 1193
 - defun, 1193
- opIsHasCat
 - calledby basicLookup, 1226
- opOf
 - calledby abbreviationsSpad2Cmd, 823
 - calledby augmentHasArgs, 1619
 - calledby conOpPage1, 1629
 - calledby dbAddDocTable, 1636
 - calledby dbSearchOrder, 1610

- calledby dbSelectCon, 1648
- calledby dbShowConditions, 1648
- calledby dbShowCons1, 1643
- calledby dbShowConsDoc, 1645
- calledby displaySpad2Cmd, 864
- calledby domainToGenvar, 101
- calledby getConstantFromDomain, 767
- calledby hasCaty, 718
- calledby isDomainOrPackage, 107
- calledby kArgumentCheck, 1625
- calledby kPage, 1593
- calledby kcPage, 1611
- calledby kcaPage1, 1617
- calledby kccPage, 1618
- calledby kcdePage, 1620
- calledby kcnPage, 1622
- calledby kcpPage, 1615
- calledby kcuPage, 1621
- calledby kePage, 1605
- calledby reportOperations, 1089
- calledby spadTrace, 132
- optionError, 789
 - calledby countCache, 980
 - calledby readSpad2Cmd, 943
 - calls commandErrorMessage, 789
 - defun, 789
- options2uc
 - calledby trace3, 83
- optionUserLevelError, 790
 - calls userLevelErrorMessage, 790
 - defun, 790
- opTran, 621
 - calledby pf2Sex1, 598
 - calledby pfApplication2Sex, 604
 - defun, 621
- orderBySlotNumber, 114
 - calls assocright, 114
 - calls exit, 114
 - calls orderList, 114
 - calls seq, 114
 - defun, 114
- orderList
 - calledby orderBySlotNumber, 114
- orderMms
 - calledby retractByFunction, 778
- orderUnionEntries
 - calledby coerceBranch2Union, 766
 - calledby coerceUnion2Branch, 775
- originsInOrder, 1636
 - calledby dbDocTable, 1635
 - calledby originsInOrder, 1636
 - calls ancestorsOf, 1636
 - calls assocleft, 1636
 - calls getdatabase, 1636
 - calls insert, 1636
 - calls originsInOrder, 1636
 - calls parentsOf, 1636
 - defun, 1636
- out-stream, 1148
 - usedby spad-long-error, 1155
 - defvar, 1148
- output
 - calledby recordAndPrint, 349
- output-library, 973
 - usedby openOutputLibrary, 973
 - defvar, 973
- outputFormat
 - calledby displayValue, 799
- outputTran2
 - calledby prinmathor0, 151
- parallel, 689
 - syntax, 689
- parentsOf
 - calledby kcpPage, 1615
 - calledby originsInOrder, 1636
- parse-integer
 - calledby bcInputEquations, 1481
 - calledby bcInputExplicitMatrix, 1443
 - calledby bcInputMatrixByFormula, 1445
- parseAndEval, 1542
 - defun, 1542
- parseAndEval1, 1542
 - defun, 1542
- parseAndInterpret, 340
 - calledby executeQuietCommand, 339
 - calledby serverReadLine, 336
 - uses \$InteractiveFrame, 340
 - uses \$InteractiveMode, 340
 - uses \$boot, 340
 - uses \$e, 340
 - uses \$spad, 340

- defun, 340
- parseFromString, 340
 - calledby parseSystemCmd, 808
 - calls StreamNull, 340
 - calls incString, 340
 - calls lineoftoks, 340
 - calls macroExpanded, 340
 - calls nloopParse, 340
 - calls next, 340
 - calls pf2Sex, 340
 - defun, 340
- parseNoMacroFromString, 1627
 - calledby mkConform, 1628
 - calls StreamNull, 1627
 - calls function, 1627
 - calls incString, 1627
 - calls lineoftoks, 1627
 - calls nloopParse, 1627
 - calls next, 1627
 - calls pf2Sex, 1627
 - defun, 1627
- ParsePair, 210
- parseSystemCmd, 808
 - calledby handleParsedSystemCommands, 808
 - calls dumbTokenize, 808
 - calls parseFromString, 808
 - calls stripSpaces, 808
 - calls tokTran, 808
 - defun, 808
- parseWord, 1558
 - defun, 1558
- pathname, 1232
 - calledby deleteFile, 1233
 - calledby editFile, 873
 - calledby editSpad2Cmd, 872
 - calledby getDependentsOfConstructor, 1621
 - calledby getUsersOfConstructor, 1622
 - calledby makePathname, 1233
 - calledby namestring, 1230
 - calledby pathnameDirectory, 1232
 - calledby pathnameName, 1231
 - calledby pathnameType, 1231
 - calledby pathname, 1232
 - calledby readSpad2Cmd, 943
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calledby setExposeAddGroup, 157
 - calledby setExpose, 159
 - calledby updateSourceFiles, 874
 - calledby workfilesSpad2Cmd, 1139
 - calls make-filename, 1232
 - calls pathname, 1232
 - defun, 1232
- pathnameDirectory, 1232
 - calledby editSpad2Cmd, 872
 - calledby loadLib, 1221
 - calledby mergePathnames, 1231
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calls pathname, 1232
 - defun, 1232
- pathnameName, 1231
 - calledby editSpad2Cmd, 872
 - calledby mergePathnames, 1231
 - calledby readSpad2Cmd, 943
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calledby updateSourceFiles, 874
 - calls pathname, 1231
 - defun, 1231
- PathnameString
 - calledby pfname, 384
- pathnameType, 1231
 - calledby editSpad2Cmd, 872
 - calledby mergePathnames, 1231
 - calledby pathnameTypeId, 1231
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054

- calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calledby updateSourceFiles, 874
 - calls pathname, 1231
 - defun, 1231
- pathnameTypeId, 1231
 - calledby readSpad2Cmd, 943
 - calledby updateSourceFiles, 874
 - calls object2Identifier, 1231
 - calls pathnameType, 1231
 - calls upcase, 1231
 - defun, 1231
- patternVarsOf, 619
 - calledby ruleLhsTran, 620
 - calledby rulePredicateTran, 618
 - calls patternVarsOf1, 619
 - defun, 619
- patternVarsOf1, 619
 - calledby patternVarsOf1, 619
 - calledby patternVarsOf, 619
 - calls patternVarsOf1, 619
 - defun, 619
- pcounters, 99
 - calledby trace1, 79
 - calls bright, 99
 - calls concat, 99
 - calls sayBrightly, 99
 - uses \$countList, 99
 - defun, 99
- peekTimedName
 - calledby interpretTopLevel, 345
- permuteToOrder, 756
 - calledby computeTTTranspositions, 754
 - calledby permuteToOrder, 756
 - calls permuteToOrder, 756
 - defun, 756
- pf0ApplicationArgs, 535
 - calledby macApplication, 519
 - calls pf0FlattenSyntacticTuple, 535
 - calls pfApplicationArg, 535
 - defun, 535
- pf0AssignLhsItems, 554
 - calledby pf2Sex1, 599
 - calls pfAssignLhsItems, 554
 - calls pfParts, 554
 - defun, 554
- pf0DefinitionLhsItems, 560
 - calledby pfDefinition2Sex, 613
 - calls pfDefinitionLhsItems, 560
 - calls pfParts, 560
 - defun, 560
- pf0FlattenSyntacticTuple, 535
 - calledby pf0ApplicationArgs, 535
 - calledby pf0FlattenSyntacticTuple, 535
 - calls pf0FlattenSyntacticTuple, 535
 - calls pf0TupleParts, 535
 - calls pfTuple?, 535
 - defun, 535
- pf0ForinLhs, 565
 - calledby pf2Sex1, 598
 - calls pfForinLhs, 565
 - calls pfParts, 565
 - defun, 565
- pf0FreeItems, 564
 - calledby pf2Sex1, 599
 - calls pfFreeItems, 564
 - calls pfParts, 564
 - defun, 564
- pf0LambdaArgs, 572
 - calledby macLambdaParameterHandling, 528
 - calledby pfLambdaTran, 614
 - calls pfLambdaArgs, 572
 - calls pfParts, 572
 - defun, 572
- pf0LocalItems, 573
 - calledby pf2Sex1, 599
 - calls pfLocalItems, 573
 - calls pfParts, 573
 - defun, 573
- pf0LoopIterators, 574
 - calledby pf0LoopIterators, 574
 - calledby pf2Sex1, 598
 - calls pf0LoopIterators, 574
 - calls pfParts, 574
 - defun, 574
- pf0MLambdaArgs, 576
 - calledby macMLambdaApply, 520
 - calledby macLambdaParameterHandling, 528
 - calls pfParts, 576
 - defun, 576

- pf0SequenceArgs, 585
 - calledby pfSequence2Sex, 608
 - calledby pfUnSequence, 591
 - calls pfParts, 585
 - calls pfSequenceArgs, 585
 - defun, 585
- pf0TupleParts, 591
 - calledby pf0FlattenSyntacticTuple, 535
 - calledby pf2Sex1, 598
 - calledby pfApplication2Sex, 604
 - calledby pfCheckArg, 539
 - calledby pfCheckItOut, 537
 - calledby pfCollectVariable1, 540
 - calledby pfSexpr,strip, 548
 - calledby pfSuchThat2Sex, 605
 - calledby pfTransformArg, 542
 - calls pfParts, 591
 - calls pfTupleParts, 591
 - defun, 591
- pf0WhereContext, 593
 - calledby pf2Sex1, 599
 - calls pfParts, 593
 - calls pfWhereContext, 593
 - defun, 593
- pf2Sex, 597
 - calledby intInterpretPform, 360
 - calledby parseFromString, 340
 - calledby parseNoMacroFromString, 1627
 - calledby pfApplication2Sex, 604
 - calledby pfSuchThat2Sex, 605
 - calls pf2Sex1, 597
 - uses \$QuietCommand, 597
 - uses \$insideApplication, 597
 - uses \$insideRule, 597
 - uses \$insideSEQ, 597
 - defun, 597
- pf2Sex1, 598
 - calledby loopIters2Sex, 610
 - calledby pf2Sex1, 598
 - calledby pf2Sex, 597
 - calledby pfApplication2Sex, 604
 - calledby pfCollect2Sex, 612
 - calledby pfDefinition2Sex, 613
 - calledby pfLambdaTran, 614
 - calledby pfLhsRule2Sex, 617
 - calledby pfOp2Sex, 606
 - calledby pfRhsRule2Sex, 617
 - calledby pfSequence2Sex, 608
 - calledby pfSuchThat2Sex, 605
 - calls keyedSystemError, 599
 - calls loopIters2Sex, 598
 - calls opTran, 598
 - calls pf0AssignLhsItems, 599
 - calls pf0ForinLhs, 598
 - calls pf0FreeItems, 599
 - calls pf0LocalItems, 599
 - calls pf0LoopIterators, 598
 - calls pf0TupleParts, 598
 - calls pf0WhereContext, 599
 - calls pf2Sex1, 598
 - calls pfAbSynOp, 600
 - calls pfAnd?, 599
 - calls pfAndLeft, 599
 - calls pfAndRight, 599
 - calls pfApplication2Sex, 598
 - calls pfApplication?, 598
 - calls pfAssign?, 599
 - calls pfAssignRhs, 599
 - calls pfBreak?, 599
 - calls pfBreakFrom, 599
 - calls pfCoerceto?, 598
 - calls pfCoercetoExpr, 598
 - calls pfCoercetoType, 598
 - calls pfCollect2Sex, 598
 - calls pfCollect?, 598
 - calls pfDefinition2Sex, 599
 - calls pfDefinition?, 599
 - calls pfDo?, 599
 - calls pfDoBody, 599
 - calls pfExit?, 598
 - calls pfExitCond, 598
 - calls pfExitExpr, 598
 - calls pfForin?, 598
 - calls pfForinWhole, 598
 - calls pfFree?, 599
 - calls pfFromdom?, 598
 - calls pfFromdomDomain, 598
 - calls pfFromdomWhat, 598
 - calls pfIdSymbol, 598
 - calls pfIf?, 598
 - calls pfIfCond, 598
 - calls pfIfElse, 598

- calls pfIfThen, 598
- calls pfIterate?, 599
- calls pfLambda2Sex, 599
- calls pfLambda?, 599
- calls pfLiteral2Sex, 598
- calls pfLiteral?, 598
- calls pfLocal?, 599
- calls pfLoop?, 598
- calls pfMLambda?, 599
- calls pfMacro?, 599
- calls pfNot?, 599
- calls pfNotArg, 599
- calls pfNothing?, 598
- calls pfNovalue?, 599
- calls pfNovalueExpr, 599
- calls pfOr?, 599
- calls pfOrLeft, 599
- calls pfOrRight, 599
- calls pfPretend?, 598
- calls pfPretendExpr, 598
- calls pfPretendType, 598
- calls pfRestrict?, 599
- calls pfRestrictExpr, 599
- calls pfRestrictType, 599
- calls pfReturn?, 599
- calls pfReturnExpr, 599
- calls pfRule2Sex, 599
- calls pfRule?, 599
- calls pfSequence2Sex, 598
- calls pfSequence?, 598
- calls pfSuchthat?, 599
- calls pfSuchthatCond, 599
- calls pfSymbol?, 598
- calls pfSymbolSymbol, 598
- calls pfTagged?, 598
- calls pfTaggedExpr, 598
- calls pfTaggedTag, 598
- calls pfTuple?, 598
- calls pfTyped?, 599
- calls pfTypedId, 599
- calls pfTypedType, 599
- calls pfWhere?, 599
- calls pfWhereExpr, 599
- calls pfWhile?, 598
- calls pfWhileCond, 598
- calls pfWrong?, 599
- calls spadThrow, 599
- calls tokPart, 600
- uses \$QuietCommand, 600
- uses \$insideRule, 600
- uses \$insideSEQ, 600
- defun, 598
- pf2sex1
 - calledby pfCollectArgTran, 615
- pfAbSynOp, 702
 - calledby macLambdaParameterHandling, 528
 - calledby pf2Sex1, 600
 - calledby pfCopyWithPos, 534
 - calledby pfLeaf?, 545
 - calledby pfLiteral?, 546
 - calledby pfLiteralClass, 546
 - calledby pfMapParts, 535
 - calledby pfSexpr,strip, 548
 - calls ifcar, 702
 - defun, 702
- pfAbSynOp?, 702
 - calledby intloopProcess, 356
 - calledby pfAnd?, 552
 - calledby pfApplication?, 553
 - calledby pfAssign?, 554
 - calledby pfBreak?, 556
 - calledby pfCoerceto?, 557
 - calledby pfCollect?, 558
 - calledby pfDefinition?, 559
 - calledby pfDo?, 560
 - calledby pfExit?, 561
 - calledby pfForin?, 564
 - calledby pfFree?, 563
 - calledby pfFromdom?, 566
 - calledby pfId?, 544
 - calledby pfIf?, 567
 - calledby pfIterate?, 569
 - calledby pfLambda?, 571
 - calledby pfLam, 570
 - calledby pfLocal?, 572
 - calledby pfLoop?, 574
 - calledby pfMLambda?, 576
 - calledby pfMacro?, 575
 - calledby pfNot?, 577
 - calledby pfNothing?, 543
 - calledby pfNovalue?, 578

- calledby pfOr?, 578
- calledby pfPretend?, 580
- calledby pfRestrict?, 581
- calledby pfReturn?, 582
- calledby pfRule?, 584
- calledby pfSequence?, 585
- calledby pfSuchthat?, 586
- calledby pfSymbol?, 550
- calledby pfTagged?, 587
- calledby pfTuple?, 590
- calledby pfTyped?, 589
- calledby pfWhere?, 592
- calledby pfWhile?, 594
- calledby pfWrong?, 595
- calls eqcar, 702
- defun, 702
- pfAdd, 550
 - calledby npAdd, 451
 - calledby npDefaultValue, 487
 - calls pfNothing, 550
 - calls pfTree, 550
 - defun, 550
- pfAnd, 551
 - calledby pfInfApplication, 569
 - calls pfTree, 551
 - defun, 551
- pfAnd?, 552
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 552
 - defun, 552
- pfAndLeft, 552
 - calledby pf2Sex1, 599
 - defun, 552
- pfAndRight, 553
 - calledby pf2Sex1, 599
 - defun, 553
- pfAppend, 553
 - calledby npPPg, 503
 - calledby npPileDefinitionlist, 481
 - calledby npSDefaultItem, 461
 - calledby npSLocalItem, 464
 - calledby npSemiListing, 485
 - calledby npSigItemList, 447
 - calledby pfUnSequence, 591
 - defun, 553
- pfApplication, 551
 - calledby npApplication2, 456
 - calledby npApplication, 454
 - calledby npEncl, 475
 - calledby npInterval, 492
 - calledby npLeftAssoc, 499
 - calledby npRightAssoc, 498
 - calledby npSelector, 455
 - calledby npSynthetic, 491
 - calledby npTerm, 494
 - calledby pfBraceBar, 555
 - calledby pfBrace, 555
 - calledby pfBracketBar, 555
 - calledby pfBracket, 555
 - calledby pfFix, 563
 - calledby pfFromDom, 565
 - calledby pfInfApplication, 569
 - calls pfTree, 551
 - defun, 551
- pfApplication2Sex, 603
 - calledby pf2Sex1, 598
 - calls hasOptArgs?, 604
 - calls opTran, 604
 - calls pf0TupleParts, 604
 - calls pf2Sex1, 604
 - calls pf2Sex, 604
 - calls pfApplicationArg, 604
 - calls pfApplicationOp, 604
 - calls pfOp2Sex, 603
 - calls pfSuchThat2Sex, 604
 - calls pfTuple?, 604
 - uses \$insideApplication, 604
 - uses \$insideRule, 604
 - defun, 603
- pfApplication?, 553
 - calledby macExpand, 519
 - calledby pf2Sex1, 598
 - calledby pfCheckItOut, 537
 - calledby pfCheckMacroOut, 538
 - calledby pfCollect1?, 540
 - calledby pfFlattenApp, 539
 - calledby pfFromDom, 565
 - calledby pfSexpr,strip, 548
 - calls pfAbSynOp?, 553
 - defun, 553
- pfApplicationArg, 552
 - calledby pf0ApplicationArgs, 535

- calledby pfApplication2Sex, 604
- calledby pfCollectVariable1, 540
- calledby pfFlattenApp, 540
- calledby pfFromDom, 565
- calledby pfSexpr,strip, 548
- defun, 552
- pfApplicationOp, 552
 - calledby macApplication, 519
 - calledby pfApplication2Sex, 604
 - calledby pfCollect1?, 540
 - calledby pfFlattenApp, 540
 - calledby pfFromDom, 565
 - calledby pfSexpr,strip, 548
 - defun, 552
- pfAssign, 553
 - calledby npAssignment, 509
 - calls pfTree, 553
 - defun, 553
- pfAssign?, 554
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 554
 - defun, 554
- pfAssignLhsItems, 554
 - calledby pf0AssignLhsItems, 554
 - defun, 554
- pfAssignRhs, 554
 - calledby pf2Sex1, 599
 - defun, 554
- pfAttribute, 551
 - calledby npSCategory, 445
 - calls pfTree, 551
 - defun, 551
- pfBrace, 555
 - calledby npBraced, 479
 - calls pfApplication, 555
 - calls pfIdPos, 555
 - calls tokPosn, 555
 - defun, 555
- pfBraceBar, 555
 - calledby npBraced, 479
 - calls pfApplication, 555
 - calls pfIdPos, 555
 - calls tokPosn, 555
 - defun, 555
- pfBracket, 555
 - calledby npBracked, 478
- calls pfApplication, 555
- calls pfIdPos, 555
- calls tokPosn, 555
- defun, 555
- pfBracketBar, 555
 - calledby npBracked, 478
 - calls pfApplication, 555
 - calls pfIdPos, 556
 - calls tokPosn, 556
 - defun, 555
- pfBreak, 556
 - calledby npBreak, 467
 - calls pfTree, 556
 - defun, 556
- pfBreak?, 556
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 556
 - defun, 556
- pfBreakFrom, 556
 - calledby pf2Sex1, 599
 - defun, 556
- pfCharPosn, 533
 - calledby ppos, 647
 - calls poCharPosn, 533
 - defun, 533
- pfCheckArg, 539
 - calledby pfCheckMacroOut, 538
 - calls pf0TupleParts, 539
 - calls pfCheckId, 539
 - calls pfListOf, 539
 - calls pfTuple?, 539
 - defun, 539
- pfCheckId, 539
 - calledby pfCheckArg, 539
 - calledby pfCheckMacroOut, 538
 - calls npTrapForm, 539
 - calls pfId?, 539
 - defun, 539
- pfCheckItOut, 537
 - calledby npDef, 480
 - calls npTrapForm, 537
 - calls pf0TupleParts, 537
 - calls pfApplication?, 537
 - calls pfCollect1?, 537
 - calls pfCollectVariable1, 537
 - calls pfDefinition?, 537

- calls pfFlattenApp, 537
- calls pfId?, 537
- calls pfListOf, 537
- calls pfNothing, 537
- calls pfTagged?, 537
- calls pfTaggedExpr, 537
- calls pfTaggedTag, 537
- calls pfTaggedToTyped1, 537
- calls pfTaggedToTyped, 537
- calls pfTransformArg, 537
- calls pfTuple?, 537
- calls pfTyped, 537
- defun, 537
- pfCheckMacroOut, 538
 - calledby npMdef, 457
 - calls npTrapForm, 538
 - calls pfApplication?, 538
 - calls pfCheckArg, 538
 - calls pfCheckId, 538
 - calls pfFlattenApp, 538
 - calls pfId?, 538
 - defun, 538
- pfCoerceto, 557
 - calledby npCoerceTo, 512
 - calls pfTree, 557
 - defun, 557
- pfCoerceto?, 557
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 557
 - defun, 557
- pfCoercetoExpr, 557
 - calledby pf2Sex1, 598
 - defun, 557
- pfCoercetoType, 557
 - calledby pf2Sex1, 598
 - defun, 557
- pfCollect, 558
 - calledby npExpress, 472
 - calls pfTree, 558
 - defun, 558
- pfCollect1?, 540
 - calledby pfCheckItOut, 537
 - calledby pfFlattenApp, 540
 - calledby pfTaggedToTyped1, 542
 - calls pfApplication?, 540
 - calls pfApplicationOp, 540
 - calls pfId?, 540
 - calls pfIdSymbol, 540
 - defun, 540
- pfCollect2Sex, 612
 - calledby pf2Sex1, 598
 - calls loopIters2Sex, 612
 - calls pf2Sex1, 612
 - calls pfCollectBody, 612
 - calls pfCollectIterators, 612
 - calls pfParts, 612
 - defun, 612
- pfCollect?, 558
 - calledby pf2Sex1, 598
 - calledby pfCollectArgTran, 615
 - calls pfAbSynOp?, 558
 - defun, 558
- pfCollectArgTran, 615
 - calledby pfLambdaTran, 614
 - calls pf2sex1, 615
 - calls pfCollect?, 615
 - calls pfCollectBody, 615
 - calls pfCollectIterators, 615
 - calls pfParts, 615
 - defun, 615
- pfCollectBody, 558
 - calledby pfCollect2Sex, 612
 - calledby pfCollectArgTran, 615
 - defun, 558
- pfCollectIterators, 558
 - calledby pfCollect2Sex, 612
 - calledby pfCollectArgTran, 615
 - defun, 558
- pfCollectVariable1, 540
 - calledby pfCheckItOut, 537
 - calledby pfTaggedToTyped1, 542
 - calls pf0TupleParts, 540
 - calls pfApplicationArg, 540
 - calls pfSuch, 540
 - calls pfTaggedToTyped, 540
 - calls pfTypedId, 540
 - calls pfTypedType, 541
 - calls pfTyped, 540
 - defun, 540
- pfCopyWithPos, 534
 - calledby macId, 523
 - calledby pfCopyWithPos, 534

- calls pfAbSynOp, 534
- calls pfCopyWithPos, 534
- calls pfLeaf?, 534
- calls pfLeaf, 534
- calls pfParts, 534
- calls pfTree, 534
- calls tokPart, 534
- defun, 534
- pfDefinition, 559
 - calledby npDef, 480
 - calls pfTree, 559
 - defun, 559
- pfDefinition2Sex, 613
 - calledby pf2Sex1, 599
 - calls pf0DefinitionLhsItems, 613
 - calls pf2Sex1, 613
 - calls pfDefinitionRhs, 613
 - calls pfLambdaTran, 613
 - calls systemError, 613
 - uses \$insideApplication, 613
 - defun, 613
- pfDefinition?, 559
 - calledby pf2Sex1, 599
 - calledby pfCheckItOut, 537
 - calledby pfTaggedToTyped1, 542
 - calls pfAbSynOp?, 559
 - defun, 559
- pfDefinitionLhsItems, 559
 - calledby pf0DefinitionLhsItems, 560
 - defun, 559
- pfDefinitionRhs, 559
 - calledby pfDefinition2Sex, 613
 - defun, 559
- pfDo, 560
 - calledby pfLoop1, 573
 - calledby pfLp, 574
 - calls pfTree, 560
 - defun, 560
- pfDo?, 560
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 560
 - defun, 560
- pfDoBody, 560
 - calledby pf2Sex1, 599
 - defun, 560
- pfDocument, 544
 - calledby npParse, 433
 - calledby npRecoverTrap, 483
 - calls pfLeaf, 544
 - defun, 544
- pfEnSequence, 561
 - calledby npEnclosed, 504
 - calledby npItem, 434
 - calledby npPDefinition, 476
 - calledby npPP, 502
 - calls pfListOf, 561
 - calls pfSequence, 561
 - calls pfTuple, 561
 - defun, 561
- pfExit, 561
 - calledby npPileExit, 509
 - calls pfTree, 561
 - defun, 561
- pfExit?, 561
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 561
 - defun, 561
- pfExitCond, 561
 - calledby pf2Sex1, 598
 - defun, 561
- pfExitExpr, 562
 - calledby pf2Sex1, 598
 - defun, 562
- pfExport, 562
 - calledby npExport, 463
 - calls pfTree, 562
 - defun, 562
- pfExpression, 562
 - calledby pfSymb, 549
 - calls ifcar, 562
 - calls pfLeaf, 562
 - defun, 562
- pfFileName, 534
 - calledby ppos, 647
 - calls poFileName, 534
 - defun, 534
- pfFirst, 562
 - calledby pfLam, 570
 - calledby pfSourceStok, 541
 - defun, 562
- pfFix, 563
 - calledby npFix, 458

- calls pfApplication, 563
- calls pfId, 563
- defun, 563
- pfFlattenApp, 539
 - calledby pfCheckItOut, 537
 - calledby pfCheckMacroOut, 538
 - calledby pfFlattenApp, 540
 - calls pfApplication?, 539
 - calls pfApplicationArg, 540
 - calls pfApplicationOp, 540
 - calls pfCollect1?, 540
 - calls pfFlattenApp, 540
 - defun, 539
- pfForin, 564
 - calledby npForIn, 470
 - calls pfTree, 564
 - defun, 564
- pfForin?, 564
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 564
 - defun, 564
- pfForinLhs, 565
 - calledby pf0ForinLhs, 565
 - defun, 565
- pfForinWhole, 565
 - calledby pf2Sex1, 598
 - defun, 565
- pfFree, 563
 - calledby npFree, 466
 - calls pfTree, 563
 - defun, 563
- pfFree?, 563
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 563
 - defun, 563
- pfFreeItems, 564
 - calledby pf0FreeItems, 564
 - defun, 564
- pfFromDom, 565
 - calledby npFromdom1, 496
 - calledby npFromdom, 495
 - calls pfApplication?, 565
 - calls pfApplicationArg, 565
 - calls pfApplicationOp, 565
 - calls pfApplication, 565
 - calls pfFromdom, 565
 - defun, 565
- pfFromdom, 566
 - calledby pfFromDom, 565
 - calls pfTree, 566
 - defun, 566
- pfFromdom?, 566
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 566
 - defun, 566
- pfFromdomDomain, 567
 - calledby pf2Sex1, 598
 - defun, 567
- pfFromdomWhat, 566
 - calledby pf2Sex1, 598
 - defun, 566
- pfGlobalLinePosn, 533
 - calledby syIgnoredFromTo, 484
 - calls poGlobalLinePosn, 533
 - defun, 533
- pfHide, 567
 - calledby npAngleBared, 479
 - calls pfTree, 567
 - defun, 567
- pfId, 544
 - calledby pfFix, 563
 - calledby pfSuch, 542
 - calledby pfTaggedToTyped, 587
 - calls pfLeaf, 544
 - defun, 544
- pfId?, 544
 - calledby mac0MLambdaApply, 520
 - calledby mac0SubstituteOuter, 527
 - calledby macExpand, 519
 - calledby macMacro, 526
 - calledby pfCheckId, 539
 - calledby pfCheckItOut, 537
 - calledby pfCheckMacroOut, 538
 - calledby pfCollect1?, 540
 - calledby pfSexpr,strip, 548
 - calledby pfTaggedToTyped, 587
 - calls pfAbSynOp?, 544
 - defun, 544
- pfIdPos, 544
 - calledby pfBraceBar, 555
 - calledby pfBrace, 555
 - calledby pfBracketBar, 556

- calledby pfBracket, 555
 - calls pfLeaf, 544
 - defun, 544
- pfIdSymbol, 545
 - calledby macId, 523
 - calledby macLambdaParameterHandling, 528
 - calledby macMacro, 526
 - calledby macSubstituteId, 528
 - calledby pf2Sex1, 598
 - calledby pfCollect1?, 540
 - calledby pfInfApplication, 569
 - calledby pfSexpr,strip, 548
 - calls tokPart, 545
 - defun, 545
- pfIf, 567
 - calledby npElse, 489
 - calledby pfIfThenOnly, 568
 - calls pfTree, 567
 - defun, 567
- pfIf?, 567
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 567
 - defun, 567
- pfIfCond, 568
 - calledby pf2Sex1, 598
 - calledby pfTweakIf, 588
 - defun, 568
- pfIfElse, 568
 - calledby pf2Sex1, 598
 - calledby pfTweakIf, 588
 - defun, 568
- pfIfThen, 568
 - calledby pf2Sex1, 598
 - calledby pfTweakIf, 588
 - defun, 568
- pfIfThenOnly, 568
 - calledby npElse, 489
 - calls pfIf, 568
 - calls pfNothing, 568
 - defun, 568
- pfImmediate?
 - calledby ppos, 647
- pfImport, 569
 - calledby npImport, 473
 - calls pfTree, 569
- defun, 569
- pfInfApplication, 569
 - calledby npInterval, 492
 - calledby npLeftAssoc, 499
 - calledby npRightAssoc, 498
 - calledby npSynthetic, 491
 - calledby pfSuch, 542
 - calledby pfTaggedToTyped, 588
 - calls pfAnd, 569
 - calls pfApplication, 569
 - calls pfIdSymbol, 569
 - calls pfListOf, 569
 - calls pfOr, 569
 - calls pfTuple, 569
 - defun, 569
- pfInline, 570
 - calledby npInline, 466
 - calls pfTree, 570
 - defun, 570
- pfIterate, 569
 - calledby npIterate, 467
 - calls pfTree, 569
 - defun, 569
- pfIterate?, 569
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 569
 - defun, 569
- pfLam, 570
 - calledby npLambda, 441
 - calls pfAbSynOp?, 570
 - calls pfFirst, 570
 - calls pfLambda, 570
 - calls pfNothing, 570
 - calls pfSecond, 570
 - defun, 570
- pfLambda, 571
 - calledby pfLam, 570
 - calledby pfPushBody, 547
 - calls pfTree, 571
 - defun, 571
- pfLambda2Sex, 616
 - calledby pf2Sex1, 599
 - calls pfLambdaTran, 616
 - defun, 616
- pfLambda?, 571
 - calledby mac0SubstituteOuter, 527

- calledby macExpand, 518
- calledby macLambdaParameterHandling, 528
- calledby pf2Sex1, 599
- calledby pfLambdaTran, 614
- calls pfAbSynOp?, 571
- defun, 571
- pfLambdaArgs, 572
 - calledby pf0LambdaArgs, 572
 - defun, 572
- pfLambdaBody, 571
 - calledby pfLambdaTran, 614
 - defun, 571
- pfLambdaRets, 571
 - calledby pfLambdaTran, 614
 - defun, 571
- pfLambdaTran, 614
 - calledby pfDefinition2Sex, 613
 - calledby pfLambda2Sex, 616
 - calls pf0LambdaArgs, 614
 - calls pf2Sex1, 614
 - calls pfCollectArgTran, 614
 - calls pfLambda?, 614
 - calls pfLambdaBody, 614
 - calls pfLambdaRets, 614
 - calls pfNothing?, 614
 - calls pfTyped?, 614
 - calls pfTypedId, 614
 - calls pfTypedType, 614
 - calls systemError, 614
 - defun, 614
- pfLeaf, 545
 - calledby macLambdaParameterHandling, 528
 - calledby pfCopyWithPos, 534
 - calledby pfDocument, 544
 - calledby pfExpression, 562
 - calledby pfIdPos, 544
 - calledby pfId, 544
 - calledby pfSymbol, 549
 - calls ifcar, 545
 - calls pfNoPosition, 545
 - calls tokConstruct, 545
 - defun, 545
- pfLeaf?, 545
 - calledby mac0SubstituteOuter, 527
- calledby macLambdaParameterHandling, 527
- calledby pfCopyWithPos, 534
- calledby pfMapParts, 534
- calledby pfSexpr,strip, 548
- calledby pfSourcePosition, 536
- calledby pfSourceStok, 541
- calledby pfSymb, 549
- calls pfAbSynOp, 545
- defun, 545
- pfLeafPosition, 546
 - calledby macLambdaParameterHandling, 528
 - calledby pfSourcePosition, 536
 - calls tokPosn, 546
 - defun, 546
- pfLeafToken, 546
 - calledby pfLiteral2Sex, 602
 - calls tokPart, 546
 - defun, 546
- pfLhsRule2Sex, 617
 - calledby pfRule2Sex, 616
 - calls pf2Sex1, 617
 - uses \$insideRule, 617
 - defun, 617
- pfLinePosn, 533
 - calledby ppos, 647
 - calls poLinePosn, 533
 - defun, 533
- pfListOf, 543
 - calledby npAssignVariable, 510
 - calledby npBPileDefinition, 480
 - calledby npEnclosed, 504
 - calledby npExpress, 472
 - calledby npListing, 447
 - calledby npParse, 433
 - calledby npRecoverTrap, 483
 - calledby npSigItemList, 447
 - calledby npTypeVariable, 448
 - calledby npVariable, 506
 - calledby pfCheckArg, 539
 - calledby pfCheckItOut, 537
 - calledby pfEnSequence, 561
 - calledby pfInfApplication, 569
 - calledby pfLoop1, 573
 - calledby pfLp, 574

- calledby pfSequenceToList, 537
- calledby pfTransformArg, 542
- calledby pfTupleListOf, 590
- calledby pfTweakIf, 588
- calledby pfUnSequence, 591
- calls pfTree, 543
- defun, 543
- pfLiteral2Sex, 602
 - calledby pf2Sex1, 598
 - calls float2Sex, 602
 - calls keyedSystemError, 602
 - calls pfLeafToken, 602
 - calls pfLiteralClass, 602
 - calls pfLiteralString, 602
 - calls pfSymbolSymbol, 602
 - uses \$insideRule, 602
 - defun, 602
- pfLiteral?, 546
 - calledby pf2Sex1, 598
 - calledby pfSexpr,strip, 548
 - calls pfAbSynOp, 546
 - defun, 546
- pfLiteralClass, 546
 - calledby pfLiteral2Sex, 602
 - calls pfAbSynOp, 546
 - defun, 546
- pfLiteralString, 547
 - calledby pfLiteral2Sex, 602
 - calledby pfSexpr,strip, 548
 - calls tokPart, 547
 - defun, 547
- pfLocal, 572
 - calledby npLocal, 466
 - calls pfTree, 572
 - defun, 572
- pfLocal?, 572
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 572
 - defun, 572
- pfLocalItems, 573
 - calledby pf0LocalItems, 573
 - defun, 573
- pfLoop, 573
 - calledby pfLoop1, 573
 - calledby pfLp, 574
 - calls pfTree, 573
 - defun, 573
- pfLoop1, 573
 - calledby npLoop, 467
 - calls pfDo, 573
 - calls pfListOf, 573
 - calls pfLoop, 573
 - defun, 573
- pfLoop?, 574
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 574
 - defun, 574
- pfLoopIterators, 574
 - defun, 574
- pfLp, 574
 - calledby npLoop, 467
 - calls pfDo, 574
 - calls pfListOf, 574
 - calls pfLoop, 574
 - defun, 574
- pfMacro, 575
 - calledby macMacro, 526
 - calledby npMdef, 457
 - calls pfTree, 575
 - defun, 575
- pfMacro?, 575
 - calledby macExpand, 519
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 575
 - defun, 575
- pfMacroLhs, 575
 - calledby macMacro, 525
 - defun, 575
- pfMacroRhs, 575
 - calledby macMacro, 526
 - defun, 575
- pfMapParts, 534
 - calledby macApplication, 519
 - calledby macExpand, 519
 - calledby macLambda,mac, 525
 - calledby macWhere,mac, 524
 - calls pfAbSynOp, 535
 - calls pfLeaf?, 534
 - calls pfParts, 534
 - calls pfTree, 535
 - defun, 534
- pfMLambda, 576

- calledby pfPushMacroBody, 541
- calls pfTree, 576
- defun, 576
- pfMLambda?, 576
 - calledby macApplication, 519
 - calledby macLambdaParameterHandling, 528
 - calledby macMacro, 526
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 576
 - defun, 576
- pfMLambdaArgs, 576
 - defun, 576
- pfMLambdaBody, 577
 - calledby mac0GetName, 522
 - calledby mac0MLambdaApply, 520
 - defun, 577
- pfname, 384
 - calledby porigin, 381
 - calledby thefname, 384
 - calls PathnameString, 384
 - defun, 384
- pfNoPosition, 704
 - calledby pfLeaf, 545
 - calledby tokPosn, 703
 - calls poNoPosition, 704
 - defun, 704
- pfNoPosition?, 702
 - calledby ppos, 647
 - calledby tokConstruct, 701
 - calls poNoPosition?, 702
 - defun, 702
- pfNot?, 577
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 577
 - defun, 577
- pfNotArg, 577
 - calledby pf2Sex1, 599
 - defun, 577
- pfNothing, 543
 - calledby macMacro, 526
 - calledby npAdd, 451
 - calledby npBreak, 467
 - calledby npDefaultValue, 487
 - calledby npIterate, 467
 - calledby npLocalDecl, 465
 - calledby npPileBracketed, 481
 - calledby npPrimary2, 450
 - calledby npQualType, 474
 - calledby npReturn, 471
 - calledby npSignature, 446
 - calledby npVariableName, 506
 - calledby npWith, 443
 - calledby pfAdd, 550
 - calledby pfCheckItOut, 537
 - calledby pfIfThenOnly, 568
 - calledby pfLam, 570
 - calledby pfPushBody, 547
 - calledby pfReturnNoName, 583
 - calledby pfTaggedToTyped1, 542
 - calledby pfTaggedToTyped, 587
 - calls pfTree, 543
 - defun, 543
- pfNothing?, 543
 - calledby macMacro, 526
 - calledby pf2Sex1, 598
 - calledby pfLambdaTran, 614
 - calledby pfTweakIf, 588
 - calls pfAbSynOp?, 543
 - defun, 543
- pfNovalue, 577
 - calledby npItem, 434
 - calledby npVoid, 471
 - calls pfTree, 577
 - defun, 577
- pfNovalue?, 578
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 578
 - defun, 578
- pfNovalueExpr, 578
 - calledby pf2Sex1, 599
 - defun, 578
- pfOp2Sex, 606
 - calledby pfApplication2Sex, 603
 - calls pf2Sex1, 606
 - calls pfSymbol?, 606
 - calls pmDontQuote?, 606
 - uses \$insideRule, 606
 - uses \$quotedOpList, 606
 - defun, 606
- pfOr, 578
 - calledby pfInfApplication, 569

- calls pfTree, 578
- defun, 578
- pfOr?, 578
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 578
 - defun, 578
- pfOrLeft, 579
 - calledby pf2Sex1, 599
 - defun, 579
- pform
 - calledby mac0InfiniteExpansion, 521
 - calledby mac0MLambdaApply, 520
 - calledby macMacro, 526
 - calledby phMacro, 518
- pfOrRight, 579
 - calledby pf2Sex1, 599
 - defun, 579
- pfParen, 579
 - calledby npParened, 478
 - defun, 579
- pfParts, 547
 - calledby mac0SubstituteOuter, 527
 - calledby macLambdaParameterHandling, 528
 - calledby npDefaultDecl, 462
 - calledby npLocalDecl, 465
 - calledby npSDefaultItem, 461
 - calledby npSLocalItem, 464
 - calledby npSQualTypelist, 473
 - calledby npSigDecl, 449
 - calledby npSigItemList, 447
 - calledby pf0AssignLhsItems, 554
 - calledby pf0DefinitionLhsItems, 560
 - calledby pf0ForinLhs, 565
 - calledby pf0FreeItems, 564
 - calledby pf0LambdaArgs, 572
 - calledby pf0LocalItems, 573
 - calledby pf0LoopIterators, 574
 - calledby pf0MLambdaArgs, 576
 - calledby pf0SequenceArgs, 585
 - calledby pf0TupleParts, 591
 - calledby pf0WhereContext, 593
 - calledby pfCollect2Sex, 612
 - calledby pfCollectArgTran, 615
 - calledby pfCopyWithPos, 534
 - calledby pfMapParts, 534
 - calledby pfSexpr,strip, 548
 - calledby pfSourcePosition, 536
 - calledby pfSourceStok, 541
 - calledby pfWDec, 592
 - defun, 547
- pfPile, 547
 - calledby npPileBracketed, 481
 - defun, 547
- pfPretend, 579
 - calledby npPretend, 512
 - calls pfTree, 579
 - defun, 579
- pfPretend?, 580
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 580
 - defun, 580
- pfPretendExpr, 580
 - calledby pf2Sex1, 598
 - defun, 580
- pfPretendType, 580
 - calledby pf2Sex1, 598
 - defun, 580
- pfPrintSrcLines
 - calledby displayParserMacro, 804
- pfPushBody, 547
 - calledby npDef, 480
 - calledby pfPushBody, 547
 - calls pfLambda, 547
 - calls pfNothing, 547
 - calls pfPushBody, 547
 - defun, 547
- pfPushMacroBody, 541
 - calledby npMdef, 457
 - calledby pfPushMacroBody, 541
 - calls pfMLambda, 541
 - calls pfPushMacroBody, 541
 - defun, 541
- pfQualType, 580
 - calledby npQualType, 474
 - calls pfTree, 580
 - defun, 580
- pfRestrict, 581
 - calledby npRestrict, 513
 - calls pfTree, 581
 - defun, 581
- pfRestrict?, 581

- calledby pf2Sex1, 599
- calls pfAbSynOp?, 581
- defun, 581
- pfRestrictExpr, 581
 - calledby pf2Sex1, 599
 - defun, 581
- pfRestrictType, 581
 - calledby pf2Sex1, 599
 - defun, 581
- pfRetractTo, 582
 - calledby npColonQuery, 512
 - calls pfTree, 582
 - defun, 582
- pfReturn, 582
 - calledby npReturn, 471
 - calledby pfReturnNoName, 583
 - calls pfTree, 582
 - defun, 582
- pfReturn?, 582
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 582
 - defun, 582
- pfReturnExpr, 582
 - calledby pf2Sex1, 599
 - defun, 582
- pfReturnNoName, 583
 - calledby npReturn, 471
 - calls pfNothing, 583
 - calls pfReturn, 583
 - defun, 583
- pfReturnTyped, 583
 - calledby npLambda, 441
 - calls pfTree, 583
 - defun, 583
- pfRhsRule2Sex, 617
 - calledby pfRule2Sex, 616
 - calls pf2Sex1, 617
 - uses \$insideRule, 617
 - defun, 617
- pfRule, 583
 - calledby npSingleRule, 486
 - calls pfTree, 583
 - defun, 583
- pfRule2Sex, 616
 - calledby pf2Sex1, 599
 - calls pfLhsRule2Sex, 616
 - calls pfRhsRule2Sex, 616
 - calls pfRuleLhsItems, 616
 - calls pfRuleRhs, 616
 - calls ruleLhsTran, 616
 - calls rulePredicateTran, 616
 - uses \$multiVarPredicateList, 616
 - uses \$predicateList, 616
 - uses \$quotedOpList, 616
 - defun, 616
- pfRule?, 584
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 584
 - defun, 584
- pfRuleLhsItems, 584
 - calledby pfRule2Sex, 616
 - defun, 584
- pfRuleRhs, 584
 - calledby pfRule2Sex, 616
 - defun, 584
- pfSecond, 584
 - calledby pfLam, 570
 - defun, 584
- pfSequence, 585
 - calledby npBPileDefinition, 480
 - calledby pfEnSequence, 561
 - calls pfTree, 585
 - defun, 585
- pfSequence2Sex, 608
 - calledby pf2Sex1, 598
 - calls pf0SequenceArgs, 608
 - calls pf2Sex1, 608
 - uses \$insideSEQ, 608
 - defun, 608
- pfSequence2Sex0, 608
 - calledby pfSequence2Sex0, 609
 - calls pfSequence2Sex0, 609
 - defun, 608
- pfSequence?, 585
 - calledby pf2Sex1, 598
 - calledby pfSequenceToList, 536
 - calledby pfUnSequence, 591
 - calls pfAbSynOp?, 585
 - defun, 585
- pfSequenceArgs, 585
 - calledby pf0SequenceArgs, 585
 - calledby pfSequenceToList, 536

- defun, 585
- pfSequenceToList, 536
 - calledby npDefinition, 459
 - calls pfListOf, 537
 - calls pfSequence?, 536
 - calls pfSequenceArgs, 536
 - defun, 536
- pfSexpr, 548
 - calledby pfSymb, 549
 - calls pfSexpr,strip, 548
 - defun, 548
- pfSexpr,strip, 548
 - calledby pfSexpr,strip, 548
 - calledby pfSexpr, 548
 - calls pf0TupleParts, 548
 - calls pfAbSynOp, 548
 - calls pfApplication?, 548
 - calls pfApplicationArg, 548
 - calls pfApplicationOp, 548
 - calls pfId?, 548
 - calls pfIdSymbol, 548
 - calls pfLeaf?, 548
 - calls pfLiteral?, 548
 - calls pfLiteralString, 548
 - calls pfParts, 548
 - calls pfSexpr,strip, 548
 - calls pfTuple?, 548
 - calls tokPart, 548
 - defun, 548
- pfSourcePosition, 536
 - calledby mac0ExpandBody, 521
 - calledby mac0MLambdaApply, 520
 - calledby macId, 523
 - calledby macMacro, 526
 - calledby pfSourcePosition, 536
 - calls pfLeaf?, 536
 - calls pfLeafPosition, 536
 - calls pfParts, 536
 - calls pfSourcePosition, 536
 - calls poNoPosition?, 536
 - uses \$nopus, 536
 - defun, 536
- pfSourceStok, 541
 - calledby npTrapForm, 505
 - calledby pfSourceStok, 541
 - calls pfFirst, 541
 - calls pfLeaf?, 541
 - calls pfParts, 541
 - calls pfSourceStok, 541
 - defun, 541
- pfSpread, 537
 - calledby npDefaultDecl, 462
 - calledby npLocalDecl, 465
 - calledby npSigDecl, 449
 - calls pfTyped, 537
 - defun, 537
- pfSuch, 542
 - calledby pfCollectVariable1, 540
 - calledby pfTaggedToTyped, 587
 - calls pfId, 542
 - calls pfInfApplication, 542
 - defun, 542
- pfSuchthat, 586
 - calledby npSuchThat, 469
 - calls pfTree, 586
 - defun, 586
- pfSuchThat2Sex, 605
 - calledby pfApplication2Sex, 604
 - calls pf0TupleParts, 605
 - calls pf2Sex1, 605
 - calls pf2Sex, 605
 - uses \$predicateList, 605
 - defun, 605
- pfSuchthat?, 586
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 586
 - defun, 586
- pfSuchthatCond, 586
 - calledby pf2Sex1, 599
 - defun, 586
- pfSymb, 549
 - calledby npConstTok, 477
 - calledby npDDInfKey, 500
 - calledby npInfixOperator, 452
 - calls ifcar, 549
 - calls pfExpression, 549
 - calls pfLeaf?, 549
 - calls pfSexpr, 549
 - calls pfSymbol, 549
 - calls tokPart, 549
 - defun, 549
- pfSymbol, 549

- calledby pfSymb, 549
- calls ifcar, 549
- calls pfLeaf, 549
- defun, 549
- pfSymbol?, 550
 - calledby pf2Sex1, 598
 - calledby pfOp2Sex, 606
 - calls pfAbSynOp?, 550
 - defun, 550
- pfSymbolSymbol, 550
 - calledby pf2Sex1, 598
 - calledby pfLiteral2Sex, 602
 - calls tokPart, 550
 - defun, 550
- pfTagged, 586
 - calledby npTagged, 510
 - calls pfTree, 586
 - defun, 586
- pfTagged?, 587
 - calledby pf2Sex1, 598
 - calledby pfCheckItOut, 537
 - calledby pfTaggedToTyped, 587
 - calls pfAbSynOp?, 587
 - defun, 587
- pfTaggedExpr, 587
 - calledby pf2Sex1, 598
 - calledby pfCheckItOut, 537
 - calledby pfTaggedToTyped, 587
 - defun, 587
- pfTaggedTag, 587
 - calledby pf2Sex1, 598
 - calledby pfCheckItOut, 537
 - calledby pfTaggedToTyped, 587
 - defun, 587
- pfTaggedToTyped, 587
 - calledby pfCheckItOut, 537
 - calledby pfCollectVariable1, 540
 - calledby pfTaggedToTyped1, 542
 - calls pfId?, 587
 - calls pfId, 587
 - calls pfInfApplication, 588
 - calls pfNothing, 587
 - calls pfSuch, 587
 - calls pfTagged?, 587
 - calls pfTaggedExpr, 587
 - calls pfTaggedTag, 587
 - calls pfTyped, 587
 - defun, 587
- pfTaggedToTyped1, 542
 - calledby pfCheckItOut, 537
 - calledby pfTransformArg, 542
 - calls pfCollect1?, 542
 - calls pfCollectVariable1, 542
 - calls pfDefinition?, 542
 - calls pfNothing, 542
 - calls pfTaggedToTyped, 542
 - calls pfTyped, 542
 - defun, 542
- pfTransformArg, 542
 - calledby pfCheckItOut, 537
 - calls pf0TupleParts, 542
 - calls pfListOf, 542
 - calls pfTaggedToTyped1, 542
 - calls pfTuple?, 542
 - defun, 542
- pfTree, 550
 - calledby pfAdd, 550
 - calledby pfAnd, 551
 - calledby pfApplication, 551
 - calledby pfAssign, 553
 - calledby pfAttribute, 551
 - calledby pfBreak, 556
 - calledby pfCoerceto, 557
 - calledby pfCollect, 558
 - calledby pfCopyWithPos, 534
 - calledby pfDefinition, 559
 - calledby pfDo, 560
 - calledby pfExit, 561
 - calledby pfExport, 562
 - calledby pfForin, 564
 - calledby pfFree, 563
 - calledby pfFromdom, 566
 - calledby pfHide, 567
 - calledby pfIf, 567
 - calledby pfImport, 569
 - calledby pfInline, 570
 - calledby pfIterate, 569
 - calledby pfLambda, 571
 - calledby pfListOf, 543
 - calledby pfLocal, 572
 - calledby pfLoop, 573
 - calledby pfMLambda, 576

- calledby pfMacro, 575
- calledby pfMapParts, 535
- calledby pfNothing, 543
- calledby pfNoValue, 577
- calledby pfOr, 578
- calledby pfPretend, 579
- calledby pfQualType, 580
- calledby pfRestrict, 581
- calledby pfRetractTo, 582
- calledby pfReturnTyped, 583
- calledby pfReturn, 582
- calledby pfRule, 583
- calledby pfSequence, 585
- calledby pfSuchthat, 586
- calledby pfTagged, 586
- calledby pfTuple, 590
- calledby pfTweakIf, 588
- calledby pfTyped, 588
- calledby pfTyping, 589
- calledby pfWDeclare, 592
- calledby pfWhere, 592
- calledby pfWhile, 593
- calledby pfWith, 594
- calledby pfWrong, 594
- defun, 550
- pfTuple, 590
 - calledby npEnclosed, 504
 - calledby pfEnSequence, 561
 - calledby pfInfApplication, 569
 - calledby pfTupleListOf, 590
 - calls pfTree, 590
 - defun, 590
- pfTuple?, 590
 - calledby pf0FlattenSyntacticTuple, 535
 - calledby pf2Sex1, 598
 - calledby pfApplication2Sex, 604
 - calledby pfCheckArg, 539
 - calledby pfCheckItOut, 537
 - calledby pfSexpr,strip, 548
 - calledby pfTransformArg, 542
 - calls pfAbSynOp?, 590
 - defun, 590
- pfTupleListOf, 590
 - calledby npTuple, 438
 - calls pfListOf, 590
 - calls pfTuple, 590
- defun, 590
- pfTupleParts, 591
 - calledby pf0TupleParts, 591
 - defun, 591
- pfTweakIf, 588
 - calledby npWConditional, 488
 - calls pfIfCond, 588
 - calls pfIfElse, 588
 - calls pfIfThen, 588
 - calls pfListOf, 588
 - calls pfNothing?, 588
 - calls pfTree, 588
 - defun, 588
- pfTyped, 588
 - calledby npDecl, 507
 - calledby npVariableName, 506
 - calledby pfCheckItOut, 537
 - calledby pfCollectVariable1, 540
 - calledby pfSpread, 537
 - calledby pfTaggedToTyped1, 542
 - calledby pfTaggedToTyped, 587
 - calls pfTree, 588
 - defun, 588
- pfTyped?, 589
 - calledby pf2Sex1, 599
 - calledby pfLambdaTran, 614
 - calls pfAbSynOp?, 589
 - defun, 589
- pfTypedId, 589
 - calledby macLambdaParameterHandling, 528
 - calledby pf2Sex1, 599
 - calledby pfCollectVariable1, 540
 - calledby pfLambdaTran, 614
 - defun, 589
- pfTypedType, 589
 - calledby pf2Sex1, 599
 - calledby pfCollectVariable1, 541
 - calledby pfLambdaTran, 614
 - defun, 589
- pfTyping, 589
 - calledby npTyping, 461
 - calls pfTree, 589
 - defun, 589
- pfUnSequence, 591
 - calledby npCategoryL, 445

- calledby npDefaultItemList, 461
- calledby npLocalItemList, 464
- calledby npQualTypelist, 473
- calls pf0SequenceArgs, 591
- calls pfAppend, 591
- calls pfListOf, 591
- calls pfSequence?, 591
- defun, 591
- pfWDec, 591
 - calledby npSignature, 446
 - calls pfParts, 592
 - calls pfWDeclare, 591
 - defun, 591
- pfWDeclare, 592
 - calledby pfWDec, 591
 - calls pfTree, 592
 - defun, 592
- pfWhere, 592
 - calledby npLetQualified, 459
 - calledby npQualified, 439
 - calls pfTree, 592
 - defun, 592
- pfWhere?, 592
 - calledby macExpand, 518
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 592
 - defun, 592
- pfWhereContext, 593
 - calledby pf0WhereContext, 593
 - defun, 593
- pfWhereExpr, 593
 - calledby pf2Sex1, 599
 - defun, 593
- pfWhile, 593
 - calledby npWhile, 470
 - calls pfTree, 593
 - defun, 593
- pfWhile?, 594
 - calledby pf2Sex1, 598
 - calls pfAbSynOp?, 594
 - defun, 594
- pfWhileCond, 594
 - calledby pf2Sex1, 598
 - defun, 594
- pfWith, 594
 - calledby npWith, 443
- calls pfTree, 594
- defun, 594
- pfWrong, 594
 - calledby npParse, 433
 - calledby npRecoverTrap, 483
 - calls pfTree, 594
 - defun, 594
- pfWrong?, 595
 - calledby pf2Sex1, 599
 - calls pfAbSynOp?, 595
 - defun, 595
- phInterpret, 359
 - calls intInterpretPform, 359
 - calls ncEltQ, 359
 - calls ncPutQ, 359
 - defun, 359
- phIntReportMsgs, 358
 - calls ncEltQ, 359
 - calls ncPutQ, 359
 - calls processMsgList, 359
 - uses \$erMsgToss, 359
 - defun, 358
- phiRatapprox, 1244
 - calledby lngammaRatapprox, 1244
 - calls horner, 1244
 - defun, 1244
- phMacro, 517
 - calls macroExpanded, 518
 - calls ncEltQ, 518
 - calls ncPutQ, 518
 - calls pform, 518
 - defun, 517
- phParse, 358
 - calls ncPutQ, 358
 - defun, 358
- pileCforest, 630
 - calledby insertpile, 625
 - calledby pileCtree, 630
 - calls enPile, 630
 - calls separatePiles, 630
 - calls tokPart, 630
 - defun, 630
- pileColumn, 627
 - calledby eqpileTree, 629
 - calledby pileTree, 627
 - calls tokPosn, 627

- defun, 627
- pileCtree, 630
 - calledby pileForests, 628
 - calls dqAppend, 630
 - calls pileCforest, 630
 - defun, 630
- pileForest, 628
 - calledby pileForests, 627
 - calls pileForest1, 628
 - calls pileTree, 628
 - defun, 628
- pileForest1, 628
 - calledby pileForest1, 629
 - calledby pileForest, 628
 - calls eqpileTree, 628
 - calls pileForest1, 629
 - defun, 628
- pileForests, 627
 - calledby eqpileTree, 629
 - calledby pileForests, 628
 - calledby pileTree, 627
 - calls npNull, 628
 - calls pileCtree, 628
 - calls pileForests, 628
 - calls pileForest, 627
 - defun, 627
- pilePlusComment, 626
 - calledby insertpile, 625
 - calledby pilePlusComments, 626
 - calls tokType, 626
 - defun, 626
- pilePlusComments, 626
 - calledby insertpile, 625
 - calledby pilePlusComments, 626
 - calls npNull, 626
 - calls pilePlusComments, 626
 - calls pilePlusComment, 626
 - defun, 626
- pileTree, 627
 - calledby insertpile, 625
 - calledby pileForest, 628
 - calls npNull, 627
 - calls pileColumn, 627
 - calls pileForests, 627
 - defun, 627
- PiMinusLogSinPi, 1249
 - calledby clngammacase1, 1248
 - calls cgammaG, 1249
 - calls logH, 1249
 - defun, 1249
- placep, 1297
 - calledby unwritable?, 916
 - calledby writify,writifyInner, 917
 - uses *read-place-holder*, 1297
 - defun, 1297
- pluralize
 - calledby dbConsHeading, 1649
 - calledby dbShowConditions, 1648
 - calledby kePageDisplay, 1608
- pluralSay
 - calledby kePage, 1606
- pmDontQuote?, 607
 - calledby pfOp2Sex, 606
 - defun, 607
- pmTransFilter
 - calledby dbShowCons, 1641
 - calledby koaPageFilterByName, 1634
- pname, 1235
 - calledby clearCmdParts, 841
 - calledby coerceTraceArgs2E, 127
 - calledby coerceTraceFunValue2E, 129
 - calledby displayType, 800
 - calledby displayValue, 799
 - calledby fixObjectForPrinting, 796
 - calledby gensymInt, 927
 - calledby getAliasIfTracedMapParameter, 140
 - calledby getStFromMsg, 645
 - calledby hasOption, 791
 - calledby isSharpVarWithNum, 109
 - calledby kcnPage, 1622
 - calledby letPrint2, 110
 - calledby letPrint3, 111
 - calledby letPrint, 108
 - calledby mac0InfiniteExpansion,name, 522
 - calledby newHelpSpad2Cmd, 881
 - calledby npMissing, 444
 - calledby reportUndo, 60
 - calledby rwrite, 914, 916
 - calledby selectOption, 819
 - calledby setFortDir, 998
 - calledby setFortTmpDir, 996

- calledby setOutputCharacters, 1039
- calledby stupidIsSpadFunction, 115
- calledby trace3, 84
- calledby undo, 52
- defun, 1235
- poCharPosn, 667
 - calledby compareposns, 660
 - calledby pfCharPosn, 533
 - calledby posPointers, 668
 - calledby processChPosesForOneLine, 666
 - calledby putFTText, 669
 - defun, 667
- poFileName, 650
 - calledby decideHowMuch, 649
 - calledby pfFileName, 534
 - calls lnFileName, 650
 - calls poGetLineObject, 651
 - defun, 650
- poGetLineObject, 651
 - calledby poFileName, 651
 - calledby poGlobalLinePosn, 364
 - calledby poLinePosn, 651
 - calledby poPosImmediate?, 650
 - defun, 651
- poGlobalLinePosn, 364
 - calledby compareposns, 660
 - calledby listDecideHowMuch, 651
 - calledby makeMsgFromLine, 661
 - calledby ncloopDQlines, 364
 - calledby pfGlobalLinePosn, 533
 - calledby processMsgList, 659
 - calledby thisPosIsEqual, 664
 - calledby thisPosIsLess, 664
 - calls lnGlobalNum, 364
 - calls ncBug, 364
 - calls poGetLineObject, 364
 - defun, 364
- poLinePosn, 651
 - calledby decideHowMuch, 650
 - calledby makeMsgFromLine, 661
 - calledby pfLinePosn, 533
 - calls lnLocalNum, 651
 - calls poGetLineObject, 651
 - defun, 651
- poNopos?, 650
 - calledby decideHowMuch, 649
 - calledby erMsgSep, 660
 - calledby listDecideHowMuch, 651
 - calledby poPosImmediate?, 650
 - calledby thisPosIsEqual, 664
 - calledby thisPosIsLess, 664
 - defun, 650
- poNoPosition, 704
 - calledby pfNoPosition, 704
 - uses \$nopos, 704
 - defun, 704
- poNoPosition?, 703
 - calledby pfNoPosition?, 702
 - calledby pfSourcePosition, 536
 - calls eqcar, 703
 - defun, 703
- poPosImmediate?, 650
 - calledby decideHowMuch, 649
 - calledby listDecideHowMuch, 651
 - calls lnImmediate?, 650
 - calls poGetLineObject, 650
 - calls poNopos?, 650
 - defun, 650
- porigin, 381
 - calledby inclmsgFileCycle, 385
 - calledby ppos, 647
 - calls pname, 381
 - defun, 381
- posend, 420
 - calledby scanW, 419
 - defun, 420
- position
 - calledby coerceBranch2Union, 766
 - calledby dbSubConform, 1640
 - calledby kArgPage, 1602
- posPointers, 668
 - calledby processChPosesForOneLine, 666
 - calls getMsgPos2, 668
 - calls getMsgPos, 668
 - calls ifcar, 668
 - calls insertPos, 668
 - calls poCharPosn, 668
 - uses getMsgFTTag, 668
 - defun, 668
- poundsign
 - calledby dewritify, dewritifyInner, 922
 - calledby displaySetVariableSettings, 965

- calledby getTraceOptions, 116
- calledby isDomainOrPackage, 107
- calledby newHelpSpad2Cmd, 881
- calledby set1, 1082
- calledby setOutputLibrary, 972
- calledby trace1, 79
- calledby traceReply, 94
- pp
 - calledby mkConform, 1628
 - calledby reportUndo, 60
- pp2Cols
 - calledby filterAndFormatConstructors, 1136
- ppos, 647
 - calledby getPosStL, 646
 - calls pfCharPosn, 647
 - calls pfFileName, 647
 - calls pfImmediate?, 647
 - calls pfLinePosn, 647
 - calls pfNoPosition?, 647
 - calls porigin, 647
 - defun, 647
- pquit, 937, 938
 - calls pquitSpad2Cmd, 938
 - defun, 938
 - manpage, 937
- pquitSpad2Cmd, 938
 - calledby pquit, 938
 - calls quitSpad2Cmd, 938
 - uses \$quitCommandType, 938
 - defun, 938
- pred2English
 - calledby displayCondition, 804
- prefix2String
 - calledby displayMode, 806
 - calledby displayProperties, 801
 - calledby displayType, 800
 - calledby displayValue, 799
 - calledby evalDomain, 1115
 - calledby spadUntrace, 143
 - calledby traceReply, 95
- previousInterpreterFrame, 33
 - calledby frameSpad2Cmd, 26
 - calls updateCurrentInterpreterFrame, 33
 - calls updateFromCurrentInterpreterFrame, 33
 - uses \$interpreterFrameRing, 33
- defun, 33
- prinmathor0, 151
 - calledby monitorPrintArgs, 148
 - calledby monitorPrintRest, 150
 - calledby monitorPrintValue, 152
 - calledby monitorPrint, 150
 - calls maprinSpecial, 151
 - calls outputTran2, 151
 - uses \$mathTrace, 151
 - uses \$monitorDepth, 151
 - defun, 151
- print
 - calledby letPrint2, 110
 - calledby letPrint3, 111
- printAsTeX, 353
 - uses \$texOutputStream, 353
 - defun, 353
- printDashedLine
 - calledby spadTrace, 133
- printLabelledList, 814
 - calledby printSynonyms, 813
 - calls blankList, 814
 - calls concat, 814
 - calls entryWidth, 814
 - calls fillerSpaces, 814
 - calls sayBrightly, 814
 - calls sayMessage, 814
 - calls substring, 814
 - defun, 814
- printStatisticsSummary, 351
 - calledby recordAndPrint, 349
 - calls sayKeyedMsg, 351
 - calls statisticsSummary, 351
 - uses \$collectOutput, 351
 - defun, 351
- printStorage, 351
 - calledby recordAndPrint, 349
 - calls makeLongSpaceString, 351
 - uses \$collectOutput, 351
 - uses \$interpreterTimedClasses, 351
 - uses \$interpreterTimedNames, 351
 - defun, 351
- printSynonyms, 813
 - calledby npProcessSynonym, 813
 - calledby synonymSpad2Cmd, 1105
 - calledby whatSpad2Cmd, 1131

- calls filterListOfStringsWithFn, 813
- calls printLabelledList, 813
- calls specialChar, 813
- calls synonymsForUserLevel, 813
- uses \$CommandSynonymAlist, 813
- uses \$linelength, 814
- defun, 813
- printTypeAndTime, 351
 - calledby recordAndPrint, 349
 - calls justifyMyType, 352
 - calls makeLongTimeString, 352
 - calls mkObjWrap, 352
 - calls msgText, 352
 - calls objMode, 352
 - calls qcar, 352
 - calls retract, 351, 352
 - calls sameUnionBranch, 352
 - calls sayKeyedMsg, 352
 - uses \$collectOutput, 352
 - uses \$interpreterTimedClasses, 352
 - uses \$interpreterTimedNames, 352
 - uses \$outputLines, 352
 - uses \$printTimeIfTrue, 352
 - uses \$printTypeIfTrue, 352
 - defun, 351
- prior-token
 - usedby token-stack-show, 1157
- probeName, 1170
 - defun, 1170
- processChPosesForOneLine, 666
 - calledby queueUpErrors, 662
 - calls concat, 666
 - calls getMsgFTTag?, 666
 - calls getMsgPos, 666
 - calls getMsgPrefix, 666
 - calls makeLeaderMsg, 666
 - calls poCharPosn, 666
 - calls posPointers, 666
 - calls putFTText, 666
 - calls setMsgPrefix, 666
 - calls size, 666
 - uses \$preLength, 666
 - defun, 666
- processInteractive, 342
 - calledby intInterpretPform, 360
 - calledby kisValidType, 1626
 - calledby topLevelInterpEval, 1626
 - calls clrhash, 342
 - calls initializeTimedNames, 342
 - calls processInteractive1, 342
 - calls qcar, 342
 - calls reportInstantiations, 342
 - calls updateHist, 342
 - calls writeHistModesAndValues, 342
 - uses \$Coerce, 342
 - uses \$ProcessInteractiveValue, 343
 - uses \$StreamFrame, 342
 - uses \$analyzingMapList, 342
 - uses \$compErrorMessageStack, 342
 - uses \$compilingLoop, 342
 - uses \$compilingMap, 342
 - uses \$declaredMode, 342
 - uses \$defaultFortVar, 343
 - uses \$domPvar, 343
 - uses \$fortVar, 342
 - uses \$freeVars, 342
 - uses \$inRetract, 343
 - uses \$instantCanCoerceCount, 342
 - uses \$instantCoerceCount, 342
 - uses \$instantMmCondCount, 342
 - uses \$instantRecord, 343
 - uses \$interpOnly, 342
 - uses \$interpreterTimedClasses, 343
 - uses \$interpreterTimedNames, 343
 - uses \$lastLineInSEQ, 342
 - uses \$localVars, 342
 - uses \$mapList, 342
 - uses \$minivectorCode, 342
 - uses \$minivectorNames, 342
 - uses \$minivector, 342
 - uses \$op, 342
 - uses \$reportInstantiations, 343
 - uses \$timeGlobalName, 342
 - uses \$whereCacheList, 342
 - defun, 342
- processInteractive1, 344
 - calledby processInteractive, 342
 - calls interpretTopLevel, 345
 - calls objMode, 345
 - calls objValUnwrap, 345
 - calls recordAndPrint, 345
 - calls recordFrame, 344

- calls startTimingProcess, 345
- calls stopTimingProcess, 345
- uses \$InteractiveFrame, 345
- uses \$ProcessInteractiveValue, 345
- uses \$e, 345
- defun, 344
- processKeyedError, 643
 - calledby ncBug, 658
 - calledby ncHardError, 642
 - calledby ncSoftError, 642
 - calls CallerName, 643
 - calls getMsgKey, 643
 - calls getMsgPrefix?, 643
 - calls getMsgTag?, 643
 - calls msgImPr?, 643
 - calls msgOutputter, 643
 - calls sayBrightly, 643
 - uses \$ncMsgList, 643
 - defun, 643
- processMsgList, 659
 - calledby phIntReportMsgs, 359
 - calls erMsgSort, 659
 - calls getMsgPos, 659
 - calls listOutputter, 659
 - calls makeMsgFromLine, 659
 - calls poGlobalLinePosn, 659
 - calls queueUpErrors, 659
 - uses \$noRepList, 659
 - uses \$outputList, 659
 - defun, 659
- processSynonymLine, 1107
 - calledby npProcessSynonym, 813
 - calledby synonymSpad2Cmd, 1105
 - calls processSynonymLine, removeKeyFromLine, 1107
 - defun, 1107
- processSynonymLine, removeKeyFromLine, 1107
 - calledby processSynonymLine, 1107
 - calls maxindex, 1107
 - defun, 1107
- processSynonyms, 325
 - calledby doSystemCommand, 786
 - calledby intProcessSynonyms, 324
 - calledby processSynonyms, 325
 - calls concat, 325
 - calls lassoc, 325
 - calls processSynonyms, 325
 - calls rplacstr, 325
 - calls size, 325
 - calls string2id-n, 325
 - calls strpos, 325
 - calls substring, 325
 - uses \$CommandSynonymAlist, 325
 - uses line, 325
 - defun, 325
- protectedEVAL, 339
 - calledby serverReadLine, 336
 - calls resetStackLimits, 339
 - calls sendHTErrorSignal, 339
 - defun, 339
- prTraceNames, 145
 - calls exit, 145
 - calls prTraceNames, fn, 145
 - calls seq, 145
 - uses \$traceNames, 145
 - defun, 145
- prTraceNames, fn, 145
 - calledby prTraceNames, 145
 - calls devaluate, 145
 - calls exit, 145
 - calls isDomainOrPackage, 145
 - calls qcar, 145
 - calls qcdr, 145
 - calls seq, 145
 - defun, 145
- pSearch, 1595
 - calls constructorSearch, 1595
 - defun, 1595
- PsiAsymptotic, 1260
 - calledby PsiBack, 1259
 - calledby rPsiW, 1258
 - calls PsiEps, 1260
 - calls rgammaImpl, 1260
 - uses \$PsiAsymptoticBern, 1260
 - defun, 1260
- PsiAsymptoticBern, 1260
 - defvar, 1260
- PsiAsymptoticOrder, 1259
 - calledby rPsiW, 1258
 - defun, 1259
- PsiBack, 1259
 - calledby rPsiW, 1258

- calls PsiAsymptotic, [1259](#)
- calls PsiIntpart, [1259](#)
- defun, [1259](#)
- PsiEps, [1261](#)
 - calledby PsiAsymptotic, [1260](#)
 - defun, [1261](#)
- PsiIntpart, [1261](#)
 - calledby PsiBack, [1259](#)
 - defun, [1261](#)
- PsiXotic, [1262](#)
 - defun, [1262](#)
- pspacers, [98](#)
 - calls bright, [98](#)
 - calls concat, [98](#)
 - calls sayBrightly, [98](#)
 - uses \$spaceList, [99](#)
 - defun, [98](#)
- ptimers, [98](#)
 - calledby trace1, [79](#)
 - calls bright, [98](#)
 - calls concat, [98](#)
 - calls float, [98](#)
 - calls quotient, [98](#)
 - calls sayBrightly, [98](#)
 - uses \$timerList, [98](#)
 - uses \$timerTicksPerSecond, [98](#)
 - defun, [98](#)
- punctuation?, [409](#)
 - calledby scanToken, [405](#)
 - defun, [409](#)
- put, [1229](#)
 - defun, [1229](#)
- putalist
 - calledby interpFunctionDepAlists, [805](#)
 - calledby npProcessSynonym, [813](#)
 - calledby synonymSpad2Cmd, [1105](#)
- putDatabaseStuff, [655](#)
 - calledby msgCreate, [638](#)
 - calls getMsgInfoFromKey, [655](#)
 - calls setMsgText, [655](#)
 - calls setMsgUnforcedAttrList, [655](#)
 - defun, [655](#)
- putFTText, [669](#)
 - calledby processChPosesForOneLine, [666](#)
 - calls getMsgFTTag?, [669](#)
 - calls getMsgPos2, [670](#)
 - calls getMsgPos, [669](#)
 - calls getMsgText, [669](#)
 - calls poCharPosn, [669](#)
 - calls setMsgText, [669](#)
 - defun, [669](#)
- putHist, [900](#)
 - calledby importFromFrame, [35](#)
 - calledby recordAndPrint, [349](#)
 - calledby restoreHistory, [907](#)
 - calledby undoChanges, [903](#)
 - calledby undoFromFile, [904](#)
 - calledby undoInCore, [902](#)
 - calledby writeHistModesAndValues, [913](#)
 - calls get, [900](#)
 - calls putIntSymTab, [900](#)
 - calls recordNewValue, [900](#)
 - calls recordOldValue, [900](#)
 - uses \$HiFiAccess, [900](#)
 - defun, [900](#)
- putIntSymTab
 - calledby putHist, [900](#)
- putTarget
 - calledby evaluateType1, [1120](#)
 - calledby interpret1, [347](#)
- pvarCondList, [1530](#)
 - defun, [1530](#)
- pvarCondList1, [1530](#)
 - defun, [1530](#)
- pvarPredTran, [620](#)
 - calledby rulePredicateTran, [618](#)
 - defun, [620](#)
- pvarsOfPattern, [1531](#)
 - defun, [1531](#)
- qassq
 - calledby getMsgCatAttr, [649](#)
 - calledby ncEltQ, [706](#)
 - calledby ncPutQ, [707](#)
 - calledby setMsgCatlessAttr, [655](#)
 - calledby setMsgUnforcedAttr, [657](#)
 - calledby tokPosn, [703](#)
- qcaar
 - calledby getConstructorDocumentation, [1647](#)
- qcadar
 - calledby dbSpecialExpandIfNecessary, [1653](#)

- calledby getConstructorDocumentation, 1647
- qcar
 - calledby /tracereply, 142
 - calledby ?t, 146
 - calledby NRTEvalDomain, 1229
 - calledby ScanOrPairVec,ScanOrInner, 926
 - calledby abbreviationsSpad2Cmd, 823
 - calledby countCache, 980
 - calledby dbGetDocTable,hn, 1637
 - calledby dbSpecialExpandIfNecessary, 1653
 - calledby dewritify,dewritifyInner, 922
 - calledby displayProperties, 801
 - calledby domainDescendantsOf, 1603
 - calledby evaluateType1, 1120
 - calledby evaluateType, 1118
 - calledby frameSpad2Cmd, 26
 - calledby funfind,LAM, 106
 - calledby getConstructorDocumentation, 1647
 - calledby getTraceOption, 118
 - calledby hasPair, 113
 - calledby kcPage, 1611
 - calledby kccPage, 1618
 - calledby kcnPage, 1622
 - calledby kcpPage, 1614
 - calledby mkEvalable, 1115
 - calledby ncAlist, 705
 - calledby ncTag, 705
 - calledby prTraceNames,fn, 145
 - calledby printTypeAndTime, 352
 - calledby processInteractive, 342
 - calledby reportOperations, 1089
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby reportSpadTrace, 141
 - calledby restoreHistory, 907
 - calledby retract, 1269
 - calledby selectOption, 819
 - calledby setExposeAddConstr, 160
 - calledby setExposeAddGroup, 157
 - calledby setExposeAdd, 159
 - calledby setExposeDropConstr, 164
 - calledby setExposeDropGroup, 162
 - calledby setExposeDrop, 162
 - calledby setExpose, 159
 - calledby setInputLibrary, 974
 - calledby setOutputAlgebra, 1035
 - calledby setOutputCharacters, 1039
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1058
 - calledby setOutputTex, 1070
 - calledby showSpad2Cmd, 1087
 - calledby spadClosure?, 921
 - calledby trace1, 79
 - calledby traceReply, 94
 - calledby transOnlyOption, 99
 - calledby undoSteps, 54
 - calledby undo, 52
 - calledby untraceDomainConstructor,keepTraced?, 138
 - calledby whatSpad2Cmd,fixpat, 1131
 - calledby writify,writifyInner, 917
- qcdar
 - calledby dbSpecialExpandIfNecessary, 1653
 - calledby getConstructorDocumentation, 1647
- qcdr
 - calledby ?t, 146
 - calledby ScanOrPairVec,ScanOrInner, 926
 - calledby abbreviationsSpad2Cmd, 823
 - calledby countCache, 980
 - calledby dbGetDocTable,hn, 1637
 - calledby dbGetDocTable, 1639
 - calledby dbSpecialExpandIfNecessary, 1653
 - calledby dewritify,dewritifyInner, 922
 - calledby displayProperties, 801
 - calledby domainDescendantsOf, 1603
 - calledby evaluateType1, 1120
 - calledby evaluateType, 1118
 - calledby frameSpad2Cmd, 25
 - calledby getTraceOption, 118
 - calledby hasPair, 113
 - calledby mkEvalable, 1115
 - calledby ncAlist, 706
 - calledby prTraceNames,fn, 145
 - calledby readHiFi, 911
 - calledby restoreHistory, 907
 - calledby selectOption, 819
 - calledby setExposeAdd, 159

- calledby setExposeDrop, 162
- calledby setExpose, 159
- calledby setInputLibrary, 974
- calledby setOutputAlgebra, 1035
- calledby setOutputCharacters, 1039
- calledby setOutputFormula, 1063
- calledby setOutputFortran, 1042
- calledby setOutputHtml, 1048
- calledby setOutputMathml, 1054
- calledby setOutputOpenMath, 1058
- calledby setOutputTex, 1070
- calledby spadClosure?, 921
- calledby trace1, 79
- calledby transOnlyOption, 99
- calledby undoSteps, 54
- calledby undo, 52
- calledby writify,writifyInner, 917
- qcsiz
 - calledby isSharpVarWithNum, 109
- qenum, 1306
 - defun, 1306
- qllesseqp
 - calledby dbShowCons1, 1643
- qrplaca
 - calledby dewritify,dewritifyInner, 922
 - calledby writify,writifyInner, 917
- qrplacd
 - calledby dewritify,dewritifyInner, 922
 - calledby writify,writifyInner, 917
- qsabsval, 1314
 - defmacro, 1314
- qsadd1, 1313
 - defmacro, 1313
- qsdifference, 1312
 - defmacro, 1312
- qsDot26432, 1325
 - defmacro, 1325
- qsDot2Mod6432, 1325
 - defmacro, 1325
- qsetvelt
 - calledby dewritify,dewritifyInner, 922
 - calledby writify,writifyInner, 917
- qslessp, 1313
 - calledby trace1, 79
 - defmacro, 1313
- qsmax, 1315
 - defmacro, 1315
- qsmin, 1315
 - defmacro, 1315
- qsminus, 1313
 - defmacro, 1313
- qsMod6432, 1324
 - defmacro, 1324
- qsMul6432, 1325
 - defmacro, 1325
- qsMulAdd6432, 1324
 - defmacro, 1324
- qsMulAddMod6432, 1324
 - defmacro, 1324
- qsMulMod32, 1324
 - defmacro, 1324
- qsoddp, 1314
 - defmacro, 1314
- qsplus, 1314
 - defmacro, 1314
- qsquotient, 1312
 - defun, 1312
- qsremainder, 1312
 - defun, 1312
- qssub1, 1313
 - defmacro, 1313
- qstimes, 1314
 - defmacro, 1314
- qszerop, 1315
 - defmacro, 1315
- queryClients, 845
 - calledby close, 845
 - calls sockGetInt, 845
 - calls sockSendInt, 845
 - uses \$QueryClients, 845
 - uses \$SessionManager, 845
 - defun, 845
- queryUserKeyedMsg
 - calledby close, 845
 - calledby historySpad2Cmd, 890
 - calledby importFromFrame, 34
 - calledby listConstructorAbbreviations, 824
 - calledby quitSpad2Cmd, 940
 - calledby yesanswer, 866
- queueUpErrors, 662
 - calledby processMsgList, 659
 - calls processChPosesForOneLine, 662

- uses \$outputList, 662
 - defun, 662
- quickAnd
 - calledby domainDescendantsOf, 1603
- quit, 939, 940
 - calls quitSpad2Cmd, 940
 - defun, 940
 - manpage, 939
- quitSpad2Cmd, 940
 - calledby pquitSpad2Cmd, 938
 - calledby quit, 940
 - calls leaveScratchpad, 940
 - calls queryUserKeyedMsg, 940
 - calls sayKeyedMsg, 940
 - calls string2id-n, 940
 - calls tersyscommand, 940
 - calls upcase, 940
 - uses \$quitCommandType, 940
 - defun, 940
- quoteString, 1544
 - defun, 1544
- quotient
 - calledby Else?, 373
 - calledby Elseif?, 372
 - calledby If?, 372
 - calledby Top?, 372
 - calledby ptimers, 98
- quotient2, 1308
 - defun, 1308
- qvlt
 - calledby dewritify,dewritifyInner, 922
 - calledby writify,writifyInner, 917
- qvlenU16, 1317
 - defmacro, 1317
- qvlenU32, 1318
 - defmacro, 1318
- qvlenU8, 1316
 - defmacro, 1316
- qymaxindex
 - calledby dewritify,dewritifyInner, 922
 - calledby writify,writifyInner, 917
- random, 1309
 - defun, 1309
- rassoc
 - calledby rassocSub, 104
 - calledby saveMapSig, 117
 - calledby trace3, 83
- rassocSub, 104
 - calledby /untrace-2, 125
 - calledby ?t, 146
 - calledby isSubForRedundantMapName, 105
 - calledby monitorXX, 90
 - calledby trace3, 83
 - calledby traceReply, 94
 - calls rassoc, 104
 - defun, 104
- rbesseli, 1280
 - calls BesselI, 1280
 - calls c-to-r, 1280
 - defun, 1280
- rbesselj, 1279
 - calls BesselJ, 1279
 - calls c-to-r, 1279
 - defun, 1279
- rdefinstream, 1309
 - calls rdefiostream, 1309
 - defun, 1309
- rdefiostream
 - calledby rdefinstream, 1309
 - calledby rdefoutstream, 1309
 - calledby readHiFi, 911
 - calledby saveHistory, 905
 - calledby setHistoryCore, 894
 - calledby writeHiFi, 912
- rdefoutstream, 1309
 - calls rdefiostream, 1309
 - defun, 1309
- rdigit?, 424
 - calls strpos, 424
 - defun, 424
- read, 942, 943
 - calledby undo, 52
 - calls readSpad2Cmd, 943
 - defun, 943
 - manpage, 942
- read-line
 - calledby serverReadLine, 336
- readHiFi, 911
 - calledby fetchOutput, 910
 - calledby restoreHistory, 907
 - calledby setHistoryCore, 894

- calledby showInOut, 910
- calledby showInput, 909
- calledby undoFromFile, 904
- calledby undoInCore, 902
- calledby writeInputLines, 896
- calls assoc, 911
- calls histFileName, 911
- calls keyedSystemError, 911
- calls object2Identifier, 911
- calls qcdr, 911
- calls rdefiostream, 911
- calls rshut, 911
- calls spadread, 911
- uses \$internalHistoryTable, 911
- uses \$useInternalHistoryTable, 911
- defun, 911
- readLibPathFast
 - calledby getDependentsOfConstructor, 1620
 - calledby getUsersOfConstructor, 1622
- readline, 1661
 - defun, 1661
- readSpad2Cmd, 943
 - calledby read, 943
 - calls /read, 943
 - calls findfile, 943
 - calls makePathname, 943
 - calls member, 943
 - calls mergePathnames, 943
 - calls namestring, 943
 - calls optionError, 943
 - calls pathnameName, 943
 - calls pathnameTypeId, 943
 - calls pathname, 943
 - calls selectOptionLC, 943
 - calls throwKeyedMsg, 943
 - calls upcase, 943
 - uses /editfile, 943
 - uses \$InteractiveMode, 943
 - uses \$UserLevel, 943
 - uses \$findfile, 943
 - uses \$options, 943
 - defun, 943
- readSpadProfileIfThere, 1147
 - calledby restart, 307
 - uses /editfile, 1147
 - defun, 1147
- reassembleTowerIntoType, 757
 - calledby computeTTTranspositions, 754
 - calledby reassembleTowerIntoType, 757
 - calls reassembleTowerIntoType, 757
 - defun, 757
- reclaim, 331
 - calledby clearCmdCompletely, 838
 - defun, 331
- recordAndPrint, 349
 - calledby processInteractive1, 345
 - calls mkCompanionPage, 349
 - calls mkObjWrap, 349
 - calls output, 349
 - calls printStatisticsSummary, 349
 - calls printStorage, 349
 - calls printTypeAndTime, 349
 - calls putHist, 349
 - calls recordAndPrintTest, 349
 - uses \$EmptyMode, 350
 - uses \$HTCompanionWindowID, 350
 - uses \$QuietCommand, 350
 - uses \$Void, 350
 - uses \$algebraOutputStream, 350
 - uses \$collectOutput, 350
 - uses \$e, 349
 - uses \$mkTestFlag, 350
 - uses \$mkTestOutputType, 349
 - uses \$outputMode, 349, 350
 - uses \$printAnyIfTrue, 350
 - uses \$printStatisticsSummaryIfTrue, 350
 - uses \$printStorageIfTrue, 350
 - uses \$printTimeIfTrue, 350
 - uses \$printTypeIfTrue, 350
 - uses \$printVoidIfTrue, 350
 - uses \$runTestFlag, 349
 - defun, 349
- recordAndPrintTest
 - calledby recordAndPrint, 349
- recordFrame, 1123
 - calledby processInteractive1, 344
 - calledby undoSteps, 54
 - calls diffAlist, 1123
 - calls exit, 1123
 - calls seq, 1123
 - uses \$InteractiveFrame, 1123
 - uses \$frameRecord, 1123

- uses \$previousBindings, 1123
 - defun, 1123
- recordNewValue, 901
 - calledby clearCmdParts, 841
 - calledby putHist, 900
 - calledby undoFromFile, 904
 - calls recordNewValue0, 901
 - calls startTimingProcess, 901
 - calls stopTimingProcess, 901
 - defun, 901
- recordNewValue0, 901
 - calledby recordNewValue, 901
 - calls assq, 901
 - uses \$HistRecord, 901
 - defun, 901
- recordOldValue, 901
 - calledby clearCmdParts, 841
 - calledby putHist, 900
 - calledby undoFromFile, 904
 - calls recordOldValue0, 901
 - calls startTimingProcess, 901
 - calls stopTimingProcess, 901
 - defun, 901
- recordOldValue0, 902
 - calledby recordOldValue, 901
 - calls assq, 901
 - uses \$HistList, 902
 - defun, 902
- reduce-stack-clear
 - calledby ioclear, 1158
- reduceAlistForDomain, 1615
 - calledby kccPage, 1618
 - calls nreverse0, 1616
 - calls simpHasPred, 1616
 - calls sublis, 1615
 - defun, 1615
- redundant, 664
 - calls msgNoRep?, 664
 - calls sameMsg?, 664
 - uses \$noRepList, 664
 - defun, 664
- refvecp
 - calledby flatBvList, 89
 - calledby isDomainOrPackage, 107
 - calledby spadTrace, 132
- regress, 946, 950
 - calls findnexttest, 950
 - calls getspoolname, 950
 - calls testpassed, 950
 - uses *all-tests-ran*, 950
 - defun, 950
 - manpage, 946
- remainder
 - calledby KeepPart?, 373
 - calledby SkipEnd?, 373
 - calledby SkipPart?, 373
- remainder2, 1308
 - defun, 1308
- remalist
 - calledby clearParserMacro, 793
- remdup
 - calledby clearCmdParts, 841
 - calledby computeTTTranspositions, 754
 - calledby dbConsHeading, 1649
 - calledby dbShowCons1, 1643
 - calledby dbShowConsDoc, 1645
 - calledby displayMacros, 867
 - calledby displayOperationsFromLisplib, 1095
 - calledby displayProperties, 801
 - calledby reportOpsFromLisplib, 1093
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby retract2Specialization, 772
- remFile, 647
 - calledby getPosStL, 646
 - calls ifcdr, 647
 - defun, 647
- remLine, 652
 - calledby getPosStL, 646
 - calls ifcar, 647
 - defun, 652
- removeAttributes
 - calledby getMsgInfoFromKey, 656
- removeOption, 100
 - calledby spadTrace, 132
 - defun, 100
- removeQuote
 - calledby coerceBranch2Union, 766
- remover, 115
 - calledby remover, 115
 - calledby spadUntrace, 143
 - calls remover, 115

- defun, 115
- removeTracedMapSigs, 127
 - calledby untrace, 123
 - uses \$tracedMapSignatures, 127
 - defun, 127
- removeUndoLines, 57
 - calls charPosition, 57
 - calls concat, 57
 - calls exit, 57
 - calls maxindex, 57
 - calls seq, 57
 - calls stringPrefix?, 57
 - calls substring, 57
 - calls trimString, 57
 - calls undoCount, 57
 - uses \$IOindex, 57
 - uses \$currentLine, 57
 - defun, 57
- removeZeroOne
 - calledby getSubDomainPredicate, 769
- removeZeroOneDestructively
 - calledby reportOperations, 1089
- remprop
 - calledby loadLib, 1222
- rempropI
 - calledby restoreHistory, 907
- renamePatternVariables, 1527
 - defun, 1527
- renamePatternVariables1, 1527
 - defun, 1527
- rep, 661
 - calledby makeMsgFromLine, 661
 - defun, 661
- repeat, 692
 - syntax, 692
- replaceFile, 1171
 - defun, 1171
- replaceLast
 - calledby coerceIntTower, 750
- replacePercentByDollar, 1504
 - defun, 1504
- replacePercentByDollar.fn, 1504
 - defun, 1504
- replaceSharps, 1144
 - calledby evaluateType1, 1120
 - calledby valueArgsEqual?, 764
- calls subCopy, 1144
- local ref \$FormalMapVariableList, 1144
- local ref \$TriangleVariableList, 1144
- defun, 1144
- reportInstantiations
 - calledby processInteractive, 342
- reportinstantiations, 1271
 - uses \$reportinstantiations, 1271
 - defun, 1271
- reportOperations, 1089
 - calledby showSpad2Cmd, 1087
 - calls bright, 1089
 - calls evaluateType, 1089
 - calls isDomainValuedVariable, 1089
 - calls isNameOfType, 1089
 - calls isType, 1090
 - calls mkAtree, 1089
 - calls opOf, 1089
 - calls qcar, 1089
 - calls removeZeroOneDestructively, 1089
 - calls reportOpsFromLisplib0, 1089
 - calls reportOpsFromUnitDirectly0, 1089
 - calls sayBrightly, 1089
 - calls sayKeyedMsg, 1089
 - calls unabbrev, 1089
 - uses \$doNotAddEmptyModelIfTrue, 1090
 - uses \$env, 1090
 - uses \$eval, 1090
 - uses \$genValue, 1090
 - uses \$quadSymbol, 1090
 - defun, 1089
- reportOpsFromLisplib, 1092
 - calledby reportOpsFromLisplib0, 1091
 - calledby reportOpsFromLisplib1, 1092
 - calls bright, 1092
 - calls concat, 1092
 - calls constructor?, 1092
 - calls dc1, 1093
 - calls displayOperationsFromLisplib, 1093
 - calls eqsubstlist, 1092
 - calls form2StringWithWhere, 1092
 - calls form2String, 1093
 - calls formatAttribute, 1093
 - calls getConstructorSignature, 1092
 - calls getdatabase, 1092
 - calls ifcdr, 1092

- calls isExposedConstructor, 1092
- calls msort, 1093
- calls namestring, 1092
- calls nreverse0, 1092
- calls remdup, 1093
- calls say2PerLine, 1093
- calls sayBrightly, 1092
- calls sayKeyedMsg, 1092
- calls selectOptionLC, 1093
- calls specialChar, 1093
- uses \$FormalMapVariableList, 1093
- uses \$linelength, 1093
- uses \$options, 1093
- uses \$showOptions, 1093
- defun, 1092
- reportOpsFromLisplib0, 1091
 - calledby reportOperations, 1089
 - calls reportOpsFromLisplib1, 1091
 - calls reportOpsFromLisplib, 1091
 - uses \$useEditorForShowOutput, 1091
 - defun, 1091
- reportOpsFromLisplib1, 1092
 - calledby reportOpsFromLisplib0, 1091
 - calls defiostream, 1092
 - calls editFile, 1092
 - calls erase, 1092
 - calls pathname, 1092
 - calls reportOpsFromLisplib, 1092
 - calls sayShowWarning, 1092
 - calls shut, 1092
 - uses \$erase, 1092
 - uses \$sayBrightlyStream, 1092
 - defun, 1092
- reportOpsFromUnitDirectly, 1096
 - calledby reportOpsFromUnitDirectly0, 1096
 - calledby reportOpsFromUnitDirectly1, 1096
 - calls bright, 1096
 - calls concat, 1096
 - calls evalDomain, 1096
 - calls formatAttribute, 1096
 - calls formatOpType, 1096
 - calls formatOperation, 1096
 - calls getOpListForConstructorForm, 1096
 - calls getdatabase, 1096
 - calls getl, 1096
 - calls isExposedConstructor, 1096
 - calls member, 1096
 - calls msort, 1096
 - calls namestring, 1096
 - calls nreverse0, 1096
 - calls qcar, 1096
 - calls remdup, 1096
 - calls say2PerLine, 1096
 - calls sayBrightly, 1096
 - calls selectOptionLC, 1096
 - calls specialChar, 1096
 - calls systemErrorHere, 1096
 - uses \$CategoryFrame, 1096
 - uses \$commentedOps, 1096
 - uses \$linelength, 1096
 - uses \$options, 1096
 - uses \$showOptions, 1096
 - defun, 1096
- reportOpsFromUnitDirectly0, 1095
 - calledby reportOperations, 1089
 - calls reportOpsFromUnitDirectly1, 1095
 - calls reportOpsFromUnitDirectly, 1096
 - uses \$useEditorForShowOutput, 1096
 - defun, 1095
- reportOpsFromUnitDirectly1, 1100
 - calledby reportOpsFromUnitDirectly0, 1095
 - calls defiostream, 1100
 - calls editFile, 1100
 - calls erase, 1100
 - calls pathname, 1100
 - calls reportOpsFromUnitDirectly, 1100
 - calls sayShowWarning, 1100
 - calls shut, 1100
 - uses \$erase, 1100
 - uses \$sayBrightlyStream, 1100
 - defun, 1100
- reportOpSymbol
 - calledby displayOperations, 866
- reportSpadTrace, 141
 - calledby ?t, 147
 - calledby spadTrace, isTraceable, 131
 - calledby spadTrace, 133
 - calls qcar, 141
 - calls sayBrightly, 141
 - uses \$traceNoisely, 141
 - defun, 141
- reportUndo, 59

- calledby diffAlist, 1125
- calls concat, 60
- calls exit, 60
- calls lassoc, 60
- calls pname, 60
- calls pp, 60
- calls sayBrightlyNT, 60
- calls sayBrightly, 60
- calls seq, 59
- defun, 59
- reportundo
 - uses \$InteractiveFrame, 60
- reportWhatOptions, 1133
 - calledby whatSpad2Cmd, 1131
 - calls sayBrightly, 1133
 - uses \$whatOptions, 1133
 - defun, 1133
- reroot, 332
 - calledby initroot, 327
 - calls make-absolute-filename, 333
 - uses \$current-directory, 333
 - uses \$directory-list, 333
 - uses \$library-directory-list, 333
 - uses \$relative-directory-list, 333
 - uses \$relative-library-directory-list, 333
 - uses \$spadroot, 333
 - defun, 332
- reset-stack-limits
 - calledby resetStackLimits, 312
- resetCounters, 98
 - calledby trace1, 79
 - calls concat, 98
 - uses \$countList, 98
 - defun, 98
- resethashtables, 1186
 - calls browseopen, 1186
 - calls categoryopen, 1186
 - calls initial-getdatabase, 1186
 - calls interpopen, 1186
 - calls operationopen, 1186
 - uses *allconstructors*, 1187
 - uses *browse-stream*, 1186
 - uses *category-stream*, 1186
 - uses *category-stream-stamp*, 1186
 - uses *hascategory-hash*, 1187
 - uses *interp-stream*, 1186
 - uses *interp-stream-stamp*, 1187
 - uses *operation-hash*, 1187
 - uses *operation-stream*, 1186
 - uses *operation-stream-stamp*, 1186
 - uses *sourcefiles*, 1186
 - defun, 1186
- resetInCoreHist, 898
 - calledby clearCmdAll, 839
 - calledby historySpad2Cmd, 890
 - uses \$HistListAct, 898
 - uses \$HistListLen, 898
 - uses \$HistList, 898
 - defun, 898
- resetSpacers, 97
 - calledby trace1, 79
 - calls concat, 97
 - uses \$spaceList, 97
 - defun, 97
- resetStackLimits, 312
 - calledby intloop, 317
 - calledby protectedEVAL, 339
 - calledby runspad, 311
 - calls reset-stack-limits, 312
 - defun, 312
- resetTimers, 97
 - calledby trace1, 79
 - calls concat, 97
 - uses \$timerList, 97
 - defun, 97
- resetWorkspaceVariables, 962
 - calls copy, 962
 - calls initializeSetVariables, 962
 - uses /editfile, 962
 - uses /pretty, 962
 - uses /sourcefiles, 962
 - uses \$CommandSynonymAlist, 963
 - uses \$IOindex, 963
 - uses \$InitialCommandSynonymAlist, 963
 - uses \$UserAbbreviationsAlist, 963
 - uses \$boot, 962
 - uses \$coerceIntByMapCounter, 963
 - uses \$compileMapFlag, 962
 - uses \$countList, 962
 - uses \$dependeeClosureAlist, 963
 - uses \$echoLineStack, 963
 - uses \$env, 963

- uses \$existingFiles, 962
 - uses \$e, 963
 - uses \$functionTable, 962
 - uses \$msgAlist, 963
 - uses \$msgDatabaseName, 963
 - uses \$msgDatabase, 963
 - uses \$operationNameList, 963
 - uses \$setOptions, 963
 - uses \$slamFlag, 963
 - uses \$sourceFiles, 962
 - uses \$spaceList, 962
 - uses \$timerList, 962
 - defun, 962
- resolveTT
 - calledby unifyStructVar, 727
- resolveTypeListAny
 - calledby retract2Specialization, 772
- Rest, 369
 - calledby incLude1, 374
 - defmacro, 369
- restart
 - calls get-current-directory, 307
 - calls init-memory-config, 307
 - calls initHist, 307
 - calls initializeInterpreterFrameRing, 307
 - calls initroot, 307
 - calls makeInitialModemapFrame, 307
 - calls openserver, 307
 - calls readSpadProfileIfThere, 307
 - calls restart0, 307
 - calls spadStartUpMsgs, 307
 - calls spad, 307
 - calls statisticsInitialization, 307
 - uses \$IOindex, 307
 - uses \$InteractiveFrame, 307
 - uses \$SpadServerName, 307
 - uses \$SpadServer, 307
 - uses \$current-directory, 307
 - uses \$currentLine, 307
 - uses \$displayStartMsgs, 307
 - uses \$openServerIfTrue, 307
 - uses \$printLoadMsgs, 307
- restart function, 306
- restart0, 309
 - calledby restart, 307
 - calls browseopen, 309
 - calls categoryopen, 309
 - calls interopen, 309
 - calls operationopen, 309
 - defun, 309
- restoreHistory, 907
 - calledby historySpad2Cmd, 890
 - calls \$fcopy, 907
 - calls clearCmdSortedCaches, 907
 - calls clearSpad2Cmd, 907
 - calls disableHist, 907
 - calls get, 907
 - calls histFileErase, 907
 - calls histFileName, 907
 - calls identp, 907
 - calls makeHistFileName, 907
 - calls makeInputFilename, 907
 - calls namestring, 907
 - calls putHist, 907
 - calls qcar, 907
 - calls qcdr, 907
 - calls readHiFi, 907
 - calls rempropI, 907
 - calls rkeyids, 907
 - calls sayKeyedMsg, 907
 - calls throwKeyedMsg, 907
 - calls updateInCoreHist, 907
 - uses \$HiFiAccess, 907
 - uses \$InteractiveFrame, 907
 - uses \$e, 907
 - uses \$internalHistoryTable, 907
 - uses \$oldHistoryFileName, 907
 - uses \$options, 907
 - uses \$useInternalHistoryTable, 907
 - defun, 907
- resultp, 956
 - calledby split, 954
 - defun, 956
- retract, 1269
 - calledby coerceOrRetract, 771
 - calledby printTypeAndTime, 351, 352
 - calledby retract2Specialization, 772
 - calls isWrapped, 1269
 - calls mkObj, 1269
 - calls objMode, 1269
 - calls objVal, 1269
 - calls qcar, 1269

- calls retract1, 1269
 - local ref \$EmptyMode, 1269
 - defun, 1269
- retract1
 - calledby retract, 1269
- retract2Specialization, 772
 - calls coerceInt, 772
 - calls coerceUnion2Branch, 772
 - calls get, 772
 - calls isPartialMode, 772
 - calls isRectangularList, 772
 - calls member, 772
 - calls mkObjWrap, 772
 - calls mkObj, 772
 - calls objMode, 772
 - calls objValUnwrap, 772
 - calls objVal, 772
 - calls remdup, 772
 - calls resolveTypeListAny, 772
 - calls retract, 772
 - calls unwrap, 772
 - calls varsInPoly, 772
 - uses \$Any, 772
 - uses \$Integer, 772
 - uses \$NonNegativeInteger, 772
 - uses \$PositiveInteger, 772
 - uses \$QuotientField, 772
 - uses \$Symbol, 772
 - uses \$e, 772
 - defun, 772
- retractByFunction, 778
 - calledby coerceRetract, 777
 - calls coerceUnion2Branch, 778
 - calls compiledLookup, 778
 - calls evalDomain, 778
 - calls findFunctionInDomain, 778
 - calls mkObjWrap, 778
 - calls objMode, 778
 - calls objValUnwrap, 778
 - calls orderMms, 778
 - calls sayFunctionSelectionResult, 778
 - calls sayFunctionSelection, 778
 - calls spadcall, 778
 - uses \$dollar, 778
 - uses \$reportBottomUpFlag, 778
 - defun, 778
- retractUnderDomain, 777
 - calls coerceInt, 777
 - calls constructT, 777
 - calls deconstructT, 777
 - calls nequal, 777
 - calls underDomainOf, 777
 - defun, 777
- rgamma, 1277
 - calls rgammaImpl, 1277
 - defun, 1277
- rgammaImpl, 1245
 - calledby PsiAsymptotic, 1260
 - calledby cgammaImpl, 1248
 - calledby rPsiImpl, 1257
 - calledby rgamma, 1277
 - calls FloatError, 1245
 - calls gammaRatapprox, 1246
 - calls gammaStirling, 1246
 - defun, 1245
- rkeyids
 - calledby restoreHistory, 907
 - calledby setHistoryCore, 894
- rlgamma, 1278
 - calls lrgamma, 1278
 - defun, 1278
- rplac
 - calledby insertAlist, 981
 - calledby spadTrace, 133
 - calledby spadUntrace, 143
- rplacstr
 - calledby processSynonyms, 325
- rpsi, 1278
 - calls rPsiImpl, 1278
 - defun, 1278
- rPsiImpl, 1257
 - calledby rPsiImpl, 1257
 - calledby rpsi, 1278
 - calls FloatError, 1257
 - calls cotdiffeval, 1257
 - calls rPsiImpl, 1257
 - calls rPsiW, 1257
 - calls rgammaImpl, 1257
 - defun, 1257
- rPsiW, 1258
 - calledby rPsiImpl, 1257
 - calls PsiAsymptoticOrder, 1258

- calls PsiAsymptotic, 1258
- calls PsiBack, 1258
- defun, 1258
- rread, 915
 - calledby SPADRREAD, 915
 - calledby getDependentsOfConstructor, 1621
 - calledby getUsersOfConstructor, 1622
 - calledby rread, 915
 - calls rread, 915
 - defun, 915
- rshut
 - calledby getDependentsOfConstructor, 1621
 - calledby getUsersOfConstructor, 1622
 - calledby readHiFi, 911
 - calledby saveDependentsHashTable, 1208
 - calledby saveHistory, 905
 - calledby saveUsersHashTable, 1209
 - calledby setHistoryCore, 894
 - calledby writeHiFi, 912
- ruleLhsTran, 620
 - calledby pfRule2Sex, 616
 - calls nsubst, 620
 - calls patternVarsOf, 620
 - uses \$multiVarPredicateList, 620
 - uses \$predicateList, 620
 - defun, 620
- rulePredicateTran, 617
 - calledby pfRule2Sex, 616
 - calls patternVarsOf, 618
 - calls pvarPredTran, 618
 - uses \$multiVarPredicateList, 618
 - defun, 617
- runspad, 311
 - calledby spad, 310
 - calls exit, 311
 - calls ncTopLevel, 311
 - calls resetStackLimits, 311
 - calls seq, 311
 - uses \$quitTag, 311
 - catches, 311
 - defun, 311
- rwrite, 914
 - calledby rwrite, 914
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1208
 - calledby spadwrite0, 914
 - calls identp, 915, 916
 - calls pname, 914, 916
 - calls rwrite, 914
 - defun, 914
- safeWritify, 917
 - calledby cbesseli, 1280
 - calledby cbesselj, 1279
 - calledby cgamma, 1278
 - calledby chyperOf1, 1280
 - calledby clngamma, 1278
 - calledby cpsi, 1279
 - defun, 1243
- sameMsg?, 666
 - calledby redundant, 664
 - calls getMsgArgL, 666
 - calls getMsgKey, 666
 - defun, 666
- sameUnionBranch, 353
 - calledby printTypeAndTime, 352
 - defun, 353
- satBreak
 - calledby kcPage, 1612
- satisfiesRegularExpressions, 1135
 - calledby filterListOfStringsWithFn, 1135
 - calledby filterListOfStrings, 1134
 - calls strpos, 1135
 - defun, 1135
- satisfiesUserLevel, 791
 - calledby commandsForUserLevel, 788
 - calledby displaySetVariableSettings, 965
 - calledby set1, 1082
 - uses \$UserLevel, 791
 - defun, 791
- save-system
 - calledby spad-save, 1176
- saveDependentsHashTable, 1208
 - calledby make-databases, 1205
 - calls erase, 1208
 - calls hget, 1208
 - calls hkeys, 1208

- calls msort, [1208](#)
- calls rshut, [1208](#)
- calls rwrite, [1208](#)
- calls writeLib1, [1208](#)
- local ref \$depTb, [1208](#)
- local ref \$erase, [1208](#)
- defun, [1208](#)
- saveHistory, [905](#)
 - calledby historySpad2Cmd, [890](#)
 - calls histFileErase, [905](#)
 - calls histFileName, [905](#)
 - calls histInputFileName, [905](#)
 - calls makeHistFileName, [905](#)
 - calls makeInputFilename, [905](#)
 - calls namestring, [905](#)
 - calls object2Identifier, [905](#)
 - calls rdefiostream, [905](#)
 - calls rshut, [905](#)
 - calls sayKeyedMsg, [905](#)
 - calls spadrrwrite0, [905](#)
 - calls throwKeyedMsg, [905](#)
 - calls writeInputLines, [905](#)
 - uses \$HiFiAccess, [905](#)
 - uses \$internalHistoryTable, [905](#)
 - uses \$seen, [905](#)
 - uses \$useInternalHistoryTable, [905](#)
 - defun, [905](#)
- saveMapSig, [117](#)
 - calledby trace1, [79](#)
 - calls addassoc, [117](#)
 - calls getMapSig, [117](#)
 - calls rassoc, [117](#)
 - uses \$mapSubNameAlist, [117](#)
 - uses \$tracedMapSignatures, [117](#)
 - defun, [117](#)
- savesystem, [958](#), [959](#)
 - calls helpSpad2Cmd, [959](#)
 - calls spad-save, [959](#)
 - defun, [959](#)
 - manpage, [958](#)
- saveUsersHashTable, [1208](#)
 - calledby make-databases, [1205](#)
 - calls erase, [1208](#)
 - calls hget, [1209](#)
 - calls hkeys, [1208](#)
 - calls msort, [1208](#)
 - calls rshut, [1209](#)
 - calls rwrite, [1208](#)
 - calls writeLib1, [1208](#)
 - local ref \$erase, [1209](#)
 - local ref \$usersTb, [1209](#)
 - defun, [1208](#)
- say
 - calledby displaySetVariableSettings, [965](#)
 - calledby newHelpSpad2Cmd, [881](#)
 - calledby trace1, [79](#)
 - calledby whatCommands, [1133](#)
 - calledby workfilesSpad2Cmd, [1139](#)
- say2PerLine
 - calledby displayOperationsFromLisplib, [1095](#)
 - calledby reportOpsFromLisplib, [1093](#)
 - calledby reportOpsFromUnitDirectly, [1096](#)
- sayAllCacheCounts, [982](#)
 - calledby setFunctionsCache, [979](#)
 - calls sayCacheCount, [982](#)
 - uses \$cacheAlist, [982](#)
 - uses \$cacheCount, [982](#)
 - defun, [982](#)
- sayAsManyPerLineAsPossible
 - calledby apropos, [1137](#)
 - calledby displayWorkspaceNames, [795](#)
 - calledby setExposeAddGroup, [157](#)
 - calledby setOutputCharacters, [1039](#)
 - calledby whatCommands, [1133](#)
- sayBrightly
 - calledby /untrace-2, [124](#)
 - calledby ?t, [147](#)
 - calledby bcError, [1496](#)
 - calledby break, [156](#)
 - calledby buildHtMacroTable, [1401](#)
 - calledby describeFortPersistence, [1029](#)
 - calledby describeInputLibraryArgs, [975](#)
 - calledby describeOutputLibraryArgs, [972](#)
 - calledby describeSetFortDir, [999](#)
 - calledby describeSetFortTmpDir, [997](#)
 - calledby describeSetFunctionsCache, [982](#)
 - calledby describeSetLinkerArgs, [1001](#)
 - calledby describeSetNagHost, [1027](#)
 - calledby describeSetOutputAlgebra, [1037](#)
 - calledby describeSetOutputFormula, [1065](#)
 - calledby describeSetOutputFortran, [1045](#)

- calledby describeSetOutputHtml, 1050
- calledby describeSetOutputMathml, 1056
- calledby describeSetOutputOpenMath, 1061
- calledby describeSetOutputTex, 1072
- calledby displayCondition, 804
- calledby displayMacros, 867
- calledby displayMacro, 794
- calledby displayModemap, 806
- calledby displayMode, 806
- calledby displaySetOptionInformation, 964
- calledby displaySetVariableSettings, 965
- calledby displayWorkspaceNames, 795
- calledby finalExactRequest, 1491
- calledby libConstructorSig, 1659
- calledby linearFinalRequest, 1490
- calledby msgOutputter, 644
- calledby pcounters, 99
- calledby printLabelledList, 814
- calledby processKeyedError, 643
- calledby pspacers, 98
- calledby ptimers, 98
- calledby reportOperations, 1089
- calledby reportOpsFromLisplib, 1092
- calledby reportOpsFromUnitDirectly, 1096
- calledby reportSpadTrace, 141
- calledby reportUndo, 60
- calledby reportWhatOptions, 1133
- calledby sayCacheCount, 983
- calledby sayShowWarning, 1100
- calledby setFortDir, 998
- calledby setFortTmpDir, 996
- calledby setOutputCharacters, 1039
- calledby trace3, 84
- calledby traceReply, 94
- calledby workfilesSpad2Cmd, 1139
- sayBrightly1, 1240
 - calledby sayMSG2File, 44
 - defun, 1240
- saybrightly1
 - calledby saymsg, 45
 - calls brightprint-0, 1240
 - calls brightprint, 1240
- sayBrightlyLength
 - calledby traceReply, 94
- sayBrightlyNT
 - calledby bcError, 1496
 - calledby letPrint, 108
 - calledby mkConform, 1628
 - calledby reportUndo, 60
- sayBrightlyNT2
 - calledby monitorEnter, 147
- sayCacheCount, 983
 - calledby countCache, 980
 - calledby sayAllCacheCounts, 982
 - calls bright, 983
 - calls linearFormatName, 983
 - calls sayBrightly, 983
 - defun, 983
- sayExample, 868
 - defun, 868
- sayexample
 - calls cleanupLine, 869
 - calls sayNewLine, 869
- sayFunctionSelection
 - calledby coerceIntByMap, 761
 - calledby retractByFunction, 778
- sayFunctionSelectionResult
 - calledby coerceIntByMap, 761
 - calledby retractByFunction, 778
- sayKeyedMsg, 43
 - calledby abbQuery, 865
 - calledby abbreviationsSpad2Cmd, 823
 - calledby apropos, 1138
 - calledby changeHistListLen, 898
 - calledby clearCmdAll, 839
 - calledby clearCmdCompletely, 838
 - calledby clearCmdParts, 841
 - calledby clearSpad2Cmd, 835
 - calledby commandAmbiguityError, 792
 - calledby commandErrorMessage, 790
 - calledby countCache, 980
 - calledby describeSetStreamsCalculate, 1075
 - calledby displayExposedConstructors, 165
 - calledby displayExposedGroups, 165
 - calledby displayHiddenConstructors, 166
 - calledby displayOperations, 866
 - calledby displayProperties, 801
 - calledby handleNoParseCommands, 810
 - calledby helpSpad2Cmd, 881
 - calledby historySpad2Cmd, 890
 - calledby history, 889
 - calledby importFromFrame, 34

- calledby listConstructorAbbreviations, 824
- calledby loadLibNoUpdate, 1223
- calledby loadLib, 1221
- calledby localdatabase, 1200
- calledby localnrlib, 1201
- calledby newHelpSpad2Cmd, 881
- calledby npsystem, 812
- calledby printStatisticsSummary, 351
- calledby printTypeAndTime, 352
- calledby quitSpad2Cmd, 940
- calledby reportOperations, 1089
- calledby reportOpsFromLisplib, 1092
- calledby restoreHistory, 907
- calledby saveHistory, 905
- calledby set1, 1082
- calledby setExposeAddConstr, 160
- calledby setExposeAddGroup, 157
- calledby setExposeAdd, 159
- calledby setExposeDropConstr, 164
- calledby setExposeDropGroup, 163
- calledby setExposeDrop, 161
- calledby setExpose, 158
- calledby setHistoryCore, 894
- calledby setOutputAlgebra, 1035
- calledby setOutputFormula, 1063
- calledby setOutputFortran, 1042
- calledby setOutputHtml, 1048
- calledby setOutputMathml, 1054
- calledby setOutputOpenMath, 1059
- calledby setOutputTex, 1070
- calledby showHistory, 892
- calledby showSpad2Cmd, 1087
- calledby spadStartUpMsgs, 309
- calledby trace1, 79
- calledby undoInCore, 902
- calledby userLevelErrorMessage, 791
- calledby whatCommands, 1134
- calledby whatSpad2Cmd, 1131
- calledby workfilesSpad2Cmd, 1139
- calledby writeInputLines, 896
- calledby writifyComplain, 916
- calls sayKeyedMsgLocal, 43
- defun, 43
- sayKeyedMsgLocal, 44
 - calledby sayKeyedMsg, 43
 - calls flowSegmentedMsg, 44
- calls sayMSG2File, 44
- calls sayMSG, 44
- calls segmentKeyedMsg, 44
- calls substituteSegmentedMsg, 44
- uses \$displayMsgNumber, 44
- uses \$linelength, 44
- uses \$margin, 44
- uses \$printMsgsToFile, 44
- defun, 44
- sayMessage
 - calledby apropos, 1137
 - calledby clearCmdParts, 841
 - calledby describeSpad2Cmd, 858
 - calledby displaySetOptionInformation, 964
 - calledby displaySpad2Cmd, 864
 - calledby displayWorkspaceNames, 794
 - calledby filterAndFormatConstructors, 1136
 - calledby printLabelledList, 814
 - calledby set1, 1083
 - calledby setFortPers, 1029
 - calledby setFunctionsCache, 979
 - calledby setOutputCharacters, 1039
 - calledby setStreamsCalculate, 1075
 - calledby traceReply, 94
 - calledby updateFromCurrentInterpreter-Frame, 30
 - calledby whatCommands, 1133
- sayMSG, 45
 - calledby ?t, 146
 - calledby commandAmbiguityError, 792
 - calledby displayProperties,sayFunctionDeps, 798
 - calledby displayProperties, 801
 - calledby displaySetOptionInformation, 964
 - calledby displayType, 800
 - calledby displayValue, 799
 - calledby evalDomain, 1115
 - calledby getAndSay, 800
 - calledby initializeSetVariables, 962
 - calledby sayKeyedMsgLocal, 44
 - calledby set1, 1083
 - calledby setExposeAddGroup, 157
 - calledby setExposeAdd, 159
 - calledby setExposeDropConstr, 164
 - calledby setExposeDropGroup, 162
 - calledby setExposeDrop, 161

- calledby setExpose, 158
- calledby showHistory, 892
- calledby showInOut, 910
- calledby showInput, 909
- calledby spadStartUpMsgs, 309
- calledby spadUntrace, 143
- calledby traceDomainLocalOps, 136
- calledby traceReply, 94
- calledby untraceDomainLocalOps, 136
- uses \$algebraOutputStream, 45
- defun, 45
- saymsg
 - calls saybrightly1, 45
- sayMSG2File, 44
 - calledby sayKeyedMsgLocal, 44
 - calls defiostream, 44
 - calls makePathname, 44
 - calls sayBrightly1, 44
 - calls shut, 44
 - defun, 44
- sayNewLine
 - calledby sayexample, 869
- sayShowWarning, 1100
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calls sayBrightly, 1100
 - defun, 1100
- scanCheckRadix, 425
 - calledby scanNumber, 423
 - uses \$linepos, 425
 - uses \$n, 425
 - defun, 425
- scanCloser, 416
 - usedby scanCloser?, 417
 - defvar, 416
- scanCloser?, 417
 - calledby scanKeyTr, 411
 - calls keyword, 417
 - uses scanCloser, 417
 - defun, 417
- scanComment, 407
 - calledby scanToken, 405
 - calls lfcomment, 407
 - calls substring, 407
 - uses \$ln, 407
 - uses \$n, 407
 - uses \$sz, 407
 - defun, 407
- scanDict, 428
 - defvar, 428
- scanDictCons, 428
 - calls hkeys, 428
 - defun, 428
- scanError, 426
 - calledby scanPunct, 409
 - calledby scanToken, 405
 - calls lferror, 426
 - calls lnExtraBlanks, 426
 - calls ncSoftError, 426
 - uses \$linepos, 426
 - uses \$ln, 426
 - uses \$n, 426
 - defun, 426
- scanEsc, 414
 - calledby scanEscape, 426
 - calledby scanEsc, 415
 - calledby scanS, 422
 - calledby scanW, 419
 - calledby spleI1, 414
 - calls nextline, 415
 - calls scanEsc, 415
 - calls startsComment?, 415
 - calls startsNegComment?, 415
 - calls strposl, 415
 - uses \$ln, 415
 - uses \$n, 415
 - uses \$r, 415
 - uses \$sz, 415
 - defun, 414
- scanEscape, 426
 - calledby scanToken, 405
 - calls scanEsc, 426
 - calls scanWord, 426
 - uses \$n, 426
 - defun, 426
- scanExponent, 417
 - calledby scanNumber, 423
 - calledby scanPossFloat, 412
 - calls concat, 418
 - calls digit?, 418
 - calls lffloat, 417
 - calls spleI, 418

- uses \$ln, 418
 - uses \$n, 418
 - uses \$sz, 418
 - defun, 417
- scanIgnoreLine, 404
 - calledby lineoftoks, 403
 - calls incPrefix?, 404
 - defun, 404
- scanInsert, 429
 - defun, 429
- scanKeyTable, 427
 - defvar, 427
- scanKeyTableCons, 427
 - defun, 427
- scanKeyTr, 411
 - calledby scanPunct, 409
 - calls keyword, 411
 - calls lfkey, 411
 - calls scanCloser?, 411
 - calls scanPossFloat, 411
 - uses \$floatok, 411
 - defun, 411
- scanKeyWords, 399
 - defvar, 399
- scanNegComment, 408
 - calledby scanToken, 405
 - calls lfnegcomment, 408
 - calls substring, 408
 - uses \$ln, 408
 - uses \$n, 408
 - uses \$sz, 408
 - defun, 408
- scanNumber, 423
 - calledby scanToken, 405
 - calls concat, 423
 - calls lfinteger, 423
 - calls lfrinteger, 423
 - calls scanCheckRadix, 423
 - calls scanExponent, 423
 - calls spleI1, 423
 - calls spleI, 423
 - uses \$floatok, 423
 - uses \$ln, 423
 - uses \$n, 423
 - uses \$sz, 423
 - defun, 423
- ScanOrPairVec, 926
 - calledby dewritify, 925
 - calledby writify, 920
 - calls ScanOrPairVec, ScanOrInner, 926
 - uses \$seen, 926
 - catches, 926
 - defun, 926
- ScanOrPairVec, ScanOrInner, 925
 - calledby ScanOrPairVec, ScanOrInner, 926
 - calledby ScanOrPairVec, 926
 - calls ScanOrPairVec, ScanOrInner, 926
 - calls hget, 926
 - calls hput, 926
 - calls qcar, 926
 - calls qcdr, 926
 - calls vecp, 926
 - uses \$seen, 926
 - defun, 925
 - throws, 925
- scanPossFloat, 412
 - calledby scanKeyTr, 411
 - calls lfkey, 412
 - calls scanExponent, 412
 - calls spleI, 412
 - uses \$ln, 412
 - uses \$n, 412
 - uses \$sz, 412
 - defun, 412
- scanPun, 430
 - defvar, 430
- scanPunCons, 430
 - calls hkeys, 431
 - defun, 430
- scanPunct, 409
 - calledby scanToken, 405
 - calls scanError, 409
 - calls scanKeyTr, 409
 - calls subMatch, 409
 - uses \$ln, 409
 - uses \$n, 409
 - defun, 409
- scanS, 422
 - calledby scanString, 421
 - calledby scanS, 422
 - calls concat, 422
 - calls lnExtraBlanks, 422

- calls ncSoftError, [422](#)
- calls scanEsc, [422](#)
- calls scanS, [422](#)
- calls scanTransform, [422](#)
- calls strpos, [422](#)
- calls substring, [422](#)
- uses \$linepos, [422](#)
- uses \$ln, [422](#)
- uses \$n, [422](#)
- uses \$sz, [422](#)
- defun, [422](#)
- scanSpace, [420](#)
 - calledby scanToken, [405](#)
 - calls lfspaces, [420](#)
 - calls strposl, [420](#)
 - uses \$floatok, [420](#)
 - uses \$ln, [420](#)
 - uses \$n, [420](#)
 - defun, [420](#)
- scanString, [421](#)
 - calledby scanToken, [405](#)
 - calls lfstring, [421](#)
 - calls scanS, [421](#)
 - uses \$floatok, [421](#)
 - uses \$n, [421](#)
 - defun, [421](#)
- scanToken, [405](#)
 - calls constoken, [406](#)
 - calls dqUnit, [406](#)
 - calls lfid, [405](#)
 - calls lnExtraBlanks, [406](#)
 - calls punctuation?, [405](#)
 - calls scanComment, [405](#)
 - calls scanError, [405](#)
 - calls scanEscape, [405](#)
 - calls scanNegComment, [405](#)
 - calls scanNumber, [405](#)
 - calls scanPunct, [405](#)
 - calls scanSpace, [405](#)
 - calls scanString, [405](#)
 - calls scanWord, [405](#)
 - calls startsComment?, [405](#)
 - calls startsId?, [405](#)
 - calls startsNegComment?, [405](#)
 - uses \$linepos, [406](#)
 - uses \$ln, [406](#)
 - uses \$n, [406](#)
 - defun, [405](#)
- scanTransform, [423](#)
 - calledby scanS, [422](#)
 - defun, [423](#)
- scanW, [419](#)
 - calledby scanWord, [417](#)
 - calledby scanW, [419](#)
 - calls concat, [419](#)
 - calls idChar?, [419](#)
 - calls posend, [419](#)
 - calls scanEsc, [419](#)
 - calls scanW, [419](#)
 - calls substring, [419](#)
 - uses \$ln, [419](#)
 - uses \$n, [419](#)
 - uses \$sz, [419](#)
 - defun, [419](#)
- scanWord, [417](#)
 - calledby scanEscape, [426](#)
 - calledby scanToken, [405](#)
 - calls keyword?, [417](#)
 - calls lfid, [417](#)
 - calls lfkey, [417](#)
 - calls scanW, [417](#)
 - uses \$floatok, [417](#)
 - defun, [417](#)
- search, [1150](#)
 - calledby getProplist, [1150](#)
 - calls searchCurrentEnv, [1150](#)
 - calls searchTailEnv, [1150](#)
 - defun, [1150](#)
- searchCurrentEnv, [1150](#)
 - calledby search, [1150](#)
 - calls assq, [1150](#)
 - defun, [1150](#)
- searchTailEnv, [1151](#)
 - calledby search, [1150](#)
 - calls assq, [1151](#)
 - defun, [1151](#)
- sec, [1291](#)
 - defun, [1291](#)
- sech, [1293](#)
 - defun, [1293](#)
- segmentKeyedMsg, [44](#)
 - calledby getMsgInfoFromKey, [656](#)

- calledby msgText, 354
 - calledby sayKeyedMsgLocal, 44
 - calls string2Words, 44
 - defun, 44
- selectLocalMms
 - calledby coerceInt1, 744
 - calledby getSubDomainPredicate, 769
- selectMms1
 - calledby coerceConvertMmSelection, 752
 - calledby coerceInt1, 744
 - calledby coerceIntByMap, 761
- selectOption, 819
 - calledby selectOptionLC, 819
 - calledby set1, 1082
 - calledby systemCommand, 788
 - calledby unAbbreviateKeyword, 809
 - calls identp, 819
 - calls member, 819
 - calls pname, 819
 - calls qcar, 819
 - calls qcdr, 819
 - calls stringPrefix?, 819
 - defun, 819
- selectOptionLC, 819
 - calledby abbreviationsSpad2Cmd, 823
 - calledby clearCmdParts, 841
 - calledby clearSpad2Cmd, 835
 - calledby close, 845
 - calledby describeSpad2Cmd, 858
 - calledby displaySpad2Cmd, 864
 - calledby frameSpad2Cmd, 25
 - calledby getTraceOption, 118
 - calledby historySpad2Cmd, 890
 - calledby newHelpSpad2Cmd, 881
 - calledby readSpad2Cmd, 943
 - calledby reportOpsFromLisplib, 1093
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby setExposeAdd, 159
 - calledby setExposeDrop, 162
 - calledby setExpose, 159
 - calledby setInputLibrary, 974
 - calledby showHistory, 892
 - calledby synonymsForUserLevel, 1106
 - calledby systemCommand, 788
 - calledby trace1, 79
 - calledby unAbbreviateKeyword, 809
 - calledby whatSpad2Cmd, 1131
 - calledby workfilesSpad2Cmd, 1139
 - calls downcase, 819
 - calls object2Identifier, 819
 - calls selectOption, 819
 - defun, 819
- sendHTErrorSignal
 - calledby protectedEVAL, 339
- separatePiles, 631
 - calledby pileCforest, 630
 - calledby separatePiles, 631
 - calls dqConcat, 631
 - calls dqUnit, 631
 - calls lastTokPosn, 631
 - calls separatePiles, 631
 - calls tokConstruct, 631
 - defun, 631
- SEQ
 - calledby funfind,LAM, 106
- seq
 - calledby /tracereply, 142
 - calledby abbreviationsSpad2Cmd, 823
 - calledby apropos, 1137
 - calledby clearCmdParts, 841
 - calledby coerceSpadArgs2E, 128
 - calledby dewritify,dewritifyInner, 922
 - calledby diffAlist, 1125
 - calledby displayMacros, 867
 - calledby displayProperties,sayFunctionDeps, 798
 - calledby displayProperties, 801
 - calledby getAliasIfTracedMapParameter, 140
 - calledby getBpiNameIfTracedMap, 140
 - calledby getPreviousMapSubNames, 103
 - calledby getTraceOption,hn, 117
 - calledby getTraceOptions, 116
 - calledby getTraceOption, 118
 - calledby historySpad2Cmd, 890
 - calledby importFromFrame, 34
 - calledby isListOfIdentifiersOrStrings, 102
 - calledby isListOfIdentifiers, 102
 - calledby orderBySlotNumber, 114
 - calledby prTraceNames,fn, 145
 - calledby prTraceNames, 145
 - calledby recordFrame, 1123

- calledby removeUndoLines, 57
- calledby reportUndo, 59
- calledby runspad, 311
- calledby set1, 1082
- calledby spadTrace,isTraceable, 131
- calledby spadTrace, 132
- calledby spadUntrace, 143
- calledby subTypes, 101
- calledby trace1, 79
- calledby traceDomainConstructor, 137
- calledby traceReply, 94
- calledby undoFromFile, 904
- calledby undoLocalModemapHack, 56
- calledby undoSingleStep, 55
- calledby untraceDomainConstructor,keepTraced, 138
- calledby untraceDomainConstructor, 139
- calledby whatConstructors, 1137
- calledby whatSpad2Cmd, 1131
- calledby writify,writifyInner, 917
- serverReadLine, 336
 - calledby intloopReadConsole, 321
 - calls addNewInterpreterFrame, 336
 - calls changeToNamedInterpreterFrame, 336
 - calls executeQuietCommand, 336
 - calls is-console[9], 336, 1199
 - calls lassoc, 336
 - calls mkprompt, 336
 - calls parseAndInterpret, 336
 - calls protectedEVAL, 336
 - calls read-line, 336
 - calls serverSwitch, 336
 - calls sockGetInt, 336
 - calls sockGetString, 336
 - calls sockSendInt, 336
 - calls sockSendString, 336
 - calls unescapeStringsInForm, 336
 - uses *eof*, 337
 - uses \$CallInterp, 337
 - uses \$CreateFrameAnswer, 337
 - uses \$CreateFrame, 337
 - uses \$EndOfOutput, 337
 - uses \$EndServerSession, 337
 - uses \$EndSession, 337
 - uses \$KillLispSystem, 336
 - uses \$LispCommand, 337
 - uses \$MenuServer, 337
 - uses \$NeedToSignalSessionManager, 337
 - uses \$NonSmanSession, 336
 - uses \$QuietSpadCommand, 336
 - uses \$SessionManager, 337
 - uses \$SpadCommand, 336
 - uses \$SpadServer, 337
 - uses \$SwitchFrames, 337
 - uses \$currentFrameNum, 337
 - uses \$frameAlist, 337
 - uses \$frameNumber, 337
 - uses \$sockBufferLength, 337
 - uses in-stream, 337
 - catches, 336
 - defun, 336
 - serverSwitch
 - calledby serverReadLine, 336
 - set, 960, 1081
 - calledby browse, 830
 - calls set1, 1081
 - uses \$setOptions, 1082
 - defun, 1081
 - manpage, 960
 - set-hole-size
 - calledby init-memory-config, 326
 - set-restart-hook, 305
 - defun, 305
 - set1, 1082
 - calledby set1, 1083
 - calledby set, 1081
 - calls bright, 1083
 - calls displaySetOptionInformation, 1082
 - calls displaySetVariableSettings, 1082
 - calls downcase, 1082
 - calls exit, 1082
 - calls lassoc, 1082
 - calls literals, 1083
 - calls object2String, 1083
 - calls poundsign, 1082
 - calls satisfiesUserLevel, 1082
 - calls sayKeyedMsg, 1082
 - calls sayMSG, 1083
 - calls sayMessage, 1083
 - calls selectOption, 1082
 - calls seq, 1082
 - calls set1, 1083

- calls translateYesNo2TrueFalse, 1083
- calls tree, 1083
- calls use-fast-links, 1083
- uses \$UserLevel, 1083
- uses \$displaySetValue, 1083
- uses \$setOptionNames, 1083
- defun, 1082
- setAref2U16, 1321
 - defmacro, 1321
- setAref2U32, 1322
 - defmacro, 1322
- setAref2U8, 1319
 - defmacro, 1319
- setCurrentLine, 333
 - calledby intloopEchoParse, 361
 - calledby intloopProcessString, 329
 - calledby intloopProcess, 356
 - calledby intloopReadConsole, 321
 - uses \$currentLine, 334
 - defun, 333
- setdatabase, 1195
 - calledby abbreviationsSpad2Cmd, 823
 - calls make-database, 1195
 - defun, 1195
- setdifference
 - calledby displayWorkspaceNames, 795
 - calledby untraceMapSubNames, 105
- seteltU16, 1318
 - defmacro, 1318
- seteltU32, 1319
 - defmacro, 1319
- seteltU8, 1317
 - defmacro, 1317
- setExpose, 158
 - calledby setExpose, 159
 - calls displayExposedConstructors, 158
 - calls displayExposedGroups, 158
 - calls displayHiddenConstructors, 158
 - calls namestring, 159
 - calls pathname, 159
 - calls qcar, 159
 - calls qcdr, 159
 - calls sayKeyedMsg, 158
 - calls sayMSG, 158
 - calls selectOptionLC, 159
 - calls setExposeAdd, 159
 - calls setExposeDrop, 159
 - calls setExpose, 159
 - defun, 158
- setExposeAdd, 159
 - calledby setExposeAdd, 160
 - calledby setExpose, 159
 - calls displayExposedConstructors, 159
 - calls displayExposedGroups, 159
 - calls qcar, 159
 - calls qcdr, 159
 - calls sayKeyedMsg, 159
 - calls sayMSG, 159
 - calls selectOptionLC, 159
 - calls setExposeAddConstr, 160
 - calls setExposeAddGroup, 159
 - calls setExposeAdd, 160
 - calls specialChar, 159
 - uses \$linelength, 160
 - defun, 159
- setExposeAddConstr, 160
 - calledby localnrlib, 1201
 - calledby setExposeAdd, 160
 - calls clearClams, 161
 - calls delete, 160
 - calls displayExposedConstructors, 161
 - calls getdatabase, 160
 - calls member, 160
 - calls msort, 161
 - calls qcar, 160
 - calls sayKeyedMsg, 160
 - calls specialChar, 161
 - calls unabbrev, 160
 - uses \$interpreterFrameName, 161
 - uses \$linelength, 161
 - uses \$localExposureData, 161
 - defun, 160
- setExposeAddGroup, 157
 - calledby setExposeAdd, 159
 - calls clearClams, 157
 - calls displayExposedConstructors, 157
 - calls displayExposedGroups, 157
 - calls displayHiddenConstructors, 157
 - calls getalist, 157
 - calls member, 157
 - calls msort, 157
 - calls namestring, 157

- calls object2String, 157
- calls pathname, 157
- calls qcar, 157
- calls sayAsManyPerLineAsPossible, 157
- calls sayKeyedMsg, 157
- calls sayMSG, 157
- calls specialChar, 157
- uses \$globalExposureGroupAlist, 157
- uses \$interpreterFrameName, 157
- uses \$linelength, 157
- uses \$localExposureData, 157
- defun, 157
- setExposeDrop, 161
 - calledby setExposeDrop, 162
 - calledby setExpose, 159
 - calls displayHiddenConstructors, 161
 - calls qcar, 162
 - calls qcdr, 162
 - calls sayKeyedMsg, 161
 - calls sayMSG, 161
 - calls selectOptionLC, 162
 - calls setExposeDropConstr, 162
 - calls setExposeDropGroup, 162
 - calls setExposeDrop, 162
 - calls specialChar, 161
 - uses \$linelength, 162
 - defun, 161
- setExposeDropConstr, 164
 - calledby setExposeDrop, 162
 - calls clearClams, 164
 - calls delete, 164
 - calls displayExposedConstructors, 164
 - calls displayHiddenConstructors, 164
 - calls getdatabase, 164
 - calls member, 164
 - calls msort, 164
 - calls qcar, 164
 - calls sayKeyedMsg, 164
 - calls sayMSG, 164
 - calls specialChar, 164
 - calls unabbrev, 164
 - uses \$interpreterFrameName, 164
 - uses \$linelength, 164
 - uses \$localExposureData, 164
 - defun, 164
- setExposeDropGroup, 162
 - calledby setExposeDrop, 162
 - calls clearClams, 162
 - calls delete, 163
 - calls displayExposedConstructors, 162
 - calls displayExposedGroups, 162
 - calls displayHiddenConstructors, 162
 - calls getalist, 163
 - calls member, 162
 - calls qcar, 162
 - calls sayKeyedMsg, 163
 - calls sayMSG, 162
 - calls specialChar, 163
 - uses \$globalExposureGroupAlist, 163
 - uses \$interpreterFrameName, 163
 - uses \$linelength, 163
 - uses \$localExposureData, 163
 - defun, 162
- setFortDir, 998
 - calls bright, 999
 - calls describeSetFortDir, 998
 - calls pname, 998
 - calls sayBrightly, 998
 - calls validateOutputDirectory, 998
 - uses \$fortranDirectory, 999
 - defun, 998
- setFortPers, 1029
 - calls bright, 1029
 - calls describeFortPersistence, 1029
 - calls sayMessage, 1029
 - calls terminateSystemCommand, 1029
 - uses \$fortPersistence, 1029
 - defun, 1029
- setFortTmpDir, 996
 - calls bright, 996
 - calls describeSetFortTmpDir, 996
 - calls pname, 996
 - calls sayBrightly, 996
 - calls validateOutputDirectory, 996
 - uses \$fortranTmpDir, 996
 - defun, 996
- setFunctionsCache, 979
 - calls bright, 979
 - calls countCache, 979
 - calls describeSetFunctionsCache, 979
 - calls object2String, 979
 - calls sayAllCacheCounts, 979

- calls sayMessage, 979
 - calls terminateSystemCommand, 979
 - uses \$cacheAlist, 980
 - uses \$cacheCount, 980
 - uses \$options, 979
 - defun, 979
- setHistoryCore, 894
 - calledby historySpad2Cmd, 890
 - calls boot-equal, 894
 - calls disableHist, 894
 - calls histFileErase, 894
 - calls histFileName, 894
 - calls object2Identifier, 894
 - calls rdefiostream, 894
 - calls readHiFi, 894
 - calls rkeyids, 894
 - calls rshut, 894
 - calls sayKeyedMsg, 894
 - calls spadwrite, 894
 - uses \$HiFiAccess, 894
 - uses \$IOindex, 894
 - uses \$internalHistoryTable, 894
 - uses \$useInternalHistoryTable, 894
 - defun, 894
- setInputLibrary, 974
 - calledby setInputLibrary, 974
 - calls addInputLibrary, 974
 - calls describeInputLibraryArgs, 974
 - calls dropInputLibrary, 974
 - calls qcar, 974
 - calls qcdr, 974
 - calls selectOptionLC, 974
 - calls setInputLibrary, 974
 - uses input-libraries, 974
 - defun, 974
- setIOindex, 909
 - calledby showHistory, 892
 - uses \$IOindex, 909
 - defun, 909
- setletprintflag
 - calledby breaklet, 155
 - calledby spadTrace, 133
 - calledby spadUntrace, 143
- setLinkerArgs, 1001
 - calls describeSetLinkerArgs, 1001
 - calls object2String, 1001
 - uses \$fortranLibraries, 1001
 - defun, 1001
- setMsgCatlessAttr, 655
 - calledby setMsgForcedAttr, 654
 - calledby setMsgUnforcedAttr, 657
 - calls ifcdr, 655
 - calls ncAlist, 655
 - calls ncPutQ, 655
 - calls qassq, 655
 - defun, 655
- setMsgForcedAttr, 654
 - calledby setMsgForcedAttrList, 654
 - calls ncPutQ, 654
 - calls setMsgCatlessAttr, 654
 - defun, 654
- setMsgForcedAttrList, 654
 - calledby msgCreate, 638
 - calls setMsgForcedAttr, 654
 - calls whichCat, 654
 - defun, 654
- setMsgPrefix, 639
 - calledby processChPosesForOneLine, 666
 - defmacro, 639
- setMsgText, 640
 - calledby putDatabaseStuff, 655
 - calledby putFTText, 669
 - defmacro, 640
- setMsgUnforcedAttr, 657
 - calledby initImPr, 657
 - calledby initToWhere, 658
 - calledby setMsgUnforcedAttrList, 656
 - calls ncAlist, 657
 - calls ncPutQ, 657
 - calls qassq, 657
 - calls setMsgCatlessAttr, 657
 - defun, 657
- setMsgUnforcedAttrList, 656
 - calledby putDatabaseStuff, 655
 - calls setMsgUnforcedAttr, 656
 - calls whichCat, 656
 - defun, 656
- setNagHost, 1027
 - calls describeSetNagHost, 1027
 - calls object2String, 1027
 - uses \$nagHost, 1027
 - defun, 1027

- setOutputAlgebra, 1035
 - calledby describeSetOutputAlgebra, 1037
 - calledby spad, 310
 - calls \$filep, 1035
 - calls concat, 1035
 - calls defiostream, 1035
 - calls describeSetOutputAlgebra, 1035
 - calls make-outstream, 1035
 - calls member, 1035
 - calls object2String, 1035
 - calls pathnameDirectory, 1035
 - calls pathnameName, 1035
 - calls pathnameType, 1035
 - calls qcar, 1035
 - calls qcdr, 1035
 - calls sayKeyedMsg, 1035
 - calls shut, 1035
 - calls upcase, 1035
 - uses \$algebraFormat, 1035
 - uses \$algebraOutputFile, 1035
 - uses \$algebraOutputStream, 1035
 - uses \$filep, 1035
 - defun, 1035
- setOutputCharacters, 1039
 - calledby setOutputCharacters, 1039
 - calls bright, 1039
 - calls concat, 1039
 - calls downcase, 1039
 - calls pname, 1039
 - calls qcar, 1039
 - calls qcdr, 1039
 - calls sayAsManyPerLineAsPossible, 1039
 - calls sayBrightly, 1039
 - calls sayMessage, 1039
 - calls setOutputCharacters, 1039
 - calls specialChar, 1039
 - uses \$RTspecialCharacters, 1039
 - uses \$plainRTspecialCharacters, 1039
 - uses \$specialCharacterAlist, 1039
 - uses \$specialCharacters, 1039
 - defun, 1039
- setOutputFormula, 1063
 - calledby describeSetOutputFormula, 1066
 - calls \$filep, 1063
 - calls concat, 1063
 - calls defiostream, 1063
 - calls describeSetOutputFormula, 1063
 - calls make-outstream, 1063
 - calls member, 1063
 - calls object2String, 1063
 - calls pathnameDirectory, 1063
 - calls pathnameName, 1063
 - calls pathnameType, 1063
 - calls qcar, 1063
 - calls qcdr, 1063
 - calls sayKeyedMsg, 1063
 - calls shut, 1063
 - calls upcase, 1063
 - uses \$filep, 1064
 - uses \$formulaFormat, 1064
 - uses \$formulaOutputFile, 1064
 - uses \$formulaOutputStream, 1063
 - defun, 1063
- setOutputFortran, 1042
 - calledby describeSetOutputFortran, 1045
 - calls \$filep, 1042
 - calls concat, 1042
 - calls defiostream, 1042
 - calls describeSetOutputFortran, 1042
 - calls makeStream, 1042
 - calls member, 1042
 - calls object2String, 1042
 - calls pathnameDirectory, 1042
 - calls pathnameName, 1042
 - calls pathnameType, 1042
 - calls qcar, 1042
 - calls qcdr, 1042
 - calls sayKeyedMsg, 1042
 - calls shut, 1042
 - calls upcase, 1042
 - uses \$filep, 1042
 - uses \$fortranFormat, 1042
 - uses \$fortranOutputFile, 1042
 - uses \$fortranOutputStream, 1042
 - defun, 1042
- setOutputHtml, 1048
 - calledby describeSetOutputHtml, 1050
 - calls \$filep, 1048
 - calls concat, 1048
 - calls defiostream, 1048
 - calls describeSetOutputHtml, 1048
 - calls make-outstream, 1048

- calls member, 1048
- calls object2String, 1048
- calls pathnameDirectory, 1048
- calls pathnameName, 1048
- calls pathnameType, 1048
- calls qcar, 1048
- calls qcdr, 1048
- calls sayKeyedMsg, 1048
- calls shut, 1048
- calls upcase, 1048
- uses \$filep, 1048
- uses \$htmlFormat, 1048
- uses \$htmlOutputFile, 1048
- uses \$htmlOutputStream, 1048
- defun, 1048
- setOutputLibrary, 972
 - calls describeOutputLibraryArgs, 972
 - calls filep, 972
 - calls openOutputLibrary, 972
 - calls poundsign, 972
 - uses \$outputLibraryName, 972
 - defun, 972
- setOutputMathml, 1054
 - calledby describeSetOutputMathml, 1056
 - calls \$filep, 1054
 - calls concat, 1054
 - calls defiostream, 1054
 - calls describeSetOutputMathml, 1054
 - calls make-outstream, 1054
 - calls member, 1054
 - calls object2String, 1054
 - calls pathnameDirectory, 1054
 - calls pathnameName, 1054
 - calls pathnameType, 1054
 - calls qcar, 1054
 - calls qcdr, 1054
 - calls sayKeyedMsg, 1054
 - calls shut, 1054
 - calls upcase, 1054
 - uses \$filep, 1054
 - uses \$mathmlFormat, 1054
 - uses \$mathmlOutputFile, 1054
 - uses \$mathmlOutputStream, 1054
 - defun, 1054
- setOutputOpenMath, 1058
 - calledby describeSetOutputOpenMath, 1061
- calls \$filep, 1059
- calls concat, 1058
- calls defiostream, 1058
- calls describeSetOutputOpenMath, 1058
- calls make-outstream, 1059
- calls member, 1058
- calls object2String, 1059
- calls pathnameDirectory, 1059
- calls pathnameName, 1059
- calls pathnameType, 1059
- calls qcar, 1058
- calls qcdr, 1058
- calls sayKeyedMsg, 1059
- calls shut, 1059
- calls upcase, 1058
- uses \$filep, 1059
- uses \$openMathFormat, 1059
- uses \$openMathOutputFile, 1059
- uses \$openMathOutputStream, 1059
- defun, 1058
- setOutputTex, 1069
 - calledby describeSetOutputTex, 1072
 - calls \$filep, 1070
 - calls concat, 1070
 - calls defiostream, 1069
 - calls describeSetOutputTex, 1070
 - calls make-outstream, 1070
 - calls member, 1070
 - calls object2String, 1070
 - calls pathnameDirectory, 1070
 - calls pathnameName, 1070
 - calls pathnameType, 1070
 - calls qcar, 1070
 - calls qcdr, 1070
 - calls sayKeyedMsg, 1070
 - calls shut, 1070
 - calls upcase, 1070
 - uses \$filep, 1070
 - uses \$texFormat, 1070
 - uses \$texOutputFile, 1070
 - uses \$texOutputStream, 1070
 - defun, 1069
- setStreamsCalculate, 1075
 - calls bright, 1075
 - calls describeSetStreamsCalculate, 1075
 - calls object2String, 1075

- calls sayMessage, 1075
 - calls terminateSystemCommand, 1075
 - uses \$streamCount, 1075
 - defun, 1075
- setUpDefault, 1525
 - defun, 1525
- shortenForPrinting, 113
 - calledby letPrint, 108
 - calls devaluate, 113
 - calls isDomainOrPackage, 113
 - defun, 113
- show, 1086, 1087
 - calls showSpad2Cmd, 1087
 - defun, 1087
 - manpage, 1086
- showdatabase, 1194
 - calls getdatabase, 1194
 - defun, 1194
- showHistory, 892
 - calledby historySpad2Cmd, 890
 - calls bright, 892
 - calls concat, 892
 - calls sayKeyedMsg, 892
 - calls sayMSG, 892
 - calls selectOptionLC, 892
 - calls setIOindex, 892
 - calls showInOut, 892
 - calls showInput, 892
 - uses \$evalTimePrint, 893
 - uses \$printTimeSum, 893
 - defun, 892
- showInOut, 910
 - calledby showHistory, 892
 - calls assq, 910
 - calls disableHist, 910
 - calls objMode, 910
 - calls objValUnwrap, 910
 - calls readHiFi, 910
 - calls sayMSG, 910
 - calls spadPrint, 910
 - defun, 910
- showInput, 909
 - calledby showHistory, 892
 - calls disableHist, 909
 - calls readHiFi, 909
 - calls sayMSG, 909
 - calls tab, 909
 - defun, 909
- showMsgPos?, 648
 - calledby getPosStL, 646
 - calls leader?, 648
 - calls msgImPr?, 648
 - uses \$erMsgToss, 648
 - defun, 648
- showSpad2Cmd, 1087
 - calledby show, 1087
 - calls helpSpad2Cmd, 1087
 - calls member, 1087
 - calls qcar, 1087
 - calls reportOperations, 1087
 - calls sayKeyedMsg, 1087
 - uses \$InteractiveFrame, 1087
 - uses \$env, 1087
 - uses \$e, 1087
 - uses \$options, 1087
 - uses \$showOptions, 1087
 - defun, 1087
- shut, 1168
 - calledby reportOpsFromLisplib1, 1092
 - calledby reportOpsFromUnitDirectly1, 1100
 - calledby sayMSG2File, 44
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1059
 - calledby setOutputTex, 1070
 - calledby writeInputLines, 896
 - calls is-console[9], 1168
 - defun, 1168
- signatures
 - /untrace-reduce, 124
 - bcGen, 1468
 - changeToNamedInterpreterFrame, 32
 - closeInterpreterFrame, 37
 - createCurrentInterpreterFrame, 29
 - Delay, 397
 - displayFrameNames, 28
 - doDoitButton, 1469
 - embed2, 88
 - emptyInterpreterFrame, 27, 28

- enum, 25
- executeInterpreterCommand, 1470
- findFrameInRing, 32
- flattenOperationAlist, 108
- fracpart, 1243
- frameEnvironment, 31
- frameExposureData, 25
- frameHiFiAccess, 23
- frameHistList, 23
- frameHistListAct, 24
- frameHistListLen, 24
- frameHistoryTable, 24
- frameHistRecord, 24
- frameInteractive, 22
- frameIOIndex, 23
- frameName, 22
- frameNames, 28
- frameSpad2Cmd, 25
- getHtMacroItem, 1402
- incCommand?, 394
- incIgen, 367
- incLude, 369
- incRenummer, 366
- incString, 331
- incZip, 366
- incZip1, 367
- intloopPrefix?, 327
- intloopProcess, 356
- intloopProcessString, 329
- intloopReadConsole, 321
- isWrapped, 1662
- mkprompt, 334
- next, 330
- next1, 330
- nextInterpreterFrame, 33
- previousInterpreterFrame, 33
- removeOption, 100
- serverReadLine, 337
- set-restart-hook, 305
- setCurrentLine, 334
- StreamNull, 623
- transTracItem, 126
- type, 22, 23
- updateCurrentInterpreterFrame, 31
- updateFromCurrentInterpreterFrame, 30
- simpCatPredicate
 - calledby dbSearchOrder, 1610
- simpHasPred
 - calledby domainDescendantsOf, 1603
 - calledby kTestPred, 1639
 - calledby kePage, 1606
 - calledby reduceAlistForDomain, 1616
- size, 1236
 - calledby abbreviationsSpad2Cmd, 823
 - calledby getPreStL, 645
 - calledby isGenvar, 126
 - calledby makeMsgFromLine, 661
 - calledby processChPosesForOneLine, 666
 - calledby processSynonyms, 325
 - calledby substringMatch, 410
 - calledby writeInputLines, 896
- defun, 1236
- SkipEnd?, 373
 - calledby incLude1, 375
 - calls remainder, 373
- defun, 373
- SkipPart?, 373
 - calledby incLude1, 375
 - calls remainder, 373
- defun, 373
- Skipping?, 374
 - calledby incLude1, 374
 - calls KeepPart?, 374
- defun, 374
- smallEnough, 153
 - calledby monitorPrintRest, 150
 - calledby monitorPrintValue, 152
 - calledby monitorPrint, 150
- defun, 153
- smallEnoughCount, 153
 - defun, 153
- sockGetInt
 - calledby queryClients, 845
 - calledby serverReadLine, 336
- sockGetString
 - calledby executeQuietCommand, 339
 - calledby serverReadLine, 336
- sockSendInt
 - calledby close, 845
 - calledby queryClients, 845
 - calledby serverReadLine, 336
- sockSendString

- calledby serverReadLine, 336
- Solve - bcInputMatrixByFormula, 1445
 - defun, 1445
- sortby
 - calledby workfilesSpad2Cmd, 1139
- spad, 310
 - calledby restart, 307
 - calls runspad, 310
 - calls setOutputAlgebra, 310
 - uses \$PrintCompilerMessageIfTrue, 310
 - defun, 310
- spad-error-loc, 1156
 - calledby spad-long-error, 1155
 - defun, 1156
- spad-long-error, 1155
 - calledby spad-syntax-error, 1155
 - calls iostat, 1155
 - calls spad-error-loc, 1155
 - uses out-stream, 1155
 - uses spaderrorstream, 1155
 - defun, 1155
- spad-save, 1175
 - calledby savesystem, 959
 - calls save-system, 1176
 - uses \$SpadServer, 1176
 - uses \$OpenServerIfTrue, 1176
 - defun, 1175
- spad-short-error, 1156
 - calledby spad-syntax-error, 1155
 - calls line-past-end-p, 1156
 - calls line-print, 1156
 - uses \$current-line, 1156
 - defun, 1156
- spad-syntax-error, 1155
 - calls bumperrorcount, 1155
 - calls consoleinputp, 1155
 - calls ioclear, 1155
 - calls spad-long-error, 1155
 - calls spad-short-error, 1155
 - uses debugmode, 1155
 - defun, 1155
 - throws, 1155
- spad2BootCoerce, 1271
 - defun, 1271
- spadcall
 - calledby algEqual, 765
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1225
 - calledby clearCmdSortedCaches, 837
 - calledby getConstantFromDomain, 767
 - calledby letPrint3, 111
 - calledby lookupInDomainVector, 1227
 - calledby retractByFunction, 778
- spadClosure?, 921
 - calledby writify, writifyInner, 917
 - calls bpname, 921
 - calls qcar, 921
 - calls qcdr, 921
 - calls vecp, 921
 - defun, 921
- spadConstant, 1373
 - defmacro, 1373
- spaderrorstream
 - usedby init-boot/spad-reader, 1154
 - usedby spad-long-error, 1155
- SpadInterpretStream, 318
 - calledby intloop, 317
 - calls intloopInclude, 319
 - calls intloopReadConsole, 319
 - calls mkprompt, 319
 - uses \$erMsgToss, 320
 - uses \$fn, 320
 - uses \$inclAssertions, 320
 - uses \$lastPos, 320
 - uses \$libQuiet, 320
 - uses \$ncMsgList, 319
 - uses \$newcompErrorCount, 320
 - uses \$nopus, 320
 - uses \$okToExecuteMachineCode, 320
 - uses \$systemCommandFunction, 319
 - defun, 318
- spadPrint
 - calledby showInOut, 910
- spadReply, 114
 - calledby spadTrace, 133
 - calledby spadUntrace, 143
 - calls isDomainOrPackage, 114
 - uses \$traceNames, 114
 - defun, 114
- SPADRREAD
 - calls dewritify, 915
 - calls rread, 915

- spadrread, 915
 - calledby readHiFi, 911
 - defun, 915
- spadrwrite, 915
 - calledby setHistoryCore, 894
 - calledby writeHiFi, 912
 - calls spadrrwrite0, 915
 - calls throwKeyedMsg, 915
 - defun, 915
- spadrwrite0, 914
 - calledby saveHistory, 905
 - calledby spadrrwrite, 915
 - calls rwrite, 914
 - calls safeWritify, 914
 - defun, 914
- spadStartUpMsgs, 309
 - calledby restart, 307
 - calls fillerSpaces, 309
 - calls sayKeyedMsg, 309
 - calls sayMSG, 309
 - calls specialChar, 309
 - uses *build-version*, 309
 - uses *yearweek*, 309
 - uses \$linelength, 309
 - uses \$msgAlist, 309
 - uses \$opSysName, 309
 - defun, 309
- spadsysnamep
 - calledby coerceTraceArgs2E, 127
 - calledby coerceTraceFunValue2E, 129
 - calledby monitorEnter, 147
- spadThrow
 - calledby pf2Sex1, 599
 - calledby tersyscommand, 792
 - calledby throwEvalTypeMsg, 1122
- spadThrowBrightly
 - calledby monitorGetValue, 94
 - calledby trace3, 84
- spadTrace, 132
 - calledby trace3, 84
 - calledby traceDomainConstructor, 137
 - calls aldorTrace, 132
 - calls as-insert, 132
 - calls assoc, 132
 - calls bpiname, 132
 - calls bpitrace, 133
 - calls constructSubst, 132
 - calls exit, 132
 - calls flattenOperationAlist, 132
 - calls getOperationAlistFromLisplib, 132
 - calls getOption, 132
 - calls isDomainOrPackage, 132
 - calls opOf, 132
 - calls printDashedLine, 133
 - calls refvecp, 132
 - calls removeOption, 132
 - calls reportSpadTrace, 133
 - calls rplac, 133
 - calls seq, 132
 - calls setletprintflag, 133
 - calls spadReply, 133
 - calls spadTrace,g, 132
 - calls spadTrace,isTraceable, 132
 - calls spadTraceAlias, 132
 - calls subTypes, 132
 - calls userError, 132
 - uses \$fromSpadTrace, 133
 - uses \$letAssoc, 133
 - uses \$reportSpadTrace, 133
 - uses \$traceNames, 133
 - uses \$traceNoisely, 133
 - uses \$tracedModemap, 133
 - defun, 132
- spadTrace,g, 131
 - calledby spadTrace, 132
 - defun, 131
- spadTrace,isTraceable, 131
 - calledby spadTrace, 132
 - calls bpiname, 132
 - calls exit, 131
 - calls gensymp, 131
 - calls reportSpadTrace, 131
 - calls seq, 131
 - defun, 131
- spadTraceAlias, 141
 - calledby spadTrace, 132
 - calls internl, 141
 - defun, 141
- spadUntrace, 143
 - calledby /untrace-2, 124
 - calls assoc, 143
 - calls bpiname, 143

- calls bpiuntrace, 143
- calls bright, 143
- calls devaluate, 143
- calls exit, 143
- calls getOption, 143
- calls isDomainOrPackage, 143
- calls prefix2String, 143
- calls remover, 143
- calls rplac, 143
- calls sayMSG, 143
- calls seq, 143
- calls setletprintflag, 143
- calls spadReply, 143
- calls userError, 143
- uses \$letAssoc, 143
- uses \$traceNames, 143
- defun, 143
- specialChar, 1166
 - calledby displayOperationsFromLisplib, 1095
 - calledby displaySetOptionInformation, 964
 - calledby displaySetVariableSettings, 965
 - calledby filterAndFormatConstructors, 1136
 - calledby printSynonyms, 813
 - calledby reportOpsFromLisplib, 1093
 - calledby reportOpsFromUnitDirectly, 1096
 - calledby setExposeAddConstr, 161
 - calledby setExposeAddGroup, 157
 - calledby setExposeAdd, 159
 - calledby setExposeDropConstr, 164
 - calledby setExposeDropGroup, 163
 - calledby setExposeDrop, 161
 - calledby setOutputCharacters, 1039
 - calledby spadStartUpMsgs, 309
 - calledby whatCommands, 1133
 - calledby workfilesSpad2Cmd, 1139
 - calls assq, 1166
 - calls ifcdr, 1166
 - uses \$specialCharacterAlist, 1166
 - uses \$specialCharacters, 1166
 - defun, 1166
- spleI, 413
 - calledby scanExponent, 418
 - calledby scanNumber, 423
 - calledby scanPossFloat, 412
 - calls spleI1, 413
 - defun, 413
- spleI1, 414
 - calledby scanNumber, 423
 - calledby spleI1, 414
 - calledby spleI, 413
 - calls concat, 414
 - calls scanEsc, 414
 - calls spleI1, 414
 - calls substring, 414
 - uses \$ln, 414
 - uses \$n, 414
 - uses \$sz, 414
 - defun, 414
- split, 953
 - calledby testpassed, 953
 - calls endedp, 954
 - calls ignorep, 954
 - calls resultp, 954
 - calls startp, 954
 - defun, 953
- splitConTable
 - calledby dbShowConditions, 1648
- splitIntoOptionBlocks, 787
 - calledby doSystemCommand, 786
 - calls stripSpaces, 787
 - defun, 787
- spool, 1101
 - manpage, 1101
- stackTraceOptionError, 100
 - calledby getTraceOption, hn, 118
 - calledby getTraceOption, 118
 - calledby traceOptionError, 121
 - calledby transOnlyOption, 99
 - uses \$traceErrorStack, 100
 - defun, 100
- startp, 955
 - calledby split, 954
 - calledby testnumberp, 952
 - calls lastcount, 955
 - uses *ok*, 955
 - defun, 955
- startsComment?, 407
 - calledby scanEsc, 415
 - calledby scanToken, 405
 - uses \$ln, 407
 - uses \$n, 407

- uses \$sz, 407
 - defun, 407
- startsId?, 1234
 - calledby scanToken, 405
 - defmacro, 1234
- startsNegComment?, 408
 - calledby scanEsc, 415
 - calledby scanToken, 405
 - uses \$ln, 408
 - uses \$n, 408
 - uses \$sz, 408
 - defun, 408
- startTimer
 - calledby monitorXX, 90
- startTimingProcess
 - calledby coerceInteractive, 740
 - calledby evalDomain, 1115
 - calledby loadLib, 1221
 - calledby localnrlib, 1201
 - calledby processInteractive1, 345
 - calledby recordNewValue, 901
 - calledby recordOldValue, 901
 - calledby updateHist, 899
- statisticsInitialization, 1221
 - calledby restart, 307
 - calls gbc-time, 1221
 - defun, 1221
- statisticsSummary
 - calledby printStatisticsSummary, 351
- stopTimer
 - calledby monitorXX, 90
- stopTimingProcess
 - calledby coerceInteractive, 740
 - calledby evalDomain, 1115
 - calledby interpretTopLevel, 345
 - calledby loadLibNoUpdate, 1223
 - calledby loadLib, 1222
 - calledby processInteractive1, 345
 - calledby recordNewValue, 901
 - calledby recordOldValue, 901
 - calledby updateHist, 899
- strconc
 - calledby trace3, 84
- streamChop, 364
 - calledby nclloopDQlines, 364
 - calledby streamChop, 365
- calls StreamNull, 365
 - calls nclloopPrefix?, 365
 - calls streamChop, 365
 - defun, 364
- StreamNil, 397
 - usedby incRgen1, 398
 - defvar, 397
- StreamNull, 623
 - calledby incAppend1, 380
 - calledby incLude1, 374
 - calledby incZip1, 367
 - calledby intloopProcess, 356
 - calledby nclloopDQlines, 364
 - calledby next1, 330
 - calledby npNull, 623
 - calledby parseFromString, 340
 - calledby parseNoMacroFromString, 1627
 - calledby streamChop, 365
 - calls eqcar, 623
 - defun, 623
- string-concatenate
 - calledby concat, 1237
- string2Constructor, 1590
 - calledby conSpecialString?, 1598
 - calls downcase, 1590
 - calls hget, 1590
 - calls ifcar, 1590
 - uses \$lowerCaseConTb, 1590
 - defun, 1590
- string2id-n
 - calledby close, 845
 - calledby historySpad2Cmd, 890
 - calledby importFromFrame, 34
 - calledby listConstructorAbbreviations, 824
 - calledby processSynonyms, 325
 - calledby quitSpad2Cmd, 940
 - calledby synonymsForUserLevel, 1106
 - calledby yesanswer, 866
- string2Integer
 - calledby dbGetDocTable, 1638
- string2pint-n
 - calledby getAliasIfTracedMapParameter, 140
- string2Words
 - calledby conSpecialString?, 1598
 - calledby segmentKeyedMsg, 44

- stringize, 1510
 - defun, 1510
- stringList2String, 1497
 - defun, 1497
- stringMatches?, 1276
 - calls basicMatch?, 1276
 - defun, 1276
- stringPrefix?, 1403
 - calledby clearCmdExcept, 840
 - calledby hasOption, 791
 - calledby removeUndoLines, 57
 - calledby selectOption, 819
 - calledby undo, 52
 - defun, 1403
- stringSpaces
 - calledby getFirstWord, 809
- StringToDir, 1298
 - calledby fnameMake, 1297
 - calls lastc, 1298
 - defun, 1298
- stripLisp, 811
 - calledby handleNoParseCommands, 810
 - defun, 811
- stripSpaces, 811
 - calledby dumbTokenize, 807
 - calledby handleNoParseCommands, 810
 - calledby parseSystemCmd, 808
 - calledby splitIntoOptionBlocks, 787
 - defun, 811
- stripUnionTags, 776
 - calledby coerceBranch2Union, 766
 - calledby coerceUnion2Branch, 775
 - defun, 776
- strpos, 1236
 - calledby getSystemCommandLine, 1106
 - calledby processSynonyms, 325
 - calledby rdigit?, 424
 - calledby satisfiesRegularExpressions, 1135
 - calledby scanS, 422
 - calledby stupidIsSpadFunction, 115
 - defun, 1236
- strposl, 1236
 - calledby nextline, 404
 - calledby scanEsc, 415
 - calledby scanSpace, 420
 - defun, 1236
- stupidIsSpadFunction, 115
 - calledby breaklet, 155
 - calledby tracelet, 154
 - calls pname, 115
 - calls strpos, 115
 - defun, 115
- subCopy
 - calledby domArg2, 720
 - calledby hasAtt, 722
 - calledby hasCate, 731
 - calledby hasCaty, 718
 - calledby hasSigAnd, 723
 - calledby hasSig, 721
 - calledby replaceSharps, 1144
 - calledby unifyStructVar, 727
- sublisFormal
 - calledby dbShowConsDoc1, 1646
 - calledby kArgPage, 1602
 - calledby kePage, 1605
- sublislis
 - calledby dbAddDocTable, 1637
 - calledby dbConstructorDoc,hn, 1634
 - calledby dbGetDocTable,hn, 1637
 - calledby dbSearchOrder, 1610
 - calledby dbShowConsDoc1, 1646
 - calledby kcnPage, 1622
 - calledby kcpPage, 1615
 - calledby libConstructorSig, 1659
 - calledby localnrlib, 1201
 - calledby mkDomTypeForm, 1603
 - calledby reduceAlistForDomain, 1615
- subMatch, 410
 - calledby scanPunct, 409
 - calls substringMatch, 410
 - defun, 410
- subseq
 - calledby getFirstWord, 809
 - calledby kdPageInfo, 1601
- substFromAlist, 1529
 - defun, 1529
- substituteSegmentedMsg
 - calledby getMsgInfoFromKey, 656
 - calledby msgText, 354
 - calledby sayKeyedMsgLocal, 44
- substring, 324

- calledby ExecuteInterpSystemCommand, 324
- calledby alqlGetKindString, 1305
- calledby alqlGetOrigin, 1304
- calledby alqlGetParams, 1305
- calledby doSystemCommand, 786
- calledby getAliasIfTracedMapParameter, 140
- calledby getSystemCommandLine, 1106
- calledby incDrop, 395
- calledby lineoftoks, 403
- calledby mkprompt, 334
- calledby printLabelledList, 814
- calledby processSynonyms, 325
- calledby removeUndoLines, 57
- calledby scanComment, 407
- calledby scanNegComment, 408
- calledby scanS, 422
- calledby scanW, 419
- calledby spleI1, 414
- calledby writeInputLines, 896
- defun, 324
- substringMatch, 410
 - calledby subMatch, 410
 - calls size, 410
 - defun, 410
- subTypes, 101
 - calledby spadTrace, 132
 - calledby subTypes, 101
 - calls exit, 101
 - calls lassoc, 101
 - calls seq, 101
 - calls subTypes, 101
 - defun, 101
- suchthat, 696
 - syntax, 696
- summary, 1102, 1103
 - calls concat, 1103
 - calls getenviron, 1103
 - calls obey, 1103
 - defun, 1103
 - manpage, 1102
- superMatch?
 - calledby dbShowCons, 1641
 - calledby koaPageFilterByName, 1634
- Sutor, Robert, 14
- syGeneralErrorHere, 484
 - calledby npListAndRecover, 482
 - calledby npTrapForm, 505
 - calls sySpecificErrorHere, 484
 - defun, 484
- syIgnoredFromTo, 484
 - calledby npRecoverTrap, 483
 - calls FromTo, 484
 - calls From, 484
 - calls To, 484
 - calls ncSoftError, 484
 - calls pfGlobalLinePosn, 484
 - defun, 484
- synonym, 1104, 1105
 - calls synonymSpad2Cmd, 1105
 - defun, 1105
 - manpage, 1104
- synonymsForUserLevel, 1106
 - calledby printSynonyms, 813
 - calls commandsForUserLevel, 1106
 - calls selectOptionLC, 1106
 - calls string2id-n, 1106
 - uses \$UserLevel, 1106
 - uses \$systemCommands, 1106
 - defun, 1106
- synonymSpad2Cmd, 1105
 - calledby synonym, 1105
 - calls getSystemCommandLine, 1105
 - calls printSynonyms, 1105
 - calls processSynonymLine, 1105
 - calls putalist, 1105
 - calls terminateSystemCommand, 1105
 - uses \$CommandSynonymAlist, 1105
 - defun, 1105
- syntax
 - assignment, 673
 - blocks, 676
 - clef, 678
 - collection, 679
 - for, 681
 - if, 685
 - iterate, 687
 - leave, 688
 - parallel, 689
 - repeat, 692
 - suchthat, 696

- while, 697
- sySpecificErrorAtToken, 485
 - calledby sySpecificErrorHere, 484
 - calls ncSoftError, 485
 - calls tokPosn, 485
 - defun, 485
- sySpecificErrorHere, 484
 - calledby syGeneralErrorHere, 484
 - calls sySpecificErrorAtToken, 484
 - uses \$stok, 485
 - defun, 484
- system, 1108
 - manpage, 1108
- systemCommand, 788
 - calledby handleParsedSystemCommands, 808
 - calledby handleTokenSizeSystemCommands, 787
 - calls commandsForUserLevel, 788
 - calls helpSpad2Cmd, 788
 - calls selectOptionLC, 788
 - calls selectOption, 788
 - uses \$CategoryFrame, 788
 - uses \$e, 788
 - uses \$options, 788
 - uses \$syscommands, 788
 - uses \$systemCommands, 788
 - defun, 788
- systemError
 - calledby bcInputEquationsEnd, 1485
 - calledby bcMatrixGen, 1472
 - calledby dbShowConsDoc, 1645
 - calledby domainDescendantsOf, 1603
 - calledby explainLinear, 1491
 - calledby koPageAux, 1633
 - calledby mkConform, 1628
 - calledby pfDefinition2Sex, 613
 - calledby pfLambdaTran, 614
- systemErrorHere
 - calledby interpret2, 348
 - calledby reportOpsFromUnitDirectly, 1096
- tab
 - calledby monitorExit, 151
 - calledby showInput, 909
- tabbing, 652
 - calledby getStFromMsg, 645
 - calls getMsgPrefix?, 652
 - uses \$preLength, 652
 - defun, 652
- take
 - calledby ?t, 147
 - calledby libConstructorSig, 1659
- tangle, 1109
 - manpage, 1109
- templateParts, 1533
 - defun, 1533
- terminateSystemCommand, 792
 - calledby commandAmbiguityError, 792
 - calledby commandErrorMessage, 790
 - calledby displayProperties, 801
 - calledby npProcessSynonym, 813
 - calledby setFortPers, 1029
 - calledby setFunctionsCache, 979
 - calledby setStreamsCalculate, 1075
 - calledby synonymSpad2Cmd, 1105
 - calledby userLevelErrorMessage, 791
 - calls tersyscommand, 792
 - defun, 792
- tersyscommand, 792
 - calledby library, 1199
 - calledby quitSpad2Cmd, 940
 - calledby terminateSystemCommand, 792
 - calls spadThrow, 792
 - defun, 792
- testBitVector
 - calledby kTestPred, 1639
- testnumberp, 952
 - calledby findnexttest, 952
 - calls startp, 952
 - defun, 952
- testpassed, 953
 - calledby regress, 950
 - calls split, 953
 - uses *ok*, 953
 - defun, 953
- The restart function, 306
- thefname, 384
 - calledby inclmsgCannotRead, 385
 - calledby inclmsgNoSuchFile, 384
 - calls pname, 384
 - defun, 384

- theid, [383](#)
 - calledby inclmsgConActive, [386](#)
 - calledby inclmsgConStill, [387](#)
 - calledby inclmsgFileCycle, [385](#)
 - calledby inclmsgIfSyntax, [390](#)
 - calledby inclmsgSay, [383](#)
 - defun, [383](#)
- theorigin, [381](#)
 - calledby inclmsgIfSyntax, [390](#)
 - calledby inclmsgPrematureEOF, [381](#)
 - calledby inclmsgPrematureFin, [389](#)
 - defun, [381](#)
- thisPosIsEqual, [664](#)
 - calls poGlobalLinePosn, [664](#)
 - calls poNopos?, [664](#)
 - defun, [664](#)
- thisPosIsLess, [664](#)
 - calls poGlobalLinePosn, [664](#)
 - calls poNopos?, [664](#)
 - defun, [664](#)
- throwEvalTypeMsg, [1122](#)
 - calledby evaluateType1, [1120](#)
 - calledby evaluateType, [1119](#)
 - calls spadThrow, [1122](#)
 - calls throwKeyedMsg, [1122](#)
 - local ref \$noEvalTypeMsg, [1122](#)
 - defun, [1122](#)
- throwKeyedMsg
 - calledby addNewInterpreterFrame, [33](#)
 - calledby closeInterpreterFrame, [37](#)
 - calledby close, [845](#)
 - calledby coerceInteractive, [740](#)
 - calledby fetchOutput, [910](#)
 - calledby frameSpad2Cmd, [25](#)
 - calledby getConstantFromDomain, [767](#)
 - calledby getTraceOptions, [116](#)
 - calledby getTraceOption, [118](#)
 - calledby importFromFrame, [34](#)
 - calledby readSpad2Cmd, [943](#)
 - calledby restoreHistory, [907](#)
 - calledby saveHistory, [905](#)
 - calledby spadwrite, [915](#)
 - calledby throwEvalTypeMsg, [1122](#)
 - calledby trace1, [79](#)
 - calledby transTraceItem, [126](#)
 - calledby workfilesSpad2Cmd, [1139](#)
 - calledby writeInputLines, [896](#)
- throwKeyedMsgCannotCoerceWithValue
 - calledby catchCoerceFailure, [760](#)
 - calledby evaluateType1, [1120](#)
 - calledby interpret2, [348](#)
- throwListOfKeyedMsgs
 - calledby getTraceOptions, [116](#)
- throws
 - coercionFailure, [763](#)
 - fin, [876](#)
 - intloopReadConsole, [321](#)
 - monitor-file, [1387](#)
 - monitor-readinterp, [1394](#)
 - monitor-spadfile, [1396](#)
 - ncError, [361](#)
 - npMissing, [443](#)
 - npTrap, [505](#)
 - npTrapForm, [505](#)
 - ScanOrPairVec,ScanOrInner, [925](#)
 - spad-syntax-error, [1155](#)
 - writify,writifyInner, [917](#)
- timedEVALFUN
 - calledby getAndEvalConstructorArgument, [1144](#)
- timedEvaluate
 - calledby coerceIntByMap, [761](#)
- timerValue
 - calledby monitorXX, [90](#)
- tmp1
 - calledby diffAlist, [1125](#)
- To, [670](#)
 - calledby syIgnoredFromTo, [484](#)
 - defun, [670](#)
- toFile?, [653](#)
 - calledby msgOutputter, [644](#)
 - calls getMsgToWhere, [653](#)
 - uses \$fn, [653](#)
 - defun, [653](#)
- tokConstruct, [701](#)
 - calledby enPile, [630](#)
 - calledby npDDInfKey, [500](#)
 - calledby npDollar, [476](#)
 - calledby npFirstTok, [435](#)
 - calledby npId, [497](#)
 - calledby npInfixOperator, [452](#)
 - calledby npPrefixColon, [453](#)

- calledby npPushId, 501
- calledby npSymbolVariable, 498
- calledby pfLeaf, 545
- calledby separatePiles, 631
- calls ifcar, 701
- calls ncPutQ, 701
- calls pfNoPosition?, 701
- defun, 701
- token-stack-show, 1157
 - calledby iostat, 1156
 - calls token-type, 1157
 - uses current-token, 1157
 - uses next-token, 1157
 - uses prior-token, 1157
 - uses valid-tokens, 1157
 - defun, 1157
- token-type
 - calledby token-stack-show, 1157
- tokPart, 703
 - calledby intloopProcess, 356
 - calledby npDDInfKey, 500
 - calledby npFirstTok, 435
 - calledby npInfixOperator, 452
 - calledby npSymbolVariable, 498
 - calledby pf2Sex1, 600
 - calledby pfCopyWithPos, 534
 - calledby pfIdSymbol, 545
 - calledby pfLeafToken, 546
 - calledby pfLiteralString, 547
 - calledby pfSexpr,strip, 548
 - calledby pfSymbolSymbol, 550
 - calledby pfSymb, 549
 - calledby pileCforest, 630
 - defun, 703
- tokPosn, 703
 - calledby firstTokPosn, 631
 - calledby lastTokPosn, 631
 - calledby ncloopDQlines, 364
 - calledby npConstTok, 477
 - calledby npDDInfKey, 500
 - calledby npDollar, 476
 - calledby npFirstTok, 435
 - calledby npId, 497
 - calledby npInfixOperator, 452
 - calledby npMissingMate, 508
 - calledby npMissing, 444
 - calledby npParse, 433
 - calledby npPrefixColon, 454
 - calledby npPushId, 501
 - calledby npRecoverTrap, 483
 - calledby npSymbolVariable, 498
 - calledby npTrapForm, 505
 - calledby npTrap, 505
 - calledby pfBraceBar, 555
 - calledby pfBrace, 555
 - calledby pfBracketBar, 556
 - calledby pfBracket, 555
 - calledby pfLeafPosition, 546
 - calledby pileColumn, 627
 - calledby sySpecificErrorAtToken, 485
 - calls ncAlist, 703
 - calls pfNoPosition, 703
 - calls qassq, 703
 - defun, 703
- tokTran, 807
 - calledby handleParsedSystemCommands, 808
 - calledby handleTokenSizeSystemCommands, 787
 - calledby parseSystemCmd, 808
 - calls isIntegerString, 807
 - defun, 807
- tokType, 703
 - calledby npConstTok, 476
 - calledby pilePlusComment, 626
 - calls ncTag, 703
 - defun, 703
- Top, 370
 - usedby incStream, 366
 - usedby incString, 331
 - defvar, 370
- Top?, 372
 - calledby incLude1, 374
 - calledby xIfSyntax, 390
 - calls quotient, 372
 - defun, 372
- toplevel
 - calledby loadLibNoUpdate, 1223
- topLevelInterpEval, 1626
 - calls processInteractive, 1626
 - uses \$ProcessInteractiveValue, 1626
 - uses \$noEvalTypeMsg, 1626

- defun, 1626
- toScreen?, 641
 - calledby msgOutputter, 644
 - calls getMsgToWhere, 641
 - defmacro, 641
- TPDHERE
 - Beware that this function occurs with lowercase also, 457
 - Beware that this function occurs with uppercase also, 457
 - Make this more international, not EBCDIC, 1166
 - Note that there is also an npADD function, 451
 - Note that there is also an npAdd function, 450
 - Remove all boot references from top level, 812
 - The pform function has a leading percent sign. fix this, 520, 521, 525
 - The pform function has a leading percent sign, 358, 517
 - The variable al is undefined, 655
 - This could probably be replaced by the default assoc using eql, 1240
 - This function should be replaced by fillerspaces, 661
 - This should probably be a macro or eliminated, 811
 - Well this makes no sense., 489
 - getMsgFTTag is nonsense, 668
 - note that the file interp.exposed no longer exists., 1394
 - rewrite this using (dolist (item seqList)...), 608
 - rewrite using dsetq, 609
- trace, 78
 - calledby ltrace, 936
 - calls traceSpad2Cmd, 78
 - defun, 78
- trace1, 79
 - calledby trace1, 79
 - calledby traceSpad2Cmd, 78
 - calls /trace,0, 79
 - calls ?t, 79
 - calls addassoc, 79
 - calls delete, 79
 - calls devalue, 79
 - calls exit, 79
 - calls getTraceOptions, 79
 - calls getTraceOption, 79
 - calls hasOption, 79
 - calls isFunctor, 79
 - calls lassoc, 79
 - calls pcounters, 79
 - calls poundsign, 79
 - calls ptimers, 79
 - calls qcar, 79
 - calls qcdr, 79
 - calls qslessp, 79
 - calls resetCounters, 79
 - calls resetSpacers, 79
 - calls resetTimers, 79
 - calls saveMapSig, 79
 - calls sayKeyedMsg, 79
 - calls say, 79
 - calls selectOptionLC, 79
 - calls seq, 79
 - calls throwKeyedMsg, 79
 - calls trace1, 79
 - calls transTraceItem, 79
 - calls unabbrev, 79
 - calls untraceDomainLocalOps, 79
 - calls untrace, 79
 - calls vecp, 79
 - uses \$lastUntraced, 80
 - uses \$optionAlist, 80
 - uses \$options, 80
 - uses \$traceNoisely, 80
 - defun, 79
- trace2, 82
 - calledby trace3, 84
 - calls trace3, 82
 - defun, 82
- trace3, 83
 - calledby bptrace, 136
 - calledby trace2, 82
 - calls adjoinEqual, 84
 - calls breaklet, 84
 - calls embed2, 84
 - calls embeddedFunction, 84
 - calls getTraceOption, 84

- calls isDomainOrPackage, 84
- calls isFunctor, 83
- calls isInterpOnlyMap, 84
- calls isSubForRedundantMapName, 83
- calls isUncompiledMap, 84
- calls options2uc, 83
- calls pname, 84
- calls rassocSub, 83
- calls rassoc, 83
- calls sayBrightly, 84
- calls spadThrowBrightly, 84
- calls spadTrace, 84
- calls strconc, 84
- calls trace2, 84
- calls traceDomainConstructor, 83
- calls tracelet, 84
- calls untrace2, 83
- uses \$countList, 84
- uses \$fromSpadTrace, 84
- uses \$mapSubNameAlist, 84
- uses \$mathTraceList, 84
- uses \$timerList, 84
- uses \$traceDomains, 84
- uses \$traceNames, 84
- uses \$traceNoisely, 84
- defun, 83
- traceDomainConstructor, 137
 - calledby trace3, 83
 - calls concat, 137
 - calls embed, 137
 - calls exit, 137
 - calls getOption, 137
 - calls loadFunctor, 137
 - calls mkq, 137
 - calls seq, 137
 - calls spadTrace, 137
 - calls traceDomainLocalOps, 137
 - uses \$ConstructorCache, 137
 - defun, 137
- traceDomainLocalOps, 136
 - calledby traceDomainConstructor, 137
 - calls sayMSG, 136
 - defun, 136
- tracelet, 154
 - calledby trace3, 84
 - calls bpiname, 154
 - calls compileBoot, 154
 - calls delete, 154
 - calls gensymp, 154
 - calls isGenvar, 154
 - calls lassoc, 154
 - calls stupidIsSpadFunction, 154
 - calls union, 154
 - uses \$QuickLet, 154
 - uses \$letAssoc, 154
 - uses \$traceletFunctions, 154
 - uses \$traceletflag, 154
 - defun, 154
- traceOptionError, 121
 - calls commandAmbiguityError, 121
 - calls stackTraceOptionError, 121
 - defun, 121
- traceReply, 94
 - calledby traceSpad2Cmd, 78
 - calls abbreviate, 95
 - calls addTraceItem, 94
 - calls concat, 94
 - calls exit, 94
 - calls flowSegmentedMsg, 94
 - calls isDomainOrPackage, 94
 - calls isFunctor, 94
 - calls isSubForRedundantMapName, 94
 - calls isgenvar, 94
 - calls poundsign, 94
 - calls prefix2String, 95
 - calls qcar, 94
 - calls rassocSub, 94
 - calls sayBrightlyLength, 94
 - calls sayBrightly, 94
 - calls sayMSG, 94
 - calls sayMessage, 94
 - calls seq, 94
 - calls userError, 94
 - uses \$constructors, 95
 - uses \$domains, 95
 - uses \$linelength, 95
 - uses \$packages, 95
 - uses \$traceNames, 95
 - defun, 94
- traceSpad2Cmd, 78
 - calledby trace, 78
 - calls augmentTraceNames, 78

- calls `getMapSubNames`, 78
 - calls `trace1`, 78
 - calls `traceReply`, 78
 - uses `$mapSubNameAlist`, 78
 - uses `$options`, 78
 - defun, 78
- trademark, 855, 1111
 - defun, 855
 - manpage, 1111
- transferPropsToNode
 - calledby `coerceInt1`, 744
 - calledby `getSubDomainPredicate`, 769
- translateTrueFalse2YesNo, 967
 - calledby `displaySetOptionInformation`, 964
 - calledby `displaySetVariableSettings`, 965
 - defun, 967
- translateYesNo2TrueFalse, 967
 - calledby `initializeSetVariables`, 962
 - calledby `set1`, 1083
 - calls `member`, 967
 - defun, 967
- translateYesNoToTrueFalse, 1560
 - defun, 1560
- transOnlyOption, 99
 - calledby `getTraceOption`, 118
 - calledby `transOnlyOption`, 99
 - calls `qcar`, 99
 - calls `qcdr`, 99
 - calls `stackTraceOptionError`, 99
 - calls `transOnlyOption`, 99
 - calls `upcase`, 99
 - defun, 99
- transTraceItem, 126
 - calledby `trace1`, 79
 - calledby `transTraceItem`, 126
 - calledby `untrace`, 122
 - calls `constructor?`, 126
 - calls `devaluate`, 126
 - calls `domainToGenvar`, 126
 - calls `get`, 126
 - calls `member`, 126
 - calls `objMode`, 126
 - calls `objVal`, 126
 - calls `throwKeyedMsg`, 126
 - calls `transTraceItem`, 126
 - calls `unabbrev`, 126
 - calls `vecp`, 126
 - uses `$doNotAddEmptyModeIfTrue`, 126
 - defun, 126
- trapNumericErrors, 1312
 - defmacro, 1312
- tree
 - calledby `displaySetVariableSettings`, 965
 - calledby `initializeSetVariables`, 962
 - calledby `set1`, 1083
- trimString
 - calledby `checkFilter`, 1589
 - calledby `removeUndoLines`, 57
- truncList, 97
 - calledby `/untrace-0`, 123
 - defun, 97
- truth-to-bit, 1301
 - defmacro, 1301
- typeCheckInputAreas, 1538
 - defun, 1538
- types
 - calledby `clearCmdParts`, 841
- unabbrev
 - calledby `kDomainName`, 1624
 - calledby `reportOperations`, 1089
 - calledby `setExposeAddConstr`, 160
 - calledby `setExposeDropConstr`, 164
 - calledby `trace1`, 79
 - calledby `transTraceItem`, 126
- unabbrevAndLoad
 - calledby `domainToGenvar`, 101
- unAbbreviateKeyword, 809
 - calledby `doSystemCommand`, 786
 - calls `commandsForUserLevel`, 809
 - calls `selectOptionLC`, 809
 - calls `selectOption`, 809
 - uses `$currentLine`, 809
 - uses `$syscommands`, 809
 - uses `$systemCommands`, 809
 - uses `line`, 809
 - defun, 809
- underbar
 - usedby `writeInputLines`, 897
- underDomainOf
 - calledby `coerceCommuteTest`, 758
 - calledby `coerceIntByMap`, 761

- calledby coerceIntCommute, 757
- calledby coerceIntX, 768
- calledby hasCateSpecialNew, 734
- calledby retractUnderDomain, 777
- undo, 52, 1112
 - calls identp, 52
 - calls pname, 52
 - calls qcar, 52
 - calls qcdr, 52
 - calls read, 52
 - calls stringPrefix?, 52
 - calls undoCount, 52
 - calls undoSteps, 52
 - calls userError, 52
 - uses \$InteractiveFrame, 52
 - uses \$options, 52
 - defun, 52
 - manpage, 1112
- undoChanges, 903
 - calledby undoChanges, 903
 - calledby undoInCore, 902
 - calls boot-equal, 903
 - calls putHist, 903
 - calls undoChanges, 903
 - uses \$HistList, 903
 - uses \$InteractiveFrame, 903
 - defun, 903
- undoCount, 61
 - calledby removeUndoLines, 57
 - calledby undo, 52
 - calls concat, 61
 - calls userError, 61
 - uses \$IOindex, 61
 - defun, 61
- undoFromFile, 904
 - calls assq, 904
 - calls disableHist, 904
 - calls exit, 904
 - calls putHist, 904
 - calls readHiFi, 904
 - calls recordNewValue, 904
 - calls recordOldValue, 904
 - calls seq, 904
 - calls updateHist, 904
 - uses \$HiFiAccess, 904
 - uses \$InteractiveFrame, 904
- defun, 904
- undoInCore, 902
 - calls assq, 902
 - calls disableHist, 902
 - calls putHist, 902
 - calls readHiFi, 902
 - calls sayKeyedMsg, 902
 - calls undoChanges, 902
 - calls updateHist, 902
 - uses \$HiFiAccess, 902
 - uses \$HistListLen, 902
 - uses \$HistList, 902
 - uses \$IOindex, 902
 - uses \$InteractiveFrame, 902
 - defun, 902
- undoLocalModemapHack, 56
 - calledby undoSingleStep, 55
 - calls exit, 56
 - calls seq, 56
 - defun, 56
- undoSingleStep, 54
 - calledby undoSteps, 54
 - calls assq, 55
 - calls exit, 55
 - calls lassoc, 55
 - calls seq, 55
 - calls undoLocalModemapHack, 55
 - defun, 54
- undoSteps, 53
 - calledby undo, 52
 - calls copy, 54
 - calls qcar, 54
 - calls qcdr, 54
 - calls recordFrame, 54
 - calls undoSingleStep, 54
 - calls writeInputLines, 53
 - uses \$IOindex, 54
 - uses \$InteractiveFrame, 54
 - uses \$frameRecord, 54
 - defun, 53
- unembed
 - calledby /untrace-2, 124
 - calledby untraceDomainConstructor, 139
- unescapeStringsInForm, 354
 - calledby serverReadLine, 336
 - calledby unescapeStringsInForm, 354

- calls unescapeStringsInForm, [354](#)
 - uses \$funnyBacks, [354](#)
 - uses \$funnyQuote, [354](#)
 - defun, [354](#)
- unifyStruct, [726](#)
 - calledby hasAtt, [722](#)
 - calledby hasCaty, [719](#)
 - calledby hasSig, [721](#)
 - calledby unifyStructVar, [727](#)
 - calledby unifyStruct, [726](#)
 - calls isPatternVar, [726](#)
 - calls unifyStructVar, [726](#)
 - calls unifyStruct, [726](#)
 - defun, [726](#)
- unifyStructVar, [727](#)
 - calledby unifyStruct, [726](#)
 - calls augmentSub, [727](#)
 - calls canCoerce, [727](#)
 - calls constructor?, [727](#)
 - calls contained, [727](#)
 - calls containsVars, [727](#)
 - calls isPatternVar, [727](#)
 - calls lassoc, [727](#)
 - calls resolveTT, [727](#)
 - calls subCopy, [727](#)
 - calls unifyStruct, [727](#)
 - local def \$hope, [727](#)
 - local ref \$Coerce, [727](#)
 - local ref \$Subst, [727](#)
 - local ref \$domPvar, [727](#)
 - defun, [727](#)
- union
 - calledby breaklet, [155](#)
 - calledby dbShowCons1, [1643](#)
 - calledby getMapSubNames, [131](#)
 - calledby tracelet, [154](#)
- unionq
 - calledby getMapSubNames, [131](#)
- unparseInputForm, [1307](#)
 - local def \$InteractiveMode, [1307](#)
 - local def \$formatSigAsTex, [1307](#)
 - defun, [1307](#)
- untrace, [122](#)
 - calledby trace1, [79](#)
 - calls /untrace,0, [123](#)
 - calls copy, [122](#)
 - calls lassocSub, [123](#)
 - calls removeTracedMapSigs, [123](#)
 - calls transTraceItem, [122](#)
 - uses \$lastUntraced, [123](#)
 - uses \$mapSubNameAlist, [123](#)
 - uses \$traceNames, [123](#)
 - defun, [122](#)
- untrace2
 - calledby trace3, [83](#)
- untraceDomainConstructor, [139](#)
 - calledby /untrace-2, [124](#)
 - calls concat, [139](#)
 - calls delete, [139](#)
 - calls exit, [139](#)
 - calls seq, [139](#)
 - calls unembed, [139](#)
 - calls untraceDomainConstructor,keepTraced?, [139](#)
 - uses \$traceNames, [139](#)
 - defun, [139](#)
- untraceDomainConstructor,keepTraced?, [138](#)
 - calledby untraceDomainConstructor, [139](#)
 - calls /untrace,0, [138](#)
 - calls boot-equal, [138](#)
 - calls devaluate, [138](#)
 - calls exit, [138](#)
 - calls isDomainOrPackage, [138](#)
 - calls qcar, [138](#)
 - calls seq, [138](#)
 - defun, [138](#)
- untraceDomainLocalOps, [136](#)
 - calledby trace1, [79](#)
 - calls sayMSG, [136](#)
 - defun, [136](#)
- untraceMapSubNames, [105](#)
 - calledby clearCmdAll, [839](#)
 - calledby clearCmdParts, [841](#)
 - calls /untrace,2, [105](#)
 - calls assocright, [105](#)
 - calls getPreviousMapSubNames, [105](#)
 - calls setdifference, [105](#)
 - uses \$lastUntraced, [105](#)
 - uses \$mapSubNameAlist, [105](#)
 - uses \$traceNames, [105](#)
 - defun, [105](#)
- unwrap

- calledby catchCoerceFailure, 760
- calledby coerceBranch2Union, 766
- calledby coerceByTable, 759
- calledby coerceInt0, 742
- calledby coerceInt1, 744
- calledby coerceIntX, 768
- calledby coerceInt, 741
- calledby retract2Specialization, 772
- unwritable?, 916
 - calls placep, 916
 - calls vecp, 916
 - defun, 916
- upcase, 1272
 - calledby close, 845
 - calledby historySpad2Cmd, 890
 - calledby importFromFrame, 34
 - calledby listConstructorAbbreviations, 824
 - calledby pathnameTypeId, 1231
 - calledby quitSpad2Cmd, 940
 - calledby readSpad2Cmd, 943
 - calledby setOutputAlgebra, 1035
 - calledby setOutputFormula, 1063
 - calledby setOutputFortran, 1042
 - calledby setOutputHtml, 1048
 - calledby setOutputMathml, 1054
 - calledby setOutputOpenMath, 1058
 - calledby setOutputTex, 1070
 - calledby transOnlyOption, 99
 - calledby upcase, 1272
 - calledby yesanswer, 866
 - calls identp, 1272
 - calls upcase, 1272
 - defun, 1272
- updateCategoryTable
 - calledby loadLib, 1222
 - calledby localnrLib, 1201
- updateCurrentInterpreterFrame, 31
 - calledby addNewInterpreterFrame, 33
 - calledby changeToNamedInterpreterFrame, 32
 - calledby clearCmdAll, 839
 - calledby clearSpad2Cmd, 835
 - calledby previousInterpreterFrame, 33
 - calledby updateHist, 899
 - calls createCurrentInterpreterFrame, 31
 - calls updateFromCurrentInterpreterFrame, 31
 - uses \$interpreterFrameRing, 31
 - defun, 31
- updateDatabase, 1203
 - calledby loadLib, 1221
 - calledby localnrLib, 1201
 - calls clearAllSlams, 1203
 - calls clearClams, 1203
 - calls constructor?, 1203
 - local ref \$forceDatabaseUpdate, 1203
 - defun, 1203
- updateFromCurrentInterpreterFrame, 30
 - calledby addNewInterpreterFrame, 34
 - calledby changeToNamedInterpreterFrame, 32
 - calledby closeInterpreterFrame, 37
 - calledby initializeInterpreterFrameRing, 27
 - calledby nextInterpreterFrame, 32
 - calledby previousInterpreterFrame, 33
 - calledby updateCurrentInterpreterFrame, 31
 - calls sayMessage, 30
 - uses \$HiFiAccess, 30
 - uses \$HistListAct, 30
 - uses \$HistListLen, 30
 - uses \$HistList, 30
 - uses \$HistRecord, 30
 - uses \$IOindex, 30
 - uses \$InteractiveFrame, 30
 - uses \$frameMessages, 30
 - uses \$internalHistoryTable, 30
 - uses \$interpreterFrameName, 30
 - uses \$interpreterFrameRing, 30
 - uses \$localExposureData, 30
 - defun, 30
- updateHist, 899
 - calledby processInteractive, 342
 - calledby undoFromFile, 904
 - calledby undoInCore, 902
 - calls disableHist, 899
 - calls startTimingProcess, 899
 - calls stopTimingProcess, 899
 - calls updateCurrentInterpreterFrame, 899
 - calls updateInCoreHist, 899

- calls writeHiFi, 899
 - uses \$HiFiAccess, 899
 - uses \$HistRecord, 899
 - uses \$IOindex, 899
 - uses \$currentLine, 899
 - uses \$mkTestInputStack, 899
 - defun, 899
- updateInCoreHist, 900
 - calledby restoreHistory, 907
 - calledby updateHist, 899
 - uses \$HistListAct, 900
 - uses \$HistListLen, 900
 - uses \$HistList, 900
 - defun, 900
- updateSourceFiles, 874
 - calledby editSpad2Cmd, 872
 - calledby workfilesSpad2Cmd, 1139
 - calls insert, 874
 - calls makeInputFilename, 874
 - calls member, 874
 - calls pathnameName, 874
 - calls pathnameTypeId, 874
 - calls pathnameType, 874
 - calls pathname, 874
 - uses \$sourceFiles, 874
 - defun, 874
- use-fast-links
 - calledby set1, 1083
- userError
 - calledby spadTrace, 132
 - calledby spadUntrace, 143
 - calledby traceReply, 94
 - calledby undoCount, 61
 - calledby undo, 52
- userLevelErrorMessage, 790
 - calledby commandUserLevelError, 790
 - calledby optionUserLevelError, 790
 - calls commandAmbiguityError, 790
 - calls sayKeyedMsg, 791
 - calls terminateSystemCommand, 791
 - uses \$UserLevel, 791
 - defun, 790
- util.ht[7.1]
 - called by buildHtMacroTable, 1401
 - called by getHtMacroItem, 1402
- valid-tokens
 - usedby token-stack-show, 1157
- validateOutputDirectory, 997
 - calledby setFortDir, 998
 - calledby setFortTmpDir, 996
 - defun, 997
- valueArgsEqual?, 764
 - calledby coerceIntByMap, 761
 - calls algEqual, 764
 - calls coerceInt, 764
 - calls getConstructorSignature, 764
 - calls getdatabase, 764
 - calls mkObjWrap, 764
 - calls objValUnwrap, 764
 - calls replaceSharps, 764
 - defun, 764
- values
 - calledby clearCmdParts, 841
- varp, 89
 - calledby flatBvList, 89
 - defun, 89
- varsInPoly
 - calledby retract2Specialization, 772
- vec2list, 1276
 - calledby computeTTTranspositions, 754
 - defun, 1276
- vecp
 - calledby ScanOrPairVec, ScanOrInner, 926
 - calledby basicLookupCheckDefaults, 1228
 - calledby basicLookup, 1226
 - calledby dewritify, dewritifyInner, 922
 - calledby spadClosure?, 921
 - calledby trace1, 79
 - calledby transTraceItem, 126
 - calledby unwritable?, 916
 - calledby writify, writifyInner, 917
- vmread, 1297
 - calledby dewritify, dewritifyInner, 922
 - defun, 1297
- voidValue, 1325
 - defun, 1325
- warn
 - calledby getdatabase, 1196
- what, 1129, 1131
 - calls whatSpad2Cmd, 1131

- defun, [1131](#)
 - manpage, [1129](#)
- whatCommands, [1133](#)
 - calledby whatSpad2Cmd, [1131](#)
 - calls blankList, [1133](#)
 - calls commandsForUserLevel, [1133](#)
 - calls concat, [1133](#)
 - calls filterListOfStrings, [1133](#)
 - calls sayAsManyPerLineAsPossible, [1133](#)
 - calls sayKeyedMsg, [1134](#)
 - calls sayMessage, [1133](#)
 - calls say, [1133](#)
 - calls specialChar, [1133](#)
 - uses \$UserLevel, [1134](#)
 - uses \$linelength, [1134](#)
 - uses \$systemCommands, [1134](#)
 - defun, [1133](#)
- whatConstructors, [1137](#)
 - calledby filterAndFormatConstructors, [1136](#)
 - calls boot-equal, [1137](#)
 - calls exit, [1137](#)
 - calls getdatabase, [1137](#)
 - calls msort, [1137](#)
 - calls seq, [1137](#)
 - defun, [1137](#)
- whatSpad2Cmd, [1131](#)
 - calledby listConstructorAbbreviations, [824](#)
 - calledby whatSpad2Cmd, [1131](#)
 - calledby what, [1131](#)
 - calls apropos, [1131](#)
 - calls exit, [1131](#)
 - calls filterAndFormatConstructors, [1131](#)
 - calls printSynonyms, [1131](#)
 - calls reportWhatOptions, [1131](#)
 - calls sayKeyedMsg, [1131](#)
 - calls selectOptionLC, [1131](#)
 - calls seq, [1131](#)
 - calls whatCommands, [1131](#)
 - calls whatSpad2Cmd,fixpat, [1131](#)
 - calls whatSpad2Cmd, [1131](#)
 - uses \$e, [1131](#)
 - uses \$whatOptions, [1132](#)
 - defun, [1131](#)
- whatSpad2Cmd,fixpat, [1131](#)
 - calledby whatSpad2Cmd, [1131](#)
 - calls downcase, [1131](#)
 - calls qcar, [1131](#)
 - defun, [1131](#)
- whichCat, [655](#)
 - calledby setMsgForcedAttrList, [654](#)
 - calledby setMsgUnforcedAttrList, [656](#)
 - calls ListMember?, [655](#)
 - uses \$attrCats, [655](#)
 - defun, [655](#)
- while, [697](#), [1230](#)
 - defmacro, [1230](#)
 - syntax, [697](#)
- whileWithResult, [1230](#)
 - defmacro, [1230](#)
- whocalled
 - calledby monitorXX, [90](#)
- workfiles, [1139](#)
 - calls workfilesSpad2Cmd, [1139](#)
 - defun, [1139](#)
 - manpage, [1139](#)
- workfilesSpad2Cmd, [1139](#)
 - calledby workfiles, [1139](#)
 - calls delete, [1139](#)
 - calls makeInputFilename, [1139](#)
 - calls namestring, [1139](#)
 - calls pathname, [1139](#)
 - calls sayBrightly, [1139](#)
 - calls sayKeyedMsg, [1139](#)
 - calls say, [1139](#)
 - calls selectOptionLC, [1139](#)
 - calls sortby, [1139](#)
 - calls specialChar, [1139](#)
 - calls throwKeyedMsg, [1139](#)
 - calls updateSourceFiles, [1139](#)
 - uses \$linelength, [1139](#)
 - uses \$options, [1139](#)
 - uses \$sourceFiles, [1139](#)
 - defun, [1139](#)
- wrap, [1233](#)
 - calledby catchCoerceFailure, [760](#)
 - calledby mkObjWrap, [515](#)
 - calledby wrap, [1233](#)
 - calls lotsof, [1233](#)
 - calls wrap, [1233](#)
 - defun, [1233](#)
- wrapped2Quote
 - calledby coerceIntByMap, [761](#)

- write-browsedb, 1215
 - calledby make-databases, 1205
 - calls allConstructors, 1215
 - uses *print-pretty*, 1215
 - uses *sourcefiles*, 1215
 - uses \$spadroot, 1215
 - defun, 1215
- write-categorydb, 1216
 - calledby make-databases, 1205
 - calls genCategoryTable, 1216
 - uses *hasCategory-hash*, 1216
 - uses *print-pretty*, 1216
 - defun, 1216
- write-interpdb, 1213
 - calledby make-databases, 1205
 - uses *ancestors-hash*, 1213
 - uses *print-pretty*, 1213
 - uses \$spadroot, 1213
 - defun, 1213
- write-operationdb, 1217
 - calledby make-databases, 1205
 - uses *operation-hash*, 1217
 - defun, 1217
- write-warndata, 1218
 - calledby make-databases, 1205
 - uses \$topicHash, 1218
 - defun, 1218
- writablep
 - calledby myWritable?, 1300
- writeHiFi, 912
 - calledby updateHist, 899
 - calls histFileName, 912
 - calls object2Identifier, 912
 - calls rdefiostream, 912
 - calls rshut, 912
 - calls spadwrite, 912
 - uses \$HistRecord, 912
 - uses \$IOindex, 912
 - uses \$currentLine, 912
 - uses \$internalHistoryTable, 912
 - uses \$useInternalHistoryTable, 912
 - defun, 912
- writeHistModesAndValues, 913
 - calledby processInteractive, 342
 - calls get, 913
 - calls putHist, 913
 - uses \$InteractiveFrame, 913
 - defun, 913
- writeInputLines, 896
 - calledby historySpad2Cmd, 890
 - calledby saveHistory, 905
 - calledby undoSteps, 53
 - calls concat, 896
 - calls defiostream, 896
 - calls histFileErase, 896
 - calls histInputFileName, 896
 - calls namestring, 896
 - calls readHiFi, 896
 - calls sayKeyedMsg, 896
 - calls shut, 896
 - calls size, 896
 - calls substring, 896
 - calls throwKeyedMsg, 896
 - uses \$HiFiAccess, 897
 - uses \$IOindex, 897
 - uses underbar, 897
 - defun, 896
- writeLib1
 - calledby saveDependentsHashTable, 1208
 - calledby saveUsersHashTable, 1208
- writify, 920
 - calledby safeWritify, 917
 - calls ScanOrPairVec, 920
 - calls function, 920
 - calls writify,writifyInner, 920
 - uses \$seen, 920
 - uses \$writifyComplained, 920
 - defun, 920
- writify,writifyInner, 917
 - calledby writify,writifyInner, 917
 - calledby writify, 920
 - calls boot-equal, 917
 - calls constructor?, 917
 - calls devaluate, 917
 - calls exit, 917
 - calls hashtable-class, 917
 - calls hget, 917
 - calls hkeys, 917
 - calls hput, 917
 - calls isDomainOrPackage, 917
 - calls mkEvalable, 917
 - calls placep, 917

- calls qcar, [917](#)
- calls qcdr, [917](#)
- calls qrplaca, [917](#)
- calls qrplacd, [917](#)
- calls qsetvelt, [917](#)
- calls qvelt, [917](#)
- calls qvmaxindex, [917](#)
- calls seq, [917](#)
- calls spadClosure?, [917](#)
- calls vecp, [917](#)
- calls writify,writifyInner, [917](#)
- uses \$NonNullStream, [918](#)
- uses \$NullStream, [918](#)
- uses \$seen, [918](#)
- defun, [917](#)
- throws, [917](#)
- writifyComplain, [916](#)
 - calls sayKeyedMsg, [916](#)
 - uses \$writifyComplained, [916](#)
 - defun, [916](#)
- xlCannotRead, [384](#)
 - calledby incLude1, [374](#)
 - calls inclmsgCannotRead, [384](#)
 - calls xMsg, [384](#)
 - defun, [384](#)
- xlCmdBug, [391](#)
 - calledby incLude1, [375](#)
 - calls inclmsgCmdBug, [391](#)
 - calls xMsg, [391](#)
 - defun, [391](#)
- xlConActive, [386](#)
 - calledby incLude1, [374](#)
 - calls inclmsgConActive, [386](#)
 - calls xMsg, [386](#)
 - defun, [386](#)
- xlConsole, [387](#)
 - calledby incLude1, [375](#)
 - calls inclmsgConsole, [387](#)
 - calls xMsg, [387](#)
 - defun, [387](#)
- xlConStill, [387](#)
 - calledby incLude1, [374](#)
 - calls inclmsgConStill, [387](#)
 - calls xMsg, [387](#)
 - defun, [387](#)
- xlFileCycle, [385](#)
 - calledby incLude1, [374](#)
 - calls inclmsgFileCycle, [385](#)
 - calls xMsg, [385](#)
 - defun, [385](#)
- xlIfBug, [391](#)
 - calledby incLude1, [375](#)
 - calls inclmsgIfBug, [391](#)
 - calls xMsg, [391](#)
 - defun, [391](#)
- xlIfSyntax, [390](#)
 - calledby incLude1, [375](#)
 - calls Else?, [390](#)
 - calls Top?, [390](#)
 - calls inclmsgIfSyntax, [390](#)
 - calls xMsg, [390](#)
 - defun, [390](#)
- xMsg, [379](#)
 - calledby xlCannotRead, [384](#)
 - calledby xlCmdBug, [391](#)
 - calledby xlConActive, [386](#)
 - calledby xlConStill, [387](#)
 - calledby xlConsole, [387](#)
 - calledby xlFileCycle, [385](#)
 - calledby xlIfBug, [391](#)
 - calledby xlIfSyntax, [390](#)
 - calledby xlNoSuchFile, [383](#)
 - calledby xIPrematureEOF, [378](#)
 - calledby xIPrematureFin, [389](#)
 - calledby xISay, [382](#)
 - calledby xISkippingFin, [388](#)
 - calls incLine, [379](#)
 - defun, [379](#)
- xlNoSuchFile, [383](#)
 - calledby incLude1, [374](#)
 - calls inclmsgNoSuchFile, [383](#)
 - calls xMsg, [383](#)
 - defun, [383](#)
- xlOK, [379](#)
 - calledby incLude1, [374](#)
 - calls xlOK1, [379](#)
 - defun, [379](#)
- xlOK1, [379](#)
 - calledby incLude1, [374](#)
 - calledby xlOK, [379](#)
 - calls incLine1, [379](#)

- defun, 379
- xlPrematureEOF, 378
 - calledby incLude1, 374
 - calls inclmsgPrematureEOF, 378
 - calls xIMsg, 378
 - defun, 378
- xlPrematureFin, 389
 - calledby incLude1, 375
 - calls inclmsgPrematureFin, 389
 - calls xIMsg, 389
 - defun, 389
- xlSay, 382
 - calledby incLude1, 374
 - calls inclmsgSay, 383
 - calls xIMsg, 382
 - defun, 382
- xlSkip, 382
 - calledby incLude1, 374
 - calls concat, 382
 - calls incLine, 382
 - defun, 382
- xlSkippingFin, 388
 - calledby incLude1, 375
 - calls inclmsgFinSkipped, 388
 - calls xIMsg, 388
 - defun, 388
- xtokenreader, 1154
 - usedby init-boot/spad-reader, 1154
 - defvar, 1154
- yesanswer, 866
 - calledby displayOperations, 866
 - calls queryUserKeyedMsg, 866
 - calls string2id-n, 866
 - calls upcase, 866
 - defun, 866
- ySearch, 1596
 - calledby conPage, 1599
 - calls constructorSearch, 1596
 - defun, 1596
- zeroOneConvert
 - calledby kePageOpAlist, 1607
- zeroOneTran, 360
 - calledby intInterpretPform, 360
 - calls nsubst, 360
- defun, 360